



PTX ISA

Release 9.1

NVIDIA Corporation

Jan 08, 2026

Contents

1	Introduction	9
1.1	Scalable Data-Parallel Computing using GPUs	9
1.2	Goals of PTX	10
1.3	PTX ISA Version 9.1	10
1.4	Document Structure	10
2	Programming Model	13
2.1	A Highly Multithreaded Coprocessor	13
2.2	Thread Hierarchy	13
2.2.1	Cooperative Thread Arrays	13
2.2.2	Cluster of Cooperative Thread Arrays	14
2.2.3	Grid of Clusters	14
2.3	Memory Hierarchy	15
3	PTX Machine Model	19
3.1	A Set of SIMT Multiprocessors	19
3.2	Independent Thread Scheduling	21
3.3	On-chip Shared Memory	21
4	Syntax	23
4.1	Source Format	23
4.2	Comments	23
4.3	Statements	24
4.3.1	Directive Statements	24
4.3.2	Instruction Statements	24
4.4	Identifiers	26
4.5	Constants	27
4.5.1	Integer Constants	27
4.5.2	Floating-Point Constants	27
4.5.3	Predicate Constants	28
4.5.4	Constant Expressions	28
4.5.5	Integer Constant Expression Evaluation	29
4.5.6	Summary of Constant Expression Evaluation Rules	30
5	State Spaces, Types, and Variables	33
5.1	State Spaces	33
5.1.1	Register State Space	34
5.1.2	Special Register State Space	35
5.1.3	Constant State Space	35
5.1.3.1	Banked Constant State Space (deprecated)	35
5.1.4	Global State Space	36
5.1.5	Local State Space	36
5.1.6	Parameter State Space	36
5.1.6.1	Kernel Function Parameters	37

5.1.6.2	Kernel Function Parameter Attributes	37
5.1.6.3	Kernel Parameter Attribute: <code>.ptr</code>	38
5.1.6.4	Device Function Parameters	39
5.1.7	Shared State Space	40
5.1.8	Texture State Space (deprecated)	40
5.2	Types	41
5.2.1	Fundamental Types	41
5.2.2	Restricted Use of Sub-Word Sizes	42
5.2.3	Alternate Floating-Point Data Formats	42
5.2.4	Fixed-point Data format	43
5.2.5	Packed Data Types	43
5.2.5.1	Packed Floating Point Data Types	44
5.2.5.2	Packed Integer Data Types	44
5.2.5.3	Packed Fixed-Point Data Types	45
5.3	Texture Sampler and Surface Types	45
5.3.1	Texture and Surface Properties	46
5.3.2	Sampler Properties	46
5.3.3	Channel Data Type and Channel Order Fields	48
5.4	Variables	49
5.4.1	Variable Declarations	49
5.4.2	Vectors	50
5.4.3	Array Declarations	50
5.4.4	Initializers	51
5.4.5	Alignment	53
5.4.6	Parameterized Variable Names	53
5.4.7	Variable Attributes	53
5.4.8	Variable and Function Attribute Directive: <code>.attribute</code>	54
5.5	Tensors	55
5.5.1	Tensor Dimension, size and format	55
5.5.1.1	Sub-byte Types	55
5.5.2	Tensor Access Modes	58
5.5.3	Tiled Mode	58
5.5.3.1	Bounding Box	58
5.5.3.2	Traversal-Stride	58
5.5.3.3	Out of Boundary Access	60
5.5.3.4	<code>.tile::scatter4</code> and <code>.tile::gather4</code> modes	61
5.5.4	<code>im2col</code> mode	62
5.5.4.1	Bounding Box	62
5.5.4.2	Traversal Stride	68
5.5.4.3	Out of Boundary Access	69
5.5.5	<code>im2col::w</code> and <code>im2col::w::128</code> modes	69
5.5.5.1	Bounding Box	69
5.5.5.2	Traversal Stride	70
5.5.5.3	<code>wHalo</code>	70
5.5.5.4	<code>wOffset</code>	73
5.5.6	Interleave layout	75
5.5.7	Swizzling Modes	77
5.5.8	Tensor-map	88
6	Instruction Operands	89
6.1	Operand Type Information	89
6.2	Source Operands	89
6.3	Destination Operands	89
6.4	Using Addresses, Arrays, and Vectors	90

6.4.1	Addresses as Operands	90
6.4.1.1	Generic Addressing	91
6.4.2	Arrays as Operands	91
6.4.3	Vectors as Operands	91
6.4.4	Labels and Function Names as Operands	92
6.5	Type Conversion	92
6.5.1	Scalar Conversions	92
6.5.2	Rounding Modifiers	94
6.6	Operand Costs	94
7	Abstracting the ABI	97
7.1	Function Declarations and Definitions	97
7.1.1	Changes from PTX ISA Version 1.x	100
7.2	Variadic Functions	100
7.3	Alloca	100
8	Memory Consistency Model	101
8.1	Scope and applicability of the model	101
8.1.1	Limitations on atomicity at system scope	101
8.2	Memory operations	101
8.2.1	Overlap	102
8.2.2	Aliases	102
8.2.3	Multimem Addresses	102
8.2.4	Memory Operations on Vector Data Types	102
8.2.5	Memory Operations on Packed Data Types	102
8.2.6	Initialization	103
8.3	State spaces	103
8.4	Operation types	103
8.4.1	mmio Operation	104
8.4.2	volatile Operation	105
8.5	Scope	105
8.6	Proxies	106
8.7	Morally strong operations	106
8.7.1	Conflict and Data-races	106
8.7.2	Limitations on Mixed-size Data-races	106
8.8	Release and Acquire Patterns	107
8.9	Ordering of memory operations	108
8.9.1	Program Order	108
8.9.1.1	Asynchronous Operations	108
8.9.2	Observation Order	108
8.9.3	Fence-SC Order	109
8.9.4	Memory synchronization	109
8.9.5	Causality Order	110
8.9.6	Coherence Order	111
8.9.7	Communication Order	111
8.10	Axioms	111
8.10.1	Coherence	111
8.10.2	Fence-SC	111
8.10.3	Atomicity	112
8.10.4	No Thin Air	113
8.10.5	Sequential Consistency Per Location	113
8.10.6	Causality	114
8.11	Special Cases	116
8.11.1	Reductions do not form Acquire Patterns	116

9	Instruction Set	119
9.1	Format and Semantics of Instruction Descriptions	119
9.2	PTX Instructions	119
9.3	Predicated Execution	120
9.3.1	Comparisons	120
9.3.1.1	Integer and Bit-Size Comparisons	120
9.3.1.2	Floating Point Comparisons	121
9.3.2	Manipulating Predicates	122
9.4	Type Information for Instructions and Operands	122
9.4.1	Operand Size Exceeding Instruction-Type Size	123
9.5	Divergence of Threads in Control Constructs	126
9.6	Semantics	126
9.6.1	Machine-Specific Semantics of 16-bit Code	126
9.7	Instructions	127
9.7.1	Integer Arithmetic Instructions	127
9.7.1.1	Integer Arithmetic Instructions: add	128
9.7.1.2	Integer Arithmetic Instructions: sub	129
9.7.1.3	Integer Arithmetic Instructions: mul	130
9.7.1.4	Integer Arithmetic Instructions: mad	131
9.7.1.5	Integer Arithmetic Instructions: mul24	133
9.7.1.6	Integer Arithmetic Instructions: mad24	134
9.7.1.7	Integer Arithmetic Instructions: sad	135
9.7.1.8	Integer Arithmetic Instructions: div	136
9.7.1.9	Integer Arithmetic Instructions: rem	137
9.7.1.10	Integer Arithmetic Instructions: abs	138
9.7.1.11	Integer Arithmetic Instructions: neg	139
9.7.1.12	Integer Arithmetic Instructions: min	140
9.7.1.13	Integer Arithmetic Instructions: max	141
9.7.1.14	Integer Arithmetic Instructions: popc	143
9.7.1.15	Integer Arithmetic Instructions: clz	144
9.7.1.16	Integer Arithmetic Instructions: bfind	145
9.7.1.17	Integer Arithmetic Instructions: fns	146
9.7.1.18	Integer Arithmetic Instructions: brev	147
9.7.1.19	Integer Arithmetic Instructions: bfe	148
9.7.1.20	Integer Arithmetic Instructions: bfi	150
9.7.1.21	Integer Arithmetic Instructions: szext	151
9.7.1.22	Integer Arithmetic Instructions: bmsk	152
9.7.1.23	Integer Arithmetic Instructions: dp4a	153
9.7.1.24	Integer Arithmetic Instructions: dp2a	155
9.7.2	Extended-Precision Integer Arithmetic Instructions	156
9.7.2.1	Extended-Precision Arithmetic Instructions: add.cc	156
9.7.2.2	Extended-Precision Arithmetic Instructions: addc	157
9.7.2.3	Extended-Precision Arithmetic Instructions: sub.cc	159
9.7.2.4	Extended-Precision Arithmetic Instructions: subc	160
9.7.2.5	Extended-Precision Arithmetic Instructions: mad.cc	161
9.7.2.6	Extended-Precision Arithmetic Instructions: madc	162
9.7.3	Floating-Point Instructions	163
9.7.3.1	Floating Point Instructions: testp	166
9.7.3.2	Floating Point Instructions: copysign	167
9.7.3.3	Floating Point Instructions: add	168
9.7.3.4	Floating Point Instructions: sub	170
9.7.3.5	Floating Point Instructions: mul	172
9.7.3.6	Floating Point Instructions: fma	174
9.7.3.7	Floating Point Instructions: mad	176

9.7.3.8	Floating Point Instructions: <code>div</code>	178
9.7.3.9	Floating Point Instructions: <code>abs</code>	180
9.7.3.10	Floating Point Instructions: <code>neg</code>	181
9.7.3.11	Floating Point Instructions: <code>min</code>	182
9.7.3.12	Floating Point Instructions: <code>max</code>	185
9.7.3.13	Floating Point Instructions: <code>rcp</code>	187
9.7.3.14	Floating Point Instructions: <code>rcp.approx.ftz.f64</code>	189
9.7.3.15	Floating Point Instructions: <code>sqrt</code>	190
9.7.3.16	Floating Point Instructions: <code>rsqrt</code>	192
9.7.3.17	Floating Point Instructions: <code>rsqrt.approx.ftz.f64</code>	194
9.7.3.18	Floating Point Instructions: <code>sin</code>	195
9.7.3.19	Floating Point Instructions: <code>cos</code>	197
9.7.3.20	Floating Point Instructions: <code>lg2</code>	198
9.7.3.21	Floating Point Instructions: <code>ex2</code>	200
9.7.3.22	Floating Point Instructions: <code>tanh</code>	201
9.7.4	Half Precision Floating-Point Instructions	202
9.7.4.1	Half Precision Floating Point Instructions: <code>add</code>	203
9.7.4.2	Half Precision Floating Point Instructions: <code>sub</code>	205
9.7.4.3	Half Precision Floating Point Instructions: <code>mul</code>	207
9.7.4.4	Half Precision Floating Point Instructions: <code>fma</code>	209
9.7.4.5	Half Precision Floating Point Instructions: <code>neg</code>	211
9.7.4.6	Half Precision Floating Point Instructions: <code>abs</code>	213
9.7.4.7	Half Precision Floating Point Instructions: <code>min</code>	214
9.7.4.8	Half Precision Floating Point Instructions: <code>max</code>	217
9.7.4.9	Half Precision Floating Point Instructions: <code>tanh</code>	219
9.7.4.10	Half Precision Floating Point Instructions: <code>ex2</code>	220
9.7.5	Mixed Precision Floating-Point Instructions	222
9.7.5.1	Mixed Precision Floating Point Instructions: <code>add</code>	223
9.7.5.2	Mixed Precision Floating Point Instructions: <code>sub</code>	224
9.7.5.3	Mixed Precision Floating Point Instructions: <code>fma</code>	225
9.7.6	Comparison and Selection Instructions	227
9.7.6.1	Comparison and Selection Instructions: <code>set</code>	227
9.7.6.2	Comparison and Selection Instructions: <code>setp</code>	229
9.7.6.3	Comparison and Selection Instructions: <code>selp</code>	231
9.7.6.4	Comparison and Selection Instructions: <code>slct</code>	232
9.7.7	Half Precision Comparison Instructions	233
9.7.7.1	Half Precision Comparison Instructions: <code>set</code>	233
9.7.7.2	Half Precision Comparison Instructions: <code>setp</code>	236
9.7.8	Logic and Shift Instructions	238
9.7.8.1	Logic and Shift Instructions: <code>and</code>	238
9.7.8.2	Logic and Shift Instructions: <code>or</code>	239
9.7.8.3	Logic and Shift Instructions: <code>xor</code>	240
9.7.8.4	Logic and Shift Instructions: <code>not</code>	241
9.7.8.5	Logic and Shift Instructions: <code>cnot</code>	242
9.7.8.6	Logic and Shift Instructions: <code>lop3</code>	243
9.7.8.7	Logic and Shift Instructions: <code>shf</code>	245
9.7.8.8	Logic and Shift Instructions: <code>shl</code>	247
9.7.8.9	Logic and Shift Instructions: <code>shr</code>	248
9.7.9	Data Movement and Conversion Instructions	249
9.7.9.1	Cache Operators	250
9.7.9.2	Cache Eviction Priority Hints	252
9.7.9.3	Data Movement and Conversion Instructions: <code>mov</code>	253
9.7.9.4	Data Movement and Conversion Instructions: <code>mov</code>	254
9.7.9.5	Data Movement and Conversion Instructions: <code>shf1</code> (deprecated)	256

9.7.9.6	Data Movement and Conversion Instructions: <code>shfl.sync</code>	259
9.7.9.7	Data Movement and Conversion Instructions: <code>prmt</code>	261
9.7.9.8	Data Movement and Conversion Instructions: <code>ld</code>	263
9.7.9.9	Data Movement and Conversion Instructions: <code>ld.global.nc</code>	268
9.7.9.10	Data Movement and Conversion Instructions: <code>ldu</code>	271
9.7.9.11	Data Movement and Conversion Instructions: <code>st</code>	272
9.7.9.12	Data Movement and Conversion Instructions: <code>st.async</code>	277
9.7.9.13	Data Movement and Conversion Instructions: <code>st.bulk</code>	279
9.7.9.14	Data Movement and Conversion Instructions: <code>multimem.ld_reduce</code> , <code>multimem.st</code> , <code>multimem.red</code>	280
9.7.9.15	Data Movement and Conversion Instructions: <code>prefetch</code> , <code>prefetchu</code>	284
9.7.9.16	Data Movement and Conversion Instructions: <code>applypriority</code>	285
9.7.9.17	Data Movement and Conversion Instructions: <code>discard</code>	286
9.7.9.18	Data Movement and Conversion Instructions: <code>createpolicy</code>	287
9.7.9.19	Data Movement and Conversion Instructions: <code>isspacep</code>	289
9.7.9.20	Data Movement and Conversion Instructions: <code>cvta</code>	290
9.7.9.21	Data Movement and Conversion Instructions: <code>cvt</code>	292
9.7.9.22	Data Movement and Conversion Instructions: <code>cvt.pack</code>	304
9.7.9.23	Data Movement and Conversion Instructions: <code>mapa</code>	305
9.7.9.24	Data Movement and Conversion Instructions: <code>getctarank</code>	306
9.7.9.25	Data Movement and Conversion Instructions: Asynchronous copy	308
9.7.9.26	Data Movement and Conversion Instructions: <code>multimem.cp.async.bulk</code>	321
9.7.9.27	Data Movement and Conversion Instructions: <code>multimem.cp.reduce.async</code> , <code>bulk</code>	323
9.7.9.28	Data Movement and Conversion Instructions: <code>tensormap.replace</code>	339
9.7.10	Texture Instructions	343
9.7.10.1	Texturing Modes	343
9.7.10.2	Mipmaps	344
9.7.10.3	Texture Instructions: <code>tex</code>	345
9.7.10.4	Texture Instructions: <code>tld4</code>	351
9.7.10.5	Texture Instructions: <code>txq</code>	354
9.7.10.6	Texture Instructions: <code>istypep</code>	356
9.7.11	Surface Instructions	357
9.7.11.1	Surface Instructions: <code>suld</code>	357
9.7.11.2	Surface Instructions: <code>sust</code>	359
9.7.11.3	Surface Instructions: <code>sured</code>	361
9.7.11.4	Surface Instructions: <code>suq</code>	363
9.7.12	Control Flow Instructions	365
9.7.12.1	Control Flow Instructions: <code>{ }</code>	365
9.7.12.2	Control Flow Instructions: <code>@</code>	366
9.7.12.3	Control Flow Instructions: <code>bra</code>	367
9.7.12.4	Control Flow Instructions: <code>brx.idx</code>	368
9.7.12.5	Control Flow Instructions: <code>call</code>	369
9.7.12.6	Control Flow Instructions: <code>ret</code>	371
9.7.12.7	Control Flow Instructions: <code>exit</code>	372
9.7.13	Parallel Synchronization and Communication Instructions	373
9.7.13.1	Parallel Synchronization and Communication Instructions: <code>bar</code> , <code>barrier</code>	374
9.7.13.2	Parallel Synchronization and Communication Instructions: <code>bar.warp.sync</code>	377
9.7.13.3	Parallel Synchronization and Communication Instructions: <code>barrier.cluster</code>	378
9.7.13.4	Parallel Synchronization and Communication Instructions: <code>membar / fence</code>	380
9.7.13.5	Parallel Synchronization and Communication Instructions: <code>atom</code>	383
9.7.13.6	Parallel Synchronization and Communication Instructions: <code>red</code>	388
9.7.13.7	Parallel Synchronization and Communication Instructions: <code>red.async</code>	392
9.7.13.8	Parallel Synchronization and Communication Instructions: <code>vote</code> (deprecated)	395

9.7.13.9	Parallel Synchronization and Communication Instructions: <code>vote.sync</code>	397
9.7.13.10	Parallel Synchronization and Communication Instructions: <code>match.sync</code>	398
9.7.13.11	Parallel Synchronization and Communication Instructions: <code>activemask</code>	399
9.7.13.12	Parallel Synchronization and Communication Instructions: <code>redux.sync</code>	400
9.7.13.13	Parallel Synchronization and Communication Instructions: <code>griddepcontrol</code>	401
9.7.13.14	Parallel Synchronization and Communication Instructions: <code>elect.sync</code>	403
9.7.13.15	Parallel Synchronization and Communication Instructions: <code>mbarrier</code>	403
9.7.13.16	Parallel Synchronization and Communication Instructions: <code>tensormap</code> , <code>cp_fenceproxy</code>	424
9.7.13.17	Parallel Synchronization and Communication Instructions: <code>clusterlaunchcontrol.try_cancel</code>	425
9.7.13.18	Parallel Synchronization and Communication Instructions: <code>clusterlaunchcontrol.query_cancel</code>	428
9.7.14	Warp Level Matrix Multiply-Accumulate Instructions	429
9.7.14.1	Matrix Shape	430
9.7.14.2	Matrix Data-types	433
9.7.14.3	Block Scaling for <code>mma.sync</code>	434
9.7.14.4	Matrix multiply-accumulate operation using <code>wmma</code> instructions	438
9.7.14.5	Matrix multiply-accumulate operation using <code>mma</code> instruction	451
9.7.14.6	Matrix multiply-accumulate operation using <code>mma.sp</code> instruction with sparse matrix A	533
9.7.15	Asynchronous Warpgroup Level Matrix Multiply-Accumulate Instructions	569
9.7.15.1	Warpgroup	570
9.7.15.2	Matrix Shape	570
9.7.15.3	Matrix Data-types	571
9.7.15.4	Async Proxy	572
9.7.15.5	Asynchronous Warpgroup Level Matrix Multiply-Accumulate Operation using <code>wgmma.mma_async</code> instruction	572
9.7.15.6	Asynchronous Warpgroup Level Multiply-and-Accumulate Operation using <code>wgmma.mma_async.sp</code> instruction	604
9.7.15.7	Asynchronous <code>wgmma</code> Proxy Operations	619
9.7.16	TensorCore 5th Generation Family Instructions	623
9.7.16.1	Tensor Memory	623
9.7.16.2	Matrix and Data Movement Shape	623
9.7.16.3	Major-ness supported by Strides	634
9.7.16.4	Matrix Descriptors	643
9.7.16.5	Issue Granularity	651
9.7.16.6	Memory Consistency Model for 5th generation of TensorCore operations	653
9.7.16.7	Tensor Memory Allocation and Management Instructions	659
9.7.16.8	Tensor Memory and Register Load/Store Instructions	661
9.7.16.9	Tensor Memory Data Movement Instructions	669
9.7.16.10	TensorCore 5th Generation Matrix Multiply and accumulate Operations	674
9.7.16.11	TensorCore 5th Generation Specialized Synchronization Operations	738
9.7.16.12	TensorCore 5th Generation Async Synchronization Operations	739
9.7.17	Stack Manipulation Instructions	741
9.7.17.1	Stack Manipulation Instructions: <code>stacksave</code>	741
9.7.17.2	Stack Manipulation Instructions: <code>stackrestore</code>	742
9.7.17.3	Stack Manipulation Instructions: <code>alloca</code>	743
9.7.18	Video Instructions	745
9.7.18.1	Scalar Video Instructions	745
9.7.18.2	SIMD Video Instructions	753
9.7.19	Miscellaneous Instructions	762
9.7.19.1	Miscellaneous Instructions: <code>brkpt</code>	763
9.7.19.2	Miscellaneous Instructions: <code>nanosleep</code>	763

9.7.19.3	Miscellaneous Instructions: <code>pmevent</code>	764
9.7.19.4	Miscellaneous Instructions: <code>trap</code>	765
9.7.19.5	Miscellaneous Instructions: <code>setmaxnreg</code>	766
10	Special Registers	769
10.1	Special Registers: <code>%tid</code>	770
10.2	Special Registers: <code>%ntid</code>	771
10.3	Special Registers: <code>%laneid</code>	772
10.4	Special Registers: <code>%warpid</code>	773
10.5	Special Registers: <code>%nwarpid</code>	774
10.6	Special Registers: <code>%ctaid</code>	774
10.7	Special Registers: <code>%nctaid</code>	775
10.8	Special Registers: <code>%smid</code>	776
10.9	Special Registers: <code>%nsmid</code>	777
10.10	Special Registers: <code>%gridid</code>	778
10.11	Special Registers: <code>%is_explicit_cluster</code>	779
10.12	Special Registers: <code>%clusterid</code>	780
10.13	Special Registers: <code>%nclusterid</code>	781
10.14	Special Registers: <code>%cluster_ctaid</code>	782
10.15	Special Registers: <code>%cluster_nctaid</code>	783
10.16	Special Registers: <code>%cluster_ctarank</code>	784
10.17	Special Registers: <code>%cluster_nctarank</code>	785
10.18	Special Registers: <code>%lanemask_eq</code>	786
10.19	Special Registers: <code>%lanemask_le</code>	786
10.20	Special Registers: <code>%lanemask_lt</code>	787
10.21	Special Registers: <code>%lanemask_ge</code>	788
10.22	Special Registers: <code>%lanemask_gt</code>	789
10.23	Special Registers: <code>%clock,%clock_hi</code>	789
10.24	Special Registers: <code>%clock64</code>	790
10.25	Special Registers: <code>%pm0 ... %pm7</code>	791
10.26	Special Registers: <code>%pm0_64 ... %pm7_64</code>	792
10.27	Special Registers: <code>%envreg<32></code>	793
10.28	Special Registers: <code>%globaltimer,%globaltimer_lo,%globaltimer_hi</code>	794
10.29	Special Registers: <code>%reserved_smem_offset_begin, %reserved_smem_offset_end, %reserved_smem_offset_cap, %reserved_smem_offset_<2></code>	795
10.30	Special Registers: <code>%total_smem_size</code>	796
10.31	Special Registers: <code>%aggr_smem_size</code>	797
10.32	Special Registers: <code>%dynamic_smem_size</code>	798
10.33	Special Registers: <code>%current_graph_exec</code>	799
11	Directives	801
11.1	PTX Module Directives	801
11.1.1	PTX Module Directives: <code>.version</code>	801
11.1.2	PTX Module Directives: <code>.target</code>	802
11.1.3	PTX Module Directives: <code>.address_size</code>	808
11.2	Specifying Kernel Entry Points and Functions	809
11.2.1	Kernel and Function Directives: <code>.entry</code>	809
11.2.2	Kernel and Function Directives: <code>.func</code>	811
11.2.3	Kernel and Function Directives: <code>.alias</code>	814
11.3	Control Flow Directives	815
11.3.1	Control Flow Directives: <code>.branchtargets</code>	815
11.3.2	Control Flow Directives: <code>.calltargets</code>	816
11.3.3	Control Flow Directives: <code>.callprototype</code>	817

11.4	Performance-Tuning Directives	819
11.4.1	Performance-Tuning Directives: <code>.maxnreg</code>	820
11.4.2	Performance-Tuning Directives: <code>.maxntid</code>	820
11.4.3	Performance-Tuning Directives: <code>.reqntid</code>	821
11.4.4	Performance-Tuning Directives: <code>.minnctapersm</code>	822
11.4.5	Performance-Tuning Directives: <code>.maxnctapersm</code> (deprecated)	823
11.4.6	Performance-Tuning Directives: <code>.noreturn</code>	824
11.4.7	Performance-Tuning Directives: <code>.pragma</code>	825
11.4.8	Performance-Tuning Directives: <code>.abi_preserve</code>	826
11.4.9	Performance-Tuning Directives: <code>.abi_preserve_control</code>	827
11.5	Debugging Directives	829
11.5.1	Debugging Directives: <code>@@dwarf</code>	829
11.5.2	Debugging Directives: <code>.section</code>	830
11.5.3	Debugging Directives: <code>.file</code>	832
11.5.4	Debugging Directives: <code>.loc</code>	833
11.6	Linking Directives	835
11.6.1	Linking Directives: <code>.extern</code>	835
11.6.2	Linking Directives: <code>.visible</code>	836
11.6.3	Linking Directives: <code>.weak</code>	836
11.6.4	Linking Directives: <code>.common</code>	837
11.7	Cluster Dimension Directives	838
11.7.1	Cluster Dimension Directives: <code>.reqnctapercluster</code>	838
11.7.2	Cluster Dimension Directives: <code>.explicitcluster</code>	839
11.7.3	Cluster Dimension Directives: <code>.maxclusterrank</code>	840
11.8	Miscellaneous Directives	841
11.8.1	Miscellaneous Directives: <code>.blocksareclusters</code>	841
12	Descriptions of <code>.pragma</code> Strings	843
12.1	Pragma Strings: <code>"nounroll"</code>	843
12.2	Pragma Strings: <code>"used_bytes_mask"</code>	844
12.3	Pragma Strings: <code>"enable_smem_spilling"</code>	845
12.4	Pragma Strings: <code>"frequency"</code>	847
13	Release Notes	849
13.1	Changes in PTX ISA Version 9.1	856
13.2	Changes in PTX ISA Version 9.0	856
13.3	Changes in PTX ISA Version 8.8	857
13.4	Changes in PTX ISA Version 8.7	860
13.5	Changes in PTX ISA Version 8.6	860
13.6	Changes in PTX ISA Version 8.5	862
13.7	Changes in PTX ISA Version 8.4	862
13.8	Changes in PTX ISA Version 8.3	862
13.9	Changes in PTX ISA Version 8.2	863
13.10	Changes in PTX ISA Version 8.1	863
13.11	Changes in PTX ISA Version 8.0	864
13.12	Changes in PTX ISA Version 7.8	865
13.13	Changes in PTX ISA Version 7.7	866
13.14	Changes in PTX ISA Version 7.6	866
13.15	Changes in PTX ISA Version 7.5	867
13.16	Changes in PTX ISA Version 7.4	867
13.17	Changes in PTX ISA Version 7.3	868
13.18	Changes in PTX ISA Version 7.2	868
13.19	Changes in PTX ISA Version 7.1	868
13.20	Changes in PTX ISA Version 7.0	869

13.21	Changes in PTX ISA Version 6.5	870
13.22	Changes in PTX ISA Version 6.4	870
13.23	Changes in PTX ISA Version 6.3	871
13.24	Changes in PTX ISA Version 6.2	872
13.25	Changes in PTX ISA Version 6.1	872
13.26	Changes in PTX ISA Version 6.0	873
13.27	Changes in PTX ISA Version 5.0	874
13.28	Changes in PTX ISA Version 4.3	874
13.29	Changes in PTX ISA Version 4.2	875
13.30	Changes in PTX ISA Version 4.1	875
13.31	Changes in PTX ISA Version 4.0	876
13.32	Changes in PTX ISA Version 3.2	876
13.33	Changes in PTX ISA Version 3.1	877
13.34	Changes in PTX ISA Version 3.0	877
13.35	Changes in PTX ISA Version 2.3	878
13.36	Changes in PTX ISA Version 2.2	879
13.37	Changes in PTX ISA Version 2.1	879
13.38	Changes in PTX ISA Version 2.0	880
14	Notices	883
14.1	Notice	883
14.2	OpenCL	884
14.3	Trademarks	884

List of Tables

1	PTX Directives	24
2	Reserved Instruction Keywords	25
3	Predefined Identifiers	26
4	Operator Precedence	29
5	Constant Expression Evaluation Rules	31
6	State Spaces	33
7	Properties of State Spaces	34
8	Fundamental Type Specifiers	41
9	Operand types for packed floating point instruction type.	44
10	Opaque Type Fields in Unified Texture Mode	46
11	Opaque Type Fields in Independent Texture Mode	47
12	OpenCL 1.0 Channel Data Type Definition	48
13	OpenCL 1.0 Channel Order Definition	49
14	Valid combination of swizzle-atomicity with swizzling-mode	88
15	Convert Instruction Precision and Format Table 1	93
16	Convert Instruction Precision and Format Table 2	93
17	Floating-Point Rounding Modifiers	94
18	Integer Rounding Modifiers	94
19	Cost Estimates for Accessing State-Spaces	95
20	Operation Types	104
21	Scopes	105
22	Operators for Signed Integer, Unsigned Integer, and Bit-Size Types	121
23	Floating-Point Comparison Operators	121
24	Floating-Point Comparison Operators Accepting NaN	121
25	Floating-Point Comparison Operators Testing for NaN	122
26	Type Checking Rules	123
27	Relaxed Type-checking Rules for Source Operands	124
28	Relaxed Type-checking Rules for Destination Operands	125
29	Summary of Floating-Point Instructions	165
30	Cache Operators for Memory Load Instructions	251
31	Cache Operators for Memory Store Instructions	252
32	Cache Eviction Priority Hints for Memory Load and Store Instructions	252
33	Tensormap new_val validity	341
34	Texture, sampler and surface limits	343
36	Shapes for scale matrices depending upon .scale_vec_size qualifier	435
37	Valid combinations of .scale_vec_size and .kind qualifier	435
38	Valid values for various selector components	437
40	Various combinations of swizzling mode, leading dimension and swizzle-atom layout	581
41	Various combinations of .kind and shapes	624
42	Shared memory descriptor layout	645

43	Starting address of repeating pattern for various swizzling modes	645
44	Instruction descriptor format for .kind::tf32, .kind::f16, .kind::f8f6f4 and .kind::i8	646
45	Instruction descriptor format for .kind::mxf8f6f4	647
46	Instruction descriptor format for .kind::mxf4 and .kind::mxf4nvf4	648
47	Zero-Column Mask descriptor layout	649
48	Execution granularity requirements for tcgen05 operations	651
49	Example of deallocating Tensor Memory before CTA exit.	652
50	Example of non-deterministic hang due to incorrect synchronization	652
51	Various-combinations of .num and .shape	664
52	Various-combinations of .num and .shape	666
53	Transpose and Negate operation for various MMA-Kind	675
54	Various combinations of N shape with .cta_group qualifier for 8bit transpose B	676
55	Major-ness for different matrices	676
56	Valid Combinations of Type-Size, Major-ness and Swizzling	676
57	Layout for swizzle atoms	687
58	Valid combinations of scale_vectorsize and shapes	693
59	Valid combinations of scale_vectorsize with types and MMA-Kind	697
64	Architecture Families	803
65	PTX Release History	849
66	Arch-specific/ Family-specific PTX Features Release History	852
67	List of features promoted to family-specific architecture	859

List of Figures

1	Grid with CTAs	15
2	Grid with clusters	16
3	Memory Hierarchy	17
4	Hardware Model	20
5	Layout for .b4x16_p64	56
6	Layout for .b6x16_p32	57
7	Layout for .b6p2x16	57
8	Tiled mode bounding box, tensor size and traversal stride	59
9	Out of boundary access	60
10	tiled::scatter4/tiled::gather4 mode bounding box example	61
11	im2col mode bounding box example 1	63
12	im2col mode bounding box example 2	64
13	im2col mode example 1	66
14	im2col mode example 2	67
15	im2col mode traversal stride example	68
16	im2col::w and im2col::w::128 modes example	69
17	im2col::w and im2col::w::128 modes first element outside Bounding Box example	70
18	tensor copy operation with im2col::w mode example	71
19	tensor copy operation with im2col::w::128 mode example	72

20	tensor copy operation to buffer 1 of Example 1	74
21	tensor copy operation to buffer 2 of Example 1	74
22	tensor copy operation to buffer 1 of Example 2	75
23	tensor copy operation to buffer 2 of Example 2	76
24	tensor copy operation to buffer 3 of Example 2	76
25	32-byte swizzle mode example	78
26	32-byte swizzle mode fragments	79
27	32-byte swizzle mode destination data layout	80
28	64-byte swizzle mode example	81
29	64-byte swizzle mode source data layout	82
30	64-byte swizzle mode destination data layout	83
31	96-byte swizzle mode example	83
32	128-byte swizzle mode example	84
33	128-byte swizzle mode source data layout	85
34	128-byte swizzle mode destination data layout	86
35	128-byte swizzle mode example with 32-byte atomicity	86
36	128-byte swizzle mode example with 32-byte atomicity with 8-byte flip	87
37	128-byte swizzle mode example with 64-byte atomicity	87
38	Random bits layout for .rs rounding with .f16 destination type	297
39	Random bits layout for .rs rounding with .bf16 destination type	297
40	Random bits layout for .rs rounding with .e2m1x4 destination type	297
41	Random bits layout for .rs rounding with .e5m2x4/.e4m3x4/.e3m2x4/.e2m3x4 destination type	298
42	mma with block scaling of .scale_vec : 2X	434
43	Selection of set of values for SF_A based on thread-id-a	436
44	Selection of set of values for SF_B based on thread-id-b	436
45	Selection of set of values for SF_A or SF_B based on byte-id-a or byte-id-b	437
46	MMA .m8n8k4 fragment layout for row-major matrix A with .f16 type	452
47	MMA .m8n8k4 fragment layout for column-major matrix A with .f16 type	453
48	MMA .m8n8k4 fragment layout for row-major matrix B with .f16 type	454
49	MMA .m8n8k4 fragment layout for column-major matrix B with .f16 type	454
50	MMA .m8n8k4 fragment layout for matrix C/D with .ctype = .f16	455
51	MMA .m8n8k4 computation 1 and 2 fragment layout for matrix C/D with .ctype = .f32	456
52	MMA .m8n8k4 computation 3 and 4 fragment layout for matrix C/D with .ctype = .f32	456
53	MMA .m8n8k4 fragment layout for matrix A with .f64 type	457
54	MMA .m8n8k4 fragment layout for matrix B with .f64 type	458
55	MMA .m8n8k4 fragment layout for accumulator matrix C/D with .f64 type	459
56	MMA .m8n8k16 fragment layout for matrix A with .u8/.s8 type	460
57	MMA .m8n8k16 fragment layout for matrix B with .u8/.s8 type	461
58	MMA .m8n8k16 fragment layout for accumulator matrix C/D with .s32 type	462
59	MMA .m8n8k32 fragment layout for matrix A with .u4/.s4 type	463
60	MMA .m8n8k32 fragment layout for matrix B with .u4/.s4 type	464
61	MMA .m8n8k32 fragment layout for accumulator matrix C/D with .s32 type	465
62	MMA .m8n8k128 fragment layout for matrix A with .b1 type.	466
63	MMA .m8n8k128 fragment layout for matrix B with .b1 type.	467
64	MMA .m8n8k128 fragment layout for accumulator matrix C/D with .s32 type	468
65	MMA .m16n8k4 fragment layout for matrix A with .tf32 type.	469
66	MMA .m16n8k4 fragment layout for matrix A with .f64 type.	471
67	MMA .m16n8k4 fragment layout for matrix B with .tf32 type.	472
68	MMA .m16n8k4 fragment layout for matrix B with .f64 type.	472
69	MMA .m16n8k4 fragment layout for accumulator matrix C/D with .f32 type.	473
70	MMA .m16n8k4 fragment layout for accumulator matrix C/D with .f64 type.	474
71	MMA .m16n8k8 fragment layout for matrix A with .f16 / .bf16 type.	475

72	MMA .m16n8k8 fragment layout for matrix A with .tf32 type.	476
73	MMA .m16n8k8 fragment layout for matrix A with .f64 type.	477
74	MMA .m16n8k8 fragment layout for matrix B with .f16 / .bf16 type.	478
75	MMA .m16n8k8 fragment layout for matrix B with .tf32 type.	479
76	MMA .m16n8k8 fragment layout for matrix B with .f64 type.	480
77	MMA .m16n8k8 fragment layout for accumulator matrix C/D with .f16x2/.f32 type. . .	481
78	MMA .m16n8k8 fragment layout for accumulator matrix C/D with .f64 type.	482
79	MMA .m16n8k16 fragment layout for matrix A with .f16 / .bf16 type.	483
80	MMA .m16n8k16 fragment layout for matrix A with .f64 type.	484
81	MMA .m16n8k16 fragment layout for matrix B with .f16 / .bf16 type.	485
82	MMA .m16n8k16 fragment layout for matrix B with .f64 type.	486
83	MMA .m16n8k16 fragment layout for accumulator matrix matrix C/D.	487
84	MMA .m16n8k16 fragment layout for matrix A with .u8 / .s8 type.	488
85	MMA .m16n8k16 fragment layout for matrix B with .u8 / .s8 type.	490
86	MMA .m16n8k16 fragment layout for accumulator matrix C/D with .s32 type.	491
87	MMA .m16n8k32 fragment layout for matrix A with .u4 / .s4 type.	492
88	MMA .m16n8k32 fragment layout for matrix A with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type.	493
89	MMA .m16n8k32 fragment layout for matrix B with .u4 / .s4 type.	494
90	MMA .m16n8k32 fragment layout for rows 0–15 of matrix B with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type.	496
91	MMA .m16n8k32 fragment layout for rows 16–31 of matrix B with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type.	497
92	MMA .m16n8k32 fragment layout for accumulator matrix C/D with .s32 / .f32 / .f16 type.	498
93	MMA .m16n8k64 fragment layout for matrix A with .u4 / .s4 / .e2m1 type.	499
94	MMA .m16n8k64 fragment layout for rows 0–31 of matrix B with .u4 / .s4 / .e2m1 type.	500
95	MMA .m16n8k64 fragment layout for rows 32–63 of matrix B with .u4 / .s4 / .e2m1 type.	501
96	MMA .m16n8k64 fragment layout for accumulator matrix C/D with .s32 / .f32 type. . .	502
97	MMA .m16n8k128 fragment layout for matrix A with .b1 type.	503
98	MMA .m16n8k128 fragment layout for matrix B with .b1 type.	505
99	MMA .m16n8k128 fragment layout for accumulator matrix C/D with .s32 type.	506
100	MMA .m16n8k256 fragment layout for matrix A with .b1 type.	506
101	MMA .m16n8k256 fragment layout for rows 0–127 of matrix B with .b1 type.	508
102	MMA .m16n8k256 fragment layout for rows 128–255 of matrix B with .b1 type.	509
103	MMA .m16n8k256 fragment layout for accumulator matrix C/D with .s32 type.	510
104	ldmatrix fragment layout for one 8x8 Matrix with 16-bit elements	523
105	ldmatrix fragment layout for one 16x16 matrix with 8-bit elements	524
106	ldmatrix fragment layout for one 8x16 matrix with 8-bit elements containing 4-bit/6-bit data	524
107	stmatrix fragment layout for one 8x8 matrix with 16-bit elements	528
108	stmatrix fragment layout for one 16x8 matrix with 8 bit elements	528
109	movmatrix source matrix fragment layout	531
110	movmatrix result matrix fragment layout	532
111	Sparse MMA storage example	533
112	Sparse MMA metadata example for .f16/.bf16 type.	534
113	Sparse MMA metadata example for .tf32 type.	535
114	Sparse MMA metadata example for .u8/.s8 type.	536
115	Sparse MMA metadata example for .u4/.s4 type.	536
116	Sparse MMA metadata example for .e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	537
117	Sparse MMA metadata example for .e2m1 type with .kind::mx4 or .kind::mx4nvf4.	538
118	Sparse MMA .m16n8k16 fragment layout for matrix A with .f16/.bf16 type.	539
119	Sparse MMA .m16n8k16 metadata layout for .f16/.bf16 type.	540
120	Sparse MMA .m16n8k32 fragment layout for matrix A with .f16/.bf16 type.	541

121	Sparse MMA .m16n8k32 fragment layout for matrix B with .f16/.bf16 type.	542
122	Sparse MMA .m16n8k32 metadata layout for .f16/.bf16 type.	543
123	Sparse MMA .m16n8k16 fragment layout for matrix A with .tf32 type.	544
124	Sparse MMA .m16n8k16 fragment layout for matrix B with .tf32 type.	545
125	Sparse MMA .m16n8k16 metadata layout for .tf32 type.	546
126	Sparse MMA .m16n8k8 fragment layout for matrix A with .tf32 type.	547
127	Sparse MMA .m16n8k8 metadata layout for .tf32 type.	548
128	Sparse MMA .m16n8k32 fragment layout for matrix A with .u8/.s8 type.	549
129	Sparse MMA .m16n8k32 metadata layout for .u8/.s8 type.	549
130	Sparse MMA .m16n8k64 fragment layout for columns 0–31 of matrix A with .u8/.s8/. e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	550
131	Sparse MMA .m16n8k64 fragment layout for columns 32–63 of matrix A with .u8/.s8/. e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	551
132	Sparse MMA .m16n8k64 fragment layout for rows 0–15 of matrix B with .u8/.s8/. e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	552
133	Sparse MMA .m16n8k64 fragment layout for rows 16–31 of matrix B with .u8/.s8/. e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	552
134	Sparse MMA .m16n8k64 fragment layout for rows 32–47 of matrix B with .u8/.s8/. e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	553
135	Sparse MMA .m16n8k64 fragment layout for rows 48–63 of matrix B with .u8/.s8/. e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.	553
136	Sparse MMA .m16n8k64 metadata layout for columns 0–31 for .u8/.s8/.e4m3/.e5m2/. e3m2/.e2m3/.e2m1 type.	554
137	Sparse MMA .m16n8k64 metadata layout for columns 32–63 for .u8/.s8/.e4m3/. e5m2/.e3m2/.e2m3/.e2m1 type.	555
138	Sparse MMA .m16n8k64 fragment layout for matrix A with .u4/.s4 type.	555
139	Sparse MMA .m16n8k64 metadata layout for .u4/.s4 type.	556
140	Sparse MMA .m16n8k128 fragment layout for columns 0–63 of matrix A with .u4/.s4/. e2m1 type.	557
141	Sparse MMA .m16n8k128 fragment layout for columns 64–127 of matrix A with .u4/. s4/.e2m1 type.	558
142	Sparse MMA .m16n8k128 fragment layout for rows 0–31 of matrix B with .u4/.s4/. e2m1 type.	559
143	Sparse MMA .m16n8k128 fragment layout for rows 32–63 of matrix B with .u4/.s4/. e2m1 type.	559
144	Sparse MMA .m16n8k128 fragment layout for rows 64–95 of matrix B with .u4/.s4/. e2m1 type.	560
145	Sparse MMA .m16n8k128 fragment layout for rows 96–127 of matrix B with .u4/.s4/. e2m1 type.	560
146	Sparse MMA .m16n8k128 metadata layout for columns 0–63 for .u4/.s4/.e2m1 type. .	561
147	Sparse MMA .m16n8k128 metadata layout for columns 64–127 for .u4/.s4/.e2m1 type.	562
148	WGMMA .m64nNk16 register fragment layout for matrix A.	573
149	WGMMA .m64nNk16 register fragment layout for accumulator matrix D.	574
150	WGMMA .m64nNk8 register fragment layout for matrix A.	575
151	WGMMA .m64nNk8 register fragment layout for accumulator matrix D.	576
152	WGMMA .m64nNk32 register fragment layout for matrix A.	577
153	WGMMA .m64nNk32 register fragment layout for accumulator matrix D.	578
154	WGMMA .m64nNk256 register fragment layout for matrix A.	579
155	WGMMA .m64nNk256 register fragment layout for accumulator matrix D.	580
156	MN major 128B swizzling	583
157	K major 128B swizzling	584
158	MN major 64B swizzling	584
159	K major 64B swizzling	585
160	MN major 32B swizzling	586

161	K major 32B swizzling	587
162	MN major interleaved	587
163	K major interleaved	588
164	K major	588
165	MN major	589
166	K major, no-swizzling and tf32 type	591
167	K major, 32B swizzling and tf32 type	592
168	MN major, no-swizzling and bf16 type	594
169	MN major, 32B swizzling and bf16 type	595
170	MN major, 64B swizzling and bf16 type	597
171	Sparse WGMMA metadata example for .f16/.bf16 type.	605
172	Sparse WGMMA metadata example for .tf32 type.	606
173	Sparse WGMMA metadata example for .e4m3/.e5m2 type.	607
174	Sparse WGMMA metadata example for .u8/.s8 type.	608
175	Sparse WGMMA .m64nNk32 fragment layout for matrix A with .f16/.bf16 type.	609
176	Sparse WGMMA .m64nNk32 metadata layout for .f16/.bf16 type.	610
177	Sparse WGMMA .m64nNk16 fragment layout for matrix A with .tf32 type.	611
178	Sparse WGMMA .m64nNk16 metadata layout for .tf32 type.	612
179	Sparse WGMMA .m64nNk64 fragment layout for matrix A with .e4m3/ .e5m2/ .s8/ .u8 type.	613
180	Sparse WGMMA .m64nNk64 metadata layout for .e4m3/ .e5m2/ .s8/ .u8 type for columns 0–31	614
181	Sparse WGMMA .m64nNk64 metadata layout for .e4m3/ .e5m2/ .s8/ .u8 type for columns 32–63	615
182	Tensor Memory Layout and Addressing	624
183	Matrix Fragment for shape .32x32b	629
184	Matrix Fragment for shape .16x64b	631
185	Matrix Fragment for shape .16x128b	632
186	Matrix Fragment for shape .16x256b	633
187	Matrix Fragment for shape .16x32bx2	635
188	K major, no-swizzling and tf32 type	638
189	K major, 32B swizzling and tf32 type	639
190	MN major, no-swizzling and bf16 type	641
191	MN major, 32B swizzling and bf16 type	642
192	MN major, 64B swizzling and bf16 type	644
193	Pack/Unpack operations for tcgen05 ld/st	662
194	Decompression from 4-bit to 8-bit	669
195	Individual decompression from 4-bit to 8-bit	670
196	Decompression from 6-bit to 8-bit	670
197	Individual decompression from 6-bit to 8-bit for E3M2 type	670
198	Individual decompression from 6-bit to 8-bit for E2M3 type	671
199	4-bit packing format with type E2M1	677
200	6-bit packing format with type E3M2	678
201	6-bit packing format with type E2M3	678
202	4-bit packing format	679
203	6-bit packing format	679
204	4-bit packing format with type E2M1	679
205	Layout organization for M = 256	681
206	Addresses to use in tcgen05.ld / tcgen05.st	681
207	Layout organization for M = 128 + .cta_group::2 + Dense A matrix	682
208	Addresses to use in tcgen05.ld / tcgen05.st	682
209	Layout organization for M = 128 + .cta_group::2 + Sparse A matrix	683
210	Addresses to use in tcgen05.ld / tcgen05.st	683
211	Layout organization for M = 128 + .cta_group::1	684

212	Addresses to use in tcgen05.ld / tcgen05.st	684
213	Layout organization for M = 64 + .ws mode	685
214	Addresses to use in tcgen05.ld / tcgen05.st	685
215	Layout organization for M = 64 + non .ws mode	685
216	Addresses to use in tcgen05.ld / tcgen05.st	686
217	Layout organization for M = 32	686
218	Addresses to use in tcgen05.ld / tcgen05.st	686
219	MN major 128B swizzling with 32B atomicity	688
220	MN major 128B swizzling	689
221	K major 128B swizzling	690
222	MN major 64B swizzling	691
223	K major 64B swizzling	692
224	MN major 32B swizzling	693
225	K major 32B swizzling	694
226	MN major no-swizzling mode	694
227	K major no-swizzling mode	695
228	K major	695
229	MN major	696
230	tcgen05.mma with block scaling of scale_vec::2X	697
231	Layout of scale factor A matrix with scale_vec::1X/block32 with K=32/K=64	698
232	Layout of scale factor A matrix with scale_vec::2X/block32 with K=64/K=128	699
233	Layout of scale factor A matrix with scale_vec::4X/block16 with K=64/K=128	700
234	Layout of scale factor A matrix with block32 with K=96 with SFA_ID=00	700
235	Layout of scale factor A matrix with block32 with K=96 with SFA_ID=01	701
236	Layout of scale factor A matrix with block32 with K=96 with SFA_ID=10	701
237	Layout of scale factor A matrix with block32 with K=96 with SFA_ID=11	702
238	Layout of scale factor A matrix with block16 with K=96 with SFA_ID=00	702
239	Layout of scale factor A matrix with block16 with K=96 with SFA_ID=10	703
240	Layout of scale factor B matrix with scale_vec::1X/block32 with K=32/K=64	704
241	Layout of scale factor B matrix with scale_vec::2X/block32 with K=64/K=128	705
242	Layout of scale factor B matrix with scale_vec::4X/block16 with K=64/K=128	705
243	Layout of scale factor B matrix with block32 with K=96 and N<=128 with SFA_ID=00	706
244	Layout of scale factor B matrix with block32 with K=96 and N<=128 with SFA_ID=01	707
245	Layout of scale factor B matrix with block32 with K=96 and N<=128 with SFA_ID=10	707
246	Layout of scale factor B matrix with block32 with K=96 and N<=128 with SFA_ID=11	708
247	Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=00	708
248	Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=01	709
249	Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=10	709
250	Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=10	710
251	Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=11	710
252	Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=11	711
253	Layout of scale factor B matrix with block16 with K=96 and N<=128 with SFA_ID=00	711
254	Layout of scale factor B matrix with block16 with K=96 and N<=128 with SFA_ID=10	712
255	Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=00	712
256	Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=00	713
257	Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=10	713
258	Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=10	714
259	Sparse tcgen05.mma metadata example for tf32 kind	715
260	Sparse tcgen05.mma metadata example for f16/f8f6f4/mxf8f6f4 kind	716
261	Sparse tcgen05.mma metadata example for mxf4 kind	717
262	Sparsity Metadata Layout for M = 64 for .kind::f16	718
263	Sparsity Metadata Layout for M = 128 / M = 256 for .kind::f16	719
264	Sparsity Metadata Layout for M = 64 for .kind::tf32	719
265	Sparsity Metadata Layout for M = 128 / M = 256 for .kind::tf32	720

266	Sparsity Metadata Layout for M = 64 for .kind::f8f6f4, .kind::mxf8f6f4, . kind::i8, .kind::mxf4, .kind::mxf4nvf4	720
267	Sparsity Metadata Layout for M = 128 / M = 256 for .kind::f8f6f4, . kind::mxf8f6f4, .kind::i8, .kind::mxf4, .kind::mxf4nvf4	721

Chapter 1. Introduction

This document describes PTX, a low-level *parallel thread execution* virtual machine and instruction set architecture (ISA). PTX exposes the GPU as a data-parallel computing *device*.

1.1. Scalable Data-Parallel Computing using GPUs

Driven by the insatiable market demand for real-time, high-definition 3D graphics, the programmable GPU has evolved into a highly parallel, multithreaded, many-core processor with tremendous computational horsepower and very high memory bandwidth. The GPU is especially well-suited to address problems that can be expressed as data-parallel computations - the same program is executed on many data elements in parallel - with high arithmetic intensity - the ratio of arithmetic operations to memory operations. Because the same program is executed for each data element, there is a lower requirement for sophisticated flow control; and because it is executed on many data elements and has high arithmetic intensity, the memory access latency can be hidden with calculations instead of big data caches.

Data-parallel processing maps data elements to parallel processing threads. Many applications that process large data sets can use a data-parallel programming model to speed up the computations. In 3D rendering large sets of pixels and vertices are mapped to parallel threads. Similarly, image and media processing applications such as post-processing of rendered images, video encoding and decoding, image scaling, stereo vision, and pattern recognition can map image blocks and pixels to parallel processing threads. In fact, many algorithms outside the field of image rendering and processing are accelerated by data-parallel processing, from general signal processing or physics simulation to computational finance or computational biology.

PTX defines a virtual machine and ISA for general purpose parallel thread execution. PTX programs are translated at install time to the target hardware instruction set. The PTX-to-GPU translator and driver enable NVIDIA GPUs to be used as programmable parallel computers.

1.2. Goals of PTX

PTX provides a stable programming model and instruction set for general purpose parallel programming. It is designed to be efficient on NVIDIA GPUs supporting the computation features defined by the NVIDIA Tesla architecture. High level language compilers for languages such as CUDA and C/C++ generate PTX instructions, which are optimized for and translated to native target-architecture instructions.

The goals for PTX include the following:

- ▶ Provide a stable ISA that spans multiple GPU generations.
- ▶ Achieve performance in compiled applications comparable to native GPU performance.
- ▶ Provide a machine-independent ISA for C/C++ and other compilers to target.
- ▶ Provide a code distribution ISA for application and middleware developers.
- ▶ Provide a common source-level ISA for optimizing code generators and translators, which map PTX to specific target machines.
- ▶ Facilitate hand-coding of libraries, performance kernels, and architecture tests.
- ▶ Provide a scalable programming model that spans GPU sizes from a single unit to many parallel units.

1.3. PTX ISA Version 9.1

PTX ISA version 9.1 introduces the following new features:

- ▶ Adds support for `.volatile` qualifier with `.local` state space for `ld` and `st` instructions.
- ▶ Adds support for `.f16x2` and `.bf16x2` source types for `cvt` instruction with destination types `.e2m1x2`, `.e2m3x2`, `.e3m2x2`, `.e4m3x2`, `.e5m2x2`.
- ▶ Adds support for `.scale_vec::4X` with `.ue8m0` as `.stype` with `.kind::mxf4nvf4` for `mma/mma.sp` instructions.
- ▶ Adds support for `.s2f6x2` instruction type for `cvt` instruction.
- ▶ Adds support for `multimem.cp.async.bulk` and `multimem.cp.reduce.async.bulk` instructions.

1.4. Document Structure

The information in this document is organized into the following Chapters:

- ▶ *Programming Model* outlines the programming model.
- ▶ *PTX Machine Model* gives an overview of the PTX virtual machine model.
- ▶ *Syntax* describes the basic syntax of the PTX language.
- ▶ *State Spaces, Types, and Variables* describes state spaces, types, and variable declarations.
- ▶ *Instruction Operands* describes instruction operands.

- ▶ *Abstracting the ABI* describes the function and call syntax, calling convention, and PTX support for abstracting the *Application Binary Interface (ABI)*.
- ▶ *Instruction Set* describes the instruction set.
- ▶ *Special Registers* lists special registers.
- ▶ *Directives* lists the assembly directives supported in PTX.
- ▶ *Release Notes* provides release notes for PTX ISA versions 2.x and beyond.

References

- ▶ 754-2008 IEEE Standard for Floating-Point Arithmetic. ISBN 978-0-7381-5752-8, 2008.
<http://ieeexplore.ieee.org/servlet/opac?punumber=4610933>
- ▶ The OpenCL Specification, Version: 1.1, Document Revision: 44, June 1, 2011.
<http://www.khronos.org/registry/cl/specs/opencl-1.1.pdf>
- ▶ CUDA Programming Guide.
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- ▶ CUDA Dynamic Parallelism Programming Guide.
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-dynamic-parallelism>
- ▶ CUDA Atomicity Requirements.
https://nvidia.github.io/cccl/libcudacxx/extended_api/memory_model.html#atomicity
- ▶ PTX Writers Guide to Interoperability.
<https://docs.nvidia.com/cuda/ptx-writers-guide-to-interoperability/index.html>

Chapter 2. Programming Model

2.1. A Highly Multithreaded Coprocessor

The GPU is a compute device capable of executing a very large number of threads in parallel. It operates as a coprocessor to the main CPU, or host: In other words, data-parallel, compute-intensive portions of applications running on the host are off-loaded onto the device.

More precisely, a portion of an application that is executed many times, but independently on different data, can be isolated into a kernel function that is executed on the GPU as many different threads. To that effect, such a function is compiled to the PTX instruction set and the resulting kernel is translated at install time to the target GPU instruction set.

2.2. Thread Hierarchy

The batch of threads that executes a kernel is organized as a grid. A grid consists of either cooperative thread arrays or clusters of cooperative thread arrays as described in this section and illustrated in [Figure 1](#) and [Figure 2](#). *Cooperative thread arrays (CTAs)* implement CUDA thread blocks and clusters implement CUDA thread block clusters.

2.2.1. Cooperative Thread Arrays

The *Parallel Thread Execution (PTX)* programming model is explicitly parallel: a PTX program specifies the execution of a given thread of a parallel thread array. A *cooperative thread array*, or CTA, is an array of threads that execute a kernel concurrently or in parallel.

Threads within a CTA can communicate with each other. To coordinate the communication of the threads within the CTA, one can specify synchronization points where threads wait until all threads in the CTA have arrived.

Each thread has a unique thread identifier within the CTA. Programs use a data parallel decomposition to partition inputs, work, and results across the threads of the CTA. Each CTA thread uses its thread identifier to determine its assigned role, assign specific input and output positions, compute addresses, and select work to perform. The thread identifier is a three-element vector `tid`, (with elements `tid.x`, `tid.y`, and `tid.z`) that specifies the thread's position within a 1D, 2D, or 3D CTA. Each thread identifier component ranges from zero up to the number of thread ids in that CTA dimension.

Each CTA has a 1D, 2D, or 3D shape specified by a three-element vector `ntid` (with elements `ntid.x`, `ntid.y`, and `ntid.z`). The vector `ntid` specifies the number of threads in each CTA dimension.

Threads within a CTA execute in SIMT (single-instruction, multiple-thread) fashion in groups called *warps*. A *warp* is a maximal subset of threads from a single CTA, such that the threads execute the same instructions at the same time. Threads within a warp are sequentially numbered. The warp size is a machine-dependent constant. Typically, a warp has 32 threads. Some applications may be able to maximize performance with knowledge of the warp size, so PTX includes a run-time immediate constant, `WARP_SZ`, which may be used in any instruction where an immediate operand is allowed.

2.2.2. Cluster of Cooperative Thread Arrays

Cluster is a group of CTAs that run concurrently or in parallel and can synchronize and communicate with each other via shared memory. The executing CTA has to make sure that the shared memory of the peer CTA exists before communicating with it via shared memory and the peer CTA hasn't exited before completing the shared memory operation.

Threads within the different CTAs in a cluster can synchronize and communicate with each other via shared memory. Cluster-wide barriers can be used to synchronize all the threads within the cluster. Each CTA in a cluster has a unique CTA identifier within its cluster (*cluster_ctaid*). Each cluster of CTAs has 1D, 2D or 3D shape specified by the parameter *cluster_nctaid*. Each CTA in the cluster also has a unique CTA identifier (*cluster_ctarank*) across all dimensions. The total number of CTAs across all the dimensions in the cluster is specified by *cluster_nctarank*. Threads may read and use these values through predefined, read-only special registers `%cluster_ctaid`, `%cluster_nctaid`, `%cluster_ctarank`, `%cluster_nctarank`.

Cluster level is applicable only on target architecture `sm_90` or higher. Specifying cluster level during launch time is optional. If the user specifies the cluster dimensions at launch time then it will be treated as explicit cluster launch, otherwise it will be treated as implicit cluster launch with default dimension 1x1x1. PTX provides read-only special register `%is_explicit_cluster` to differentiate between explicit and implicit cluster launch.

2.2.3. Grid of Clusters

There is a maximum number of threads that a CTA can contain and a maximum number of CTAs that a cluster can contain. However, clusters with CTAs that execute the same kernel can be batched together into a grid of clusters, so that the total number of threads that can be launched in a single kernel invocation is very large. This comes at the expense of reduced thread communication and synchronization, because threads in different clusters cannot communicate and synchronize with each other.

Each cluster has a unique cluster identifier (*clusterid*) within a grid of clusters. Each grid of clusters has a 1D, 2D, or 3D shape specified by the parameter *nclusterid*. Each grid also has a unique temporal grid identifier (*gridid*). Threads may read and use these values through predefined, read-only special registers `%tid`, `%ntid`, `%clusterid`, `%nclusterid`, and `%gridid`.

Each CTA has a unique identifier (*ctaid*) within a grid. Each grid of CTAs has 1D, 2D, or 3D shape specified by the parameter *nctaid*. Thread may use and read these values through predefined, read-only special registers `%ctaid` and `%nctaid`.

Each kernel is executed as a batch of threads organized as a grid of clusters consisting of CTAs where cluster is optional level and is applicable only for target architectures `sm_90` and higher. [Figure 1](#) shows a grid consisting of CTAs and [Figure 2](#) shows a grid consisting of clusters.

Grids may be launched with dependencies between one another - a grid may be a dependent grid and/or a prerequisite grid. To understand how grid dependencies may be defined, refer to the section on *CUDA Graphs* in the *Cuda Programming Guide*.

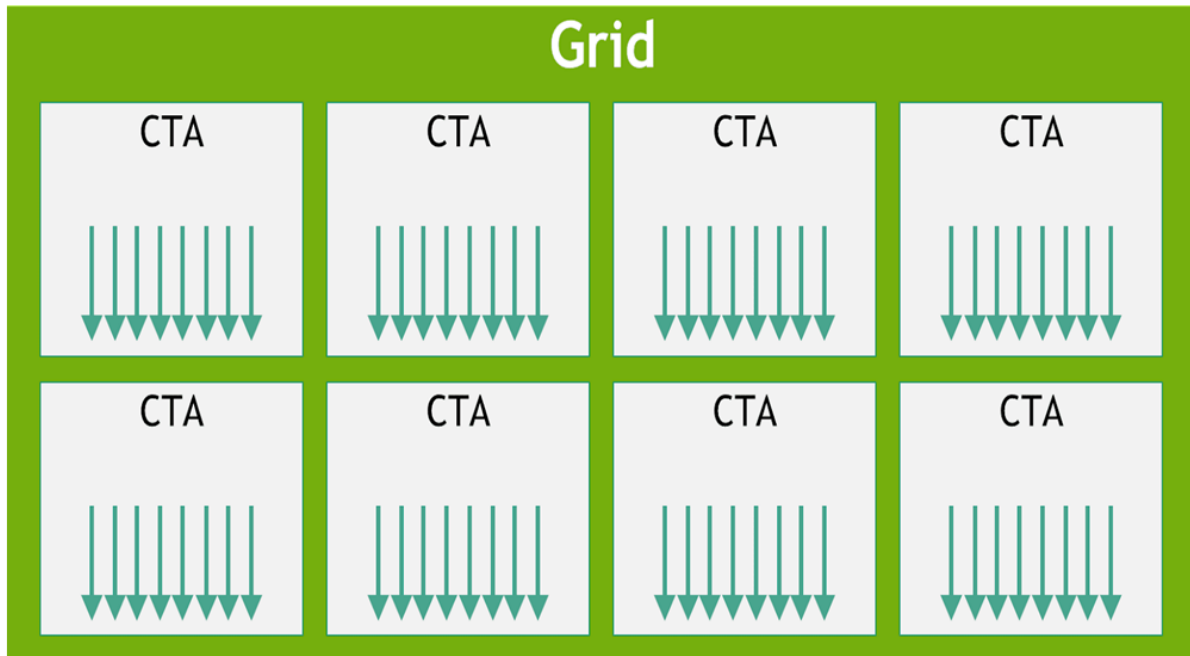


Figure 1: Grid with CTAs

2.3. Memory Hierarchy

PTX threads may access data from multiple state spaces during their execution as illustrated by [Figure 3](#) where cluster level is introduced from target architecture `sm_90` onwards. Each thread has a private local memory. Each thread block (CTA) has a shared memory visible to all threads of the block and to all active blocks in the cluster and with the same lifetime as the block. Finally, all threads have access to the same global memory.

There are additional state spaces accessible by all threads: the constant, param, texture, and surface state spaces. Constant and texture memory are read-only; surface memory is readable and writable. The global, constant, param, texture, and surface state spaces are optimized for different memory usages. For example, texture memory offers different addressing modes as well as data filtering for specific data formats. Note that texture and surface memory is cached, and within the same kernel call, the cache is not kept coherent with respect to global memory writes and surface memory writes, so any texture fetch or surface read to an address that has been written to via a global or a surface write in the same kernel call returns undefined data. In other words, a thread can safely read some texture or surface memory location only if this memory location has been updated by a previous kernel call or memory copy, but not if it has been previously updated by the same thread or another thread from the same kernel call.

The global, constant, and texture state spaces are persistent across kernel launches by the same application.

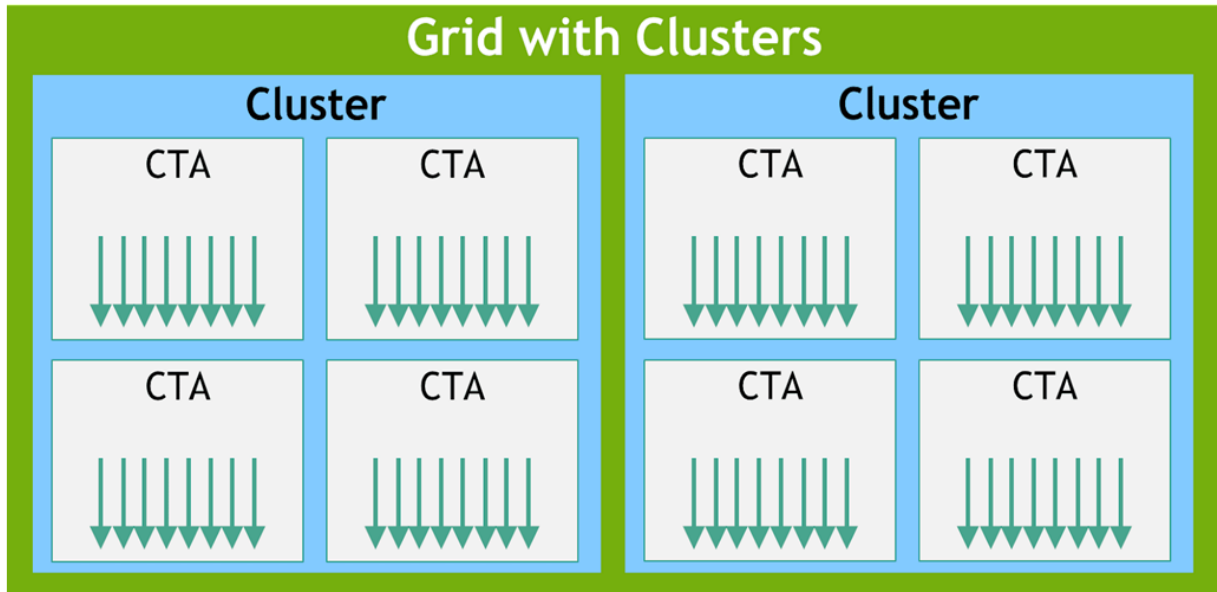


Figure 2: Grid with clusters

A cluster is a set of cooperative thread arrays (CTAs) where a CTA is a set of concurrent threads that execute the same kernel program. A grid is a set of clusters consisting of CTAs that execute independently.

Both the host and the device maintain their own local memory, referred to as *host memory* and *device memory*, respectively. The device memory may be mapped and read or written by the host, or, for more efficient transfer, copied from the host memory through optimized API calls that utilize the device's high-performance *Direct Memory Access (DMA)* engine.

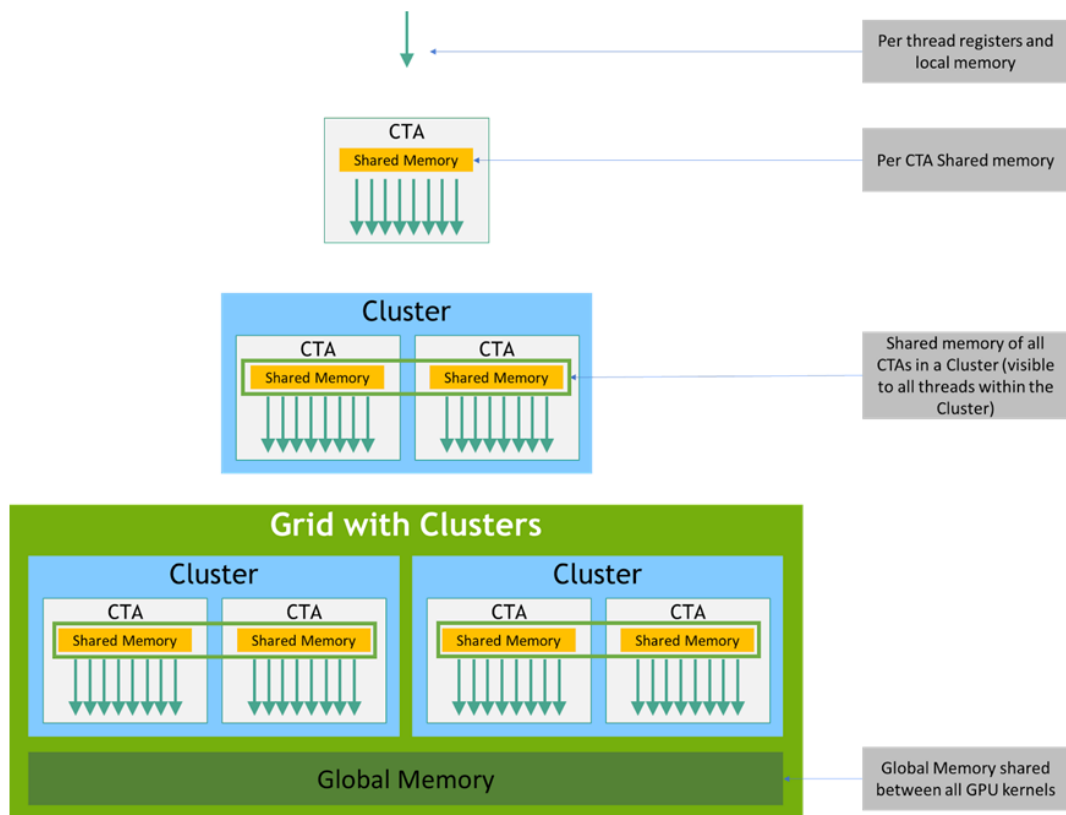


Figure 3: Memory Hierarchy

Chapter 3. PTX Machine Model

3.1. A Set of SIMT Multiprocessors

The NVIDIA GPU architecture is built around a scalable array of multithreaded *Streaming Multiprocessors (SMs)*. When a host program invokes a kernel grid, the blocks of the grid are enumerated and distributed to multiprocessors with available execution capacity. The threads of a thread block execute concurrently on one multiprocessor. As thread blocks terminate, new blocks are launched on the vacated multiprocessors.

A multiprocessor consists of multiple *Scalar Processor (SP)* cores, a multithreaded instruction unit, and on-chip shared memory. The multiprocessor creates, manages, and executes concurrent threads in hardware with zero scheduling overhead. It implements a single-instruction barrier synchronization. Fast barrier synchronization together with lightweight thread creation and zero-overhead thread scheduling efficiently support very fine-grained parallelism, allowing, for example, a low granularity decomposition of problems by assigning one thread to each data element (such as a pixel in an image, a voxel in a volume, a cell in a grid-based computation).

To manage hundreds of threads running several different programs, the multiprocessor employs an architecture we call *SIMT (single-instruction, multiple-thread)*. The multiprocessor maps each thread to one scalar processor core, and each scalar thread executes independently with its own instruction address and register state. The multiprocessor SIMT unit creates, manages, schedules, and executes threads in groups of parallel threads called *warps*. (This term originates from weaving, the first parallel thread technology.) Individual threads composing a SIMT warp start together at the same program address but are otherwise free to branch and execute independently.

When a multiprocessor is given one or more thread blocks to execute, it splits them into warps that get scheduled by the SIMT unit. The way a block is split into warps is always the same; each warp contains threads of consecutive, increasing thread IDs with the first warp containing thread 0.

At every instruction issue time, the SIMT unit selects a warp that is ready to execute and issues the next instruction to the active threads of the warp. A warp executes one common instruction at a time, so full efficiency is realized when all threads of a warp agree on their execution path. If threads of a warp diverge via a data-dependent conditional branch, the warp serially executes each branch path taken, disabling threads that are not on that path, and when all paths complete, the threads converge back to the same execution path. Branch divergence occurs only within a warp; different warps execute independently regardless of whether they are executing common or disjointed code paths.

SIMT architecture is akin to SIMD (Single Instruction, Multiple Data) vector organizations in that a single instruction controls multiple processing elements. A key difference is that SIMD vector organizations expose the SIMD width to the software, whereas SIMT instructions specify the execution and branching behavior of a single thread. In contrast with SIMD vector machines, SIMT enables programmers to write thread-level parallel code for independent, scalar threads, as well as data-parallel code for coordinated threads. For the purposes of correctness, the programmer can essentially ignore the

SIMT behavior; however, substantial performance improvements can be realized by taking care that the code seldom requires threads in a warp to diverge. In practice, this is analogous to the role of cache lines in traditional code: Cache line size can be safely ignored when designing for correctness but must be considered in the code structure when designing for peak performance. Vector architectures, on the other hand, require the software to coalesce loads into vectors and manage divergence manually.

How many blocks a multiprocessor can process at once depends on how many registers per thread and how much shared memory per block are required for a given kernel since the multiprocessor's registers and shared memory are split among all the threads of the batch of blocks. If there are not enough registers or shared memory available per multiprocessor to process at least one block, the kernel will fail to launch.

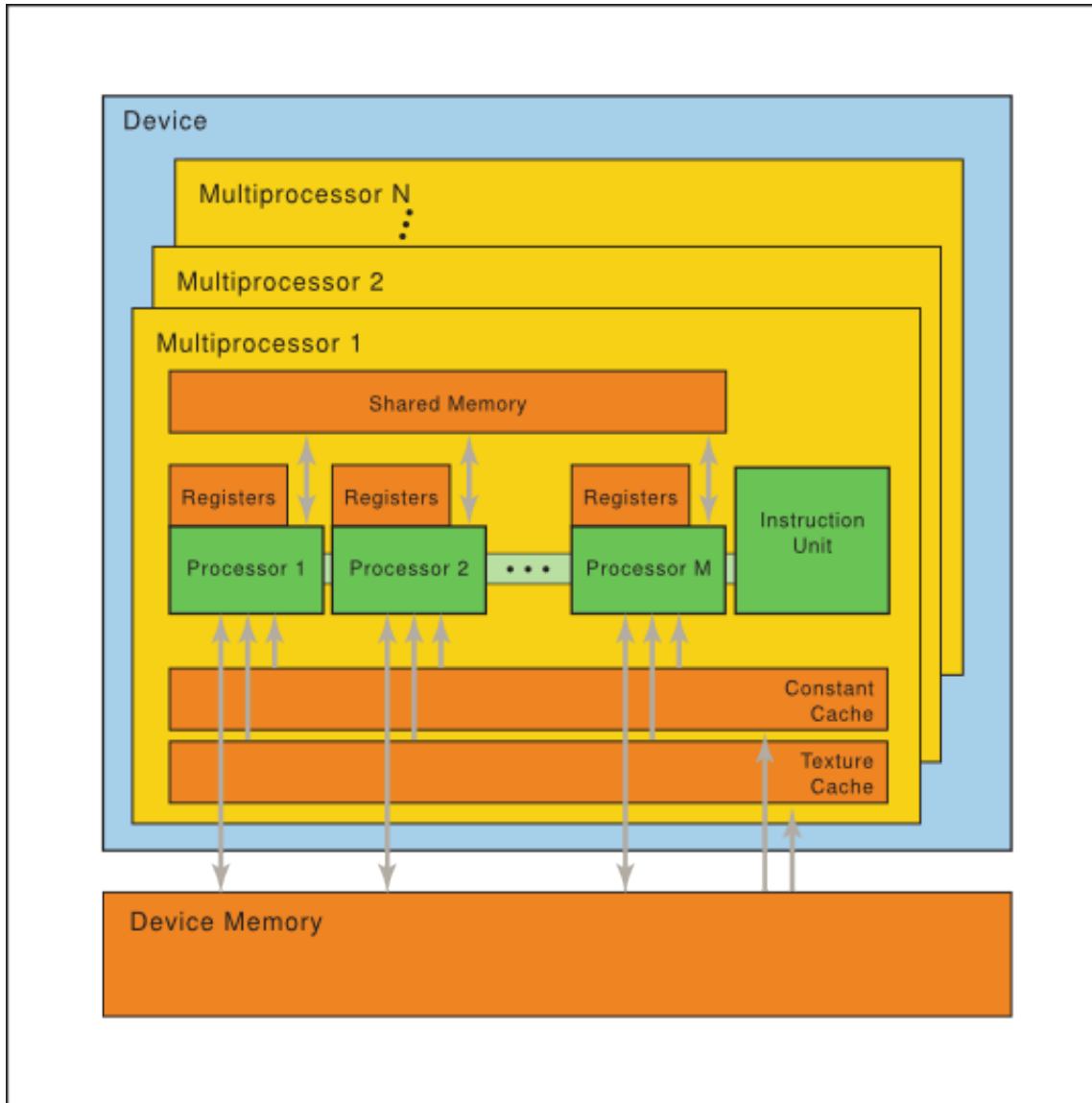


Figure 4: Hardware Model

A set of SIMT multiprocessors with on-chip shared memory.

3.2. Independent Thread Scheduling

On architectures prior to Volta, warps used a single program counter shared amongst all 32 threads in the warp together with an active mask specifying the active threads of the warp. As a result, threads from the same warp in divergent regions or different states of execution cannot signal each other or exchange data, and algorithms requiring fine-grained sharing of data guarded by locks or mutexes can easily lead to deadlock, depending on which warp the contending threads come from.

Starting with the Volta architecture, *Independent Thread Scheduling* allows full concurrency between threads, regardless of warp. With *Independent Thread Scheduling*, the GPU maintains execution state per thread, including a program counter and call stack, and can yield execution at a per-thread granularity, either to make better use of execution resources or to allow one thread to wait for data to be produced by another. A schedule optimizer determines how to group active threads from the same warp together into SIMT units. This retains the high throughput of SIMT execution as in prior NVIDIA GPUs, but with much more flexibility: threads can now diverge and reconverge at sub-warp granularity.

Independent Thread Scheduling can lead to a rather different set of threads participating in the executed code than intended if the developer made assumptions about warp-synchronicity of previous hardware architectures. In particular, any warp-synchronous code (such as synchronization-free, intra-warp reductions) should be revisited to ensure compatibility with Volta and beyond. See the section on Compute Capability 7.x in the *Cuda Programming Guide* for further details.

3.3. On-chip Shared Memory

As illustrated by [Figure 4](#), each multiprocessor has on-chip memory of the four following types:

- ▶ One set of local 32-bit *registers* per processor,
- ▶ A parallel data cache or *shared memory* that is shared by all scalar processor cores and is where the shared memory space resides,
- ▶ A read-only *constant cache* that is shared by all scalar processor cores and speeds up reads from the constant memory space, which is a read-only region of device memory,
- ▶ A read-only *texture cache* that is shared by all scalar processor cores and speeds up reads from the texture memory space, which is a read-only region of device memory; each multiprocessor accesses the texture cache via a *texture unit* that implements the various addressing modes and data filtering.

The local and global memory spaces are read-write regions of device memory.

Chapter 4. Syntax

PTX programs are a collection of text source modules (files). PTX source modules have an assembly-language style syntax with instruction operation codes and operands. Pseudo-operations specify symbol and addressing management. The ptxas optimizing backend compiler optimizes and assembles PTX source modules to produce corresponding binary object files.

4.1. Source Format

Source modules are ASCII text. Lines are separated by the newline character (`\n`).

All whitespace characters are equivalent; whitespace is ignored except for its use in separating tokens in the language.

The C preprocessor `cpp` may be used to process PTX source modules. Lines beginning with `#` are preprocessor directives. The following are common preprocessor directives:

`#include`, `#define`, `#if`, `#ifdef`, `#else`, `#endif`, `#line`, `#file`

C: A Reference Manual by Harbison and Steele provides a good description of the C preprocessor.

PTX is case sensitive and uses lowercase for keywords.

Each PTX module must begin with a `.version` directive specifying the PTX language version, followed by a `.target` directive specifying the target architecture assumed. See [PTX Module Directives](#) for a more information on these directives.

4.2. Comments

Comments in PTX follow C/C++ syntax, using non-nested `/*` and `*/` for comments that may span multiple lines, and using `//` to begin a comment that extends up to the next newline character, which terminates the current line. Comments cannot occur within character constants, string literals, or within other comments.

Comments in PTX are treated as whitespace.

4.3. Statements

A PTX statement is either a directive or an instruction. Statements begin with an optional label and end with a semicolon.

Examples

```
.reg      .b32 r1, r2;
.global   .f32 array[N];

start:    mov.b32    r1, %tid.x;
          shl.b32    r1, r1, 2;           // shift thread id by 2 bits
          ld.global.b32 r2, array[r1];    // thread[tid] gets array[tid]
          add.f32     r2, r2, 0.5;        // add 1/2
```

4.3.1. Directive Statements

Directive keywords begin with a dot, so no conflict is possible with user-defined identifiers. The directives in PTX are listed in [Table 1](#) and described in [State Spaces, Types, and Variables](#) and [Directives](#).

Table 1: PTX Directives

.address_size	.explicitcluster	.maxnreg	.section
.alias	.extern	.maxntid	.shared
.align	.file	.minnctapersm	.sreg
.branchtargets	.func	.noreturn	.target
.callprototype	.global	.param	.tex
.calltargets	.loc	.pragma	.version
.common	.local	.reg	.visible
.const	.maxclusterrank	.reqnctapercluster	.weak
.entry	.maxnctapersm	.reqntid	

4.3.2. Instruction Statements

Instructions are formed from an instruction opcode followed by a comma-separated list of zero or more operands, and terminated with a semicolon. Operands may be register variables, constant expressions, address expressions, or label names. Instructions have an optional guard predicate which controls conditional execution. The guard predicate follows the optional label and precedes the opcode, and is written as @p, where p is a predicate register. The guard predicate may be optionally negated, written as @!p.

The destination operand is first, followed by source operands.

Instruction keywords are listed in [Table 2](#). All instruction keywords are reserved tokens in PTX.

Table 2: Reserved Instruction Keywords

abs	cvta	membar	setp	vabsdiff
activemask	discard	min	shf	vabsdiff2
add	div	mma	shfl	vabsdiff4
addc	dp2a	mov	shl	vadd
alloca	dp4a	movmatrix	shr	vadd2
and	elect	mul	sin	vadd4
applypriority	ex2	mul24	slct	vavrg2
atom	exit	multimem	sqr	vavrg4
bar	fence	nanosleep	st	vmad
barrier	fma	neg	stackrestore	vmax
bfe	fns	not	stacksave	vmax2
bfi	getctarank	or	stmatrix	vmax4
bfind	griddepcontrol	pmevent	sub	vmin
bmsk	isspacep	popc	subc	vmin2
bra	istypep	prefetch	suld	vmin4
brev	ld	prefetchu	suq	vote
brkpt	ldmatrix	prmt	sured	vset
brx	ldu	rcp	sust	vset2
call	lg2	red	szext	vset4
clz	lop3	redux	tanh	vshl
cnot	mad	rem	tcgen05	vshr
copysign	mad24	ret	tensormap	vsub
cos	madc	rsqrt	testp	vsub2
clusterlaunchcontrol	mapa	sad	tex	vsub4
cp	match	selp	tld4	wgmma
createpolicy	max	set	trap	wmma
cvt	mbarrier	setmaxnreg	txq	xor

4.4. Identifiers

User-defined identifiers follow extended C++ rules: they either start with a letter followed by zero or more letters, digits, underscore, or dollar characters; or they start with an underscore, dollar, or percentage character followed by one or more letters, digits, underscore, or dollar characters:

```
followsym:  [a-zA-Z0-9_$]
identifier: [a-zA-Z]{followsym}* | {[_$%]{followsym}+
```

PTX does not specify a maximum length for identifiers and suggests that all implementations support a minimum length of at least 1024 characters.

Many high-level languages such as C and C++ follow similar rules for identifier names, except that the percentage sign is not allowed. PTX allows the percentage sign as the first character of an identifier. The percentage sign can be used to avoid name conflicts, e.g., between user-defined variable names and compiler-generated names.

PTX predefines one constant and a small number of special registers that begin with the percentage sign, listed in [Table 3](#).

Table 3: Predefined Identifiers

%aggr_smem_size	%dynamic_smem_size	%lanemask_gt	%reserved_smem_offset_begin
%clock	%envreg<32>	%lanemask_le	%reserved_smem_offset_cap
%clock64	%globaltimer	%lanemask_lt	%reserved_smem_offset_end
%cluster_ctaid	%globaltimer_hi	%nclusterid	%smid
%cluster_ctarank	%globaltimer_lo	%nctaid	%tid
%cluster_nctaid	%gridid	%nsmid	%total_smem_size
%cluster_nctarank	%is_explicit_cluster	%tid	%warpid
%clusterid	%laneid	%nwarpid	WARP_SZ
%ctaid	%lanemask_eq	%pm0, ..., %pm7	
%current_graph_exec	%lanemask_ge	%reserved_smem_offset_<2>	

4.5. Constants

PTX supports integer and floating-point constants and constant expressions. These constants may be used in data initialization and as operands to instructions. Type checking rules remain the same for integer, floating-point, and bit-size types. For predicate-type data and instructions, integer constants are allowed and are interpreted as in C, i.e., zero values are `False` and non-zero values are `True`.

4.5.1. Integer Constants

Integer constants are 64-bits in size and are either signed or unsigned, i.e., every integer constant has type `.s64` or `.u64`. The signed/unsigned nature of an integer constant is needed to correctly evaluate constant expressions containing operations such as division and ordered comparisons, where the behavior of the operation depends on the operand types. When used in an instruction or data initialization, each integer constant is converted to the appropriate size based on the data or instruction type at its use.

Integer literals may be written in decimal, hexadecimal, octal, or binary notation. The syntax follows that of C. Integer literals may be followed immediately by the letter `U` to indicate that the literal is unsigned.

```
hexadecimal literal: 0[xX]{hexdigit}+U?
octal literal:       0{octal digit}+U?
binary literal:      0[bB]{bit}+U?
decimal literal      {nonzero-digit}{digit}*U?
```

Integer literals are non-negative and have a type determined by their magnitude and optional type suffix as follows: literals are signed (`.s64`) unless the value cannot be fully represented in `.s64` or the unsigned suffix is specified, in which case the literal is unsigned (`.u64`).

The predefined integer constant `WARP_SZ` specifies the number of threads per warp for the target platform; to date, all target architectures have a `WARP_SZ` value of 32.

4.5.2. Floating-Point Constants

Floating-point constants are represented as 64-bit double-precision values, and all floating-point constant expressions are evaluated using 64-bit double precision arithmetic. The only exception is the 32-bit hex notation for expressing an exact single-precision floating-point value; such values retain their exact 32-bit single-precision value and may not be used in constant expressions. Each 64-bit floating-point constant is converted to the appropriate floating-point size based on the data or instruction type at its use.

Floating-point literals may be written with an optional decimal point and an optional signed exponent. Unlike C and C++, there is no suffix letter to specify size; literals are always represented in 64-bit double-precision format.

PTX includes a second representation of floating-point constants for specifying the exact machine representation using a hexadecimal constant. To specify IEEE 754 double-precision floating point values, the constant begins with `0d` or `0D` followed by 16 hex digits. To specify IEEE 754 single-precision floating point values, the constant begins with `0f` or `0F` followed by 8 hex digits.

```
0[fF]{hexdigit}{8}      // single-precision floating point  
0[dD]{hexdigit}{16}     // double-precision floating point
```

Example

```
mov.f32 $f3, 0F3f800000;    // 1.0
```

4.5.3. Predicate Constants

In PTX, integer constants may be used as predicates. For predicate-type data initializers and instruction operands, integer constants are interpreted as in C, i.e., zero values are `False` and non-zero values are `True`.

4.5.4. Constant Expressions

In PTX, constant expressions are formed using operators as in C and are evaluated using rules similar to those in C, but simplified by restricting types and sizes, removing most casts, and defining full semantics to eliminate cases where expression evaluation in C is implementation dependent.

Constant expressions are formed from constant literals, unary plus and minus, basic arithmetic operators (addition, subtraction, multiplication, division), comparison operators, the conditional ternary operator (`? :`), and parentheses. Integer constant expressions also allow unary logical negation (`!`), bitwise complement (`~`), remainder (`%`), shift operators (`<<` and `>>`), bit-type operators (`&`, `|`, and `^`), and logical operators (`&&`, `||`).

Constant expressions in PTX do not support casts between integer and floating-point.

Constant expressions are evaluated using the same operator precedence as in C. [Table 4](#) gives operator precedence and associativity. Operator precedence is highest for unary operators and decreases with each line in the chart. Operators on the same line have the same precedence and are evaluated right-to-left for unary operators and left-to-right for binary operators.

Table 4: Operator Precedence

Kind	Operator Symbols	Operator Names	Associates
Primary	()	parenthesis	n/a
Unary	+- ! ~	plus, minus, negation, complement	right
	(.s64) (.u64)	casts	right
Binary	* / %	multiplication, division, remainder	left
	+-	addition, subtraction	
	>> <<	shifts	
	< > <= >=	ordered comparisons	
	== !=	equal, not equal	
	&	bitwise AND	
	^	bitwise XOR	
	 	bitwise OR	
	&&	logical AND	
	 	logical OR	
Ternary	? :	conditional	right

4.5.5. Integer Constant Expression Evaluation

Integer constant expressions are evaluated at compile time according to a set of rules that determine the type (signed `.s64` versus unsigned `.u64`) of each sub-expression. These rules are based on the rules in C, but they've been simplified to apply only to 64-bit integers, and behavior is fully defined in all cases (specifically, for remainder and shift operators).

- ▶ Literals are signed unless unsigned is needed to prevent overflow, or unless the literal uses a U suffix. For example:
 - ▶ 42, 0x1234, 0123 are signed.
 - ▶ 0xfabc123400000000, 42U, 0x1234U are unsigned.
- ▶ Unary plus and minus preserve the type of the input operand. For example:
 - ▶ +123, -1, -(-42) are signed.
 - ▶ -1U, -0xfabc123400000000 are unsigned.
- ▶ Unary logical negation (!) produces a signed result with value 0 or 1.
- ▶ Unary bitwise complement (~) interprets the source operand as unsigned and produces an unsigned result.
- ▶ Some binary operators require normalization of source operands. This normalization is known as *the usual arithmetic conversions* and simply converts both operands to unsigned type if either operand is unsigned.

- ▶ Addition, subtraction, multiplication, and division perform the usual arithmetic conversions and produce a result with the same type as the converted operands. That is, the operands and result are unsigned if either source operand is unsigned, and is otherwise signed.
- ▶ Remainder (%) interprets the operands as unsigned. Note that this differs from C, which allows a negative divisor but defines the behavior to be implementation dependent.
- ▶ Left and right shift interpret the second operand as unsigned and produce a result with the same type as the first operand. Note that the behavior of right-shift is determined by the type of the first operand: right shift of a signed value is arithmetic and preserves the sign, and right shift of an unsigned value is logical and shifts in a zero bit.
- ▶ AND (&), OR (|), and XOR (^) perform the usual arithmetic conversions and produce a result with the same type as the converted operands.
- ▶ AND_OP (&&), OR_OP (| |), Equal (==), and Not_Equal (!=) produce a signed result. The result value is 0 or 1.
- ▶ Ordered comparisons (<, <=, >, >=) perform the usual arithmetic conversions on source operands and produce a signed result. The result value is 0 or 1.
- ▶ Casting of expressions to signed or unsigned is supported using (.s64) and (.u64) casts.
- ▶ For the conditional operator (? :), the first operand must be an integer, and the second and third operands are either both integers or both floating-point. The usual arithmetic conversions are performed on the second and third operands, and the result type is the same as the converted type.

4.5.6. Summary of Constant Expression Evaluation Rules

Table 5 contains a summary of the constant expression evaluation rules.

Table 5: Constant Expression Evaluation Rules

Kind	Operator	Operand Types	Operand Interpretation	Result Type
Primary	()	any type	same as source	same as source
	constant literal	n/a	n/a	.u64, .s64, or .f64
Unary	+ -	any type	same as source	same as source
	!	integer	zero or non-zero	.s64
	~	integer	.u64	.u64
Cast	(.u64)	integer	.u64	.u64
	(.s64)	integer	.s64	.s64
Binary	+ - * /	.f64	.f64	.f64
		integer	use usual conversions	converted type
	< > <= >=	.f64	.f64	.s64
		integer	use usual conversions	.s64
	== !=	.f64	.f64	.s64
		integer	use usual conversions	.s64
	%	integer	.u64	.s64
	>> <<	integer	1st unchanged, 2nd is .u64	same as 1st operand
	& ^	integer	.u64	.u64
Ternary	?:	int ? .f64 : .f64	same as sources	.f64
		int ? int : int	use usual conversions	converted type

Chapter 5. State Spaces, Types, and Variables

While the specific resources available in a given target GPU will vary, the kinds of resources will be common across platforms, and these resources are abstracted in PTX through state spaces and data types.

5.1. State Spaces

A state space is a storage area with particular characteristics. All variables reside in some state space. The characteristics of a state space include its size, addressability, access speed, access rights, and level of sharing between threads.

The state spaces defined in PTX are a byproduct of parallel programming and graphics programming. The list of state spaces is shown in [Table 6](#), and properties of state spaces are shown in [Table 7](#).

Table 6: State Spaces

Name	Description
.reg	Registers, fast.
.sreg	Special registers. Read-only; pre-defined; platform-specific.
.const	Shared, read-only memory.
.global	Global memory, shared by all threads.
.local	Local memory, private to each thread.
.param	Kernel parameters, defined per-grid; or Function or local parameters, defined per-thread.
.shared	Addressable memory, defined per CTA, accessible to all threads in the cluster throughout the lifetime of the CTA that defines it.
.tex	Global texture memory (deprecated).

Table 7: Properties of State Spaces

Name	Addressable	Initializable	Access	Sharing
<code>.reg</code>	No	No	R/W	per-thread
<code>.sreg</code>	No	No	RO	per-CTA
<code>.const</code>	Yes	Yes ¹	RO	per-grid
<code>.global</code>	Yes	Yes ¹	R/W	Context
<code>.local</code>	Yes	No	R/W	per-thread
<code>.param</code> (as input to kernel)	Yes ²	No	RO	per-grid
<code>.param</code> (used in functions)	Restricted ³	No	R/W	per-thread
<code>.shared</code>	Yes	No	R/W	per-cluster ⁵
<code>.tex</code>	No ⁴	Yes, via driver	RO	Context

Notes:

¹ Variables in `.const` and `.global` state spaces are initialized to zero by default.

² Accessible only via the `ld.param{::entry}` instruction. Address may be taken via `mov` instruction.

³ Accessible via `ld.param{::func}` and `st.param{::func}` instructions. Device function input and return parameters may have their address taken via `mov`; the parameter is then located on the stack frame and its address is in the `.local` state space.

⁴ Accessible only via the `tex` instruction.

⁵ Visible to the owning CTA and other active CTAs in the cluster.

5.1.1. Register State Space

Registers (`.reg` state space) are fast storage locations. The number of registers is limited, and will vary from platform to platform. When the limit is exceeded, register variables will be spilled to memory, causing changes in performance. For each architecture, there is a recommended maximum number of registers to use (see the *CUDA Programming Guide* for details).

Registers may be typed (signed integer, unsigned integer, floating point, predicate) or untyped. Register size is restricted; aside from predicate registers which are 1-bit, scalar registers have a width of 8-, 16-, 32-, 64-, or 128-bits, and vector registers have a width of 16-, 32-, 64-, or 128-bits. The most common use of 8-bit registers is with `ld`, `st`, and `cvt` instructions, or as elements of vector tuples.

Registers differ from the other state spaces in that they are not fully addressable, i.e., it is not possible to refer to the address of a register. When compiling to use the Application Binary Interface (ABI), register variables are restricted to function scope and may not be declared at module scope. When compiling legacy PTX code (ISA versions prior to 3.0) containing module-scoped `.reg` variables, the compiler silently disables use of the ABI. Registers may have alignment boundaries required by multi-word loads and stores.

5.1.2. Special Register State Space

The special register (`.sreg`) state space holds predefined, platform-specific registers, such as grid, cluster, CTA, and thread parameters, clock counters, and performance monitoring registers. All special registers are predefined.

5.1.3. Constant State Space

The constant (`.const`) state space is a read-only memory initialized by the host. Constant memory is accessed with a `ld.const` instruction. Constant memory is restricted in size, currently limited to 64 KB which can be used to hold statically-sized constant variables. There is an additional 640 KB of constant memory, organized as ten independent 64 KB regions. The driver may allocate and initialize constant buffers in these regions and pass pointers to the buffers as kernel function parameters. Since the ten regions are not contiguous, the driver must ensure that constant buffers are allocated so that each buffer fits entirely within a 64 KB region and does not span a region boundary.

Statically-sized constant variables have an optional variable initializer; constant variables with no explicit initializer are initialized to zero by default. Constant buffers allocated by the driver are initialized by the host, and pointers to such buffers are passed to the kernel as parameters. See the description of kernel parameter attributes in [Kernel Function Parameter Attributes](#) for more details on passing pointers to constant buffers as kernel parameters.

5.1.3.1 Banked Constant State Space (deprecated)

Previous versions of PTX exposed constant memory as a set of eleven 64 KB banks, with explicit bank numbers required for variable declaration and during access.

Prior to PTX ISA version 2.2, the constant memory was organized into fixed size banks. There were eleven 64 KB banks, and banks were specified using the `.const[bank]` modifier, where *bank* ranged from 0 to 10. If no bank number was given, bank zero was assumed.

By convention, bank zero was used for all statically-sized constant variables. The remaining banks were used to declare *incomplete* constant arrays (as in C, for example), where the size is not known at compile time. For example, the declaration

```
.extern .const[2] .b32 const_buffer[];
```

resulted in `const_buffer` pointing to the start of constant bank two. This pointer could then be used to access the entire 64 KB constant bank. Multiple incomplete array variables declared in the same bank were aliased, with each pointing to the start address of the specified constant bank.

To access data in constant banks 1 through 10, the bank number was required in the state space of the load instruction. For example, an incomplete array in bank 2 was accessed as follows:

```
.extern .const[2] .b32 const_buffer[];
ld.const[2].b32 %r1, [const_buffer+4]; // load second word
```

In PTX ISA version 2.2, we eliminated explicit banks and replaced the incomplete array representation of driver-allocated constant buffers with kernel parameter attributes that allow pointers to constant buffers to be passed as kernel parameters.

5.1.4. Global State Space

The global (`.global`) state space is memory that is accessible by all threads in a context. It is the mechanism by which threads in different CTAs, clusters, and grids can communicate. Use `ld.global`, `st.global`, and `atom.global` to access global variables.

Global variables have an optional variable initializer; global variables with no explicit initializer are initialized to zero by default.

5.1.5. Local State Space

The local state space (`.local`) is private memory for each thread to keep its own data. It is typically standard memory with cache. The size is limited, as it must be allocated on a per-thread basis. Use `ld.local` and `st.local` to access local variables.

When compiling to use the *Application Binary Interface (ABI)*, `.local` state-space variables must be declared within function scope and are allocated on the stack. In implementations that do not support a stack, all local memory variables are stored at fixed addresses, recursive function calls are not supported, and `.local` variables may be declared at module scope. When compiling legacy PTX code (ISA versions prior to 3.0) containing module-scoped `.local` variables, the compiler silently disables use of the ABI.

5.1.6. Parameter State Space

The parameter (`.param`) state space is used (1) to pass input arguments from the host to the kernel, (2a) to declare formal input and return parameters for device functions called from within kernel execution, and (2b) to declare locally-scoped byte array variables that serve as function call arguments, typically for passing large structures by value to a function. Kernel function parameters differ from device function parameters in terms of access and sharing (read-only versus read-write, per-kernel versus per-thread). Note that PTX ISA versions 1.x supports only kernel function parameters in `.param` space; device function parameters were previously restricted to the register state space. The use of parameter state space for device function parameters was introduced in PTX ISA version 2.0 and requires target architecture `sm_20` or higher. Additional sub-qualifiers `::entry` or `::func` can be specified on instructions with `.param` state space to indicate whether the address refers to kernel function parameter or device function parameter. If no sub-qualifier is specified with the `.param` state space, then the default sub-qualifier is specific to and dependent on the exact instruction. For example, `st.param` is equivalent to `st.param::func` whereas `isspacep.param` is equivalent to `isspacep.param::entry`. Refer to the instruction description for more details on default sub-qualifier assumption.

Note: The location of parameter space is implementation specific. For example, in some implementations kernel parameters reside in global memory. No access protection is provided between parameter and global space in this case. Though the exact location of the kernel parameter space is implementation specific, the kernel parameter space window is always contained within the global space window. Similarly, function parameters are mapped to parameter passing registers and/or stack locations based on the function calling conventions of the *Application Binary Interface (ABI)*. Therefore, PTX code should make no assumptions about the relative locations or ordering of `.param` space variables.

5.1.6.1 Kernel Function Parameters

Each kernel function definition includes an optional list of parameters. These parameters are addressable, read-only variables declared in the `.param` state space. Values passed from the host to the kernel are accessed through these parameter variables using `ld.param` instructions. The kernel parameter variables are shared across all CTAs from all clusters within a grid.

The address of a kernel parameter may be moved into a register using the `mov` instruction. The resulting address is in the `.param` state space and is accessed using `ld.param` instructions.

Example

```
.entry foo ( .param .b32 N, .param .align 8 .b8 buffer[64] )
{
    .reg .u32 %n;
    .reg .f64 %d;

    ld.param.u32 %n, [N];
    ld.param.f64 %d, [buffer];
    ...
}
```

Example

```
.entry bar ( .param .b32 len )
{
    .reg .u32 %ptr, %n;

    mov.u32      %ptr, len;
    ld.param.u32 %n, [%ptr];
    ...
}
```

Kernel function parameters may represent normal data values, or they may hold addresses to objects in constant, global, local, or shared state spaces. In the case of pointers, the compiler and runtime system need information about which parameters are pointers, and to which state space they point. Kernel parameter attribute directives are used to provide this information at the PTX level. See [Kernel Function Parameter Attributes](#) for a description of kernel parameter attribute directives.

Note: The current implementation does not allow creation of generic pointers to constant variables (`cvta.const`) in programs that have pointers to constant buffers passed as kernel parameters.

5.1.6.2 Kernel Function Parameter Attributes

Kernel function parameters may be declared with an optional `.ptr` attribute to indicate that a parameter is a pointer to memory, and also indicate the state space and alignment of the memory being pointed to. [Kernel Parameter Attribute: `.ptr`](#) describes the `.ptr` kernel parameter attribute.

5.1.6.3 Kernel Parameter Attribute: `.ptr`

`.ptr`

Kernel parameter alignment attribute.

Syntax

```
.param .type .ptr .space .align N varname  
.param .type .ptr          .align N varname  
  
.space = { .const, .global, .local, .shared };
```

Description

Used to specify the state space and, optionally, the alignment of memory pointed to by a pointer type kernel parameter. The alignment value *N*, if present, must be a power of two. If no state space is specified, the pointer is assumed to be a generic address pointing to one of const, global, local, or shared memory. If no alignment is specified, the memory pointed to is assumed to be aligned to a 4 byte boundary.

Spaces between `.ptr`, `.space`, and `.align` may be eliminated to improve readability.

PTX ISA Notes

- ▶ Introduced in PTX ISA version 2.2.
- ▶ Support for generic addressing of `.const` space added in PTX ISA version 3.1.

Target ISA Notes

- ▶ Supported on all target architectures.

Examples

```
.entry foo ( .param .u32 param1,  
            .param .u32 .ptr.global.align 16 param2,  
            .param .u32 .ptr.const.align 8 param3,  
            .param .u32 .ptr.align 16 param4 // generic address  
                                                // pointer  
) { .. }
```

5.1.6.4 Device Function Parameters

PTX ISA version 2.0 extended the use of parameter space to device function parameters. The most common use is for passing objects by value that do not fit within a PTX register, such as C structures larger than 8 bytes. In this case, a byte array in parameter space is used. Typically, the caller will declare a locally-scoped `.param` byte array variable that represents a flattened C structure or union. This will be passed by value to a callee, which declares a `.param` formal parameter having the same size and alignment as the passed argument.

Example

```
// pass object of type struct { double d; int y; };
.func foo ( .reg .b32 N, .param .align 8 .b8 buffer[12] )
{
    .reg .f64 %d;
    .reg .s32 %y;

    ld.param.f64 %d, [buffer];
    ld.param.s32 %y, [buffer+8];
    ...
}

// code snippet from the caller
// struct { double d; int y; } mystruct; is flattened, passed to foo
...
.reg .f64 dbl;
.reg .s32 x;
.param .align 8 .b8 mystruct;
...
st.param.f64 [mystruct+0], dbl;
st.param.s32 [mystruct+8], x;
call foo, (4, mystruct);
...
```

See the section on function call syntax for more details.

Function input parameters may be read via `ld.param` and function return parameters may be written using `st.param`; it is illegal to write to an input parameter or read from a return parameter.

Aside from passing structures by value, `.param` space is also required whenever a formal parameter has its address taken within the called function. In PTX, the address of a function input parameter may be moved into a register using the `mov` instruction. Note that the parameter will be copied to the stack if necessary, and so the address will be in the `.local` state space and is accessed via `ld.local` and `st.local` instructions. It is not possible to use `mov` to get the address of or a locally-scoped `.param` space variable. Starting PTX ISA version 6.0, it is possible to use `mov` instruction to get address of return parameter of device function.

Example

```
// pass array of up to eight floating-point values in buffer
.func foo ( .param .b32 N, .param .b32 buffer[32] )
{
    .reg .u32   %n, %r;
    .reg .f32   %f;
    .reg .pred  %p;

    ld.param.u32 %n, [N];
    mov.u32      %r, buffer; // forces buffer to .local state space
Loop:
    setp.eq.u32  %p, %n, 0;
    @%p bra      Done;
    ld.local.f32 %f, [%r];
    ...
    add.u32      %r, %r, 4;
    sub.u32      %n, %n, 1;
    bra         Loop;
Done:
    ...
}
```

5.1.7. Shared State Space

The shared (`.shared`) state space is a memory that is owned by an executing CTA and is accessible to the threads of all the CTAs within a cluster. An address in shared memory can be read and written by any thread in a CTA cluster.

Additional sub-qualifiers `::cta` or `::cluster` can be specified on instructions with `.shared` state space to indicate whether the address belongs to the shared memory window of the executing CTA or of any CTA in the cluster respectively. The addresses in the `.shared::cta` window also fall within the `.shared::cluster` window. If no sub-qualifier is specified with the `.shared` state space, then it defaults to `::cta`. For example, `ld.shared` is equivalent to `ld.shared::cta`.

Variables declared in `.shared` state space refer to the memory addresses in the current CTA. Instruction `mapa` gives the `.shared::cluster` address of the corresponding variable in another CTA in the cluster.

Shared memory typically has some optimizations to support the sharing. One example is broadcast; where all threads read from the same address. Another is sequential access from sequential threads.

5.1.8. Texture State Space (deprecated)

The texture (`.tex`) state space is global memory accessed via the texture instruction. It is shared by all threads in a context. Texture memory is read-only and cached, so accesses to texture memory are not coherent with global memory stores to the texture image.

The GPU hardware has a fixed number of texture bindings that can be accessed within a single kernel (typically 128). The `.tex` directive will bind the named texture memory variable to a hardware texture identifier, where texture identifiers are allocated sequentially beginning with zero. Multiple names may be bound to the same physical texture identifier. An error is generated if the maximum number of physical resources is exceeded. The texture name must be of type `.u32` or `.u64`.

Physical texture resources are allocated on a per-kernel granularity, and `.tex` variables are required to be defined in the global scope.

Texture memory is read-only. A texture's base address is assumed to be aligned to a 16 byte boundary.

Example

```
.tex .u32 tex_a;           // bound to physical texture 0
.tex .u32 tex_c, tex_d;    // both bound to physical texture 1
.tex .u32 tex_d;           // bound to physical texture 2
.tex .u32 tex_f;           // bound to physical texture 3
```

Note: Explicit declarations of variables in the texture state space is deprecated, and programs should instead reference texture memory through variables of type `.texref`. The `.tex` directive is retained for backward compatibility, and variables declared in the `.tex` state space are equivalent to module-scoped `.texref` variables in the `.global` state space.

For example, a legacy PTX definitions such as

```
.tex .u32 tex_a;
```

is equivalent to:

```
.global .texref tex_a;
```

See [Texture Sampler and Surface Types](#) for the description of the `.texref` type and [Texture Instructions](#) for its use in texture instructions.

5.2. Types

5.2.1. Fundamental Types

In PTX, the fundamental types reflect the native data types supported by the target architectures. A fundamental type specifies both a basic type and a size. Register variables are always of a fundamental type, and instructions operate on these types. The same type-size specifiers are used for both variable definitions and for typing instructions, so their names are intentionally short.

[Table 8](#) lists the fundamental type specifiers for each basic type:

Table 8: Fundamental Type Specifiers

Basic Type	Fundamental Type Specifiers
Signed integer	<code>.s8</code> , <code>.s16</code> , <code>.s32</code> , <code>.s64</code>
Unsigned integer	<code>.u8</code> , <code>.u16</code> , <code>.u32</code> , <code>.u64</code>
Floating-point	<code>.f16</code> , <code>.f16x2</code> , <code>.f32</code> , <code>.f64</code>
Bits (untyped)	<code>.b8</code> , <code>.b16</code> , <code>.b32</code> , <code>.b64</code> , <code>.b128</code>
Predicate	<code>.pred</code>

Most instructions have one or more type specifiers, needed to fully specify instruction behavior. Operand types and sizes are checked against instruction types for compatibility.

Two fundamental types are compatible if they have the same basic type and are the same size. Signed and unsigned integer types are compatible if they have the same size. The bit-size type is compatible with any fundamental type having the same size.

In principle, all variables (aside from predicates) could be declared using only bit-size types, but typed variables enhance program readability and allow for better operand type checking.

5.2.2. Restricted Use of Sub-Word Sizes

The `.u8`, `.s8`, and `.b8` instruction types are restricted to `ld`, `st`, and `cvt` instructions. The `.f16` floating-point type is allowed only in conversions to and from `.f32`, `.f64` types, in half precision floating point instructions and texture fetch instructions. The `.f16x2` floating point type is allowed only in half precision floating point arithmetic instructions and texture fetch instructions.

For convenience, `ld`, `st`, and `cvt` instructions permit source and destination data operands to be wider than the instruction-type size, so that narrow values may be loaded, stored, and converted using regular-width registers. For example, 8-bit or 16-bit values may be held directly in 32-bit or 64-bit registers when being loaded, stored, or converted to other types and sizes.

5.2.3. Alternate Floating-Point Data Formats

The fundamental floating-point types supported in PTX have implicit bit representations that indicate the number of bits used to store exponent and mantissa. For example, the `.f16` type indicates 5 bits reserved for exponent and 10 bits reserved for mantissa. In addition to the floating-point representations assumed by the fundamental types, PTX allows the following alternate floating-point data formats:

bf16 data format:

This data format is a 16-bit floating point format with 8 bits for exponent and 7 bits for mantissa. A register variable containing bf16 data must be declared with `.b16` type.

e4m3 data format:

This data format is an 8-bit floating point format with 4 bits for exponent and 3 bits for mantissa. The e4m3 encoding does not support infinity and NaN values are limited to `0x7f` and `0xff`. A register variable containing e4m3 value must be declared using bit-size type.

e5m2 data format:

This data format is an 8-bit floating point format with 5 bits for exponent and 2 bits for mantissa. A register variable containing e5m2 value must be declared using bit-size type.

tf32 data format:

This data format is a special 32-bit floating point format supported by the matrix multiply-and-accumulate instructions, with the same range as `.f32` and reduced precision (≥ 10 bits). The internal layout of tf32 format is implementation defined. PTX facilitates conversion from single precision `.f32` type to tf32 format. A register variable containing tf32 data must be declared with `.b32` type.

e2m1 data format:

This data format is a 4-bit floating point format with 2 bits for exponent and 1 bit for mantissa. The e2m1 encoding does not support infinity and NaN. e2m1 values must be used in a packed

format specified as `e2m1x2`. A register variable containing two `e2m1` values must be declared with `.b8` type.

e2m3 data format:

This data format is a 6-bit floating point format with 2 bits for exponent and 3 bits for mantissa. The `e2m3` encoding does not support infinity and NaN. `e2m3` values must be used in a packed format specified as `e2m3x2`. A register variable containing two `e2m3` values must be declared with `.b16` type where each `.b8` element has 6-bit floating point value and 2 MSB bits padded with zeros.

e3m2 data format:

This data format is a 6-bit floating point format with 3 bits for exponent and 2 bits for mantissa. The `e3m2` encoding does not support infinity and NaN. `e3m2` values must be used in a packed format specified as `e3m2x2`. A register variable containing two `e3m2` values must be declared with `.b16` type where each `.b8` element has 6-bit floating point value and 2 MSB bits padded with zeros.

ue8m0 data format:

This data format is an 8-bit unsigned floating-point format with 8 bits for exponent and 0 bits for mantissa. The `ue8m0` encoding does not support infinity. NaN value is limited to `0xff`. `ue8m0` values must be used in a packed format specified as `ue8m0x2`. A register variable containing two `ue8m0` values must be declared with `.b16` type.

ue4m3 data format:

This data format is a 7-bit unsigned floating-point format with 4 bits for exponent and 3 bits for mantissa. The `ue4m3` encoding does not support infinity. NaN value is limited to `0x7f`. A register variable containing single `ue4m3` value must be declared with `.b8` type having MSB bit padded with zero.

Alternate data formats cannot be used as fundamental types. They are supported as source or destination formats by certain instructions.

5.2.4. Fixed-point Data format

PTX supports following fixed-point data formats:

s2f6 data format:

This data format is 8-bit signed 2's complement integer with 2 sign-integer bits and 6 fractional bits with form `xx.xxxxxx`. The `s2f6` encoding does not support infinity and NaN.

`s2f6` value = `s8` value * 2^{-6} Positive max representation = `01.111111` = $127 * 2^{-6}$ = 1.984375
Negative max representation = `10.000000` = $-128 * 2^{-6}$ = -2.0

5.2.5. Packed Data Types

Certain PTX instructions operate on two or more sets of inputs in parallel, and produce two or more outputs. Such instructions can use the data stored in a packed format. PTX supports packing two or four values of the same scalar data type into a single, larger value. The packed value is considered as a value of a *packed data type*. In this section we describe the packed data types supported in PTX.

5.2.5.1 Packed Floating Point Data Types

PTX supports various variants of packed floating point data types. Out of them, only `.f16x2` is supported as a fundamental type, while others cannot be used as fundamental types - they are supported as instruction types on certain instructions. When using an instruction with such non-fundamental types, the operand data variables must be of bit type of appropriate size. For example, all of the operand variables must be of type `.b32` for an instruction with instruction type as `.bf16x2`. Table 9 described various variants of packed floating point data types in PTX.

Table 9: Operand types for packed floating point instruction type.

Packed floating point type	Number of elements contained in a packed format	Type of each element	Register variable type to be used in the declaration
<code>.f16x2</code>	Two	<code>.f16</code>	<code>.f16x2</code> or <code>.b32</code>
<code>.f32x2</code>		<code>.f32</code>	<code>.b64</code>
<code>.bf16x2</code>		<code>.bf16</code>	<code>.b32</code>
<code>.e4m3x2</code>		<code>.e4m3</code>	<code>.b16</code>
<code>.e5m2x2</code>		<code>.e5m2</code>	
<code>.e2m3x2</code>		<code>.e2m3</code>	
<code>.e3m2x2</code>		<code>.e3m2</code>	
<code>.ue8m0x2</code>		<code>.ue8m0</code>	
<code>.s2f6x2</code>		<code>.s2f6</code>	
<code>.e2m1x2</code>		<code>.e2m1</code>	<code>.b8</code>
<code>.e4m3x4</code>	Four	<code>.e4m3</code>	<code>.b32</code>
<code>.e5m2x4</code>		<code>.e5m2</code>	
<code>.e2m3x4</code>		<code>.e2m3</code>	
<code>.e3m2x4</code>		<code>.e3m2</code>	
<code>.e2m1x4</code>		<code>.e2m1</code>	<code>.b16</code>

5.2.5.2 Packed Integer Data Types

PTX supports two variants of packed integer data types: `.u16x2` and `.s16x2`. The packed data type consists of two `.u16` or `.s16` values. A register variable containing `.u16x2` or `.s16x2` data must be declared with `.b32` type. Packed integer data types cannot be used as fundamental types. They are supported as instruction types on certain instructions.

5.2.5.3 Packed Fixed-Point Data Types

PTX supports `.s2f6x2` packed fixed-point data type consisting of two `.s2f6` packed fixed-point values. A register variable containing `.s2f6x2` value must be declared with `.b16` type. Packed fixed-point data type cannot be used as fundamental type and is only supported as instruction type.

5.3. Texture Sampler and Surface Types

PTX includes built-in *opaque* types for defining texture, sampler, and surface descriptor variables. These types have named fields similar to structures, but all information about layout, field ordering, base address, and overall size is hidden to a PTX program, hence the term *opaque*. The use of these opaque types is limited to:

- ▶ Variable definition within global (module) scope and in kernel entry parameter lists.
- ▶ Static initialization of module-scope variables using comma-delimited static assignment expressions for the named members of the type.
- ▶ Referencing textures, samplers, or surfaces via texture and surface load/store instructions (`tex`, `suld`, `sust`, `sured`).
- ▶ Retrieving the value of a named member via query instructions (`txq`, `suq`).
- ▶ Creating pointers to opaque variables using `mov`, e.g., `mov .u64 reg, opaque_var ;`. The resulting pointer may be stored to and loaded from memory, passed as a parameter to functions, and de-referenced by texture and surface load, store, and query instructions, but the pointer cannot otherwise be treated as an address, i.e., accessing the pointer with `ld` and `st` instructions, or performing pointer arithmetic will result in undefined results.
- ▶ Opaque variables may not appear in initializers, e.g., to initialize a pointer to an opaque variable.

Note: Indirect access to textures and surfaces using pointers to opaque variables is supported beginning with PTX ISA version 3.1 and requires target `sm_20` or later.

Indirect access to textures is supported only in unified texture mode (see below).

The three built-in types are `.texref`, `.samplerref`, and `.surfref`. For working with textures and samplers, PTX has two modes of operation. In the *unified mode*, texture and sampler information is accessed through a single `.texref` handle. In the *independent mode*, texture and sampler information each have their own handle, allowing them to be defined separately and combined at the site of usage in the program. In independent mode, the fields of the `.texref` type that describe sampler properties are ignored, since these properties are defined by `.samplerref` variables.

Table 10 and Table 11 list the named members of each type for unified and independent texture modes. These members and their values have precise mappings to methods and values defined in the texture HW class as well as exposed values via the API.

Table 10: Opaque Type Fields in Unified Texture Mode

Member	.texref values	.surfref values
width	in elements	
height	in elements	
depth	in elements	
channel_data_type	enum type corresponding to source language API	
channel_order	enum type corresponding to source language API	
normalized_coords	0, 1	N/A
filter_mode	nearest, linear	N/A
addr_mode_0, addr_mode_1, addr_mode_2	wrap, mirror, clamp_ogl, clamp_to_edge, clamp_to_border	N/A
array_size	as number of textures in a texture array	as number of surfaces in a surface array
num_mipmap_levels	as number of levels in a mipmapped texture	N/A
num_samples	as number of samples in a multi-sample texture	N/A
memory_layout	N/A	1 for linear memory layout; 0 otherwise

5.3.1. Texture and Surface Properties

Fields width, height, and depth specify the size of the texture or surface in number of elements in each dimension.

The channel_data_type and channel_order fields specify these properties of the texture or surface using enumeration types corresponding to the source language API. For example, see [Channel Data Type and Channel Order Fields](#) for the OpenCL enumeration types currently supported in PTX.

5.3.2. Sampler Properties

The normalized_coords field indicates whether the texture or surface uses normalized coordinates in the range [0.0, 1.0) instead of unnormalized coordinates in the range [0, N). If no value is specified, the default is set by the runtime system based on the source language.

The filter_mode field specifies how the values returned by texture reads are computed based on the input texture coordinates.

The addr_mode_{0, 1, 2} fields define the addressing mode in each dimension, which determine how out-of-range coordinates are handled.

See the *CUDA C++ Programming Guide* for more details of these properties.

Table 11: Opaque Type Fields in Independent Texture Mode

Member	.samplerref values	.texref values	.surfref values
width	N/A	in elements	
height	N/A	in elements	
depth	N/A	in elements	
channel_data_type	N/A	enum type corresponding to source language API	
channel_order	N/A	enum type corresponding to source language AP	
normalized_coords	N/A	0, 1	N/A
force_unnormalized_coords	0, 1	N/A	N/A
filter_mode	nearest, linear	ignored	N/A
addr_mode_0, addr_mode_1, addr_mode_2	wrap, mirror, clamp_ogl, clamp_to_edge, clamp_to_border	N/A	N/A
array_size	N/A	as number of textures in a texture array	as number of surfaces in a surface array
num_mipmap_levels	N/A	as number of levels in a mipmapped texture	N/A
num_samples	N/A	as number of samples in a multi-sample texture	N/A
memory_layout	N/A	N/A	1 for linear memory layout; 0 otherwise

In independent texture mode, the sampler properties are carried in an independent `.samplerref` variable, and these fields are disabled in the `.texref` variables. One additional sampler property, `force_unnormalized_coords`, is available in independent texture mode.

The `force_unnormalized_coords` field is a property of `.samplerref` variables that allows the sampler to override the texture header `normalized_coords` property. This field is defined only in independent texture mode. When `True`, the texture header setting is overridden and unnormalized coordinates are used; when `False`, the texture header setting is used.

The `force_unnormalized_coords` property is used in compiling OpenCL; in OpenCL, the property of normalized coordinates is carried in sampler headers. To compile OpenCL to PTX, texture headers are always initialized with `normalized_coords` set to `True`, and the OpenCL sampler-based `normalized_coords` flag maps (negated) to the PTX-level `force_unnormalized_coords` flag.

Variables using these types may be declared at module scope or within kernel entry parameter lists. At module scope, these variables must be in the `.global` state space. As kernel parameters, these

variables are declared in the `.param` state space.

Example

```
.global .texref      my_texture_name;
.global .samplerref  my_sampler_name;
.global .surfref     my_surface_name;
```

When declared at module scope, the types may be initialized using a list of static expressions assigning values to the named members.

Example

```
.global .texref tex1;
.global .samplerref tsamp1 = { addr_mode_0 = clamp_to_border,
                               filter_mode = nearest
                               };
```

5.3.3. Channel Data Type and Channel Order Fields

The `channel_data_type` and `channel_order` fields have enumeration types corresponding to the source language API. Currently, OpenCL is the only source language that defines these fields. [Table 13](#) and [Table 12](#) show the enumeration values defined in OpenCL version 1.0 for channel data type and channel order.

Table 12: OpenCL 1.0 Channel Data Type Definition

CL_SNORM_INT8	0x10D0
CL_SNORM_INT16	0x10D1
CL_UNORM_INT8	0x10D2
CL_UNORM_INT16	0x10D3
CL_UNORM_SHORT_565	0x10D4
CL_UNORM_SHORT_555	0x10D5
CL_UNORM_INT_101010	0x10D6
CL_SIGNED_INT8	0x10D7
CL_SIGNED_INT16	0x10D8
CL_SIGNED_INT32	0x10D9
CL_UNSIGNED_INT8	0x10DA
CL_UNSIGNED_INT16	0x10DB
CL_UNSIGNED_INT32	0x10DC
CL_HALF_FLOAT	0x10DD
CL_FLOAT	0x10DE

Table 13: OpenCL 1.0 Channel Order Definition

CL_R	0x10B0
CL_A	0x10B1
CL_RG	0x10B2
CL_RA	0x10B3
CL_RGB	0x10B4
CL_RGBA	0x10B5
CL_BGRA	0x10B6
CL_ARGB	0x10B7
CL_INTENSITY	0x10B8
CL_LUMINANCE	0x10B9

5.4. Variables

In PTX, a variable declaration describes both the variable's type and its state space. In addition to fundamental types, PTX supports types for simple aggregate objects such as vectors and arrays.

5.4.1. Variable Declarations

All storage for data is specified with variable declarations. Every variable must reside in one of the state spaces enumerated in the previous section.

A variable declaration names the space in which the variable resides, its type and size, its name, an optional array size, an optional initializer, and an optional fixed address for the variable.

Predicate variables may only be declared in the register state space.

Examples

```
.global .u32 loc;
.reg    .s32 i;
.const  .f32 bias[] = {-1.0, 1.0};
.global .u8  bg[4] = {0, 0, 0, 0};
.reg    .v4 .f32 accel;
.reg    .pred p, q, r;
```

5.4.2. Vectors

Limited-length vector types are supported. Vectors of length 2 and 4 of any non-predicate fundamental type can be declared by prefixing the type with `.v2` or `.v4`. Vectors must be based on a fundamental type, and they may reside in the register space. Vectors cannot exceed 128-bits in length; for example, `.v4 .f64` is not allowed. Three-element vectors may be handled by using a `.v4` vector, where the fourth element provides padding. This is a common case for three-dimensional grids, textures, etc.

Examples

```
.global .v4 .f32 V;    // a length-4 vector of floats
.shared .v2 .u16 uv;   // a length-2 vector of unsigned ints
.global .v4 .b8 v;     // a length-4 vector of bytes
```

By default, vector variables are aligned to a multiple of their overall size (vector length times base-type size), to enable vector load and store instructions which require addresses aligned to a multiple of the access size.

5.4.3. Array Declarations

Array declarations are provided to allow the programmer to reserve space. To declare an array, the variable name is followed with dimensional declarations similar to fixed-size array declarations in C. The size of each dimension is a constant expression.

Examples

```
.local .u16 kernel[19][19];
.shared .u8 mailbox[128];
```

The size of the array specifies how many elements should be reserved. For the declaration of array *kernel* above, $19 \times 19 = 361$ halfwords are reserved, for a total of 722 bytes.

When declared with an initializer, the first dimension of the array may be omitted. The size of the first array dimension is determined by the number of elements in the array initializer.

Examples

```
.global .u32 index[] = { 0, 1, 2, 3, 4, 5, 6, 7 };
.global .s32 offset[][2] = { {-1, 0}, {0, -1}, {1, 0}, {0, 1} };
```

Array *index* has eight elements, and array *offset* is a 4x2 array.

5.4.4. Initializers

Declared variables may specify an initial value using a syntax similar to C/C++, where the variable name is followed by an equals sign and the initial value or values for the variable. A scalar takes a single value, while vectors and arrays take nested lists of values inside of curly braces (the nesting matches the dimensionality of the declaration).

As in C, array initializers may be incomplete, i.e., the number of initializer elements may be less than the extent of the corresponding array dimension, with remaining array locations initialized to the default value for the specified array type.

Examples

```
.const .f32 vals[8] = { 0.33, 0.25, 0.125 };
.global .s32 x[3][2] = { {1,2}, {3} };
```

is equivalent to

```
.const .f32 vals[8] = { 0.33, 0.25, 0.125, 0.0, 0.0, 0.0, 0.0, 0.0 };
.global .s32 x[3][2] = { {1,2}, {3,0}, {0,0} };
```

Currently, variable initialization is supported only for constant and global state spaces. Variables in constant and global state spaces with no explicit initializer are initialized to zero by default. Initializers are not allowed in external variable declarations.

Variable names appearing in initializers represent the address of the variable; this can be used to statically initialize a pointer to a variable. Initializers may also contain *var+offset* expressions, where *offset* is a byte offset added to the address of *var*. Only variables in `.global` or `.const` state spaces may be used in initializers. By default, the resulting address is the offset in the variable's state space (as is the case when taking the address of a variable with a `mov` instruction). An operator, `generic()`, is provided to create a generic address for variables used in initializers.

Starting PTX ISA version 7.1, an operator `mask()` is provided, where `mask` is an integer immediate. The only allowed expressions in the `mask()` operator are integer constant expression and symbol expression representing address of variable. The `mask()` operator extracts *n* consecutive bits from the expression used in initializers and inserts these bits at the lowest position of the initialized variable. The number *n* and the starting position of the bits to be extracted is specified by the integer immediate `mask`. PTX ISA version 7.1 only supports extracting a single byte starting at byte boundary from the address of the variable. PTX ISA version 7.3 supports Integer constant expression as an operand in the `mask()` operator.

Supported values for `mask` are: `0xFF`, `0xFF00`, `0xFF0000`, `0xFF000000`, `0xFF00000000`, `0xFF0000000000`, `0xFF000000000000`.

Examples

```
.const .u32 foo = 42;
.global .u32 bar[] = { 2, 3, 5 };
.global .u32 p1 = foo;           // offset of foo in .const space
.global .u32 p2 = generic(foo); // generic address of foo

// array of generic-address pointers to elements of bar
.global .u32 parr[] = { generic(bar), generic(bar)+4,
generic(bar)+8 };

// examples using mask() operator are pruned for brevity
.global .u8 addr[] = {0xff(foo), 0xff00(foo), 0xff0000(foo), ...};

.global .u8 addr2[] = {0xff(foo+4), 0xff00(foo+4), 0xff0000(foo+4),...}

.global .u8 addr3[] = {0xff(generic(foo)), 0xff00(generic(foo)),...}

.global .u8 addr4[] = {0xff(generic(foo)+4), 0xff00(generic(foo)+4),...}

// mask() operator with integer const expression
.global .u8 addr5[] = { 0xFF(1000 + 546), 0xFF00(131187), ...};
```

Note: PTX 3.1 redefines the default addressing for global variables in initializers, from generic addresses to offsets in the global state space. Legacy PTX code is treated as having an implicit `generic()` operator for each global variable used in an initializer. PTX 3.1 code should either include explicit `generic()` operators in initializers, use `cvta.global` to form generic addresses at runtime, or load from the non-generic address using `ld.global`.

Device function names appearing in initializers represent the address of the first instruction in the function; this can be used to initialize a table of function pointers to be used with indirect calls. Beginning in PTX ISA version 3.1, kernel function names can be used as initializers e.g. to initialize a table of kernel function pointers, to be used with CUDA Dynamic Parallelism to launch kernels from GPU. See the *CUDA Dynamic Parallelism Programming Guide* for details.

Labels cannot be used in initializers.

Variables that hold addresses of variables or functions should be of type `.u8` or `.u32` or `.u64`.

Type `.u8` is allowed only if the `mask()` operator is used.

Initializers are allowed for all types except `.f16`, `.f16x2` and `.pred`.

Examples

```
.global .s32 n = 10;
.global .f32 blur_kernel[][3]
    = {{.05, .1, .05}, {.1, .4, .1}, {.05, .1, .05}};

.global .u32 foo[] = { 2, 3, 5, 7, 9, 11 };
.global .u64 ptr = generic(foo); // generic address of foo[0]
.global .u64 ptr = generic(foo)+8; // generic address of foo[2]
```

5.4.5. Alignment

Byte alignment of storage for all addressable variables can be specified in the variable declaration. Alignment is specified using an optional `.align byte-count` specifier immediately following the state-space specifier. The variable will be aligned to an address which is an integer multiple of *byte-count*. The alignment value *byte-count* must be a power of two. For arrays, alignment specifies the address alignment for the starting address of the entire array, not for individual elements.

The default alignment for scalar and array variables is to a multiple of the base-type size. The default alignment for vector variables is to a multiple of the overall vector size.

Examples

```
// allocate array at 4-byte aligned address.  Elements are bytes.
.const .align 4 .b8 bar[8] = {0,0,0,0,2,0,0,0};
```

Note that all PTX instructions that access memory require that the address be aligned to a multiple of the access size. The access size of a memory instruction is the total number of bytes accessed in memory. For example, the access size of `ld.v4.b32` is 16 bytes, while the access size of `atom.f16x2` is 4 bytes.

5.4.6. Parameterized Variable Names

Since PTX supports virtual registers, it is quite common for a compiler frontend to generate a large number of register names. Rather than require explicit declaration of every name, PTX supports a syntax for creating a set of variables having a common prefix string appended with integer suffixes.

For example, suppose a program uses a large number, say one hundred, of `.b32` variables, named `%r0`, `%r1`, ..., `%r99`. These 100 register variables can be declared as follows:

```
.reg .b32 %r<100>;    // declare %r0, %r1, ..., %r99
```

This shorthand syntax may be used with any of the fundamental types and with any state space, and may be preceded by an alignment specifier. Array variables cannot be declared this way, nor are initializers permitted.

5.4.7. Variable Attributes

Variables may be declared with an optional `.attribute` directive which allows specifying special attributes of variables. Keyword `.attribute` is followed by attribute specification inside parenthesis. Multiple attributes are separated by comma.

Variable and Function Attribute Directive: `.attribute` describes the `.attribute` directive.

5.4.8. Variable and Function Attribute Directive: `.attribute`

`.attribute`

Variable and function attributes

Description

Used to specify special attributes of a variable or a function.

The following attributes are supported.

`.managed`

`.managed` attribute specifies that variable will be allocated at a location in unified virtual memory environment where host and other devices in the system can reference the variable directly. This attribute can only be used with variables in `.global` state space. See the *CUDA UVM-Lite Programming Guide* for details.

`.unified`

`.unified` attribute specifies that function has the same memory address on the host and on other devices in the system. Integer constants `uuid1` and `uuid2` respectively specify upper and lower 64 bits of the unique identifier associated with the function or the variable. This attribute can only be used on device functions or on variables in the `.global` state space. Variables with `.unified` attribute are read-only and must be loaded by specifying `.unified` qualifier on the address operand of `ld` instruction, otherwise the behavior is undefined.

PTX ISA Notes

- ▶ Introduced in PTX ISA version 4.0.
- ▶ Support for function attributes introduced in PTX ISA version 8.0.

Target ISA Notes

- ▶ `.managed` attribute requires `sm_30` or higher.
- ▶ `.unified` attribute requires `sm_90` or higher.

Examples

```
.global .attribute(.managed) .s32 g;  
.global .attribute(.managed) .u64 x;  
  
.global .attribute(.unified(19,95)) .f32 f;  
  
.func .attribute(.unified(0xAB, 0xCD)) bar() { ... }
```

5.5. Tensors

A tensor is a multi-dimensional matrix structure in the memory. Tensor is defined by the following properties:

- ▶ Dimensionality
- ▶ Dimension sizes across each dimension
- ▶ Individual element types
- ▶ Tensor stride across each dimension

PTX supports instructions which can operate on the tensor data. PTX Tensor instructions include:

- ▶ Copying data between global and shared memories
- ▶ Reducing the destination tensor data with the source.

The Tensor data can be operated on by various `wmma.mma`, `mma` and `wgmma.mma_async` instructions.

PTX Tensor instructions treat the tensor data in the global memory as a multi-dimensional structure and treat the data in the shared memory as a linear data.

5.5.1. Tensor Dimension, size and format

Tensors can have dimensions: 1D, 2D, 3D, 4D or 5D.

Each dimension has a size which represents the number of elements along the dimension. The elements can have one the following types:

- ▶ Bit-sized type: `.b32`, `.b64`
- ▶ Sub-byte types: `.b4x16`, `.b4x16_p64`, `.b6x16_p32`, `.b6p2x16`
- ▶ Integer: `.u8`, `.u16`, `.u32`, `.s32`, `.u64`, `.s64`
- ▶ Floating point and alternate floating point: `.f16`, `.bf16`, `.tf32`, `.f32`, `.f64` (rounded to nearest even).

Tensor can have padding at the end in each of the dimensions to provide alignment for the data in the subsequent dimensions. Tensor stride can be used to specify the amount of padding in each dimension.

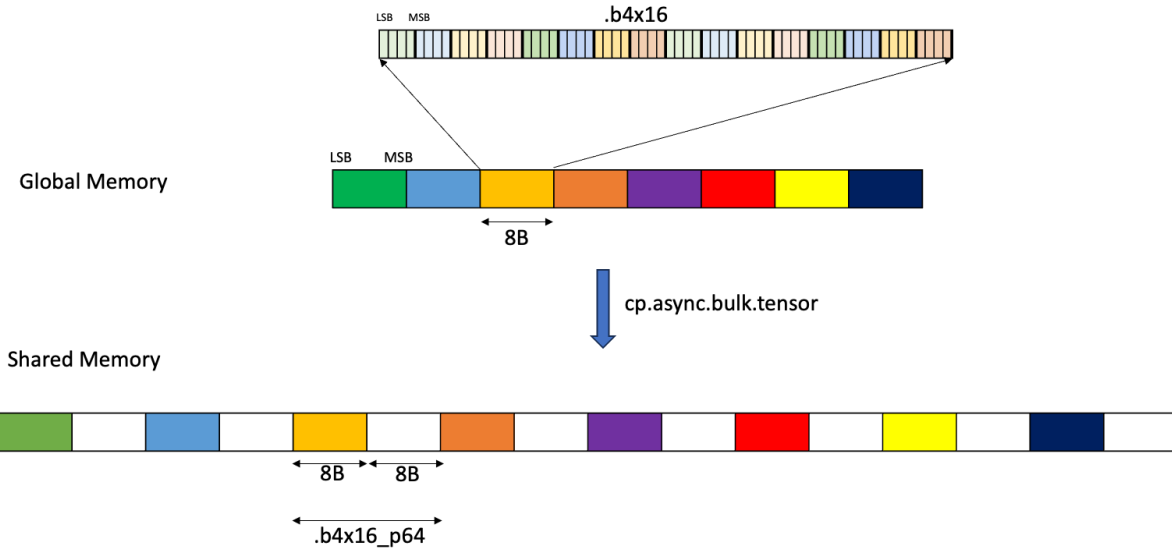
5.5.1.1 Sub-byte Types

5.5.1.1.1 Padding and alignment of the sub-byte types

The sub-byte types are expected to be packed contiguously in the global memory and the Tensor copy instruction will expand them by appending empty spaces as shown below:

1. Type `.b4x16`: With this type, there is no padding involved and the packed sixteen `.b4` elements in a 64-bits container is copied as is between the shared memory and the global memory.
2. Type `.b4x16_p64`: With this type, sixteen contiguous 4-bits of data is copied from global memory to the shared memory with the append of 64-bits of padding as shown in [Figure 5](#)

The padded region that gets added is un-initialized.

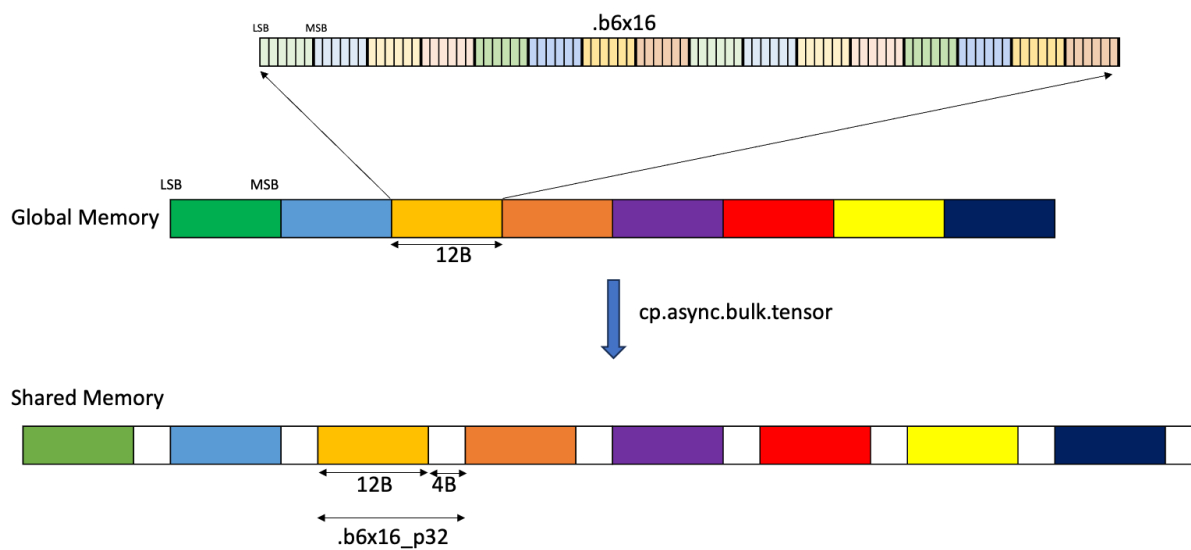
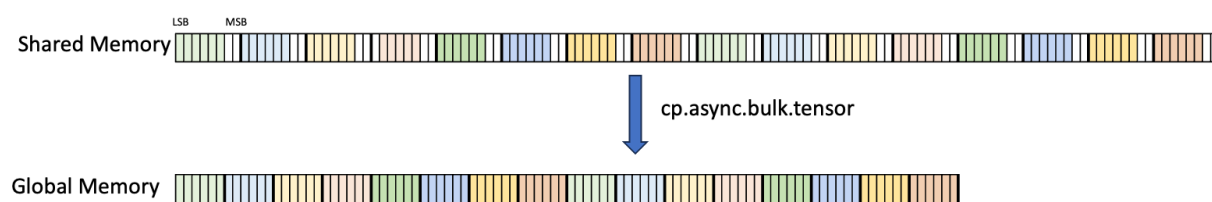
Figure 5: Layout for `.b4x16_p64`

3. Type `.b6x16_p32`: With this type, sixteen 6-bits of data is copied from global memory to the shared memory with an append of 32-bits of padding as shown in [Figure 6](#). The padded region that gets added is un-initialized.
4. Type `.b6p2x16`: With this type, sixteen elements, each containing 6-bits of data at the LSB and 2-bits of padding at the MSB, are copied from shared memory into the global memory by discarding the 2-bits of padding data and packing the 6-bits data contiguously as shown in [Figure 7](#).

In case of `.b6x16_p32` and `.b4x16_p64`, the padded region that gets added is un-initialized.

The types `.b6x16_p32` and `.b6p2x16` share the same encoding value in the descriptor (value 15) as the two types are applicable for different types of tensor copy operations:

Type	Valid Tensor Copy Direction
<code>.b6x16_p32</code>	<code>.shared::cluster.global, .shared::cta.global</code>
<code>.b6p2x16</code>	<code>.global.shared::cta</code>

Figure 6: Layout for `.b6x16_p32`Figure 7: Layout for `.b6p2x16`

5.5.2. Tensor Access Modes

Tensor data can be accessed in two modes:

- ▶ Tiled mode:

In tiled mode, the source multi-dimensional tensor layout is preserved at the destination.

- ▶ Im2col mode:

In im2col mode, the elements in the Bounding Box of the source tensor are rearranged into columns at the destination. Refer [here](#) for more details.

5.5.3. Tiled Mode

This section talks about how Tensor and Tensor access work in tiled mode.

5.5.3.1 Bounding Box

A tensor can be accessed in chunks known as *Bounding Box*. The Bounding Box has the same dimensionality as the tensor they are accessing into. Size of each bounding Box must be a multiple of 16 bytes. The address of the bounding Box must also be aligned to 16 bytes.

Bounding Box has the following access properties:

- ▶ Bounding Box dimension sizes
- ▶ Out of boundary access mode
- ▶ Traversal strides

The tensor-coordinates, specified in the PTX tensor instructions, specify the starting offset of the bounding box. Starting offset of the bounding box along with the rest of the bounding box information together are used to determine the elements which are to be accessed.

5.5.3.2 Traversal-Stride

While the Bounding Box is iterating the tensor across a dimension, the traversal stride specifies the exact number of elements to be skipped. If no jump over is required, default value of 1 must be specified.

The traversal stride in dimension 0 can be used for the *Interleave layout*. For non-interleaved layout, the traversal stride in dimension 0 must always be 1.

[Figure 8](#) illustrates tensor, tensor size, tensor stride, Bounding Box size and traversal stride.

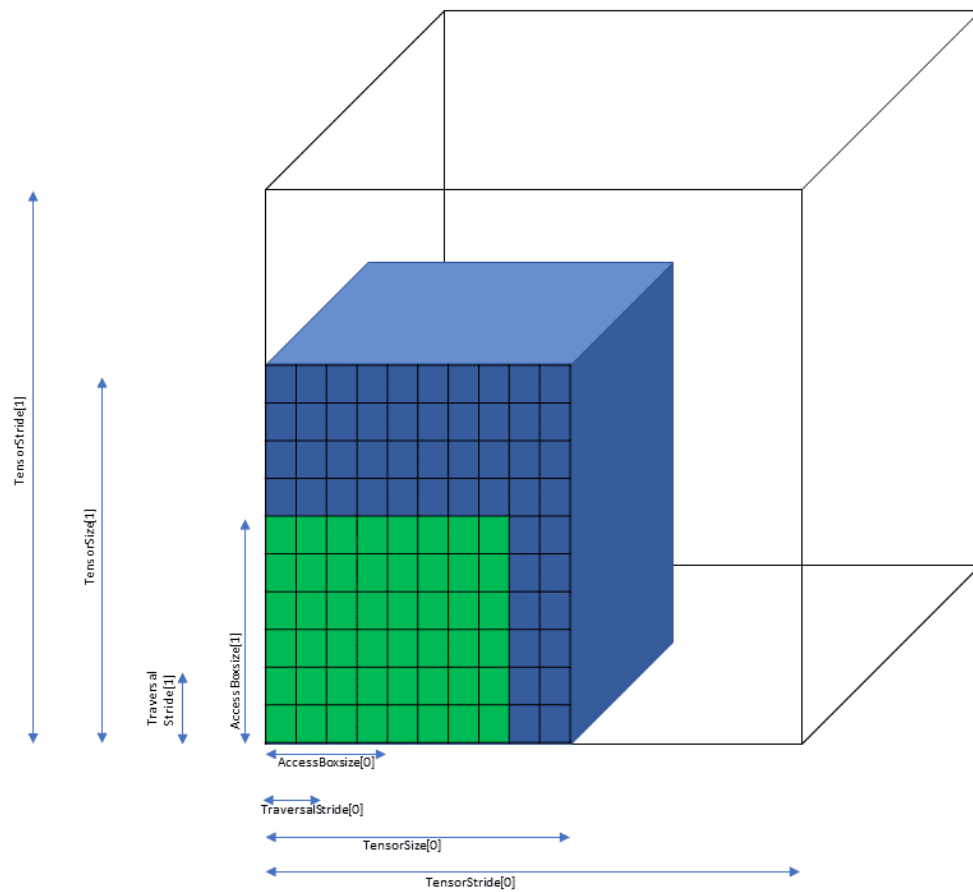


Figure 8: Tiled mode bounding box, tensor size and traversal stride

5.5.3.3 Out of Boundary Access

PTX Tensor operation can detect and handle the case when the Bounding Box crosses the tensor boundary in any dimension. There are 2 modes:

- ▶ Zero fill mode:

Elements in the Bounding Box which fall outside of the tensor boundary are set to 0.

- ▶ 00B-NaN fill mode:

Elements in the Bounding Box which fall outside of the tensor boundary are set to a special NaN called 00B-NaN.

Figure 9 shows an example of the out of boundary access.

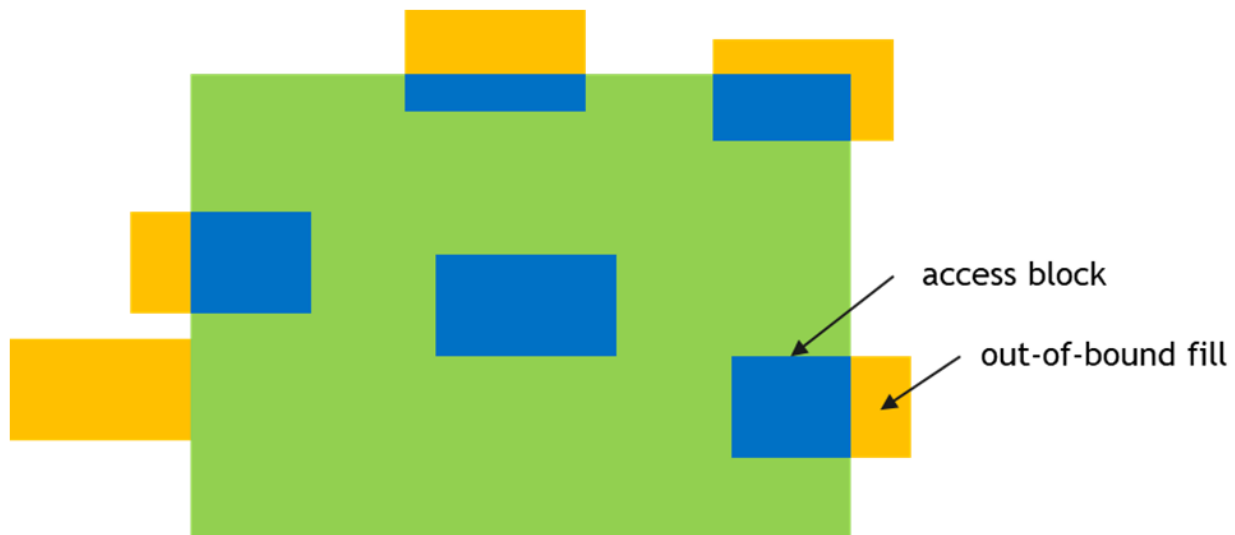


Figure 9: Out of boundary access

5.5.3.4 .tile::scatter4 and .tile::gather4 modes

These modes are similar to the tiled mode with restriction that these modes work only on 2D tensor data. `Tile::scatter4` and `Tile::gather4` modes are used to access multiple non-contiguous rows of tensor data.

In `Tile::scatter4` mode single 2D source tensor is divided into four rows in the 2D destination tensor. In `Tile::gather4` mode four rows in the source 2D tensor are combined to form single 2D destination tensor.

These modes work on four rows and hence the instruction will take:

1. four tensor coordinates across the dimension 0
2. one tensor coordinate across the dimension 1

The interleave layout is not supported for `.tile::scatter4` and `.tile::gather4` modes.

All other constraints and rules of the tile mode apply to these modes as well.

5.5.3.4.1 Bounding Box

For `Tile::scatter4` and `Tile::gather4` modes, four request coordinates will form four Bounding Boxes in the tensor space.

Figure 10 shows an example of the same with start coordinates (1, 2), (1, 5), (1, 0) and (1, 9).

The size of the bounding box in the dimension 0 represents the length of the rows. The size of the bounding box in the dimension 1 must be one.

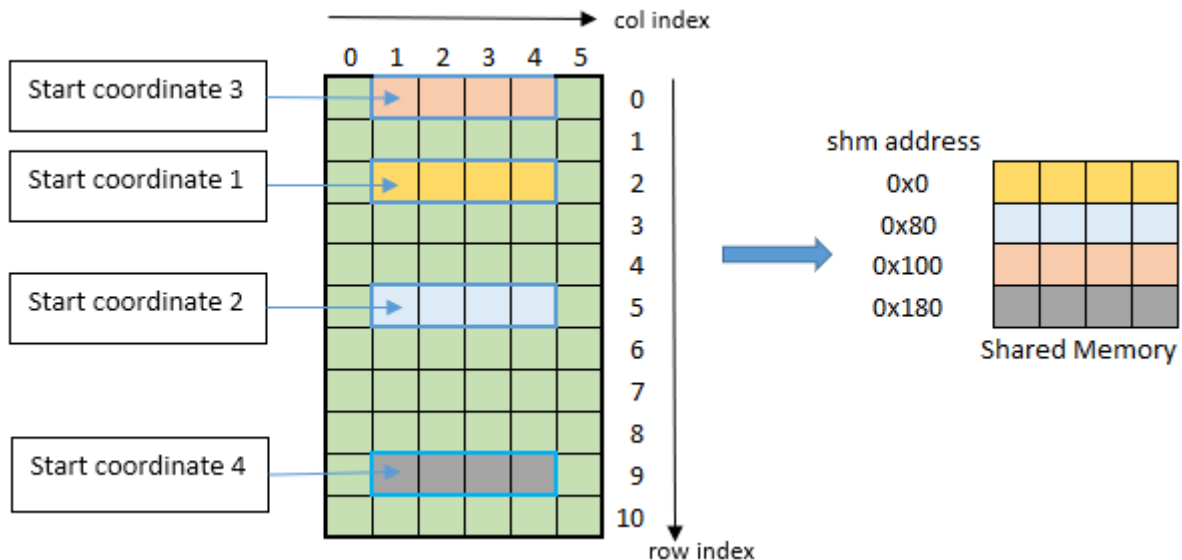


Figure 10: tiled::scatter4/tiled::gather4 mode bounding box example

5.5.4. im2col mode

Im2col mode supports the following tensor dimensions : 3D, 4D and 5D. In this mode, the tensor data is treated as a batch of images with the following properties:

- N : number of images in the batch
- D, H, W : size of a 3D image (depth, height and width)
- C: channels per image element

The above properties are associated with 3D, 4D and 5D tensors as follows:

Dimension	N/D/H/W/C applicability
3D	NWC
4D	NHWC
5D	NDHWC

5.5.4.1 Bounding Box

In im2col mode, the Bounding Box is defined in DHW space. Boundaries along other dimensions are specified by Pixels-per-Column and Channels-per-Pixel parameters as described below.

The dimensionality of the Bounding Box is two less than the tensor dimensionality.

The following properties describe how to access of the elements in im2col mode:

- Bounding-Box Lower-Corner
- Bounding-Box Upper-Corner
- Pixels-per-Column
- Channels-per-Pixel

Bounding-box Lower-Corner and *Bounding-box Upper-Corner* specify the two opposite corners of the Bounding Box in the DHW space. *Bounding-box Lower-Corner* specifies the corner with the smallest coordinate and *Bounding-box Upper-Corner* specifies the corner with the largest coordinate.

Bounding-box Upper- and *Lower-Corners* are 16-bit signed values whose limits varies across the dimensions and are as shown below:

	3D	4D	5D
Upper- / Lower- Corner sizes	$[-2^{15}, 2^{15}-1]$	$[-2^7, 2^7-1]$	$[-2^4, 2^4-1]$

Figure 11 and Figure 12 show the Upper-Corners and Lower-Corners.

The *Bounding-box Upper-* and *Lower- Corners* specify only the boundaries and not the number of elements to be accessed. *Pixels-per-Column* specifies the number of elements to be accessed in the NDHW space.

Channels-per-Pixel specifies the number of elements to access across the C dimension.

The tensor coordinates, specified in the PTX tensor instructions, behaves differently in different dimensions:

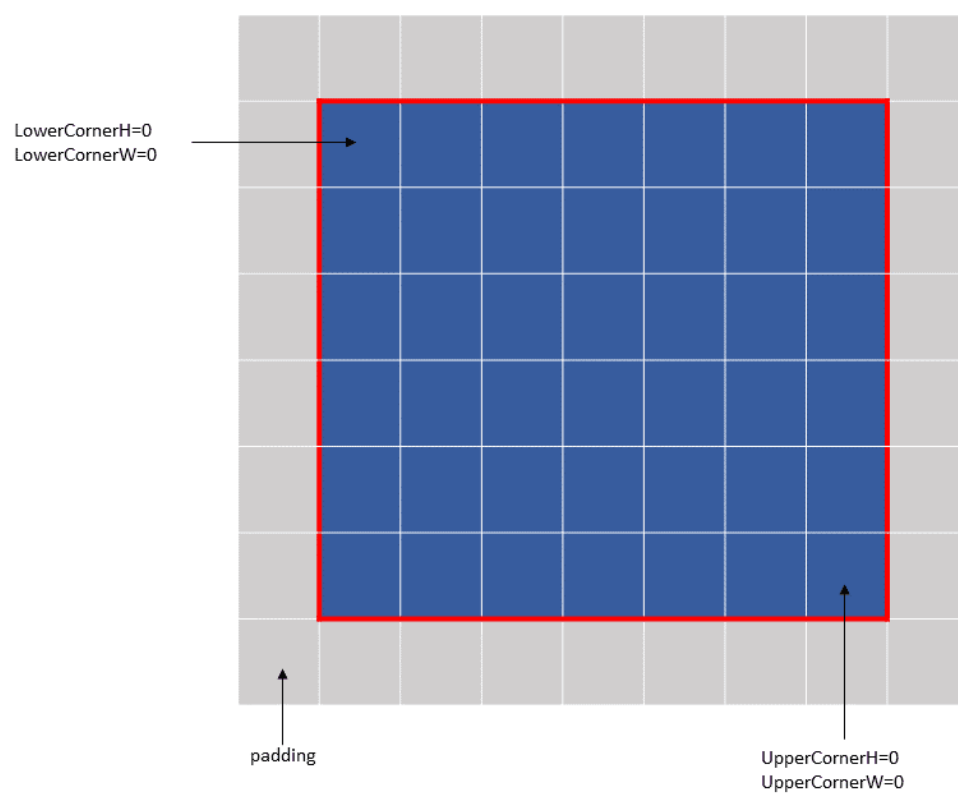


Figure 11: im2col mode bounding box example 1

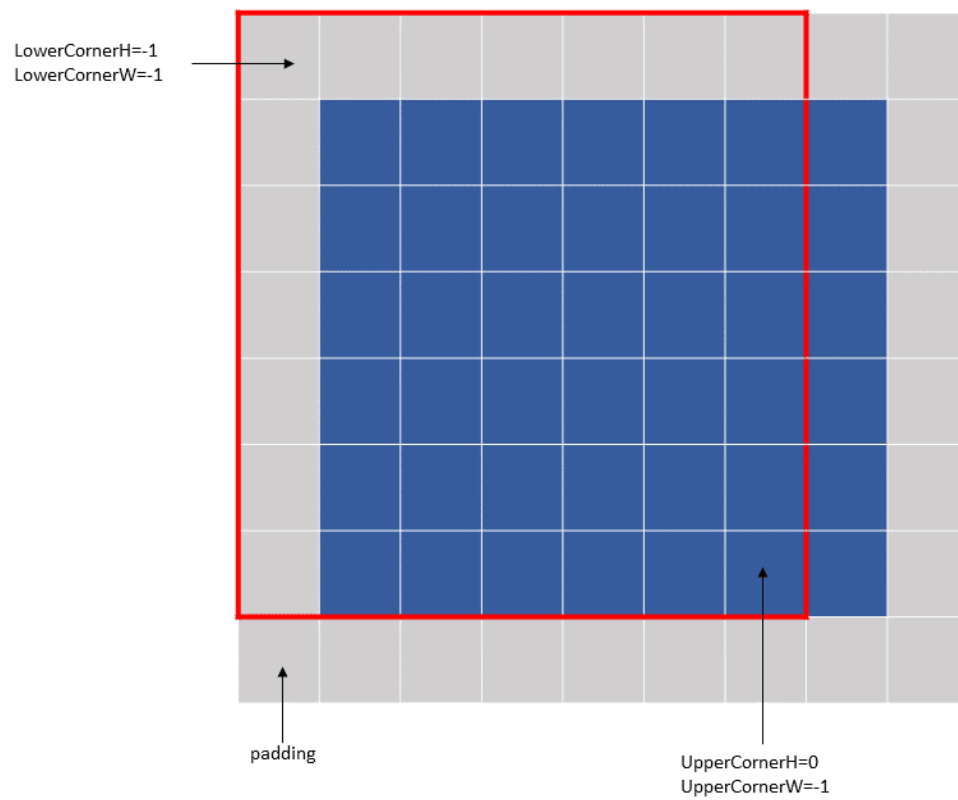


Figure 12: im2col mode bounding box example 2

- Across N and C dimensions: specify the starting offsets along the dimension, similar to the tiled mode.
- Across DHW dimensions: specify the location of the convolution filter base in the tensor space. The filter corner location must be within the bounding box.

The im2col offsets, specified in the PTX tensor instructions in im2col mode, are added to the filter base coordinates to determine the starting location in the tensor space from where the elements are accessed.

The size of the im2col offsets varies across the dimensions and their valid ranges are as shown below:

	3D	4D	5D
im2col offsets range	$[0, 2^{16}-1]$	$[0, 2^8-1]$	$[0, 2^5-1]$

Following are some examples of the im2col mode accesses:

- Example 1 (Figure 13):

```
Tensor Size[0] = 64
Tensor Size[1] = 9
Tensor Size[2] = 14
Tensor Size[3] = 64
Pixels-per-Column = 64
channels-per-pixel = 8
Bounding-Box Lower-Corner W = -1
Bounding-Box Lower-Corner H = -1
Bounding-Box Upper-Corner W = -1
Bounding-Box Upper-Corner H = -1.

tensor coordinates = (7, 7, 4, 0)
im2col offsets : (0, 0)
```

- Example 2 (Figure 14):

```
Tensor Size[0] = 64
Tensor Size[1] = 9
Tensor Size[2] = 14
Tensor Size[3] = 64
Pixels-per-Column = 64
channels-per-pixel = 8
Bounding-Box Lower-Corner W = 0
Bounding-Box Lower-Corner H = 0
Bounding-Box Upper-Corner W = -2
Bounding-Box Upper-Corner H = -2

tensor coordinates = (7, 7, 4, 0)
im2col offsets: (2, 2)
```

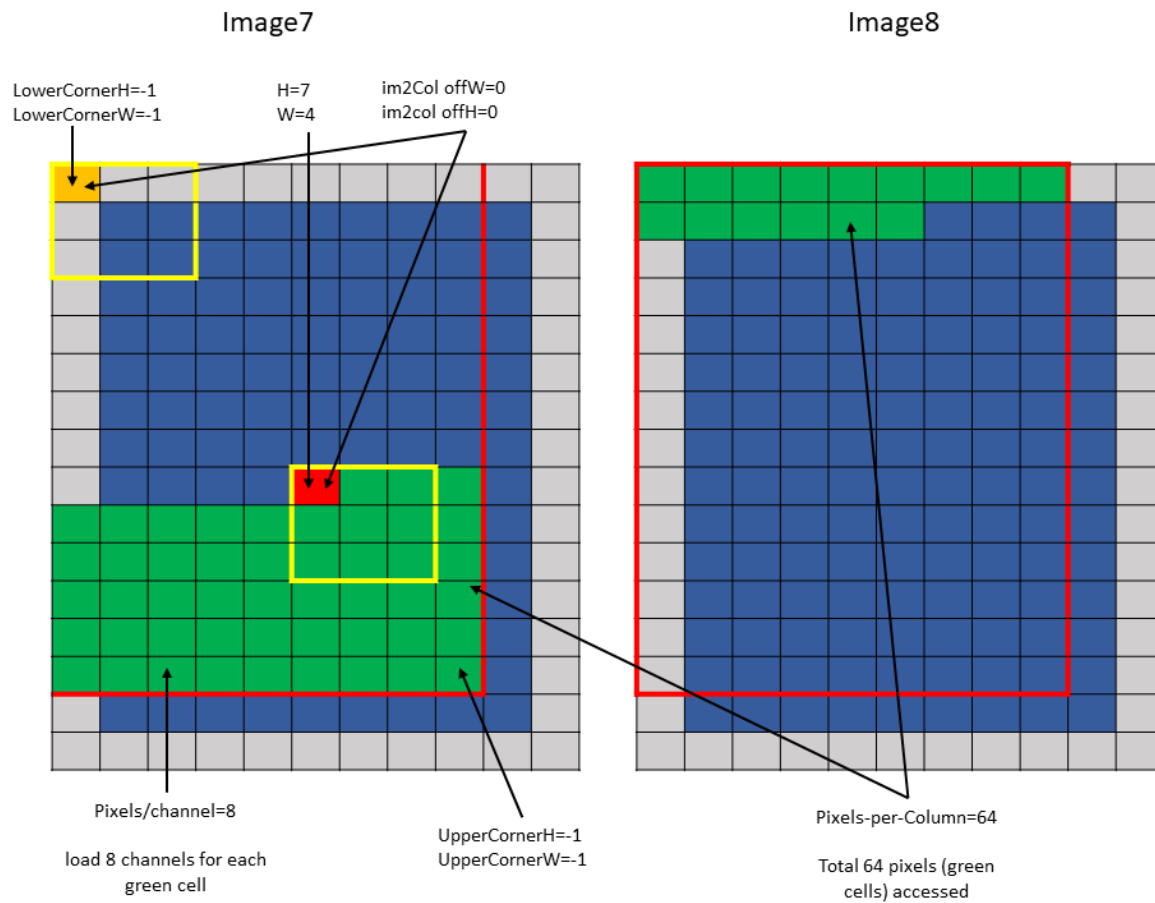


Figure 13: im2col mode example 1

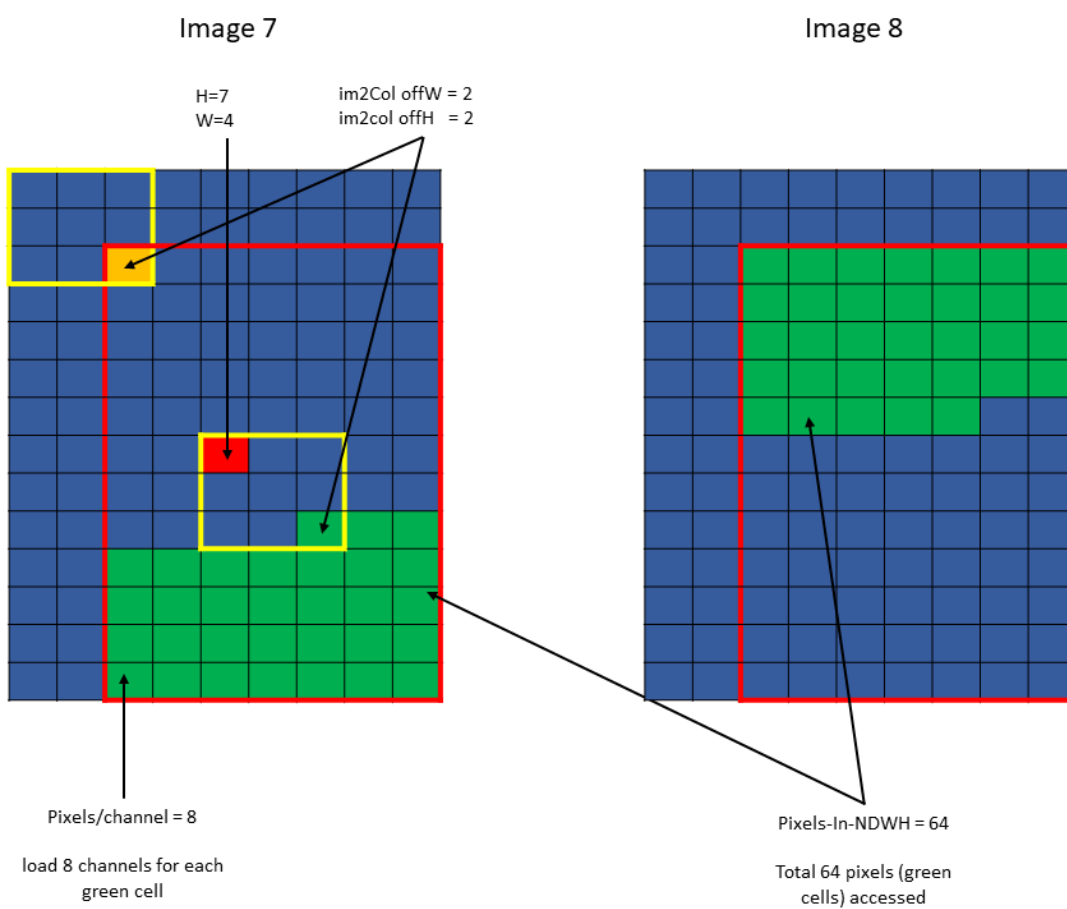


Figure 14: im2col mode example 2

5.5.4.2 Traversal Stride

The traversal stride, in im2col mode, does not impact the total number of elements (or pixels) being accessed unlike the tiled mode. Pixels-per-Column determines the total number of elements being accessed, in im2col mode.

The number of elements traversed along the D, H and W dimensions is strided by the traversal stride for that dimension.

The following example with Figure 15 illustrates accesses with traversal-strides:

```
Tensor Size[0] = 64
Tensor Size[1] = 8
Tensor Size[2] = 14
Tensor Size[3] = 64
Traversal Stride = 2
Pixels-per-Column = 32
channels-per-pixel = 16
Bounding-Box Lower-Corner W = -1
Bounding-Box Lower-Corner H = -1
Bounding-Box Upper-Corner W = -1
Bounding-Box Upper-Corner H = -1.
Tensor coordinates in the instruction = (7, 7, 5, 0)
Im2col offsets in the instruction : (1, 1)
```

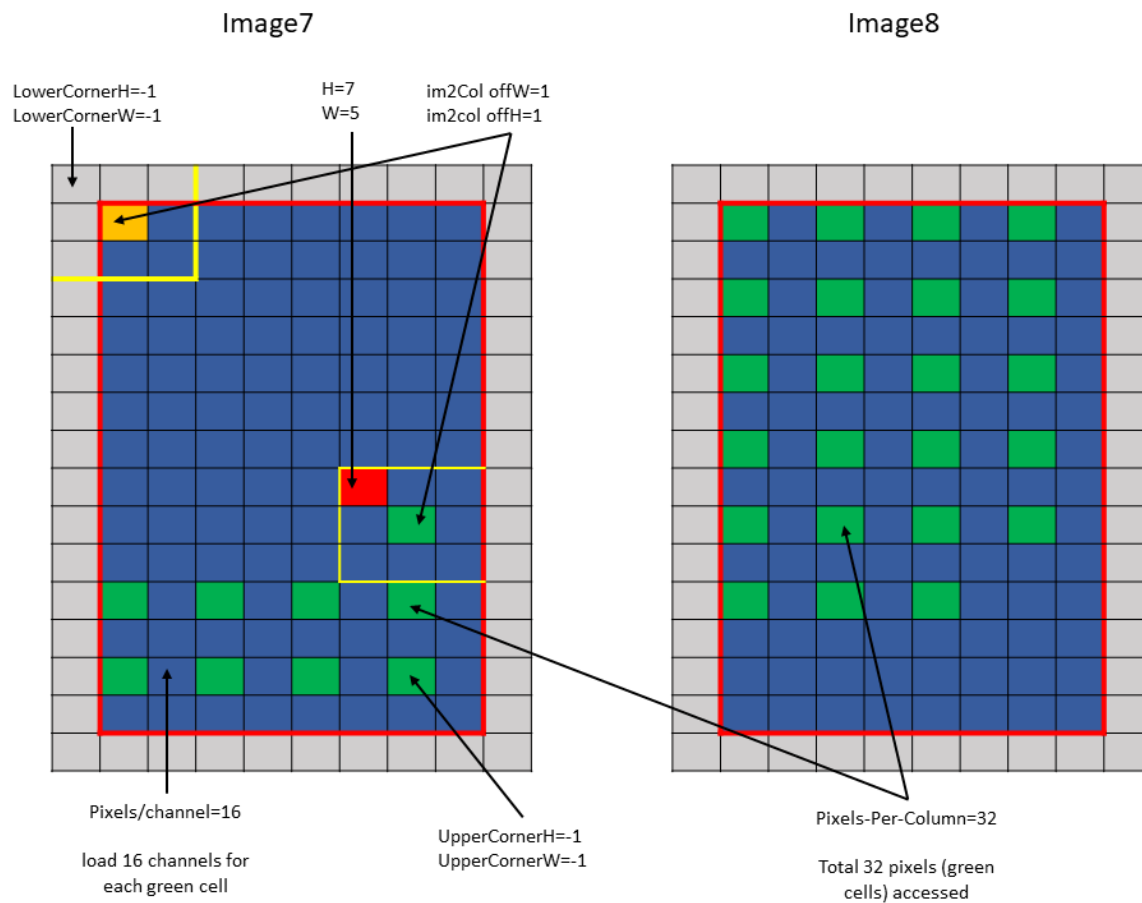


Figure 15: im2col mode traversal stride example

5.5.4.3 Out of Boundary Access

In `im2col` mode, when the number of requested pixels in NDHW space specified by *Pixels-per-Column* exceeds the number of available pixels in the image batch then out-of-bounds access is performed.

Similar to tiled mode, zero fill or 00B-NaN fill can be performed based on the Fill-Mode specified.

5.5.5. `im2col::w` and `im2col::w::128` modes

These modes are similar to the `im2col` mode with the restriction that elements are accessed across the W dimension only while keeping the H and D dimension constant.

All the constraints and rules of the `im2col` mode apply to these modes as well.

The number of elements accessed in the `im2col::w::128` mode is fixed and is equal to 128. The number of elements accessed in the `im2col::w` mode depends on the field *Pixels-per-Column* field in the *TensorMap*.

5.5.5.1 Bounding Box

In these modes, the size of the bounding box in D and H dimensions are 1.

The D and H dimensions in the tensor coordinates argument in the PTX instruction specify the position of the bounding box in the tensor space.

The Bounding-Box Lower-Corner-W and Bounding-Box Upper-Corner-W specify the two opposite corners of the Bounding Box in the W dimension.

The W dimension in the tensor coordinates argument in the PTX instruction specify the location of the first element that is to be accessed in the bounding box.

Number of pixels loaded in `im2col::w` mode is as specified by *Pixels-per-Column* in the *TensorMap*. Number of pixels loaded in `im2col::w::128` mode is always 128. So, *Pixels-per-Column* is ignored in `im2col::w::128` mode.

Figure 16 shows an example of the `im2col::w` and `im2col::w::128` modes.

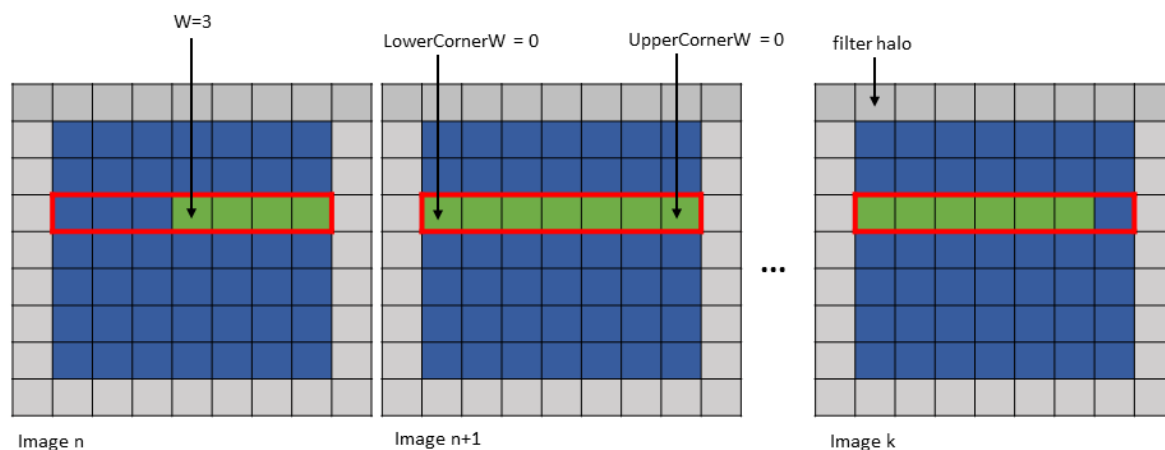


Figure 16: `im2col::w` and `im2col::w::128` modes example

The first element can lie outside of the Bounding Box in the W-dimension only and only on the left side of the Bounding Box. Figure 17 shows of an example of this.

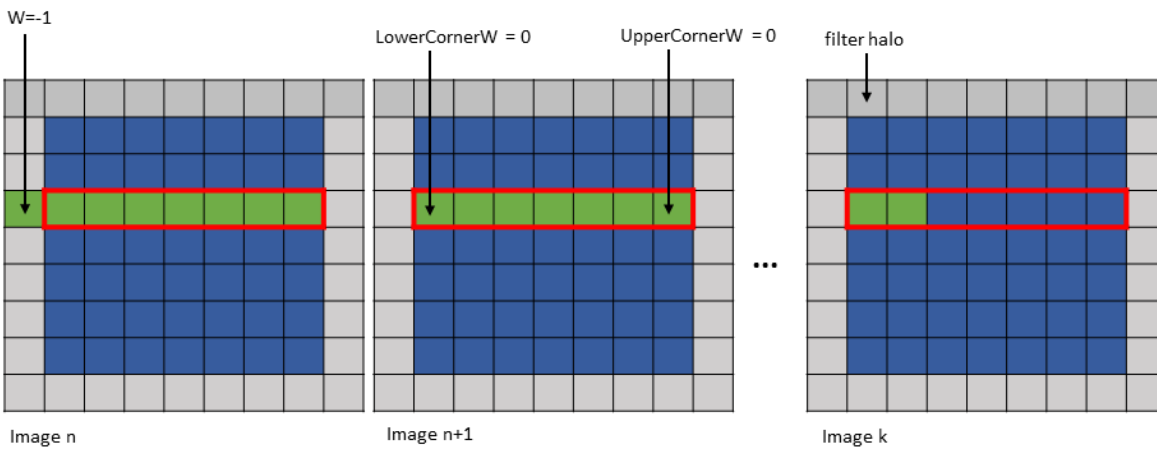


Figure 17: im2col::w and im2col::w::128 modes first element outside Bounding Box example

5.5.5.2 Traversal Stride

This is similar to im2col mode with the exception of that the number of elements traversed along only the W dimension is strided by the traversal stride as specified in the TensorMap.

5.5.5.3 wHalo

In im2col::w mode, the wHalo argument in the PTX instruction specifies how many filter halo elements must be loaded at the end of the image.

In im2col::w::128 mode, the halo elements are loaded after every 32 elements in the bounding box along the W dimension. The wHalo argument in the PTX instruction specifies how many halo elements must be loaded after every 32 elements.

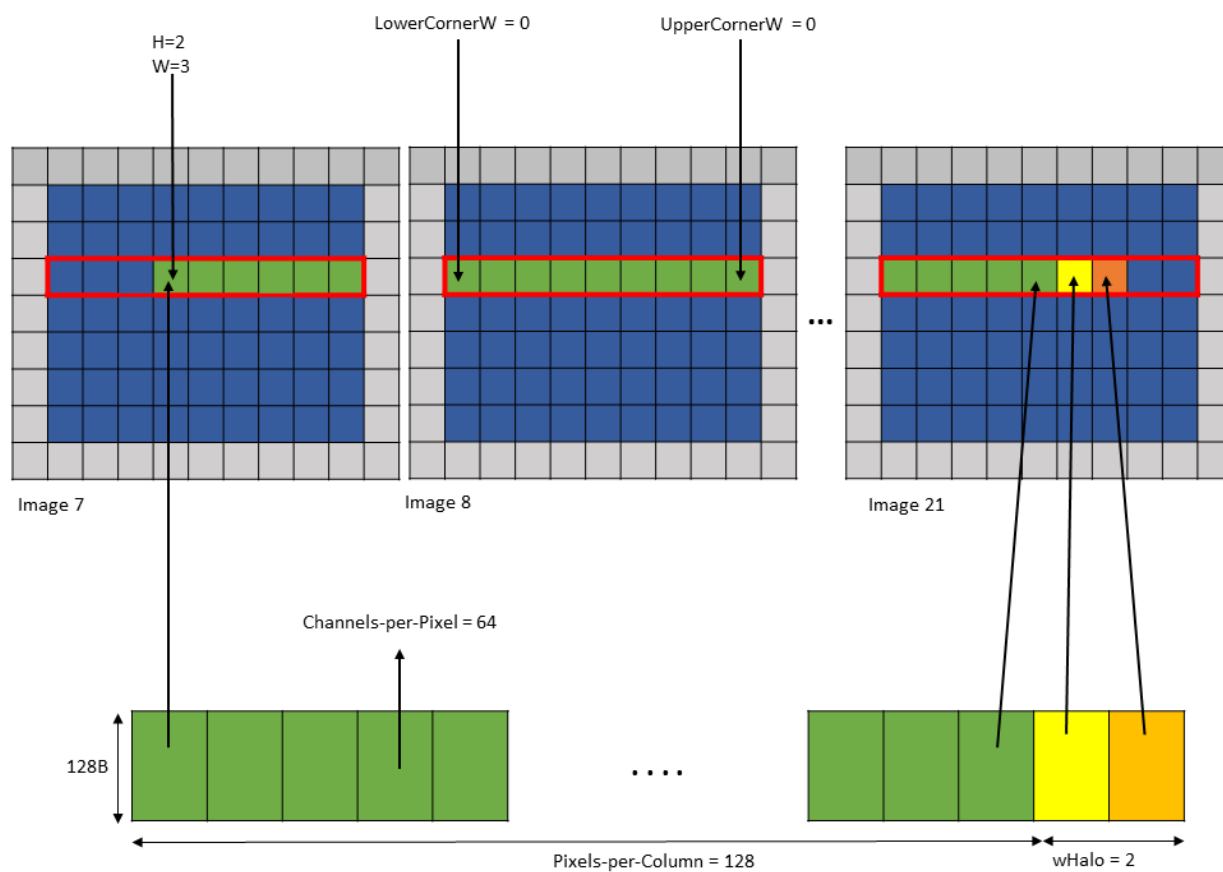
Following is an example of .im2col::w mode access:

```
Tensor Size [0] = 128
Tensor Size [1] = 9
Tensor Size [2] = 7
Tensor Size [3] = 64
Pixels-per-column = 128
Channels-per-pixel = 64
Bounding Box Lower Corner W = 0
Bounding Box Upper Corner W = 0

Tensor Coordinates in the instruction = (7, 2, 3, 0)
wHalo in the instruction = 2 (as 3x3 convolution filter is used)
```

A tensor copy operation with the above parameters loads 128 pixels and the two halo pixels as shown in Figure 18.

The halo pixels are always loaded in the shared memory next to the main row pixels as shown in Figure 18.

Figure 18: tensor copy operation with `im2col:w` mode example

Following is an example of `.im2col::w::128` mode access:

```
Tensor Size [0] = 128
Tensor Size [1] = 9
Tensor Size [2] = 7
Tensor Size [3] = 64
Channels-per-pixel = 64
Bounding Box Lower Corner W = 0
Bounding Box Upper Corner W = 0

Tensor Coordinates in the instruction = (7, 2, 3, 0)
wHalo in the instruction = 2 (as 3x3 convolution filter is used)
```

A tensor copy operation with the above parameters loads 128 elements such that after every 32 elements, wHalo number of elements are loaded as shown in [Figure 19](#).

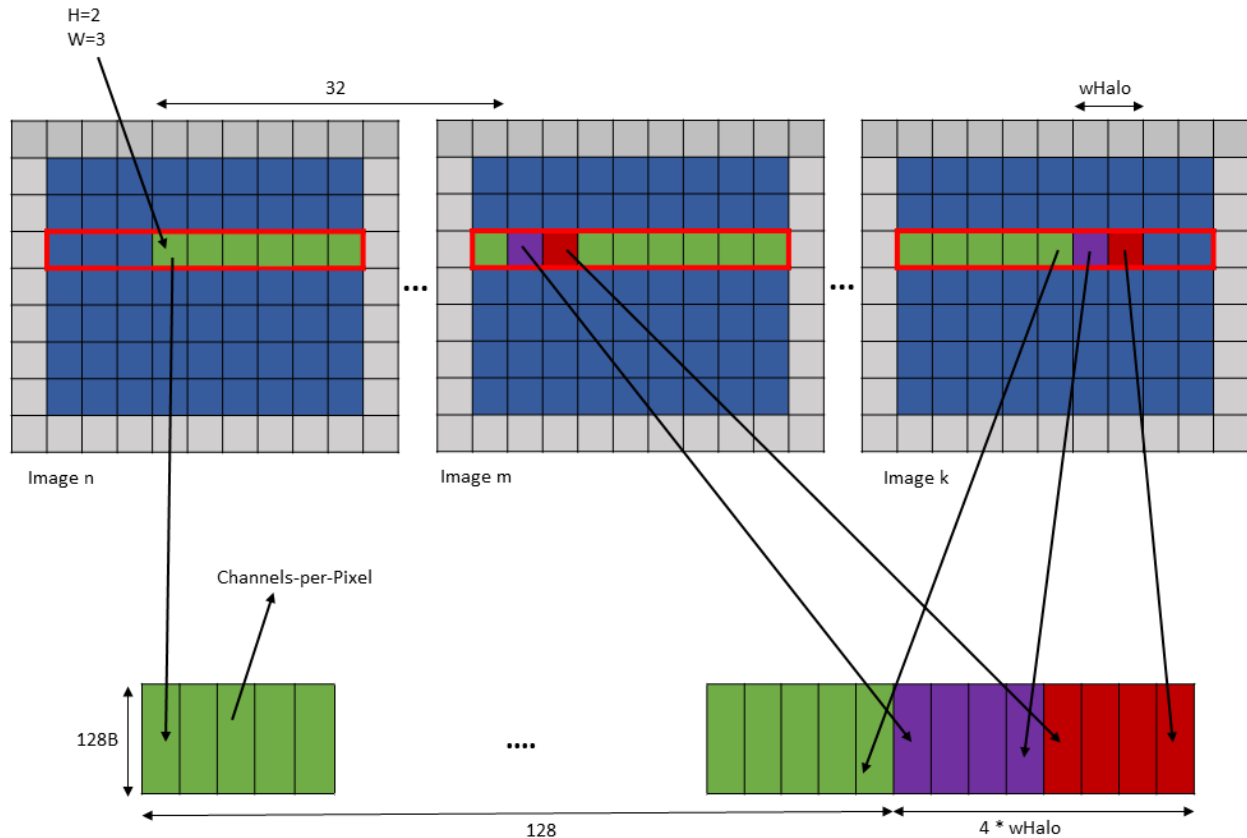


Figure 19: tensor copy operation with `im2col::w::128` mode example

5.5.5.4 wOffset

In the convolution calculations, the same elements along the W dimension are reused for different locations within the convolution filter footprint. Based on the number of times a pixel is used, the pixels may be loaded into different shared memory buffers. Each buffer can be loaded by a separate tensor copy operation.

The wOffset argument in the tensor copy and prefetch instruction adjusts the source pixel location for each buffer. The exact position of the buffer is adjusted along the W dimension using the following formula:

```
Bounding Box Lower Corner W += wOffset
Bounding Box Upper Corner W += wOffset
W += wOffset
```

Following are examples of tensor copy to multiple buffers with various wHalo and wOffset values:

Example 1:

```
Tensor Size [0] = 128
Tensor Size [1] = 9
Tensor Size [2] = 67
Tensor Size [3] = 64
Pixels-per-Column = 128
Channels-per-pixel = 64
Bounding Box Lower Corner W = -1
Bounding Box Upper Corner W = 0
Traversal Stride = 2

Tensor Coordinates in the instruction = (7, 2, -1, 0)

Shared memory buffer 1:
    wHalo = 1
    wOffset = 0

Shared memory buffer 2:
    wHalo = 0
    wOffset = 1
```

Example 2:

```
Tensor Size [0] = 128
Tensor Size [1] = 7
Tensor Size [2] = 7
Tensor Size [3] = 64
Pixels-per-Column = 128
Channels-per-pixel = 64
Bounding Box Lower Corner W = -1
Bounding Box Upper Corner W = -1
Traversal Stride = 3

Tensor Coordinates in the instruction = (7, 2, -1, 0)

Shared memory buffer 1:
    wHalo = 0
    wOffset = 0

Shared memory buffer 2:
```

(continues on next page)

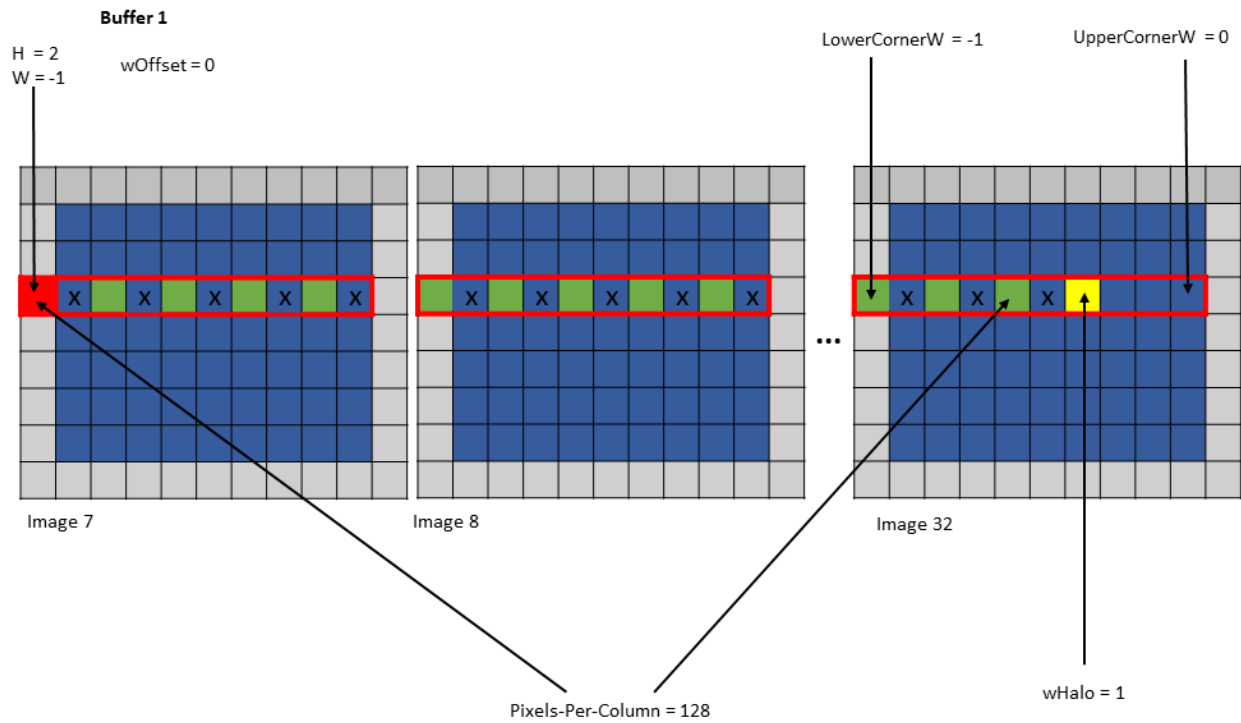


Figure 20: tensor copy operation to buffer 1 of Example 1

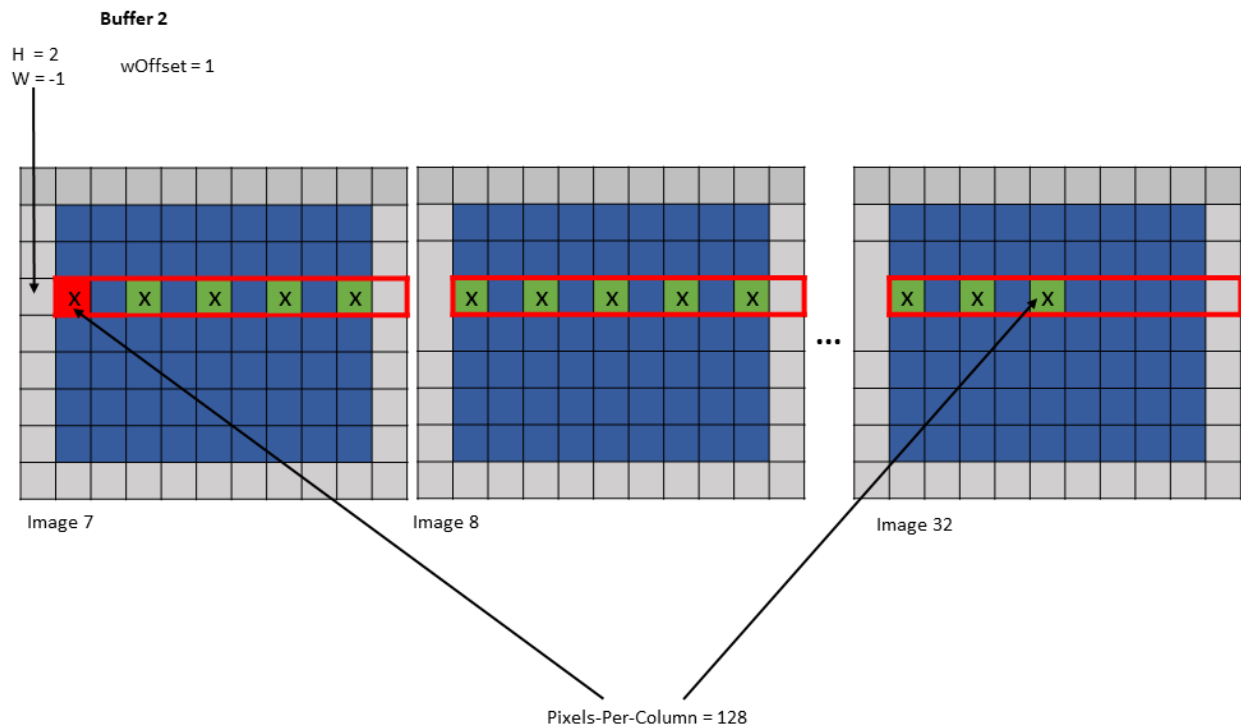


Figure 21: tensor copy operation to buffer 2 of Example 1

(continued from previous page)

```
wHalo = 0
wOffset = 1
```

Shared memory buffer 3:

```
wHalo = 0
wOffset = 2
```

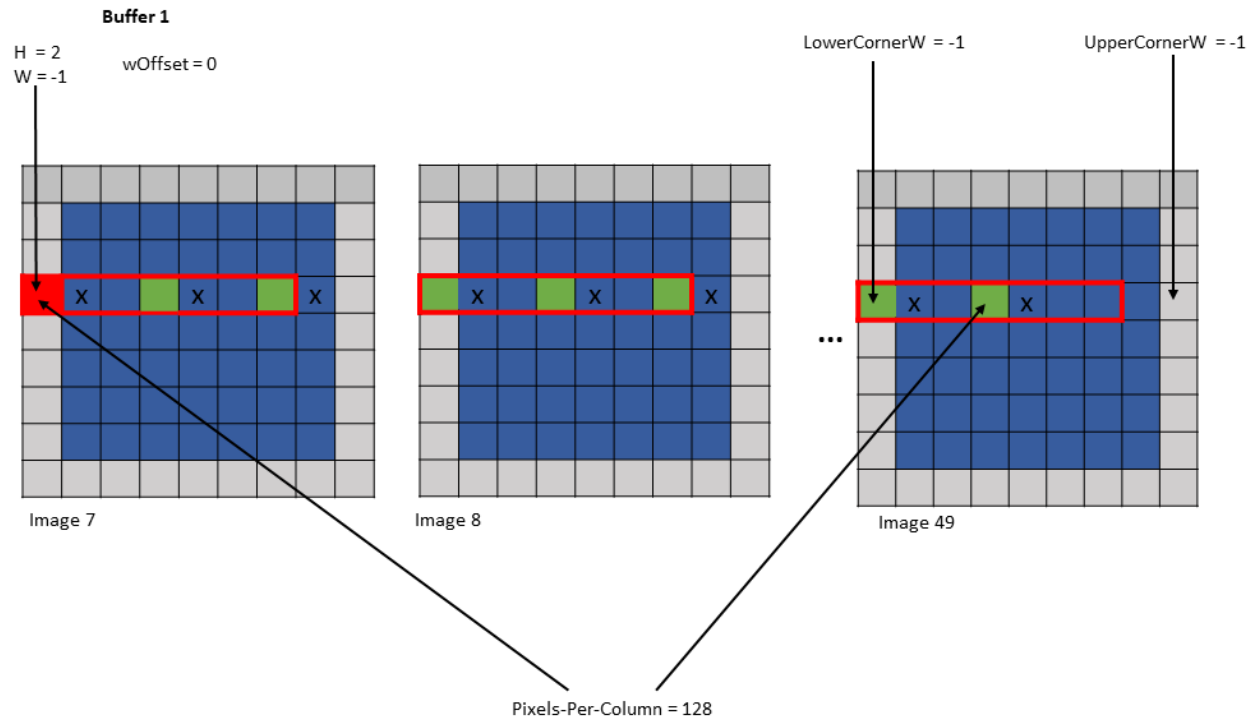


Figure 22: tensor copy operation to buffer 1 of Example 2

5.5.6. Interleave layout

Tensor can be interleaved and the following interleave layouts are supported:

- ▶ No interleave (NDHWC)
- ▶ 8 byte interleave (NC/8DHWC8) : C8 utilizes 16 bytes in memory assuming 2B per channel.
- ▶ 16 byte interleave (NC/16HWC16) : C16 utilizes 32 bytes in memory assuming 4B per channel.

The C information is organized in slices where sequential C elements are grouped in 16 byte or 32 byte quantities.

If the total number of channels is not a multiple of the number of channels per slice, then the last slice must be padded with zeros to make it complete 16B or 32B slice.

Interleaved layouts are supported only for the dimensionalities : 3D, 4D and 5D.

The interleave layout is not supported for .im2col:w and .im2col:w::128 modes.

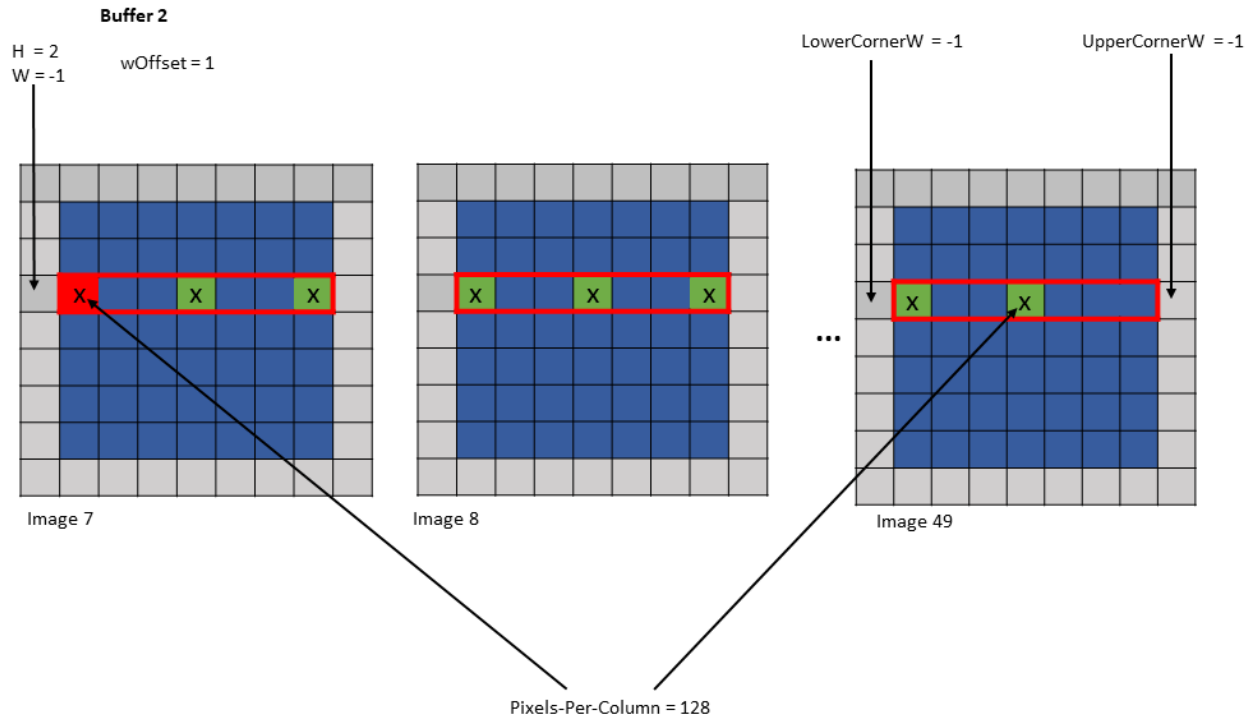


Figure 23: tensor copy operation to buffer 2 of Example 2

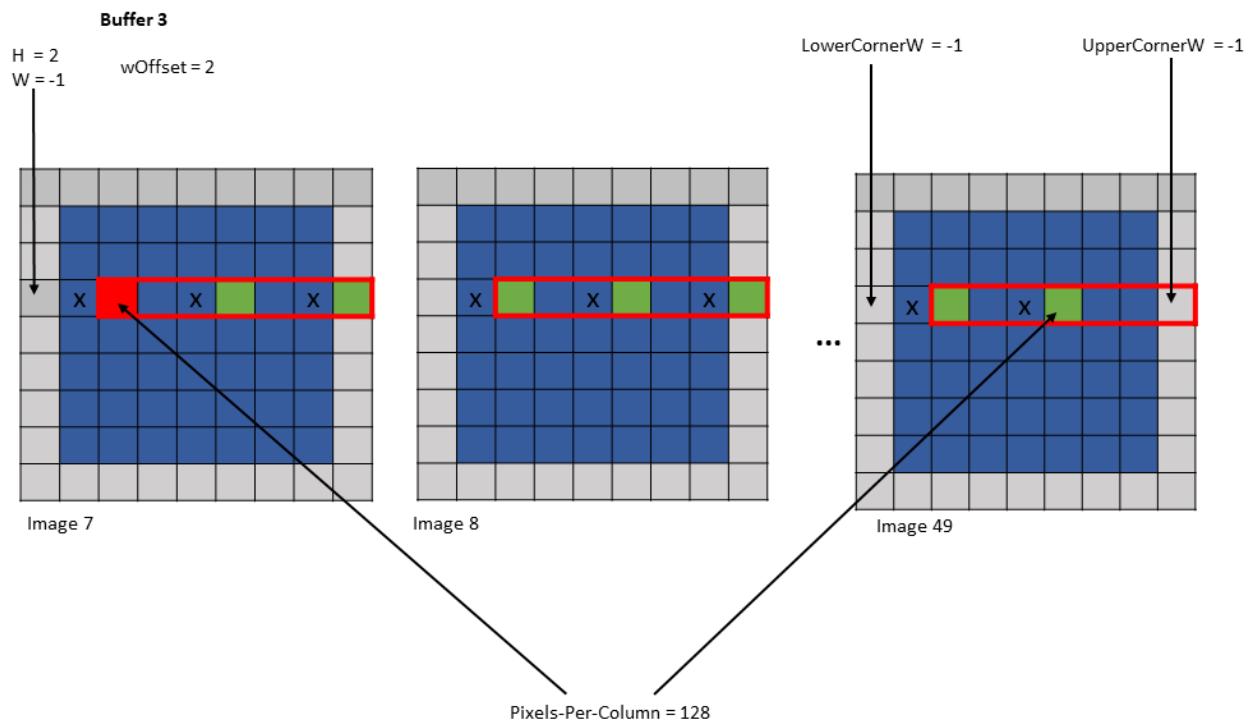


Figure 24: tensor copy operation to buffer 3 of Example 2

5.5.7. Swizzling Modes

The layout of the data in the shared memory can be different to that of global memory, for access performance reasons. The following describes various swizzling modes:

► No swizzle mode:

There is no swizzling in this mode and the destination data layout is exactly similar to the source data layout.

0	1	2	3	4	5	6	7
0	1	2	3	4	5	6	7
... Pattern repeats ...							

► 32 byte swizzle mode:

The following table, where each elements (numbered cell) is 16 byte and the starting address is 256 bytes aligned, shows the pattern of the destination data layout:

0	1	2	3	4	5	6	7
1	0	3	2	5	4	7	6
... Pattern repeats ...							

An example of the 32 byte swizzle mode for NC/(32B)HWC(32B) tensor of 1x2x10x10xC16 dimension, with the innermost dimension holding slice of 16 channels with 2 byte/channel, is shown in [Figure 25](#).

[Figure 26](#) shows the two fragments of the tensor : one for $C/(32B) = 0$ and another for $C/(32B) = 1$.

[Figure 27](#) shows the destination data layout with 32 byte swizzling.

► 64 byte swizzle mode:

The following table, where each elements (numbered cell) is 16 byte and the starting address is 512 bytes aligned, shows the pattern of the destination data layout:

0	1	2	3	4	5	6	7
1	0	3	2	5	4	7	6
2	3	0	1	6	7	4	5
3	2	1	0	7	6	5	4
... Pattern repeats ...							

An example of the 64 byte swizzle mode for NHWC tensor of 1x10x10x64 dimension, with 2 bytes / channel and 32 channels, is shown in [Figure 28](#).

Each colored cell represents 8 channels. [Figure 29](#) shows the source data layout.

[Figure 30](#) shows the destination data layout with 64 byte swizzling.

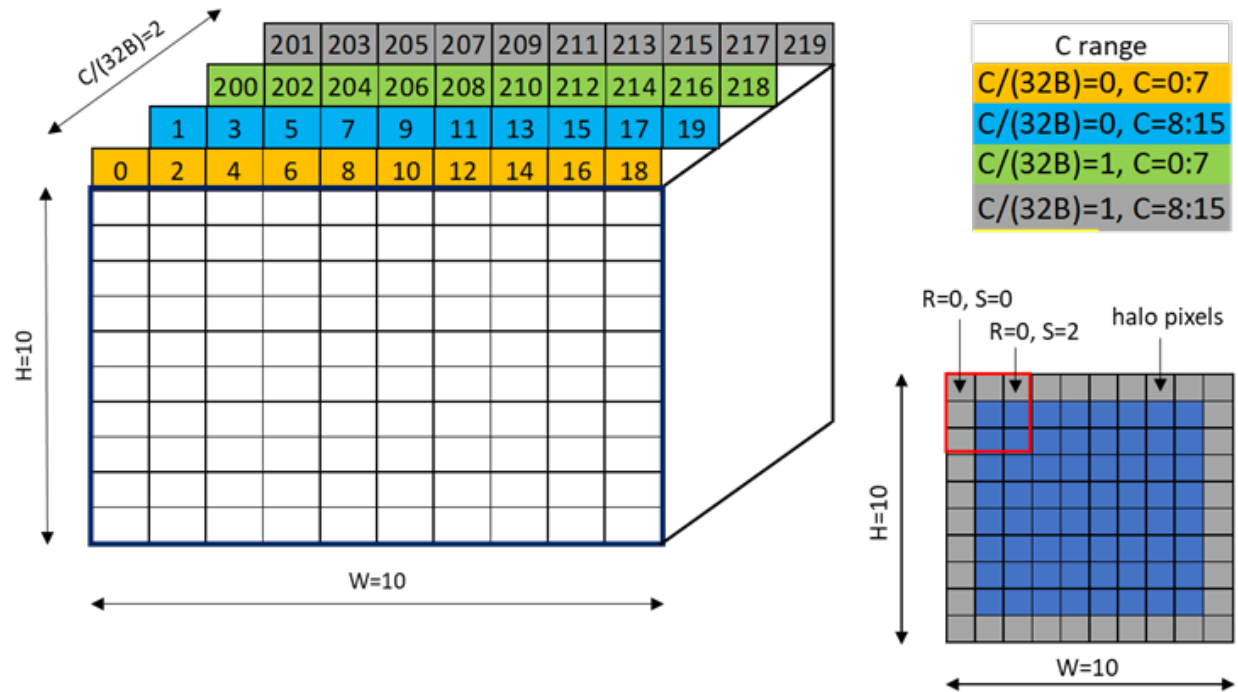


Figure 25: 32-byte swizzle mode example

► 96 byte swizzle mode:

The following table where each element (numbered cell) is 16 byte shows the swizzling pattern at the destination data layout:

0	1	2	3	4	5	6	7
1	0	3	2	5	4	7	6
... Pattern repeats ...							

An example of the data layout in global memory and its swizzled data layout in shared memory where each element (colored cell) is 16 bytes and the starting address is 256 bytes aligned is shown in [Figure 31](#).

► 128 byte swizzle mode:

The 128-byte swizzling mode supports the following sub-modes:

► 16-byte atomicity sub-mode:

In this sub-mode, the 16-byte of data is kept intact while swizzling.

The following table, where each elements (numbered cell) is 16 byte and the starting address is 1024 bytes aligned, shows the pattern of the destination data layout:

C/(32B)	H	C(32B)x4							
0	0	0	1	2	3	4	5	6	7
		8	9	10	11	12	13	14	15
	0,1	16	17	18	19	20	21	22	23
		24	25	26	27	28	29	30	31
	1	32	33	34	35	36	37	38	39
		40	41	42	43	44	45	46	47
	2	48	49	50	51	52	53	54	55
		56	57	58	59	60	61	62	63
	2,3	64	65	66	67	68	69	70	71
		72	73	74	75	76	77	78	79

C/(32B)	H	C(32B)x4							
1	0	200	201	202	203	204	205	206	207
	0,1	208	209	210	211	212	213	214	215
	0,1	216	217	218	219	220	221	222	223
	1	224	225	226	227	228	229	230	231
	8,9	232	233	234	235	236	237	238	239
	2	240	241	242	243	244	245	246	247
	2,3	248	249	250	251	252	253	254	255
	2,3	256	257	258	259	260	261	262	263
	3	264	265	266	267	268	269	270	271
	8,9	272	273	274	275	276	277	278	279

Figure 26: 32-byte swizzle mode fragments

C/(32B)	H	C(32B)x4							
0	0	0	1	2	3	4	5	6	7
		9	8	11	10	13	12	15	14
	0,1	16	17	18	19	20	21	22	23
		25	24	27	26	29	28	31	30
	1	32	33	34	35	36	37	38	39
		41	40	43	42	45	44	47	46
	2	48	49	50	51	52	53	54	55
		57	56	59	58	61	60	63	62
	2,3	64	65	66	67	68	69	70	71
		73	72	75	74	77	76	79	78
1	0	200	201	202	203	204	205	206	207
	0,1	209	208	211	210	213	212	215	214
	0,1	216	217	218	219	220	221	222	223
	1	225	224	227	226	229	228	231	230
	8,9	232	233	234	235	236	237	238	239
	2	241	240	243	242	245	244	247	246
	2,3	248	249	250	251	252	253	254	255
	2,3	257	256	259	258	261	260	263	262
	3	264	265	266	267	268	269	270	271
	8,9	273	272	275	274	277	276	279	278

Figure 27: 32-byte swizzle mode destination data layout

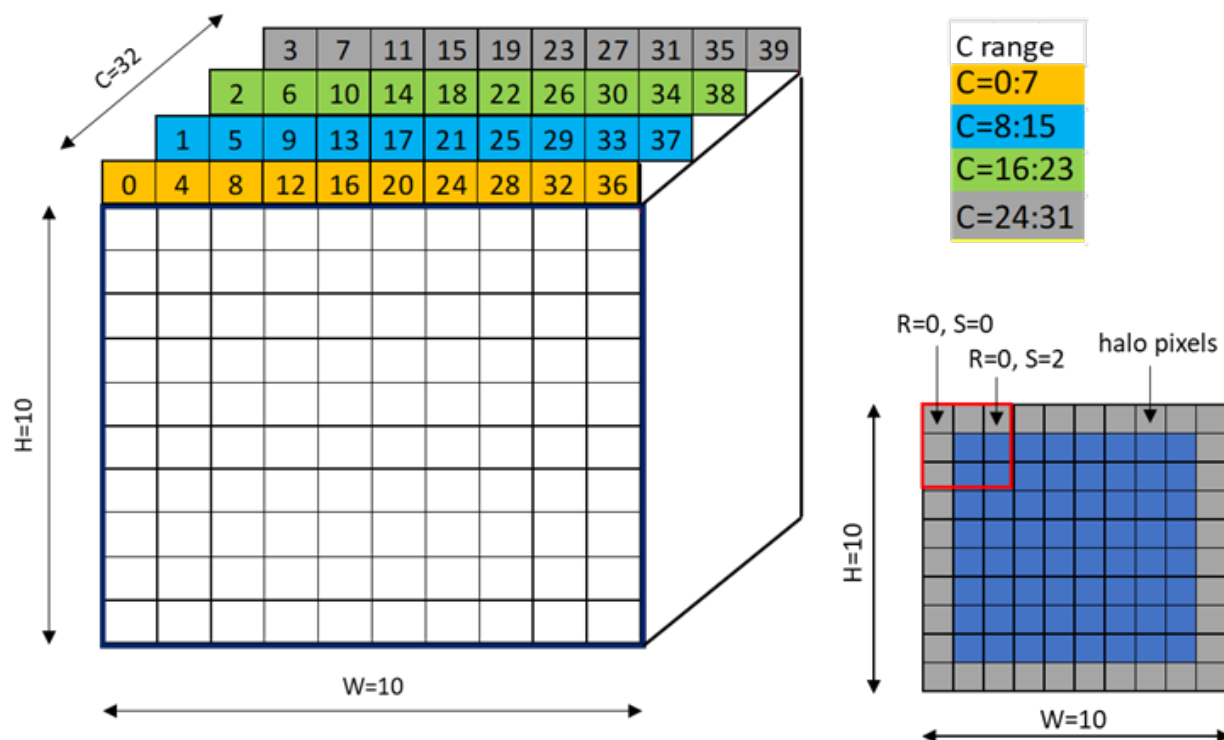


Figure 28: 64-byte swizzle mode example

0	1	2	3	4	5	6	7
1	0	3	2	5	4	7	6
2	3	0	1	6	7	4	5
3	2	1	0	7	6	5	4
4	5	6	7	0	1	2	3
5	4	7	6	1	0	3	2
6	7	4	5	2	3	0	1
7	6	5	4	3	2	1	0
... Pattern repeats ...							

An example of the 128 byte swizzle mode for NHWC tensor of 1x10x10x64 dimension, with 2 bytes / channel and 64 channels, is shown in [Figure 32](#).

Each colored cell represents 8 channels. [Figure 33](#) shows the source data layout.

[Figure 34](#) shows the destination data layout with 128 byte swizzling.

► 32-byte atomicity sub-mode:

In this sub-mode, the 32-byte of data is kept intact while swizzling.

The following table where each element (numbered cell) is 16 byte shows the swizzling pattern at the destination data layout:

W	H	C = 0-63							
0,1	0	0	1	2	3	4	5	6	7
2,3		8	9	10	11	12	13	14	15
4,5		16	17	18	19	20	21	22	23
6,7		24	25	26	27	28	29	30	31
8,9		32	33	34	35	36	37	38	39
0,1	1	40	41	42	43	44	45	46	47
2,3		48	49	50	51	52	53	54	55
4,5		56	57	58	59	60	61	62	63
6,7		64	65	66	67	68	69	70	71
8,9		72	73	74	75	76	77	78	79

Figure 29: 64-byte swizzle mode source data layout

W	H	C = 0-63							
0,1	0	0	1	2	3	4	5	6	7
2,3		9	8	11	10	13	12	15	14
4,5		18	19	16	17	22	23	20	21
6,7		27	26	25	24	31	30	29	28
8,9		32	33	34	35	36	37	38	39
0,1	1	41	40	43	42	45	44	47	46
2,3		50	51	48	49	54	55	52	53
4,5		59	58	57	56	63	62	61	60
6,7		64	65	66	67	68	69	70	71
8,9		73	72	75	74	77	76	79	78

Figure 30: 64-byte swizzle mode destination data layout

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

global memory layout

0	1	2	3	4	5	8	9
11	10	13	12	17	16	19	18
20	21	24	25	26	27	28	29
33	32	35	34	37	36	41	40
42	43	44	45	48	49	50	51
53	52	57	56	59	58	61	60

swizzled shared memory layout

Figure 31: 96-byte swizzle mode example

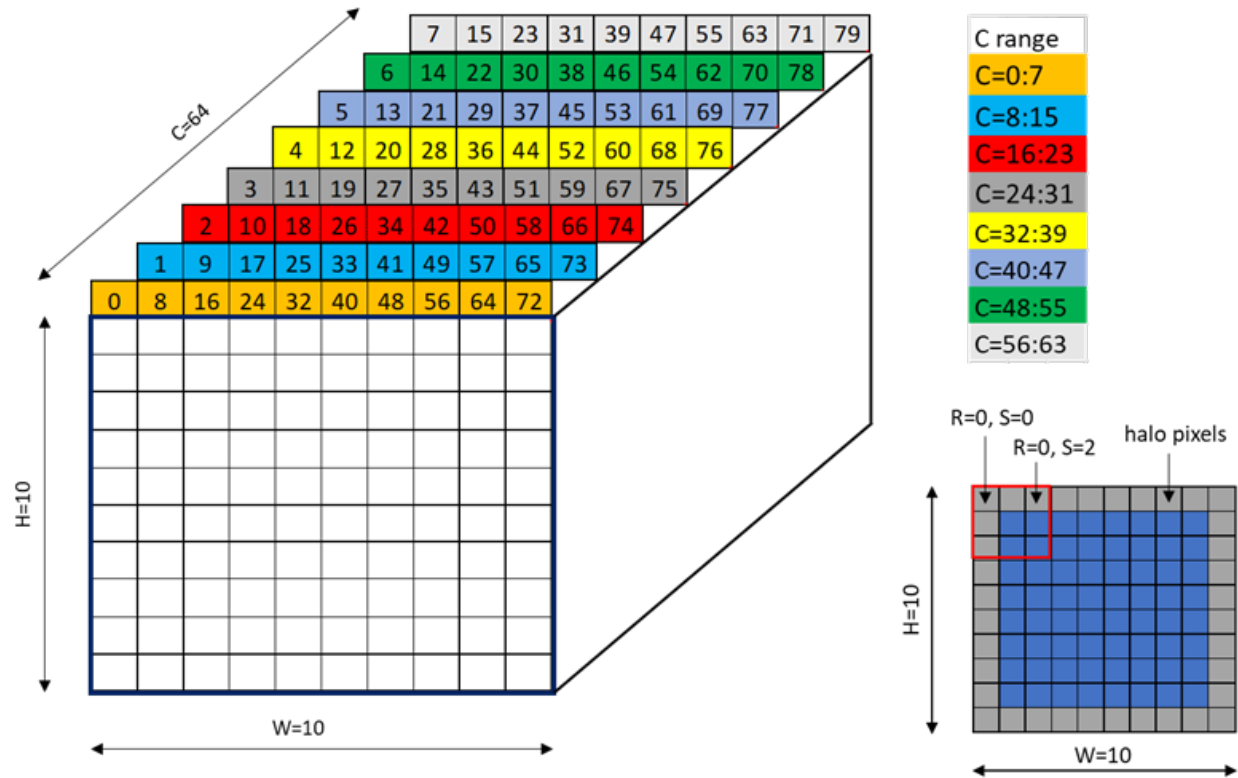


Figure 32: 128-byte swizzle mode example

0 1	2 3	4 5	6 7
2 3	0 1	6 7	4 5
4 5	6 7	0 1	2 3
6 7	4 5	2 3	0 1
... Pattern repeats ...			

This sub-mode requires 32 byte alignment at shared memory.

An example of the data layout in global memory and its swizzled data layout in shared memory where each element (colored cell) is 16 bytes is shown in [Figure 35](#)

► 32-byte atomicity with 8-byte flip sub-mode:

The swizzling pattern for this sub-mode is similar to the 32-byte atomicity sub-mode except that there is a flip of adjacent 8-bytes within the 16-byte data at every alternate shared memory line.

An example of the data layout in global memory and its swizzled data layout in shared memory where each element (colored cell) is 16 bytes (two 8-byte sub-elements for each 16-byte colored cell are shown to show the flip) is shown in [Figure 36](#)

► 64-byte atomicity sub-mode:

In this sub-mode, the 64-byte of data is kept intact while swizzling.

W	H	C = 0-63							
0	0	0	1	2	3	4	5	6	7
1		8	9	10	11	12	13	14	15
2		16	17	18	19	20	21	22	23
3		24	25	26	27	28	29	30	31
4		32	33	34	35	36	37	38	39
5		40	41	42	43	44	45	46	47
6		48	49	50	51	52	53	54	55
7		56	57	58	59	60	61	62	63
8		64	65	66	67	68	69	70	71
9		72	73	74	75	76	77	78	79
0	1	80	81	82	83	84	85	86	87
1		88	89	90	91	92	93	94	95
2		96	97	98	99	100	101	102	103
3		104	105	106	107	108	109	110	111
4		112	113	114	115	116	117	118	119
5		120	121	122	123	124	125	126	127
6		128	129	130	131	132	133	134	135
7		136	137	138	139	140	141	142	143
8		144	145	146	147	148	149	150	151
9		152	153	154	155	156	157	158	159

Figure 33: 128-byte swizzle mode source data layout

W	H	C = 0-63							
0	0	0	1	2	3	4	5	6	7
1		9	8	11	10	13	12	15	14
2		18	19	16	17	22	23	20	21
3		27	26	25	24	31	30	29	28
4		36	37	38	39	32	33	34	35
5		45	44	47	46	41	40	43	42
6		54	55	52	53	50	51	48	49
7		63	62	61	60	59	58	57	56
8		64	65	66	67	68	69	70	71
9		73	72	75	74	77	76	79	78
0	1	82	83	80	81	86	87	84	85
1		91	90	89	88	95	94	93	92
2		100	101	102	103	96	97	98	99
3		109	108	111	110	105	104	107	106
4		118	119	116	117	114	115	112	113
5		127	126	125	124	123	122	121	120
6		128	129	130	131	132	133	134	135
7		137	136	139	138	141	140	143	142
8		146	147	144	145	150	151	148	149
9		155	154	153	152	159	158	157	156

Figure 34: 128-byte swizzle mode destination data layout

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

global memory layout

0	1	2	3	4	5	6	7
10	11	8	9	14	15	12	13
20	21	22	23	16	17	18	19
30	31	28	29	26	27	24	25

swizzled shared memory layout

Figure 35: 128-byte swizzle mode example with 32-byte atomicity

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63

global memory layout

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
21	20	23	22	17	16	19	18	29	28	31	30	25	24	27	26
40	41	42	43	44	45	46	47	32	33	34	35	36	37	38	39
61	60	63	62	57	56	59	58	53	52	55	54	49	48	51	50

swizzled shared memory layout

Figure 36: 128-byte swizzle mode example with 32-byte atomicity with 8-byte flip

The following table where each element (numbered cell) is 16 byte shows the swizzling pattern at the destination data layout:

0 1 2 3	4 5 6 7
4 5 6 7	0 1 2 3
... Pattern repeats ...	

This sub-mode requires 64-byte alignment at shared memory.

An example of the data layout in global memory and its swizzled data layout in shared memory where each element (colored cell) is 16 bytes is shown in [Figure 37](#)

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15

global memory layout

0	1	2	3	4	5	6	7
12	13	14	15	8	9	10	11

swizzled shared memory layout

Figure 37: 128-byte swizzle mode example with 64-byte atomicity

[Table 14](#) lists the valid combination of swizzle-atomicity with the swizzling-mode.

Table 14: Valid combination of swizzle-atomicity with swizzling-mode

Swizzling Mode	Swizzle-Atomicity
No Swizzling	–
32B Swizzling Mode	16B
64B Swizzling Mode	16B
96B Swizzling Mode	16B
128B Swizzling Mode	▶ 16B ▶ 32B ▶ 32B + 8B-flip ▶ 64B

The value of swizzle base offset is 0 when the `dstMem` shared memory address is located at the following boundary:

Swizzling Mode	Starting address of the repeating pattern
128-Byte swizzle	1024-Byte boundary
96-Byte swizzle	256-Byte boundary
64-Byte swizzle	512-Byte boundary
32-Byte swizzle	256-Byte boundary

Otherwise, the swizzle base offset is a non-zero value, computed using following formula:

Swizzling Mode	Formula
128-Byte swizzle	$\text{base offset} = (\text{dstMem} / 128) \% 8$
96-Byte swizzle	$\text{base offset} = (\text{dstMem} / 128) \% 2$
64-Byte swizzle	$\text{base offset} = (\text{dstMem} / 128) \% 4$
32-Byte swizzle	$\text{base offset} = (\text{dstMem} / 128) \% 2$

5.5.8. Tensor-map

The tensor-map is a 128-byte opaque object either in `.const` space or `.param` (kernel function parameter) space or `.global` space which describes the tensor properties and the access properties of the tensor data described in previous sections.

Tensor-Map can be created using CUDA APIs. Refer to *CUDA programming guide* for more details.

Chapter 6. Instruction Operands

6.1. Operand Type Information

All operands in instructions have a known type from their declarations. Each operand type must be compatible with the type determined by the instruction template and instruction type. There is no automatic conversion between types.

The bit-size type is compatible with every type having the same size. Integer types of a common size are compatible with each other. Operands having type different from but compatible with the instruction type are silently cast to the instruction type.

6.2. Source Operands

The source operands are denoted in the instruction descriptions by the names *a*, *b*, and *c*. PTX describes a load-store machine, so operands for ALU instructions must all be in variables declared in the `.reg` register state space. For most operations, the sizes of the operands must be consistent.

The `cvt` (convert) instruction takes a variety of operand types and sizes, as its job is to convert from nearly any data type to any other data type (and size).

The `ld`, `st`, `mov`, and `cvt` instructions copy data from one location to another. Instructions `ld` and `st` move data from/to addressable state spaces to/from registers. The `mov` instruction copies data between registers.

Most instructions have an optional predicate guard that controls conditional execution, and a few instructions have additional predicate source operands. Predicate operands are denoted by the names *p*, *q*, *r*, *s*.

6.3. Destination Operands

PTX instructions that produce a single result store the result in the field denoted by *d* (for destination) in the instruction descriptions. The result operand is a scalar or vector variable in the register state space.

6.4. Using Addresses, Arrays, and Vectors

Using scalar variables as operands is straightforward. The interesting capabilities begin with addresses, arrays, and vectors.

6.4.1. Addresses as Operands

All the memory instructions take an address operand that specifies the memory location being accessed. This addressable operand is one of:

[var]

the name of an addressable variable `var`.

[reg]

an integer or bit-size type register `reg` containing a byte address.

[reg+immOff]

a sum of register `reg` containing a byte address plus a constant integer byte offset (signed, 32-bit).

[var+immOff]

a sum of address of addressable variable `var` containing a byte address plus a constant integer byte offset (signed, 32-bit).

[immAddr]

an immediate absolute byte address (unsigned, 32-bit).

var[immOff]

an array element as described in *Arrays as Operands*.

The register containing an address may be declared as a bit-size type or integer type.

The access size of a memory instruction is the total number of bytes accessed in memory. For example, the access size of `ld.v4.b32` is 16 bytes, while the access size of `atom.f16x2` is 4 bytes.

The address must be naturally aligned to a multiple of the access size. If an address is not properly aligned, the resulting behavior is undefined. For example, among other things, the access may proceed by silently masking off low-order address bits to achieve proper rounding, or the instruction may fault.

The address size may be either 32-bit or 64-bit. 128-bit addresses are not supported. Addresses are zero-extended to the specified width as needed, and truncated if the register width exceeds the state space address width for the target architecture.

Address arithmetic is performed using integer arithmetic and logical instructions. Examples include pointer arithmetic and pointer comparisons. All addresses and address computations are byte-based; there is no support for C-style pointer arithmetic.

The `mov` instruction can be used to move the address of a variable into a pointer. The address is an offset in the state space in which the variable is declared. Load and store operations move data between registers and locations in addressable state spaces. The syntax is similar to that used in many assembly languages, where scalar variables are simply named and addresses are de-referenced by enclosing the address expression in square brackets. Address expressions include variable names, address registers, address register plus byte offset, and immediate address expressions which evaluate at compile-time to a constant address.

Here are a few examples:

```

.shared .u16 x;
.reg    .u16 r0;
.global .v4 .f32 V;
.reg    .v4 .f32 W;
.const  .s32 tbl[256];
.reg    .b32 p;
.reg    .s32 q;

ld.shared.u16    r0, [x];
ld.global.v4.f32 W, [V];
ld.const.s32     q, [tbl+12];
mov.u32         p, tbl;

```

6.4.1.1 Generic Addressing

If a memory instruction does not specify a state space, the operation is performed using generic addressing. The state spaces `.const`, *Kernel Function Parameters* (`.param`), `.local` and `.shared` are modeled as windows within the generic address space. Each window is defined by a window base and a window size that is equal to the size of the corresponding state space. A generic address maps to global memory unless it falls within the window for `const`, `local`, or `shared` memory. The *Kernel Function Parameters* (`.param`) window is contained within the `.global` window. Within each window, a generic address maps to an address in the underlying state space by subtracting the window base from the generic address.

6.4.2. Arrays as Operands

Arrays of all types can be declared, and the identifier becomes an address constant in the space where the array is declared. The size of the array is a constant in the program.

Array elements can be accessed using an explicitly calculated byte address, or by indexing into the array using square-bracket notation. The expression within square brackets is either a constant integer, a register variable, or a simple *register with constant offset* expression, where the offset is a constant expression that is either added or subtracted from a register variable. If more complicated indexing is desired, it must be written as an address calculation prior to use. Examples are:

```

ld.global.u32    s, a[0];
ld.global.u32    s, a[N-1];
mov.u32         s, a[1]; // move address of a[1] into s

```

6.4.3. Vectors as Operands

Vector operands can be specified as source and destination operands for instructions. However, when specified as destination operand, all elements in vector expression must be unique, otherwise behavior is undefined. Vectors may also be passed as arguments to called functions.

Vector elements can be extracted from the vector with the suffixes `.x`, `.y`, `.z` and `.w`, as well as the typical color fields `.r`, `.g`, `.b` and `.a`.

A brace-enclosed list is used for pattern matching to pull apart vectors.

```
.reg .v4 .f32 V;  
.reg .f32      a, b, c, d;  
  
mov.v4.f32 {a,b,c,d}, V;
```

Vector loads and stores can be used to implement wide loads and stores, which may improve memory performance. The registers in the load/store operations can be a vector, or a brace-enclosed list of similarly typed scalars. Here are examples:

```
ld.global.v4.f32 {a,b,c,d}, [addr+16];  
ld.global.v2.u32 V2, [addr+8];
```

Elements in a brace-enclosed vector, say {Ra, Rb, Rc, Rd}, correspond to extracted elements as follows:

```
Ra = V.x = V.r  
Rb = V.y = V.g  
Rc = V.z = V.b  
Rd = V.w = V.a
```

6.4.4. Labels and Function Names as Operands

Labels and function names can be used only in `bra/brx.idx` and `call` instructions respectively. Function names can be used in `mov` instruction to get the address of the function into a register, for use in an indirect call.

Beginning in PTX ISA version 3.1, the `mov` instruction may be used to take the address of kernel functions, to be passed to a system call that initiates a kernel launch from the GPU. This feature is part of the support for CUDA Dynamic Parallelism. See the *CUDA Dynamic Parallelism Programming Guide* for details.

6.5. Type Conversion

All operands to all arithmetic, logic, and data movement instruction must be of the same type and size, except for operations where changing the size and/or type is part of the definition of the instruction. Operands of different sizes or types must be converted prior to the operation.

6.5.1. Scalar Conversions

[Table 15](#) and [Table 16](#) show what precision and format the `cvt` instruction uses given operands of differing types. For example, if a `cvt.s32.u16` instruction is given a `u16` source operand and `s32` as a destination operand, the `u16` is zero-extended to `s32`.

Conversions to floating-point that are beyond the range of floating-point numbers are represented with the maximum floating-point value (IEEE 754 Inf for `f32` and `f64`, and $\sim 131,000$ for `f16`).

Table 15: Convert Instruction Precision and Format Table 1

		Destination Format												
		s8	s16	s32	s64	u8	u16	u32	u64	f16	f32	f64	bf16	tf32
Source For- mat	s8	–	sext	sext	sext	–	sext	sext	sext	s2f	s2f	s2f	s2f	–
	s16	chop ¹	–	sext	sext	chop ¹	–	sext	sext	s2f	s2f	s2f	s2f	–
	s32	chop ¹	chop ¹	–	sext	chop ¹	chop ¹	–	sext	s2f	s2f	s2f	s2f	–
	s64	chop ¹	chop ¹	chop ¹	–	chop ¹	chop ¹	chop ¹	–	s2f	s2f	s2f	s2f	–
	u8	–	zext	zext	zext	–	zext	zext	zext	u2f	u2f	u2f	u2f	–
	u16	chop ¹	–	zext	zext	chop ¹	–	zext	zext	u2f	u2f	u2f	u2f	–
	u32	chop ¹	chop ¹	–	zext	chop ¹	chop ¹	–	zext	u2f	u2f	u2f	u2f	–
	u64	chop ¹	chop ¹	chop ¹	–	chop ¹	chop ¹	chop ¹	–	u2f	u2f	u2f	u2f	–
	f16	f2s	f2s	f2s	f2s	f2u	f2u	f2u	f2u	–	f2f	f2f	f2f	–
	f32	f2s	f2s	f2s	f2s	f2u	f2u	f2u	f2u	f2f	–	f2f	f2f	f2f
	f64	f2s	f2s	f2s	f2s	f2u	f2u	f2u	f2u	f2f	f2f	–	f2f	–
	bf16	f2s	f2s	f2s	f2s	f2u	f2u	f2u	f2u	f2f	f2f	f2f	f2f	–
	tf32	–	–	–	–	–	–	–	–	–	–	–	–	–

Table 16: Convert Instruction Precision and Format Table 2

		Destination Format									
		f16	f32	bf16	e4m3	e5m2	e2m3	e3m2	e2m1	ue8m0	s2f6
Source For- mat	f16	–	f2f	f2f	f2f	f2f	f2f	f2f	f2f	–	–
	f32	f2f	–	f2f	f2f	f2f	f2f	f2f	f2f	f2f	f2f
	bf16	f2f	f2f	–	f2f	f2f	f2f	f2f	f2f	f2f	f2f
	e4m3	f2f	–	–	–	–	–	–	–	–	–
	e5m2	f2f	–	–	–	–	–	–	–	–	–
	e2m3	f2f	–	–	–	–	–	–	–	–	–
	e3m2	f2f	–	–	–	–	–	–	–	–	–
	e2m1	f2f	–	–	–	–	–	–	–	–	–
	ue8m0	–	–	f2f	–	–	–	–	–	–	–
	s2f6	–	–	f2f	–	–	–	–	–	–	–

Notes

sext = sign-extend; zext = zero-extend; chop = keep only low bits that fit;

s2f = signed-to-float; f2s = float-to-signed; u2f = unsigned-to-float;

f2u = float-to-unsigned; f2f = float-to-float.

¹ If the destination register is wider than the destination format, the result is extended to the desti-

nation register width after chopping. The type of extension (sign or zero) is based on the destination format. For example, `cvt.s16.u32` targeting a 32-bit register first chops to 16-bit, then sign-extends to 32-bit.

6.5.2. Rounding Modifiers

Conversion instructions may specify a rounding modifier. In PTX, there are four integer rounding modifiers and six floating-point rounding modifiers. [Table 17](#) and [Table 18](#) summarize the rounding modifiers.

Table 17: Floating-Point Rounding Modifiers

Modifier	Description
<code>.rn</code>	rounds to nearest even
<code>.rna</code>	rounds to nearest, ties away from zero
<code>.rz</code>	rounds towards zero
<code>.rm</code>	rounds towards negative infinity
<code>.rp</code>	rounds towards positive infinity
<code>.rs</code>	rounds either towards zero or away from zero based on the carry out of the integer addition of random bits and the discarded bits of mantissa

Table 18: Integer Rounding Modifiers

Modifier	Description
<code>.rni</code>	round to nearest integer, choosing even integer if source is equidistant between two integers.
<code>.rzi</code>	round to nearest integer in the direction of zero
<code>.rmi</code>	round to nearest integer in direction of negative infinity
<code>.rpi</code>	round to nearest integer in direction of positive infinity

6.6. Operand Costs

Operands from different state spaces affect the speed of an operation. Registers are fastest, while global memory is slowest. Much of the delay to memory can be hidden in a number of ways. The first is to have multiple threads of execution so that the hardware can issue a memory operation and then switch to other execution. Another way to hide latency is to issue the load instructions as early as possible, as execution is not blocked until the desired result is used in a subsequent (in time) instruction. The register in a store operation is available much more quickly. [Table 19](#) gives estimates of the costs of using different kinds of memory.

Table 19: Cost Estimates for Accessing State-Spaces

Space	Time	Notes
Register	0	
Shared	0	
Constant	0	Amortized cost is low, first access is high
Local	> 100 clocks	
Parameter	0	
Immediate	0	
Global	> 100 clocks	
Texture	> 100 clocks	
Surface	> 100 clocks	

Chapter 7. Abstracting the ABI

Rather than expose details of a particular calling convention, stack layout, and Application Binary Interface (ABI), PTX provides a slightly higher-level abstraction and supports multiple ABI implementations. In this section, we describe the features of PTX needed to achieve this hiding of the ABI. These include syntax for function definitions, function calls, parameter passing, and memory allocated on the stack (`alloca`).

Refer to *PTX Writers Guide to Interoperability* for details on generating PTX compliant with Application Binary Interface (ABI) for the CUDA[®] architecture.

7.1. Function Declarations and Definitions

In PTX, functions are declared and defined using the `.func` directive. A function *declaration* specifies an optional list of return parameters, the function name, and an optional list of input parameters; together these specify the function's interface, or prototype. A function *definition* specifies both the interface and the body of the function. A function must be declared or defined prior to being called.

The simplest function has no parameters or return values, and is represented in PTX as follows:

```
.func foo
{
    ...
    ret;
}

...
call foo;
...
```

Here, execution of the `call` instruction transfers control to `foo`, implicitly saving the return address. Execution of the `ret` instruction within `foo` transfers control to the instruction following the call.

Scalar and vector base-type input and return parameters may be represented simply as register variables. At the call, arguments may be register variables or constants, and return values may be placed directly into register variables. The arguments and return variables at the call must have type and size that match the callee's corresponding formal parameters.

Example

```
.func (.reg .u32 %res) inc_ptr ( .reg .u32 %ptr, .reg .u32 %inc )
{
    add.u32 %res, %ptr, %inc;
    ret;
}

...
call (%r1), inc_ptr, (%r1,4);
...
```

When using the ABI, `.reg` state space parameters must be at least 32-bits in size. Subword scalar objects in the source language should be promoted to 32-bit registers in PTX, or use `.param` state space byte arrays described next.

Objects such as C structures and unions are flattened into registers or byte arrays in PTX and are represented using `.param` space memory. For example, consider the following C structure, passed by value to a function:

```
struct {
    double dbl;
    char   c[4];
};
```

In PTX, this structure will be flattened into a byte array. Since memory accesses are required to be aligned to a multiple of the access size, the structure in this example will be a 12 byte array with 8 byte alignment so that accesses to the `.f64` field are aligned. The `.param` state space is used to pass the structure by value:

Example

```
.func (.reg .s32 out) bar (.reg .s32 x, .param .align 8 .b8 y[12])
{
    .reg .f64 f1;
    .reg .b32 c1, c2, c3, c4;
    ...
    ld.param.f64 f1, [y+0];
    ld.param.b8  c1, [y+8];
    ld.param.b8  c2, [y+9];
    ld.param.b8  c3, [y+10];
    ld.param.b8  c4, [y+11];
    ...
    ... // computation using x, f1, c1, c2, c3, c4;
}

{
    .param .b8 .align 8 py[12];
    ...
    st.param.b64 [py+ 0], %rd;
    st.param.b8  [py+ 8], %rc1;
    st.param.b8  [py+ 9], %rc2;
    st.param.b8  [py+10], %rc1;
    st.param.b8  [py+11], %rc2;
    // scalar args in .reg space, byte array in .param space
```

(continues on next page)

(continued from previous page)

```
call (%out), bar, (%x, py);
...
```

In this example, note that `.param` space variables are used in two ways. First, a `.param` variable `y` is used in function definition `bar` to represent a formal parameter. Second, a `.param` variable `py` is declared in the body of the calling function and used to set up the structure being passed to `bar`.

The following is a conceptual way to think about the `.param` state space use in device functions.

For a caller,

- ▶ The `.param` state space is used to set values that will be passed to a called function and/or to receive return values from a called function. Typically, a `.param` byte array is used to collect together fields of a structure being passed by value.

For a callee,

- ▶ The `.param` state space is used to receive parameter values and/or pass return values back to the caller.

The following restrictions apply to parameter passing.

For a caller,

- ▶ Arguments may be `.param` variables, `.reg` variables, or constants.
- ▶ In the case of `.param` space formal parameters that are byte arrays, the argument must also be a `.param` space byte array with matching type, size, and alignment. A `.param` argument must be declared within the local scope of the caller.
- ▶ In the case of `.param` space formal parameters that are base-type scalar or vector variables, the corresponding argument may be either a `.param` or `.reg` space variable with matching type and size, or a constant that can be represented in the type of the formal parameter.
- ▶ In the case of `.reg` space formal parameters, the corresponding argument may be either a `.param` or `.reg` space variable of matching type and size, or a constant that can be represented in the type of the formal parameter.
- ▶ In the case of `.reg` space formal parameters, the register must be at least 32-bits in size.
- ▶ All `st.param` instructions used for passing arguments to function call must immediately precede the corresponding `call` instruction and `ld.param` instruction used for collecting return value must immediately follow the `call` instruction without any control flow alteration. `st.param` and `ld.param` instructions used for argument passing cannot be predicated. This enables compiler optimization and ensures that the `.param` variable does not consume extra space in the caller's frame beyond that needed by the ABI. The `.param` variable simply allows a mapping to be made at the call site between data that may be in multiple locations (e.g., structure being manipulated by caller is located in registers and memory) to something that can be passed as a parameter or return value to the callee.

For a callee,

- ▶ Input and return parameters may be `.param` variables or `.reg` variables.
- ▶ Parameters in `.param` memory must be aligned to a multiple of 1, 2, 4, 8, or 16 bytes.
- ▶ Parameters in the `.reg` state space must be at least 32-bits in size.
- ▶ The `.reg` state space can be used to receive and return base-type scalar and vector values, including sub-word size objects when compiling in non-ABI mode. Supporting the `.reg` state space provides legacy support.

Note that the choice of `.reg` or `.param` state space for parameter passing has no impact on whether the parameter is ultimately passed in physical registers or on the stack. The mapping of parameters to physical registers and stack locations depends on the ABI definition and the order, size, and alignment of parameters.

7.1.1. Changes from PTX ISA Version 1.x

In PTX ISA version 1.x, formal parameters were restricted to `.reg` state space, and there was no support for array parameters. Objects such as C structures were flattened and passed or returned using multiple registers. PTX ISA version 1.x supports multiple return values for this purpose.

Beginning with PTX ISA version 2.0, formal parameters may be in either `.reg` or `.param` state space, and `.param` space parameters support arrays. For targets `sm_20` or higher, PTX restricts functions to a single return value, and a `.param` byte array should be used to return objects that do not fit into a register. PTX continues to support multiple return registers for `sm_1x` targets.

Note: PTX implements a stack-based ABI only for targets `sm_20` or higher.

PTX ISA versions prior to 3.0 permitted variables in `.reg` and `.local` state spaces to be defined at module scope. When compiling to use the ABI, PTX ISA version 3.0 and later disallows module-scoped `.reg` and `.local` variables and restricts their use to within function scope. When compiling without use of the ABI, module-scoped `.reg` and `.local` variables are supported as before. When compiling legacy PTX code (ISA versions prior to 3.0) containing module-scoped `.reg` or `.local` variables, the compiler silently disables use of the ABI.

7.2. Variadic Functions

Note: Support for variadic functions which was unimplemented has been removed from the spec.

PTX version 6.0 supports passing unsized array parameter to a function which can be used to implement variadic functions.

Refer to *Kernel and Function Directives: `.func`* for details

7.3. Alloca

PTX provides `alloca` instruction for allocating storage at runtime on the per-thread local memory stack. The allocated stack memory can be accessed with `ld.local` and `st.local` instructions using the pointer returned by `alloca`.

In order to facilitate deallocation of memory allocated with `alloca`, PTX provides two additional instructions: `stacksave` which allows reading the value of stack pointer in a local variable, and `stackrestore` which can restore the stack pointer with the saved value.

`alloca`, `stacksave`, and `stackrestore` instructions are described in *Stack Manipulation Instructions*.

Chapter 8. Memory Consistency Model

In multi-threaded executions, the side-effects of memory operations performed by each thread become visible to other threads in a partial and non-identical order. This means that any two operations may appear to happen in no order, or in different orders, to different threads. The axioms introduced by the memory consistency model specify exactly which contradictions are forbidden between the orders observed by different threads.

In the absence of any constraint, each read operation returns the value committed by some write operation to the same memory location, including the initial write to that memory location. The memory consistency model effectively constrains the set of such candidate writes from which a read operation can return a value.

8.1. Scope and applicability of the model

The constraints specified under this model apply to PTX programs with any PTX ISA version number, running on `sm_70` or later architectures.

The memory consistency model does not apply to texture (including `ld.global.nc`) and surface accesses.

8.1.1. Limitations on atomicity at system scope

When communicating with the host CPU, certain strong operations with system scope may not be performed atomically on some systems. For more details on atomicity guarantees to host memory, see the *CUDA Atomicity Requirements*.

8.2. Memory operations

The fundamental storage unit in the PTX memory model is a byte, consisting of 8 bits. Each state space available to a PTX program is a sequence of contiguous bytes in memory. Every byte in a PTX state space has a unique address relative to all threads that have access to the same state space.

Each PTX memory instruction specifies an address operand and a data type. The address operand contains a virtual address that gets converted to a physical address during memory access. The physical address and the size of the data type together define a physical memory location, which is the range of bytes starting from the physical address and extending up to the size of the data type in bytes.

The memory consistency model specification uses the terms “address” or “memory address” to indicate a virtual address, and the term “memory location” to indicate a physical memory location.

Each PTX memory instruction also specifies the operation — either a read, a write or an atomic read-modify-write — to be performed on all the bytes in the corresponding memory location.

8.2.1. Overlap

Two memory locations are said to overlap when the starting address of one location is within the range of bytes constituting the other location. Two memory operations are said to overlap when they specify the same virtual address and the corresponding memory locations overlap. The overlap is said to be complete when both memory locations are identical, and it is said to be partial otherwise.

8.2.2. Aliases

Two distinct virtual addresses are said to be aliases if they map to the same memory location.

8.2.3. Multimem Addresses

A multimem address is a virtual address which points to multiple distinct memory locations across devices.

Only *multimem.** operations are valid on multimem addresses. That is, the behavior of accessing a multimem address in any other memory operation is undefined.

8.2.4. Memory Operations on Vector Data Types

The memory consistency model relates operations executed on memory locations with scalar data types, which have a maximum size and alignment of 64 bits. Memory operations with a vector data type are modelled as a set of equivalent memory operations with a scalar data type, executed in an unspecified order on the elements in the vector.

8.2.5. Memory Operations on Packed Data Types

A packed data type consists of two values of the same scalar data type, as described in *Packed Data Types*. These values are accessed in adjacent memory locations. A memory operation on a packed data type is modelled as a pair of equivalent memory operations on the scalar data type, executed in an unspecified order on each element of the packed data.

8.2.6. Initialization

Each byte in memory is initialized by a hypothetical write *WO* executed before starting any thread in the program. If the byte is included in a program variable, and that variable has an initial value, then *WO* writes the corresponding initial value for that byte; else *WO* is assumed to have written an unknown but constant value to the byte.

8.3. State spaces

The relations defined in the memory consistency model are independent of state spaces. In particular, causality order closes over all memory operations across all the state spaces. But the side-effect of a memory operation in one state space can be observed directly only by operations that also have access to the same state space. This further constrains the synchronizing effect of a memory operation in addition to scope. For example, the synchronizing effect of the PTX instruction `ld.relaxed.shared.sys` is identical to that of `ld.relaxed.shared.cluster`, since no thread outside the same cluster can execute an operation that accesses the same memory location.

8.4. Operation types

For simplicity, the rest of the document refers to the following operation types, instead of mentioning specific instructions that give rise to them.

Table 20: Operation Types

Operation Type	Instruction/Operation
atomic operation	atom or red instruction.
read operation	All variants of ld instruction and atom instruction (but not red instruction).
write operation	All variants of st instruction, and <i>atomic</i> operations if they result in a write.
memory operation	A <i>read</i> or <i>write</i> operation.
volatile operation	An instruction with .volatile qualifier.
acquire operation	A <i>memory</i> operation with .acquire or .acq_rel qualifier.
release operation	A <i>memory</i> operation with .release or .acq_rel qualifier.
mmio operation	An ld or st instruction with .mmio qualifier.
memory fence operation	A membar, fence.sc or fence.acq_rel instruction.
proxy fence operation	A fence.proxy or a membar.proxy instruction.
strong operation	A <i>memory fence</i> operation, or a <i>memory</i> operation with a .relaxed, .acquire, .release, .acq_rel, .volatile, or .mmio qualifier.
weak operation	An ld or st instruction with a .weak qualifier.
synchronizing operation	A barrier instruction, <i>fence</i> operation, <i>release</i> operation or <i>acquire</i> operation.

8.4.1. mmio Operation

An *mmio* operation is a memory operation with .mmio qualifier specified. It is usually performed on a memory location which is mapped to the control registers of peer I/O devices. It can also be used for communication between threads but has poor performance relative to non-*mmio* operations.

The semantic meaning of *mmio* operations cannot be defined precisely as it is defined by the underlying I/O device. For formal specification of semantics of *mmio* operation from Memory Consistency Model perspective, it is equivalent to the semantics of a *strong* operation. But it follows a few implementation-specific properties, if it meets the *CUDA atomicity requirements* at the specified scope:

- ▶ Writes are always performed and are never combined within the scope specified.
- ▶ Reads are always performed, and are not forwarded, prefetched, combined, or allowed to hit any cache within the scope specified.
 - ▶ As an exception, in some implementations, the surrounding locations may also be loaded. In such cases the amount of data loaded is implementation specific and varies between 32 and 128 bytes in size.

8.4.2. volatile Operation

A *volatile* operation is a memory operation with `.volatile` qualifier specified. The semantics of volatile operations are equivalent to a relaxed memory operation with system-scope but with the following extra implementation-specific constraints:

- ▶ The number of volatile *instructions* (not operations) executed by a program is preserved. Hardware may combine and merge volatile *operations* issued by multiple different volatile *instructions*, that is, the number of volatile *operations* in the program is not preserved.
- ▶ Volatile *instructions* are not re-ordered around other volatile *instructions*, but the memory *operations* performed by those *instructions* may be re-ordered around each other.

Note: PTX volatile operations are intended for compilers to lower volatile read and write operations from CUDA C++, and other programming languages sharing CUDA C++ volatile semantics, to PTX.

Since volatile operations are relaxed at system-scope with extra constraints, prefer using other *strong* read or write operations (e.g. `ld.relaxed.sys` or `st.relaxed.sys`) for **Inter-Thread Synchronization** instead, which may deliver better performance.

PTX volatile operations are not suited for **Memory Mapped IO (MMIO)** because volatile operations do not preserve the number of memory operations performed, and may perform more or less operations than requested in a non-deterministic way. Use *.mmio operations* instead, which strictly preserve the number of operations performed.

8.5. Scope

Each *strong* operation must specify a *scope*, which is the set of threads that may interact directly with that operation and establish any of the relations described in the memory consistency model. There are four scopes:

Table 21: Scopes

Scope	Description
<code>.cta</code>	The set of all threads executing in the same CTA as the current thread.
<code>.cluster</code>	The set of all threads executing in the same cluster as the current thread.
<code>.gpu</code>	The set of all threads in the current program executing on the same compute device as the current thread. This also includes other kernel grids invoked by the host program on the same compute device.
<code>.sys</code>	The set of all threads in the current program, including all kernel grids invoked by the host program on all compute devices, and all threads constituting the host program itself.

Note that the warp is not a *scope*; the CTA is the smallest collection of threads that qualifies as a *scope* in the memory consistency model.

8.6. Proxies

A *memory proxy*, or a *proxy* is an abstract label applied to a method of memory access. When two memory operations use distinct methods of memory access, they are said to be different *proxies*.

Memory operations as defined in *Operation types* use *generic* method of memory access, i.e. a *generic proxy*. Other operations such as textures and surfaces all use distinct methods of memory access, also distinct from the *generic* method.

A *proxy fence* is required to synchronize memory operations across different *proxies*. Although virtual aliases use the *generic* method of memory access, since using distinct virtual addresses behaves as if using different *proxies*, they require a *proxy fence* to establish memory ordering.

8.7. Morally strong operations

Two operations are said to be *morally strong* relative to each other if they satisfy all of the following conditions:

1. The operations are related in *program order* (i.e, they are both executed by the same thread), or each operation is *strong* and specifies a *scope* that includes the thread executing the other operation.
2. Both operations are performed via the same *proxy*.
3. If both are memory operations, then they overlap completely.

Most (but not all) of the axioms in the memory consistency model depend on relations between *morally strong* operations.

8.7.1. Conflict and Data-races

Two *overlapping* memory operations are said to *conflict* when at least one of them is a *write*.

Two *conflicting* memory operations are said to be in a *data-race* if they are not related in *causality order* and they are not *morally strong*.

8.7.2. Limitations on Mixed-size Data-races

A *data-race* between operations that *overlap* completely is called a *uniform-size data-race*, while a *data-race* between operations that *overlap* partially is called a *mixed-size data-race*.

The axioms in the memory consistency model do not apply if a PTX program contains one or more *mixed-size data-races*. But these axioms are sufficient to describe the behavior of a PTX program with only *uniform-size data-races*.

Atomicity of mixed-size RMW operations

In any program with or without *mixed-size data-races*, the following property holds for every pair of *overlapping atomic* operations A1 and A2 such that each specifies a *scope* that includes the other: Either the *read-modify-write* operation specified by A1 is performed completely before A2 is initiated, or vice versa. This property holds irrespective of whether the two operations A1 and A2 overlap partially or completely.

8.8. Release and Acquire Patterns

Some sequences of instructions give rise to patterns that participate in memory synchronization as described later. The *release* pattern makes prior operations from the current thread¹ visible to some operations from other threads. The *acquire* pattern makes some operations from other threads visible to later operations from the current thread.

A *release* pattern on a location M consists of one of the following:

1. A *release* operation on M
E.g.: `st.release [M];` or `atom.release [M];` or `mbARRIER.arrive.release [M];`
2. Or a *release* or *acquire-release* operation on M followed by a *strong* write on M in *program order*
E.g.: `st.release [M]; st.relaxed [M];`
3. Or a *release* or *acquire-release memory fence* followed by a *strong* write on M in *program order*
E.g.: `fence.release;` `st.relaxed [M];` or `fence.release;` `atom.relaxed [M];`

Any *memory synchronization* established by a *release* pattern only affects operations occurring in *program order* before the first instruction in that pattern.

An *acquire* pattern on a location M consists of one of the following:

1. An *acquire* operation on M
E.g.: `ld.acquire [M];` or `atom.acquire [M];` or `mbARRIER.test_wait.acquire [M];`
2. Or a *strong* read on M followed by an *acquire* operation on M in *program order*
E.g.: `ld.relaxed [M]; ld.acquire [M];`
3. Or a *strong* read on M followed by an *acquire memory fence* in *program order*
E.g.: `ld.relaxed [M]; fence.acquire;` or `atom.relaxed [M]; fence.acquire;`

Any *memory synchronization* established by an *acquire* pattern only affects operations occurring in *program order* after the last instruction in that pattern.

Note that while atomic reductions conceptually perform a strong read as part of its read-modify-write sequence, this strong read does not form an acquire pattern.

E.g.: `red.add [M], 1; fence.acquire;` is not an acquire pattern.

¹ For both *release* and *acquire* patterns, this effect is further extended to operations in other threads through the transitive nature of *causality order*.

8.9. Ordering of memory operations

The sequence of operations performed by each thread is captured as *program order* while *memory synchronization* across threads is captured as *causality order*. The visibility of the side-effects of memory operations to other memory operations is captured as *communication order*. The memory consistency model defines contradictions that are disallowed between communication order on the one hand, and *causality order* and *program order* on the other.

8.9.1. Program Order

The *program order* relates all operations performed by a thread to the order in which a sequential processor will execute instructions in the corresponding PTX source. It is a transitive relation that forms a total order over the operations performed by the thread, but does not relate operations from different threads.

8.9.1.1 Asynchronous Operations

Some PTX instructions (all variants of `cp.async`, `cp.async.bulk`, `cp.reduce.async.bulk`, `wgmma.mma_async`) perform operations that are asynchronous to the thread that executed the instruction. These asynchronous operations are ordered after prior instructions in the same thread (except in the case of `wgmma.mma_async`), but they are not part of the program order for that thread. Instead, they provide weaker ordering guarantees as documented in the instruction description.

For example, the loads and stores performed as part of a `cp.async` are ordered with respect to each other, but not to those of any other `cp.async` instructions initiated by the same thread, nor any other instruction subsequently issued by the thread with the exception of `cp.async.commit_group` or `cp.async.mbarrier.arrive`. The asynchronous `mbarrier arrive-on` operation performed by a `cp.async.mbarrier.arrive` instruction is ordered with respect to the memory operations performed by all prior `cp.async` operations initiated by the same thread, but not to those of any other instruction issued by the thread. The implicit `mbarrier complete-tx` operation that is part of all variants of `cp.async.bulk` and `cp.reduce.async.bulk` instructions is ordered only with respect to the memory operations performed by the same asynchronous instruction, and in particular it does not transitively establish ordering with respect to prior instructions from the issuing thread.

8.9.2. Observation Order

Observation order relates a write *W* to a read *R* through an optional sequence of atomic read-modify-write operations.

A write *W* precedes a read *R* in *observation order* if:

1. *R* and *W* are *morally strong* and *R* reads the value written by *W*, or
2. For some atomic operation *Z*, *W* precedes *Z* and *Z* precedes *R* in *observation order*.

8.9.3. Fence-SC Order

The *Fence-SC* order is an acyclic partial order, determined at runtime, that relates every pair of *morally strong fence.sc* operations.

8.9.4. Memory synchronization

Synchronizing operations performed by different threads synchronize with each other at runtime as described here. The effect of such synchronization is to establish *causality order* across threads.

1. A `fence.sc` operation *X* *synchronizes* with a `fence.sc` operation *Y* if *X* precedes *Y* in the *Fence-SC* order.
2. A `bar{.cta}.sync` or `bar{.cta}.red` or `bar{.cta}.arrive` operation *synchronizes* with a `bar{.cta}.sync` or `bar{.cta}.red` operation executed on the same barrier.
3. A `barrier.cluster.arrive` operation *synchronizes* with a `barrier.cluster.wait` operation.
4. A *release* pattern *X* *synchronizes* with an *acquire* pattern *Y*, if a *write* operation in *X* precedes a *read* operation in *Y* in *observation order*, and the first operation in *X* and the last operation in *Y* are *morally strong*.

API synchronization

A *synchronizes* relation can also be established by certain CUDA APIs.

1. Completion of a task enqueued in a CUDA stream *synchronizes* with the start of the following task in the same stream, if any.
2. For purposes of the above, recording or waiting on a CUDA event in a stream, or causing a cross-stream barrier to be inserted due to `cudaStreamLegacy`, enqueues tasks in the associated streams even if there are no direct side effects. An event record task *synchronizes* with matching event wait tasks, and a barrier arrival task *synchronizes* with matching barrier wait tasks.
3. Start of a CUDA kernel *synchronizes* with start of all threads in the kernel. End of all threads in a kernel *synchronizes* with end of the kernel.
4. Start of a CUDA graph *synchronizes* with start of all source nodes in the graph. Completion of all sink nodes in a CUDA graph *synchronizes* with completion of the graph. Completion of a graph node *synchronizes* with start of all nodes with a direct dependency.
5. Start of a CUDA API call to enqueue a task *synchronizes* with start of the task.
6. Completion of the last task queued to a stream, if any, *synchronizes* with return from `cudaStreamSynchronize`. Completion of the most recently queued matching event record task, if any, *synchronizes* with return from `cudaEventSynchronize`. Synchronizing a CUDA device or context behaves as if synchronizing all streams in the context, including ones that have been destroyed.
7. Returning `cudaSuccess` from an API to query a CUDA handle, such as a stream or event, behaves the same as return from the matching synchronization API.

In addition to establishing a *synchronizes* relation, the CUDA API synchronization mechanisms above also participate in *proxy-preserved base causality order*.

8.9.5. Causality Order

Causality order captures how memory operations become visible across threads through synchronizing operations. The axiom “Causality” uses this order to constrain the set of write operations from which a read operation may read a value.

Relations in the *causality order* primarily consist of relations in *Base causality order*¹, which is a transitive order, determined at runtime.

Base causality order

An operation X precedes an operation Y in *base causality order* if:

1. X precedes Y in *program order*, or
2. X *synchronizes* with Y, or
3. For some operation Z,
 - a. X precedes Z in *program order* and Z precedes Y in *base causality order*, or
 - b. X precedes Z in *base causality order* and Z precedes Y in *program order*, or
 - c. X precedes Z in *base causality order* and Z precedes Y in *base causality order*.

Proxy-preserved base causality order

A memory operation X precedes a memory operation Y in *proxy-preserved base causality order* if X precedes Y in *base causality order*, and:

1. X and Y are performed to the same address, using the *generic proxy*, or
2. X and Y are performed to the same address, using the same *proxy*, and by the same thread block, or
3. X and Y are aliases and there is an *alias proxy fence* along the base causality path from X to Y.

Causality order

Causality order combines *base causality order* with some non-transitive relations as follows:

An operation X precedes an operation Y in *causality order* if:

1. X precedes Y in *proxy-preserved base causality order*, or
2. For some operation Z, X precedes Z in *observation order*, and Z precedes Y in *proxy-preserved base causality order*.

¹ The transitivity of *base causality order* accounts for the “cumulativity” of synchronizing operations.

8.9.6. Coherence Order

There exists a partial transitive order that relates *overlapping* write operations, determined at runtime, called the *coherence order*¹. Two *overlapping* write operations are related in *coherence order* if they are *morally strong* or if they are related in *causality order*. Two *overlapping* writes are unrelated in *coherence order* if they are in a *data-race*, which gives rise to the partial nature of *coherence order*.

¹ *Coherence order* cannot be observed directly since it consists entirely of write operations. It may be observed indirectly by its use in constraining the set of candidate writes that a read operation may read from.

8.9.7. Communication Order

The *communication order* is a non-transitive order, determined at runtime, that relates write operations to other *overlapping* memory operations.

1. A write W precedes an *overlapping* read R in *communication order* if R returns the value of any byte that was written by W.
2. A write W precedes a write W' in *communication order* if W precedes W' in *coherence order*.
3. A read R precedes an *overlapping* write W in *communication order* if, for any byte accessed by both R and W, R returns the value written by a write W' that precedes W in *coherence order*.

Communication order captures the visibility of memory operations — when a memory operation X1 precedes a memory operation X2 in *communication order*, X1 is said to be visible to X2.

8.10. Axioms

8.10.1. Coherence

If a write W precedes an *overlapping* write W' in *causality order*, then W must precede W' in *coherence order*.

8.10.2. Fence-SC

Fence-SC order cannot contradict *causality order*. For a pair of *morally strong fence.sc* operations F1 and F2, if F1 precedes F2 in *causality order*, then F1 must precede F2 in *Fence-SC* order.

8.10.3. Atomicity

Single-Copy Atomicity

Conflicting *morally strong* operations are performed with *single-copy atomicity*. When a read R and a write W are *morally strong*, then the following two communications cannot both exist in the same execution, for the set of bytes accessed by both R and W:

1. R reads any byte from W.
2. R reads any byte from any write W' which precedes W in *coherence order*.

Atomicity of read-modify-write (RMW) operations

When an *atomic* operation A and a write W *overlap* and are *morally strong*, then the following two communications cannot both exist in the same execution, for the set of bytes accessed by both A and W:

1. A reads any byte from a write W' that precedes W in *coherence order*.
2. A follows W in *coherence order*.

Litmus Test 1

<code>.global .u32 x = 0;</code>	
T1	T2
A1 : <code>atom.sys.inc.u32 %r0, [x];</code>	A2 : <code>atom.sys.inc.u32 %r0, [x];</code>
FINAL STATE: <code>x == 2</code>	

Atomicity is guaranteed when the operations are *morally strong*.

Litmus Test 2

<code>.global .u32 x = 0;</code>	
T1	T2 (In a different CTA)
A1 : <code>atom.cta.inc.u32 %r0, [x];</code>	A2 : <code>atom.gpu.inc.u32 %r0, [x];</code>
FINAL STATE: <code>x == 1 OR x == 2</code>	

Atomicity is not guaranteed if the operations are not *morally strong*.

8.10.4. No Thin Air

Values may not appear “out of thin air”: an execution cannot speculatively produce a value in such a way that the speculation becomes self-satisfying through chains of instruction dependencies and inter-thread communication. This matches both programmer intuition and hardware reality, but is necessary to state explicitly when performing formal analysis.

Litmus Test: Load Buffering

<pre>.global .u32 x = 0; .global .u32 y = 0;</pre>	
T1	T2
<pre>A1: ld.global.u32 %r0, [x]; B1: st.global.u32 [y], %r0;</pre>	<pre>A2: ld.global.u32 %r1, [y]; B2: st.global.u32 [x], %r1;</pre>
FINAL STATE: $x == 0$ AND $y == 0$	

The litmus test known as “LB” (Load Buffering) checks such forbidden values that may arise out of thin air. Two threads T1 and T2 each read from a first variable and copy the observed result into a second variable, with the first and second variable exchanged between the threads. If each variable is initially zero, the final result shall also be zero. If A1 reads from B2 and A2 reads from B1, then values passing through the memory operations in this example form a cycle: A1→B1→A2→B2→A1. Only the values $x == 0$ and $y == 0$ are allowed to satisfy this cycle. If any of the memory operations in this example were to speculatively associate a different value with the corresponding memory location, then such a speculation would become self-fulfilling, and hence forbidden.

8.10.5. Sequential Consistency Per Location

Within any set of *overlapping* memory operations that are pairwise *morally strong*, *communication order* cannot contradict *program order*, i.e., a concatenation of *program order* between *overlapping* operations and *morally strong* relations in *communication order* cannot result in a cycle. This ensures that each program slice of *overlapping* pairwise *morally strong* operations is strictly *sequentially-consistent*.

Litmus Test: CoRR

<code>.global .u32 x = 0;</code>	
T1	T2
<code>W1: st.global.relaxed.sys.u32 [x], 1;</code>	<code>R1: ld.global.relaxed.sys.u32 %r0, [x];</code> <code>R2: ld.global.relaxed.sys.u32 %r1, [x];</code>
IF %r0 == 1 THEN %r1 == 1	

The litmus test “CoRR” (Coherent Read-Read), demonstrates one consequence of this guarantee. A thread T1 executes a write W1 on a location x, and a thread T2 executes two (or an infinite sequence of) reads R1 and R2 on the same location x. No other writes are executed on x, except the one modelling the initial value. The operations W1, R1 and R2 are pairwise *morally strong*. If R1 reads from W1, then the subsequent read R2 must also observe the same value. If R2 observed the initial value of x instead, then this would form a sequence of *morally-strong* relations R2->W1->R1 in *communication order* that contradicts the *program order* R1->R2 in thread T2. Hence R2 cannot read the initial value of x in such an execution.

8.10.6. Causality

Relations in *communication order* cannot contradict *causality order*. This constrains the set of candidate write operations that a read operation may read from:

1. If a read R precedes an *overlapping* write W in *causality order*, then R cannot read from W.
2. If a write W precedes an *overlapping* read R in *causality order*, then for any byte accessed by both R and W, R cannot read from any write W' that precedes W in *coherence order*.

Litmus Test: Message Passing

<code>.global .u32 data = 0;</code> <code>.global .u32 flag = 0;</code>	
T1	T2
<code>W1: st.global.u32 [data], 1;</code> <code>F1: fence.sys;</code> <code>W2: st.global.relaxed.sys.u32 [flag], 1;</code>	<code>R1: ld.global.relaxed.sys.u32 %r0,</code> <code>↪ [flag];</code> <code>F2: fence.sys;</code> <code>R2: ld.global.u32 %r1, [data];</code>
IF %r0 == 1 THEN %r1 == 1	

The litmus test known as “MP” (Message Passing) represents the essence of typical synchronization algorithms. A vast majority of useful programs can be reduced to sequenced applications of this pattern.

Thread T1 first writes to a data variable and then to a flag variable while a second thread T2 first reads from the flag variable and then from the data variable. The operations on the flag are *morally strong* and the memory operations in each thread are separated by a *fence*, and these *fences* are *morally strong*.

If R1 observes W2, then the release pattern “F1; W2” *synchronizes* with the *acquire pattern* “R1; F2”. This establishes the *causality order* W1 -> F1 -> W2 -> R1 -> F2 -> R2. Then axiom *causality* guarantees that R2 cannot read from any write that precedes W1 in *coherence order*. In the absence of any other writes in this example, R2 must read from W1.

Litmus Test: CoWR

<pre>// These addresses are aliases .global .u32 data_alias_1; .global .u32 data_alias_2;</pre>	
T1	
<pre>W1: st.global.u32 [data_alias_1], 1; F1: fence.proxy.alias; R1: ld.global.u32 %r1, [data_alias_2];</pre>	
	<pre>%r1 == 1</pre>

Virtual aliases require an alias *proxy fence* along the synchronization path.

Litmus Test: Store Buffering

The litmus test known as “SB” (Store Buffering) demonstrates the *sequential consistency* enforced by the `fence.sc`. A thread T1 writes to a first variable, and then reads the value of a second variable, while a second thread T2 writes to the second variable and then reads the value of the first variable. The memory operations in each thread are separated by `fence.sc` instructions, and these *fences* are *morally strong*.

<pre>.global .u32 x = 0; .global .u32 y = 0;</pre>	
T1	T2
<pre>W1: st.global.u32 [x], 1; F1: fence.sc.sys; R1: ld.global.u32 %r0, [y];</pre>	<pre>W2: st.global.u32 [y], 1; F2: fence.sc.sys; R2: ld.global.u32 %r1, [x];</pre>
<pre>%r0 == 1 OR %r1 == 1</pre>	

In any execution, either F1 precedes F2 in *Fence-SC* order, or vice versa. If F1 precedes F2 in *Fence-SC* order, then F1 *synchronizes* with F2. This establishes the *causality order* in W1 -> F1 -> F2 -> R2. Axiom *causality* ensures that R2 cannot read from any write that precedes W1 in *coherence order*. In the absence of any other write to that variable, R2 must read from W1. Similarly, in the case where F2 precedes F1 in *Fence-SC* order, R1 must read from W2. If each `fence.sc` in this example were replaced by a `fence.acq_rel` instruction, then this outcome is not guaranteed. There may be an execution where the write from each thread remains unobserved from the other thread, i.e., an execution is possible, where both R1 and R2 return the initial value “0” for variables y and x respectively.

8.11. Special Cases

8.11.1. Reductions do not form Acquire Patterns

Atomic reduction operations like `red` do not form acquire patterns with acquire fences.

Litmus Test: Message Passing with a Red Instruction

<pre>.global .u32 x = 0; .global .u32 flag = 0;</pre>	
T1	T2
<pre>W1: st.u32 [x], 42; W2: st.release.gpu.u32 [flag], 1;</pre>	<pre>RMW1: red.sys.global.add.u32 [flag], 1; F2: fence.acquire.gpu; R2: ld.weak.u32 %r1, [x];</pre>
<pre>%r1 == 0 AND flag == 2</pre>	

The litmus test known as “MP” (Message Passing) demonstrates the consequence of reductions being excluded from acquire patterns. It is possible to observe the outcome where R2 reads the value 0 from x and flag has the final value of 2. This outcome is possible since the release pattern in T1 does not synchronize with any acquire pattern in T2. Using the atom instruction instead of red forbids this outcome.

Chapter 9. Instruction Set

9.1. Format and Semantics of Instruction Descriptions

This section describes each PTX instruction. In addition to the name and the format of the instruction, the semantics are described, followed by some examples that attempt to show several possible instantiations of the instruction.

9.2. PTX Instructions

PTX instructions generally have from zero to four operands, plus an optional guard predicate appearing after an @ symbol to the left of the opcode:

- ▶ @p opcode ;
- ▶ @p opcode a ;
- ▶ @p opcode d, a ;
- ▶ @p opcode d, a, b ;
- ▶ @p opcode d, a, b, c ;

For instructions that create a result value, the d operand is the destination operand, while a, b, and c are source operands.

The `setp` instruction writes two destination registers. We use a | symbol to separate multiple destination registers.

```
setp.lt.s32 p|q, a, b; // p = (a < b); q = !(a < b);
```

For some instructions the destination operand is optional. A *bit bucket* operand denoted with an underscore (_) may be used in place of a destination register.

9.3. Predicated Execution

In PTX, predicate registers are virtual and have `.pred` as the type specifier. So, predicate registers can be declared as

```
.reg .pred p, q, r;
```

All instructions have an optional *guard predicate* which controls conditional execution of the instruction. The syntax to specify conditional execution is to prefix an instruction with `@{!}p`, where `p` is a predicate variable, optionally negated. Instructions without a guard predicate are executed unconditionally.

Predicates are most commonly set as the result of a comparison performed by the `setp` instruction.

As an example, consider the high-level code

```
if (i < n)
    j = j + 1;
```

This can be written in PTX as

```
@p    setp.lt.s32  p, i, n;    // p = (i < n)
      add.s32     j, j, 1;    // if i < n, add 1 to j
```

To get a conditional branch or conditional function call, use a predicate to control the execution of the branch or call instructions. To implement the above example as a true conditional branch, the following PTX instruction sequence might be used:

```
@!p    setp.lt.s32  p, i, n;    // compare i to n
      bra  L1;          // if False, branch over
      add.s32     j, j, 1;
L1:    ...
```

9.3.1. Comparisons

9.3.1.1 Integer and Bit-Size Comparisons

The signed integer comparisons are the traditional `eq` (equal), `ne` (not-equal), `lt` (less-than), `le` (less-than-or-equal), `gt` (greater-than), and `ge` (greater-than-or-equal). The unsigned comparisons are `eq`, `ne`, `lo` (lower), `ls` (lower-or-same), `hi` (higher), and `hs` (higher-or-same). The bit-size comparisons are `eq` and `ne`; ordering comparisons are not defined for bit-size types.

Table 22 shows the operators for signed integer, unsigned integer, and bit-size types.

Table 22: Operators for Signed Integer, Unsigned Integer, and Bit-Size Types

Meaning	Signed Operator	Unsigned Operator	Bit-Size Operator
$a == b$	eq	eq	eq
$a != b$	ne	ne	ne
$a < b$	lt	lo	n/a
$a <= b$	le	ls	n/a
$a > b$	gt	hi	n/a
$a >= b$	ge	hs	n/a

9.3.1.2 Floating Point Comparisons

The ordered floating-point comparisons are eq, ne, lt, le, gt, and ge. If either operand is NaN, the result is False. Table 23 lists the floating-point comparison operators.

Table 23: Floating-Point Comparison Operators

Meaning	Floating-Point Operator
$a == b \ \&\& \ !\text{isNaN}(a) \ \&\& \ !\text{isNaN}(b)$	eq
$a != b \ \&\& \ !\text{isNaN}(a) \ \&\& \ !\text{isNaN}(b)$	ne
$a < b \ \&\& \ !\text{isNaN}(a) \ \&\& \ !\text{isNaN}(b)$	lt
$a <= b \ \&\& \ !\text{isNaN}(a) \ \&\& \ !\text{isNaN}(b)$	le
$a > b \ \&\& \ !\text{isNaN}(a) \ \&\& \ !\text{isNaN}(b)$	gt
$a >= b \ \&\& \ !\text{isNaN}(a) \ \&\& \ !\text{isNaN}(b)$	ge

To aid comparison operations in the presence of NaN values, unordered floating-point comparisons are provided: equ, neu, ltu, leu, gtu, and geu. If both operands are numeric values (not NaN), then the comparison has the same result as its ordered counterpart. If either operand is NaN, then the result of the comparison is True.

Table 24 lists the floating-point comparison operators accepting NaN values.

Table 24: Floating-Point Comparison Operators Accepting NaN

Meaning	Floating-Point Operator
$a == b \ \ \text{isNaN}(a) \ \ \text{isNaN}(b)$	equ
$a != b \ \ \text{isNaN}(a) \ \ \text{isNaN}(b)$	neu
$a < b \ \ \text{isNaN}(a) \ \ \text{isNaN}(b)$	ltu
$a <= b \ \ \text{isNaN}(a) \ \ \text{isNaN}(b)$	leu
$a > b \ \ \text{isNaN}(a) \ \ \text{isNaN}(b)$	gtu
$a >= b \ \ \text{isNaN}(a) \ \ \text{isNaN}(b)$	geu

To test for NaN values, two operators `num` (numeric) and `nan` (`isNaN`) are provided. `num` returns `True` if both operands are numeric values (not NaN), and `nan` returns `True` if either operand is NaN. Table 25 lists the floating-point comparison operators testing for NaN values.

Table 25: Floating-Point Comparison Operators Testing for NaN

Meaning	Floating-Point Operator
<code>!isNaN(a) && !isNaN(b)</code>	<code>num</code>
<code>isNaN(a) isNaN(b)</code>	<code>nan</code>

9.3.2. Manipulating Predicates

Predicate values may be computed and manipulated using the following instructions: `and`, `or`, `xor`, `not`, and `mov`.

There is no direct conversion between predicates and integer values, and no direct way to load or store predicate register values. However, `setp` can be used to generate a predicate from an integer, and the predicate-based select (`selp`) instruction can be used to generate an integer value based on the value of a predicate; for example:

```
selp.u32 %r1,1,0,%p;    // convert predicate to 32-bit value
```

9.4. Type Information for Instructions and Operands

Typed instructions must have a type-size modifier. For example, the `add` instruction requires type and size information to properly perform the addition operation (signed, unsigned, float, different sizes), and this information must be specified as a suffix to the opcode.

Example

```
.reg .u16 d, a, b;

add.u16 d, a, b;    // perform a 16-bit unsigned add
```

Some instructions require multiple type-size modifiers, most notably the data conversion instruction `cvt`. It requires separate type-size modifiers for the result and source, and these are placed in the same order as the operands. For example:

```
.reg .u16 a;
.reg .f32 d;

cvt.f32.u16 d, a;    // convert 16-bit unsigned to 32-bit float
```

In general, an operand's type must agree with the corresponding instruction-type modifier. The rules for operand and instruction type conformance are as follows:

- ▶ Bit-size types agree with any type of the same size.
- ▶ Signed and unsigned integer types agree provided they have the same size, and integer operands are silently cast to the instruction type if needed. For example, an unsigned integer operand used in a signed integer instruction will be treated as a signed integer by the instruction.
- ▶ Floating-point types agree only if they have the same size; i.e., they must match exactly.

Table 26 summarizes these type checking rules.

Table 26: Type Checking Rules

		Operand Type			
		.bX	.sX	.uX	.fX
Instruction Type	.bX	okay	okay	okay	okay
	.sX	okay	okay	okay	invalid
	.uX	okay	okay	okay	invalid
	.fX	okay	invalid	invalid	okay

Note: Some operands have their type and size defined independently from the instruction type-size. For example, the shift amount operand for left and right shift instructions always has type `.u32`, while the remaining operands have their type and size determined by the instruction type.

Example

```
// 64-bit arithmetic right shift; shift amount 'b' is .u32
shr.s64 d,a,b;
```

9.4.1. Operand Size Exceeding Instruction-Type Size

For convenience, `ld`, `st`, and `cvt` instructions permit source and destination data operands to be wider than the instruction-type size, so that narrow values may be loaded, stored, and converted using regular-width registers. For example, 8-bit or 16-bit values may be held directly in 32-bit or 64-bit registers when being loaded, stored, or converted to other types and sizes. The operand type checking rules are relaxed for bit-size and integer (signed and unsigned) instruction types; floating-point instruction types still require that the operand type-size matches exactly, unless the operand is of bit-size type.

When a source operand has a size that exceeds the instruction-type size, the source data is truncated (chopped) to the appropriate number of bits specified by the instruction type-size.

Table 27 summarizes the relaxed type-checking rules for source operands. Note that some combinations may still be invalid for a particular instruction; for example, the `cvt` instruction does not support `.bX` instruction types, so those rows are invalid for `cvt`.

Table 27: Relaxed Type-checking Rules for Source Operands

		Source Operand Type															
		b8	b16	b32	b64	b128	s8	s16	s32	s64	u8	u16	u32	u64	f16	f32	f64
In- struc- tion Type	b8	–	chop	chop	chop	chop	–	chop	chop	chop	–	chop	chop	chop	chop	chop	chop
	b16	inv	–	chop	chop	chop	inv	–	chop	chop	inv	–	chop	chop	–	chop	chop
	b32	inv	inv	–	chop	chop	inv	inv	–	chop	inv	inv	–	chop	inv	–	chop
	b64	inv	inv	inv	–	chop	inv	inv	inv	–	inv	inv	inv	–	inv	inv	–
	b128	inv	inv	inv	inv	–	inv	inv	inv	inv	inv	inv	inv	inv	inv	inv	inv
	s8	–	chop	chop	chop	chop	–	chop	chop	chop	–	chop	chop	chop	inv	inv	inv
	s16	inv	–	chop	chop	chop	inv	–	chop	chop	inv	–	chop	chop	inv	inv	inv
	s32	inv	inv	–	chop	chop	inv	inv	–	chop	inv	inv	–	chop	inv	inv	inv
	s64	inv	inv	inv	–	chop	inv	inv	inv	–	inv	inv	inv	–	inv	inv	inv
	u8	–	chop	chop	chop	chop	–	chop	chop	chop	–	chop	chop	chop	inv	inv	inv
	u16	inv	–	chop	chop	chop	inv	–	chop	chop	inv	–	chop	chop	inv	inv	inv
	u32	inv	inv	–	chop	chop	inv	inv	–	chop	inv	inv	–	chop	inv	inv	inv
	u64	inv	inv	inv	–	chop	inv	inv	inv	–	inv	inv	inv	–	inv	inv	inv
	f16	inv	–	chop	chop	chop	inv	inv	inv	inv	inv	inv	inv	inv	–	inv	inv
	f32	inv	inv	–	chop	chop	inv	inv	inv	inv	inv	inv	inv	inv	inv	–	inv
	f64	inv	inv	inv	–	chop	inv	inv	inv	inv	inv	inv	inv	inv	inv	inv	–
Notes		chop = keep only low bits that fit; “–” = allowed, but no conversion needed; inv = invalid, parse error. 1. Source register size must be of equal or greater size than the instruction-type size. 2. Bit-size source registers may be used with any appropriately-sized instruction type. The data are truncated (“chopped”) to the instruction-type size and interpreted according to the instruction type. 3. Integer source registers may be used with any appropriately-sized bit-size or integer instruction type. The data are truncated to the instruction-type size and interpreted according to the instruction type. 4. Floating-point source registers can only be used with bit-size or floating-point instruction types. When used with a narrower bit-size instruction type, the data are truncated. When used with a floating-point instruction type, the size must match exactly.															

When a destination operand has a size that exceeds the instruction-type size, the destination data is zero- or sign-extended to the size of the destination register. If the corresponding instruction type is signed integer, the data is sign-extended; otherwise, the data is zero-extended.

Table 28 summarizes the relaxed type-checking rules for destination operands.

Table 28: Relaxed Type-checking Rules for Destination Operands

		Destination Operand Type															
		b8	b16	b32	b64	b128	s8	s16	s32	s64	u8	u16	u32	u64	f16	f32	f64
In- struc- tion Type	b8	–	zext	zext	zext	zext	–	zext	zext	zext	–	zext	zext	zext	zext	zext	zext
	b16	inv	–	zext	zext	zext	inv	–	zext	zext	inv	–	zext	zext	–	zext	zext
	b32	inv	inv	–	zext	zext	inv	inv	–	zext	inv	inv	–	zext	inv	–	zext
	b64	inv	inv	inv	–	zext	inv	inv	inv	–	inv	inv	inv	–	inv	inv	–
	b128	inv	inv	inv	inv	–	inv	inv	inv	inv	inv	inv	inv	inv	inv	inv	inv
	s8	–	sext	sext	sext	sext	–	sext	sext	sext	–	sext	sext	sext	inv	inv	inv
	s16	inv	–	sext	sext	sext	inv	–	sext	sext	inv	–	sext	sext	inv	inv	inv
	s32	inv	inv	–	sext	sext	inv	inv	–	sext	inv	inv	–	sext	inv	inv	inv
	s64	inv	inv	inv	–	sext	inv	inv	inv	–	inv	inv	inv	–	inv	inv	inv
	u8	–	zext	zext	zext	zext	–	zext	zext	zext	–	zext	zext	zext	inv	inv	inv
	u16	inv	–	zext	zext	zext	inv	–	zext	zext	inv	–	zext	zext	inv	inv	inv
	u32	inv	inv	–	zext	zext	inv	inv	–	zext	inv	inv	–	zext	inv	inv	inv
	u64	inv	inv	inv	–	zext	inv	inv	inv	–	inv	inv	inv	–	inv	inv	inv
	f16	inv	–	zext	zext	zext	inv	inv	inv	inv	inv	inv	inv	inv	–	inv	inv
	f32	inv	inv	–	zext	zext	inv	inv	inv	inv	inv	inv	inv	inv	inv	–	inv
	f64	inv	inv	inv	–	zext	inv	inv	inv	inv	inv	inv	inv	inv	inv	inv	–
Notes		sext = sign-extend; zext = zero-extend; “–” = allowed, but no conversion needed; inv = invalid, parse error. 1. Destination register size must be of equal or greater size than the instruction-type size. 2. Bit-size destination registers may be used with any appropriately-sized instruction type. The data are sign-extended to the destination register width for signed integer instruction types, and are zero-extended to the destination register width otherwise. 3. Integer destination registers may be used with any appropriately-sized bit-size or integer instruction type. The data are sign-extended to the destination register width for signed integer instruction types, and are zero-extended to the destination register width for bit-size and unsigned integer instruction types. 4. Floating-point destination registers can only be used with bit-size or floating-point instruction types. When used with a narrower bit-size instruction type, the data are zero-extended. When used with a floating-point instruction type, the size must match exactly.															

9.5. Divergence of Threads in Control Constructs

Threads in a CTA execute together, at least in appearance, until they come to a conditional control construct such as a conditional branch, conditional function call, or conditional return. If threads execute down different control flow paths, the threads are called *divergent*. If all of the threads act in unison and follow a single control flow path, the threads are called *uniform*. Both situations occur often in programs.

A CTA with divergent threads may have lower performance than a CTA with uniformly executing threads, so it is important to have divergent threads re-converge as soon as possible. All control constructs are assumed to be divergent points unless the control-flow instruction is marked as uniform, using the `.uni` suffix. For divergent control flow, the optimizing code generator automatically determines points of re-convergence. Therefore, a compiler or code author targeting PTX can ignore the issue of divergent threads, but has the opportunity to improve performance by marking branch points as uniform when the compiler or author can guarantee that the branch point is non-divergent.

9.6. Semantics

The goal of the semantic description of an instruction is to describe the results in all cases in as simple language as possible. The semantics are described using C, until C is not expressive enough.

9.6.1. Machine-Specific Semantics of 16-bit Code

A PTX program may execute on a GPU with either a 16-bit or a 32-bit data path. When executing on a 32-bit data path, 16-bit registers in PTX are mapped to 32-bit physical registers, and 16-bit computations are *promoted* to 32-bit computations. This can lead to computational differences between code run on a 16-bit machine versus the same code run on a 32-bit machine, since the promoted computation may have bits in the high-order half-word of registers that are not present in 16-bit physical registers. These extra precision bits can become visible at the application level, for example, by a right-shift instruction.

At the PTX language level, one solution would be to define semantics for 16-bit code that is consistent with execution on a 16-bit data path. This approach introduces a performance penalty for 16-bit code executing on a 32-bit data path, since the translated code would require many additional masking instructions to suppress extra precision bits in the high-order half-word of 32-bit registers.

Rather than introduce a performance penalty for 16-bit code running on 32-bit GPUs, the semantics of 16-bit instructions in PTX is machine-specific. A compiler or programmer may choose to enforce portable, machine-independent 16-bit semantics by adding explicit conversions to 16-bit values at appropriate points in the program to guarantee portability of the code. However, for many performance-critical applications, this is not desirable, and for many applications the difference in execution is preferable to limiting performance.

9.7. Instructions

All PTX instructions may be predicated. In the following descriptions, the optional guard predicate is omitted from the syntax.

9.7.1. Integer Arithmetic Instructions

Integer arithmetic instructions operate on the integer types in register and constant immediate forms. The integer arithmetic instructions are:

- ▶ add
- ▶ sub
- ▶ mul
- ▶ mad
- ▶ mul24
- ▶ mad24
- ▶ sad
- ▶ div
- ▶ rem
- ▶ abs
- ▶ neg
- ▶ min
- ▶ max
- ▶ popc
- ▶ clz
- ▶ bfind
- ▶ fns
- ▶ brev
- ▶ bfe
- ▶ bfi
- ▶ bmsk
- ▶ szext
- ▶ dp4a
- ▶ dp2a

9.7.1.1 Integer Arithmetic Instructions: add

add

Add two values.

Syntax

```
add.type      d, a, b;
add{.sat}.s32 d, a, b;    // .sat applies only to .s32

.type = { .u16, .u32, .u64,
          .s16, .s32, .s64,
          .u16x2, .s16x2 };
```

Description

Performs addition and writes the resulting value into a destination register.

For `.u16x2`, `.s16x2` instruction types, forms input vectors by half word values from source operands. Half-word operands are then added in parallel to produce `.u16x2`, `.s16x2` result in destination.

Operands `d`, `a` and `b` have type `.type`. For instruction types `.u16x2`, `.s16x2`, operands `d`, `a` and `b` have type `.b32`.

Semantics

```
if (type == u16x2 || type == s16x2) {
    iA[0] = a[0:15];
    iA[1] = a[16:31];
    iB[0] = b[0:15];
    iB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = iA[i] + iB[i];
    }
} else {
    d = a + b;
}
```

Notes

Saturation modifier:

.sat limits result to `MININT` . `MAXINT` (no overflow) for the size of the operation. Applies only to `.s32` type.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`add.u16x2` and `add.s16x2` introduced in PTX ISA version 8.0.

Target ISA Notes

Supported on all target architectures.

`add.u16x2` and `add.s16x2` require `sm_90` or higher.

Examples

```
@p  add.u32      x,y,z;
    add.sat.s32  c,c,1;
    add.u16x2    u,v,w;
```

9.7.1.2 Integer Arithmetic Instructions: `sub`

`sub`

Subtract one value from another.

Syntax

```
sub.type      d, a, b;
sub{.sat}.s32 d, a, b;    // .sat applies only to .s32

.type = { .u16, .u32, .u64,
          .s16, .s32, .s64 };
```

Description

Performs subtraction and writes the resulting value into a destination register.

Semantics

```
d = a - b;
```

Notes

Saturation modifier:

.sat

limits result to MININT..MAXINT (no overflow) for the size of the operation. Applies only to .s32 type.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
sub.s32 c, a, b;
```

9.7.1.3 Integer Arithmetic Instructions: mul

mul

Multiply two values.

Syntax

```
mul.mode.type d, a, b;  
  
.mode = { .hi, .lo, .wide };  
.type = { .u16, .u32, .u64,  
          .s16, .s32, .s64 };
```

Description

Compute the product of two values.

Semantics

```
t = a * b;
n = bitwidth of type;
d = t;           // for .wide
d = t<2n-1..n>; // for .hi variant
d = t<n-1..0>;  // for .lo variant
```

Notes

The type of the operation represents the types of the `a` and `b` operands. If `.hi` or `.lo` is specified, then `d` is the same size as `a` and `b`, and either the upper or lower half of the result is written to the destination register. If `.wide` is specified, then `d` is twice as wide as `a` and `b` to receive the full result of the multiplication.

The `.wide` suffix is supported only for 16- and 32-bit integer types.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
mul.wide.s16 fa,fxs,fys; // 16*16 bits yields 32 bits
mul.lo.s16 fa,fxs,fys;  // 16*16 bits, save only the low 16 bits
mul.wide.s32 z,x,y;      // 32*32 bits, creates 64 bit result
```

9.7.1.4 Integer Arithmetic Instructions: `mad`

`mad`

Multiply two values, optionally extract the high or low half of the intermediate result, and add a third value.

Syntax

```
mad.mode.type d, a, b, c;
mad.hi.sat.s32 d, a, b, c;

.mode = { .hi, .lo, .wide };
.type = { .u16, .u32, .u64,
          .s16, .s32, .s64 };
```

Description

Multiplies two values, optionally extracts the high or low half of the intermediate result, and adds a third value. Writes the result into a destination register.

Semantics

```
t = a * b;  
n = bitwidth of type;  
d = t + c;           // for .wide  
d = t<2n-1..n> + c;  // for .hi variant  
d = t<n-1..0> + c;   // for .lo variant
```

Notes

The type of the operation represents the types of the *a* and *b* operands. If *.hi* or *.lo* is specified, then *d* and *c* are the same size as *a* and *b*, and either the upper or lower half of the result is written to the destination register. If *.wide* is specified, then *d* and *c* are twice as wide as *a* and *b* to receive the result of the multiplication.

The *.wide* suffix is supported only for 16-bit and 32-bit integer types.

Saturation modifier:

.sat

limits result to MININT..MAXINT (no overflow) for the size of the operation.

Applies only to *.s32* type in *.hi* mode.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
@p  mad.lo.s32 d,a,b,c;  
     mad.lo.s32 r,p,q,r;
```

9.7.1.5 Integer Arithmetic Instructions: mul24

mul24

Multiply two 24-bit integer values.

Syntax

```
mul24.mode.type  d, a, b;  
  
.mode = { .hi, .lo };  
.type  = { .u32, .s32 };
```

Description

Compute the product of two 24-bit integer values held in 32-bit source registers, and return either the high or low 32-bits of the 48-bit result.

Semantics

```
t = a * b;  
d = t<47..16>;    // for .hi variant  
d = t<31..0>;     // for .lo variant
```

Notes

Integer multiplication yields a result that is twice the size of the input operands, i.e., 48-bits.

mul24.hi performs a 24x24-bit multiply and returns the high 32 bits of the 48-bit result.

mul24.lo performs a 24x24-bit multiply and returns the low 32 bits of the 48-bit result.

All operands are of the same type and size.

mul24.hi may be less efficient on machines without hardware support for 24-bit multiply.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
mul24.lo.s32 d,a,b;    // low 32-bits of 24x24-bit signed multiply.
```

9.7.1.6 Integer Arithmetic Instructions: mad24

mad24

Multiply two 24-bit integer values and add a third value.

Syntax

```
mad24.mode.type  d, a, b, c;  
mad24.hi.sat.s32 d, a, b, c;  
  
.mode = { .hi, .lo };  
.type = { .u32, .s32 };
```

Description

Compute the product of two 24-bit integer values held in 32-bit source registers, and add a third, 32-bit value to either the high or low 32-bits of the 48-bit result. Return either the high or low 32-bits of the 48-bit result.

Semantics

```
t = a * b;  
d = t<47..16> + c;    // for .hi variant  
d = t<31..0> + c;     // for .lo variant
```

Notes

Integer multiplication yields a result that is twice the size of the input operands, i.e., 48-bits.

`mad24.hi` performs a 24x24-bit multiply and adds the high 32 bits of the 48-bit result to a third value.

`mad24.lo` performs a 24x24-bit multiply and adds the low 32 bits of the 48-bit result to a third value.

All operands are of the same type and size.

Saturation modifier:

.sat

limits result of 32-bit signed addition to `MININT..MAXINT` (no overflow). Applies only to `.s32` type in `.hi` mode.

`mad24.hi` may be less efficient on machines without hardware support for 24-bit multiply.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
mad24.lo.s32 d,a,b,c;    // low 32-bits of 24x24-bit signed multiply.
```

9.7.1.7 Integer Arithmetic Instructions: `sad`

`sad`

Sum of absolute differences.

Syntax

```
sad.type d, a, b, c;
.type = { .u16, .u32, .u64,
          .s16, .s32, .s64 };
```

Description

Adds the absolute value of `a-b` to `c` and writes the resulting value into `d`.

Semantics

```
d = c + ((a<b) ? b-a : a-b);
```

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
sad.s32 d,a,b,c;  
sad.u32 d,a,b,d; // running sum
```

9.7.1.8 Integer Arithmetic Instructions: `div`

`div`

Divide one value by another.

Syntax

```
div.type d, a, b;  
.type = { .u16, .u32, .u64,  
          .s16, .s32, .s64 };
```

Description

Divides `a` by `b`, stores result in `d`.

Semantics

```
d = a / b;
```

Notes

Division by zero yields an unspecified, machine-specific value.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
div.s32 b,n,i;
```

9.7.1.9 Integer Arithmetic Instructions: rem

rem

The remainder of integer division.

Syntax

```
rem.type d, a, b;
.type = { .u16, .u32, .u64,
          .s16, .s32, .s64 };
```

Description

Divides a by b, store the remainder in d.

Semantics

```
d = a % b;
```

Notes

The behavior for negative numbers is machine-dependent and depends on whether divide rounds towards zero or negative infinity.

Division by zero yields an unspecified, machine-specific value.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
rem.s32 x,x,8;    // x = x%8;
```

9.7.1.10 Integer Arithmetic Instructions: abs

abs

Absolute value.

Syntax

```
abs.type d, a;  
.type = { .s16, .s32, .s64 };
```

Description

Take the absolute value of a and store it in d.

Semantics

```
d = |a|;
```

Notes

Only for signed integers.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
abs.s32  r0, a;
```

9.7.1.11 Integer Arithmetic Instructions: neg

neg

Arithmetic negate.

Syntax

```
neg.type  d, a;  
.type = { .s16, .s32, .s64 };
```

Description

Negate the sign of **a** and store the result in **d**.

Semantics

```
d = -a;
```

Notes

Only for signed integers.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
neg.s32  r0, a;
```

9.7.1.12 Integer Arithmetic Instructions: min

min

Find the minimum of two values.

Syntax

```
min.atype      d, a, b;  
min{.relu}.btype d, a, b;  
  
.atype = { .u16, .u32, .u64,  
           .u16x2, .s16, .s64 };  
.btype = { .s16x2, .s32 };
```

Description

Store the minimum of a and b in d.

For `.u16x2`, `.s16x2` instruction types, forms input vectors by half word values from source operands. Half-word operands are then processed in parallel to produce `.u16x2`, `.s16x2` result in destination.

Operands d, a and b have the same type as the instruction type. For instruction types `.u16x2`, `.s16x2`, operands d, a and b have type `.b32`.

Semantics

```
if (type == u16x2 || type == s16x2) {  
    iA[0] = a[0:15];  
    iA[1] = a[16:31];  
    iB[0] = b[0:15];  
    iB[1] = b[16:31];  
    for (i = 0; i < 2; i++) {  
        d[i] = (iA[i] < iB[i]) ? iA[i] : iB[i];  
    }  
} else {  
    d = (a < b) ? a : b; // Integer (signed and unsigned)  
}
```

Notes

Signed and unsigned differ.

Saturation modifier:

`min.relu.{s16x2, s32}` clamps the result to 0 if negative.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`min.u16x2`, `min{.relu}.s16x2` and `min.relu.s32` introduced in PTX ISA version 8.0.

Target ISA Notes

Supported on all target architectures.

`min.u16x2`, `min{.relu}.s16x2` and `min.relu.s32` require `sm_90` or higher.

Examples

```
min.s32  r0,a,b;
@p min.u16  h,i,j;
min.s16x2.relu u,v,w;
```

9.7.1.13 Integer Arithmetic Instructions: max

max

Find the maximum of two values.

Syntax

```
max.atype      d, a, b;
max{.relu}.btype d, a, b;

.atype = { .u16, .u32, .u64,
           .u16x2, .s16, .s64 };
.btype = { .s16x2, .s32 };
```

Description

Store the maximum of a and b in d.

For `.u16x2`, `.s16x2` instruction types, forms input vectors by half word values from source operands. Half-word operands are then processed in parallel to produce `.u16x2`, `.s16x2` result in destination.

Operands d, a and b have the same type as the instruction type. For instruction types `.u16x2`, `.s16x2`, operands d, a and b have type `.b32`.

Semantics

```
if (type == u16x2 || type == s16x2) {
    iA[0] = a[0:15];
    iA[1] = a[16:31];
    iB[0] = b[0:15];
    iB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = (iA[i] > iB[i]) ? iA[i] : iB[i];
    }
} else {
    d = (a > b) ? a : b; // Integer (signed and unsigned)
}
```

Notes

Signed and unsigned differ.

Saturation modifier:

`max.relu.{s16x2, s32}` clamps the result to 0 if negative.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`max.u16x2`, `max{.relu}.s16x2` and `max.relu.s32` introduced in PTX ISA version 8.0.

Target ISA Notes

Supported on all target architectures.

`max.u16x2`, `max{.relu}.s16x2` and `max.relu.s32` require `sm_90` or higher.

Examples

```
max.u32  d,a,b;
max.s32  q,q,0;
max.relu.s16x2 t,t,u;
```

9.7.1.14 Integer Arithmetic Instructions: popc

popc

Population count.

Syntax

```
popc.type  d, a;
.type = { .b32, .b64 };
```

Description

Count the number of one bits in *a* and place the resulting *population count* in 32-bit destination register *d*. Operand *a* has the instruction type and destination *d* has type `.u32`.

Semantics

```
.u32  d = 0;
while (a != 0) {
    if (a & 0x1) d++;
    a = a >> 1;
}
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

`popc` requires `sm_20` or higher.

Examples

```
popc.b32  d, a;
popc.b64  cnt, X;  // cnt is .u32
```

9.7.1.15 Integer Arithmetic Instructions: clz

clz

Count leading zeros.

Syntax

```
clz.type  d, a;
.type = { .b32, .b64 };
```

Description

Count the number of leading zeros in *a* starting with the most-significant bit and place the result in 32-bit destination register *d*. Operand *a* has the instruction type, and destination *d* has type `.u32`. For `.b32` type, the number of leading zeros is between 0 and 32, inclusively. For `.b64` type, the number of leading zeros is between 0 and 64, inclusively.

Semantics

```
.u32  d = 0;
if (.type == .b32) { max = 32; mask = 0x80000000; }
else { max = 64; mask = 0x8000000000000000; }

while (d < max && (a&mask == 0) ) {
    d++;
    a = a << 1;
}
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

clz requires sm_20 or higher.

Examples

```
clz.b32 d, a;
clz.b64 cnt, X; // cnt is .u32
```

9.7.1.16 Integer Arithmetic Instructions: bfind

bfind

Find most significant non-sign bit.

Syntax

```
bfind.type          d, a;
bfind.shiftamt.type d, a;

.type = { .u32, .u64,
          .s32, .s64 };

```

Description

Find the bit position of the most significant non-sign bit in a and place the result in d. Operand a has the instruction type, and destination d has type .u32. For unsigned integers, bfind returns the bit position of the most significant 1. For signed integers, bfind returns the bit position of the most significant 0 for negative inputs and the most significant 1 for non-negative inputs.

If .shiftamt is specified, bfind returns the shift amount needed to left-shift the found bit into the most-significant bit position.

bfind returns 0xffffffff if no non-sign bit is found.

Semantics

```
msb = (.type==.u32 || .type==.s32) ? 31 : 63;
// negate negative signed inputs
if ( (.type==.s32 || .type==.s64) && (a & (1<<msb)) ) {
    a = ~a;
}
.u32 d = 0xffffffff;
for (.s32 i=msb; i>=0; i--) {
    if (a & (1<<i)) { d = i; break; }
}
if (.shiftamt && d != 0xffffffff) { d = msb - d; }
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

bfind requires sm_20 or higher.

Examples

```
bfind.u32 d, a;
bfind.shiftamt.s64 cnt, X; // cnt is .u32
```

9.7.1.17 Integer Arithmetic Instructions: fns

fns

Find the n-th set bit

Syntax

```
fns.b32 d, mask, base, offset;
```

Description

Given a 32-bit value mask and an integer value base (between 0 and 31), find the n-th (given by offset) set bit in mask from the base bit, and store the bit position in d. If not found, store 0xffffffff in d.

Operand mask has a 32-bit type. Operand base has .b32, .u32 or .s32 type. Operand offset has .s32 type. Destination d has type .b32.

Operand base must be ≤ 31 , otherwise behavior is undefined.

Semantics

```
d = 0xffffffff;
if (offset == 0) {
    if (mask[base] == 1) {
        d = base;
    }
} else {
    pos = base;
    count = |offset| - 1;
    inc = (offset > 0) ? 1 : -1;

    while ((pos >= 0) && (pos < 32)) {
        if (mask[pos] == 1) {
```

(continues on next page)

(continued from previous page)

```

        if (count == 0) {
            d = pos;
            break;
        } else {
            count = count - 1;
        }
    }
    pos = pos + inc;
}

```

PTX ISA Notes

Introduced in PTX ISA version 6.0.

Target ISA Notes

fns requires sm_30 or higher.

Examples

```

fns.b32 d, 0xffffffff, 3, 1;    // d = 3
fns.b32 d, 0xffffffff, 3, -1;   // d = 3
fns.b32 d, 0xffffffff, 2, 1;    // d = 3
fns.b32 d, 0xffffffff, 2, -1;   // d = 1

```

9.7.1.18 Integer Arithmetic Instructions: brev

brev

Bit reverse.

Syntax

```

brev.type d, a;

.type = { .b32, .b64 };

```

Description

Perform bitwise reversal of input.

Semantics

```
msb = (.type==.b32) ? 31 : 63;
for (i=0; i<=msb; i++) {
    d[i] = a[msb-i];
}
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

brev requires sm_20 or higher.

Examples

```
brev.b32 d, a;
```

9.7.1.19 Integer Arithmetic Instructions: bfe

bfe

Bit Field Extract.

Syntax

```
bfe.type d, a, b, c;
.type = { .u32, .u64,
          .s32, .s64 };
```

Description

Extract bit field from *a* and place the zero or sign-extended result in *d*. Source *b* gives the bit field starting bit position, and source *c* gives the bit field length in bits.

Operands *a* and *d* have the same type as the instruction type. Operands *b* and *c* are type `.u32`, but are restricted to the 8-bit value range 0..255.

The sign bit of the extracted field is defined as:

.u32, .u64:

zero

.s32, .s64:

msb of input *a* if the extracted field extends beyond the msb of *a*; msb of extracted field, otherwise

If the bit field length is zero, the result is zero.

The destination *d* is padded with the sign bit of the extracted field. If the start position is beyond the msb of the input, the destination *d* is filled with the replicated sign bit of the extracted field.

Semantics

```
msb = (.type==.u32 || .type==.s32) ? 31 : 63;
pos = b & 0xff; // pos restricted to 0..255 range
len = c & 0xff; // len restricted to 0..255 range

if (.type==.u32 || .type==.u64 || len==0)
    sbit = 0;
else
    sbit = a[min(pos+len-1,msb)];

d = 0;
for (i=0; i<=msb; i++) {
    d[i] = (i<len && pos+i<=msb) ? a[pos+i] : sbit;
}
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

`bfe` requires `sm_20` or higher.

Examples

```
bfe.b32 d,a,start,len;
```

9.7.1.20 Integer Arithmetic Instructions: bfi

bfi

Bit Field Insert.

Syntax

```
bfi.type f, a, b, c, d;  
.type = { .b32, .b64 };
```

Description

Align and insert a bit field from *a* into *b*, and place the result in *f*. Source *c* gives the starting bit position for the insertion, and source *d* gives the bit field length in bits.

Operands *a*, *b*, and *f* have the same type as the instruction type. Operands *c* and *d* are type `.u32`, but are restricted to the 8-bit value range `0..255`.

If the bit field length is zero, the result is *b*.

If the start position is beyond the msb of the input, the result is *b*.

Semantics

```
msb = (.type==.b32) ? 31 : 63;  
pos = c & 0xff; // pos restricted to 0..255 range  
len = d & 0xff; // len restricted to 0..255 range  
  
f = b;  
for (i=0; i<len && pos+i<=msb; i++) {  
    f[pos+i] = a[i];  
}
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

`bfi` requires `sm_20` or higher.

Examples

```
bfi.b32 d,a,b,start,len;
```

9.7.1.21 Integer Arithmetic Instructions: `szext`

`szext`

Sign-extend or Zero-extend.

Syntax

```
szext.mode.type d, a, b;

.mode = { .clamp, .wrap };
.type = { .u32, .s32 };
```

Description

Sign-extends or zero-extends an N-bit value from operand `a` where `N` is specified in operand `b`. The resulting value is stored in the destination operand `d`.

For the `.s32` instruction type, the value in `a` is treated as an N-bit signed value and the most significant bit of this N-bit value is replicated up to bit 31. For the `.u32` instruction type, the value in `a` is treated as an N-bit unsigned number and is zero-extended to 32 bits. Operand `b` is an unsigned 32-bit value.

If the value of `N` is 0, then the result of `szext` is 0. If the value of `N` is 32 or higher, then the result of `szext` depends upon the value of the `.mode` qualifier as follows:

- If `.mode` is `.clamp`, then the result is the same as the source operand `a`.
- If `.mode` is `.wrap`, then the result is computed using the wrapped value of `N`.

Semantics

```
b1      = b & 0x1f;
too_large = (b >= 32 && .mode == .clamp) ? true : false;
mask     = too_large ? 0 : (~0) << b1;
sign_pos  = (b1 - 1) & 0x1f;

if (b1 == 0 || too_large || .type != .s32) {
    sign_bit = false;
} else {
    sign_bit = (a >> sign_pos) & 1;
}
d = (a & ~mask) | (sign_bit ? mask | 0);
```

PTX ISA Notes

Introduced in PTX ISA version 7.6.

Target ISA Notes

szext requires sm_70 or higher.

Examples

```
szext.clamp.s32 rd, ra, rb;  
szext.wrap.u32  rd, 0xffffffff, 0; // Result is 0.
```

9.7.1.22 Integer Arithmetic Instructions: bmsk

bmsk

Bit Field Mask.

Syntax

```
bmsk.mode.b32  d, a, b;  
  
.mode = { .clamp, .wrap };
```

Description

Generates a 32-bit mask starting from the bit position specified in operand a, and of the width specified in operand b. The generated bitmask is stored in the destination operand d.

The resulting bitmask is 0 in the following cases:

- ▶ When the value of a is 32 or higher and .mode is .clamp.
- ▶ When either the specified value of b or the wrapped value of b (when .mode is specified as .wrap) is 0.

Semantics

```
a1      = a & 0x1f;  
mask0   = (~0) << a1;  
b1      = b & 0x1f;  
sum     = a1 + b1;  
mask1   = (~0) << sum;  
  
sum-overflow      = sum >= 32 ? true : false;  
bit-position-overflow = false;  
bit-width-overflow  = false;
```

(continues on next page)

(continued from previous page)

```

if (.mode == .clamp) {
    if (a >= 32) {
        bit-position-overflow = true;
        mask0 = 0;
    }
    if (b >= 32) {
        bit-width-overflow = true;
    }
}

if (sum-overflow || bit-position-overflow || bit-width-overflow) {
    mask1 = 0;
} else if (b1 == 0) {
    mask1 = ~0;
}
d = mask0 & ~mask1;

```

Notes

The bitmask width specified by operand b is limited to range 0..32 in .clamp mode and to range 0..31 in .wrap mode.

PTX ISA Notes

Introduced in PTX ISA version 7.6.

Target ISA Notes

bmsk requires sm_70 or higher.

Examples

```

bmsk.clamp.b32  rd, ra, rb;
bmsk.wrap.b32   rd, 1, 2; // Creates a bitmask of 0x00000006.

```

9.7.1.23 Integer Arithmetic Instructions: dp4a

dp4a

Four-way byte dot product-accumulate.

Syntax

```
dp4a.atype.btype  d, a, b, c;  
.atype = .btype = { .u32, .s32 };
```

Description

Four-way byte dot product which is accumulated in 32-bit result.

Operand a and b are 32-bit inputs which hold 4 byte inputs in packed form for dot product.

Operand c has type .u32 if both .atype and .btype are .u32 else operand c has type .s32.

Semantics

```
d = c;  
  
// Extract 4 bytes from a 32bit input and sign or zero extend  
// based on input type.  
Va = extractAndSignOrZeroExt_4(a, .atype);  
Vb = extractAndSignOrZeroExt_4(b, .btype);  
  
for (i = 0; i < 4; ++i) {  
    d += Va[i] * Vb[i];  
}
```

PTX ISA Notes

Introduced in PTX ISA version 5.0.

Target ISA Notes

Requires sm_61 or higher.

Examples

```
dp4a.u32.u32      d0, a0, b0, c0;  
dp4a.u32.s32      d1, a1, b1, c1;
```

9.7.1.24 Integer Arithmetic Instructions: dp2a

dp2a

Two-way dot product-accumulate.

Syntax

```
dp2a.mode.atype.btype d, a, b, c;

.atype = .btype = { .u32, .s32 };
.mode = { .lo, .hi };
```

Description

Two-way 16-bit to 8-bit dot product which is accumulated in 32-bit result.

Operand a and b are 32-bit inputs. Operand a holds two 16-bits inputs in packed form and operand b holds 4 byte inputs in packed form for dot product.

Depending on the .mode specified, either lower half or upper half of operand b will be used for dot product.

Operand c has type .u32 if both .atype and .btype are .u32 else operand c has type .s32.

Semantics

```
d = c;
// Extract two 16-bit values from a 32-bit input and sign or zero extend
// based on input type.
Va = extractAndSignOrZeroExt_2(a, .atype);

// Extract four 8-bit values from a 32-bit input and sign or zer extend
// based on input type.
Vb = extractAndSignOrZeroExt_4(b, .btype);

b_select = (.mode == .lo) ? 0 : 2;

for (i = 0; i < 2; ++i) {
    d += Va[i] * Vb[b_select + i];
}
```

PTX ISA Notes

Introduced in PTX ISA version 5.0.

Target ISA Notes

Requires sm_61 or higher.

Examples

```
dp2a.lo.u32.u32      d0, a0, b0, c0;  
dp2a.hi.u32.s32      d1, a1, b1, c1;
```

9.7.2. Extended-Precision Integer Arithmetic Instructions

Instructions `add.cc`, `addc`, `sub.cc`, `subc`, `mad.cc` and `madc` reference an implicitly specified condition code register (CC) having a single carry flag bit (CC.CF) holding carry-in/carry-out or borrow-in/borrow-out. These instructions support extended-precision integer addition, subtraction, and multiplication. No other instructions access the condition code, and there is no support for setting, clearing, or testing the condition code. The condition code register is not preserved across calls and is mainly intended for use in straight-line code sequences for computing extended-precision integer addition, subtraction, and multiplication.

The extended-precision arithmetic instructions are:

- ▶ `add.cc`, `addc`
- ▶ `sub.cc`, `subc`
- ▶ `mad.cc`, `madc`

9.7.2.1 Extended-Precision Arithmetic Instructions: `add.cc`

`add.cc`

Add two values with carry-out.

Syntax

```
add.cc.type d, a, b;  
.type = { .u32, .s32, .u64, .s64 };
```

Description

Performs integer addition and writes the carry-out value into the condition code register.

Semantics

```
d = a + b;
```

carry-out written to CC.CF

Notes

No integer rounding modifiers.

No saturation.

Behavior is the same for unsigned and signed integers.

PTX ISA Notes

32-bit add.cc introduced in PTX ISA version 1.2.

64-bit add.cc introduced in PTX ISA version 4.3.

Target ISA Notes

32-bit add.cc is supported on all target architectures.

64-bit add.cc requires sm_20 or higher.

Examples

```

@p add.cc.u32    x1,y1,z1;    // extended-precision addition of
@p addc.cc.u32   x2,y2,z2;    // two 128-bit values
@p addc.cc.u32   x3,y3,z3;
@p addc.u32      x4,y4,z4;

```

9.7.2.2 Extended-Precision Arithmetic Instructions: addc

addc

Add two values with carry-in and optional carry-out.

Syntax

```
addc{.cc}.type  d, a, b;  
.type = { .u32, .s32, .u64, .s64 };
```

Description

Performs integer addition with carry-in and optionally writes the carry-out value into the condition code register.

Semantics

```
d = a + b + CC.CF;
```

if `.cc` specified, carry-out written to `CC.CF`

Notes

No integer rounding modifiers.

No saturation.

Behavior is the same for unsigned and signed integers.

PTX ISA Notes

32-bit `addc` introduced in PTX ISA version 1.2.

64-bit `addc` introduced in PTX ISA version 4.3.

Target ISA Notes

32-bit `addc` is supported on all target architectures.

64-bit `addc` requires `sm_20` or higher.

Examples

```
@p  add.cc.u32    x1,y1,z1;    // extended-precision addition of  
@p  addc.cc.u32   x2,y2,z2;    // two 128-bit values  
@p  addc.cc.u32   x3,y3,z3;  
@p  addc.u32      x4,y4,z4;
```

9.7.2.3 Extended-Precision Arithmetic Instructions: `sub.cc`

`sub.cc`

Subtract one value from another, with borrow-out.

Syntax

```
sub.cc.type d, a, b;  
.type = { .u32, .s32, .u64, .s64 };
```

Description

Performs integer subtraction and writes the borrow-out value into the condition code register.

Semantics

```
d = a - b;
```

borrow-out written to `CC.CF`

Notes

No integer rounding modifiers.

No saturation.

Behavior is the same for unsigned and signed integers.

PTX ISA Notes

32-bit `sub.cc` introduced in PTX ISA version 1.2.

64-bit `sub.cc` introduced in PTX ISA version 4.3.

Target ISA Notes

32-bit `sub.cc` is supported on all target architectures.

64-bit `sub.cc` requires `sm_20` or higher.

Examples

```
@p sub.cc.u32    x1,y1,z1;    // extended-precision subtraction
@p subc.cc.u32   x2,y2,z2;    // of two 128-bit values
@p subc.cc.u32   x3,y3,z3;
@p subc.u32      x4,y4,z4;
```

9.7.2.4 Extended-Precision Arithmetic Instructions: subc

subc

Subtract one value from another, with borrow-in and optional borrow-out.

Syntax

```
subc{.cc}.type  d, a, b;

.type = { .u32, .s32, .u64, .s64 };
```

Description

Performs integer subtraction with borrow-in and optionally writes the borrow-out value into the condition code register.

Semantics

```
d = a - (b + CC.CF);
```

if .cc specified, borrow-out written to CC.CF

Notes

No integer rounding modifiers.

No saturation.

Behavior is the same for unsigned and signed integers.

PTX ISA Notes

32-bit subc introduced in PTX ISA version 1.2.

64-bit subc introduced in PTX ISA version 4.3.

Target ISA Notes

32-bit subc is supported on all target architectures.

64-bit subc requires sm_20 or higher.

Examples

```
@p sub.cc.u32   x1,y1,z1;    // extended-precision subtraction
@p subc.cc.u32  x2,y2,z2;    // of two 128-bit values
@p subc.cc.u32  x3,y3,z3;
@p subc.u32     x4,y4,z4;
```

9.7.2.5 Extended-Precision Arithmetic Instructions: mad.cc

mad.cc

Multiply two values, extract high or low half of result, and add a third value with carry-out.

Syntax

```
mad{.hi,.lo}.cc.type  d, a, b, c;
.type = { .u32, .s32, .u64, .s64 };
```

Description

Multiplies two values, extracts either the high or low part of the result, and adds a third value. Writes the result to the destination register and the carry-out from the addition into the condition code register.

Semantics

```
t = a * b;
d = t<63..32> + c;    // for .hi variant
d = t<31..0> + c;     // for .lo variant
```

carry-out from addition is written to CC.CF

Notes

Generally used in combination with `madc` and `addc` to implement extended-precision multi-word multiplication. See `madc` for an example.

PTX ISA Notes

32-bit `mad.lo.cc` introduced in PTX ISA version 3.0.

64-bit `mad.lo.cc` introduced in PTX ISA version 4.3.

Target ISA Notes

Requires target `sm_20` or higher.

Examples

```
@p  mad.lo.cc.u32 d,a,b,c;  
    mad.lo.cc.u32 r,p,q,r;
```

9.7.2.6 Extended-Precision Arithmetic Instructions: `madc`

`madc`

Multiply two values, extract high or low half of result, and add a third value with carry-in and optional carry-out.

Syntax

```
madc{.hi,.lo}{.cc}.type d, a, b, c;  
.type = { .u32, .s32, .u64, .s64 };
```

Description

Multiplies two values, extracts either the high or low part of the result, and adds a third value along with carry-in. Writes the result to the destination register and optionally writes the carry-out from the addition into the condition code register.

Semantics

```
t = a * b;
d = t<63..32> + c + CC.CF;    // for .hi variant
d = t<31..0> + c + CC.CF;    // for .lo variant
```

if .cc specified, carry-out from addition is written to CC.CF

Notes

Generally used in combination with mad.cc and addc to implement extended-precision multi-word multiplication. See example below.

PTX ISA Notes

32-bit maddc introduced in PTX ISA version 3.0.

64-bit maddc introduced in PTX ISA version 4.3.

Target ISA Notes

Requires target sm_20 or higher.

Examples

```
// extended-precision multiply: [r3,r2,r1,r0] = [r5,r4] * [r7,r6]
mul.lo.u32    r0,r4,r6;    // r0=(r4*r6).[31:0], no carry-out
mul.hi.u32    r1,r4,r6;    // r1=(r4*r6).[63:32], no carry-out
mad.lo.cc.u32 r1,r5,r6,r1; // r1+=(r5*r6).[31:0], may carry-out
maddc.hi.u32  r2,r5,r6,0;  // r2=(r5*r6).[63:32]+carry-in,
                          // no carry-out
mad.lo.cc.u32 r1,r4,r7,r1; // r1+=(r4*r7).[31:0], may carry-out
maddc.hi.cc.u32 r2,r4,r7,r2; // r2+=(r4*r7).[63:32]+carry-in,
                          // may carry-out
addc.u32      r3,0,0;      // r3 = carry-in, no carry-out
mad.lo.cc.u32 r2,r5,r7,r2; // r2+=(r5*r7).[31:0], may carry-out
maddc.hi.u32  r3,r5,r7,r3; // r3+=(r5*r7).[63:32]+carry-in
```

9.7.3. Floating-Point Instructions

Floating-point instructions operate on .f32 and .f64 register operands and constant immediate values. The floating-point instructions are:

- testp
- copysign
- add
- sub

- ▶ `mul`
- ▶ `fma`
- ▶ `mad`
- ▶ `div`
- ▶ `abs`
- ▶ `neg`
- ▶ `min`
- ▶ `max`
- ▶ `rcp`
- ▶ `sqrt`
- ▶ `rsqrt`
- ▶ `sin`
- ▶ `cos`
- ▶ `lg2`
- ▶ `ex2`
- ▶ `tanh`

Instructions that support rounding modifiers are IEEE-754 compliant. Double-precision instructions support subnormal inputs and results. Single-precision instructions support subnormal inputs and results by default for `sm_20` and subsequent targets, and flush subnormal inputs and results to sign-preserving zero for `sm_1x` targets. The optional `.ftz` modifier on single-precision instructions provides backward compatibility with `sm_1x` targets by flushing subnormal inputs and results to sign-preserving zero regardless of the target architecture.

Single-precision `add`, `sub`, `mul`, and `mad` support saturation of results to the range `[0.0, 1.0]`, with NaNs being flushed to positive zero. NaN payloads are supported for double-precision instructions (except for `rcp.approx.ftz.f64` and `rsqrt.approx.ftz.f64`, which maps input NaNs to a canonical NaN). Single-precision instructions return an unspecified NaN. Note that future implementations may support NaN payloads for single-precision instructions, so PTX programs should not rely on the specific single-precision NaNs being generated.

Table 29 summarizes floating-point instructions in PTX.

Table 29: Summary of Floating-Point Instructions

Instruction	.rn	.rz	.rm	.rp	.ftz	.sat	Notes
{add, sub, mul}.rnd.f32	x	x	x	x	x	x	If no rounding modifier is specified, default is .rn and instructions may be folded into a multiply-add.
{add, sub, mul}.rnd.f64	x	x	x	x	n/a	n/a	If no rounding modifier is specified, default is .rn and instructions may be folded into a multiply-add.
mad.f32	n/a	n/a	n/a	n/a	x	x	.target sm_1x No rounding modifier.
{mad, fma}.rnd.f32	x	x	x	x	x	x	.target sm_20 or higher mad.f32 and fma.f32 are the same.
{mad, fma}.rnd.f64	x	x	x	x	n/a	n/a	mad.f64 and fma.f64 are the same.
div.full.f32	n/a	n/a	n/a	n/a	x	n/a	No rounding modifier.
{div, rcp, sqrt}.approx.f32	n/a	n/a	n/a	n/a	x	n/a	n/a
rcp.approx.ftz.f64	n/a	n/a	n/a	n/a	x	n/a	.target sm_20 or higher
{div, rcp, sqrt}.rnd.f32	x	x	x	x	x	n/a	.target sm_20 or higher
{div, rcp, sqrt}.rnd.f64	x	x	x	x	n/a	n/a	.target sm_20 or higher
{abs, neg, min, max}.f32	n/a	n/a	n/a	n/a	x	n/a	
{abs, neg, min, max}.f64	n/a	n/a	n/a	n/a	n/a	n/a	
rsqrt.approx.f32	n/a	n/a	n/a	n/a	x	n/a	
rsqrt.approx.f64	n/a	n/a	n/a	n/a	n/a	n/a	
rsqrt.approx.ftz.f64	n/a	n/a	n/a	n/a	x	n/a	.target sm_20 or higher
{sin, cos, lg2, ex2}.approx.f32	n/a	n/a	n/a	n/a	x	n/a	
tanh.approx.f32	n/a	n/a	n/a	n/a	n/a	n/a	.target sm_75 or higher

9.7.3.1 Floating Point Instructions: testp

testp

Test floating-point property.

Syntax

```
testp.op.type p, a; // result is .pred

.op    = { .finite, .infinite,
           .number, .notanumber,
           .normal, .subnormal };
.type  = { .f32, .f64 };
```

Description

testp tests common properties of floating-point numbers and returns a predicate value of 1 if True and 0 if False.

testp.finite

True if the input is not infinite or NaN

testp.infinite

True if the input is positive or negative infinity

testp.number

True if the input is not NaN

testp.notanumber

True if the input is NaN

testp.normal

True if the input is a normal number (not NaN, not infinity)

testp.subnormal

True if the input is a subnormal number (not NaN, not infinity)

As a special case, positive and negative zero are considered normal numbers.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

Requires sm_20 or higher.

Examples

```
testp.notanumber.f32  isnan, f0;
testp.infinite.f64    p, X;
```

9.7.3.2 Floating Point Instructions: copysign

copysign

Copy sign of one input to another.

Syntax

```
copysign.type  d, a, b;
.type = { .f32, .f64 };
```

Description

Copy sign bit of a into value of b, and return the result as d.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

Requires sm_20 or higher.

Examples

```
copysign.f32  x, y, z;
copysign.f64  A, B, C;
```

9.7.3.3 Floating Point Instructions: add

add

Add two values.

Syntax

```
add{.rnd}{.ftz}{.sat}.f32  d, a, b;  
add{.rnd}{.ftz}.f32x2     d, a, b;  
add{.rnd}.f64             d, a, b;  
  
.rnd = { .rn, .rz, .rm, .rp };
```

Description

Performs addition and writes the resulting value into a destination register.

For `.f32x2` instruction type, forms input vectors of single precision (`.f32`) values from source operands. Single precision (`.f32`) operands are then added in parallel to produce `.f32x2` result in destination.

For `.f32x2` instruction type, operands `d`, `a` and `b` have `.b64` type.

Semantics

```
if (type == f32 || type == f64) {  
    d = a + b;  
} else if (type == f32x2) {  
    fA[0] = a[0:31];  
    fA[1] = a[32:63];  
    fB[0] = b[0:31];  
    fB[1] = b[32:63];  
    for (i = 0; i < 2; i++) {  
        d[i] = fA[i] + fB[i];  
    }  
}
```

Notes

Rounding modifiers:

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

The default value of rounding modifier is `.rn`. Note that an `add` instruction with an explicit rounding modifier is treated conservatively by the code optimizer. An `add` instruction with no rounding modifier

defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, `mul/add` sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`add.ftz.f32`, `add.ftz.f32x2` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`add.f64` supports subnormal numbers.

`add.f32` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`add.sat.f32` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`add.f32x2` introduced in PTX ISA version 8.6.

Target ISA Notes

`add.f32` supported on all target architectures.

`add.f64` requires `sm_13` or higher.

Rounding modifiers have the following target requirements:

.rn, .rz

available for all targets

.rm, .rp

for `add.f64`, requires `sm_13` or higher.

for `add.f32`, requires `sm_20` or higher.

`add.f32x2` requires `sm_100` or higher.

Examples

```
@p add.rz.ftz.f32 f1, f2, f3;
add.rp.ftz.f32x2 d, a, b;
```

9.7.3.4 Floating Point Instructions: sub

sub

Subtract one value from another.

Syntax

```
sub{.rnd}{.ftz}{.sat}.f32  d, a, b;  
sub{.rnd}{.ftz}.f32x2     d, a, b;  
sub{.rnd}.f64             d, a, b;  
  
.rnd = { .rn, .rz, .rm, .rp };
```

Description

Performs subtraction and writes the resulting value into a destination register.

For `.f32x2` instruction type, forms input vectors of single precision (`.f32`) values from source operands. Single precision (`.f32`) operands are then subtracted in parallel to produce `.f32x2` result in destination.

For `.f32x2` instruction type, operands `d`, `a` and `b` have `.b64` type.

Semantics

```
if (type == f32 || type == f64) {  
    d = a - b;  
} else if (type == f32x2) {  
    fA[0] = a[0:31];  
    fA[1] = a[32:63];  
    fB[0] = b[0:31];  
    fB[1] = b[32:63];  
    for (i = 0; i < 2; i++) {  
        d[i] = fA[i] - fB[i];  
    }  
}
```

Notes

Rounding modifiers:

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

The default value of rounding modifier is `.rn`. Note that a `sub` instruction with an explicit rounding modifier is treated conservatively by the code optimizer. A `sub` instruction with no rounding modifier

defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, `mul/sub` sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`sub.ftz.f32`, `sub.ftz.f32x2` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`sub.f64` supports subnormal numbers.

`sub.f32` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`sub.sat.f32` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`sub.f32x2` introduced in PTX ISA version 8.6.

Target ISA Notes

`sub.f32` supported on all target architectures.

`sub.f64` requires `sm_13` or higher.

Rounding modifiers have the following target requirements:

.rn, .rz

available for all targets

.rm, .rp

for `sub.f64`, requires `sm_13` or higher.

for `sub.f32`, requires `sm_20` or higher.

`sub.f32x2` requires `sm_100` or higher.

Examples

```
sub.f32 c,a,b;
sub.rn.ftz.f32 f1,f2,f3;
```

9.7.3.5 Floating Point Instructions: mul

mul

Multiply two values.

Syntax

```
mul{.rnd}{.ftz}{.sat}.f32  d, a, b;  
mul{.rnd}{.ftz}.f32x2      d, a, b;  
mul{.rnd}.f64              d, a, b;  
  
.rnd = { .rn, .rz, .rm, .rp };
```

Description

Compute the product of two values.

For `.f32x2` instruction type, forms input vectors of single precision (`.f32`) values from source operands. Single precision (`.f32`) operands are then multiplied in parallel to produce `.f32x2` result in destination.

For `.f32x2` instruction type, operands `d`, `a` and `b` have `.b64` type.

Semantics

```
if (type == f32 || type == f64) {  
    d = a * b;  
} else if (type == f32x2) {  
    fA[0] = a[0:31];  
    fA[1] = a[32:63];  
    fB[0] = b[0:31];  
    fB[1] = b[32:63];  
    for (i = 0; i < 2; i++) {  
        d[i] = fA[i] * fB[i];  
    }  
}
```

Notes

For floating-point multiplication, all operands must be the same size.

Rounding modifiers:

- `.rn` mantissa LSB rounds to nearest even
- `.rz` mantissa LSB rounds towards zero
- `.rm` mantissa LSB rounds towards negative infinity
- `.rp` mantissa LSB rounds towards positive infinity

The default value of rounding modifier is `.rn`. Note that a `mul` instruction with an explicit rounding modifier is treated conservatively by the code optimizer. A `mul` instruction with no rounding modifier defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, `mul/add` and `mul/sub` sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`mul.ftz.f32`, `mul.ftz.f32x2` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`mul.f64` supports subnormal numbers.

`mul.f32` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`mul.sat.f32` clamps the result to `[0.0, 1.0]`. NaN results are flushed to `+0.0f`.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`mul.f32x2` introduced in PTX ISA version 8.6.

Target ISA Notes

`mul.f32` supported on all target architectures.

`mul.f64` requires `sm_13` or higher.

Rounding modifiers have the following target requirements:

.rn, .rz

available for all targets

.rm, .rp

for `mul.f64`, requires `sm_13` or higher.

for `mul.f32`, requires `sm_20` or higher.

`mul.f32x2` requires `sm_100` or higher.

Examples

```
mul.ftz.f32 circumf,radius,pi // a single-precision multiply
```

9.7.3.6 Floating Point Instructions: fma

fma

Fused multiply-add.

Syntax

```
fma.rnd{.ftz}{.sat}.f32  d, a, b, c;  
fma.rnd{.ftz}.f32x2     d, a, b, c;  
fma.rnd.f64             d, a, b, c;  
  
.rnd = { .rn, .rz, .rm, .rp };
```

Description

Performs a fused multiply-add with no loss of precision in the intermediate product and addition.

For `.f32x2` instruction type, forms input vectors of single precision (`.f32`) values from source operands. Single precision (`.f32`) operands are then operated in parallel to produce `.f32x2` result in destination.

For `.f32x2` instruction type, operands `d`, `a`, `b` and `c` have `.b64` type.

Semantics

```
if (type == f32 || type == f64) {  
    d = a * b + c;  
} else if (type == f32x2) {  
    fA[0] = a[0:31];  
    fA[1] = a[32:63];  
    fB[0] = b[0:31];  
    fB[1] = b[32:63];  
    fC[0] = c[0:31];  
    fC[1] = c[32:63];  
    for (i = 0; i < 2; i++) {  
        d[i] = fA[i] * fB[i] + fC[i];  
    }  
}
```

Notes

`fma.f32` computes the product of `a` and `b` to infinite precision and then adds `c` to this product, again in infinite precision. The resulting value is then rounded to single precision using the rounding mode specified by `.rnd`.

`fma.f64` computes the product of `a` and `b` to infinite precision and then adds `c` to this product, again in infinite precision. The resulting value is then rounded to double precision using the rounding mode specified by `.rnd`.

`fma.f64` is the same as `mad.f64`.

Rounding modifiers (no default):

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`fma.ftz.f32`, `fma.ftz.f32x2` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`fma.f64` supports subnormal numbers.

`fma.f32` is unimplemented for `sm_1x` targets.

Saturation:

`fma.sat.f32` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`.

PTX ISA Notes

`fma.f64` introduced in PTX ISA version 1.4.

`fma.f32` introduced in PTX ISA version 2.0.

`fma.f32x2` introduced in PTX ISA version 8.6.

Target ISA Notes

`fma.f32` requires `sm_20` or higher.

`fma.f64` requires `sm_13` or higher.

`fma.f32x2` requires `sm_100` or higher.

Examples

```
@p fma.rn.ftz.f32 w,x,y,z;
    fma.rn.f64    d,a,b,c;
    fma.rp.ftz.f32x2 p,q,r,s;
```

9.7.3.7 Floating Point Instructions: mad

mad

Multiply two values and add a third value.

Syntax

```
mad{.ftz}{.sat}.f32    d, a, b, c;    // .target sm_1x
mad.rnd{.ftz}{.sat}.f32 d, a, b, c;    // .target sm_20
mad.rnd.f64           d, a, b, c;    // .target sm_13 and higher

.rnd = { .rn, .rz, .rm, .rp };
```

Description

Multiplies two values and adds a third, and then writes the resulting value into a destination register.

Semantics

```
d = a*b + c;
```

Notes

For `.target sm_20` and higher:

- `mad.f32` computes the product of `a` and `b` to infinite precision and then adds `c` to this product, again in infinite precision. The resulting value is then rounded to single precision using the rounding mode specified by `.rnd`.
- `mad.f64` computes the product of `a` and `b` to infinite precision and then adds `c` to this product, again in infinite precision. The resulting value is then rounded to double precision using the rounding mode specified by `.rnd`.
- `mad.{f32, f64}` is the same as `fma.{f32, f64}`.

For `.target sm_1x`:

- `mad.f32` computes the product of `a` and `b` at double precision, and then the mantissa is truncated to 23 bits, but the exponent is preserved. Note that this is different from computing the product with `mul`, where the mantissa can be rounded and the exponent will be clamped. The exception for `mad.f32` is when `c = +/-0.0`, `mad.f32` is identical to the result computed using separate `mul` and `add` instructions. When JIT-compiled for SM 2.0 devices, `mad.f32` is implemented as a fused multiply-add (i.e., `fma.rn.ftz.f32`). In this case, `mad.f32` can produce slightly different numeric results and backward compatibility is not guaranteed in this case.
- `mad.f64` computes the product of `a` and `b` to infinite precision and then adds `c` to this product, again in infinite precision. The resulting value is then rounded to double precision using the rounding mode specified by `.rnd`. Unlike `mad.f32`, the treatment of subnormal inputs and output follows IEEE 754 standard.
- `mad.f64` is the same as `fma.f64`.

Rounding modifiers (no default):

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`mad.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`mad.f64` supports subnormal numbers.

`mad.f32` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`mad.sat.f32` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

In PTX ISA versions 1.4 and later, a rounding modifier is required for `mad.f64`.

Legacy `mad.f64` instructions having no rounding modifier will map to `mad.rn.f64`.

In PTX ISA versions 2.0 and later, a rounding modifier is required for `mad.f32` for `sm_20` and higher targets.

Errata

`mad.f32` requires a rounding modifier for `sm_20` and higher targets. However for PTX ISA version 3.0 and earlier, `ptxas` does not enforce this requirement and `mad.f32` silently defaults to `mad.rn.f32`. For PTX ISA version 3.1, `ptxas` generates a warning and defaults to `mad.rn.f32`, and in subsequent releases `ptxas` will enforce the requirement for PTX ISA version 3.2 and later.

Target ISA Notes

`mad.f32` supported on all target architectures.

`mad.f64` requires `sm_13` or higher.

Rounding modifiers have the following target requirements:

- ▶ `.rn`, `.rz`, `.rm`, `.rp` for `mad.f64`, requires `sm_13` or higher.
- ▶ `.rn`, `.rz`, `.rm`, `.rp` for `mad.f32`, requires `sm_20` or higher.

Examples

```
@p mad.f32 d,a,b,c;
```

9.7.3.8 Floating Point Instructions: div

div

Divide one value by another.

Syntax

```
div.approx{.ftz}.f32 d, a, b; // fast, approximate divide
div.full{.ftz}.f32   d, a, b; // full-range approximate divide
div.rnd{.ftz}.f32    d, a, b; // IEEE 754 compliant rounding
div.rnd.f64          d, a, b; // IEEE 754 compliant rounding

.rnd = { .rn, .rz, .rm, .rp };
```

Description

Divides a by b, stores result in d.

Semantics

```
d = a / b;
```

Notes

Fast, approximate single-precision divides:

- ▶ `div.approx.f32` implements a fast approximation to divide, computed as $d = a * (1/b)$. For $|b|$ in $[2^{-126}, 2^{126}]$, the maximum ulp error is 2. For $2^{126} < |b| < 2^{128}$, if a is infinity, `div.approx.f32` returns NaN, otherwise it returns a sign-preserving zero.
- ▶ `div.full.f32` implements a relatively fast, full-range approximation that scales operands to achieve better accuracy, but is not fully IEEE 754 compliant and does not support rounding modifiers. The maximum ulp error is 2 across the full range of inputs.

Divide with IEEE 754 compliant rounding:

Rounding modifiers (no default):

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`div.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`div.f64` supports subnormal numbers.

`div.f32` flushes subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

`div.f32` and `div.f64` introduced in PTX ISA version 1.0.

Explicit modifiers `.approx`, `.full`, `.ftz`, and rounding introduced in PTX ISA version 1.4.

For PTX ISA version 1.4 and later, one of `.approx`, `.full`, or `.rnd` is required.

For PTX ISA versions 1.0 through 1.3, `div.f32` defaults to `div.approx.ftz.f32`, and `div.f64` defaults to `div.rn.f64`.

Target ISA Notes

`div.approx.f32` and `div.full.f32` supported on all target architectures.

`div.rnd.f32` requires `sm_20` or higher.

`div.rn.f64` requires `sm_13` or higher, or `.target map_f64_to_f32`.

`div.{rz, rm, rp}.f64` requires `sm_20` or higher.

Examples

```
div.approx.ftz.f32  diam, circum, 3.14159;
div.full.ftz.f32    x, y, z;
div.rn.f64          xd, yd, zd;
```

9.7.3.9 Floating Point Instructions: abs

abs

Absolute value.

Syntax

```
abs{.ftz}.f32  d, a;  
abs.f64       d, a;
```

Description

Take the absolute value of a and store the result in d.

Semantics

```
d = |a|;
```

Notes

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

abs.ftz.f32 flushes subnormal inputs and results to sign-preserving zero.

sm_1x

abs.f64 supports subnormal numbers.

abs.f32 flushes subnormal inputs and results to sign-preserving zero.

For abs.f32, NaN input yields unspecified NaN. For abs.f64, NaN input is passed through unchanged. Future implementations may comply with the IEEE 754 standard by preserving payload and modifying only the sign bit.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

`abs.f32` supported on all target architectures.

`abs.f64` requires `sm_13` or higher.

Examples

```
abs.ftz.f32  x, f0;
```

9.7.3.10 Floating Point Instructions: `neg`

`neg`

Arithmetic negate.

Syntax

```
neg{.ftz}.f32  d, a;
neg.f64        d, a;
```

Description

Negate the sign of `a` and store the result in `d`.

Semantics

```
d = -a;
```

Notes

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`neg.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`neg.f64` supports subnormal numbers.

`neg.f32` flushes subnormal inputs and results to sign-preserving zero.

NaN inputs yield an unspecified NaN. Future implementations may comply with the IEEE 754 standard by preserving payload and modifying only the sign bit.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

`neg.f32` supported on all target architectures.

`neg.f64` requires `sm_13` or higher.

Examples

```
neg.ftz.f32  x, f0;
```

9.7.3.11 Floating Point Instructions: `min`

`min`

Find the minimum of given values.

Syntax

```
min{.ftz}{.NaN}{.xorsign.abs}.f32  d, a, b;  
min{.ftz}{.NaN}{.abs}.f32          d, a, b, c;  
min.f64                             d, a, b;
```

Description

Store the minimum of `a`, `b` and optionally `c` in `d`.

If `.NaN` modifier is specified, then the result is canonical `NaN` if any of the inputs is `NaN`.

If `.abs` modifier is specified, the magnitude of destination operand `d` is the minimum of absolute values of both input arguments.

If `.xorsign` modifier is specified, the sign bit of destination `d` is equal to the XOR of the sign bits of both inputs `a` and `b`. The `.xorsign` qualifier cannot be specified for three inputs operation.

Qualifier `.xorsign` requires qualifier `.abs` to be specified. In such cases, `.xorsign` considers the sign bit of both inputs before applying `.abs` operation.

If the result of `min` is `NaN` then the `.xorsign` and `.abs` modifiers will be ignored.

Semantics

```

def min_num (z, x, y) {
    if (isNaN(x) && isNaN(y))
        z = NaN;
    else if (isNaN(x))
        z = y;
    else if (isNaN(y))
        z = x;
    else
        // note: -0.0 < +0.0 here
        z = (x < y) ? x : y;
    return z;
}

def min_nan (z, x, y) {
    if (isNaN(x) || isNaN(y))
        z = NaN;
    else
        // note: -0.0 < +0.0 here
        z = (x < y) ? x : y;
    return z;
}

def two_inputs_min (z, x, y) {
    if (.NaN)
        z = min_nan(z, x, y);
    else
        z = min_num(z, x, y);
    return z;
}

if (.xorsign && !isPresent(c)) {
    xorsign = getSignBit(a) ^ getSignBit(b);
}

if (.abs) {
    a = |a|;
    b = |b|;
    if (isPresent(c)) {
        c = |c|;
    }
}

d = two_inputs_min(d, a, b)
if (isPresent(c)) {
    d = two_inputs_min(d, d, c)
}
if (.xorsign && !isPresent(c) && !isNaN(d)) {
    setSignBit(d, xorsign);
}

```

Notes

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`min.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`min.f64` supports subnormal numbers.

`min.f32` flushes subnormal inputs and results to sign-preserving zero.

If values of both inputs are 0.0, then $+0.0 > -0.0$.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`min.NaN` introduced in PTX ISA version 7.0.

`min.xorsign.abs` introduced in PTX ISA version 7.2.

`min` with three input arguments introduced in PTX ISA version 8.8.

Target ISA Notes

`min.f32` supported on all target architectures.

`min.f64` requires `sm_13` or higher.

`min.NaN` requires `sm_80` or higher.

`min.xorsign.abs` requires `sm_86` or higher.

`min` with three input arguments requires `sm_100` or higher.

Examples

```
@p min.ftz.f32  z,z,x;
    min.f64     a,b,c;
    // fp32 min with .NaN
    min.NaN.f32  f0,f1,f2;
    // fp32 min with .xorsign.abs
    min.xorsign.abs.f32 Rd, Ra, Rb;
```


9.7.3.12 Floating Point Instructions: max

max

Find the maximum of given values.

Syntax

```
max{.ftz}{.NaN}{.xorsign.abs}.f32  d, a, b;
max{.ftz}{.NaN}{.abs}.f32         d, a, b, c;
max.f64                           d, a, b;
```

Description

Store the maximum of a, b and optionally c in d.

If `.NaN` modifier is specified, the result is canonical NaN if any of the inputs is NaN.

If `.abs` modifier is specified, the magnitude of destination operand d is the maximum of absolute values of the input arguments.

If `.xorsign` modifier is specified, the sign bit of destination d is equal to the XOR of the sign bits of the inputs: a and b. The `.xorsign` qualifier cannot be specified for three inputs operation.

Qualifier `.xorsign` requires qualifier `.abs` to be specified. In such cases, `.xorsign` considers the sign bit of both inputs before applying `.abs` operation.

If the result of max is NaN then the `.xorsign` and `.abs` modifiers will be ignored.

Semantics

```
def max_num (z, x, y) {
    if (isNaN(x) && isNaN(y))
        z = NaN;
    else if (isNaN(x))
        z = y;
    else if (isNaN(y))
        z = x;
    else
        // note: +0.0 > -0.0 here
        z = (x > y) ? x : y;
    return z;
}

def max_nan (z, x, y) {
    if (isNaN(x) || isNaN(y))
        z = NaN;
    else
        // note: +0.0 > -0.0 here
        z = (x > y) ? x : y;
    return z;
}
```

(continues on next page)

(continued from previous page)

```
def two_inputs_max (z, x, y) {
    if (.NaN)
        z = max_nan(z, x, y);
    else
        z = max_num(z, x, y);
    return z;
}

if (.xorsign && !isPresent(c)) {
    xorsign = getSignBit(a) ^ getSignBit(b);
}
if (.abs) {
    a = |a|;
    b = |b|;
    if (isPresent(c)) {
        c = |c|;
    }
}

d = two_inputs_max (d, a, b)
if (isPresent(c)) {
    d = two_inputs_max (d, d, c)
}

if (.xorsign && !isPresent(c) !isNaN(d)) {
    setSignBit(d, xorsign);
}
```

Notes

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`max.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`max.f64` supports subnormal numbers.

`max.f32` flushes subnormal inputs and results to sign-preserving zero.

If values of both inputs are 0.0, then `+0.0 > -0.0`.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`max.NaN` introduced in PTX ISA version 7.0.

`max.xorsign.abs` introduced in PTX ISA version 7.2.

`max` with three input arguments introduced in PTX ISA version 8.8.

Target ISA Notes

`max.ftz.f32` supported on all target architectures.

`max.f64` requires `sm_13` or higher.

`max.NaN.f32` requires `sm_80` or higher.

`max.xorsign.abs` requires `sm_86` or higher.

`max` with three input arguments requires `sm_100` or higher.

Examples

```
max.ftz.f32  f0, f1, f2;
max.f64      a, b, c;
// fp32 max with .NaN
max.NaN.f32  f0, f1, f2;
// fp32 max with .xorsign.abs
max.xorsign.abs.f32 Rd, Ra, Rb;
```

9.7.3.13 Floating Point Instructions: `rcp`

`rcp`

Take the reciprocal of a value.

Syntax

```
rcp.approx{.ftz}.f32  d, a; // fast, approximate reciprocal
rcp.rnd{.ftz}.f32     d, a; // IEEE 754 compliant rounding
rcp.rnd.f64           d, a; // IEEE 754 compliant rounding

.rnd = { .rn, .rz, .rm, .rp };
```

Description

Compute $1/a$, store result in `d`.

Semantics

```
d = 1 / a;
```

Notes

Fast, approximate single-precision reciprocal:

`rcp.approx.f32` implements a fast approximation to reciprocal. The maximum ulp error is 1 across the full range of inputs.

Input	Result
-Inf	-0.0
-0.0	-Inf
+0.0	+Inf
+Inf	+0.0
NaN	NaN

Reciprocal with IEEE 754 compliant rounding:

Rounding modifiers (no default):

- `.rn` mantissa LSB rounds to nearest even
- `.rz` mantissa LSB rounds towards zero
- `.rm` mantissa LSB rounds towards negative infinity
- `.rp` mantissa LSB rounds towards positive infinity

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`rcp.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`rcp.f64` supports subnormal numbers.

`rcp.f32` flushes subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

`rcp.f32` and `rcp.f64` introduced in PTX ISA version 1.0. `rcp.rn.f64` and explicit modifiers `.approx` and `.ftz` were introduced in PTX ISA version 1.4. General rounding modifiers were added in PTX ISA version 2.0.

For PTX ISA version 1.4 and later, one of `.approx` or `.rnd` is required.

For PTX ISA versions 1.0 through 1.3, `rcp.f32` defaults to `rcp.approx.ftz.f32`, and `rcp.f64` defaults to `rcp.rn.f64`.

Target ISA Notes

`rcp.approx.ftz.f32` supported on all target architectures.

`rcp.rnd.f32` requires `sm_20` or higher.

`rcp.rn.f64` requires `sm_13` or higher, or `.target map_f64_to_f32`.

`rcp.{rz,rm,rp}.f64` requires `sm_20` or higher.

Examples

```
rcp.approx.ftz.f32  ri,r;
rcp.rn.ftz.f32     xi,x;
rcp.rn.f64         xi,x;
```

9.7.3.14 Floating Point Instructions: `rcp.approx.ftz.f64`

`rcp.approx.ftz.f64`

Compute a fast, gross approximation to the reciprocal of a value.

Syntax

```
rcp.approx.ftz.f64  d, a;
```

Description

Compute a fast, gross approximation to the reciprocal as follows:

1. extract the most-significant 32 bits of `.f64` operand `a` in 1.11.20 IEEE floating-point format (i.e., ignore the least-significant 32 bits of `a`),
2. compute an approximate `.f64` reciprocal of this value using the most-significant 20 bits of the mantissa of operand `a`,
3. place the resulting 32-bits in 1.11.20 IEEE floating-point format in the most-significant 32-bits of destination `d`, and
4. zero the least significant 32 mantissa bits of `.f64` destination `d`.

Semantics

```
tmp = a[63:32]; // upper word of a, 1.11.20 format
d[63:32] = 1.0 / tmp;
d[31:0] = 0x00000000;
```

Notes

`rcp.approx.ftz.f64` implements a fast, gross approximation to reciprocal.

Input a[63:32]	Result d[63:32]
-Inf	-0.0
-subnormal	-Inf
-0.0	-Inf
+0.0	+Inf
+subnormal	+Inf
+Inf	+0.0
NaN	NaN

Input NaNs map to a canonical NaN with encoding `0x7fffffff00000000`.

Subnormal inputs and results are flushed to sign-preserving zero.

PTX ISA Notes

`rcp.approx.ftz.f64` introduced in PTX ISA version 2.1.

Target ISA Notes

`rcp.approx.ftz.f64` requires `sm_20` or higher.

Examples

```
rcp.approx.ftz.f64  xi,x;
```

9.7.3.15 Floating Point Instructions: `sqr`

`sqr`

Take the square root of a value.

Syntax

```

sqrt.approx{.ftz}.f32  d, a; // fast, approximate square root
sqrt.rnd{.ftz}.f32    d, a; // IEEE 754 compliant rounding
sqrt.rnd.f64          d, a; // IEEE 754 compliant rounding

.rnd = { .rn, .rz, .rm, .rp };

```

Description

Compute $\text{sqrt}(a)$ and store the result in d .

Semantics

```
d = sqrt(a);
```

Notes

`sqrt.approx.f32` implements a fast approximation to square root. The maximum relative error over the entire positive finite floating-point range is 2^{-23} .

For various corner-case inputs, results of `sqrt` instruction are shown in below table:

Input	Result
-Inf	NaN
-normal	NaN
-0.0	-0.0
+0.0	+0.0
+Inf	+Inf
NaN	NaN

Square root with IEEE 754 compliant rounding:

Rounding modifiers (no default):

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`sqrt.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`sqrt.f64` supports subnormal numbers.

`sqrt.f32` flushes subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

`sqrt.f32` and `sqrt.f64` introduced in PTX ISA version 1.0. `sqrt.rn.f64` and explicit modifiers `.approx` and `.ftz` were introduced in PTX ISA version 1.4. General rounding modifiers were added in PTX ISA version 2.0.

For PTX ISA version 1.4 and later, one of `.approx` or `.rnd` is required.

For PTX ISA versions 1.0 through 1.3, `sqrt.f32` defaults to `sqrt.approx.ftz.f32`, and `sqrt.f64` defaults to `sqrt.rn.f64`.

Target ISA Notes

`sqrt.approx.f32` supported on all target architectures.

`sqrt.rnd.f32` requires `sm_20` or higher.

`sqrt.rn.f64` requires `sm_13` or higher, or `.target map_f64_to_f32`.

`sqrt.{rz, rm, rp}.f64` requires `sm_20` or higher.

Examples

```
sqrt.approx.ftz.f32  r,x;  
sqrt.rn.ftz.f32      r,x;  
sqrt.rn.f64          r,x;
```

9.7.3.16 Floating Point Instructions: rsqrt**rsqrt**

Take the reciprocal of the square root of a value.

Syntax

```
rsqrt.approx{.ftz}.f32  d, a;  
rsqrt.approx.f64        d, a;
```


Description

Compute $1/\text{sqrt}(a)$ and store the result in `d`.

Semantics

```
d = 1/sqrt(a);
```

Notes

`rsqrt.approx` implements an approximation to the reciprocal square root.

Input	Result
-Inf	NaN
-normal	NaN
-0.0	-Inf
+0.0	+Inf
+Inf	+0.0
NaN	NaN

The maximum relative error for `rsqrt.f32` over the entire positive finite floating-point range is $2^{-22.9}$.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`rsqrt.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

`rsqrt.f64` supports subnormal numbers.

`rsqrt.f32` flushes subnormal inputs and results to sign-preserving zero.

Note that `rsqrt.approx.f64` is emulated in software and are relatively slow.

PTX ISA Notes

`rsqrt.f32` and `rsqrt.f64` were introduced in PTX ISA version 1.0. Explicit modifiers `.approx` and `.ftz` were introduced in PTX ISA version 1.4.

For PTX ISA version 1.4 and later, the `.approx` modifier is required.

For PTX ISA versions 1.0 through 1.3, `rsqrt.f32` defaults to `rsqrt.approx.ftz.f32`, and `rsqrt.f64` defaults to `rsqrt.approx.f64`.

Target ISA Notes

`rsqrt.f32` supported on all target architectures.

`rsqrt.f64` requires `sm_13` or higher.

Examples

```
rsqrt.approx.ftz.f32  isr, x;  
rsqrt.approx.f64      ISR, X;
```

9.7.3.17 Floating Point Instructions: `rsqrt.approx.ftz.f64`

`rsqrt.approx.ftz.f64`

Compute an approximation of the square root reciprocal of a value.

Syntax

```
rsqrt.approx.ftz.f64 d, a;
```

Description

Compute a double-precision (`.f64`) approximation of the square root reciprocal of a value. The least significant 32 bits of the double-precision (`.f64`) destination `d` are all zeros.

Semantics

```
tmp = a[63:32]; // upper word of a, 1.11.20 format  
d[63:32] = 1.0 / sqrt(tmp);  
d[31:0] = 0x00000000;
```

Notes

`rsqrt.approx.ftz.f64` implements a fast approximation of the square root reciprocal of a value.

Input	Result
-Inf	NaN
-subnormal	-Inf
-0.0	-Inf
+0.0	+Inf
+subnormal	+Inf
+Inf	+0.0
NaN	NaN

Input NaNs map to a canonical NaN with encoding `0x7fffffff00000000`.

Subnormal inputs and results are flushed to sign-preserving zero.

PTX ISA Notes

`rsqrt.approx.ftz.f64` introduced in PTX ISA version 4.0.

Target ISA Notes

`rsqrt.approx.ftz.f64` requires `sm_20` or higher.

Examples

```
rsqrt.approx.ftz.f64 xi,x;
```

9.7.3.18 Floating Point Instructions: `sin`

`sin`

Find the sine of a value.

Syntax

```
sin.approx{.ftz}.f32 d, a;
```

Description

Find the sine of the angle a (in radians).

Semantics

```
d = sin(a);
```

Notes

`sin.approx.f32` implements a fast approximation to sine.

Input	Result
-Inf	NaN
-0.0	-0.0
+0.0	+0.0
+Inf	NaN
NaN	NaN

The maximum absolute error over input range is as follows:

Range	[-2pi .. 2pi]	[-100pi .. +100pi]
Error	$2^{-20.5}$	$2^{-14.7}$

Outside of the range [-100pi .. +100pi], only best effort is provided. There are no defined error guarantees.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`sin.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

Subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

`sin.f32` introduced in PTX ISA version 1.0. Explicit modifiers `.approx` and `.ftz` introduced in PTX ISA version 1.4.

For PTX ISA version 1.4 and later, the `.approx` modifier is required.

For PTX ISA versions 1.0 through 1.3, `sin.f32` defaults to `sin.approx.ftz.f32`.

Target ISA Notes

Supported on all target architectures.

Examples

```
sin.approx.ftz.f32  sa, a;
```

9.7.3.19 Floating Point Instructions: cos

cos

Find the cosine of a value.

Syntax

```
cos.approx{.ftz}.f32  d, a;
```

Description

Find the cosine of the angle a (in radians).

Semantics

```
d = cos(a);
```

Notes

`cos.approx.f32` implements a fast approximation to cosine.

Input	Result
-Inf	NaN
-0.0	+1.0
+0.0	+1.0
+Inf	NaN
NaN	NaN

The maximum absolute error over input range is as follows:

Range	[-2pi .. 2pi]	[-100pi .. +100pi]
Error	$2^{-20.5}$	$2^{-14.7}$

Outside of the range $[-100\pi .. +100\pi]$, only best effort is provided. There are no defined error guarantees.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`cos.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

Subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

`cos.f32` introduced in PTX ISA version 1.0. Explicit modifiers `.approx` and `.ftz` introduced in PTX ISA version 1.4.

For PTX ISA version 1.4 and later, the `.approx` modifier is required.

For PTX ISA versions 1.0 through 1.3, `cos.f32` defaults to `cos.approx.ftz.f32`.

Target ISA Notes

Supported on all target architectures.

Examples

```
cos.approx.ftz.f32  ca, a;
```

9.7.3.20 Floating Point Instructions: lg2**lg2**

Find the base-2 logarithm of a value.

Syntax

```
lg2.approx{.ftz}.f32  d, a;
```

Description

Determine the $\log_2$ of `a`.

Semantics

```
d = log(a) / log(2);
```

Notes

`lg2 . approx . f32` implements a fast approximation to $\log_2(a)$.

Input	Result
-Inf	NaN
-normal	NaN
-0.0	-Inf
+0.0	-Inf
+Inf	+Inf
NaN	NaN

The maximum absolute error is 2^{-22} when the input operand is in the range (0.5, 2). For positive finite inputs outside of this interval, maximum relative error is 2^{-22} .

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`lg2 . ftz . f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

Subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

`lg2 . f32` introduced in PTX ISA version 1.0. Explicit modifiers `. approx` and `. ftz` introduced in PTX ISA version 1.4.

For PTX ISA version 1.4 and later, the `. approx` modifier is required.

For PTX ISA versions 1.0 through 1.3, `lg2 . f32` defaults to `lg2 . approx . ftz . f32`.

Target ISA Notes

Supported on all target architectures.

Examples

```
lg2.approx.ftz.f32 1a, a;
```

9.7.3.21 Floating Point Instructions: ex2

ex2

Find the base-2 exponential of a value.

Syntax

```
ex2.approx{.ftz}.f32 d, a;
```

Description

Raise 2 to the power a.

Semantics

```
d = 2 ^ a;
```

Notes

`ex2.approx.f32` implements a fast approximation to 2^a .

Input	Result
-Inf	+0.0
-0.0	+1.0
+0.0	+1.0
+Inf	+Inf
NaN	NaN

The maximum ulp error is 2 ulp from correctly rounded result across the full range of inputs.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`ex2.ftz.f32` flushes subnormal inputs and results to sign-preserving zero.

sm_1x

Subnormal inputs and results to sign-preserving zero.

PTX ISA Notes

ex2.f32 introduced in PTX ISA version 1.0. Explicit modifiers .approx and .ftz introduced in PTX ISA version 1.4.

For PTX ISA version 1.4 and later, the .approx modifier is required.

For PTX ISA versions 1.0 through 1.3, ex2.f32 defaults to ex2.approx.ftz.f32.

Target ISA Notes

Supported on all target architectures.

Examples

```
ex2.approx.ftz.f32  xa, a;
```

9.7.3.22 Floating Point Instructions: tanh

tanh

Find the hyperbolic tangent of a value (in radians)

Syntax

```
tanh.approx.f32 d, a;
```

Description

Take hyperbolic tangent value of a.

The operands d and a are of type .f32.

Semantics

```
d = tanh(a);
```

Notes

tanh.approx.f32 implements a fast approximation to FP32 hyperbolic-tangent.

Results of tanh for various corner-case inputs are as follows:

Input	Result
-Inf	-1.0
-0.0	-0.0
+0.0	+0.0
+Inf	1.0
NaN	NaN

The maximum relative error over the entire floating point range is 2^{-11} . The subnormal numbers are supported.

Note: The subnormal inputs gets passed through to the output since the value of $\tanh(x)$ for small values of x is approximately the same as x .

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Target ISA Notes

Requires sm_75 or higher.

Examples

```
tanh.approx.f32 ta, a;
```

9.7.4. Half Precision Floating-Point Instructions

Half precision floating-point instructions operate on .f16 and .f16x2 register operands. The half precision floating-point instructions are:

- ▶ add
- ▶ sub
- ▶ mul
- ▶ fma
- ▶ neg
- ▶ abs
- ▶ min
- ▶ max
- ▶ tanh

► ex2

Half-precision add, sub, mul, and fma support saturation of results to the range [0.0, 1.0], with NaNs being flushed to positive zero. Half-precision instructions return an unspecified NaN.

9.7.4.1 Half Precision Floating Point Instructions: add

add

Add two values.

Syntax

```
add{.rnd}{.ftz}{.sat}.f16    d, a, b;
add{.rnd}{.ftz}{.sat}.f16x2 d, a, b;

add{.rnd}.bf16    d, a, b;
add{.rnd}.bf16x2 d, a, b;

.rnd = { .rn };
```

Description

Performs addition and writes the resulting value into a destination register.

For `.f16x2` and `.bf16x2` instruction type, forms input vectors by half word values from source operands. Half-word operands are then added in parallel to produce `.f16x2` or `.bf16x2` result in destination.

For `.f16` instruction type, operands `d`, `a` and `b` have `.f16` or `.b16` type. For `.f16x2` instruction type, operands `d`, `a` and `b` have `.b32` type. For `.bf16` instruction type, operands `d`, `a`, `b` have `.b16` type. For `.bf16x2` instruction type, operands `d`, `a`, `b` have `.b32` type.

Semantics

```
if (type == f16 || type == bf16) {
    d = a + b;
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = fA[i] + fB[i];
    }
}
```

Notes

Rounding modifiers:

.rn mantissa LSB rounds to nearest even

The default value of rounding modifier is **.rn**. Note that an add instruction with an explicit rounding modifier is treated conservatively by the code optimizer. An add instruction with no rounding modifier defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, mul/add sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

By default, subnormal numbers are supported. `add.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`add.sat.{f16, f16x2}` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`.

PTX ISA Notes

Introduced in PTX ISA version 4.2.

`add{.rnd}.bf16` and `add{.rnd}.bf16x2` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_53 or higher.

`add{.rnd}.bf16` and `add{.rnd}.bf16x2` requires sm_90 or higher.

Examples

```
// scalar f16 additions
add.f16      d0, a0, b0;
add.rn.f16   d1, a1, b1;
add.bf16     bd0, ba0, bb0;
add.rn.bf16  bd1, ba1, bb1;

// SIMD f16 addition
cvt.rn.f16.f32 h0, f0;
cvt.rn.f16.f32 h1, f1;
cvt.rn.f16.f32 h2, f2;
cvt.rn.f16.f32 h3, f3;
mov.b32  p1, {h0, h1};    // pack two f16 to 32bit f16x2
mov.b32  p2, {h2, h3};    // pack two f16 to 32bit f16x2
add.f16x2 p3, p1, p2;     // SIMD f16x2 addition

// SIMD bf16 addition
cvt.rn.bf16x2.f32 p4, f4, f5; // Convert two f32 into packed bf16x2
cvt.rn.bf16x2.f32 p5, f6, f7; // Convert two f32 into packed bf16x2
add.bf16x2  p6, p4, p5;      // SIMD bf16x2 addition

// SIMD fp16 addition
```

(continues on next page)

(continued from previous page)

```
ld.global.b32    f0, [addr];      // load 32 bit which hold packed f16x2
ld.global.b32    f1, [addr + 4];  // load 32 bit which hold packed f16x2
add.f16x2        f2, f0, f1;      // SIMD f16x2 addition

ld.global.b32    f3, [addr + 8];  // load 32 bit which hold packed bf16x2
ld.global.b32    f4, [addr + 12]; // load 32 bit which hold packed bf16x2
add.bf16x2       f5, f3, f4;      // SIMD bf16x2 addition
```

9.7.4.2 Half Precision Floating Point Instructions: sub

sub

Subtract two values.

Syntax

```
sub{.rnd}{.ftz}{.sat}.f16    d, a, b;
sub{.rnd}{.ftz}{.sat}.f16x2 d, a, b;

sub{.rnd}.bf16    d, a, b;
sub{.rnd}.bf16x2 d, a, b;

.rnd = { .rn };
```

Description

Performs subtraction and writes the resulting value into a destination register.

For `.f16x2` and `.bf16x2` instruction type, forms input vectors by half word values from source operands. Half-word operands are then subtracted in parallel to produce `.f16x2` or `.bf16x2` result in destination.

For `.f16` instruction type, operands `d`, `a` and `b` have `.f16` or `.b16` type. For `.f16x2` instruction type, operands `d`, `a` and `b` have `.b32` type. For `.bf16` instruction type, operands `d`, `a`, `b` have `.b16` type. For `.bf16x2` instruction type, operands `d`, `a`, `b` have `.b32` type.

Semantics

```
if (type == f16 || type == bf16) {
    d = a - b;
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = fA[i] - fB[i];
    }
}
```

Notes

Rounding modifiers:

.rn mantissa LSB rounds to nearest even

The default value of rounding modifier is **.rn**. Note that a **sub** instruction with an explicit rounding modifier is treated conservatively by the code optimizer. A **sub** instruction with no rounding modifier defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, **mul/sub** sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

By default, subnormal numbers are supported. **sub.ftz.{f16, f16x2}** flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

sub.sat.{f16, f16x2} clamps the result to [0.0, 1.0]. NaN results are flushed to +0.0f.

PTX ISA Notes

Introduced in PTX ISA version 4.2.

sub{.rnd}.bf16 and **sub{.rnd}.bf16x2** introduced in PTX ISA version 7.8.

Target ISA Notes

Requires **sm_53** or higher.

sub{.rnd}.bf16 and **sub{.rnd}.bf16x2** requires **sm_90** or higher.

Examples

```
// scalar f16 subtractions
sub.f16      d0, a0, b0;
sub.rn.f16   d1, a1, b1;
sub.bf16     bd0, ba0, bb0;
sub.rn.bf16  bd1, ba1, bb1;

// SIMD f16 subtraction
cvt.rn.f16.f32 h0, f0;
cvt.rn.f16.f32 h1, f1;
cvt.rn.f16.f32 h2, f2;
cvt.rn.f16.f32 h3, f3;
mov.b32  p1, {h0, h1};    // pack two f16 to 32bit f16x2
mov.b32  p2, {h2, h3};    // pack two f16 to 32bit f16x2
sub.f16x2 p3, p1, p2;     // SIMD f16x2 subtraction

// SIMD bf16 subtraction
cvt.rn.bf16x2.f32 p4, f4, f5; // Convert two f32 into packed bf16x2
cvt.rn.bf16x2.f32 p5, f6, f7; // Convert two f32 into packed bf16x2
sub.bf16x2  p6, p4, p5;      // SIMD bf16x2 subtraction

// SIMD fp16 subtraction
```

(continues on next page)

(continued from previous page)

```
ld.global.b32    f0, [addr];      // load 32 bit which hold packed f16x2
ld.global.b32    f1, [addr + 4]; // load 32 bit which hold packed f16x2
sub.f16x2        f2, f0, f1;      // SIMD f16x2 subtraction

// SIMD bf16 subtraction
ld.global.b32    f3, [addr + 8]; // load 32 bit which hold packed bf16x2
ld.global.b32    f4, [addr + 12]; // load 32 bit which hold packed bf16x2
sub.bf16x2       f5, f3, f4;      // SIMD bf16x2 subtraction
```

9.7.4.3 Half Precision Floating Point Instructions: mul

mul

Multiply two values.

Syntax

```
mul{.rnd}{.ftz}{.sat}.f16    d, a, b;
mul{.rnd}{.ftz}{.sat}.f16x2 d, a, b;

mul{.rnd}.bf16    d, a, b;
mul{.rnd}.bf16x2 d, a, b;

.rnd = { .rn };
```

Description

Performs multiplication and writes the resulting value into a destination register.

For `.f16x2` and `.bf16x2` instruction type, forms input vectors by half word values from source operands. Half-word operands are then multiplied in parallel to produce `.f16x2` or `.bf16x2` result in destination.

For `.f16` instruction type, operands `d`, `a` and `b` have `.f16` or `.b16` type. For `.f16x2` instruction type, operands `d`, `a` and `b` have `.b32` type. For `.bf16` instruction type, operands `d`, `a`, `b` have `.b16` type. For `.bf16x2` instruction type, operands `d`, `a`, `b` have `.b32` type.

Semantics

```
if (type == f16 || type == bf16) {
    d = a * b;
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = fA[i] * fB[i];
    }
}
```

Notes

Rounding modifiers:

.rn mantissa LSB rounds to nearest even

The default value of rounding modifier is **.rn**. Note that a `mul` instruction with an explicit rounding modifier is treated conservatively by the code optimizer. A `mul` instruction with no rounding modifier defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, `mul/add` and `mul/sub` sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

By default, subnormal numbers are supported. `mul.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`mul.sat.{f16, f16x2}` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`.

PTX ISA Notes

Introduced in PTX ISA version 4.2.

`mul{.rnd}.bf16` and `mul{.rnd}.bf16x2` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_53` or higher.

`mul{.rnd}.bf16` and `mul{.rnd}.bf16x2` requires `sm_90` or higher.

Examples

```
// scalar f16 multiplications
mul.f16      d0, a0, b0;
mul.rn.f16   d1, a1, b1;
mul.bf16     bd0, ba0, bb0;
mul.rn.bf16  bd1, ba1, bb1;

// SIMD f16 multiplication
cvt.rn.f16.f32 h0, f0;
cvt.rn.f16.f32 h1, f1;
cvt.rn.f16.f32 h2, f2;
cvt.rn.f16.f32 h3, f3;
mov.b32 p1, {h0, h1};    // pack two f16 to 32bit f16x2
mov.b32 p2, {h2, h3};    // pack two f16 to 32bit f16x2
mul.f16x2 p3, p1, p2;    // SIMD f16x2 multiplication

// SIMD bf16 multiplication
cvt.rn.bf16x2.f32 p4, f4, f5; // Convert two f32 into packed bf16x2
cvt.rn.bf16x2.f32 p5, f6, f7; // Convert two f32 into packed bf16x2
mul.bf16x2 p6, p4, p5;      // SIMD bf16x2 multiplication

// SIMD fp16 multiplication
```

(continues on next page)

(continued from previous page)

```
ld.global.b32    f0, [addr];    // load 32 bit which hold packed f16x2
ld.global.b32    f1, [addr + 4]; // load 32 bit which hold packed f16x2
mul.f16x2        f2, f0, f1;    // SIMD f16x2 multiplication

// SIMD bf16 multiplication
ld.global.b32    f3, [addr + 8]; // load 32 bit which hold packed bf16x2
ld.global.b32    f4, [addr + 12]; // load 32 bit which hold packed bf16x2
mul.bf16x2       f5, f3, f4;    // SIMD bf16x2 multiplication
```

9.7.4.4 Half Precision Floating Point Instructions: fma

fma

Fused multiply-add

Syntax

```
fma.rnd{.ftz}{.sat}.f16      d, a, b, c;
fma.rnd{.ftz}{.sat}.f16x2    d, a, b, c;
fma.rnd{.ftz}.relu.f16      d, a, b, c;
fma.rnd{.ftz}.relu.f16x2    d, a, b, c;
fma.rnd{.relu}.bf16         d, a, b, c;
fma.rnd{.relu}.bf16x2       d, a, b, c;
fma.rnd.oob.{relu}.type     d, a, b, c;

.rnd = { .rn };
```

Description

Performs a fused multiply-add with no loss of precision in the intermediate product and addition.

For .f16x2 and .bf16x2 instruction type, forms input vectors by half word values from source operands. Half-word operands are then operated in parallel to produce .f16x2 or .bf16x2 result in destination.

For .f16 instruction type, operands d, a, b and c have .f16 or .b16 type. For .f16x2 instruction type, operands d, a, b and c have .b32 type. For .bf16 instruction type, operands d, a, b and c have .b16 type. For .bf16x2 instruction type, operands d, a, b and c have .b32 type.

Semantics

```
if (type == f16 || type == bf16) {
    d = a * b + c;
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    fC[0] = c[0:15];
```

(continues on next page)

(continued from previous page)

```

    fC[1] = c[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = fA[i] * fB[i] + fC[i];
    }
}

```

Notes

Rounding modifiers (default is `.rn`):

`.rn` mantissa LSB rounds to nearest even

Subnormal numbers:

By default, subnormal numbers are supported. `fma.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

Saturation modifier:

`fma.sat.{f16, f16x2}` clamps the result to [0.0, 1.0]. NaN results are flushed to `+0.0f`. `fma.relu.{f16, f16x2, bf16, bf16x2}` clamps the result to 0 if negative. NaN result is converted to canonical NaN.

Out Of Bounds modifier:

`fma.oob.{f16, f16x2, bf16, bf16x2}` clamps the result to 0 if either of the operands is 00B NaN (defined under *Tensors*) value. The test for the special NaN value and resultant forcing of the result to +0.0 is performed independently for each of the two SIMD operations.

PTX ISA Notes

Introduced in PTX ISA version 4.2.

`fma.relu.{f16, f16x2}` and `fma{.relu}.{bf16, bf16x2}` introduced in PTX ISA version 7.0.

Support for modifier `.oob` introduced in PTX ISA version 8.1.

Target ISA Notes

Requires `sm_53` or higher.

`fma.relu.{f16, f16x2}` and `fma{.relu}.{bf16, bf16x2}` require `sm_80` or higher.

`fma{.oob}.{f16, f16x2, bf16, bf16x2}` requires `sm_90` or higher.

Examples

```

// scalar f16 fused multiply-add
fma.rn.f16      d0, a0, b0, c0;
fma.rn.f16      d1, a1, b1, c1;
fma.rn.relu.f16  d1, a1, b1, c1;
fma.rn.oob.f16   d1, a1, b1, c1;
fma.rn.oob.relu.f16 d1, a1, b1, c1;

// scalar bf16 fused multiply-add

```

(continues on next page)

(continued from previous page)

```

fma.rn.bf16      d1, a1, b1, c1;
fma.rn.relu.bf16 d1, a1, b1, c1;
fma.rn.oob.bf16   d1, a1, b1, c1;
fma.rn.oob.relu.bf16 d1, a1, b1, c1;

// SIMD f16 fused multiply-add
cvt.rn.f16.f32 h0, f0;
cvt.rn.f16.f32 h1, f1;
cvt.rn.f16.f32 h2, f2;
cvt.rn.f16.f32 h3, f3;
mov.b32 p1, {h0, h1}; // pack two f16 to 32bit f16x2
mov.b32 p2, {h2, h3}; // pack two f16 to 32bit f16x2
fma.rn.f16x2 p3, p1, p2, p2; // SIMD f16x2 fused multiply-add
fma.rn.relu.f16x2 p3, p1, p2, p2; // SIMD f16x2 fused multiply-add with relu
↪saturation mode
fma.rn.oob.f16x2 p3, p1, p2, p2; // SIMD f16x2 fused multiply-add with oob modifier
fma.rn.oob.relu.f16x2 p3, p1, p2, p2; // SIMD f16x2 fused multiply-add with oob
↪modifier and relu saturation mode

// SIMD fp16 fused multiply-add
ld.global.b32 f0, [addr]; // load 32 bit which hold packed f16x2
ld.global.b32 f1, [addr + 4]; // load 32 bit which hold packed f16x2
fma.rn.f16x2 f2, f0, f1, f1; // SIMD f16x2 fused multiply-add

// SIMD bf16 fused multiply-add
fma.rn.bf16x2 f2, f0, f1, f1; // SIMD bf16x2 fused multiply-add
fma.rn.relu.bf16x2 f2, f0, f1, f1; // SIMD bf16x2 fused multiply-add with relu
↪saturation mode
fma.rn.oob.bf16x2 f2, f0, f1, f1; // SIMD bf16x2 fused multiply-add with oob modifier
fma.rn.oob.relu.bf16x2 f2, f0, f1, f1; // SIMD bf16x2 fused multiply-add with oob
↪modifier and relu saturation mode

```

9.7.4.5 Half Precision Floating Point Instructions: neg

neg

Arithmetic negate.

Syntax

```
neg{.ftz}.f16      d, a;  
neg{.ftz}.f16x2    d, a;  
neg.bf16           d, a;  
neg.bf16x2         d, a;
```

Description

Negate the sign of *a* and store the result in *d*.

For `.f16x2` and `.bf16x2` instruction type, forms input vector by extracting half word values from the source operand. Half-word operands are then negated in parallel to produce `.f16x2` or `.bf16x2` result in destination.

For `.f16` instruction type, operands *d* and *a* have `.f16` or `.b16` type. For `.f16x2` instruction type, operands *d* and *a* have `.b32` type. For `.bf16` instruction type, operands *d* and *a* have `.b16` type. For `.bf16x2` instruction type, operands *d* and *a* have `.b32` type.

Semantics

```
if (type == f16 || type == bf16) {  
    d = -a;  
} else if (type == f16x2 || type == bf16x2) {  
    fA[0] = a[0:15];  
    fA[1] = a[16:31];  
    for (i = 0; i < 2; i++) {  
        d[i] = -fA[i];  
    }  
}
```

Notes

Subnormal numbers:

By default, subnormal numbers are supported. `neg.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

NaN inputs yield an unspecified NaN. Future implementations may comply with the IEEE 754 standard by preserving payload and modifying only the sign bit.

PTX ISA Notes

Introduced in PTX ISA version 6.0.

`neg.bf16` and `neg.bf16x2` introduced in PTX ISA 7.0.

Target ISA Notes

Requires sm_53 or higher.

neg.bf16 and neg.bf16x2 requires architecture sm_80 or higher.

Examples

```
neg.ftz.f16    x, f0;
neg.bf16       x, b0;
neg.bf16x2     x1, b1;
```

9.7.4.6 Half Precision Floating Point Instructions: abs

abs

Absolute value

Syntax

```
abs{.ftz}.f16    d, a;
abs{.ftz}.f16x2  d, a;
abs.bf16         d, a;
abs.bf16x2       d, a;
```

Description

Take absolute value of a and store the result in d.

For .f16x2 and .bf16x2 instruction type, forms input vector by extracting half word values from the source operand. Absolute values of half-word operands are then computed in parallel to produce .f16x2 or .bf16x2 result in destination.

For .f16 instruction type, operands d and a have .f16 or .b16 type. For .f16x2 instruction type, operands d and a have .f16x2 or .b32 type. For .bf16 instruction type, operands d and a have .b16 type. For .bf16x2 instruction type, operands d and a have .b32 type.

Semantics

```
if (type == f16 || type == bf16) {
    d = |a|;
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    for (i = 0; i < 2; i++) {
        d[i] = |fA[i]|;
    }
}
```

Notes

Subnormal numbers:

By default, subnormal numbers are supported. `abs.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

NaN inputs yield an unspecified NaN. Future implementations may comply with the IEEE 754 standard by preserving payload and modifying only the sign bit.

PTX ISA Notes

Introduced in PTX ISA version 6.5.

`abs.bf16` and `abs.bf16x2` introduced in PTX ISA 7.0.

Target ISA Notes

Requires `sm_53` or higher.

`abs.bf16` and `abs.bf16x2` requires architecture `sm_80` or higher.

Examples

```
abs.ftz.f16    x, f0;  
abs.bf16       x, b0;  
abs.bf16x2     x1, b1;
```

9.7.4.7 Half Precision Floating Point Instructions: `min`

`min`

Find the minimum of two values.

Syntax

```
min{.ftz}{.NaN}{.xorsign.abs}.f16      d, a, b;  
min{.ftz}{.NaN}{.xorsign.abs}.f16x2    d, a, b;  
min{.NaN}{.xorsign.abs}.bf16           d, a, b;  
min{.NaN}{.xorsign.abs}.bf16x2         d, a, b;
```

Description

Store the minimum of *a* and *b* in *d*.

For `.f16x2` and `.bf16x2` instruction types, input vectors are formed with half-word values from source operands. Half-word operands are then processed in parallel to store `.f16x2` or `.bf16x2` result in destination.

For `.f16` instruction type, operands *d* and *a* have `.f16` or `.b16` type. For `.f16x2` instruction type, operands *d* and *a* have `.f16x2` or `.b32` type. For `.bf16` instruction type, operands *d* and *a* have `.b16` type. For `.bf16x2` instruction type, operands *d* and *a* have `.b32` type.

If `.NaN` modifier is specified, then the result is canonical NaN if either of the inputs is NaN.

If `.abs` modifier is specified, the magnitude of destination operand *d* is the minimum of absolute values of both the input arguments.

If `.xorsign` modifier is specified, the sign bit of destination *d* is equal to the XOR of the sign bits of both the inputs.

Modifiers `.abs` and `.xorsign` must be specified together and `.xorsign` considers the sign bit of both inputs before applying `.abs` operation.

If the result of `min` is NaN then the `.xorsign` and `.abs` modifiers will be ignored.

Semantics

```

if (type == f16 || type == bf16) {
    if (.xorsign) {
        xorsign = getSignBit(a) ^ getSignBit(b);
        if (.abs) {
            a = |a|;
            b = |b|;
        }
    }
    if (isNaN(a) && isNaN(b))           d = NaN;
    if (.NaN && (isNaN(a) || isNaN(b))) d = NaN;
    else if (isNaN(a))                 d = b;
    else if (isNaN(b))                 d = a;
    else                               d = (a < b) ? a : b;
    if (.xorsign && !isNaN(d)) {
        setSignBit(d, xorsign);
    }
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        if (.xorsign) {
            xorsign = getSignBit(fA[i]) ^ getSignBit(fB[i]);
            if (.abs) {
                fA[i] = |fA[i]|;
                fB[i] = |fB[i]|;
            }
        }
        if (isNaN(fA[i]) && isNaN(fB[i])) d[i] = NaN;
        if (.NaN && (isNaN(fA[i]) || isNaN(fB[i]))) d[i] = NaN;
    }
}

```

(continues on next page)

(continued from previous page)

```

        else if (isNaN(fA[i]))      d[i] = fB[i];
        else if (isNaN(fB[i]))      d[i] = fA[i];
        else                        d[i] = (fA[i] < fB[i]) ? fA[i]
↪: fB[i];
        if (.xorsign && !isNaN(d[i])) {
            setSignBit(d[i], xorsign);
        }
    }
}

```

Notes

Subnormal numbers:

By default, subnormal numbers are supported. `min.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

If values of both inputs are 0.0, then $+0.0 > -0.0$.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

`min.xorsign` introduced in PTX ISA version 7.2.

Target ISA Notes

Requires `sm_80` or higher.

`min.xorsign.abs` support requires `sm_86` or higher.

Examples

```

min.ftz.f16      h0,h1,h2;
min.f16x2        b0,b1,b2;
// SIMD fp16 min with .NaN
min.NaN.f16x2    b0,b1,b2;
min.bf16         h0, h1, h2;
// SIMD bf16 min with NaN
min.NaN.bf16x2   b0, b1, b2;
// scalar bf16 min with xorsign.abs
min.xorsign.abs.bf16 Rd, Ra, Rb

```


9.7.4.8 Half Precision Floating Point Instructions: max

max

Find the maximum of two values.

Syntax

```
max{.ftz}{.NaN}{.xorsign.abs}.f16      d, a, b;
max{.ftz}{.NaN}{.xorsign.abs}.f16x2    d, a, b;
max{.NaN}{.xorsign.abs}.bf16           d, a, b;
max{.NaN}{.xorsign.abs}.bf16x2         d, a, b;
```

Description

Store the maximum of a and b in d.

For `.f16x2` and `.bf16x2` instruction types, input vectors are formed with half-word values from source operands. Half-word operands are then processed in parallel to store `.f16x2` or `.bf16x2` result in destination.

For `.f16` instruction type, operands d and a have `.f16` or `.b16` type. For `.f16x2` instruction type, operands d and a have `.f16x2` or `.b32` type. For `.bf16` instruction type, operands d and a have `.b16` type. For `.bf16x2` instruction type, operands d and a have `.b32` type.

If `.NaN` modifier is specified, the result is canonical NaN if either of the inputs is NaN.

If `.abs` modifier is specified, the magnitude of destination operand d is the maximum of absolute values of both the input arguments.

If `.xorsign` modifier is specified, the sign bit of destination d is equal to the XOR of the sign bits of both the inputs.

Modifiers `.abs` and `.xorsign` must be specified together and `.xorsign` considers the sign bit of both inputs before applying `.abs` operation.

If the result of `max` is NaN then the `.xorsign` and `.abs` modifiers will be ignored.

Semantics

```
if (type == f16 || type == bf16) {
    if (.xorsign) {
        xorsign = getSignBit(a) ^ getSignBit(b);
        if (.abs) {
            a = |a|;
            b = |b|;
        }
    }
    if (isNaN(a) && isNaN(b))           d = NaN;
    if (.NaN && (isNaN(a) || isNaN(b))) d = NaN;
    else if (isNaN(a))                 d = b;
    else if (isNaN(b))                 d = a;
    else                               d = (a > b) ? a : b;
    if (.xorsign && !isNaN(d)) {
```

(continues on next page)

(continued from previous page)

```

        setSignBit(d, xorsign);
    }
} else if (type == f16x2 || type == bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    for (i = 0; i < 2; i++) {
        if (.xorsign) {
            xorsign = getSignBit(fA[i]) ^ getSignBit(fB[i]);
            if (.abs) {
                fA[i] = |fA[i]|;
                fB[i] = |fB[i]|;
            }
        }
        if (isNaN(fA[i]) && isNaN(fB[i]))          d[i] = NaN;
        if (.NaN && (isNaN(fA[i]) || isNaN(fB[i]))) d[i] = NaN;
        else if (isNaN(fA[i]))                    d[i] = fB[i];
        else if (isNaN(fB[i]))                    d[i] = fA[i];
        else                                       d[i] = (fA[i] > fB[i]) ? fA[i]
↪: fB[i];
        if (.xorsign && !isNaN(fA[i])) {
            setSignBit(d[i], xorsign);
        }
    }
}

```

Notes

Subnormal numbers:

By default, subnormal numbers are supported. `max.ftz.{f16, f16x2}` flushes subnormal inputs and results to sign-preserving zero.

If values of both inputs are 0.0, then `+0.0 > -0.0`.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

`max.xorsign.abs` introduced in PTX ISA version 7.2.

Target ISA Notes

Requires `sm_80` or higher.

`max.xorsign.abs` support requires `sm_86` or higher.

Examples

```
max.ftz.f16      h0,h1,h2;
max.f16x2        b0,b1,b2;
// SIMD fp16 max with NaN
max.NaN.f16x2    b0,b1,b2;
// scalar f16 max with xorsign.abs
max.xorsign.abs.f16 Rd, Ra, Rb;
max.bf16         h0, h1, h2;
// scalar bf16 max and NaN
max.NaN.bf16x2   b0, b1, b2;
// SIMD bf16 max with xorsign.abs
max.xorsign.abs.bf16x2 Rd, Ra, Rb;
```

9.7.4.9 Half Precision Floating Point Instructions: tanh

tanh

Find the hyperbolic tangent of a value (in radians)

Syntax

```
tanh.approx.type d, a;
.type = { .f16, .f16x2, .bf16, .bf16x2 }
```

Description

Take hyperbolic tangent value of a.

The type of operands d and a are as specified by .type.

For .f16x2 or .bf16x2 instruction type, each of the half-word operands are operated in parallel and the results are packed appropriately into a .f16x2 or .bf16x2.

Semantics

```
if (.type == .f16 || .type == .bf16) {
    d = tanh(a)
} else if (.type == .f16x2 || .type == .bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    d[0] = tanh(fA[0])
    d[1] = tanh(fA[1])
}
```

Notes

`tanh.approx.{f16, f16x2, bf16, bf16x2}` implements an approximate hyperbolic tangent in the target format.

Results of `tanh` for various corner-case inputs are as follows:

Input	Result
-Inf	-1.0
-0.0	-0.0
+0.0	+0.0
+Inf	1.0
NaN	NaN

The maximum absolute error for `.f16` type is $2^{-10.987}$. The maximum absolute error for `.bf16` type is 2^{-8} .

The subnormal numbers are supported.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

`tanh.approx.{bf16/bf16x2}` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_75` or higher.

`tanh.approx.{bf16/bf16x2}` requires `sm_90` or higher.

Examples

```
tanh.approx.f16    h1, h0;
tanh.approx.f16x2  hd1, hd0;
tanh.approx.bf16   b1, b0;
tanh.approx.bf16x2 hb1, hb0;
```

9.7.4.10 Half Precision Floating Point Instructions: `ex2`

`ex2`

Find the base-2 exponent of input.

Syntax

```
ex2.approx.ctype    d, a;
ex2.approx.ftz.btype d, a;

.type = { .f16,    .f16x2}
.btype = { .bf16,  .bf16x2}
```

Description

Raise 2 to the power a.

The type of operands d and a are as specified by .type.

For .f16x2 or .bf16x2 instruction type, each of the half-word operands are operated in parallel and the results are packed appropriately into a .f16x2 or .bf16x2.

Semantics

```
if (.type == .f16 || .type == .bf16) {
    d = 2 ^ a
} else if (.type == .f16x2 || .type == .bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    d[0] = 2 ^ fA[0]
    d[1] = 2 ^ fA[1]
}
```

Notes

ex2.approx.{f16, f16x2, bf16, bf16x2} implement a fast approximation to 2^a .

For the .f16 type, subnormal inputs are supported. ex2.approx.ftz.bf16 flushes subnormal inputs and results to sign-preserving zero.

Results of ex2.approx.ftz.bf16 for various corner-case inputs are as follows:

Input	Result
-Inf	+0.0
-subnormal	+1.0
-0.0	+1.0
+0.0	+1.0
+subnormal	+1.0
+Inf	+Inf
NaN	NaN

Results of ex2.approx.f16 for various corner-case inputs are as follows:

Input	Result
-Inf	+0.0
-0.0	+1.0
+0.0	+1.0
+Inf	+Inf
NaN	NaN

The maximum relative error for `.f16` type is 2-9.9. The maximum relative error for `.bf16` type is 2-7.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

`ex2.approx.ftz.{bf16/bf16x2}` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_75` or higher.

`ex2.approx.ftz.{bf16/bf16x2}` requires `sm_90` or higher.

Examples

```
ex2.approx.f16      h1, h0;
ex2.approx.f16x2    hd1, hd0;
ex2.approx.ftz.bf16 b1, b2;
ex2.approx.ftz.bf16x2 hb1, hb2;
```

9.7.5. Mixed Precision Floating-Point Instructions

Mixed precision floating-point instructions operate on data with varied floating point precision. Before executing the specified operation, operands with different precision needs to be converted such that all the instruction operands can be represented with a consistent floating-point precision. The register variable to be used for holding a particular operand depends upon the combination of the instruction types. Refer *Fundamental Types* and *Alternate Floating-Point Data Formats* for more details around exact register operand to be used for a given data type.

The mixed precision floating point instructions are:

- ▶ `add`
- ▶ `sub`
- ▶ `fma`

Mixed precision `add`, `sub`, `fma` support saturation of results to the range `[0.0, 1.0]`, with NaN being flushed to positive zero.

9.7.5.1 Mixed Precision Floating Point Instructions: add

add

Add 2 values.

Syntax

```
add{.rnd}{.sat}.f32.atype d, a, c;

.atype = { .f16, .bf16 };
.rnd    = { .rn, .rz, .rm, .rp };
```

Description

Converts input operand `a` from `.atype` into `.f32` type. The converted value is then used for the addition. The resulting value is stored in the destination operand `d`.

Semantics

```
d = convert(a) + c;
```

Notes

Rounding modifiers:

- .rn** mantissa LSB rounds to nearest even
- .rz** mantissa LSB rounds towards zero
- .rm** mantissa LSB rounds towards negative infinity
- .rp** mantissa LSB rounds towards positive infinity

The default value of rounding modifier is `.rn`. Note that an `add` instruction with an explicit rounding modifier is treated conservatively by the code optimizer. An `add` instruction with no rounding modifier defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, `mul/add` sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

By default, subnormal numbers are supported.

Saturation modifier:

`add .sat` clamps the result to `[0.0, 1.0]`. NaN results are flushed to `+0.0f`.

PTX ISA Notes

`add.f32.{f16/bf16}` introduced in PTX ISA version 8.6.

Target ISA Notes

`add.f32.{f16/bf16}` requires `sm_100` or higher.

Examples

```
.reg .f32 fc, fd;  
.reg .b16 ba;  
add.rz.f32.bf16.sat    fd, fa, fc;
```

9.7.5.2 Mixed Precision Floating Point Instructions: `sub`

`sub`

Subtract one value from another.

Syntax

```
sub{.rnd}{.sat}.f32.atype d, a, c;  
  
.atype = { .f16, .bf16 };  
.rnd    = { .rn, .rz, .rm, .rp };
```

Description

Converts input operand `a` from `.atype` into `.f32` type. The converted value is then used for the subtraction. The resulting value is stored in the destination operand `d`.

Semantics

```
d = convert(a) - c;
```

Notes

Rounding modifiers:

- `.rn` mantissa LSB rounds to nearest even
- `.rz` mantissa LSB rounds towards zero
- `.rm` mantissa LSB rounds towards negative infinity
- `.rp` mantissa LSB rounds towards positive infinity

The default value of rounding modifier is `.rn`. Note that an `sub` instruction with an explicit rounding modifier is treated conservatively by the code optimizer. An `sub` instruction with no rounding modifier defaults to round-to-nearest-even and may be optimized aggressively by the code optimizer. In particular, `mul/sub` sequences with no rounding modifiers may be optimized to use fused-multiply-add instructions on the target device.

Subnormal numbers:

By default, subnormal numbers are supported.

Saturation modifier:

`sub.sat` clamps the result to `[0.0, 1.0]`. NaN results are flushed to `+0.0f`.

PTX ISA Notes

`sub.f32.{f16/bf16}` introduced in PTX ISA version 8.6.

Target ISA Notes

`sub.f32.{f16/bf16}` requires `sm_100` or higher.

Examples

```
.reg .f32 fc, fd;
.reg .f16 ha;
sub.rz.f32.f16.sat    fd, ha, fc;
```

9.7.5.3 Mixed Precision Floating Point Instructions: `fma`

`fma`

Fused multiply-add.

Syntax

```
fma.rnd{.sat}.f32.abtype  d, a, b, c;

.abtype = { .f16, .bf16 };
.rnd    = { .rn, .rz, .rm, .rp };
```

Description

Converts input operands a and b from .atype into .f32 type. The converted values are then used to perform fused multiply-add operation with no loss of precision in the intermediate product and addition. The resulting value is stored in the destination operand d.

Semantics

```
d = convert(a) * convert(b) + c;
```

Notes

`fma.f32.{f16/bf16}` computes the product of a and b to infinite precision and then adds c to this product, again in infinite precision. The resulting value is then rounded to single precision using the rounding mode specified by .rnd.

Rounding modifiers(no default):

- .rn mantissa LSB rounds to nearest even
- .rz mantissa LSB rounds towards zero
- .rm mantissa LSB rounds towards negative infinity
- .rp mantissa LSB rounds towards positive infinity

Subnormal numbers:

By default, subnormal numbers are supported.

Saturation modifier:

`fma.sat` clamps the result to [0.0, 1.0]. NaN results are flushed to +0.0f.

PTX ISA Notes

`fma.f32.{f16/bf16}` introduced in PTX ISA version 8.6.

Target ISA Notes

`fma.f32.{f16/bf16}` requires sm_100 or higher.

Examples

```
.reg .f32 fc, fd;  
.reg .f16 ha, hb;  
fma.rz.sat.f32.f16.sat    fd, ha, hb, fc;
```

9.7.6. Comparison and Selection Instructions

The comparison select instructions are:

- ▶ `set`
- ▶ `setp`
- ▶ `selp`
- ▶ `slct`

As with single-precision floating-point instructions, the `set`, `setp`, and `slct` instructions support subnormal numbers for `sm_20` and higher targets and flush single-precision subnormal inputs to sign-preserving zero for `sm_1x` targets. The optional `.ftz` modifier provides backward compatibility with `sm_1x` targets by flushing subnormal inputs and results to sign-preserving zero regardless of the target architecture.

9.7.6.1 Comparison and Selection Instructions: `set`

`set`

Compare two numeric values with a relational operator, and optionally combine this result with a predicate value by applying a Boolean operator.

Syntax

```
set.CmpOp{.ftz}.dtype.stype      d, a, b;
set.CmpOp.BoolOp{.ftz}.dtype.stype d, a, b, {!}c;

.CmpOp = { eq, ne, lt, le, gt, ge, lo, ls, hi, hs,
            equ, neu, ltu, leu, gtu, geu, num, nan };
.BoolOp = { and, or, xor };
.dtype   = { .u32, .s32, .f32 };
.stype   = { .b16, .b32, .b64,
            .u16, .u32, .u64,
            .s16, .s32, .s64,
            .f32, .f64 };
```

Description

Compares two numeric values and optionally combines the result with another predicate value by applying a Boolean operator. If this result is True, `1.0f` is written for floating-point destination types, and `0xffffffff` is written for integer destination types. Otherwise, `0x00000000` is written.

Operand `d` has type `.dtype`; operands `a` and `b` have type `.stype`; operand `c` has type `.pred`.

Semantics

```
t = (a CmpOp b) ? 1 : 0;
if (isFloat(dtype))
    d = BoolOp(t, c) ? 1.0f : 0x00000000;
else
    d = BoolOp(t, c) ? 0xffffffff : 0x00000000;
```

Integer Notes

The signed and unsigned comparison operators are `eq`, `ne`, `lt`, `le`, `gt`, `ge`.

For unsigned values, the comparison operators `lo`, `ls`, `hi`, and `hs` for lower, lower-or-same, higher, and higher-or-same may be used instead of `lt`, `le`, `gt`, `ge`, respectively.

The untyped, bit-size comparisons are `eq` and `ne`.

Floating Point Notes

The ordered comparisons are `eq`, `ne`, `lt`, `le`, `gt`, `ge`. If either operand is NaN, the result is False.

To aid comparison operations in the presence of NaN values, unordered versions are included: `equ`, `neu`, `ltu`, `leu`, `gtu`, `geu`. If both operands are numeric values (not NaN), then these comparisons have the same result as their ordered counterparts. If either operand is NaN, then the result of these comparisons is True.

`num` returns True if both operands are numeric values (not NaN), and `nan` returns True if either operand is NaN.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`set.ftz.dtype.f32` flushes subnormal inputs to sign-preserving zero.

sm_1x

`set.dtype.f64` supports subnormal numbers.

`set.dtype.f32` flushes subnormal inputs to sign-preserving zero.

Modifier `.ftz` applies only to `.f32` comparisons.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

set with .f64 source type requires sm_13 or higher.

Examples

```
@p  set.lt.and.f32.s32  d,a,b,r;
    set.eq.u32.u32      d,i,n;
```

9.7.6.2 Comparison and Selection Instructions: setp

setp

Compare two numeric values with a relational operator, and (optionally) combine this result with a predicate value by applying a Boolean operator.

Syntax

```
setp.CmpOp{.ftz}.type      p[|q], a, b;
setp.CmpOp.BoolOp{.ftz}.type p[|q], a, b, {!}c;

.CmpOp = { eq, ne, lt, le, gt, ge, lo, ls, hi, hs,
            equ, neu, ltu, leu, gtu, geu, num, nan };
.BoolOp = { and, or, xor };
.type    = { .b16, .b32, .b64,
            .u16, .u32, .u64,
            .s16, .s32, .s64,
            .f32, .f64 };
```

Description

Compares two values and combines the result with another predicate value by applying a Boolean operator. This result is written to the first destination operand. A related value computed using the complement of the compare result is written to the second destination operand.

Applies to all numeric types. Operands a and b have type .type; operands p, q, and c have type .pred. The sink symbol '_' may be used in place of any one of the destination operands.

Semantics

```
t = (a CmpOp b) ? 1 : 0;
p = BoolOp(t, c);
q = BoolOp(!t, c);
```

Integer Notes

The signed and unsigned comparison operators are `eq`, `ne`, `lt`, `le`, `gt`, `ge`.

For unsigned values, the comparison operators `lo`, `ls`, `hi`, and `hs` for lower, lower-or-same, higher, and higher-or-same may be used instead of `lt`, `le`, `gt`, `ge`, respectively.

The untyped, bit-size comparisons are `eq` and `ne`.

Floating Point Notes

The ordered comparisons are `eq`, `ne`, `lt`, `le`, `gt`, `ge`. If either operand is NaN, the result is `False`.

To aid comparison operations in the presence of NaN values, unordered versions are included: `equ`, `neu`, `ltu`, `leu`, `gtu`, `geu`. If both operands are numeric values (not NaN), then these comparisons have the same result as their ordered counterparts. If either operand is NaN, then the result of these comparisons is `True`.

`num` returns `True` if both operands are numeric values (not NaN), and `nan` returns `True` if either operand is NaN.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`setp.ftz.dtype.f32` flushes subnormal inputs to sign-preserving zero.

sm_1x

`setp.dtype.f64` supports subnormal numbers.

`setp.dtype.f32` flushes subnormal inputs to sign-preserving zero.

Modifier `.ftz` applies only to `.f32` comparisons.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

`setp` with `.f64` source type requires `sm_13` or higher.

Examples

```
@q    setp.lt.and.s32  p|q,a,b,r;
      setp.eq.u32     p,i,n;
```

9.7.6.3 Comparison and Selection Instructions: `selp`

`selp`

Select between source operands, based on the value of the predicate source operand.

Syntax

```
selp.type d, a, b, c;

.type = { .b16, .b32, .b64,
          .u16, .u32, .u64,
          .s16, .s32, .s64,
          .f32, .f64 };
```

Description

Conditional selection. If `c` is True, `a` is stored in `d`, `b` otherwise. Operands `d`, `a`, and `b` must be of the same type. Operand `c` is a predicate.

Semantics

```
d = (c == 1) ? a : b;
```

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

`selp.f64` requires `sm_13` or higher.

Examples

```
    selp.s32  r0, r, g, p;
@q  selp.f32  f0, t, x, xp;
```

9.7.6.4 Comparison and Selection Instructions: `slect`

`slect`

Select one source operand, based on the sign of the third operand.

Syntax

```
slect.dtype.s32      d, a, b, c;  
slect{.ftz}.dtype.f32 d, a, b, c;  
  
.dtype = { .b16, .b32, .b64,  
           .u16, .u32, .u64,  
           .s16, .s32, .s64,  
           .f32, .f64 };
```

Description

Conditional selection. If $c \geq 0$, a is stored in d , otherwise b is stored in d . Operands d , a , and b are treated as a bitsize type of the same width as the first instruction type; operand c must match the second instruction type (`.s32` or `.f32`). The selected input is copied to the output without modification.

Semantics

```
d = (c >= 0) ? a : b;
```

Floating Point Notes

For `.f32` comparisons, negative zero equals zero.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

`slect.ftz.dtype.f32` flushes subnormal values of operand c to sign-preserving zero, and operand a is selected.

sm_1x

`slect.dtype.f32` flushes subnormal values of operand c to sign-preserving zero, and operand a is selected.

Modifier `.ftz` applies only to `.f32` comparisons.

If operand c is NaN, the comparison is ordered and operand b is selected.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

`slct.f64` requires `sm_13` or higher.

Examples

```
slct.u32.s32  x, y, z, val;
slct.ftz.u64.f32  A, B, C, fval;
```

9.7.7. Half Precision Comparison Instructions

The comparison instructions are:

- `set`
- `setp`

9.7.7.1 Half Precision Comparison Instructions: `set`

`set`

Compare two numeric values with a relational operator, and optionally combine this result with a predicate value by applying a Boolean operator.

Syntax

```
set.CmpOp{.ftz}.f16.stype      d, a, b;
set.CmpOp.BoolOp{.ftz}.f16.stype  d, a, b, {!}c;

set.CmpOp.bf16.stype          d, a, b;
set.CmpOp.BoolOp.bf16.stype    d, a, b, {!}c;

set.CmpOp{.ftz}.dtype.f16      d, a, b;
set.CmpOp.BoolOp{.ftz}.dtype.f16  d, a, b, {!}c;
.dtype = { .u16, .s16, .u32, .s32}

set.CmpOp.dtype.bf16           d, a, b;
set.CmpOp.BoolOp.dtype.bf16     d, a, b, {!}c;
.dtype = { .u16, .s16, .u32, .s32}

set.CmpOp{.ftz}.dtype.f16x2     d, a, b;
set.CmpOp.BoolOp{.ftz}.dtype.f16x2  d, a, b, {!}c;
.dtype = { .f16x2, .u32, .s32}
```

(continues on next page)

(continued from previous page)

```

set.CmpOp.dtype.bf16x2      d, a, b;
set.CmpOp.BoolOp.dtype.bf16x2 d, a, b, {!}c;
.dtype = { .bf16x2, .u32, .s32}

.CmpOp = { eq, ne, lt, le, gt, ge,
           equ, neu, ltu, leu, gt, geu, num, nan };
.BoolOp = { and, or, xor };
.stype = { .b16, .b32, .b64,
           .u16, .u32, .u64,
           .s16, .s32, .s64,
           .f16, .f32, .f64};

```

Description

Compares two numeric values and optionally combines the result with another predicate value by applying a Boolean operator.

Result of this computation is written in destination register in the following way:

- ▶ If result is True,
 - ▶ 0xffffffff is written for destination types .u32/.s32.
 - ▶ 0xffff is written for destination types .u16/.s16.
 - ▶ 1.0 in target precision floating point format is written for destination type .f16, .bf16.
- ▶ If result is False,
 - ▶ 0x0 is written for all integer destination types.
 - ▶ 0.0 in target precision floating point format is written for destination type .f16, .bf16.

If the source type is .f16x2 or .bf16x2 then result of individual operations are packed in the 32-bit destination operand.

Operand c has type .pred.

Semantics

```

if (stype == .f16x2 || stype == .bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    t[0] = (fA[0] CmpOp fB[0]) ? 1 : 0;
    t[1] = (fA[1] CmpOp fB[1]) ? 1 : 0;
    if (dtype == .f16x2 || stype == .bf16x2) {
        for (i = 0; i < 2; i++) {
            d[i] = BoolOp(t[i], c) ? 1.0 : 0.0;
        }
    } else {
        for (i = 0; i < 2; i++) {
            d[i] = BoolOp(t[i], c) ? 0xffff : 0;
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

} else if (dtype == .f16 || stype == .bf16) {
    t = (a CmpOp b) ? 1 : 0;
    d = BoolOp(t, c) ? 1.0 : 0.0;
} else { // Integer destination type
    trueVal = (isU16(dtype) || isS16(dtype)) ? 0xffff : 0xffffffff;
    t = (a CmpOp b) ? 1 : 0;
    d = BoolOp(t, c) ? trueVal : 0;
}

```

Floating Point Notes

The ordered comparisons are eq, ne, lt, le, gt, ge. If either operand is NaN, the result is False.

To aid comparison operations in the presence of NaN values, unordered versions are included: equ, neu, ltu, leu, gtu, geu. If both operands are numeric values (not NaN), then these comparisons have the same result as their ordered counterparts. If either operand is NaN, then the result of these comparisons is True.

num returns True if both operands are numeric values (not NaN), and nan returns True if either operand is NaN.

Subnormal numbers:

By default, subnormal numbers are supported.

When .ftz modifier is specified then subnormal inputs and results are flushed to sign preserving zero.

PTX ISA Notes

Introduced in PTX ISA version 4.2.

set.{u16, u32, s16, s32}.f16 and set.{u32, s32}.f16x2 are introduced in PTX ISA version 6.5.

set.{u16, u32, s16, s32}.bf16, set.{u32, s32, bf16x2}.bf16x2, set.bf16.{s16, u16, f16, b16, s32, u32, f32, b32, s64, u64, f64, b64} are introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_53 or higher.

set.{u16, u32, s16, s32}.bf16, set.{u32, s32, bf16x2}.bf16x2, set.bf16.{s16, u16, f16, b16, s32, u32, f32, b32, s64, u64, f64, b64} require sm_90 or higher.

Examples

```
set.lt.and.f16.f16    d,a,b,r;
set.eq.f16x2.f16x2    d,i,n;
set.eq.u32.f16x2      d,i,n;
set.lt.and.u16.f16    d,a,b,r;
set.ltu.or.bf16.f16   d,u,v,s;
set.equ.bf16x2.bf16x2 d,j,m;
set.geu.s32.bf16x2    d,j,m;
set.num.xor.s32.bf16  d,u,v,s;
```

9.7.7.2 Half Precision Comparison Instructions: setp

setp

Compare two numeric values with a relational operator, and optionally combine this result with a predicate value by applying a Boolean operator.

Syntax

```
setp.CmpOp{.ftz}.f16      p, a, b;
setp.CmpOp.BoolOp{.ftz}.f16 p, a, b, {!}c;

setp.CmpOp{.ftz}.f16x2    p|q, a, b;
setp.CmpOp.BoolOp{.ftz}.f16x2 p|q, a, b, {!}c;

setp.CmpOp.bf16          p, a, b;
setp.CmpOp.BoolOp.bf16   p, a, b, {!}c;

setp.CmpOp.bf16x2        p|q, a, b;
setp.CmpOp.BoolOp.bf16x2 p|q, a, b, {!}c;

.CmpOp = { eq, ne, lt, le, gt, ge,
            equ, neu, ltu, leu, gtu, geu, num, nan };
.BoolOp = { and, or, xor };
```

Description

Compares two values and combines the result with another predicate value by applying a Boolean operator. This result is written to the destination operand.

Operand c, p and q has type .pred.

For instruction type .f16, operands a and b have type .b16 or .f16.

For instruction type .f16x2, operands a and b have type .b32.

For instruction type .bf16, operands a and b have type .b16.

For instruction type .bf16x2, operands a and b have type .b32.

Semantics

```

if (type == .f16 || type == .bf16) {
    t = (a CmpOp b) ? 1 : 0;
    p = BoolOp(t, c);
} else if (type == .f16x2 || type == .bf16x2) {
    fA[0] = a[0:15];
    fA[1] = a[16:31];
    fB[0] = b[0:15];
    fB[1] = b[16:31];
    t[0] = (fA[0] CmpOp fB[0]) ? 1 : 0;
    t[1] = (fA[1] CmpOp fB[1]) ? 1 : 0;
    p = BoolOp(t[0], c);
    q = BoolOp(t[1], c);
}

```

Floating Point Notes

The ordered comparisons are eq, ne, lt, le, gt, ge. If either operand is NaN, the result is False.

To aid comparison operations in the presence of NaN values, unordered versions are included: equ, neu, ltu, leu, gtu, geu. If both operands are numeric values (not NaN), then these comparisons have the same result as their ordered counterparts. If either operand is NaN, then the result of these comparisons is True.

num returns True if both operands are numeric values (not NaN), and nan returns True if either operand is NaN.

Subnormal numbers:

By default, subnormal numbers are supported.

setp.ftz.{f16, f16x2} flushes subnormal inputs to sign-preserving zero.

PTX ISA Notes

Introduced in PTX ISA version 4.2.

setp.{bf16/bf16x2} introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_53 or higher.

setp.{bf16/bf16x2} requires sm_90 or higher.

Examples

```
setp.lt.and.f16x2  p|q,a,b,r;  
@q setp.eq.f16     p,i,n;  
  
setp.gt.or.bf16x2 u|v,c,d,s;  
@q setp.eq.bf16    u,j,m;
```

9.7.8. Logic and Shift Instructions

The logic and shift instructions are fundamentally untyped, performing bit-wise operations on operands of any type, provided the operands are of the same size. This permits bit-wise operations on floating point values without having to define a union to access the bits. Instructions and, or, xor, and not also operate on predicates.

The logical shift instructions are:

- ▶ and
- ▶ or
- ▶ xor
- ▶ not
- ▶ cnot
- ▶ lop3
- ▶ shf
- ▶ shl
- ▶ shr

9.7.8.1 Logic and Shift Instructions: and

and

Bitwise AND.

Syntax

```
and.type d, a, b;  
  
.type = { .pred, .b16, .b32, .b64 };
```

Description

Compute the bit-wise and operation for the bits in a and b.

Semantics

```
d = a & b;
```

Notes

The size of the operands must match, but not necessarily the type.
Allowed types include predicate registers.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
and.b32 x,q,r;  
and.b32 sign,fpvalue,0x80000000;
```

9.7.8.2 Logic and Shift Instructions: or

or

Biwise OR.

Syntax

```
or.type d, a, b;  
.type = { .pred, .b16, .b32, .b64 };
```

Description

Compute the bit-wise or operation for the bits in a and b.

Semantics

```
d = a | b;
```

Notes

The size of the operands must match, but not necessarily the type.
Allowed types include predicate registers.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
or.b32 mask mask, 0x00010001  
or.pred p, q, r;
```

9.7.8.3 Logic and Shift Instructions: xor

xor

Bitwise exclusive-OR (inequality).

Syntax

```
xor.type d, a, b;  
.type = { .pred, .b16, .b32, .b64 };
```


Description

Compute the bit-wise exclusive-or operation for the bits in a and b.

Semantics

```
d = a ^ b;
```

Notes

The size of the operands must match, but not necessarily the type.
Allowed types include predicate registers.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
xor.b32 d,q,r;  
xor.b16 d,x,0x0001;
```

9.7.8.4 Logic and Shift Instructions: not

not

Bitwise negation; one's complement.

Syntax

```
not.type d, a;  
.type = { .pred, .b16, .b32, .b64 };
```

Description

Invert the bits in a.

Semantics

```
d = ~a;
```

Notes

The size of the operands must match, but not necessarily the type.
Allowed types include predicates.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
not.b32 mask,mask;  
not.pred p,q;
```

9.7.8.5 Logic and Shift Instructions: cnot

cnot

C/C++ style logical negation.

Syntax

```
cnot.type d, a;  
.type = { .b16, .b32, .b64 };
```

Description

Compute the logical negation using C/C++ semantics.

Semantics

```
d = (a==0) ? 1 : 0;
```

Notes

The size of the operands must match, but not necessarily the type.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
cnot.b32 d,a;
```

9.7.8.6 Logic and Shift Instructions: `lop3`

`lop3`

Arbitrary logical operation on 3 inputs.

Syntax

```
lop3.b32 d, a, b, c, immLut;  
lop3.BoolOp.b32 d|p, a, b, c, immLut, q;  
  
.BoolOp    = { .or , .and };
```

Description

Compute bitwise logical operation on inputs a, b, c and store the result in destination d.

Optionally, `.BoolOp` can be specified to compute the predicate result p by performing a Boolean operation on the destination operand d with the predicate q in the following manner:

```
p = (d != 0) BoolOp q;
```

The sink symbol `'_'` may be used in place of the destination operand d when `.BoolOp` qualifier is specified.

The logical operation is defined by a look-up table which, for 3 inputs, can be represented as an 8-bit value specified by operand `immLut` as described below. `immLut` is an integer constant that can take values from 0 to 255, thereby allowing up to 256 distinct logical operations on inputs a, b, c.

For a logical operation $F(a, b, c)$ the value of `immLut` can be computed by applying the same operation to three predefined constant values as follows:

```
ta = 0xF0;
tb = 0xCC;
tc = 0xAA;

immLut = F(ta, tb, tc);
```

Examples:

```
If F = (a & b & c);
immLut = 0xF0 & 0xCC & 0xAA = 0x80

If F = (a | b | c);
immLut = 0xF0 | 0xCC | 0xAA = 0xFE

If F = (a & b & ~c);
immLut = 0xF0 & 0xCC & (~0xAA) = 0x40

If F = ((a & b | c) ^ a);
immLut = (0xF0 & 0xCC | 0xAA) ^ 0xF0 = 0x1A
```

The following table illustrates computation of `immLut` for various logical operations:

ta	tb	tc	Oper 0 (False)	Oper 1 (ta & tb & tc)	Oper 2 (ta & tb & ~tc)	...	Oper 254 (ta tb tc)	Oper 255 (True)
0	0	0	0	0	0	...	0	1
0	0	1	0	0	0		1	1
0	1	0	0	0	0		1	1
0	1	1	0	0	0		1	1
1	0	0	0	0	0		1	1
1	0	1	0	0	0		1	1
1	1	0	0	0	1		1	1
1	1	1	0	1	0		1	1
immLut			0x0	0x80	0x40	...	0xFE	0xFF

Semantics

```
F = GetFunctionFromTable(immLut); // returns the function corresponding to immLut
↪value
d = F(a, b, c);
if (BoolOp specified) {
    p = (d != 0) BoolOp q;
}
```

PTX ISA Notes

Introduced in PTX ISA version 4.3.

Support for `.BoolOp` qualifier introduced in PTX ISA version 8.2.

Target ISA Notes

Requires `sm_50` or higher.

Qualifier `.BoolOp` requires `sm_70` or higher.

Examples

```
lop3.b32      d, a, b, c, 0x40;
lop3.or.b32   d|p, a, b, c, 0x3f, q;
lop3.and.b32  _|p, a, b, c, 0x3f, q;
```

9.7.8.7 Logic and Shift Instructions: `shf`

`shf`

Funnel shift.

Syntax

```
shf.l.mode.b32 d, a, b, c; // left shift
shf.r.mode.b32 d, a, b, c; // right shift

.mode = { .clamp, .wrap };
```

Description

Shift the 64-bit value formed by concatenating operands *a* and *b* left or right by the amount specified by the unsigned 32-bit value in *c*. Operand *b* holds bits 63:32 and operand *a* holds bits 31:0 of the 64-bit source value. The source is shifted left or right by the clamped or wrapped value in *c*. For *shf.l*, the most-significant 32-bits of the result are written into *d*; for *shf.r*, the least-significant 32-bits of the result are written into *d*.

Semantics

```
u32 n = (.mode == .clamp) ? min(c, 32) : c & 0x1f;
switch (shf.dir) { // shift concatenation of [b, a]
    case shf.l:    // extract 32 msbs
        u32 d = (b << n) | (a >> (32-n));
    case shf.r:    // extract 32 lsbs
        u32 d = (b << (32-n)) | (a >> n);
}
```

Notes

Use funnel shift for multi-word shift operations and for rotate operations. The shift amount is limited to the range 0..32 in clamp mode and 0..31 in wrap mode, so shifting multi-word values by distances greater than 32 requires first moving 32-bit words, then using *shf* to shift the remaining 0..31 distance.

To shift data sizes greater than 64 bits to the right, use repeated *shf.r* instructions applied to adjacent words, operating from least-significant word towards most-significant word. At each step, a single word of the shifted result is computed. The most-significant word of the result is computed using a *shr.{u32,s32}* instruction, which zero or sign fills based on the instruction type.

To shift data sizes greater than 64 bits to the left, use repeated *shf.l* instructions applied to adjacent words, operating from most-significant word towards least-significant word. At each step, a single word of the shifted result is computed. The least-significant word of the result is computed using a *shl* instruction.

Use funnel shift to perform 32-bit left or right rotate by supplying the same value for source arguments *a* and *b*.

PTX ISA Notes

Introduced in PTX ISA version 3.1.

Target ISA Notes

Requires sm_32 or higher.

Example

```
shf.l.clamp.b32  r3,r1,r0,16;

// 128-bit left shift; n < 32
// [r7,r6,r5,r4] = [r3,r2,r1,r0] << n
shf.l.clamp.b32  r7,r2,r3,n;
shf.l.clamp.b32  r6,r1,r2,n;
shf.l.clamp.b32  r5,r0,r1,n;
shl.b32          r4,r0,n;

// 128-bit right shift, arithmetic; n < 32
// [r7,r6,r5,r4] = [r3,r2,r1,r0] >> n
shf.r.clamp.b32  r4,r0,r1,n;
shf.r.clamp.b32  r5,r1,r2,n;
shf.r.clamp.b32  r6,r2,r3,n;
shr.s32          r7,r3,n;    // result is sign-extended

shf.r.clamp.b32  r1,r0,r0,n; // rotate right by n; n < 32
shf.l.clamp.b32  r1,r0,r0,n; // rotate left by n; n < 32

// extract 32-bits from [r1,r0] starting at position n < 32
shf.r.clamp.b32  r0,r0,r1,n;
```

9.7.8.8 Logic and Shift Instructions: shl

shl

Shift bits left, zero-fill on right.

Syntax

```
shl.type d, a, b;

.type = { .b16, .b32, .b64 };
```

Description

Shift a left by the amount specified by unsigned 32-bit value in b.

Semantics

```
d = a << b;
```

Notes

Shift amounts greater than the register width N are clamped to N .

The sizes of the destination and first source operand must match, but not necessarily the type. The b operand must be a 32-bit value, regardless of the instruction type.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Example

```
shl.b32 q, a, 2;
```

9.7.8.9 Logic and Shift Instructions: shr

shr

Shift bits right, sign or zero-fill on left.

Syntax

```
shr.type d, a, b;  
  
.type = { .b16, .b32, .b64,  
          .u16, .u32, .u64,  
          .s16, .s32, .s64 };
```


Description

Shift *a* right by the amount specified by unsigned 32-bit value in *b*. Signed shifts fill with the sign bit, unsigned and untyped shifts fill with 0.

Semantics

```
d = a >> b;
```

Notes

Shift amounts greater than the register width *N* are clamped to *N*.

The sizes of the destination and first source operand must match, but not necessarily the type. The *b* operand must be a 32-bit value, regardless of the instruction type.

Bit-size types are included for symmetry with `shl`.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Example

```
shr.u16  c,a,2;
shr.s32  i,i,1;
shr.b16  k,i,j;
```

9.7.9. Data Movement and Conversion Instructions

These instructions copy data from place to place, and from state space to state space, possibly converting it from one format to another. `mov`, `ld`, `ldu`, and `st` operate on both scalar and vector types. The `isspacep` instruction is provided to query whether a generic address falls within a particular state space window. The `cvta` instruction converts addresses between generic and `const`, `global`, `local`, or shared state spaces.

Instructions `ld`, `st`, `suld`, and `sust` support optional cache operations.

The Data Movement and Conversion Instructions are:

- ▶ `mov`
- ▶ `shfl.sync`
- ▶ `prmt`

- ▶ `ld`
- ▶ `ldu`
- ▶ `st`
- ▶ `st.async`
- ▶ `st.bulk`
- ▶ `multimem.ld_reduce, multimem.st, multimem.red`
- ▶ `prefetch, prefetchu`
- ▶ `isspacep`
- ▶ `cvta`
- ▶ `cvt`
- ▶ `cvt.pack`
- ▶ `cp.async`
- ▶ `cp.async.commit_group`
- ▶ `cp.async.wait_group, cp.async.wait_all`
- ▶ `cp.async.bulk`
- ▶ `cp.reduce.async.bulk`
- ▶ `cp.async.bulk.prefetch`
- ▶ `multimem.cp.async.bulk`
- ▶ `multimem.cp.reduce.async.bulk`
- ▶ `cp.async.bulk.tensor`
- ▶ `cp.reduce.async.bulk.tensor`
- ▶ `cp.async.bulk.prefetch.tensor`
- ▶ `cp.async.bulk.commit_group`
- ▶ `cp.async.bulk.wait_group`
- ▶ `tensormap.replace`

9.7.9.1 Cache Operators

PTX ISA version 2.0 introduced optional cache operators on load and store instructions. The cache operators require a target architecture of `sm_20` or higher.

Cache operators on load or store instructions are treated as performance hints only. The use of a cache operator on an `ld` or `st` instruction does not change the memory consistency behavior of the program.

For `sm_20` and higher, the cache operators have the following definitions and behavior.

Table 30: Cache Operators for Memory Load Instructions

Operator	Meaning
.ca	Cache at all levels, likely to be accessed again. The default load instruction cache operation is <code>ld.ca</code> , which allocates cache lines in all levels (L1 and L2) with normal eviction policy. Global data is coherent at the L2 level, but multiple L1 caches are not coherent for global data. If one thread stores to global memory via one L1 cache, and a second thread loads that address via a second L1 cache with <code>ld.ca</code> , the second thread may get stale L1 cache data, rather than the data stored by the first thread. The driver must invalidate global L1 cache lines between dependent grids of parallel threads. Stores by the first grid program are then correctly fetched by the second grid program issuing default <code>ld.ca</code> loads cached in L1.
.cg	Cache at global level (cache in L2 and below, not L1). Use <code>ld.cg</code> to cache loads only globally, bypassing the L1 cache, and cache only in the L2 cache.
.cs	Cache streaming, likely to be accessed once. The <code>ld.cs</code> load cached streaming operation allocates global lines with evict-first policy in L1 and L2 to limit cache pollution by temporary streaming data that may be accessed once or twice. When <code>ld.cs</code> is applied to a Local window address, it performs the <code>ld.lu</code> operation.
.lu	Last use. The compiler/programmer may use <code>ld.lu</code> when restoring spilled registers and popping function stack frames to avoid needless write-backs of lines that will not be used again. The <code>ld.lu</code> instruction performs a load cached streaming operation (<code>ld.cs</code>) on global addresses.
.cv	Don't cache and fetch again (consider cached system memory lines stale, fetch again). The <code>ld.cv</code> load operation applied to a global System Memory address invalidates (discards) a matching L2 line and re-fetches the line on each new load.

Table 31: Cache Operators for Memory Store Instructions

Operator	Meaning
<code>.wb</code>	Cache write-back all coherent levels. The default store instruction cache operation is <code>st .wb</code> , which writes back cache lines of coherent cache levels with normal eviction policy. If one thread stores to global memory, bypassing its L1 cache, and a second thread in a different SM later loads from that address via a different L1 cache with <code>ld .ca</code> , the second thread may get a hit on stale L1 cache data, rather than get the data from L2 or memory stored by the first thread. The driver must invalidate global L1 cache lines between dependent grids of thread arrays. Stores by the first grid program are then correctly missed in L1 and fetched by the second grid program issuing default <code>ld .ca</code> loads.
<code>.cg</code>	Cache at global level (cache in L2 and below, not L1). Use <code>st .cg</code> to cache global store data only globally, bypassing the L1 cache, and cache only in the L2 cache.
<code>.cs</code>	Cache streaming, likely to be accessed once. The <code>st .cs</code> store cached-streaming operation allocates cache lines with evict-first policy to limit cache pollution by streaming output data.
<code>.wt</code>	Cache write-through (to system memory). The <code>st .wt</code> store write-through operation applied to a global System Memory address writes through the L2 cache.

9.7.9.2 Cache Eviction Priority Hints

PTX ISA version 7.4 adds optional cache eviction priority hints on load and store instructions. Cache eviction priority requires target architecture `sm_70` or higher.

Cache eviction priority on load or store instructions is treated as a performance hint. It is supported for `.global` state space and generic addresses where the address points to `.global` state space.

Table 32: Cache Eviction Priority Hints for Memory Load and Store Instructions

Cache Eviction Priority	Meaning
<code>evict_normal</code>	Cache data with normal eviction priority. This is the default eviction priority.
<code>evict_first</code>	Data cached with this priority will be first in the eviction priority order and will likely be evicted when cache eviction is required. This priority is suitable for streaming data.
<code>evict_last</code>	Data cached with this priority will be last in the eviction priority order and will likely be evicted only after other data with <code>evict_normal</code> or <code>evict_first</code> eviction priority is already evicted. This priority is suitable for data that should remain persistent in cache.
<code>evict_unchanged</code>	Do not change eviction priority order as part of this operation.
<code>no_allocate</code>	Do not allocate data to cache. This priority is suitable for streaming data.

9.7.9.3 Data Movement and Conversion Instructions: mov

mov

Set a register variable with the value of a register variable or an immediate value. Take the non-generic address of a variable in global, local, or shared state space.

Syntax

```
mov.type d, a;
mov.type d, sreg;
mov.type d, avar;           // get address of variable
mov.type d, avar+imm;       // get address of variable with offset
mov.u32 d, fname;           // get address of device function
mov.u64 d, fname;           // get address of device function
mov.u32 d, kernel;          // get address of entry function
mov.u64 d, kernel;          // get address of entry function

.type = { .pred,
          .b16, .b32, .b64,
          .u16, .u32, .u64,
          .s16, .s32, .s64,
          .f32, .f64 };
```

Description

Write register d with the value of a.

Operand a may be a register, special register, variable with optional offset in an addressable memory space, or function name.

For variables declared in `.const`, `.global`, `.local`, and `.shared` state spaces, `mov` places the non-generic address of the variable (i.e., the address of the variable in its state space) into the destination register. The generic address of a variable in `const`, `global`, `local`, or `shared` state space may be generated by first taking the address within the state space with `mov` and then converting it to a generic address using the `cvta` instruction; alternately, the generic address of a variable declared in `const`, `global`, `local`, or `shared` state space may be taken directly using the `cvta` instruction.

Note that if the address of a device function parameter is moved to a register, the parameter will be copied onto the stack and the address will be in the local state space.

Semantics

```
d = a;
d = sreg;
d = &avar;           // address is non-generic; i.e., within the variable's declared
↳state space
d = &avar+imm;
```

Notes

- ▶ Although only predicate and bit-size types are required, we include the arithmetic types for the programmer's convenience: their use enhances program readability and allows additional type checking.
- ▶ When moving address of a kernel or a device function, only `.u32` or `.u64` instruction types are allowed. However, if a signed type is used, it is not treated as a compilation error. The compiler issues a warning in this case.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Taking the address of kernel entry functions requires PTX ISA version 3.1 or later. Kernel function addresses should only be used in the context of CUDA Dynamic Parallelism system calls. See the *CUDA Dynamic Parallelism Programming Guide* for details.

Target ISA Notes

`mov.f64` requires `sm_13` or higher.

Taking the address of kernel entry functions requires `sm_35` or higher.

Examples

```
mov.f32  d,a;
mov.u16  u,v;
mov.f32  k,0.1;
mov.u32  ptr, A;           // move address of A into ptr
mov.u32  ptr, A[5];        // move address of A[5] into ptr
mov.u32  ptr, A+20;        // move address with offset into ptr
mov.u32  addr, myFunc;     // get address of device function 'myFunc'
mov.u64  kptr, main;       // get address of entry function 'main'
```

9.7.9.4 Data Movement and Conversion Instructions: `mov`

`mov`

Move vector-to-scalar (pack) or scalar-to-vector (unpack).

Syntax

```
mov.type  d, a;

.type = { .b16, .b32, .b64, .b128 };
```

Description

Write scalar register *d* with the packed value of vector register *a*, or write vector register *d* with the unpacked values from scalar register *a*.

When destination operand *d* is a vector register, the sink symbol '_' may be used for one or more elements provided that at least one element is a scalar register.

For bit-size types, *mov* may be used to pack vector elements into a scalar register or unpack sub-fields of a scalar register into a vector. Both the overall size of the vector and the size of the scalar must match the size of the instruction type.

Semantics

```
// pack two 8-bit elements into .b16
d = a.x | (a.y << 8)
// pack four 8-bit elements into .b32
d = a.x | (a.y << 8) | (a.z << 16) | (a.w << 24)
// pack two 16-bit elements into .b32
d = a.x | (a.y << 16)
// pack four 16-bit elements into .b64
d = a.x | (a.y << 16) | (a.z << 32) | (a.w << 48)
// pack two 32-bit elements into .b64
d = a.x | (a.y << 32)
// pack four 32-bit elements into .b128
d = a.x | (a.y << 32) | (a.z << 64) | (a.w << 96)
// pack two 64-bit elements into .b128
d = a.x | (a.y << 64)

// unpack 8-bit elements from .b16
{ d.x, d.y } = { a[0..7], a[8..15] }
// unpack 8-bit elements from .b32
{ d.x, d.y, d.z, d.w }
    { a[0..7], a[8..15], a[16..23], a[24..31] }

// unpack 16-bit elements from .b32
{ d.x, d.y } = { a[0..15], a[16..31] }
// unpack 16-bit elements from .b64
{ d.x, d.y, d.z, d.w } =
    { a[0..15], a[16..31], a[32..47], a[48..63] }

// unpack 32-bit elements from .b64
{ d.x, d.y } = { a[0..31], a[32..63] }

// unpack 32-bit elements from .b128
{ d.x, d.y, d.z, d.w } =
    { a[0..31], a[32..63], a[64..95], a[96..127] }
// unpack 64-bit elements from .b128
{ d.x, d.y } = { a[0..63], a[64..127] }
```

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Support for `.b128` type introduced in PTX ISA version 8.3.

Target ISA Notes

Supported on all target architectures.

Support for `.b128` type requires `sm_70` or higher.

Examples

```
mov.b32 %r1, {a,b};           // a,b have type .u16
mov.b64 {lo,hi}, %x;          // %x is a double; lo,hi are .u32
mov.b32 %r1, {x,y,z,w};        // x,y,z,w have type .b8
mov.b32 {r,g,b,a}, %r1;        // r,g,b,a have type .u8
mov.b64 {%r1, _}, %x;          // %x is .b64, %r1 is .b32
mov.b128 {%b1, %b2}, %y;        // %y is .b128, %b1 and %b2 are .b64
mov.b128 %y, {%b1, %b2};        // %y is .b128, %b1 and %b2 are .b64
```

9.7.9.5 Data Movement and Conversion Instructions: `shfl` (deprecated)

`shfl` (deprecated)

Register data shuffle within threads of a warp.

Syntax

```
shfl.mode.b32 d[|p], a, b, c;

.mode = { .up, .down, .bfly, .idx };
```

Deprecation Note

The `shfl` instruction without a `.sync` qualifier is deprecated in PTX ISA version 6.0.

- Support for this instruction with `.target` lower than `sm_70` may be removed in a future PTX ISA version.

Removal Note

Support for `shfl` instruction without a `.sync` qualifier is removed in PTX ISA version 6.4 for `.target sm_70` or higher.

Description

Exchange register data between threads of a warp.

Each thread in the currently executing warp will compute a source lane index j based on input operands b and c and the *mode*. If the computed source lane index j is in range, the thread will copy the input operand a from lane j into its own destination register d ; otherwise, the thread will simply copy its own input a to destination d . The optional destination predicate p is set to `True` if the computed source lane is in range, and otherwise set to `False`.

Note that an out of range value of b may still result in a valid computed source lane index j . In this case, a data transfer occurs and the destination predicate p is `True`.

Note that results are undefined in divergent control flow within a warp, if an active thread sources a register from an inactive thread.

Operand b specifies a source lane or source lane offset, depending on the mode.

Operand c contains two packed values specifying a mask for logically splitting warps into sub-segments and an upper bound for clamping the source lane index.

Semantics

```
lane[4:0] = [Thread].laneid; // position of thread in warp
bval[4:0] = b[4:0];           // source lane or lane offset (0..31)
cval[4:0] = c[4:0];           // clamp value
mask[4:0] = c[12:8];

// get value of source register a if thread is active and
// guard predicate true, else unpredictable
if (isActive(Thread) && isGuardPredicateTrue(Thread)) {
    SourceA[lane] = a;
} else {
    // Value of SourceA[lane] is unpredictable for
    // inactive/predicated-off threads in warp
}
maxLane = (lane[4:0] & mask[4:0]) | (cval[4:0] & ~mask[4:0]);
minLane = (lane[4:0] & mask[4:0]);

switch (.mode) {
    case .up:    j = lane - bval; pval = (j >= maxLane); break;
    case .down:  j = lane + bval; pval = (j <= maxLane); break;
    case .bfly:  j = lane ^ bval; pval = (j <= maxLane); break;
    case .idx:   j = minLane | (bval[4:0] & ~mask[4:0]);
                 pval = (j <= maxLane); break;
}
if (!pval) j = lane; // copy from own lane
d = SourceA[j];      // copy input a from lane j
if (dest predicate selected)
    p = pval;
```

PTX ISA Notes

Introduced in PTX ISA version 3.0.

Deprecated in PTX ISA version 6.0 in favor of `shfl.sync`.

Not supported in PTX ISA version 6.4 for `.target sm_70` or higher.

Target ISA Notes

`shfl` requires `sm_30` or higher.

`shfl` is not supported on `sm_70` or higher starting PTX ISA version 6.4.

Examples

```
// Warp-level INCLUSIVE PLUS SCAN:
//
// Assumes input in following registers:
//   - Rx = sequence value for this thread
//
shfl.up.b32 Ry|p, Rx, 0x1, 0x0;
@p add.f32   Rx, Ry, Rx;
shfl.up.b32 Ry|p, Rx, 0x2, 0x0;
@p add.f32   Rx, Ry, Rx;
shfl.up.b32 Ry|p, Rx, 0x4, 0x0;
@p add.f32   Rx, Ry, Rx;
shfl.up.b32 Ry|p, Rx, 0x8, 0x0;
@p add.f32   Rx, Ry, Rx;
shfl.up.b32 Ry|p, Rx, 0x10, 0x0;
@p add.f32   Rx, Ry, Rx;

// Warp-level INCLUSIVE PLUS REVERSE-SCAN:
//
// Assumes input in following registers:
//   - Rx = sequence value for this thread
//
shfl.down.b32 Ry|p, Rx, 0x1, 0x1f;
@p add.f32   Rx, Ry, Rx;
shfl.down.b32 Ry|p, Rx, 0x2, 0x1f;
@p add.f32   Rx, Ry, Rx;
shfl.down.b32 Ry|p, Rx, 0x4, 0x1f;
@p add.f32   Rx, Ry, Rx;
shfl.down.b32 Ry|p, Rx, 0x8, 0x1f;
@p add.f32   Rx, Ry, Rx;
shfl.down.b32 Ry|p, Rx, 0x10, 0x1f;
@p add.f32   Rx, Ry, Rx;

// BUTTERFLY REDUCTION:
//
// Assumes input in following registers:
//   - Rx = sequence value for this thread
//
```

(continues on next page)

(continued from previous page)

```

shfl.bfly.b32  Ry, Rx, 0x10, 0x1f;    // no predicate dest
add.f32        Rx, Ry, Rx;
shfl.bfly.b32  Ry, Rx, 0x8,  0x1f;
add.f32        Rx, Ry, Rx;
shfl.bfly.b32  Ry, Rx, 0x4,  0x1f;
add.f32        Rx, Ry, Rx;
shfl.bfly.b32  Ry, Rx, 0x2,  0x1f;
add.f32        Rx, Ry, Rx;
shfl.bfly.b32  Ry, Rx, 0x1,  0x1f;
add.f32        Rx, Ry, Rx;
//
// All threads now hold sum in Rx

```

9.7.9.6 Data Movement and Conversion Instructions: shfl.sync

shfl.sync

Register data shuffle within threads of a warp.

Syntax

```

shfl.sync.mode.b32  d[|p], a, b, c, membermask;

.mode = { .up, .down, .bfly, .idx };

```

Description

Exchange register data between threads of a warp.

`shfl.sync` will cause executing thread to wait until all non-exited threads corresponding to `membermask` have executed `shfl.sync` with the same qualifiers and same `membermask` value before resuming execution.

Operand `membermask` specifies a 32-bit integer which is a mask indicating threads participating in barrier where the bit position corresponds to thread's `laneid`.

`shfl.sync` exchanges register data between threads in `membermask`.

Each thread in the currently executing warp will compute a source lane index j based on input operands `b` and `c` and the `mode`. If the computed source lane index j is in range, the thread will copy the input operand `a` from lane j into its own destination register `d`; otherwise, the thread will simply copy its own input `a` to destination `d`. The optional destination predicate `p` is set to `True` if the computed source lane is in range, and otherwise set to `False`.

Note that an out of range value of `b` may still result in a valid computed source lane index j . In this case, a data transfer occurs and the destination predicate `p` is `True`.

Note that results are undefined if a thread sources a register from an inactive thread or a thread that is not in `membermask`.

Operand `b` specifies a source lane or source lane offset, depending on the mode.

Operand `c` contains two packed values specifying a mask for logically splitting warps into sub-segments and an upper bound for clamping the source lane index.

The behavior of `shfl.sync` is undefined if the executing thread is not in the `membermask`.

Note: For `.target sm_6x` or below, all threads in `membermask` must execute the same `shfl.sync` instruction in convergence, and only threads belonging to some `membermask` can be active when the `shfl.sync` instruction is executed. Otherwise, the behavior is undefined.

Semantics

```
// wait for all threads in membermask to arrive
wait_for_specified_threads(membermask);

lane[4:0] = [Thread].laneid; // position of thread in warp
bval[4:0] = b[4:0];          // source lane or lane offset (0..31)
cval[4:0] = c[4:0];          // clamp value
segmask[4:0] = c[12:8];

// get value of source register a if thread is active and
// guard predicate true, else unpredictable
if (isActive(Thread) && isGuardPredicateTrue(Thread)) {
    SourceA[lane] = a;
} else {
    // Value of SourceA[lane] is unpredictable for
    // inactive/predicated-off threads in warp
}
maxLane = (lane[4:0] & segmask[4:0]) | (cval[4:0] & ~segmask[4:0]);
minLane = (lane[4:0] & segmask[4:0]);

switch (.mode) {
    case .up:    j = lane - bval; pval = (j >= maxLane); break;
    case .down:  j = lane + bval; pval = (j <= maxLane); break;
    case .bfly:  j = lane ^ bval; pval = (j <= maxLane); break;
    case .idx:   j = minLane | (bval[4:0] & ~segmask[4:0]);
                 pval = (j <= maxLane); break;
}
if (!pval) j = lane; // copy from own lane
d = SourceA[j];      // copy input a from lane j
if (dest predicate selected)
    p = pval;
```

PTX ISA Notes

Introduced in PTX ISA version 6.0.

Target ISA Notes

Requires sm_30 or higher.

Examples

```
shfl.sync.up.b32 Ry|p, Rx, 0x1, 0x0, 0xffffffff;
```

9.7.9.7 Data Movement and Conversion Instructions: prmt

prmt

Permute bytes from register pair.

Syntax

```
prmt.b32{.mode} d, a, b, c;
.mode = { .f4e, .b4e, .rc8, .ec1, .ecr, .rc16 };
```

Description

Pick four arbitrary bytes from two 32-bit registers, and reassemble them into a 32-bit destination register.

In the generic form (no mode specified), the permute control consists of four 4-bit selection values. The bytes in the two source registers are numbered from 0 to 7: {b, a} = {{b7, b6, b5, b4}, {b3, b2, b1, b0}}. For each byte in the target register, a 4-bit selection value is defined.

The 3 lsbs of the selection value specify which of the 8 source bytes should be moved into the target position. The msb defines if the byte value should be copied, or if the sign (msb of the byte) should be replicated over all 8 bits of the target position (sign extend of the byte value); msb=0 means copy the literal value; msb=1 means replicate the sign. Note that the sign extension is only performed as part of generic form.

Thus, the four 4-bit values fully specify an arbitrary byte permute, as a 16b permute code.

default mode	d.b3 source select	d.b2 source select	d.b1 source select	d.b0 source select
index	c[15:12]	c[11:8]	c[7:4]	c[3:0]

The more specialized form of the permute control uses the two lsb's of operand c (which is typically an address pointer) to control the byte extraction.

mode	selector c[1:0]	d.b3 source	d.b2 source	d.b1 source	d.b0 source
f4e (forward 4 extract)	0	3	2	1	0
	1	4	3	2	1
	2	5	4	3	2
	3	6	5	4	3
b4e (backward 4 extract)	0	5	6	7	0
	1	6	7	0	1
	2	7	0	1	2
	3	0	1	2	3
rc8 (replicate 8)	0	0	0	0	0
	1	1	1	1	1
	2	2	2	2	2
	3	3	3	3	3
ec1 (edge clamp left)	0	3	2	1	0
	1	3	2	1	1
	2	3	2	2	2
	3	3	3	3	3
ecr (edge clamp right)	0	0	0	0	0
	1	1	1	1	0
	2	2	2	1	0
	3	3	2	1	0
rc16 (replicate 16)	0	1	0	1	0
	1	3	2	3	2
	2	1	0	1	0
	3	3	2	3	2

Semantics

```
tmp64 = (b<<32) | a; // create 8 byte source
```

```
if ( ! mode ) {
    ctl[0] = (c >> 0) & 0xf;
    ctl[1] = (c >> 4) & 0xf;
    ctl[2] = (c >> 8) & 0xf;
    ctl[3] = (c >> 12) & 0xf;
} else {
```

(continues on next page)

(continued from previous page)

```

    ctl[0] = ctl[1] = ctl[2] = ctl[3] = (c >> 0) & 0x3;
}

tmp[07:00] = ReadByte( mode, ctl[0], tmp64 );
tmp[15:08] = ReadByte( mode, ctl[1], tmp64 );
tmp[23:16] = ReadByte( mode, ctl[2], tmp64 );
tmp[31:24] = ReadByte( mode, ctl[3], tmp64 );

```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

prmt requires sm_20 or higher.

Examples

```

prmt.b32      r1, r2, r3, r4;
prmt.b32.f4e  r1, r2, r3, r4;

```

9.7.9.8 Data Movement and Conversion Instructions: ld

ld

Load a register variable from an addressable state space variable.

Syntax

```

ld{.weak}{.ss}{.cop}{.level::cache_hint}{.level::prefetch_size}{.vec}.type  d, [a]{.
↳unified}{, cache-policy};

ld{.weak}{.ss}{.level1::eviction_priority}{.level2::eviction_priority}{.level::cache_
↳hint}{.level::prefetch_size}{.vec}.type  d, [a]{.unified}{, cache-policy};

ld.volatile{.ss}{.level::prefetch_size}{.vec}.type  d, [a];

ld.relaxed.scope{.ss}{.level1::eviction_priority}{.level2::eviction_priority}{.
↳level::cache_hint}{.level::prefetch_size}{.vec}.type  d, [a]{, cache-policy};

ld.acquire.scope{.ss}{.level1::eviction_priority}{.level2::eviction_priority}{.
↳level::cache_hint}{.level::prefetch_size}{.vec}.type  d, [a]{, cache-policy};

ld.mmio.relaxed.sys{.global}.type  d, [a];

.ss =
↳shared{::cta, ::cluster} };
.cop =
{ .ca, .cg, .cs, .lu, .cv };

```

(continues on next page)

(continued from previous page)

```

.level1::eviction_priority = { .L1::evict_normal, .L1::evict_unchanged,
                               .L1::evict_first, .L1::evict_last, .L1::no_allocate };
.level2::eviction_priority = { .L2::evict_normal, .L2::evict_first, .L2::evict_last };
.level::cache_hint =        { .L2::cache_hint };
.level::prefetch_size =     { .L2::64B, .L2::128B, .L2::256B };
.scope =                    { .cta, .cluster, .gpu, .sys };
.vec =                      { .v2, .v4, .v8 };
.type =                     { .b8, .b16, .b32, .b64, .b128,
                               .u8, .u16, .u32, .u64,
                               .s8, .s16, .s32, .s64,
                               .f32, .f64 };

```

Description

Load register variable `d` from the location specified by the source address operand `a` in specified state space. If no state space is given, perform the load using [Generic Addressing](#).

If no sub-qualifier is specified with `.shared` state space, then `::cta` is assumed by default.

Supported addressing modes for operand `a` and alignment requirements are described in [Addresses as Operands](#)

If no sub-qualifier is specified with `.param` state space, then:

- ▶ `::func` is assumed when access is inside a device function.
- ▶ `::entry` is assumed when accessing kernel function parameters from entry function. Otherwise, when accessing device function parameters or any other `.param` variables from entry function `::func` is assumed by default.

For `ld.param::entry` instruction, operand `a` must be a kernel parameter address, otherwise behavior is undefined. For `ld.param::func` instruction, operand `a` must be a device function parameter address, otherwise behavior is undefined.

Instruction `ld.param{::func}` used for reading value returned from device function call cannot be predicated. See [Parameter State Space](#) and [Function Declarations and Definitions](#) for descriptions of the proper use of `ld.param`.

The `.relaxed` and `.acquire` qualifiers indicate memory synchronization as described in the [Memory Consistency Model](#). The `.scope` qualifier indicates the set of threads with which an `ld.relaxed` or `ld.acquire` instruction can directly synchronize¹. The `.weak` qualifier indicates a memory instruction with no synchronization. The effects of this instruction become visible to other threads only when synchronization is established by other means.

The semantic details of `.mmio` qualifier are described in the [Memory Consistency Model](#). Only `.sys` thread scope is valid for `ld.mmio` operation. The qualifiers `.mmio` and `.relaxed` must be specified together.

The semantic details of `.volatile` qualifier are described in the [Memory Consistency Model](#).

The `.weak`, `.volatile`, `.relaxed` and `.acquire` qualifiers are mutually exclusive. When none of these is specified, the `.weak` qualifier is assumed by default.

`.relaxed` and `.acquire`:

- ▶ May be used with `.global`, `.shared` spaces, or with generic addressing where the address points to `.global` or `.shared` space.
- ▶ Cache operations are not allowed.

`.volatile:`

- ▶ May be used with `.global`, `.shared`, `.local` spaces, or with generic addressing where the address points to `.global`, `.shared`, or `.local` space.
- ▶ Cache operations are not allowed.

`.mmio:`

- ▶ May be used only with `.global` space or with generic addressing where the address points to `.global` space.

The optional qualifier `.unified` must be specified on operand `a` if `a` is the address of a variable declared with `.unified` attribute as described in *Variable and Function Attribute Directive: `.attribute`*.

The `.v8 (.vec)` qualifier is supported if:

- ▶ `.type` is `.b32` or `.s32` or `.u32` or `.f32` AND
- ▶ State space is `.global` or with generic addressing where address points to `.global` state space

The `.v4 (.vec)` qualifier with type `.b64` or `.s64` or `.u64` or `.f64` is supported if:

- ▶ State space is `.global` or with generic addressing where address points to `.global` state space

Qualifiers `.level1::eviction_priority` and `.level2::eviction_priority` specify the eviction policy for L1 and L2 cache respectively which may be applied during memory access.

Qualifier `.level2::eviction_priority` is supported if:

- ▶ `.vec` is `.v8` and `.type` is `.b32` or `.s32` or `.u32` or `.f32`
 - ▶ AND Operand `d` is vector of 8 registers with type specified with `.type`
- ▶ OR `.vec` is `.v4` and `.type` is `.b64` or `.s64` or `.u64` or `.f64`
 - ▶ AND Operand `d` is vector of 4 registers with type specified with `.type`

Optionally, sink symbol `'_'` can be used in vector expression `d` when:

- ▶ `.vec` is `.v8` and `.type` is `.b32` or `.s32` or `.u32` or `.f32` OR
- ▶ `.vec` is `.v4` and `.type` is `.b64` or `.s64` or `.u64` or `.f64`

which indicates that data from corresponding memory location is not read.

The `.level::prefetch_size` qualifier is a hint to fetch additional data of the specified size into the respective cache level. The sub-qualifier `prefetch_size` can be set to either of 64B, 128B, 256B thereby allowing the prefetch size to be 64 Bytes, 128 Bytes or 256 Bytes respectively.

The qualifier `.level::prefetch_size` may only be used with `.global` state space and with generic addressing where the address points to `.global` state space. If the generic address does not fall within the address window of the global memory, then the prefetching behavior is undefined.

The `.level::prefetch_size` qualifier is treated as a performance hint only.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

The qualifiers `.unified` and `.level::cache_hint` are only supported for `.global` state space and for generic addressing where the address points to the `.global` state space.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

¹ This synchronization is further extended to other threads through the transitive nature of *causality order*, as described in the memory consistency model.

Semantics

```
d = a;           // named variable a
d = *(&a+immOff) // variable-plus-offset
d = *a;          // register
d = *(a+immOff); // register-plus-offset
d = *(immAddr);  // immediate address
```

Notes

Destination `d` must be in the `.reg` state space.

A destination register wider than the specified type may be used. The value loaded is sign-extended to the destination register width for signed integers, and is zero-extended to the destination register width for unsigned and bit-size types. See [Table 28](#) for a description of these relaxed type-checking rules.

`.f16` data may be loaded using `ld.b16`, and then converted to `.f32` or `.f64` using `cvt` or can be used in half precision floating point instructions.

`.f16x2` data may be loaded using `ld.b32` and then used in half precision floating point instructions.

PTX ISA Notes

`ld` introduced in PTX ISA version 1.0. `ld.volatile` introduced in PTX ISA version 1.1.

Generic addressing and cache operations introduced in PTX ISA version 2.0.

Support for scope qualifier, `.relaxed`, `.acquire`, `.weak` qualifiers introduced in PTX ISA version 6.0.

Support for generic addressing of `.const` space added in PTX ISA version 3.1.

Support for `.level1::eviction_priority`, `.level::prefetch_size` and `.level::cache_hint` qualifiers introduced in PTX ISA version 7.4.

Support for `.cluster` scope qualifier introduced in PTX ISA version 7.8.

Support for `::cta` and `::cluster` sub-qualifiers introduced in PTX ISA version 7.8.

Support for `.unified` qualifier introduced in PTX ISA version 8.0.

Support for `.mmio` qualifier introduced in PTX ISA version 8.2.

Support for `::entry` and `::func` sub-qualifiers on `.param` space introduced in PTX ISA version 8.3.

Support for `.b128` type introduced in PTX ISA version 8.3.

Support for `.sys` scope with `.b128` type introduced in PTX ISA version 8.4.

Support for `.level2::eviction_priority` qualifier and `.v8.b32/.v4.b64` introduced in PTX ISA version 8.8.

Support for `.volatile` qualifier with `.local` state space introduced in PTX ISA version 9.1.

Target ISA Notes

`ld.f64` requires `sm_13` or higher.

Support for scope qualifier, `.relaxed`, `.acquire`, `.weak` qualifiers require `sm_70` or higher.

Generic addressing requires `sm_20` or higher.

Cache operations require `sm_20` or higher.

Support for `.level::eviction_priority` qualifier requires `sm_70` or higher.

Support for `.level::prefetch_size` qualifier requires `sm_75` or higher.

Support for `.L2::256B` and `.L2::cache_hint` qualifiers requires `sm_80` or higher.

Support for `.cluster` scope qualifier requires `sm_90` or higher.

Sub-qualifier `::cta` requires `sm_30` or higher.

Sub-qualifier `::cluster` requires `sm_90` or higher.

Support for `.unified` qualifier requires `sm_90` or higher.

Support for `.mmio` qualifier requires `sm_70` or higher.

Support for `.b128` type requires `sm_70` or higher.

Support for `.level2::eviction_priority` qualifier and `.v8.b32/.v4.b64` require `sm_100` or higher.

Examples

```
ld.global.f32    d,[a];
ld.shared.v4.b32 Q,[p];
ld.const.s32     d,[p+4];
ld.local.b32     x,[p+-8]; // negative offset
ld.local.b64     x,[240]; // immediate address

ld.global.b16     %r,[fs]; // load .f16 data into 32-bit reg
cvt.f32.f16      %r,%r;    // up-convert f16 data to f32

ld.global.b32     %r0,[fs]; // load .f16x2 data in 32-bit reg
ld.global.b32     %r1,[fs + 4]; // load .f16x2 data in 32-bit reg
add.rn.f16x2      %d0,%r0,%r1; // addition of f16x2 data
ld.global.relaxed.gpu.u32 %r0,[gbl];
ld.shared.acquire.gpu.u32 %r1,[sh];
ld.global.relaxed.cluster.u32 %r2,[gbl];
ld.shared::cta.acquire.gpu.u32 %r2,[sh + 4];
ld.shared::cluster.u32 %r3,[sh + 8];
ld.global.mmio.relaxed.sys.u32 %r3,[gbl];
ld.local.volatile.u32 %r4,[lcl];

ld.global.f32     d,[ugbl].unified;
ld.b32            %r0,[%r1].unified;

ld.global.L1::evict_last.u32 d,[p];

ld.global.L2::64B.b32 %r0,[gbl]; // Prefetch 64B to L2
ld.L2::128B.f64     %r1,[gbl]; // Prefetch 128B to L2
```

(continues on next page)

(continued from previous page)

```
ld.global.L2::256B.f64 %r2, [gb1]; // Prefetch 256B to L2

createpolicy.fractional.L2::evict_last.L2::evict_unchanged.b64 cache-policy, 1;
ld.global.L2::cache_hint.b64 x, [p], cache-policy;
ld.param::entry.b32 %rp1, [kparam1];

ld.global.b128 %r0, [gb1]; // 128-bit load

// 256-bit load
ld.global.L2::evict_last.v8.f32 { %reg0, _, %reg2, %reg3, %reg4, %reg5, %reg6, %reg7},
↳ [addr];
ld.global.L2::evict_last.L1::evict_last.v4.u64 { %reg0, %reg1, %reg2, %reg3}, [addr];
```

9.7.9.9 Data Movement and Conversion Instructions: `ld.global.nc`

`ld.global.nc`

Load a register variable from global state space via non-coherent cache.

Syntax

```
ld.global{.cop}.nc{.level::cache_hint}{.level::prefetch_size}.type          d,
↳ [a]{, cache-policy};
ld.global{.cop}.nc{.level::cache_hint}{.level::prefetch_size}.vec.type      d,
↳ [a]{, cache-policy};

ld.global.nc{.level1::eviction_priority}{.level2::eviction_priority}{.level::cache_
↳ hint}{.level::prefetch_size}.type      d, [a]{, cache-policy};
ld.global.nc{.level1::eviction_priority}{.level2::eviction_priority}{.level::cache_
↳ hint}{.level::prefetch_size}.vec.type  d, [a]{, cache-policy};

.cop = { .ca, .cg, .cs }; // cache operation
.level1::eviction_priority = { .L1::evict_normal, .L1::evict_unchanged,
                              .L1::evict_first, .L1::evict_last, .L1::no_allocate};
.level2::eviction_priority = { .L2::evict_normal, .L2::evict_first, .L2::evict_last};
.level::cache_hint = { .L2::cache_hint };
.level::prefetch_size = { .L2::64B, .L2::128B, .L2::256B }
.vec = { .v2, .v4, .v8 };
.type = { .b8, .b16, .b32, .b64, .b128,
          .u8, .u16, .u32, .u64,
          .s8, .s16, .s32, .s64,
          .f32, .f64 };
```

Description

Load register variable `d` from the location specified by the source address operand `a` in the global state space, and optionally cache in non-coherent read-only cache.

Note: On some architectures, the texture cache is larger, has higher bandwidth, and longer latency than the global memory cache. For applications with sufficient parallelism to cover the longer latency, `ld.global.nc` should offer better performance than `ld.global` on such architectures.

The address operand `a` shall contain a global address. Supported addressing modes for operand `a` and alignment requirements are described in [Addresses as Operands](#).

The `.v8 (.vec)` qualifier is supported if:

- ▶ `.type` is `.b32`, `.s32`, `.u32`, or `.f32` AND
- ▶ State space is `.global` or with generic addressing where address points to `.global` state space

The `.v4 (.vec)` qualifier with type `.b64` or `.s64` or `.u64` or `.f64` is supported if:

- ▶ State space is `.global` or with generic addressing where address points to `.global` state space

Qualifiers `.level1::eviction_priority` and `.level2::eviction_priority` specify the eviction policy for L1 and L2 cache respectively which may be applied during memory access.

Qualifier `.level2::eviction_priority` is supported if:

- ▶ `.vec` is `.v8` and `.type` is `.b32` or `.s32` or `.u32` or `.f32`
 - ▶ AND Operand `d` is vector of 8 registers with type specified with `.type`
- ▶ OR `.vec` is `.v4` and `.type` is `.b64` or `.s64` or `.u64` or `.f64`
 - ▶ AND Operand `d` is vector of 4 registers with type specified with `.type`

Optionally, sink symbol `'_'` can be used in vector expression `d` when:

- ▶ `.vec` is `.v8` and `.type` is `.b32` or `.s32` or `.u32` or `.f32` OR
- ▶ `.vec` is `.v4` and `.type` is `.b64` or `.s64` or `.u64` or `.f64`

which indicates that data from corresponding memory location is not read.

The `.level::prefetch_size` qualifier is a hint to fetch additional data of the specified size into the respective cache level. The sub-qualifier `prefetch_size` can be set to either of 64B, 128B, 256B thereby allowing the prefetch size to be 64 Bytes, 128 Bytes or 256 Bytes respectively.

The `.level::prefetch_size` qualifier is treated as a performance hint only.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

Semantics

```
d = a;           // named variable a
d = *(&a+immOff) // variable-plus-offset
d = *a;          // register
d = *(a+immOff); // register-plus-offset
d = *(immAddr);  // immediate address
```

Notes

Destination `d` must be in the `.reg` state space.

A destination register wider than the specified type may be used. The value loaded is sign-extended to the destination register width for signed integers, and is zero-extended to the destination register width for unsigned and bit-size types.

`.f16` data may be loaded using `ld.b16`, and then converted to `.f32` or `.f64` using `cvt`.

PTX ISA Notes

Introduced in PTX ISA version 3.1.

Support for `.level::eviction_priority`, `.level::prefetch_size` and `.level::cache_hint` qualifiers introduced in PTX ISA version 7.4.

Support for `.b128` type introduced in PTX ISA version 8.3.

Support for `.level2::eviction_priority` qualifier and `.v8.b32/.v4.b64` introduced in PTX ISA version 8.8.

Target ISA Notes

Requires `sm_32` or higher.

Support for `.level1::eviction_priority` qualifier requires `sm_70` or higher.

Support for `.level::prefetch_size` qualifier requires `sm_75` or higher.

Support for `.level::cache_hint` qualifier requires `sm_80` or higher.

Support for `.b128` type requires `sm_70` or higher.

Support for `.level2::eviction_priority` qualifier and `.v8.b32/.v4.b64` require `sm_100` or higher.

Examples

```
ld.global.nc.f32      d, [a];
ld.global.nc.L1::evict_last.u32 d, [a];

createpolicy.fractional.L2::evict_last.b64 cache-policy, 0.5;
ld.global.nc.L2::cache_hint.f32 d, [a], cache-policy;

ld.global.nc.L2::64B.b32      d, [a];      // Prefetch 64B to L2
ld.global.nc.L2::256B.f64     d, [a];      // Prefetch 256B to L2

ld.global.nc.b128        d, [a];

ld.global.nc.L2::evict_first.v4.f64 {%reg0, %reg1, %reg2, %reg3}. [a]; // 256-bit load
```

9.7.9.10 Data Movement and Conversion Instructions: `ldu`

`ldu`

Load read-only data from an address that is common across threads in the warp.

Syntax

```
ldu{.ss}.type      d, [a];      // load from address
ldu{.ss}.vec.type   d, [a];      // vec load from address

.ss   = { .global };           // state space
.vec  = { .v2, .v4 };
.type = { .b8, .b16, .b32, .b64, .b128,
          .u8, .u16, .u32, .u64,
          .s8, .s16, .s32, .s64,
          .f32, .f64 };;
```

Description

Load *read-only* data into register variable `d` from the location specified by the source address operand `a` in the global state space, where the address is guaranteed to be the same across all threads in the warp. If no state space is given, perform the load using *Generic Addressing*.

Supported addressing modes for operand `a` and alignment requirements are described in *Addresses as Operands*.

Semantics

```
d = a;           // named variable a
d = *(&a+immOff) // variable-plus-offset
d = *a;          // register
d = *(a+immOff); // register-plus-offset
d = *(immAddr);  // immediate address
```

Notes

Destination `d` must be in the `.reg` state space.

A destination register wider than the specified type may be used. The value loaded is sign-extended to the destination register width for signed integers, and is zero-extended to the destination register width for unsigned and bit-size types. See [Table 28](#) for a description of these relaxed type-checking rules.

`.f16` data may be loaded using `ldu.b16`, and then converted to `.f32` or `.f64` using `cvt` or can be used in half precision floating point instructions.

`.f16x2` data may be loaded using `ldu.b32` and then used in half precision floating point instructions.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Support for `.b128` type introduced in PTX ISA version 8.3.

Target ISA Notes

`ldu.f64` requires `sm_13` or higher.

Support for `.b128` type requires `sm_70` or higher.

Examples

```
ldu.global.f32    d,[a];
ldu.global.b32    d,[p+4];
ldu.global.v4.f32 Q,[p];
ldu.global.b128   d,[a];
```

9.7.9.11 Data Movement and Conversion Instructions: `st`

`st`

Store data to an addressable state space variable.

Syntax

```

st{.weak}{.ss}{.cop}{.level::cache_hint}{.vec}.type    [a], b{, cache-policy};
st{.weak}{.ss}{.level1::eviction_priority}{.level2::eviction_priority}{.level::cache_
↪hint}{.vec}.type                                     [a], b{, cache-policy};
st.volatile{.ss}{.vec}.type                           [a], b;
st.relaxed.scope{.ss}{.level1::eviction_priority}{.level2::eviction_priority}{.
↪level::cache_hint}{.vec}.type                       [a], b{, cache-policy};
st.release.scope{.ss}{.level1::eviction_priority}{.level2::eviction_priority}{.
↪level::cache_hint}{.vec}.type                       [a], b{, cache-policy};
st.mmio.relaxed.sys{.global}.type                     [a], b;

.ss = { .global, .local, .param{::func}, .shared{::cta,
↪::cluster} };
.level1::eviction_priority = { .L1::evict_normal, .L1::evict_unchanged,
                              .L1::evict_first, .L1::evict_last, .L1::no_allocate };
.level2::eviction_priority = { .L2::evict_normal, .L2::evict_first, .L2::evict_last };
.level::cache_hint = { .L2::cache_hint };
.cop = { .wb, .cg, .cs, .wt };
.sem = { .relaxed, .release };
.scope = { .cta, .cluster, .gpu, .sys };
.vec = { .v2, .v4, .v8 };
.type = { .b8, .b16, .b32, .b64, .b128,
          .u8, .u16, .u32, .u64,
          .s8, .s16, .s32, .s64,
          .f32, .f64 };

```

Description

Store the value of operand *b* in the location specified by the destination address operand *a* in specified state space. If no state space is given, perform the store using *Generic Addressing*. Stores to const memory are illegal.

If no sub-qualifier is specified with `.shared` state space, then `::cta` is assumed by default.

Supported addressing modes for operand *a* and alignment requirements are described in *Addresses as Operands*.

If `.param` is specified without any sub-qualifiers then it defaults to `.param::func`.

Instruction `st.param{::func}` used for passing arguments to device function cannot be predicated. See *Parameter State Space* and *Function Declarations and Definitions* for descriptions of the proper use of `st.param`.

The qualifiers `.relaxed` and `.release` indicate memory synchronization as described in the *Memory Consistency Model*. The `.scope` qualifier indicates the set of threads with which an `st.relaxed` or `st.release` instruction can directly synchronize¹. The `.weak` qualifier indicates a memory instruction with no synchronization. The effects of this instruction become visible to other threads only when synchronization is established by other means.

The semantic details of `.mmio` qualifier are described in the *Memory Consistency Model*. Only `.sys` thread scope is valid for `st.mmio` operation. The qualifiers `.mmio` and `.relaxed` must be specified together.

The semantic details of `.volatile` qualifier are described in the *Memory Consistency Model*.

The `.weak`, `.volatile`, `.relaxed` and `.release` qualifiers are mutually exclusive. When none of these is specified, the `.weak` qualifier is assumed by default.

`.relaxed` and `.release`:

- ▶ May be used with `.global`, `.shared` spaces or with generic addressing where the address points to `.global` or `.shared` space.
- ▶ Cache operations are not allowed.

`.volatile`:

- ▶ May be used with `.global`, `.shared`, `.local` spaces or with generic addressing where the address points to `.global`, `.shared`, or `.local` space.
- ▶ Cache operations are not allowed.

`.mmio`:

- ▶ May be used only with `.global` space or with generic addressing where the address points to `.global` space.

The `.v8` (`.vec`) qualifier is supported if:

- ▶ `.type` is `.b32`, `.s32`, `.u32`, or `.f32` AND
- ▶ State space is `.global` or with generic addressing where address points to `.global` state space

The `.v4` (`.vec`) qualifier with type `.b64` or `.s64` or `.u64` or `.f64` is supported if:

- ▶ State space is `.global` or with generic addressing where address points to `.global` state space

Qualifiers `.level1::eviction_priority` and `.level2::eviction_priority` specify the eviction policy for L1 and L2 cache respectively which may be applied during memory access.

Qualifier `.level2::eviction_priority` is supported if:

- ▶ `.vec` is `.v8` and `.type` is `.b32` or `.s32` or `.u32` or `.f32`
 - ▶ AND Operand `d` is vector of 8 registers with type specified with `.type`
- ▶ OR `.vec` is `.v4` and `.type` is `.b64` or `.s64` or `.u64` or `.f64`
 - ▶ AND Operand `d` is vector of 4 registers with type specified with `.type`

Optionally, sink symbol `'_'` can be used in vector expression `b` when:

- ▶ `.vec` is `.v8` and `.type` is `.b32` or `.s32` or `.u32` or `.f32` OR
- ▶ `.vec` is `.v4` and `.type` is `.b64` or `.s64` or `.u64` or `.f64`

which indicates that no data is being written at the corresponding destination address.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

The qualifier `.level::cache_hint` is only supported for `.global` state space and for generic addressing where the address points to the `.global` state space.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

¹ This synchronization is further extended to other threads through the transitive nature of *causality order*, as described in the memory consistency model.

Semantics

```
d = a;           // named variable d
*(&a+immOffset) = b; // variable-plus-offset
*a = b;         // register
*(a+immOffset) = b; // register-plus-offset
*(immAddr) = b;   // immediate address
```

Notes

Operand `b` must be in the `.reg` state space.

A source register wider than the specified type may be used. The lower `n` bits corresponding to the instruction-type width are stored to memory. See [Table 27](#) for a description of these relaxed type-checking rules.

`.f16` data resulting from a `cvt` instruction may be stored using `st.b16`.

`.f16x2` data may be stored using `st.b32`.

PTX ISA Notes

`st` introduced in PTX ISA version 1.0. `st.volatile` introduced in PTX ISA version 1.1.

Generic addressing and cache operations introduced in PTX ISA version 2.0.

Support for scope qualifier, `.relaxed`, `.release`, `.weak` qualifiers introduced in PTX ISA version 6.0.

Support for `.level1::eviction_priority` and `.level::cache_hint` qualifiers introduced in PTX ISA version 7.4.

Support for `.cluster` scope qualifier introduced in PTX ISA version 7.8.

Support for `::cta` and `::cluster` sub-qualifiers introduced in PTX ISA version 7.8.

Support for `.mmio` qualifier introduced in PTX ISA version 8.2.

Support for `::func` sub-qualifier on `.param` space introduced in PTX ISA version 8.3.

Support for `.b128` type introduced in PTX ISA version 8.3.

Support for `.sys` scope with `.b128` type introduced in PTX ISA version 8.4.

Support for `.level2::eviction_priority` qualifier and `.v8.b32/.v4.b64` introduced in PTX ISA version 8.8.

Support for `.volatile` qualifier with `.local` state space introduced in PTX ISA version 9.1.

Target ISA Notes

st.f64 requires sm_13 or higher.

Support for scope qualifier, .relaxed, .release, .weak qualifiers require sm_70 or higher.

Generic addressing requires sm_20 or higher.

Cache operations require sm_20 or higher.

Support for .level1::eviction_priority qualifier requires sm_70 or higher.

Support for .level::cache_hint qualifier requires sm_80 or higher.

Support for .cluster scope qualifier requires sm_90 or higher.

Sub-qualifier ::cta requires sm_30 or higher.

Sub-qualifier ::cluster requires sm_90 or higher.

Support for .mmio qualifier requires sm_70 or higher.

Support for .b128 type requires sm_70 or higher.

Support for .level2::eviction_priority qualifier and .v8.b32/.v4.b64 require sm_100 or higher.

Examples

```
st.global.f32    [a],b;
st.local.b32     [q+4],a;
st.global.v4.s32 [p],Q;
st.local.b32     [q+-8],a; // negative offset
st.local.s32     [100],r7; // immediate address

cvt.f16.f32      %r,%r;    // %r is 32-bit register
st.b16           [fs],%r;  // store lower
st.global.relaxed.sys.u32 [gb1], %r0;
st.shared.release.cta.u32 [sh], %r1;
st.global.relaxed.cluster.u32 [gb1], %r2;
st.shared::cta.release.cta.u32 [sh + 4], %r1;
st.shared::cluster.u32 [sh + 8], %r1;
st.global.mmio.relaxed.sys.u32 [gb1], %r1;
st.local.volatile.u32 [lcl], %r2;

st.global.L1::no_allocate.f32 [p], a;

createpolicy.fractional.L2::evict_last.b64 cache-policy, 0.25;
st.global.L2::cache_hint.b32 [a], b, cache-policy;

st.param::func.b64 [param1], %rp1;

st.global.b128   [a], b; // 128-bit store

// 256-bit store
st.global.L2::evict_last.v8.f32 [addr], { %reg0, _, %reg2, %reg3, %reg4, %reg5, %reg6,
↪ %reg7};
```

9.7.9.12 Data Movement and Conversion Instructions: `st.async`

`st.async`

Asynchronous store operation.

Syntax

```
st.async{.weak}{.ss}.completion_mechanism{.vec}.type [a], b, [mbar];
st.async{.scope}{.ss}.completion_mechanism{.vec}.type [a], b, [mbar];

.scope =          { .cluster };
.ss    =          { .shared::cluster };
.type  =          { .b32, .b64,
                    .u32, .u64,
                    .s32, .s64,
                    .f32, .f64 };

.vec    =          { .v2, .v4 };
.completion_mechanism = { .mbarrier::complete_tx::bytes };

st.async{.mmio}.sem.scope{.ss}.type [a], b;

.sem    =          { .release };
.scope  =          { .gpu, .sys };
.ss     =          { .global };
.type   =          { .b8, .b16, .b32, .b64,
                    .u8, .u16, .u32, .u64,
                    .s8, .s16, .s32, .s64,
                    .f32, .f64 };
```

Description

`st.async` is a non-blocking instruction which initiates an asynchronous store operation that stores the value specified by source operand `b` to the destination memory location specified by operand `a`.

Operands

- ▶ `a` is a destination address, and must be either a register, or of the form `register + immOff`, as described in [Addresses as Operands](#).
- ▶ `b` is a source value, of the type indicated by qualifier `.type`.
- ▶ `mbar` is an `mbarrier` object address.

Qualifiers

- ▶ `.mmio` indicates whether this is an *mmio Operation*.
- ▶ `.sem` specifies the memory ordering semantics as described in the *Memory Consistency Model*.
 - ▶ If `.sem` is not specified, it defaults to `.weak`.
- ▶ `.scope` specifies the set of threads with which this instruction can directly synchronize.
- ▶ `.ss` specifies the state space of the destination operand `a` and the `mbarrier` operand `mbar`.
 - ▶ If `.ss` is not specified, *Generic Addressing* is used.
- ▶ `.completion_mechanism` specifies the mechanism for observing the completion of the asynchronous operation.
 - ▶ When `.completion_mechanism` is `.mbarrier::complete_tx::bytes`: upon completion of the asynchronous operation, a *complete-tx* operation will be performed on the `mbarrier` object specified by the operand `mbar`, with `completeCount` argument equal to the amount of data stored in `bytes`. This instruction accesses its `mbarrier` operand using generic-proxy.
- ▶ `.type` specifies the type of the source operand `b`.

Conditions

When `.sem` is `.weak`:

- ▶ This is a weak store to shared memory, which signals its completion through an `mbarrier` object.
- ▶ The store operation is treated as a weak memory operation.
- ▶ The `complete-tx` operation on the `mbarrier` has `.release` semantics at `.cluster` scope.
- ▶ Requires:
 - ▶ The shared memory addresses of destination operand `a` and the *mbarrier object* `mbar` belong to the same CTA within the same cluster as the executing thread.
 - ▶ The number of CTAs within the cluster is strictly greater than one; `%cluster_nctarank > 1` is true.

Otherwise, the behavior is undefined.

- ▶ `.mmio` must not be specified.
- ▶ If `.ss` is specified, it must be `.shared::cluster`.
- ▶ If `.ss` is not specified, generic addressing is used for operands `a` and `mbar`. If the generic addresses specified do not fall within the address window of `.shared::cluster` state space, the behavior is undefined.
- ▶ `.completion_mechanism` must be specified and must be `.mbarrier::complete_tx::bytes`.

When `.sem` is `.release`:

- ▶ This is a release store to global memory.
- ▶ The store operation is a strong memory operation with `.release` semantics at the scope specified by `.scope`.
- ▶ If `.mmio` is specified, `.scope` must be `.sys`.

- ▶ If `.ss` is specified, it must be `.global`.
- ▶ If `.ss` is not specified, generic addressing is used for operand `a`. If the generic address specified does not fall within the address window of `.global` state space, the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.1.

Support for `.mmio` qualifier, `.release` semantics, `.global` state space, and `.scope` qualifier introduced in PTX ISA version 8.7.

Target ISA Notes

Requires `sm_90` or higher.

`.mmio` qualifier, `.release` semantics, `.global` state space, and `.scope` qualifier require `sm_100` or higher.

Examples

```
st.async.shared::cluster.mbarrier::complete_tx::bytes.u32 [addr], b, [mbar_addr];
st.async.sys.release.global.u32 [addr], b;
```

9.7.9.13 Data Movement and Conversion Instructions: `st.bulk`

`st.bulk`

Initializes a region of memory as specified by state space.

Syntax

```
st.bulk{.weak}{.shared::cta} [a], size, initval; // initval must be zero
```

Description

`st.bulk` instruction initializes a region of shared memory starting from the location specified by destination address operand `a`.

The 32-bit or 64-bit integer operand `size` specifies the amount of memory to be initialized in terms of number of bytes. `size` must be a multiple of 8. If the value is not a multiple of 8, then the behavior is undefined. The maximum value of `size` operand can be 16777216.

The integer immediate operand `initval` specifies the initialization value for the memory locations. The only numeric value allowed for operand `initval` is 0.

If no state space is specified then *Generic Addressing* is used. If the address specified by `a` does not fall within the address window of `.shared` state space then the behavior is undefined.

The optional qualifier `.weak` specify the memory synchronizing effect of the `st.bulk` instruction as described in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Support for size operand with 32-bit length is introduced in PTX ISA version 9.0.

Target ISA Notes

Requires `sm_100` or higher.

Examples

```
st.bulk.weak.shared::cta [dst], n, 0;
st.bulk                  [gdst], 4096, 0;
```

9.7.9.14 Data Movement and Conversion Instructions: `multimem.ld_reduce`, `multimem.st`, `multimem.red`

The `multimem.*` operations operate on `multimem` addresses and accesses all of the multiple memory locations which the `multimem` address points to.

`Multimem` addresses can only be accessed only by `multimem.*` operations. Accessing a `multimem` address with `ld`, `st` or any other memory operations results in undefined behavior.

Refer to *CUDA programming guide* for creation and management of the `multimem` addresses.

`multimem.ld_reduce`, `multimem.st`, `multimem.red`

Perform memory operations on the `multimem` address.

Syntax

```
// Integer type:

multimem.ld_reduce{.ldsem}{.scope}{.ss}.op.type      d, [a];
multimem.ld_reduce.weak{.ss}.op.type                 d, [a];

multimem.st{.stsem}{.scope}{.ss}.type                [a], b;
multimem.st.weak{.ss}.type                           [a], b;

multimem.red{.redsem}{.scope}{.ss}.op.type           [a], b;

.ss =          { .global }
.ldsem =       { .relaxed, .acquire }
.stsem =       { .relaxed, .release }
```

(continues on next page)

(continued from previous page)

```

.redsem = { .relaxed, .release }
.scope = { .cta, .cluster, .gpu, .sys }
.op = { .min, .max, .add, .and, .or, .xor }
.type = { .b32, .b64, .u32, .u64, .s32, .s64 }

// Floating point type:

multimem.ld_reduce{.ldsem}{.scope}{.ss}.op{.acc_prec}{.vec}.type    d, [a];
multimem.ld_reduce.weak{.ss}.op{.acc_prec}{.vec}.type              d, [a];

multimem.st{.stsem}{.scope}{.ss}{.vec}.type                        [a], b;
multimem.st.weak{.ss}{.vec}.type                                    [a], b;

multimem.red{.redsem}{.scope}{.ss}.redop{.vec}.redtype             [a], b;

.ss = { .global }
.ldsem = { .relaxed, .acquire }
.stsem = { .relaxed, .release }
.redsem = { .relaxed, .release }
.scope = { .cta, .cluster, .gpu, .sys }
.op = { .min, .max, .add }
.redop = { .add }
.acc_prec = { .acc::f32, .acc::f16 }
.vec = { .v2, .v4, .v8 }
.type = { .f16, .f16x2, .bf16, .bf16x2, .f32, .f64, .e5m2, .e5m2x2, .e5m2x4, .
→e4m3, .e4m3x2, .e4m3x4 }
.redtype = { .f16, .f16x2, .bf16, .bf16x2, .f32, .f64 }

```

Description

Instruction `multimem.ld_reduce` performs the following operations:

- load operation on the multimem address `a`, which involves loading of data from all of the multiple memory locations pointed to by the multimem address `a`,
- reduction operation specified by `.op` on the multiple data loaded from the multimem address `a`.

The result of the reduction operation is returned in register `d`.

Instruction `multimem.st` performs a store operation of the input operand `b` to all the memory locations pointed to by the multimem address `a`.

Instruction `multimem.red` performs a reduction operation on all the memory locations pointed to by the multimem address `a`, with operand `b`.

Instruction `multimem.ld_reduce` performs reduction on the values loaded from all the memory locations that the multimem address points to. In contrast, the `multimem.red` perform reduction on all the memory locations that the multimem address points to.

Address operand `a` must be a multimem address. Otherwise, the behavior is undefined. Supported addressing modes for operand `a` and alignment requirements are described in [Addresses as Operands](#).

If no state space is specified then [Generic Addressing](#) is used. If the address specified by `a` does not fall within the address window of `.global` state space then the behavior is undefined.

For floating-point type multi- operations, the size of the specified type along with `.vec` must equal either 32-bits or 64-bits or 128-bits. No other combinations of `.vec` and type are allowed. Type `.f64`

cannot be used with `.vec` qualifier. The following table describes the valid usage of `.vec` and base floating-point type:

.vec	Base float-type supported
No <code>.vec</code> specified	<code>.f16x2</code> , <code>.bf16x2</code> , <code>.f32</code> , <code>.f64</code> , <code>.e5m2x4</code> , <code>.e4m3x4</code>
<code>.v2</code>	<code>.f16</code> , <code>.f16x2</code> , <code>.bf16</code> , <code>.bf16x2</code> , <code>.f32</code> , <code>.e5m2x2</code> , <code>.e5m2x4</code> , <code>.e4m3x2</code> , <code>.e4m3x4</code>
<code>.v4</code>	<code>.f16</code> , <code>.f16x2</code> , <code>.bf16</code> , <code>.bf16x2</code> , <code>.f32</code> , <code>.e5m2</code> , <code>.e5m2x2</code> , <code>.e5m2x4</code> , <code>.e4m3</code> , <code>.e4m3x2</code> , <code>.e4m3x4</code>
<code>.v8</code>	<code>.f16</code> , <code>.bf16</code> , <code>.e5m2</code> , <code>.e4m3</code> , <code>.e5m2x2</code> , <code>.e4m3x2</code>

The following table describes the valid combinations of `.op` and base type:

op	Base type
<code>.add</code>	<code>.u32</code> , <code>.u64</code> , <code>.s32</code> , <code>.f16</code> , <code>.f16x2</code> , <code>.bf16</code> , <code>.bf16x2</code> , <code>.f32</code> , <code>.f64</code> , <code>.e5m2</code> , <code>.e5m2x2</code> , <code>.e5m2x4</code> , <code>.e4m3</code> , <code>.e4m3x2</code> , <code>.e4m3x4</code>
<code>.and</code> , <code>.or</code> , <code>.xor</code>	<code>.b32</code> , <code>.b64</code>
<code>.min</code> , <code>.max</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.s64</code> , <code>.f16</code> , <code>.f16x2</code> , <code>.bf16</code> , <code>.bf16x2</code> , <code>.e5m2</code> , <code>.e5m2x2</code> , <code>.e5m2x4</code> , <code>.e4m3</code> , <code>.e4m3x2</code> , <code>.e4m3x4</code>

For `multimem.ld_reduce`, the default precision of the intermediate accumulation is same as the specified type.

Optionally, `.acc_prec` qualifier can be specified to change the precision of intermediate accumulation as follows:

.type	.acc::prec	Changes precision to
<code>.f16</code> , <code>.f16x2</code> , <code>.bf16</code> , <code>.bf16x2</code>	<code>.acc::f32</code>	<code>.f32</code>
<code>.e5m2</code> , <code>.e4m3</code> , <code>.e5m2x2</code> , <code>.e4m3x2</code> , <code>.e4m3x4</code> , <code>.e5m2x4</code>	<code>.acc::f16</code>	<code>.f16</code>

Optional qualifiers `.ldsem`, `.stsem` and `.redsem` specify the memory synchronizing effect of the `multimem.ld_reduce`, `multimem.st` and `multimem.red` respectively, as described in *Memory Consistency Model*. If explicit semantics qualifiers are not specified, then `multimem.ld_reduce` and `multimem.st` default to `.weak` and `multimem.red` defaults to `.relaxed`.

The optional `.scope` qualifier specifies the set of threads that can directly observe the memory synchronizing effect of this operation, as described in *Memory Consistency Model*. If the `.scope` qualifier is not specified for `multimem.red` then `.sys` scope is assumed by default.

PTX ISA Notes

Introduced in PTX ISA version 8.1.

Support for `.acc::f32` qualifier introduced in PTX ISA version 8.2.

Support for types `.e5m2`, `.e5m2x2`, `.e5m2x4`, `.e4m3`, `.e4m3x2`, `.e4m3x4` introduced in PTX ISA version 8.6.

Support for `.acc::f16` qualifier introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_90` or higher.

Types `.e5m2`, `.e5m2x2`, `.e5m2x4`, `.e4m3`, `.e4m3x2`, `.e4m3x4` are supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ `sm_121a`
- ▶ And are supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.acc::f16` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ `sm_121a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
multimem.ld_reduce.and.b32          val1_b32, [addr1];
multimem.ld_reduce.acquire.gpu.global.add.u32 val2_u32, [addr2];

multimem.st.relaxed.gpu.b32         [addr3], val3_b32;
multimem.st.release.cta.global.u32  [addr4], val4_u32;

multimem.red.relaxed.gpu.max.f64    [addr5], val5_f64;
multimem.red.release.cta.global.add.v4.f32 [addr6], {val6, val7, val8, val9};
multimem.ld_reduce.add.acc::f32.v2.f16x2 {val_10, val_11}, [addr7];
```

(continues on next page)

(continued from previous page)

```

multimem.ld_reduce.relaxed.cta.min.v2.e4m3x2 {val_12, val_13}, [addr8];
multimem.ld_reduce.relaxed.cta.add.v4.e4m3    {val_14, val_15, val_16, val_17},
↪ [addr9];
multimem.ld_reduce.add.acc::f16.v4.e5m2      {val_18, val_19, val_20, val_21},
↪ [addr10];

```

9.7.9.15 Data Movement and Conversion Instructions: `prefetch`, `prefetchu`

`prefetch`, `prefetchu`

Prefetch line containing a generic address at a specified level of memory hierarchy, in specified state space.

Syntax

```

prefetch{.space}.level           [a];    // prefetch to data cache
prefetch.global.level::eviction_priority [a]; // prefetch to data cache

prefetchu.L1 [a];                  // prefetch to uniform cache

prefetch{.tensormap_space}.tensormap [a]; // prefetch the tensormap

.space = { .global, .local };
.level = { .L1, .L2 };
.level::eviction_priority = { .L2::evict_last, .L2::evict_normal };
.tensormap_space = { .const, .param };

```

Description

The `prefetch` instruction brings the cache line containing the specified address in global or local memory state space into the specified cache level.

If the `.tensormap` qualifier is specified then the `prefetch` instruction brings the cache line containing the specified address in the `.const` or `.param` memory state space for subsequent use by the `cp.async.bulk.tensor` instruction.

If no state space is given, the `prefetch` uses *Generic Addressing*.

Optionally, the eviction priority to be applied on the prefetched cache line can be specified by the modifier `.level::eviction_priority`.

Supported addressing modes for operand `a` and alignment requirements are described in *Addresses as Operands*.

The `prefetchu` instruction brings the cache line containing the specified generic address into the specified uniform cache level.

A `prefetch` to a shared memory location performs no operation.

A `prefetch` into the uniform cache requires a generic address, and no operation occurs if the address maps to a `const`, `local`, or shared memory location.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Support for `.level::eviction_priority` qualifier introduced in PTX ISA version 7.4.

Support for the `.tensormap` qualifier is introduced in PTX ISA version 8.0.

Target ISA Notes

`prefetch` and `prefetchu` require `sm_20` or higher.

Support for `.level::eviction_priority` qualifier requires `sm_80` or higher.

Support for the `.tensormap` qualifier requires `sm_90` or higher.

Examples

```
prefetch.global.L1          [ptr];
prefetch.global.L2::evict_last [ptr];
prefetchu.L1 [addr];
prefetch.const.tensormap    [ptr];
```

9.7.9.16 Data Movement and Conversion Instructions: `applypriority`

`applypriority`

Apply the cache eviction priority to the specified address in the specified cache level.

Syntax

```
applypriority{.global}.level::eviction_priority [a], size;
.level::eviction_priority = { .L2::evict_normal };
```

Description

The `applypriority` instruction applies the cache eviction priority specified by the `.level::eviction_priority` qualifier to the address range `[a..a+size)` in the specified cache level.

If no state space is specified then *Generic Addressing* is used. If the specified address does not fall within the address window of `.global` state space then the behavior is undefined.

The operand size is an integer constant that specifies the amount of data, in bytes, in the specified cache level on which the priority is to be applied. The only supported value for the size operand is 128.

Supported addressing modes for operand `a` are described in *Addresses as Operands*. `a` must be aligned to 128 bytes.

PTX ISA Notes

Introduced in PTX ISA version 7.4.

Target ISA Notes

Requires sm_80 or higher.

Examples

```
applypriority.global.L2::evict_normal [ptr], 128;
```

9.7.9.17 Data Movement and Conversion Instructions: **discard**

discard

Discard the data at the specified address range and cache level.

Syntax

```
discard{.global}.level [a], size;  
.level = { .L2 };
```

Description

Semantically, this behaves like a weak write of an *unstable indeterminate value*: reads of memory locations with *unstable indeterminate values* may return different bit patterns each time until the memory is overwritten. This operation *hints* to the implementation that data in the specified cache `.level` can be destructively discarded without writing it back to memory.

The operand `size` is an integer constant that specifies the length in bytes of the address range `[a, a + size)` to write *unstable indeterminate values* into. The only supported value for the `size` operand is 128.

If no state space is specified then *Generic Addressing* is used. If the specified address does not fall within the address window of `.global` state space then the behavior is undefined.

Supported addressing modes for address operand `a` are described in *Addresses as Operands*. `a` must be aligned to 128 bytes.

PTX ISA Notes

Introduced in PTX ISA version 7.4.

Target ISA Notes

Requires sm_80 or higher.

Examples

```
discard.global.L2 [ptr], 128;
ld.weak.u32 r0, [ptr];
ld.weak.u32 r1, [ptr];
// The values in r0 and r1 may differ!
```

9.7.9.18 Data Movement and Conversion Instructions: `createpolicy`

`createpolicy`

Create a cache eviction policy for the specified cache level.

Syntax

```
// Range-based policy
createpolicy.range{.global}.level::primary_priority{.level::secondary_priority}.b64
    cache-policy, [a], primary-size, total-size;

// Fraction-based policy
createpolicy.fractional.level::primary_priority{.level::secondary_priority}.b64
    cache-policy{, fraction};

// Converting the access property from CUDA APIs
createpolicy.cvt.L2.b64    cache-policy, access-property;

.level::primary_priority = { .L2::evict_last, .L2::evict_normal,
                             .L2::evict_first, .L2::evict_unchanged };
.level::secondary_priority = { .L2::evict_first, .L2::evict_unchanged };
```

Description

The `createpolicy` instruction creates a cache eviction policy for the specified cache level in an opaque 64-bit register specified by the destination operand `cache-policy`. The cache eviction policy specifies how cache eviction priorities are applied to global memory addresses used in memory operations with `.level::cache_hint` qualifier.

There are two types of cache eviction policies:

► Range-based policy

The cache eviction policy created using `createpolicy.range` specifies the cache eviction behaviors for the following three address ranges:

- $[a \dots a + (\text{primary-size} - 1)]$ referred to as primary range.
- $[a + \text{primary-size} \dots a + (\text{total-size} - 1)]$ referred to as trailing secondary range.
- $[a - (\text{total-size} - \text{primary-size}) \dots (a - 1)]$ referred to as preceding secondary range.

When a range-based cache eviction policy is used in a memory operation with `.level::cache_hint` qualifier, the eviction priorities are applied as follows:

- If the memory address falls in the primary range, the eviction priority specified by `.L2::primary_priority` is applied.
- If the memory address falls in any of the secondary ranges, the eviction priority specified by `.L2::secondary_priority` is applied.
- If the memory address does not fall in either of the above ranges, then the applied eviction priority is unspecified.

The 32-bit operand `primary-size` specifies the size, in bytes, of the primary range. The 32-bit operand `total-size` specifies the combined size, in bytes, of the address range including primary and secondary ranges. The value of `primary-size` must be less than or equal to the value of `total-size`. Maximum allowed value of `total-size` is 4GB.

If `.L2::secondary_priority` is not specified, then it defaults to `.L2::evict_unchanged`.

If no state space is specified then *Generic Addressing* is used. If the specified address does not fall within the address window of `.global` state space then the behavior is undefined.

► Fraction-based policy

A memory operation with `.level::cache_hint` qualifier can use the fraction-based cache eviction policy to request the cache eviction priority specified by `.L2::primary_priority` to be applied to a fraction of cache accesses specified by the 32-bit floating point operand `fraction`. The remainder of the cache accesses get the eviction priority specified by `.L2::secondary_priority`. This implies that in a memory operation that uses a fraction-based cache policy, the memory access has a probability specified by the operand `fraction` of getting the cache eviction priority specified by `.L2::primary_priority`.

The valid range of values for the operand `fraction` is $(0.0, \dots, 1.0]$. If the operand `fraction` is not specified, it defaults to 1.0.

If `.L2::secondary_priority` is not specified, then it defaults to `.L2::evict_unchanged`.

The access property created using the CUDA APIs can be converted into cache eviction policy by the instruction `createpolicy.cvt`. The source operand `access-property` is a 64-bit opaque register. Refer to *CUDA programming guide* for more details.

PTX ISA Notes

Introduced in PTX ISA version 7.4.

Target ISA Notes

Requires sm_80 or higher.

Examples

```
createpolicy.fractional.L2::evict_last.b64      policy, 1.0;
createpolicy.fractional.L2::evict_last.L2::evict_unchanged.b64  policy, 0.5;

createpolicy.range.L2::evict_last.L2::evict_first.b64
                                                policy, [ptr], 0x100000, 0x200000;

// access-prop is created by CUDA APIs.
createpolicy.cvt.L2.b64 policy, access-prop;
```

9.7.9.19 Data Movement and Conversion Instructions: `isspacep`

`isspacep`

Query whether a generic address falls within a specified state space window.

Syntax

```
isspacep.space p, a;    // result is .pred

.space = { const, .global, .local, .shared{::cta, ::cluster}, .param{::entry} };
```

Description

Write predicate register `p` with 1 if generic address `a` falls within the specified state space window and with 0 otherwise. Destination `p` has type `.pred`; the source address operand must be of type `.u32` or `.u64`.

`isspacep.param{::entry}` returns 1 if the generic address falls within the window of *Kernel Function Parameters*, otherwise returns 0. If `.param` is specified without any sub-qualifiers then it defaults to `.param::entry`.

`isspacep.global` returns 1 for *Kernel Function Parameters* as `.param` window is contained within the `.global` window.

If no sub-qualifier is specified with `.shared` state space, then `::cta` is assumed by default.

Note: `isspacep.shared::cluster` will return 1 for every shared memory address that is accessible to the threads in the cluster, whereas `isspacep.shared::cta` will return 1 only if the address is of a variable declared in the executing CTA.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

`isspacep.const` introduced in PTX ISA version 3.1.

`isspacep.param` introduced in PTX ISA version 7.7.

Support for `::cta` and `::cluster` sub-qualifiers introduced in PTX ISA version 7.8.

Support for sub-qualifier `::entry` on `.param` space introduced in PTX ISA version 8.3.

Target ISA Notes

`isspacep` requires `sm_20` or higher.

`isspacep.param{::entry}` requires `sm_70` or higher.

Sub-qualifier `::cta` requires `sm_30` or higher.

Sub-qualifier `::cluster` requires `sm_90` or higher.

Examples

```
isspacep.const      iscnst, cptr;
isspacep.global     isglbl, gptr;
isspacep.local      islcl, lptr;
isspacep.shared     isshrd, sptr;
isspacep.param::entry isparam, pptr;
isspacep.shared::cta isshrdcta, sptr;
isspacep.shared::cluster ishrdany sptr;
```

9.7.9.20 Data Movement and Conversion Instructions: `cvta`

`cvta`

Convert address from `.const`, *Kernel Function Parameters* (`.param`), `.global`, `.local`, or `.shared` state space to generic, or vice-versa. Take the generic address of a variable declared in `.const`, *Kernel Function Parameters* (`.param`), `.global`, `.local`, or `.shared` state space.

Syntax

```
// convert const, global, local, or shared address to generic address
cvta.space.size p, a;           // source address in register a
cvta.space.size p, var;         // get generic address of var
cvta.space.size p, var+imm;     // generic address of var+offset

// convert generic address to const, global, local, or shared address
cvta.to.space.size p, a;

.space = { .const, .global, .local, .shared{::cta, ::cluster}, .param{::entry} };
.size = { .u32, .u64 };
```

Description

Convert a const, *Kernel Function Parameters* (.param), global, local, or shared address to a generic address, or vice-versa. The source and destination addresses must be the same size. Use `cvt.u32.u64` or `cvt.u64.u32` to truncate or zero-extend addresses.

For variables declared in .const, *Kernel Function Parameters* (.param), .global, .local, or .shared state space, the generic address of the variable may be taken using `cvta`. The source is either a register or a variable defined in const, *Kernel Function Parameters* (.param), global, local, or shared memory with an optional offset.

When converting a generic address into a const, *Kernel Function Parameters* (.param), global, local, or shared address, the resulting address is undefined in cases where the generic address does not fall within the address window of the specified state space. A program may use `isspacep` to guard against such incorrect behavior.

For `cvta` with .shared state space, the address must belong to the space specified by `::cta` or `::cluster` sub-qualifier, otherwise the behavior is undefined. If no sub-qualifier is specified with .shared state space, then `::cta` is assumed by default.

If .param is specified without any sub-qualifiers then it defaults to `.param::entry`. For `.param{::entry}` state space, operand a must be a kernel parameter address, otherwise behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

`cvta.const` and `cvta.to.const` introduced in PTX ISA version 3.1.

`cvta.param` and `cvta.to.param` introduced in PTX ISA version 7.7.

Note: The current implementation does not allow generic pointers to const space variables in programs that contain pointers to constant buffers passed as kernel parameters.

Support for `::cta` and `::cluster` sub-qualifiers introduced in PTX ISA version 7.8.

Support for sub-qualifier `::entry` on .param space introduced in PTX ISA version 8.3.

Target ISA Notes

`cvta` requires sm_20 or higher.

`cvta.param{::entry}` and `cvta.to.param{::entry}` requires sm_70 or higher.

Sub-qualifier `::cta` requires sm_30 or higher.

Sub-qualifier `::cluster` requires sm_90 or higher.

Examples

```
cvta.const.u32    ptr,cvar;
cvta.local.u32    ptr,lptr;
cvta.shared::cta.u32 p,As+4;
cvta.shared::cluster.u32 ptr, As;
cvta.to.global.u32 p,gptr;
cvta.param.u64    ptr,pvar;
cvta.to.param::entry.u64  ep_ptr, ptr;
```

9.7.9.21 Data Movement and Conversion Instructions: cvt

cvt

Convert a value from one type to another.

Syntax

```
cvt{.irnd}{.ftz}{.sat}.dtype.atype    d, a; // integer rounding
cvt{.frnd}{.ftz}{.sat}.dtype.atype    d, a; // fp rounding

cvt.frnd2{.relu}{.satfinite}.f16.f32  d, a;
cvt.frnd2{.relu}{.satfinite}.f16x2.f32 d, a, b;
cvt.rs{.relu}{.satfinite}.f16x2.f32    d, a, b, rbits;

cvt.frnd2{.relu}{.satfinite}.bf16.f32  d, a;
cvt.frnd2{.relu}{.satfinite}.bf16x2.f32 d, a, b;
cvt.rs{.relu}{.satfinite}.bf16x2.f32    d, a, b, rbits;

cvt.rna{.satfinite}.tf32.f32           d, a;
cvt.frnd2{.satfinite}{.relu}.tf32.f32  d, a;

cvt.rn.satfinite{.relu}.f8x2type.f32    d, a, b;
cvt.rn.satfinite{.relu}.f8x2type.fp16x2 d, a;
cvt.rn{.relu}.f16x2.f8x2type            d, a;
cvt.rs{.relu}.satfinite.f8x4type.f32    d, {a, b, e, f}, rbits;

cvt.rn.satfinite{.relu}.f4x2type.f32    d, a, b;
cvt.rn.satfinite{.relu}.f4x2type.fp16x2type d, a;
cvt.rn{.relu}.f16x2.f4x2type            d, a;
cvt.rs{.relu}.satfinite.f4x4type.f32    d, {a, b, e, f}, rbits;

cvt.rn.satfinite{.relu}.f6x2type.f32    d, a, b;
cvt.rn.satfinite{.relu}.f6x2type.fp16x2type d, a;
cvt.rn{.relu}.f16x2.f6x2type            d, a;
cvt.rs{.relu}.satfinite.f6x4type.f32    d, {a, b, e, f}, rbits;

cvt.frnd3{.satfinite}.ue8m0x2.f32      d, a, b;
cvt.frnd3{.satfinite}.ue8m0x2.bf16x2   d, a;
cvt.rn.bf16x2.ue8m0x2                  d, a;

cvt.rn.satfinite{.relu}{.scaled::n2::ue8m0}.s2f6x2.f32  d, a, b{, scale-factor};
cvt.rn.satfinite{.relu}{.scaled::n2::ue8m0}.s2f6x2.bf16x2 d, a{, scale-factor};
```

(continues on next page)

(continued from previous page)

```

cvt.rn{.satfinite}{.relu}{.scaled::n2::ue8m0}.bf16x2.s2f6x2 d, a{, scale-factor};

.irnd  = { .rni, .rzi, .rmi, .rpi };
.frnd  = { .rn,  .rz,  .rm,  .rp  };
.frnd2 = { .rn,  .rz  };
.frnd3 = { .rz,  .rp  };
.dtype = .atype = { .u8,   .u16, .u32, .u64,
                    .s8,   .s16, .s32, .s64,
                    .bf16, .f16, .f32, .f64 };
.f8x2type = { .e4m3x2, .e5m2x2 };
.f4x2type = { .e2m1x2 };
.f6x2type = { .e2m3x2, .e3m2x2 };
.f4x4type = { .e2m1x4 };
.f8x4type = { .e4m3x4, .e5m2x4 };
.f6x4type = { .e2m3x4, .e3m2x4 };
.fp16x2type = { .f16x2, .bf16x2 };

```

Description

Convert between different types and sizes.

For `.f16x2` and `.bf16x2` instruction type, two inputs `a` and `b` of `.f32` type are converted into `.f16` or `.bf16` type and the converted values are packed in the destination register `d`, such that the value converted from input `a` is stored in the upper half of `d` and the value converted from input `b` is stored in the lower half of `d`.

For `.f16x2` instruction type, destination operand `d` has `.f16x2` or `.b32` type. For `.bf16` instruction type, operand `d` has `.b16` type. For `.bf16x2` instruction type, operand `d` has `.b32` type. For `.tf32` instruction type, operand `d` has `.b32` type.

When converting to `.e4m3x2/.e5m2x2` data formats, the destination operand `d` has `.b16` type. When converting two `.f32` inputs to `.e4m3x2/.e5m2x2`, each input is converted to the specified format, and the converted values are packed in the destination operand `d` such that the value converted from input `a` is stored in the upper 8 bits of `d` and the value converted from input `b` is stored in the lower 8 bits of `d`. When converting an `.f16x2/.bf16x2` input to `.e4m3x2/.e5m2x2`, each `.f16/.bf16` input from operand `a` is converted to the specified format. The converted values are packed in the destination operand `d` such that the value converted from the upper 16 bits of input `a` is stored in the upper 8 bits of `d` and the value converted from the lower 16 bits of input `a` is stored in the lower 8 bits of `d`.

When converting from `.e4m3x2/.e5m2x2` to `.f16x2`, source operand `a` has `.b16` type. Each 8-bit input value in operand `a` is converted to `.f16` type. The converted values are packed in the destination operand `d` such that the value converted from the upper 8 bits of `a` is stored in the upper 16 bits of `d` and the value converted from the lower 8 bits of `a` is stored in the lower 16 bits of `d`.

When converting to `.e2m1x2` data formats, the destination operand `d` has `.b8` type. When converting two `.f32` inputs to `.e2m1x2`, each input is converted to the specified format, and the converted values are packed in the destination operand `d` such that the value converted from input `a` is stored in the upper 4 bits of `d` and the value converted from input `b` is stored in the lower 4 bits of `d`. When converting an `.f16x2/.bf16x2` input to `.e2m1x2`, each `.f16/.bf16` input from operand `a` is converted to the specified format. The converted values are packed in `d` so that the value from the upper 16 bits of `a` is stored in the upper 4 bits of `d`, and the value from the lower 16 bits of `a` is stored in the lower 4 bits of `d`.

When converting from `.e2m1x2` to `.f16x2`, source operand `a` has `.b8` type. Each 4-bit input value in operand `a` is converted to `.f16` type. The converted values are packed in the destination operand `d`.

such that the value converted from the upper 4 bits of *a* is stored in the upper 16 bits of *d* and the value converted from the lower 4 bits of *a* is stored in the lower 16 bits of *d*.

When converting to `.e2m1x4` data format, the destination operand *d* has `.b16` type. When converting four `.f32` inputs to `.e2m1x4`, each input is converted to the specified format, and the converted values are packed in the destination operand *d* such that the value converted from inputs *a*, *b*, *e*, *f* are stored in each 4 bits starting from upper bits of *d*.

When converting to `.e2m3x2/.e3m2x2` data formats, the destination operand *d* has `.b16` type. When converting two `.f32` inputs to `.e2m3x2/.e3m2x2`, each input is converted to the specified format, and the converted values are packed in the destination operand *d* such that the value converted from input *a* is stored in the upper 8 bits of *d* with 2 MSB bits padded with zeros and the value converted from input *b* is stored in the lower 8 bits of *d* with 2 MSB bits padded with zeros. When converting an `.f16x2/.bf16x2` input to `.e2m3x2/.e3m2x2`, each `.f16/.bf16` input from operand *a* is converted to the specified format. The converted values are packed in the destination operand *d* so that the value from the upper 16 bits of input *a* is stored in the upper 8 bits of *d* with 2 MSB bits padded with zeros, and the value from the lower 16 bits of input *a* is stored in the lower 8 bits of *d* with 2 MSB bits padded with zeros.

When converting from `.e2m3x2/.e3m2x2` to `.f16x2`, source operand *a* has `.b16` type. Each 8-bit input value with 2 MSB bits 0 in operand *a* is converted to `.f16` type. The converted values are packed in the destination operand *d* such that the value converted from the upper 8 bits of *a* is stored in the upper 16 bits of *d* and the value converted from the lower 8 bits of *a* is stored in the lower 16 bits of *d*.

When converting to `.e5m2x4/.e4m3x4/.e3m2x4/.e2m3x4` data format, the destination operand *d* has `.b32` type. When converting four `.f32` inputs to `.e5m2x4/.e4m3x4/.e3m2x4/.e2m3x4`, each input is converted to the specified format, and the converted values are packed in the destination operand *d* such that the value converted from inputs *a*, *b*, *e*, *f* are stored in each 8 bits starting from upper bits of *d*. For `.e3m2x4/.e2m3x4`, each 8-bit output will have 2 MSB bits padded with zeros.

When converting to `.ue8m0x2` data formats, the destination operand *d* has `.b16` type. When converting two `.f32` or two packed `.bf16` inputs to `.ue8m0x2`, each input is converted to the specified format, and the converted values are packed in the destination operand *d* such that the value converted from input *a* is stored in the upper 8 bits of *d* and the value converted from input *b* is stored in the lower 8 bits of *d*.

When converting from `.ue8m0x2` to `.bf16x2`, source operand *a* has `.b16` type. Each 8-bit input value in operand *a* is converted to `.bf16` type. The converted values are packed in the destination operand *d* such that the value converted from the upper 8 bits of *a* is stored in the upper 16 bits of *d* and the value converted from the lower 8 bits of *a* is stored in the lower 16 bits of *d*.

When converting to `.s2f6x2` data formats, the destination operand *d* has `.b16` type. When converting two `.f32` inputs to `.s2f6x2`, each input is converted to the `.s2f6` value, and the converted values are packed in the destination operand *d* such that the value converted from input *a* is stored in the upper 8 bits of *d* and the value converted from input *b* is stored in the lower 8 bits of *d*. When converting from `.bf16x2`, each input packed in operand *a* is converted to `.s2f6` value and converted values are packed in the destination operand *d* such that value converted from upper 8 bits of input *a* is stored in upper 8 bits of operand *d* and the value converted from lower 8 bits of input *a* is stored in lower 8 bits of operand *d*. Optional operand `scale-factor` has type `.b16` and stores two packed scaling factors of type `.ue8m0`. For down conversion, inputs are divided by `scale-factor` and then conversion is performed. The scaling factor of input *a/.bf16* input from upper 16 bits of *a* is stored in the upper 8 bits of operand `scale-factor` and the scaling factor of input *b/.bf16* input from lower 16 bits of *a* is stored in the lower 8 bits of operand `scale-factor`. If not specified explicitly, value of `scale-factor` will be assumed to be `0x7f7f`, that is, assumed as value 1 for both input scale factors.

When converting from `.s2f6x2` to `.bf16x2`, source operand *a* has `.b16` type. Each 8-bit input value in operand *a* is converted to `.bf16` type. The converted values are packed in the destination operand *d*

such that the value converted from the upper 8 bits of *a* is stored in the upper 16 bits of *d* and the value converted from the lower 8 bits of *a* is stored in the lower 16 bits of *d*. Optional operand *scale-factor* has type `.b16` and stores two packed scaling factors of type `.ue8m0`. For up-conversion, inputs are converted to destination type and then multiplied by *scale-factor*. The scaling factor of `.s2f6` input from upper 8 bits of *a* is stored in the upper 8 bits of operand *scale-factor* and the scaling factor of `.s2f6` input from lower 8 bits of *a* is stored in the lower 8 bits of operand *scale-factor*. If not specified explicitly, value of *scale-factor* will be assumed to be `0x7f7f`, that is, assumed as value 1 for both input scale factors.

Optional qualifier `.scaled::n2::ue8m0` specifies that the instruction uses packed *scale-factor* with 2 scale values of `ue8m0` type. Operand *scale-factor* and qualifier `.scaled::n2::ue8m0` must be used together.

rbits is a `.b32` type register operand used for providing random bits for *.rs* rounding mode.

When converting to `.f16x2`, two 16-bit values are provided from *rbits* where 13 LSBs from upper 16-bits are used as random bits for operand *a* with 3 MSBs are 0 and 13 LSBs from lower 16-bits are used as random bits for operand *b* with 3 MSBs are 0.

When converting to `.bf16x2`, two 16-bit values are provided from *rbits* where upper 16-bits are used as random bits for operand *a* and lower 16-bits are used as random bits for operand *b*.

When converting to `.e4m3x4/.e5m2x4/.e2m3x4/.e3m2x4`, two 16-bit values are provided from *rbits* where lower 16-bits are used for operands *e*, *f* and upper 16 bits are used for operands *a*, *b*.

When converting to `.e2m1x4`, two 16-bit values are provided from *rbits* where lower 8-bits from both 16-bits half of *rbits* are used for operands *e*, *f* and upper 8-bits from both 16-bits half of *rbits* are used for operands *a*, *b*.

Rounding modifier is mandatory in all of the following cases:

- ▶ float-to-float conversions, when destination type is smaller than source type
- ▶ All float-to-int conversions
- ▶ All int-to-float conversions
- ▶ All conversions involving `.f16x2`, `.e4m3x2`, `.e5m2x2`, `.bf16x2`, `.tf32`, `.e2m1x2`, `.e2m3x2`, `.e3m2x2`, `.e4m3x4`, `.e5m2x4`, `.e2m1x4`, `.e2m3x4`, `.e3m2x4`, `.s2f6x2` and `.ue8m0x2` instruction types.

`.satfinite` modifier is only supported for conversions involving the following types:

- ▶ `.e4m3x2`, `.e5m2x2`, `.e2m1x2`, `.e2m3x2`, `.e3m2x2`, `.e4m3x4`, `.e5m2x4`, `.e2m1x4`, `.e2m3x4`, `.e3m2x4`, `.s2f6x2` destination types. `.satfinite` modifier is mandatory for such conversions.
- ▶ `.f16`, `.bf16`, `.f16x2`, `.bf16x2`, `.tf32`, `.ue8m0x2` as destination types.

Semantics

```
if (/* inst type is .f16x2 or .bf16x2 */) {
    d[31:16] = convert(a);
    d[15:0]  = convert(b);
} else if (/* inst destination type is .e5m2x2 or .e4m3x2 or .ue8m0x2 */) {
    if (/* inst source type is .f32 */) {
        d[15:8] = convert(a);
        d[7:0]  = convert(b);
    } else {
        d[15:8] = convert(a[31:16]);
        d[7:0]  = convert(a[15:0]);
    }
}
```

(continues on next page)

(continued from previous page)

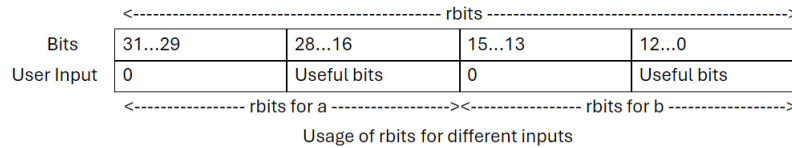
```

    }
} else if (/* inst destination type is .s2f6x2 */) {
    if (/* inst source type is .f32 */) {
        d[15:8] = convert(a / scale-factor[15:8]);
        d[7:0] = convert(b / scale-factor[7:0]);
    } else {
        d[15:8] = convert(a[15:8] / scale-factor[15:8]);
        d[7:0] = convert(a[7:0] / scale-factor[7:0]);
    }
} else if (/* inst source type is .s2f6x2 */) {
    d[31:16] = convert(a[15:8]) * scale-factor[15:8];
    d[15:0] = convert(a[7:0]) * scale-factor[7:0];
} else if (/* inst destination type is .e2m1x2 */) {
    if (/* inst source type is .f32 */) {
        d[7:4] = convert(a);
        d[3:0] = convert(b);
    } else {
        d[7:4] = convert(a[31:16]);
        d[3:0] = convert(a[15:0]);
    }
} else if (/* inst destination type is .e2m3x2 or .e3m2x2 */) {
    if (/* inst source type is .f32 */) {
        d[15:14] = 0;
        d[13:8] = convert(a);
        d[7:6] = 0;
        d[5:0] = convert(b);
    } else {
        d[15:14] = 0;
        d[13:8] = convert(a[31:16]);
        d[7:6] = 0;
        d[5:0] = convert(a[15:0]);
    }
} else if (/* inst destination type is .e2m1x4 */) {
    d[15:12] = convert(a);
    d[11:8] = convert(b);
    d[7:4] = convert(e);
    d[3:0] = convert(f);
} else if (/* inst destination type is .e4m3x4 or .e5m2x4 */) {
    d[31:24] = convert(a);
    d[23:16] = convert(b);
    d[15:8] = convert(e);
    d[7:0] = convert(f);
} else if (/* inst destination type is .e2m3x4 or .e3m2x4 */) {
    d[31:30] = 0;
    d[29:24] = convert(a);
    d[23:22] = 0;
    d[21:16] = convert(b);
    d[15:14] = 0;
    d[13:8] = convert(e);
    d[7:6] = 0;
    d[5:0] = convert(f);
} else {
    d = convert(a);
}

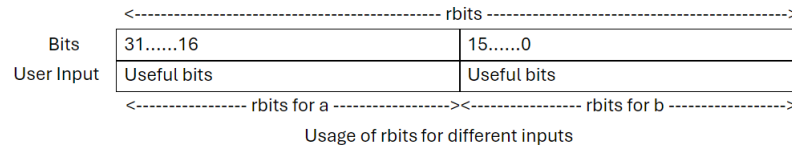
```

// Random bits rbits semantics for .rs rounding:

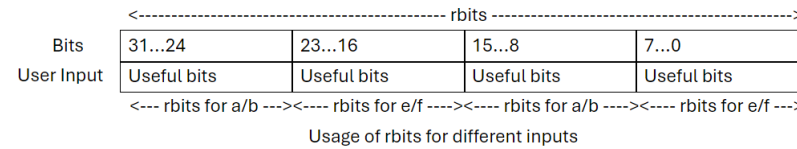
1. Destination type .f16: Refer [Figure 38](#) for random bits layout details.

Figure 38: Random bits layout for `.rs` rounding with `.f16` destination type

2. Destination type `.bf16`: Refer [Figure 39](#) for random bits layout details.

Figure 39: Random bits layout for `.rs` rounding with `.bf16` destination type

3. Destination type `.e2m1x4`: Refer [Figure 40](#) for random bits layout details.

Figure 40: Random bits layout for `.rs` rounding with `.e2m1x4` destination type

4. Destination type `.e5m2x4`, `.e4m3x4`, `.e3m2x4`, `.e2m3x4`: Refer [Figure 41](#) for random bits layout details.

Integer Notes

Integer rounding is required for float-to-integer conversions, and for same-size float-to-float conversions where the value is rounded to an integer. Integer rounding is illegal in all other instances.

Integer rounding modifiers:

`.rni`

round to nearest integer, choosing even integer if source is equidistant between two integers

`.rzi`

round to nearest integer in the direction of zero

`.rmi`

round to nearest integer in direction of negative infinity

`.rpi`

round to nearest integer in direction of positive infinity

In float-to-integer conversions, depending upon conversion types, NaN input results in following value:

1. Zero if source is not `.f64` and destination is not `.s64`, `.u64`.
2. Otherwise $1 \ll (\text{BitWidth}(\text{dst}) - 1)$ corresponding to the value of $(\text{MAXINT} \gg 1) + 1$ for unsigned type or MININT for signed type.

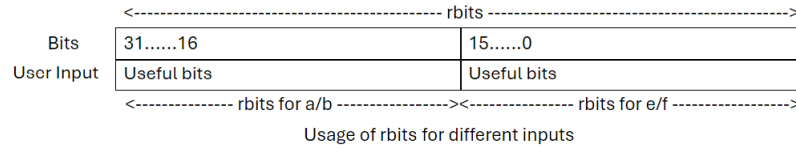


Figure 41: Random bits layout for `.rs` rounding with `.e5m2x4/` `.e4m3x4/` `.e3m2x4/` `.e2m3x4` destination type

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported.

For `cvt.ftz.dtype.f32` float-to-integer conversions and `cvt.ftz.f32.f32` float-to-float conversions with integer rounding, subnormal inputs are flushed to sign-preserving zero. Modifier `.ftz` can only be specified when either `.dtype` or `.atype` is `.f32` and applies only to single precision (`.f32`) inputs and results.

sm_1x

For `cvt.ftz.dtype.f32` float-to-integer conversions and `cvt.ftz.f32.f32` float-to-float conversions with integer rounding, subnormal inputs are flushed to sign-preserving zero. The optional `.ftz` modifier may be specified in these cases for clarity.

Note: In PTX ISA versions 1.4 and earlier, the `cvt` instruction did not flush single-precision subnormal inputs or results to zero if the destination type size was 64-bits. The compiler will preserve this behavior for legacy PTX code.

Saturation modifier:

.sat

For integer destination types, `.sat` limits the result to `MININT` `.MAXINT` for the size of the operation. Note that saturation applies to both signed and unsigned integer types.

The saturation modifier is allowed only in cases where the destination type's value range is not a superset of the source type's value range; i.e., the `.sat` modifier is illegal in cases where saturation is not possible based on the source and destination types.

For float-to-integer conversions, the result is clamped to the destination range by default; i.e., `.sat` is redundant.

Floating Point Notes

Floating-point rounding is required for float-to-float conversions that result in loss of precision, and for integer-to-float conversions. Floating-point rounding is illegal in all other instances.

Floating-point rounding modifiers:

.rn rounding to nearest, with ties to even

.rna
rounding to nearest, with ties away from zero

.rz rounding toward zero

.rm rounding toward negative infinity

.rp rounding toward positive infinity

- .rs** Stochastic rounding is achieved through the use of the supplied random bits. Operation's result is rounded in the direction toward zero or away from zero based on the carry out of the integer addition of the supplied random bits (rbits) to the truncated off (discarded) bits of mantissa from the input.

A floating-point value may be rounded to an integral value using the integer rounding modifiers (see Integer Notes). The operands must be of the same size. The result is an integral value, stored in floating-point format.

Subnormal numbers:

sm_20+

By default, subnormal numbers are supported. Modifier `.ftz` may be specified to flush single-precision subnormal inputs and results to sign-preserving zero. Modifier `.ftz` can only be specified when either `.dtype` or `.atype` is `.f32` and applies only to single precision (`.f32`) inputs and results.

sm_1x

Single-precision subnormal inputs and results are flushed to sign-preserving zero. The optional `.ftz` modifier may be specified in these cases for clarity.

Note: In PTX ISA versions 1.4 and earlier, the `cvt` instruction did not flush single-precision subnormal inputs or results to zero if either source or destination type was `.f64`. The compiler will preserve this behavior for legacy PTX code. Specifically, if the PTX ISA version is 1.4 or earlier, single-precision subnormal inputs and results are flushed to sign-preserving zero only for `cvt.f32.f16`, `cvt.f16.f32`, and `cvt.f32.f32` instructions.

Saturation modifier:

.sat:

For floating-point destination types, `.sat` limits the result to the range [0.0, 1.0]. NaN results are flushed to positive zero. Applies to `.f16`, `.f32`, and `.f64` types.

.relu:

For `.f16`, `.f16x2`, `.bf16`, `.bf16x2`, `.e4m3x2`, `.e5m2x2`, `.e2m1x2`, `.e2m3x2`, `.e3m2x2`, `.e4m3x4`, `.e5m2x4`, `.e2m1x4`, `.e2m3x4`, `.e3m2x4`, `.s2f6x2` and `.tf32` destination types, `.relu` clamps the result to 0 if negative. NaN results are converted to canonical NaN.

.satfinite:

For `.f16`, `.f16x2`, `.bf16`, `.bf16x2`, `.e4m3x2`, `.e5m2x2`, `.ue8m0x2`, `.e4m3x4`, `.e5m2x4` and `.tf32` destination formats, if the input value is NaN, then the result is NaN in the specified destination format. For `.e2m1x2`, `.e2m3x2`, `.e3m2x2`, `.e2m1x4`, `.e2m3x4`, `.e3m2x4`, `.s2f6x2` destination formats NaN results are converted to positive `MAX_NORM`. If the absolute value of input (ignoring sign) is greater than `MAX_NORM` of the specified destination format, then the result is sign-preserved `MAX_NORM` of the destination format and a positive `MAX_NORM` in `.ue8m0x2` for which the destination sign is not supported.

Notes

A source register wider than the specified type may be used, except when the source operand has `.bf16` or `.bf16x2` format. The lower `n` bits corresponding to the instruction-type width are used in the conversion. See *Operand Size Exceeding Instruction-Type Size* for a description of these relaxed type-checking rules.

A destination register wider than the specified type may be used, except when the destination operand has `.bf16`, `.bf16x2` or `.tf32` format. The result of conversion is sign-extended to the destination register width for signed integers, and is zero-extended to the destination register width for unsigned,

bit-size, and floating-point types. See *Operand Size Exceeding Instruction-Type Size* for a description of these relaxed type-checking rules.

For `cvt.f32.bf16`, NaN input yields unspecified NaN.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`.relu` modifier and `{.f16x2, .bf16, .bf16x2, .tf32}` destination formats introduced in PTX ISA version 7.0.

`cvt.f32.bf16` introduced in PTX ISA version 7.1.

`cvt.bf16.{u8/s8/u16/s16/u32/s32/u64/s64/f16/f64/bf16}`, `cvt.{u8/s8/u16/s16/u32/s32/u64/s64/f16/f64}.bf16`, and `cvt.tf32.f32.{relu}.{rn/rz}` introduced in PTX ISA version 7.8.

`.ftz` qualifier for `cvt.f32.bf16` introduced in PTX ISA version 7.8.

`cvt` with `.e4m3x2/.e5m2x2` for `sm_90` or higher introduced in PTX ISA version 7.8.

`cvt.satfinite.{e4m3x2, e5m2x2}.{f32, f16x2}` for `sm_90` or higher introduced in PTX ISA version 7.8.

`cvt` with `.e4m3x2/.e5m2x2` for `sm_89` introduced in PTX ISA version 8.1.

`cvt.satfinite.{e4m3x2, e5m2x2}.{f32, f16x2}` for `sm_89` introduced in PTX ISA version 8.1.

`cvt.satfinite.{f16, bf16, f16x2, bf16x2, tf32}.f32` introduced in PTX ISA version 8.1.

`cvt.{rn/rz}.satfinite.tf32.f32` introduced in PTX ISA version 8.6.

`cvt.rn.satfinite{.relu}.{e2m1x2/e2m3x2/e3m2x2/ue8m0x2}.f32` introduced in PTX ISA version 8.6.

`cvt.rn{.relu}.f16x2.{e2m1x2/e2m3x2/e3m2x2}` introduced in PTX ISA version 8.6.

`cvt.{rp/rz}{.satfinite}{.relu}.ue8m0x2.bf16x2` introduced in PTX ISA version 8.6.

`cvt.{rz/rp}.satfinite.ue8m0x2.f32` introduced in PTX ISA version 8.6.

`cvt.rn.bf16x2.ue8m0x2` introduced in PTX ISA version 8.6.

`.rs` rounding mode introduced in PTX ISA version 8.7.

`cvt.rs{.e2m1x4/.e4m3x4/.e5m2x4/.e3m2x4/.e2m3x4}.f32` introduced in PTX ISA version 8.7.

`cvt.rn.satfinite{.relu}{.e5m2x2/.e4m3x2}{.bf16x2}` introduced in PTX ISA version 9.1.

`cvt.rn.satfinite{.relu}{.e2m3x2/.e3m2x2/.e2m1x2}{.f16x2/.bf16x2}` introduced in PTX ISA version 9.1.

`cvt` with `.s2f6x2` instruction type introduced in PTX ISA version 9.1.

Target ISA Notes

`cvt` to or from `.f64` requires `sm_13` or higher.

`.relu` modifier and `{.f16x2, .bf16, .bf16x2, .tf32}` destination formats require `sm_80` or higher.

`cvt.f32.bf16` requires `sm_80` or higher.

`cvt.bf16.{u8/s8/u16/s16/u32/s32/u64/s64/f16/f64/bf16}`, `cvt.{u8/s8/u16/s16/u32/s32/u64/s64/f16/f64}.bf16`, and `cvt.tf32.f32.{relu}.{rn/rz}` require `sm_90` or higher.

`.ftz` qualifier for `cvt.f32.bf16` requires `sm_90` or higher.

`cvt` with `.e4m3x2/.e5m2x2` requires `sm_89` or higher.

`cvt.satfinite.{e4m3x2, e5m2x2}.{f32, f16x2}` requires `sm_89` or higher.

`cvt.{rn/rz}.satfinite.tf32.f32` requires `sm_100` or higher.

`cvt.rn.satfinite{.relu}.{e2m1x2/e2m3x2/e3m2x2/ue8m0x2}.f32` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

`cvt.rn{.relu}.f16x2.{e2m1x2/e2m3x2/e3m2x2}` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

`cvt.{rz/rp}{.satfinite}{.relu}.ue8m0x2.bf16x2` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)

- ▶ sm_120f or higher in the same family
- ▶ sm_110f or higher in the same family

`cvt.{rz/rp}.satfinite.ue8m0x2.f32` is supported on following architectures:

- ▶ sm_100a
- ▶ sm_101a (Renamed to sm_110a from PTX ISA version 9.0)
- ▶ sm_120a
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ sm_100f or higher in the same family
 - ▶ sm_101f or higher in the same family (Renamed to sm_110f from PTX ISA version 9.0)
 - ▶ sm_120f or higher in the same family
- ▶ sm_110f or higher in the same family

`cvt.rn.bf16x2.ue8m0x2` is supported on following architectures:

- ▶ sm_100a
- ▶ sm_101a (Renamed to sm_110a from PTX ISA version 9.0)
- ▶ sm_120a
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ sm_100f or higher in the same family
 - ▶ sm_101f or higher in the same family (Renamed to sm_110f from PTX ISA version 9.0)
 - ▶ sm_120f or higher in the same family
- ▶ sm_110f or higher in the same family

`.rs` rounding mode is supported on following architectures:

- ▶ sm_100a
- ▶ sm_103a

`cvt.rs{.e2m1x4/.e4m3x4/.e5m2x4/.e3m2x4/.e2m3x4}.f32` is supported on following architectures:

- ▶ sm_100a
- ▶ sm_103a

`cvt.rn.satfinite{.relu}{.e5m2x2/.e4m3x2}{.bf16x2}` is supported on following family-specific architectures:

- ▶ sm_100f or higher in the same family
- ▶ sm_110f or higher in the same family
- ▶ sm_120f or higher in the same family

`cvt.rn.satfinite{.relu}{.e2m3x2/.e3m2x2/.e2m1x2}{.f16x2/.bf16x2}` is supported on following family-specific architectures:

- ▶ sm_100f or higher in the same family
- ▶ sm_110f or higher in the same family
- ▶ sm_120f or higher in the same family

cvt with .s2f6x2 instruction type is supported on following architectures:

- sm_100a
- sm_103a
- sm_110a
- sm_120a
- sm_121a

Examples

```

cvt.f32.s32 f,i;
cvt.s32.f64 j,r;      // float-to-int saturates by default
cvt.rni.f32.f32 x,y;  // round to nearest int, result is fp
cvt.f32.f32 x,y;      // note .ftz behavior for sm_1x targets
cvt.rn.relu.f16.f32   b, f;      // result is saturated with .relu saturation
↪mode
cvt.rz.f16x2.f32      b1, f, f1;  // convert two fp32 values to packed fp16
↪outputs
cvt.rn.relu.satfinite.f16x2.f32   b1, f, f1;  // convert two fp32 values to packed
↪fp16 outputs with .relu saturation on each output
cvt.rn.bf16.f32       b, f;      // convert fp32 to bf16
cvt.rz.relu.satfinite.bf16.f32 2   b, f;      // convert fp32 to bf16 with .relu
↪and .satfinite saturation
cvt.rz.satfinite.bf16x2.f32      b1, f, f1;  // convert two fp32 values to packed
↪bf16 outputs
cvt.rn.relu.bf16x2.f32  b1, f, f1;  // convert two fp32 values to packed bf16
↪outputs with .relu saturation on each output
cvt.rna.satfinite.tf32.f32      b1, f;      // convert fp32 to tf32 format
cvt.rn.relu.tf32.f32   d, a;      // convert fp32 to tf32 format
cvt.f64.bf16.rp        f, b;      // convert bf16 to f64 format
cvt.bf16.f16.rz        b, f       // convert f16 to bf16 format
cvt.bf16.u64.rz        b, u       // convert u64 to bf16 format
cvt.s8.bf16.rpi        s, b       // convert bf16 to s8 format
cvt.bf16.bf16.rpi      b1, b2     // convert bf16 to corresponding int
↪represented in bf16 format
cvt.rn.satfinite.e4m3x2.f32 d, a, b; // convert a, b to .e4m3 and pack as .e4m3x2
↪output
cvt.rn.relu.satfinite.e5m2x2.f16x2 d, a; // unpack a and convert the values to .e5m2
↪outputs with .relu
                                     // saturation on each output and pack as .
↪e5m2x2
cvt.rn.f16x2.e4m3x2 d, a;           // unpack a, convert two .e4m3 values to packed
↪f16x2 output
cvt.rn.satfinite.tf32.f32 d, a;     // convert fp32 to tf32 format
cvt.rn.relu.f16x2.e2m1x2 d, a;     // unpack a, convert two .e2m1 values to packed
↪f16x2 output
cvt.rn.satfinite.e2m3x2.f32 d, a, b; // convert a, b to .e2m3 and pack as .e2m3x2
↪output
cvt.rn.relu.f16x2.e3m2x2 d, a;     // unpack a, convert two .e3m2 values to packed
↪f16x2 output

cvt.rs.f16x2.f32   d, a, b, rbits; // convert 2 fp32 values to packed fp16 with
↪applying .rs rounding
cvt.rs.satfinite.e2m1x4.f32 d, {a, b, e, f}, rbits; // convert 4 fp32 values to
↪packed 4 e2m1 values with applying .rs rounding

```

(continues on next page)

(continued from previous page)

```

cvt.rn.satfinite.relu.e2m1x2type.f16x2 d, a; // unpack a and covert to two .e2m1
↪ values
cvt.rn.satfinite.e2m3x2type.bf16x2 d, a; // unpack a and covert to two .e2m3 values

// Convert 2 f32 values to s2f6 after applying scale factor for dividing the value
cvt.rn.satfinite.scaled::n2::ue8m0.s2f6x2.f32 d, a, b, scale-factor;

```

9.7.9.22 Data Movement and Conversion Instructions: `cvt.pack`

`cvt.pack`

Convert two integer values from one integer type to another and pack the results.

Syntax

```

cvt.pack.sat.convertType.abType d, a, b;
    .convertType = { .u16, .s16 }
    .abType      = { .s32 }

cvt.pack.sat.convertType.abType.cType d, a, b, c;
    .convertType = { .u2, .s2, .u4, .s4, .u8, .s8 }
    .abType      = { .s32 }
    .cType       = { .b32 }

```

Description

Convert two 32-bit integers `a` and `b` into specified type and pack the results into `d`.

Destination `d` is an unsigned 32-bit integer. Source operands `a` and `b` are integers of type `.abType` and the source operand `c` is an integer of type `.cType`.

The inputs `a` and `b` are converted to values of type specified by `.convertType` with saturation and the results after conversion are packed into lower bits of `d`.

If operand `c` is specified then remaining bits of `d` are copied from lower bits of `c`.

Semantics

```

ta = a < MIN(convertType) ? MIN(convertType) : a;
ta = a > MAX(convertType) ? MAX(convertType) : a;
tb = b < MIN(convertType) ? MIN(convertType) : b;
tb = b > MAX(convertType) ? MAX(convertType) : b;

size = sizeInBits(convertType);
td = tb;
for (i = size; i <= 2 * size - 1; i++) {
    td[i] = ta[i - size];
}

```

(continues on next page)

(continued from previous page)

```

if (isU16(convertType) || isS16(convertType)) {
    d = td;
} else {
    for (i = 0; i < 2 * size; i++) {
        d[i] = td[i];
    }
    for (i = 2 * size; i <= 31; i++) {
        d[i] = c[i - 2 * size];
    }
}

```

.sat modifier limits the converted values to MIN(convertType).. MAX(convertedType) (no overflow) if the corresponding inputs are not in the range of datatype specified as .convertType.

PTX ISA Notes

Introduced in PTX ISA version 6.5.

Target ISA Notes

Requires sm_72 or higher.

Sub byte types (.u4/.s4 and .u2/.s2) requires sm_75 or higher.

Examples

```

cvt.pack.sat.s16.s32    %r1, %r2, %r3;           // 32-bit to 16-bit conversion
cvt.pack.sat.u8.s32.b32 %r4, %r5, %r6, 0;        // 32-bit to 8-bit conversion
cvt.pack.sat.u8.s32.b32 %r7, %r8, %r9, %r4;      // %r7 = { %r5, %r6, %r8, %r9 }
cvt.pack.sat.u4.s32.b32 %r10, %r12, %r13, %r14;  // 32-bit to 4-bit conversion
cvt.pack.sat.s2.s32.b32 %r15, %r16, %r17, %r18; // 32-bits to 2-bit conversion

```

9.7.9.23 Data Movement and Conversion Instructions: mapa

mapa

Map the address of the shared variable in the target CTA.

Syntax

```

mapa{.space}.type      d, a, b;

// Maps shared memory address in register a into CTA b.
mapa.shared::cluster.type d, a, b;

// Maps shared memory variable into CTA b.
mapa.shared::cluster.type d, sh, b;

```

(continues on next page)

(continued from previous page)

```
// Maps shared memory variable into CTA b.
mapa.shared::cluster.type  d, sh + imm, b;

// Maps generic address in register a into CTA b.
mapa.type                  d, a, b;

.space = { .shared::cluster }
.type   = { .u32, .u64 }
```

Description

Get address in the CTA specified by operand b which corresponds to the address specified by operand a.

Instruction type `.type` indicates the type of the destination operand d and the source operand a.

When space is `.shared::cluster`, source a is either a shared memory variable or a register containing a valid shared memory address and register d contains a shared memory address. When the optional qualifier `.space` is not specified, both a and d are registers containing generic addresses pointing to shared memory.

b is a 32-bit integer operand representing the rank of the target CTA.

Destination register d will hold an address in CTA b corresponding to operand a.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
mapa.shared::cluster.u64 d1, %reg1, cta;
mapa.shared::cluster.u32 d2, sh, 3;
mapa.u64                  d3, %reg2, cta;
```

9.7.9.24 Data Movement and Conversion Instructions: `getctarank`

`getctarank`

Generate the CTA rank of the address.

Syntax

```

getctarank{.space}.type d, a;

// Get cta rank from source shared memory address in register a.
getctarank.shared::cluster.type d, a;

// Get cta rank from shared memory variable.
getctarank.shared::cluster.type d, var;

// Get cta rank from shared memory variable+offset.
getctarank.shared::cluster.type d, var + imm;

// Get cta rank from generic address of shared memory variable in register a.
getctarank.type d, a;

.space = { .shared::cluster }
.type   = { .u32, .u64 }

```

Description

Write the destination register `d` with the rank of the CTA which contains the address specified in operand `a`.

Instruction type `.type` indicates the type of source operand `a`.

When `space` is `.shared::cluster`, source `a` is either a shared memory variable or a register containing a valid shared memory address. When the optional qualifier `.space` is not specified, `a` is a register containing a generic addresses pointing to shared memory. Destination `d` is always a 32-bit register which holds the rank of the CTA.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```

getctarank.shared::cluster.u32 d1, addr;
getctarank.shared::cluster.u64 d2, sh + 4;
getctarank.u64                  d3, src;

```

9.7.9.25 Data Movement and Conversion Instructions: Asynchronous copy

An asynchronous copy operation performs the underlying operation asynchronously in the background, thus allowing the issuing threads to perform subsequent tasks.

An asynchronous copy operation can be a *bulk* operation that operates on a large amount of data, or a *non-bulk* operation that operates on smaller sized data. The amount of data handled by a bulk asynchronous operation must be a multiple of 16 bytes.

An asynchronous copy operation typically includes the following sequence:

- ▶ Optionally, reading from the tensormap.
- ▶ Reading data from the source location(s).
- ▶ Writing data to the destination location(s).
- ▶ Writes being made visible to the executing thread or other threads.

9.7.9.25.1 Completion Mechanisms for Asynchronous Copy Operations

A thread must explicitly wait for the completion of an asynchronous copy operation in order to access the result of the operation. Once an asynchronous copy operation is initiated, modifying the source memory location or tensor descriptor or reading from the destination memory location before the asynchronous operation completes, exhibits undefined behavior.

This section describes two asynchronous copy operation completion mechanisms supported in PTX: Async-group mechanism and mbarrier-based mechanism.

Asynchronous operations may be tracked by either of the completion mechanisms or both mechanisms. The tracking mechanism is instruction/instruction-variant specific.

9.7.9.25.1.1 Async-group mechanism

When using the async-group completion mechanism, the issuing thread specifies a group of asynchronous operations, called *async-group*, using a *commit* operation and tracks the completion of this group using a *wait* operation. The thread issuing the asynchronous operation must create separate *async-groups* for bulk and non-bulk asynchronous operations.

A *commit* operation creates a per-thread *async-group* containing all prior asynchronous operations tracked by *async-group* completion and initiated by the executing thread but none of the asynchronous operations following the commit operation. A committed asynchronous operation belongs to a single *async-group*.

When an *async-group* completes, all the asynchronous operations belonging to that group are complete and the executing thread that initiated the asynchronous operations can read the result of the asynchronous operations. All *async-groups* committed by an executing thread always complete in the order in which they were committed. There is no ordering between asynchronous operations within an *async-group*.

A typical pattern of using *async-group* as the completion mechanism is as follows:

- ▶ Initiate the asynchronous operations.
- ▶ Group the asynchronous operations into an *async-group* using a *commit* operation.
- ▶ Wait for the completion of the *async-group* using the *wait* operation.

- Once the *async-group* completes, access the results of all asynchronous operations in that *async-group*.

9.7.9.25.1.2 Mbarrier-based mechanism

A thread can track the completion of one or more asynchronous operations using the current phase of an *mbarrier object*. When the current phase of the *mbarrier object* is complete, it implies that all asynchronous operations tracked by this phase are complete, and all threads participating in that *mbarrier object* can access the result of the asynchronous operations.

The *mbarrier object* to be used for tracking the completion of an asynchronous operation can be either specified along with the asynchronous operation as part of its syntax, or as a separate operation. For a bulk asynchronous operation, the *mbarrier object* must be specified in the asynchronous operation, whereas for non-bulk operations, it can be specified after the asynchronous operation.

A typical pattern of using mbarrier-based completion mechanism is as follows:

- Initiate the asynchronous operations.
- Set up an *mbarrier object* to track the asynchronous operations in its current phase, either as part of the asynchronous operation or as a separate operation.
- Wait for the *mbarrier object* to complete its current phase using `mbarrier.test_wait` or `mbarrier.try_wait`.
- Once the `mbarrier.test_wait` or `mbarrier.try_wait` operation returns True, access the results of the asynchronous operations tracked by the *mbarrier object*.

9.7.9.25.2 Async Proxy

The `cp{.reduce}.async.bulk` operations are performed in the *asynchronous proxy* (or *async proxy*).

Accessing the same memory location across multiple proxies needs a cross-proxy fence. For the *async proxy*, `fence.proxy.async` should be used to synchronize memory between *generic proxy* and the *async proxy*.

The completion of a `cp{.reduce}.async.bulk` operation is followed by an implicit *generic-async proxy fence*. So the result of the asynchronous operation is made visible to the generic proxy as soon as its completion is observed. *Async-group* OR *mbarrier-based* completion mechanism must be used to wait for the completion of the `cp{.reduce}.async.bulk` instructions.

9.7.9.25.3 Data Movement and Conversion Instructions: Non-bulk copy

9.7.9.25.3.1 Data Movement and Conversion Instructions: `cp.async`

`cp.async`

Initiates an asynchronous copy operation from one state space to another.

Syntax

```

cp.async.ca.shared{::cta}.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], cp-size{, src-size}{, cache-policy} ;
cp.async.cg.shared{::cta}.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], 16{, src-size}{, cache-policy} ;
cp.async.ca.shared{::cta}.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], cp-size{, ignore-src}{, cache-policy} ;
cp.async.cg.shared{::cta}.global{.level::cache_hint}{.level::prefetch_size}
    [dst], [src], 16{, ignore-src}{, cache-policy} ;

.level::cache_hint =    { .L2::cache_hint }
.level::prefetch_size = { .L2::64B, .L2::128B, .L2::256B }
cp-size =               { 4, 8, 16 }

```

Description

`cp.async` is a non-blocking instruction which initiates an asynchronous copy operation of data from the location specified by source address operand `src` to the location specified by destination address operand `dst`. Operand `src` specifies a location in the global state space and `dst` specifies a location in the shared state space.

Operand `cp-size` is an integer constant which specifies the size of data in bytes to be copied to the destination `dst`. `cp-size` can only be 4, 8 and 16.

Instruction `cp.async` allows optionally specifying a 32-bit integer operand `src-size`. Operand `src-size` represents the size of the data in bytes to be copied from `src` to `dst` and must be less than `cp-size`. In such case, remaining bytes in destination `dst` are filled with zeros. Specifying `src-size` larger than `cp-size` results in undefined behavior.

The optional and non-immediate predicate argument `ignore-src` specifies whether the data from the source location `src` should be ignored completely. If the source data is ignored then zeros will be copied to destination `dst`. If the argument `ignore-src` is not specified then it defaults to `False`.

Supported alignment requirements and addressing modes for operand `src` and `dst` are described in [Addresses as Operands](#).

The mandatory `.async` qualifier indicates that the `cp` instruction will initiate the memory copy operation asynchronously and control will return to the executing thread before the copy operation is complete. The executing thread can then use [async-group based completion mechanism](#) or the [mbarrier based completion mechanism](#) to wait for completion of the asynchronous copy operation. No other synchronization mechanism guarantees the completion of the asynchronous copy operations.

There is no ordering guarantee between two `cp.async` operations if they are not explicitly synchronized using `cp.async.wait_all` or `cp.async.wait_group` or [mbarrier instructions](#).

As described in [Cache Operators](#), the `.cg` qualifier indicates caching of data only at global level cache L2 and not at L1 whereas `.ca` qualifier indicates caching of data at all levels including L1 cache. Cache operator are treated as performance hints only.

`cp.async` is treated as a weak memory operation in the [Memory Consistency Model](#).

The `.level::prefetch_size` qualifier is a hint to fetch additional data of the specified size into the respective cache level. The sub-qualifier `prefetch_size` can be set to either of 64B, 128B, 256B thereby allowing the prefetch size to be 64 Bytes, 128 Bytes or 256 Bytes respectively.

The qualifier `.level::prefetch_size` may only be used with `.global` state space and with generic addressing where the address points to `.global` state space. If the generic address does not fall

within the address window of the global memory, then the prefetching behavior is undefined.

The `.level::prefetch_size` qualifier is treated as a performance hint only.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

The qualifier `.level::cache_hint` is only supported for `.global` state space and for generic addressing where the address points to the `.global` state space.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for `.level::cache_hint` and `.level::prefetch_size` qualifiers introduced in PTX ISA version 7.4.

Support for `ignore-src` operand introduced in PTX ISA version 7.5.

Support for sub-qualifier `::cta` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_80` or higher.

Sub-qualifier `::cta` requires `sm_30` or higher.

Examples

```
cp.async.ca.shared.global [shrd], [gbl + 4], 4;
cp.async.ca.shared::cta.global [%r0 + 8], [%r1], 8;
cp.async.cg.shared.global [%r2], [%r3], 16;

cp.async.cg.shared.global.L2::64B [%r2], [%r3], 16;
cp.async.cg.shared.global.L2::128B [%r0 + 16], [%r1], 16;
cp.async.cg.shared.global.L2::256B [%r2 + 32], [%r3], 16;

createpolicy.fractional.L2::evict_last.L2::evict_unchanged.b64 cache-policy, 0.25;
cp.async.ca.shared.global.L2::cache_hint [%r2], [%r1], 4, cache-policy;

cp.async.ca.shared.global [shrd], [gbl], 4, p;
cp.async.cg.shared.global.L2::cache_hint [%r0], [%r2], 16, q, cache-policy;
```

9.7.9.25.3.2 Data Movement and Conversion Instructions: `cp.async.commit_group`

`cp.async.commit_group`

Commits all prior initiated but uncommitted `cp.async` instructions into a *cp.async-group*.

Syntax

```
cp.async.commit_group ;
```

Description

`cp.async.commit_group` instruction creates a new *cp.async-group* per thread and batches all prior `cp.async` instructions initiated by the executing thread but not committed to any *cp.async-group* into the new *cp.async-group*. If there are no uncommitted `cp.async` instructions then `cp.async.commit_group` results in an empty *cp.async-group*.

An executing thread can wait for the completion of all `cp.async` operations in a *cp.async-group* using `cp.async.wait_group`.

There is no memory ordering guarantee provided between any two `cp.async` operations within the same *cp.async-group*. So two or more `cp.async` operations within a *cp.async-group* copying data to the same location results in undefined behavior.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
// Example 1:
cp.async.ca.shared.global [shrd], [gb1], 4;
cp.async.commit_group ; // Marks the end of a cp.async group

// Example 2:
cp.async.ca.shared.global [shrd1], [gb11], 8;
cp.async.ca.shared.global [shrd1+8], [gb11+8], 8;
cp.async.commit_group ; // Marks the end of cp.async group 1

cp.async.ca.shared.global [shrd2], [gb12], 16;
cp.async.cg.shared.global [shrd2+16], [gb12+16], 16;
cp.async.commit_group ; // Marks the end of cp.async group 2
```


9.7.9.25.3.3 Data Movement and Conversion Instructions: `cp.async.wait_group` / `cp.async.wait_all`

`cp.async.wait_group`, `cp.async.wait_all`

Wait for completion of prior asynchronous copy operations.

Syntax

```
cp.async.wait_group N;
cp.async.wait_all ;
```

Description

`cp.async.wait_group` instruction will cause executing thread to wait till only N or fewer of the most recent *cp.async-groups* are pending and all the prior *cp.async-groups* committed by the executing threads are complete. For example, when N is 0, the executing thread waits on all the prior *cp.async-groups* to complete. Operand N is an integer constant.

`cp.async.wait_all` is equivalent to :

```
cp.async.commit_group;
cp.async.wait_group 0;
```

An empty *cp.async-group* is considered to be trivially complete.

Writes performed by `cp.async` operations are made visible to the executing thread only after:

1. The completion of `cp.async.wait_all` or
2. The completion of `cp.async.wait_group` on the *cp.async-group* in which the `cp.async` belongs to or
3. *mbarrier.test_wait* returns True on an *mbarrier object* which is tracking the completion of the `cp.async` operation.

There is no ordering between two `cp.async` operations that are not synchronized with `cp.async.wait_all` or `cp.async.wait_group` or *mbarrier objects*.

`cp.async.wait_group` and `cp.async.wait_all` does not provide any ordering and visibility guarantees for any other memory operation apart from `cp.async`.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Target ISA Notes

Requires sm_80 or higher.

Examples

```
// Example of .wait_all:
cp.async.ca.shared.global [shrd1], [gbl1], 4;
cp.async.cg.shared.global [shrd2], [gbl2], 16;
cp.async.wait_all; // waits for all prior cp.async to complete

// Example of .wait_group :
cp.async.ca.shared.global [shrd3], [gbl3], 8;
cp.async.commit_group; // End of group 1

cp.async.cg.shared.global [shrd4], [gbl4], 16;
cp.async.commit_group; // End of group 2

cp.async.cg.shared.global [shrd5], [gbl5], 16;
cp.async.commit_group; // End of group 3

cp.async.wait_group 1; // waits for group 1 and group 2 to complete
```

9.7.9.25.4 Data Movement and Conversion Instructions: Bulk copy

9.7.9.25.4.1 Data Movement and Conversion Instructions: `cp.async.bulk`

`cp.async.bulk`

Initiates an asynchronous copy operation from one state space to another.

Syntax

```
// global -> shared::cta
cp.async.bulk.dst.src.completion_mechanism{.level::cache_hint}
    [dstMem], [srcMem], size, [mbar] {, cache-policy}

.dst =          { .shared::cta }
.src =          { .global }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.level::cache_hint =   { .L2::cache_hint }

// global -> shared::cluster
cp.async.bulk.dst.src.completion_mechanism{.multicast}{.level::cache_hint}
    [dstMem], [srcMem], size, [mbar] {, ctaMask} {, cache-policy}

.dst =          { .shared::cluster }
.src =          { .global }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.level::cache_hint =   { .L2::cache_hint }
```

(continues on next page)

(continued from previous page)

```

.multicast =          { .multicast::cluster  }

// shared::cta -> shared::cluster
cp.async.bulk.dst.src.completion_mechanism [dstMem], [srcMem], size, [mbar]

.dst =                { .shared::cluster  }
.src =                { .shared::cta  }
.completion_mechanism = { .mbarrier::complete_tx::bytes  }

// shared::cta -> global
cp.async.bulk.dst.src.completion_mechanism{.level::cache_hint}{.cp_mask}
                                [dstMem], [srcMem], size {, cache-policy} {, byteMask}

.dst =                { .global  }
.src =                { .shared::cta  }
.completion_mechanism = { .bulk_group  }
.level::cache_hint =  { .L2::cache_hint  }

```

Description

`cp.async.bulk` is a non-blocking instruction which initiates an asynchronous bulk-copy operation from the location specified by source address operand `srcMem` to the location specified by destination address operand `dstMem`.

The direction of bulk-copy is from the state space specified by the `.src` modifier to the state space specified by the `.dst` modifiers.

The 32-bit operand `size` specifies the amount of memory to be copied, in terms of number of bytes. `size` must be a multiple of 16. If the value is not a multiple of 16, then the behavior is undefined. The memory range `[dstMem, dstMem + size - 1]` must not overflow the destination memory space and the memory range `[srcMem, srcMem + size - 1]` must not overflow the source memory space. Otherwise, the behavior is undefined. The addresses `dstMem` and `srcMem` must be aligned to 16 bytes.

When the destination of the copy is `.shared::cta` the destination address has to be in the shared memory of the executing CTA within the cluster, otherwise the behavior is undefined.

When the source of the copy is `.shared::cta` and the destination is `.shared::cluster`, the destination has to be in the shared memory of a different CTA within the cluster.

The modifier `.completion_mechanism` specifies the completion mechanism that is supported on the instruction variant. The completion mechanisms that are supported for different variants are summarized in the following table:

.completion-mechanism	.dst	.src	Completion mechanism
<code>.mbarrier::...</code>	<code>.shared::cta</code>	<code>.global</code>	mbarrier based
	<code>.shared::cluster</code>	<code>.global</code>	
	<code>.shared::cluster</code>	<code>.shared::cta</code>	
<code>.bulk_group</code>	<code>.global</code>	<code>.shared::cta</code>	<i>Bulk async-group based</i>

The modifier `.mbarrier::complete_tx::bytes` specifies that the `cp.async.bulk` variant uses mbarrier based completion mechanism. The *complete-tx* operation, with `completeCount` argument equal to amount of data copied in bytes, will be performed on the mbarrier object specified by the operand `mbar`. This instruction accesses its `mbarrier` operand using generic-proxy.

The modifier `.bulk_group` specifies that the `cp.async.bulk` variant uses *bulk async-group* based completion mechanism.

The optional modifier `.multicast::cluster` allows copying of data from global memory to shared memory of multiple CTAs in the cluster. Operand `ctaMask` specifies the destination CTAs in the cluster such that each bit position in the 16-bit `ctaMask` operand corresponds to the `%ctaid` of the destination CTA. The source data is multicast to the same CTA-relative offset as `dstMem` in the shared memory of each destination CTA. The mbarrier signal is also multicast to the same CTA-relative offset as `mbar` in the shared memory of the destination CTA.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program. The qualifier `.level::cache_hint` is only supported when at least one of the `.src` or `.dst` state spaces is `.global` state space.

When the optional qualifier `.cp_mask` is specified, the argument `byteMask` is required. The *i*-th bit in the 16-bit wide `byteMask` operand specifies whether the *i*-th byte of each 16-byte wide chunk of source data is copied to the destination. If the bit is set, the byte is copied.

The copy operation in `cp.async.bulk` is treated as a weak memory operation and the *complete-tx* operation on the mbarrier has `.release` semantics at the `.cluster` scope as described in the *Memory Consistency Model*.

Notes

`.multicast::cluster` qualifier is optimized for target architecture `sm_90a/sm_100f/sm_100a/sm_103f/sm_103a/sm_110f/sm_110a` and may have substantially reduced performance on other targets and hence `.multicast::cluster` is advised to be used with `.target sm_90a/sm_100f/sm_100a/sm_103f/sm_103a/sm_110f/sm_110a`.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Support for `.shared::cta` as destination state space is introduced in PTX ISA version 8.6.

Support for `.cp_mask` qualifier introduced in PTX ISA version 8.6.

Target ISA Notes

Requires sm_90 or higher.

.multicast::cluster qualifier advised to be used with .target sm_90a or sm_100f or sm_100a or sm_103f or sm_103a or sm_110f or sm_110a.

Support for .cp_mask qualifier requires sm_100 or higher.

Examples

```
// .global -> .shared::cta (strictly non-remote):
cp.async.bulk.shared::cta.global.mbarrier::complete_tx::bytes [dstMem], [srcMem],
↳size, [mbar];

cp.async.bulk.shared::cta.global.mbarrier::complete_tx::bytes.L2::cache_hint
[dstMem], [srcMem], size, [mbar], cache-
↳policy;

// .global -> .shared::cluster:
cp.async.bulk.shared::cluster.global.mbarrier::complete_tx::bytes [dstMem], [srcMem],
↳size, [mbar];

cp.async.bulk.shared::cluster.global.mbarrier::complete_tx::bytes.multicast::cluster
[dstMem], [srcMem], size, [mbar],
↳ctaMask;

cp.async.bulk.shared::cluster.global.mbarrier::complete_tx::bytes.L2::cache_hint
[dstMem], [srcMem], size, [mbar], cache-
↳policy;

// .shared::cta -> .shared::cluster (strictly remote):
cp.async.bulk.shared::cluster.shared::cta.mbarrier::complete_tx::bytes [dstMem],
↳[srcMem], size, [mbar];

// .shared::cta -> .global:
cp.async.bulk.global.shared::cta.bulk_group [dstMem], [srcMem], size;

cp.async.bulk.global.shared::cta.bulk_group.L2::cache_hint} [dstMem], [srcMem], size,
↳cache-policy;

// .shared::cta -> .global with .cp_mask:
cp.async.bulk.global.shared::cta.bulk_group.L2::cache_hint.cp_mask [dstMem], [srcMem],
↳size, cache-policy, byteMask;
```

9.7.9.25.4.2 Data Movement and Conversion Instructions: `cp.reduce.async.bulk`

`cp.reduce.async.bulk`

Initiates an asynchronous reduction operation.

Syntax

```

cp.reduce.async.bulk.dst.src.completion_mechanism.redOp.type
    [dstMem], [srcMem], size, [mbar]

.dst =          { .shared::cluster }
.src =          { .shared::cta }
.completion_mechanism = { .mbARRIER::complete_tx::bytes }
.redOp=         { .and, .or, .xor,
                  .add, .inc, .dec,
                  .min, .max }
.type =         { .b32, .u32, .s32, .b64, .u64 }

cp.reduce.async.bulk.dst.src.completion_mechanism{.level::cache_hint}.redOp.type
    [dstMem], [srcMem], size{, cache-policy}

.dst =          { .global          }
.src =          { .shared::cta }
.completion_mechanism = { .bulk_group }
.level::cache_hint = { .L2::cache_hint }
.redOp=         { .and, .or, .xor,
                  .add, .inc, .dec,
                  .min, .max }
.type =         { .f16, .bf16, .b32, .u32, .s32, .b64, .u64, .s64, .f32, .f64
    ↪ }

cp.reduce.async.bulk.dst.src.completion_mechanism{.level::cache_hint}.add.noftz.type
    [dstMem], [srcMem], size{, cache-policy}
.dst =          { .global }
.src =          { .shared::cta }
.completion_mechanism = { .bulk_group }
.type =         { .f16, .bf16 }

```

Description

`cp.reduce.async.bulk` is a non-blocking instruction which initiates an asynchronous reduction operation on an array of memory locations specified by the destination address operand `dstMem` with the source array whose location is specified by the source address operand `srcMem`. The size of the source and the destination array must be the same and is specified by the operand `size`.

Each data element in the destination array is reduced inline with the corresponding data element in the source array with the reduction operation specified by the modifier `.redOp`. The type of each data element in the source and the destination array is specified by the modifier `.type`.

The source address operand `srcMem` is located in the state space specified by `.src` and the destination address operand `dstMem` is located in the state specified by the `.dst`.

The 32-bit operand size specifies the amount of memory to be copied from the source location and used in the reduction operation, in terms of number of bytes. `size` must be a multiple of 16. If the value is not a multiple of 16, then the behavior is undefined. The memory range `[dstMem, dstMem + size - 1]` must not overflow the destination memory space and the memory range `[srcMem, srcMem + size - 1]` must not overflow the source memory space. Otherwise, the behavior is undefined. The addresses `dstMem` and `srcMem` must be aligned to 16 bytes.

The operations supported by `.redOp` are classified as follows:

- ▶ The bit-size operations are `.and`, `.or`, and `.xor`.
- ▶ The integer operations are `.add`, `.inc`, `.dec`, `.min`, and `.max`. The `.inc` and `.dec` operations return a result in the range `[0..x]` where `x` is the value at the source state space.
- ▶ The floating point operation `.add` rounds to the nearest even. The current implementation of `cp.reduce.async.bulk.add.f32` flushes subnormal inputs and results to sign-preserving zero. The `cp.reduce.async.bulk.add.f16` and `cp.reduce.async.bulk.add.bf16` operations require `.noftz` qualifier. It preserves input and result subnormals, and does not flush them to zero.

The following table describes the valid combinations of `.redOp` and element type:

<code>.dst</code>	<code>.redOp</code>	Element type
<code>.shared::cluster</code>	<code>.add</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code>
	<code>.min</code> , <code>.max</code>	<code>.u32</code> , <code>.s32</code>
	<code>.inc</code> , <code>.dec</code>	<code>.u32</code>
	<code>.and</code> , <code>.or</code> , <code>.xor</code>	<code>.b32</code>
<code>.global</code>	<code>.add</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.f32</code> , <code>.f64</code> , <code>.f16</code> , <code>.bf16</code>
	<code>.min</code> , <code>.max</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.s64</code> , <code>.f16</code> , <code>.bf16</code>
	<code>.inc</code> , <code>.dec</code>	<code>.u32</code>
	<code>.and</code> , <code>.or</code> , <code>.xor</code>	<code>.b32</code> , <code>.b64</code>

The modifier `.completion_mechanism` specifies the completion mechanism that is supported on the instruction variant. The completion mechanisms that are supported for different variants are summarized in the following table:

<code>.completion-mechanism</code>	<code>.dst</code>	<code>.src</code>	Completion mechanism
<code>.mbarrier::...</code>	<code>.shared::cluster</code>	<code>.global</code>	mbarrier based
	<code>.shared::cluster</code>	<code>.shared::cta</code>	
<code>.bulk_group</code>	<code>.global</code>	<code>.shared::cta</code>	<i>Bulk async-group</i> based

The modifier `.mbarrier::complete_tx::bytes` specifies that the `cp.reduce.async.bulk` variant uses mbarrier based completion mechanism. The *complete-tx* operation, with `completeCount` argument equal to amount of data copied in bytes, will be performed on the mbarrier object specified by the operand `mbar`. This instruction accesses its `mbarrier` operand using generic-proxy.

The modifier `.bulk_group` specifies that the `cp.reduce.async.bulk` variant uses *bulk async-group* based completion mechanism.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program. The qualifier `.level::cache_hint` is only supported when at least one of the `.src` or `.dst` state spaces is `.global` state space.

Each reduction operation performed by the `cp.reduce.async.bulk` has individually `.relaxed.gpu` memory ordering semantics. The load operations in `cp.reduce.async.bulk` are treated as weak memory operation and the *complete-tx* operation on the mbarrier has `.release` semantics at the `.cluster` scope as described in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
cp.reduce.async.bulk.shared::cluster.shared::cta.mbarrier::complete_tx::bytes.add.u64
    [dstMem], [srcMem],
↪size, [mbar];

cp.reduce.async.bulk.shared::cluster.shared::cta.mbarrier::complete_tx::bytes.min.s32
    [dstMem], [srcMem],
↪size, [mbar];

cp.reduce.async.bulk.global.shared::cta.bulk_group.min.f16 [dstMem], [srcMem], size;

cp.reduce.async.bulk.global.shared::cta.bulk_group.L2::cache_hint.xor.s32 [dstMem],
↪[srcMem], size, policy;

cp.reduce.async.bulk.global.shared::cta.bulk_group.add.noftz.f16 [dstMem], [srcMem],
↪size;
```

9.7.9.25.4.3 Data Movement and Conversion Instructions: `cp.async.bulk.prefetch`

`cp.async.bulk.prefetch`

Provides a hint to the system to initiate the asynchronous prefetch of data to the cache.

Syntax

```
cp.async.bulk.prefetch.L2.src{.level::cache_hint}    [srcMem], size {, cache-policy}

.src =                { .global }
.level::cache_hint = { .L2::cache_hint }
```

Description

`cp.async.bulk.prefetch` is a non-blocking instruction which may initiate an asynchronous prefetch of data from the location specified by source address operand `srcMem`, in `.src` statespace, to the L2 cache.

The 32-bit operand `size` specifies the amount of memory to be prefetched in terms of number of bytes. `size` must be a multiple of 16. If the value is not a multiple of 16, then the behavior is undefined. The address `srcMem` must be aligned to 16 bytes.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

`cp.async.bulk.prefetch` is treated as a weak memory operation in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
cp.async.bulk.prefetch.L2.global                [srcMem], size;
cp.async.bulk.prefetch.L2.global.L2::cache_hint [srcMem], size, policy;
```

9.7.9.26 Data Movement and Conversion Instructions: `multimem.cp.async.bulk`

`multimem.cp.async.bulk`

Initiates an asynchronous copy operation to a `multimem` address range.

Syntax

```
multimem.cp.async.bulk.dst.src.completion_mechanism{.cp_mask}  
    [dstMem], [srcMem], size{, byteMask};  
  
    .dst                = { .global }  
    .src                = { .shared::cta }  
    .completion_mechanism = { .bulk_group }
```

Description

Instruction `multimem.cp.async.bulk` initiates an asynchronous bulk-copy operation from source address range `[srcMem, srcMem + size)` to memory locations residing on each GPU's memory referred to by the destination `multimem` address range `[dstMem, dstMem + size)`. The direction of bulk-copy is from the state space specified by the `.src` modifier to the state space specified by the `.dst` modifiers.

The 32-bit operand `size` specifies the amount of memory to be copied, in terms of number of bytes. Operand `size` must be a multiple of 16. The memory range `[dstMem, dstMem + size)` must not overflow the destination `multimem` memory space. The memory range `[srcMem, srcMem + size)` must not overflow the source memory space. The addresses `dstMem` and `srcMem` must be aligned to 16 bytes. If any of these pre-conditions is not met, the behavior is undefined.

The modifier `.completion_mechanism` specifies the completion mechanism that is supported by the instruction. The modifier `.bulk_group` specifies that the `multimem.cp.async.bulk` instruction uses bulk async-group based completion mechanism.

When the optional modifier `.cp_mask` is specified, the argument `byteMask` is required. The *i*-th bit in the 16-bit wide `byteMask` operand specifies whether the *i*-th byte of each 16-byte wide chunk of source data is copied to the destination. If the bit is set, the byte is copied.

The reads and writes of the copy operation in `multimem.cp.async.bulk` are weak memory operations as described in the [Memory Consistency Model](#).

PTX ISA Notes

Introduced in PTX ISA version 9.1.

Target ISA Notes

Requires `sm_90` or higher.

Support for `.cp_mask` qualifier requires `sm_100` or higher.

Examples

```
multimem.cp.async.bulk.global.shared::cta.bulk_group [dstMem], [srcMem], size;
multimem.cp.async.bulk.global.shared::cta.bulk_group [dstMem], [srcMem], 512;
multimem.cp.async.bulk.global.shared::cta.bulk_group.cp_mask [dstMem], [srcMem], size,
↪ byteMask;
```

9.7.9.27 Data Movement and Conversion Instructions: `multimem.cp.reduce.async.bulk`

`multimem.cp.reduce.async.bulk`

Initiates an asynchronous reduction operation to a multimem address range.

Syntax

```
multimem.cp.reduce.async.bulk.dst.src.completion_mechanism.redOp.type [dstMem],
↪ [srcMem], size;

    .dst                = { .global }
    .src                = { .shared::cta }
    .completion_mechanism = { .bulk_group }
    .redOp              = { .and, .or, .xor,
                           .add, .inc, .dec,
                           .min, .max }
    .type               = { .f16, .bf16,
                           .b32, .u32, .s32,
                           .b64, .u64, .s64,
                           .f32, .f64 }

multimem.cp.reduce.async.bulk.dst.src.completion_mechanism.add.noftz.type [dstMem],
↪ [srcMem], size;

    .dst                = { .global }
    .src                = { .shared::cta }
    .completion_mechanism = { .bulk_group }
    .type               = { .f16, .bf16 }
```

Description

Instruction `multimem.cp.reduce.async.bulk` initiates an element-wise asynchronous reduction operation with elements from source memory address range `[srcMem, srcMem + size)` to memory locations residing on each GPU's memory referred to by the multimem destination address range `[dstMem, dstMem + size)`.

Each data element in the destination array is reduced inline with the corresponding data element in the source array with the reduction operation specified by the modifier `.redOp`. The type of each data element in the source and the destination array is specified by the modifier `.type`.

The source address operand `srcMem` is in the state space specified by `.src` and the destination address operand `dstMem` is in the state specified by the `.dst`.

The 32-bit operand `size` specifies the amount of memory to be copied from the source location and used in the reduction operation, in terms of number of bytes. Operand `size` must be a multiple of 16. The memory range `[dstMem, dstMem + size)` must not overflow the destination multimem memory space. The memory range `[srcMem, srcMem + size)` must not overflow the source memory space. The addresses `dstMem` and `srcMem` must be aligned to 16 bytes. If any of these preconditions is not met, the behavior is undefined.

The operations supported by `.redOp` are classified as follows:

The bit-size operations are `.and`, `.or`, and `.xor`.

The integer operations are `.add`, `.inc`, `.dec`, `.min`, and `.max`. The `.inc` and `.dec` operations return a result in the range `[0..x]` where `x` is the value at the source state space.

The floating point operation `.add` rounds to the nearest even, preserve input and result subnormals, and does not flush them to zero, except for the current implementation of `multimem.cp.reduce.async.bulk.add.f32` which flushes subnormal inputs and results to sign-preserving zero. The `multimem.cp.reduce.async.bulk.add.f16` and `multimem.cp.reduce.async.bulk.add.bf16` operations require `.noftz` qualifier. It preserves input and result subnormals, and does not flush them to zero.

The following table describes the valid combinations of `.redOp` and element type:

.redOp	element type
<code>.add</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.f32</code> , <code>.f64</code> , <code>.f16</code> , <code>.bf16</code>
<code>.min</code> , <code>.max</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.s64</code> , <code>.f16</code> , <code>.bf16</code>
<code>.inc</code> , <code>.dec</code>	<code>.u32</code>
<code>.and</code> , <code>.or</code> , <code>.xor</code>	<code>.b32</code> , <code>.b64</code>

The modifier `.completion_mechanism` specifies the completion mechanism that is supported by the instruction. The modifier `.bulk_group` specifies that the `multimem.cp.reduce.async.bulk` uses bulk async-group based completion mechanism.

Each reduction operation performed by the `multimem.cp.reduce.async.bulk` has individually `.relaxed.sys` memory ordering semantics. The load operations in `multimem.cp.reduce.async.bulk` are treated as weak memory operations as described in the [Memory Consistency Model](#).

PTX ISA Notes

Introduced in PTX ISA version 9.1.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.add.u32 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.xor.b64 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.inc.u32 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.dec.u32 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.max.s32 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.add.noftz.f16 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.min.bf16 [dstMem],
↪[srcMem], size;

multimem.cp.reduce.async.bulk.global.shared::cta.bulk_group.add.noftz.bf16 [dstMem],
↪[srcMem], size;

```

9.7.9.27.1 Data Movement and Conversion Instructions: Tensor copy

9.7.9.27.1.1 Restriction on Tensor Copy instructions

Following are the restrictions on the types `.b4x16`, `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16`:

1. `cp.reduce.async.bulk` doesn't support the types `.b4x16`, `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16`.
2. `cp.async.bulk.tensor` with the direction `.global.shared::cta` doesn't support the type `.b4x16_p64`.
3. `cp.async.bulk.tensor` with the direction `.shared::cluster.global` doesn't support the sub-byte types on `sm_120a`.
4. OOB-NaN fill mode doesn't support the types `.b4x16`, `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16`.
5. `Box-Size[0]` must be exactly:
 1. 96B for `b6x16_p32` and `.b6p2x16`.
 2. 64B for `b4x16_p64`.
6. `Tensor-Size[0]` must be a multiple of:
 1. 96B for `b6x16_p32` and `.b6p2x16`.
 2. 64B for `b4x16_p64`.
7. For `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16`, the first coordinate in the `tensorCoords` argument vector must be a multiple of 128.

8. For `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16`, the global memory address must be 32B aligned. Additionally, tensor stride in every dimension must be 32B aligned.
9. `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16` supports the following swizzling modes:
 1. None.
 2. 128B (With all potential swizzle atomicity values except: 32B with 8B flip)

Following are the restrictions on the 96B swizzle mode:

1. The `.swizzle_atomicity` must be 16B.
2. The `.interleave_layout` must not be set.
3. `Box-Size[0]` must be less than or equal to 96B.
4. The type must not be among following: `.b4x16_p64`, `.b6x16_p32` and `.b6p2x16`.
5. The `.load_mode` must not be set to `.im2col:w:128`.

Following are the restrictions on the `.global.shared::cta` direction:

1. Starting co-ordinates for Bounding Box (`tensorCoords`) must be non-negative.
2. The bounding box along the D, W and H dimensions must stay within the tensor boundaries. This implies:
 1. Bounding-Box Lower-Corner must be non-negative.
 2. Bounding-Box Upper-Corner must be non-positive.

Following are the restrictions for `sm_120a`:

1. `cp.async.bulk.tensor` with the direction `.shared::cluster.global` doesn't support:
 1. the sub-byte types
 2. the qualifier `.swizzle_atomicity`

Following are the restrictions for `sm_103a` while using type `.b6p2x16` on `cp.async.bulk.tensor` with the direction `.global.shared::cta`:

1. `Box-Size[0]` must be exactly either of 48B or 96B.
2. The global memory address must be 16B aligned.
3. Tensor Stride in every dimension must be 16B aligned.
4. The first coordinate in the `tensorCoords` argument vector must be a multiple of 64.
5. `Tensor-Size[0]` must be a multiple of 48B.
6. The following swizzle modes are supported:
 1. None.
 2. 128B (With all potential swizzle atomicity values except: 32B with 8B flip)
 3. 64B swizzle with 16B swizzle atomicity

9.7.9.27.1.2 Data Movement and Conversion Instructions: `cp.async.bulk.tensor`

`cp.async.bulk.tensor`

Initiates an asynchronous copy operation on the tensor data from one state space to another.

Syntax

```
// global -> shared::cta
cp.async.bulk.tensor.dim.dst.src{.load_mode}.completion_mechanism{.cta_group}{.
↪level::cache_hint}
                                [dstMem], [tensorMap, tensorCoords], [mbar]{,
↪im2colInfo} {, cache-policy}

.dst =          { .shared::cta }
.src =          { .global }
.dim =          { .1d, .2d, .3d, .4d, .5d }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.cta_group =    { .cta_group::1, .cta_group::2 }
.load_mode =    { .tile, .tile::gather4, .im2col, .im2col::w, .im2col::w::128
↪}
.level::cache_hint = { .L2::cache_hint }

// global -> shared::cluster
cp.async.bulk.tensor.dim.dst.src{.load_mode}.completion_mechanism{.multicast}{.cta_
↪group}{.level::cache_hint}
                                [dstMem], [tensorMap, tensorCoords], [mbar]{,
↪im2colInfo}
                                {, ctaMask} {, cache-policy}

.dst =          { .shared::cluster }
.src =          { .global }
.dim =          { .1d, .2d, .3d, .4d, .5d }
.completion_mechanism = { .mbarrier::complete_tx::bytes }
.cta_group =    { .cta_group::1, .cta_group::2 }
.load_mode =    { .tile, .tile::gather4, .im2col, .im2col::w, .im2col::w::128
↪}
.level::cache_hint = { .L2::cache_hint }
.multicast =    { .multicast::cluster }

// shared::cta -> global
cp.async.bulk.tensor.dim.dst.src{.load_mode}.completion_mechanism{.level::cache_hint}
                                [tensorMap, tensorCoords], [srcMem] {, cache-
↪policy}

.dst =          { .global }
.src =          { .shared::cta }
.dim =          { .1d, .2d, .3d, .4d, .5d }
.completion_mechanism = { .bulk_group }
.load_mode =    { .tile, .tile::scatter4, .im2col_no_offs }
.level::cache_hint = { .L2::cache_hint }
```

Description

`cp.async.bulk.tensor` is a non-blocking instruction which initiates an asynchronous copy operation of tensor data from the location in `.src` state space to the location in the `.dst` state space.

The operand `dstMem` specifies the location in the `.dst` state space into which the tensor data has to be copied and `srcMem` specifies the location in the `.src` state space from which the tensor data has to be copied.

When `.dst` is specified as `.shared::cta`, the address `dstMem` must be in the shared memory of the executing CTA within the cluster, otherwise the behavior is undefined.

When `.dst` is specified as `.shared::cluster`, the address `dstMem` can be in the shared memory of any of the CTAs within the current cluster.

The operand `tensorMap` is the generic address of the opaque tensor-map object which resides in `.param` space or `.const` space or `.global` space. The operand `tensorMap` specifies the properties of the tensor copy operation, as described in [Tensor-map](#). The `tensorMap` is accessed in `tensormap` proxy. Refer to the *CUDA programming guide* for creating the tensor-map objects on the host side.

The dimension of the tensor data is specified by the `.dim` modifier.

The vector operand `tensorCoords` specifies the starting coordinates in the tensor data in the global memory from or to which the copy operation has to be performed. The individual tensor coordinates in `tensorCoords` are of type `.s32`. The format of vector argument `tensorCoords` is dependent on `.load_mode` specified and is as follows:

<code>.load_mode</code>	<code>tensorCoords</code>	Semantics
<code>.tile::scatter4</code>	{ <code>col_idx</code> , <code>row_idx0</code> , <code>row_idx1</code> , <code>row_idx2</code> , <code>row_idx3</code> }	Fixed length vector of size 5. The five elements together specify the start coordinates of the four rows.
<code>.tile::gather4</code>		
Rest all	{ <code>d0</code> , ..., <code>dn</code> } for <code>n = .dim</code>	Vector of <code>n</code> elements where <code>n = .dim</code> . The elements indicate the offset in each of the dimension.

The modifier `.completion_mechanism` specifies the completion mechanism that is supported on the instruction variant. The completion mechanisms that are supported for different variants are summarized in the following table:

.completion-mechanism	.dst	.src	Completion mechanism	
			Needed for completion of entire Async operation	optionally can be used for the completion of reading of the tensormap object
.mbarrier::...	.shared::cta	.global	mbarrier based	<i>Bulk async-group based</i>
	.shared::cluster	.global		
.bulk_group	.global	.shared::cta	<i>Bulk async-group based</i>	

The modifier `.mbarrier::complete_tx::bytes` specifies that the `cp.async.bulk.tensor` variant uses mbarrier based completion mechanism. Upon the completion of the asynchronous copy operation, the *complete-tx* operation, with `completeCount` argument equal to amount of data copied in bytes, will be performed on the mbarrier object specified by the operand `mbar`. This instruction accesses its `mbarrier` operand using generic-proxy.

The modifier `.cta_group` can only be specified with the mbarrier based completion mechanism. The modifier `.cta_group` is used to signal either the odd numbered CTA or the even numbered CTA among the *CTA-Pair*. When `.cta_group::1` is specified, the mbarrier object `mbar` that is specified must be in the shared memory of the same CTA as the shared memory destination `dstMem`. When `.cta_group::2` is specified, the mbarrier object `mbar` can be in shared memory of either the same CTA as the shared memory destination `dstMem` or in its *peer-CTA*. If `.cta_group` is not specified, then it defaults to `.cta_group::1`.

The modifier `.bulk_group` specifies that the `cp.async.bulk.tensor` variant uses *bulk async-group* based completion mechanism.

The qualifier `.load_mode` specifies how the data in the source location is copied into the destination location. If `.load_mode` is not specified, it defaults to `.tile`.

In `.tile` mode, the multi-dimensional layout of the source tensor is preserved at the destination. In `.tile::gather4` mode, four rows in 2-dimnesional source tensor are combined to form a single 2-dimensional destination tensor. In `.tile::scatter4` mode, single 2-dimensional source tensor is divided into four rows in 2-dimensional destination tensor. Details of `.tile::scatter4/`. `tile::gather4` modes are described in *tile::scatter4 and tile::gather4 modes*.

In `.im2col` and `.im2col::*` modes, some dimensions of the source tensors are unrolled in a single dimensional column at the destination. Details of the `im2col` and `.im2col::*` modes are described in *im2col mode* and *im2col::w and im2col::w::128 modes* respectively. In `.im2col` and `.im2col::*` modes, the tensor has to be at least 3-dimensional. The vector operand `im2colInfo` can be specified only when `.load_mode` is `.im2col` or `.im2col::w` or `.im2col::w::128`. The format of the vector argument `im2colInfo` is dependent on the exact `im2col` mode and is as follows:

Exact im2col mode	im2colInfo argument	Semantics
<code>.im2col</code>	{ i2cOffW , i2cOffH , i2cOffD } for <code>.dim = .5d</code>	A vector of im2col offsets whose vector size is two less than number of dimensions <code>.dim</code> .
<code>.im2col::w</code>	{ wHalo, wOffset }	A vector of 2 arguments containing <i>wHalo</i> and <i>wOffset</i> arguments.
<code>.im2col::w::128</code>		
<code>.im2col_no_offs</code>	im2colInfo is not applicable.	im2colInfo is not applicable.

Argument *wHalo* is a 16bit unsigned integer whose valid set of values differs on the load-mode and is as follows: - `Im2col::w` mode : valid range is [0, 512). - `Im2col::w::128` mode : valid range is [0, 32).

Argument *wOffset* is a 16bit unsigned integer whose valid range of values is [0, 32).

The optional modifier `.multicast::cluster` allows copying of data from global memory to shared memory of multiple CTAs in the cluster. Operand *ctaMask* specifies the destination CTAs in the cluster such that each bit position in the 16-bit *ctaMask* operand corresponds to the %*ctaid* of the destination CTA. The source data is multicast to the same offset as *dstMem* in the shared memory of each destination CTA. When `.cta_group` is specified as:

- ▶ `.cta_group::1` : The mbarrier signal is also multicast to the same offset as *mbar* in the shared memory of the destination CTA.
- ▶ `.cta_group::2` : The mbarrier signal is multicast either to all the odd numbered CTAs or the even numbered CTAs within the corresponding *CTA-Pair*. For each destination CTA specified in the *ctaMask*, the mbarrier signal is sent either to the destination CTA or its *peer-CTA* based on CTAs %*cluster_ctarank* parity of shared memory where the mbarrier object *mbar* resides.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

The copy operation in `cp.async.bulk.tensor` is treated as a weak memory operation and the *complete-tx* operation on the mbarrier has `.release` semantics at the `.cluster` scope as described in the *Memory Consistency Model*.

Notes

`.multicast::cluster` qualifier is optimized for target architecture `sm_90a/sm_100f/sm_100a/sm_103f/sm_103a/sm_110f/sm_110a` and may have substantially reduced performance on other targets and hence `.multicast::cluster` is advised to be used with `.target sm_90a/sm_100f/sm_100a/sm_103f/sm_103a/sm_110f/sm_110a`.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Support for `.shared::cta` as destination state space is introduced in PTX ISA version 8.6.

Support for qualifiers `.tile::gather4` and `.tile::scatter4` introduced in PTX ISA version 8.6.

Support for qualifiers `.im2col::w` and `.im2col::w::128` introduced in PTX ISA version 8.6.

Support for qualifier `.cta_group` introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_90` or higher.

`.multicast::cluster` qualifier advised to be used with `.target sm_90a` or `sm_100f` or `sm_100a` or `sm_103f` or `sm_103a` or `sm_110f` or `sm_110a`.

Qualifiers `.tile::gather4` and `.im2col::w` require:

- ▶ `sm_100a` when destination state space is `.shared::cluster` and is supported on `sm_100f` from PTX ISA version 8.8.
- ▶ `sm_100` or higher when destination state space is `.shared::cta`.

Qualifier `.tile::scatter4` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.im2col::w::128` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.cta_group` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```

.reg .b16 ctaMask;
.reg .u16 i2c0ffW, i2c0ffH, i2c0ffD;
.reg .b64 l2CachePolicy;

cp.async.bulk.tensor.1d.shared::cta.global.mbarrier::complete_tx::bytes.tile [sMem0],
↳ [tensorMap0, {tc0}], [mbar0];

@p cp.async.bulk.tensor.5d.shared::cta.global.im2col.mbarrier::complete_tx::bytes
    [sMem2], [tensorMap2, {tc0, tc1, tc2, tc3, tc4}], [mbar2],
↳ {i2c0ffW, i2c0ffH, i2c0ffD};

cp.async.bulk.tensor.1d.shared::cluster.global.mbarrier::complete_tx::bytes.tile
↳ [sMem0], [tensorMap0, {tc0}], [mbar0];

@p cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes.
↳ multicast::cluster
    [sMem1], [tensorMap1, {tc0, tc1}], [mbar2], ctaMask;

@p cp.async.bulk.tensor.5d.shared::cluster.global.im2col.mbarrier::complete_tx::bytes
    [sMem2], [tensorMap2, {tc0, tc1, tc2, tc3, tc4}], [mbar2],
↳ {i2c0ffW, i2c0ffH, i2c0ffD};

@p cp.async.bulk.tensor.3d.im2col.shared::cluster.global.mbarrier::complete_tx::bytes.
↳ L2::cache_hint
    [sMem3], [tensorMap3, {tc0, tc1, tc2}], [mbar3], {i2c0ffW},
↳ policy;

@p cp.async.bulk.tensor.1d.global.shared::cta.bulk_group [tensorMap3, {tc0}],
↳ [sMem3];

cp.async.bulk.tensor.2d.tile::gather4.shared::cluster.global.mbarrier::complete_
↳ tx::bytes
    [sMem5], [tensorMap6, {x0, y0, y1, y2, y3}], [mbar5];

cp.async.bulk.tensor.3d.im2col::w.shared::cluster.global.mbarrier::complete_tx::bytes
    [sMem4], [tensorMap5, {t0, t1, t2}], [mbar4], {im2colwHalo,
↳ im2col0ff};

cp.async.bulk.tensor.1d.shared::cluster.global.tile.cta_group::2
    [sMem6], [tensorMap7, {tc0}], [peerMbar];

```

9.7.9.27.1.3 Data Movement and Conversion Instructions: **cp.reduce.async.bulk.tensor****cp.reduce.async.bulk.tensor**

Initiates an asynchronous reduction operation on the tensor data.

Syntax

```
// shared::cta -> global:
cp.reduce.async.bulk.tensor.dim.dst.src.redOp{.load_mode}.completion_mechanism{.
↪level::cache_hint}
                                [tensorMap, tensorCoords], [srcMem] {,cache-
↪policy}

.dst =          { .global }
.src =          { .shared::cta }
.dim =          { .1d, .2d, .3d, .4d, .5d }
.completion_mechanism = { .bulk_group }
.load_mode =    { .tile, .im2col_no_offs }
.redOp =        { .add, .min, .max, .inc, .dec, .and, .or, .xor }
```

Description

`cp.reduce.async.bulk.tensor` is a non-blocking instruction which initiates an asynchronous reduction operation of tensor data in the `.dst` state space with tensor data in the `.src` state space.

The operand `srcMem` specifies the location of the tensor data in the `.src` state space using which the reduction operation has to be performed.

The operand `tensorMap` is the generic address of the opaque tensor-map object which resides in `.` param space or `.const` space or `.global` space. The operand `tensorMap` specifies the properties of the tensor copy operation, as described in [Tensor-map](#). The `tensorMap` is accessed in `tensormap` proxy. Refer to the *CUDA programming guide* for creating the tensor-map objects on the host side.

Each element of the tensor data in the `.dst` state space is reduced inline with the corresponding element from the tensor data in the `.src` state space. The modifier `.redOp` specifies the reduction operation used for the inline reduction. The type of each tensor data element in the source and the destination tensor is specified in [Tensor-map](#).

The dimension of the tensor is specified by the `.dim` modifier.

The vector operand `tensorCoords` specifies the starting coordinates of the tensor data in the global memory on which the reduce operation is to be performed. The number of tensor coordinates in the vector argument `tensorCoords` should be equal to the dimension specified by the modifier `.dim`. The individual tensor coordinates are of the type `.s32`.

The following table describes the valid combinations of `.redOp` and element type:

<code>.redOp</code>	Element type
<code>.add</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.f32</code> , <code>.f16</code> , <code>.bf16</code>
<code>.min</code> , <code>.max</code>	<code>.u32</code> , <code>.s32</code> , <code>.u64</code> , <code>.s64</code> , <code>.f16</code> , <code>.bf16</code>
<code>.inc</code> , <code>.dec</code>	<code>.u32</code>
<code>.and</code> , <code>.or</code> , <code>.xor</code>	<code>.b32</code> , <code>.b64</code>

The modifier `.completion_mechanism` specifies the completion mechanism that is supported on the instruction variant. Value `.bulk_group` of the modifier `.completion_mechanism` specifies that `cp.reduce.async.bulk.tensor` instruction uses *bulk async-group* based completion mechanism.

The qualifier `.load_mode` specifies how the data in the source location is copied into the destination location. If `.load_mode` is not specified, it defaults to `.tile`. In `.tile` mode, the multi-dimensional

layout of the source tensor is preserved at the destination. In `.im2col_no_offs` mode, some dimensions of the source tensors are unrolled in a single dimensional column at the destination. Details of the `im2col` mode are described in [im2col mode](#). In `.im2col` mode, the tensor has to be at least 3-dimensional.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program. The qualifier `.level::cache_hint` is only supported when at least one of the `.src` or `.dst` state spaces is `.global` state space.

Each reduction operation performed by `cp.reduce.async.bulk.tensor` has individually `.relaxed.gpu` memory ordering semantics. The load operations in `cp.reduce.async.bulk.tensor` are treated as weak memory operations and the [complete-tx](#) operation on the `mbarrier` has `.release` semantics at the `.cluster` scope as described in the [Memory Consistency Model](#).

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
cp.reduce.async.bulk.tensor.1d.global.shared::cta.add.tile.bulk_group
    [tensorMap0, {tc0}], [sMem0];

cp.reduce.async.bulk.tensor.2d.global.shared::cta.and.bulk_group.L2::cache_hint
    [tensorMap1, {tc0, tc1}], [sMem1] ,
↪policy;

cp.reduce.async.bulk.tensor.3d.global.shared::cta.xor.im2col.bulk_group
    [tensorMap2, {tc0, tc1, tc2}], [sMem2]
```

9.7.9.27.1.4 Data Movement and Conversion Instructions: `cp.async.bulk.prefetch.tensor`

`cp.async.bulk.prefetch.tensor`

Provides a hint to the system to initiate the asynchronous prefetch of tensor data to the cache.

Syntax

```
// global -> shared::cluster:
cp.async.bulk.prefetch.tensor.dim.L2.src{.load_mode}{.level::cache_hint} [tensorMap,
↪ tensorCoords]
                                {, im2colInfo } {, cache-
↪ policy}

.src =          { .global }
.dim =          { .1d, .2d, .3d, .4d, .5d }
.load_mode =    { .tile, .tile::gather4, .im2col, .im2col::w, .im2col::w::128 }
.level::cache_hint = { .L2::cache_hint }
```

Description

`cp.async.bulk.prefetch.tensor` is a non-blocking instruction which may initiate an asynchronous prefetch of tensor data from the location in `.src` statespace to the L2 cache.

The operand `tensorMap` is the generic address of the opaque tensor-map object which resides in `.` param space or `.const` space or `.global` space. The operand `tensorMap` specifies the properties of the tensor copy operation, as described in [Tensor-map](#). The `tensorMap` is accessed in `tensormap` proxy. Refer to the *CUDA programming guide* for creating the tensor-map objects on the host side.

The dimension of the tensor data is specified by the `.dim` modifier.

The vector operand `tensorCoords` specifies the starting coordinates in the tensor data in the global memory from which the copy operation has to be performed. The individual tensor coordinates in `tensorCoords` are of type `.s32`. The format of vector argument `tensorCoords` is dependent on `.load_mode` specified and is as follows:

<code>.load_mode</code>	<code>tensorCoords</code>	Semantics
<code>.tile::gather4</code>	{col_idx, row_idx0, row_idx1, row_idx2, row_idx3}	Fixed length vector of size 5. The five elements together specify the start coordinates of the four rows.
Rest all	{d0, .., dn} for n = .dim	Vector of n elements where n = .dim. The elements indicate the offset in each of the dimension.

The qualifier `.load_mode` specifies how the data in the source location is copied into the destination location. If `.load_mode` is not specified, it defaults to `.tile`.

In `.tile` mode, the multi-dimensional layout of the source tensor is preserved at the destination. In `.tile::gather4` mode, four rows in the 2-dimnesional source tensor are fetched to L2 cache. Details of `.tile::gather4` modes are described in [.tile::scatter4 and .tile::gather4 modes](#).

In `.im2col` and `.im2col::*` modes, some dimensions of the source tensors are unrolled in a single dimensional column at the destination. Details of the `im2col` and `.im2col::*` modes are described in [im2col mode](#) and [im2col::w and im2col::w::128 modes](#) respectively. In `.im2col` and `.im2col::*`

modes, the tensor has to be at least 3-dimensional. The vector operand `im2colInfo` can be specified only when `.load_mode` is `.im2col` or `.im2col::w` or `.im2col::w::128`. The format of the vector argument `im2colInfo` is dependent on the exact `im2col` mode and is as follows:

Exact <code>im2col</code> mode	<code>im2colInfo</code> argument	Semantics
<code>.im2col</code>	{ <code>i2cOffW</code> , <code>i2cOffH</code> , <code>i2cOffD</code> } for <code>.dim = .5d</code>	A vector of <code>im2col</code> offsets whose vector size is two less than number of dimensions <code>.dim</code> .
<code>.im2col::w</code>	{ <code>wHalo</code> , <code>wOffset</code> }	A vector of 2 arguments containing <i>wHalo</i> and <i>wOffset</i> arguments.
<code>.im2col::w::128</code>		

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

`cp.async.bulk.prefetch.tensor` is treated as a weak memory operation in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Support for qualifier `.tile::gather4` introduced in PTX ISA version 8.6.

Support for qualifiers `.im2col::w` and `.im2col::w::128` introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_90` or higher.

Qualifier `.tile::gather4` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifiers `.im2col::w` and `.im2col::w::128` are supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)

- ▶ And are supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ sm_100f or higher in the same family
 - ▶ sm_101f or higher in the same family (Renamed to sm_110f from PTX ISA version 9.0)
- ▶ sm_110f or higher in the same family

Examples

```
.reg .b16 ctaMask, im2colwHalo, im2col0ff;
.reg .u16 i2c0ffW, i2c0ffH, i2c0ffD;
.reg .b64 l2CachePolicy;

cp.async.bulk.prefetch.tensor.1d.L2.global.tile [tensorMap0, {tc0}];

@p cp.async.bulk.prefetch.tensor.2d.L2.global [tensorMap1, {tc0, tc1}];

@p cp.async.bulk.prefetch.tensor.5d.L2.global.im2col
    [tensorMap2, {tc0, tc1, tc2, tc3, tc4}], {i2c0ffW, i2c0ffH,
    ↪ i2c0ffD};

@p cp.async.bulk.prefetch.tensor.3d.L2.global.im2col.L2::cache_hint
    [tensorMap3, {tc0, tc1, tc2}], {i2c0ffW}, policy;

cp.async.bulk.prefetch.tensor.2d.L2.global.tile::gather4 [tensorMap5, {col_idx, row_
    ↪ idx0, row_idx1, row_idx2, row_idx3}];

cp.async.bulk.prefetch.tensor.4d.L2.global.im2col::w::128
    [tensorMap4, {t0, t1, t2, t3}], {im2colwHalo, im2col0ff};
```

9.7.9.27.2 Data Movement and Conversion Instructions: Bulk and Tensor copy completion instructions

9.7.9.27.2.1 Data Movement and Conversion Instructions: `cp.async.bulk.commit_group`

`cp.async.bulk.commit_group`

Commits all prior initiated but uncommitted `cp.async.bulk` instructions into a *cp.async.bulk-group*.

Syntax

```
cp.async.bulk.commit_group;
```

Description

`cp.async.bulk.commit_group` instruction creates a new per-thread *bulk async-group* and batches all prior `cp{.reduce}.async.bulk.{.prefetch}{.tensor}` instructions satisfying the following conditions into the new *bulk async-group*:

- ▶ The prior `cp{.reduce}.async.bulk.{.prefetch}{.tensor}` instructions use *bulk_group* based completion mechanism, and
- ▶ They are initiated by the executing thread but not committed to any *bulk async-group*.

If there are no uncommitted `cp{.reduce}.async.bulk.{.prefetch}{.tensor}` instructions then `cp.async.bulk.commit_group` results in an empty *bulk async-group*.

An executing thread can wait for the completion of all `cp{.reduce}.async.bulk.{.prefetch}{.tensor}` operations in a *bulk async-group* using `cp.async.bulk.wait_group`.

There is no memory ordering guarantee provided between any two `cp{.reduce}.async.bulk.{.prefetch}{.tensor}` operations within the same *bulk async-group*.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
cp.async.bulk.commit_group;
```

9.7.9.27.2.2 Data Movement and Conversion Instructions: `cp.async.bulk.wait_group`

`cp.async.bulk.wait_group`

Wait for completion of *bulk async-groups*.

Syntax

```
cp.async.bulk.wait_group{.read} N;
```

Description

`cp.async.bulk.wait_group` instruction will cause the executing thread to wait until only N or fewer of the most recent *bulk async-groups* are pending and all the prior *bulk async-groups* committed by the executing threads are complete. For example, when N is 0, the executing thread waits on all the prior *bulk async-groups* to complete. Operand N is an integer constant.

By default, `cp.async.bulk.wait_group` instruction will cause the executing thread to wait until completion of all the bulk async operations in the specified *bulk async-group*. A bulk async operation includes the following:

- ▶ Optionally, reading from the tensormap.
- ▶ Reading from the source locations.
- ▶ Writing to their respective destination locations.
- ▶ Writes being made visible to the executing thread.

The optional `.read` modifier indicates that the waiting has to be done until all the bulk async operations in the specified *bulk async-group* have completed:

1. reading from the tensormap
2. the reading from their source locations.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
cp.async.bulk.wait_group.read    0;
cp.async.bulk.wait_group        2;
```

9.7.9.28 Data Movement and Conversion Instructions: `tensormap.replace`

`tensormap.replace`

Modifies the field of a tensor-map object.

Syntax

```
tensormap.replace.mode.field1{.ss}.b1024.type  [addr], new_val;  
tensormap.replace.mode.field2{.ss}.b1024.type  [addr], ord, new_val;  
tensormap.replace.mode.field3{.ss}.b1024.type  [addr], new_val;  
  
.mode      = { .tile }  
.field1    = { .global_address, .rank }  
.field2    = { .box_dim, .global_dim, .global_stride, .element_stride }  
.field3    = { .elemtype, .interleave_layout, .swizzle_mode, .swizzle_atomicsity, .fill_  
↪mode }  
.ss        = { .global, .shared::cta }  
.type      = { .b32, .b64 }
```

Description

The `tensormap.replace` instruction replaces the field, specified by `.field` qualifier, of the tensor-map object at the location specified by the address operand `addr` with a new value. The new value is specified by the argument `new_val`.

Qualifier `.mode` specifies the mode of the *tensor-map* object located at the address operand `addr`.

Instruction type `.b1024` indicates the size of the *tensor-map* object, which is 1024 bits.

Operand `new_val` has the type `.type`. When `.field` is specified as `.global_address` or `.global_stride`, `.type` must be `.b64`. Otherwise, `.type` must be `.b32`.

The immediate integer operand `ord` specifies the ordinal of the field across the rank of the tensor which needs to be replaced in the *tensor-map* object.

For field `.rank`, the operand `new_val` must be ones less than the desired tensor rank as this field uses zero-based numbering.

When `.field3` is specified, the operand `new_val` must be an immediate and the [Table 33](#) shows the mapping of the operand `new_val` across various fields.

Table 33: Tensormap new_val validity

new_val	.field3				
	.elemtype	.inter-leave_layout	.swiz-zle_mode	.swiz-zle_atomicity	.fill_mode
0	.u8	No interleave	No swizzling	16B	Zero fill
1	.u16	16B interleave	32B swiz-zling	32B	OOB-NaN fill
2	.u32	32B interleave	64B swiz-zling	32B + 8B flip	x
3	.s32	x	128B swiz-zling	64B	x
4	.u64	x	96B swiz-zling	x	x
5	.s64	x	x	x	x
6	.f16	x	x	x	x
7	.f32	x	x	x	x
8	.f32.ftz	x	x	x	x
9	.f64	x	x	x	x
10	.bf16	x	x	x	x
11	.tf32	x	x	x	x
12	.tf32.ftz	x	x	x	x
13	.b4x16	x	x	x	x
14	.b4x16_p64	x	x	x	x
15	.b6x16_p32 or .b6p2x16	x	x	x	x

Note: The values of `.elemtype` do not correspond to the values of the `CUTensorMapDataType` enum used in the driver API.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.global` or `.shared::cta` state space then the behavior is undefined.

`tensormap.replace` is treated as a weak memory operation, on the entire 1024-bit opaque *tensor-map* object, in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.3.

Qualifier `.swizzle_atomics` introduced in PTX ISA version 8.6.

Qualifier `.elemtype` with values from 13 to 15, both inclusive, is supported in PTX ISA version 8.7 onwards.

Qualifier `.swizzle_mode` with value 4 is supported from PTX ISA version 8.8 onwards.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_90a`
- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Qualifier `.swizzle_atomics` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a` (refer to [section](#) for restrictions on `sm_120a`)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

`.field3` variant `.elemtype` corresponding to `new_val` values 13, 14 and 15 is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a` (refer to [section](#) for restrictions on `sm_120a`)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family

- ▶ `sm_110f` or higher in the same family

`.field3` variant `.swizzle_mode` corresponding to `new_val` value 4 is supported on following architectures:

- ▶ `sm_103a` (refer to [section](#) for restrictions on `sm_103a`)

Examples

```
tensormap.replace.tile.global_address.shared::cta.b1024.b64    [sMem], new_val;
```

9.7.10. Texture Instructions

This section describes PTX instructions for accessing textures and samplers. PTX supports the following operations on texture and sampler descriptors:

- ▶ Static initialization of texture and sampler descriptors.
- ▶ Module-scope and per-entry scope definitions of texture and sampler descriptors.
- ▶ Ability to query fields within texture and sampler descriptors.

9.7.10.1 Texturing Modes

For working with textures and samplers, PTX has two modes of operation. In the *unified mode*, texture and sampler information is accessed through a single `.texref` handle. In the *independent mode*, texture and sampler information each have their own handle, allowing them to be defined separately and combined at the site of usage in the program.

The advantage of unified mode is that it allows 256 samplers per kernel (128 for architectures prior to `sm_3x`), with the restriction that they correspond 1-to-1 with the 256 possible textures per kernel (128 for architectures prior to `sm_3x`). The advantage of independent mode is that textures and samplers can be mixed and matched, but the number of samplers is greatly restricted to 32 per kernel (16 for architectures prior to `sm_3x`).

[Table 34](#) summarizes the number of textures, samplers and surfaces available in different texturing modes.

Table 34: Texture, sampler and surface limits

Texturing mode	Resource	sm_1x, sm_2x	sm_3x+
Unified mode	Textures	128	256
	Samplers	128	256
	Surfaces	8	16
Independent mode	Textures	128	256
	Samplers	16	32
	Surfaces	8	16

The texturing mode is selected using `.target` options `texmode_unified` and `texmode_independent`. A PTX module may declare only one texturing mode. If no texturing mode is declared, the module is assumed to use unified mode.

Example: calculate an element's power contribution as element's power/total number of elements.

```
.target texmode_independent
.global .samplerref tsamp1 = { addr_mode_0 = clamp_to_border,
                               filter_mode = nearest
                               };
...
.entry compute_power
( .param .texref tex1 )
{
    txq.width.b32 r6, [tex1]; // get tex1's width
    txq.height.b32 r5, [tex1]; // get tex1's height
    tex.2d.v4.f32.f32 {r1,r2,r3,r4}, [tex1, tsamp1, {f1,f2}];
    mul.u32 r5, r5, r6;
    add.f32 r1, r1, r2;
    add.f32 r3, r3, r4;
    add.f32 r1, r1, r3;
    cvt.f32.u32 r5, r5;
    div.f32 r1, r1, r5;
}
```

9.7.10.2 Mipmaps

A *mipmap* is a sequence of textures, each of which is a progressively lower resolution representation of the same image. The height and width of each image, or level of detail (LOD), in the mipmap is a power of two smaller than the previous level. Mipmaps are used in graphics applications to improve rendering speed and reduce aliasing artifacts. For example, a high-resolution mipmap image is used for objects that are close to the user; lower-resolution images are used as the object appears farther away. Mipmap filtering modes are provided when switching between two levels of detail (LODs) in order to avoid abrupt changes in visual fidelity.

Example: If the texture has a basic size of 256 by 256 pixels, then the associated mipmap set may contain a series of eight images, each one-fourth the total area of the previous one: 128x128 pixels, 64x64, 32x32, 16x16, 8x8, 4x4, 2x2, 1x1 (a single pixel). If, for example, a scene is rendering this texture in a space of 40x40 pixels, then either a scaled up version of the 32x32 (without trilinear interpolation) or an interpolation of the 64x64 and the 32x32 mipmaps (with trilinear interpolation) would be used.

The total number of LODs in a complete mipmap pyramid is calculated through the following equation:

$$\text{numLODs} = 1 + \text{floor}(\log_2(\max(w, h, d)))$$

The finest LOD is called the base level and is the 0th level. The next (coarser) level is the 1st level, and so on. The coarsest level is the level of size (1 x 1 x 1). Each successively smaller mipmap level has half the {width, height, depth} of the previous level, but if this half value is a fractional value, it's rounded down to the next largest integer. Essentially, the size of a mipmap level can be specified as:

```
max(1, floor(w_b / 2^i)) x
max(1, floor(h_b / 2^i)) x
max(1, floor(d_b / 2^i))
```

where *i* is the *i*th level beyond the 0th level (the base level). And *w_b*, *h_b* and *d_b* are the width, height and depth of the base level respectively.

PTX support for mipmaps

The PTX `tex` instruction supports three modes for specifying the LOD: *base*, *level*, and *gradient*. In base mode, the instruction always picks level 0. In level mode, an additional argument is provided to specify the LOD to fetch from. In gradmode, two floating-point vector arguments provide *partials* (e.g., $\{ds/dx, dt/dx\}$ and $\{ds/dy, dt/dy\}$ for a 2d texture), which the `tex` instruction uses to compute the LOD.

These instructions provide access to texture memory.

- ▶ `tex`
- ▶ `tld4`
- ▶ `txq`

9.7.10.3 Texture Instructions: `tex`

`tex`

Perform a texture memory lookup.

Syntax

```
tex.geom.v4.dtype.ctype d, [a, c] {, e} {, f};
tex.geom.v4.dtype.ctype d[|p], [a, b, c] {, e} {, f}; // explicit sampler

tex.geom.v2.f16x2.ctype d[|p], [a, c] {, e} {, f};
tex.geom.v2.f16x2.ctype d[|p], [a, b, c] {, e} {, f}; // explicit sampler

// mipmaps
tex.base.geom.v4.dtype.ctype d[|p], [a, {b,} c] {, e} {, f};
tex.level.geom.v4.dtype.ctype d[|p], [a, {b,} c], lod {, e} {, f};
tex.grad.geom.v4.dtype.ctype d[|p], [a, {b,} c], dPdx, dPdy {, e} {, f};

tex.base.geom.v2.f16x2.ctype d[|p], [a, {b,} c] {, e} {, f};
tex.level.geom.v2.f16x2.ctype d[|p], [a, {b,} c], lod {, e} {, f};
tex.grad.geom.v2.f16x2.ctype d[|p], [a, {b,} c], dPdx, dPdy {, e} {, f};

.geom = { .1d, .2d, .3d, .a1d, .a2d, .cube, .acube, .2dms, .a2dms };
.dtype = { .u32, .s32, .f16, .f32 };
.ctype = {          .s32, .f32 };           // .cube, .acube require .f32
                                           // .2dms, .a2dms require .s32
```

Description

`tex. {1d, 2d, 3d}`

Texture lookup using a texture coordinate vector. The instruction loads data from the texture named by operand `a` at coordinates given by operand `c` into destination `d`. Operand `c` is a scalar or singleton tuple for 1d textures; is a two-element vector for 2d textures; and is a four-element vector for 3d textures, where the fourth element is ignored. An optional texture sampler `b` may be specified. If no sampler is specified, the sampler behavior is a property of the named texture. The optional destination predicate `p` is set to `True` if data from texture at specified coordinates is resident in memory, `False`

otherwise. When optional destination predicate `p` is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

An optional operand `e` may be specified. Operand `e` is a vector of `.s32` values that specifies coordinate offset. Offset is applied to coordinates before doing texture lookup. Offset value is in the range of -8 to +7. Operand `e` is a singleton tuple for 1d textures; is a two element vector 2d textures; and is four-element vector for 3d textures, where the fourth element is ignored.

An optional operand `f` may be specified for depth textures. Depth textures are special type of textures which hold data from the depth buffer. Depth buffer contains depth information of each pixel. Operand `f` is `.f32` scalar value that specifies depth compare value for depth textures. Each element fetched from texture is compared against value given in `f` operand. If comparison passes, result is 1.0; otherwise result is 0.0. These per-element comparison results are used for the filtering. When using depth compare operand, the elements in texture coordinate vector `c` have `.f32` type.

Depth compare operand is not supported for 3d textures.

The instruction returns a two-element vector for destination type `.f16x2`. For all other destination types, the instruction returns a four-element vector. Coordinates may be given in either signed 32-bit integer or 32-bit floating point form.

A texture base address is assumed to be aligned to a 16 byte boundary, and the address given by the coordinate vector must be naturally aligned to a multiple of the access size. If an address is not properly aligned, the resulting behavior is undefined; i.e., the access may proceed by silently masking off low-order address bits to achieve proper rounding, or the instruction may fault.

`tex. {a1d, a2d}`

Texture array selection, followed by texture lookup. The instruction first selects a texture from the texture array named by operand `a` using the index given by the first element of the array coordinate vector `c`. The instruction then loads data from the selected texture at coordinates given by the remaining elements of operand `c` into destination `d`. Operand `c` is a bit-size type vector or tuple containing an index into the array of textures followed by coordinates within the selected texture, as follows:

- For 1d texture arrays, operand `c` has type `.v2.b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the texture array, and the second element is interpreted as a 1d texture coordinate of type `.ctype`.
- For 2d texture arrays, operand `c` has type `.v4.b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the texture array, and the next two elements are interpreted as 2d texture coordinates of type `.ctype`. The fourth element is ignored.

An optional texture sampler `b` may be specified. If no sampler is specified, the sampler behavior is a property of the named texture.

An optional operand `e` may be specified. Operand `e` is a vector of `.s32` values that specifies coordinate offset. Offset is applied to coordinates before doing texture lookup. Offset value is in the range of -8 to +7. Operand `e` is a singleton tuple for 1d texture arrays; and is a two element vector 2d texture arrays.

An optional operand `f` may be specified for depth textures arrays. Operand `f` is `.f32` scalar value that specifies depth compare value for depth textures. When using depth compare operand, the coordinates in texture coordinate vector `c` have `.f32` type.

The instruction returns a two-element vector for destination type `.f16x2`. For all other destination types, the instruction returns a four-element vector. The texture array index is a 32-bit unsigned integer, and texture coordinate elements are 32-bit signed integer or floating point values.

The optional destination predicate `p` is set to `True` if data from texture at specified coordinates is resident in memory, `False` otherwise. When optional destination predicate `p` is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

`tex.cube`

Cubemap texture lookup. The instruction loads data from the cubemap texture named by operand `a` at coordinates given by operand `c` into destination `d`. Cubemap textures are special two-dimensional layered textures consisting of six layers that represent the faces of a cube. All layers in a cubemap are of the same size and are square (i.e., width equals height).

When accessing a cubemap, the texture coordinate vector `c` has type `.v4.f32`, and comprises three floating-point coordinates (`s`, `t`, `r`) and a fourth padding argument which is ignored. Coordinates (`s`, `t`, `r`) are projected onto one of the six cube faces. The (`s`, `t`, `r`) coordinates can be thought of as a direction vector emanating from the center of the cube. Of the three coordinates (`s`, `t`, `r`), the coordinate of the largest magnitude (the major axis) selects the cube face. Then, the other two coordinates (the minor axes) are divided by the absolute value of the major axis to produce a new (`s`, `t`) coordinate pair to lookup into the selected cube face.

An optional texture sampler `b` may be specified. If no sampler is specified, the sampler behavior is a property of the named texture.

Offset vector operand `e` is not supported for cubemap textures.

an optional operand `f` may be specified for cubemap depth textures. operand `f` is `.f32` scalar value that specifies depth compare value for cubemap depth textures.

The optional destination predicate `p` is set to `True` if data from texture at specified coordinates is resident in memory, `False` otherwise. When optional destination predicate `p` is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

`tex.acube`

Cubemap array selection, followed by cubemap lookup. The instruction first selects a cubemap texture from the cubemap array named by operand `a` using the index given by the first element of the array coordinate vector `c`. The instruction then loads data from the selected cubemap texture at coordinates given by the remaining elements of operand `c` into destination `d`.

Cubemap array textures consist of an array of cubemaps, i.e., the total number of layers is a multiple of six. When accessing a cubemap array texture, the coordinate vector `c` has type `.v4.b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the cubemap array, and the remaining three elements are interpreted as floating-point cubemap coordinates (`s`, `t`, `r`), used to lookup in the selected cubemap as described above.

An optional texture sampler `b` may be specified. If no sampler is specified, the sampler behavior is a property of the named texture.

Offset vector operand `e` is not supported for cubemap texture arrays.

An optional operand `f` may be specified for cubemap depth texture arrays. Operand `f` is `.f32` scalar value that specifies depth compare value for cubemap depth textures.

The optional destination predicate `p` is set to `True` if data from texture at specified coordinates is resident in memory, `False` otherwise. When optional destination predicate `p` is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

tex.2dms

Multi-sample texture lookup using a texture coordinate vector. Multi-sample textures consist of multiple samples per data element. The instruction loads data from the texture named by operand *a* from sample number given by first element of the operand *c*, at coordinates given by remaining elements of operand *c* into destination *d*. When accessing a multi-sample texture, texture coordinate vector *c* has type `.v4.b32`. The first element in operand *c* is interpreted as unsigned integer sample number (`.u32`), and the next two elements are interpreted as signed integer (`.s32`) 2d texture coordinates. The fourth element is ignored. An optional texture sampler *b* may be specified. If no sampler is specified, the sampler behavior is a property of the named texture.

An optional operand *e* may be specified. Operand *e* is a vector of type `.v2.s32` that specifies coordinate offset. Offset is applied to coordinates before doing texture lookup. Offset value is in the range of -8 to +7.

Depth compare operand *f* is not supported for multi-sample textures.

The optional destination predicate *p* is set to `True` if data from texture at specified coordinates is resident in memory, `False` otherwise. When optional destination predicate *p* is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

tex.a2dms

Multi-sample texture array selection, followed by multi-sample texture lookup. The instruction first selects a multi-sample texture from the multi-sample texture array named by operand *a* using the index given by the first element of the array coordinate vector *c*. The instruction then loads data from the selected multi-sample texture from sample number given by second element of the operand *c*, at coordinates given by remaining elements of operand *c* into destination *d*. When accessing a multi-sample texture array, texture coordinate vector *c* has type `.v4.b32`. The first element in operand *c* is interpreted as unsigned integer sampler number, the second element is interpreted as unsigned integer index (`.u32`) into the multi-sample texture array and the next two elements are interpreted as signed integer (`.s32`) 2d texture coordinates. An optional texture sampler *b* may be specified. If no sampler is specified, the sampler behavior is a property of the named texture.

An optional operand *e* may be specified. Operand *e* is a vector of type `.v2.s32` values that specifies coordinate offset. Offset is applied to coordinates before doing texture lookup. Offset value is in the range of -8 to +7.

Depth compare operand *f* is not supported for multi-sample texture arrays.

The optional destination predicate *p* is set to `True` if data from texture at specified coordinates is resident in memory, `False` otherwise. When optional destination predicate *p* is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

Mipmaps

.base (lod zero)

Pick level 0 (base level). This is the default if no mipmap mode is specified. No additional arguments.

.level1 (lod explicit)

Requires an additional 32-bit scalar argument, `lod`, which contains the LOD to fetch from. The type of `lod` follows `.ctype` (either `.s32` or `.f32`). Geometries `.2dms` and `.a2dms` are not supported in this mode.

.grad (lod gradient)

Requires two `.f32` vectors, `dPdx` and `dPdy`, that specify the partials. The vectors are singletons for 1d and a1d textures; are two-element vectors for 2d and a2d textures; and are four-element vectors for 3d, cube and acube textures, where the fourth element is ignored for 3d and cube geometries. Geometries `.2dms` and `.a2dms` are not supported in this mode.

For mipmap texture lookup, an optional operand `e` may be specified. Operand `e` is a vector of `.s32` that specifies coordinate offset. Offset is applied to coordinates before doing texture lookup. Offset value is in the range of -8 to +7. Offset vector operand is not supported for cube and cubemap geometries.

An optional operand `f` may be specified for mipmap textures. Operand `f` is `.f32` scalar value that specifies depth compare value for depth textures. When using depth compare operand, the coordinates in texture coordinate vector `c` have `.f32` type.

The optional destination predicate `p` is set to `True` if data from texture at specified coordinates is resident in memory, `False` otherwise. When optional destination predicate `p` is set to `False`, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

Depth compare operand is not supported for 3d textures.

Indirect texture access

Beginning with PTX ISA version 3.1, indirect texture access is supported in unified mode for target architecture `sm_20` or higher. In indirect access, operand `a` is a `.u64` register holding the address of a `.texref` variable.

Notes

For compatibility with prior versions of PTX, the square brackets are not required and `.v4` coordinate vectors are allowed for any geometry, with the extra elements being ignored.

PTX ISA Notes

Unified mode texturing introduced in PTX ISA version 1.0. Extension using opaque `.texref` and `.samplerref` types and independent mode texturing introduced in PTX ISA version 1.5.

Texture arrays `tex.{a1d, a2d}` introduced in PTX ISA version 2.3.

Cubemaps and cubemap arrays introduced in PTX ISA version 3.0.

Support for mipmaps introduced in PTX ISA version 3.1.

Indirect texture access introduced in PTX ISA version 3.1.

Multi-sample textures and multi-sample texture arrays introduced in PTX ISA version 3.2.

Support for textures returning .f16 and .f16x2 data introduced in PTX ISA version 4.2.

Support for `tex.grad.{cube, acube}` introduced in PTX ISA version 4.3.

Offset vector operand introduced in PTX ISA version 4.3.

Depth compare operand introduced in PTX ISA version 4.3.

Support for optional destination predicate introduced in PTX ISA version 7.1.

Target ISA Notes

Supported on all target architectures.

The cubemap array geometry (.acube) requires `sm_20` or higher.

Mipmaps require `sm_20` or higher.

Indirect texture access requires `sm_20` or higher.

Multi-sample textures and multi-sample texture arrays require `sm_30` or higher.

Texture fetch returning .f16 and .f16x2 data require `sm_53` or higher.

`tex.grad.{cube, acube}` requires `sm_20` or higher.

Offset vector operand requires `sm_30` or higher.

Depth compare operand requires `sm_30` or higher.

Support for optional destination predicate requires `sm_60` or higher.

Examples

```
// Example of unified mode texturing
// - f4 is required to pad four-element tuple and is ignored
tex.3d.v4.s32.s32 {r1,r2,r3,r4}, [tex_a,{f1,f2,f3,f4}];

// Example of independent mode texturing
tex.1d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a,smpl_x,{f1}];

// Example of 1D texture array, independent texturing mode
tex.a1d.v4.s32.s32 {r1,r2,r3,r4}, [tex_a,smpl_x,{idx,s1}];

// Example of 2D texture array, unified texturing mode
// - f3 is required to pad four-element tuple and is ignored
tex.a2d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a,{idx,f1,f2,f3}];

// Example of cubemap array, unified textureing mode
tex.acube.v4.f32.f32 {r0,r1,r2,r3}, [tex_cuarray,{idx,f1,f2,f3}];

// Example of multi-sample texture, unified texturing mode
tex.2dms.v4.s32.s32 {r0,r1,r2,r3}, [tex_ms,{sample,r6,r7,r8}];

// Example of multi-sample texture, independent texturing mode
tex.2dms.v4.s32.s32 {r0,r1,r2,r3}, [tex_ms, smpl_x,{sample,r6,r7,r8}];

// Example of multi-sample texture array, unified texturing mode
```

(continues on next page)

(continued from previous page)

```

tex.a2dms.v4.s32.s32 {r0,r1,r2,r3}, [tex_ams,{idx,sample,r6,r7}];

// Example of texture returning .f16 data
tex.1d.v4.f16.f32 {h1,h2,h3,h4}, [tex_a,smpl_x,{f1}];

// Example of texture returning .f16x2 data
tex.1d.v2.f16x2.f32 {h1,h2}, [tex_a,smpl_x,{f1}];

// Example of 3d texture array access with tex.grad,unified texturing mode
tex.grad.3d.v4.f32.f32 {%f4,%f5,%f6,%f7},[tex_3d,{%f0,%f0,%f0,%f0}],
    {f10,f11,f12,f13},{f10,f11,f12,f13};

// Example of cube texture array access with tex.grad,unified texturing mode
tex.grad.cube.v4.f32.f32{%f4,%f5,%f6,%f7},[tex_cube,{%f0,%f0,%f0,%f0}],
    {f10,f11,f12,f13},{f10,f11,f12,f13};

// Example of 1d texture lookup with offset, unified texturing mode
tex.1d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a, {f1}], {r5};

// Example of 2d texture array lookup with offset, unified texturing mode
tex.a2d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a,{idx,f1,f2}], {f5,f6};

// Example of 2d mipmap texture lookup with offset, unified texturing mode
tex.level.2d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a,{f1,f2}],
    flv1, {r7, r8};

// Example of 2d depth texture lookup with compare, unified texturing mode
tex.1d.v4.f32.f32 {f1,f2,f3,f4}, [tex_a, {f1}], f0;

// Example of depth 2d texture array lookup with offset, compare
tex.a2d.v4.s32.f32 {f0,f1,f2,f3}, [tex_a,{idx,f4,f5}], {r5,r6}, f6;

// Example of destination predicate use
tex.3d.v4.s32.s32 {r1,r2,r3,r4}|p, [tex_a,{f1,f2,f3,f4}];

```

9.7.10.4 Texture Instructions: tld4

tld4

Perform a texture fetch of the 4-textel bilerp footprint.

Syntax

```

tld4.comp.2d.v4.dtype.f32    d[|p], [a, c] {, e} {, f};
tld4.comp.geom.v4.dtype.f32  d[|p], [a, b, c] {, e} {, f}; // explicit sampler

.comp = { .r, .g, .b, .a };
.geom = { .2d, .a2d, .cube, .acube };
.dtype = { .u32, .s32, .f32 };

```

Description

Texture fetch of the 4-textel bilerp footprint using a texture coordinate vector. The instruction loads the bilerp footprint from the texture named by operand *a* at coordinates given by operand *c* into vector destination *d*. The texture component fetched for each texel sample is specified by *.comp*. The four texel samples are placed into destination vector *d* in counter-clockwise order starting at lower left.

An optional texture sampler *b* may be specified. If no sampler is specified, the sampler behavior is a property of the named texture.

The optional destination predicate *p* is set to *True* if data from texture at specified coordinates is resident in memory, *False* otherwise. When optional destination predicate *p* is set to *False*, data loaded will be all zeros. Memory residency of Texture Data at specified coordinates is dependent on execution environment setup using Driver API calls, prior to kernel launch. Refer to Driver API documentation for more details including any system/implementation specific behavior.

An optional operand *f* may be specified for *depth textures*. Depth textures are special type of textures which hold data from the depth buffer. Depth buffer contains depth information of each pixel. Operand *f* is *.f32* scalar value that specifies depth compare value for depth textures. Each element fetched from texture is compared against value given in *f* operand. If comparison passes, result is 1.0; otherwise result is 0.0. These per-element comparison results are used for the filtering.

A texture base address is assumed to be aligned to a 16 byte boundary, and the address given by the coordinate vector must be naturally aligned to a multiple of the access size. If an address is not properly aligned, the resulting behavior is undefined; i.e., the access may proceed by silently masking off low-order address bits to achieve proper rounding, or the instruction may fault.

tld4.2d

For 2D textures, operand *c* specifies coordinates as a two-element, 32-bit floating-point vector.

An optional operand *e* may be specified. Operand *e* is a vector of type *.v2.s32* that specifies coordinate offset. Offset is applied to coordinates before doing texture fetch. Offset value is in the range of -8 to +7.

tld4.a2d

Texture array selection, followed by *tld4* texture fetch of 2d texture. For 2d texture arrays operand *c* is a four element, 32-bit vector. The first element in operand *c* is interpreted as an unsigned integer index (*.u32*) into the texture array, and the next two elements are interpreted as 32-bit floating point coordinates of 2d texture. The fourth element is ignored.

An optional operand *e* may be specified. Operand *e* is a vector of type *.v2.s32* that specifies coordinate offset. Offset is applied to coordinates before doing texture fetch. Offset value is in the range of -8 to +7.

tld4.cube

For cubemap textures, operand *c* specifies four-element vector which comprises three floating-point coordinates (*s*, *t*, *r*) and a fourth padding argument which is ignored.

Cubemap textures are special two-dimensional layered textures consisting of six layers that represent the faces of a cube. All layers in a cubemap are of the same size and are square (i.e., width equals height).

Coordinates (*s*, *t*, *r*) are projected onto one of the six cube faces. The (*s*, *t*, *r*) coordinates can be thought of as a direction vector emanating from the center of the cube. Of the three coordinates (*s*, *t*, *r*), the coordinate of the largest magnitude (the major axis) selects the cube face. Then, the other two coordinates (the minor axes) are divided by the absolute value of the major axis to produce a new (*s*, *t*) coordinate pair to lookup into the selected cube face.

Offset vector operand `e` is not supported for cubemap textures.

`tld4.acube`

Cubemap array selection, followed by `tld4` texture fetch of cubemap texture. The first element in operand `c` is interpreted as an unsigned integer index (`.u32`) into the cubemap texture array, and the remaining three elements are interpreted as floating-point cubemap coordinates (`s`, `t`, `r`), used to lookup in the selected cubemap.

Offset vector operand `e` is not supported for cubemap texture arrays.

Indirect texture access

Beginning with PTX ISA version 3.1, indirect texture access is supported in unified mode for target architecture `sm_20` or higher. In indirect access, operand `a` is a `.u64` register holding the address of a `.texref` variable.

PTX ISA Notes

Introduced in PTX ISA version 2.2.

Indirect texture access introduced in PTX ISA version 3.1.

`tld4.{a2d, cube, acube}` introduced in PTX ISA version 4.3.

Offset vector operand introduced in PTX ISA version 4.3.

Depth compare operand introduced in PTX ISA version 4.3.

Support for optional destination predicate introduced in PTX ISA version 7.1.

Target ISA Notes

`tld4` requires `sm_20` or higher.

Indirect texture access requires `sm_20` or higher.

`tld4.{a2d, cube, acube}` requires `sm_30` or higher.

Offset vector operand requires `sm_30` or higher.

Depth compare operand requires `sm_30` or higher.

Support for optional destination predicate requires `sm_60` or higher.

Examples

```
//Example of unified mode texturing
tld4.r.2d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a,{f1,f2}];

// Example of independent mode texturing
tld4.r.2d.v4.u32.f32 {u1,u2,u3,u4}, [tex_a,smpl_x,{f1,f2}];

// Example of unified mode texturing using offset
tld4.r.2d.v4.s32.f32 {r1,r2,r3,r4}, [tex_a,{f1,f2}], {r5, r6};
```

(continues on next page)

(continued from previous page)

```
// Example of unified mode texturing using compare
tld4.r.2d.v4.f32.f32 {f1,f2,f3,f4}, [tex_a,{f5,f6}], f7;

// Example of optional destination predicate
tld4.r.2d.v4.f32.f32 {f1,f2,f3,f4}|p, [tex_a,{f5,f6}], f7;
```

9.7.10.5 Texture Instructions: txq

txq

Query texture and sampler attributes.

Syntax

```
txq.tquery.b32      d, [a];      // texture attributes
txq.level.tlquery.b32 d, [a], lod; // texture attributes
txq.squery.b32      d, [a];      // sampler attributes

.tquery = { .width, .height, .depth,
             .channel_data_type, .channel_order,
             .normalized_coords, .array_size,
             .num_mipmap_levels, .num_samples};

.tlquery = { .width, .height, .depth };

.squery = { .force_unnormalized_coords, .filter_mode,
            .addr_mode_0, addr_mode_1, addr_mode_2 };
```

Description

Query an attribute of a texture or sampler. Operand a is either a `.texref` or `.samplerref` variable, or a `.u64` register.

Query	Returns
.width .height .depth	value in elements
.channel_data_type	Unsigned integer corresponding to source language's channel data type enumeration. If the source language combines channel data type and channel order into a single enumeration type, that value is returned for both <code>channel_data_type</code> and <code>channel_order</code> queries.
.channel_order	Unsigned integer corresponding to source language's channel order enumeration. If the source language combines channel data type and channel order into a single enumeration type, that value is returned for both <code>channel_data_type</code> and <code>channel_order</code> queries.
.normalized_coords	1 (True) or 0 (False).
.force_unnormalized_coords	1 (True) or 0 (False). Defined only for <code>.samplerref</code> variables in independent texture mode. Overrides the <code>normalized_coords</code> field of a <code>.texref</code> variable used with a <code>.samplerref</code> in a <code>tex</code> instruction.
.filter_mode	Integer from enum { nearest, linear }
.addr_mode_0 .addr_mode_1 .addr_mode_2	Integer from enum { wrap, mirror, clamp_ogl, clamp_to_edge, clamp_to_border }
.array_size	For a texture array, number of textures in array, 0 otherwise.
.num_mipmap_levels	For a mipmapped texture, number of levels of details (LOD), 0 otherwise.
.num_samples	For a multi-sample texture, number of samples, 0 otherwise.

Texture attributes are queried by supplying a `.texref` argument to `txq`. In unified mode, sampler attributes are also accessed via a `.texref` argument, and in independent mode sampler attributes are accessed via a separate `.samplerref` argument.

`txq.level`

`txq.level` requires an additional 32bit integer argument, `lod`, which specifies LOD and queries requested attribute for the specified LOD.

Indirect texture access

Beginning with PTX ISA version 3.1, indirect texture access is supported in unified mode for target architecture `sm_20` or higher. In indirect access, operand `a` is a `.u64` register holding the address of a `.texref` variable.

PTX ISA Notes

Introduced in PTX ISA version 1.5.

Channel data type and channel order queries were added in PTX ISA version 2.1.

The `.force_unnormalized_coords` query was added in PTX ISA version 2.2.

Indirect texture access introduced in PTX ISA version 3.1.

`.array_size`, `.num_mipmap_levels`, `.num_samples` queries were added in PTX ISA version 4.1.

`txq.level` introduced in PTX ISA version 4.3.

Target ISA Notes

Supported on all target architectures.

Indirect texture access requires `sm_20` or higher.

Querying the number of mipmap levels requires `sm_20` or higher.

Querying the number of samples requires `sm_30` or higher.

`txq.level` requires `sm_30` or higher.

Examples

```
txq.width.b32      %r1, [tex_A];
txq.filter_mode.b32 %r1, [tex_A]; // unified mode
txq.addr_mode_0.b32 %r1, [smp1_B]; // independent mode
txq.level.width.b32 %r1, [tex_A], %r_lod;
```

9.7.10.6 Texture Instructions: `istypep`

`istypep`

Query whether a register points to an opaque variable of a specified type.

Syntax

```
istypep.type  p, a; // result is .pred
.type = { .texref, .samplerref, .surfref };
```

Description

Write predicate register `p` with 1 if register `a` points to an opaque variable of the specified type, and with 0 otherwise. Destination `p` has type `.pred`; the source address operand must be of type `.u64`.

PTX ISA Notes

Introduced in PTX ISA version 4.0.

Target ISA Notes

`istypep` requires `sm_30` or higher.

Examples

```
istypep.texref istex, tptr;
istypep.samplerref issampler, sptr;
istypep.surfref issurface, surfptr;
```

9.7.11. Surface Instructions

This section describes PTX instructions for accessing surfaces. PTX supports the following operations on surface descriptors:

- ▶ Static initialization of surface descriptors.
- ▶ Module-scope and per-entry scope definitions of surface descriptors.
- ▶ Ability to query fields within surface descriptors.

These instructions provide access to surface memory.

- ▶ `suld`
- ▶ `sust`
- ▶ `sured`
- ▶ `suq`

9.7.11.1 Surface Instructions: `suld`

`suld`

Load from surface memory.

Syntax

```
suld.b.geom{.cop}.vec.dtype.clamp d, [a, b]; // unformatted

.geom = { .1d, .2d, .3d, .a1d, .a2d };
.cop  = { .ca, .cg, .cs, .cv };           // cache operation
.vec  = { none, .v2, .v4 };
.dtype = { .b8, .b16, .b32, .b64 };
.clamp = { .trap, .clamp, .zero };
```

Description

suld.b.{1d,2d,3d}

Load from surface memory using a surface coordinate vector. The instruction loads data from the surface named by operand *a* at coordinates given by operand *b* into destination *d*. Operand *a* is a `.surfref` variable or `.u64` register. Operand *b* is a scalar or singleton tuple for 1d surfaces; is a two-element vector for 2d surfaces; and is a four-element vector for 3d surfaces, where the fourth element is ignored. Coordinate elements are of type `.s32`.

suld.b performs an unformatted load of binary data. The lowest dimension coordinate represents a byte offset into the surface and is not scaled, and the size of the data transfer matches the size of destination operand *d*.

suld.b.{a1d,a2d}

Surface layer selection, followed by a load from the selected surface. The instruction first selects a surface layer from the surface array named by operand *a* using the index given by the first element of the array coordinate vector *b*. The instruction then loads data from the selected surface at coordinates given by the remaining elements of operand *b* into destination *d*. Operand *a* is a `.surfref` variable or `.u64` register. Operand *b* is a bit-size type vector or tuple containing an index into the array of surfaces followed by coordinates within the selected surface, as follows:

For 1d surface arrays, operand *b* has type `.v2.b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the surface array, and the second element is interpreted as a 1d surface coordinate of type `.s32`.

For 2d surface arrays, operand *b* has type `.v4.b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the surface array, and the next two elements are interpreted as 2d surface coordinates of type `.s32`. The fourth element is ignored.

A surface base address is assumed to be aligned to a 16 byte boundary, and the address given by the coordinate vector must be naturally aligned to a multiple of the access size. If an address is not properly aligned, the resulting behavior is undefined; i.e., the access may proceed by silently masking off low-order address bits to achieve proper rounding, or the instruction may fault.

The `.clamp` field specifies how to handle out-of-bounds addresses:

.trap

causes an execution trap on out-of-bounds addresses

.clamp

loads data at the nearest surface location (sized appropriately)

.zero

loads zero for out-of-bounds addresses

Indirect surface access

Beginning with PTX ISA version 3.1, indirect surface access is supported for target architecture `sm_20` or higher. In indirect access, operand `a` is a `.u64` register holding the address of a `.surfref` variable.

PTX ISA Notes

`suld.b.trap` introduced in PTX ISA version 1.5.

Additional clamp modifiers and cache operations introduced in PTX ISA version 2.0.

`suld.b.3d` and `suld.b.{a1d,a2d}` introduced in PTX ISA version 3.0.

Indirect surface access introduced in PTX ISA version 3.1.

Target ISA Notes

`suld.b` supported on all target architectures.

`sm_1x` targets support only the `.trap` clamping modifier.

`suld.3d` and `suld.{a1d,a2d}` require `sm_20` or higher.

Indirect surface access requires `sm_20` or higher.

Cache operations require `sm_20` or higher.

Examples

```
suld.b.1d.v4.b32.trap {s1,s2,s3,s4}, [surf_B, {x}];
suld.b.3d.v2.b64.trap {r1,r2}, [surf_A, {x,y,z,w}];
suld.b.a1d.v2.b32     {r0,r1}, [surf_C, {idx,x}];
suld.b.a2d.b32        r0, [surf_D, {idx,x,y,z}]; // z ignored
```

9.7.11.2 Surface Instructions: `sust`

`sust`

Store to surface memory.

Syntax

```
sust.b.{1d,2d,3d}{.cop}.vec.ctype.clamp [a, b], c; // unformatted
sust.p.{1d,2d,3d}.vec.b32.clamp         [a, b], c; // formatted

sust.b.{a1d,a2d}{.cop}.vec.ctype.clamp  [a, b], c; // unformatted

.cop   = { .wb, .cg, .cs, .wt }; // cache operation
.vec   = { none, .v2, .v4 };
.ctype = { .b8, .b16, .b32, .b64 };
.clamp = { .trap, .clamp, .zero };
```

Description

`sust . {1d, 2d, 3d}`

Store to surface memory using a surface coordinate vector. The instruction stores data from operand `c` to the surface named by operand `a` at coordinates given by operand `b`. Operand `a` is a `.surfref` variable or `.u64` register. Operand `b` is a scalar or singleton tuple for 1d surfaces; is a two-element vector for 2d surfaces; and is a four-element vector for 3d surfaces, where the fourth element is ignored. Coordinate elements are of type `.s32`.

`sust . b` performs an unformatted store of binary data. The lowest dimension coordinate represents a byte offset into the surface and is not scaled. The size of the data transfer matches the size of source operand `c`.

`sust . p` performs a formatted store of a vector of 32-bit data values to a surface sample. The source vector elements are interpreted left-to-right as R, G, B, and A surface components. These elements are written to the corresponding surface sample components. Source elements that do not occur in the surface sample are ignored. Surface sample components that do not occur in the source vector will be written with an unpredictable value. The lowest dimension coordinate represents a sample offset rather than a byte offset.

The source data interpretation is based on the surface sample format as follows: If the surface format contains `UNORM`, `SNORM`, or `FLOAT` data, then `.f32` is assumed; if the surface format contains `UINT` data, then `.u32` is assumed; if the surface format contains `SINT` data, then `.s32` is assumed. The source data is then converted from this type to the surface sample format.

`sust . b . {a1d, a2d}`

Surface layer selection, followed by an unformatted store to the selected surface. The instruction first selects a surface layer from the surface array named by operand `a` using the index given by the first element of the array coordinate vector `b`. The instruction then stores the data in operand `c` to the selected surface at coordinates given by the remaining elements of operand `b`. Operand `a` is a `.surfref` variable or `.u64` register. Operand `b` is a bit-size type vector or tuple containing an index into the array of surfaces followed by coordinates within the selected surface, as follows:

- For 1d surface arrays, operand `b` has type `.v2 . b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the surface array, and the second element is interpreted as a 1d surface coordinate of type `.s32`.
- For 2d surface arrays, operand `b` has type `.v4 . b32`. The first element is interpreted as an unsigned integer index (`.u32`) into the surface array, and the next two elements are interpreted as 2d surface coordinates of type `.s32`. The fourth element is ignored.

A surface base address is assumed to be aligned to a 16 byte boundary, and the address given by the coordinate vector must be naturally aligned to a multiple of the access size. If an address is not properly aligned, the resulting behavior is undefined; i.e., the access may proceed by silently masking off low-order address bits to achieve proper rounding, or the instruction may fault.

The `.clamp` field specifies how to handle out-of-bounds addresses:

.trap

causes an execution trap on out-of-bounds addresses

.clamp

stores data at the nearest surface location (sized appropriately)

.zero

drops stores to out-of-bounds addresses

Indirect surface access

Beginning with PTX ISA version 3.1, indirect surface access is supported for target architecture `sm_20` or higher. In indirect access, operand `a` is a `.u64` register holding the address of a `.surfref` variable.

PTX ISA Notes

`sust.b.trap` introduced in PTX ISA version 1.5. `sust.p`, additional clamp modifiers, and cache operations introduced in PTX ISA version 2.0.

`sust.b.3d` and `sust.b.{a1d,a2d}` introduced in PTX ISA version 3.0.

Indirect surface access introduced in PTX ISA version 3.1.

Target ISA Notes

`sust.b` supported on all target architectures.

`sm_1x` targets support only the `.trap` clamping modifier.

`sust.3d` and `sust.{a1d,a2d}` require `sm_20` or higher.

`sust.p` requires `sm_20` or higher.

Indirect surface access requires `sm_20` or higher.

Cache operations require `sm_20` or higher.

Examples

```
sust.p.1d.v4.b32.trap [surf_B, {x}], {f1,f2,f3,f4};
sust.b.3d.v2.b64.trap [surf_A, {x,y,z,w}], {r1,r2};
sust.b.a1d.v2.b64     [surf_C, {idx,x}], {r1,r2};
sust.b.a2d.b32        [surf_D, {idx,x,y,z}], r0; // z ignored
```

9.7.11.3 Surface Instructions: `sured`

`sured`

Reduce surface memory.

Syntax

```
sured.b.op.geom.ctype.clamp [a,b],c; // byte addressing
sured.p.op.geom.ctype.clamp [a,b],c; // sample addressing

.op    = { .add, .min, .max, .and, .or };
.geom  = { .1d, .2d, .3d };
.ctype = { .u32, .u64, .s32, .b32, .s64 }; // for sured.b
.ctype = { .b32, .b64 };                  // for sured.p
.clamp = { .trap, .clamp, .zero };
```

Description

Reduction to surface memory using a surface coordinate vector. The instruction performs a reduction operation with data from operand `c` to the surface named by operand `a` at coordinates given by operand `b`. Operand `a` is a `.surfref` variable or `.u64` register. Operand `b` is a scalar or singleton tuple for 1d surfaces; is a two-element vector for 2d surfaces; and is a four-element vector for 3d surfaces, where the fourth element is ignored. Coordinate elements are of type `.s32`.

`sured.b` performs an unformatted reduction on `.u32`, `.s32`, `.b32`, `.u64`, or `.s64` data. The lowest dimension coordinate represents a byte offset into the surface and is not scaled. Operation `add` applies to `.u32`, `.u64`, and `.s32` types; `min` and `max` apply to `.u32`, `.s32`, `.u64` and `.s64` types; operations `and` and `or` apply to `.b32` type.

`sured.p` performs an unformatted reduction on sample-addressed data. The lowest dimension coordinate represents a sample offset rather than a byte offset and is scaled by the data-size given by `.ctype`. The instruction type `.b64` is restricted to `min` and `max` operations. For type `.b32`, the data is interpreted as `.u32` or `.s32` based on the surface sample format as follows: if the surface format contains `UINT` data, then `.u32` is assumed; if the surface format contains `SINT` data, then `.s32` is assumed. For type `.b64`, if the surface format contains `UINT` data, then `.u64` is assumed; if the surface format contains `SINT` data, then `.s64` is assumed.

A surface base address is assumed to be aligned to a 16 byte boundary, and the address given by the coordinate vector must be naturally aligned to a multiple of the access size. If an address is not properly aligned, the resulting behavior is undefined; i.e., the access may proceed by silently masking off low-order address bits to achieve proper rounding, or the instruction may fault.

The `.clamp` field specifies how to handle out-of-bounds addresses:

.trap

causes an execution trap on out-of-bounds addresses

.clamp

stores data at the nearest surface location (sized appropriately)

.zero

drops stores to out-of-bounds addresses

Indirect surface access

Beginning with PTX ISA version 3.1, indirect surface access is supported for target architecture `sm_20` or higher. In indirect access, operand `a` is a `.u64` register holding the address of a `.surfref` variable.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Indirect surface access introduced in PTX ISA version 3.1.

`.u64/.s64/.b64` types with `.min/.max` operations introduced in PTX ISA version 8.1.

Target ISA Notes

sured requires sm_20 or higher.

Indirect surface access requires sm_20 or higher.

.u64/.s64/.b64 types with .min/.max operations requires sm_50 or higher.

Examples

```
sured.b.add.2d.u32.trap [surf_A, {x,y}], r1;
sured.p.min.1d.u32.trap [surf_B, {x}], r1;
sured.b.max.1d.u64.trap [surf_C, {x}], r1;
sured.p.min.1d.b64.trap [surf_D, {x}], r1;
```

9.7.11.4 Surface Instructions: suq

suq

Query a surface attribute.

Syntax

```
suq.query.b32    d, [a];

.query = { .width, .height, .depth,
           .channel_data_type, .channel_order,
           .array_size, .memory_layout };
```

Description

Query an attribute of a surface. Operand *a* is a `.surfref` variable or a `.u64` register.

Query	Returns
<code>.width</code> <code>.height</code> <code>.depth</code>	value in elements
<code>.channel_data_type</code>	Unsigned integer corresponding to source language's channel data type enumeration. If the source language combines channel data type and channel order into a single enumeration type, that value is returned for both <code>channel_data_type</code> and <code>channel_order</code> queries.
<code>.channel_order</code>	Unsigned integer corresponding to source language's channel order enumeration. If the source language combines channel data type and channel order into a single enumeration type, that value is returned for both <code>channel_data_type</code> and <code>channel_order</code> queries.
<code>.array_size</code>	For a surface array, number of surfaces in array, 0 otherwise.
<code>.memory_layout</code>	1 for surface with linear memory layout; 0 otherwise

Indirect surface access

Beginning with PTX ISA version 3.1, indirect surface access is supported for target architecture `sm_20` or higher. In indirect access, operand *a* is a `.u64` register holding the address of a `.surfref` variable.

PTX ISA Notes

Introduced in PTX ISA version 1.5.

Channel data type and channel order queries added in PTX ISA version 2.1.

Indirect surface access introduced in PTX ISA version 3.1.

The `.array_size` query was added in PTX ISA version 4.1.

The `.memory_layout` query was added in PTX ISA version 4.2.

Target ISA Notes

Supported on all target architectures.

Indirect surface access requires `sm_20` or higher.

Examples

```
suq.width.b32      %r1, [surf_A];
```

9.7.12. Control Flow Instructions

The following PTX instructions and syntax are for controlling execution in a PTX program:

- ▶ {}
- ▶ @
- ▶ bra
- ▶ call
- ▶ ret
- ▶ exit

9.7.12.1 Control Flow Instructions: {}

```
{}
```

Instruction grouping.

Syntax

```
{ instructionList }
```

Description

The curly braces create a group of instructions, used primarily for defining a function body. The curly braces also provide a mechanism for determining the scope of a variable: any variable declared within a scope is not available outside the scope.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
{ add.s32  a,b,c; mov.s32  d,a; }
```

9.7.12.2 Control Flow Instructions: @

@

Predicated execution.

Syntax

```
@{!}p    instruction;
```

Description

Execute an instruction or instruction block for threads that have the guard predicate True. Threads with a False guard predicate do nothing.

Semantics

If `{!}p` then instruction

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
    setp.eq.f32  p,y,0;    // is y zero?
@!p div.f32     ratio,x,y  // avoid division by zero
@q  bra L23;           // conditional branch
```

9.7.12.3 Control Flow Instructions: bra

bra

Branch to a target and continue execution there.

Syntax

```
@p    bra{.uni}  tgt;           // tgt is a label
      bra{.uni}  tgt;           // unconditional branch
```

Description

Continue execution at the target. Conditional branches are specified by using a guard predicate. The branch target must be a label.

`bra .uni` is guaranteed to be non-divergent, i.e. all active threads in a warp that are currently executing this instruction have identical values for the guard predicate and branch target.

Semantics

```
if (p) {
    pc = tgt;
}
```

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Unimplemented indirect branch introduced in PTX ISA version 2.1 has been removed from the spec.

Target ISA Notes

Supported on all target architectures.

Examples

```
bra.uni  L_exit;    // uniform unconditional jump
@q bra    L23;      // conditional branch
```

9.7.12.4 Control Flow Instructions: `brx.idx`

`brx.idx`

Branch to a label indexed from a list of potential branch targets.

Syntax

```
@p    brx.idx{.uni} index, tlist;  
      brx.idx{.uni} index, tlist;
```

Description

Index into a list of possible destination labels, and continue execution from the chosen label. Conditional branches are specified by using a guard predicate.

`brx.idx.uni` guarantees that the branch is non-divergent, i.e. all active threads in a warp that are currently executing this instruction have identical values for the guard predicate and the `index` argument.

The `index` operand is a `.u32` register. The `tlist` operand must be the label of a `.branchtargets` directive. It is accessed as a zero-based sequence using `index`. Behaviour is undefined if the value of `index` is greater than or equal to the length of `tlist`.

The `.branchtargets` directive must be defined in the local function scope before it is used. It must refer to labels within the current function.

Semantics

```
if (p) {  
    if (index < length(tlist)) {  
        pc = tlist[index];  
    } else {  
        pc = undefined;  
    }  
}
```

PTX ISA Notes

Introduced in PTX ISA version 6.0.

Target ISA Notes

Requires sm_30 or higher.

Examples

```
.function foo () {
    .reg .u32 %r0;
    ...
    L1:
    ...
    L2:
    ...
    L3:
    ...
    ts: .branchtargets L1, L2, L3;
    @p brx.idx %r0, ts;
    ...
}
```

9.7.12.5 Control Flow Instructions: call

call

Call a function, recording the return location.

Syntax

```
// direct call to named function, func is a symbol
call{.uni} (ret-param), func, (param-list);
call{.uni} func, (param-list);
call{.uni} func;

// indirect call via pointer, with full list of call targets
call{.uni} (ret-param), fptr, (param-list), flist;
call{.uni} fptr, (param-list), flist;
call{.uni} fptr, flist;

// indirect call via pointer, with no knowledge of call targets
call{.uni} (ret-param), fptr, (param-list), fproto;
call{.uni} fptr, (param-list), fproto;
call{.uni} fptr, fproto;
```

Description

The `call` instruction stores the address of the next instruction, so execution can resume at that point after executing a `ret` instruction. A `call` is assumed to be divergent unless the `.uni` suffix is present. The `.uni` suffix indicates that the `call` is guaranteed to be non-divergent, i.e. all active threads in a warp that are currently executing this instruction have identical values for the guard predicate and `call` target.

For direct calls, the called location `func` must be a symbolic function name; for indirect calls, the called location `fptr` must be an address of a function held in a register. Input arguments and return values are optional. Arguments may be registers, immediate constants, or variables in `.param` space. Arguments are pass-by-value.

Indirect calls require an additional operand, `flist` or `fproto`, to communicate the list of potential `call` targets or the common function prototype of all `call` targets, respectively. In the first case, `flist` gives a complete list of potential `call` targets and the optimizing backend is free to optimize the calling convention. In the second case, where the complete list of potential `call` targets may not be known, the common function prototype is given and the `call` must obey the ABI's calling convention.

The `flist` operand is either the name of an array (call table) initialized to a list of function names; or a label associated with a `.calltargets` directive, which declares a list of potential `call` targets. In both cases the `fptr` register holds the address of a function listed in the call table or `.calltargets` list, and the `call` operands are type-checked against the type signature of the functions indicated by `flist`.

The `fproto` operand is the name of a label associated with a `.callprototype` directive. This operand is used when a complete list of potential targets is not known. The `call` operands are type-checked against the prototype, and code generation will follow the ABI calling convention. If a function that doesn't match the prototype is called, the behavior is undefined.

Call tables may be declared at module scope or local scope, in either the constant or global state space. The `.calltargets` and `.callprototype` directives must be declared within a function body. All functions must be declared prior to being referenced in a `call` table initializer or `.calltargets` directive.

PTX ISA Notes

Direct `call` introduced in PTX ISA version 1.0. Indirect `call` introduced in PTX ISA version 2.1.

Target ISA Notes

Direct `call` supported on all target architectures. Indirect `call` requires `sm_20` or higher.

Examples

```
// examples of direct call
    call    init;    // call function 'init'
    call.uni g, (a); // call function 'g' with parameter 'a'
@p call    (d), h, (a, b); // return value into register d

// call-via-pointer using jump table
.func (.reg .u32 rv) foo (.reg .u32 a, .reg .u32 b) ...
.func (.reg .u32 rv) bar (.reg .u32 a, .reg .u32 b) ...
.func (.reg .u32 rv) baz (.reg .u32 a, .reg .u32 b) ...

.global .u32 jmptbl[5] = { foo, bar, baz };
...
@p ld.global.u32 %r0, [jmptbl+4];
@p ld.global.u32 %r0, [jmptbl+8];
    call (retval), %r0, (x, y), jmptbl;

// call-via-pointer using .calltargets directive
.func (.reg .u32 rv) foo (.reg .u32 a, .reg .u32 b) ...
.func (.reg .u32 rv) bar (.reg .u32 a, .reg .u32 b) ...
.func (.reg .u32 rv) baz (.reg .u32 a, .reg .u32 b) ...
...
@p mov.u32 %r0, foo;
@q mov.u32 %r0, baz;
Ftgt: .calltargets foo, bar, baz;
    call (retval), %r0, (x, y), Ftgt;

// call-via-pointer using .callprototype directive
.func dispatch (.reg .u32 fptr, .reg .u32 idx)
{
...
Fproto: .callprototype _ (.param .u32 _, .param .u32 _);
    call %fptr, (x, y), Fproto;
...

```

9.7.12.6 Control Flow Instructions: ret

ret

Return from function to instruction after call.

Syntax

```
ret{.uni};
```

Description

Return execution to caller's environment. A divergent return suspends threads until all threads are ready to return to the caller. This allows multiple divergent `ret` instructions.

A `ret` is assumed to be divergent unless the `.uni` suffix is present, indicating that the return is guaranteed to be non-divergent.

Any values returned from a function should be moved into the return parameter variables prior to executing the `ret` instruction.

A return instruction executed in a top-level entry routine will terminate thread execution.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
ret;  
@p ret;
```

9.7.12.7 Control Flow Instructions: `exit`

`exit`

Terminate a thread.

Syntax

```
exit;
```

Description

Ends execution of a thread.

Barriers exclusively waiting on arrivals from exited threads are always released.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
    exit;  
@p  exit;
```

9.7.13. Parallel Synchronization and Communication Instructions

These instructions are:

- ▶ `bar{.cta}, barrier{.cta}`
- ▶ `bar.warp.sync`
- ▶ `barrier.cluster`
- ▶ `membar`
- ▶ `atom`
- ▶ `red`
- ▶ `red.async`
- ▶ `vote`
- ▶ `match.sync`
- ▶ `activemask`
- ▶ `redux.sync`
- ▶ `griddepcontrol`
- ▶ `elect.sync`
- ▶ `mbarrier.init`
- ▶ `mbarrier.inval`
- ▶ `mbarrier.arrive`
- ▶ `mbarrier.arrive_drop`
- ▶ `mbarrier.test_wait`
- ▶ `mbarrier.try_wait`
- ▶ `mbarrier.pending_count`
- ▶ `cp.async.mbarrier.arrive`

- ▶ `tensormap.cp_fenceproxy`
- ▶ `clusterlaunchcontrol.try_cancel`
- ▶ `clusterlaunchcontrol.query_cancel`

9.7.13.1 Parallel Synchronization and Communication Instructions: `bar`, `barrier`

`bar`, `bar.cta`, `barrier`, `barrier.cta`

Barrier synchronization.

Syntax

```

barrier{.cta}.sync{.aligned}    a{, b};
barrier{.cta}.arrive{.aligned}  a, b;

barrier{.cta}.red.popc{.aligned}.u32  d, a{, b}, {!}c;
barrier{.cta}.red.op{.aligned}.pred    p, a{, b}, {!}c;

bar{.cta}.sync    a{, b};
bar{.cta}.arrive  a, b;

bar{.cta}.red.popc.u32  d, a{, b}, {!}c;
bar{.cta}.red.op.pred   p, a{, b}, {!}c;

.op = { .and, .or };

```

Description

Performs barrier synchronization and communication within a CTA. Each CTA instance has sixteen barriers numbered 0..15.

`barrier{.cta}` instructions can be used by the threads within the CTA for synchronization and communication.

Operands `a`, `b`, and `d` have type `.u32`; operands `p` and `c` are predicates. Source operand `a` specifies a logical barrier resource as an immediate constant or register with value 0 through 15. Operand `b` specifies the number of threads participating in the barrier. If no thread count is specified, all threads in the CTA participate in the barrier. When specifying a thread count, the value must be a multiple of the warp size. Note that a non-zero thread count is required for `barrier{.cta}.arrive`.

Depending on operand `b`, either specified number of threads (in multiple of warp size) or all threads in the CTA participate in `barrier{.cta}` instruction. The `barrier{.cta}` instructions signal the arrival of the executing threads at the named barrier.

`barrier{.cta}` instruction causes executing thread to wait for all non-exited threads from its warp and marks warps' arrival at barrier. In addition to signaling its arrival at the barrier, the `barrier{.cta}.red` and `barrier{.cta}.sync` instructions causes executing thread to wait for non-exited threads of all other warps participating in the barrier to arrive. `barrier{.cta}.arrive` does not cause executing thread to wait for threads of other participating warps.

When a barrier completes, the waiting threads are restarted without delay, and the barrier is reinitialized so that it can be immediately reused.

The `barrier{.cta}.sync` or `barrier{.cta}.red` or `barrier{.cta}.arrive` instruction guarantees that when the barrier completes, prior memory accesses requested by this thread are performed relative to all threads participating in the barrier. The `barrier{.cta}.sync` and `barrier{.cta}.red` instruction further guarantees that no new memory access is requested by this thread before the barrier completes.

A memory read (e.g., by `ld` or `atom`) has been performed when the value read has been transmitted from memory and cannot be modified by another thread participating in the barrier. A memory write (e.g., by `st`, `red` or `atom`) has been performed when the value written has become visible to other threads participating in the barrier, that is, when the previous value can no longer be read.

`barrier{.cta}.red` performs a reduction operation across threads. The `c` predicate (or its complement) from all threads in the CTA are combined using the specified reduction operator. Once the barrier count is reached, the final value is written to the destination register in all threads waiting at the barrier.

The reduction operations for `barrier{.cta}.red` are population-count (`.popc`), all-threads-True (`.and`), and any-thread-True (`.or`). The result of `.popc` is the number of threads with a True predicate, while `.and` and `.or` indicate if all the threads had a True predicate or if any of the threads had a True predicate.

Instruction `barrier{.cta}` has optional `.aligned` modifier. When specified, it indicates that all threads in CTA will execute the same `barrier{.cta}` instruction. In conditionally executed code, an aligned `barrier{.cta}` instruction should only be used if it is known that all threads in CTA evaluate the condition identically, otherwise behavior is undefined.

Different warps may execute different forms of the `barrier{.cta}` instruction using the same barrier name and thread count. One example mixes `barrier{.cta}.sync` and `barrier{.cta}.arrive` to implement producer/consumer models. The producer threads execute `barrier{.cta}.arrive` to announce their arrival at the barrier and continue execution without delay to produce the next value, while the consumer threads execute the `barrier{.cta}.sync` to wait for a resource to be produced. The roles are then reversed, using a different barrier, where the producer threads execute a `barrier{.cta}.sync` to wait for a resource to be consumed, while the consumer threads announce that the resource has been consumed with `barrier{.cta}.arrive`. Care must be taken to keep a warp from executing more `barrier{.cta}` instructions than intended (`barrier{.cta}.arrive` followed by any other `barrier{.cta}` instruction to the same barrier) prior to the reset of the barrier. `barrier{.cta}.red` should not be intermixed with `barrier{.cta}.sync` or `barrier{.cta}.arrive` using the same active barrier. Execution in this case is unpredictable.

The optional `.cta` qualifier simply indicates CTA-level applicability of the barrier and it doesn't change the semantics of the instruction.

`bar{.cta}.sync` is equivalent to `barrier{.cta}.sync.aligned`. `bar{.cta}.arrive` is equivalent to `barrier{.cta}.arrive.aligned`. `bar{.cta}.red` is equivalent to `barrier{.cta}.red.aligned`.

Note: For `.target sm_6x` or below,

1. `barrier{.cta}` instruction without `.aligned` modifier is equivalent to `.aligned` variant and has the same restrictions as of `.aligned` variant.
 2. All threads in warp (except for those have exited) must execute `barrier{.cta}` instruction in convergence.
-

PTX ISA Notes

`bar.sync` without a thread count introduced in PTX ISA version 1.0.

Register operands, thread count, and `bar.{arrive, red}` introduced in PTX ISA version 2.0.

`barrier` instruction introduced in PTX ISA version 6.0.

`.cta` qualifier introduced in PTX ISA version 7.8.

Target ISA Notes

Register operands, thread count, and `bar{.cta}.{arrive, red}` require `sm_20` or higher.

Only `bar{.cta}.sync` with an immediate barrier number is supported for `sm_1x` targets.

`barrier{.cta}` instruction requires `sm_30` or higher.

Examples

```
// Use bar.sync to arrive at a pre-computed barrier number and
// wait for all threads in CTA to also arrive:
    st.shared [r0],r1; // write my result to shared memory
    bar.cta.sync 1;    // arrive, wait for others to arrive
    ld.shared r2,[r3]; // use shared results from other threads

// Use bar.sync to arrive at a pre-computed barrier number and
// wait for fixed number of cooperating threads to arrive:
    #define CNT1 (8*12) // Number of cooperating threads

    st.shared [r0],r1; // write my result to shared memory
    bar.cta.sync 1, CNT1; // arrive, wait for others to arrive
    ld.shared r2,[r3]; // use shared results from other threads

// Use bar.red.and to compare results across the entire CTA:
    setp.eq.u32 p,r1,r2; // p is True if r1==r2
    bar.cta.red.and.pred r3,1,p; // r3=AND(p) forall threads in CTA

// Use bar.red.popc to compute the size of a group of threads
// that have a specific condition True:
    setp.eq.u32 p,r1,r2; // p is True if r1==r2
    bar.cta.red.popc.u32 r3,1,p; // r3=SUM(p) forall threads in CTA

// Examples of barrier.cta.sync
    st.shared [r0],r1;
    barrier.cta.sync 0;
    ld.shared r1, [r0];

/* Producer/consumer model. The producer deposits a value in
 * shared memory, signals that it is complete but does not wait
 * using bar.arrive, and begins fetching more data from memory.
 * Once the data returns from memory, the producer must wait
 * until the consumer signals that it has read the value from
 * the shared memory location. In the meantime, a consumer
 * thread waits until the data is stored by the producer, reads
 * it, and then signals that it is done (without waiting).
```

(continues on next page)

(continued from previous page)

```

*/
// Producer code places produced value in shared memory.
st.shared    [r0],r1;
bar.arrive   0,64;
ld.global    r1,[r2];
bar.sync     1,64;
...

// Consumer code, reads value from shared memory
bar.sync     0,64;
ld.shared    r1,[r0];
bar.arrive   1,64;
...

```

9.7.13.2 Parallel Synchronization and Communication Instructions: `bar.warp.sync`

`bar.warp.sync`

Barrier synchronization for threads in a warp.

Syntax

```
bar.warp.sync    membermask;
```

Description

`bar.warp.sync` will cause executing thread to wait until all threads corresponding to `membermask` have executed a `bar.warp.sync` with the same `membermask` value before resuming execution.

Operand `membermask` specifies a 32-bit integer which is a mask indicating threads participating in barrier where the bit position corresponds to thread's `laneid`.

The behavior of `bar.warp.sync` is undefined if the executing thread is not in the `membermask`.

`bar.warp.sync` also guarantee memory ordering among threads participating in barrier. Thus, threads within warp that wish to communicate via memory can store to memory, execute `bar.warp.sync`, and then safely read values stored by other threads in warp.

Note: For `.target sm_6x` or below, all threads in `membermask` must execute the same `bar.warp.sync` instruction in convergence, and only threads belonging to some `membermask` can be active when the `bar.warp.sync` instruction is executed. Otherwise, the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 6.0.

Target ISA Notes

Requires sm_30 or higher.

Examples

```
st.shared.u32 [r0],r1;           // write my result to shared memory
bar.warp.sync  0xffffffff;       // arrive, wait for others to arrive
ld.shared.u32 r2,[r3];          // read results written by other threads
```

9.7.13.3 Parallel Synchronization and Communication Instructions: `barrier.cluster`

`barrier.cluster`

Barrier synchronization within a cluster.

Syntax

```
barrier.cluster.arrive{.sem}{.aligned};
barrier.cluster.wait{.acquire}{.aligned};

.sem = {.release, .relaxed}
```

Description

Performs barrier synchronization and communication within a cluster.

`barrier.cluster` instructions can be used by the threads within the cluster for synchronization and communication.

`barrier.cluster.arrive` instruction marks warps' arrival at barrier without causing executing thread to wait for threads of other participating warps.

`barrier.cluster.wait` instruction causes the executing thread to wait for all non-exited threads of the cluster to perform `barrier.cluster.arrive`.

In addition, `barrier.cluster` instructions cause the executing thread to wait for all non-exited threads from its warp.

When all non-exited threads in the cluster have executed `barrier.cluster.arrive`, the barrier completes and is automatically reinitialized. After using `barrier.cluster.wait` to detect completion of the barrier, a thread may immediately arrive at the barrier once again. Each thread must arrive at the barrier only once before the barrier completes.

The `barrier.cluster.wait` instruction guarantees that when it completes the execution, memory accesses (except asynchronous operations) requested, in program order, prior to the preceding

`barrier.cluster.arrive` by all threads in the cluster are complete and visible to the executing thread.

There is no memory ordering and visibility guarantee for memory accesses requested by the executing thread, in program order, after `barrier.cluster.arrive` and prior to `barrier.cluster.wait`.

The optional `.relaxed` qualifier on `barrier.cluster.arrive` specifies that there are no memory ordering and visibility guarantees provided for the memory accesses performed prior to `barrier.cluster.arrive`.

The optional `.sem` and `.acquire` qualifiers on instructions `barrier.cluster.arrive` and `barrier.cluster.wait` specify the memory synchronization as described in the *Memory Consistency Model*. If the optional `.sem` qualifier is absent for `barrier.cluster.arrive`, `.release` is assumed by default. If the optional `.acquire` qualifier is absent for `barrier.cluster.wait`, `.acquire` is assumed by default.

The optional `.aligned` qualifier indicates that all threads in the warp must execute the same `barrier.cluster` instruction. In conditionally executed code, an aligned `barrier.cluster` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Support for `.acquire`, `.relaxed`, `.release` qualifiers introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
// use of arrive followed by wait
ld.shared::cluster.u32 r0, [addr];
barrier.cluster.arrive.aligned;
...
barrier.cluster.wait.aligned;
st.shared::cluster.u32 [addr], r1;

// use memory fence prior to arrive for relaxed barrier
@cta0 ld.shared::cluster.u32 r0, [addr];
fence.cluster.acq_rel;
barrier.cluster.arrive.relaxed.aligned;
...
barrier.cluster.wait.aligned;
@cta1 st.shared::cluster.u32 [addr], r1;
```

9.7.13.4 Parallel Synchronization and Communication Instructions: `membar` / `fence`

`membar`, `fence`

Enforce an ordering of memory operations.

Syntax

```
// Thread fence:
fence{.sem}.scope;

// Thread fence:
fence.acquire.sync_restrict::shared::cluster.cluster;
fence.release.sync_restrict::shared::cta.cluster;

// Operation fence:
fence.op_restrict.release.cluster;

// Proxy fence (bi-directional):
fence.proxy.proxykind;

// Proxy fence (uni-directional):
fence.proxy.to_proxykind::from_proxykind.release.scope;
fence.proxy.to_proxykind::from_proxykind.acquire.scope [addr], size;
fence.proxy.async::generic.acquire.sync_restrict::shared::cluster.cluster;
fence.proxy.async::generic.release.sync_restrict::shared::cta.cluster;

// Old style membar:
membar.level;
membar.proxy.proxykind;

.sem      = { .sc, .acq_rel, .acquire, .release };
.scope    = { .cta, .cluster, .gpu, .sys };
.level    = { .cta, .gl, .sys };
.proxykind = { .alias, .async, .async.global, .async.shared::{cta, cluster} };
.op_restrict = { .mbARRIER_init };
.to_proxykind::from_proxykind = { .tensormap::generic};
```

Description

The `membar` instruction guarantees that prior memory accesses requested by this thread (`ld`, `st`, `atom` and `red` instructions) are performed at the specified `level`, before later memory operations requested by this thread following the `membar` instruction. The `level` qualifier specifies the set of threads that may observe the ordering effect of this operation.

A memory read (e.g., by `ld` or `atom`) has been performed when the value read has been transmitted from memory and cannot be modified by another thread at the indicated level. A memory write (e.g., by `st`, `red` or `atom`) has been performed when the value written has become visible to other threads at the specified level, that is, when the previous value can no longer be read.

The `fence` instruction establishes an ordering between memory accesses requested by this thread (`ld`, `st`, `atom` and `red` instructions) as described in the [Memory Consistency Model](#). The scope qualifier specifies the set of threads that may observe the ordering effect of this operation.

`fence.acq_rel` is a light-weight fence that is sufficient for memory synchronization in most programs. Instances of `fence.acq_rel` synchronize when combined with additional memory operations as described in acquire and release patterns in the *Memory Consistency Model*. If the optional `.sem` qualifier is absent, `.acq_rel` is assumed by default.

`fence.sc` is a slower fence that can restore *sequential consistency* when used in sufficient places, at the cost of performance. Instances of `fence.sc` with sufficient scope always synchronize by forming a total order per scope, determined at runtime. This total order can be constrained further by other synchronization in the program.

Qualifiers `.op_restrict` and `.sync_restrict` restrict the class of memory operations for which the fence instruction provides the memory ordering guarantees. When `.op_restrict` is `.mbarrier_init`, the synchronizing effect of the fence only applies to the prior `mbarrier.init` operations executed by the same thread on *mbarrier objects* in `.shared::cta` state space. When `.sync_restrict` is `.sync_restrict::shared::cta`, `.sem` must be `.release`, and the effect of the fence only applies to operations performed on objects in `.shared::cta` state space. Likewise, when `.sync_restrict` is `.sync_restrict::shared::cluster`, `.sem` must be `.acquire`, and the effect of the fence only applies to operations performed on objects in `.shared::cluster` state space. When either `.sync_restrict::shared::cta` or `.sync_restrict::shared::cluster` is present, the `.scope` must be specified as `.cluster`.

The address operand `addr` and the operand size together specify the memory range `[addr, addr+size-1]` on which the ordering guarantees on the memory accesses across the proxies is to be provided. The only supported value for the size operand is 128, which must be a constant integer literal. *Generic Addressing* is used unconditionally, and the address specified by the operand `addr` must fall within the `.global` state space. Otherwise, the behavior is undefined.

On `sm_70` and higher `membar` is a synonym for `fence.sc`¹, and the `membar` levels `cta`, `gl` and `sys` are synonymous with the fence scopes `cta`, `gpu` and `sys` respectively.

`membar.proxy` and `fence.proxy` instructions establish an ordering between memory accesses that may happen through different *proxies*.

A *uni-directional* proxy ordering from the *from-proxykind* to the *to-proxykind* establishes ordering between a prior memory access performed via the *from-proxykind* and a subsequent memory access performed via the *to-proxykind*.

A *bi-directional* proxy ordering between two proxykinds establishes two *uni-directional* proxy orderings: one from the first proxykind to the second proxykind and the other from the second proxykind to the first proxykind.

The `.proxykind` qualifier indicates the *bi-directional* proxy ordering that is established between the memory accesses done between the generic proxy and the proxy specified by `.proxykind`.

Value `.alias` of the `.proxykind` qualifier refers to memory accesses performed using virtually aliased addresses to the same memory location. Value `.async` of the `.proxykind` qualifier specifies that the memory ordering is established between the *async proxy* and the generic proxy. The memory ordering is limited only to operations performed on objects in the state space specified. If no state space is specified, then the memory ordering applies on all state spaces.

A *.release proxy fence* can form a release sequence that synchronizes with an acquire sequence that contains a *.acquire proxy fence*. The `.to_proxykind` and `.from_proxykind` qualifiers indicate the *uni-directional* proxy ordering that is established.

On `sm_70` and higher, `membar.proxy` is a synonym for `fence.proxy`.

¹ The semantics of `fence.sc` introduced with `sm_70` is a superset of the semantics of `membar` and the two are compatible; when executing on `sm_70` or later architectures, `membar` acquires the full semantics of `fence.sc`.

PTX ISA Notes

`membar.{cta,gl}` introduced in PTX ISA version 1.4.
`membar.sys` introduced in PTX ISA version 2.0.
`fence` introduced in PTX ISA version 6.0.
`membar.proxy` and `fence.proxy` introduced in PTX ISA version 7.5.
`.cluster` scope qualifier introduced in PTX ISA version 7.8.
`.op_restrict` qualifier introduced in PTX ISA version 8.0.
`fence.proxy.async` is introduced in PTX ISA version 8.0.
`.to_proxykind::from_proxykind` qualifier introduced in PTX ISA version 8.3.
`.acquire` and `.release` qualifiers for `fence` instruction introduced in PTX ISA version 8.6.
`.sync_restrict` qualifier introduced in PTX ISA version 8.6.

Target ISA Notes

`membar.{cta,gl}` supported on all target architectures.
`membar.sys` requires `sm_20` or higher.
`fence` requires `sm_70` or higher.
`membar.proxy` requires `sm_60` or higher.
`fence.proxy` requires `sm_70` or higher.
`.cluster` scope qualifier requires `sm_90` or higher.
`.op_restrict` qualifier requires `sm_90` or higher.
`fence.proxy.async` requires `sm_90` or higher.
`.to_proxykind::from_proxykind` qualifier requires `sm_90` or higher.
`.acquire` and `.release` qualifiers for `fence` instruction require `sm_90` or higher..
`.sync_restrict` qualifier requires `sm_90` or higher..

Examples

```
membar.gl;  
membar.cta;  
membar.sys;  
fence.sc.cta;  
fence.sc.cluster;  
fence.proxy.alias;  
membar.proxy.alias;  
fence.mbarrier_init.release.cluster;  
fence.proxy.async;  
fence.proxy.async.shared::cta;  
fence.proxy.async.shared::cluster;  
fence.proxy.async.global;
```

(continues on next page)

(continued from previous page)

```

tensormap.replace.tile.global_address.global.b1024.b64    [gbl], new_addr;
fence.proxy.tensormap::generic.release.gpu;
cvta.global.u64  tmap, gbl;
fence.proxy.tensormap::generic.acquire.gpu [tmap], 128;
cp.async.bulk.tensor.1d.shared::cluster.global.tile  [addr0], [tmap, {tc0}], [mbar0];

// Acquire remote barrier state via async proxy.
barrier.cluster.wait.acquire;
fence.proxy.async::generic.acquire.sync_restrict::shared::cluster.cluster;

// Release local barrier state via generic proxy.
mbarrier.init [bar];
fence.mbarrier_init.release.cluster;
barrier.cluster.arrive.relaxed;

// Acquire local shared memory via generic proxy.
mbarrier.try_wait.relaxed.cluster.shared::cta.b64 complete, [addr], parity;
fence.acquire.sync_restrict::shared::cluster.cluster;

// Release local shared memory via generic proxy.
fence.release.sync_restrict::shared::cta.cluster;
mbarrier.arrive.relaxed.cluster.shared::cluster.b64 state, [bar];

```

9.7.13.5 Parallel Synchronization and Communication Instructions: **atom**

atom

Atomic reduction operations for thread-to-thread communication.

Syntax

Atomic operation with scalar type:

```

atom{.sem}{.scope}{.space}.op{.level::cache_hint}.type d, [a], b{, cache-policy};
atom{.sem}{.scope}{.space}.op.type d, [a], b, c;

atom{.sem}{.scope}{.space}.cas.b16 d, [a], b, c;

atom{.sem}{.scope}{.space}.cas.b128 d, [a], b, c;
atom{.sem}{.scope}{.space}.exch{.level::cache_hint}.b128 d, [a], b {, cache-policy};

atom{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.f16      d, [a], b{, cache-
↪policy};
atom{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.f16x2   d, [a], b{, cache-
↪policy};

atom{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.bf16    d, [a], b{, cache-
↪policy};
atom{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.bf16x2  d, [a], b{, cache-
↪policy};

.space =          { .global, .shared{::cta, ::cluster} };
.sem =           { .relaxed, .acquire, .release, .acq_rel };

```

(continues on next page)

(continued from previous page)

```
.scope =          { .cta, .cluster, .gpu, .sys };

.op =             { .and, .or, .xor,
                   .cas, .exch,
                   .add, .inc, .dec,
                   .min, .max };
.level::cache_hint = { .L2::cache_hint };
.type =           { .b32, .b64, .u32, .u64, .s32, .s64, .f32, .f64 };
```

Atomic operation with vector type:

```
atom{.sem}{.scope}{.global}.add{.level::cache_hint}.vec_32_bit.f32      d,
↪ [a], b{, cache-policy};
atom{.sem}{.scope}{.global}.op.noftz{.level::cache_hint}.vec_16_bit.half_word_type d,
↪ [a], b{, cache-policy};
atom{.sem}{.scope}{.global}.op.noftz{.level::cache_hint}.vec_32_bit.packed_type d,
↪ [a], b{, cache-policy};

.sem =           { .relaxed, .acquire, .release, .acq_rel };
.scope =         { .cta, .cluster, .gpu, .sys };
.op =            { .add, .min, .max };
.half_word_type = { .f16, .bf16 };
.packed_type =   { .f16x2, .bf16x2 };
.vec_16_bit =     { .v2, .v4, .v8 };
.vec_32_bit =     { .v2, .v4 };
.level::cache_hint = { .L2::cache_hint }
```

Description

Atomically loads the original value at location *a* into destination register *d*, performs a reduction operation with operand *b* and the value in location *a*, and stores the result of the specified operation at location *a*, overwriting the original value. Operand *a* specifies a location in the specified state space. If no state space is given, perform the memory accesses using *Generic Addressing*. *atom* with scalar type may be used only with *.global* and *.shared* spaces and with generic addressing, where the address points to *.global* or *.shared* space. *atom* with vector type may be used only with *.global* space and with generic addressing where the address points to *.global* space.

For *atom* with vector type, operands *d* and *b* are brace-enclosed vector expressions, size of which is equal to the size of vector qualifier.

If no sub-qualifier is specified with *.shared* state space, then *:cta* is assumed by default.

The optional *.sem* qualifier specifies a memory synchronizing effect as described in the *Memory Consistency Model*. If the *.sem* qualifier is absent, *.relaxed* is assumed by default.

The optional *.scope* qualifier specifies the set of threads that can directly observe the memory synchronizing effect of this operation, as described in the *Memory Consistency Model*. If the *.scope* qualifier is absent, *.gpu* scope is assumed by default.

For *atom* with vector type, the supported combinations of vector qualifier and types, and atomic operations supported on these combinations are depicted in the following table:

Vector qualifier	Types		
	.f16/ bf16	.f16x2/ bf16x2	.f32
.v2	.add, .min, .max	.add, .min, .max	.add
.v4	.add, .min, .max	.add, .min, .max	.add
.v8	.add, .min, .max	Not supported	Not Supported

Two atomic operations (`atom` or `red`) are performed atomically with respect to each other only if each operation specifies a scope that includes the other. When this condition is not met, each operation observes the other operation being performed as if it were split into a read followed by a dependent write.

`atom` instruction on packed type or vector type, accesses adjacent scalar elements in memory. In such cases, the atomicity is guaranteed separately for each of the individual scalar elements; the entire `atom` is not guaranteed to be atomic as a single access.

For `sm_6x` and earlier architectures, `atom` operations on `.shared` state space do not guarantee atomicity with respect to normal store instructions to the same address. It is the programmer's responsibility to guarantee correctness of programs that use shared memory atomic instructions, e.g., by inserting barriers between normal stores and atomic operations to a common address, or by using `atom.exch` to store to locations accessed by other atomic operations.

Supported addressing modes for operand `a` and alignment requirements are described in [Addresses as Operands](#)

The bit-size operations are `.and`, `.or`, `.xor`, `.cas` (compare-and-swap), and `.exch` (exchange).

The integer operations are `.add`, `.inc`, `.dec`, `.min`, `.max`. The `.inc` and `.dec` operations return a result in the range `[0..b]`.

The floating-point operation `.add` operation rounds to nearest even. Current implementation of `atom.add.f32` on global memory flushes subnormal inputs and results to sign-preserving zero; whereas `atom.add.f32` on shared memory supports subnormal inputs and results and doesn't flush them to zero.

`atom.add.f16`, `atom.add.f16x2`, `atom.add.bf16` and `atom.add.bf16x2` operation requires the `.noftz` qualifier; it preserves subnormal inputs and results, and does not flush them to zero.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

The qualifier `.level::cache_hint` is only supported for `.global` state space and for generic addressing where the address points to the `.global` state space.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

Semantics

```
atomic {  
    d = *a;  
    *a = (operation == cas) ? operation(*a, b, c)  
        : operation(*a, b);  
}  
where  
    inc(r, s) = (r >= s) ? 0 : r+1;  
    dec(r, s) = (r==0 || r > s) ? s : r-1;  
    exch(r, s) = s;  
    cas(r,s,t) = (r == s) ? t : r;
```

Notes

Simple reductions may be specified by using the *bit bucket* destination operand `_`.

PTX ISA Notes

32-bit `atom.global` introduced in PTX ISA version 1.1.

`atom.shared` and 64-bit `atom.global`. {`add`, `cas`, `exch`} introduced in PTX ISA 1.2.

`atom.add.f32` and 64-bit `atom.shared`. {`add`, `cas`, `exch`} introduced in PTX ISA 2.0.

64-bit `atom`. {`and`, `or`, `xor`, `min`, `max`} introduced in PTX ISA 3.1.

`atom.add.f64` introduced in PTX ISA 5.0.

`.scope` qualifier introduced in PTX ISA 5.0.

`.sem` qualifier introduced in PTX ISA version 6.0.

`atom.add.noftz.f16x2` introduced in PTX ISA 6.2.

`atom.add.noftz.f16` and `atom.cas.b16` introduced in PTX ISA 6.3.

Per-element atomicity of `atom.f16x2` clarified in PTX ISA version 6.3, with retrospective effect from PTX ISA version 6.2.

Support for `.level::cache_hint` qualifier introduced in PTX ISA version 7.4.

`atom.add.noftz.bf16` and `atom.add.noftz.bf16x2` introduced in PTX ISA 7.8.

Support for `.cluster` scope qualifier introduced in PTX ISA version 7.8.

Support for `::cta` and `::cluster` sub-qualifiers introduced in PTX ISA version 7.8.

Support for vector types introduced in PTX ISA version 8.1.

Support for `.b128` type introduced in PTX ISA version 8.3.

Support for `.sys` scope with `.b128` type introduced in PTX ISA version 8.4.

Target ISA Notes

atom.global requires sm_11 or higher.
 atom.shared requires sm_12 or higher.
 64-bit atom.global.{add,cas,exch} require sm_12 or higher.
 64-bit atom.shared.{add,cas,exch} require sm_20 or higher.
 64-bit atom.{and,or,xor,min,max} require sm_32 or higher.
 atom.add.f32 requires sm_20 or higher.
 atom.add.f64 requires sm_60 or higher.
 .scope qualifier requires sm_60 or higher.
 .sem qualifier requires sm_70 or higher.
 Use of generic addressing requires sm_20 or higher.
 atom.add.noftz.f16x2 requires sm_60 or higher.
 atom.add.noftz.f16 and atom.cas.b16 requires sm_70 or higher.
 Support for .level::cache_hint qualifier requires sm_80 or higher.
 atom.add.noftz.bf16 and atom.add.noftz.bf16x2 require sm_90 or higher.
 Support for .cluster scope qualifier requires sm_90 or higher.
 Sub-qualifier ::cta requires sm_30 or higher.
 Sub-qualifier ::cluster requires sm_90 or higher.
 Support for vector types requires sm_90 or higher.
 Support for .b128 type requires sm_90 or higher.

Examples

```

atom.global.add.s32 d,[a],1;
atom.shared::cta.max.u32 d,[x+4],0;
@p atom.global.cas.b32 d,[p],my_val,my_new_val;
atom.global.sys.add.u32 d,[a],1;
atom.global.acquire.sys.inc.u32 ans,[gbl],%r0;
atom.add.noftz.f16x2 d,[a],b;
atom.add.noftz.f16 hd,[ha],hb;
atom.global.cas.b16 hd,[ha],hb,hc;
atom.add.noftz.bf16 hd,[a],hb;
atom.add.noftz.bf16x2 bd,[b],bb;
atom.add.shared::cluster.noftz.f16 hd,[ha],hb;
atom.shared.b128.cas d,a,b,c; // 128-bit atom
atom.global.b128.exch d,a,b; // 128-bit atom

atom.global.cluster.relaxed.add.u32 d,[a],1;

createpolicy.fractional.L2::evict_last.b64 cache-policy,0.25;
atom.global.add.L2::cache_hint.s32 d,[a],1,cache-policy;

atom.global.v8.f16.max.noftz {%hd0,%hd1,%hd2,%hd3,%hd4,%hd5,%hd6,%hd7},[gbl],
  
```

(continues on next page)

(continued from previous page)

```

                                {%h0, %h1, %h2, %h3, %h4, %h5, %h6, %h7}
↪;
atom.global.v8.bf16.add.noftz  {%hd0, %hd1, %hd2, %hd3, %hd4, %hd5, %hd6, %hd7},
↪[gb1],
                                {%h0, %h1, %h2, %h3, %h4, %h5, %h6, %h7}
↪;
atom.global.v2.f16.add.noftz  {%hd0, %hd1}, [gb1], {%h0, %h1};
atom.global.v2.bf16.add.noftz {%hd0, %hd1}, [gb1], {%h0, %h1};
atom.global.v4.b16x2.min.noftz {%hd0, %hd1, %hd2, %hd3}, [gb1], {%h0, %h1, %h2, %h3};
atom.global.v4.f32.add  {%f0, %f1, %f2, %f3}, [gb1], {%f0, %f1, %f2, %f3};
atom.global.v2.f16x2.min.noftz {%bd0, %bd1}, [g], {%b0, %b1};
atom.global.v2.bf16x2.max.noftz {%bd0, %bd1}, [g], {%b0, %b1};
atom.global.v2.f32.add  {%f0, %f1}, [g], {%f0, %f1};

```

9.7.13.6 Parallel Synchronization and Communication Instructions: red

red

Reduction operations on global and shared memory.

Syntax

Reduction operation with scalar type:

```

red{.sem}{.scope}{.space}.op{.level::cache_hint}.type      [a], b{, cache-policy};
red{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.f16  [a], b{, cache-policy};
red{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.f16x2 [a], b{, cache-policy};
red{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.bf16
                                                                [a], b {, cache-policy};
red{.sem}{.scope}{.space}.add.noftz{.level::cache_hint}.bf16x2
                                                                [a], b {, cache-policy};

.space =                { .global, .shared{::cta, ::cluster} };
.sem =                  { .relaxed, .release };
.scope =                { .cta, .cluster, .gpu, .sys };

.op =                  { .and, .or, .xor,
                        .add, .inc, .dec,
                        .min, .max };
.level::cache_hint =  { .L2::cache_hint };
.type =                { .b32, .b64, .u32, .u64, .s32, .s64, .f32, .f64 };

```

Reduction operation with vector type:

```

red{.sem}{.scope}{.global}.add{.level::cache_hint}.vec_32_bit.f32 [a], b{, cache-
↪policy};
red{.sem}{.scope}{.global}.op.noftz{.level::cache_hint}. vec_16_bit.half_word_type
↪[a], b{, cache-policy};
red{.sem}{.scope}{.global}.op.noftz{.level::cache_hint}.vec_32_bit.packed_type [a], b
↪{, cache-policy};

```

(continues on next page)

(continued from previous page)

```
.sem =           { .relaxed, .release };
.scope =        { .cta, .cluster, .gpu, .sys };
.op =           { .add, .min, .max };
.half_word_type = { .f16, .bf16 };
.packed_type =  { .f16x2, .bf16x2 };
.vec_16_bit =   { .v2, .v4, .v8 };
.vec_32_bit =   { .v2, .v4 };
.level::cache_hint = { .L2::cache_hint }
```

Description

Performs a reduction operation with operand *b* and the value in location *a*, and stores the result of the specified operation at location *a*, overwriting the original value. Operand *a* specifies a location in the specified state space. If no state space is given, perform the memory accesses using [Generic Addressing](#). *red* with scalar type may be used only with *.global* and *.shared* spaces and with generic addressing, where the address points to *.global* or *.shared* space. *red* with vector type may be used only with *.global* space and with generic addressing where the address points to *.global* space.

For *red* with vector type, operand *b* is brace-enclosed vector expressions, size of which is equal to the size of vector qualifier.

If no sub-qualifier is specified with *.shared* state space, then *::cta* is assumed by default.

The optional *.sem* qualifier specifies a memory synchronizing effect as described in the [Memory Consistency Model](#). If the *.sem* qualifier is absent, *.relaxed* is assumed by default.

The optional *.scope* qualifier specifies the set of threads that can directly observe the memory synchronizing effect of this operation, as described in the [Memory Consistency Model](#). If the *.scope* qualifier is absent, *.gpu* scope is assumed by default.

For *red* with vector type, the supported combinations of vector qualifier, types and reduction operations supported on these combinations are depicted in following table:

Vector qualifier	Types		
	.f16/ bf16	.f16x2/ bf16x2	.f32
.v2	.add, .min, .max	.add, .min, .max	.add
.v4	.add, .min, .max	.add, .min, .max	.add
.v8	.add, .min, .max	Not supported	Not Supported

Two atomic operations (*atom* or *red*) are performed atomically with respect to each other only if each operation specifies a scope that includes the other. When this condition is not met, each operation observes the other operation being performed as if it were split into a read followed by a dependent write.

red instruction on packed type or vector type, accesses adjacent scalar elements in memory. In such case, the atomicity is guaranteed separately for each of the individual scalar elements; the entire *red* is not guaranteed to be atomic as a single access.

For *sm_6x* and earlier architectures, *red* operations on *.shared* state space do not guarantee atomicity with respect to normal store instructions to the same address. It is the programmer's responsibility to guarantee correctness of programs that use shared memory reduction instructions, e.g., by

inserting barriers between normal stores and reduction operations to a common address, or by using `atom.exch` to store to locations accessed by other reduction operations.

Supported addressing modes for operand `a` and alignment requirements are described in [Addresses as Operands](#)

The bit-size operations are `.and`, `.or`, and `.xor`.

The integer operations are `.add`, `.inc`, `.dec`, `.min`, `.max`. The `.inc` and `.dec` operations return a result in the range `[0..b]`.

The floating-point operation `.add` operation rounds to nearest even. Current implementation of `red.add.f32` on global memory flushes subnormal inputs and results to sign-preserving zero; whereas `red.add.f32` on shared memory supports subnormal inputs and results and doesn't flush them to zero.

`red.add.f16`, `red.add.f16x2`, `red.add.bf16` and `red.add.bf16x2` operation requires the `.noftz` qualifier; it preserves subnormal inputs and results, and does not flush them to zero.

When the optional argument `cache-policy` is specified, the qualifier `.level::cache_hint` is required. The 64-bit operand `cache-policy` specifies the cache eviction policy that may be used during the memory access.

The qualifier `.level::cache_hint` is only supported for `.global` state space and for generic addressing where the address points to the `.global` state space.

`cache-policy` is a hint to the cache subsystem and may not always be respected. It is treated as a performance hint only, and does not change the memory consistency behavior of the program.

Semantics

```
*a = operation(*a, b);

where
    inc(r, s) = (r >= s) ? 0 : r+1;
    dec(r, s) = (r==0 || r > s) ? s : r-1;
```

PTX ISA Notes

Introduced in PTX ISA version 1.2.

`red.add.f32` and `red.shared.add.u64` introduced in PTX ISA 2.0.

64-bit `red.{and,or,xor,min,max}` introduced in PTX ISA 3.1.

`red.add.f64` introduced in PTX ISA 5.0.

`.scope` qualifier introduced in PTX ISA 5.0.

`.sem` qualifier introduced in PTX ISA version 6.0.

`red.add.noftz.f16x2` introduced in PTX ISA 6.2.

`red.add.noftz.f16` introduced in PTX ISA 6.3.

Per-element atomicity of `red.f16x2` clarified in PTX ISA version 6.3, with retrospective effect from PTX ISA version 6.2

Support for `.level::cache_hint` qualifier introduced in PTX ISA version 7.4.

`red.add.noftz.bf16` and `red.add.noftz.bf16x2` introduced in PTX ISA 7.8.

Support for `.cluster` scope qualifier introduced in PTX ISA version 7.8.

Support for `::cta` and `::cluster` sub-qualifiers introduced in PTX ISA version 7.8.

Support for vector types introduced in PTX ISA version 8.1.

Target ISA Notes

`red.global` requires `sm_11` or higher

`red.shared` requires `sm_12` or higher.

`red.global.add.u64` requires `sm_12` or higher.

`red.shared.add.u64` requires `sm_20` or higher.

64-bit `red.{and, or, xor, min, max}` require `sm_32` or higher.

`red.add.f32` requires `sm_20` or higher.

`red.add.f64` requires `sm_60` or higher.

`.scope` qualifier requires `sm_60` or higher.

`.sem` qualifier requires `sm_70` or higher.

Use of generic addressing requires `sm_20` or higher.

`red.add.noftz.f16x2` requires `sm_60` or higher.

`red.add.noftz.f16` requires `sm_70` or higher.

Support for `.level::cache_hint` qualifier requires `sm_80` or higher.

`red.add.noftz.bf16` and `red.add.noftz.bf16x2` require `sm_90` or higher.

Support for `.cluster` scope qualifier requires `sm_90` or higher.

Sub-qualifier `::cta` requires `sm_30` or higher.

Sub-qualifier `::cluster` requires `sm_90` or higher.

Support for vector types requires `sm_90` or higher.

Examples

```
red.global.add.s32 [a],1;
red.shared::cluster.max.u32 [x+4],0;
@p red.global.and.b32 [p],my_val;
red.global.sys.add.u32 [a], 1;
red.global.acquire.sys.add.u32 [gb1], 1;
red.add.noftz.f16x2 [a], b;
red.add.noftz.bf16 [a], hb;
red.add.noftz.bf16x2 [b], bb;
red.global.cluster.relaxed.add.u32 [a], 1;
red.shared::cta.min.u32 [x+4],0;

createpolicy.fractional.L2::evict_last.b64 cache-policy, 0.25;
red.global.and.L2::cache_hint.b32 [a], 1, cache-policy;

red.global.v8.f16.add.noftz [gb1], {%h0, %h1, %h2, %h3, %h4, %h5, %h6, %h7};
red.global.v8.bf16.min.noftz [gb1], {%h0, %h1, %h2, %h3, %h4, %h5, %h6, %h7};
```

(continues on next page)

(continued from previous page)

```

red.global.v2.f16.add.noftz [gbl], {%h0, %h1};
red.global.v2.bf16.add.noftz [gbl], {%h0, %h1};
red.global.v4.f16x2.max.noftz [gbl], {%h0, %h1, %h2, %h3};
red.global.v4.f32.add [gbl], {%f0, %f1, %f2, %f3};
red.global.v2.f16x2.max.noftz {%bd0, %bd1}, [g], {%b0, %b1};
red.global.v2.bf16x2.add.noftz {%bd0, %bd1}, [g], {%b0, %b1};
red.global.v2.f32.add {%f0, %f1}, [g], {%f0, %f1};

```

9.7.13.7 Parallel Synchronization and Communication Instructions: `red.async`

`red.async`

Asynchronous reduction operation.

Syntax

```

// Increment and Decrement reductions
red.async.sem.scope{.ss}.completion_mechanism.op.type [a], b, [mbar];

.sem = { .relaxed };
.scope = { .cluster };
.ss = { .shared::cluster };
.op = { .inc, .dec };
.type = { .u32 };
.completion_mechanism = { .mbarrier::complete_tx::bytes };

// MIN and MAX reductions
red.async.sem.scope{.ss}.completion_mechanism.op.type [a], b, [mbar];

.sem = { .relaxed };
.scope = { .cluster };
.ss = { .shared::cluster };
.op = { .min, .max };
.type = { .u32, .s32 };
.completion_mechanism = { .mbarrier::complete_tx::bytes };

// Bitwise AND, OR and XOR reductions
red.async.sem.scope{.ss}.completion_mechanism.op.type [a], b, [mbar];

.sem = { .relaxed };
.scope = { .cluster };
.ss = { .shared::cluster };
.op = { .and, .or, .xor };
.type = { .b32 };
.completion_mechanism = { .mbarrier::complete_tx::bytes };

// ADD reductions
red.async.sem.scope{.ss}.completion_mechanism.add.type [a], b, [mbar];

.sem = { .relaxed };
.scope = { .cluster };
.ss = { .shared::cluster };

```

(continues on next page)

(continued from previous page)

```
.type = { .u32, .s32, .u64 };
.completion_mechanism = { .mbarrier::complete_tx::bytes };

red.async{.mmio}.sem.scope{.ss}.add.type [a], b;

.sem = { .release };
.scope = { .gpu, .cluster };
.ss = { .global };
.type = { .u32, .s32, .u64, .s64 };
```

Description

`red.async` is a non-blocking instruction which initiates an asynchronous reduction operation specified by `.op`, with the operand `b` and the value at destination shared memory location specified by operand `a`.

Operands

- ▶ `a` is a destination address, and must be either a register, or of the form `register + immOff`, as described in *Addresses as Operands*.
- ▶ `b` is a source value, of the type indicated by qualifier `.type`.
- ▶ `mbar` is an mbarrier object address.

Qualifiers

- ▶ `.mmio` indicates whether this is an *mmio Operation*.
- ▶ `.sem` specifies the memory ordering semantics as described in the *Memory Consistency Model*.
- ▶ `.scope` specifies the set of threads with which this instruction can directly synchronize.
- ▶ `.ss` specifies the state space of the destination operand `a` and the mbarrier operand `mbar`.
 - ▶ If `.ss` is not specified, *Generic Addressing* is used.
- ▶ `.completion_mechanism` specifies the mechanism for observing the completion of the asynchronous operation.
 - ▶ When `.completion_mechanism` is `.mbarrier::complete_tx::bytes`: upon completion of the asynchronous operation, a *complete-tx* operation will be performed on the mbarrier object specified by the operand `mbar`, with `completeCount` argument equal to the amount of data stored in bytes.
 - ▶ When `.completion_mechanism` is not specified: the completion of the store synchronizes with the end of the CTA. This instruction accesses its mbarrier operand using generic-proxy.
- ▶ `.op` specifies the reduction operation.
 - ▶ The `.inc` and `.dec` operations return a result in the range `[0..b]`.
- ▶ `.type` specifies the type of the source operand `b`.

Conditions

When `.sem` is `.relaxed`:

- ▶ The reduce operation is a relaxed memory operation.
- ▶ The complete-tx operation on the mbarrier has `.release` semantics at `.cluster` scope.
- ▶ The shared-memory addresses of the destination operand `a` and the mbarrier operand `mbar` must meet all of the following conditions:
 - ▶ They belong to the same CTA.
 - ▶ The CTA to which they belong is different from the CTA of the executing thread, but must be within the same cluster.

Otherwise, the behavior is undefined.

- ▶ `.mmio` must not be specified.
- ▶ If `.ss` is specified, it must be `.shared::cluster`.
- ▶ If `.ss` is not specified, generic addressing is used for operands `a` and `mbar`. If the generic addresses specified do not fall within the address window of `.shared::cluster` state space, the behavior is undefined.
- ▶ If `.completion_mechanism` is specified, it must be `.mbarrier::complete_tx::bytes`.
- ▶ If `.completion_mechanism` is not specified, it defaults to `.mbarrier::complete_tx::bytes`.

When `.sem` is `.release`:

- ▶ The reduce operation is a strong memory operation with `.release` semantics at the scope specified by `.scope`.
- ▶ If `.mmio` is specified, `.scope` must be `.sys`.
- ▶ If `.ss` is specified, it must be `.global`.
- ▶ If `.ss` is not specified, generic addressing is used for operand `a`. If the generic address specified does not fall within the address window of `.global` state space, the behavior is undefined.
- ▶ `.completion_mechanism` must not be specified.

PTX ISA Notes

Introduced in PTX ISA version 8.1.

Support for `.mmio` qualifier, `.release` semantics, `.global` state space, and `.gpu` and `.sys` scopes introduced in PTX ISA version 8.7.

Target ISA Notes

Requires sm_90 or higher.

.mmio qualifier, .release semantics, .global state space, and .gpu and .sys scopes require sm_100 or higher.

Examples

```
red.async.relaxed.cluster.shared::cluster.mbarrier::complete_tx::bytes.min.u32 [addr],
↪ b, [mbar_addr];

red.async.release.sys.global.add.u32 [addr], b;
```

9.7.13.8 Parallel Synchronization and Communication Instructions: vote (deprecated)

vote (deprecated)

Vote across thread group.

Syntax

```
vote.mode.pred d, {!}a;
vote.ballot.b32 d, {!}a; // 'ballot' form, returns bitmask

.mode = { .all, .any, .uni };
```

Deprecation Note

The vote instruction without a .sync qualifier is deprecated in PTX ISA version 6.0.

- Support for this instruction with .target lower than sm_70 may be removed in a future PTX ISA version.

Removal Note

Support for vote instruction without a .sync qualifier is removed in PTX ISA version 6.4 for .target sm_70 or higher.

Description

Performs a reduction of the source predicate across all active threads in a warp. The destination predicate value is the same across all threads in the warp.

The reduction modes are:

.all

True if source predicate is True for all active threads in warp. Negate the source predicate to compute `.none`.

.any

True if source predicate is True for some active thread in warp. Negate the source predicate to compute `.not_all`.

.uni

True if source predicate has the same value in all active threads in warp. Negating the source predicate also computes `.uni`.

In the *ballot* form, `vote.ballot.b32` simply copies the predicate from each thread in a warp into the corresponding bit position of destination register `d`, where the bit position corresponds to the thread's lane id.

An inactive thread in warp will contribute a 0 for its entry when participating in `vote.ballot.b32`.

PTX ISA Notes

Introduced in PTX ISA version 1.2.

Deprecated in PTX ISA version 6.0 in favor of `vote.sync`.

Not supported in PTX ISA version 6.4 for `.target sm_70` or higher.

Target ISA Notes

`vote` requires `sm_12` or higher.

`vote.ballot.b32` requires `sm_20` or higher.

`vote` is not supported on `sm_70` or higher starting PTX ISA version 6.4.

Release Notes

Note that `vote` applies to threads in a single warp, not across an entire CTA.

Examples

```
vote.all.pred    p,q;
vote.uni.pred    p,q;
vote.ballot.b32  r1,p; // get 'ballot' across warp
```

9.7.13.9 Parallel Synchronization and Communication Instructions: `vote.sync`

`vote.sync`

Vote across thread group.

Syntax

```
vote.sync.mode.pred d, {!}a, membermask;
vote.sync.ballot.b32 d, {!}a, membermask; // 'ballot' form, returns bitmask

.mode = { .all, .any, .uni };
```

Description

`vote.sync` will cause executing thread to wait until all non-exited threads corresponding to `membermask` have executed `vote.sync` with the same qualifiers and same `membermask` value before resuming execution.

Operand `membermask` specifies a 32-bit integer which is a mask indicating threads participating in this instruction where the bit position corresponds to thread's `laneid`. Operand `a` is a predicate register.

In the *mode* form, `vote.sync` performs a reduction of the source predicate across all non-exited threads in `membermask`. The destination operand `d` is a predicate register and its value is the same across all threads in `membermask`.

The reduction modes are:

.all

True if source predicate is True for all non-exited threads in `membermask`. Negate the source predicate to compute `.none`.

.any

True if source predicate is True for some thread in `membermask`. Negate the source predicate to compute `.not_all`.

.uni

True if source predicate has the same value in all non-exited threads in `membermask`. Negating the source predicate also computes `.uni`.

In the *ballot* form, the destination operand `d` is a `.b32` register. In this form, `vote.sync.ballot.b32` simply copies the predicate from each thread in `membermask` into the corresponding bit position of destination register `d`, where the bit position corresponds to the thread's lane id.

A thread not specified in `membermask` will contribute a 0 for its entry in `vote.sync.ballot.b32`.

The behavior of `vote.sync` is undefined if the executing thread is not in the `membermask`.

Note: For `.target sm_6x` or below, all threads in `membermask` must execute the same `vote.sync` instruction in convergence, and only threads belonging to some `membermask` can be active when the `vote.sync` instruction is executed. Otherwise, the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 6.0.

Target ISA Notes

Requires sm_30 or higher.

Examples

```
vote.sync.all.pred    p,q,0xffffffff;  
vote.sync.ballot.b32  r1,p,0xffffffff; // get 'ballot' across warp
```

9.7.13.10 Parallel Synchronization and Communication Instructions: `match.sync`

`match.sync`

Broadcast and compare a value across threads in warp.

Syntax

```
match.any.sync.type  d, a, membermask;  
match.all.sync.type  d[|p], a, membermask;  
  
.type = { .b32, .b64 };
```

Description

`match.sync` will cause executing thread to wait until all non-exited threads from `membermask` have executed `match.sync` with the same qualifiers and same `membermask` value before resuming execution.

Operand `membermask` specifies a 32-bit integer which is a mask indicating threads participating in this instruction where the bit position corresponds to thread's laneid.

`match.sync` performs broadcast and compare of operand `a` across all non-exited threads in `membermask` and sets destination `d` and optional predicate `p` based on mode.

Operand `a` has instruction type and `d` has `.b32` type.

Destination `d` is a 32-bit mask where bit position in mask corresponds to thread's laneid.

The matching operation modes are:

.all

`d` is set to mask corresponding to non-exited threads in `membermask` if all non-exited threads in `membermask` have same value of operand `a`; otherwise `d` is set to 0. Optionally predicate `p` is set to true if all non-exited threads in `membermask` have same value of operand `a`; otherwise `p` is set to false. The sink symbol `'_'` may be used in place of any one of the destination operands.

.any

`d` is set to mask of non-exited threads in `membermask` that have same value of operand `a`.

The behavior of `match.sync` is undefined if the executing thread is not in the `membermask`.

PTX ISA Notes

Introduced in PTX ISA version 6.0.

Target ISA Notes

Requires `sm_70` or higher.

Release Notes

Note that `match.sync` applies to threads in a single warp, not across an entire CTA.

Examples

```
match.any.sync.b32    d, a, 0xffffffff;
match.all.sync.b64    d|p, a, mask;
```

9.7.13.11 Parallel Synchronization and Communication Instructions: `activemask`**`activemask`**

Queries the active threads within a warp.

Syntax

```
activemask.b32 d;
```

Description

`activemask` queries predicated-on active threads from the executing warp and sets the destination `d` with 32-bit integer mask where bit position in the mask corresponds to the thread's `laneid`.

Destination `d` is a 32-bit destination register.

An active thread will contribute 1 for its entry in the result and exited or inactive or predicated-off thread will contribute 0 for its entry in the result.

PTX ISA Notes

Introduced in PTX ISA version 6.2.

Target ISA Notes

Requires sm_30 or higher.

Examples

```
activemask.b32 %r1;
```

9.7.13.12 Parallel Synchronization and Communication Instructions: `redux.sync`

`redux.sync`

Perform reduction operation on the data from each predicated active thread in the thread group.

Syntax

```
redux.sync.op.type dst, src, membermask;  
.op    = { .add, .min, .max }  
.type  = { .u32, .s32 }  
  
redux.sync.op.b32 dst, src, membermask;  
.op    = { .and, .or, .xor }  
  
redux.sync.op{ .abs. }{ .NaN }.f32 dst, src, membermask;  
.op    = { .min, .max }
```

Description

`redux.sync` will cause the executing thread to wait until all non-exited threads corresponding to `membermask` have executed `redux.sync` with the same qualifiers and same `membermask` value before resuming execution.

Operand `membermask` specifies a 32-bit integer which is a mask indicating threads participating in this instruction where the bit position corresponds to thread's `laneid`.

`redux.sync` performs a reduction operation `.op` of the 32 bit source register `src` across all non-exited threads in the `membermask`. The result of the reduction operation is written to the 32 bit destination register `dst`.

Reduction operation can be one of the bitwise operation in `.and`, `.or`, `.xor` or arithmetic operation in `.add`, `.min`, `.max`.

For the `.add` operation result is truncated to 32 bits.

For `.f32` instruction type, if the input value is 0.0 then `+0.0 > -0.0`.

If `.abs` qualifier is specified, then the absolute value of the input is considered for the reduction operation.

If the `.NaN` qualifier is specified, then the result of the reduction operation is canonical NaN if the input to the reduction operation from any participating thread is NaN.

In the absence of `.NaN` qualifier, only non-NaN values are considered for the reduction operation and the result will be canonical NaN when all inputs are NaNs.

The behavior of `redux.sync` is undefined if the executing thread is not in the `membermask`.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for `.f32` type is introduced in PTX ISA version 8.6.

Support for `.abs` and `.NaN` qualifiers is introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_80` or higher.

`.f32` type requires `sm_100a` and is supported on `sm_100f` from PTX ISA version 8.8.

Qualifiers `.abs` and `.NaN` require `sm_100a` and are supported on `sm_100f` or higher in the same family from PTX ISA version 8.8.

Release Notes

Note that `redux.sync` applies to threads in a single warp, not across an entire CTA.

Examples

```
.reg .b32 dst, src, init, mask;
redux.sync.add.s32 dst, src, 0xff;
redux.sync.xor.b32 dst, src, mask;

redux.sync.min.abs.NaN.f32 dst, src, mask;
```

9.7.13.13 Parallel Synchronization and Communication Instructions: `griddepcontrol`

`griddepcontrol`

Control execution of dependent grids.

Syntax

```
griddepcontrol.action;  
  
.action    = { .launch_dependents, .wait }
```

Description

The `griddepcontrol` instruction allows the dependent grids and prerequisite grids as defined by the runtime, to control execution in the following way:

`.launch_dependents` modifier signals that specific dependents the runtime system designated to react to this instruction can be scheduled as soon as all other CTAs in the grid issue the same instruction or have completed. The dependent may launch before the completion of the current grid. There is no guarantee that the dependent will launch before the completion of the current grid. Repeated invocations of this instruction by threads in the current CTA will have no additional side effects past that of the first invocation.

`.wait` modifier causes the executing thread to wait until all prerequisite grids in flight have completed and all the memory operations from the prerequisite grids are performed and made visible to the current grid.

Note: If the prerequisite grid is using `griddepcontrol.launch_dependents`, then the dependent grid must use `griddepcontrol.wait` to ensure correct functional execution.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
griddepcontrol.launch_dependents;  
griddepcontrol.wait;
```

9.7.13.14 Parallel Synchronization and Communication Instructions: `elect.sync`

`elect.sync`

Elect a leader thread from a set of threads.

Syntax

```
elect.sync d|p, membermask;
```

Description

`elect.sync` elects one predicated active leader thread from among a set of threads specified by `membermask`. `laneid` of the elected thread is returned in the 32-bit destination operand `d`. The sink symbol `'_'` can be used for destination operand `d`. The predicate destination `p` is set to `True` for the leader thread, and `False` for all other threads.

Operand `membermask` specifies a 32-bit integer indicating the set of threads from which a leader is to be elected. The behavior is undefined if the executing thread is not in `membermask`.

Election of a leader thread happens deterministically, i.e. the same leader thread is elected for the same `membermask` every time.

The mandatory `.sync` qualifier indicates that `elect` causes the executing thread to wait until all threads in the `membermask` execute the `elect` instruction before resuming execution.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
elect.sync    %r0|%p0, 0xffffffff;
```

9.7.13.15 Parallel Synchronization and Communication Instructions: `mbarrier`

`mbarrier` is a barrier created in shared memory that supports :

- ▶ Synchronizing any subset of threads within a CTA
- ▶ One-way synchronization of threads across CTAs of a cluster. As noted in *[mbarrier support with shared memory](#)*, threads can perform only *arrive* operations but not **_wait* on an `mbarrier` located in `shared::cluster` space.

- ▶ Waiting for completion of asynchronous memory operations initiated by a thread and making them visible to other threads.

An *mbarrier object* is an opaque object in memory which can be initialized and invalidated using :

- ▶ `mbarrier.init`
- ▶ `mbarrier.inval`

Operations supported on *mbarrier objects* are :

- ▶ `mbarrier.expect_tx`
- ▶ `mbarrier.complete_tx`
- ▶ `mbarrier.arrive`
- ▶ `mbarrier.arrive_drop`
- ▶ `mbarrier.test_wait`
- ▶ `mbarrier.try_wait`
- ▶ `mbarrier.pending_count`
- ▶ `cp.async.mbarrier.arrive`

Performing any *mbarrier* operation except `mbarrier.init` on an uninitialized *mbarrier object* results in undefined behavior. Performing any *non-mbarrier* or `mbarrier.init` operations on an initialized *mbarrier object* results in undefined behavior.

Unlike `bar{.cta}/barrier{.cta}` instructions which can access a limited number of barriers per CTA, *mbarrier objects* are user defined and are only limited by the total shared memory size available.

mbarrier operations enable threads to perform useful work after the arrival at the *mbarrier* and before waiting for the *mbarrier* to complete.

9.7.13.15.1 Size and alignment of mbarrier object

An *mbarrier object* is an opaque object with the following type and alignment requirements :

Type	Alignment (bytes)	Memory space
.b64	8	.shared

9.7.13.15.2 Contents of the mbarrier object

An opaque *mbarrier object* keeps track of the following information :

- ▶ Current phase of the *mbarrier object*
- ▶ Count of pending arrivals for the current phase of the *mbarrier object*
- ▶ Count of expected arrivals for the next phase of the *mbarrier object*
- ▶ Count of pending asynchronous memory operations (or transactions) tracked by the current phase of the *mbarrier object*. This is also referred to as *tx-count*.

An *mbarrier object* progresses through a sequence of phases where each phase is defined by threads performing an expected number of *arrive-on* operations.

The valid range of each of the counts is as shown below:

Count name	Minimum value	Maximum value
Expected arrival count	1	$2^{20} - 1$
Pending arrival count	0	$2^{20} - 1$
tx-count	$-(2^{20} - 1)$	$2^{20} - 1$

9.7.13.15.3 Lifecycle of the mbarrier object

The *mbarrier object* must be initialized prior to use.

An *mbarrier object* is used to synchronize threads and asynchronous memory operations.

An *mbarrier object* may be used to perform a sequence of such synchronizations.

An *mbarrier object* must be invalidated to repurpose its memory for any purpose, including repurposing it for another *mbarrier object*.

9.7.13.15.4 Phase of the mbarrier object

The phase of an *mbarrier object* is the number of times the *mbarrier object* has been used to synchronize threads and *asynchronous* operations. In each phase {0, 1, 2, ...}, threads perform in program order :

- *arrive-on* operations to complete the current phase and
- *test_wait* / *try_wait* operations to check for the completion of the current phase.

An *mbarrier object* is automatically reinitialized upon completion of the current phase for immediate use in the next phase. The current phase is incomplete and all prior phases are complete.

For each phase of the *mbarrier object*, at least one *test_wait* or *try_wait* operation must be performed which returns True for `waitComplete` before an *arrive-on* operation in the subsequent phase.

9.7.13.15.5 Tracking asynchronous operations by the mbarrier object

Starting with the Hopper architecture (sm_9x), *mbarrier object* supports a new count, called *tx-count*, which is used for tracking the completion of asynchronous memory operations or transactions. *tx-count* tracks the number of asynchronous transactions, in units specified by the asynchronous memory operation, that are outstanding and yet to be complete.

The *tx-count* of an *mbarrier object* must be set to the total amount of asynchronous memory operations, in units as specified by the asynchronous operations, to be tracked by the current phase. Upon completion of each of the asynchronous operations, the *complete-tx* operation will be performed on the *mbarrier object* and thus progress the *mbarrier* towards the completion of the current phase.

9.7.13.15.5.1 expect-tx operation

The *expect-tx* operation, with an `expectCount` argument, increases the *tx-count* of an *mbarrier object* by the value specified by `expectCount`. This sets the current phase of the *mbarrier object* to expect and track the completion of additional asynchronous transactions.

9.7.13.15.5.2 complete-tx operation

The *complete-tx* operation, with an `completeCount` argument, on an *mbarrier object* consists of the following:

mbarrier signaling

Signals the completion of asynchronous transactions that were tracked by the current phase. As a result of this, *tx-count* is decremented by `completeCount`.

mbarrier potentially completing the current phase

If the current phase has been completed then the *mbarrier* transitions to the next phase. Refer to *Phase Completion of the mbarrier object* for details on phase completion requirements and phase transition process.

9.7.13.15.6 Phase Completion of the mbarrier object

The requirements for completion of the current phase are described below. Upon completion of the current phase, the phase transitions to the subsequent phase as described below.

Current phase completion requirements

An *mbarrier object* completes the current phase when all of the following conditions are met:

- ▶ The count of the pending arrivals has reached zero.
- ▶ The *tx-count* has reached zero.

Phase transition

When an *mbarrier object* completes the current phase, the following actions are performed atomically:

- ▶ The *mbarrier object* transitions to the next phase.
- ▶ The pending arrival count is reinitialized to the expected arrival count.

9.7.13.15.7 Arrive-on operation on mbarrier object

An *arrive-on* operation, with an optional *count* argument, on an *mbarrier object* consists of the following 2 steps :

- ▶ *mbarrier signalling*:

Signals the arrival of the executing thread OR completion of the asynchronous instruction which signals the arrive-on operation initiated by the executing thread on the *mbarrier object*. As a result of this, the pending arrival count is decremented by *count*. If the *count* argument is not specified, then it defaults to 1.

- ▶ *mbarrier potentially completing the current phase*:

If the current phase has been completed then the mbarrier transitions to the next phase. Refer to *Phase Completion of the mbarrier object* for details on phase completion requirements and phase transition process.

9.7.13.15.8 mbarrier support with shared memory

The following table summarizes the support of various mbarrier operations on *mbarrier objects* located at different shared memory locations:

mbarrier operations	.shared::cta	.shared::cluster
mbarrier.arrive	Supported	Supported, cannot return result
mbarrier.expect_tx	Supported	Supported
mbarrier.complete_tx	Supported	Supported
Other mbarrier operations	Supported	Not supported

9.7.13.15.9 Parallel Synchronization and Communication Instructions: mbarrier.init

mbarrier.init

Initialize the *mbarrier object*.

Syntax

```
mbarrier.init{.shared{::cta}}.b64 [addr], count;
```

Description

`mbarrier.init` initializes the *mbarrier object* at the location specified by the address operand `addr` with the unsigned 32-bit integer `count`. The value of operand `count` must be in the range as specified in *Contents of the mbarrier object*.

Initialization of the *mbarrier object* involves :

- Initializing the current phase to 0.
- Initializing the expected arrival count to `count`.
- Initializing the pending arrival count to `count`.
- Initializing the *tx-count* to 0.

The valid range of values for the operand `count` is $[1, \dots, 2^{20} - 1]$. Refer *Contents of the mbarrier object* for the valid range of values for the various constituents of the mbarrier.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` state space then the behavior is undefined.

Supported addressing modes for operand `addr` is as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

The behavior of performing an `mbarrier.init` operation on a memory location containing a valid *mbarrier object* is undefined; invalidate the *mbarrier object* using `mbarrier.inval` first, before repurposing the memory location for any other purpose, including another *mbarrier object*.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for sub-qualifier `::cta` on `.shared` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
.shared .b64 shMem, shMem2;
.reg   .b64 addr;
.reg   .b32 %r1;

cvta.shared.u64      addr, shMem2;
mbarrier.init.b64    [addr], %r1;
bar.cta.sync         0;
// ... other mbarrier operations on addr

mbarrier.init.shared::cta.b64 [shMem], 12;
bar.sync             0;
// ... other mbarrier operations on shMem
```

9.7.13.15.10 Parallel Synchronization and Communication Instructions: `mbarrier.inval`

`mbarrier.inval`

Invalidates the *mbarrier object*.

Syntax

```
mbarrier.inval{.shared{::cta}}.b64 [addr];
```


Description

`mbarrier.inval` invalidates the *mbarrier object* at the location specified by the address operand `addr`.

An *mbarrier object* must be invalidated before using its memory location for any other purpose.

Performing any *mbarrier* operation except `mbarrier.init` on a memory location that does not contain a valid *mbarrier object*, results in undefined behaviour.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` state space then the behavior is undefined.

Supported addressing modes for operand `addr` is as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for sub-qualifier `::cta` on `.shared` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
.shared .b64 shmem;
.reg    .b64 addr;
.reg    .b32 %r1;
.reg    .pred t0;

// Example 1 :
bar.sync                0;
@t0 mbarrier.init.b64    [addr], %r1;
// ... other mbarrier operations on addr
bar.sync                0;
@t0 mbarrier.inval.b64   [addr];

// Example 2 :
bar.cta.sync            0;
mbarrier.init.shared.b64 [shmem], 12;
// ... other mbarrier operations on shmem
bar.cta.sync            0;
@t0 mbarrier.inval.shared.b64 [shmem];

// shmem can be reused here for unrelated use :
bar.cta.sync            0;
st.shared.b64           [shmem], ...;

// shmem can be re-initialized as mbarrier object :
bar.cta.sync            0;
```

(continues on next page)

(continued from previous page)

```
@t0 mbarrier.init.shared.b64      [shmem], 24;  
// ... other mbarrier operations on shmem  
bar.cta.sync                      0;  
@t0 mbarrier.inval.shared::cta.b64 [shmem];
```

9.7.13.15.11 Parallel Synchronization and Communication Instructions: `mbarrier.expect_tx`

`mbarrier.expect_tx`

Performs *expect-tx* operation on the *mbarrier object*.

Syntax

```
mbarrier.expect_tx{.sem.scope}{.space}.b64 [addr], txCount;  
  
.sem    = { .relaxed }  
.scope  = { .cta, .cluster }  
.space  = { .shared{::cta}, .shared::cluster }
```

Description

A thread executing `mbarrier.expect_tx` performs an *expect-tx* operation on the *mbarrier object* at the location specified by the address operand `addr`. The 32-bit unsigned integer operand `txCount` specifies the `expectCount` argument to the *expect-tx* operation.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` or `.shared::cluster` state space then the behavior is undefined.

Supported addressing modes for operand `addr` are as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

The optional `.sem` qualifier specifies a memory synchronizing effect as described in the *Memory Consistency Model*. The `.relaxed` qualifier does not provide any memory ordering semantics and visibility guarantees.

The optional `.scope` qualifier indicates the set of threads that directly observe the memory synchronizing effect of this operation, as described in the *Memory Consistency Model*.

Qualifiers `.sem` and `.scope` must be specified together.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
mbarrier.expect_tx.b64          [addr], 32;
mbarrier.expect_tx.relaxed.cta.shared.b64 [mbarObj1], 512;
mbarrier.expect_tx.relaxed.cta.shared.b64 [mbarObj2], 512;
```

9.7.13.15.12 Parallel Synchronization and Communication Instructions: `mbarrier.complete_tx`

`mbarrier.complete_tx`

Performs *complete-tx* operation on the *mbarrier object*.

Syntax

```
mbarrier.complete_tx{.sem.scope}{.space}.b64 [addr], txCount;

.sem    = { .relaxed }
.scope  = { .cta, .cluster }
.space  = { .shared{:cta}, .shared::cluster }
```

Description

A thread executing `mbarrier.complete_tx` performs a *complete-tx* operation on the *mbarrier object* at the location specified by the address operand `addr`. The 32-bit unsigned integer operand `txCount` specifies the `completeCount` argument to the *complete-tx* operation.

`mbarrier.complete_tx` does not involve any asynchronous memory operations and only simulates the completion of an asynchronous memory operation and its side effect of signaling to the *mbarrier object*.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` or `.shared::cluster` state space then the behavior is undefined.

Supported addressing modes for operand `addr` are as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

The optional `.sem` qualifier specifies a memory synchronizing effect as described in the *Memory Consistency Model*. The `.relaxed` qualifier does not provide any memory ordering semantics and visibility guarantees.

The optional `.scope` qualifier indicates the set of threads that directly observe the memory synchronizing effect of this operation, as described in the [Memory Consistency Model](#).

Qualifiers `.sem` and `.scope` must be specified together.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
mbarrier.complete_tx.b64      [addr],      32;
mbarrier.complete_tx.shared.b64 [mbarObj1], 512;
mbarrier.complete_tx.relaxed.cta.b64 [addr2], 32;
```

9.7.13.15.13 Parallel Synchronization and Communication Instructions: `mbarrier.arrive`

`mbarrier.arrive`

Performs *arrive-on operation* on the *mbarrier object*.

Syntax

```
mbarrier.arrive{.sem.scope}{.shared{::cta}}.b64      state, [addr]{, count};
mbarrier.arrive{.sem.scope}{.shared::cluster}.b64    _, [addr] {,count}
mbarrier.arrive.expect_tx{.sem.scope}{.shared{::cta}}.b64 state, [addr], txCount;
mbarrier.arrive.expect_tx{.sem.scope}{.shared::cluster}.b64  _, [addr], txCount;
mbarrier.arrive.noComplete{.release.cta}{.shared{::cta}}.b64 state, [addr], count;

.sem    = { .release, .relaxed }
.scope  = { .cta, .cluster }
```

Description

A thread executing `mbarrier.arrive` performs an *arrive-on* operation on the *mbarrier object* at the location specified by the address operand `addr`. The 32-bit unsigned integer operand `count` specifies the *count* argument to the *arrive-on* operation.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` state space then the behavior is undefined.

Supported addressing modes for operand `addr` is as described in [Addresses as Operands](#). Alignment for operand `addr` is as described in the [Size and alignment of mbarrier object](#).

The optional qualifier `.expect_tx` specifies that an *expect-tx* operation is performed prior to the *arrive-on* operation. The 32-bit unsigned integer operand `txCount` specifies the *expectCount* argument to the *expect-tx* operation. When both qualifiers `.arrive` and `.expect_tx` are specified, then the count argument of the *arrive-on* operation is assumed to be 1.

A `mbarrier.arrive` operation with `.noComplete` qualifier must not cause the `mbarrier` to complete its current phase, otherwise the behavior is undefined.

The value of the operand count must be in the range as specified in *Contents of the mbarrier object*.

Note: for `sm_8x`, when the argument count is specified, the modifier `.noComplete` is required.

`mbarrier.arrive` operation on an *mbarrier object* located in `.shared::cta` returns an opaque 64-bit register capturing the phase of the *mbarrier object* prior to the *arrive-on operation* in the destination operand state. Contents of the state operand are implementation specific. Optionally, sink symbol `'_'` can be used for the state argument.

`mbarrier.arrive` operation on an *mbarrier object* located in `.shared::cluster` but not in `.shared::cta` cannot return a value. Sink symbol `'_'` is mandatory for the destination operand for such cases.

The optional `.sem` qualifier specifies a memory synchronizing effect as described in the *Memory Consistency Model*. If the `.sem` qualifier is absent, `.release` is assumed by default.

The `.relaxed` qualifier does not provide any memory ordering semantics and visibility guarantees.

The optional `.scope` qualifier indicates the set of threads that directly observe the memory synchronizing effect of this operation, as described in the *Memory Consistency Model*. If the `.scope` qualifier is not specified then it defaults to `.cta`. In contrast, the `.shared::<scope>` indicates the state space where the `mbarrier` resides.

Qualifiers `.sem` and `.scope` must be specified together.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for sink symbol `'_'` as the destination operand is introduced in PTX ISA version 7.1.

Support for sub-qualifier `::cta` on `.shared` introduced in PTX ISA version 7.8.

Support for count argument without the modifier `.noComplete` introduced in PTX ISA version 7.8.

Support for sub-qualifier `::cluster` introduced in PTX ISA version 8.0.

Support for qualifier `.expect_tx` is introduced in PTX ISA version 8.0.

Support for `.scope` and `.sem` qualifiers introduced in PTX ISA version 8.0

Support for `.relaxed` qualifier introduced in PTX ISA version 8.6.

Target ISA Notes

Requires sm_80 or higher.

Support for count argument without the modifier `.noComplete` requires sm_90 or higher.

Qualifier `.expect_tx` requires sm_90 or higher.

Sub-qualifier `::cluster` requires sm_90 or higher.

Support for `.cluster` scope requires sm_90 or higher.

Support for `.relaxed` qualifier requires sm_90 or higher.

Examples

```
.reg .b32 cnt, remoteAddr32, remoteCTAId, addr32;
.reg .b64 %r<5>, addr, remoteAddr64;
.shared .b64 shMem, shMem2;

cvta.shared.u64      addr, shMem2;
mov.b32             addr32, shMem2;
mapa.shared::cluster.u32 remoteAddr32, addr32, remoteCTAId;
mapa.u64            remoteAddr64, addr, remoteCTAId;

cvta.shared.u64      addr, shMem2;

mbarrier.arrive.shared.b64      %r0, [shMem];
mbarrier.arrive.shared::cta.b64 %r0, [shMem2];
mbarrier.arrive.release.cta.shared::cluster.b64 _, [remoteAddr32];
mbarrier.arrive.release.cluster.b64 _, [remoteAddr64], cnt;
mbarrier.arrive.expect_tx.release.cluster.b64 _, [remoteAddr64], tx_count;
mbarrier.arrive.noComplete.b64 %r1, [addr], 2;
mbarrier.arrive.relaxed.cta.b64 %r2, [addr], 4;
mbarrier.arrive.b64 %r2, [addr], cnt;
```

9.7.13.15.14 Parallel Synchronization and Communication Instructions: `mbarrier.arrive_drop`

`mbarrier.arrive_drop`

Decrements the expected count of the *mbarrier object* and performs *arrive-on operation*.

Syntax

```
mbarrier.arrive_drop{.sem.scope}{.shared{::cta}}.b64 state, [addr]{, count};
mbarrier.arrive_drop{.sem.scope}{.shared::cluster}.b64 _, [addr]{, count};
mbarrier.arrive_drop.expect_tx{.sem.scope}{.shared{::cta}}.b64 state, [addr], tx_
↳count;
mbarrier.arrive_drop.expect_tx{.sem.scope}{.shared::cluster}.b64 _, [addr], tx_
↳count;
mbarrier.arrive_drop.noComplete{.release.cta}{.shared{::cta}}.b64 state, [addr],
↳count;
```

(continues on next page)

(continued from previous page)

```
.sem    = { .release, .relaxed }
.scope = { .cta, .cluster }
```

Description

A thread executing `mbarrier.arrive_drop` on the *mbarrier object* at the location specified by the address operand `addr` performs the following steps:

- ▶ Decrements the expected arrival count of the *mbarrier object* by the value specified by the 32-bit integer operand `count`. If `count` operand is not specified, it defaults to 1.
- ▶ Performs an *arrive-on operation* on the *mbarrier object*. The operand `count` specifies the *count* argument to the *arrive-on operation*.

The decrement done in the expected arrivals count of the *mbarrier object* will be for all the subsequent phases of the *mbarrier object*.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` or `.shared::cluster` state space then the behavior is undefined.

Supported addressing modes for operand `addr` is as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

The optional qualifier `.expect_tx` specifies that an *expect-tx* operation is performed prior to the *arrive_drop* operation, i.e. the decrement of arrival count and *arrive-on* operation. The 32-bit unsigned integer operand `txCount` specifies the *expectCount* argument to the *expect-tx* operation. When both qualifiers `.arrive_drop` and `.expect_tx` are specified, then the `count` argument of the *arrive-on* operation is assumed to be 1.

`mbarrier.arrive_drop` operation with `.release` qualifier forms the *release* pattern as described in the Memory Consistency Model and synchronizes with the *acquire* patterns.

The optional `.sem` qualifier specifies a memory synchronizing effect as described in the *Memory Consistency Model*. If the `.sem` qualifier is absent, `.release` is assumed by default. The `.relaxed` qualifier does not provide any memory ordering semantics and visibility guarantees.

The optional `.scope` qualifier indicates the set of threads that an `mbarrier.arrive_drop` instruction can directly synchronize. If the `.scope` qualifier is not specified then it defaults to `.cta`. In contrast, the `.shared::<scope>` indicates the state space where the *mbarrier* resides.

A `mbarrier.arrive_drop` with `.noComplete` qualifier must not complete the *mbarrier*, otherwise the behavior is undefined.

The value of the operand `count` must be in the range as specified in *Contents of the mbarrier object*.

Note: for `sm_8x`, when the argument `count` is specified, the modifier `.noComplete` is required.

A thread that wants to either exit or opt out of participating in the *arrive-on operation* can use `mbarrier.arrive_drop` to drop itself from the *mbarrier*.

`mbarrier.arrive_drop` operation on an *mbarrier object* located in `.shared::cta` returns an opaque 64-bit register capturing the phase of the *mbarrier object* prior to the *arrive-on operation* in the destination operand state. Contents of the returned state are implementation specific. Optionally, sink symbol `'_'` can be used for the state argument.

`mbarrier.arrive_drop` operation on an *mbarrier object* located in `.shared::cluster` but not in `.shared::cta` cannot return a value. Sink symbol `'_'` is mandatory for the destination operand for such cases.

Qualifiers `.sem` and `.scope` must be specified together.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for sub-qualifier `::cta` on `.shared` introduced in PTX ISA version 7.8.

Support for count argument without the modifier `.noComplete` introduced in PTX ISA version 7.8.

Support for qualifier `.expect_tx` is introduced in PTX ISA version 8.0.

Support for sub-qualifier `::cluster` introduced in PTX ISA version 8.0.

Support for `.scope` and `.sem` qualifiers introduced in PTX ISA version 8.0

Support for `.relaxed` qualifier introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_80` or higher.

Support for count argument without the modifier `.noComplete` requires `sm_90` or higher.

Qualifier `.expect_tx` requires `sm_90` or higher.

Sub-qualifier `::cluster` requires `sm_90` or higher.

Support for `.cluster` scope requires `sm_90` or higher.

Support for `.relaxed` qualifier requires `sm_90` or higher.

Examples

```
.reg .b32 cnt;
.reg .b64 %r1;
.shared .b64 shMem;

// Example 1
@p mbarrier.arrive_drop.shared.b64 _, [shMem];
@p exit;
@p2 mbarrier.arrive_drop.noComplete.shared.b64 _, [shMem], %a;
@p2 exit;
..
@!p mbarrier.arrive.shared.b64 %r1, [shMem];
@!p mbarrier.test_wait.shared.b64 q, [shMem], %r1;

// Example 2
mbarrier.arrive_drop.shared::cluster.b64 _, [addr];
mbarrier.arrive_drop.shared::cta.release.cluster.b64 _, [addr], cnt;

// Example 3
mbarrier.arrive_drop.expect_tx.shared::cta.relaxed.cluster.b64 state, [addr], tx_
↪count;
```


9.7.13.15.15 Parallel Synchronization and Communication Instructions: `cp.async.mbarrier.arrive`

`cp.async.mbarrier.arrive`

Makes the *mbarrier object* track all prior `cp.async` operations initiated by the executing thread.

Syntax

```
cp.async.mbarrier.arrive{.noinc}{.shared{:cta}}.b64 [addr];
```

Description

Causes an *arrive-on operation* to be triggered by the system on the *mbarrier object* upon the completion of all prior `cp.async` operations initiated by the executing thread. The *mbarrier object* is at the location specified by the operand `addr`. The *arrive-on operation* is asynchronous to execution of `cp.async.mbarrier.arrive`.

When `.noinc` modifier is not specified, the pending count of the *mbarrier object* is incremented by 1 prior to the asynchronous *arrive-on operation*. This results in a zero-net change for the pending count from the asynchronous *arrive-on operation* during the current phase. The pending count of the *mbarrier object* after the increment should not exceed the limit as mentioned in *Contents of the mbarrier object*. Otherwise, the behavior is undefined.

When the `.noinc` modifier is specified, the increment to the pending count of the *mbarrier object* is not performed. Hence the decrement of the pending count done by the asynchronous *arrive-on operation* must be accounted for in the initialization of the *mbarrier object*.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared{:cta}` state space then the behavior is undefined.

Supported addressing modes for operand `addr` is as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Support for sub-qualifier `:cta` on `.shared` introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
// Example 1: no .noinc
mbarrier.init.shared.b64 [shMem], threadCount;
....
cp.async.ca.shared.global [shard1], [gb11], 4;
cp.async.cg.shared.global [shard2], [gb12], 16;
....
// Absence of .noinc accounts for arrive-on from completion of prior cp.async
↳operations.
// So mbarrier.init must only account for arrive-on from mbarrier.arrive.
cp.async.mbarrier.arrive.shared.b64 [shMem];
....
mbarrier.arrive.shared.b64 state, [shMem];

waitLoop:
mbarrier.test_wait.shared.b64 p, [shMem], state;
@!p bra waitLoop;

// Example 2: with .noinc

// Tracks arrive-on from mbarrier.arrive and cp.async.mbarrier.arrive.

// All threads participating in the mbarrier perform cp.async
mov.b32 copyOperationCnt, threadCount;

// 3 arrive-on operations will be triggered per-thread
mul.lo.u32 copyArrivalCnt, copyOperationCnt, 3;

add.u32 totalCount, threadCount, copyArrivalCnt;

mbarrier.init.shared.b64 [shMem], totalCount;
....
cp.async.ca.shared.global [shard1], [gb11], 4;
cp.async.cg.shared.global [shard2], [gb12], 16;
...
// Presence of .noinc requires mbarrier initialization to have accounted for arrive-on
↳from cp.async
cp.async.mbarrier.arrive.noinc.shared.b64 [shMem]; // 1st instance
....
cp.async.ca.shared.global [shard3], [gb13], 4;
cp.async.ca.shared.global [shard4], [gb14], 16;
cp.async.mbarrier.arrive.noinc.shared.cta.b64 [shMem]; // 2nd instance
....
cp.async.ca.shared.global [shard5], [gb15], 4;
cp.async.cg.shared.global [shard6], [gb16], 16;
cp.async.mbarrier.arrive.noinc.shared.b64 [shMem]; // 3rd and last instance
....
mbarrier.arrive.shared.b64 state, [shMem];

waitLoop:
mbarrier.test_wait.shared.b64 p, [shMem], state;
@!p bra waitLoop;
```

9.7.13.15.16 Parallel Synchronization and Communication Instructions: `mbarrier.test_wait` / `mbarrier.try_wait`

`mbarrier.test_wait`, `mbarrier.try_wait`

Checks whether the *mbarrier object* has completed the phase.

Syntax

```
mbarrier.test_wait{.sem.scope}{.shared{::cta}}.b64      waitComplete, [addr], state;
mbarrier.test_wait.parity{.sem.scope}{.shared{::cta}}.b64 waitComplete, [addr],
↳phaseParity;

mbarrier.try_wait{.sem.scope}{.shared{::cta}}.b64      waitComplete, [addr], state
                                                         {, suspendTimeHint};

mbarrier.try_wait.parity{.sem.scope}{.shared{::cta}}.b64 waitComplete, [addr],
↳phaseParity
                                                         {, suspendTimeHint};

.sem    = { .acquire, .relaxed }
.scope  = { .cta, .cluster }
```

Description

The *test_wait* and *try_wait* operations test for the completion of the current or the immediately preceding phase of an *mbarrier object* at the location specified by the operand *addr*.

`mbarrier.test_wait` is a non-blocking instruction which tests for the completion of the phase.

`mbarrier.try_wait` is a potentially blocking instruction which tests for the completion of the phase. If the phase is not complete, the executing thread may be suspended. Suspended thread resumes execution when the specified phase completes OR before the phase completes following a system-dependent time limit. The optional 32-bit unsigned integer operand *suspendTimeHint* specifies the time limit, in nanoseconds, that may be used for the time limit instead of the system-dependent limit.

`mbarrier.test_wait` and `mbarrier.try_wait` test for completion of the phase :

- Specified by the 64-bit unsigned integer operand *state*, which was returned by an `mbarrier.arrive` instruction on the same *mbarrier object* during the current or the immediately preceding phase. Or
- Indicated by the 32-bit unsigned integer operand *phaseParity*, which is the integer parity of either the current phase or the immediately preceding phase of the *mbarrier object*.

The *.parity* variant of the instructions test for the completion of the phase indicated by the operand *phaseParity*, which is the integer parity of either the current phase or the immediately preceding phase of the *mbarrier object*. An even phase has integer parity 0 and an odd phase has integer parity of 1. So the valid values of *phaseParity* operand are 0 and 1.

Note: the use of the *.parity* variants of the instructions requires tracking the phase of an *mbarrier object* throughout its lifetime.

The *test_wait* and *try_wait* operations are valid only for :

- the current incomplete phase, for which *waitComplete* returns *False*.

- the immediately preceding phase, for which `waitComplete` returns `True`.

If no state space is specified then *Generic Addressing* is used. If the address specified by `addr` does not fall within the address window of `.shared::cta` state space then the behavior is undefined.

Supported addressing modes for operand `addr` is as described in *Addresses as Operands*. Alignment for operand `addr` is as described in the *Size and alignment of mbarrier object*.

When `mbarrier.test_wait` and `mbarrier.try_wait` operations with `.acquire` qualifier returns `True`, they form the *acquire* pattern as described in the *Memory Consistency Model*.

The optional `.sem` qualifier specifies a memory synchronizing effect as described in the *Memory Consistency Model*. If the `.sem` qualifier is absent, `.acquire` is assumed by default. The `.relaxed` qualifier does not provide any memory ordering semantics and visibility guarantees.

The optional `.scope` qualifier indicates the set of threads that the `mbarrier.test_wait` and `mbarrier.try_wait` instructions can directly synchronize. If the `.scope` qualifier is not specified then it defaults to `.cta`. In contrast, the `.shared::<scope>` indicates the state space where the `mbarrier` resides.

Qualifiers `.sem` and `.scope` must be specified together.

The following ordering of memory operations hold for the executing thread when `mbarrier.test_wait` or `mbarrier.try_wait` having *acquire* semantics returns `True`:

1. All memory accesses (except *async operations*) requested prior, in program order, to `mbarrier.arrive` having release semantics during the completed phase by the participating threads of the CTA are performed and are visible to the executing thread.
2. All *cp.async* operations requested prior, in program order, to `cp.async.mbarrier.arrive` during the completed phase by the participating threads of the CTA are performed and made visible to the executing thread.
3. All `cp.async.bulk` asynchronous operations using the same *mbarrier object* requested prior, in program order, to `mbarrier.arrive` having release semantics during the completed phase by the participating threads of the CTA are performed and made visible to the executing thread.
4. All memory accesses requested after the `mbarrier.test_wait` or `mbarrier.try_wait`, in program order, are not performed and not visible to memory accesses performed prior to `mbarrier.arrive` having release semantics, in program order, by other threads participating in the `mbarrier`.
5. There is no ordering and visibility guarantee for memory accesses requested by the thread after `mbarrier.arrive` having release semantics and prior to `mbarrier.test_wait`, in program order.

PTX ISA Notes

`mbarrier.test_wait` introduced in PTX ISA version 7.0.

Modifier `.parity` is introduced in PTX ISA version 7.1.

`mbarrier.try_wait` introduced in PTX ISA version 7.8.

Support for sub-qualifier `::cta` on `.shared` introduced in PTX ISA version 7.8.

Support for `.scope` and `.sem` qualifiers introduced in PTX ISA version 8.0

Support for `.relaxed` qualifier introduced in PTX ISA version 8.6.

Target ISA Notes

`mbarrier.test_wait` requires `sm_80` or higher.
`mbarrier.try_wait` requires `sm_90` or higher.
Support for `.cluster` scope requires `sm_90` or higher.
Support for `.relaxed` qualifier requires `sm_90` or higher.

Examples

```
// Example 1a, thread synchronization with test_wait:

.reg .b64 %r1;
.shared .b64 shMem;

mbarrier.init.shared.b64 [shMem], N; // N threads participating in the mbarrier.
...
mbarrier.arrive.shared.b64 %r1, [shMem]; // N threads executing mbarrier.arrive

// computation not requiring mbarrier synchronization...

waitLoop:
mbarrier.test_wait.shared.b64    complete, [shMem], %r1;
@!complete nanosleep.u32 20;
@!complete bra waitLoop;

// Example 1b, thread synchronization with try_wait :

.reg .b64 %r1;
.shared .b64 shMem;

mbarrier.init.shared.b64 [shMem], N; // N threads participating in the mbarrier.
...
mbarrier.arrive.shared.b64 %r1, [shMem]; // N threads executing mbarrier.arrive

// computation not requiring mbarrier synchronization...

waitLoop:
mbarrier.try_wait.relaxed.cluster.shared.b64    complete, [shMem], %r1;
@!complete bra waitLoop;

// Example 2, thread synchronization using phase parity :

.reg .b32 i, parArg;
.reg .b64 %r1;
.shared .b64 shMem;

mov.b32 i, 0;
mbarrier.init.shared.b64 [shMem], N; // N threads participating in the mbarrier.
...
loopStart :                                // One phase per loop iteration
...
    mbarrier.arrive.shared.b64 %r1, [shMem]; // N threads
...
```

(continues on next page)

(continued from previous page)

```

    and.b32 parArg, i, 1;
    waitLoop:
    mbarrier.test_wait.parity.shared.b64 complete, [shMem], parArg;
    @!complete nanosleep.u32 20;
    @!complete bra waitLoop;
    ...
    add.u32 i, i, 1;
    setp.lt.u32 p, i, IterMax;
    @p bra loopStart;

// Example 3, Asynchronous copy completion waiting :

.reg .b64 state;
.shared .b64 shMem2;
.shared .b64 shard1, shard2;
.global .b64 gbl1, gbl2;

mbarrier.init.shared.b64 [shMem2], threadCount;
...
cp.async.ca.shared.global [shard1], [gbl1], 4;
cp.async.cg.shared.global [shard2], [gbl2], 16;

// Absence of .noinc accounts for arrive-on from prior cp.async operation
cp.async.mbarrier.arrive.shared.b64 [shMem2];
...
mbarrier.arrive.shared.b64 state, [shMem2];

waitLoop:
mbarrier.test_wait.shared::cta.b64 p, [shMem2], state;
@!p bra waitLoop;

// Example 4, Synchronizing the CTA0 threads with cluster threads
.reg .b64 %r1, addr, remAddr;
.shared .b64 shMem;

cvta.shared.u64          addr, shMem;
mapa.u64                remAddr, addr, 0;    // CTA0's shMem instance

// One thread from CTA0 executing the below initialization operation
@p0 mbarrier.init.shared::cta.b64 [shMem], N; // N = no of cluster threads

barrier.cluster.arrive;
barrier.cluster.wait;

// Entire cluster executing the below arrive operation
mbarrier.arrive.release.cluster.b64          _, [remAddr];

// computation not requiring mbarrier synchronization ...

// Only CTA0 threads executing the below wait operation
waitLoop:
mbarrier.try_wait.parity.acquire.cluster.shared::cta.b64 complete, [shMem], 0;
@!complete bra waitLoop;

```

9.7.13.15.17 Parallel Synchronization and Communication Instructions: `mbarrier.pending_count`

`mbarrier.pending_count`

Query the pending arrival count from the opaque `mbarrier` state.

Syntax

```
mbarrier.pending_count.b64 count, state;
```

Description

The pending count can be queried from the opaque `mbarrier` state using `mbarrier.pending_count`.

The state operand is a 64-bit register that must be the result of a prior `mbarrier.arrive.noComplete` or `mbarrier.arrive_drop.noComplete` instruction. Otherwise, the behavior is undefined.

The destination register count is a 32-bit unsigned integer representing the pending count of the *mbarrier object* prior to the *arrive-on operation* from which the state register was obtained.

PTX ISA Notes

Introduced in PTX ISA version 7.0.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
.reg .b32 %r1;
.reg .b64 state;
.shared .b64 shMem;

mbarrier.arrive.noComplete.b64 state, [shMem], 1;
mbarrier.pending_count.b64 %r1, state;
```

9.7.13.16 Parallel Synchronization and Communication Instructions: `tensormap.cp_fenceproxy`

`tensormap.cp_fenceproxy`

A fused copy and fence operation.

Syntax

```
tensormap.cp_fenceproxy.cp_qualifiers.fence_qualifiers.sync.aligned [dst], [src],
↪ size;

.cp_qualifiers      = { .global.shared::cta }
.fence_qualifiers = { .to_proxy::from_proxy.release.scope }
.to_proxy::from_proxy = { .tensormap::generic }
.scope              = { .cta, .cluster, .gpu , .sys }
```

Description

The `tensormap.cp_fenceproxy` instructions perform the following operations in order :

- ▶ Copies data of size specified by the `size` argument, in bytes, from the location specified by the address operand `src` in shared memory to the location specified by the address operand `dst` in the global memory, in the generic proxy.
- ▶ Establishes a *uni-directional* proxy release pattern on the ordering from the copy operation to the subsequent access performed in the `tensormap.proxy` on the address `dst`.

The valid value of immediate operand `size` is 128.

The operands `src` and `dst` specify non-generic addresses in `shared::cta` and `global` state space respectively.

The `.scope` qualifier specifies the set of threads that can directly observe the proxy synchronizing effect of this operation, as described in [Memory Consistency Model](#).

The mandatory `.sync` qualifier indicates that `tensormap.cp_fenceproxy` causes the executing thread to wait until all threads in the warp execute the same `tensormap.cp_fenceproxy` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tensormap.cp_fenceproxy` instruction. In conditionally executed code, an aligned `tensormap.cp_fenceproxy` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.3.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
// Example: manipulate a tensor-map object and then consume it in cp.async.bulk.tensor
.reg .b64 new_addr;
.global .align 128 .b8 gbl[128];
.shared .align 128 .b8 sMem[128];

cp.async.bulk.shared::cluster.global.mbarrier::complete_tx::bytes [sMem], [gMem], 128,
↪ [mbar];
...
try_wait_loop:
mbarrier.try_wait.shared.b64 p, [mbar], state;
@!p bra try_wait loop;

tensormap.replace.tile.global_address.shared.b1024.b64 [sMem], new_addr;
tensormap.cp_fenceproxy.global.shared::cta.tensormap::generic.release.gpu.sync.aligned
[gbl], [sMem], 128;
fence.proxy.tensormap::generic.acquire.gpu [gbl], 128;
cp.async.bulk.tensor.1d.shared::cluster.global.tile [addr0], [gbl, {tc0}], [mbar0];
```

9.7.13.17 Parallel Synchronization and Communication Instructions: `clusterlaunchcontrol.try_cancel`

`clusterlaunchcontrol.try_cancel`

Requests cancellation of cluster which is not launched yet.

Syntax

```
clusterlaunchcontrol.try_cancel.async{.space}.completion_mechanism{.
↪multicast::cluster::all}.b128 [addr], [mbar];

.completion_mechanism = { .mbarrier::complete_tx::bytes };
.space = { .shared::cta };
```

Description

The `clusterlaunchcontrol.try_cancel` instruction requests atomically cancelling the launch of a cluster that has not started running yet. It asynchronously writes an opaque response to shared memory indicating whether the operation succeeded or failed. The completion of the asynchronous operation is tracked using the mbarrier completion mechanism at `.cluster` scope. This instruction accesses its mbarrier operand using generic-proxy.

On success, the opaque response contains the `ctaid` of the first CTA of the canceled cluster; no other successful response from other `clusterlaunchcontrol.try_cancel` operations from the same grid will contain that id.

The mandatory `.async` qualifier indicates that the instruction will initiate the cancellation operation asynchronously and control will return to the executing thread before the requested operation is complete.

The `.space` qualifier is specified, both operands `addr` and `mbar` must be in the `.shared::cta` state space. Otherwise, generic addressing will be assumed for both. The result is undefined if any of address operands do not fall within the address window of `.shared::cta`.

The qualifier `.completion_mechanism` specifies that upon completion of the asynchronous operation, *complete-tx* operation, with `completeCount` argument equal to amount of data stored in bytes, will be performed on the mbarrier object specified by the operand `mbar`.

The executing thread can then use *mbarrier instructions* to wait for completion of the asynchronous operation. No other synchronization mechanisms described in *Memory Consistency Model* can be used to guarantee the completion of the asynchronous copy operations.

The `.multicast::cluster::all` qualifier indicates that the response is asynchronously written using weak async-proxy writes to the corresponding local shared memory `addr` of each CTA in the requesting cluster. The completion of the writes to `addr` of a particular CTA is signaled via a *complete-tx* operation to the mbarrier object on the shared memory of that CTA.

The behavior of instruction with `.multicast::cluster::all` qualifier is undefined if any CTA in the cluster is exited.

Operand `addr` specifies the naturally aligned address of the 16-byte wide shared memory location where the request's response is written.

The response of `clusterlaunchcontrol.try_cancel` instruction will be 16-byte opaque value and will be it available at location specified by operand `addr`. After loading this response into 16-byte register, instruction `clusterlaunchcontrol.query_cancel` can be used to check if request was successful and to retrieve `ctaid` of the first CTA of the canceled cluster.

If the executing CTA has already observed the completion of a `clusterlaunchcontrol.try_cancel` instruction as failed, then the behavior of issuing a subsequent `clusterlaunchcontrol.try_cancel` instruction is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_100` or higher.

Qualifier `.multicast::cluster::all` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family

- sm_110f or higher in the same family

Examples

```
// Assumption: 1D cluster (cluster_ctaid.y/.z == 1) with 1 thread per CTA.

// Current Cluster to be processed: initially the launched cluster:
mov.b32 xctaid, %ctaid.x;

// Establish full synchronization across all CTAs of the cluster for the first
↳ iteration.
// Weaker synchronization may suffice depending on initialization sequence.
barrier.cluster.arrive;
barrier.cluster.wait;

// Iteration loop over all cluster CTAs
processCluster:
    mov.u32 %r0, %tid.x;
    setp.u32.eq p0, %r0, 0x0;
    // Elect a leader thread (thread idx 0) for each CTA to arrive and expect_tx at
    // each CTA local shared memory barrier:
    mov.u32 %r0, %tid.x;
    setp.u32.eq p0, %r0, 0x0;
    // All other threads skip to processing the work of the current cluster:
    @!p0 bra processCurrentCluster;

    // All CTAs in the cluster arrive at their local SMEM barrier and set 16B handle tx
    ↳ count:
    mbarrier.arrive.expect_tx.cluster.relaxed.shared::cta.b64 state, [mbar], 16;

    // First CTA in Cluster attempts to cancel a not-yet-started cluster:
    mov.u32 %r0, %cluster_ctaid.x;
    setp.u32.eq p0, %r0, 0x0;
    @p0 clusterlaunchcontrol.try_cancel.async.mbarrier::complete_tx::bytes.
    ↳ multicast::cluster::all.b128 [addr], [mbar];

    processCurrentCluster:
        // ...process current cluster ("xctaid") while cancellation request for next
        ↳ cluster runs asynchronously...

        // After processing current cluster, wait on cancellation request response for next
        ↳ cluster via specified mbarrier:
        waitLoop:
            // .acquire prevents weak handle read ("ld.shared handle, [addr]") from
            ↳ overtaking this mbarrier.try_wait:
            mbarrier.try_wait.cluster.acquire.shared::cta.b64 complete, [mbar], state;
            @!complete bra waitLoop;
            // Cancellation request has completed.

        // Generic-proxy weak read of cancellation request into 16-byte wide register:
        ld.shared.b128 handle, [addr];

        // Check whether cancellation succeeded:
        clusterlaunchcontrol.query_cancel.is_canceled.pred.b128 p, handle;
        // If cancellation request failed, we couldn't cancel any other cluster, so all
        ↳ current cluster CTAs exit.
```

(continues on next page)

(continued from previous page)

```

@!p ret;

// Otherwise, cancellation request succeeded.
// Extract "ctaid" of first cancelled-cluster CTA which we'll process in next
↪ "processCluster" loop iteration:
@p clusterlaunchcontrol.query_cancel.get_first_ctaid.v4.b32.b128 {xctaid, _, _, _},
↪ handle;

// Release current iteration generic-proxy weak read of handle ("ld.shared handle,
↪ [addr]")
// before next iteration async-proxy write to handle ("clusterlaunchcontrol.try_
↪ cancel [addr]")
fence.proxy.async::generic.release.sync_restrict::shared::cta.cluster;

// Arrive and wait at the next iteration cluster barrier with relaxed semantics.
barrier.cluster.arrive.relaxed;
barrier.cluster.wait;

// Acquire prior iteration generic-proxy weak read of handle ("ld.shared handle,
↪ [addr]")
// before current iteration async-proxy write to handle ("clusterlaunchcontrol.try_
↪ cancel [addr]")
fence.proxy.async::generic.acquire.sync_restrict::shared::cluster.cluster;

bra processCluster;

```

9.7.13.18 Parallel Synchronization and Communication Instructions: `clusterlaunchcontrol.query_cancel`

`clusterlaunchcontrol.query_cancel`

Queries response of `clusterlaunchcontrol.try_cancel` operation.

Syntax

```

clusterlaunchcontrol.query_cancel.is_canceled.pred.b128 pred, try_cancel_response;

clusterlaunchcontrol.query_cancel.get_first_ctaid.v4.b32.b128 {xdim, ydim, zdim, _},
↪ try_cancel_response;

clusterlaunchcontrol.query_cancel.get_first_ctaid{::dimension}.b32.b128 reg, try_
↪ cancel_response;

::dimension = { ::x, ::y, ::z };

```

Description

Instruction `clusterlaunchcontrol.query_cancel` can be used to decode opaque response written by instruction `clusterlaunchcontrol.try_cancel`.

After loading response from `clusterlaunchcontrol.try_cancel` instruction into 16-byte register it can be further queried using `clusterlaunchcontrol.query_cancel` instruction as follows:

`clusterlaunchcontrol.query_cancel.is_canceled.pred.b128`: If the cluster is canceled successfully, predicate `p` is set to true; otherwise, it is set to false.

If the request succeeded, the instruction `clusterlaunchcontrol.query_cancel.get_first_ctaid` extracts the CTA id of the first CTA in the canceled cluster. By default, the instruction returns a `.v4` vector whose first three elements are the `x`, `y` and `z` coordinate of first CTA in canceled cluster. The contents of the 4th element are unspecified. The explicit `.get_first_ctaid::x`, `.get_first_ctaid::y`, or `.get_first_ctaid::z` qualifiers can be used to extract individual `x`, `y` or `z` coordinates into a 32-bit register.

If the request fails the behavior of `clusterlaunchcontrol.query_cancel.get_first_ctaid` is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_100` or higher.

Examples

```
clusterlaunchcontrol.query_cancel.is_canceled pred.b128 p, handle;

@p clusterlaunchcontrol.query_cancel.get_first_ctaid.v4.b32.b128 {xdim, ydim, zdim,
    ↪ignr} handle;

clusterlaunchcontrol.query_cancel.get_first_ctaid::x.b32.b128 reg0, handle;
clusterlaunchcontrol.query_cancel.get_first_ctaid::y.b32.b128 reg1, handle;
clusterlaunchcontrol.query_cancel.get_first_ctaid::z.b32.b128 reg2, handle;
```

9.7.14. Warp Level Matrix Multiply-Accumulate Instructions

The matrix multiply and accumulate operation has the following form:

$$D = A * B + C$$

where D and C are called accumulators and may refer to the same matrix.

PTX provides two ways to perform matrix multiply-and-accumulate computation:

- ▶ Using `wmma` instructions:
 - ▶ This warp-level computation is performed collectively by all threads in the warp as follows:
 - ▶ Load matrices A, B and C from memory into registers using the `wmma.load` operation. When the operation completes, the destination registers in each thread hold a fragment of the loaded matrix.
 - ▶ Perform the matrix multiply and accumulate operation using the `wmma.mma` operation on the loaded matrices. When the operation completes, the destination registers in each thread hold a fragment of the result matrix returned by the `wmma.mma` operation.
 - ▶ Store result Matrix D back to memory using the `wmma.store` operation. Alternately, result matrix D can also be used as argument C for a subsequent `wmma.mma` operation.

The `wmma.load` and `wmma.store` instructions implicitly handle the organization of matrix elements when loading the input matrices from memory for the `wmma.mma` operation and when storing the result back to memory.

- ▶ Using `mma` instruction:
 - ▶ Similar to `wmma`, `mma` also requires computation to be performed collectively by all threads in the warp however distribution of matrix elements across different threads in warp needs to be done explicitly before invoking the `mma` operation. The `mma` instruction supports both dense as well as sparse matrix A. The sparse variant can be used when A is a structured sparse matrix as described in [Sparse matrix storage](#).

9.7.14.1 Matrix Shape

The matrix multiply and accumulate operations support a limited set of shapes for the operand matrices A, B and C. The shapes of all three matrix operands are collectively described by the tuple $M \times N \times K$, where A is an $M \times K$ matrix, B is a $K \times N$ matrix, while C and D are $M \times N$ matrices.

The following matrix shapes are supported for the specified types:

In-struction	Scale	Sparsity	Multiplicand Data-type	Shape	PTX ISA version
<code>wmma</code>	NA	Dense	Floating-point - <code>.f16</code>	<code>.m16n16k16</code> , <code>.m8n32k16</code> , and <code>.m32n8k16</code>	PTX ISA version 6.0
<code>wmma</code>		Dense	Alternate floating-point format - <code>.bf16</code>	<code>.m16n16k16</code> , <code>.m8n32k16</code> , and <code>.m32n8k16</code>	PTX ISA version 7.0
<code>wmma</code>		Dense	Alternate floating-point format - <code>.tf32</code>	<code>.m16n16k8</code>	PTX ISA version 7.0
<code>wmma</code>		Dense	Integer - <code>.u8/.s8</code>	<code>.m16n16k16</code> , <code>.m8n32k16</code> , and <code>.m32n8k16</code>	PTX ISA version 6.3
<code>wmma</code>		Dense	Sub-byte integer - <code>.u4/.s4</code>	<code>.m8n8k32</code>	PTX ISA version 6.3
<code>wmma</code>		Dense	Single-bit - <code>.b1</code>	<code>.m8n8k128</code>	PTX ISA version 6.3

continues on next page

Table 35 – continued from previous page

In-struction	Scale	Sparsity	Multiplicand Data-type	Shape	PTX ISA version
mma	NA	Dense	Floating-point - .f64	.m8n8k4	PTX ISA version 7.0
				.m16n8k4, .m16n8k8, and .m16n8k16	PTX ISA version 7.8
mma		Dense	Floating-point - .f16	.m8n8k4	PTX ISA version 6.4
				.m16n8k8	PTX ISA version 6.5
				.m16n8k16	PTX ISA version 7.0
mma		Dense	Alternate floating-point format - .bf16	.m16n8k8 and .m16n8k16	PTX ISA version 7.0
mma		Dense	Alternate floating-point format - .tf32	.m16n8k4 and .m16n8k8	PTX ISA version 7.0
mma		Dense	Integer - .u8/.s8	.m8n8k16	PTX ISA version 6.5
				.m16n8k16 and .m16n8k32	PTX ISA version 7.0
mma		Dense	Sub-byte integer - .u4/.s4	.m8n8k32	PTX ISA version 6.5
				.m16n8k32 and .m16n8k64	PTX ISA version 7.0
mma		Dense	Single-bit - .b1	.m8n8k128, .m16n8k128, and .m16n8k256	PTX ISA version 7.0
mma		Dense	Alternate floating-point format - .e4m3 / .e5m2	.m16n8k32	PTX ISA version 8.4
mma		Dense	Alternate floating-point format - .e4m3 / .e5m2	.m16n8k16	PTX ISA version 8.7
mma		Dense	Alternate floating-point format - .e3m2 / .e2m3/.e2m1	.m16n8k32	PTX ISA version 8.7
mma	Yes	Dense	Alternate floating-point format - .e4m3 / .e5m2/.e3m2/.e2m3/.e2m1 X (Scale) .ue8m0	.m16n8k32	PTX ISA version 8.7
mma		Dense	Alternate floating-point format - .e2m1 X (Scale) .ue8m0/.ue4m3	.m16n8k64	PTX ISA version 8.7
mma	NA	Sparse	Floating-point - .f16	.m16n8k16 and .m16n8k32	PTX ISA version 7.1
mma		Sparse	Alternate floating-point format - .bf16	.m16n8k16 and .m16n8k32	PTX ISA version 7.1
mma		Sparse	Alternate floating-point format - .tf32	.m16n8k8 and .m16n8k16	PTX ISA version 7.1
mma		Sparse	Integer - .u8/.s8	.m16n8k32 and .m16n8k64	PTX ISA version 7.1

continues on next page

Table 35 – continued from previous page

In- struc- tion	Scale	Sparsity	Multiplicand Data-type	Shape	PTX ISA version
mma		Sparse	Sub-byte integer - .u4/.s4	.m16n8k64 and .m16n8k128	PTX ISA version 7.1
mma		Sparse	Alternate floating-point format - .e4m3 / .e5m2	.m16n8k64	PTX ISA version 8.4
mma		Sparse with ordered meta- data	Floating-point - .f16	.m16n8k16 and .m16n8k32	PTX ISA version 8.5
mma		Sparse with ordered meta- data	Alternate floating-point format - .bf16	.m16n8k16 and .m16n8k32	PTX ISA version 8.5
mma		Sparse with ordered meta- data	Alternate floating-point format - .tf32	.m16n8k8 and .m16n8k16	PTX ISA version 8.5
mma		Sparse with ordered meta- data	Integer - .u8/.s8	.m16n8k32 and .m16n8k64	PTX ISA version 8.5
mma		Sparse with ordered meta- data	Sub-byte integer - .u4/.s4	.m16n8k64 and .m16n8k128	PTX ISA version 8.5
mma		Sparse with ordered meta- data	Alternate floating-point format - .e4m3 / .e5m2	.m16n8k64	PTX ISA version 8.5
mma		Sparse with ordered meta- data	Alternate floating- point format - .e3m2 / .e2m3/.e2m1	.m16n8k64	PTX ISA version 8.7

continues on next page

Table 35 – continued from previous page

In-struction	Scale	Sparsity	Multiplicand Data-type	Shape	PTX ISA version
mma	Yes	Sparse with ordered meta-data	Alternate floating-point format - .e4m3 / .e5m2/ .e3m2/ .e2m3/ .e2m1 X (Scale) .ue8m0	.m16n8k64	PTX ISA version 8.7
mma		Sparse with ordered meta-data	Alternate floating-point format - .e2m1 X (Scale) .ue8m0/ .ue4m3	.m16n8k128	PTX ISA version 8.7

9.7.14.2 Matrix Data-types

The matrix multiply and accumulate operation is supported separately on integer, floating-point, sub-byte integer and single bit data-types. All operands must contain the same basic type kind, i.e., integer or floating-point.

For floating-point matrix multiply and accumulate operation, different matrix operands may have different precision, as described later.

Data-type	Multiplicands (A or B)	Accumulators (C or D)
Integer	.u8, .s8	.s32
Floating Point	.f16	.f16, .f32
Alternate floating Point	.bf16	.f32
Alternate floating Point	.tf32	.f32
Alternate floating Point	.e4m3 or .e5m2 or .e3m2 or .e2m3 or .e2m1	.f16, .f32
Alternate floating Point with scale	.e4m3 or .e5m2 or .e3m2 or .e2m3 or .e2m1 X (Scale) .ue8m0	.f32
Alternate floating Point with scale	.e2m1 X (Scale) .ue8m0 or .ue4m3	.f32
Floating Point	.f64	.f64
Sub-byte integer	both .u4 or both .s4	.s32
Single-bit integer	.b1	.s32

9.7.14.3 Block Scaling for `mma.sync`

The `mma` instruction with the following `.kind` qualifier:

- ▶ `.kind::mxf8f6f4`
- ▶ `.kind::mxf4`
- ▶ `.kind::mxf4nfvf4`

perform matrix multiplication with block scaling. This operation has the following form: $D = (A * \text{scale_A}) * (B * \text{scale_B}) + C$.

For a `scale_A` matrix of shape $M \times SFA_N$, each row of matrix A is divided into SFA_N number of chunks and each chunk of a row is multiplied with the corresponding element (henceforth referred as SF_A) from the same row of `scale_A`.

Similarly, for a `scale_B` matrix of shape $SFB_M \times N$, each column of matrix B is divided into the SFB_M number of chunks and each chunk of a column is multiplied with the corresponding element (henceforth referred as SF_B) from the same column of `scale_B`.

Figure 42 shows an example of `mma` with block scaling of `scale_vec::2X`.

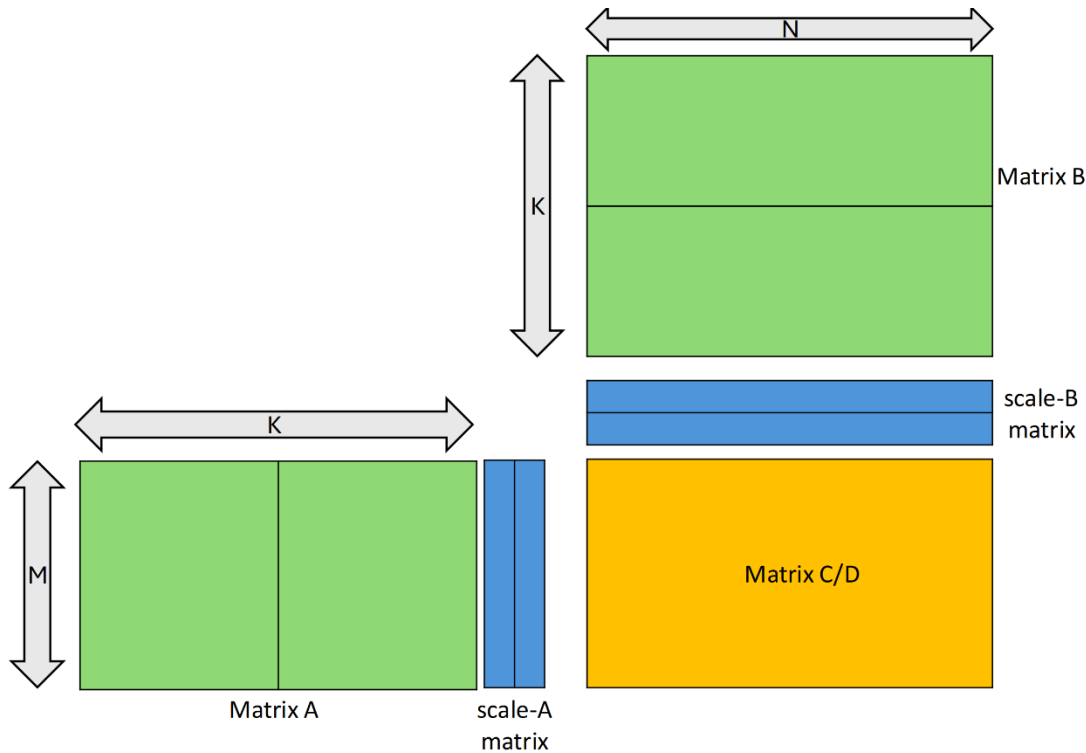


Figure 42: `mma` with block scaling of `.scale_vec::2X`

The shapes for `scale_A` and `scale_B` matrices depend upon the qualifier `.scale_vec_size` as shown in Table 36.

Table 36: Shapes for scale matrices depending upon `.scale_vec_size` qualifier

<code>.scale_vec_size</code>	Shape of <code>scale_A</code>	Shape of <code>scale_B</code>
<code>.scale_vec::1X</code>	$M \times 1$	$1 \times N$
<code>.scale_vec::2X</code>	$M \times 2$	$2 \times N$
<code>.scale_vec::4X</code>	$M \times 4$	$4 \times N$

The valid combination of the exact element types and the `.scale_vec_size` are listed in [Table 37](#).

Table 37: Valid combinations of `.scale_vec_size` and `.kind` qualifier

<code>.kind::*</code>	Element Data Type and <code>.btype</code>	Scale Data Type <code>.stype</code>	<code>.scale_vec_size</code>
<code>.kind::mxf8f6f4</code>	<code>.e4m3</code> , <code>.e5m2</code> , <code>.e3m2</code> , <code>.e2m3</code> <code>.e2m1</code>	<code>.ue8m0</code>	<code>.scale_vec::1X</code>
<code>.kind::mxf4</code>	<code>.e2m1</code>	<code>.ue8m0</code>	<code>.scale_vec::2X</code>
<code>.kind::mxf4nvf4</code>	<code>.e2m1</code>	<code>.ue8m0</code>	<code>.scale_vec::2X</code> , <code>scale_vec::4X</code>
	<code>.e2m1</code>	<code>.ue4m3</code>	<code>.scale_vec::4X</code>

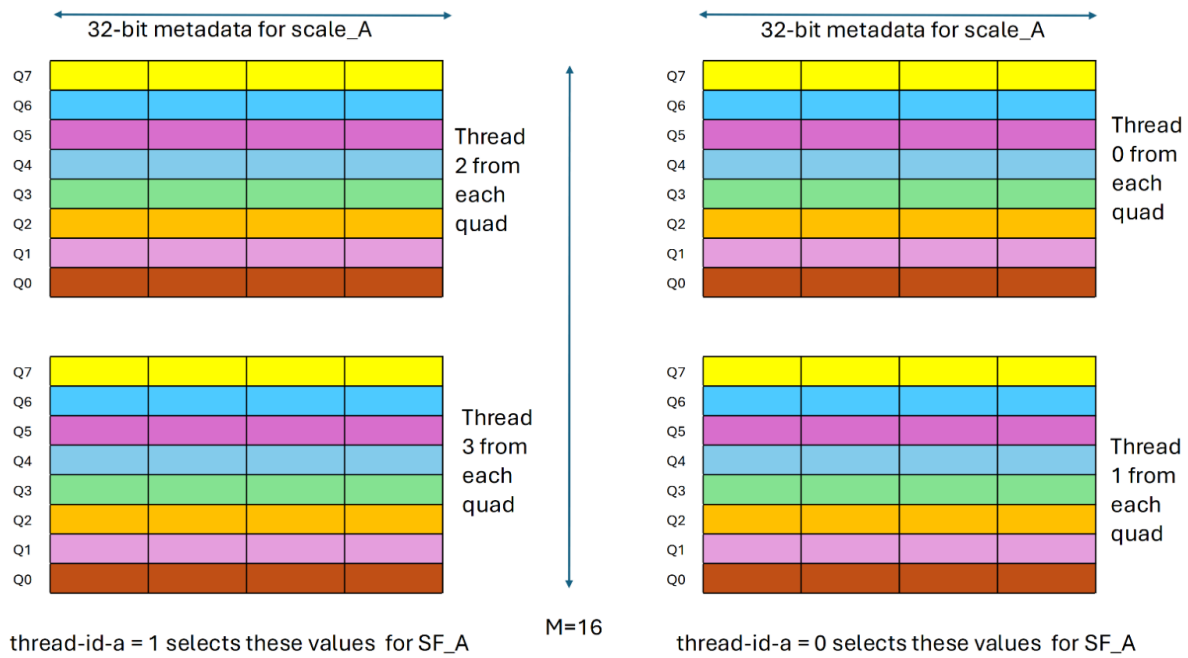
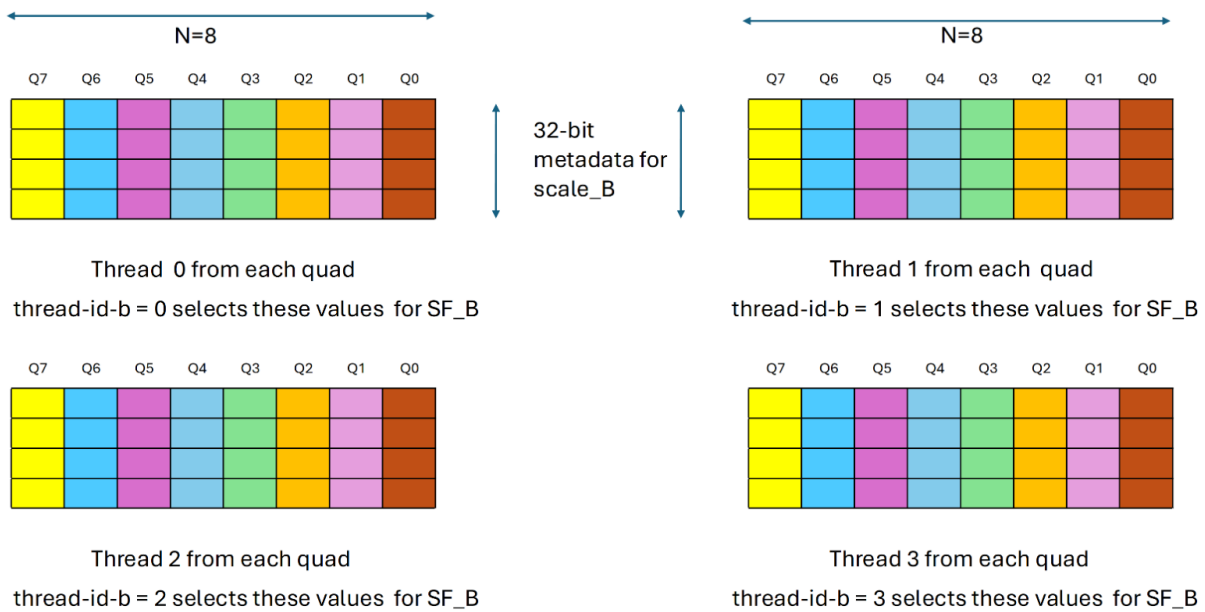
The `scale-a-data` and `scale-b-data` argument provides metadata for `scale_A` and `scale_B` matrices respectively. The tuple `{byte-id-a, thread-id-a}` and `{byte-id-b, thread-id-b}` provides the selector information to choose elements `SF_A` and `SF_B` from corresponding metadata arguments `scale-a-data` and `scale-b-data`. The tuple `{byte-id-a, thread-id-a}` allows to select the scale matrix element `SF_A` from `scale-a-data`. Similarly, the tuple `{byte-id-b, thread-id-b}` allows to select the scale matrix element `SF_B` from `scale-b-data`.

The components `thread-id-a`, `thread-id-b` decides which threads among the quad contribute the `SF_A` and `SF_B` values. The following listing describes the impact of thread selector component `thread-id-a`, `thread-id-b`:

- One thread-pair within the quad determined by `thread-id-a`, contributes the `SF_A` values. The value of 0 selects lower two threads whereas value of 1 selects upper two threads from the quad. In other words, when `thread-id-a` set to 0, thread-pair satisfying: `%laneid % 4 == 0` or `1` provides the `SF_A`. In contrast when `thread-id-a` set to 1, thread-pair satisfying: `%laneid % 4 == 2` or `3` provides the `SF_A`. Refer [Figure 43](#) for more details.
- One thread within the quad, determined by `thread-id-b`, contributes the `SF_B` value. In other words, each thread satisfying: `%laneid % 4 == thread-id-b` provides the `SF_B`. Refer [Figure 44](#) for more details.

The arguments `byte-id-a`, `byte-id-b` selects which bytes from the `scale-a-data`, `scale-b-data` contribute the `SF_A` and `SF_B` values. The following listing describes implications of `.scale_vec_size` qualifier on byte selector component `byte-id-a`, `byte-id-b`:

- When `.scale_vec_size` is `.scale_vec::1X`
 - One byte each within `scale-a-data` and `scale-b-data` determined by `byte-id-a`, `byte-id-b` respectively contributes the `SF_A` and `SF_B` values.
- When `.scale_vec_size` is `.scale_vec::2X`

Figure 43: Selection of set of values for SF_A based on thread-id-aFigure 44: Selection of set of values for SF_B based on thread-id-b

- One byte-pair (two bytes) within `scale-a-data` and `scale-b-data` determined by `byte-id-a` and `byte-id-b` contributes the `SF_A` and `SF_B` values. The value of 0 selects lower two bytes whereas value of 2 selects upper two bytes from the corresponding meta-data value.
- When `.scale_vec_size` is `.scale_vec::4X`
 - All four bytes within `scale-a-data` and `scale-b-data` contribute the values. Hence, `byte-id-a`, `byte-id-b` must be zero.

Refer Figure 45 for more details.

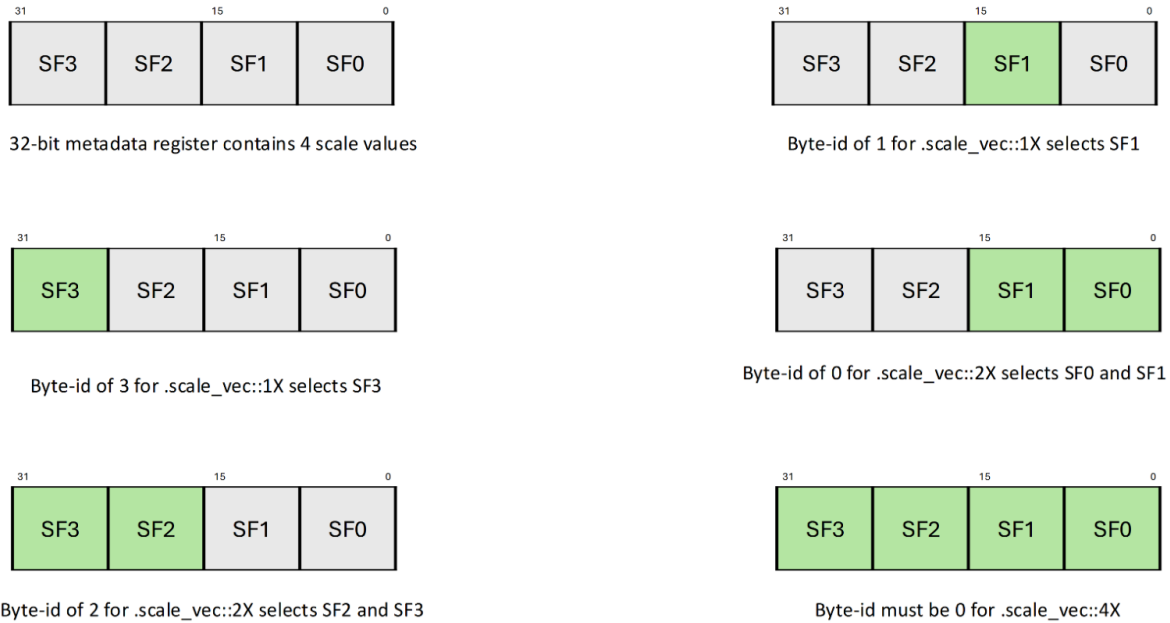


Figure 45: Selection of set of values for `SF_A` or `SF_B` based on `byte-id-a` or `byte-id-b`

Table 38 enumerates the valid values for various selector components. Any other value results in an undefined behavior.

Table 38: Valid values for various selector components

<code>.scale_vec_size</code>	Selector Components			
	<code>byte-id-a</code>	<code>thread-id-a</code>	<code>byte-id-b</code>	<code>thread-id-b</code>
<code>scale_vec::1X</code>	[0, 1, 2, 3]	[0, 1]	[0, 1, 2, 3]	[0, 1, 2, 3]
<code>scale_vec::2X</code>	[0, 2]		[0, 2]	
<code>scale_vec::4X</code>	0		0	

9.7.14.4 Matrix multiply-accumulate operation using wmma instructions

This section describes warp level `wmma.load`, `wmma.mma` and `wmma.store` instructions and the organization of various matrices involved in these instructions.

9.7.14.4.1 Matrix Fragments for WMMA

Each thread in the warp holds a fragment of the matrix. The distribution of fragments loaded by the threads in a warp is unspecified and is target architecture dependent, and hence the identity of the fragment within the matrix is also unspecified and is target architecture dependent. The fragment returned by a `wmma` operation can be used as an operand for another `wmma` operation if the shape, layout and element type of the underlying matrix matches. Since fragment layout is architecture dependent, using the fragment returned by a `wmma` operation in one function as an operand for a `wmma` operation in a different function may not work as expected if the two functions are linked together but were compiled for different link-compatible SM architectures. Note passing `wmma` fragment to a function having `.weak` linkage is unsafe since at link time references to such function may get resolved to a function in different compilation module.

Each fragment is a vector expression whose contents are determined as follows. The identity of individual matrix elements in the fragment is unspecified.

Integer fragments

Multiplicands (A or B):

Data-type	Shape	Matrix	Fragment
.u8 or .s8	.m16n16k16	A	A vector expression of two .b32 registers, with each register containing four elements from the matrix.
		B	A vector expression of two .b32 registers, with each register containing four elements from the matrix.
	.m8n32k16	A	A vector expression containing a single .b32 register containing four elements from the matrix.
		B	A vector expression of four .b32 registers, with each register containing four elements from the matrix.
	.m32n8k16	A	A vector expression of four .b32 registers, with each register containing four elements from the matrix.
		B	A vector expression containing single .b32 register, with each containing four elements from the matrix.

Accumulators (C or D):

Data-type	Shape	Fragment
.s32	.m16n16k16	A vector expression of eight .s32 registers.
	.m8n32k16	
	.m32n8k16	

Floating point fragments

Data-type	Matrix	Fragment
.f16	A or B	A vector expression of eight .f16x2 registers.
.f16	C or D	A vector expression of four .f16x2 registers.
.f32		A vector expression of eight .f32 registers.

Floating point fragments for .bf16 data format

Multiplicands (A or B):

Data-type	Shape	Matrix	Fragment
.bf16	.m16n16k16	A	A vector expression of four .b32 registers, with each register containing two elements from the matrix.
		B	
	.m8n32k16	A	A vector expression containing a two .b32 registers, with containing two elements from the matrix.
		B	A vector expression of eight .b32 registers, with each register containing two elements from the matrix.
	.m32n8k16	A	A vector expression of eight .b32 registers, with each register containing two elements from the matrix.
		B	A vector expression containing two .b32 registers, with each containing two elements from the matrix.

Accumulators (C or D):

Data-type	Matrix	Fragment
.f32	C or D	A vector expression containing eight .f32 registers.

Floating point fragments for .tf32 data format

Multiplicands (A or B):

Data-type	Shape	Matrix	Fragment
.tf32	.m16n16k8	A	A vector expression of four .b32 registers.
		B	A vector expression of four .b32 registers.

Accumulators (C or D):

Data-type	Shape	Matrix	Fragment
.f32	.m16n16k8	C or D	A vector expression containing eight .f32 registers.

Double precision floating point fragments

Multiplicands (A or B):

Data-type	Shape	Matrix	Fragment
.f64	.m8n8k4	A or B	A vector expression of single .f64 register.

Accumulators (C or D):

Data-type	Shape	Matrix	Fragment
.f64	.m8n8k4	C or D	A vector expression containing single .f64 register.

Sub-byte integer and single-bit fragments

Multiplicands (A or B):

Data-type	Shape	Fragment
.u4 or .s4	.m8n8k32	A vector expression containing a single .b32 register, containing eight elements from the matrix.
.b1	.m8n8k128	A vector expression containing a single .b32 register, containing 32 elements from the matrix.

Accumulators (C or D):

Data-type	Shape	Fragment
.s32	.m8n8k32	A vector expression of two .s32 registers.
	.m8n8k128	A vector expression of two .s32 registers.

Manipulating fragment contents

The contents of a matrix fragment can be manipulated by reading and writing to individual registers in the fragment, provided the following conditions are satisfied:

- All matrix element in the fragment are operated on uniformly across threads, using the same parameters.
- The order of the matrix elements is not changed.

For example, if each register corresponding to a given matrix is multiplied by a uniform constant value, then the resulting matrix is simply the scaled version of the original matrix.

Note that type conversion between .f16 and .f32 accumulator fragments is not supported in either direction. The result is undefined even if the order of elements in the fragment remains unchanged.

9.7.14.4.2 Matrix Storage for WMMA

Each matrix can be stored in memory with a *row-major* or *column-major* layout. In a *row-major* format, consecutive elements of each row are stored in contiguous memory locations, and the row is called the *leading dimension* of the matrix. In a *column-major* format, consecutive elements of each column are stored in contiguous memory locations and the column is called the *leading dimension* of the matrix.

Consecutive instances of the *leading dimension* (rows or columns) need not be stored contiguously in memory. The `wmma.load` and `wmma.store` operations accept an optional argument `stride` that specifies the offset from the beginning of each row (or column) to the next, in terms of matrix elements (and not bytes). For example, the matrix being accessed by a `wmma` operation may be a submatrix from a larger matrix stored in memory. This allows the programmer to compose a multiply-and-accumulate operation on matrices that are larger than the shapes supported by the `wmma` operation.

Address Alignment

The starting address of each instance of the leading dimension (row or column) must be aligned with the size of the corresponding fragment in bytes. Note that the starting address is determined by the base pointer and the optional `stride`.

Consider the following instruction as an example:

```
wmma.load.a.sync.aligned.row.m16n16k16.f16 {x0,...,x7}, [p], s;
```

- Fragment size in bytes = 32 (eight elements of type `.f16x2`)
- Actual `stride` in bytes = $2 * s$ (since `stride` is specified in terms of `.f16` elements, not bytes)
- For each row of this matrix to be aligned at fragment size the following must be true:
 1. `p` is a multiple of 32.
 2. $2*s$ is a multiple of 32.

Default value for stride

The default value of the `stride` is the size of the *leading dimension* of the matrix. For example, for an $M \times K$ matrix, the `stride` is K for a *row-major* layout and M for a *column-major* layout. In particular, the default strides for the supported matrix shapes are as follows:

Shape	A (row)	A (column)	B (row)	B (column)	Accumulator (row)	Accumulator (column)
16x16x16	16	16	16	16	16	16
8x32x16	16	8	32	16	32	8
32x8x16	16	32	8	16	8	32
8x8x32	32	8	8	32	8	8
8x8x128	128	8	8	128	8	8
16x16x8	8	16	16	8	16	16
8x8x4	4	8	8	4	8	8

9.7.14.4.3 Warp-level Matrix Load Instruction: `wmma.load`

`wmma.load`

Collectively load a matrix from memory for WMMA

Syntax

Floating point format `.f16` loads:

```
wmma.load.a.sync.aligned.layout.shape{.ss}.atype r, [p] {, stride};
wmma.load.b.sync.aligned.layout.shape{.ss}.btype r, [p] {, stride};
wmma.load.c.sync.aligned.layout.shape{.ss}.ctype r, [p] {, stride};

.layout = {.row, .col};
.shape  = {.m16n16k16, .m8n32k16, .m32n8k16};
.ss      = {.global, .shared{::cta}};
.atype   = {.f16, .s8, .u8};
.btype   = {.f16, .s8, .u8};
.ctype   = {.f16, .f32, .s32};
```

Alternate floating point format `.bf16` loads:

```
wmma.load.a.sync.aligned.layout.shape{.ss}.atype r, [p] {, stride}
wmma.load.b.sync.aligned.layout.shape{.ss}.btype r, [p] {, stride}
wmma.load.c.sync.aligned.layout.shape{.ss}.ctype r, [p] {, stride}
.layout = {.row, .col};
.shape  = {.m16n16k16, .m8n32k16, .m32n8k16};
.ss      = {.global, .shared{::cta}};
.atype   = {.bf16 };
.btype   = {.bf16 };
.ctype   = {.f32  };
```

Alternate floating point format `.tf32` loads:

```
wmma.load.a.sync.aligned.layout.shape{.ss}.atype r, [p] {, stride}
wmma.load.b.sync.aligned.layout.shape{.ss}.btype r, [p] {, stride}
wmma.load.c.sync.aligned.layout.shape{.ss}.ctype r, [p] {, stride}
.layout = {.row, .col};
.shape  = {.m16n16k8 };
.ss      = {.global, .shared{::cta}};
.atype   = {.tf32 };
.btype   = {.tf32 };
.ctype   = {.f32  };
```

Double precision Floating point `.f64` loads:

```
wmma.load.a.sync.aligned.layout.shape{.ss}.atype r, [p] {, stride}
wmma.load.b.sync.aligned.layout.shape{.ss}.btype r, [p] {, stride}
wmma.load.c.sync.aligned.layout.shape{.ss}.ctype r, [p] {, stride}
.layout = {.row, .col};
.shape  = {.m8n8k4 };
.ss      = {.global, .shared{::cta}};
.atype   = {.f64 };
.btype   = {.f64 };
.ctype   = {.f64 };
```

Sub-byte loads:

```
wmma.load.a.sync.aligned.row.shape{.ss}.atype r, [p] {, stride}
wmma.load.b.sync.aligned.col.shape{.ss}.btype r, [p] {, stride}
wmma.load.c.sync.aligned.layout.shape{.ss}.ctype r, [p] {, stride}
.layout = {.row, .col};
.shape = {.m8n8k32};
.ss = {.global, .shared{::cta}};
.atype = {.s4, .u4};
.btype = {.s4, .u4};
.ctype = {.s32};
```

Single-bit loads:

```
wmma.load.a.sync.aligned.row.shape{.ss}.atype r, [p] {, stride}
wmma.load.b.sync.aligned.col.shape{.ss}.btype r, [p] {, stride}
wmma.load.c.sync.aligned.layout.shape{.ss}.ctype r, [p] {, stride}
.layout = {.row, .col};
.shape = {.m8n8k128};
.ss = {.global, .shared{::cta}};
.atype = {.b1};
.btype = {.b1};
.ctype = {.s32};
```

Description

Collectively load a matrix across all threads in a warp from the location indicated by address operand *p* in the specified state space into destination register *r*.

If no state space is given, perform the memory accesses using [Generic Addressing](#). `wmma.load` operation may be used only with `.global` and `.shared` spaces and with generic addressing, where the address points to `.global` or `.shared` space.

The mutually exclusive qualifiers `.a`, `.b` and `.c` indicate whether matrix A, B or C is being loaded respectively for the `wmma` computation.

The destination operand *r* is a brace-enclosed vector expression that can hold the fragment returned by the load operation, as described in [Matrix Fragments for WMMA](#).

The `.shape` qualifier indicates the dimensions of all the matrix arguments involved in the intended `wmma` computation.

The `.layout` qualifier indicates whether the matrix to be loaded is stored in *row-major* or *column-major* format.

`stride` is an optional 32-bit integer operand that provides an offset in terms of matrix elements between the start of consecutive instances of the *leading dimension* (rows or columns). The default value of `stride` is described in [Matrix Storage for WMMA](#) and must be specified if the actual value is larger than the default. For example, if the matrix is a sub-matrix of a larger matrix, then the value of `stride` is the leading dimension of the larger matrix. Specifying a value lower than the default value results in undefined behavior.

The required alignment for address *p* and `stride` is described in the [Matrix Storage for WMMA](#).

The mandatory `.sync` qualifier indicates that `wmma.load` causes the executing thread to wait until all threads in the warp execute the same `wmma.load` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `wmma.load` instruction. In conditionally executed code, a `wmma.load` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `wmma.load` is undefined if all threads do not use the same qualifiers and the same values of `p` and `stride`, or if any thread in the warp has exited.

`wmma.load` is treated as a weak memory operation in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 6.0.

`.m8n32k16` and `.m32n8k16` introduced in PTX ISA version 6.1.

Integer, sub-byte integer and single-bit `wmma` introduced in PTX ISA version 6.3.

`.m8n8k4` and `.m16n16k8` on `wmma` introduced in PTX ISA version 7.0.

Double precision and alternate floating point precision `wmma` introduced in PTX ISA version 7.0.

Modifier `.aligned` is required from PTX ISA version 6.3 onwards, and considered implicit in PTX ISA versions less than 6.3.

Support for `::cta` sub-qualifier introduced in PTX ISA version 7.8.

Target ISA Notes

Floating point `wmma` requires `sm_70` or higher.

Integer `wmma` requires `sm_72` or higher.

Sub-byte and single-bit `wmma` requires `sm_75` or higher.

Double precision and alternate floating point precision `wmma` requires `sm_80` or higher.

Examples

```
// Load elements from f16 row-major matrix B
.reg .b32 x<8>;

wmma.load.b.sync.aligned.m16n16k16.row.f16 {x0,x1,x2,x3,x4,x5,x,x7}, [ptr];
// Now use {x0, ..., x7} for the actual wmma.mma

// Load elements from f32 column-major matrix C and scale the values:
.reg .b32 x<8>;

wmma.load.c.sync.aligned.m16n16k16.col.f32
    {x0,x1,x2,x3,x4,x5,x6,x7}, [ptr];

mul.f32 x0, x0, 0.1;
// repeat for all registers x<8>;
...
mul.f32 x7, x7, 0.1;
// Now use {x0, ..., x7} for the actual wmma.mma

// Load elements from integer matrix A:
```

(continues on next page)

(continued from previous page)

```
.reg .b32 x<4>
// destination registers x<4> contain four packed .u8 values each
wmma.load.a.sync.aligned.m32n8k16.row.u8 {x0,x1,x2,x3}, [ptr];

// Load elements from sub-byte integer matrix A:
.reg .b32 x0;
// destination register x0 contains eight packed .s4 values
wmma.load.a.sync.aligned.m8n8k32.row.s4 {x0}, [ptr];

// Load elements from .bf16 matrix A:
.reg .b32 x<4>;
wmma.load.a.sync.aligned.m16n16k16.row.bf16
    {x0,x1,x2,x3}, [ptr];

// Load elements from .tf32 matrix A:
.reg .b32 x<4>;
wmma.load.a.sync.aligned.m16n16k8.row.tf32
    {x0,x1,x2,x3}, [ptr];

// Load elements from .f64 matrix A:
.reg .b32 x<4>;
wmma.load.a.sync.aligned.m8n8k4.row.f64
    {x0}, [ptr];
```

9.7.14.4.4 Warp-level Matrix Store Instruction: `wmma.store`

`wmma.store`

Collectively store a matrix into memory for WMMA

Syntax

```
wmma.store.d.sync.aligned.layout.shape{.ss}.type [p], r {, stride};

.layout = {.row, .col};
.shape  = {.m16n16k16, .m8n32k16, .m32n8k16};
.ss     = {.global, .shared{::cta}};
.type   = {.f16, .f32, .s32};

wmma.store.d.sync.aligned.layout.shape{.ss}.type [p], r {, stride}
.layout = {.row, .col};
.shape  = {.m8n8k32, .m8n8k128};
.ss     = {.global, .shared{::cta}};
.type   = {.s32};

wmma.store.d.sync.aligned.layout.shape{.ss}.type [p], r {, stride}
.layout = {.row, .col};
.shape  = {.m16n16k8};
.ss     = {.global, .shared{::cta}};
.type   = {.f32};

wmma.store.d.sync.aligned.layout.shape{.ss}.type [p], r {, stride}
```

(continues on next page)

(continued from previous page)

```
.layout = {.row, .col};
.shape  = {.m8n8k4 };
.ss     = {.global, .shared{::cta}};
.type   = {.f64};
```

Description

Collectively store a matrix across all threads in a warp at the location indicated by address operand `p` in the specified state space from source register `r`.

If no state space is given, perform the memory accesses using [Generic Addressing](#). `wmma.load` operation may be used only with `.global` and `.shared` spaces and with generic addressing, where the address points to `.global` or `.shared` space.

The source operand `r` is a brace-enclosed vector expression that matches the shape of the fragment expected by the store operation, as described in [Matrix Fragments for WMMA](#).

The `.shape` qualifier indicates the dimensions of all the matrix arguments involved in the intended `wmma` computation. It must match the `.shape` qualifier specified on the `wmma.mma` instruction that produced the D matrix being stored.

The `.layout` qualifier indicates whether the matrix to be loaded is stored in *row-major* or *column-major* format.

`stride` is an optional 32-bit integer operand that provides an offset in terms of matrix elements between the start of consecutive instances of the *leading dimension* (rows or columns). The default value of `stride` is described in [Matrix Storage for WMMA](#) and must be specified if the actual value is larger than the default. For example, if the matrix is a sub-matrix of a larger matrix, then the value of `stride` is the leading dimension of the larger matrix. Specifying a value lower than the default value results in undefined behavior.

The required alignment for address `p` and `stride` is described in the [Matrix Storage for WMMA](#).

The mandatory `.sync` qualifier indicates that `wmma.store` causes the executing thread to wait until all threads in the warp execute the same `wmma.store` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `wmma.store` instruction. In conditionally executed code, a `wmma.store` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `wmma.store` is undefined if all threads do not use the same qualifiers and the same values of `p` and `stride`, or if any thread in the warp has exited.

`wmma.store` is treated as a *weak* memory operation in the [Memory Consistency Model](#).

PTX ISA Notes

Introduced in PTX ISA version 6.0.

`.m8n32k16` and `.m32n8k16` introduced in PTX ISA version 6.1.

Integer, sub-byte integer and single-bit `wmma` introduced in PTX ISA version 6.3.

`.m16n16k8` introduced in PTX ISA version 7.0.

Double precision `wmma` introduced in PTX ISA version 7.0.

Modifier `.aligned` is required from PTX ISA version 6.3 onwards, and considered implicit in PTX ISA versions less than 6.3.

Support for `::cta` sub-qualifier introduced in PTX ISA version 7.8.

Target ISA Notes

Floating point `wmma` requires `sm_70` or higher.

Integer `wmma` requires `sm_72` or higher.

Sub-byte and single-bit `wmma` requires `sm_75` or higher.

Double precision `wmma` and shape `.m16n16k8` requires `sm_80` or higher.

Examples

```
// Storing f32 elements computed by a wmma.mma
.reg .b32 x<8>;

wmma.mma.sync.m16n16k16.row.col.f32.f32
    {d0, d1, d2, d3, d4, d5, d6, d7}, ...;
wmma.store.d.sync.m16n16k16.row.f32
    [ptr], {d0, d1, d2, d3, d4, d5, d6, d7};

// Store s32 accumulator for m16n16k16 shape:
.reg .b32 d<8>;
wmma.store.d.sync.aligned.m16n16k16.row.s32
    [ptr], {d0, d1, d2, d3, d4, d5, d6, d7};

// Store s32 accumulator for m8n8k128 shape:
.reg .b32 d<2>;
wmma.store.d.sync.aligned.m8n8k128.row.s32
    [ptr], {d0, d1};

// Store f64 accumulator for m8n8k4 shape:
.reg .f64 d<2>;
wmma.store.d.sync.aligned.m8n8k4.row.f64
    [ptr], {d0, d1};
```

9.7.14.4.5 Warp-level Matrix Multiply-and-Accumulate Instruction: `wmma.mma`

`wmma.mma`

Perform a single matrix multiply-and-accumulate operation across a warp

Syntax

```
// Floating point (.f16 multiplicands) wmma.mma
wmma.mma.sync.aligned.alayout.blayout.shape.dtype.ctype d, a, b, c;

// Integer (.u8/.s8 multiplicands) wmma.mma
wmma.mma.sync.aligned.alayout.blayout.shape.s32.atype.btype.s32{.satfinite} d, a, b,
↪ c;

.alayout = {.row, .col};
.blayout = {.row, .col};
.shape   = {.m16n16k16, .m8n32k16, .m32n8k16};
.dtype   = {.f16, .f32};
.atype   = {.s8, .u8};
.btype   = {.s8, .u8};
.ctype   = {.f16, .f32};
```

Floating point format .bf16 wmma.mma:

```
wmma.mma.sync.aligned.alayout.blayout.shape.f32.atype.btype.f32 d, a, b, c;
.alayout = {.row, .col};
.blayout = {.row, .col};
.shape   = {.m16n16k16, .m8n32k16, .m32n8k16};
.atype   = {.bf16 };
.btype   = {.bf16};
```

Floating point format .tf32 wmma.mma:

```
wmma.mma.sync.aligned.alayout.blayout.shape.f32.atype.btype.f32 d, a, b, c;
.alayout = {.row, .col};
.blayout = {.row, .col};
.shape   = {.m16n16k8 };
.atype   = {.tf32 };
.btype   = {.tf32};
```

Floating point Double precision wmma.mma:

```
wmma.mma.sync.aligned.alayout.blayout.shape{.rnd}.f64.f64.f64.f64 d, a, b, c;
.alayout = {.row, .col};
.blayout = {.row, .col};
.shape   = {.m8n8k4 };
.rnd     = { .rn, .rz, .rm, .rp };
```

Sub-byte (.u4/.s4 multiplicands) wmma.mma:

```
wmma.mma.sync.aligned.row.col.shape.s32.atype.btype.s32{.satfinite} d, a, b, c;
.shape   = {.m8n8k32};
.atype   = {.s4, .u4};
.btype   = {.s4, .u4};
```

Single-bit (.b1 multiplicands) wmma.mma:

```
wmma.mma.op.popc.sync.aligned.row.col.shape.s32.atype.btype.s32 d, a, b, c;
.shape   = {.m8n8k128};
.atype   = {.b1};
.btype   = {.b1};
.op      = {.xor, .and}
```


Description

Perform a warp-level matrix multiply-and-accumulate computation $D = A * B + C$ using matrices A, B and C loaded in registers a, b and c respectively, and store the result matrix in register d. The register arguments a, b, c and d hold unspecified fragments of the corresponding matrices as described in *Matrix Fragments for WMMA*.

The qualifiers `.dtype`, `.atype`, `.btype` and `.ctype` indicate the data-type of the elements in the matrices D, A, B and C respectively.

For `wmma.mma` without explicit `.atype` and `.btype`: `.atype` and `.btype` are implicitly set to `.f16`.

For integer `wmma`, `.ctype` and `.dtype` must be specified as `.s32`. Also, the values for `.atype` and `.btype` must be the same, i.e., either both are `.s8` or both are `.u8`.

For sub-byte single-bit `wmma`, `.ctype` and `.dtype` must be specified as `.s32`. Also, the values for `.atype` and `.btype` must be the same; i.e., either both are `.s4`, both are `.u4`, or both are `.b1`.

For single-bit `wmma`, multiplication is replaced by a sequence of logical operations; specifically, `wmma.xor.popc` and `wmma.and.popc` computes the XOR, AND respectively of a 128-bit row of A with a 128-bit column of B, then counts the number of set bits in the result (`popc`). This result is added to the corresponding element of C and written into D.

The qualifiers `.alayout` and `.blayout` must match the layout specified on the `wmma.load` instructions that produce the contents of operands a and b respectively. Similarly, the qualifiers `.atype`, `.btype` and `.ctype` must match the corresponding qualifiers on the `wmma.load` instructions that produce the contents of operands a, b and c respectively.

The `.shape` qualifier must match the `.shape` qualifier used on the `wmma.load` instructions that produce the contents of all three input operands a, b and c respectively.

The destination operand d is a brace-enclosed vector expression that matches the `.shape` of the fragment computed by the `wmma.mma` instruction.

Saturation at the output:

The optional qualifier `.satfinite` indicates that the final values in the destination register are saturated as follows:

- The output is clamped to the minimum or maximum 32-bit signed integer value. Otherwise, if the accumulation would overflow, the value wraps.

Precision and rounding for `.f16` floating point operations:

Element-wise multiplication of matrix A and B is performed with at least single precision. When `.ctype` or `.dtype` is `.f32`, accumulation of the intermediate values is performed with at least single precision. When both `.ctype` and `.dtype` are specified as `.f16`, the accumulation is performed with at least half precision.

The accumulation order, rounding and handling of subnormal inputs is unspecified.

Precision and rounding for `.bf16`, `.tf32` floating point operations:

Element-wise multiplication of matrix A and B is performed with specified precision. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding and handling of subnormal inputs is unspecified.

Rounding modifiers on double precision `wmma.mma` (default is `.rn`):

- `.rn` mantissa LSB rounds to nearest even
- `.rz` mantissa LSB rounds towards zero
- `.rm` mantissa LSB rounds towards negative infinity

.rp mantissa LSB rounds towards positive infinity

The mandatory **.sync** qualifier indicates that **wmma.mma** causes the executing thread to wait until all threads in the warp execute the same **wmma.mma** instruction before resuming execution.

The mandatory **.aligned** qualifier indicates that all threads in the warp must execute the same **wmma.mma** instruction. In conditionally executed code, a **wmma.mma** instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of **wmma.mma** is undefined if all threads in the same warp do not use the same qualifiers, or if any thread in the warp has exited.

PTX ISA Notes

Introduced in PTX ISA version 6.0.

.m8n32k16 and **.m32n8k16** introduced in PTX ISA version 6.1.

Integer, sub-byte integer and single-bit **wmma** introduced in PTX ISA version 6.3.

Double precision and alternate floating point precision **wmma** introduced in PTX ISA version 7.0.

Support for **.and** operation in single-bit **wmma** introduced in PTX ISA version 7.1.

Modifier **.aligned** is required from PTX ISA version 6.3 onwards, and considered implicit in PTX ISA versions less than 6.3.

Support for **.satfinite** on floating point **wmma.mma** is deprecated in PTX ISA version 6.4 and is removed from PTX ISA version 6.5.

Target ISA Notes

Floating point **wmma** requires **sm_70** or higher.

Integer **wmma** requires **sm_72** or higher.

Sub-byte and single-bit **wmma** requires **sm_75** or higher.

Double precision, alternate floating point precision **wmma** require **sm_80** or higher.

.and operation in single-bit **wmma** requires **sm_80** or higher.

Examples

```
.global .align 32 .f16 A[256], B[256];
.global .align 32 .f32 C[256], D[256];
.reg .b32 a<8> b<8> c<8> d<8>;

wmma.load.a.sync.aligned.m16n16k16.global.row.f16
    {a0, a1, a2, a3, a4, a5, a6, a7}, [A];
wmma.load.b.sync.aligned.m16n16k16.global.col.f16
    {b0, b1, b2, b3, b4, b5, b6, b7}, [B];

wmma.load.c.sync.aligned.m16n16k16.global.row.f32
    {c0, c1, c2, c3, c4, c5, c6, c7}, [C];

wmma.mma.sync.aligned.m16n16k16.row.col.f32.f32
```

(continues on next page)

(continued from previous page)

```

        {d0, d1, d2, d3, d4, d5, d6, d7},
        {a0, a1, a2, a3, a4, a5, a6, a7},
        {b0, b1, b2, b3, b4, b5, b6, b7},
        {c0, c1, c2, c3, c4, c5, c6, c7};

mma.store.d.sync.aligned.m16n16k16.global.col.f32
    [D], {d0, d1, d2, d3, d4, d5, d6, d7};

// Compute an integer WMMA:
.reg .b32 a, b<4>;
.reg .b32 c<8>, d<8>;
mma.mma.sync.aligned.m8n32k16.row.col.s32.s8.s8.s32
    {d0, d1, d2, d3, d4, d5, d6, d7},
    {a}, {b0, b1, b2, b3},
    {c0, c1, c2, c3, c4, c5, c6, c7};

// Compute sub-byte WMMA:
.reg .b32 a, b, c<2> d<2>
mma.mma.sync.aligned.m8n8k32.row.col.s32.s4.s4.s32
    {d0, d1}, {a}, {b}, {c0, c1};

// Compute single-bit type WMMA:
.reg .b32 a, b, c<2> d<2>
mma.mma.xor.popc.sync.aligned.m8n8k128.row.col.s32.b1.b1.s32
    {d0, d1}, {a}, {b}, {c0, c1};

// Compute double precision wmma
.reg .f64 a, b, c<2>, d<2>;
mma.mma.sync.aligned.m8n8k4.row.col.f64.f64.f64.f64
    {d0, d1}, {a}, {b}, {c0, c1};

// Compute alternate floating point precision wmma
.reg .b32 a<2>, b<2>, c<8>, d<8>;
mma.mma.sync.aligned.m16n16k8.row.col.f32.tf32.tf32.f32
    {d0, d1, d2, d3, d4, d5, d6, d7},
    {a0, a1, a2, a3}, {b0, b1, b2, b3},
    {c0, c1, c2, c3, c4, c5, c6, c7};

```

9.7.14.5 Matrix multiply-accumulate operation using mma instruction

This section describes warp-level mma, ldmatrix, stmatrix, and movmatrix instructions and the organization of various matrices involved in these instructions.

9.7.14.5.1 Matrix Fragments for mma.m8n8k4 with .f16 floating point type

A warp executing mma.m8n8k4 with .f16 floating point type will compute 4 MMA operations of shape .m8n8k4.

Elements of 4 matrices need to be distributed across the threads in a warp. The following table shows distribution of matrices for MMA operations.

MMA Computation	Threads participating in MMA computation
MMA computation 1	Threads with %laneid 0-3 (low group) and 16-19 (high group)
MMA computation 2	Threads with %laneid 4-7 (low group) and 20-23 (high group)
MMA computation 3	Threads with %laneid 8-11 (low group) and 24-27 (high group)
MMA computation 4	Threads with %laneid 12-15 (low group) and 28-31 (high group)

For each of the individual MMA computation shown above, each of the required thread holds a fragment of the matrix for performing mma operation as follows:

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix A.	a0, a1, a2, a3

The layout of the fragments held by different threads is shown below:

► Fragment layout for Row Major matrix A is shown in Figure 46.

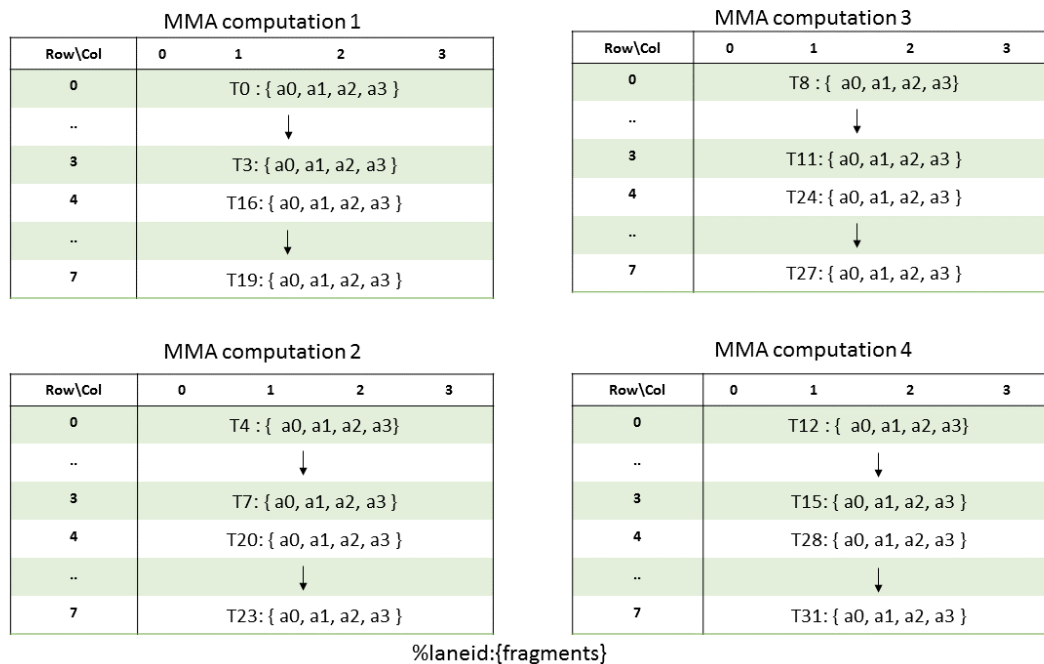


Figure 46: MMA .m8n8k4 fragment layout for row-major matrix A with .f16 type

The row and column of a matrix fragment can be computed as:

```
row =          %laneid % 4      if %laneid < 16
              (%laneid % 4) + 4 otherwise
```

(continues on next page)

(continued from previous page)

```
col =          i          for ai where i = {0,...,3}
```

- Fragment layout for Column Major matrix A is shown in Figure 47.

The layout of the fragments held by different threads is shown below:

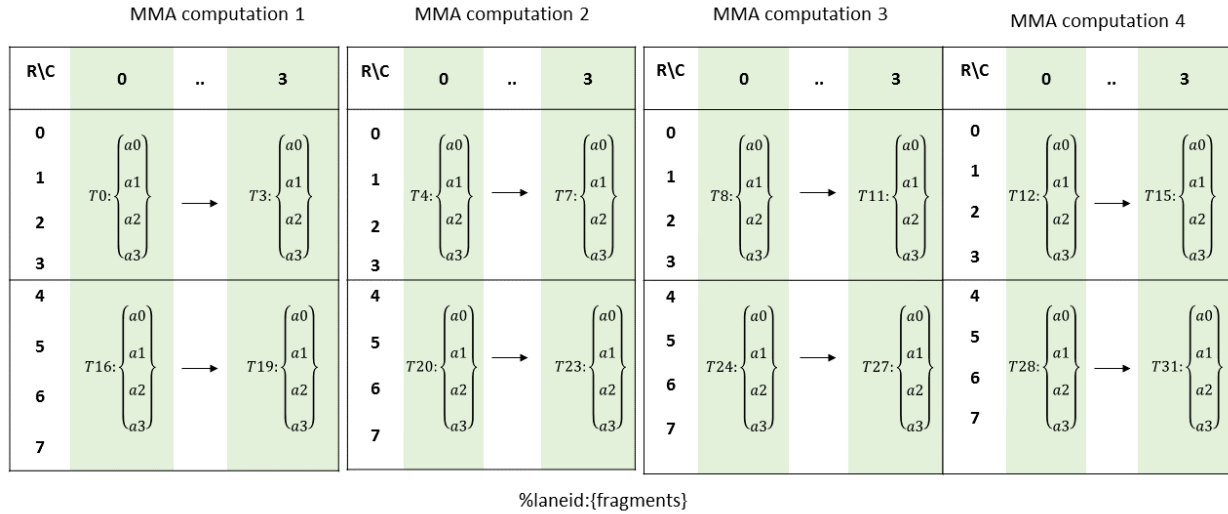


Figure 47: MMA .m8n8k4 fragment layout for column-major matrix A with .f16 type

The row and column of a matrix fragment can be computed as:

```
row =          i % 4          for ai where i = {0,...,3}  if %laneid < 16
      (i % 4) + 4          for ai where i = {0,...,3}  otherwise

col =          %laneid % 4
```

- Multiplicand B:

.btype	Fragment	Elements (low to high)
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown below:

- Fragment layout for Row Major matrix B is shown in Figure 48.

The row and column of a matrix fragment can be computed as:

```
row =          %laneid % 4

col =          i          for bi where i = {0,...,3}  if %laneid < 16
      i+4        for bi where i = {0,...,3}  otherwise
```

- Fragment layout for Column Major matrix B is shown in Figure 49.

The row and column of a matrix fragment can be computed as:

MMA computation 1

Row\Col	0	1	2	3	4	5	6	7
0	T0 : { b0, b1, b2, b3 }				T16 : { b0, b1, b2, b3 }			
..	↓				↓			
3	T3 : { b0, b1, b2, b3 }				T19 : { b0, b1, b2, b3 }			

MMA computation 2

Row\Col	0	1	2	3	4	5	6	7
0	T4 : { b0, b1, b2, b3 }				T20 : { b0, b1, b2, b3 }			
..	↓				↓			
3	T7 : { b0, b1, b2, b3 }				T23 : { b0, b1, b2, b3 }			

MMA computation 3

Row\Col	0	1	2	3	4	5	6	7
0	T8 : { b0, b1, b2, b3 }				T24 : { b0, b1, b2, b3 }			
..	↓				↓			
3	T11 : { b0, b1, b2, b3 }				T27 : { b0, b1, b2, b3 }			

MMA computation 4

Row\Col	0	1	2	3	4	5	6	7
0	T12 : { b0, b1, b2, b3 }				T28 : { b0, b1, b2, b3 }			
..	↓				↓			
3	T15 : { b0, b1, b2, b3 }				T31 : { b0, b1, b2, b3 }			

Figure 48: MMA .m8n8k4 fragment layout for row-major matrix B with .f16 type

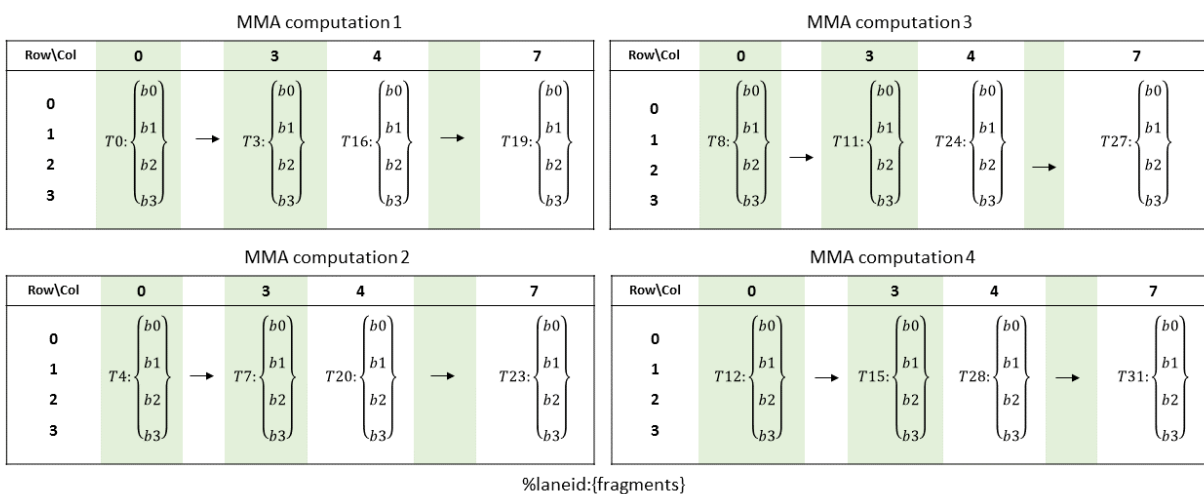


Figure 49: MMA .m8n8k4 fragment layout for column-major matrix B with .f16 type

```

row =      i          for bi   where i = {0,...,3}

col =      %laneid % 4    if %laneid < 16
      (%laneid % 4) + 4  otherwise

```

► Accumulators C (or D):

.ctype / .dtype	Fragment	Elements (low to high)
.f16	A vector expression containing four .f16x2 registers, with each register containing two .f16 elements from the matrix C (or D).	c0, c1, c2, c3, c4, c5, c6, c7
.f32	A vector expression of eight .f32 registers.	

The layout of the fragments held by different threads is shown below:

► Fragment layout for accumulator matrix when .ctype is .f16 is shown in Figure 50.

MMA computation 1

Row\Col	0	1	2	3	4	5	6	7
0	T0 : { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
3	T3: { c0, c1, c2, c3, c4, c5, c6, c7 }							
4	T16: { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
7	T19: { c0, c1, c2, c3, c4, c5, c6, c7 }							

MMA computation 3

Row\Col	0	1	2	3	4	5	6	7
0	T8 : { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
3	T11: { c0, c1, c2, c3, c4, c5, c6, c7 }							
4	T24: { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
7	T27: { c0, c1, c2, c3, c4, c5, c6, c7 }							

MMA computation 2

Row\Col	0	1	2	3	4	5	6	7
0	T4 : { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
3	T7: { c0, c1, c2, c3, c4, c5, c6, c7 }							
4	T20: { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
7	T23: { c0, c1, c2, c3, c4, c5, c6, c7 }							

MMA computation 4

Row\Col	0	1	2	3	4	5	6	7
0	T12 : { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
3	T15: { c0, c1, c2, c3, c4, c5, c6, c7 }							
4	T28: { c0, c1, c2, c3, c4, c5, c6, c7 }							
..								
7	T31: { c0, c1, c2, c3, c4, c5, c6, c7 }							

Figure 50: MMA .m8n8k4 fragment layout for matrix C/D with .ctype = .f16

The row and column of a matrix fragment can be computed as:

```

row =      %laneid % 4    if %laneid < 16
      (%laneid % 4) + 4  otherwise

col =      i          for ci   where i = {0,...,7}

```

► Fragment layout for accumulator matrix when .ctype is .f32 is shown in Figure 51 and Figure 52.

The row and column of a matrix fragment can be computed as:

MMA computation 1								
R\C	0	1	2	3	4	5	6	7
0	T0 : { c0, c1 }		T2 : { c0, c1 }		T0 : { c4, c5 }		T2 : { c4, c5 }	
1	T1 : { c0, c1 }		T3 : { c0, c1 }		T1 : { c4, c5 }		T3 : { c4, c5 }	
2	T0 : { c2, c3 }		T2 : { c2, c3 }		T0 : { c6, c7 }		T2 : { c6, c7 }	
3	T1 : { c2, c3 }		T3 : { c2, c3 }		T1 : { c6, c7 }		T3 : { c6, c7 }	
4	T16 : { c0, c1 }		T18 : { c0, c1 }		T16 : { c4, c5 }		T18 : { c4, c5 }	
5	T17 : { c0, c1 }		T19 : { c0, c1 }		T17 : { c4, c5 }		T19 : { c4, c5 }	
6	T16 : { c2, c3 }		T18 : { c2, c3 }		T16 : { c6, c7 }		T18 : { c6, c7 }	
7	T17 : { c2, c3 }		T19 : { c2, c3 }		T17 : { c6, c7 }		T19 : { c6, c7 }	

MMA computation 2								
R\C	0	1	2	3	4	5	6	7
0	T4 : { c0, c1 }		T6 : { c0, c1 }		T4 : { c4, c5 }		T6 : { c4, c5 }	
1	T5 : { c0, c1 }		T7 : { c0, c1 }		T5 : { c4, c5 }		T7 : { c4, c5 }	
2	T4 : { c2, c3 }		T6 : { c2, c3 }		T4 : { c6, c7 }		T6 : { c6, c7 }	
3	T5 : { c2, c3 }		T7 : { c2, c3 }		T5 : { c6, c7 }		T7 : { c6, c7 }	
4	T20 : { c0, c1 }		T22 : { c0, c1 }		T20 : { c4, c5 }		T22 : { c4, c5 }	
5	T21 : { c0, c1 }		T23 : { c0, c1 }		T21 : { c4, c5 }		T23 : { c4, c5 }	
6	T20 : { c2, c3 }		T22 : { c2, c3 }		T20 : { c6, c7 }		T22 : { c6, c7 }	
7	T21 : { c2, c3 }		T23 : { c2, c3 }		T21 : { c6, c7 }		T23 : { c6, c7 }	

Figure 51: MMA .m8n8k4 computation 1 and 2 fragment layout for matrix C/D with .ctype = .f32

MMA computation 3								
R\C	0	1	2	3	4	5	6	7
0	T8 : { c0, c1 }	T10 : { c0, c1 }	T8 : { c4, c5 }	T10 : { c4, c5 }				
1	T9 : { c0, c1 }	T11 : { c0, c1 }	T9 : { c4, c5 }	T11 : { c4, c5 }				
2	T8 : { c2, c3 }	T10 : { c2, c3 }	T8 : { c6, c7 }	T10 : { c6, c7 }				
3	T9 : { c2, c3 }	T11 : { c2, c3 }	T9 : { c6, c7 }	T11 : { c6, c7 }				
4	T24 : { c0, c1 }	T26 : { c0, c1 }	T24 : { c4, c5 }	T26 : { c4, c5 }				
5	T25 : { c0, c1 }	T27 : { c0, c1 }	T25 : { c4, c5 }	T27 : { c4, c5 }				
6	T24 : { c2, c3 }	T26 : { c2, c3 }	T24 : { c6, c7 }	T26 : { c6, c7 }				
7	T25 : { c2, c3 }	T27 : { c2, c3 }	T25 : { c6, c7 }	T27 : { c6, c7 }				

MMA computation 4								
R\C	0	1	2	3	4	5	6	7
0	T12 : { c0, c1 }	T14 : { c0, c1 }	T12 : { c4, c5 }	T14 : { c4, c5 }				
1	T13 : { c0, c1 }	T15 : { c0, c1 }	T13 : { c4, c5 }	T15 : { c4, c5 }				
2	T12 : { c2, c3 }	T14 : { c2, c3 }	T12 : { c6, c7 }	T14 : { c6, c7 }				
3	T13 : { c2, c3 }	T15 : { c2, c3 }	T13 : { c6, c7 }	T15 : { c6, c7 }				
4	T28 : { c0, c1 }	T30 : { c0, c1 }	T28 : { c4, c5 }	T30 : { c4, c5 }				
5	T29 : { c0, c1 }	T31 : { c0, c1 }	T29 : { c4, c5 }	T31 : { c4, c5 }				
6	T28 : { c2, c3 }	T30 : { c2, c3 }	T28 : { c6, c7 }	T30 : { c6, c7 }				
7	T29 : { c2, c3 }	T31 : { c2, c3 }	T29 : { c6, c7 }	T31 : { c6, c7 }				

Figure 52: MMA .m8n8k4 computation 3 and 4 fragment layout for matrix C/D with .ctype = .f32


```

row =      X      if %laneid < 16
      X + 4      otherwise

      where X = (%laneid & 0b1) + (i & 0b10) for ci where i = {0,...,7}
col = (i & 0b100) + (%laneid & 0b10) + (i & 0b1) for ci where i = {0,...,7}

```

9.7.14.5.2 Matrix Fragments for mma.m8n8k4 with .f64 floating point type

A warp executing mma.m8n8k4 with .f64 floating point type will compute an MMA operation of shape .m8n8k4.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.f64	A vector expression containing a single .f64 register, containing single .f64 element from the matrix A.	a0

The layout of the fragments held by different threads is shown in Figure 53.

Row\Col	0	1	2	3
0	T0:a0	T1:a0	T2:a0	T3:a0
1	T4:a0	T5:a0	T6:a0	T7:a0
2				
..				
7	T28:a0	T29:a0	T30:a0	T31:a0

%laneid:{fragments}

Figure 53: MMA .m8n8k4 fragment layout for matrix A with .f64 type

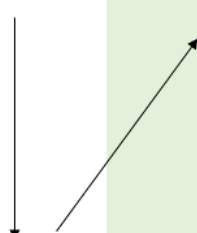
The row and column of a matrix fragment can be computed as:

```
row =    %laneid >> 2
col =    %laneid % 4
```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.f64	A vector expression containing a single .f64 register, containing a single .f64 element from the matrix B.	b0

The layout of the fragments held by different threads is shown in Figure 54.

Row\Column	0	1	2	..	7
0	T0:b0	T4:b0			T28:b0
1	T1:b0	T5:b0			T29:b0
2	T2:b0	T6:b0			T30:b0
3	T3:b0	T7:b0			T31:b0

%laneid:{fragments}

Figure 54: MMA .m8n8k4 fragment layout for matrix B with **.f64** type

The row and column of a matrix fragment can be computed as:

```
row =    %laneid % 4
col =    %laneid >> 2
```

► Accumulators (C or D):

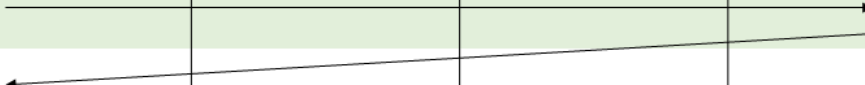
.ctype / .dtype	Fragment	Elements (low to high)
.f64	A vector expression containing of two .f64 registers containing two .f64 elements from the matrix C.	c0, c1

The layout of the fragments held by different threads is shown in Figure 55.

The row and column of a matrix fragment can be computed as:

```
groupID      = %laneid >> 2
threadID_in_group = %laneid % 4
```

(continues on next page)

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
..								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	

`%laneid:{fragments}`

Figure 55: MMA .m8n8k4 fragment layout for accumulator matrix C/D with .f64 type

(continued from previous page)

```

row =      groupID
col =      (threadID_in_group * 2) + (i & 0x1)      for ci  where i = {0, 1}

```

9.7.14.5.3 Matrix Fragments for `mma.m8n8k16`

A warp executing `mma.m8n8k16` will compute an MMA operation of shape .m8n8k16.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.s8 / .u8	A vector expression containing a single .b32 register, containing four .s8 or .u8 elements from the matrix A.	a0, a1, a2, a3

The layout of the fragments held by different threads is shown in Figure 56.

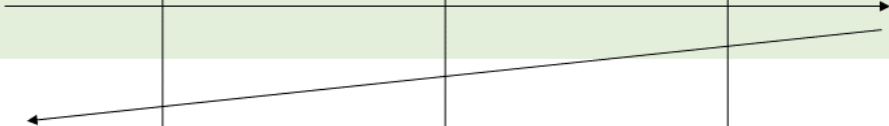
The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

```

(continues on next page)

Row \ Col	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:{a0, a1, a2, a3}				T1:{a0, a1, a2, a3}				T2:{a0, a1, a2, a3}				T3:{a0, a1, a2, a3}			
1	T4:{a0, a1, a2, a3}				T5:{a0, a1, a2, a3}				T6:{a0, a1, a2, a3}				T7:{a0, a1, a2, a3}			
2																
..																
7	T28:{a0, a1, a2, a3}				T29:{a0, a1, a2, a3}				T30:{a0, a1, a2, a3}				T31:{a0, a1, a2, a3}			

%laneid:{fragments}

Figure 56: MMA .m8n8k16 fragment layout for matrix A with .u8/.s8 type

(continued from previous page)

```

row = groupID
col = (threadID_in_group * 4) + i    for ai    where i = {0,...,3}

```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.s8 / .u8	A vector expression containing a single .b32 register, containing four .s8 or .u8 elements from the matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in [Figure 57](#).

The row and column of a matrix fragment can be computed as:

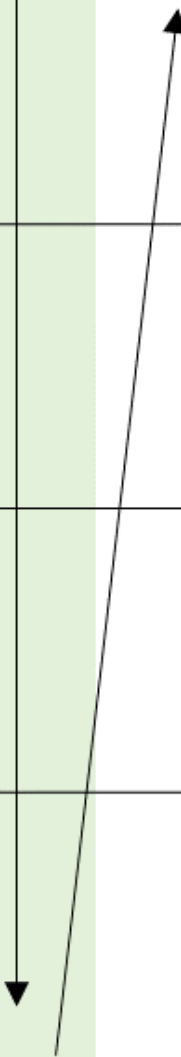
```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 4) + i    for bi    where i = {0,...,3}
col = groupID

```

► Accumulators (C or D):

Row\Col	0	1	2	..	7		
0	$T0: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$		
1							
2							
3							
4	$T1: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$					$T29: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
5							
6							
7							
8	$T2: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$					$T30: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
9							
10							
11							
12	$T3: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$					$T31: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
13							
14							
15							

`%laneid:{fragment}`

Figure 57: MMA .m8n8k16 fragment layout for matrix B with .u8/.s8 type

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing of two .s32 registers.	c0, c1

The layout of the fragments held by different threads is shown in Figure 58.

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}				
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}				
2								
..								
7	T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}				

%laneid:{fragments}

Figure 58: MMA .m8n8k16 fragment layout for accumulator matrix C/D with .s32 type

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row = groupID

col = (threadID_in_group * 2) + i      for ci    where i = {0, 1}

```

9.7.14.5.4 Matrix Fragments for mma.m8n8k32

A warp executing mma.m8n8k32 will compute an MMA operation of shape .m8n8k32.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.s4 / .u4	A vector expression containing a single .b32 register, containing eight .s4 or .u4 elements from the matrix A.	a0, a1, a2, a3, a4, a5, a6, a7

The layout of the fragments held by different threads is shown in Figure 59.

Row\Col	0 .. 7	8 ... 15	16 ... 23	24 ... 31
0	T0:{a0, a1, ..., a7}	T1:{a0, a1, ..., a7}	T2:{a0, a1, ..., a7}	T3:{a0, a1, ..., a7}
1	T4:{a0, a1, ..., a7}	T5:{a0, a1, ..., a7}	T6:{a0, a1, ..., a7}	T7:{a0, a1, ..., a7}
2				
..				
7	T28:{a0, a1, ..., a7}	T29:{a0, a1, ..., a7}	T30:{a0, a1, ..., a7}	T31:{a0, a1, ..., a7}

%laneid:{fragments}

Figure 59: MMA .m8n8k32 fragment layout for matrix A with .u4/.s4 type

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID
col = (threadID_in_group * 8) + i      for ai    where i = {0,...,7}

```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.s4 / .u4	A vector expression containing a single .b32 register, containing eight .s4 or .u4 elements from the matrix B.	b0, b1, b2, b3, b4, b5, b6, b7

The layout of the fragments held by different threads is shown in Figure 60.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 8) + i      for bi    where i = {0,...,7}
col = groupID

```

► Accumulators (C or D):

Row \ Col	0	1	2	..	7
0	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
..	$T0: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
7	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
8	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
..	$T1: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
15	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
16	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
..	$T2: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
23	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
24	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
..	$T3: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$
31	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$	$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$			$\begin{Bmatrix} b0 \\ b1 \\ \vdots \\ b6 \\ b7 \end{Bmatrix}$

`%laneid:{fragments}`

Figure 60: MMA .m8n8k32 fragment layout for matrix B with .u4/.s4 type

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression of two .s32 registers.	c0, c1

The layout of the fragments held by different threads is shown in Figure 61:

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
..								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	

`%laneid:{fragments}`

Figure 61: MMA .m8n8k32 fragment layout for accumulator matrix C/D with .s32 type

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = groupID
col = (threadID_in_group * 2) + i    for ci where i = {0, 1}

```

9.7.14.5.5 Matrix Fragments for `mma.m8n8k128`

A warp executing `mma.m8n8k128` will compute an MMA operation of shape .m8n8k128.

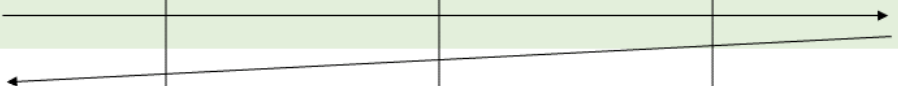
Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.b1	A vector expression containing a single .b32 register, containing thirty two .b1 elements from the matrix A.	a0, a1, ... a30, a31

The layout of the fragments held by different threads is shown in Figure 62.

The row and column of a matrix fragment can be computed as:

R \ C	0 1 .. 31	32 33 .. 63	64 65 .. 95	96 97 .. 127
0	T0:{a0, a1, .. a31}	T1:{a0, a1, .. a31}	T2:{a0, a1, .. a31}	T3:{a0, a1, .. a31}
1	T4:{a0, a1, .. a31}	T5:{a0, a1, .. a31}	T6:{a0, a1, .. a31}	T7:{a0, a1, .. a31}
2				
..				
7	T28:{a0, a1, .. a31}	T29:{a0, a1, .. a31}	T30:{a0, a1, .. a31}	T31:{a0, a1, .. a31}

%laneid:{fragments}

Figure 62: MMA .m8n8k128 fragment layout for matrix A with .b1 type.

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = groupID
col = (threadID_in_group * 32) + i      for ai where i = {0,..,31}

```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.b1	A vector expression containing a single .b32 register, containing thirty two .b1 elements from the matrix B.	b0, b1, ..., b30, b31

The layout of the fragments held by different threads is shown in [Figure 63](#).

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 32) + i      for bi where i = {0,..,31}
col = groupID

```

► Accumulators (C or D):

Row\Col	0	1	2	..	7
0	$T0: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
1					
..					
31					
32	$T1: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
33					
..					
63					
64	$T2: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
65					
..					
95					
96	$T3: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
97					
..					
127					

%laneid:{fragment}

Figure 63: MMA .m8n8k128 fragment layout for matrix B with .b1 type.

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing two .s32 registers, containing two .s32 elements from the matrix C (or D).	c0, c1

The layout of the fragments held by different threads is shown in Figure 64.

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}				
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}				
2								
..								
7	T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}				

%laneid:{fragments}

Figure 64: MMA .m8n8k128 fragment layout for accumulator matrix C/D with .s32 type

The row and column of a matrix fragment can be computed as:

```
groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID
col = (threadID_in_group * 2) + i    for ci where i = {0, 1}
```

9.7.14.5.6 Matrix Fragments for mma.m16n8k4

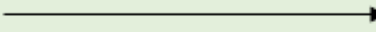
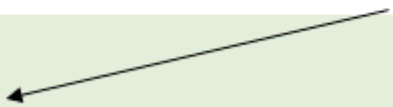
A warp executing mma.m16n8k4 will compute an MMA operation of shape .m16n8k4.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

- Multiplicand A:
 - .tf32:

.atype	Fragment	Elements (low to high)
.tf32	A vector expression containing two .b32 registers, containing two .tf32 elements from the matrix A.	a0, a1

The layout of the fragments held by different threads is shown in [Figure 65](#).

Row\Col	0	1	2	3
0	T0:a0	T1:a0	T2:a0	T3:a0
1	T4:a0	T5:a0	T6:a0	T7:a0
2				
..				
7	T28:a0	T29:a0	T30:a0	T31:a0
8	T0:a1	T1:a1	T2:a1	T3:a1
9	T4:a1	T5:a1	T6:a1	T7:a1
10				
..				
15	T28:a1	T29:a1	T30:a1	T31:a1

%laneid:{fragments}

Figure 65: MMA .m16n8k4 fragment layout for matrix A with .tf32 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for a0
        groupID + 8    for a1

col =  threadID_in_group

```

► .f64:

.atype	Fragment	Elements (low to high)
.f64	A vector expression containing two .f64 registers, containing two .f64 elements from the matrix A.	a0, a1

The layout of the fragments held by different threads is shown in [Figure 66](#).

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for a0
        groupID + 8    for a1

col =  threadID_in_group

```

► Multiplicand B:

► .tf32:

.btype	Fragment	Elements (low to high)
.tf32	A vector expression of a single .b32 register, containing a single .tf32 element from the matrix B.	b0

The layout of the fragments held by different threads is shown in [Figure 67](#).

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

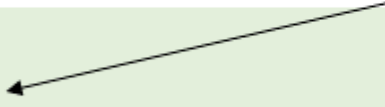
row =  threadID_in_group

col =  groupID

```

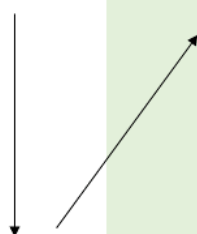
► .f64:

.btype	Fragment	Elements (low to high)
.f64	A vector expression of a single .f64 register, containing a single .f64 element from the matrix B.	b0

Row\Col	0	1	2	3
0	T0:a0	T1:a0	T2:a0	T3:a0
1	T4:a0	T5:a0	T6:a0	T7:a0
2				
..				
7	T28:a0	T29:a0	T30:a0	T31:a0
8	T0:a1	T1:a1	T2:a1	T3:a1
9	T4:a1	T5:a1	T6:a1	T7:a1
10				
..				
15	T28:a1	T29:a1	T30:a1	T31:a1

%laneid:{fragments}

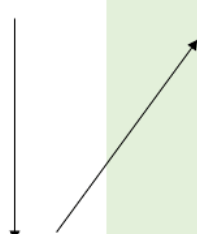
Figure 66: MMA .m16n8k4 fragment layout for matrix A with .f64 type.

Row\Column	0	1	2	..	7
0	T0:b0	T4:b0			T28:b0
1	T1:b0	T5:b0			T29:b0
2	T2:b0	T6:b0			T30:b0
3	T3:b0	T7:b0			T31:b0

%laneid:{fragments}

Figure 67: MMA .m16n8k4 fragment layout for matrix B with .tf32 type.

The layout of the fragments held by different threads is shown in [Figure 68](#).

Row\Column	0	1	2	..	7
0	T0:b0	T4:b0			T28:b0
1	T1:b0	T5:b0			T29:b0
2	T2:b0	T6:b0			T30:b0
3	T3:b0	T7:b0			T31:b0

%laneid:{fragments}

Figure 68: MMA .m16n8k4 fragment layout for matrix B with .f64 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = threadID_in_group
col = groupID

```

► Accumulators (C or D):

► .tf32:

.ctype / .dtype	Fragment	Elements (low to high)
.f32	A vector expression containing four .f32 registers, containing four .f32 elements from the matrix C (or D).	c0, c1, c2, c3

The layout of the fragments held by different threads is shown in [Figure 69](#).

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
..								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
..								
15	T28: {c2, c3}		T29: {c2, c3}		T30: {c2, c3}		T31: {c2, c3}	

%laneid:{fragments}

Figure 69: MMA .m16n8k4 fragment layout for accumulator matrix C/D with **.f32** type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID
          groupID + 8      for c0 and c1
                           for c2 and c3

col = (threadID_in_group * 2) + (i & 0x1)  for ci  where i = {0, ...,
↪ 3}

```

► **.f64:**

.ctype / .dtype	Fragment	Elements (low to high)
.f64	A vector expression containing four .f64 registers, containing four .f64 elements from the matrix C (or D).	c0, c1, c2, c3

The layout of the fragments held by different threads is shown in [Figure 70](#).

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
..								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
..								
15	T28: {c2, c3}		T29: {c2, c3}		T30: {c2, c3}		T31: {c2, c3}	

%laneid:{fragments}

Figure 70: MMA .m16n8k4 fragment layout for accumulator matrix C/D with .f64 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for c0 and c1
          groupID + 8    for c2 and c3

col = (threadID_in_group * 2) + (i & 0x1) for ci where i = {0, ...,
↪ 3}

```

9.7.14.5.7 Matrix Fragments for `mma.m16n8k8`

A warp executing `mma.m16n8k8` will compute an MMA operation of shape `.m16n8k8`.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

► `.f16` and `.bf16`:

.atype	Fragment	Elements (low to high)
<code>.f16</code> / <code>.bf16</code>	A vector expression containing two <code>.f16x2</code> registers, with each register containing two <code>.f16</code> / <code>.bf16</code> elements from the matrix A.	a0, a1, a2, a3

The layout of the fragments held by different threads is shown in Figure 71.

Row\Col	0	1	2	3	4	5	6	7
0	T0: {a0, a1}	T1: {a0, a1}	T2: {a0, a1}	T3: {a0, a1}				
1	T4: {a0, a1}	T5: {a0, a1}	T6: {a0, a1}	T7: {a0, a1}				
2								
..								
7	T28: {a0, a1}	T29: {a0, a1}	T30: {a0, a1}	T31: {a0, a1}				
8	T0: {a2, a3}	T1: {a2, a3}	T2: {a2, a3}	T3: {a2, a3}				
9	T4: {a2, a3}	T5: {a2, a3}	T6: {a2, a3}	T7: {a2, a3}				
10								
..								
15	T28: {a2, a3}	T29: {a2, a3}	T30: {a2, a3}	T31: {a2, a3}				

`%laneid:{fragments}`

Figure 71: MMA `.m16n8k8` fragment layout for matrix A with `.f16` / `.bf16` type.

The row and column of a matrix fragment can be computed as:

```

groupId          = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupId          for a0 and a1
        groupId + 8        for a2 and a3

col =  threadID_in_group * 2 + (i & 0x1)  for ai    where i = {0, ...,
↪ 3}

```

► .tf32:

.atype	Fragment	Elements (low to high)
.tf32	A vector expression containing four .b32 registers, containing four .tf32 elements from the matrix A.	a0, a1, a2, a3

The layout of the fragments held by different threads is shown in Figure 72.

Row\Col	0	1	2	3	4	5	6	7
0	T0:a0	T1:a0	T2:a0	T3:a0	T0:a2	T1:a2	T2:a2	T3:a2
1	T4:a0	T5:a0	T6:a0	T7:a0	T4:a2	T5:a2	T6:a2	T7:a2
2	→				→			
..	↙				↙			
7	T28:a0	T29:a0	T30:a0	T31:a0	T28:a2	T29:a2	T30:a2	T31:a2
8	T0:a1	T1:a1	T2:a1	T3:a1	T0:a3	T1:a3	T2:a3	T3:a3
9	T4:a1	T5:a1	T6:a1	T7:a1	T4:a3	T5:a3	T6:a3	T7:a3
10	→				→			
..	↙				↙			
15	T28:a1	T29:a1	T30:a1	T31:a1	T28:a3	T29:a3	T30:a3	T31:a3

%laneid:{fragments}

Figure 72: MMA .m16n8k8 fragment layout for matrix A with .tf32 type.

The row and column of a matrix fragment can be computed as:

```

groupId      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupId      for a0 and a2
          groupId + 8   for a1 and a3

col =  threadID_in_group for a0 and a1
      threadID_in_group + 4 for a2 and a3

```

► .f64:

.atype	Fragment	Elements (low to high)
.f64	A vector expression containing four .f64 registers, containing four .f64 elements from the matrix A.	a0, a1, a2, a3

The layout of the fragments held by different threads is shown in Figure 73.

Row\Col	0	1	2	3	4	5	6	7
0	T0:a0	T1:a0	T2:a0	T3:a0	T0:a2	T1:a2	T2:a2	T3:a2
1	T4:a0	T5:a0	T6:a0	T7:a0	T4:a2	T5:a2	T6:a2	T7:a2
2	→				→			
..	↖				↖			
7	T28:a0	T29:a0	T30:a0	T31:a0	T28:a2	T29:a2	T30:a2	T31:a2
8	T0:a1	T1:a1	T2:a1	T3:a1	T0:a3	T1:a3	T2:a3	T3:a3
9	T4:a1	T5:a1	T6:a1	T7:a1	T4:a3	T5:a3	T6:a3	T7:a3
10	→				→			
..	↖				↖			
15	T28:a1	T29:a1	T30:a1	T31:a1	T28:a3	T29:a3	T30:a3	T31:a3

%laneid:{fragments}

Figure 73: MMA .m16n8k8 fragment layout for matrix A with .f64 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for a0 and a2
          groupID + 8    for a1 and a3

col =  threadID_in_group for a0 and a1
      threadID_in_group + 4 for a2 and a3

```

► Multiplicand B:

► .f16 and .bf16 :

.btype	Fragment	Elements (low to high)
.f16 / .bf16	A vector expression containing a single .f16x2 register, containing two .f16 / .bf16 elements from the matrix B.	b0, b1

The layout of the fragments held by different threads is shown in Figure 74.

Row\Column	0	1	2	..	7
0	$T0: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$
1					
2	$T1: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$
3					
4	$T2: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$
5					
6	$T3: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \end{Bmatrix}$
7					

$\%laneid:\{\text{fragments}\}$

Figure 74: MMA .m16n8k8 fragment layout for matrix B with .f16 / .bf16 type.

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 2) + i      for bi    where i = {0, 1}
col = groupID

```

► .tf32:

.btype	Fragment	Elements (low to high)
.tf32	A vector expression containing two .b32 registers, containing two .tf32 elements from the matrix B.	b0, b1

The layout of the fragments held by different threads is shown in Figure 75.

Row\Column	0	1	2	..	7
0	T0:b0	T4:b0	↓ ↗		T28:b0
1	T1:b0	T5:b0			T29:b0
2	T2:b0	T6:b0			T30:b0
3	T3:b0	T7:b0			T31:b0
4	T0:b1	T4:b1	↓ ↗		T28:b1
5	T1:b1	T5:b1			T29:b1
6	T2:b1	T6:b1			T30:b1
7	T3:b1	T7:b1			T31:b1

%laneid:{fragments}

Figure 75: MMA .m16n8k8 fragment layout for matrix B with .tf32 type.

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row =      threadID_in_group      for b0
        threadID_in_group + 4    for b1
col = groupID

```

► .f64:

.btype	Fragment	Elements (low to high)
.f64	A vector expression containing two .f64 registers, containing two .f64 elements from the matrix B.	b0, b1

The layout of the fragments held by different threads is shown in Figure 76.

Row\Column	0	1	2	..	7
0	T0:b0	T4:b0			T28:b0
1	T1:b0	T5:b0			T29:b0
2	T2:b0	T6:b0			T30:b0
3	T3:b0	T7:b0			T31:b0
4	T0:b1	T4:b1			T28:b1
5	T1:b1	T5:b1			T29:b1
6	T2:b1	T6:b1			T30:b1
7	T3:b1	T7:b1			T31:b1

`%laneid:{fragments}`

Figure 76: MMA .m16n8k8 fragment layout for matrix B with .f64 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      threadID_in_group      for b0
      threadID_in_group + 4      for b1
col = groupID

```

► Accumulators (C or D):

► .f16, .bf16 and .tf32:

.ctype / .dtype	Fragment	Elements (low to high)
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix C (or D).	c0, c1, c2, c3
.f32	A vector expression of four .f32 registers.	

The layout of the fragments held by different threads is shown in [Figure 77](#).

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
..								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
..								
15	T28: {c2, c3}		T29: {c2, c3}		T30: {c2, c3}		T31: {c2, c3}	

%laneid:{fragments}

Figure 77: MMA .m16n8k8 fragment layout for accumulator matrix C/D with .f16x2/.f32 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID
        groupID + 8      for c0 and c1
                        for c2 and c3

col = (threadID_in_group * 2) + (i & 0x1)  for ci  where i = {0, ...,
↪ 3}

```

► .f64:

.ctype / .dtype	Fragment	Elements (low to high)
<code>.f64</code>	A vector expression of four <code>.f64</code> registers containing four <code>.f64</code> elements from the matrix C (or D).	<code>c0, c1, c2, c3</code>

The layout of the fragments held by different threads is shown in [Figure 78](#).

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}				
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}				
2								
..								
7	T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}				
8	T0: {c2, c3}	T1: {c2, c3}	T2: {c2, c3}	T3: {c2, c3}				
9	T4: {c2, c3}	T5: {c2, c3}	T6: {c2, c3}	T7: {c2, c3}				
10								
..								
15	T28: {c2, c3}	T29: {c2, c3}	T30: {c2, c3}	T31: {c2, c3}				

%laneid:{fragments}

Figure 78: MMA .m16n8k8 fragment layout for accumulator matrix C/D with `.f64` type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for c0 and c1
          groupID + 8    for c2 and c3

col = (threadID_in_group * 2) + (i & 0x1)  for ci  where i = {0, ...,
↪ 3}

```

9.7.14.5.8 Matrix Fragments for `mma.m16n8k16` with floating point type

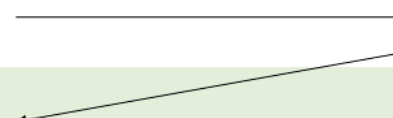
A warp executing `mma.m16n8k16` floating point types will compute an MMA operation of shape `.m16n8k16`.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

- Multiplicand A:
 - `.f16` and `.bf16`:

<code>.atype</code>	Fragment	Elements (low to high)
<code>.f16</code> / <code>.bf16</code>	A vector expression containing four <code>.f16x2</code> registers, with each register containing two <code>.f16</code> / <code>.bf16</code> elements from the matrix A.	a0, a1, a2, a3, a4, a5, a6, a7

The layout of the fragments held by different threads is shown in Figure 79.

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:{a0,a1}	T1:{a0,a1}	T2:{a0,a1}	T3:{a0,a1}					T0:{a4,a5}	T1:{a4,a5}	T2:{a4,a5}	T3:{a4,a5}				
1	T4:{a0,a1}	T5:{a0,a1}	T6:{a0,a1}	T7:{a0,a1}					T4:{a4,a5}	T5:{a4,a5}	T6:{a4,a5}	T7:{a4,a5}				
2																
..																
7	T28:{a0,a1}	T29:{a0,a1}	T30:{a0,a1}	T31:{a0,a1}					T28:{a4,a5}	T29:{a4,a5}	T30:{a4,a5}	T31:{a4,a5}				
8	T0:{a2,a3}	T1:{a2,a3}	T2:{a2,a3}	T3:{a2,a3}					T0:{a6,a7}	T1:{a6,a7}	T2:{a6,a7}	T3:{a6,a7}				
9	T4:{a2,a3}	T5:{a2,a3}	T6:{a2,a3}	T7:{a2,a3}					T4:{a6,a7}	T5:{a6,a7}	T6:{a6,a7}	T7:{a6,a7}				
10																
..																
15	T28:{a2,a3}	T29:{a2,a3}	T30:{a2,a3}	T31:{a2,a3}					T28:{a6,a7}	T29:{a6,a7}	T30:{a6,a7}	T31:{a6,a7}				

%laneid:{fragments}

Figure 79: MMA `.m16n8k16` fragment layout for matrix A with `.f16` / `.bf16` type.

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row =            groupID          for ai where 0 <= i < 2 || 4 <= i < 6
            groupID + 8          Otherwise

col = (threadID_in_group * 2) + (i & 0x1)          for ai where i < 4
      (threadID_in_group * 2) + (i & 0x1) + 8      for ai where i >= 4

```

► .f64:

.atype	Fragment	Elements (low to high)
.f64	A vector expression containing eight .f64 registers, with each register containing one .f64 element from the matrix A.	a0, a1, a2, a3, a4, a5, a6, a7

The layout of the fragments held by different threads is shown in Figure 80.

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:a0	T1:a0	T2:a0	T3:a0	T0:a2	T1:a2	T2:a2	T3:a2	T0:a4	T1:a4	T2:a4	T3:a4	T0:a6	T1:a6	T2:a6	T3:a6
1	T4:a0	T5:a0	T6:a0	T7:a0	T4:a2	T5:a2	T6:a2	T7:a2	T4:a4	T5:a4	T6:a4	T7:a4	T4:a6	T5:a6	T6:a6	T7:a6
2																
..																
7	T28:a0	T29:a0	T30:a0	T31:a0	T28:a2	T29:a2	T30:a2	T31:a2	T28:a4	T29:a4	T30:a4	T31:a4	T28:a6	T29:a6	T30:a6	T31:a6
8	T0:a1	T1:a1	T2:a1	T3:a1	T0:a3	T1:a3	T2:a3	T3:a3	T0:a5	T1:a5	T2:a5	T3:a5	T0:a7	T1:a7	T2:a7	T3:a7
9	T4:a1	T5:a1	T6:a1	T7:a1	T4:a3	T5:a3	T6:a3	T7:a3	T4:a5	T5:a5	T6:a5	T7:a5	T4:a7	T5:a7	T6:a7	T7:a7
10																
..																
15	T28:a1	T29:a1	T30:a1	T31:a1	T28:a3	T29:a3	T30:a3	T31:a3	T28:a5	T29:a5	T30:a5	T31:a5	T28:a7	T29:a7	T30:a7	T31:a7

laneid:{fragments}

Figure 80: MMA .m16n8k16 fragment layout for matrix A with .f64 type.

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row =            groupID
            groupID + 8          for ai where i % 2 = 0
                                Otherwise

```

(continues on next page)

(continued from previous page)

```

col = (i * 2) + threadID_in_group      for ai where i % 2 = 0
      (i * 2) - 2 + (threadID_in_group Otherwise

```

► Multiplicand B:

► .f16 and .bf16 :

.btype	Fragment	Elements (low to high)
.f16 / .bf16	A vector expression containing two .f16x2 registers, with each register containing two .f16 / .bf16 elements from the matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in Figure 81.

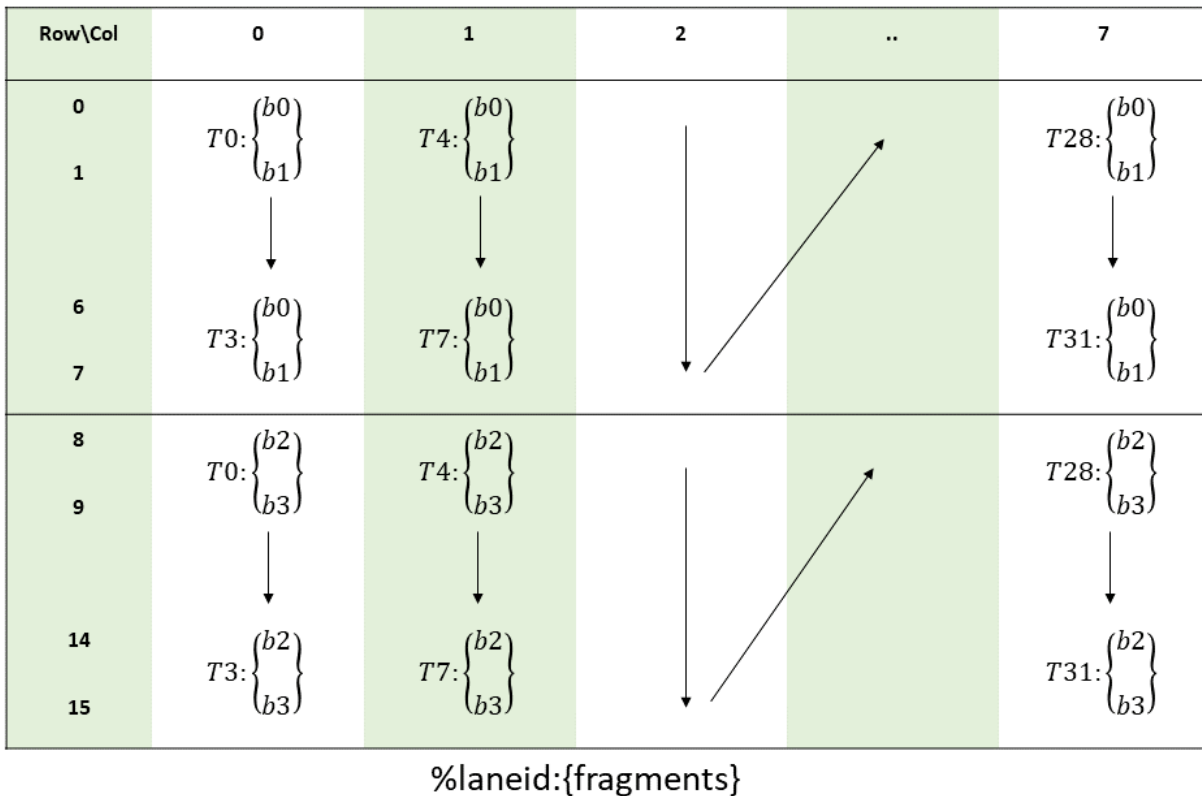


Figure 81: MMA .m16n8k16 fragment layout for matrix B with .f16 / .bf16 type.

where the row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 2) + (i & 0x1)      for bi where i <
↪2      (threadID_in_group * 2) + (i & 0x1) + 8  for bi where i >=
↪2      (continues on next page)

```

(continued from previous page)

```
col = groupID
```

► .f64:

.atype	Fragment	Elements (low to high)
.f64	A vector expression containing four .f64 registers, with each register containing one .f64 element from the matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in Figure 82.

Row\Column	0	1	2	...	7
0	T0:B0	T4:B0	↓		T28:B0
1	T1:B0	T5:B0			T29:B0
2	T2:B0	T6:B0			T30:B0
3	T3:B0	T7:B0			T31:B0
4	T0:B1	T4:B1	↓		T28:B1
5	T1:B1	T5:B1			T29:B1
6	T2:B1	T6:B1			T30:B1
7	T3:B1	T7:B1			T31:B1
8	T0:B2	T4:B2	↓		T28:B2
9	T1:B2	T5:B2			T29:B2
10	T2:B2	T6:B2			T30:B2
11	T3:B2	T7:B2			T31:B2
12	T0:B3	T4:B3	↓		T28:B3
13	T1:B3	T5:B3			T29:B3
14	T2:B3	T6:B3			T30:B3
15	T3:B3	T7:B3			T31:B3

%laneid:{fragments}

Figure 82: MMA .m16n8k16 fragment layout for matrix B with .f64 type.

The row and column of a matrix fragment can be computed as:

```
groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row = threadID_in_group + (i * 4)      for bi where i < 4
col = groupID
```

► Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.f64	A vector expression containing four .f64 registers containing .f64 elements from the matrix C (or D).	c0, c1, c2, c3
.f32	A vector expression containing four .f32 registers containing four .f32 elements from the matrix C (or D).	
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix C (or D).	

The layout of the fragments held by different threads is shown in [Figure 83](#).

Row\Col	0	1	2	3	4	5	6	7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}				
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}				
2								
..								
7	T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}				
8	T0: {c2, c3}	T1: {c2, c3}	T2: {c2, c3}	T3: {c2, c3}				
9	T4: {c2, c3}	T5: {c2, c3}	T6: {c2, c3}	T7: {c2, c3}				
10								
..								
15	T28: {c2, c3}	T29: {c2, c3}	T30: {c2, c3}	T31: {c2, c3}				

%laneid:{fragments}

Figure 83: MMA .m16n8k16 fragment layout for accumulator matrix matrix C/D.

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row =            groupID
           groupID + 8
col = (threadID_in_group * 2) + (i & 0x1)

for ci where i < 2
for ci where i >= 2
for ci where i = {0,...,3}

```

9.7.14.5.9 Matrix Fragments for `mma.m16n8k16` with integer type

A warp executing `mma.m16n8k16` will compute an MMA operation of shape `.m16n8k16`.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.u8 / .s8	A vector expression containing two <code>.b32</code> registers, with each register containing four <code>.u8 / .s8</code> elements from the matrix A.	a0, a1, a2, a3, a4, a5, a6, a7
.e4m3 / .e5m2	A vector expression containing two <code>.b32</code> registers, with each register containing four <code>.e4m3 / .e5m2</code> elements from the matrix A.	a0, a1, a2, a3, a4, a5, a6, a7

The layout of the fragments held by different threads is shown in Figure 84.

R \ C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:{a0, a1, a2, a3}				T1:{a0, a1, a2, a3}				T2:{a0, a1, a2, a3}				T3:{a0, a1, a2, a3}			
1	T4:{a0, a1, a2, a3}				T5:{a0, a1, a2, a3}				T6:{a0, a1, a2, a3}				T7:{a0, a1, a2, a3}			
2																
..																
7	T28:{a0, a1, a2, a3}				T29:{a0, a1, a2, a3}				T30:{a0, a1, a2, a3}				T31:{a0, a1, a2, a3}			
8	T0:{a4, a5, a6, a7}				T1:{a4, a5, a6, a7}				T2:{a4, a5, a6, a7}				T3:{a4, a5, a6, a7}			
9	T4:{a4, a5, a6, a7}				T5:{a4, a5, a6, a7}				T6:{a4, a5, a6, a7}				T7:{a4, a5, a6, a7}			
10																
..																
15	T28:{a4, a5, a6, a7}				T29:{a4, a5, a6, a7}				T30:{a4, a5, a6, a7}				T31:{a4, a5, a6, a7}			

`%laneid:{fragments}`

Figure 84: MMA `.m16n8k16` fragment layout for matrix A with `.u8 / .s8` type.

The row and column of a matrix fragment can be computed as:

```

groupId          = %laneid >> 2
threadID_in_group = %laneid % 4

row =            groupId
      groupId + 8

for ai where i < 4
for ai where i >= 4

```

(continues on next page)

(continued from previous page)

```
col = (threadID_in_group * 4) + (i & 0x3)    for ai where i = {0,...,7}
```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.u8 / .s8	A vector expression containing a single .b32 register, containing four .u8 / .s8 elements from the matrix B.	b0, b1, b2, b3
.e4m3 / .e5m2	A vector expression containing a single .b32 register, containing four .e4m3 / .e5m2 elements from the matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in [Figure 85](#).

The row and column of a matrix fragment can be computed as:

```
groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 4) + i    for bi where i = {0,...,3}
col = groupID
```

► Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing four .s32 registers, containing four .s32 elements from the matrix C (or D).	c0, c1, c2, c3
.f32	A vector expression containing four .f32 registers, containing four .f32 elements from the matrix C (or D).	c0, c1, c2, c3
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix C (or D).	c0, c1, c1, c2

The layout of the fragments held by different threads is shown in [Figure 86](#).

The row and column of a matrix fragment can be computed as:

```
groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID          for ci where i < 2
      groupID + 8          for ci where i >= 2

col = (threadID_in_group * 2) + (i & 0x1)    for ci where i = {0,...,3}
```

Row\Col	0	1	2	..	7
0	$T0: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
1					
2					
3					
4	$T1: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
5					
6					
7					
8	$T2: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
9					
10					
11					
12	$T3: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$
13					
14					
15					

`%laneid:{fragment}`

Figure 85: MMA .m16n8k16 fragment layout for matrix B with .u8 / .s8 type.

R\C	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
7								
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
15								

Figure 86: MMA .m16n8k16 fragment layout for accumulator matrix C/D with .s32 type.

9.7.14.5.10 Matrix Fragments for mma.m16n8k32

A warp executing `mma.m16n8k32` will compute an MMA operation of shape .m16n8k32.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

- Multiplicand A:
 - .s4 or .u4 :

.atype	Fragment	Elements (low to high)
.s4 / .u4	A vector expression containing two .b32 registers, with each register containing eight .u4 / .s4 elements from the matrix A.	a0, a1, ..., a14, a15

The layout of the fragments held by different threads is shown in Figure 87.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for ai where i < 8
      groupID + 8      for ai where i >= 8

col = (threadID_in_group * 8) + (i & 0x7)  for ai where i = {0, ...,
↪ 15}

```

R\C	0 ... 7	8 ... 15	16 ... 23	24 ... 31
0	T0:{a0, .., a7}	T1:{a0, .., a7}	T2:{a0, .., a7}	T3:{a0, .., a7}
1	T4:{a0, .., a7}	T5:{a0, .., a7}	T6:{a0, .., a7}	T7:{a0, .., a7}
2				
..				
7	T28:{a0, .., a7}	T29:{a0, .., a7}	T30:{a0, .., a7}	T31:{a0, .., a7}
8	T0:{a8, .., a15}	T1:{a8, .., a15}	T2:{a8, .., a15}	T3:{a8, .., a15}
9	T4:{a8, .., a15}	T5:{a8, .., a15}	T6:{a8, .., a15}	T7:{a8, .., a15}
10				
..				
15	T28:{a8, .., a15}	T29:{a8, .., a15}	T30:{a8, .., a15}	T31:{a8, .., a15}

`%laneid:{fragments}`

Figure 87: MMA .m16n8k32 fragment layout for matrix A with .u4 / .s4 type.

► .s8 or .u8 or .e4m3 or .e5m2 or .e3m2 or .e2m3 or .e2m1:

.atype	Fragment	Elements (low to high)
.s8 / .u8	A vector expression containing four .b32 registers, with each register containing four .s8 / .u8 elements from the matrix A.	a0, a1, ..., a14, a15
.e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1	A vector expression containing four .b32 registers, with each register containing four .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 elements from the matrix A.	a0, a1, ..., a14, a15

The layout of the fragments held by different threads is shown in [Figure 88](#).

The row and column of a matrix fragment can be computed as:

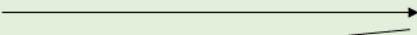
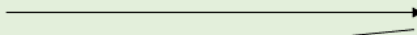
```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =    groupID                                for ai where 0
    <=& i < 4 || 8 <= i < 12                    otherwise
        groupID + 8

col =    (threadID_in_group * 4) + (i & 0x3)      for ai where i
    < 8
  
```

(continues on next page)

R\C	0	...	4	5	...	7	8	...	11	12	...	15	16	..	19	20	..	23	24	..	27	28	..	31								
0	T0:{a0, a1, ..., a3}				T1:{a0, a1, ..., a3}				T2:{a0, a1, ..., a3}				T3:{a0, a1, ..., a3}				T0:{a8, ..., a11}				T1:{a8, ..., a11}				T2:{a8, ..., a11}				T3:{a8, ..., a11}			
1	T4:{a0, a1, ..., a3}				T5:{a0, a1, ..., a3}				T6:{a0, a1, ..., a3}				T7:{a0, a1, ..., a3}				T4:{a8, ..., a11}				T5:{a8, ..., a11}				T6:{a8, ..., a11}				T7:{a8, ..., a11}			
2																																
..																																
7	T28:{a0, a1, ..., a3}				T29:{a0, a1, ..., a3}				T30:{a0, a1, ..., a3}				T31:{a0, a1, ..., a3}				T28:{a8, ..., a11}				T29:{a8, ..., a11}				T30:{a8, ..., a11}				T31:{a8, ..., a11}			
8	T0:{a4, ..., a7}				T1:{a4, ..., a7}				T2:{a4, ..., a7}				T3:{a4, ..., a7}				T0:{a12, ..., a15}				T1:{a12, ..., a15}				T2:{a12, ..., a15}				T3:{a12, ..., a15}			
9	T4:{a4, ..., a7}				T5:{a4, ..., a7}				T6:{a4, ..., a7}				T7:{a4, ..., a7}				T4:{a12, ..., a15}				T5:{a12, ..., a15}				T6:{a12, ..., a15}				T7:{a12, ..., a15}			
10																																
..																																
15	T28:{a4, ..., a7}				T29:{a4, ..., a7}				T30:{a4, ..., a7}				T31:{a4, ..., a7}				T28:{a12, ..., a15}				T29:{a12, ..., a15}				T30:{a12, ..., a15}				T31:{a12, ..., a15}			

%laneid:{fragments}

Figure 88: MMA .m16n8k32 fragment layout for matrix A with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type.

(continued from previous page)

```

    (threadID_in_group * 4) + (i & 0x3) + 16    for ai where i >
    ↪ = 8

```

► Multiplicand B:

► .s4 or .u4 :

.btype	Fragment	Elements (low to high)
.s4 / .u4	A vector expression containing a single .b32 register, containing eight .s4 / .u4 elements from the matrix B.	b0, b1, b2, b3, b4, b5, b6, b7

The layout of the fragments held by different threads is shown in [Figure 89](#).

The row and column of a matrix fragment can be computed as:

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 8) + (i & 0x7)    for bi where i = {0, ..., 7}
col = groupID

```

► .s8 or .u8 or .e4m3 or .e5m2 or .e3m2 or .e2m3 or .e2m1:

Row \ Col	0	1	2	..	7
0 .. 7	$T0: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$
8 .. 15	$T1: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$
16 .. 23	$T2: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$
24 .. 31	$T3: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$

Figure 89: MMA .m16n8k32 fragment layout for matrix B with .u4 / .s4 type.

.btype	Fragment	Elements (low to high)
.s8 / .u8	A vector expression containing two .b32 registers, with each register containing four .s8 / .u8 elements from the matrix B.	b0, b1, b2, b3, b4, b5, b6, b7
.e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1	A vector expression containing two .b32 registers, with each register containing four .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 elements from the matrix B.	b0, b1, b2, b3, b4, b5, b6, b7

The layout of the fragments held by different threads is shown in [Figure 90](#) and [Figure 91](#).

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =        (threadID_in_group * 4) + (i & 0x3)      for bi where
↪ i < 4      (threadID_in_group * 4) + (i & 0x3) + 16  for bi where
↪ i >= 4

col =    groupID

```

► Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing four .s32 registers, containing four .s32 elements from the matrix C (or D).	c0, c1, c2, c3
.f32	A vector expression containing four .f32 registers, containing four .f32 elements from the matrix C (or D).	c0, c1, c2, c3
.f16	A vector expression containing two .f16x2 registers, with each register containing two .f16 elements from the matrix C (or D).	c0, c1, c2, c3

The layout of the fragments held by different threads is shown in [Figure 92](#).

The row and column of a matrix fragment can be computed as:

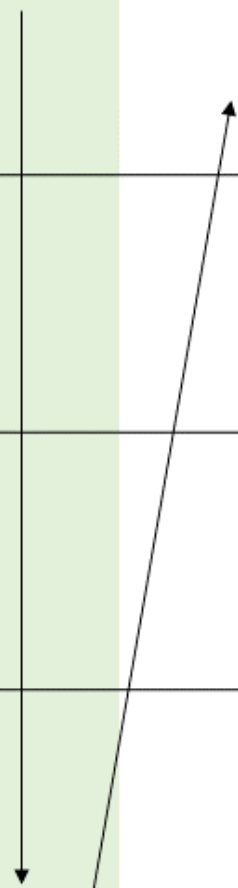
```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =        groupID      for ci where i < 2
              groupID + 8  for ci where i >= 2

col =    (threadID_in_group * 2) + (i & 0x1)  for ci where i = {0, ..., 3}

```

Row \ Col	0	1	2	..	7
0	$T0: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$		$T28: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	
..					
3					
4	$T1: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$		$T29: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	
..					
7					
8	$T2: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$		$T30: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	
..					
11					
12	$T3: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$		$T31: \begin{Bmatrix} b0 \\ b1 \\ b2 \\ b3 \end{Bmatrix}$	
..					
15					

`%laneid:{fragments}`

Figure 90: MMA .m16n8k32 fragment layout for rows 0–15 of matrix B with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type.

Row \ Col	0	1	2	..	7
16	$T0: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	$T4: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$		$T28: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	
..					
19					
20	$T1: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	$T5: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$		$T29: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	
..					
23					
24	$T2: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	$T6: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$		$T30: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	
..					
27					
28	$T3: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	$T7: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$		$T31: \begin{Bmatrix} b4 \\ b5 \\ b6 \\ b7 \end{Bmatrix}$	
..					
31					

`%laneid:{fragments}`

Figure 91: MMA.m16n8k32 fragment layout for rows 16–31 of matrix B with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type.

R\C	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
..								
15	T28: {c2, c3}		T29: {c2, c3}		T30: {c2, c3}		T31: {c2, c3}	

Figure 92: MMA.m16n8k32 fragment layout for accumulator matrix C/D with .s32 / .f32 / .f16 type.

9.7.14.5.11 Matrix Fragments for mma.m16n8k64

A warp executing `mma.m16n8k64` will compute an MMA operation of shape .m16n8k64.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.s4 / .u4	A vector expression containing four .b32 registers, with each register containing eight .s4 / .u4 elements from the matrix A.	a0, a1, ..., a30, a31
.e2m1	A vector expression containing four .b32 registers, with each register containing eight .e2m1 elements from the matrix A.	a0, a1, ..., a30, a31

The layout of the fragments held by different threads is shown in Figure 93.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID                                for ai where 0 <= i < 8 ||
↪ 16 <= i < 24                                otherwise
      groupID + 8

```

(continues on next page)

R\C	0	...	7	8	...	15	16	...	23	24	...	31	32	..	39	40	..	47	48	..	55	56	..	63								
0	T0:{a0, a1, ..., a7}				T1:{a0, a1, ..., a7}				T2:{a0, a1, ..., a7}				T3:{a0, a1, ..., a7}				T0:{a16, ..., a23}				T1:{a16, ..., a23}				T2:{a16, ..., a23}				T3:{a16, ..., a23}			
1	T4:{a0, a1, ..., a7}				T5:{a0, a1, ..., a7}				T6:{a0, a1, ..., a7}				T7:{a0, a1, ..., a7}				T4:{a16, ..., a23}				T5:{a16, ..., a23}				T6:{a16, ..., a23}				T7:{a16, ..., a23}			
2																																
..																																
7	T28:{a0, a1, ..., a7}				T29:{a0, a1, ..., a7}				T30:{a0, a1, ..., a7}				T31:{a0, a1, ..., a7}				T28:{a16, ..., a23}				T29:{a16, ..., a23}				T30:{a16, ..., a23}				T31:{a16, ..., a23}			
8	T0:{a8, ..., a15}				T1:{a8, ..., a15}				T2:{a8, ..., a15}				T3:{a8, ..., a15}				T0:{a24, ..., a31}				T1:{a24, ..., a31}				T2:{a24, ..., a31}				T3:{a24, ..., a31}			
9	T4:{a8, ..., a15}				T5:{a8, ..., a15}				T6:{a8, ..., a15}				T7:{a8, ..., a15}				T4:{a24, ..., a31}				T5:{a24, ..., a31}				T6:{a24, ..., a31}				T7:{a24, ..., a31}			
10																																
..																																
15	T28:{a8, ..., a15}				T29:{a8, ..., a15}				T30:{a8, ..., a15}				T31:{a8, ..., a15}				T28:{a24, ..., a31}				T29:{a24, ..., a31}				T30:{a24, ..., a31}				T31:{a24, ..., a31}			

`%laneid:{fragments}`

Figure 93: MMA .m16n8k64 fragment layout for matrix A with .u4 / .s4 / .e2m1 type.

(continued from previous page)

```
col =      (threadID_in_group * 8) + (i & 0x7)      for ai where i < 16
          (threadID_in_group * 8) + (i & 0x7) + 32  for ai where i >= 16
```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.s4 / .u4	A vector expression containing two .b32 registers, with each register containing eight .s4 / .u4 elements from the matrix B.	b0, b1, ..., b14, b15
.e2m1	A vector expression containing two .b32 registers, with each register containing eight .e2m1 elements from the matrix B.	b0, b1, ..., b14, b15

The layout of the fragments held by different threads is shown in [Figure 94](#) and [Figure 95](#).

The row and column of a matrix fragment can be computed as:

```
groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      (threadID_in_group * 8) + (i & 0x7)      for bi where i < 8
          (threadID_in_group * 8) + (i & 0x7) + 32  for bi where i >= 8

col =      groupID
```

Row \ Col	0	1	2	..	7
0 .. 7	$T0: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$
8 .. 15	$T1: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$
16 .. 23	$T2: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$
24 .. 31	$T3: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b6 \\ b7 \end{Bmatrix}$

Figure 94: MMA .m16n8k64 fragment layout for rows 0–31 of matrix B with .u4 / .s4 / .e2m1 type.

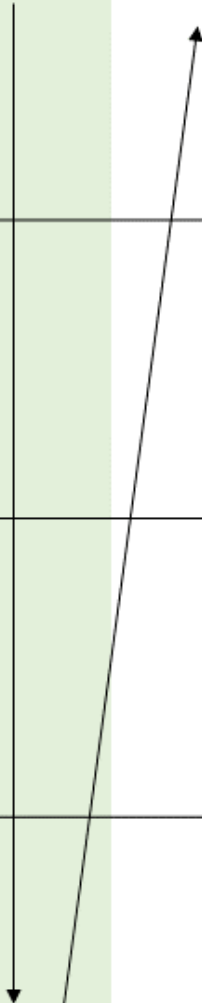
Row \ Col	0	1	2	..	7
32	$T0: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$	$T4: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$			$T28: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$
..					
39					
40	$T1: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$	$T5: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$			$T29: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$
..					
47					
48	$T2: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$	$T6: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$			$T30: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$
..					
55					
56	$T3: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$	$T7: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$			$T31: \begin{Bmatrix} b8 \\ b9 \\ .. \\ b14 \\ b15 \end{Bmatrix}$
..					
64					

Figure 95: MMA .m16n8k64 fragment layout for rows 32–63 of matrix B with .u4 / .s4 / .e2m1 type.

► Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing four .s32 registers, containing four .s32 elements from the matrix C (or D).	c0, c1, c2, c3
.f32	A vector expression containing four .f32 registers, containing four .f32 elements from the matrix C (or D).	c0, c1, c2, c3

The layout of the fragments held by different threads is shown in Figure 96.

R\C	0	1	2	3	4	5	6	7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}				
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}				
2								
7					T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}
8	T0: {c2, c3}	T1: {c2, c3}	T2: {c2, c3}	T3: {c2, c3}				
9	T4: {c2, c3}	T5: {c2, c3}	T6: {c2, c3}	T7: {c2, c3}				
10								
15					T28: {c2, c3}	T29: {c2, c3}	T30: {c2, c3}	T31: {c2, c3}

Figure 96: MMA .m16n8k64 fragment layout for accumulator matrix C/D with .s32 / .f32 type.

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID
        groupID + 8
        for ci where i < 2
        for ci where i >= 2

col = (threadID_in_group * 2) + (i & 0x1)    for ci where i = {0,...,3}

```

9.7.14.5.12 Matrix Fragments for `mma.m16n8k128`

A warp executing `mma.m16n8k128` will compute an MMA operation of shape `.m16n8k128`.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
<code>.b1</code>	A vector expression containing two <code>.b32</code> registers, with each register containing thirty two <code>.b1</code> elements from the matrix A.	<code>a0, a1, ..., a62, a63</code>

The layout of the fragments held by different threads is shown in Figure 97.

R \ C	0 1 .. 31	32 33 .. 63	64 65 .. 95	96 97 .. 127
0	T0:{a0, a1, ..., a31}	T1:{a0, a1, ..., a31}	T2:{a0, a1, ..., a31}	T3:{a0, a1, ..., a31}
1	T4:{a0, a1, ..., a31}	T5:{a0, a1, ..., a31}	T6:{a0, a1, ..., a31}	T7:{a0, a1, ..., a31}
2				
..				
7	T28:{a0, a1, ..., a31}	T29:{a0, a1, ..., a31}	T30:{a0, a1, ..., a31}	T31:{a0, a1, ..., a31}
8	T0:{a32, a33, ..., a63}	T1:{a32, a33, ..., a63}	T2:{a32, a33, ..., a63}	T3:{a32, a33, ..., a63}
9	T4:{a32, a33, ..., a63}	T5:{a32, a33, ..., a63}	T6:{a32, a33, ..., a63}	T7:{a32, a33, ..., a63}
10				
..				
15	T28:{a32, a33, ..., a63}	T29:{a32, a33, ..., a63}	T30:{a32, a33, ..., a63}	T31:{a32, a33, ..., a63}

`%laneid:{fragments}`

Figure 97: MMA `.m16n8k128` fragment layout for matrix A with `.b1` type.

The row and column of a matrix fragment can be computed as:

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID
      groupID + 8
col = (threadID_in_group * 32) + (i & 0x1F)

for ai where i < 32
for ai where i >= 32
for ai where i = {0, ..., 63}

```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.b1	A vector expression containing a single .b32 register containing thirty two .b1 elements from the matrix B.	b0, b1, ..., b30, b31

The layout of the fragments held by different threads is shown in [Figure 98](#).

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row = (threadID_in_group * 32) + i      for bi where i = {0,...,31}
col = groupID

```

► Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing four .s32 registers, containing four .s32 elements from the matrix C (or D).	c0, c1, c2, c3

The layout of the fragments held by different threads is shown in [Figure 99](#).

The row and column of a matrix fragment can be computed as:

```

groupID          = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for ci where i < 2
      groupID + 8      for ci where i >= 2

col = (threadID_in_group * 2) + (i & 0x1)  for ci where i = {0, 1, 2, 3}

```

9.7.14.5.13 Matrix Fragments for **mma.m16n8k256**

A warp executing **mma.m16n8k256** will compute an MMA operation of shape **.m16n8k256**.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements (low to high)
.b1	A vector expression containing four .b32 registers, with each register containing thirty two .b1 elements from the matrix A.	a0, a1, ..., a126, a127

The layout of the fragments held by different threads is shown in [Figure 100](#).

The row and column of a matrix fragment can be computed as:

Row\Col	0	1	2	..	7
0	$T0: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
1					
..					
31					
32	$T1: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
33					
..					
63					
64	$T2: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
65					
..					
95					
96	$T3: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ .. \\ b31 \end{Bmatrix}$
97					
..					
127					

%laneid:{fragment}

Figure 98: MMA .m16n8k128 fragment layout for matrix B with .b1 type.

R\C	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
7	T28: {c0, c1}		T29: {c0, c1}		T30: {c0, c1}		T31: {c0, c1}	
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
..								
15	T28: {c2, c3}		T29: {c2, c3}		T30: {c2, c3}		T31: {c2, c3}	

Figure 99: MMA .m16n8k128 fragment layout for accumulator matrix C/D with .s32 type.

R\C	0	...	31	32	...	63	64	...	95	96	...	127	128	..	159	160	..	191	192	..	223	224	..	255
0	T0:{a0, a1, ..., a31}			T1:{a0, a1, ..., a31}			T2:{a0, a1, ..., a31}			T3:{a0, a1, ..., a31}			T0:{a64, ..., a95}			T1:{a64, ..., a95}			T2:{a64, ..., a95}			T3:{a64, ..., a95}		
1	T4:{a0, a1, ..., a31}			T5:{a0, a1, ..., a31}			T6:{a0, a1, ..., a31}			T7:{a0, a1, ..., a31}			T4:{a64, ..., a95}			T5:{a64, ..., a95}			T6:{a64, ..., a95}			T7:{a64, ..., a95}		
2																								
..																								
7	T28:{a0, a1, ..., a31}			T29:{a0, a1, ..., a31}			T30:{a0, a1, ..., a31}			T31:{a0, a1, ..., a31}			T28:{a64, ..., a95}			T29:{a64, ..., a95}			T30:{a64, ..., a95}			T31:{a64, ..., a95}		
8	T0:{a32, ..., a63}			T1:{a32, ..., a63}			T2:{a32, ..., a63}			T3:{a32, ..., a63}			T0:{a96, ..., a127}			T1:{a96, ..., a127}			T2:{a96, ..., a127}			T3:{a96, ..., a127}		
9	T4:{a32, ..., a63}			T5:{a32, ..., a63}			T6:{a32, ..., a63}			T7:{a32, ..., a63}			T4:{a96, ..., a127}			T5:{a96, ..., a127}			T6:{a96, ..., a127}			T7:{a96, ..., a127}		
10																								
..																								
15	T28:{a32, ..., a63}			T29:{a32, ..., a63}			T30:{a32, ..., a63}			T31:{a32, ..., a63}			T28:{a96, ..., a127}			T29:{a96, ..., a127}			T30:{a96, ..., a127}			T31:{a96, ..., a127}		

%laneid:{fragments}

Figure 100: MMA .m16n8k256 fragment layout for matrix A with .b1 type.

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID                                for ai where 0 <= i <
↪32 || 64 <= i < 96                                otherwise
      groupID + 8

col =      (threadID_in_group * 32) + i              for ai where i < 64
          (threadID_in_group * 32) + (i & 0x1F) + 128 for ai where i >= 64

```

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.b1	A vector expression containing two .b32 registers, with each register containing thirty two .b1 elements from the matrix B.	b0, b1, ..., b62, b63

The layout of the fragments held by different threads is shown in [Figure 101](#) and [Figure 102](#).

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      (threadID_in_group * 32) + (i & 0x1F)      for bi where i < 32
          (threadID_in_group * 32) + (i & 0x1F) + 128 for bi where i >= 32

col =      groupID

```

► Accumulators (C or D):

.ctype / .dtype	Fragment	Elements (low to high)
.s32	A vector expression containing four .s32 registers, containing four .s32 elements from the matrix C (or D).	c0, c1, c2, c3

The layout of the fragments held by different threads is shown in [Figure 103](#).

The row and column of a matrix fragment can be computed as:

```

groupID      = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID                                for ci where i < 2
          groupID + 8                             for ci where i >= 2

col =      (threadID_in_group * 2) + (i & 0x1)      for ci where i = {0, 1, 2, 3}

```

Row \ Col	0	1	2	..	7
0 .. 31	$T0: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$
32 .. 63	$T1: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$
64 .. 95	$T2: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$			$T30: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$
96 .. 127	$T3: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b31 \end{Bmatrix}$

`%laneid:{fragments}`

Figure 101: MMA .m16n8k256 fragment layout for rows 0–127 of matrix B with .b1 type.

Row \ Col	0	1	2	..	7
128	$T0: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$	$T4: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$			$T28: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$
..					
159					
160	$T1: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$	$T5: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$			$T29: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$
..					
191					
192	$T2: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$	$T6: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$			$T30: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$
..					
223					
224	$T3: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$	$T7: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$			$T31: \begin{Bmatrix} b_{32} \\ b_{33} \\ \vdots \\ b_{63} \end{Bmatrix}$
..					
255					

`%laneid:{fragments}`

Figure 102: MMA .m16n8k256 fragment layout for rows 128–255 of matrix B with .b1 type.

R\C	0	1	2	3	4	5	6	7
0	T0: {c0, c1}		T1: {c0, c1}		T2: {c0, c1}		T3: {c0, c1}	
1	T4: {c0, c1}		T5: {c0, c1}		T6: {c0, c1}		T7: {c0, c1}	
2								
7								
8	T0: {c2, c3}		T1: {c2, c3}		T2: {c2, c3}		T3: {c2, c3}	
9	T4: {c2, c3}		T5: {c2, c3}		T6: {c2, c3}		T7: {c2, c3}	
10								
..								
15	T28: {c2, c3}		T29: {c2, c3}		T30: {c2, c3}		T31: {c2, c3}	

Figure 103: MMA .m16n8k256 fragment layout for accumulator matrix C/D with .s32 type.

9.7.14.5.14 Multiply-and-Accumulate Instruction: mma

mma

Perform matrix multiply-and-accumulate operation

Syntax

Half precision floating point type:

```
mma.sync.aligned.m8n8k4.alayout.blayout.dtype.f16.f16.ctype d, a, b, c;
mma.sync.aligned.m16n8k8.row.col.dtype.f16.f16.ctype d, a, b, c;
mma.sync.aligned.m16n8k16.row.col.dtype.f16.f16.ctype d, a, b, c;

.alayout = {.row, .col};
.blayout = {.row, .col};
.ctype   = {.f16, .f32};
.dtype   = {.f16, .f32};
```

Alternate floating point type:

```
mma.sync.aligned.m16n8k4.row.col.f32.tf32.tf32.f32 d, a, b, c;
mma.sync.aligned.m16n8k8.row.col.f32.atype.btype.f32 d, a, b, c;
mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32 d, a, b, c;
mma.sync.aligned.shape.row.col.dtype.f8type.f8type.ctype d, a, b, c;
mma.sync.aligned.m16n8k32.row.col.kind.dtype.f8f6f4type.f8f6f4type.ctype d, a, b, c;

.atype    = {.bf16, .tf32};
```

(continues on next page)

(continued from previous page)

```
.btype      = {.bf16, .tf32};
.f8type     = {.e4m3, .e5m2};
.f8f6f4type = {.e4m3, .e5m2, .e3m2, .e2m3, .e2m1};
.ctype      = {.f16, .f32};
.dtype      = {.f16, .f32};
.shape      = {.m16n8k16, .m16n8k32};
.kind       = {.kind::f8f6f4};
```

Alternate floating point type with block scaling:

```
mma.sync.aligned.m16n8k64.row.col.kind.block_scale{.scale_vec_size}.f32.e2m1.e2m1.f32.
↳stype d, a, b, c, scale-a-data, {byte-id-a, thread-id-a}, scale-b-data, {byte-id-b,
↳thread-id-b};

.kind          = {.kind::mxf4};
.scale_vec_size = {.scale_vec::2X};
.stype         = {.ue8m0};

mma.sync.aligned.m16n8k64.row.col.kind.block_scale.scale_vec_size.f32.e2m1.e2m1.f32.
↳stype d, a, b, c, scale-a-data, {byte-id-a, thread-id-a}, scale-b-data, {byte-id-b,
↳thread-id-b};

.kind          = {.kind::mxf4nvf4};
.scale_vec_size = {.scale_vec::2X, .scale_vec::4X};
.stype         = {.ue8m0, .ue4m3};

mma.sync.aligned.m16n8k32.row.col.kind.block_scale{.scale_vec_size}.f32.f8f6f4type.
↳f8f6f4type.f32.stype d, a, b, c, scale-a-data, {byte-id-a, thread-id-a}, scale-b-
↳data, {byte-id-b, thread-id-b};

.kind          = {.kind::mxf8f6f4};
.scale_vec_size = {.scale_vec::1X};
.f8f6f4type    = {.e4m3, .e5m2, .e3m2, .e2m3, .e2m1};
.stype         = {.ue8m0};
```

Double precision floating point type:

```
mma.sync.aligned.shape.row.col.f64.f64.f64.f64 d, a, b, c;

.shape      = {.m8n84, .m16n8k4, .m16n8k8, .m16n8k16};
```

Integer type:

```
mma.sync.aligned.shape.row.col{.satfinite}.s32.atype.btype.s32 d, a, b, c;

.shape      = {.m8n8k16, .m16n8k16, .m16n8k32}
.atype      = {.u8, .s8};
.btype      = {.u8, .s8};

mma.sync.aligned.shape.row.col{.satfinite}.s32.atype.btype.s32 d, a, b, c;

.shape      = {.m8n8k32, .m16n8k32, .m16n8k64}
.atype      = {.u4, .s4};
.btype      = {.u4, .s4};
```

Single bit:

```
mma.sync.aligned.shape.row.col.s32.b1.b1.s32.bitOp.popc d, a, b, c;

.bitOp = {.xor, .and}
.shape = {.m8n8k128, .m16n8k128, .m16n8k256}
```

Description

Perform a $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + C$, where the A matrix is $M \times K$, the B matrix is $K \times N$, and the C and D matrices are $M \times N$.

Qualifier `.block_scale` specifies that the matrices A and B are scaled with `scale_A` and `scale_B` matrices respectively before performing the matrix multiply and accumulate operation as specified in the section [Block Scaling](#). The data type corresponding to each of the element within `scale_A` and `scale_B` matrices is specified by `.stype`. Qualifier `.scale_vec_size` specifies the number of columns of `scale_A` matrix and number of rows in the matrix `scale_B`.

The valid combinations of `.kind`, `.stype` and `.scale_vec_size` are described in [Table 37](#). For `mma` with `.kind::mxf4` when the qualifier `.scale_vec_size` is not specified, then it defaults to 2X. In contrast, when `.kind` is specified as `.kind::mxf8f6f4` then the qualifier `.scale_vec_size` defaults to 1X. However, for `.kind::mxf4nvvf4`, it is mandatory to provide valid `.scale_vec_size`.

A warp executing `mma.sync.m8n8k4` instruction computes 4 matrix multiply and accumulate operations. Rest of the `mma.sync` operations compute a single matrix multiply and accumulate operation per warp.

For single-bit `mma.sync`, multiplication is replaced by a sequence of logical operations; specifically, `mma.xor.popc` and `mma.and.popc` computes the XOR, AND respectively of a k-bit row of A with a k-bit column of B, then counts the number of set bits in the result (`popc`). This result is added to the corresponding element of C and written into D.

Operands `a` and `b` represent two multiplicand matrices A and B, while `c` and `d` represent the accumulator and destination matrices, distributed across the threads in warp. When `.block_scale` qualifier is specified, operand `scale-a-data`, `scale-b-data` represents the scale matrix metadata corresponding to `scale_A` and `scale_B` matrices respectively. The tuple `{byte-id-a, thread-id-a}` and `{byte-id-b, thread-id-b}` represent selectors for matrices `scale_A` and `scale_B` respectively from their corresponding metadata arguments `scale-a-data`, `scale-b-data`. The operands `scale-a-data`, `scale-b-data` are of type `.b32`. The operands `byte-id-a`, `thread-id-a`, `byte-id-b`, `thread-id-b` are unsigned 16-bit integer values. For more details on selector arguments refer [Block Scaling](#) section.

The registers in each thread hold a fragment of matrix as described in [Matrix multiply-accumulate operation using mma instruction](#).

The qualifiers `.dtype`, `.atype`, `.btype` and `.ctype` indicate the data-type of the elements in the matrices D, A, B and C respectively. The qualifier `.stype` indicate the data-type of the elements in the matrices `scale_A` and `scale_B`. Specific shapes have type restrictions :

- ▶ `.m8n8k4` : When `.ctype` is `.f32`, `.dtype` must also be `.f32`.
- ▶ `.m16n8k8` :
 - ▶ `.dtype` must be the same as `.ctype`.
 - ▶ `.atype` must be the same as `.btype`.
- ▶ `.m16n8k16` and `.m16n8k32` :
 - ▶ `.dtype` must be the same as `.ctype`.

The qualifiers `.alayout` and `.blayout` indicate the row-major or column-major layouts of matrices A and B respectively.

When `.kind` is either of `.kind::mxf8f6f4` or `.kind::f8f6f4`, the individual 4-bit and the 6-bit floating point type elements must be packed in an 8-bit container. The matrix element of type `.e2m1` resides in central 4 bits of the 8-bit container with padding in the upper 2 bits and lower 2 bits of the container. When the matrix element is of type `.e3m2` or `.e2m3`, the matrix element resides in the lower 6 bits of the 8-bit container with padding in the upper 2 bits of the container. In contrast, note that when using `mma` with `.kind::mxf4` or `.kind::mxf4nvf4`, no explicit padding is necessary even though matrix elements are of type `.e2m1`.

Precision and rounding :

- ▶ `.f16` floating point operations:

Element-wise multiplication of matrix A and B is performed with at least single precision. When `.ctype` or `.dtype` is `.f32`, accumulation of the intermediate values is performed with at least single precision. When both `.ctype` and `.dtype` are specified as `.f16`, the accumulation is performed with at least half precision.

The accumulation order, rounding and handling of subnormal inputs are unspecified.

- ▶ `.e4m3`, `.e5m2`, `.e3m2`, `.e2m3`, `.e2m1` floating point operations :

Element-wise multiplication of matrix A and B is performed with specified precision. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding, and handling of subnormal inputs are unspecified.

- ▶ `.bf16` and `.tf32` floating point operations :

Element-wise multiplication of matrix A and B is performed with specified precision. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding, and handling of subnormal inputs are unspecified.

- ▶ `.f64` floating point operations :

Precision of the element-wise multiplication and addition operation is identical to that of `.f64` precision fused multiply-add. Supported rounding modifiers are :

- ▶ `.rn` : mantissa LSB rounds to nearest even. This is the default.
- ▶ `.rz` : mantissa LSB rounds towards zero.
- ▶ `.rm` : mantissa LSB rounds towards negative infinity.
- ▶ `.rp` : mantissa LSB rounds towards positive infinity.

- ▶ Integer operations :

The integer `mma` operation is performed with `.s32` accumulators. The `.satfinite` qualifier indicates that on overflow, the accumulated value is limited to the range `MIN_INT32..MAX_INT32` (where the bounds are defined as the minimum negative signed 32-bit integer and the maximum positive signed 32-bit integer respectively).

If `.satfinite` is not specified, the accumulated value is wrapped instead.

The mandatory `.sync` qualifier indicates that `mma` instruction causes the executing thread to wait until all threads in the warp execute the same `mma` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `mma` instruction. In conditionally executed code, a `mma` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `mma` instruction is undefined if all threads in the same warp do not use the same qualifiers, or if any thread in the warp has exited.

Notes

Programs using double precision floating point `mma` instruction with shapes `.m16n8k4`, `.m16n8k8`, and `.m16n8k16` require at least 64 registers for compilation.

PTX ISA Notes

Introduced in PTX ISA version 6.4.

`.f16` floating point type `mma` operation with `.m8n8k4` shape introduced in PTX ISA version 6.4.

`.f16` floating point type `mma` operation with `.m16n8k8` shape introduced in PTX ISA version 6.5.

`.u8/.s8` integer type `mma` operation with `.m8n8k16` shape introduced in PTX ISA version 6.5.

`.u4/.s4` integer type `mma` operation with `.m8n8k32` shape introduced in PTX ISA version 6.5.

`.f64` floating point type `mma` operation with `.m8n8k4` shape introduced in PTX ISA version 7.0.

`.f16` floating point type `mma` operation with `.m16n8k16` shape introduced in PTX ISA version 7.0.

`.bf16` alternate floating point type `mma` operation with `.m16n8k8` and `.m16n8k16` shapes introduced in PTX ISA version 7.0.

`.tf32` alternate floating point type `mma` operation with `.m16n8k4` and `.m16n8k8` shapes introduced in PTX ISA version 7.0.

`.u8/.s8` integer type `mma` operation with `.m16n8k16` and `.m16n8k32` shapes introduced in PTX ISA version 7.0.

`.u4/.s4` integer type `mma` operation with `.m16n8k32` and `.m16n8k64` shapes introduced in PTX ISA version 7.0.

`.b1` single-bit integer type `mma` operation with `.m8n8k128`, `.m16n8k128` and `.m16n8k256` shapes introduced in PTX ISA version 7.0.

Support for `.and` operation in single-bit `mma` introduced in PTX ISA version 7.1.

`.f64` floating point type `mma` operation with `.m16n8k4`, `.m16n8k8`, and `.m16n8k16` shapes introduced in PTX ISA version 7.8.

Support for `.e4m3` and `.e5m2` alternate floating point type `mma` operation introduced in PTX ISA version 8.4.

Support for shape `.m16n8k16` and `.f16 dtype/ctype` with `.e4m3/.e5m2` alternate floating point type `mma` operation introduced in PTX ISA version 8.7.

Support for `.e3m2`, `.e2m3`, `.e2m1` alternate floating point type `mma` operation introduced in PTX ISA version 8.7.

Support for `.kind`, `.block_scale`, `.scale_vec_size` qualifier introduced in PTX ISA version 8.7.

Support for `.scale_vec::4X` on `.ue8m0` as `.stype` with `.kind::mxf4nvf4` is introduced in PTX ISA version 9.1.

Target ISA Notes

Requires sm_70 or higher.

.f16 floating point type mma operation with .m8n8k4 shape requires sm_70 or higher.

Note: mma.sync.m8n8k4 is optimized for target architecture sm_70 and may have substantially reduced performance on other target architectures.

.f16 floating point type mma operation with .m16n8k8 shape requires sm_75 or higher.

.u8/.s8 integer type mma operation with .m8n8k16 shape requires sm_75 or higher.

.u4/.s4 integer type mma operation with .m8n8k32 shape sm_75 or higher.

.b1 single-bit integer type mma operation with .m8n8k128 shape sm_75 or higher.

.f64 floating point type mma operation with .m8n8k4 shape requires sm_80 or higher.

.f16 floating point type mma operation with .m16n8k16 shape requires sm_80 or higher.

.bf16 alternate floating point type mma operation with .m16n8k8 and .m16n8k16 shapes requires sm_80 or higher.

.tf32 alternate floating point type mma operation with .m16n8k4 and .m16n8k8 shapes requires sm_80 or higher.

.u8/.s8 integer type mma operation with .m16n8k16 and .m16n8k32 shapes requires sm_80 or higher.

.u4/.s4 integer type mma operation with .m16n8k32 and .m16n8k64 shapes requires sm_80 or higher.

.b1 single-bit integer type mma operation with .m16n8k128 and .m16n8k256 shapes requires sm_80 or higher.

.and operation in single-bit mma requires sm_80 or higher.

.f64 floating point type mma operation with .m16n8k4, .m16n8k8, and .m16n8k16 shapes require sm_90 or higher.

.e4m3 and .e5m2 alternate floating point type mma operation requires sm_89 or higher.

.e3m2, .e2m3 and .e2m1 alternate floating point type mma operation requires sm_120a and is supported on sm_120f from PTX ISA version 8.8.

Support for .kind, .block_scale, .scale_vec_size qualifier requires sm_120a and are supported on sm_120f or higher in the same family from PTX ISA version 8.8.

Examples of half precision floating point type

```
// f16 elements in C and D matrix
.reg .f16x2 %Ra<2> %Rb<2> %Rc<4> %Rd<4>
mma.sync.aligned.m8n8k4.row.col.f16.f16.f16.f16
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1},
{%Rb0, %Rb1},
{%Rc0, %Rc1, %Rc2, %Rc3};
```

```
// f16 elements in C and f32 elements in D
.reg .f16x2 %Ra<2> %Rb<2> %Rc<4>
```

(continues on next page)

(continued from previous page)

```

.reg .f32 %Rd<8>
mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f16
{%Rd0, %Rd1, %Rd2, %Rd3, %Rd4, %Rd5, %Rd6, %Rd7},
{%Ra0, %Ra1},
{%Rb0, %Rb1},
{%Rc0, %Rc1, %Rc2, %Rc3};

// f32 elements in C and D
.reg .f16x2 %Ra<2>, %Rb<1>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k8.row.col.f32.f16.f16.f32
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1},
{%Rb0},
{%Rc0, %Rc1, %Rc2, %Rc3};

.reg .f16x2 %Ra<4>, %Rb<2>, %Rc<2>, %Rd<2>;
mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16
{%Rd0, %Rd1},
{%Ra0, %Ra1, %Ra2, %Ra3},
{%Rb0, %Rb1},
{%Rc0, %Rc1};

.reg .f16 %Ra<4>, %Rb<2>;
.reg .f32 %Rc<2>, %Rd<2>;
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1, %Ra2, %Ra3},
{%Rb0, %Rb1},
{%Rc0, %Rc1, %Rc2, %Rc3};

```

Examples of alternate floating point type

```

.reg .b32 %Ra<2>, %Rb<1>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k4.row.col.f32.tf32.tf32.f32
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1},
{%Rb0},
{%Rc0, %Rc1, %Rc2, %Rc3};

.reg .f16x2 %Ra<2>, %Rb<1>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k8.row.col.f32.bf16.bf16.f32
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1},
{%Rb0},
{%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<2>, %Rb<1>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k8.row.col.f32.tf32.tf32.f32
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1, %Rb2, %Rb3},

```

(continues on next page)

(continued from previous page)

```

    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .f16x2 %Ra<2>, %Rb<1>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e5m2.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k16.row.col.f32.e5m2.e4m3.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .b32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k32.row.col.f16.e4m3.e5m2.f16
    {%Rd0, %Rd1},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .b32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k16.row.col.f16.e5m2.e5m2.f16
    {%Rd0, %Rd1},
    {%Ra0, %Ra1},
    {%Rb0},
    {%Rc0, %Rc1};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k32.row.col.kind::f8f6f4.f32.e3m2.e2m3.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .b32 %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k32.row.col.kind::f8f6f4.f16.e2m3.e2m1.f16
    {%Rd0, %Rd1},
    {%Ra0, %Ra1, %Ra2, %Ra3},

```

(continues on next page)

(continued from previous page)

```
{%Rb0, %Rb1},
{%Rc0, %Rc1};
```

Examples of integer type

```
.reg .b32 %Ra, %Rb, %Rc<2>, %Rd<2>;

// s8 elements in A and u8 elements in B
mma.sync.aligned.m8n8k16.row.col.satfinite.s32.s8.u8.s32
    {%Rd0, %Rd1},
    {%Ra},
    {%Rb},
    {%Rc0, %Rc1};

// u4 elements in A and B matrix
mma.sync.aligned.m8n8k32.row.col.satfinite.s32.u4.u4.s32
    {%Rd0, %Rd1},
    {%Ra},
    {%Rb},
    {%Rc0, %Rc1};

// s8 elements in A and u8 elements in B
.reg .b32 %Ra<2>, %Rb, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k16.row.col.satfinite.s32.s8.u8.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb},
    {%Rc0, %Rc1, %Rc2, %Rc3};

// u4 elements in A and s4 elements in B
.reg .b32 %Ra<2>, %Rb, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k32.row.col.satfinite.s32.u4.s4.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb},
    {%Rc0, %Rc1, %Rc2, %Rc3};

// s8 elements in A and s8 elements in B
.reg .b32 %Ra<4>, %Rb<2>, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k32.row.col.satfinite.s32.s8.s8.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

// u8 elements in A and u8 elements in B
.reg .b32 %Ra<4>, %Rb<2>, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k64.row.col.satfinite.s32.u4.u4.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1 },
    {%Rc0, %Rc1, %Rc2, %Rc3};
```

Examples of single bit type

```
// b1 elements in A and B
.reg .b32 %Ra, %Rb, %Rc<2>, %Rd<2>;
mma.sync.aligned.m8n8k128.row.col.s32.b1.b1.s32.and.popc
    {%Rd0, %Rd1},
    {%Ra},
    {%Rb},
    {%Rc0, %Rc1};

// b1 elements in A and B
.reg .b32 %Ra, %Rb, %Rc<2>, %Rd<2>;
mma.sync.aligned.m8n8k128.row.col.s32.b1.b1.s32.xor.popc
    {%Rd0, %Rd1},
    {%Ra},
    {%Rb},
    {%Rc0, %Rc1};

.reg .b32 %Ra<2>, %Rb, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k128.row.col.s32.b1.b1.s32.xor.popc
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<2>, %Rb, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k128.row.col.s32.b1.b1.s32.and.popc
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<4>, %Rb<2>, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k256.row.col.s32.b1.b1.s32.xor.popc
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

.reg .b32 %Ra<4>, %Rb<2>, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k256.row.col.s32.b1.b1.s32.and.popc
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};
```

Examples of .f64 floating point type

```
.reg .f64 %Ra, %Rb, %Rc<2>, %Rd<2>;
mma.sync.aligned.m8n8k4.row.col.f64.f64.f64
    {%Rd0, %Rd1},
    {%Ra},
    {%Rb},
    {%Rc0, %Rc1};
```

(continues on next page)

(continued from previous page)

```
.reg .f64 %Ra<8>, %Rb<4>, %Rc<4>, %Rd<4>;
mma.sync.aligned.m16n8k4.row.col.f64.f64.f64.f64.rn
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0},
    {%Rc0, %Rc1, %Rc2, %Rc3};

mma.sync.aligned.m16n8k8.row.col.f64.f64.f64.f64.rn
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3};

mma.sync.aligned.m16n8k16.row.col.f64.f64.f64.f64.rn
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3, %Ra4, %Ra5, %Ra6, %Ra7},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3};
```

Examples of mma with block scale

```
.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
mma.sync.aligned.m16n8k64.row.col.kind::mxf4.block_scale.f32.e2m1.e2m1.f32.ue8m0
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    scaleAData, {2, 1}, scaleBData, {2, 3};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .u16 bidA, bidB, tidA, tidB;
mma.sync.aligned.m16n8k64.row.col.kind::mxf4nvf4.block_scale.scale_vec::4X.f32.e2m1.
↪ e2m1.f32.ue4m3
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    scaleAData, {bidA, tidA}, scaleBData, {bidB, tidB};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .u16 bidA, bidB, tidA, tidB;
mma.sync.aligned.m16n8k64.row.col.kind::mxf4nvf4.block_scale.scale_vec::4X.f32.e2m1.
↪ e2m1.f32.ue8m0
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    scaleAData, {bidA, tidA}, scaleBData, {bidB, tidB};
```

(continues on next page)

(continued from previous page)

```

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
mma.sync.aligned.m16n8k32.row.col.kind::mxf8f6f4.block_scale.scale_vec::1X.f32.e3m2.
↪e2m1.f32.ue8m0
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    scaleAData, {0, 1}, scaleBData, {0, 1};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
mma.sync.aligned.m16n8k32.row.col.kind::mxf8f6f4.block_scale.scale_vec::1X.f32.e4m3.
↪e5m2.f32.ue8m0
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    scaleAData, {0, 1}, scaleBData, {0, 0};

```

9.7.14.5.15 Warp-level matrix load instruction: **ldmatrix**

ldmatrix

Collectively load one or more matrices from shared memory for mma instruction

Syntax

```

ldmatrix.sync.aligned.shape.num{.trans}{.ss}.type r, [p];

ldmatrix.sync.aligned.m8n16.num{.ss}.dst_fmt.src_fmt      r, [p];
ldmatrix.sync.aligned.m16n16.num.trans{.ss}.dst_fmt.src_fmt r, [p];

.shape    = { .m8n8, .m16n16 };
.num      = { .x1, .x2, .x4 };
.ss       = { .shared{::cta} };
.type     = { .b16, .b8 };
.dst_fmt  = { .b8x16 };
.src_fmt  = { .b6x16_p32, .b4x16_p64 };

```

Description

Collectively load one or more matrices across all threads in a warp from the location indicated by the address operand `p`, from `.shared` state space into destination register `r`. If no state space is provided, generic addressing is used, such that the address in `p` points into `.shared` space. If the generic address doesn't fall in `.shared` state space, then the behavior is undefined.

The `.shape` qualifier indicates the dimensions of the matrices being loaded. Each matrix element holds 16-bit or 8-bit or 6-bit or 4-bit data.

Following table shows the matrix load case for each `.shape`.

<code>.shape</code>	Matrix shape	Element size
<code>.m8n8</code>	8x8	16-bit
<code>.m16n16</code>	16x16	8-bit or 6-bit or 4-bit
<code>.m8n16</code>	8x16	6-bit or 4-bit

Following table shows the valid use of 6-bit or 4-bit data load.

.src_fmt	.shape	Source data	Padding	.dst_fmt
.b6x16_p32	.m8n16	16 6-bit elements	32 bits	.b8x16 (16 8-bit elements)
	.m16n16			
.b4x16_p64	.m8n16	16 4-bit elements	64 bits	
	.m16n16			

For `.b6x16_p32` format source data is 16 unsigned 6-bit elements with 32 bits padding. For `.b4x16_p64` format source data is 16 unsigned 4-bit elements with 64 bits padding.

The values `.x1`, `.x2` and `.x4` for `.num` indicate one, two or four matrices respectively. When `.shape` is `.m16n16`, only `.x1` and `.x2` are valid values for `.num`.

The mandatory `.sync` qualifier indicates that `ldmatrix` causes the executing thread to wait until all threads in the warp execute the same `ldmatrix` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `ldmatrix` instruction. In conditionally executed code, an `ldmatrix` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise the behavior is undefined.

The behavior of `ldmatrix` is undefined if all threads do not use the same qualifiers, or if any thread in the warp has exited.

The destination operand `r` is a brace-enclosed vector expression consisting of 1, 2, or 4 32-bit registers as per the value of `.num`. Each component of the vector expression holds a fragment from the corresponding matrix.

Supported addressing modes for `p` are described in [Addresses as Operands](#).

Consecutive instances of row need not be stored contiguously in memory. The eight addresses required for each matrix are provided by eight threads, depending upon the value of `.num` as shown in the following table. Each address corresponds to the start of a matrix row. Addresses `addr0`–`addr7` correspond to the rows of the first matrix, addresses `addr8`–`addr15` correspond to the rows of the second matrix, and so on.

.num	Threads 0–7	Threads 8–15	Threads 16–23	Threads 24–31
.x1	addr0–addr7	–	–	–
.x2	addr0–addr7	addr8–addr15	–	–
.x4	addr0–addr7	addr8–addr15	addr16–addr23	addr24–addr31

Note: For .target sm_75 or below, all threads must contain valid addresses. Otherwise, the behavior is undefined. For .num = .x1 and .num = .x2, addresses contained in lower threads can be copied to higher threads to achieve the expected behavior.

When reading 8x8 matrices, a group of four consecutive threads loads 16 bytes. The matrix addresses must be naturally aligned accordingly.

Each thread in a warp loads fragments of a row, with thread 0 receiving the first fragment in its register r, and so on. A group of four threads loads an entire row of the matrix as shown in [Figure 104](#).

Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

Figure 104: ldmatrix fragment layout for one 8x8 Matrix with 16-bit elements

When .num = .x2, the elements of the second matrix are loaded in the next destination register in each thread as per the layout in above table. Similarly, when .num = .x4, elements of the third and fourth matrices are loaded in the subsequent destination registers in each thread.

For matrix shape 16x16, two destination registers r0 and r1 of type .b32 must be specified and in each register four 8-bit elements are loaded. For 4-bit or 6-bit data, 8-bit element will have 4 bits or 2 bits of padding respectively. Refer [Optional Decompression](#) for more details on these formats.

An entire row of the matrix can be loaded by a group of four consecutive and aligned threads. Each thread in a warp loads 4 consecutive columns across 2 rows as shown in the [Figure 105](#).

For matrix shape 8x16, one destination register r0 of type .b32 must be specified where four 8-bit elements are loaded in the register. For 4-bit or 6-bit data, 8-bit element will have 4 bits or 2 bits of padding respectively.

An entire row of the matrix can be loaded by a group of four consecutive and aligned threads. Each thread in a warp loads 4 consecutive columns as shown in [Figure 106](#).

Optional qualifier .trans indicates that the matrix is loaded in column-major format. However, for 16x16 matrices, .trans is mandatory.

The ldmatrix instruction is treated as a weak memory operation in the [Memory Consistency Model](#).

Row\Col	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:r0	T0:r0	T0:r0	T0:r0	T1:r0	T1:r0	T1:r0	T1:r0	T2:r0	T2:r0	T2:r0	T2:r0	T3:r0	T3:r0	T3:r0	T3:r0
1	T4:r0	T4:r0	T4:r0	T4:r0	T5:r0	T5:r0	T5:r0	T5:r0	T6:r0	T6:r0	T6:r0	T6:r0	T7:r0	T7:r0	T7:r0	T7:r0
2	T8:r0	T8:r0	T8:r0	T8:r0	T9:r0	T9:r0	T9:r0	T9:r0	T10:r0	T10:r0	T10:r0	T10:r0	T11:r0	T11:r0	T11:r0	T11:r0
3	T12:r0	T12:r0	T12:r0	T12:r0	T13:r0	T13:r0	T13:r0	T13:r0	T14:r0	T14:r0	T14:r0	T14:r0	T15:r0	T15:r0	T15:r0	T15:r0
4	T16:r0	T16:r0	T16:r0	T16:r0	T17:r0	T17:r0	T17:r0	T17:r0	T18:r0	T18:r0	T18:r0	T18:r0	T19:r0	T19:r0	T19:r0	T19:r0
5	T20:r0	T20:r0	T20:r0	T20:r0	T21:r0	T21:r0	T21:r0	T21:r0	T22:r0	T22:r0	T22:r0	T22:r0	T23:r0	T23:r0	T23:r0	T23:r0
6	T24:r0	T24:r0	T24:r0	T24:r0	T25:r0	T25:r0	T25:r0	T25:r0	T26:r0	T26:r0	T26:r0	T26:r0	T27:r0	T27:r0	T27:r0	T27:r0
7	T28:r0	T28:r0	T28:r0	T28:r0	T29:r0	T29:r0	T29:r0	T29:r0	T30:r0	T30:r0	T30:r0	T30:r0	T31:r0	T31:r0	T31:r0	T31:r0
8	T0:r1	T0:r1	T0:r1	T0:r1	T1:r1	T1:r1	T1:r1	T1:r1	T2:r1	T2:r1	T2:r1	T2:r1	T3:r1	T3:r1	T3:r1	T3:r1
9	T4:r1	T4:r1	T4:r1	T4:r1	T5:r1	T5:r1	T5:r1	T5:r1	T6:r1	T6:r1	T6:r1	T6:r1	T7:r1	T7:r1	T7:r1	T7:r1
10	T8:r1	T8:r1	T8:r1	T8:r1	T9:r1	T9:r1	T9:r1	T9:r1	T10:r1	T10:r1	T10:r1	T10:r1	T11:r1	T11:r1	T11:r1	T11:r1
11	T12:r1	T12:r1	T12:r1	T12:r1	T13:r1	T13:r1	T13:r1	T13:r1	T14:r1	T14:r1	T14:r1	T14:r1	T15:r1	T15:r1	T15:r1	T15:r1
12	T16:r1	T16:r1	T16:r1	T16:r1	T17:r1	T17:r1	T17:r1	T17:r1	T18:r1	T18:r1	T18:r1	T18:r1	T19:r1	T19:r1	T19:r1	T19:r1
13	T20:r1	T20:r1	T20:r1	T20:r1	T21:r1	T21:r1	T21:r1	T21:r1	T22:r1	T22:r1	T22:r1	T22:r1	T23:r1	T23:r1	T23:r1	T23:r1
14	T24:r1	T24:r1	T24:r1	T24:r1	T25:r1	T25:r1	T25:r1	T25:r1	T26:r1	T26:r1	T26:r1	T26:r1	T27:r1	T27:r1	T27:r1	T27:r1
15	T28:r1	T28:r1	T28:r1	T28:r1	T29:r1	T29:r1	T29:r1	T29:r1	T30:r1	T30:r1	T30:r1	T30:r1	T31:r1	T31:r1	T31:r1	T31:r1

Figure 105: Idmatrix fragment layout for one 16x16 matrix with 8-bit elements

Row\Col	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:r0	T0:r0	T0:r0	T0:r0	T1:r0	T1:r0	T1:r0	T1:r0	T2:r0	T2:r0	T2:r0	T2:r0	T3:r0	T3:r0	T3:r0	T3:r0
1	T4:r0	T4:r0	T4:r0	T4:r0	T5:r0	T5:r0	T5:r0	T5:r0	T6:r0	T6:r0	T6:r0	T6:r0	T7:r0	T7:r0	T7:r0	T7:r0
2	T8:r0	T8:r0	T8:r0	T8:r0	T9:r0	T9:r0	T9:r0	T9:r0	T10:r0	T10:r0	T10:r0	T10:r0	T11:r0	T11:r0	T11:r0	T11:r0
3	T12:r0	T12:r0	T12:r0	T12:r0	T13:r0	T13:r0	T13:r0	T13:r0	T14:r0	T14:r0	T14:r0	T14:r0	T15:r0	T15:r0	T15:r0	T15:r0
4	T16:r0	T16:r0	T16:r0	T16:r0	T17:r0	T17:r0	T17:r0	T17:r0	T18:r0	T18:r0	T18:r0	T18:r0	T19:r0	T19:r0	T19:r0	T19:r0
5	T20:r0	T20:r0	T20:r0	T20:r0	T21:r0	T21:r0	T21:r0	T21:r0	T22:r0	T22:r0	T22:r0	T22:r0	T23:r0	T23:r0	T23:r0	T23:r0
6	T24:r0	T24:r0	T24:r0	T24:r0	T25:r0	T25:r0	T25:r0	T25:r0	T26:r0	T26:r0	T26:r0	T26:r0	T27:r0	T27:r0	T27:r0	T27:r0
7	T28:r0	T28:r0	T28:r0	T28:r0	T29:r0	T29:r0	T29:r0	T29:r0	T30:r0	T30:r0	T30:r0	T30:r0	T31:r0	T31:r0	T31:r0	T31:r0

Figure 106: Idmatrix fragment layout for one 8x16 matrix with 8-bit elements containing 4-bit/6-bit data

PTX ISA Notes

Introduced in PTX ISA version 6.5.

Support for `::cta` sub-qualifier introduced in PTX ISA version 7.8.

Support for `.m16n16`, `.m8n16` shapes introduced in PTX ISA version 8.6.

Support for `.b8` type with `ldmatrix` is introduced in PTX ISA version 8.6.

Support for `.src_fmt`, `.dst_fmt` qualifiers introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_75` or higher.

Shapes `.m16n16`, `.m8n16` are supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And are supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Type `.b8` with `ldmatrix` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And are supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Qualifiers `.src_fmt`, `.dst_fmt` are supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And are supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family

- sm_110f or higher in the same family

Examples

```
// Load a single 8x8 matrix using 64-bit addressing
.reg .b64 addr;
.reg .b32 d;
ldmatrix.sync.aligned.m8n8.x1.shared::cta.b16 {d}, [addr];

// Load two 8x8 matrices in column-major format
.reg .b64 addr;
.reg .b32 d<2>;
ldmatrix.sync.aligned.m8n8.x2.trans.shared.b16 {d0, d1}, [addr];

// Load four 8x8 matrices
.reg .b64 addr;
.reg .b32 d<4>;
ldmatrix.sync.aligned.m8n8.x4.b16 {d0, d1, d2, d3}, [addr];

// Load one 16x16 matrices of 64-bit elements and transpose them
.reg .b64 addr;
.reg .b32 d<2>;
ldmatrix.sync.aligned.m16n16.x1.trans.shared.b8 {d0, d1}, [addr];

// Load two 16x16 matrices of 64-bit elements and transpose them
.reg .b64 addr;
.reg .b32 d<4>;
ldmatrix.sync.aligned.m16n16.x2.trans.shared::cta.b8 {d0, d1, d2, d3}, [addr];

// Load two 16x16 matrices of 6-bit elements and transpose them
.reg .b64 addr;
.reg .b32 d<4>;
ldmatrix.sync.aligned.m16n16.x2.trans.shared::cta.b8x16.b6x16_p32 {d0, d1, d2, d3},
↪ [addr];
```

9.7.14.5.16 Warp-level matrix store instruction: **stmatrix**

stmatrix

Collectively store one or more matrices to shared memory.

Syntax

```
stmatrix.sync.aligned.shape.num{.trans}{.ss}.type [p], r;

.shape = {.m8n8, .m16n8};
.num   = {.x1, .x2, .x4};
.ss    = {.shared{::cta}};
.type  = {.b16, .b8};
```

Description

Collectively store one or more matrices across all threads in a warp to the location indicated by the address operand *p*, in `.shared` state space. If no state space is provided, generic addressing is used, such that the address in *p* points into `.shared` space. If the generic address doesn't fall in `.shared` state space, then the behavior is undefined.

The `.shape` qualifier indicates the dimensions of the matrices being loaded. Each matrix element holds 16-bit or 8-bit data as indicated by the `.type` qualifier.

`.m16n8` shape is valid only for `.b8` type.

The values `.x1`, `.x2` and `.x4` for `.num` indicate one, two or four matrices respectively.

The mandatory `.sync` qualifier indicates that `stmatrix` causes the executing thread to wait until all threads in the warp execute the same `stmatrix` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `stmatrix` instruction. In conditionally executed code, an `stmatrix` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise the behavior is undefined.

The behavior of `stmatrix` is undefined if all threads do not use the same qualifiers, or if any thread in the warp has exited.

The source operand *r* is a brace-enclosed vector expression consisting of 1, 2, or 4 32-bit registers as per the value of `.num`. Each component of the vector expression holds a fragment from the corresponding matrix.

Supported addressing modes for *p* are described in [Addresses as Operands](#).

Consecutive instances of row need not be stored contiguously in memory. The eight addresses required for each matrix are provided by eight threads, depending upon the value of `.num` as shown in the following table. Each address corresponds to the start of a matrix row. Addresses `addr0`–`addr7` correspond to the rows of the first matrix, addresses `addr8`–`addr15` correspond to the rows of the second matrix, and so on.

<code>.num</code>	Threads 0–7	Threads 8–15	Threads 16–23	Threads 24–31
<code>.x1</code>	<code>addr0</code> – <code>addr7</code>	–	–	–
<code>.x2</code>	<code>addr0</code> – <code>addr7</code>	<code>addr8</code> – <code>addr15</code>	–	–
<code>.x4</code>	<code>addr0</code> – <code>addr7</code>	<code>addr8</code> – <code>addr15</code>	<code>addr16</code> – <code>addr23</code>	<code>addr24</code> – <code>addr31</code>

When storing 8x8 matrices, a group of four consecutive threads stores 16 bytes. The matrix addresses must be naturally aligned accordingly.

Each thread in a warp stores fragments of a row, with thread 0 storing the first fragment from its register *r*, and so on. A group of four threads stores an entire row of the matrix as shown in [Figure 107](#).

When `.num` = `.x2`, the elements of the second matrix are stored from the next source register in each thread as per the layout in above table. Similarly, when `.num` = `.x4`, elements of the third and fourth matrices are stored from the subsequent source registers in each thread.

For 16x8 matrix shape, each of the 32 threads in the warp provides four elements of data per matrix.

Each element in the source operand *r* is of type `.b32` and contains four 8 bit elements *e0*, *e1*, *e2*, *e3* with *e0* and *e3* containing the LSB and MSB respectively of register *r*.

Row\Col	0	1	2	3	4	5	6	7
0	T0:r	T0:r	T1:r	T1:r	T2:r	T2:r	T3:r	T3:r
1	T4:r	T4:r	T5:r	T5:r	T6:r	T6:r	T7:r	T7:r
2	T8:r	T8:r	T9:r	T9:r	T10:r	T10:r	T11:r	T11:r
3	T12:r	T12:r	T13:r	T13:r	T14:r	T14:r	T15:r	T15:r
4	T16:r	T16:r	T17:r	T17:r	T18:r	T18:r	T19:r	T19:r
5	T20:r	T20:r	T21:r	T21:r	T22:r	T22:r	T23:r	T23:r
6	T24:r	T24:r	T25:r	T25:r	T26:r	T26:r	T27:r	T27:r
7	T28:r	T28:r	T29:r	T29:r	T30:r	T30:r	T31:r	T31:r

Figure 107: stmatrix fragment layout for one 8x8 matrix with 16-bit elements

Row\Col	0	1	2	3	4	5	6	7
0	T0:e0	T0:e1	T1:e0	T1:e1	T2:e0	T2:e1	T3:e0	T3:e1
1	T4:e0	T4:e1	T5:e0	T5:e1	T6:e0	T6:e1	T7:e0	T7:e1
2	T8:e0	T8:e1	T9:e0	T9:e1	T10:e0	T10:e1	T11:e0	T11:e1
3	T12:e0	T12:e1	T13:e0	T13:e1	T14:e0	T14:e1	T15:e0	T15:e1
4	T16:e0	T16:e1	T17:e0	T17:e1	T18:e0	T18:e1	T19:e0	T19:e1
5	T20:e0	T20:e1	T21:e0	T21:e1	T22:e0	T22:e1	T23:e0	T23:e1
6	T24:e0	T24:e1	T25:e0	T25:e1	T26:e0	T26:e1	T27:e0	T27:e1
7	T28:e0	T28:e1	T29:e0	T29:e1	T30:e0	T30:e1	T31:e0	T31:e1
8	T0:e2	T0:e3	T1:e2	T1:e3	T2:e2	T2:e3	T3:e2	T3:e3
9	T4:e2	T4:e3	T5:e2	T5:e3	T6:e2	T6:e3	T7:e2	T7:e3
10	T8:e2	T8:e3	T9:e2	T9:e3	T10:e2	T10:e3	T11:e2	T11:e3
11	T12:e2	T12:e3	T13:e2	T13:e3	T14:e2	T14:e3	T15:e2	T15:e3
12	T16:e2	T16:e3	T17:e2	T17:e3	T18:e2	T18:e3	T19:e2	T19:e3
13	T20:e2	T20:e3	T21:e2	T21:e3	T22:e2	T22:e3	T23:e2	T23:e3
14	T24:e2	T24:e3	T25:e2	T25:e3	T26:e2	T26:e3	T27:e2	T27:e3
15	T28:e2	T28:e3	T29:e2	T29:e3	T30:e2	T30:e3	T31:e2	T31:e3

Figure 108: stmatrix fragment layout for one 16x8 matrix with 8 bit elements

Optional qualifier `.trans` indicates that the matrix is stored in column-major format. However, for 16x8 matrices, `.trans` is mandatory.

The `stmatrix` instruction is treated as a weak memory operation in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Support for `.m16n8` shape is introduced in PTX ISA version 8.6.

Support for `.b8` type with `stmatrix` is introduced in PTX ISA version 8.6.

Target ISA Notes

Requires `sm_90` or higher.

Shape `.m16n8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Type `.b8` with `stmatrix` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Examples

```
// Store a single 8x8 matrix using 64-bit addressing
.reg .b64 addr;
.reg .b32 r;
stmatrix.sync.aligned.m8n8.x1.shared.b16 [addr], {r};

// Store two 8x8 matrices in column-major format
.reg .b64 addr;
.reg .b32 r<2>;
stmatrix.sync.aligned.m8n8.x2.trans.shared::cta.b16 [addr], {r0, r1};

// Store four 8x8 matrices
.reg .b64 addr;
.reg .b32 r<4>;
stmatrix.sync.aligned.m8n8.x4.b16 [addr], {r0, r1, r2, r3};

// Store a single 16x8 matrix using generic addressing
.reg .b64 addr;
.reg .b32 r;
stmatrix.sync.aligned.m16n8.x1.trans.shared.b8 [addr], {r};

// Store two 16x8 matrices
.reg .b64 addr;
.reg .b32 r<2>;
stmatrix.sync.aligned.m16n8.x2.trans.shared::cta.b8 [addr], {r0, r1};

// Store four 16x8 matrices
.reg .b64 addr;
.reg .b32 r<4>;
stmatrix.sync.aligned.m16n8.x4.b8 [addr], {r0, r1, r2, r3};
```

9.7.14.5.17 Warp-level matrix transpose instruction: **movmatrix**

movmatrix

Transpose a matrix in registers across the warp.

Syntax

```
movmatrix.sync.aligned.shape.trans.type d, a;

.shape = {.m8n8};
.type  = {.b16};
```

Description

Move a row-major matrix across all threads in a warp, reading elements from source *a*, and writing the transposed elements to destination *d*.

The `.shape` qualifier indicates the dimensions of the matrix being transposed. Each matrix element holds 16-bit data as indicated by the `.type` qualifier.

The mandatory `.sync` qualifier indicates that `movmatrix` causes the executing thread to wait until all threads in the warp execute the same `movmatrix` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `movmatrix` instruction. In conditionally executed code, a `movmatrix` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise the behavior is undefined.

Operands *a* and *d* are 32-bit registers containing fragments of the input matrix and the resulting matrix respectively. The mandatory qualifier `.trans` indicates that the resulting matrix in *d* is a transpose of the input matrix specified by *a*.

Each thread in a warp holds a fragment of a row of the input matrix, with thread 0 holding the first fragment in register *a*, and so on. A group of four threads holds an entire row of the input matrix as shown in [Figure 109](#).

movmatrix: matrix fragment in source operand a								
	col0	col1	col2	col3	col4	col5	col6	col7
row0	%laneid = 0 src=a		%laneid = 1 src=a		%laneid = 2 src=a		%laneid = 3 src=a	
row1	%laneid = 4 src=a		%laneid = 5 src=a		%laneid = 6 src=a		%laneid = 7 src=a	
row2	%laneid = 8 src=a		%laneid = 9 src=a		%laneid = 10 src=a		%laneid = 11 src=a	
row3	%laneid = 12 src=a		%laneid = 13 src=a		%laneid = 14 src=a		%laneid = 15 src=a	
row4	%laneid = 16 src=a		%laneid = 17 src=a		%laneid = 18 src=a		%laneid = 19 src=a	
row5	%laneid = 20 src=a		%laneid = 21 src=a		%laneid = 22 src=a		%laneid = 23 src=a	
row6	%laneid = 24 src=a		%laneid = 25 src=a		%laneid = 26 src=a		%laneid = 27 src=a	
row7	%laneid = 28 src=a		%laneid = 29 src=a		%laneid = 30 src=a		%laneid = 31 src=a	

Figure 109: movmatrix source matrix fragment layout

Each thread in a warp holds a fragment of a column of the result matrix, with thread 0 holding the first fragment in register d, and so on. A group of four threads holds an entire column of the result matrix as shown in [Figure 110](#).

movmatrix: matrix fragment in destination operand d								
	row0	row1	row2	row3	row4	row5	row6	row7
col0	%laneid = 0 dst=d		%laneid = 1 dst=d		%laneid = 2 dst=d		%laneid = 3 dst=d	
col1	%laneid = 4 dst=d		%laneid = 5 dst=d		%laneid = 6 dst=d		%laneid = 7 dst=d	
col2	%laneid = 8 dst=d		%laneid = 9 dst=d		%laneid = 10 dst=d		%laneid = 11 dst=d	
col3	%laneid = 12 dst=d		%laneid = 13 dst=d		%laneid = 14 dst=d		%laneid = 15 dst=d	
col4	%laneid = 16 dst=d		%laneid = 17 dst=d		%laneid = 18 dst=d		%laneid = 19 dst=d	
col5	%laneid = 20 dst=d		%laneid = 21 dst=d		%laneid = 22 dst=d		%laneid = 23 dst=d	
col6	%laneid = 24 dst=d		%laneid = 25 dst=d		%laneid = 26 dst=d		%laneid = 27 dst=d	
col7	%laneid = 28 dst=d		%laneid = 29 dst=d		%laneid = 30 dst=d		%laneid = 31 dst=d	

Figure 110: movmatrix result matrix fragment layout

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_75 or higher.

Examples

```
.reg .b32 d, a;
movmatrix.sync.aligned.m8n8.trans.b16 d, a;
```

9.7.14.6 Matrix multiply-accumulate operation using `mma.sp` instruction with sparse matrix A

This section describes warp-level `mma.sp{:ordered_metadata}` instruction with sparse matrix A. This variant of the `mma` operation can be used when A is a structured sparse matrix with 50% zeros in each row distributed in a shape-specific granularity. For an $M \times N \times K$ sparse `mma.sp{:ordered_metadata}` operation, the $M \times K$ matrix A is packed into $M \times K/2$ elements. For each K -wide row of matrix A, 50% elements are zeros and the remaining $K/2$ non-zero elements are packed in the operand representing matrix A. The mapping of these $K/2$ elements to the corresponding K -wide row is provided explicitly as metadata. The mapping of these $K/2$ elements to the corresponding K -wide row is provided explicitly as metadata.

9.7.14.6.1 Sparse matrix storage

Granularity of sparse matrix A is defined as the ratio of the number of non-zero elements in a sub-chunk of the matrix row to the total number of elements in that sub-chunk where the size of the sub-chunk is shape-specific. For example, in a 16×16 matrix A, sparsity is expected to be at 2:4 granularity, i.e. each 4-element vector (i.e. a sub-chunk of 4 consecutive elements) of a matrix row contains 2 zeros. Index of each non-zero element in a sub-chunk is stored in the metadata operand. Values `0b0000`, `0b0101`, `0b1010`, `0b1111` are invalid values for metadata and will result in undefined behavior. In a group of four consecutive threads, one or more threads store the metadata for the whole group depending upon the matrix shape. These threads are specified using an additional *sparsity selector* operand.

Figure 111 shows an example of a 16×16 matrix A represented in sparse format and sparsity selector indicating which thread in a group of four consecutive threads stores the metadata.

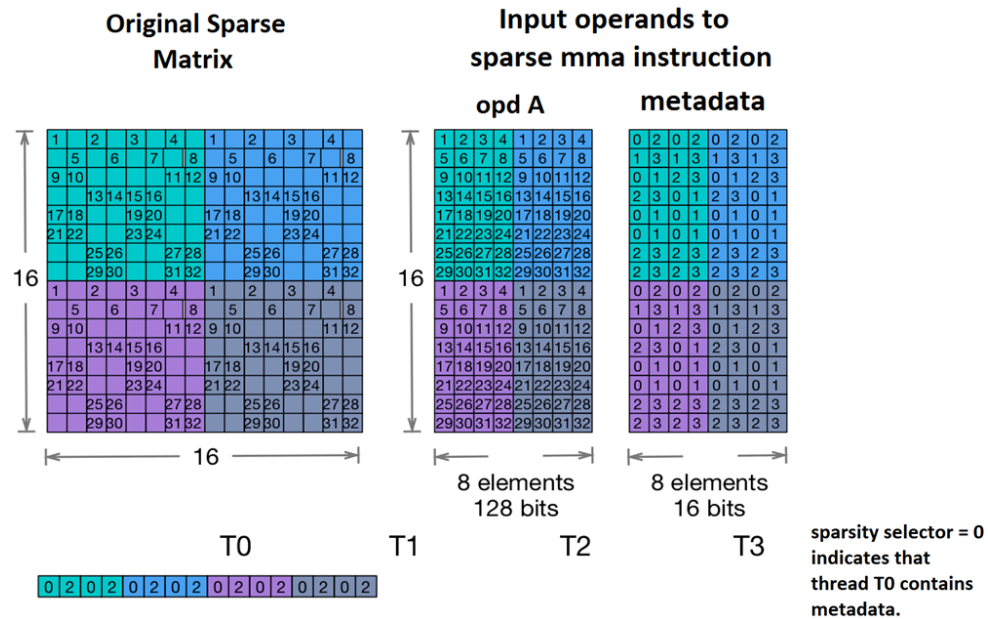


Figure 111: Sparse MMA storage example

Granularities for different matrix shapes and data types are described below.

Sparse `mma.sp{: :ordered_metadata}` with half-precision and `.bf16` type

For the `.m16n8k16` and `.m16n8k32` `mma.sp{: :ordered_metadata}` operations, matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A has two zeros and two non-zero elements. Only the two non-zero elements are stored in the operand representing matrix A and their positions in the four-wide chunk in matrix A are indicated by two 2-bit indices in the metadata operand. For `mma.sp{: :ordered_metadata}`, `0b0100`, `0b1000`, `0b1001`, `0b1100`, `0b1101`, `0b1110` are the meaningful values of indices; any other values result in an undefined behavior.

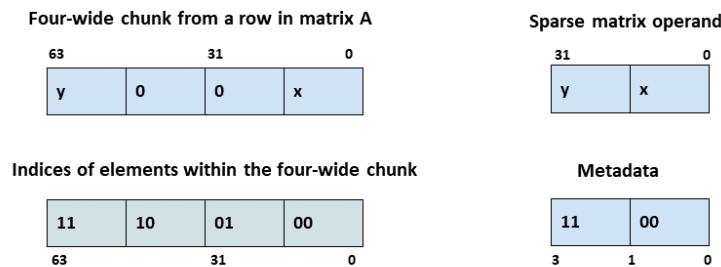


Figure 112: Sparse MMA metadata example for `.f16/.bf16` type.

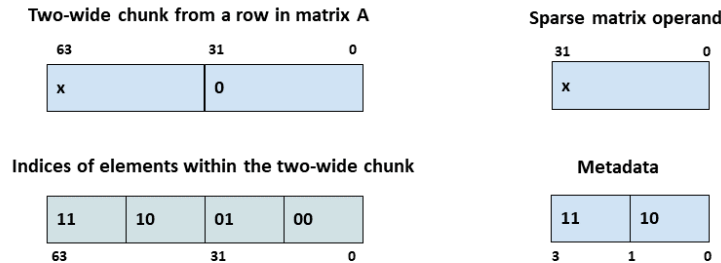
The sparsity selector indicates the threads which contribute metadata as listed below:

- `m16n8k16`: One thread within a group of four consecutive threads contributes the metadata for the entire group. This thread is indicated by a value in `{0, 1, 2, 3}`.
- `m16n8k32`: A thread-pair within a group of four consecutive threads contributes the sparsity metadata. Hence, the sparsity selector must be either 0 (threads T0, T1) or 1 (threads T2, T3); any other value results in an undefined behavior.

Sparse `mma.sp{: :ordered_metadata}` with `.tf32` type

When matrix A has `.tf32` elements, matrix A is structured sparse at a granularity of 1:2. In other words, each chunk of two adjacent elements in a row of matrix A has one zero and one non-zero element. Only the non-zero elements are stored in the operand for matrix A and their positions in a two-wide chunk in matrix A are indicated by the 4-bit index in the metadata. `0b1110` and `0b0100` are the only meaningful index values; any other values result in an undefined behavior.

The sparsity selector indicates the threads which contribute metadata as listed below:

Figure 113: Sparse MMA metadata example for `.tf32` type.

- `m16n8k8`: One thread within a group of four consecutive threads contributes the metadata for the entire group. This thread is indicated by a value in `{0, 1, 2, 3}`.
- `m16n8k16`: A thread-pair within a group of four consecutive threads contributes the sparsity metadata. Hence, the sparsity selector must be either 0 (threads T0, T1) or 1 (threads T2, T3); any other value results in an undefined behavior.

Sparse `mma.sp{:ordered_metadata}` with integer type

When matrices A and B have `.u8/.s8` elements, matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A have two zeroes and two non-zero elements. Only the two non-zero elements are stored in sparse matrix and their positions in the four-wide chunk are indicated by two 2-bit indices in the metadata. For `mma.sp{:ordered_metadata}`, `0b0100`, `0b1000`, `0b1001`, `0b1100`, `0b1101`, `0b1110` are the meaningful values of indices; any other values result in an undefined behavior.

when matrices A and B have `.u4/.s4` elements, matrix A is pair-wise structured sparse at a granularity of 4:8. In other words, each chunk of eight adjacent elements in a row of matrix A has four zeroes and four non-zero values. Further, the zero and non-zero values are clustered in sub-chunks of two elements each within the eight-wide chunk. i.e., each two-wide sub-chunk within the eight-wide chunk must be all zeroes or all non-zeros. Only the four non-zero values are stored in sparse matrix and the positions of the two two-wide sub-chunks with non-zero values in the eight-wide chunk of a row of matrix A are indicated by two 2-bit indices in the metadata. For `mma.sp{:ordered_metadata}`, `0b0100`, `0b1000`, `0b1001`, `0b1100`, `0b1101`, `0b1110` are the meaningful values of indices; any other values result in an undefined behavior.

The sparsity selector indicates the threads which contribute metadata as listed below:

- `m16n8k32` with `.u8/.s8` type and `m16n8k64` with `.u4/.s4` type: A thread-pair within a group of

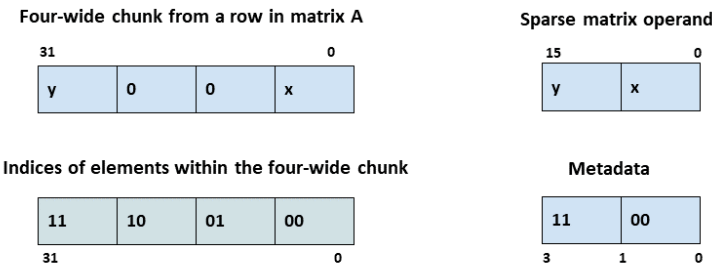


Figure 114: Sparse MMA metadata example for .u8/.s8 type.

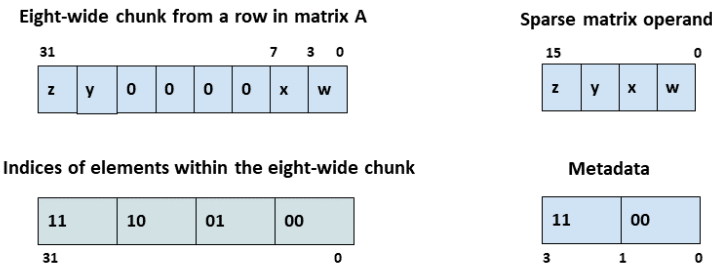


Figure 115: Sparse MMA metadata example for .u4/.s4 type.

four consecutive threads contributes the sparsity metadata. Hence, the sparsity selector must be either 0 (threads T0, T1) or 1 (threads T2, T3); any other value results in an undefined behavior.

- **m16n8k64** with `.u8/.s8` type and **m16n8k128** with `.u4/.s4` type: All threads within a group of four consecutive threads contribute the sparsity metadata. Hence, the sparsity selector in this case must be 0. Any other value of sparsity selector results in an undefined behavior.

Sparse `mma.sp{:ordered_metadata}` operating on `.e4m3/.e5m2/.e3m2/.e2m3/.e2m1` type with `.kind::f8f6f4` or `.kind::mxf8f6f4`

When matrices A and B have `.e4m3/.e5m2/.e3m2/.e2m3/.e2m1` elements, matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A have two zeroes and two non-zero elements. Only the two non-zero elements are stored in sparse matrix and their positions in the four-wide chunk are indicated by two 2-bit indices in the metadata. `0b0100`, `0b1000`, `0b1001`, `0b1100`, `0b1101`, `0b1110` are the meaningful values of indices; any other values result in an undefined behavior.

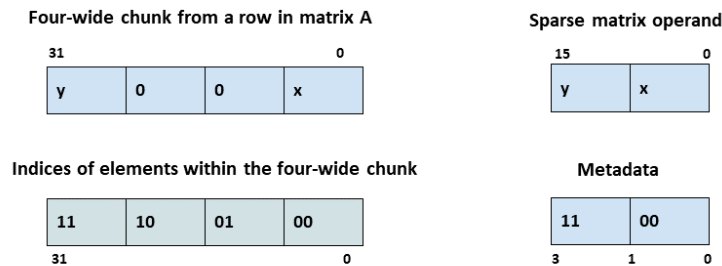


Figure 116: Sparse MMA metadata example for `.e4m3/.e5m2/.e3m2/.e2m3/.e2m1` type.

The sparsity selector indicates the threads which contribute metadata as listed below:

- **m16n8k64:** All threads within a group of four consecutive threads contribute the sparsity metadata. Hence, the sparsity selector in this case must be 0. Any other value of sparsity selector results in an undefined behavior.

Sparse `mma.sp::ordered_metadata` operating on `.e2m1` type with `.kind::mxf4` or `.kind::mxf4nvf4`

When matrices A and B have `.e2m1` elements, matrix A is pair-wise structured sparse at a granularity of 4:8. In other words, each chunk of eight adjacent elements in a row of matrix A has four zeroes and four non-zero values. Further, the zero and non-zero values are clustered in sub-chunks of two elements each within the eight-wide chunk. i.e., each two-wide sub-chunk within the eight-wide chunk must be all zeroes or all non-zeros. Only the four non-zero values are stored in sparse matrix and the positions of the two two-wide sub-chunks with non-zero values in the eight-wide chunk of a row of matrix A are indicated by two 2-bit indices in the metadata. `0b0100`, `0b1000`, `0b1001`, `0b1100`, `0b1101`, `0b1110` are the meaningful values of indices; any other values result in an undefined behavior.

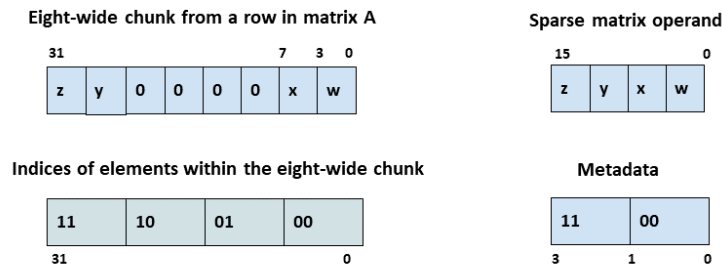


Figure 117: Sparse MMA metadata example for `.e2m1` type with `.kind::mxf4` or `.kind::mxf4nvf4`

The sparsity selector indicates the threads which contribute metadata as listed below:

- **m16n8k128:** All threads within a group of four consecutive threads contribute the sparsity metadata. Hence, the sparsity selector in this case must be 0. Any other value of sparsity selector results in an undefined behavior.

9.7.14.6.2 Matrix fragments for multiply-accumulate operation with sparse matrix A

In this section we describe how the contents of thread registers are associated with fragments of various matrices and the sparsity metadata. The following conventions are used throughout this section:

- For matrix A, only the layout of a fragment is described in terms of register vector sizes and their association with the matrix data.
- For matrix B, when the combination of matrix dimension and the supported data type is not already covered in *Matrix multiply-accumulate operation using mma instruction*, a pictorial representation of matrix fragments is provided.

- For matrices C and D, since the matrix dimension - data type combination is the same for all supported shapes, and is already covered in *Matrix multiply-accumulate operation using mma instruction*, the pictorial representations of matrix fragments are not included in this section.
- For the metadata operand, pictorial representations of the association between indices of the elements of matrix A and the contents of the metadata operand are included. $T_k: [m..n]$ present in cell $[x][y..z]$ indicates that bits m through n (with m being higher) in the metadata operand of thread with $\%laneid=k$ contains the indices of the non-zero elements from the chunk $[x][y]..[x][z]$ of matrix A.

9.7.14.6.2.1 Matrix Fragments for sparse mma.m16n8k16 with .f16 and .bf16 types

A warp executing sparse mma.m16n8k16 with .f16 / .bf16 floating point type will compute an MMA operation of shape .m16n8k16.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

- Multiplicand A:

.atype	Fragment	Elements
.f16 / .bf16	A vector expression containing two .b32 registers, with each register containing two non-zero .f16 / .bf16 elements out of 4 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .

The layout of the fragments held by different threads is shown in Figure 118.

Row\Column	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:{a0,a1}				T1:{a0,a1}				T2:{a0,a1}				T3:{a0,a1}			
1	T4:{a0,a1}				T5:{a0,a1}				T6:{a0,a1}				T6:{a0,a1}			
2																
...																
7	T28:{a0,a1}				T29:{a0,a1}				T30:{a0,a1}				T31:{a0,a1}			
8	T0:{a2,a3}				T1:{a2,a3}				T2:{a2,a3}				T3:{a2,a3}			
9	T4:{a2,a3}				T5:{a2,a3}				T6:{a2,a3}				T7:{a2,a3}			
10																
...																
15	T28:{a2,a3}				T29:{a2,a3}				T30:{a2,a3}				T31:{a2,a3}			

Figure 118: Sparse MMA .m16n8k16 fragment layout for matrix A with .f16/.bf16 type.

The row and column of a matrix fragment can be computed as:

```
groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for a0 and a1
      groupID + 8      for a2 and a3

col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                               // as described in Sparse matrix storage

Where
firstcol = threadID_in_group * 4
lastcol  = firstcol + 3
```

- Matrix fragments for multiplicand B and accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k16 with floating point type* for .f16/.b16 formats.
- Metadata: A .b32 register containing 16 2-bit vectors each storing the index of a non-zero element of a 4-wide chunk of matrix A as shown in Figure 119.

Row\Column	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T _i : [3..0]				T _i : [7..4]				T _i : [11..8]				T _i : [15..12]			
1	T _{i+4} : [3..0]				T _{i+4} : [7..4]				T _{i+4} : [11..8]				T _{i+4} : [15..12]			
2																
...																
7	T _{i+28} : [3..0]				T _{i+28} : [7..4]				T _{i+28} : [11..8]				T _{i+28} : [15..12]			
8	T _i : [19..16]				T _i : [23..20]				T _i : [27..24]				T _i : [31..28]			
9	T _{i+4} : [19..16]				T _{i+4} : [23..20]				T _{i+4} : [27..24]				T _{i+4} : [31..28]			
10																
...																
15	T _{i+28} : [19..16]				T _{i+28} : [23..20]				T _{i+28} : [27..24]				T _{i+28} : [31..28]			

Figure 119: Sparse MMA .m16n8k16 metadata layout for .f16/.bf16 type.

9.7.14.6.2.2 Matrix Fragments for sparse mma.m16n8k32 with .f16 and .bf16 types

A warp executing sparse mma.m16n8k32 with .f16 / .bf16 floating point type will compute an MMA operation of shape .m16n8k32.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements
.f16 / .bf16	A vector expression containing four .b32 registers, with each register containing two non-zero .f16 / .bf16 elements out of 4 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .

The layout of the fragments held by different threads is shown in Figure 120.

Row\Column	0..3	4..7	8..11	12..15	16..19	20..23	24..27	28..31
0	T0:{a0,a1}	T1:{a0,a1}	T2:{a0,a1}	T3:{a0,a1}	T0:{a4,a5}	T1:{a4,a5}	T2:{a4,a5}	T3:{a4,a5}
1	T4:{a0,a1}	T5:{a0,a1}	T6:{a0,a1}	T7:{a0,a1}	T4:{a4,a5}	T5:{a4,a5}	T6:{a4,a5}	T7:{a4,a5}
2								
...								
7	T28:{a0,a1}	T29:{a0,a1}	T30:{a0,a1}	T31:{a0,a1}	T28:{a4,a5}	T29:{a4,a5}	T30:{a4,a5}	T31:{a4,a5}
8	T0:{a2,a3}	T1:{a2,a3}	T2:{a2,a3}	T3:{a2,a3}	T0:{a6,a7}	T1:{a6,a7}	T2:{a6,a7}	T3:{a6,a7}
9	T4:{a2,a3}	T5:{a2,a3}	T6:{a2,a3}	T7:{a2,a3}	T4:{a6,a7}	T5:{a6,a7}	T6:{a6,a7}	T7:{a6,a7}
10								
...								
15	T28:{a2,a3}	T29:{a2,a3}	T30:{a2,a3}	T31:{a2,a3}	T28:{a6,a7}	T29:{a6,a7}	T30:{a6,a7}	T31:{a6,a7}

Figure 120: Sparse MMA .m16n8k32 fragment layout for matrix A with .f16/.bf16 type.

The row and column of a matrix fragment can be computed as:

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for ai where  0 <= i < 2 || 4 <= i < 6
      groupID + 8      Otherwise

col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                             // as described in Sparse matrix storage

```

(continues on next page)

(continued from previous page)

Where
 $\text{firstcol} = \text{threadID_in_group} * 4$ For a_i where $i < 4$
 $(\text{threadID_in_group} * 4) + 16$ for a_i where $i \geq 4$
 $\text{lastcol} = \text{firstcol} + 3$

► Multiplicand B:

.atype	Fragment	Elements (low to high)
.f16 / .bf16	A vector expression containing four .b32 registers, each containing two .f16 / .bf16 elements from matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in Figure 121.

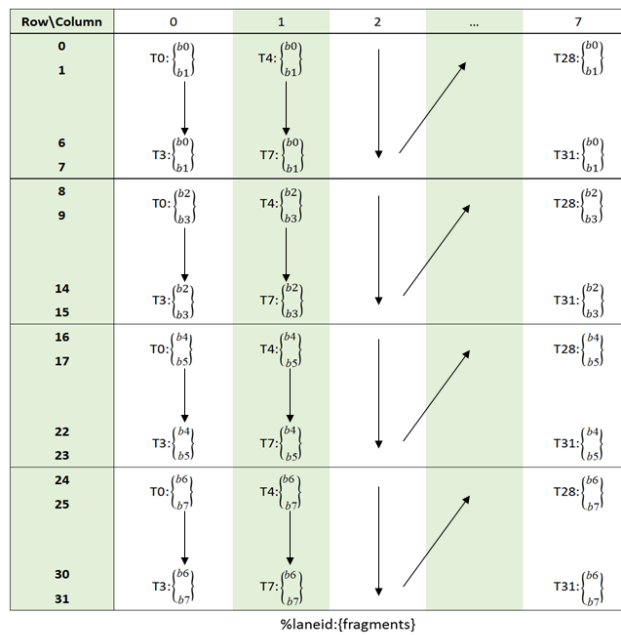


Figure 121: Sparse MMA .m16n8k32 fragment layout for matrix B with .f16/.bf16 type.

- Matrix fragments for accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k16 with floating point type* for .f16/.b16 formats.
- Metadata: A .b32 register containing 16 2-bit vectors with each pair of 2-bit vectors storing the indices of two non-zero element from a 4-wide chunk of matrix A as shown in Figure 122.

Row\Column	0..3	4..7	8..11	12..15	16..19	20..23	24..27	28..31
0	$T_{2i}: [3..0]$	$T_{2i}: [7..4]$	$T_{2i}: [11..8]$	$T_{2i}: [15..12]$	$T_{2i+1}: [3..0]$	$T_{2i+1}: [7..4]$	$T_{2i+1}: [11..8]$	$T_{2i+1}: [15..12]$
1	$T_{2i+4}: [3..0]$	$T_{2i+4}: [7..4]$	$T_{2i+4}: [11..8]$	$T_{2i+4}: [15..12]$	$T_{2i+5}: [3..0]$	$T_{2i+5}: [7..4]$	$T_{2i+5}: [11..8]$	$T_{2i+5}: [15..12]$
2								
...								
7	$T_{2i+28}: [3..0]$	$T_{2i+28}: [7..4]$	$T_{2i+28}: [11..8]$	$T_{2i+28}: [15..12]$	$T_{2i+29}: [3..0]$	$T_{2i+29}: [7..4]$	$T_{2i+29}: [11..8]$	$T_{2i+29}: [15..12]$
8	$T_{2i}: [19..16]$	$T_{2i}: [23..20]$	$T_{2i}: [27..24]$	$T_{2i}: [31..28]$	$T_{2i+1}: [19..16]$	$T_{2i+1}: [23..20]$	$T_{2i+1}: [27..24]$	$T_{2i+1}: [31..28]$
9	$T_{2i+4}: [19..16]$	$T_{2i+4}: [23..20]$	$T_{2i+4}: [27..24]$	$T_{2i+4}: [31..28]$	$T_{2i+5}: [19..16]$	$T_{2i+5}: [23..20]$	$T_{2i+5}: [27..24]$	$T_{2i+5}: [31..28]$
10								
...								
15	$T_{2i+28}: [19..16]$	$T_{2i+28}: [23..20]$	$T_{2i+28}: [27..24]$	$T_{2i+28}: [31..28]$	$T_{2i+29}: [19..16]$	$T_{2i+29}: [23..20]$	$T_{2i+29}: [27..24]$	$T_{2i+29}: [31..28]$

Figure 122: Sparse MMA .m16n8k32 metadata layout for .f16/.bf16 type.

9.7.14.6.2.3 Matrix Fragments for sparse mma.m16n8k16 with .tf32 floating point type

A warp executing sparse mma.m16n8k16 with .tf32 floating point type will compute an MMA operation of shape .m16n8k16.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements
.tf32	A vector expression containing four .b32 registers, with each register containing one non-zero .tf32 element out of 2 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .

The layout of the fragments held by different threads is shown in Figure 123.

The row and column of a matrix fragment can be computed as:

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for a0 and a2
          groupID + 8    for a1 and a3

col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                               // as described in Sparse matrix storage

```

(continues on next page)

Row\Column	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:a0	T1:a0	T2:a0	T3:a0	T0:a2	T1:a2	T2:a2	T3:a2								
1	T4:a0	T5:a0	T6:a0	T7:a0	T4:a2	T5:a2	T6:a2	T7:a2								
2																
...																
7	T28:a0	T29:a0	T30:a0	T31:a0	T28:a2	T29:a2	T30:a2	T31:a2								
8	T0:a1	T1:a1	T2:a1	T3:a1	T0:a3	T1:a3	T2:a3	T3:a3								
9	T4:a1	T5:a1	T6:a1	T7:a1	T4:a3	T5:a3	T6:a3	T7:a3								
10																
...																
15	T28:a1	T29:a1	T30:a1	T31:a1	T28:a3	T29:a3	T30:a3	T31:a3								

Figure 123: Sparse MMA .m16n8k16 fragment layout for matrix A with .tf32 type.

(continued from previous page)

Where
`firstcol = threadID_in_group * 2` **for** a0 and a1
 `(threadID_in_group * 2) + 8` **for** a2 and a3
`lastcol = firstcol + 1`

► Multiplicand B:

.atype	Fragment	Elements (low to high)
.tf32	A vector expression containing four .b32 registers, each containing four .tf32 elements from matrix B.	b0, b1, b2, b3

The layout of the fragments held by different threads is shown in Figure 124.

- Matrix fragments for accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k16 with floating point type*.
- Metadata: A .b32 register containing 8 4-bit vectors each storing the index of a non-zero element of a 2-wide chunk of matrix A as shown in Figure 125.

Row\Column	0	1	2	...	7
0	T0:B0	T4:B0	↓		T28:B0
1	T1:B0	T5:B0			T29:B0
2	T2:B0	T6:B0			T30:B0
3	T3:B0	T7:B0			T31:B0
4	T0:B1	T4:B1	↓		T28:B1
5	T1:B1	T5:B1			T29:B1
6	T2:B1	T6:B1			T30:B1
7	T3:B1	T7:B1			T31:B1
8	T0:B2	T4:B2	↓		T28:B2
9	T1:B2	T5:B2			T29:B2
10	T2:B2	T6:B2			T30:B2
11	T3:B2	T7:B2			T31:B2
12	T0:B3	T4:B3	↓		T28:B3
13	T1:B3	T5:B3			T29:B3
14	T2:B3	T6:B3			T30:B3
15	T3:B3	T7:B3			T31:B3

%laneid:{fragments}

Figure 124: Sparse MMA .m16n8k16 fragment layout for matrix B with .tf32 type.

Row\Column	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	$T_{2i}:[3..0]$	$T_{2i}:[7..4]$	$T_{2i}:[11..8]$	$T_{2i}:[15..12]$	$T_{2i+1}:[3..0]$	$T_{2i+1}:[7..4]$	$T_{2i+1}:[11..8]$	$T_{2i+1}:[15..12]$								
1	$T_{2i+4}:[3..0]$	$T_{2i+4}:[7..4]$	$T_{2i+4}:[11..8]$	$T_{2i+4}:[15..12]$	$T_{2i+5}:[3..0]$	$T_{2i+5}:[7..4]$	$T_{2i+5}:[11..8]$	$T_{2i+5}:[15..12]$								
2																
...																
7	$T_{2i+28}:[3..0]$	$T_{2i+28}:[7..4]$	$T_{2i+28}:[11..8]$	$T_{2i+28}:[15..12]$	$T_{2i+29}:[3..0]$	$T_{2i+29}:[7..4]$	$T_{2i+29}:[11..8]$	$T_{2i+29}:[15..12]$								
8	$T_{2i}:[19..16]$	$T_{2i}:[23..20]$	$T_{2i}:[27..24]$	$T_{2i}:[31..28]$	$T_{2i+1}:[19..16]$	$T_{2i+1}:[23..20]$	$T_{2i+1}:[27..24]$	$T_{2i+1}:[31..28]$								
9	$T_{2i+4}:[19..16]$	$T_{2i+4}:[23..20]$	$T_{2i+4}:[27..24]$	$T_{2i+4}:[31..28]$	$T_{2i+5}:[19..16]$	$T_{2i+5}:[23..20]$	$T_{2i+5}:[27..24]$	$T_{2i+5}:[31..28]$								
10																
...																
15	$T_{2i+28}:[19..16]$	$T_{2i+28}:[23..20]$	$T_{2i+28}:[27..24]$	$T_{2i+28}:[31..28]$	$T_{2i+29}:[19..16]$	$T_{2i+29}:[23..20]$	$T_{2i+29}:[27..24]$	$T_{2i+29}:[31..28]$								

Figure 125: Sparse MMA .m16n8k16 metadata layout for .tf32 type.

9.7.14.6.2.4 Matrix Fragments for sparse mma.m16n8k8 with .tf32 floating point type

A warp executing sparse mma.m16n8k8 with .tf32 floating point type will compute an MMA operation of shape .m16n8k8.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements
.tf32	A vector expression containing two .b32 registers, each containing one non-zero .tf32 element out of 2 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .

The layout of the fragments held by different threads is shown in Figure 126.

The row and column of a matrix fragment can be computed as:

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for a0
        groupID + 8    for a1

col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                                // as described in Sparse matrix storage

```

(continues on next page)

Row\Column	0	1	2	3	4	5	6	7
0	T0:a0		T1:a0		T2:a0		T3:a0	
1	T4:a0		T5:a0		T6:a0		T7:a0	
2								
...								
7	T28:a0		T29:a0		T30:a0		T31:a0	
8	T0:a1		T1:a1		T2:a1		T3:a1	
9	T4:a1		T5:a1		T6:a1		T7:a1	
10								
...								
15	T28:a1		T29:a1		T30:a1		T31:a1	

Figure 126: Sparse MMA .m16n8k8 fragment layout for matrix A with .tf32 type.

(continued from previous page)

Where
`firstcol = threadID_in_group * 2`
`lastcol = firstcol + 1`

- Matrix fragments for multiplicand B and accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k8* for .tf32 format.
- Metadata: A .b32 register containing 8 4-bit vectors each storing the index of a non-zero element of a 2-wide chunk of matrix A as shown in Figure 127.

9.7.14.6.2.5 Matrix Fragments for sparse mma.m16n8k32 with .u8 / .s8 integer type

A warp executing sparse mma.m16n8k32 with .u8 / .s8 integer type will compute an MMA operation of shape .m16n8k32.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

- Multiplicand A:

.atype	Fragment	Elements
.u8 / .s8	A vector expression containing two .b32 registers, with each register containing four non-zero .u8 / .s8 elements out of 8 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .

Row\Column	0	1	2	3	4	5	6	7
0	$T_i: [3..0]$		$T_i: [7..4]$		$T_i: [11..8]$		$T_i: [15..12]$	
1	$T_{i+4}: [3..0]$		$T_{i+4}: [7..4]$		$T_{i+4}: [11..8]$		$T_{i+4}: [15..12]$	
2								
...								
7	$T_{i+28}: [3..0]$		$T_{i+28}: [7..4]$		$T_{i+28}: [11..8]$		$T_{i+28}: [15..12]$	
8	$T_i: [19..16]$		$T_i: [23..20]$		$T_i: [27..24]$		$T_i: [31..28]$	
9	$T_{i+4}: [19..16]$		$T_{i+4}: [23..20]$		$T_{i+4}: [27..24]$		$T_{i+4}: [31..28]$	
10								
...								
15	$T_{i+28}: [19..16]$		$T_{i+28}: [23..20]$		$T_{i+28}: [27..24]$		$T_{i+28}: [31..28]$	

Figure 127: Sparse MMA .m16n8k8 metadata layout for .tf32 type.

The layout of the fragments held by different threads is shown in Figure 128.

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for ai where 0 <= i < 4
      groupID + 8      Otherwise

col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                               // as described in Sparse matrix storage

Where
firstcol = threadID_in_group * 8
lastcol  = firstcol + 7

```

- Matrix fragments for multiplicand B and accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k32*.
- Metadata: A .b32 register containing 16 2-bit vectors with each pair of 2-bit vectors storing the indices of two non-zero elements from a 4-wide chunk of matrix A as shown in Figure 129.

Row\Column	0 ... 7	8 ... 15	16 ... 23	24 ... 31
0	T0:{a0,a1,a2,a3}	T1:{a0,a1,a2,a3}	T2:{a0,a1,a2,a3}	T3:{a0,a1,a2,a3}
1	T4:{a0,a1,a2,a3}	T5:{a0,a1,a2,a3}	T6:{a0,a1,a2,a3}	T7:{a0,a1,a2,a3}
2				
...				
7	T28:{a0,a1,a2,a3}	T29:{a0,a1,a2,a3}	T30:{a0,a1,a2,a3}	T31:{a0,a1,a2,a3}
8	T0:{a4,a5,a6,a7}	T1:{a4,a5,a6,a7}	T2:{a4,a5,a6,a7}	T3:{a4,a5,a6,a7}
9	T4:{a4,a5,a6,a7}	T5:{a4,a5,a6,a7}	T6:{a4,a5,a6,a7}	T7:{a4,a5,a6,a7}
10				
...				
15	T28:{a4,a5,a6,a7}	T29:{a4,a5,a6,a7}	T30:{a4,a5,a6,a7}	T31:{a4,a5,a6,a7}

Figure 128: Sparse MMA .m16n8k32 fragment layout for matrix A with .u8/.s8 type.

Row\Column	0..3	4..7	8..11	12..15	16..19	20..23	24..27	28..31
0	T _{2i} : [3..0]	T _{2i} : [7..4]	T _{2i} : [11..8]	T _{2i} : [15..12]	T _{2i} : [19..16]	T _{2i} : [23..20]	T _{2i} : [27..24]	T _{2i} : [31..28]
1	T _{2i+4} : [3..0]	T _{2i+4} : [7..4]	T _{2i+4} : [11..8]	T _{2i+4} : [15..12]	T _{2i+4} : [19..16]	T _{2i+4} : [23..20]	T _{2i+4} : [27..24]	T _{2i+4} : [31..28]
2								
...								
7	T _{2i+28} : [3..0]	T _{2i+28} : [7..4]	T _{2i+28} : [11..8]	T _{2i+28} : [15..12]	T _{2i+28} : [19..16]	T _{2i+28} : [23..20]	T _{2i+28} : [27..24]	T _{2i+28} : [31..28]
8	T _{2i+1} : [3..0]	T _{2i+1} : [7..4]	T _{2i+1} : [11..8]	T _{2i+1} : [15..12]	T _{2i+1} : [19..16]	T _{2i+1} : [23..20]	T _{2i+1} : [27..24]	T _{2i+1} : [31..28]
9	T _{2i+5} : [3..0]	T _{2i+5} : [7..4]	T _{2i+5} : [11..8]	T _{2i+5} : [15..12]	T _{2i+5} : [19..16]	T _{2i+5} : [23..20]	T _{2i+5} : [27..24]	T _{2i+5} : [31..28]
10								
...								
15	T _{2i+29} : [3..0]	T _{2i+29} : [7..4]	T _{2i+29} : [11..8]	T _{2i+29} : [15..12]	T _{2i+29} : [19..16]	T _{2i+29} : [23..20]	T _{2i+29} : [27..24]	T _{2i+29} : [31..28]

Figure 129: Sparse MMA .m16n8k32 metadata layout for .u8/.s8 type.

9.7.14.6.2.6 Matrix Fragments for sparse mma.m16n8k64 with .u8 / .s8 / .e4m3 / .e5m2 type

A warp executing sparse mma.m16n8k64 with .u8 / .s8 / .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 type will compute an MMA operation of shape .m16n8k64.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements
.u8 / .s8	A vector expression containing four .b32 registers, with each register containing four non-zero .u8 / .s8 elements out of 8 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .
.e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1	A vector expression containing four .b32 registers, with each register containing four non-zero .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 elements out of 8 consecutive elements from matrix A.	

The layout of the fragments held by different threads is shown in Figure 130 and Figure 131.

Row\Column	0 ... 7	8 ... 15	16 ... 23	24 ... 31
0	T0:{a0,a1,a2,a3}	T1:{a0,a1,a2,a3}	T2:{a0,a1,a2,a3}	T3:{a0,a1,a2,a3}
1	T4:{a0,a1,a2,a3}	T5:{a0,a1,a2,a3}	T6:{a0,a1,a2,a3}	T7:{a0,a1,a2,a3}
2				
...				
7	T28:{a0,a1,a2,a3}	T29:{a0,a1,a2,a3}	T30:{a0,a1,a2,a3}	T31:{a0,a1,a2,a3}
8	T0:{a4,a5,a6,a7}	T1:{a4,a5,a6,a7}	T2:{a4,a5,a6,a7}	T3:{a4,a5,a6,a7}
9	T4:{a4,a5,a6,a7}	T5:{a4,a5,a6,a7}	T6:{a4,a5,a6,a7}	T7:{a4,a5,a6,a7}
10				
...				
15	T28:{a4,a5,a6,a7}	T29:{a4,a5,a6,a7}	T30:{a4,a5,a6,a7}	T31:{a4,a5,a6,a7}

Figure 130: Sparse MMA .m16n8k64 fragment layout for columns 0–31 of matrix A with .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID          for ai where  0 <= i < 4 || 8 <= i < 12

```

(continues on next page)

Row\Column	32 ... 39	40 ... 47	48 ... 55	56 ... 63
0	T0:{a8,a9,a10,a11}	T1:{a8,a9,a10,a11}	T2:{a8,a9,a10,a11}	T3:{a8,a9,a10,a11}
1	T4:{a8,a9,a10,a11}	T5:{a8,a9,a10,a11}	T6:{a8,a9,a10,a11}	T7:{a8,a9,a10,a11}
2				
...				
7	T28:{a8,a9,a10,a11}	T29:{a8,a9,a10,a11}	T30:{a8,a9,a10,a11}	T31:{a8,a9,a10,a11}
8	T0:{a12,a13,a14,a15}	T1:{a12,a13,a14,a15}	T2:{a12,a13,a14,a15}	T3:{a12,a13,a14,a15}
9	T4:{a12,a13,a14,a15}	T5:{a12,a13,a14,a15}	T6:{a12,a13,a14,a15}	T7:{a12,a13,a14,a15}
10				
...				
15	T28:{a12,a13,a14,a15}	T29:{a12,a13,a14,a15}	T30:{a12,a13,a14,a15}	T31:{a12,a13,a14,a15}

Figure 131: Sparse MMA .m16n8k64 fragment layout for columns 32–63 of matrix A with .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

(continued from previous page)

groupID + 8	Otherwise
col = [firstcol ... lastcol]	// As per the mapping of non-zero elements // as described in Sparse matrix storage
Where	
firstcol = threadID_in_group * 8	For ai where i < 8
(threadID_in_group * 8) + 32	For ai where i >= 8
lastcol = firstcol + 7	

► Multiplicand B:

.btype	Fragment	Elements (low to high)
.u8 / .s8	A vector expression containing four .b32 registers, each containing four .u8 / .s8 elements from matrix B.	b0, b1, b2, b3, ..., b15
.e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1	A vector expression containing four .b32 registers, each containing four .e4m3 / .e5m2 / .e3m2 / .e2m3 / .e2m1 elements from matrix B.	

The layout of the fragments held by different threads is shown in [Figure 132](#), [Figure 133](#), [Figure 134](#) and [Figure 135](#).

Row\Col	0	1	2	...	7
0 ... 3	T0: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$	T4: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$			T28: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$
4 ... 7	T1: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$	T5: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$			T29: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$
8 ... 11	T2: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$	T6: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$			T30: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$
12 ... 15	T3: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$	T7: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$			T31: $\begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$

%laneid:{fragments}

Figure 132: Sparse MMA .m16n8k64 fragment layout for rows 0–15 of matrix B with .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

Row\Col	0	1	2	...	7
16 ... 19	T0: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$	T4: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$			T28: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$
20 ... 23	T1: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$	T5: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$			T29: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$
24 ... 27	T2: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$	T6: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$			T30: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$
28 ... 31	T3: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$	T7: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$			T31: $\begin{Bmatrix} b_4 \\ b_5 \\ b_6 \\ b_7 \end{Bmatrix}$

%laneid:{fragments}

Figure 133: Sparse MMA .m16n8k64 fragment layout for rows 16–31 of matrix B with .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

Row\Col	0	1	2	...	7
32 ... 35	T0: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$	T4: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$			T28: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$
36 ... 39	T1: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$	T5: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$			T29: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$
40 ... 43	T2: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$	T6: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$			T30: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$
44 ... 47	T3: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$	T7: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$			T31: $\begin{Bmatrix} b8 \\ b9 \\ b10 \\ b11 \end{Bmatrix}$
%laneid:{fragments}					

Figure 134: Sparse MMA .m16n8k64 fragment layout for rows 32–47 of matrix B with .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

Row\Col	0	1	2	...	7
48 ... 51	T0: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$	T4: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$			T28: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$
52 ... 55	T1: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$	T5: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$			T29: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$
56 ... 59	T2: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$	T6: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$			T30: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$
60 ... 63	T3: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$	T7: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$			T31: $\begin{Bmatrix} b12 \\ b13 \\ b14 \\ b15 \end{Bmatrix}$
%laneid:{fragments}					

Figure 135: Sparse MMA .m16n8k64 fragment layout for rows 48–63 of matrix B with .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

- Matrix fragments for accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k16 with integer type*.
- Metadata: A .b32 register containing 16 2-bit vectors with each pair of 2-bit vectors storing the indices of two non-zero elements from a 4-wide chunk of matrix A as shown in Figure 136 and Figure 137.

Row\Column	0..3	4..7	8..11	12..15	16..19	20..23	24..27	28..31
0	T ₀ : [3..0]	T ₀ : [7..4]	T ₀ : [11..8]	T ₀ : [15..12]	T ₀ : [19..16]	T ₀ : [23..20]	T ₀ : [27..24]	T ₀ : [31..28]
1	T ₄ : [3..0]	T ₄ : [7..4]	T ₄ : [11..8]	T ₄ : [15..12]	T ₄ : [19..16]	T ₄ : [23..20]	T ₄ : [27..24]	T ₄ : [31..28]
2								
...								
7	T ₂₈ : [3..0]	T ₂₈ : [7..4]	T ₂₈ : [11..8]	T ₂₈ : [15..12]	T ₂₈ : [19..16]	T ₂₈ : [23..20]	T ₂₈ : [27..24]	T ₂₈ : [31..28]
8	T ₁ : [3..0]	T ₁ : [7..4]	T ₁ : [11..8]	T ₁ : [15..12]	T ₁ : [19..16]	T ₁ : [23..20]	T ₁ : [27..24]	T ₁ : [31..28]
9	T ₅ : [3..0]	T ₅ : [7..4]	T ₅ : [11..8]	T ₅ : [15..12]	T ₅ : [19..16]	T ₅ : [23..20]	T ₅ : [27..24]	T ₅ : [31..28]
10								
...								
15	T ₂₉ : [3..0]	T ₂₉ : [7..4]	T ₂₉ : [11..8]	T ₂₉ : [15..12]	T ₂₉ : [19..16]	T ₂₉ : [23..20]	T ₂₉ : [27..24]	T ₂₉ : [31..28]

Figure 136: Sparse MMA .m16n8k64 metadata layout for columns 0–31 for .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

9.7.14.6.2.7 Matrix Fragments for sparse mma.m16n8k64 with .u4 / .s4 integer type

A warp executing sparse mma.m16n8k64 with .u4 / .s4 integer type will compute an MMA operation of shape .m16n8k64.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

- Multiplicand A:

.atype	Fragment	Elements
.u4 / .s4	A vector expression containing two .b32 registers, with each register containing eight non-zero .u4 / .s4 elements out of 16 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .

The layout of the fragments held by different threads is shown in Figure 138.

Row\Column	35..32	39..36	43..40	47..44	51..48	55..52	59..56	63..60
0	T ₂ : [3..0]	T ₂ : [7..4]	T ₂ : [11..8]	T ₂ : [15..12]	T ₂ : [19..16]	T ₂ : [23..20]	T ₂ : [27..24]	T ₂ : [31..28]
1	T ₆ : [3..0]	T ₆ : [7..4]	T ₆ : [11..8]	T ₆ : [15..12]	T ₆ : [19..16]	T ₆ : [23..20]	T ₆ : [27..24]	T ₆ : [31..28]
2								
...								
7	T ₃₀ : [3..0]	T ₃₀ : [7..4]	T ₃₀ : [11..8]	T ₃₀ : [15..12]	T ₃₀ : [19..16]	T ₃₀ : [23..20]	T ₃₀ : [27..24]	T ₃₀ : [31..28]
8	T ₃ : [3..0]	T ₃ : [7..4]	T ₃ : [11..8]	T ₃ : [15..12]	T ₃ : [19..16]	T ₃ : [23..20]	T ₃ : [27..24]	T ₃ : [31..28]
9	T ₇ : [3..0]	T ₇ : [7..4]	T ₇ : [11..8]	T ₇ : [15..12]	T ₇ : [19..16]	T ₇ : [23..20]	T ₇ : [27..24]	T ₇ : [31..28]
10								
...								
15	T ₃₁ : [3..0]	T ₃₁ : [7..4]	T ₃₁ : [11..8]	T ₃₁ : [15..12]	T ₃₁ : [19..16]	T ₃₁ : [23..20]	T ₃₁ : [27..24]	T ₃₁ : [31..28]

Figure 137: Sparse MMA .m16n8k64 metadata layout for columns 32–63 for .u8/.s8/.e4m3/.e5m2/.e3m2/.e2m3/.e2m1 type.

Row\Column	0 ... 15	16 ... 31	32 ... 47	48 ... 63
0	T0:{a0 ... a7}	T1:{a0 ... a7}	T2:{a0 ... a7}	T3:{a0 ... a7}
1	T4:{a0 ... a7}	T5:{a0 ... a7}	T6:{a0 ... a7}	T7:{a0 ... a7}
2				
...				
7	T28:{a0 ... a7}	T29:{a0 ... a7}	T30:{a0 ... a7}	T31:{a0 ... a7}
8	T0:{a8 ... a15}	T1:{a8 ... a15}	T2:{a8 ... a15}	T3:{a8 ... a15}
9	T4:{a8 ... a15}	T5:{a8 ... a15}	T6:{a8 ... a15}	T7:{a8 ... a15}
10				
...				
15	T28:{a8 ... a15}	T29:{a8 ... a15}	T30:{a8 ... a15}	T31:{a8 ... a15}

Figure 138: Sparse MMA .m16n8k64 fragment layout for matrix A with .u4/.s4 type.

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for ai where 0 <= i < 8
      groupID + 8      Otherwise

col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                             // as described in Sparse matrix storage

Where
firstcol = threadID_in_group * 16
lastcol  = firstcol + 15

```

- Matrix fragments for multiplicand B and accumulators C and D are the same as in case of *Matrix Fragments for mma.m16n8k64*.
- Metadata: A .b32 register containing 16 2-bit vectors with each pair of 2-bit vectors storing the indices of four non-zero elements from a 8-wide chunk of matrix A as shown in *Figure 139*.

Row\Column	0..7	8..15	16..23	24..31	32..39	40..47	48..55	56..63
0	T _{2i} : [3..0]	T _{2i} : [7..4]	T _{2i} : [11..8]	T _{2i} : [15..12]	T _{2i} : [19..16]	T _{2i} : [23..20]	T _{2i} : [27..24]	T _{2i} : [31..28]
1	T _{2i+4} : [3..0]	T _{2i+4} : [7..4]	T _{2i+4} : [11..8]	T _{2i+4} : [15..12]	T _{2i+4} : [19..16]	T _{2i+4} : [23..20]	T _{2i+4} : [27..24]	T _{2i+4} : [31..28]
2								
...								
7	T _{2i+28} : [3..0]	T _{2i+28} : [7..4]	T _{2i+28} : [11..8]	T _{2i+28} : [15..12]	T _{2i+28} : [19..16]	T _{2i+28} : [23..20]	T _{2i+28} : [27..24]	T _{2i+28} : [31..28]
8	T _{2i+1} : [3..0]	T _{2i+1} : [7..4]	T _{2i+1} : [11..8]	T _{2i+1} : [15..12]	T _{2i+1} : [19..16]	T _{2i+1} : [23..20]	T _{2i+1} : [27..24]	T _{2i+1} : [31..28]
9	T _{2i+5} : [3..0]	T _{2i+5} : [7..4]	T _{2i+5} : [11..8]	T _{2i+5} : [15..12]	T _{2i+5} : [19..16]	T _{2i+5} : [23..20]	T _{2i+5} : [27..24]	T _{2i+5} : [31..28]
10								
...								
15	T _{2i+29} : [3..0]	T _{2i+29} : [7..4]	T _{2i+29} : [11..8]	T _{2i+29} : [15..12]	T _{2i+29} : [19..16]	T _{2i+29} : [23..20]	T _{2i+29} : [27..24]	T _{2i+29} : [31..28]

Figure 139: Sparse MMA .m16n8k64 metadata layout for .u4/.s4 type.

9.7.14.6.2.8 Matrix Fragments for sparse mma.m16n8k128 with .u4 / .s4 integer type

A warp executing sparse mma.m16n8k128 with .u4 / .s4 / .e2m1 integer type will compute an MMA operation of shape .m16n8k128.

Elements of the matrix are distributed across the threads in a warp so each thread of the warp holds a fragment of the matrix.

► Multiplicand A:

.atype	Fragment	Elements
.u4 / .s4	A vector expression containing four .b32 registers, with each register containing eight non-zero .u4 / .s4 elements out of 16 consecutive elements from matrix A.	Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i> .
.e2m1	A vector expression containing four .b32 registers, with each register containing eight non-zero .e2m1 elements out of 16 consecutive elements from matrix A.	

The layout of the fragments held by different threads is shown in Figure 140 and Figure 141.

Row\Column	0 ... 15	16 ... 31	32 ... 47	48 ... 63
0	T0:{a0 ... a7}	T1:{a0 ... a7}	T2:{a0 ... a7}	T3:{a0 ... a7}
1	T4:{a0 ... a7}	T5:{a0 ... a7}	T6:{a0 ... a7}	T7:{a0 ... a7}
2				
...				
7	T28:{a0 ... a7}	T29:{a0 ... a7}	T30:{a0 ... a7}	T31:{a0 ... a7}
8	T0:{a8 ... a15}	T1:{a8 ... a15}	T2:{a8 ... a15}	T3:{a8 ... a15}
9	T4:{a8 ... a15}	T5:{a8 ... a15}	T6:{a8 ... a15}	T7:{a8 ... a15}
10				
...				
15	T28:{a8 ... a15}	T29:{a8 ... a15}	T30:{a8 ... a15}	T31:{a8 ... a15}

Figure 140: Sparse MMA .m16n8k128 fragment layout for columns 0–63 of matrix A with .u4/.s4/.e2m1 type.

```

groupID = %laneid >> 2
threadID_in_group = %laneid % 4

row =      groupID      for ai where  0 <= i < 8 || 16 <= i < 24
          groupID + 8    Otherwise

```

(continues on next page)

Row\Column	64 ... 79	80 ... 95	96 ... 111	112 ... 127
0	T0:{a16 ... a23}	T1:{a16 ... a23}	T2:{a16 ... a23}	T3:{a16 ... a23}
1	T4:{a16 ... a23}	T5:{a16 ... a23}	T6:{a16 ... a23}	T7:{a16 ... a23}
2				
...				
7	T28:{a16 ... a23}	T29:{a16 ... a23}	T30:{a16 ... a23}	T31:{a16 ... a23}
8	T0:{a24 ... a31}	T1:{a24 ... a31}	T2:{a24 ... a31}	T3:{a24 ... a31}
9	T4:{a24 ... a31}	T5:{a24 ... a31}	T6:{a24 ... a31}	T7:{a24 ... a31}
10				
...				
15	T28:{a24 ... a31}	T29:{a24 ... a31}	T30:{a24 ... a31}	T31:{a24 ... a31}

Figure 141: Sparse MMA .m16n8k128 fragment layout for columns 64–127 of matrix A with .u4/.s4/.e2m1 type.

(continued from previous page)

```
col = [firstcol ... lastcol] // As per the mapping of non-zero elements
                             // as described in Sparse matrix storage
```

Where

```
firstcol = threadID_in_group * 16           For ai where i < 16
          (threadID_in_group * 16) + 64     For ai where i >= 16
lastcol  = firstcol + 15
```

► Multiplicand B:

.atype	Fragment	Elements (low to high)
.u4 / .s4	A vector expression containing four .b32 registers, each containing eight .u4 / .s4 elements from matrix B.	b0, b1, b2, b3, ..., b31
.e2m1	A vector expression containing four .b32 registers, each containing eight .e2m1 elements from matrix B.	

The layout of the fragments held by different threads is shown in [Figure 142](#), [Figure 143](#), [Figure 144](#), [Figure 145](#).

- Matrix fragments for accumulators C and D are the same as in case of [Matrix Fragments for mma.m16n8k64](#).
- Metadata: A .b32 register containing 16 2-bit vectors with each pair of 2-bit vectors storing the indices of four non-zero elements from a 8-wide chunk of matrix A as shown in [Figure 146](#) and [Figure 147](#).

Row\Col	0	1	2	...	7
0 ... 7	$T0: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$	$T4: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$			$T28: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$
8 ... 15	$T1: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$	$T5: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$			$T29: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$
16 ... 23	$T2: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$	$T6: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$			$R30: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$
24 ... 31	$T3: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$	$T7: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$			$T31: \begin{Bmatrix} b0 \\ b1 \\ \dots \\ b6 \\ b7 \end{Bmatrix}$

%laneid:{fragments}

Figure 142: Sparse MMA .m16n8k128 fragment layout for rows 0–31 of matrix B with .u4/.s4/.e2m1 type.

Row\Col	0	1	2	...	7
32 ... 39	$T0: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$	$T4: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$			$T28: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$
40 ... 47	$T1: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$	$T5: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$			$T29: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$
48 ... 55	$T2: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$	$T6: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$			$R30: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$
56 ... 63	$T3: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$	$T7: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$			$T31: \begin{Bmatrix} b8 \\ b9 \\ \dots \\ b14 \\ b15 \end{Bmatrix}$

%laneid:{fragments}

Figure 143: Sparse MMA .m16n8k128 fragment layout for rows 32–63 of matrix B with .u4/.s4/.e2m1 type.

Row\Col	0	1	2	...	7
64 ... 71	$T0: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$	$T4: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$			$T28: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$
72 ... 79	$T1: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$	$T5: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$			$T29: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$
80 ... 87	$T2: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$	$T6: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$			$R30: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$
88 ... 95	$T3: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$	$T7: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$			$T31: \begin{Bmatrix} b16 \\ b17 \\ .. \\ b22 \\ b23 \end{Bmatrix}$

%laneid:{fragments}

Figure 144: Sparse MMA .m16n8k128 fragment layout for rows 64–95 of matrix B with .u4/.s4/.e2m1 type.

Row\Col	0	1	2	...	7
96 ... 103	$T0: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$	$T4: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$			$T28: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$
104 ... 111	$T1: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$	$T5: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$			$T29: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$
112 ... 119	$T2: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$	$T6: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$			$R30: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$
120 ... 127	$T3: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$	$T7: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$			$T31: \begin{Bmatrix} b24 \\ b25 \\ .. \\ b30 \\ b31 \end{Bmatrix}$

%laneid:{fragments}

Figure 145: Sparse MMA .m16n8k128 fragment layout for rows 96–127 of matrix B with .u4/.s4/.e2m1 type.

Row\Column	0..7	8..15	16..23	24..31	32..39	40..47	48..55	56..63
0	$T_{2i}:[3..0]$	$T_{2i}:[7..4]$	$T_{2i}:[11..8]$	$T_{2i}:[15..12]$	$T_{2i}:[19..16]$	$T_{2i}:[23..20]$	$T_{2i}:[27..24]$	$T_{2i}:[31..28]$
1	$T_{2i+4}:[3..0]$	$T_{2i+4}:[7..4]$	$T_{2i+4}:[11..8]$	$T_{2i+4}:[15..12]$	$T_{2i+4}:[19..16]$	$T_{2i+4}:[23..20]$	$T_{2i+4}:[27..24]$	$T_{2i+4}:[31..28]$
2								
...								
7	$T_{2i+28}:[3..0]$	$T_{2i+28}:[7..4]$	$T_{2i+28}:[11..8]$	$T_{2i+28}:[15..12]$	$T_{2i+28}:[19..16]$	$T_{2i+28}:[23..20]$	$T_{2i+28}:[27..24]$	$T_{2i+28}:[31..28]$
8	$T_{2i+1}:[3..0]$	$T_{2i+1}:[7..4]$	$T_{2i+1}:[11..8]$	$T_{2i+1}:[15..12]$	$T_{2i+1}:[19..16]$	$T_{2i+1}:[23..20]$	$T_{2i+1}:[27..24]$	$T_{2i+1}:[31..28]$
9	$T_{2i+5}:[3..0]$	$T_{2i+5}:[7..4]$	$T_{2i+5}:[11..8]$	$T_{2i+5}:[15..12]$	$T_{2i+5}:[19..16]$	$T_{2i+5}:[23..20]$	$T_{2i+5}:[27..24]$	$T_{2i+5}:[31..28]$
10								
...								
15	$T_{2i+29}:[3..0]$	$T_{2i+29}:[7..4]$	$T_{2i+29}:[11..8]$	$T_{2i+29}:[15..12]$	$T_{2i+29}:[19..16]$	$T_{2i+29}:[23..20]$	$T_{2i+29}:[27..24]$	$T_{2i+29}:[31..28]$

Figure 146: Sparse MMA .m16n8k128 metadata layout for columns 0–63 for .u4/.s4/.e2m1 type.

9.7.14.6.3 Multiply-and-Accumulate Instruction: `mma.sp / mma.sp::ordered_metadata`

`mma.sp, mma.sp::ordered_metadata`

Perform matrix multiply-and-accumulate operation with sparse matrix A

Syntax

Half precision floating point type:

```
mma.spvariant.sync.aligned.m16n8k16.row.col.dtype.f16.f16.ctype d, a, b, c, e, f;
mma.spvariant.sync.aligned.m16n8k32.row.col.dtype.f16.f16.ctype d, a, b, c, e, f;

.ctype      = {.f16, .f32};
.dtype      = {.f16, .f32};
.spvariant  = {.sp, .sp::ordered_metadata};
```

Alternate floating point type:

```
mma.spvariant.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32 d, a, b, c, e, f;
mma.spvariant.sync.aligned.m16n8k32.row.col.f32.bf16.bf16.f32 d, a, b, c, e, f;
mma.spvariant.sync.aligned.m16n8k8.row.col.f32.tf32.tf32.f32 d, a, b, c, e, f;
mma.spvariant.sync.aligned.m16n8k16.row.col.f32.tf32.tf32.f32 d, a, b, c, e, f;
mma.spvariant.sync.aligned.m16n8k64.row.col.f32.f8type.f8type.f32 d, a, b, c, e, f;
mma.sp::ordered_metadata.sync.aligned.m16n8k64.row.col.kind.dtype.f8f6f4type.
↪ f8f6f4type.ctype d, a, b, c, e, f;
```

(continues on next page)

Row\Column	64..71	72..79	80..87	88..95	96..103	104..111	112..119	120..127
0	T ₂ : [3..0]	T ₂ : [7..4]	T ₂ : [11..8]	T ₂ : [15..12]	T ₂ : [19..16]	T ₂ : [23..20]	T ₂ : [27..24]	T ₂ : [31..28]
1	T ₆ : [3..0]	T ₆ : [7..4]	T ₆ : [11..8]	T ₆ : [15..12]	T ₆ : [19..16]	T ₆ : [23..20]	T ₆ : [27..24]	T ₆ : [31..28]
2								
...								
7	T ₃₀ : [3..0]	T ₃₀ : [7..4]	T ₃₀ : [11..8]	T ₃₀ : [15..12]	T ₃₀ : [19..16]	T ₃₀ : [23..20]	T ₃₀ : [27..24]	T ₃₀ : [31..28]
8	T ₃ : [3..0]	T ₃ : [7..4]	T ₃ : [11..8]	T ₃ : [15..12]	T ₃ : [19..16]	T ₃ : [23..20]	T ₃ : [27..24]	T ₃ : [31..28]
9	T ₇ : [3..0]	T ₇ : [7..4]	T ₇ : [11..8]	T ₇ : [15..12]	T ₇ : [19..16]	T ₇ : [23..20]	T ₇ : [27..24]	T ₇ : [31..28]
10								
...								
15	T ₃₁ : [3..0]	T ₃₁ : [7..4]	T ₃₁ : [11..8]	T ₃₁ : [15..12]	T ₃₁ : [19..16]	T ₃₁ : [23..20]	T ₃₁ : [27..24]	T ₃₁ : [31..28]

Figure 147: Sparse MMA .m16n8k128 metadata layout for columns 64–127 for .u4/.s4/.e2m1 type.

(continued from previous page)

```
.f8type      = {.e4m3, .e5m2};
.spvariant   = {.sp, .sp::ordered_metadata};
.f8f6f4type = {.e4m3, .e5m2, .e3m2, .e2m3, .e2m1};
.kind        = {kind::f8f6f4};
.ctype       = {.f16, .f32};
.dtype       = {.f16, .f32};
```

Alternate floating point type with block scaling:

```
mma.spvariant.sync.aligned.m16n8k128.row.col.kind.block_scale{.scale_vec_size}.f32.
↳ e2m1.e2m1.f32.stype d, a, b, c, e, f, scale-a-data, {byte-id-a, thread-id-a}, scale-
↳ b-data, {byte-id-b, thread-id-b};
```

```
.spvariant    = {.sp::ordered_metadata};
.kind         = {.kind::mxf4};
.scale_vec_size = {.scale_vec::2X};
.stype        = {.ue8m0};
```

```
mma.spvariant.sync.aligned.m16n8k128.row.col.kind.block_scale.scale_vec_size.f32.e2m1.
↳ e2m1.f32.stype d, a, b, c, e, f, scale-a-data, {byte-id-a, thread-id-a}, scale-b-
↳ data, {byte-id-b, thread-id-b};
```

```
.spvariant    = {.sp::ordered_metadata};
.kind         = {.kind::mxf4nvf4};
.scale_vec_size = {.scale_vec::2X, .scale_vec::4X};
.stype        = {.ue8m0, .ue4m3};
```

```
mma.spvariant.sync.aligned.m16n8k64.row.col.kind.block_scale{.scale_vec_size}.f32.
↳ f8f6f4type.f8f6f4type.f32.stype d, a, b, c, e, f, scale-a-data, {byte-id-a, thread-id-a}, scale-b-
↳ data, {byte-id-b, thread-id-b};
```

(continued from previous page)

```
.spvariant      = {.sp::ordered_metadata};
.kind           = {.kind::mxf8f6f4};
.scale_vec_size = {.scale_vec::1X};
.f8f6f4type     = {.e4m3, .e5m2, .e3m2, .e2m3, .e2m1};
.stype          = {.ue8m0};
```

Integer type:

```
mma.spvariant.sync.aligned.shape.row.col{.satfinite}.s32.atype.btype.s32 d, a, b, c,
↪e, f;

.shape      = {.m16n8k32, .m16n8k64}
.atype      = {.u8, .s8};
.btype      = {.u8, .s8};
.spvariant  = {.sp, .sp::ordered_metadata};

mma.spvariant.sync.aligned.shape.row.col{.satfinite}.s32.atype.btype.s32 d, a, b, c,
↪e, f;

.shape      = {.m16n8k64, .m16n8k128}
.atype      = {.u4, .s4};
.btype      = {.u4, .s4};
.spvariant  = {.sp, .sp::ordered_metadata};
```

Description

Perform a $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + C$, where the A matrix is $M \times K$, the B matrix is $K \times N$, and the C and D matrices are $M \times N$.

A warp executing `mma.sp.sync/mma.sp::ordered_metadata.sync` instruction compute a single matrix multiply and accumulate operation.

Qualifier `.block_scale` specifies that the matrices A and B are scaled with `scale_A` and `scale_B` matrices respectively before performing the matrix multiply and accumulate operation as specified in the section [Block Scaling](#). The data type corresponding to each of the element within `scale_A` and `scale_B` matrices is specified by `.stype`. Qualifier `.scale_vec_size` specifies the number of columns of `scale_A` matrix and number of rows in the matrix `scale_B`.

The valid combinations of `.kind`, `.stype` and `.scale_vec_size` are described in [Table 37](#). For `mma` with `.kind::mxf4` when the qualifier `.scale_vec_size` is not specified, then it defaults to 2X. In contrast, when `.kind` is specified as `.kind::mxf8f6f4` then the qualifier `.scale_vec_size` defaults to 1X. However, for `.kind::mxf4nvf4`, it is mandatory to provide valid `.scale_vec_size`.

Operands `a` and `b` represent two multiplicand matrices A and B, while `c` and `d` represent the accumulator and destination matrices, distributed across the threads in warp. Matrix A is structured sparse as described in [Sparse matrix storage](#). Operands `e` and `f` represent sparsity metadata and sparsity selector respectively. Operand `e` is a 32-bit integer and operand `f` is a 32-bit integer constant with values in the range 0..3. When `.block_scale` qualifier is specified, operand `scale-a-data`, `scale-b-data` represents the scale matrix metadata corresponding to `scale_A` and `scale_B` matrices respectively. The tuple `{byte-id-a, thread-id-a}` and `{byte-id-b, thread-id-b}` represent selectors for matrices `scale_A` and `scale_B` respectively from their corresponding metadata arguments `scale-a-data`, `scale-b-data`. The operands `scale-a-data`, `scale-b-data` are of type `.b32`. The operands `byte-id-a`, `thread-id-a`, `byte-id-b`, `thread-id-b` are unsigned 16-bit integer values. For more details on selector arguments refer [Block Scaling for mma.sync](#) section.

Instruction `mma.sp::ordered_metadata` requires the indices in the sparsity metadata to be sorted in an increasing order starting from LSB, otherwise behavior is undefined.

The registers in each thread hold a fragment of matrix as described in *Matrix fragments for multiply-accumulate operation with sparse matrix A*.

The qualifiers `.dtype`, `.atype`, `.btype` and `.ctype` indicate the data-type of the elements in the matrices D, A, B and C respectively. The qualifier `.stype` indicate the data-type of the elements in the matrices `scale_A` and `scale_B`. In case of shapes `.m16n8k16`, `.m16n8k32` and `.m16n8k64`, `.dtype` must be the same as `.ctype`.

When `.kind` is either of `.kind::mxf8f6f4` or `.kind::f8f6f4`, the individual 4-bit and the 6-bit floating point type elements must be packed in an 8-bit container. The matrix element of type `.e2m1` resides in central 4 bits of the 8-bit container with padding in the upper 2 bits and lower 2 bits of the container. When the matrix element is of type `.e3m2` or `.e2m3`, the matrix element resides in the lower 6 bits of the 8-bit container with padding in the upper 2 bits of the container. In contrast, note that when using `mma` with `.kind::mxf4` or `.kind::mxf4nvf4`, no explicit padding is necessary even though matrix elements are of type `.e2m1`.

Precision and rounding :

- ▶ `.f16` floating point operations :

Element-wise multiplication of matrix A and B is performed with at least single precision. When `.ctype` or `.dtype` is `.f32`, accumulation of the intermediate values is performed with at least single precision. When both `.ctype` and `.dtype` are specified as `.f16`, the accumulation is performed with at least half precision.

The accumulation order, rounding and handling of subnormal inputs are unspecified.

- ▶ `.e4m3`, `.e5m2`, `.e3m2`, `.e2m3`, `.e2m1` floating point operations :

Element-wise multiplication of matrix A and B is performed with specified precision. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding, and handling of subnormal inputs are unspecified.

- ▶ `.bf16` and `.tf32` floating point operations :

Element-wise multiplication of matrix A and B is performed with specified precision. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding, and handling of subnormal inputs are unspecified.

- ▶ Integer operations :

The integer `mma.sp/mma.sp::ordered_metadata` operation is performed with `.s32` accumulators. The `.satfinite` qualifier indicates that on overflow, the accumulated value is limited to the range `MIN_INT32..MAX_INT32` (where the bounds are defined as the minimum negative signed 32-bit integer and the maximum positive signed 32-bit integer respectively).

If `.satfinite` is not specified, the accumulated value is wrapped instead.

The mandatory `.sync` qualifier indicates that `mma.sp/mma.sp::ordered_metadata` instruction causes the executing thread to wait until all threads in the warp execute the same `mma.sp/mma.sp::ordered_metadata` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `mma.sp/mma.sp::ordered_metadata` instruction. In conditionally executed code, a `mma.sp/mma.sp::ordered_metadata` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `mma.sp/mma.sp::ordered_metadata` instruction is undefined if all threads in the same warp do not use the same qualifiers, or if any thread in the warp has exited.

Notes

`mma.sp` instruction may have substantially reduced performance on some target architectures. Hence, it is advised to use `mma.sp::ordered_metadata` instruction.

PTX ISA Notes

Introduced in PTX ISA version 7.1.

Support for `.e4m3` and `.e5m2` alternate floating point type `mma` operation introduced in PTX ISA version 8.4.

`mma.sp::ordered_metadata` introduced in PTX ISA version 8.5.

Support for shape `.m16n8k32` and `.f16` dtype/ctype with `.e4m3/.e5m2` alternate floating point type `mma` operation introduced in PTX ISA version 8.7.

Support for `.e3m2`, `.e2m3`, `.e2m1` alternate floating point type `mma` operation introduced in PTX ISA version 8.7.

Support for `.kind`, `.block_scale`, `.scale_vec_size` qualifier introduced in PTX ISA version 8.7.

Support for `.scale_vec::4X` on `.ue8m0` as `.stype` with `.kind::mxf4nvf4` is introduced in PTX ISA version 9.1

Target ISA Notes

Requires `sm_80` or higher.

`.e4m3` and `.e5m2` alternate floating point type `mma` operation requires `sm_89` or higher.

`mma.sp::ordered_metadata` requires `sm_80` or higher.

Support for shape `.m16n8k32` and `.f16` dtype/ctype with `.e4m3/.e5m2` alternate floating point type `mma` operation requires `sm_120`.

`.e3m2`, `.e2m3` and `.e2m1` alternate floating point type `mma` operation requires `sm_120a` and are supported on `sm_120f` or higher in the same family from PTX ISA version 8.8.

Support for `.kind`, `.block_scale`, `.scale_vec_size` qualifier requires `sm_120a` and are supported on `sm_120f` and later generation targets in the same family from PTX ISA version 8.8 except for `.kind::mxf4nvf4/.kind::mxf4`.

Qualifiers `.kind::mxf4nvf4` and `.kind::mxf4` are supported on following architectures:

- ▶ `sm_120a`
- ▶ `sm_121a`

Examples of half precision floating point type

```
// f16 elements in C and D matrix
.reg .f16x2 %Ra<2> %Rb<2> %Rc<2> %Rd<2>
.reg .b32 %Re;
mma.sp.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16
    {%Rd0, %Rd1},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1}, %Re, 0x1;

.reg .f16x2 %Ra<2> %Rb<2> %Rc<2> %Rd<2>
.reg .b32 %Re;

mma.sp::ordered_metadata.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16
    {%Rd0, %Rd1},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1}, %Re, 0x1;
```

Examples of alternate floating point type

```
.reg .b32 %Ra<2>, %Rb<2>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp.sync.aligned.m16n8k8.row.col.f32.tf32.tf32.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x1;

.reg .b32 %Ra<2>, %Rb<2>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x1;

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp.sync.aligned.m16n8k32.row.col.f32.bf16.bf16.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x1;

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp.sync.aligned.m16n8k64.row.col.f32.e5m2.e4m3.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
```

(continues on next page)

(continued from previous page)

```

    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0;

.reg .b32 %Ra<2>, %Rb<2>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x1;

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k64.row.col.kind::f8f6f4.f32.e3m2.e2m3.f32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0;

.reg .b32 %Ra<4>, %Rb<4>;
.reg .b32 %Rc<4>, %Rd<4>;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k64.row.col.kind::f8f6f4.f16.e2m3.e2m1.f16
    {%Rd0, %Rd1},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1}, %Re, 0;

```

Examples of integer type

```

.reg .b32 %Ra<4>, %Rb<4>, %Rc<4>, %Rd<4>;
.reg .u32 %Re;

// u8 elements in A and B matrix
mma.sp.sync.aligned.m16n8k32.row.col.satfinite.s32.u8.u8.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x1;

// s8 elements in A and B matrix
mma.sp.sync.aligned.m16n8k64.row.col.satfinite.s32.s8.s8.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x0;

// s8 elements in A and B matrix with ordered metadata
mma.sp::ordered_metadata.sync.aligned.m16n8k64.row.col.satfinite.s32.s8.s8.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},

```

(continues on next page)

(continued from previous page)

```

    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x0;

// u4 elements in A and B matrix
mma.sp.sync.aligned.m16n8k64.row.col.s32.s4.s4.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1},
    {%Rb0, %Rb1},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x1;

// u4 elements in A and B matrix
mma.sp.sync.aligned.m16n8k128.row.col.satfinite.s32.u4.u4.s32
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3}, %Re, 0x0;

```

Examples of mma with block scale

```

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k128.row.col.kind::mxf4.block_scale.f32.
↪e2m1.e2m1.f32.ue8m0
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    %Re, 0,
    scaleAData, {2, 1}, scaleBData, {2, 3};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .u16 bidA, bidB, tidA, tidB;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k128.row.col.kind::mxf4nvf4.block_scale.
↪scale_vec::4X.f32.e2m1.e2m1.f32.ue4m3
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},
    {%Rb0, %Rb1, %Rb2, %Rb3},
    {%Rc0, %Rc1, %Rc2, %Rc3},
    %Re, 0,
    scaleAData, {bidA, tidA}, scaleBData, {bidB, tidB};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .u16 bidA, bidB, tidA, tidB;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k128.row.col.kind::mxf4nvf4.block_scale.
↪scale_vec::4X.f32.e2m1.e2m1.f32.ue8m0
    {%Rd0, %Rd1, %Rd2, %Rd3},
    {%Ra0, %Ra1, %Ra2, %Ra3},

```

(continues on next page)

(continued from previous page)

```

{%Rb0, %Rb1, %Rb2, %Rb3},
{%Rc0, %Rc1, %Rc2, %Rc3},
%Re, 0,
scaleAData, {bidA, tidA}, scaleBData, {bidB, tidB};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k64.row.col.kind::mxf8f6f4.block_scale.
↪scale_vec::1X.f32.e3m2.e2m1.f32.ue8m0
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1, %Ra2, %Ra3},
{%Rb0, %Rb1, %Rb2, %Rb3},
{%Rc0, %Rc1, %Rc2, %Rc3},
%Re, 0,
scaleAData, {0, 1}, scaleBData, {0, 1};

.reg .b32 %Ra<4>, %Rb<4>;
.reg .f32 %Rc<4>, %Rd<4>;
.reg .b32 scaleAData, scaleBData;
.reg .b32 %Re;
mma.sp::ordered_metadata.sync.aligned.m16n8k64.row.col.kind::mxf8f6f4.block_scale.
↪scale_vec::1X.f32.e4m3.e5m2.f32.ue8m0
{%Rd0, %Rd1, %Rd2, %Rd3},
{%Ra0, %Ra1, %Ra2, %Ra3},
{%Rb0, %Rb1, %Rb2, %Rb3},
{%Rc0, %Rc1, %Rc2, %Rc3},
%Re, 0,
scaleAData, {0, 1}, scaleBData, {0, 0};

```

9.7.15. Asynchronous Warpgroup Level Matrix Multiply-Accumulate Instructions

The warpgroup level matrix multiply and accumulate operation has either of the following forms, where matrix D is called accumulator:

- ▶ $D = A * B + D$
- ▶ $D = A * B$, where the input from accumulator D is disabled.

The `wgmma` instructions perform warpgroup level matrix multiply-and-accumulate operation by having all threads in a warpgroup collectively perform the following actions:

1. Load matrices A, B and D into registers or into shared memory.
2. Perform the following fence operations:
 - ▶ `wgmma.fence` operations to indicate that the register/shared-memory across the warp-group have been written into.
 - ▶ `fence.proxy.async` operation to make the generic proxy operations visible to the async proxy.

3. Issue the asynchronous matrix multiply and accumulate operations using the `wgmma.mma_async` operation on the input matrices. The `wgmma.mma_async` operation is performed in the `async` proxy.
4. Create a `wgmma-group` and commit all the prior outstanding `wgmma.mma_async` operations into the group, by using `wgmma.commit_group` operation.
5. Wait for the completion of the required `wgmma-group`.
6. Once the `wgmma-group` completes, all the `wgmma.mma_async` operations have been performed and completed.

9.7.15.1 Warpgroup

A warpgroup is a set of four contiguous warps such that the *warp-rank* of the first warp is a multiple of 4.

warp-rank of a warp is defined as:

```
(%tid.x + %tid.y * %ntid.x + %tid.z * %ntid.x * %ntid.y) / 32
```

9.7.15.2 Matrix Shape

The matrix multiply and accumulate operations support a limited set of shapes for the operand matrices A, B and D. The shapes of all three matrix operands are collectively described by the tuple `MxNxK`, where A is an `MxK` matrix, B is a `KxN` matrix, while D is a `MxN` matrix.

The following matrix shapes are supported for the specified types for the `wgmma.mma_async` operation:

Multiplicand type	Data	Spar-sity	Shape
Floating-point - .f16		Dense	.m64n8k16, .m64n16k16, .m64n24k16, .m64n32k16, .m64n40k16, .m64n48k16, .m64n56k16, .m64n64k16, .m64n72k16, .m64n80k16, .m64n88k16, .m64n96k16, .m64n104k16, .m64n112k16, .m64n120k16, .m64n128k16, .m64n136k16, .m64n144k16, .m64n152k16, .m64n160k16, .m64n168k16, .m64n176k16, .m64n184k16, .m64n192k16, .m64n200k16, .m64n208k16, .m64n216k16, .m64n224k16, .m64n232k16, .m64n240k16, .m64n248k16, .m64n256k16
Alternate floating-point format - .bf16			
Alternate floating-point format - .tf32		Sparse	
Alternate floating-point format - .tf32		Dense	.m64n8k8, .m64n16k8, .m64n24k8, .m64n32k8, .m64n40k8, .m64n48k8, .m64n56k8, .m64n64k8, .m64n72k8, .m64n80k8, .m64n88k8, .m64n96k8, .m64n104k8, .m64n112k8, .m64n120k8, .m64n128k8, .m64n136k8, .m64n144k8, .m64n152k8, .m64n160k8, .m64n168k8, .m64n176k8, .m64n184k8, .m64n192k8, .m64n200k8, .m64n208k8, .m64n216k8, .m64n224k8, .m64n232k8, .m64n240k8, .m64n248k8, .m64n256k8

continues on next page

Table 39 – continued from previous page

Multiplicand type	Data	Spar- sity	Shape
Alternate floating-point format - .e4m3/.e5m2		Dense	.m64n8k32, .m64n16k32, .m64n24k32, .m64n32k32, .m64n40k32, .m64n48k32, .m64n56k32, .m64n64k32, .m64n72k32, .m64n80k32, .m64n88k32, .m64n96k32, .m64n104k32, .m64n112k32, .m64n120k32, .m64n128k32, .m64n136k32, .m64n144k32, .m64n152k32, .m64n160k32, .m64n168k32, .m64n176k32, .m64n184k32, .m64n192k32, .m64n200k32, .m64n208k32, .m64n216k32, .m64n224k32, .m64n232k32, .m64n240k32, .m64n248k32, .m64n256k32
Floating point - .f16		Sparse	
Alternate floating-point format - .bf16			
Integer - .u8/.s8		Dense	.m64n8k32, .m64n16k32, .m64n24k32, .m64n32k32, .m64n48k32, .m64n64k32, .m64n80k32, .m64n96k32, .m64n112k32, .m64n128k32, .m64n144k32, .m64n160k32, .m64n176k32, .m64n192k32, .m64n208k32, .m64n224k32, .m64n240k32, .m64n256k32
Alternate floating-point format - .e4m3/.e5m2		Sparse	.m64n8k64, .m64n16k64, .m64n24k64, .m64n32k64, .m64n40k64, .m64n48k64, .m64n56k64, .m64n64k64, .m64n72k64, .m64n80k64, .m64n88k64, .m64n96k64, .m64n104k64, .m64n112k64, .m64n120k64, .m64n128k64, .m64n136k64, .m64n144k64, .m64n152k64, .m64n160k64, .m64n168k64, .m64n176k64, .m64n184k64, .m64n192k64, .m64n200k64, .m64n208k64, .m64n216k64, .m64n224k64, .m64n232k64, .m64n240k64, .m64n248k64, .m64n256k64
Integer - .u8/.s8		Sparse	.m64n8k64, .m64n16k64, .m64n24k64, .m64n32k64, .m64n48k64, .m64n64k64, .m64n80k64, .m64n96k64, .m64n112k64, .m64n128k64, .m64n144k64, .m64n160k64, .m64n176k64, .m64n192k64, .m64n208k64, .m64n224k64, .m64n240k64, .m64n256k64
Single-bit - .b1		Dense	.m64n8k256, .m64n16k256, .m64n24k256, .m64n32k256, .m64n48k256, .m64n64k256, .m64n80k256, .m64n96k256, .m64n112k256, .m64n128k256, .m64n144k256, .m64n160k256, .m64n176k256, .m64n192k256, .m64n208k256, .m64n224k256, .m64n240k256, .m64n256k256

9.7.15.3 Matrix Data-types

The matrix multiply and accumulate operation is supported separately on integer, floating-point, sub-byte integer and single bit data-types. All operands must contain the same basic type kind, i.e., integer or floating-point.

For floating-point matrix multiply and accumulate operation, different matrix operands may have different precision, as described later.

For integer matrix multiply and accumulate operation, both multiplicand matrices (A and B) must have elements of the same data-type, e.g. both signed integer or both unsigned integer.

Data-type	Multiplicands (A or B)	Accumulator (D)
Integer	both .u8 or both .s8	.s32
Floating Point	.f16	.f16, .f32
Alternate floating Point	.bf16	.f32
Alternate floating Point	.tf32	.f32
Alternate floating Point	.e4m3, .e5m2	.f16, .f32
Single-bit integer	.b1	.s32

9.7.15.4 Async Proxy

The `wgmma.mma_async` operations are performed in the asynchronous proxy (or async proxy).

Accessing the same memory location across multiple proxies needs a cross-proxy fence. For the async proxy, `fence.proxy.async` should be used to synchronize memory between generic proxy and the async proxy.

The completion of a `wgmma.mma_async` operation is followed by an implicit generic-async proxy fence. So the result of the asynchronous operation is made visible to the generic proxy as soon as its completion is observed. `wgmma.commit_group` and `wgmma.wait_group` operations must be used to wait for the completion of the `wgmma.mma_async` instructions.

9.7.15.5 Asynchronous Warpgroup Level Matrix Multiply-Accumulate Operation using `wgmma.mma_async` instruction

This section describes warpgroup level `wgmma.mma_async` instruction and the organization of various matrices involved in this instruction.

9.7.15.5.1 Register Fragments and Shared Memory Matrix Layouts

The input matrix A of the warpgroup wide MMA operations can be either in registers or in the shared memory. The input matrix B of the warpgroup wide MMA operations must be in the shared memory. This section describes the layouts of register fragments and shared memory expected by the warpgroup MMA instructions.

When the matrices are in shared memory, their starting addresses must be aligned to 16 bytes.

9.7.15.5.1.1 Register Fragments

This section describes the organization of various matrices located in register operands of the `wgmma.mma_async` instruction.

9.7.15.5.1.2 Matrix Fragments for `wgmma.mma_async.m64nNk16`

A warpgroup executing `wgmma.mma_async.m64nNk16` will compute an MMA operation of shape `.m64nNk16` where N is a valid n dimension as listed in [Matrix Shape](#).

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warpgroup holds a fragment of the matrix.

- Multiplicand A in registers:

.atype	Fragment	Elements (low to high)
.f16/.bf16	A vector expression containing four .f16x2 registers, with each register containing two .f16/ .bf16 elements from matrix A.	a0, a1, a2, a3, a4, a5, a6, a7

The layout of the fragments held by different threads is shown in [Figure 148](#).

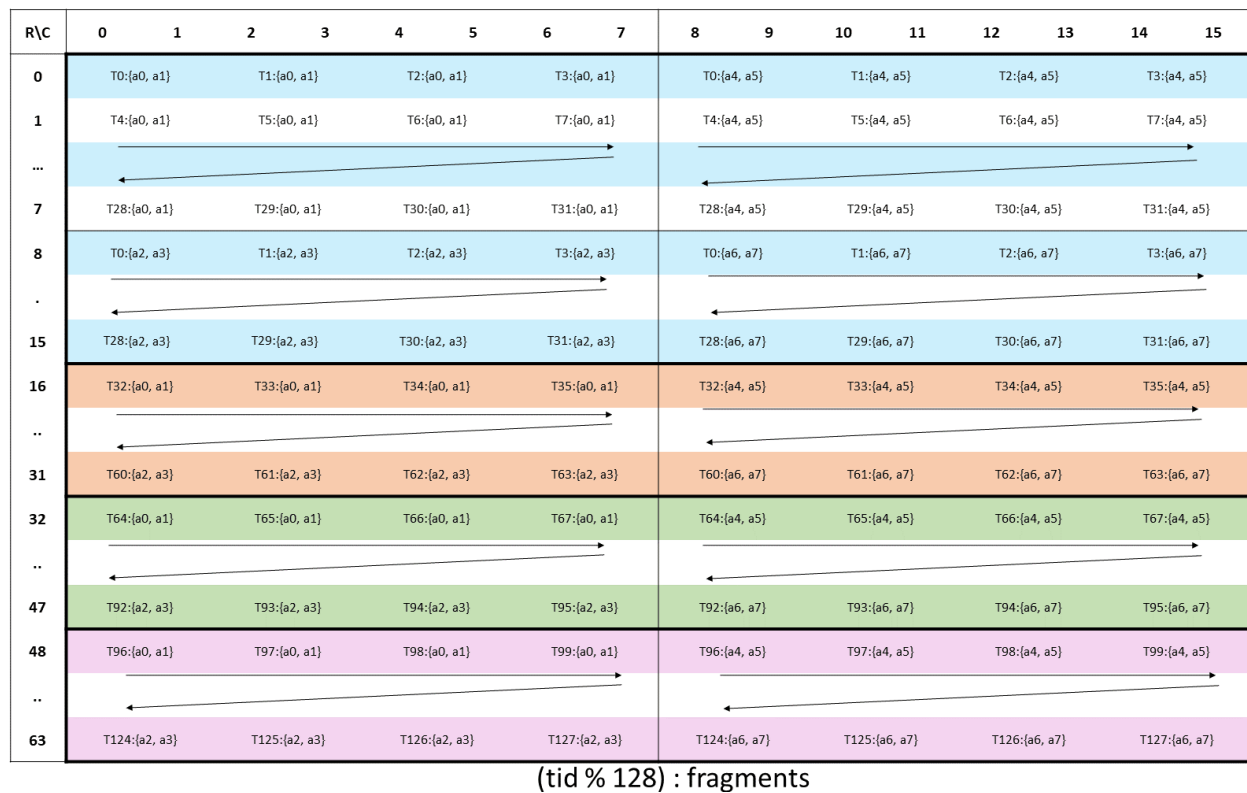
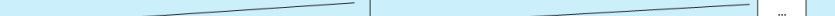
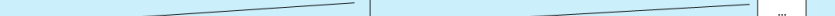


Figure 148: WGMMMA .m64nNk16 register fragment layout for matrix A.

- Accumulator D:

.dtype	Fragment	Elements (low to high)
.f16	A vector expression containing $N/4$ number of .f16x2 registers, with each register containing two .f16 elements from matrix D.	d0, d1, d2, d3, ..., dX, dY, dZ, dW where $X = N/2 - 4$ $Y = N/2 - 3$ $Z = N/2 - 2$ $W = N/2 - 1$ $N = 8*i$ where $i = \{1, 2, \dots, 32\}$
.f32	A vector expression containing $N/2$ number of .f32 registers.	

The layout of the fragments held by different threads is shown in [Figure 149](#).

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...	N-8	N-7	N-6	N-5	N-4	N-3	N-2	N-1
0	T0:{d0, d1}	T1:{d0, d1}	T2:{d0, d1}	T3:{d0, d1}					T0:{d4, d5}	T1:{d4, d5}	T2:{d4, d5}	T3:{d4, d5}					...	T0:{dX,dY}	T1:{dX,dY}	T2:{dX,dY}	T3:{dX,dY}				
1	T4:{d0, d1}	T5:{d0, d1}	T6:{d0, d1}	T7:{d0, d1}					T4:{d4, d5}	T5:{d4, d5}	T6:{d4, d5}	T7:{d4, d5}					...	T4:{dX,dY}	T5:{dX,dY}	T6:{dX,dY}	T7:{dX,dY}				
...																	...								
7	T28:{d0, d1}	T29:{d0, d1}	T30:{d0, d1}	T31:{d0, d1}					T28:{d4, d5}	T29:{d4, d5}	T30:{d4, d5}	T31:{d4, d5}					...	T28:{dX,dY}	T29:{dX,dY}	T30:{dX,dY}	T31:{dX,dY}				
8	T0:{d2, d3}	T1:{d2, d3}	T2:{d2, d3}	T3:{d2, d3}					T0:{d6, d7}	T1:{d6, d7}	T2:{d6, d7}	T3:{d6, d7}					...	T0:{dZ, dW}	T1:{dZ, dW}	T2:{dZ, dW}	T3:{dZ, dW}				
.																	...								
15	T28:{d2, d3}	T29:{d2, d3}	T30:{d2, d3}	T31:{d2, d3}					T28:{d6, d7}	T29:{d6, d7}	T30:{d6, d7}	T31:{d6, d7}					...	T28:{dZ, dW}	T29:{dZ, dW}	T30:{dZ, dW}	T31:{dZ, dW}				
16	T32:{d0, d1}	T33:{d0, d1}	T34:{d0, d1}	T35:{d0, d1}					T32:{d4, d5}	T33:{d4, d5}	T34:{d4, d5}	T35:{d4, d5}					...	T32:{dX, dY}	T33:{dX, dY}	T34:{dX, dY}	T35:{dX, dY}				
..																	...								
31	T60:{d2, d3}	T61:{d2, d3}	T62:{d2, d3}	T63:{d2, d3}					T60:{d6, d7}	T61:{d6, d7}	T62:{d6, d7}	T63:{d6, d7}					...	T60:{dZ, dW}	T61:{dZ, dW}	T62:{dZ, dW}	T63:{dZ, dW}				
32	T64:{d0, d1}	T65:{d0, d1}	T66:{d0, d1}	T67:{d0, d1}					T64:{d4, d5}	T65:{d4, d5}	T66:{d4, d5}	T67:{d4, d5}					...	T64:{dX, dY}	T65:{dX, dY}	T66:{dX, dY}	T67:{dX, dY}				
..																	...								
47	T92:{d2, d3}	T93:{d2, d3}	T94:{d2, d3}	T95:{d2, d3}					T92:{d6, d7}	T93:{d6, d7}	T94:{d6, d7}	T95:{d6, d7}					...	T92:{dZ, dW}	T93:{dZ, dW}	T94:{dZ, dW}	T95:{dZ, dW}				
48	T96:{d0, d1}	T97:{d0, d1}	T98:{d0, d1}	T99:{d0, d1}					T96:{d4, d5}	T97:{d4, d5}	T98:{d4, d5}	T99:{d4, d5}					...	T96:{dX, dY}	T97:{dX, dY}	T98:{dX, dY}	T99:{dX, dY}				
..																	...								
63	T124:{d2, d3}	T125:{d2, d3}	T126:{d2, d3}	T127:{d2, d3}					T124:{d6, d7}	T125:{d6, d7}	T126:{d6, d7}	T127:{d6, d7}					...	T124:{dZ, dW}	T125:{dZ, dW}	T126:{dZ, dW}	T127:{dZ, dW}				

(tid % 128) : fragments

Figure 149: WGMMA .m64nNk16 register fragment layout for accumulator matrix D.

9.7.15.5.1.3 Matrix Fragments for `wgmma.mma_async.m64nNk8`

A warpgroup executing `wgmma.mma_async.m64nNk8` will compute an MMA operation of shape `.m64nNk8` where N is a valid n dimension as listed in [Matrix Shape](#).

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warp-group holds a fragment of the matrix.

- Multiplicand A in registers:

.atype	Fragment	Elements (low to high)
.tf32	A vector expression containing four .b32 registers containing four .tf32 elements from matrix A.	a0, a1, a2, a3

The layout of the fragments held by different threads is shown in [Figure 150](#).

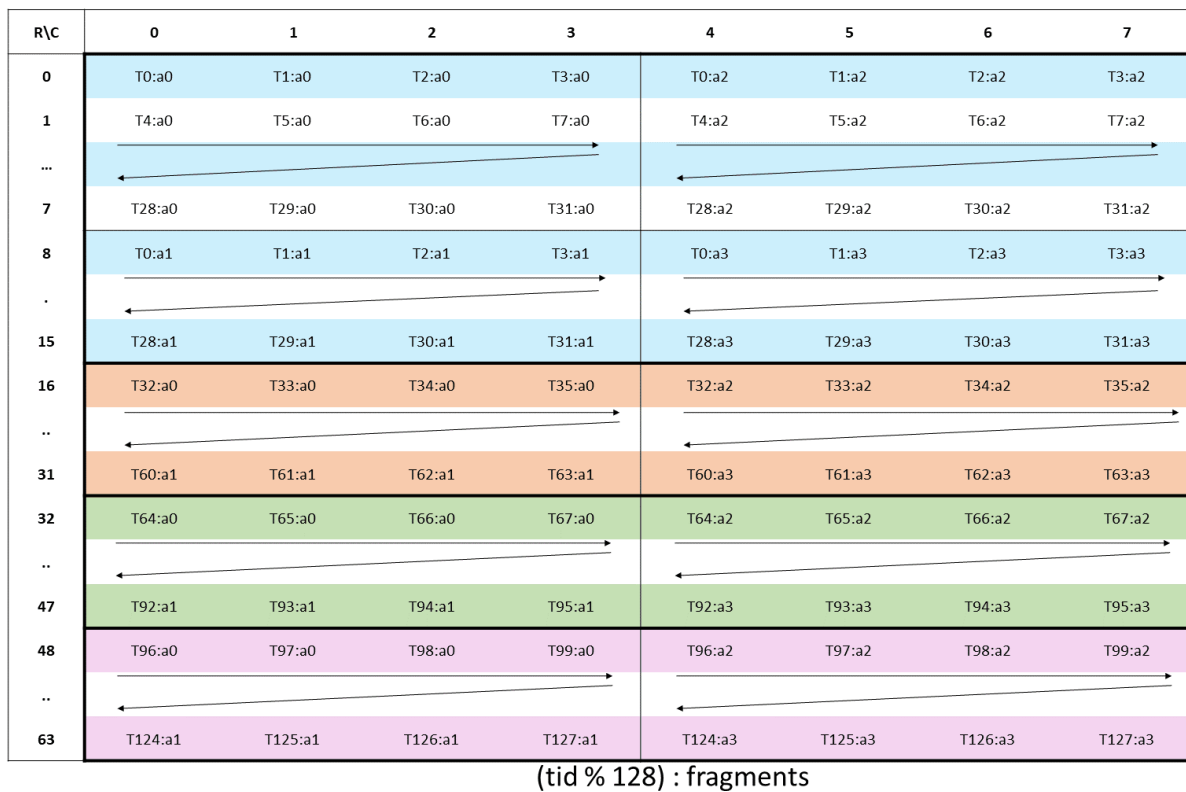


Figure 150: WGMMMA .m64nNk8 register fragment layout for matrix A.

- Accumulator D:

.dtype	Fragment	Elements (low to high)
.f32	A vector expression containing $N/2$ number of .f32 registers.	$d0, d1, d2, d3, \dots, dX, dY, dZ, dW$ where $X = N/2 - 4$ $Y = N/2 - 3$ $Z = N/2 - 2$ $W = N/2 - 1$ $N = 8*i$ where $i = \{1, 2, \dots, 32\}$

The layout of the fragments held by different threads is shown in [Figure 151](#).

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...	N-8	N-7	N-6	N-5	N-4	N-3	N-2	N-1
0	T0:{d0, d1}	T1:{d0, d1}	T2:{d0, d1}	T3:{d0, d1}					T0:{d4, d5}	T1:{d4, d5}	T2:{d4, d5}	T3:{d4, d5}					...	T0:{dX, dY}	T1:{dX, dY}	T2:{dX, dY}	T3:{dX, dY}				
1	T4:{d0, d1}	T5:{d0, d1}	T6:{d0, d1}	T7:{d0, d1}					T4:{d4, d5}	T5:{d4, d5}	T6:{d4, d5}	T7:{d4, d5}					...	T4:{dX, dY}	T5:{dX, dY}	T6:{dX, dY}	T7:{dX, dY}				
...																	...								
7	T28:{d0, d1}	T29:{d0, d1}	T30:{d0, d1}	T31:{d0, d1}					T28:{d4, d5}	T29:{d4, d5}	T30:{d4, d5}	T31:{d4, d5}					...	T28:{dX, dY}	T29:{dX, dY}	T30:{dX, dY}	T31:{dX, dY}				
8	T0:{d2, d3}	T1:{d2, d3}	T2:{d2, d3}	T3:{d2, d3}					T0:{d6, d7}	T1:{d6, d7}	T2:{d6, d7}	T3:{d6, d7}					...	T0:{dZ, dW}	T1:{dZ, dW}	T2:{dZ, dW}	T3:{dZ, dW}				
.																	...								
15	T28:{d2, d3}	T29:{d2, d3}	T30:{d2, d3}	T31:{d2, d3}					T28:{d6, d7}	T29:{d6, d7}	T30:{d6, d7}	T31:{d6, d7}					...	T28:{dZ, dW}	T29:{dZ, dW}	T30:{dZ, dW}	T31:{dZ, dW}				
16	T32:{d0, d1}	T33:{d0, d1}	T34:{d0, d1}	T35:{d0, d1}					T32:{d4, d5}	T33:{d4, d5}	T34:{d4, d5}	T35:{d4, d5}					...	T32:{dX, dY}	T33:{dX, dY}	T34:{dX, dY}	T35:{dX, dY}				
..																	...								
31	T60:{d2, d3}	T61:{d2, d3}	T62:{d2, d3}	T63:{d2, d3}					T60:{d6, d7}	T61:{d6, d7}	T62:{d6, d7}	T63:{d6, d7}					...	T60:{dZ, dW}	T61:{dZ, dW}	T62:{dZ, dW}	T63:{dZ, dW}				
32	T64:{d0, d1}	T65:{d0, d1}	T66:{d0, d1}	T67:{d0, d1}					T64:{d4, d5}	T65:{d4, d5}	T66:{d4, d5}	T67:{d4, d5}					...	T64:{dX, dY}	T65:{dX, dY}	T66:{dX, dY}	T67:{dX, dY}				
..																	...								
47	T92:{d2, d3}	T93:{d2, d3}	T94:{d2, d3}	T95:{d2, d3}					T92:{d6, d7}	T93:{d6, d7}	T94:{d6, d7}	T95:{d6, d7}					...	T92:{dZ, dW}	T93:{dZ, dW}	T94:{dZ, dW}	T95:{dZ, dW}				
48	T96:{d0, d1}	T97:{d0, d1}	T98:{d0, d1}	T99:{d0, d1}					T96:{d4, d5}	T97:{d4, d5}	T98:{d4, d5}	T99:{d4, d5}					...	T96:{dX, dY}	T97:{dX, dY}	T98:{dX, dY}	T99:{dX, dY}				
..																	...								
63	T124:{d2, d3}	T125:{d2, d3}	T126:{d2, d3}	T127:{d2, d3}					T124:{d6, d7}	T125:{d6, d7}	T126:{d6, d7}	T127:{d6, d7}					...	T124:{dZ, dW}	T125:{dZ, dW}	T126:{dZ, dW}	T127:{dZ, dW}				

(tid % 128) : fragments

Figure 151: WGMMA .m64nNk8 register fragment layout for accumulator matrix D.

9.7.15.5.1.4 Matrix Fragments for wgmma.mma_async.m64nNk32

A warpgroup executing `wgmma.mma_async.m64nNk32` will compute an MMA operation of shape .m64nNk32 where N is a valid n dimension as listed in [Matrix Shape](#).

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warpgroup holds a fragment of the matrix.

- Multiplicand A in registers:

.atype	Fragment	Elements (low to high)
.s8/.u8	A vector expression containing four .b32 registers, with each register containing four .u8/.s8 elements from matrix A.	a0, a1, a2, a3, ..., a14, a15
.e4m3/.e5m2	A vector expression containing four .b32 registers, with each register containing four .e4m3/.e5m2 elements from matrix A.	

The layout of the fragments held by different threads is shown in [Figure 152](#).

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	T0:{a0, a1, a2, a3}				T1:{a0, a1, a2, a3}				T2:{a0, a1, a2, a3}				T3:{a0, a1, a2, a3}				T0:{a8, a9, a10, a11}				T1:{a8, a9, a10, a11}				T2:{a8, a9, a10, a11}				T3:{a8, a9, a10, a11}			
1	T4:{a0, a1, a2, a3}				T5:{a0, a1, a2, a3}				T6:{a0, a1, a2, a3}				T7:{a0, a1, a2, a3}				T4:{a8, a9, a10, a11}				T5:{a8, a9, a10, a11}				T6:{a8, a9, a10, a11}				T7:{a8, a9, a10, a11}			
...																																
7	T28:{a0, a1, a2, a3}				T29:{a0, a1, a2, a3}				T30:{a0, a1, a2, a3}				T31:{a0, a1, a2, a3}				T28:{a8, a9, a10, a11}				T29:{a8, a9, a10, a11}				T30:{a8, a9, a10, a11}				T31:{a8, a9, a10, a11}			
8	T0:{a4, a5, a6, a7}				T1:{a4, a5, a6, a7}				T2:{a4, a5, a6, a7}				T3:{a4, a5, a6, a7}				T0:{a12, a13, a14, a15}				T1:{a12, a13, a14, a15}				T2:{a12, a13, a14, a15}				T3:{a12, a13, a14, a15}			
.																																
15	T28:{a4, a5, a6, a7}				T29:{a4, a5, a6, a7}				T30:{a4, a5, a6, a7}				T31:{a4, a5, a6, a7}				T28:{a12, a13, a14, a15}				T29:{a12, a13, a14, a15}				T30:{a12, a13, a14, a15}				T31:{a12, a13, a14, a15}			
16	T32:{a0, a1, a2, a3}				T33:{a0, a1, a2, a3}				T34:{a0, a1, a2, a3}				T35:{a0, a1, a2, a3}				T32:{a8, a9, a10, a11}				T33:{a8, a9, a10, a11}				T34:{a8, a9, a10, a11}				T35:{a8, a9, a10, a11}			
..																																
31	T60:{a4, a5, a6, a7}				T61:{a4, a5, a6, a7}				T62:{a4, a5, a6, a7}				T63:{a4, a5, a6, a7}				T60:{a12, a13, a14, a15}				T61:{a12, a13, a14, a15}				T62:{a12, a13, a14, a15}				T63:{a12, a13, a14, a15}			
32	T64:{a0, a1, a2, a3}				T65:{a0, a1, a2, a3}				T66:{a0, a1, a2, a3}				T67:{a0, a1, a2, a3}				T64:{a8, a9, a10, a11}				T65:{a8, a9, a10, a11}				T66:{a8, a9, a10, a11}				T67:{a8, a9, a10, a11}			
..																																
47	T92:{a4, a5, a6, a7}				T93:{a4, a5, a6, a7}				T94:{a4, a5, a6, a7}				T95:{a4, a5, a6, a7}				T92:{a12, a13, a14, a15}				T93:{a12, a13, a14, a15}				T94:{a12, a13, a14, a15}				T95:{a12, a13, a14, a15}			
48	T96:{a0, a1, a2, a3}				T97:{a0, a1, a2, a3}				T98:{a0, a1, a2, a3}				T99:{a0, a1, a2, a3}				T96:{a8, a9, a10, a11}				T97:{a8, a9, a10, a11}				T98:{a8, a9, a10, a11}				T99:{a8, a9, a10, a11}			
..																																
63	T124:{a4, a5, a6, a7}				T125:{a4, a5, a6, a7}				T126:{a4, a5, a6, a7}				T127:{a4, a5, a6, a7}				T124:{a12, a13, a14, a15}				T125:{a12, a13, a14, a15}				T126:{a12, a13, a14, a15}				T127:{a12, a13, a14, a15}			

(tid % 128) : fragments

Figure 152: WGMMA .m64nNk32 register fragment layout for matrix A.

► Accumulator D:

.dtype	Fragment	Elements (low to high)	Miscellaneous Information
.s32	A vector expression containing N/2 number of .s32 registers.	d0, d1, d2, d3, ..., dX, dY, dZ, dW where $X = N/2 - 4$ $Y = N/2 - 3$ $Z = N/2 - 2$ $W = N/2 - 1$ N depends on .dtype, as described in the next column.	$N = 8*i$ where $i = \{1, 2, 3, 4\}$ $= 16*i$ where $i = \{3, 4, \dots, 15, 16\}$
.f32	A vector expression containing N/2 number of .f32 registers.		$N = 8*i$ where $i = \{1, 2, \dots, 32\}$
.f16	A vector expression containing N/4 number of .f16x2 registers, with each register containing two .f16 elements from matrix D.		

The layout of the fragments held by different threads is shown in [Figure 153](#).

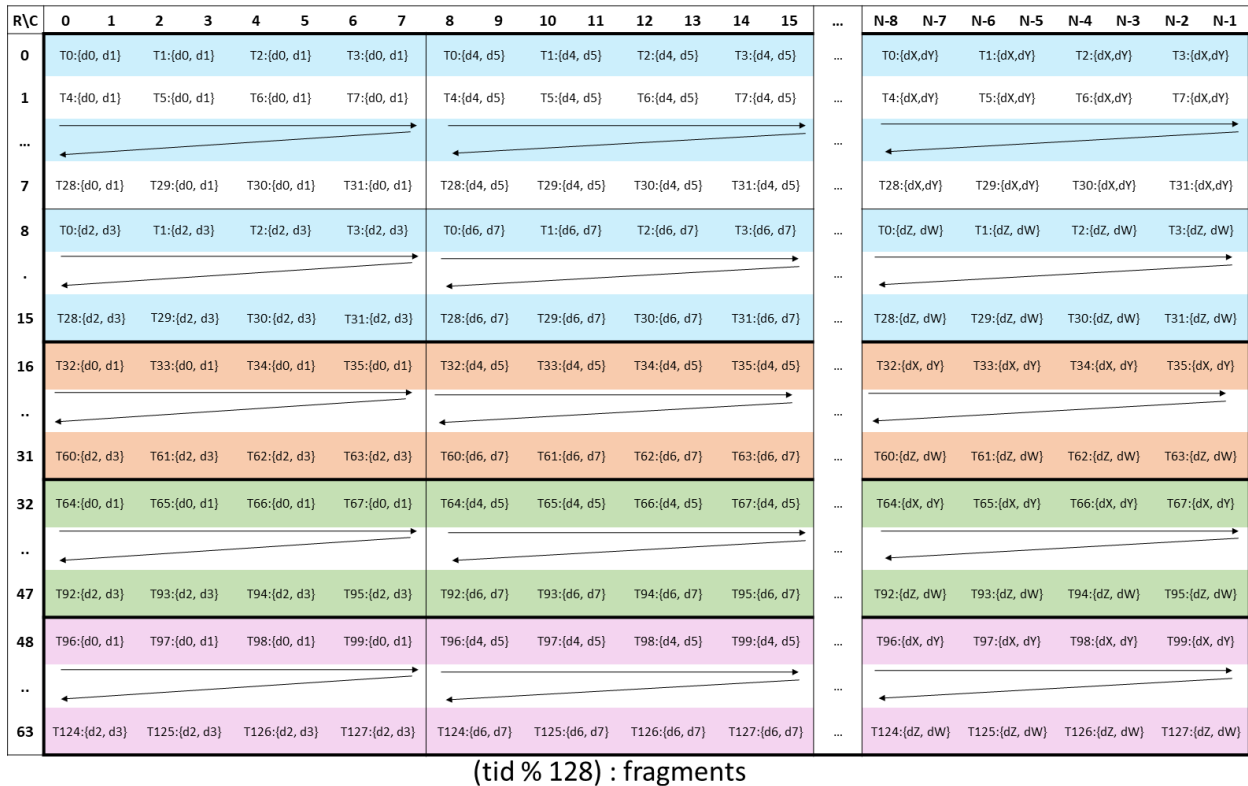


Figure 153: WGMMA .m64nNk32 register fragment layout for accumulator matrix D.

9.7.15.5.1.5 Matrix Fragments for `wgmma.mma_async.m64nNk256`

A warpgroup executing `wgmma.mma_async.m64nNk256` will compute an MMA operation of shape `.m64nNk256` where N is a valid n dimension as listed in [Matrix Shape](#).

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warpgroup holds a fragment of the matrix.

- Multiplicand A in registers:

.atype	Fragment	Elements (low to high)
.b1	A vector expression containing four .b32 registers, with each register containing thirty two .b1 element from matrix A.	a0, a1, a2, ..., a127

The layout of the fragments held by different threads is shown in [Figure 154](#).

R\C	0	...	31	32	..	64	64	..	95	96	..	127	128	..	159	160	..	191	192	..	223	224	..	255
0	T0:{a0, a1, ..., a31}		T1:{a0, a1, ..., a31}			T2:{a0, a1, ..., a31}				T3:{a0, a1, ..., a31}			T0:{a64 a65, ..., a95}			T1:{a64 a65, ..., a95}			T2:{a64 a65, ..., a95}			T3:{a64 a65, ..., a95}		
1	T4:{a0, a1, ..., a31}		T5:{a0, a1, ..., a31}			T6:{a0, a1, ..., a31}				T7:{a0, a1, ..., a31}			T4:{a64 a65, ..., a95}			T5:{a64 a65, ..., a95}			T6:{a64 a65, ..., a95}			T7:{a64 a65, ..., a95}		
...																								
7	T28:{a0, a1, ..., a31}		T29:{a0, a1, ..., a31}			T30:{a0, a1, ..., a31}				T31:{a0, a1, ..., a31}			T28:{a64 a65, ..., a95}			T29:{a64 a65, ..., a95}			T30:{a64 a65, ..., a95}			T31:{a64 a65, ..., a95}		
8	T0:{a32 a33, ..., a63}		T1:{a32 a33, ..., a63}			T2:{a32 a33, ..., a63}				T3:{a32 a33, ..., a63}			T0:{a96 a97, ..., a127}			T1:{a96 a97, ..., a127}			T2:{a96 a97, ..., a127}			T3:{a96 a97, ..., a127}		
...																								
15	T28:{a32 a33, ..., a63}		T29:{a32 a33, ..., a63}			T30:{a32 a33, ..., a63}				T31:{a32 a33, ..., a63}			T28:{a96 a97, ..., a127}			T29:{a96 a97, ..., a127}			T30:{a96 a97, ..., a127}			T31:{a96 a97, ..., a127}		
16	T32:{a0, a1, ..., a31}		T33:{a0, a1, ..., a31}			T34:{a0, a1, ..., a31}				T35:{a0, a1, ..., a31}			T32:{a64 a65, ..., a95}			T33:{a64 a65, ..., a95}			T34:{a64 a65, ..., a95}			T35:{a64 a65, ..., a95}		
...																								
31	T60:{a32 a33, ..., a63}		T61:{a32 a33, ..., a63}			T62:{a32 a33, ..., a63}				T63:{a32 a33, ..., a63}			T60:{a96 a97, ..., a127}			T61:{a96 a97, ..., a127}			T62:{a96 a97, ..., a127}			T63:{a96 a97, ..., a127}		
32	T64:{a0, a1, ..., a31}		T65:{a0, a1, ..., a31}			T66:{a0, a1, ..., a31}				T67:{a0, a1, ..., a31}			T64:{a64 a65, ..., a95}			T65:{a64 a65, ..., a95}			T66:{a64 a65, ..., a95}			T67:{a64 a65, ..., a95}		
...																								
47	T92:{a32 a33, ..., a63}		T93:{a32 a33, ..., a63}			T94:{a32 a33, ..., a63}				T95:{a32 a33, ..., a63}			T92:{a96 a97, ..., a127}			T93:{a96 a97, ..., a127}			T94:{a96 a97, ..., a127}			T95:{a96 a97, ..., a127}		
48	T96:{a0, a1, ..., a31}		T97:{a0, a1, ..., a31}			T98:{a0, a1, ..., a31}				T99:{a0, a1, ..., a31}			T96:{a64 a65, ..., a95}			T97:{a64 a65, ..., a95}			T98:{a64 a65, ..., a95}			T99:{a64 a65, ..., a95}		
...																								
63	T124:{a32 a33, ..., a63}		T125:{a32 a33, ..., a63}			T126:{a32 a33, ..., a63}				T127:{a32 a33, ..., a63}			T124:{a96 a97, ..., a127}			T125:{a96 a97, ..., a127}			T126:{a96 a97, ..., a127}			T127:{a96 a97, ..., a127}		

(tid % 128) : fragments

Figure 154: WGMMA .m64nNk256 register fragment layout for matrix A.

- Accumulator D:

R/C	0	1	2	3	4	5	6	7	...	N-8	N-7	N-6	N-5	N-4	N-3	N-2	N-1
0	T0:d0	T1:d0	T2:d0	T3:d0	T0:d2	T1:d2	T2:d2	T3:d2	...	T0:dX	T1:dX	T2:dX	T3:dX	T0:dZ	T1:dZ	T2:dZ	T3:dZ
1	T4:d0	T5:d0	T6:d0	T7:d0	T4:d2	T5:d2	T6:d2	T7:d2	...	T4:dX	T5:dX	T6:dX	T7:dX	T4:dZ	T5:dZ	T6:dZ	T7:dZ
...									...								
7	T28:d0	T29:d0	T30:d0	T31:d0	T28:d2	T29:d2	T30:d2	T31:d2	...	T28:dX	T29:dX	T30:dX	T31:dX	T28:dZ	T29:dZ	T30:dZ	T31:dZ
8	T0:d1	T1:d1	T2:d1	T3:d1	T0:d3	T1:d3	T2:d3	T3:d3	...	T0:dY	T1:dY	T2:dY	T3:dY	T0:dW	T1:dW	T2:dW	T3:dW
...									...								
15	T28:d1	T29:d1	T30:d1	T31:d1	T28:d3	T29:d3	T30:d3	T31:d1	...	T28:dY	T29:dY	T30:dY	T31:dY	T28:dW	T29:dW	T30:dW	T31:dW
16	T32:d0	T33:d0	T34:d0	T35:d0	T32:d2	T33:d2	T34:d2	T35:d2	...	T32:dX	T33:dX	T34:dX	T35:dX	T32:dZ	T33:dZ	T34:dZ	T35:dZ
...									...								
31	T60:d1	T61:d1	T62:d1	T63:d1	T60:d3	T61:d3	T62:d3	T63:d3	...	T60:dY	T61:dY	T62:dY	T63:dY	T60:dW	T61:dW	T62:dW	T63:dW
32	T64:d0	T65:d0	T66:d0	T67:d0	T64:d2	T65:d2	T66:d2	T67:d2	...	T64:dX	T65:dX	T66:dX	T67:dX	T64:dZ	T65:dZ	T66:dZ	T67:dZ
...									...								
47	T92:d1	T93:d1	T94:d1	T95:d1	T92:d3	T93:d3	T94:d3	T95:d3	...	T92:dY	T93:dY	T94:dY	T95:dY	T92:dW	T93:dW	T94:dW	T95:dW
48	T96:d0	T97:d0	T98:d0	T99:d0	T96:d2	T97:d2	T98:d2	T99:d2	...	T96:dX	T97:dX	T98:dX	T99:dX	T96:dZ	T97:dZ	T98:dZ	T99:dZ
...									...								
63	T124:d1	T125:d1	T126:d1	T127:d1	T124:d3	T125:d3	T126:d3	T127:d3	...	T124:dY	T125:dY	T126:dY	T127:dY	T124:dW	T125:dW	T126:dW	T127:dW

Figure 155: WGMMMA .m64nNk256 register fragment layout for accumulator matrix D.

9.7.15.5.1.6 Shared Memory Matrix Layout

If the argument `imm-trans-a / imm-trans-b` of the instruction `wmma.mma_async{.sp}` is 0, then *K-major* is used for matrix A / B respectively. If the value of argument `imm-trans-a` is 1 then *M-major* is used for matrix A. If the value of the argument `imm-trans-b` is 1, then *N-major* is used for matrix B.

In a column-major default BLAS library such as cuBLAS, the matrices A and B with and without transpose can be classified as either *K-Major* or *M-or-N-Major* as shown in the following table:

	Non-Transposed	Transposed
A	K-major	M-major
B	K-major	N-major

To avoid confusion with A, B, row-major, col-major, transpose, and non-transpose, we will use *MN-Major* and *K-Major* throughout this section.

The matrices in the shared memory are made up of one or more “swizzle layout atom”. The exact layout of these swizzle atoms depends on the swizzling mode, swizzle-atomicity, and the leading dimension. The layout of the swizzle are shown in [Table 40](#).

Table 40: Various combinations of swizzling mode, leading dimension and swizzle-atom layout

Swizzling mode	Leading Dimension / Major-ness	Swizzle atom layout (128b element)
128B Swizzling Mode	M/N	8x8
	K	8x8
64B Swizzling Mode	M/N	4x8
	K	8x4
32B Swizzling Mode	M/N	2x8
	K	8x2
None	M/N	1x8
	K	8x1

The above shapes are for elements of size 128 bits. For smaller elements sizes, the same shapes would get multiplied along the leading dimension by a factor of $128/\text{sizeof_bits}(\text{Element})$. For example, 128B MN major swizzle atom would have a shape of $(8*(128/32)) \times 8 = 32 \times 8$ for tf32 tensor core inputs.

Examples

The following are some example layouts of $M \times K$ or $K \times N$ matrices with various swizzling modes, and are in units of 128b elements as shown by each colored cell as shown in [Figure 156](#), [Figure 157](#), [Figure 158](#), [Figure 159](#), [Figure 160](#), [Figure 161](#), [Figure 162](#), [Figure 163](#).

Following are some of the examples of the 128B swizzling layout for `tf32` element type.

- ▶ K-Major: [Figure 164](#)
- ▶ MN-Major: [Figure 165](#)

9.7.15.5.1.7 Major-ness supported by Strides

There are two strides involved while accessing a matrix from shared memory:

1. Leading dimension byte offset
2. Stride dimension byte offset

9.7.15.5.1.8 Leading Dimension Byte Offset

The leading dimension byte offset is defined differently for transposed and non-transposed matrices. The leading byte offset is defined as follows for matrices whose element types are normalized to 128-bits:

Major-ness	Definition
K-Major	▶ No-Swizzling: the offset from the first column to the second columns of the 8×2 tile in the 128-bit element type normalized matrix. ▶ Swizzled layouts: not used, assumed to be 1.
MN-Major	▶ Interleave: offset from the first 8 columns to the next 8 columns. ▶ Swizzled layouts: offset from the first ($\text{swizzle-byte-size}/16$) rows to the next ($\text{swizzle-byte-size}/16$) rows.

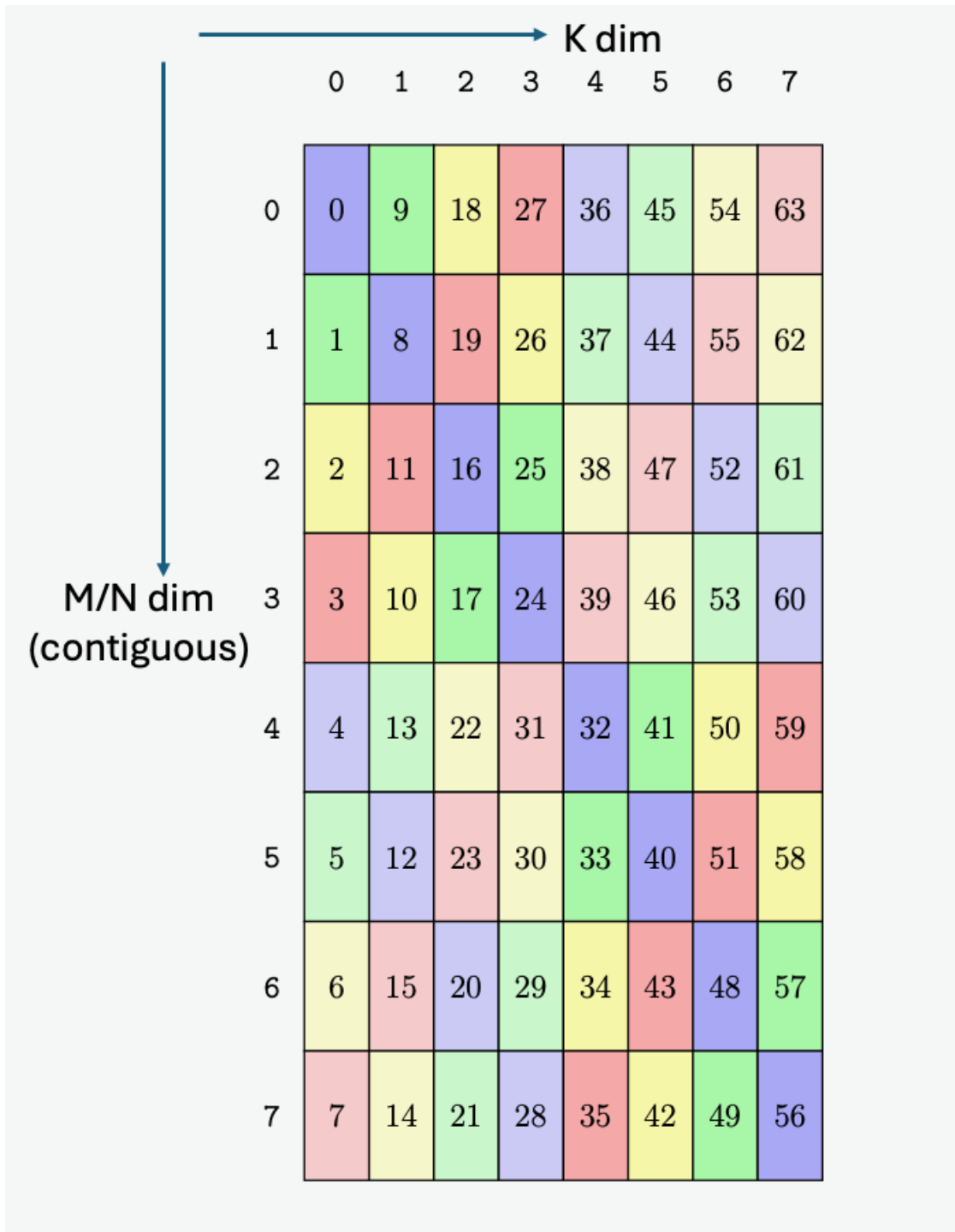


Figure 156: MN major 128B swizzling

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	9	8	11	10	13	12	15	14
2	18	19	16	17	22	23	20	21
3	27	26	25	24	31	30	29	28
4	36	37	38	39	32	33	34	35
5	45	44	47	46	41	40	43	42
6	54	55	52	53	50	51	48	49
7	63	62	61	60	59	58	57	56

Figure 157: K major 128B swizzling

	0	1	2	3	4	5	6	7
0	0	4	9	13	18	22	27	31
1	1	5	8	12	19	23	26	30
2	2	6	11	15	16	20	25	29
3	3	7	10	14	17	21	24	28

Figure 158: MN major 64B swizzling

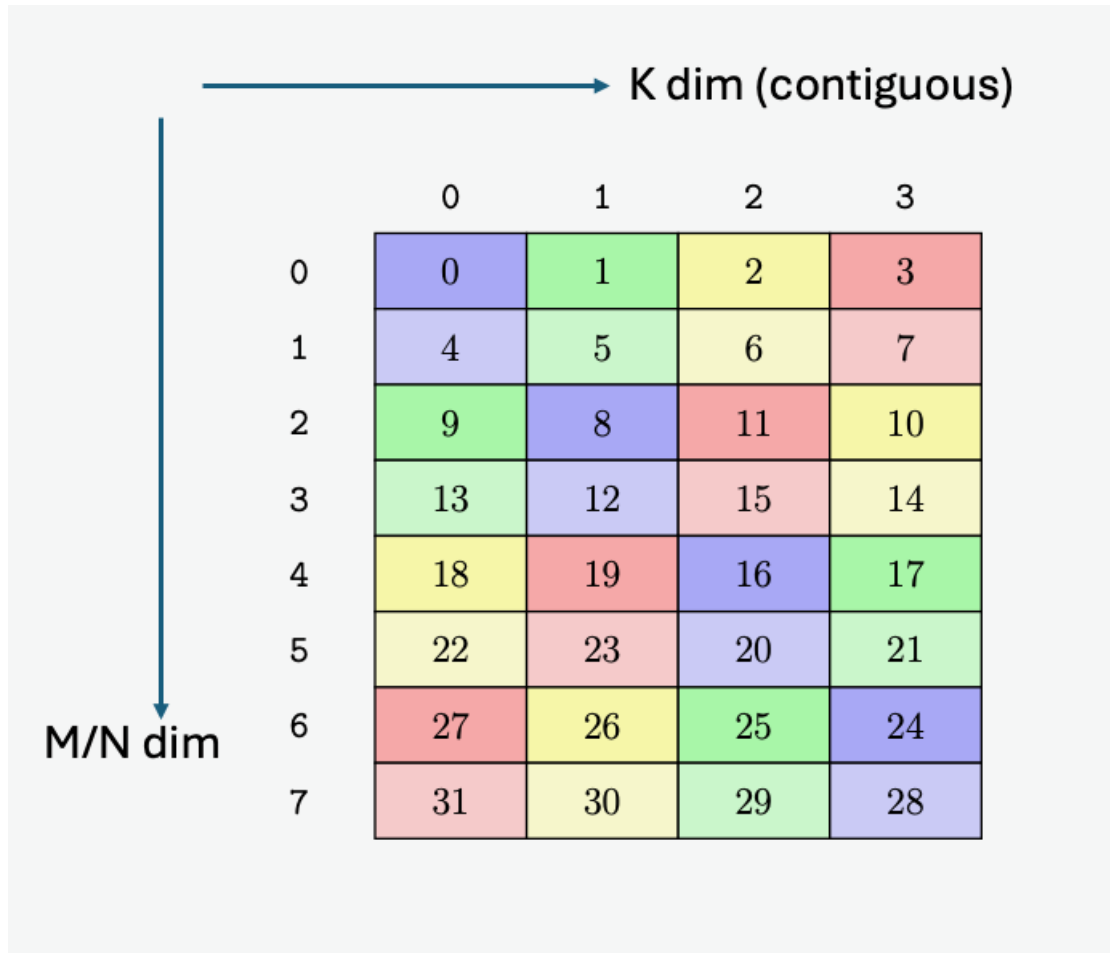


Figure 159: K major 64B swizzling

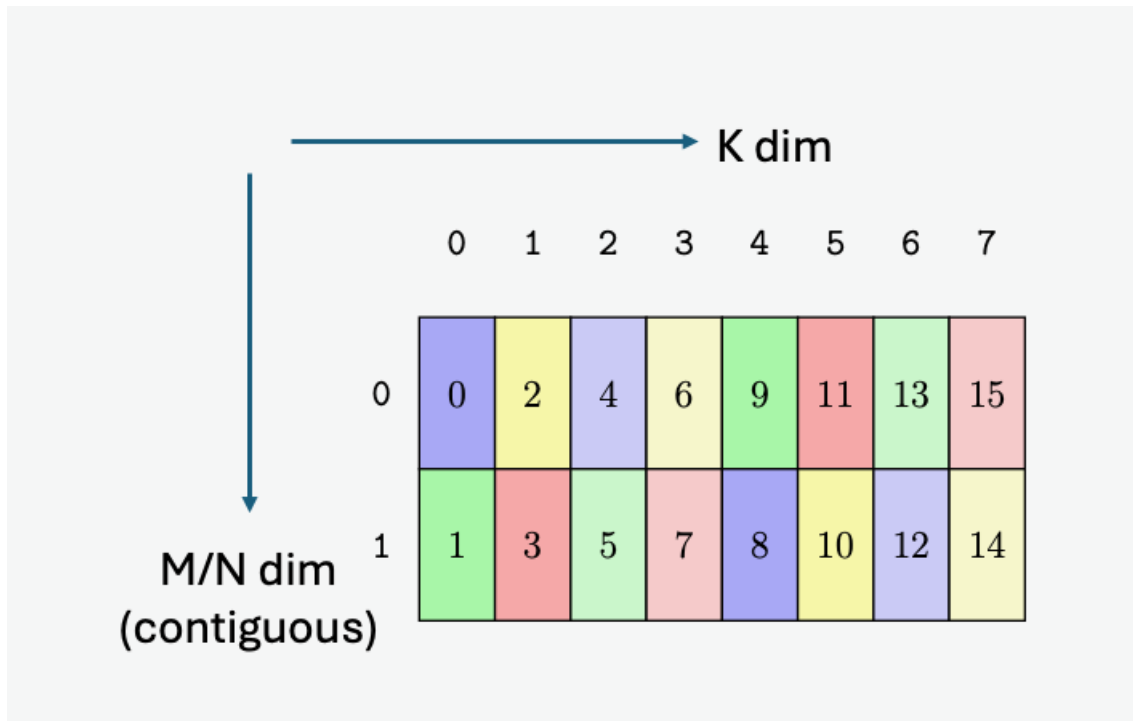


Figure 160: MN major 32B swizzling

9.7.15.5.1.9 Stride Dimension Byte Offset

The stride dimension byte offset is defined differently for transposed and non-transposed matrices. The stride dimension byte offset is defined as follows for matrices whose element types are normalized to 128-bits:

Major-ness	Definition
K-Major	The offset from the first 8 rows to the next 8 rows.
MN-Major	► Interleave: offset from the first row to the next row. ► Swizzled layout: offset from the first 8 columns to the next 8 columns

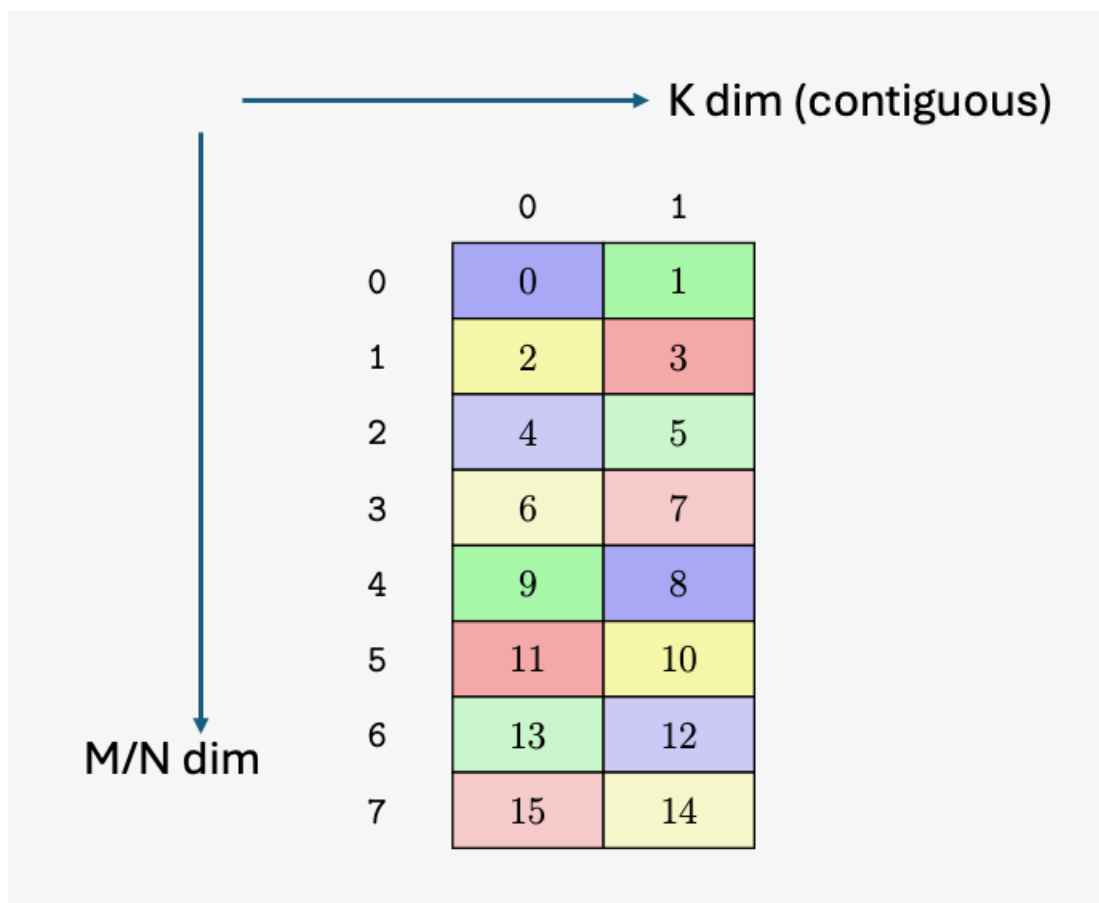


Figure 161: K major 32B swizzling

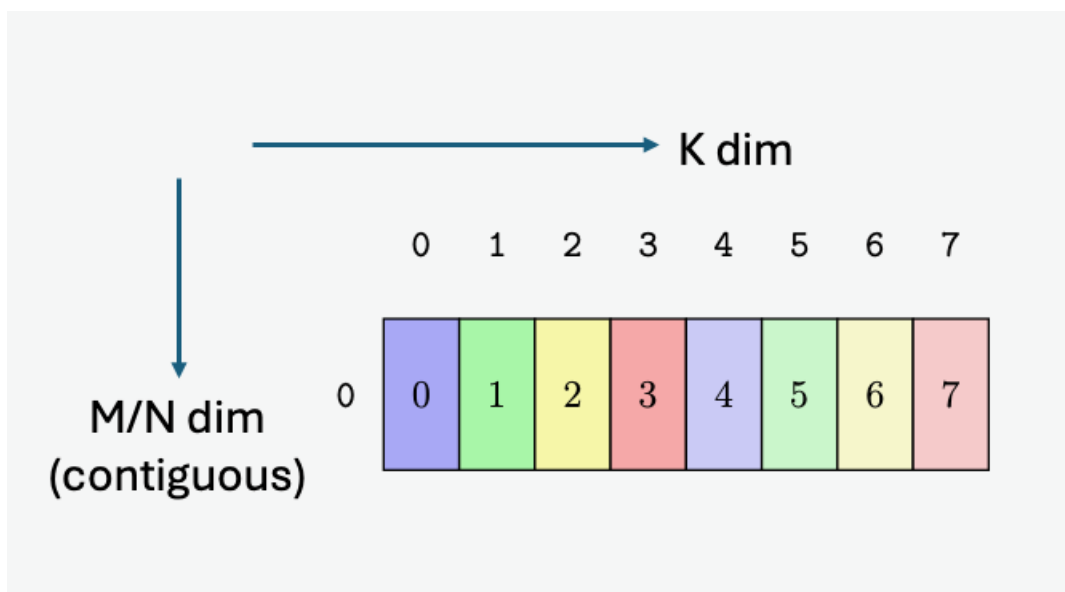


Figure 162: MN major interleaved

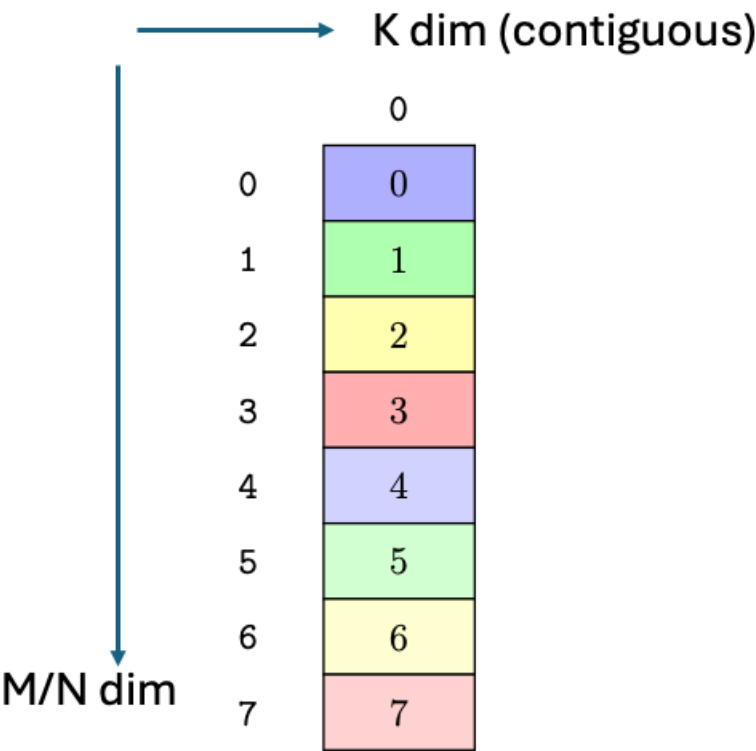


Figure 163: K major interleaved



Figure 164: K major

	K dim							
	0	1	2	3	4	5	6	7
0	0	36	72	108	144	180	216	252
1	1	37	73	109	145	181	217	253
2	2	38	74	110	146	182	218	254
3	3	39	75	111	147	183	219	255
4	4	32	76	104	148	176	220	248
5	5	33	77	105	149	177	221	249
6	6	34	78	106	150	178	222	250
7	7	35	79	107	151	179	223	251
8	8	44	64	100	152	188	208	244
9	9	45	65	101	153	189	209	245
10	10	46	66	102	154	190	210	246
11	11	47	67	103	155	191	211	247
12	12	40	68	96	156	184	212	240
13	13	41	69	97	157	185	213	241
14	14	42	70	98	158	186	214	242
15	15	43	71	99	159	187	215	243
16	16	52	88	124	128	164	200	236
17	17	53	89	125	129	165	201	237
18	18	54	90	126	130	166	202	238
19	19	55	91	127	131	167	203	239
20	20	48	92	120	132	160	204	232
21	21	49	93	121	133	161	205	233
22	22	50	94	122	134	162	206	234
23	23	51	95	123	135	163	207	235
24	24	60	80	116	136	172	192	228
25	25	61	81	117	137	173	193	229
26	26	62	82	118	138	174	194	230
27	27	63	83	119	139	175	195	231
28	28	56	84	112	140	168	196	224
29	29	57	85	113	141	169	197	225
30	30	58	86	114	142	170	198	226
31	31	59	87	115	143	171	199	227

Figure 165: MN major

9.7.15.5.1.10 Canonical Layouts

In terms of [CuTe layouts](#) the canonical layout can be expressed as follows:

Major-ness	Swizzling mode	Canonical Layout without swizzling	Swizzling on the previous column
MN- major	No-swizzling or Inter-leaved	$((T, 1, m), (8, k)) : ((1, T, SBO), (1, T, LBO))$	Swizzle<0, 4, 3>
	32B Swizzling	$((T, 2, m), (8, k)) : ((1, T, LBO), (2, T, SBO))$	Swizzle<1, 4, 3>
	64B Swizzling	$((T, 4, m), (8, k)) : ((1, T, LBO), (4, T, SBO))$	Swizzle<2, 4, 3>
	128B Swizzling	$((T, 8, m), (8, k)) : ((1, T, LBO), (8, T, SBO))$	Swizzle<3, 4, 3>
K- major	No-swizzling or Inter-leaved	$((8, m), (T, 2k)) : ((1, T, SBO), (1, LBO))$	Swizzle<0, 4, 3>
	32B Swizzling	$((8, m), (T, 2k)) : ((2, T, SBO), (1, T))$	Swizzle<1, 4, 3>
	64B Swizzling	$((8, m), (T, 2k)) : ((4, T, SBO), (1, T))$	Swizzle<2, 4, 3>
	128B Swizzling	$((8, m), (T, 2k)) : ((8, T, SBO), (1, T))$	Swizzle<3, 4, 3>

where

- $T = 128 / \text{sizeof-elements-in-bits}$ T represents scale factor which normalizes matrix element types to 128-bits.
- m represents the number of repeating patterns across rows.
- k represents the number of repeating patterns across columns.

Examples

- K-Major, no-swizzling and tf32 type: [Figure 166](#)

the strides and related details are as follows:

Exact layout : Swizzle<0,4,3> o $((8, 2), (4, 4)) : ((4, 32), (1, 64))$

Canonical Layout : Swizzle<0,4,3> o $((8, m), (T, 2k)) : ((1, T, SBO), (1, LBO))$

Parameters	Value
T	4
m	2
k	2
LBO	$64 * \text{sizeof}(\text{tf32})$
SBO	$32 * \text{sizeof}(\text{tf32})$
Encoding of LBO in descriptor	$(LBO) \gg 4 = 16$
Encoding of SBO in descriptor	$(SBO) \gg 4 = 8$

- K-Major, 32B swizzling and tf32 type: [Figure 167](#)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	64	65	66	67	128	129	130	131	192	193	194	195
1	4	5	6	7	68	69	70	71	132	133	134	135	196	197	198	199
2	8	9	10	11	72	73	74	75	136	137	138	139	200	201	202	203
3	12	13	14	15	76	77	78	79	140	141	142	143	204	205	206	207
4	16	17	18	19	80	81	82	83	144	145	146	147	208	209	210	211
5	20	21	22	23	84	85	86	87	148	149	150	151	212	213	214	215
6	24	25	26	27	88	89	90	91	152	153	154	155	216	217	218	219
7	28	29	30	31	92	93	94	95	156	157	158	159	220	221	222	223
8	32	33	34	35	96	97	98	99	160	161	162	163	224	225	226	227
9	36	37	38	39	100	101	102	103	164	165	166	167	228	229	230	231
10	40	41	42	43	104	105	106	107	168	169	170	171	232	233	234	235
11	44	45	46	47	108	109	110	111	172	173	174	175	236	237	238	239
12	48	49	50	51	112	113	114	115	176	177	178	179	240	241	242	243
13	52	53	54	55	116	117	118	119	180	181	182	183	244	245	246	247
14	56	57	58	59	120	121	122	123	184	185	186	187	248	249	250	251
15	60	61	62	63	124	125	126	127	188	189	190	191	252	253	254	255

Figure 166: K major, no-swizzling and tf32 type

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	4	5	6	7	128	129	130	131	132	133	134	135
1	8	9	10	11	12	13	14	15	136	137	138	139	140	141	142	143
2	16	17	18	19	20	21	22	23	144	145	146	147	148	149	150	151
3	24	25	26	27	28	29	30	31	152	153	154	155	156	157	158	159
4	36	37	38	39	32	33	34	35	164	165	166	167	160	161	162	163
5	44	45	46	47	40	41	42	43	172	173	174	175	168	169	170	171
6	52	53	54	55	48	49	50	51	180	181	182	183	176	177	178	179
7	60	61	62	63	56	57	58	59	188	189	190	191	184	185	186	187
8	64	65	66	67	68	69	70	71	192	193	194	195	196	197	198	199
9	72	73	74	75	76	77	78	79	200	201	202	203	204	205	206	207
10	80	81	82	83	84	85	86	87	208	209	210	211	212	213	214	215
11	88	89	90	91	92	93	94	95	216	217	218	219	220	221	222	223
12	100	101	102	103	96	97	98	99	228	229	230	231	224	225	226	227
13	108	109	110	111	104	105	106	107	236	237	238	239	232	233	234	235
14	116	117	118	119	112	113	114	115	244	245	246	247	240	241	242	243
15	124	125	126	127	120	121	122	123	252	253	254	255	248	249	250	251

Figure 167: K major, 32B swizzling and tf32 type

the strides and related details are as follows:

Exact layout : Swizzle<1,4,3> o ((8,2),(4,4)):((8,64),(1,4))

Canonical Layout :Swizzle<1,4,3> o ((8,m),(T,2k)):((2T,SBO),(1,T))

Parameters	Value
T	4
m	2
k	2
LBO	NA
SBO	64*sizeof(tf32)
Encoding of LBO in descriptor	1 (assumed)
Encoding of SBO in descriptor	(SBO) >> 4 = 16

- MN-Major, no-swizzling and bf16 type: [Figure 168](#)

the strides and related details are as follows:

Exact layout : Swizzle<0,4,3> o ((8,1,2),(8,2)):((1,8,64),(8,128))

Canonical Layout :Swizzle<0,4,3> o ((T,1,m),(8,k)):((1,T,SBO),(1T,LBO))

Parameters	Value
T	8
m	2
k	2
LBO	128*sizeof(bf16)
SBO	64*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 16
Encoding of SBO in descriptor	(SBO) >> 4 = 8

- MN-Major, 32B swizzling and bf16 type: [Figure 169](#)

the strides and related details are as follows:

Exact layout : Swizzle<1,4,3> o ((8,2,2),(8,2)):((1,8,128),(16,256))

Canonical Layout :Swizzle<1,4,3> o ((T,2,m),(8,k)):((1,T,LBO),(2T,SBO))

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	8	16	24	32	40	48	56	128	136	144	152	160	168	176	184
1	1	9	17	25	33	41	49	57	129	137	145	153	161	169	177	185
2	2	10	18	26	34	42	50	58	130	138	146	154	162	170	178	186
3	3	11	19	27	35	43	51	59	131	139	147	155	163	171	179	187
4	4	12	20	28	36	44	52	60	132	140	148	156	164	172	180	188
5	5	13	21	29	37	45	53	61	133	141	149	157	165	173	181	189
6	6	14	22	30	38	46	54	62	134	142	150	158	166	174	182	190
7	7	15	23	31	39	47	55	63	135	143	151	159	167	175	183	191
8	64	72	80	88	96	104	112	120	192	200	208	216	224	232	240	248
9	65	73	81	89	97	105	113	121	193	201	209	217	225	233	241	249
10	66	74	82	90	98	106	114	122	194	202	210	218	226	234	242	250
11	67	75	83	91	99	107	115	123	195	203	211	219	227	235	243	251
12	68	76	84	92	100	108	116	124	196	204	212	220	228	236	244	252
13	69	77	85	93	101	109	117	125	197	205	213	221	229	237	245	253
14	70	78	86	94	102	110	118	126	198	206	214	222	230	238	246	254
15	71	79	87	95	103	111	119	127	199	207	215	223	231	239	247	255

Figure 168: MN major, no-swizzling and bf16 type

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	16	32	48	72	88	104	120	256	272	288	304	328	344	360	376
1	1	17	33	49	73	89	105	121	257	273	289	305	329	345	361	377
2	2	18	34	50	74	90	106	122	258	274	290	306	330	346	362	378
3	3	19	35	51	75	91	107	123	259	275	291	307	331	347	363	379
4	4	20	36	52	76	92	108	124	260	276	292	308	332	348	364	380
5	5	21	37	53	77	93	109	125	261	277	293	309	333	349	365	381
6	6	22	38	54	78	94	110	126	262	278	294	310	334	350	366	382
7	7	23	39	55	79	95	111	127	263	279	295	311	335	351	367	383
8	8	24	40	56	80	96	112	128	264	280	296	312	320	336	352	368
9	9	25	41	57	81	97	113	129	265	281	297	313	321	337	353	369
10	10	26	42	58	82	98	114	130	266	282	298	314	322	338	354	370
11	11	27	43	59	83	99	115	131	267	283	299	315	323	339	355	371
12	12	28	44	60	84	100	116	132	268	284	300	316	324	340	356	372
13	13	29	45	61	85	101	117	133	269	285	301	317	325	341	357	373
14	14	30	46	62	86	102	118	134	270	286	302	318	326	342	358	374
15	15	31	47	63	87	103	119	135	271	287	303	319	327	343	359	375
16	128	144	160	176	200	216	232	248	384	400	416	432	456	472	488	504
17	129	145	161	177	201	217	233	249	385	401	417	433	457	473	489	505
18	130	146	162	178	202	218	234	250	386	402	418	434	458	474	490	506
19	131	147	163	179	203	219	235	251	387	403	419	435	459	475	491	507
20	132	148	164	180	204	220	236	252	388	404	420	436	460	476	492	508
21	133	149	165	181	205	221	237	253	389	405	421	437	461	477	493	509
22	134	150	166	182	206	222	238	254	390	406	422	438	462	478	494	510
23	135	151	167	183	207	223	239	255	391	407	423	439	463	479	495	511
24	136	152	168	184	192	208	224	240	392	408	424	440	448	464	480	496
25	137	153	169	185	193	209	225	241	393	409	425	441	449	465	481	497
26	138	154	170	186	194	210	226	242	394	410	426	442	450	466	482	498
27	139	155	171	187	195	211	227	243	395	411	427	443	451	467	483	499
28	140	156	172	188	196	212	228	244	396	412	428	444	452	468	484	500
29	141	157	173	189	197	213	229	245	397	413	429	445	453	469	485	501
30	142	158	174	190	198	214	230	246	398	414	430	446	454	470	486	502
31	143	159	175	191	199	215	231	247	399	415	431	447	455	471	487	503

Figure 169: MN major, 32B swizzling and bf16 type

Parameters	Value
T	8
m	2
k	2
LBO	128*sizeof(bf16)
SBO	256*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 16
Encoding of SBO in descriptor	(SBO) >> 4 = 32

- MN-Major, 64B swizzling and bf16 type: [Figure 170](#)

the strides and related details are as follows:

Exact layout : Swizzle<2,4,3> o ((8,4,2),(8,2)):((1,8,256),(32,512))

Canonical Layout :Swizzle<2,4,3> o ((T,4,m),(8,k)):((1,T,LBO),(4T,SBO))

Parameters	Value
T	8
m	2
k	2
LBO	256*sizeof(bf16)
SBO	512*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 32
Encoding of SBO in descriptor	(SBO) >> 4 = 64

9.7.15.5.1.11 Matrix Descriptor Format

Matrix descriptor specifies the properties of the matrix in shared memory that is a multiplicand in the matrix multiply and accumulate operation. It is a 64-bit value contained in a register with the following layout:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	9	32	72	104	144	176	216	248	312	344	384	436	488	528	576	608
1	1	33	73	105	145	177	217	249	313	345	385	437	489	529	577	609
2	2	34	74	106	146	178	218	250	314	346	386	438	490	530	578	610
3	3	35	75	107	147	179	219	251	315	347	387	439	491	531	579	611
4	4	36	76	108	148	180	220	252	316	348	388	440	492	532	580	612
5	5	37	77	109	149	181	221	253	317	349	389	441	493	533	581	613
6	6	38	78	110	150	182	222	254	318	350	390	442	494	534	582	614
7	7	39	79	111	151	183	223	255	319	351	391	443	495	535	583	615
8	8	40	80	112	152	184	224	256	320	352	392	444	496	536	584	616
9	9	41	81	113	153	185	225	257	321	353	393	445	497	537	585	617
10	10	42	82	114	154	186	226	258	322	354	394	446	498	538	586	618
11	11	43	83	115	155	187	227	259	323	355	395	447	499	539	587	619
12	12	44	84	116	156	188	228	260	324	356	396	448	500	540	588	620
13	13	45	85	117	157	189	229	261	325	357	397	449	501	541	589	621
14	14	46	86	118	158	190	230	262	326	358	398	450	502	542	590	622
15	15	47	87	119	159	191	231	263	327	359	399	451	503	543	591	623
16	16	48	88	120	160	192	232	264	328	360	400	452	504	544	592	624
17	17	49	89	121	161	193	233	265	329	361	401	453	505	545	593	625
18	18	50	90	122	162	194	234	266	330	362	402	454	506	546	594	626
19	19	51	91	123	163	195	235	267	331	363	403	455	507	547	595	627
20	20	52	92	124	164	196	236	268	332	364	404	456	508	548	596	628
21	21	53	93	125	165	197	237	269	333	365	405	457	509	549	597	629
22	22	54	94	126	166	198	238	270	334	366	406	458	510	550	598	630
23	23	55	95	127	167	199	239	271	335	367	407	459	511	551	599	631
24	24	56	96	128	168	200	240	272	336	368	408	460	512	552	600	632
25	25	57	97	129	169	201	241	273	337	369	409	461	513	553	601	633
26	26	58	98	130	170	202	242	274	338	370	410	462	514	554	602	634
27	27	59	99	131	171	203	243	275	339	371	411	463	515	555	603	635
28	28	60	100	132	172	204	244	276	340	372	412	464	516	556	604	636
29	29	61	101	133	173	205	245	277	341	373	413	465	517	557	605	637
30	30	62	102	134	174	206	246	278	342	374	414	466	518	558	606	638
31	31	63	103	135	175	207	247	279	343	375	415	467	519	559	607	639
32	32	64	104	136	176	208	248	280	344	376	416	468	520	560	608	640
33	33	65	105	137	177	209	249	281	345	377	417	469	521	561	609	641
34	34	66	106	138	178	210	250	282	346	378	418	470	522	562	610	642
35	35	67	107	139	179	211	251	283	347	379	419	471	523	563	611	643
36	36	68	108	140	180	212	252	284	348	380	420	472	524	564	612	644
37	37	69	109	141	181	213	253	285	349	381	421	473	525	565	613	645
38	38	70	110	142	182	214	254	286	350	382	422	474	526	566	614	646
39	39	71	111	143	183	215	255	287	351	383	423	475	527	567	615	647
40	40	72	112	144	184	216	256	288	352	384	424	476	528	568	616	648
41	41	73	113	145	185	217	257	289	353	385	425	477	529	569	617	649
42	42	74	114	146	186	218	258	290	354	386	426	478	530	570	618	650
43	43	75	115	147	187	219	259	291	355	387	427	479	531	571	619	651
44	44	76	116	148	188	220	260	292	356	388	428	480	532	572	620	652
45	45	77	117	149	189	221	261	293	357	389	429	481	533	573	621	653
46	46	78	118	150	190	222	262	294	358	390	430	482	534	574	622	654
47	47	79	119	151	191	223	263	295	359	391	431	483	535	575	623	655
48	48	80	120	152	192	224	264	296	360	392	432	484	536	576	624	656
49	49	81	121	153	193	225	265	297	361	393	433	485	537	577	625	657
50	50	82	122	154	194	226	266	298	362	394	434	486	538	578	626	658
51	51	83	123	155	195	227	267	299	363	395	435	487	539	579	627	659
52	52	84	124	156	196	228	268	300	364	396	436	488	540	580	628	660
53	53	85	125	157	197	229	269	301	365	397	437	489	541	581	629	661
54	54	86	126	158	198	230	270	302	366	398	438	490	542	582	630	662
55	55	87	127	159	199	231	271	303	367	399	439	491	543	583	631	663
56	56	88	128	160	200	232	272	304	368	400	440	492	544	584	632	664
57	57	89	129	161	201	233	273	305	369	401	441	493	545	585	633	665
58	58	90	130	162	202	234	274	306	370	402	442	494	546	586	634	666
59	59	91	131	163	203	235	275	307	371	403	443	495	547	587	635	667
60	60	92	132	164	204	236	276	308	372	404	444	496	548	588	636	668
61	61	93	133	165	205	237	277	309	373	405	445	497	549	589	637	669
62	62	94	134	166	206	238	278	310	374	406	446	498	550	590	638	670
63	63	95	135	167	207	239	279	311	375	407	447	499	551	591	639	671

Figure 170: MN major, 64B swizzling and bf16 type

Bit-field	Size in bits	Description
13–0	14	matrix-descriptor-encode(Matrix start address)
29–16	14	matrix-descriptor-encode (<i>Leading dimension byte offset</i>)
45–32	14	matrix-descriptor-encode (<i>Stride dimension byte offset</i>)
51–49	3	Matrix base offset. This is valid for all swizzling modes except the no-swizzle mode.
63–62	2	Specifies the swizzling mode to be used: ▶ 0: No swizzle ▶ 1: 128-Byte swizzle ▶ 2: 64-Byte swizzle ▶ 3: 32-Byte swizzle

where

```
matrix-descriptor-encode(x) = (x & 0x3FFFF) >> 4
```

The value of base offset is 0 when the repeating pattern of the specified swizzling mode starts as per the below table:

Swizzling mode	Starting address of the repeating pattern
128-Byte swizzle	1024-Byte boundary
64-Byte swizzle	512-Byte boundary
32-Byte swizzle	256-Byte boundary

Otherwise, the base offset must be a non-zero value, computed using the following formula:

```
base offset = (pattern start addr >> 0x7) & 0x7
```

9.7.15.5.2 Asynchronous Multiply-and-Accumulate Instruction: `wgmma.mma_async`

`wgmma.mma_async`

Perform matrix multiply-and-accumulate operation across warpgroup

Syntax

Half precision floating point type:

```
wgmma.mma_async.sync.aligned.shape.dtype.f16.f16 d, a-desc, b-desc, scale-d, imm-
↳scale-a, imm-scale-b, imm-trans-a, imm-trans-b;

wgmma.mma_async.sync.aligned.shape.dtype.f16.f16 d, a, b-desc, scale-d, imm-scale-a,
↳imm-scale-b, imm-trans-b;

.shape = { .m64n8k16, .m64n16k16, .m64n24k16, .m64n32k16,
           .m64n40k16, .m64n48k16, .m64n56k16, .m64n64k16,
           .m64n72k16, .m64n80k16, .m64n88k16, .m64n96k16,
           .m64n104k16, .m64n112k16, .m64n120k16, .m64n128k16,
           .m64n136k16, .m64n144k16, .m64n152k16, .m64n160k16,
           .m64n168k16, .m64n176k16, .m64n184k16, .m64n192k16,
           .m64n200k16, .m64n208k16, .m64n216k16, .m64n224k16,
           .m64n232k16, .m64n240k16, .m64n248k16, .m64n256k16 };
.dtype = { .f16, .f32 };
```

Alternate floating point type :

```
.bf16 floating point type:

wgmma.mma_async.sync.aligned.shape.dtype.bf16.bf16 d, a-desc, b-desc, scale-d, imm-
↳scale-a, imm-scale-b, imm-trans-a, imm-trans-b;

wgmma.mma_async.sync.aligned.shape.dtype.bf16.bf16 d, a, b-desc, scale-d, imm-scale-
↳a, imm-scale-b, imm-trans-b;

.shape = { .m64n8k16, .m64n16k16, .m64n24k16, .m64n32k16,
           .m64n40k16, .m64n48k16, .m64n56k16, .m64n64k16,
           .m64n72k16, .m64n80k16, .m64n88k16, .m64n96k16,
           .m64n104k16, .m64n112k16, .m64n120k16, .m64n128k16,
           .m64n136k16, .m64n144k16, .m64n152k16, .m64n160k16,
           .m64n168k16, .m64n176k16, .m64n184k16, .m64n192k16,
           .m64n200k16, .m64n208k16, .m64n216k16, .m64n224k16,
           .m64n232k16, .m64n240k16, .m64n248k16, .m64n256k16 };
.dtype = { .f32 };

.tf32 floating point type:

wgmma.mma_async.sync.aligned.shape.dtype.tf32.tf32 d, a-desc, b-desc, scale-d, imm-
↳scale-a, imm-scale-b;

wgmma.mma_async.sync.aligned.shape.dtype.tf32.tf32 d, a, b-desc, scale-d, imm-scale-
↳a, imm-scale-b;

.shape = { .m64n8k8, .m64n16k8, .m64n24k8, .m64n32k8,
           .m64n40k8, .m64n48k8, .m64n56k8, .m64n64k8,
```

(continues on next page)

(continued from previous page)

```

        .m64n72k8, .m64n80k8, .m64n88k8, .m64n96k8,
        .m64n104k8, .m64n112k8, .m64n120k8, .m64n128k8,
        .m64n136k8, .m64n144k8, .m64n152k8, .m64n160k8,
        .m64n168k8, .m64n176k8, .m64n184k8, .m64n192k8,
        .m64n200k8, .m64n208k8, .m64n216k8, .m64n224k8,
        .m64n232k8, .m64n240k8, .m64n248k8, .m64n256k8};
.dtype = {.f32};

FP8 floating point type

wgmma.mma_async.sync.aligned.shape.dtype.atype.btype d, a-desc, b-desc, scale-d, imm-
↪scale-a, imm-scale-b;

wgmma.mma_async.sync.aligned.shape.dtype.atype.btype d, a, b-desc, scale-d, imm-
↪scale-a, imm-scale-b;

.shape = {.m64n8k32, .m64n16k32, .m64n24k32, .m64n32k32,
        .m64n40k32, .m64n48k32, .m64n56k32, .m64n64k32,
        .m64n72k32, .m64n80k32, .m64n88k32, .m64n96k32,
        .m64n104k32, .m64n112k32, .m64n120k32, .m64n128k32,
        .m64n136k32, .m64n144k32, .m64n152k32, .m64n160k32,
        .m64n168k32, .m64n176k32, .m64n184k32, .m64n192k32,
        .m64n200k32, .m64n208k32, .m64n216k32, .m64n224k32,
        .m64n232k32, .m64n240k32, .m64n248k32, .m64n256k32};
.atype = {.e4m3, .e5m2};
.btype = {.e4m3, .e5m2};
.dtype = {.f16, .f32};

```

Integer type:

```

wgmma.mma_async.sync.aligned.shape{.satfinite}.s32.atype.btype d, a-desc, b-desc,
↪scale-d;

wgmma.mma_async.sync.aligned.shape{.satfinite}.s32.atype.btype d, a, b-desc, scale-d;

.shape = {.m64n8k32, .m64n16k32, .m64n24k32, .m64n32k32,
        .m64n48k32, .m64n64k32, .m64n80k32, .m64n96k32,
        .m64n112k32, .m64n128k32, .m64n144k32, .m64n160k32,
        .m64n176k32, .m64n192k32, .m64n208k32, .m64n224k32};
.atype = {.s8, .u8};
.btype = {.s8, .u8};

```

Single bit:

```

wgmma.mma_async.sync.aligned.shape.s32.b1.b1.op.popc d, a-desc, b-desc, scale-d;

wgmma.mma_async.sync.aligned.shape.s32.b1.b1.op.popc d, a, b-desc, scale-d;

.shape = {.m64n8k256, .m64n16k256, .m64n24k256, .m64n32k256,
        .m64n48k256, .m64n64k256, .m64n80k256, .m64n96k256,
        .m64n112k256, .m64n128k256, .m64n144k256, .m64n160k256,
        .m64n176k256, .m64n192k256, .m64n208k256, .m64n224k256,
        .m64n240k256, .m64n256k256};
.op = {.and};

```


Description

Instruction `wgmma.mma_async` issues a $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$, where the A matrix is $M \times K$, the B matrix is $K \times N$, and the D matrix is $M \times N$.

The operation of the form $D = A * B$ is issued when the input predicate argument `scale-d` is false.

`wgmma.fence` instruction must be used to fence the register accesses of `wgmma.mma_async` instruction from their prior accesses. Otherwise, the behavior is undefined.

`wgmma.commit_group` and `wgmma.wait_group` operations must be used to wait for the completion of the asynchronous matrix multiply and accumulate operations before the results are accessed.

Register operand `d` represents the accumulator matrix as well as the destination matrix, distributed across the participating threads. Register operand `a` represents the multiplicand matrix A in register distributed across the participating threads. The 64-bit register operands `a-desc` and `b-desc` are the matrix descriptors which represent the multiplicand matrices A and B in shared memory respectively. The contents of a matrix descriptor must be same across all the warps in the warpgroup. The format of the matrix descriptor is described in [Matrix Descriptor Format](#).

Matrices A and B are stored in row-major and column-major format respectively. For certain floating point variants, the input matrices A and B can be transposed by specifying the value 1 for the immediate integer arguments `imm-trans-a` and `imm-trans-b` respectively. A value of 0 can be used to avoid the transpose operation. The valid values of `imm-trans-a` and `imm-trans-b` are 0 and 1. The transpose operation is only supported for the `wgmma.mma_async` variants with `.f16/` `.bf16` types on matrices accessed from shared memory using matrix descriptors.

For the floating point variants of the `wgmma.mma_async` operation, each element of the input matrices A and B can be negated by specifying the value -1 for operands `imm-scale-a` and `imm-scale-b` respectively. A value of 1 can be used to avoid the negate operation. The valid values of `imm-scale-a` and `imm-scale-b` are -1 and 1.

The qualifiers `.dtype`, `.atype` and `.btype` indicate the data type of the elements in matrices D, A and B respectively. `.atype` and `.btype` must be the same for all floating point `wgmma.mma_async` variants except for the FP8 floating point variants. The sizes of individual data elements of matrices A and B in alternate floating point variants of the `wgmma.mma_async` operation are as follows:

- ▶ Matrices A and B have 8-bit data elements when `.atype/` `.btype` is `.e4m3/` `.e5m2`.
- ▶ Matrices A and B have 16-bit data elements when `.atype/` `.btype` is `.bf16`.
- ▶ Matrices A and B have 32-bit data elements when `.atype/` `.btype` is `.tf32`.

Precision and rounding:

- ▶ Floating point operations:

Element-wise multiplication of matrix A and B is performed with at least single precision. When `.dtype` is `.f32`, accumulation of the intermediate values is performed with at least single precision. When `.dtype` is `.f16`, the accumulation is performed with at least half precision.

The accumulation order, rounding and handling of subnormal inputs are unspecified.

- ▶ `.bf16` and `.tf32` floating point operations:

Element-wise multiplication of matrix A and B is performed with specified precision. `wgmma.mma_async` operation involving type `.tf32` will truncate lower 13 bits of the 32-bit input data before multiplication is issued. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding, and handling of subnormal inputs are unspecified.

► Integer operations:

The integer `wmma.mma_async` operation is performed with `.s32` accumulators. The `.satfinite` qualifier indicates that on overflow, the accumulated value is limited to the range `MIN_INT32..MAX_INT32` (where the bounds are defined as the minimum negative signed 32-bit integer and the maximum positive signed 32-bit integer respectively).

If `.satfinite` is not specified, the accumulated value is wrapped instead.

The mandatory `.sync` qualifier indicates that `wmma.mma_async` instruction causes the executing thread to wait until all threads in the warp execute the same `wmma.mma_async` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warpgroup must execute the same `wmma.mma_async` instruction. In conditionally executed code, a `wmma.mma_async` instruction should only be used if it is known that all threads in the warpgroup evaluate the condition identically, otherwise behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Support for `.u8.s8` and `.s8.u8` as `.atype.btype` introduced in PTX ISA version 8.4.

Target ISA Notes

Requires `sm_90a`.

Examples of half precision floating point type

```
.reg .f16x2 f16a<40>, f16d<40>;
.reg .f32   f32d<40>;
.reg .b64   descA, descB;
.reg .pred  scaled;
wmma.mma_async.sync.aligned.m64n8k16.f32.f16.f16
    {f32d0, f32d1, f32d2, f32d3},
    {f16a0, f16a1, f16a2, f16a3},
    descB,
    1, -1, -1, 1;

wmma.mma_async.sync.aligned.m64n72k16.f16.f16.f16
    {f16d0, f16d1, f16d2, f16d3, f16d4, f16d5, f16d6, f16d7, f16d8,
     f16d9, f16d10, f16d11, f16d12, f16d13, f16d14, f16d15, f16d16, f16d17},
    descA,
    descB,
    scaled, -1, 1, 1, 0;
```

Examples of alternate floating point type

```
.reg .f32    f32d<40>;
.reg .b32    bf16a<40>
.reg .b64    descA, descB;

wmma.mma_async.sync.aligned.m64n120k16.f32.bf16.bf16
{f32d0, f32d1, f32d2, f32d3, f32d4, f32d5, f32d6, f32d7, f32d8, f32d9,
 f32d10, f32d11, f32d12, f32d13, f32d14, f32d15, f32d16, f32d17, f32d18, f32d19,
 f32d20, f32d21, f32d22, f32d23, f32d24, f32d25, f32d26, f32d27, f32d28, f32d29,
 f32d30, f32d31, f32d32, f32d33, f32d34, f32d35, f32d36, f32d37, f32d38, f32d39,
 f32d40, f32d41, f32d42, f32d43, f32d44, f32d45, f32d46, f32d47, f32d48, f32d49,
 f32d50, f32d51, f32d52, f32d53, f32d54, f32d55, f32d56, f32d57, f32d58, f32d59},
{bf16a0, bf16a1, bf16a2, bf16a3},
descB,
scaleD, -1, -1, 0;

.reg .f32    f32d<40>;
.reg .b64    descA, descB;

wmma.mma_async.sync.aligned.m64n16k8.f32.tf32.tf32
{f32d0, f32d1, f32d2, f32d3, f32d4, f32d5, f32d6, f32d7},
descA,
descB,
0, -1, -1;

.reg .b32 f16d<8>, f16a<8>;
.reg .f32 f32d<8>;
.reg .b64    descA, descB;

wmma.mma_async.sync.aligned.m64n8k32.f16.e4m3.e5m2
{f16d0, f16d1},
descA,
descB,
scaleD, -1, 1;

wmma.mma_async.sync.aligned.m64n8k32.f32.e5m2.e4m3
{f32d0, f32d1, f32d2, f32d3},
{f16a0, f16a1, f16a2, f16a3},
descB,
1, -1, -1;
```

Examples of integer type

```
.reg .s32 s32d<8>, s32a<8>;
.reg .u32 u32a<8>;
.reg .pred scaleD;
.reg .b64    descA, descB;

wmma.mma_async.sync.aligned.m64n8k32.s32.s8.s8.satfinite
{s32d0, s32d1, s32d2, s32d3},
{s32a0, s32a1, s32a2, s32a3},
descB,
1;
```

(continues on next page)

(continued from previous page)

```

wgmma.mma_async.sync.aligned.m64n8k32.s32.u8.u8
    {s32d0, s32d1, s32d2, s32d3},
    descA,
    descB,
    scaled;

wgmma.mma_async.sync.aligned.m64n8k32.s32.s8.u8.satfinite
    {s32d0, s32d1, s32d2, s32d3},
    {s32a0, s32a1, s32a2, s32a3},
    descB,
    scaled;

wgmma.mma_async.sync.aligned.m64n8k32.s32.u8.s8
    {s32d0, s32d1, s32d2, s32d3},
    descA,
    descB,
    scaled;

```

Examples of single bit type

```

.reg .s32 s32d<4>;
.reg .b32 b32a<4>;
.reg .pred scaled;
.reg .b64 descA, descB;

wgmma.mma_async.sync.aligned.m64n8k256.s32.b1.b1.and.popc
    {s32d0, s32d1, s32d2, s32d3},
    {b32a0, b32a1, b32a2, b32a3},
    descB,
    scaled;

```

9.7.15.6 Asynchronous Warpgroup Level Multiply-and-Accumulate Operation using wgmma.mma_async.sp instruction

This section describes warp-level `wgmma.mma_async.sp` instruction with sparse matrix A. This variant of the `wgmma.mma_async` operation can be used when A is a structured sparse matrix with 50% zeros in each row distributed in a shape-specific granularity. For an $M \times N \times K$ sparse `wgmma.mma_async.sp` operation, the $M \times K$ matrix A is packed into $M \times K/2$ elements. For each K -wide row of matrix A, 50% elements are zeros and the remaining $K/2$ non-zero elements are packed in the operand representing matrix A. The mapping of these $K/2$ elements to the corresponding K -wide row is provided explicitly as metadata.

9.7.15.6.1 Sparse matrix storage

Granularity of sparse matrix A is defined as the ratio of the number of non-zero elements in a sub-chunk of the matrix row to the total number of elements in that sub-chunk where the size of the sub-chunk is shape-specific. For example, in a 64x32 matrix A used in floating point `wgmma.mma_async` operations, sparsity is expected to be at 2:4 granularity, i.e. each 4-element vector (i.e. a sub-chunk of 4 consecutive elements) of a matrix row contains 2 zeros. Index of each non-zero element in a sub-chunk is stored in the metadata operand. Values `0b0000`, `0b0101`, `0b1010`, `0b1111` are invalid values for metadata and will result in undefined behavior. In a group of four consecutive threads, one or more threads store the metadata for the whole group depending upon the matrix shape. These threads are specified using an additional sparsity selector operand.

Matrix A and its corresponding input operand to the sparse `wgmma` is similar to the diagram shown in [Figure 111](#), with an appropriate matrix size.

Granularities for different matrix shapes and data types are described below.

Sparse `wgmma.mma_async.sp` with half-precision and `.bf16` type

For `.f16` and `.bf16` types, for all supported 64xNx32 shapes, matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A have two zeroes and two non-zero elements. Only the two non-zero elements are stored in matrix A and their positions in the four-wide chunk in Matrix A are indicated by two 2-bits indices in the metadata operand.

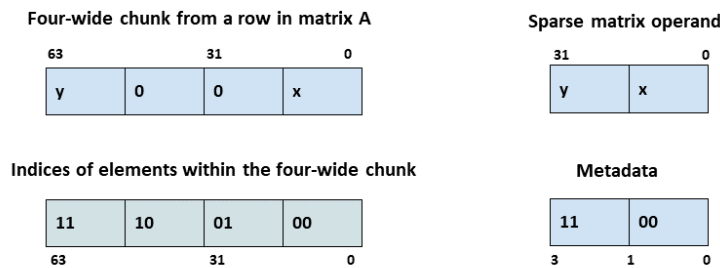


Figure 171: Sparse WGMMMA metadata example for `.f16/.bf16` type.

The sparsity selector indicates a thread-pair within a group of four consecutive threads which contributes the sparsity metadata. Hence, the sparsity selector must be either 0 (threads T0, T1) or 1 (threads T2, T3); any other value results in an undefined behavior.

Sparse wmma.mma_async.sp with .tf32 type

For .tf32 type, for all supported 64xNx16 shapes, matrix A is structured sparse at a granularity of 1:2. In other words, each chunk of two adjacent elements in a row of matrix A have one zero and one non-zero element. Only the non-zero element is stored in operand for matrix A and the 4-bit index in the metadata indicates the position of the non-zero element in the two-wide chunk. 0b1110 and 0b0100 are the only meaningful values of the index, the remaining values result in an undefined behavior.

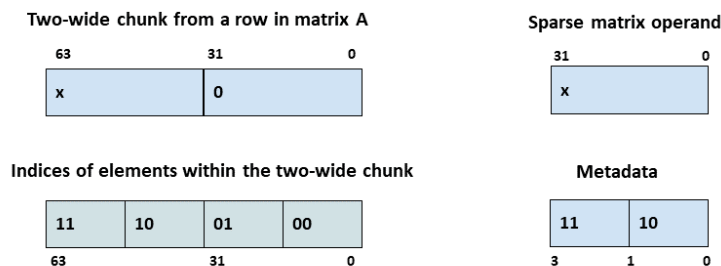


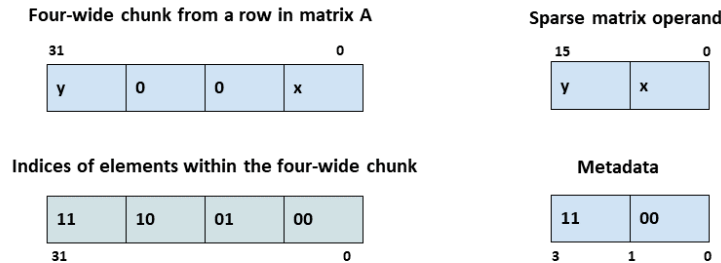
Figure 172: Sparse WGMMMA metadata example for .tf32 type.

The sparsity selector indicates a thread-pair within a group of four consecutive threads which contributes the sparsity metadata. Hence, the sparsity selector must be either 0 (threads T0, T1) or 1 (threads T2, T3); any other value results in an undefined behavior.

Sparse wmma.mma_async.sp with .e4m3 and .e5m2 floating point type

For .e4m3 and .e5m2 types, for all supported 64xNx64 shapes, matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A have two zeroes and two non-zero elements. Only the two non-zero elements are stored in matrix A and their positions in the four-wide chunk in Matrix A are indicated by two 2-bits indices in the metadata operand.

All threads contribute the sparsity metadata and the sparsity selector must be 0; any other value results in an undefined behavior.

Figure 173: Sparse WGMMMA metadata example for `.e4m3/.e5m2` type.

Sparse `wgmma.mma_async.sp` with integer type

For the integer type, for all supported $64 \times N \times 64$ shapes, matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A have two zeroes and two non-zero elements. Only the two non-zero elements are stored in matrix A and two 2-bit indices in the metadata indicate the position of these two non-zero elements in the four-wide chunk.

All threads contribute the sparsity metadata and the sparsity selector must be 0; any other value results in an undefined behavior.

9.7.15.6.2 Matrix fragments for warpgroup-level multiply-accumulate operation with sparse matrix A

In this section we describe how the contents of thread registers are associated with fragments of A matrix and the sparsity metadata.

Each warp in the warpgroup provides sparsity information for 16 rows of matrix A. The following table shows the assignment of warps to rows of matrix A:

Warp	Sparsity information for rows of matrix A
<code>%warpid % 4 = 3</code>	48-63
<code>%warpid % 4 = 2</code>	32-47
<code>%warpid % 4 = 1</code>	16-31
<code>%warpid % 4 = 0</code>	0-15

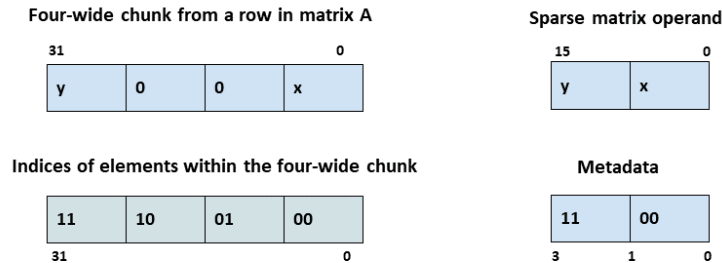


Figure 174: Sparse WGMMMA metadata example for .u8/.s8 type.

The following conventions are used throughout this section:

- For matrix A, only the layout of a fragment is described in terms of register vector sizes and their association with the matrix data.
- For matrix D, since the matrix dimension - data type combination is the same for all supported shapes, and is already covered in *Asynchronous Warpgroup Level Matrix Multiply-Accumulate Operation using wgmma.mma_async instruction*, the pictorial representations of matrix fragments are not included in this section.
- For the metadata operand, pictorial representations of the association between indices of the elements of matrix A and the contents of the metadata operand are included. Tk: [m..n] present in cell [x][y..z] indicates that bits m through n (with m being higher) in the metadata operand of thread with %laneid=k contains the indices of the non-zero elements from the chunk [x][y]..[x][z] of matrix A.

9.7.15.6.2.1 Matrix Fragments for sparse wgmma.mma_async.m64nNk32

A warpgroup executing sparse wgmma.mma_async.m64nNk32 will compute an MMA operation of shape .m64nNk32 where N is a valid n dimension as listed in *Matrix Shape*.

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warp-group holds a fragment of the matrix.

- Multiplicand A, from shared memory is documented in *Shared Memory Matrix Layout*.
- Multiplicand A, from registers:

.atype	Fragments	Elements
.f16 / .bf16	A vector expression containing four .b32 registers, with each register containing two non-zero .f16 /.bf16 elements out of 4 consecutive elements from matrix A.	Non-zero elements: a0, a1, a2, a3, a4, a5, a6, a7 Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i>

The layout of the fragments held by different threads is shown in [Figure 175](#).

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	T0:[a0, a1]				T1:[a0, a1]				T2:[a0, a1]				T3:[a0, a1]				T0:[a4, a5]				T1:[a4, a5]				T2:[a4, a5]				T3:[a4, a5]			
1	T4:[a0, a1]				T5:[a0, a1]				T6:[a0, a1]				T7:[a0, a1]				T4:[a4, a5]				T5:[a4, a5]				T6:[a4, a5]				T7:[a4, a5]			
...																																
7	T28:[a0, a1]				T29:[a0, a1]				T30:[a0, a1]				T31:[a0, a1]				T28:[a4, a5]				T29:[a4, a5]				T30:[a4, a5]				T31:[a4, a5]			
8	T0:[a2, a3]				T1:[a2, a3]				T2:[a2, a3]				T3:[a2, a3]				T0:[a6, a7]				T1:[a6, a7]				T2:[a6, a7]				T3:[a6, a7]			
.																																
15	T28:[a2, a3]				T29:[a2, a3]				T30:[a2, a3]				T31:[a2, a3]				T28:[a6, a7]				T29:[a6, a7]				T30:[a6, a7]				T31:[a6, a7]			
16	T32:[a0, a1]				T33:[a0, a1]				T34:[a0, a1]				T35:[a0, a1]				T32:[a4, a5]				T33:[a4, a5]				T34:[a4, a5]				T35:[a4, a5]			
..																																
31	T60:[a2, a3]				T61:[a2, a3]				T62:[a2, a3]				T63:[a2, a3]				T60:[a6, a7]				T61:[a6, a7]				T62:[a6, a7]				T63:[a6, a7]			
32	T64:[a0, a1]				T65:[a0, a1]				T66:[a0, a1]				T67:[a0, a1]				T64:[a4, a5]				T65:[a4, a5]				T66:[a4, a5]				T67:[a4, a5]			
..																																
47	T92:[a2, a3]				T93:[a2, a3]				T94:[a2, a3]				T95:[a2, a3]				T92:[a6, a7]				T93:[a6, a7]				T94:[a6, a7]				T95:[a6, a7]			
48	T96:[a0, a1]				T97:[a0, a1]				T98:[a0, a1]				T99:[a0, a1]				T96:[a4, a5]				T97:[a4, a5]				T98:[a4, a5]				T99:[a4, a5]			
..																																
63	T124:[a2, a3]				T125:[a2, a3]				T126:[a2, a3]				T127:[a2, a3]				T124:[a6, a7]				T125:[a6, a7]				T126:[a6, a7]				T127:[a6, a7]			

(tid % 128) : fragments

Figure 175: Sparse WGMMA .m64nNk32 fragment layout for matrix A with .f16/.bf16 type.

► Accumulator D:

Matrix fragments for accumulator D are the same as in case of *Matrix Fragments for wgmma.mma_async.m64nNk32* for the same .dtype format.

► Multiplicand B:

Shared memory layout for Matrix B is documented in *Shared Memory Matrix Layout*.

► Metadata operand is a .b32 register containing 16 2-bit vectors each storing the index of a non-zero element of a 4-wide chunk of matrix A.

[Figure 176](#) shows the mapping of the metadata bits to the elements of matrix A for a warp. In this figure, variable *i* represents the value of the sparsity selector operand.

Row\Column	0..3	4..7	8..11	12..15	16..19	20..23	24..27	28..31
0	$T_{2i}: [3..0]$	$T_{2i}: [7..4]$	$T_{2i}: [11..8]$	$T_{2i}: [15..12]$	$T_{2i+1}: [3..0]$	$T_{2i+1}: [7..4]$	$T_{2i+1}: [11..8]$	$T_{2i+1}: [15..12]$
1	$T_{2i+4}: [3..0]$	$T_{2i+4}: [7..4]$	$T_{2i+4}: [11..8]$	$T_{2i+4}: [15..12]$	$T_{2i+5}: [3..0]$	$T_{2i+5}: [7..4]$	$T_{2i+5}: [11..8]$	$T_{2i+5}: [15..12]$
2								
...								
7	$T_{2i+28}: [3..0]$	$T_{2i+28}: [7..4]$	$T_{2i+28}: [11..8]$	$T_{2i+28}: [15..12]$	$T_{2i+29}: [3..0]$	$T_{2i+29}: [7..4]$	$T_{2i+29}: [11..8]$	$T_{2i+29}: [15..12]$
8	$T_{2i}: [19..16]$	$T_{2i}: [23..20]$	$T_{2i}: [27..24]$	$T_{2i}: [31..28]$	$T_{2i+1}: [19..16]$	$T_{2i+1}: [23..20]$	$T_{2i+1}: [27..24]$	$T_{2i+1}: [31..28]$
9	$T_{2i+4}: [19..16]$	$T_{2i+4}: [23..20]$	$T_{2i+4}: [27..24]$	$T_{2i+4}: [31..28]$	$T_{2i+5}: [19..16]$	$T_{2i+5}: [23..20]$	$T_{2i+5}: [27..24]$	$T_{2i+5}: [31..28]$
10								
...								
15	$T_{2i+28}: [19..16]$	$T_{2i+28}: [23..20]$	$T_{2i+28}: [27..24]$	$T_{2i+28}: [31..28]$	$T_{2i+29}: [19..16]$	$T_{2i+29}: [23..20]$	$T_{2i+29}: [27..24]$	$T_{2i+29}: [31..28]$

Figure 176: Sparse WGMMA .m64nNk32 metadata layout for .f16/.bf16 type.

9.7.15.6.2.2 Matrix Fragments for sparse wgmma.mma_async.m64nNk16

A warpgroup executing sparse wgmma.mma_async.m64nNk16 will compute an MMA operation of shape .m64nNk16 where N is a valid n dimension as listed in [Matrix Shape](#).

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warpgroup holds a fragment of the matrix.

- Multiplicand A, from shared memory is documented in [Shared Memory Matrix Layout](#).
- Multiplicand A, from registers:

.atype	Fragments	Elements
.tf32	A vector expression containing four .b32 registers, containing four non-zero .tf32 elements out of eight consecutive elements from matrix A.	Non-zero elements: a0, a1, a2, a3 Mapping of the non-zero elements is as described in Sparse matrix storage

The layout of the fragments held by different threads is shown in [Figure 177](#).

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:a0	T1:a0	T2:a0	T3:a0					T0:a2	T1:a2	T2:a2	T3:a2				
1	T4:a0	T5:a0	T6:a0	T7:a0					T4:a2	T5:a2	T6:a2	T7:a2				
...																
7	T28:a0	T29:a0	T30:a0	T31:a0					T28:a2	T29:a2	T30:a2	T31:a2				
8	T0:a1	T1:a1	T2:a1	T3:a1					T0:a3	T1:a3	T2:a3	T3:a3				
.																
15	T28:a1	T29:a1	T30:a1	T31:a1					T28:a3	T29:a3	T30:a3	T31:a3				
16	T32:a0	T33:a0	T34:a0	T35:a0					T32:a2	T33:a2	T34:a2	T35:a2				
..																
31	T60:a1	T61:a1	T62:a1	T63:a1					T60:a3	T61:a3	T62:a3	T63:a3				
32	T64:a0	T65:a0	T66:a0	T67:a0					T64:a2	T65:a2	T66:a2	T67:a2				
..																
47	T92:a1	T93:a1	T94:a1	T95:a1					T92:a3	T93:a3	T94:a3	T95:a3				
48	T96:a0	T97:a0	T98:a0	T99:a0					T96:a2	T97:a2	T98:a2	T99:a2				
..																
63	T124:a1	T125:a1	T126:a1	T127:a1					T124:a3	T125:a3	T126:a3	T127:a3				

(tid % 128) : fragments

Figure 177: Sparse WGMMA .m64nNk16 fragment layout for matrix A with .tf32 type.

► Accumulator D:

Matrix fragments for accumulator D are the same as in case of *Matrix Fragments for wgmma.mma_async.m64nNk8* for the same .dtype format.

► Multiplicand B:

Shared memory layout for Matrix B is documented in *Shared Memory Matrix Layout*.

► Metadata operand is a .b32 register containing eight 4-bit vectors each storing the index of a non-zero element of a 2-wide chunk of matrix A.

Figure 178 shows the mapping of the metadata bits to the elements of matrix A for a warp. In this figure, variable *i* represents the value of the sparsity selector operand.

9.7.15.6.2.3 Matrix Fragments for sparse wgmma.mma_async.m64nNk64

A warpgroup executing sparse wgmma.mma_async.m64nNk64 will compute an MMA operation of shape .m64nNk64 where N is a valid n dimension as listed in *Matrix Shape*.

Elements of the matrix are distributed across the threads in a warpgroup so each thread of the warpgroup holds a fragment of the matrix.

- Multiplicand A, from shared memory is documented in *Matrix Fragments for sparse wgmma.mma_async.m64nNk64*.
- Multiplicand A, from registers:

Row\Column	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	$T_{2i}:[3..0]$		$T_{2i}:[7..4]$		$T_{2i}:[11..8]$		$T_{2i}:[15..12]$		$T_{2i+1}:[3..0]$		$T_{2i+1}:[7..4]$		$T_{2i+1}:[11..8]$		$T_{2i+1}:[15..12]$	
1	$T_{2i+4}:[3..0]$		$T_{2i+4}:[7..4]$		$T_{2i+4}:[11..8]$		$T_{2i+4}:[15..12]$		$T_{2i+5}:[3..0]$		$T_{2i+5}:[7..4]$		$T_{2i+5}:[11..8]$		$T_{2i+5}:[15..12]$	
2																
...																
7	$T_{2i+28}:[3..0]$		$T_{2i+28}:[7..4]$		$T_{2i+28}:[11..8]$		$T_{2i+28}:[15..12]$		$T_{2i+29}:[3..0]$		$T_{2i+29}:[7..4]$		$T_{2i+29}:[11..8]$		$T_{2i+29}:[15..12]$	
8	$T_{2i}:[19..16]$		$T_{2i}:[23..20]$		$T_{2i}:[27..24]$		$T_{2i}:[31..28]$		$T_{2i+1}:[19..16]$		$T_{2i+1}:[23..20]$		$T_{2i+1}:[27..24]$		$T_{2i+1}:[31..28]$	
9	$T_{2i+4}:[19..16]$		$T_{2i+4}:[23..20]$		$T_{2i+4}:[27..24]$		$T_{2i+4}:[31..28]$		$T_{2i+5}:[19..16]$		$T_{2i+5}:[23..20]$		$T_{2i+5}:[27..24]$		$T_{2i+5}:[31..28]$	
10																
...																
15	$T_{2i+28}:[19..16]$		$T_{2i+28}:[23..20]$		$T_{2i+28}:[27..24]$		$T_{2i+28}:[31..28]$		$T_{2i+29}:[19..16]$		$T_{2i+29}:[23..20]$		$T_{2i+29}:[27..24]$		$T_{2i+29}:[31..28]$	

Figure 178: Sparse WGMMA .m64nNk16 metadata layout for .tf32 type.

.atype	Fragments	Elements
.e4m3 / .e5m2	A vector expression containing four .b32 registers, with each register containing four non-zero .e4m3 / .e5m2 elements out of eight consecutive elements from matrix A.	Non-zero elements: a0, a1, a2, ..., a15 Mapping of the non-zero elements is as described in <i>Sparse matrix storage</i>
.s8 / .u8	A vector expression containing four .b32 registers, with each register containing four non-zero .s8 / .u8 elements out of eight consecutive elements from matrix A.	

The layout of the fragments held by different threads is shown in Figure 179.

► Accumulator D:

Matrix fragments for accumulator D are the same as in case of *Matrix Fragments for wgmma.mma_async.m64nNk32* for the same .dtype format.

► Multiplicand B:

Shared memory layout for Matrix B is documented in *Matrix Fragments for sparse wgmma.mma_async.m64nNk64*.

R\C	0	...	7	8	...	15	16	...	23	24	...	31	32	...	39	40	...	47	48	...	55	56	...	63
0	T0:{a0, a1, a2, a3}			T1:{a0, a1, a2, a3}			T2:{a0, a1, a2, a3}			T3:{a0, a1, a2, a3}			T0:{a8, a9, a10, a11}			T1:{a8, a9, a10, a11}			T2:{a8, a9, a10, a11}			T3:{a8, a9, a10, a11}		
1	T4:{a0, a1, a2, a3}			T5:{a0, a1, a2, a3}			T6:{a0, a1, a2, a3}			T7:{a0, a1, a2, a3}			T4:{a8, a9, a10, a11}			T5:{a8, a9, a10, a11}			T6:{a8, a9, a10, a11}			T7:{a8, a9, a10, a11}		
...																								
7	T28:{a0, a1, a2, a3}			T29:{a0, a1, a2, a3}			T30:{a0, a1, a2, a3}			T31:{a0, a1, a2, a3}			T28:{a8, a9, a10, a11}			T29:{a8, a9, a10, a11}			T30:{a8, a9, a10, a11}			T31:{a8, a9, a10, a11}		
8	T0:{a4, a5, a6, a7}			T1:{a4, a5, a6, a7}			T2:{a4, a5, a6, a7}			T3:{a4, a5, a6, a7}			T0:{a12, a13, a14, a15}			T1:{a12, a13, a14, a15}			T2:{a12, a13, a14, a15}			T3:{a12, a13, a14, a15}		
...																								
15	T28:{a4, a5, a6, a7}			T29:{a4, a5, a6, a7}			T30:{a4, a5, a6, a7}			T31:{a4, a5, a6, a7}			T28:{a12, a13, a14, a15}			T29:{a12, a13, a14, a15}			T30:{a12, a13, a14, a15}			T31:{a12, a13, a14, a15}		
16	T32:{a0, a1, a2, a3}			T33:{a0, a1, a2, a3}			T34:{a0, a1, a2, a3}			T35:{a0, a1, a2, a3}			T32:{a8, a9, a10, a11}			T33:{a8, a9, a10, a11}			T34:{a8, a9, a10, a11}			T35:{a8, a9, a10, a11}		
...																								
31	T60:{a4, a5, a6, a7}			T61:{a4, a5, a6, a7}			T62:{a4, a5, a6, a7}			T63:{a4, a5, a6, a7}			T60:{a12, a13, a14, a15}			T61:{a12, a13, a14, a15}			T62:{a12, a13, a14, a15}			T63:{a12, a13, a14, a15}		
32	T64:{a0, a1, a2, a3}			T65:{a0, a1, a2, a3}			T66:{a0, a1, a2, a3}			T67:{a0, a1, a2, a3}			T64:{a8, a9, a10, a11}			T65:{a8, a9, a10, a11}			T66:{a8, a9, a10, a11}			T67:{a8, a9, a10, a11}		
...																								
47	T92:{a4, a5, a6, a7}			T93:{a4, a5, a6, a7}			T94:{a4, a5, a6, a7}			T95:{a4, a5, a6, a7}			T92:{a12, a13, a14, a15}			T93:{a12, a13, a14, a15}			T94:{a12, a13, a14, a15}			T95:{a12, a13, a14, a15}		
48	T96:{a0, a1, a2, a3}			T97:{a0, a1, a2, a3}			T98:{a0, a1, a2, a3}			T99:{a0, a1, a2, a3}			T96:{a8, a9, a10, a11}			T97:{a8, a9, a10, a11}			T98:{a8, a9, a10, a11}			T99:{a8, a9, a10, a11}		
...																								
63	T124:{a4, a5, a6, a7}			T125:{a4, a5, a6, a7}			T126:{a4, a5, a6, a7}			T127:{a4, a5, a6, a7}			T124:{a12, a13, a14, a15}			T125:{a12, a13, a14, a15}			T126:{a12, a13, a14, a15}			T127:{a12, a13, a14, a15}		

(tid % 128) : fragments

Figure 179: Sparse WGMMMA .m64nNk64 fragment layout for matrix A with .e4m3/ .e5m2/ .s8/ .u8 type.

- Metadata operand is a .b32 register containing 16 4-bit vectors each storing the indices of two non-zero elements of a 4-wide chunk of matrix A.

Figure 180 shows the mapping of the metadata bits to the elements of columns 0–31 of matrix A.

Row\Column	0..3	4..7	8..11	12..15	16..19	20..23	24..27	28..31
0	T ₀ : [3..0]	T ₀ : [7..4]	T ₀ : [11..8]	T ₀ : [15..12]	T ₀ : [19..16]	T ₀ : [23..20]	T ₀ : [27..24]	T ₀ : [31..28]
1	T ₄ : [3..0]	T ₄ : [7..4]	T ₄ : [11..8]	T ₄ : [15..12]	T ₄ : [19..16]	T ₄ : [23..20]	T ₄ : [27..24]	T ₄ : [31..28]
2								
...								
7	T ₂₈ : [3..0]	T ₂₈ : [7..4]	T ₂₈ : [11..8]	T ₂₈ : [15..12]	T ₂₈ : [19..16]	T ₂₈ : [23..20]	T ₂₈ : [27..24]	T ₂₈ : [31..28]
8	T ₁ : [3..0]	T ₁ : [7..4]	T ₁ : [11..8]	T ₁ : [15..12]	T ₁ : [19..16]	T ₁ : [23..20]	T ₁ : [27..24]	T ₁ : [31..28]
9	T ₅ : [3..0]	T ₅ : [7..4]	T ₅ : [11..8]	T ₅ : [15..12]	T ₅ : [19..16]	T ₅ : [23..20]	T ₅ : [27..24]	T ₅ : [31..28]
10								
...								
15	T ₂₉ : [3..0]	T ₂₉ : [7..4]	T ₂₉ : [11..8]	T ₂₉ : [15..12]	T ₂₉ : [19..16]	T ₂₉ : [23..20]	T ₂₉ : [27..24]	T ₂₉ : [31..28]

Figure 180: Sparse WGMMA .m64nNk64 metadata layout for .e4m3/ .e5m2/ .s8/ .u8 type for columns 0–31

Figure 181 shows the mapping of the metadata bits to the elements of columns 32–63 of matrix A.

9.7.15.6.3 Asynchronous Multiply-and-Accumulate Instruction: `wgmma.mma_async.sp`

`wgmma.mma_async.sp`

Perform matrix multiply-and-accumulate operation with sparse matrix A across warpgroup

Row\Column	35..32	39..36	43..40	47..44	51..48	55..52	59..56	63..60
0	T ₂ : [3..0]	T ₂ : [7..4]	T ₂ : [11..8]	T ₂ : [15..12]	T ₂ : [19..16]	T ₂ : [23..20]	T ₂ : [27..24]	T ₂ : [31..28]
1	T ₆ : [3..0]	T ₆ : [7..4]	T ₆ : [11..8]	T ₆ : [15..12]	T ₆ : [19..16]	T ₆ : [23..20]	T ₆ : [27..24]	T ₆ : [31..28]
2								
...								
7	T ₃₀ : [3..0]	T ₃₀ : [7..4]	T ₃₀ : [11..8]	T ₃₀ : [15..12]	T ₃₀ : [19..16]	T ₃₀ : [23..20]	T ₃₀ : [27..24]	T ₃₀ : [31..28]
8	T ₃ : [3..0]	T ₃ : [7..4]	T ₃ : [11..8]	T ₃ : [15..12]	T ₃ : [19..16]	T ₃ : [23..20]	T ₃ : [27..24]	T ₃ : [31..28]
9	T ₇ : [3..0]	T ₇ : [7..4]	T ₇ : [11..8]	T ₇ : [15..12]	T ₇ : [19..16]	T ₇ : [23..20]	T ₇ : [27..24]	T ₇ : [31..28]
10								
...								
15	T ₃₁ : [3..0]	T ₃₁ : [7..4]	T ₃₁ : [11..8]	T ₃₁ : [15..12]	T ₃₁ : [19..16]	T ₃₁ : [23..20]	T ₃₁ : [27..24]	T ₃₁ : [31..28]

Figure 181: Sparse WGMMMA .m64nNk64 metadata layout for .e4m3/ .e5m2/ .s8/ .u8 type for columns 32–63

Syntax

Half precision floating point type:

```
wgmma.mma_async.sp.sync.aligned.shape.dtype.f16.f16 d, a-desc, b-desc, sp-meta, sp-
↪ sel, scale-d, imm-scale-a, imm-scale-b, imm-trans-a, imm-trans-b;
```

```
wgmma.mma_async.sp.sync.aligned.shape.dtype.f16.f16 d, a, b-desc, sp-meta, sp-sel,
↪ scale-d, imm-scale-a, imm-scale-b, imm-trans-b;
```

```
.shape = { .m64n8k32, .m64n16k32, .m64n24k32, .m64n32k32,
            .m64n40k32, .m64n48k32, .m64n56k32, .m64n64k32,
            .m64n72k32, .m64n80k32, .m64n88k32, .m64n96k32,
            .m64n104k32, .m64n112k32, .m64n120k32, .m64n128k32,
            .m64n136k32, .m64n144k32, .m64n152k32, .m64n160k32,
            .m64n168k32, .m64n176k32, .m64n184k32, .m64n192k32,
            .m64n200k32, .m64n208k32, .m64n216k32, .m64n224k32,
            .m64n232k32, .m64n240k32, .m64n248k32, .m64n256k32 };
.dtype = { .f16, .f32 };
```

Alternate floating point type :

.bf16 floating point type:

```
wgmma.mma_async.sp.sync.aligned.shape.dtype.bf16.bf16 d, a-desc, b-desc, sp-meta, sp-
↪ sel, scale-d, imm-scale-a, imm-scale-b, imm-trans-a, imm-trans-b;
```

```
wgmma.mma_async.sp.sync.aligned.shape.dtype.bf16.bf16 d, a, b-desc, sp-meta, sp-sel,
↪ scale-d, imm-scale-a, imm-scale-b, imm-trans-b;
```

(continues on next page)

(continued from previous page)

```

.shape    = {.m64n8k32, .m64n16k32, .m64n24k32, .m64n32k32,
             .m64n40k32, .m64n48k32, .m64n56k32, .m64n64k32,
             .m64n72k32, .m64n80k32, .m64n88k32, .m64n96k32,
             .m64n104k32, .m64n112k32, .m64n120k32, .m64n128k32,
             .m64n136k32, .m64n144k32, .m64n152k32, .m64n160k32,
             .m64n168k32, .m64n176k32, .m64n184k32, .m64n192k32,
             .m64n200k32, .m64n208k32, .m64n216k32, .m64n224k32,
             .m64n232k32, .m64n240k32, .m64n248k32, .m64n256k32};
.dtype    = {.f32};

.tf32 floating point type:

wgmma.mma_async.sp.sync.aligned.shape.dtype.tf32.tf32  d, a-desc, b-desc, sp-meta, sp-
↪sel, scale-d, imm-scale-a, imm-scale-b;

wgmma.mma_async.sp.sync.aligned.shape.dtype.tf32.tf32  d, a, b-desc, sp-meta, sp-sel,
↪scale-d, imm-scale-a, imm-scale-b;

.shape    = {.m64n8k16, .m64n16k16, .m64n24k16, .m64n32k16,
             .m64n40k16, .m64n48k16, .m64n56k16, .m64n64k16,
             .m64n72k16, .m64n80k16, .m64n88k16, .m64n96k16,
             .m64n104k16, .m64n112k16, .m64n120k16, .m64n128k16,
             .m64n136k16, .m64n144k16, .m64n152k16, .m64n160k16,
             .m64n168k16, .m64n176k16, .m64n184k16, .m64n192k16,
             .m64n200k16, .m64n208k16, .m64n216k16, .m64n224k16,
             .m64n232k16, .m64n240k16, .m64n248k16, .m64n256k16};
.dtype    = {.f32};

FP8 floating point type

wgmma.mma_async.sp.sync.aligned.shape.dtype.atype.btype  d, a-desc, b-desc, sp-meta,
↪sp-sel, scale-d, imm-scale-a, imm-scale-b;

wgmma.mma_async.sp.sync.aligned.shape.dtype.atype.btype  d, a, b-desc, sp-meta, sp-
↪sel, scale-d, imm-scale-a, imm-scale-b;

.shape    = {.m64n8k64, .m64n16k64, .m64n24k64, .m64n32k64,
             .m64n40k64, .m64n48k64, .m64n56k64, .m64n64k64,
             .m64n72k64, .m64n80k64, .m64n88k64, .m64n96k64,
             .m64n104k64, .m64n112k64, .m64n120k64, .m64n128k64,
             .m64n136k64, .m64n144k64, .m64n152k64, .m64n160k64,
             .m64n168k64, .m64n176k64, .m64n184k64, .m64n192k64,
             .m64n200k64, .m64n208k64, .m64n216k64, .m64n224k64,
             .m64n232k64, .m64n240k64, .m64n248k64, .m64n256k64};
.atype    = {.e4m3, .e5m2};
.btype    = {.e4m3, .e5m2};
.dtype    = {.f16, .f32};

```

Integer type:

```

wgmma.mma_async.sp.sync.aligned.shape{.satfinite}.s32.atype.btype  d, a-desc, b-desc,
↪sp-meta, sp-sel, scale-d;

wgmma.mma_async.sp.sync.aligned.shape{.satfinite}.s32.atype.btype  d, a, b-desc, sp-
↪meta, sp-sel, scale-d;

```

(continues on next page)

(continued from previous page)

```
.shape = { .m64n8k64, .m64n16k64, .m64n24k64, .m64n32k64,
           .m64n48k64, .m64n64k64, .m64n80k64, .m64n96k64,
           .m64n112k64, .m64n128k64, .m64n144k64, .m64n160k64,
           .m64n176k64, .m64n192k64, .m64n208k64, .m64n224k64,
           .m64n240k64, .m64n256k64 };
.atype = { .s8, .u8 };
.btype = { .s8, .u8 };
```

Description

Instruction `wgmma.mma_async` issues a $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$, where the A matrix is $M \times K$, the B matrix is $K \times N$, and the D matrix is $M \times N$.

The matrix A is stored in the packed format $M \times (K/2)$ as described in [Sparse matrix storage](#).

The operation of the form $D = A * B$ is issued when the input predicate argument `scale-d` is false.

`wgmma.fence` instruction must be used to fence the register accesses of `wgmma.mma_async` instruction from their prior accesses. Otherwise, the behavior is undefined.

`wgmma.commit_group` and `wgmma.wait_group` operations must be used to wait for the completion of the asynchronous matrix multiply and accumulate operations before the results are accessed.

Register operand `d` represents the accumulator matrix as well as the destination matrix, distributed across the participating threads. Register operand `a` represents the multiplicand matrix A in register distributed across the participating threads. The 64-bit register operands `a-desc` and `b-desc` are the matrix descriptors which represent the multiplicand matrices A and B in shared memory respectively. The contents of a matrix descriptor must be same across all the warps in the warpgroup. The format of the matrix descriptor is described in [Matrix Descriptor Format](#). Matrix A is structured sparse as described in [Sparse matrix storage](#). Operands `sp-meta` and `sp-se1` represent sparsity metadata and sparsity selector respectively. Operand `sp-meta` is a 32-bit integer and operand `sp-se1` is a 32-bit integer constant with values in the range 0..3.

The valid values of `sp-meta` and `sp-se1` for each shape is specified in [Sparse matrix storage](#) and are summarized here :

Matrix shape	.atype	Valid values of sp-meta	Valid values of sp-se1
.m64nNk16	.tf32	0b11110, 0b0100	0 (threads T0, T1) or 1 (threads T2, T3)
.m64nNk32	.f16/ .bf16	0b00, 0b01, 0b10, 0b11	0 (threads T0, T1) or 1 (threads T2, T3)
.m64nNk64	.e4m3 / .e5m2 / .s8 / .u8	0b00, 0b01, 0b10, 0b11	0 (all threads contribute)

Matrices A and B are stored in row-major and column-major format respectively. For certain floating point variants, the input matrices A and B can be transposed by specifying the value 1 for the immediate integer arguments `imm-trans-a` and `imm-trans-b` respectively. A value of 0 can be used to avoid the transpose operation. The valid values of `imm-trans-a` and `imm-trans-b` are 0 and 1. The transpose operation is only supported for the `wgmma.mma_async` variants with `.f16/ .bf16` types on matrices accessed from shared memory using matrix descriptors.

For the floating point variants of the `wgmma.mma_async` operation, each element of the input matrices A and B can be negated by specifying the value -1 for operands `imm-scale-a` and `imm-scale-b`

respectively. A value of 1 can be used to avoid the negate operation. The valid values of `imm-scale-a` and `imm-scale-b` are -1 and 1.

The qualifiers `.dtype`, `.atype` and `.btype` indicate the data type of the elements in matrices D, A and B respectively. `.atype` and `.btype` must be the same for all floating point `wgmma.mma_async` variants except for the FP8 floating point variants. The sizes of individual data elements of matrices A and B in alternate floating point variants of the `wgmma.mma_async` operation are as follows:

- ▶ Matrices A and B have 8-bit data elements when `.atype/.btype` is `.e4m3/.e5m2`.
- ▶ Matrices A and B have 16-bit data elements when `.atype/.btype` is `.bf16`.
- ▶ Matrices A and B have 32-bit data elements when `.atype/.btype` is `.tf32`.

Precision and rounding:

- ▶ Floating point operations:

Element-wise multiplication of matrix A and B is performed with at least single precision. When `.dtype` is `.f32`, accumulation of the intermediate values is performed with at least single precision. When `.dtype` is `.f16`, the accumulation is performed with at least half precision.

The accumulation order, rounding and handling of subnormal inputs are unspecified.

- ▶ `.bf16` and `.tf32` floating point operations:

Element-wise multiplication of matrix A and B is performed with specified precision. `wgmma.mma_async` operation involving type `.tf32` will truncate lower 13 bits of the 32-bit input data before multiplication is issued. Accumulation of the intermediate values is performed with at least single precision.

The accumulation order, rounding, and handling of subnormal inputs are unspecified.

- ▶ Integer operations:

The integer `wgmma.mma_async` operation is performed with `.s32` accumulators. The `.satfinite` qualifier indicates that on overflow, the accumulated value is limited to the range `MIN_INT32..MAX_INT32` (where the bounds are defined as the minimum negative signed 32-bit integer and the maximum positive signed 32-bit integer respectively).

If `.satfinite` is not specified, the accumulated value is wrapped instead.

The mandatory `.sync` qualifier indicates that `wgmma.mma_async` instruction causes the executing thread to wait until all threads in the warp execute the same `wgmma.mma_async` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warpgroup must execute the same `wgmma.mma_async` instruction. In conditionally executed code, a `wgmma.mma_async` instruction should only be used if it is known that all threads in the warpgroup evaluate the condition identically, otherwise behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.2.

Support for `.u8.s8` and `.s8.u8` as `.atype.btype` introduced in PTX ISA version 8.4.

Target ISA Notes

Requires sm_90a.

Examples of integer type

```
wgmma.fence.sync.aligned;
wgmma.mma_async.sp.sync.aligned.m64n8k64.s32.u8.u8 {s32d0, s32d1, s32d2, s32d3},
descA, descB, spMeta, 0, scaleD;
wgmma.mma_async.sp.sync.aligned.m64n8k64.s32.s8.u8 {s32d0, s32d1, s32d2, s32d3},
descA, descB, spMeta, 0, scaleD;
wgmma.commit_group.sync.aligned;
wgmma.wait_group.sync.aligned 0;
```

9.7.15.7 Asynchronous wgmma Proxy Operations

This section describes warpgroup level `wgmma.fence`, `wgmma.commit_group` and `wgmma.wait_group` instructions.

9.7.15.7.1 Asynchronous Multiply-and-Accumulate Instruction: `wgmma.fence`

`wgmma.fence`

Enforce an ordering of register accesses between `wgmma.mma_async` and other operations.

Syntax

```
wgmma.fence.sync.aligned;
```

Description

`wgmma.fence` instruction establishes an ordering between prior accesses to any warpgroup registers and subsequent accesses to the same registers by a `wgmma.mma_async` instruction. Only the accumulator register and the input registers containing the fragments of matrix A require this ordering.

The `wgmma.fence` instruction must be issued by all warps of the warpgroup at the following locations:

- Before the first `wgmma.mma_async` operation in a warpgroup.
- Between a register access by a thread in the warpgroup and any `wgmma.mma_async` instruction that accesses the same registers, either as accumulator or input register containing fragments of matrix A, except when these are accumulator register accesses across multiple `wgmma.mma_async` instructions of the same shape. In the latter case, an ordering guarantee is provided by default.

Otherwise, the behavior is undefined.

An async proxy fence must be used to establish an ordering between prior writes to shared memory matrices and subsequent reads of the same matrices in a `wgmma.mma_async` instruction.

The mandatory `.sync` qualifier indicates that `wgmma.fence` instruction causes the executing thread to wait until all threads in the warp execute the same `wgmma.fence` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warpgroup must execute the same `wgmma.fence` instruction. In conditionally executed code, an `wgmma.fence` instruction should only be used if it is known that all threads in the warpgroup evaluate the condition identically, otherwise the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90a`.

Examples

```
// Example 1, first use example:
wgmma.fence.sync.aligned;    // Establishes an ordering w.r.t. prior accesses to the
    ↪ registers s32d<0-3>
wgmma.mma_async.sync.aligned.m64n8k32.s32.u8.u8  {s32d0, s32d1, s32d2, s32d3},
                                                    descA, descB, scaled;

wgmma.commit_group.sync.aligned;
wgmma.wait_group.sync.aligned 0;

// Example 2, use-case with the input value updated in between:
wgmma.fence.sync.aligned;
wgmma.mma_async.sync.aligned.m64n8k32.s32.u8.u8  {s32d0, s32d1, s32d2, s32d3},
                                                    descA, descB, scaled;

...
mov.b32 s32d0, new_val;
wgmma.fence.sync.aligned;
wgmma.mma_async.sync.aligned.m64n8k32.s32.u8.u8  {s32d4, s32d5, s32d6, s32d7},
                                                    {s32d0, s32d1, s32d2, s32d3},
                                                    descB, scaled;

wgmma.commit_group.sync.aligned;
wgmma.wait_group.sync.aligned 0;
```

9.7.15.7.2 Asynchronous Multiply-and-Accumulate Instruction: `wgmma.commit_group`

`wgmma.commit_group`

Commits all prior uncommitted `wgmma.mma_async` operations into a *wgmma-group*.

Syntax

```
wgmma.commit_group.sync.aligned;
```

Description

`wgmma.commit_group` instruction creates a new `wgmma`-group per warpgroup and batches all prior `wgmma.mma_async` instructions initiated by the executing warp but not committed to any `wgmma`-group into the new `wgmma`-group. If there are no uncommitted `wgmma.mma_async` instructions then `wgmma.commit_group` results in an empty `wgmma`-group.

An executing thread can wait for the completion of all `wgmma.mma_async` operations in a `wgmma`-group by using `wgmma.wait_group`.

The mandatory `.sync` qualifier indicates that `wgmma.commit_group` instruction causes the executing thread to wait until all threads in the warp execute the same `wgmma.commit_group` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warpgroup must execute the same `wgmma.commit_group` instruction. In conditionally executed code, an `wgmma.commit_group` instruction should only be used if it is known that all threads in the warpgroup evaluate the condition identically, otherwise the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90a`.

Examples

```
wgmma.commit_group.sync.aligned;
```

9.7.15.7.3 Asynchronous Multiply-and-Accumulate Instruction: `wgmma.wait_group`

`wgmma.wait_group`

Signal the completion of a preceding warpgroup operation.

Syntax

```
wgmma.wait_group.sync.aligned N;
```

Description

`wgmma.wait_group` instruction will cause the executing thread to wait until only `N` or fewer of the most recent `wgmma`-groups are pending and all the prior `wgmma`-groups committed by the executing threads are complete. For example, when `N` is 0, the executing thread waits on all the prior `wgmma`-groups to complete. Operand `N` is an integer constant.

Accessing the accumulator register or the input register containing the fragments of matrix `A` of a `wgmma.mma_async` instruction without first performing a `wgmma.wait_group` instruction that waits on a *wgmma-group* including that `wgmma.mma_async` instruction is undefined behavior.

The mandatory `.sync` qualifier indicates that `wgmma.wait_group` instruction causes the executing thread to wait until all threads in the warp execute the same `wgmma.wait_group` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warpgroup must execute the same `wgmma.wait_group` instruction. In conditionally executed code, an `wgmma.wait_group` instruction should only be used if it is known that all threads in the warpgroup evaluate the condition identically, otherwise the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires `sm_90a`.

Examples

```
wgmma.fence.sync.aligned;

wgmma.mma_async.sync.aligned.m64n8k32.s32.u8.u8  {s32d0, s32d1, s32d2, s32d3},
                                                    descA, descB, scaled;
wgmma.commit_group.sync.aligned;

wgmma.mma_async.sync.aligned.m64n8k16.f32.f16.f16 {f32d0, f32d1, f32d2, f32d3},
                                                    {f16a0, f16a1, f16a2, f16a3},
                                                    descB, 1, -1, -1, 1;
wgmma.commit_group.sync.aligned;

wgmma.wait_group.sync.aligned 0;
```

9.7.16. TensorCore 5th Generation Family Instructions

9.7.16.1 Tensor Memory

The 5th generation TensorCore has dedicated on-chip memory that is specialized for use by TensorCore operations. This Tensor Memory is organized as a two-dimensional matrix where the horizontal rows are called lanes and the vertical columns are called columns.

On architecture sm_100a/sm_100f, the 5th generation TensorCore's Tensor Memory has a two-dimensional structure of 512 columns and 128 rows per CTA, with each cell being 32-bits in size.

Restrictions on threads accessing the Tensor Memory via the load and store operations are specified in [Access restrictions](#).

9.7.16.1.1 Tensor Memory Addressing

Tensor Memory addresses are 32-bit wide and specify two components.

1. Lane index
2. Column index

The layout is as follows:

31 16	15 0
Lane index	Column index

Figure 182 shows the view of the Tensor Memory Layout within CTA.

9.7.16.1.2 Tensor Memory Allocation

The Tensor Memory is dynamically allocated. The Tensor Memory must be allocated by a single warp in a CTA using the [Tensor Memory Allocation and Management Instructions](#).

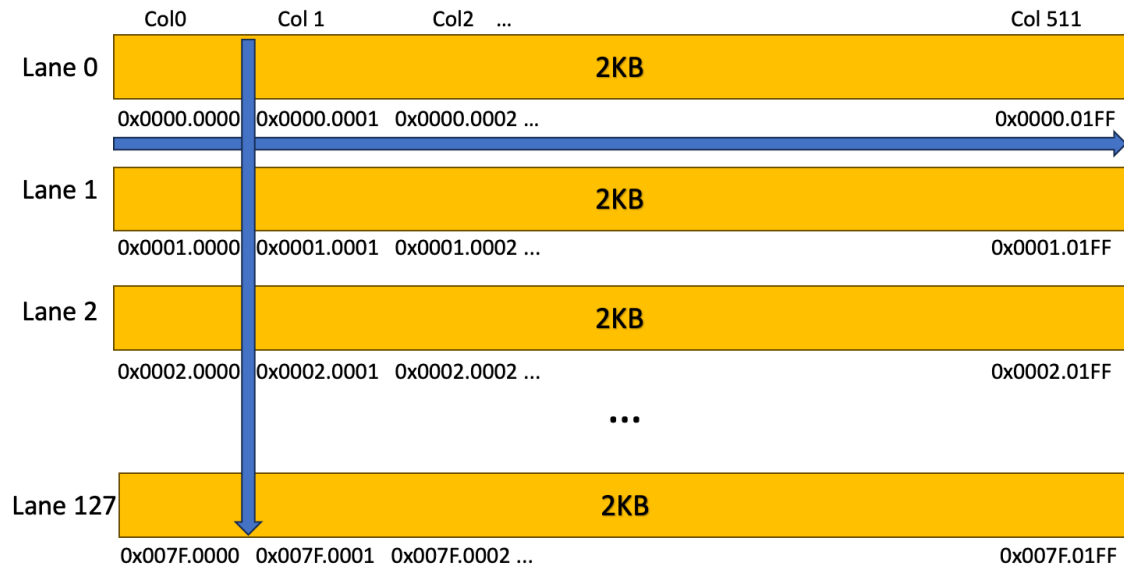
The allocation and deallocation of [Tensor Memory](#) is performed in terms of columns. The unit of allocation is 32 columns and the number of columns being allocated must be a power of 2. When a column is allocated, all 128 lanes of the column are allocated.

All of the Tensor Memory that was allocated in a kernel, must be explicitly deallocated before the kernel exits.

9.7.16.2 Matrix and Data Movement Shape

There are two kinds of shapes involved.

1. Shapes in the data movement operations
2. Shapes in the MMA operations



Tensor Memory Layout and Addressing

Figure 182: Tensor Memory Layout and Addressing

9.7.16.2.1 Matrix Shape

The matrix multiply and accumulate operations support a limited set of shapes for the operand matrices A, B and D. The shapes of all three matrix operands are collectively described by the tuple $M \times N \times K$ where A is $M \times K$ matrix, B is a $K \times N$ matrix, and D is a $M \times N$ matrix.

Table 41 shows matrix shapes that are supported for the specified types for the `tcgen05.mma` operation.

Table 41: Various combinations of .kind and shapes

Various Combinations						Shapes Supported		
.kind::*	Has .ws	CTA Group	Spar-sity	dtype	atype/btype			
kind::f16	No .ws	1	Dense	.f16	.f16	64xNxK 128xNxK	N = {8, 16, 24, ... 256} steps of 8	K = 16
				.f32	.f16, .bf16			
			Sparse	.f16	.f16			K = 32
				.f32	.f16, .bf16			
		2	Dense	.f16	.f16	128xNxK 256xNxK	N = {16, 32, ... 256} steps of 16	K = 16
				.f16	.f16			

continues on next page

Table 41 – continued from previous page

Various Combinations						Shapes Supported			
.kind:: *	Has .ws	CTA Group	Spar- sity	dtype	atype/btype				
				.f32	.f16, .bf16			K = 32	
			Sparse	.f16	.f16				
				.f32	.f16, .bf16				
		.ws	1	Dense	.f16	.f16	32xNxK 64xNxK 128xNxK	N = {64, 128, 256}	K = 16
					.f32	.f16, .bf16			
				Sparse	.f16	.f16		N = {64, 128}	K = 32
	.f32				.f16, .bf16				
	2	Either	.f16	.f16	Invalid				
			.f32	.f16, .bf16					
	.kind:: tf32	No .ws	1	Dense	.f32	.tf32	64xNxK 128xNxK	N = {8, 16, 24, ... 256} steps of 8	K = 8
Sparse				K = 16					
2			Dense	128xNxK 256xNxK			N = {16, 32, ... 256} steps of 16	K = 8	
			Sparse					K = 16	
.ws		1	Dense	32xNxK 64xNxK 128xNxK	N = {64, 128, 256}	K = 8			
			Sparse			K = 16			
		2	Dense		Invalid				
			Sparse						
.kind:: f8f6f4		No .ws	1	Dense	.f32 .f16	.e4m3, .e5m2, .e2m3, .e3m2, .e2m1	64xNxK 128xNxK	N = {8, 16, ... 256} steps of 8	K = 32
				Sparse					K = 64
	2		Dense	128xNxK 256xNxK			N = {16, 32, ... 256} steps of 16	K = 32	

continues on next page

Table 41 – continued from previous page

Various Combinations						Shapes Supported					
.kind::*	Has .ws	CTA Group	Spar-sity	dtype	atype/btype						
		1	Sparse			32xNxK 64xNxK 128xNxK	N = {64, 128, 256}	K = 64			
			Dense					K = 32			
			Sparse				N = {64, 128}	K = 64			
		2	Dense			Invalid					
			Sparse								
.kind::mxf8f6f4	No .ws	1	Dense	.f32	.e4m3, .e5m2, .e2m3, .e3m2, .e2m1 X (Scale) .ue8m0	128xNxK	N = {8, 16, ... 256} steps of 8	K = 32			
			Sparse					K = 64			
		2	Dense			128xNxK 256xNxK		N = {16, 32, ... 256} steps of 16	K = 32		
			Sparse						K = 64		
		256xNxK	Invalid								
	.ws	1				Dense	Invalid				
						Sparse					
		2	Dense								
			Sparse								
	.kind::i8	No .ws	1			Dense	.s32	.s8, .u8	64xNxK 128xNxK	N = {8, 16, 24, 32, 48, ... 256} steps of 16 after N > 32	K = 32
						Sparse					K = 64
2			Dense	128xNxK 256xNxK	N = {32, 64, ... 256} steps of 32	K = 32					
			Sparse			K = 64					
.ws		1	Dense	32xNxK 64xNxK 128xNxK		N = {64, 128, 256}			K = 32		
			Sparse						K = 64		
		2	Dense		Invalid						
			Sparse								

continues on next page

Table 41 – continued from previous page

Various Combinations						Shapes Supported			
.kind::*	Has .ws	CTA Group	Spar-sity	dtype	atype/btype				
.kind::mxf4	No .ws	1	Dense	.f32	.e2m1X (Scale) .ue8m0	128xNxK	N = {8, 16, ... 256} steps of 8	K = 64	
			Sparse					K = 128	
		2	Dense			128xNxK 256xNxK 256xNxK ¹	N = {16, 32, ... 256} steps of 16	K = 64 K ¹ = 96	
			Sparse					256xNxK	K = 128
	.ws	1 / 2	Either			Invalid			
.kind::mxf4nvf4	No .ws	1	Dense	.f32	.e2m1X (Scale) .ue8m0, .ue4m3	128xNxK	N = {8, 16, ... 256} steps of 8	K = 64	
			Sparse					K = 128	
		2	Dense			128xNxK 256xNxK 256xNxK ¹	N = {16, 32, ... 256} steps of 16	K = 64 K ¹ = 96	
			Sparse					256xNxK	K = 128
	.ws	1 / 2	Either			Invalid			

9.7.16.2.1.1 Target ISA Note

- K = 96 is only supported for target architecture sm_103a.

9.7.16.2.2 Specifying Matrix Shape

M and N can be specified in the *Instruction descriptor*.

K cannot be explicitly specified but is implicitly determined by the MMA-kind and the sparsity, as shown in the [Table 41](#).

9.7.16.2.3 Data Movement Shape

The data movement shape indicates the dimension of the data to be moved to or from the *Tensor Memory*. These shapes are described as a tuple `lane x size` where:

- ▶ `lane` indicates the number of rows in the *Tensor Memory*; and
- ▶ `size` indicates the amount of data, in units of bits (b), across the columns in the *Tensor Memory*.

The following shapes are supported by various `tcgen05` operations:

Shape	tcgen05.<op>
.16x64b, .16x128b, .16x256b, .16x32bx2, .32x32b	.ld / .st
.4x256b, .32x128b, .64x128b, .128x256b, .128x128b	.cp
.31x256b (implicit)	.shift

9.7.16.2.3.1 Memory Layout

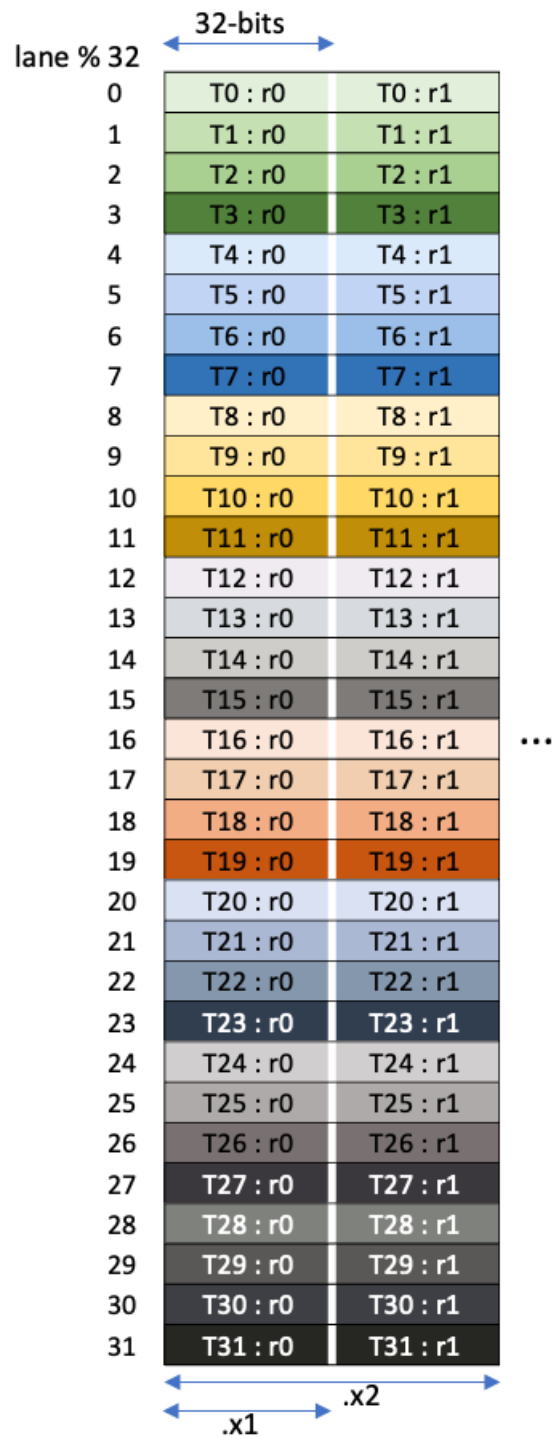
The following shows the layout of the matrix fragments across threads of the warp.

9.7.16.2.3.2 Matrix fragments for shape .32x32b

A `tcgen05{.ld, .st}.32x32b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 51 .	<code>r0, r1, ...</code>

A warp executing `tcgen05{.ld, .st}.32x32b` will access 32 lanes of the Tensor Memory. It loads from or stores to each of the lane ($32 * .num$)-bits of data as shown in [Figure 183](#).

Figure 183: Matrix Fragment for shape `.32x32b`

9.7.16.2.3.3 Matrix fragments for shape .16x64b

A `tcgen05{.ld, .st}.16x64b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 51 .	<code>r0, r1, ...</code>

A warp executing `tcgen05{.ld, .st}.16x64b` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(64 * .num)$ -bits of data as shown in [Figure 184](#).

9.7.16.2.3.4 Matrix fragments for shape .16x128b

A `tcgen05{.ld, .st}.16x128b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 51 .	<code>r0, r1, ...</code>

A warp executing `tcgen05{.ld, .st}.16x128b` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(128 * .num)$ -bits of data as shown in [Figure 185](#).

9.7.16.2.3.5 Matrix fragments for shape .16x256b

A `tcgen05{.ld, .st}.16x256b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 51 .	<code>r0, r1, r2, r3, ...</code>

A warp executing `tcgen05{.ld, .st}.16x256b` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(256 * .num)$ -bits of data as shown in [Figure 186](#).

9.7.16.2.3.6 Matrix fragments for shape .16x32bx2

A `tcgen05{.ld, .st}.16x32bx2` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 51 .	<code>r0, r1, ...</code>

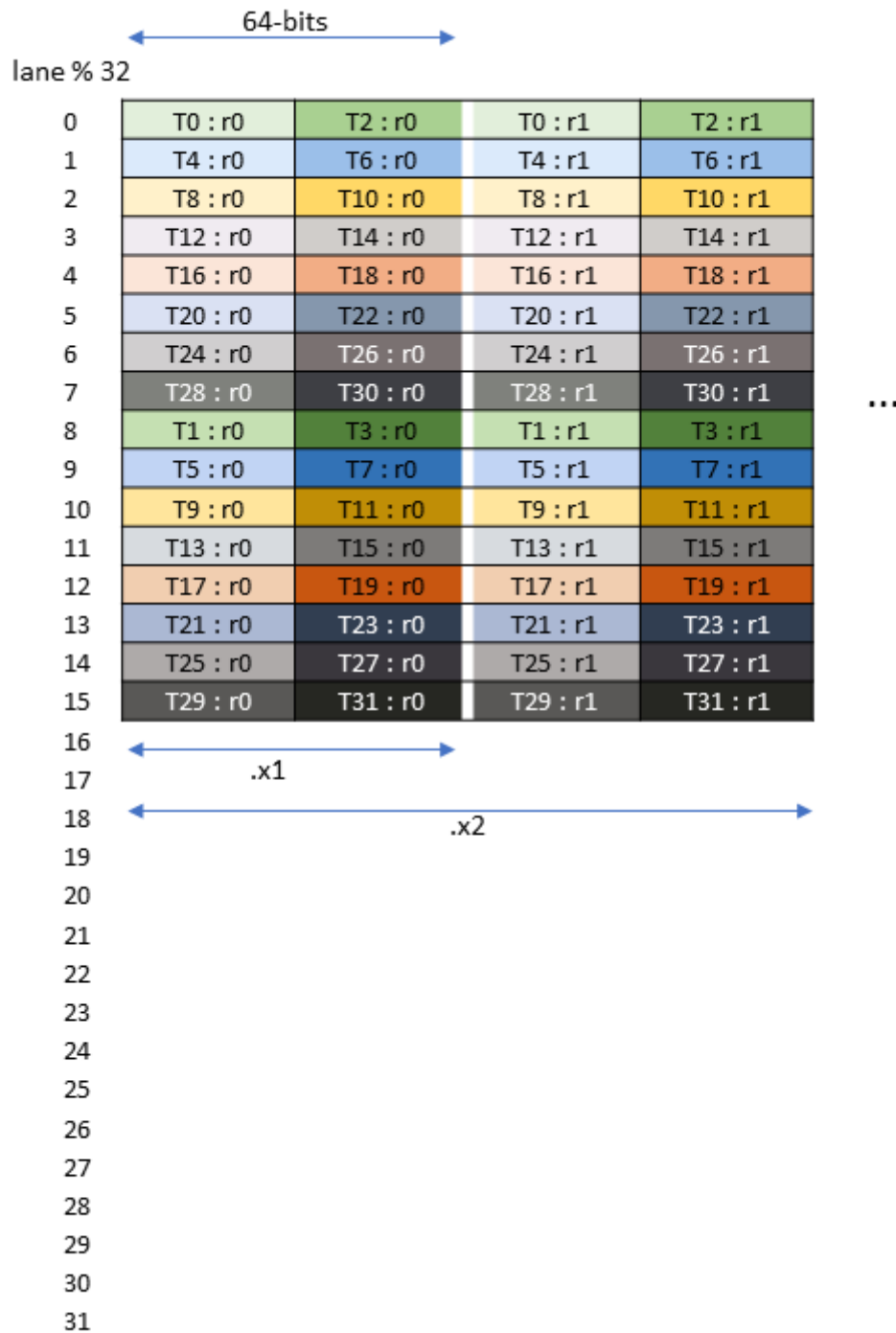


Figure 184: Matrix Fragment for shape .16x64b

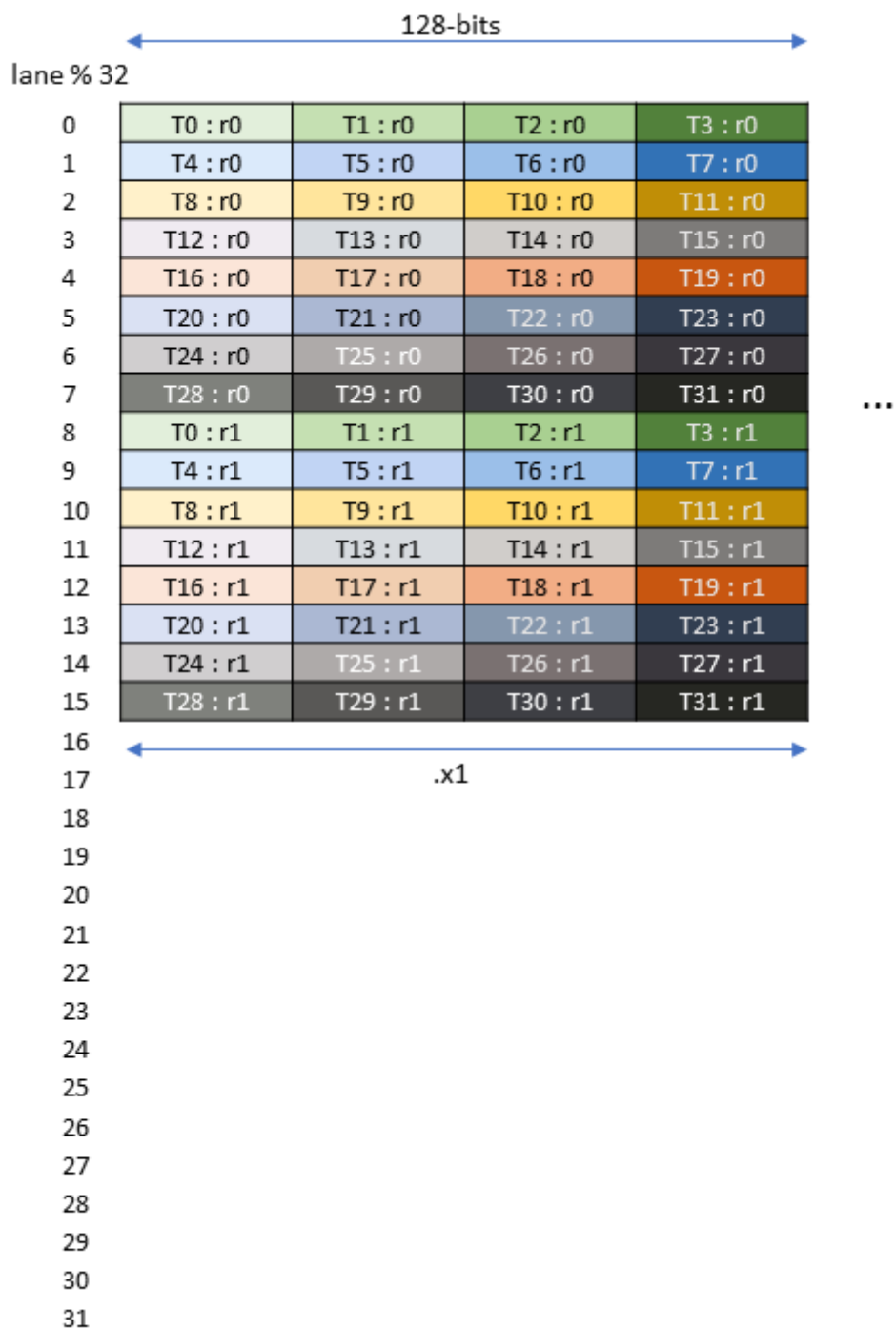
Figure 185: Matrix Fragment for shape `.16x128b`

Figure 186: Matrix Fragment for shape .16x256b

A warp executing `tcgen05{.ld,.st}.16x32bx2` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(32 * .num)$ -bits of data as shown in [Figure 187](#).

9.7.16.3 Major-ness supported by Strides

There are two strides involved while accessing a matrix from shared memory:

1. Leading dimension stride (byte offset or absolute address)
2. Stride dimension byte offset

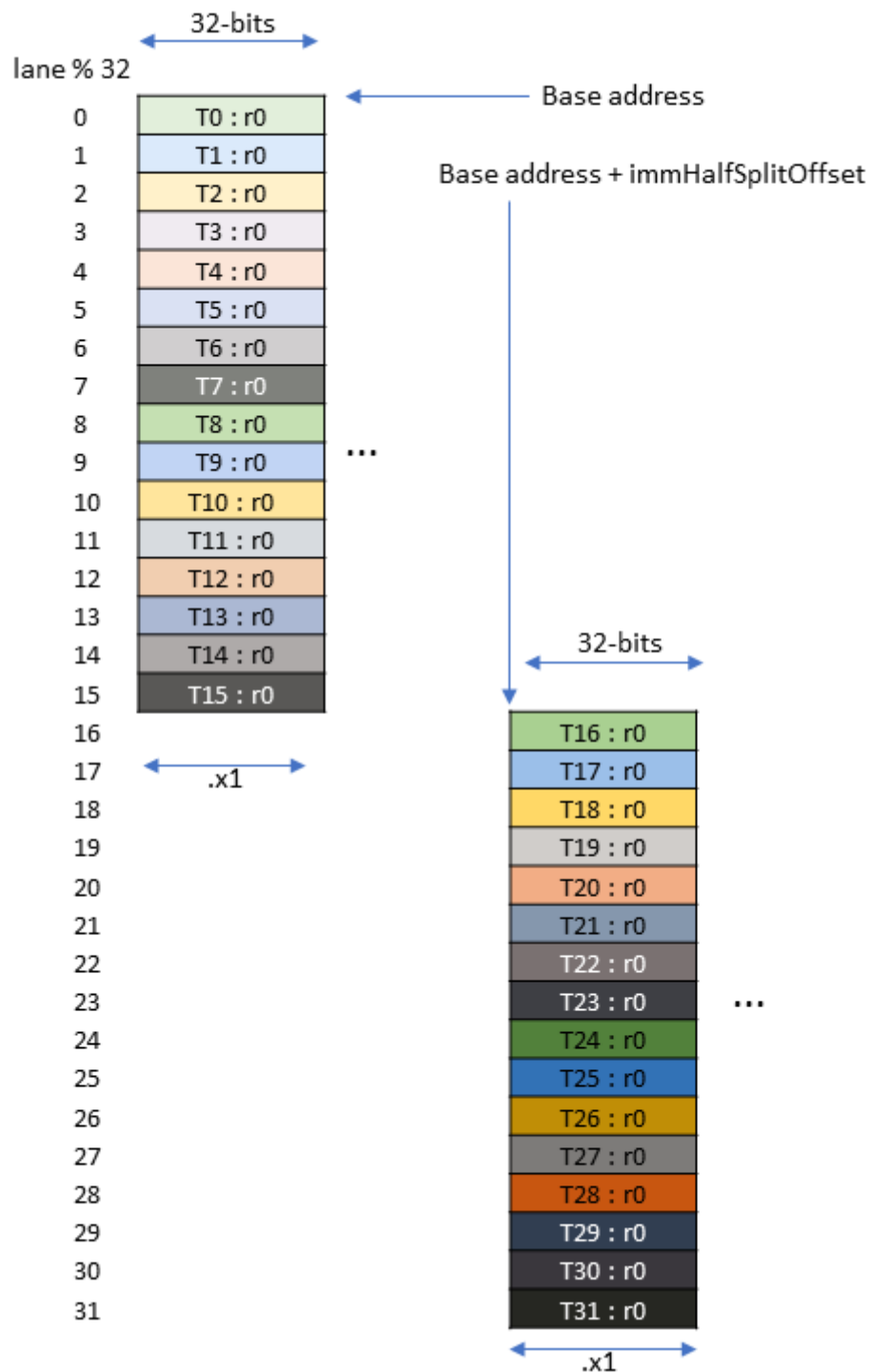
9.7.16.3.1 Leading Dimension Stride: relative offset or absolute address

There are two modes of Leading Dimension Strides as described below. Bit #52 in the *Shared memory descriptor* is used to distinguish between two modes.

9.7.16.3.1.1 Relative offset mode

In this mode, the leading dimension stride is specified as a relative byte offset between the columns as explained in the below table. The leading dimension stride can either be specified as a relative offset between the columns or as an absolute byte address of next buffer. The leading dimension stride is defined differently for transposed and non-transposed matrices. The leading dimension stride is defined as follows for matrices whose element types are normalized to 128-bits:

Major-ness	Definition
K-Major	▶ No-Swizzling: the stride from the first column to the second column of the 8x2 tile in the 128-bit element type normalized matrix. ▶ Swizzled layouts: not used, assumed to be 1.
MN-Major	▶ Interleave: stride from the first 8 columns to the next 8 columns. ▶ Swizzled layouts: stride from the first $(\text{swizzle-byte-size}/16)$ rows to the next $(\text{swizzle-byte-size}/16)$ rows.

Figure 187: Matrix Fragment for shape `.16x32bx2`

9.7.16.3.1.2 Absolute address mode for K dimension being 48B

The `tcgen05.mma` instruction with *K-dimension* of 48B would overflow the 128B shared memory boundary if the data is packed contiguously.

In this case, the absolute address mode can be used to break up the data in the shared memory into two chunks such that both these chunks are laid out within the aligned 128-byte address boundary. The leading dimension absolute address can point to the second data chunk in the shared memory.

9.7.16.3.1.3 Restrictions on the Leading Dimension Absolute Address Stride

Following are the restrictions on the absolute address stride mode:

1. Only 128B swizzle (with 16B atomicity) is supported.
2. Only K-Major mode is supported. That is, the transpose bits(bits #15 and #16) in *Instruction descriptor* must be 0.
3. The matrix base offset must be 0.

9.7.16.3.2 Stride Dimension Byte Offset

The stride dimension byte offset is defined differently for transposed and non-transposed matrices. The stride dimension byte offset is defined as follows for matrices whose element types are normalized to 128-bits:

Major-ness	Definition
K-Major	The offset from the first 8 rows to the next 8 rows.
MN-Major	▶ Interleave: offset from the first row to the next row. ▶ Swizzled layout: offset from the first 8 columns to the next 8 columns

9.7.16.3.3 Canonical Layouts

In terms of *CuTe layouts* the canonical layout can be expressed as follows:

Major-ness	Swizzling mode	Canonical Layout without swizzling	Swizzling on the previous column
MN- major	No-swizzling or Inter-leaved	$((T,1,m),(8,k)):((1,T,SBO),(1T,LBO))$	Swizzle<0, 4, 3>
	32B Swizzling	$((T,2,m),(8,k)):((1,T,LBO),(2T,SBO))$	Swizzle<1, 4, 3>
	64B Swizzling	$((T,4,m),(8,k)):((1,T,LBO),(4T,SBO))$	Swizzle<2, 4, 3>
	128B Swizzling	$((T,8,m),(8,k)):((1,T,LBO),(8T,SBO))$	Swizzle<3, 4, 3>
K- major	No-swizzling or Inter-leaved	$((8,m),(T,2k)):((1T,SBO),(1,LBO))$	Swizzle<0, 4, 3>
	32B Swizzling	$((8,m),(T,2k)):((2T,SBO),(1,T))$	Swizzle<1, 4, 3>
	64B Swizzling	$((8,m),(T,2k)):((4T,SBO),(1,T))$	Swizzle<2, 4, 3>
	128B Swizzling	$((8,m),(T,2k)):((8T,SBO),(1,T))$	Swizzle<3, 4, 3>

where

- $T = 128 / \text{sizeof-elements-in-bits}$ T represents scale factor which normalizes matrix element types to 128-bits.
- m represents the number of repeating patterns across rows.
- k represents the number of repeating patterns across columns.

Examples

- K-Major, no-swizzling and tf32 type: [Figure 188](#)

the strides and related details are as follows:

Exact layout : Swizzle<0,4,3> o $((8,2),(4,4)):((4,32),(1,64))$

Canonical Layout : Swizzle<0,4,3> o $((8,m),(T,2k)):((1T,SBO),(1,LBO))$

Parameters	Value
T	4
m	2
k	2
LBO (relative offset)	$64 * \text{sizeof}(\text{tf32})$
SBO	$32 * \text{sizeof}(\text{tf32})$
Encoding of LBO in descriptor	$(LBO) \gg 4 = 16$
Encoding of SBO in descriptor	$(SBO) \gg 4 = 8$

- K-Major, 32B swizzling and tf32 type: [Figure 189](#)

the strides and related details are as follows:

Exact layout : Swizzle<1,4,3> o $((8,2),(4,4)):((8,64),(1,4))$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	64	65	66	67	128	129	130	131	192	193	194	195
1	4	5	6	7	68	69	70	71	132	133	134	135	196	197	198	199
2	8	9	10	11	72	73	74	75	136	137	138	139	200	201	202	203
3	12	13	14	15	76	77	78	79	140	141	142	143	204	205	206	207
4	16	17	18	19	80	81	82	83	144	145	146	147	208	209	210	211
5	20	21	22	23	84	85	86	87	148	149	150	151	212	213	214	215
6	24	25	26	27	88	89	90	91	152	153	154	155	216	217	218	219
7	28	29	30	31	92	93	94	95	156	157	158	159	220	221	222	223
8	32	33	34	35	96	97	98	99	160	161	162	163	224	225	226	227
9	36	37	38	39	100	101	102	103	164	165	166	167	228	229	230	231
10	40	41	42	43	104	105	106	107	168	169	170	171	232	233	234	235
11	44	45	46	47	108	109	110	111	172	173	174	175	236	237	238	239
12	48	49	50	51	112	113	114	115	176	177	178	179	240	241	242	243
13	52	53	54	55	116	117	118	119	180	181	182	183	244	245	246	247
14	56	57	58	59	120	121	122	123	184	185	186	187	248	249	250	251
15	60	61	62	63	124	125	126	127	188	189	190	191	252	253	254	255

Figure 188: K major, no-swizzling and tf32 type

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	4	5	6	7	128	129	130	131	132	133	134	135
1	8	9	10	11	12	13	14	15	136	137	138	139	140	141	142	143
2	16	17	18	19	20	21	22	23	144	145	146	147	148	149	150	151
3	24	25	26	27	28	29	30	31	152	153	154	155	156	157	158	159
4	36	37	38	39	32	33	34	35	164	165	166	167	160	161	162	163
5	44	45	46	47	40	41	42	43	172	173	174	175	168	169	170	171
6	52	53	54	55	48	49	50	51	180	181	182	183	176	177	178	179
7	60	61	62	63	56	57	58	59	188	189	190	191	184	185	186	187
8	64	65	66	67	68	69	70	71	192	193	194	195	196	197	198	199
9	72	73	74	75	76	77	78	79	200	201	202	203	204	205	206	207
10	80	81	82	83	84	85	86	87	208	209	210	211	212	213	214	215
11	88	89	90	91	92	93	94	95	216	217	218	219	220	221	222	223
12	100	101	102	103	96	97	98	99	228	229	230	231	224	225	226	227
13	108	109	110	111	104	105	106	107	236	237	238	239	232	233	234	235
14	116	117	118	119	112	113	114	115	244	245	246	247	240	241	242	243
15	124	125	126	127	120	121	122	123	252	253	254	255	248	249	250	251

Figure 189: K major, 32B swizzling and tf32 type

Canonical Layout :Swizzle<1,4,3> o ((8,m),(T,2k)):((2T,SBO),(1,T))

Parameters	Value
T	4
m	2
k	2
LBO (relative offset)	NA
SBO	64*sizeof(tf32)
Encoding of LBO in descriptor	1 (assumed)
Encoding of SBO in descriptor	(SBO) >> 4 = 16

- MN-Major, no-swizzling and bf16 type: [Figure 190](#)

the strides and related details are as follows:

Exact layout : Swizzle<0,4,3> o ((8,1,2),(8,2)):((1,8,64),(8,128))

Canonical Layout :Swizzle<0,4,3> o ((T,1,m),(8,k)):((1,T,SBO),(1T,LBO))

Parameters	Value
T	8
m	2
k	2
LBO (relative offset)	128*sizeof(bf16)
SBO	64*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 16
Encoding of SBO in descriptor	(SBO) >> 4 = 8

- MN-Major, 32B swizzling and bf16 type: [Figure 191](#)

the strides and related details are as follows:

Exact layout : Swizzle<1,4,3> o ((8,2,2),(8,2)):((1,8,128),(16,256))

Canonical Layout :Swizzle<1,4,3> o ((T,2,m),(8,k)):((1,T,LBO),(2T,SBO))

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	8	16	24	32	40	48	56	128	136	144	152	160	168	176	184
1	1	9	17	25	33	41	49	57	129	137	145	153	161	169	177	185
2	2	10	18	26	34	42	50	58	130	138	146	154	162	170	178	186
3	3	11	19	27	35	43	51	59	131	139	147	155	163	171	179	187
4	4	12	20	28	36	44	52	60	132	140	148	156	164	172	180	188
5	5	13	21	29	37	45	53	61	133	141	149	157	165	173	181	189
6	6	14	22	30	38	46	54	62	134	142	150	158	166	174	182	190
7	7	15	23	31	39	47	55	63	135	143	151	159	167	175	183	191
8	64	72	80	88	96	104	112	120	192	200	208	216	224	232	240	248
9	65	73	81	89	97	105	113	121	193	201	209	217	225	233	241	249
10	66	74	82	90	98	106	114	122	194	202	210	218	226	234	242	250
11	67	75	83	91	99	107	115	123	195	203	211	219	227	235	243	251
12	68	76	84	92	100	108	116	124	196	204	212	220	228	236	244	252
13	69	77	85	93	101	109	117	125	197	205	213	221	229	237	245	253
14	70	78	86	94	102	110	118	126	198	206	214	222	230	238	246	254
15	71	79	87	95	103	111	119	127	199	207	215	223	231	239	247	255

Figure 190: MN major, no-swizzling and bf16 type

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	16	32	48	72	88	104	120	256	272	288	304	328	344	360	376
1	1	17	33	49	73	89	105	121	257	273	289	305	329	345	361	377
2	2	18	34	50	74	90	106	122	258	274	290	306	330	346	362	378
3	3	19	35	51	75	91	107	123	259	275	291	307	331	347	363	379
4	4	20	36	52	76	92	108	124	260	276	292	308	332	348	364	380
5	5	21	37	53	77	93	109	125	261	277	293	309	333	349	365	381
6	6	22	38	54	78	94	110	126	262	278	294	310	334	350	366	382
7	7	23	39	55	79	95	111	127	263	279	295	311	335	351	367	383
8	8	24	40	56	80	96	112	128	264	280	296	312	320	336	352	368
9	9	25	41	57	81	97	113	129	265	281	297	313	321	337	353	369
10	10	26	42	58	82	98	114	130	266	282	298	314	322	338	354	370
11	11	27	43	59	83	99	115	131	267	283	299	315	323	339	355	371
12	12	28	44	60	84	100	116	132	268	284	300	316	324	340	356	372
13	13	29	45	61	85	101	117	133	269	285	301	317	325	341	357	373
14	14	30	46	62	86	102	118	134	270	286	302	318	326	342	358	374
15	15	31	47	63	87	103	119	135	271	287	303	319	327	343	359	375
16	128	144	160	176	200	216	232	248	384	400	416	432	456	472	488	504
17	129	145	161	177	201	217	233	249	385	401	417	433	457	473	489	505
18	130	146	162	178	202	218	234	250	386	402	418	434	458	474	490	506
19	131	147	163	179	203	219	235	251	387	403	419	435	459	475	491	507
20	132	148	164	180	204	220	236	252	388	404	420	436	460	476	492	508
21	133	149	165	181	205	221	237	253	389	405	421	437	461	477	493	509
22	134	150	166	182	206	222	238	254	390	406	422	438	462	478	494	510
23	135	151	167	183	207	223	239	255	391	407	423	439	463	479	495	511
24	136	152	168	184	192	208	224	240	392	408	424	440	448	464	480	496
25	137	153	169	185	193	209	225	241	393	409	425	441	449	465	481	497
26	138	154	170	186	194	210	226	242	394	410	426	442	450	466	482	498
27	139	155	171	187	195	211	227	243	395	411	427	443	451	467	483	499
28	140	156	172	188	196	212	228	244	396	412	428	444	452	468	484	500
29	141	157	173	189	197	213	229	245	397	413	429	445	453	469	485	501
30	142	158	174	190	198	214	230	246	398	414	430	446	454	470	486	502
31	143	159	175	191	199	215	231	247	399	415	431	447	455	471	487	503

Figure 191: MN major, 32B swizzling and bf16 type

Parameters	Value
T	8
m	2
k	2
LBO (relative offset)	128*sizeof(bf16)
SBO	256*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 16
Encoding of SBO in descriptor	(SBO) >> 4 = 32

- MN-Major, 64B swizzling and bf16 type: [Figure 192](#)

the strides and related details are as follows:

Exact layout : Swizzle<2,4,3> o ((8,4,2),(8,2)):((1,8,256),(32,512))

Canonical Layout :Swizzle<2,4,3> o ((T,4,m),(8,k)):((1,T,LBO),(4T,SBO))

Parameters	Value
T	8
m	2
k	2
LBO (relative offset)	256*sizeof(bf16)
SBO	512*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 32
Encoding of SBO in descriptor	(SBO) >> 4 = 64

9.7.16.4 Matrix Descriptors

There are three kinds of matrix descriptors used by the `tcgen05` family of instructions.

9.7.16.4.1 Shared memory descriptor

The shared memory descriptor describes the properties of multiplicand matrix in shared memory including its location in the shared memory of the current *CTA*. It is a 64-bit value contained in a register with the following layout:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	9	32	72	104	144	176	216	248	312	344	384	436	488	528	568	600
1	1	33	73	105	145	177	217	249	313	345	385	437	489	529	569	601
2	2	34	74	106	146	178	218	250	314	346	386	438	490	530	570	602
3	3	35	75	107	147	179	219	251	315	347	387	439	491	531	571	603
4	4	36	76	108	148	180	220	252	316	348	388	440	492	532	572	604
5	5	37	77	109	149	181	221	253	317	349	389	441	493	533	573	605
6	6	38	78	110	150	182	222	254	318	350	390	442	494	534	574	606
7	7	39	79	111	151	183	223	255	319	351	391	443	495	535	575	607
8	8	40	80	112	152	184	224	256	320	352	392	444	496	536	576	608
9	9	41	81	113	153	185	225	257	321	353	393	445	497	537	577	609
10	10	42	82	114	154	186	226	258	322	354	394	446	498	538	578	610
11	11	43	83	115	155	187	227	259	323	355	395	447	499	539	579	611
12	12	44	84	116	156	188	228	260	324	356	396	448	500	540	580	612
13	13	45	85	117	157	189	229	261	325	357	397	449	501	541	581	613
14	14	46	86	118	158	190	230	262	326	358	398	450	502	542	582	614
15	15	47	87	119	159	191	231	263	327	359	399	451	503	543	583	615
16	16	48	88	120	160	192	232	264	328	360	400	452	504	544	584	616
17	17	49	89	121	161	193	233	265	329	361	401	453	505	545	585	617
18	18	50	90	122	162	194	234	266	330	362	402	454	506	546	586	618
19	19	51	91	123	163	195	235	267	331	363	403	455	507	547	587	619
20	20	52	92	124	164	196	236	268	332	364	404	456	508	548	588	620
21	21	53	93	125	165	197	237	269	333	365	405	457	509	549	589	621
22	22	54	94	126	166	198	238	270	334	366	406	458	510	550	590	622
23	23	55	95	127	167	199	239	271	335	367	407	459	511	551	591	623
24	24	56	96	128	168	200	240	272	336	368	408	460	512	552	592	624
25	25	57	97	129	169	201	241	273	337	369	409	461	513	553	593	625
26	26	58	98	130	170	202	242	274	338	370	410	462	514	554	594	626
27	27	59	99	131	171	203	243	275	339	371	411	463	515	555	595	627
28	28	60	100	132	172	204	244	276	340	372	412	464	516	556	596	628
29	29	61	101	133	173	205	245	277	341	373	413	465	517	557	597	629
30	30	62	102	134	174	206	246	278	342	374	414	466	518	558	598	630
31	31	63	103	135	175	207	247	279	343	375	415	467	519	559	599	631
32	32	64	104	136	176	208	248	280	344	376	416	468	520	560	600	632
33	33	65	105	137	177	209	249	281	345	377	417	469	521	561	601	633
34	34	66	106	138	178	210	250	282	346	378	418	470	522	562	602	634
35	35	67	107	139	179	211	251	283	347	379	419	471	523	563	603	635
36	36	68	108	140	180	212	252	284	348	380	420	472	524	564	604	636
37	37	69	109	141	181	213	253	285	349	381	421	473	525	565	605	637
38	38	70	110	142	182	214	254	286	350	382	422	474	526	566	606	638
39	39	71	111	143	183	215	255	287	351	383	423	475	527	567	607	639
40	40	72	112	144	184	216	256	288	352	384	424	476	528	568	608	640
41	41	73	113	145	185	217	257	289	353	385	425	477	529	569	609	641
42	42	74	114	146	186	218	258	290	354	386	426	478	530	570	610	642
43	43	75	115	147	187	219	259	291	355	387	427	479	531	571	611	643
44	44	76	116	148	188	220	260	292	356	388	428	480	532	572	612	644
45	45	77	117	149	189	221	261	293	357	389	429	481	533	573	613	645
46	46	78	118	150	190	222	262	294	358	390	430	482	534	574	614	646
47	47	79	119	151	191	223	263	295	359	391	431	483	535	575	615	647
48	48	80	120	152	192	224	264	296	360	392	432	484	536	576	616	648
49	49	81	121	153	193	225	265	297	361	393	433	485	537	577	617	649
50	50	82	122	154	194	226	266	298	362	394	434	486	538	578	618	650
51	51	83	123	155	195	227	267	299	363	395	435	487	539	579	619	651
52	52	84	124	156	196	228	268	300	364	396	436	488	540	580	620	652
53	53	85	125	157	197	229	269	301	365	397	437	489	541	581	621	653
54	54	86	126	158	198	230	270	302	366	398	438	490	542	582	622	654
55	55	87	127	159	199	231	271	303	367	399	439	491	543	583	623	655
56	56	88	128	160	200	232	272	304	368	400	440	492	544	584	624	656
57	57	89	129	161	201	233	273	305	369	401	441	493	545	585	625	657
58	58	90	130	162	202	234	274	306	370	402	442	494	546	586	626	658
59	59	91	131	163	203	235	275	307	371	403	443	495	547	587	627	659
60	60	92	132	164	204	236	276	308	372	404	444	496	548	588	628	660
61	61	93	133	165	205	237	277	309	373	405	445	497	549	589	629	661
62	62	94	134	166	206	238	278	310	374	406	446	498	550	590	630	662
63	63	95	135	167	207	239	279	311	375	407	447	499	551	591	631	663

Figure 192: MN major, 64B swizzling and bf16 type

Table 42: Shared memory descriptor layout

Bit-field	Size in bits	Description
0-13	14	matrix-descriptor-encode (Matrix start address)
16-29	14	matrix-descriptor-encode (<i>Leading dimension byte offset relative</i>) OR matrix-descriptor-encode (<i>Leading dimension byte address absolute</i>)
32-45	14	matrix-descriptor-encode (<i>Stride dimension byte offset</i>)
46-48	3	Fixed constant value of 0b001
49-51	3	Matrix base offset
52	1	Leading dimension stride mode: - 0: byte offset relative - 1: byte address absolute
53-60	8	Fixed constant value of 0xb00000000
61-63	3	Specifies the swizzling mode to be used: 0. No swizzling 1. 128-Byte with 32B atomic swizzling 2. 128-Byte swizzling 4. 64-Byte swizzling 6. 32-Byte swizzling Note: Values 3, 5 and 7 are invalid

where $\text{matrix-descriptor-encode}(x) = (x \& 0x3FFFF) \gg 4$

The value of base offset is 0 when the repeating pattern of the specified swizzling mode starts as per shown in Table 43.

Table 43: Starting address of repeating pattern for various swizzling modes

Swizzling mode	Starting address of the repeating pattern
128-Byte swizzle	1024-Byte boundary
64-Byte swizzle	512-Byte boundary
32-Byte swizzle	256-Byte boundary

Otherwise, the base offset must be a non-zero value, computed using the following formula: $\text{base offset} = (\text{pattern start addr} \gg 0x7) \& 0x7$

The following must be 16-byte aligned:

1. Matrix start address
2. Leading dimension byte offset
3. Stride dimension byte offset

9.7.16.4.1.1 Target ISA Note

- The byte address mode for the leading dimension stride is supported on sm_103a.

9.7.16.4.2 Instruction descriptor

The instruction descriptor describes the shapes, types and other details of all the matrices and the matrix-multiplication-and-accumulation operation. It is a 32-bit value in registers and the exact layout is dependent on the MMA-Kind:

Table 44: Instruction descriptor format for .kind::tf32, .kind::f16, .kind::f8f6f4 and .kind::i8

Bits	Size (bits)	Description	Values			
			.kind::tf32	.kind::f16	.kind::f8f6f4	.kind::i8
0-1	2	<i>Sparsity selector</i> , if Sparsity is enabled	0-3			
2	1	Sparsity	Dense = 0 Sparse = 1			
3	1	Saturate for integer types	0 (NA)			No Saturate = 0 Saturate = 1
4-5	2	dtype (Matrix D type)	F32 = 1	F16 = 0 F32 = 1		S32 = 2
6	1	Reserved	0			
7-9	3	atype (Matrix A type)	TF32 = 2	F16 = 0 BF16 = 1	E4M3 = 0 E5M2 = 1 E2M3 = 3 E3M2 = 4 E2M1 = 5	Unsigned 8b = 0 Signed 8b = 1
10-12	3	btype (Matrix B type)				
13	1	Negate A Matrix	No Negate = 0 Negate = 1			No Negate = 0
14	1	Negate B Matrix				
15	1	Transpose A Matrix	No Transpose = 0 Transpose = 1			
16	1	Transpose B Matrix				
17-22	6	N, Dimension of Matrix B (3 LSBs not included)	N >> 3			
23	1	Reserved	0			
24-28	5	M, Dimension of Matrix A (4 LSBs not included)	M >> 4			
29	1	Reserved	0			
30-31	2	Maximum shift while attempting B matrix-reuse in .ws	no shift = 0 maximum shift of 8 = 1 maximum shift of 16 = 2 maximum shift of 32 = 3			

Table 45: Instruction descriptor format for .kind::mxf8f6f4

Bits	Size (bits)	Description	Values
			.kind::mxf8f6f4
0-1	2	Reserved	0
2	1	Sparsity	Dense = 0 Sparse = 1
3	1	Reserved	0
4-5	2	<i>Matrix B Scale Factor Data ID</i>	0-3
6	1	Reserved	0
7-9	3	atype (Matrix A type)	E4M3 = 0 E5M2 = 1 E2M3 = 3 E3M2 = 4 E2M1 = 5
10-12	3	btype (Matrix B type)	
13	1	Negate A Matrix	No Negate = 0 Negate = 1
14	1	Negate B Matrix	
15	1	Transpose A Matrix	No Transpose = 0 Transpose = 1
16	1	Transpose B Matrix	
17-22	6	N, Dimension of Matrix B (3 LSBs not included)	N >> 3
23	1	Scale Matrix Type, for both scale_A / scale_B	UE8M0 = 1
24-26	3	Reserved	0
27-28	2	M, Dimension of Matrix A (7 LSBs not included)	M >> 7
29-30	2	<i>Matrix A Scale Factor Data ID</i>	0-3
31	1	Reserved	0

Table 46: Instruction descriptor format for .kind::mxf4 and .kind::mxf4nrv4

Bits	Size (bits)	Description	Values	
			.kind::mxf4	.kind::mxf4nrv4
0-1	2	Reserved	0	
2	1	Sparsity	Dense = 0 Sparse = 1	
3	1	Reserved	0	
4-5	2	<i>Matrix B Scale Factor Data ID</i>	0 or 2	
6	1	Reserved	0	
7-9	3	atype (Matrix A type)	E2M1 = 1	
10-11	2	btype (Matrix B type)		
12	1	Reserved	0	
13	1	Negate A Matrix	No Negate = 0 Negate = 1	
14	1	Negate B Matrix		
15	1	Transpose A Matrix	No Transpose = 0	
16	1	Transpose B Matrix		
17-22	6	N, Dimension of Matrix B (3 LSBs not included)	N >> 3	
23	1	Scale Matrix Type, for both scale_A / scale_B	UE8M0 = 1	UE4M3 = 0
24-26	3	Reserved	0	
27-28	2	M, Dimension of Matrix A (7 LSBs not included)	M >> 7	
29-30	2	<i>Matrix A Scale Factor Data ID</i>	0 or 2	
31	1	K Dimension	(Dense K=64 / Sparse K=128) = 0 (Dense K=96) = 1	

9.7.16.4.3 Zero-Column Mask Descriptor

The zero-column mask descriptor is used to generate a mask that specifies which columns of B matrix will have zero value for the MMA operation regardless of the values present in the shared memory. The total size of the generated mask is N-bits.

A 0-bit in the mask specifies that values of the corresponding column in matrix B should be used for the MMA operation. A 1-bit in the mask specifies 0s must be used for the entire column for the MMA operation.

The zero-column mask descriptor is a 64-bit value in registers with the following layout:

Table 47: Zero-Column Mask descriptor layout

Bits	Size (bits)	Field Name	Description
0-7	8	Start Count 0 (sc0)	Specifies the LSBs that must be skipped for sub-mask mask-i
8-15	8	Start Count 1 (sc1)	
16-23	8	Start Count 2 (sc2)	
24-31	8	Start Count 3 (sc3)	
32	1	First Span 0 (fs0)	Specifies the starting value for sub-mask mask-i
33	1	First Span 1 (fs1)	
34	1	First Span 2 (fs2)	
35	1	First Span 3 (fs3)	
36-38	3	Reserved	
39	1	Non-Zero Mask	Value 0 indicates generated mask will have all 0s Value 1 indicates the mask has to be generated
40-47	8	Skip Span	(Count of consecutive columns where B matrix is used) - 1
48-55	8	Use Span	(Count of consecutive columns where 0s are used) - 1
56-61	6	Column Shift	Shifts column by specified amount. Thus allows MMA on non-0 starting column. Max shift amount = 16 for M=32 Max shift amount = 32 otherwise

The zero-column mask is made up of one or more sub-mask depending on M, as shown in the table:

M	Zero-Column Mask breakup	Sub-masks	First Span used	Start Column used
128	Single sub-mask of size N-bits	mask0	fs0	sc0
64	Two sub-masks, each with size of N/2 bits	mask0, mask1	fs0, fs1	sc0, sc1
32	Four sub-masks, each with size of N/4 bits	mask0, mask1 mask2, mask3	fs0, fs1, fs2, fs3	sc0, sc1, sc2, sc3

The following table shows the coverage of the sub-masks across N-dimension:

Sub-mask	M		
	128	64	32
mask0	Columns [0, N-1]	Columns [0, N/2-1]	Columns [0, N/4-1]
mask1	–	Columns [N/2, N-1]	Columns [N/4, N/2-1]
mask2	–	–	Columns [N/2, (N/4*3)-1]
mask3	–	–	Columns [(N/4*3), N-1]

The following examples shows zero-column mask descriptor and their corresponding mask generated:

1. Example 1: M = 128

Input zero-column mask descriptor:

Start count	First span	Non-Zero Mask	Skip Span	Use Span	Shift
{0, 0, 0, 0}	{0, 0, 0, 0}	0	4	3	0

Output zero-column mask: 0x0.

As Non-Zero Mask field is 0, the mask is 0x0. All the columns of the matrix B will be used for the MMA operation.

2. Example 2: M = 128

Input zero-column mask descriptor:

Start count	First span	Non-Zero Mask	Skip Span	Use Span	Shift
{-, -, -, 0}	{-, -, -, 0}	1	2	3	0

Output mask0: 0b ... 111 0000 111 0000 (size = N)

3. Example 3: M = 64

Input zero-column mask descriptor:

Start count {..., sc1, sc0}	First span {..., fs1, fs0}	Non-Zero Mask	Skip Span	Use Span	Shift
{-, -, 0, 0}	{-, -, 0, 1}	1	2	3	0

Output mask0: 0b ... 111 0000 111 0000 111

Output masl1: 0b ... 0000 111 0000 111 0000

4. Example 4: M = 32

Input zero-column mask descriptor:

Start count {sc3, sc2, sc1, sc0}	First span {fs3, fs2, fs1, fs0}	Non-Zero Mask	Skip Span	Use Span	Shift
{1, 2, 1, 0}	{0, 0, 1, 1}	1	2	3	2

Output mask0: 0b ... 0000 111 0000 111

Output mask1: 0b ... 0000 111 0000 11

Output mask2: 0b ... 111 0000 111 00

Output mask3: 0b ... 111 0000 111 000

If $N = 128$ then B Matrix with columns from 2 to 129 will be used for the MMA operation, due to the shift of 2.

9.7.16.5 Issue Granularity

Each of the tcgen05 operation has different requirements for the number of threads/warps that needs to issue them.

The following table lists the execution granularity requirements of each of the tcgen05 operation:

Table 48: Execution granularity requirements for tcgen05 operations

tcgen05 operation	.cta_group	Issue Granularity
.mma, .cp, .shift, .commit	::1	An issue from a single thread in the current CTA would initiate the base operation.
	::2	Issue from a single thread from the <i>CTA-Pair</i> would initiate the base operation. When the current CTA issues the operation, the peer CTA should be active and should not have exited.
.alloc, .dealloc, .relinquish_alloc_permit	::1	Issue from a single warp in the current CTA would initiate the allocation management instruction.
	::2	Issue from two warps, one in each of the current CTA and its <i>Peer CTA</i> , in order to collectively perform the operation, i.e., the first warp to perform the operation could block until the the second warp in the <i>Peer CTA</i> also performs the operation (see examples below).

continues on next page

Table 48 – continued from previous page

tcgen05 operation	.cta_group	Issue Granularity
.ld, .st, .wait::{ld, st}	N/A	Issue from a warp in the current CTA can access only 1/4 of the Tensor Memory of the current CTA. So, a warp-group is needed to access the entire Tensor Memory of the current CTA.
.fence::*	N/A	A thread needs to fence all its accesses to the tensor memory that it wants to order with other accesses to the tensor memory from other threads.

The following example shows that:

- Before attempting to deallocate Tensor Memory, it suffices to ensure that there are no concurrent Tensor Memory accesses from the *Peer CTA*.
- Warps can immediately exit after deallocating Tensor Memory; no extra synchronization required.

Table 49: Example of deallocating Tensor Memory before CTA exit.

CTA0 Warp	CTA1 Warp
barrier.cluster.arrive; barrier.cluster.wait; tcgen05.dealloc.2cta.sync.aligned; exit;	barrier.cluster.arrive; barrier.cluster.wait; tcgen05.dealloc.2cta.sync.aligned; exit;

This example uses a cluster barrier for illustration purposes but in practice other synchronization mechanisms are often used.

The following example illustrates a scenario in which the program exhibits non-deterministic behavior due to incorrect synchronizaton because `.dealloc` may or may not block:

Table 50: Example of non-deterministic hang due to incorrect synchronization

CTA0 Warp	CTA1 Warp
barrier.cluster.arrive; barrier.cluster.wait; tcgen05.dealloc.2cta.sync.aligned; exit;	tcgen05.dealloc.2cta.sync.aligned; barrier.cluster.arrive; barrier.cluster.wait; exit;

9.7.16.5.1 CTA Pair

Any 2 CTAs within the cluster whose `%cluster_ctarank` differs by the last bit only is said to form a CTA pair.

Within a CTA pair, the CTA whose last bit in the `%cluster_ctarank` is:

- ▶ 0 is termed the even numbered CTA within the CTA pair.
- ▶ 1 is termed as the odd numbered CTA within the CTA pair.

Most of the `tcgen05` operations can either execute at a single CTA level granularity OR at a CTA pair level granularity. When a `tcgen05` operation is performed at CTA pair granularity, the Tensor Memory of both the CTAs within the CTA pair are accessed. The set of threads that need to issue the `tcgen05` operation is listed in the *Issue Granularity*.

9.7.16.5.2 Peer CTA

The peer CTA of the odd CTA within the CTA pair is the even CTA in the same pair. Similarly, the peer CTA of the even CTA within the CTA pair is the odd CTA in the same pair.

9.7.16.6 Memory Consistency Model for 5th generation of TensorCore operations

Ordering of `tcgen05` instructions is described in terms of two key concepts:

1. Pipelined `tcgen05` instructions
2. Specialized `tcgen05`-specific inter-thread synchronization mechanisms.

These concepts combine to form four canonical synchronization patterns, as described further below.

9.7.16.6.1 Asynchronous Operations

The `tcgen05` family of instructions are divided into 2 categories:

1. Asynchronous instructions:

These `tcgen05` operations are not inherently ordered with respect to other `tcgen05` operations in the same thread (unless pipelined as mentioned below).

2. Synchronous instructions:

These `tcgen05` operations are inherently ordered with respect to other `tcgen05` operations in the same order.

The Tensor Memory allocation related instructions that access shared memory maintain same-address ordering with respect to non-`tcgen05` instructions.

The following table lists the category of each of the `tcgen05` instruction:

tcgen05.* operation	Category
.alloc	Synchronous instructions
.dealloc	
.relinquish_alloc_permit	
.fence::*	
.wait::*	
.commit	
.mma	Asynchronous instructions
.cp	
.shift	
.ld	
.st	

9.7.16.6.2 Pipelined tcgen05 Instructions

The asynchronous tcgen05 operations may execute and complete in a different order than they were issued. However, some specific pairs of the asynchronous tcgen05 instructions form tcgen05 pipelines, where in the two asynchronous operations are guaranteed to execute in the same order as the instructions that issued them. The specific pairings are as follows:

1. tcgen05.mma.cta_group::N -> tcgen05.mma.cta_group::N (same N and accumulator and shape)
2. tcgen05.copy.cta_group::N -> tcgen05.mma.cta_group::N (same N)
3. tcgen05.shift.cta_group::N -> tcgen05.mma.cta_group::N (same N)
4. tcgen05.shift.cta_group::N -> tcgen05.cp.4x256b.cta_group::N (same N)
5. tcgen05.mma.cta_group::N -> tcgen05.shift.cta_group::N (same N)

9.7.16.6.2.1 Implicitly pipelined tcgen05 Instructions

Instructions `tcgen05.commit` and `tcgen05.wait` are implicitly pipelined with respect to previously issued `tcgen05.{mma, cp, shift}` and `tcgen05.{ld, st}` instructions respectively that they track from the same thread.

9.7.16.6.2.2 mbarrier based completion mechanism

Completion of the following instruction's asynchronous operations is observed through the mbarrier based waiting mechanism:

1. `tcgen05.mma`
2. `tcgen05.cp`
3. `tcgen05.shift`

`tcgen05.commit` is used to track the completion of the above asynchronous instructions.

Following are the implicitly pipelined `tcgen05` instruction pairing that uses mbarrier based completion mechanism:

- ▶ `tcgen05.mma.cta_group::N -> tcgen05.commit.cta_group::N` (same N)
- ▶ `tcgen05.cp.cta_group::N -> tcgen05.commit.cta_group::N` (same N)
- ▶ `tcgen05.shift.cta_group::N -> tcgen05.commit.cta_group::N` (same N)

9.7.16.6.2.3 tcgen05.wait instruction based completion mechanism

Completion of the following instruction's asynchronous operations is observed through `tcgen05.wait` based waiting mechanism:

1. `tcgen05.ld`
2. `tcgen05.st`

`tcgen05.wait::ld` and `tcgen05.wait::st` is used to track the completion of the `tcgen05.ld` and `tcgen05.st` asynchronous instructions.

Following are the implicitly pipelined `tcgen05` instruction pairing that uses `tcgen05.wait` based completion mechanism:

- ▶ `tcgen05.ld -> tcgen05.wait::ld`
- ▶ `tcgen05.st -> tcgen05.wait::st`

9.7.16.6.3 Specialized Inter-thread Synchronization for tcgen05 instructions

The `tcgen05` instructions support a specialized inter-thread synchronization which are optimized for `tcgen05` family of instructions. The standard memory consistency model synchronization mechanisms also apply to the `tcgen05` family of instructions.

The *TensorCore 5th Generation Specialized Synchronization Operations* section contains the specialized inter-thread synchronization for `tcgen05` instructions.

The `tcgen05.fence::before_thread_sync` and `tcgen05.fence::after_thread_sync` composes with execution ordering instructions, like morally strong `ld/st/atom` instructions, `mbarrier`

instruction, barrier instructions and so on, to establish an ordering between the `tcgen05` operations across threads. The asynchronous `tcgen05` instructions that are ordered across threads also form a `tcgen05` pipeline.

An asynchronous `tcgen05` operation prior to a `tcgen05.fence::before_thread_sync` is ordered before all subsequent `tcgen05` and the execution ordering operations.

An asynchronous `tcgen05` operation subsequent to a `tcgen05.fence::after_thread_sync` is ordered after all the prior `tcgen05` and the execution ordering operations.

9.7.16.6.4 Canonical synchronization patterns

Using the above rules, the following are the five canonical synchronization patterns:

9.7.16.6.4.1 Pipelined instructions, same thread

In this pattern, no explicit ordering mechanism is needed and the ordering guarantee is provided by the pipelined instruction pairing.

Example:

```
tcgen05.mma
tcgen05.mma (same shape and accumulator)
```

The two instructions will be executed in program order.

9.7.16.6.4.2 Non-pipelined instructions, same thread

In this pattern, explicit waiting mechanisms are used to wait for the completion of the asynchronous `tcgen05` operations.

Example 1:

```
tcgen05.st
tcgen05.wait::st
tcgen05.ld
```

`tcgen05.wait::st` is used to wait for the completion of the prior asynchronous instruction `tcgen05.st`.

Example 2:

```
tcgen05.mma [d], ...
tcgen05.commit.mbarrier::arrive::one
mbarrier.try_wait.relaxed.cluster (loop until successful)
tcgen05.fence::after_thread_sync
tcgen05.ld [d], ...
```

For the completion of the asynchronous `tcgen05.mma`, `tcgen05.commit` is used.

As `tcgen05.ld` is an asynchronous operation, the instruction `tcgen05.fence::after_thread_sync` is needed.

No explicit `tcgen05.fence::before_thread_sync` is needed as this is implicitly performed by `tcgen05.commit`. The combination of `tcgen05.mma` and `tcgen05.commit` forms a conceptual asynchronous pipeline and establishes execution ordering.


```
tcgen05.mma [d], ...
tcgen05.fence::before_thread_sync
mbarrier::arrive
```

9.7.16.6.4.3 Pipelined instructions, different thread

In this pattern, no explicit waiting mechanism is needed but proper synchronization between threads is needed.

Example:

Thread 0	Thread 1
<pre>tcgen05.cp tcgen05.fence::before_thread_sync mbarrier.arrive.relaxed.cluster</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster // ↪ loop till success tcgen05.fence::after_thread_sync tcgen05.mma</pre>

9.7.16.6.4.4 Non-pipelined instructions, different thread

In this pattern, the producer threads that issue the asynchronous `tcgen05` instructions must explicitly wait for the instructions' completion before synchronizing with the consumer threads.

Example 1:

Thread 0	Thread 1
<pre>tcgen05.ld tcgen05.wait::ld tcgen05.fence::before_thread_sync mbarrier.arrive.relaxed.cluster</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster // ↪ loop till success tcgen05.fence::after_thread_sync tcgen05.mma</pre>

Example 1:

Thread 0	Thread 1
<pre>tcgen05.mma tcgen05.commit.mbarrier::arrive::one ↪ [mbar]</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster [mbar] ↪ // loop till success tcgen05.fence::after_thread_sync tcgen05.ld</pre>

The synchronization mechanisms can also be composed with each other. For example:

Thread 0	Thread 1
<pre>tcgen05.mma tcgen05.commit.mbarrier::arrive::one ↪ [bar1] mbarrier.try_wait.relaxed.cluster [bar1] ↪ // loop ... tcgen05.fence::after_thread_sync ... // completion is guaranteed tcgen05.fence::before_thread_sync mbarrier.arrive.relaxed.cluster [bar2] ↪ // loop ...</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster [bar2] ↪ // loop ... tcgen05.fence::after_thread_sync tcgen05.ld</pre>

9.7.16.6.4.5 Register dependencies, same thread

For `tcgen05.ld`, an intra-thread ordering through true register dependency will be respected regardless of the presence or absence of other forms of synchronization. This form of register dependency does not imply any other form of ordering. For example, a register dependency does not imply that a dependee instruction's memory accesses will be performed before a dependent instruction's memory accesses. To enforce such memory orderings and avoiding anti-dependency hazards around `tcgen05.ld`, `tcgen05.wait::ld` must be used.

Example:

```
tcgen05.ld %r1, ...;
tcgen05.mma ..., %r1, ...;
```

9.7.16.6.5 Shared Memory Accesses

The shared memory accesses by `tcgen05.mma` and `tcgen05.cp` operations are performed in the asynchronous proxy (async proxy).

Accessing the same memory location across multiple proxies needs a cross-proxy fence. For the async proxy, `fence.proxy.async` should be used to synchronize memory between generic proxy and the async proxy.

9.7.16.7 Tensor Memory Allocation and Management Instructions

9.7.16.7.1 Tensorcore 5th Generation Instructions: `tcgen05.alloc`, `tcgen05.dealloc`, `tcgen05.relinquish_alloc_permit`

`tcgen05.alloc`, `tcgen05.dealloc`, `tcgen05.relinquish_alloc_permit`

Dynamic *Tensor Memory* allocation management instructions

Syntax

```
tcgen05.alloc.cta_group.sync.aligned{.shared::cta}.b32  [dst], nCols;
tcgen05.dealloc.cta_group.sync.aligned.b32             taddr, nCols;
tcgen05.relinquish_alloc_permit.cta_group.sync.aligned;
.cta_group = { .cta_group::1, .cta_group::2 }
```

Description

`tcgen05.alloc` is a potentially blocking instruction which dynamically allocates the specified number of columns in the *Tensor Memory* and writes the address of the allocated *Tensor Memory* into shared memory at the location specified by address operand `dst`. The `tcgen05.alloc` blocks if the requested amount of *Tensor Memory* is not available and unblocks as soon as the requested amount of *Tensor Memory* becomes available for allocation.

`tcgen05.dealloc` is a potentially blocking instruction which deallocates the *Tensor Memory* specified by the *Tensor Memory* address `taddr`. The operand `taddr` must point to a previous *Tensor Memory* allocation. If `.cta_group::2` is specified,

- ▶ issuing warp and *peer CTA* warp must synchronize *Tensor Memory* accesses before attempting to collectively deallocate the *Tensor Memory*, and
- ▶ `tcgen05.dealloc` may block to collectively performs the deallocation with the other *peer CTA*'s warp.

All of the *Tensor Memory* that was allocated using `tcgen05.alloc` instruction in a kernel, must be explicitly deallocated using `tcgen05.dealloc` before the kernel exits.

The unsigned 32-bit operand `nCols` specify the number of columns to be allocated or de-allocated. The unit of allocation and de-allocation is 32 columns and all of lanes per column. The number of columns must be a power of 2. The operand `nCols` must be within the range [32, 512]. The number of

columns allocated should not increase between any two allocations in the execution order within the CTA. Operand `nCols` must be power of 2.

Instruction `tcgen05.relinquish_alloc_permit` specifies that the CTA of the executing thread is relinquishing the right to allocate *Tensor Memory*. So, it is illegal for a CTA to perform `tcgen05.alloc` after any of its constituent threads execute `tcgen05.relinquish_alloc_permit`.

If no state space is specified then *Generic Addressing* is used. If the address specified by `dst` does not fall within the address window of `.shared::cta` state space then the behavior is undefined.

Qualifier `.cta_group` specifies the number of CTAs involved in the allocation and de-allocation operation. When `.cta_group::1` is specified, one warp from the CTA must perform the allocation and de-allocation. When `.cta_group::2` is specified, one warp from each of the *peer CTAs* must collectively perform the allocation and de-allocation. Refer to the *Issue Granularity* section. When `.cta_group::2` is specified, the issuing warp must make sure that peer CTA is launched and its warps eventually participate in collective operations.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The mandatory `.sync` qualifier indicates that the instruction causes the executing thread to wait until all threads in the warp execute the same instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same instruction. In conditionally executed code, the instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of the instruction is undefined if all the threads in the warp do not use the same values of `nCols`, or if any thread in the warp has exited.

The store operation in `tcgen05.alloc` is treated as a weak memory operation in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
// Example 1:

tcgen05.alloc.cta_group::1.sync.aligned.shared::cta.b32 [sMemAddr1], 32;
ld.shared.b32 taddr, [sMemAddr1];
// use taddr ...
// more allocations and its usages ...
tcgen05.dealloc.cta_group::1.sync.aligned.b32 taddr, 32;
// more deallocations ...
tcgen05.relinquish_alloc_permit.cta_group::1.sync.aligned;

// Example 2:

// Following instructions are performed by current warp and the warp in the peer-CTA:
tcgen05.alloc.cta_group::2.sync.aligned.shared::cta.b32 [sMemAddr2], 32;
ld.shared.b32 taddr, [sMemAddr2];
// use taddr ...
// more allocations and its usages ...
tcgen05.dealloc.cta_group::2.sync.aligned.b32 taddr, 32;
// more deallocations ...
tcgen05.relinquish_alloc_permit.cta_group::2.sync.aligned;
```

9.7.16.8 Tensor Memory and Register Load/Store Instructions

The threads of the CTA can perform the loads and stores to the *Tensor Memory* of the CTA and move data between registers and Tensor Memory. The loads and stores of data can be performed in certain shapes as specified in the *Matrix and Data Movement Shape* section.

9.7.16.8.1 Access restrictions

Not all threads of the CTA can access the entire Tensor Memory via the `tcgen05.ld` and `tcgen05.st` operations.

The Tensor Memory of a CTA is divided into 4 equal chunks such that each warp of a warpgroup in the CTA can access a chunk of the Tensor Memory. All the columns of the Tensor Memory can be accessed by all the four warps of a warpgroup. A lane of the Tensor Memory can be accessed by a single warp in the warpgroup. The following table describes the access restriction.

ID of the warp within the warpgroup	Accessible Lanes
0	0-31
1	32-63
2	64-95
3	96-127

9.7.16.8.2 Packing and Unpacking

Optionally, the following pack and unpack operations can be performed during the load and store:

1. Packing: two 16-bit chunks can be packed into a single 32-bit chunk in the register in `tcgen05.ld`
2. Unpacking: a single 32-bit chunk in the register can be unpacked into two 16-bit chunks in `tcgen05.st`

as shown in the Figure 193.

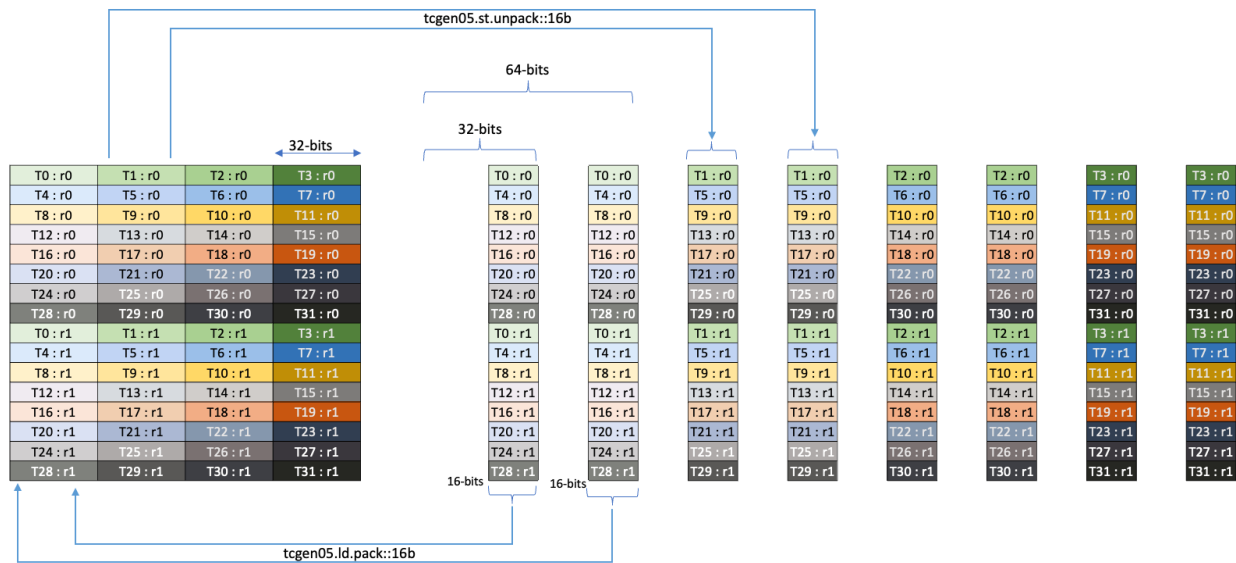


Figure 193: Pack/Unpack operations for `tcgen05 ld/st`

9.7.16.8.3 Tensorcore 5th Generation Instructions: `tcgen05.ld`

`tcgen05.ld`

Asynchronous collective load from tensor memory into registers.

Syntax

```
// Base load instruction:

tcgen05.ld.sync.aligned.shape1.num{.pack}.b32    r, [taddr];

tcgen05.ld.sync.aligned.shape2.num{.pack}.b32    r, [taddr], immHalfSplitoff;

.shape1 = { .16x64b, .16x128b, .16x256b, .32x32b }
.shape2 = { .16x32bx2 }
.num     = { .x1, .x2, .x4, .x8, .x16, .x32, .x64, .x128 }
.pack    = { .pack::16b }

// Floating point type load along with reduction :

tcgen05.ld.red.sync.aligned.shape3.num.redOp{.abs}{.NaN}.f32 r, redval, [taddr];

tcgen05.ld.red.sync.aligned.shape4.num.redOp{.abs}{.NaN}.f32 r, redval, [taddr],
↪immHalfSplitoff;

// Integer type load along with reduction :

tcgen05.ld.red.sync.aligned.shape3.num.redOp.type r, redval, [taddr];

tcgen05.ld.red.sync.aligned.shape4.num.redOp.type r, redval, [taddr], immHalfSplitoff;

.shape3 = { .32x32b    }
.shape4 = { .16x32bx2 }
.redOp  = { .min, .max }
.type   = { .u32, .s32 }
```

Description

Instruction `tcgen05.ld` asynchronously loads data from the *Tensor Memory* at the location specified by the 32-bit address operand `taddr` into the destination register `r`, collectively across all threads of the warps.

All the threads in the warp must specify the same value of `taddr`, which must be the base address of the collective load operation. Otherwise, the behavior is undefined.

The `.shape` qualifier and the `.num` qualifier together determines the total dimension of the data which is loaded from the *Tensor Memory*. The `.shape` qualifier indicates the base dimension of data to be accessed as described in the *Data Movement Shape*. The `.num` qualifier indicates the repeat factor on the base dimension resulting in the total dimension of the data that is accessed.

The shape `.16x32bx2` performs two accesses into Tensor Memory of the shape `.16x32b`. The base address of the first access is specified by `taddr` and the base address of the second access is specified by `taddr+immHalfSplitoff`, where `immHalfSplitoff` is an immediate argument.

The destination operand `r` is a brace-enclosed vector expression consisting of one or more 32-bit registers as per the value of `.shape` and `.num`. The size of the vector for various combinations of `.num` and `.shape` is shown in [Table 51](#).

Table 51: Various-combinations of .num and .shape

.num	.shape		
	.16x32bx2 / .16x64b / .32x32b	.16x128b	.16x256b
.x1	1	2	4
.x2	2	4	8
.x4	4	8	16
.x8	8	16	32
.x16	16	32	64
.x32	32	64	128
.x64	64	128	NA
.x128	128	NA	NA

The qualifier `.red` specifies that the reduction operation specified by `.redOp` is performed on the data that is loaded across columns in each lane. The result of the reduction operation is written into the corresponding thread's 32-bit destination register operand `redVal`. When `.red` qualifier is specified, `.num` modifier must be at least `.x2`.

The optional qualifier `.pack : :16b` can be used to pack two 16-bit elements from adjacent columns into a single 32-bit element during the load as shown in the section [Packing and Unpacking](#).

The mandatory `.sync` qualifier indicates that `tcgen05.ld` causes the executing thread to wait until all threads in the warp execute the same `tcgen05.ld` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tcgen05.ld` instruction. In conditionally executed code, a `tcgen05.ld` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `tcgen05.ld` is undefined if all threads do not use the same values of `taddr`, or if any thread in the warp has exited.

The instruction `tcgen05.ld` is performed asynchronously and more details are specified in the section [Memory Consistency Model for 5th generation of TensorCore operations](#).

PTX ISA Notes

Introduced in PTX ISA version 8.6.

`tcgen05.ld.red` is introduced in PTX ISA version 8.8.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

`tcgen05.ld.red` is supported on following architectures:

- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_103f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Examples

```
tcgen05.ld.sync.aligned.32x32b.x2.b32    {r0, r1}, [taddr1];
tcgen05.ld.sync.aligned.16x128b.x4.b32    {r0, r1, r2, r3, r4, r5, r6, r7}, [taddr2];
tcgen05.ld.red.sync.aligned.16x32bx2.x8.u32.max {r0, r1, r2, r3, r4, r5, r6, r7},
                                           redVal, [taddr3], 16;
```

9.7.16.8.4 Tensorcore 5th Generation Instructions: `tcgen05.st`

`tcgen05.st`

Asynchronous collective store to tensor memory from registers.

Syntax

```
tcgen05.st.sync.aligned.shape1.num{.unpack}.b32    [taddr], r;
tcgen05.st.sync.aligned.shape2.num{.unpack}.b32    [taddr], immHalfSplitoff, r;

.shape1 = { .16x64b, .16x128b, .16x256b, .32x32b }
.shape2 = { .16x32bx2 }
.num     = { .x1, .x2, .x4, .x8, .x16, .x32, .x64, .x128 }
.unpack  = { .unpack::16b }
```

Description

Instruction `tcgen05.st` asynchronously stores data from the source register `r` into the *Tensor Memory* at the location specified by the 32-bit address operand `taddr`, collectively across all threads of the warps.

All the threads in the warp must specify the same value of `taddr`, which must be the base address of the collective store operation. Otherwise, the behavior is undefined.

The `.shape` qualifier and the `.num` qualifier together determines the total dimension of the data which is stored to the Tensor Memory. The `.shape` qualifier indicates the base dimension of data to be accessed as described in the *Data Movement Shape*. The `.num` qualifier indicates the repeat factor on the base dimension resulting in the total dimension of the data that is accessed.

The shape `.16x32bx2` performs two accesses into Tensor Memory of the shape `.16x32b`. The base address of the first access is specified by `taddr` and the base address of the second access is specified by `taddr+immHalfSplitoff`, where `immHalfSplitoff` is an immediate argument.

The source operand `r` is a brace-enclosed vector expression consisting of one or more 32-bit registers as per the value of `.shape` and `.num`. The size of the vector for various combinations of `.num` and `.shape` is shown in [Table 52](#).

Table 52: Various-combinations of `.num` and `.shape`

.num	.shape		
	.16x32bx2 / .16x64b / .32x32b	.16x128b	.16x256b
.x1	1	2	4
.x2	2	4	8
.x4	4	8	16
.x8	8	16	32
.x16	16	32	64
.x32	32	64	128
.x64	64	128	NA
.x128	128	NA	NA

The optional qualifier `.unpack::16b` can be used to unpack a 32-bit element in the register into two 16-bit elements and store them in adjacent columns as shown in the section *Packing and Unpacking*.

The mandatory `.sync` qualifier indicates that `tcgen05.st` causes the executing thread to wait until all threads in the warp execute the same `tcgen05.st` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tcgen05.st` instruction. In conditionally executed code, a `tcgen05.st` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `tcgen05.st` is undefined if all threads do not use the same values of `taddr`, or if any thread in the warp has exited.

The instruction `tcgen05.st` is performed asynchronously and more details are specified in the section *Memory Consistency Model for 5th generation of TensorCore operations*.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
tcgen05.st.sync.aligned.16x64b.x4.b32      [taddr0], {r0, r1, r2, r3};
tcgen05.st.sync.aligned.16x128b.x1.unpack::16b.b32 [taddr1], {r0, r1};
```

9.7.16.8.5 Tensorcore 5th Generation Instructions: `tcgen05.wait`

`tcgen05.wait`

Waits for the completion of all prior asynchronous `tcgen05.ld` / `tcgen05.st` instructions.

Syntax

```
tcgen05.wait_operation.sync.aligned;  
.wait_operation = { .wait::ld, .wait::st }
```

Description

Instruction `tcgen05.wait::st` causes the executing thread to block until all prior `tcgen05.st` operations issued by the executing thread have completed.

Instruction `tcgen05.wait::ld` causes the executing thread to block until all prior `tcgen05.ld` operations issued by the executing thread have completed.

The mandatory `.sync` qualifier indicates that `tcgen05.wait_operation` causes the executing thread to wait until all threads in the warp execute the same `tcgen05.wait_operation` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tcgen05.wait_operation` instruction.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

Example 1:

```
tcgen05.ld.sync.aligned.32x32b.x2.b32    {r0, r1}, [taddr0];  
  
// Prevents subsequent tcgen05.mma from racing ahead of the tcgen05.ld  
  
tcgen05.wait::ld.sync.aligned;  
  
tcgen05.mma.cta_group::1.kind::f16    [taddr0], a-desc, b-desc, idesc, p;
```

Example 2:

(continues on next page)

(continued from previous page)

```
tcgen05.st.sync.aligned.32x32b.x2.b32    [taddr0], {r0, r1};
// Prevents the write to taddr0 in tcgen05.mma from racing ahead of the tcgen05.st
tcgen05.wait::st.sync.aligned;
tcgen05.mma.cta_group::1.kind::f16    [taddr0], a-desc, b-desc, idesc, p;
```

9.7.16.9 Tensor Memory Data Movement Instructions

Data from the shared memory can be copied asynchronously to the *Tensor Memory* using the *Tensor-core 5th Generation Instructions: tcgen05.cp* operation.

9.7.16.9.1 Optional Decompression

Optionally, during the copy, a vector of 4-bit and 6-bit custom floating point types can be decompressed into 8-bit types.

9.7.16.9.1.1 Decompression of 4-bit floating point to 8-bit type

A contiguous set of 16 elements of 4-bits each followed by 8 bytes of padding can be converted into 16 elements of 8-bits each as shown in [Figure 194](#).

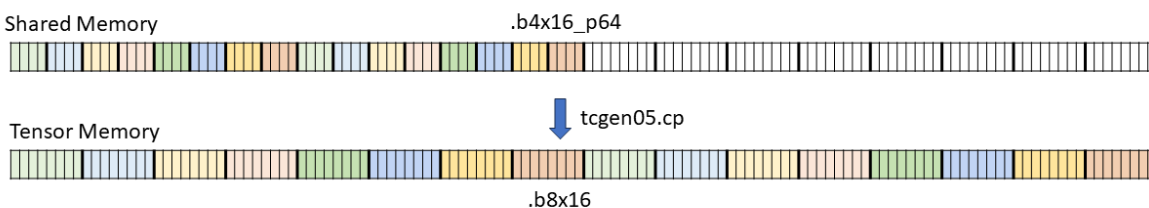


Figure 194: Decompression from 4-bit to 8-bit

The individual 4-bit to 8-bit decompression would look like as shown in [Figure 195](#).

9.7.16.9.1.2 Decompression of 6-bit floating point to 8-bit type

A contiguous set of 16 elements of 6-bits each followed by 4 bytes of padding is decompressed into 16 elements of 8-bits each as shown in [Figure 196](#).

The individual 6-bit to 8-bit decompression for types E3M2 and E2M3 is shown in [Figure 197](#) and [Figure 198](#) respectively.

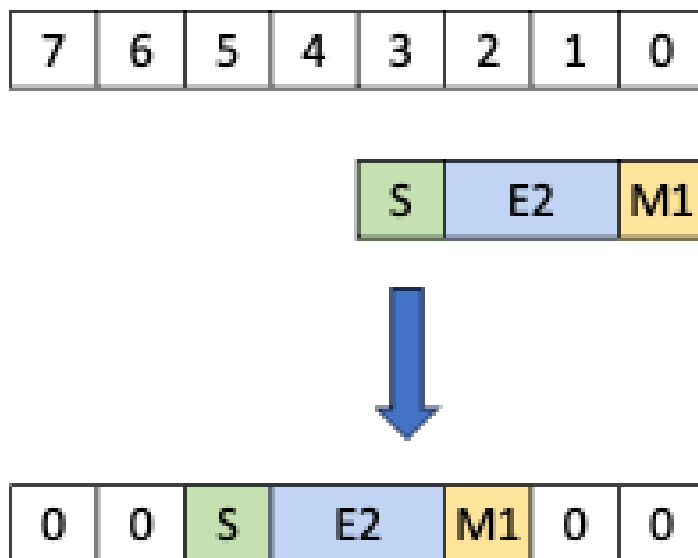


Figure 195: Individual decomposition from 4-bit to 8-bit

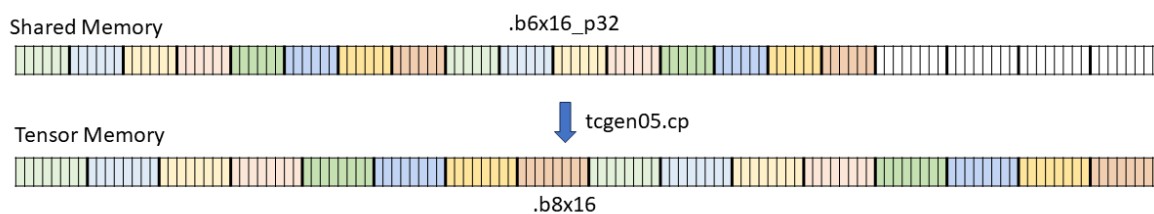


Figure 196: Decompression from 6-bit to 8-bit

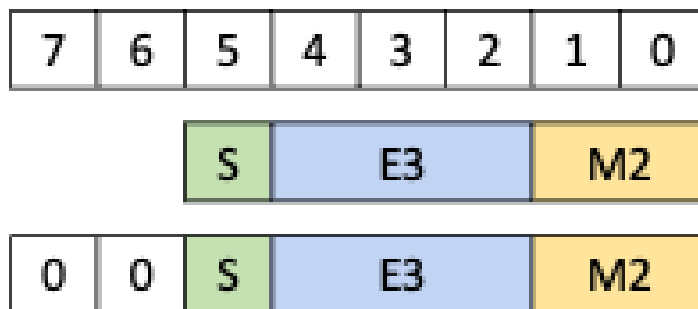


Figure 197: Individual decomposition from 6-bit to 8-bit for E3M2 type

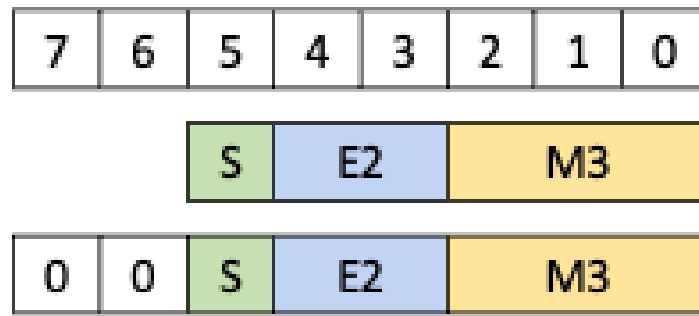


Figure 198: Individual decompression from 6-bit to 8-bit for E2M3 type

9.7.16.9.2 Tensorcore 5th Generation Instructions: `tcgen05.cp`

`tcgen05.cp`

Initiates an asynchronous copy operation from shared memory to the *Tensor Memory*.

Syntax

```
tcgen05.cp.cta_group.shape{.multicast}{.dst_fmt.src_fmt} [taddr], s-desc;

.cta_group = { .cta_group::1, .cta_group::2 }
.src_fmt   = { .b6x16_p32 , .b4x16_p64 }
.dst_fmt   = { .b8x16 }
.shape     = { .128x256b, .4x256b, .128x128b, .64x128b**, .32x128b*** }
.multicast = { .warp2::02_13** , .warp2::01_23** , .warp4*** }
```

Description

Instruction `tcgen05.cp` initiates an asynchronous copy operation from shared memory to the location specified by the address operand `taddr` in the *Tensor Memory*.

The 64-bit register operand `s-desc` is the matrix descriptor which represents the source matrix in the shared memory that needs to be copied. The format of the matrix descriptor is described in *Matrix Descriptors*.

The `.shape` qualifier indicates the dimension of data to be copied as described in the *Data Movement Shape*.

Qualifier `.cta_group` specifies the number of CTAs whose *Tensor Memory* is accessed when a single thread of a single CTA executes the `tcgen05.cp` instruction. When `.cta_group::1` is specified, the data is copied into the *Tensor Memory* of the current CTA. When `.cta_group::2` is specified, the data is copied into the *Tensor Memory* of both the current and the *peer CTAs*.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

When the qualifiers `.dst_fmt` and `.src_fmt` are specified, the data is decompressed from the source format `.src_fmt` in the shared memory to the destination format `.dst_fmt` in *Tensor Memory* by the

copy operation. The details of source and the destination formats as specified in the section *Optional Decompression*.

Some of the `.shape` qualifiers require certain `.multicast` qualifiers.

1. `.64x128b` requires `.warpx2::02_13` or `.warpx2::01_23`
2. `.32x128b` requires `.warpx4`

When the `.multicast` qualifier is specified as either `.warpx2::02_13` or `.warpx2::01_23` then the data being copied is multicasted into warp pairs and each warp in the warp pair receive half of the data. Warp pairs are formed as follows:

1. `.warpx2::02_13` : warps 0 and 2 form a pair; warps 1 and 3 form a pair.
2. `.warpx2::01_23` : warps 0 and 1 form a pair; warps 2 and 3 form a pair.

When the `.multicast` modifier is specified as `.warpx4` then the data being copied is multicasted into all 4 warps.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
tcgen05.cp.cta_group::1.128x256b      [taddr0], sdesc0;  
tcgen05.cp.cta_group::2.128x128b.b8x16.b6x16_p32 [taddr1], sdesc1;  
tcgen05.cp.cta_group::1.64x128b.warpx2::02_13 [taddr2], sdesc2;
```

9.7.16.9.3 Tensorcore 5th Generation Instructions: `tcgen05.shift`

`tcgen05.shift`

Asynchronously shift down the rows of the matrix in the *Tensor Memory* for a warp.

Syntax

```
tcgen05.shift.cta_group.down [taddr];
.cta_group = { .cta_group::1, .cta_group::2 }
```

Description

Instruction `tcgen05.shift` is an asynchronous instruction which initiates the shifting of 32-byte elements downwards across all the rows, except the last, by one row. The address operand `taddr` specifies the base address of the matrix in the *Tensor Memory* whose rows must be down shifted.

The lane of the address operand `taddr` must be aligned to 32.

Qualifier `.cta_group` specifies the number of CTAs whose *Tensor Memory* is touched when a single thread of a single CTA executes the `tcgen05.shift` instruction. When `.cta_group::1` is specified, the shift operation is performed in the *Tensor Memory* of the current CTA. When `.cta_group::2` is specified, the shift operation is performed in the *Tensor Memory* of both the current and the *peer CTAs*.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_103a`
- ▶ `sm_110a`

Examples

```
tcgen05.shift.down.cta_group::1 [taddr0];
tcgen05.shift.down.cta_group::2 [taddr1];
```

9.7.16.10 TensorCore 5th Generation Matrix Multiply and accumulate Operations

The 5th generation of TensorCore operations of shape $M \times N \times K$ perform matrix multiplication and accumulation of the form:

$$D = A * B + D$$

where:

- ▶ the A matrix has shape $M \times K$, in either Tensor Memory or Shared Memory
- ▶ the B matrix has shape $K \times N$, in Shared Memory of the current CTA and optionally in peer CTA
- ▶ the D matrix is of the shape $M \times N$, in Tensor Memory

Optionally an input predicate can be used to disable the input from the accumulator matrix and the following operation can be performed as

$$D = A * B$$

The matrix multiplication and accumulation operations are categorized into various kinds based on input types and the throughput of the multiplication operation. The following shows the different kinds of MMA operations that are supported:

1. f16 : supports f16 and bf16 input types.
2. tf32 : supports tf32 input types.
3. f8f6f4 : supports all input combinations of f8, f6 and f4 types.
4. i8 : supports signed and unsigned 8-bit integer input types.
5. mxf8f6f4/mxf4 : supports mx-floating points input types.
6. mxf4nvf4 : supports mxf4 type and a custom NVIDIA floating-point type for inputs where the type of the vector elements is 4 bits and requires a common scaling factor to form the complete floating-point type, similar to other mx-types.

Optionally, the 5th generation of TensorCore MMAs support dense and sparse matrix A. [Sparse Matrices](#) describes the details of the sparse matrices.

Some of the MMA-kinds requires scaling of input matrices from memory to form the matrix A and matrix B before performing the MMA operation. [Block Scaling for tcgen05.mma.sync](#) describes the details of the scaling of matrices.

The following table show the various matrices involved in the MMA operations and the memory in which they can reside:

Matrix Type	Memory
A	Tensor Memory OR Shared Memory
B	Shared Memory
D	Tensor Memory
Sparse Meta Data	
A-Scale / B-Scale	

A sequence of MMA instructions may reuse the same A matrix with a sequence of B matrices or may reuse the same B matrix with a sequence of A matrices. In these patterns the TensorCore may be able to load the unchanged matrix once and reuse it through the sequence without multiple reloads. The A or B matrices are loaded into a TensorCore collector buffer (i.e., special cache).

An MMA instruction has an optional `collector` qualifier to specify when an A or B matrix is new to the sequence and should be loaded, unchanged within the sequence and should be reused, or the last use in the sequence and should be discarded. The `collector` qualifier is used to give the TensorCore permission to reuse a previously loaded A or B matrix; however reuse is opportunistic in that the TensorCore may reload a matrix even when it has permission to reuse that matrix. Thus, the source memory of an A or B matrix must not be modified while the MMA instruction using those matrices has not completed - regardless of `collector` qualifier permissions.

The 5th generation of TensorCore MMAs can be used for general matrix multiplication OR for convolution operations. In case of convolutions, the activations can be stored in either matrix A or matrix B while the weights will be stored in the other matrix.

Activation Matrix	Weights Matrix	Name of the op	Instruction Name	Collector Buffer Applicability
A	B	Activation Stationary	(default <code>tcgen05.mma</code>)	Collector buffer is applicable on matrix A
B	A	Weights Stationary	<code>.ws</code>	Collector buffer is applicable on matrix B

9.7.16.10.1 Transpose and Negate operations

The matrices A and B can be transposed by specifying the Transpose A Matrix and Transpose B Matrix bits in the instruction descriptor respectively.

The elements of the matrices A and B can be negated by specifying the Negate A Matrix and Negate B Matrix bits in the instruction descriptor respectively.

The support for Transpose and Negate operation for various MMA-Kind are shown in [Table 53](#).

Table 53: Transpose and Negate operation for various MMA-Kind

MMA-Kind	Is Transpose A/B supported	Is Negate A/B supported
<code>.kind::tf32</code>	Yes	Yes
<code>.kind::f16</code>	Yes	Yes
<code>.kind::f8f6f4</code>	Yes	Yes
<code>.kind::mxf8f6f4</code>	Yes	Yes
<code>.kind::i8</code>	Yes	No
<code>.kind::mxf4</code>	No	Yes
<code>.kind::mxf4nvf4</code>	No	Yes

For `.kind::tf32`, the transpose operations on matrices A and B are supported only with 128B swizzling mode with 32B swizzle-atomicity.

For all other MMA-Kinds, the transpose operations on matrices A and B are not supported on 128B swizzling mode with 32B swizzle-atomicity.

[Table 54](#) shows the valid combinations of N shape with `.cta_group` qualifier for 8bit transpose B.

Table 54: Various combinations of N shape with .cta_group qualifier for 8bit transpose B

.cta_group	N shape
1	16 <= N <= 256, step 16
2	32 <= N <= 256, step 32

9.7.16.10.2 Matrix Layout Organization

Table 55 describes the major-ness used for different matrices.

Table 55: Major-ness for different matrices

Ma-trix	Residing in Memory	Default Major-ness
D	Tensor Memory	Row-Major
A	Tensor Memory	
	Shared Memory	Depends on swizzling mode. Refer <i>Shared Memory Layout and Swizzling</i>
B	Shared Memory	

9.7.16.10.3 Valid Combinations of Type-Size, Major-ness and Swizzling

Table 56: Valid Combinations of Type-Size, Major-ness and Swizzling

Type-Size	Major-ness	Ma-trix	Supported Swizzle
4-bit, 6-bit, 8-bit, 16-bit, 32-bit	Row	A	All swizzling modes
	Column	B	
8-bit 16-bit	Column (transpose)	A	All except 128B swizzling with 32B atomicity
	Row (transpose)	B	
32-bit	Column (transpose)	A	Only 128B swizzling with 32B atomicity
	Row (transpose)	B	

9.7.16.10.4 Packing formats of elements in Tensor and Shared memory

9.7.16.10.4.1 Packing format for matrix D in Tensor Memory

The sub-word elements of matrix D are expected not to be packed within a 32-bit Tensor Memory word. For example, if the type of elements of the matrix D is 16 bits then a Tensor Memory word would contain a single 16-bit element in its lower 16 bits.

9.7.16.10.4.2 Packing format for matrix A and B

The 6-bit and 4-bit floating point types have different packing format requirements for different MMA kinds in both Tensor memory and Shared memory. The requirements are as follows.

9.7.16.10.4.3 Packing format used for matrix A by `.kind::mxf8f6f4` in Tensor Memory

The individual 4-bit and the 6-bit floating point type elements must be packed in an 8-bit container in Tensor memory as shown below. The 8-bit containers must be contiguously packed in a 32-bit Tensor Memory word. For example, if the type of elements of the matrix A is 6 bits then 4 consecutive A elements should be packed in one 32-bit Tensor Memory word.

- 4-bit packing format as shown in [Figure 199](#)

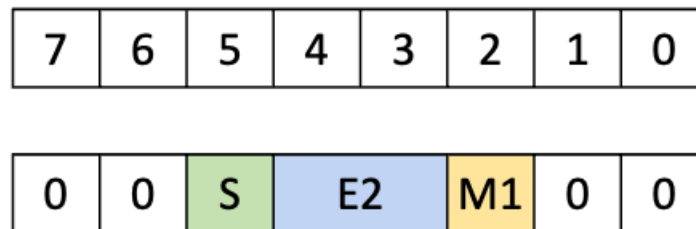


Figure 199: 4-bit packing format with type E2M1

- 6-bit packing format
 - Type E3M2 as shown in [Figure 200](#)
 - Type E2M3 as shown in [Figure 201](#)

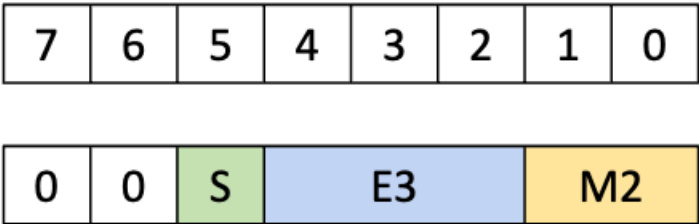


Figure 200: 6-bit packing format with type E3M2

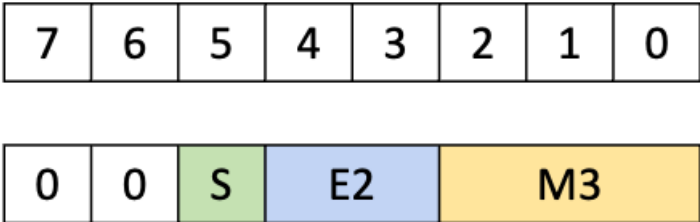


Figure 201: 6-bit packing format with type E2M3

9.7.16.10.4.4 Packing format used for matrix A and B by `.kind::mxf8f6f4` in Shared Memory

The 4-bit and 6-bit floating point elements in shared memory must be contiguously packed along with padding as follows.

- 4-bit packing format as shown in Figure 202

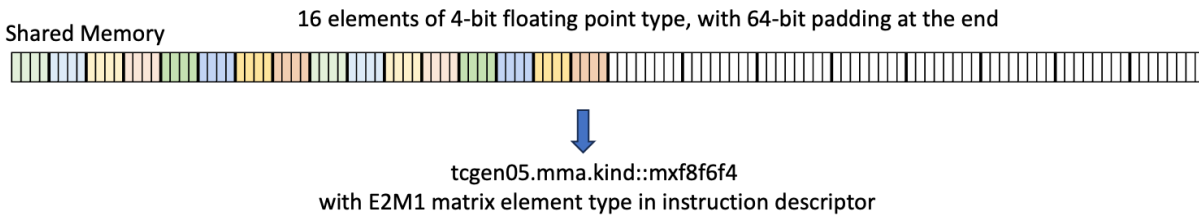


Figure 202: 4-bit packing format

- 6-bit packing format as shown in Figure 203

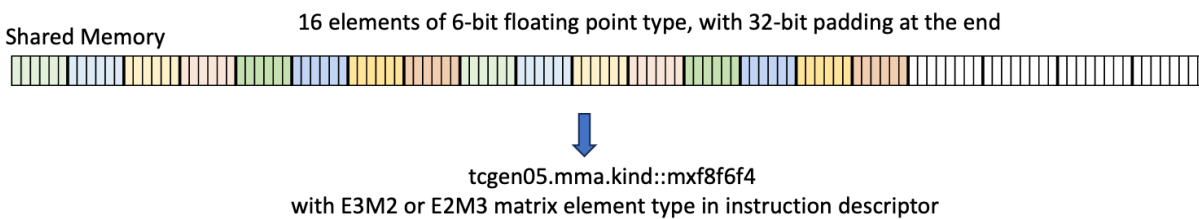


Figure 203: 6-bit packing format

9.7.16.10.4.5 Packing format used for matrix A by `.kind::mxf4` and `.kind::mxf4nvf4` in Tensor Memory

Two 4-bit floating point type elements must be packed in an 8-bit container in Tensor memory as shown in Figure 204 for `mxf4`.

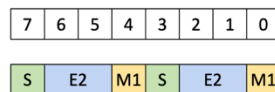


Figure 204: 4-bit packing format with type E2M1

9.7.16.10.4.6 Packing format used for matrix A and B by `.kind::mxf4` and `.kind::mxf4nvf4` in Shared Memory

The packing format for 4-bit floating point elements in shared memory is to pack two 4-bit elements in a 8-bit container, with no padding.

9.7.16.10.5 Data Path Layout Organization

Different MMA variants access the tensor memory with different layout organization. The following table lists the various layouts:

M	cta_group	A-Sparsity	Is .ws mode	Datapath organization	Layout ID	Tensor Memory Lane Alignment	Datapath
32	::1	Either	Yes	1x4	<i>Layout G</i>	0	
64	::1	Either	Yes	2x3	<i>Layout E</i>	0	
64	::1	Either	No	4x1 (1/2 datapath utilized)	<i>Layout F</i>	0 or 16	
128	::1	Either	Either	4x1	<i>Layout D</i>	0	
128	::2	Dense	N/A	2x2	<i>Layout B</i>	0	
128	::2	Sparse	N/A	4x1 (1/2 datapath utilized)	<i>Layout C</i>	0 or 16	
256	::2	Either	N/A	4x1	<i>Layout A</i>	0	

The layouts which utilize only half the datapath lanes, i.e., *Layout F* and *Layout C*, must use the same Tensor Memory lane alignment across matrices A, D and the sparsity metadata matrix.

The following shows the warps that can access the Tensor Memory regions via `tcgen05.ld` / `tcgen05.st` along with the addresses for various Tensor Memory Layouts.

9.7.16.10.5.1 Layout A (M = 256)

Layout organization for M = 256 is shown in [Figure 205](#).

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in [Figure 206](#)

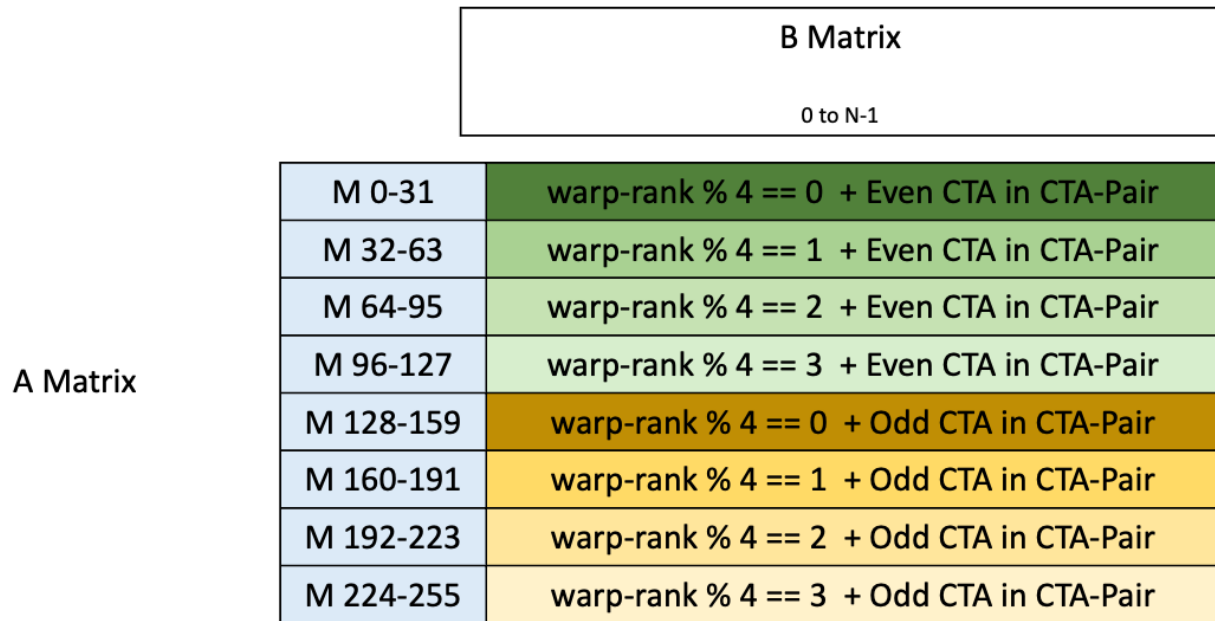


Figure 205: Layout organization for M = 256

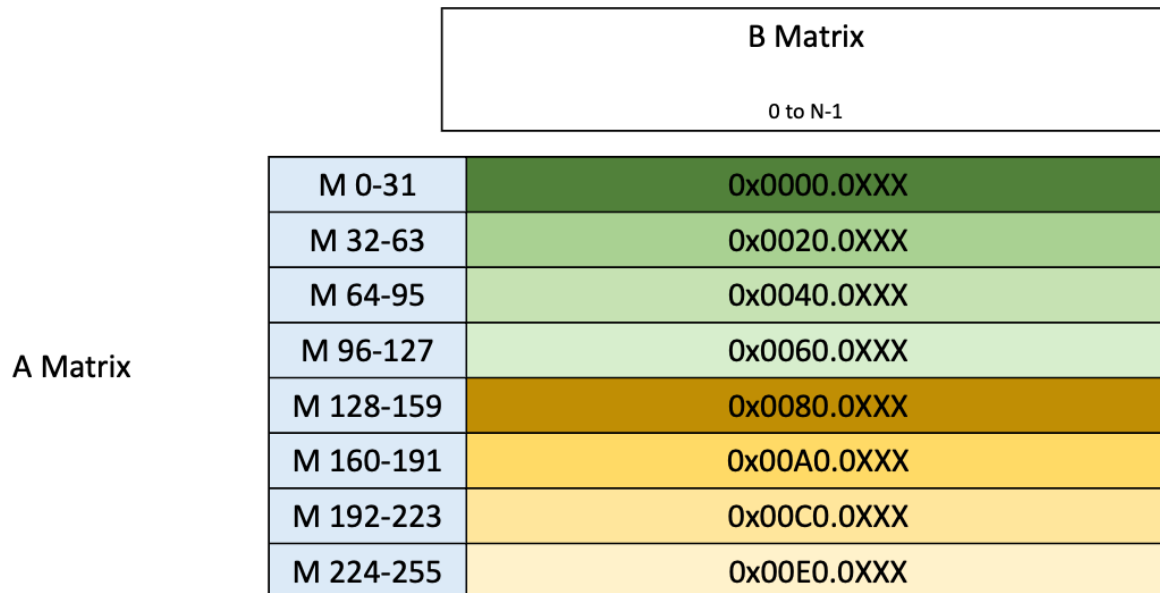


Figure 206: Addresses to use in tcgen05.ld / tcgen05.st

9.7.16.10.5.2 Layout B (M = 128 + cta-group::2 + Dense A matrix)

Layout organization for M = 128 + .cta_group::2 + Dense A matrix is shown in Figure 207.

B Matrix		B Matrix	
0 to (N/2)-1		(N/2) to N-1	
A Matrix	M 0-31	warp-rank % 4 == 0 + Even CTA in CTA-Pair	warp-rank % 4 == 2 + Even CTA in CTA-Pair
	M 32-63	warp-rank % 4 == 1 + Even CTA in CTA-Pair	warp-rank % 4 == 3 + Even CTA in CTA-Pair
	M 64-95	warp-rank % 4 == 0 + Odd CTA in CTA-Pair	warp-rank % 4 == 2 + Odd CTA in CTA-Pair
	M 96-127	warp-rank % 4 == 1 + Odd CTA in CTA-Pair	warp-rank % 4 == 3 + Odd CTA in CTA-Pair

Figure 207: Layout organization for M = 128 + .cta_group::2 + Dense A matrix

Addresses for the above region to be used in tcgen05.ld / tcgen05.st is shown in Figure 208

B Matrix		B Matrix	
0 to (N/2)-1		(N/2) to N-1	
A Matrix	M 0-31	0x0000.0XXX	0x0000.0XXX
	M 32-63	0x0020.0XXX	0x0020.0XXX
	M 64-95	0x0040.0XXX	0x0040.0XXX
	M 96-127	0x0060.0XXX	0x0060.0XXX

Figure 208: Addresses to use in tcgen05.ld / tcgen05.st

9.7.16.10.5.3 Layout C (M = 128 + cta-group::2 + Sparse A matrix)

Layout organization for M = 128 + .cta_group::2 + Sparse A matrix is shown in Figure 209.

Addresses for the above region to be used in tcgen05.ld / tcgen05.st is shown in Figure 210

9.7.16.10.5.4 Layout D (M = 128 + cta-group::1)

Layout organization for M = 128 + .cta_group::1 is shown in Figure 211.

Addresses for the above region to be used in tcgen05.ld / tcgen05.st is shown in Figure 212

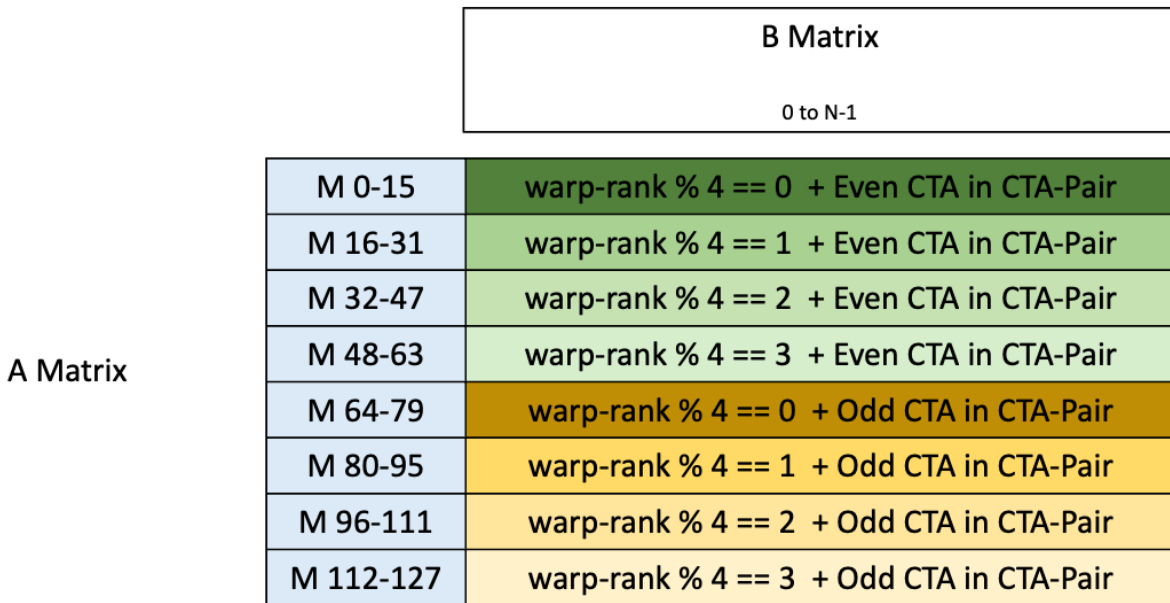


Figure 209: Layout organization for M = 128 + .cta_group::2 + Sparse A matrix

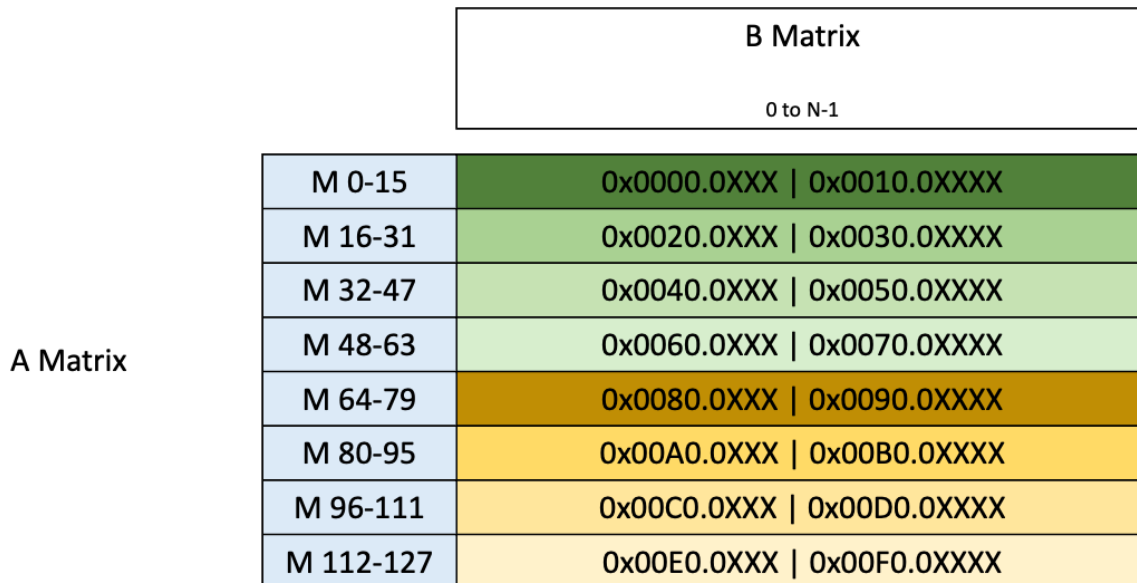


Figure 210: Addresses to use in tcgen05.ld / tcgen05.st

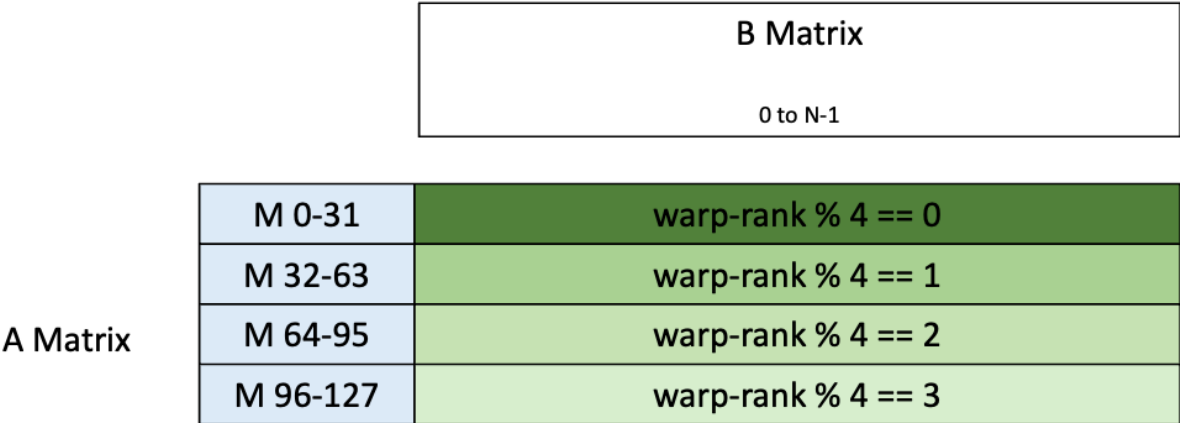


Figure 211: Layout organization for M = 128 + .cta_group::1

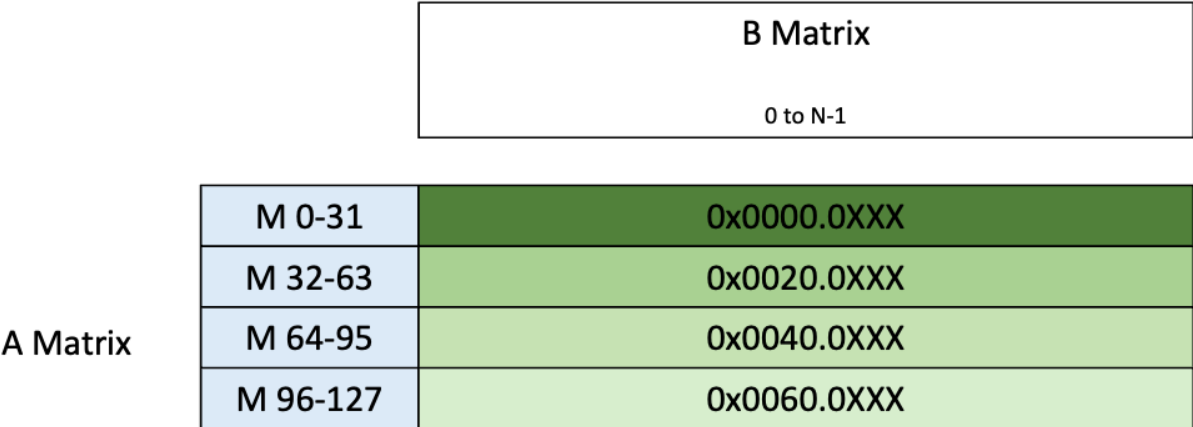


Figure 212: Addresses to use in tcgen05.ld / tcgen05.st

9.7.16.10.5.5 Layout E (M = 64 + .ws mode)

Layout organization for M = 64 + .ws mode is shown in Figure 213.

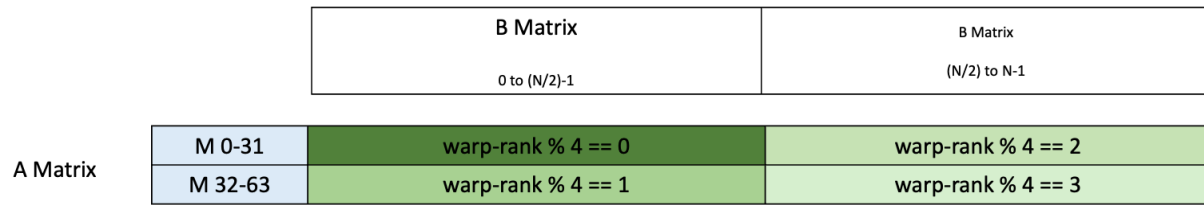


Figure 213: Layout organization for M = 64 + .ws mode

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 214

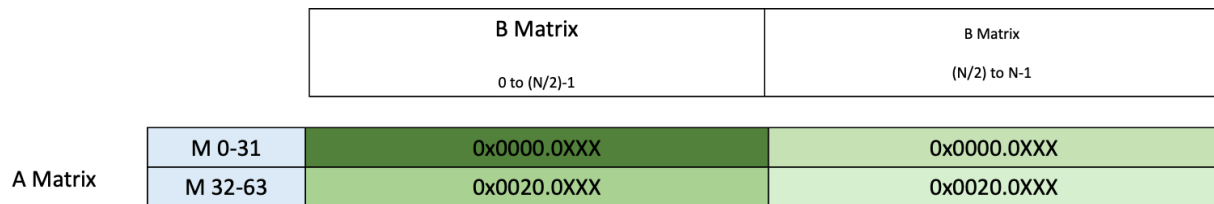


Figure 214: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.5.6 Layout F (M = 64 + non .ws mode)

Layout organization for M = 64 + non .ws mode is shown in Figure 215.

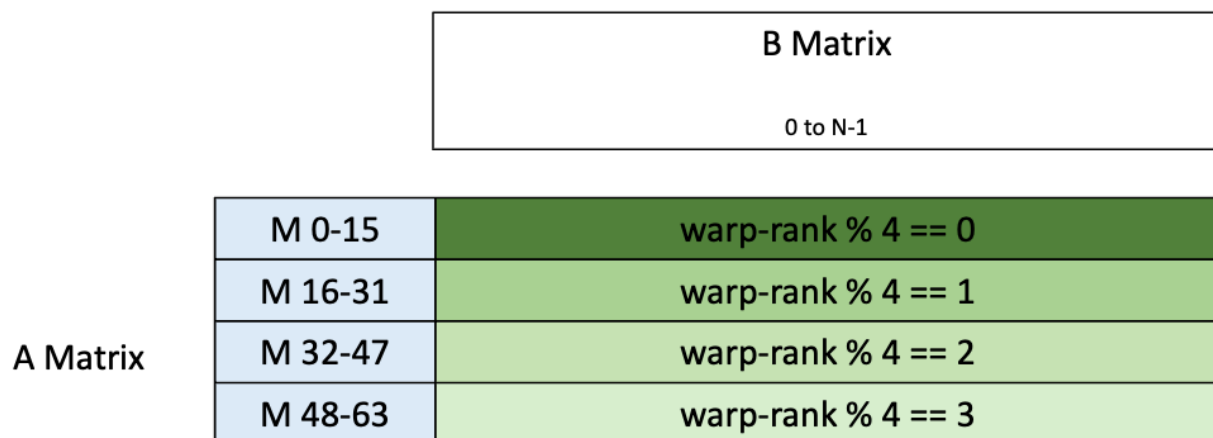
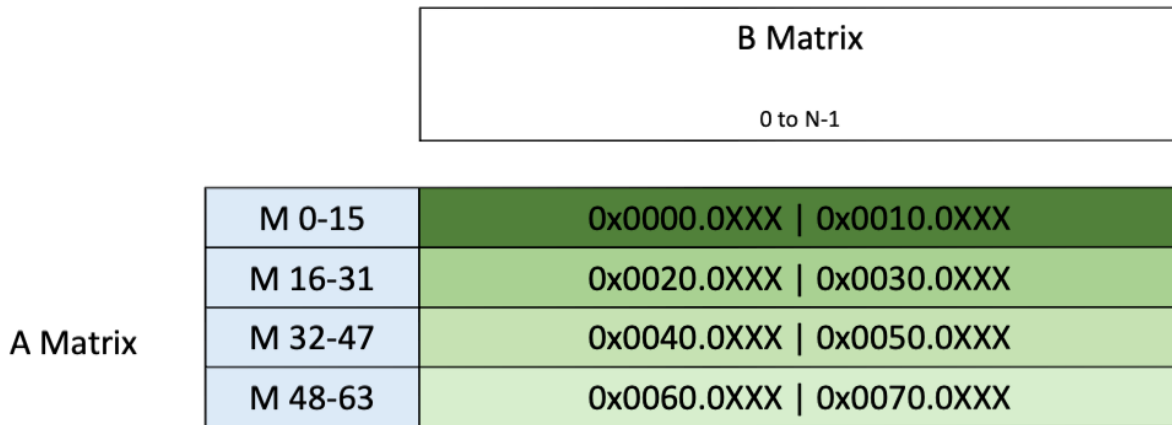


Figure 215: Layout organization for M = 64 + non .ws mode

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 216

Figure 216: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.5.7 Layout G (M = 32)

Layout organization for M = 32 is shown in Figure 217.

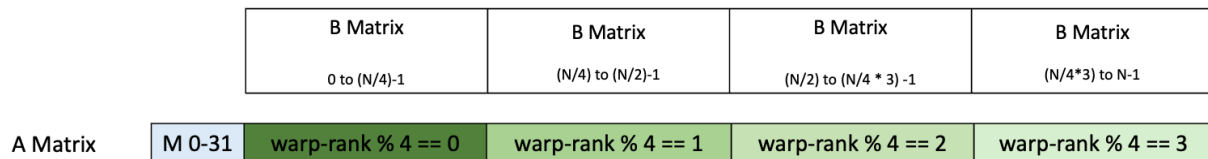
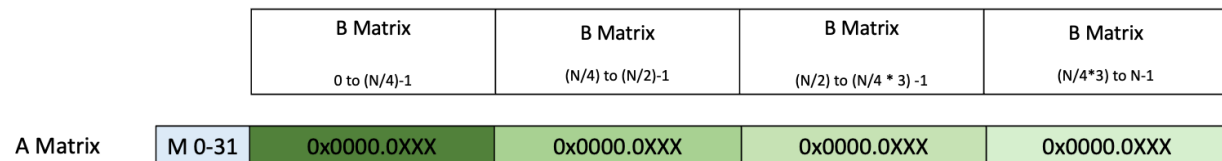


Figure 217: Layout organization for M = 32

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 218

Figure 218: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.6 Shared Memory Layout and Swizzling

If the bit `Transpose A Matrix / Transpose B Matrix` in the *Instruction descriptor* is 0, then *K-major* is used for matrix A / B respectively. If the bit `Transpose A Matrix` in the *Instruction descriptor* is 1 then *M-major* is used for matrix A. If the bit `Transpose B Matrix` in the *Instruction descriptor* is 1, then *N-major* is used for matrix B.

In a column-major default BLAS library such as cuBLAS, the matrices A and B with and without transpose can be classified as either *K-Major* or *M-or-N-Major* as shown in the following table:

	Non-Transposed	Transposed
A	K-major	M-major
B	K-major	N-major

To avoid confusion with A, B, row-major, col-major, transpose, and non-transpose, we will use *MN-Major* and *K-Major* throughout this section.

The matrices in the shared memory are made up of one or more “swizzle layout atom”. The exact layout of these swizzle atoms depends on the swizzling mode, swizzle-atomicity, and the leading dimension. The layout of the swizzle are shown in [Table 57](#)

Table 57: Layout for swizzle atoms

Swizzling mode and Swizzle-Atomicity	Leading Dimension	Swizzle atom layout (128b element)
128B Swizzling with 32B atomicity	M/N	8x4
	–	–
128B Swizzling with 16B atomicity	M/N	8x8
	K	8x8
64B Swizzling Mode	M/N	4x8
	K	8x4
32B Swizzling Mode	M/N	2x8
	K	8x2
None	M/N	1x8
	K	8x1

The above shapes are for elements of size 128 bits. For smaller element sizes, the same shapes would get multiplied along the leading dimension by a factor of $128 / \text{sizeof_bits}(\text{Element})$. For example, 128B MN major swizzle atom would have a shape of $(8 * (128/32)) \times 8 = 32 \times 8$ for tf32 tensor core inputs.

Some example Layouts of $M \times K$ or $K \times N$ matrices with various swizzling modes, and are in units of 128b elements as shown by each colored cell as shown in [Figure 219](#), [Figure 220](#), [Figure 221](#), [Figure 222](#), [Figure 223](#), [Figure 224](#), [Figure 225](#), [Figure 226](#), [Figure 227](#).

Following are some of the examples of the 128B swizzling layout for tf32 element type.

- K-Major: [Figure 228](#)

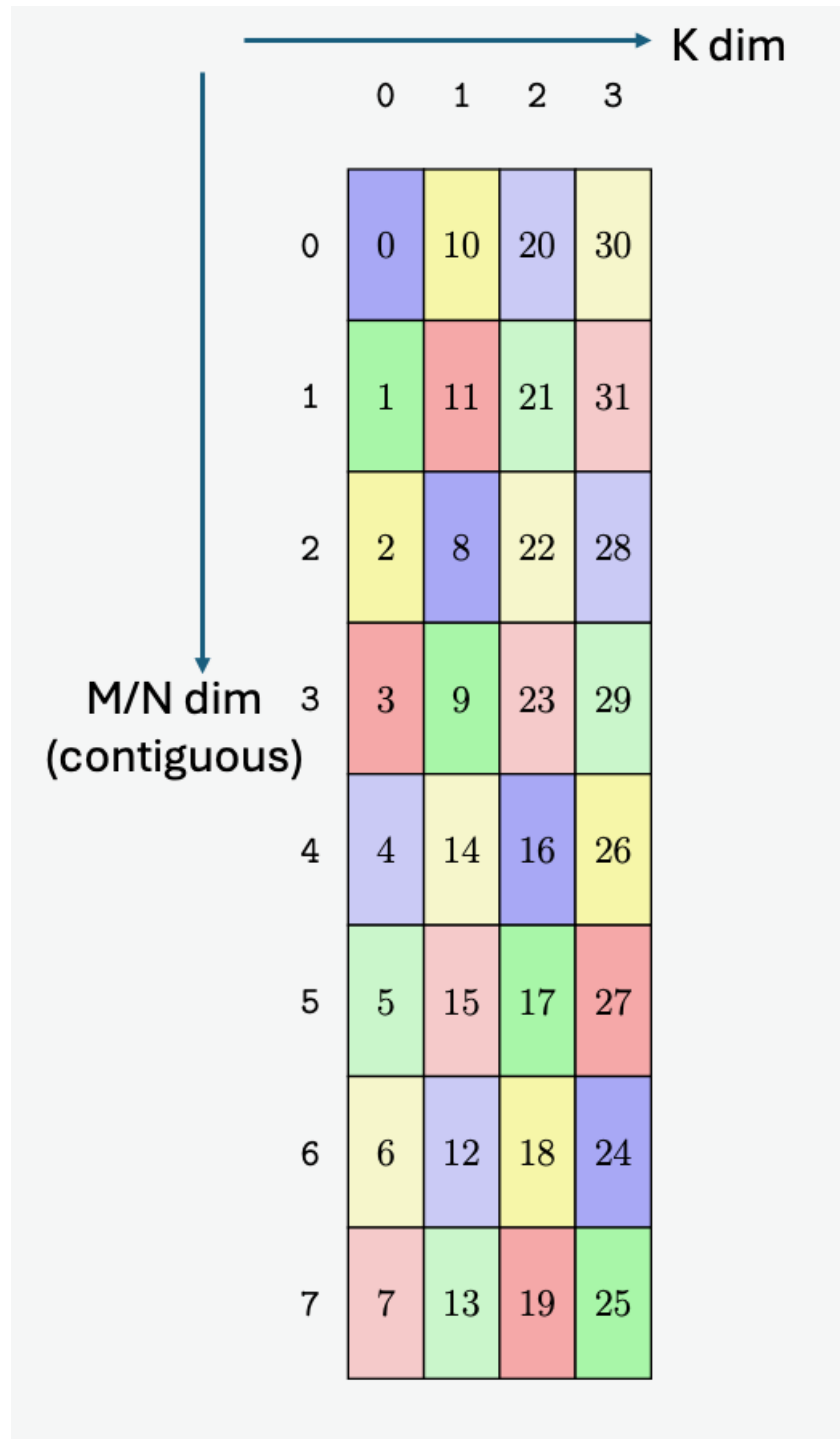


Figure 219: MN major 128B swizzling with 32B atomicity

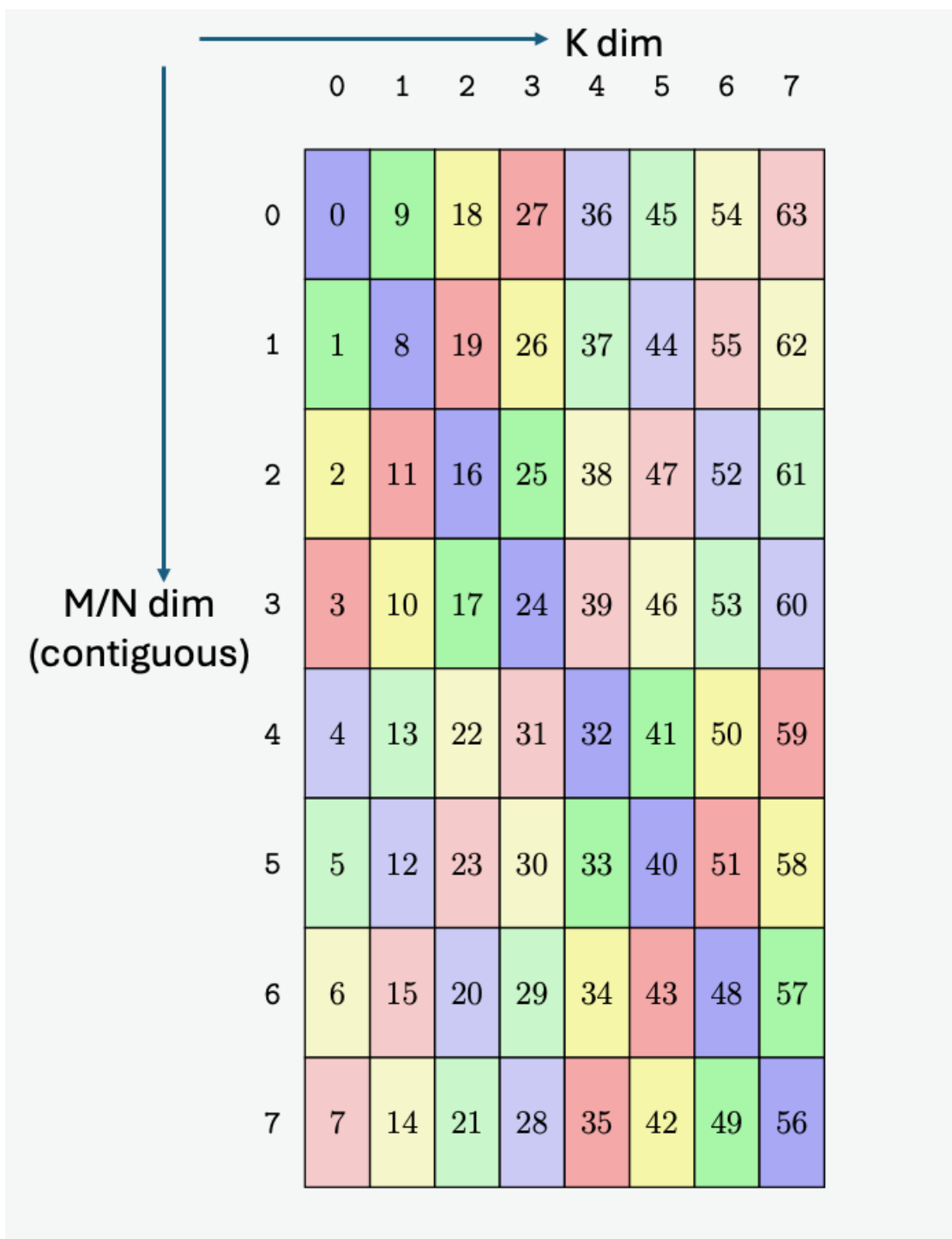


Figure 220: MN major 128B swizzling

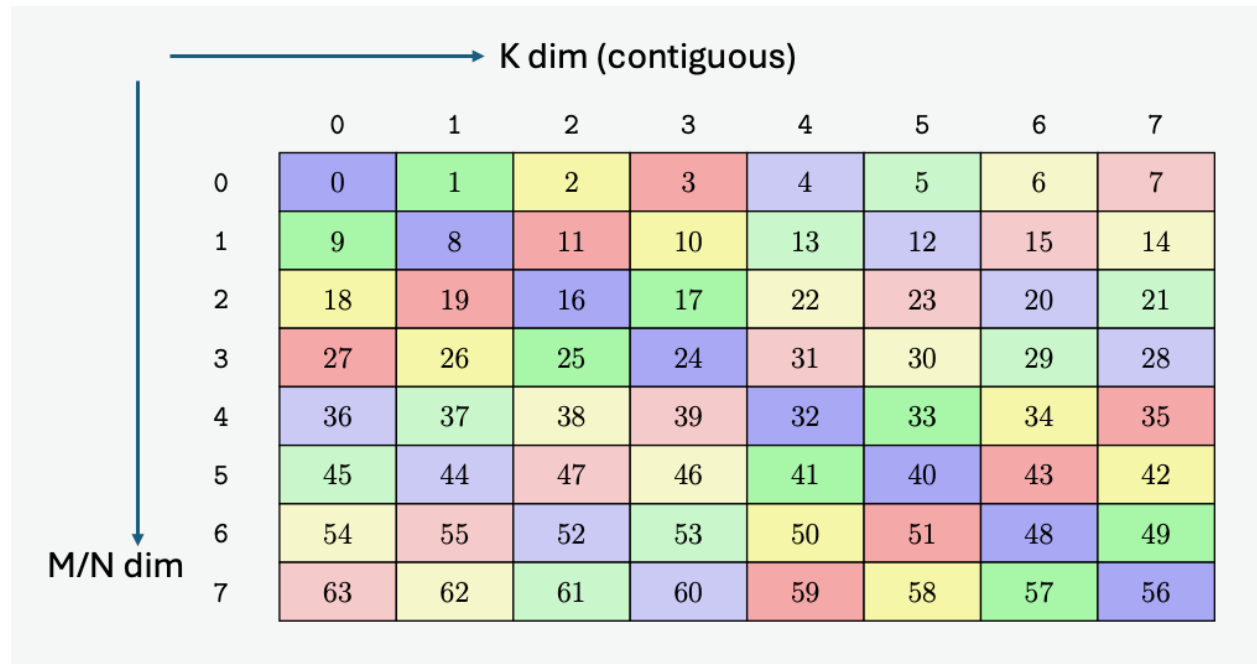


Figure 221: K major 128B swizzling

► MN-Major: [Figure 229](#)

9.7.16.10.7 Block Scaling for `tcgen05.mma.sync`

The `tcgen05.mma` instructions with the following `.kind` qualifier:

- `.kind::mxf8f6f4`
- `.kind::mxf4`
- `.kind::mxf4nvf4`

perform matrix multiplication with block scaling. This operation has the following form:

$$(A * \text{scale_A}) * (B * \text{scale_B}) + D$$

where `scale_A` and `scale_B` are matrices residing in *Tensor Memory*.

For a `scale_A` matrix of shape $M \times SFA_N$, each row of matrix A is divided into SFA_N number of chunks and each chunk of a row is multiplied with the corresponding element in the SF_A of the same row.

Similarly, for a `scale_B` matrix of shape $SFB_M \times N$, each column of matrix B is divided into the SFB_M number of chunks and each chunk of a column is multiplied with the corresponding element in the SF_B of the same column.

Scale factors for A and B matrices need to be duplicated to all 32 lane partitions of tensor memory.

[Figure 230](#) shows an example of `tcgen05.mma` with block scaling of `scale_vec::2X`.

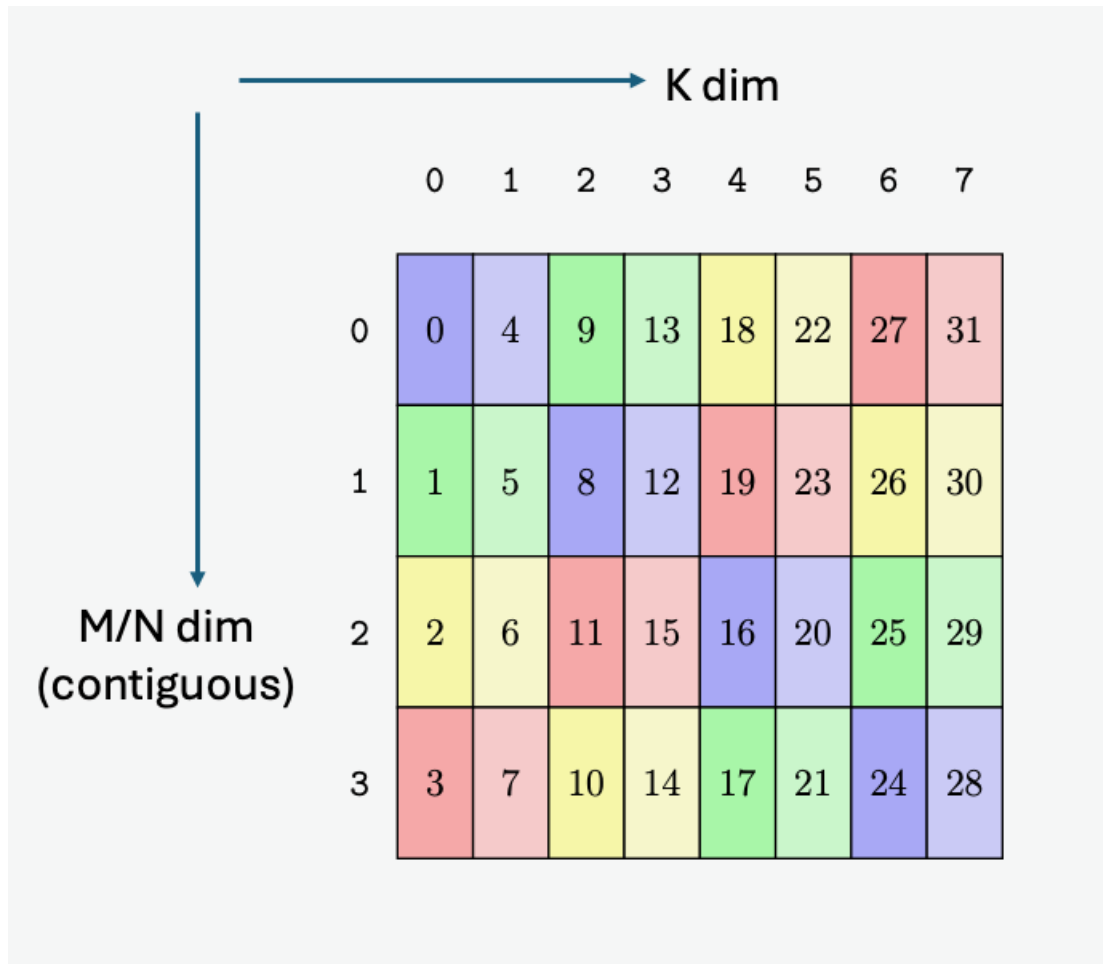


Figure 222: MN major 64B swizzling

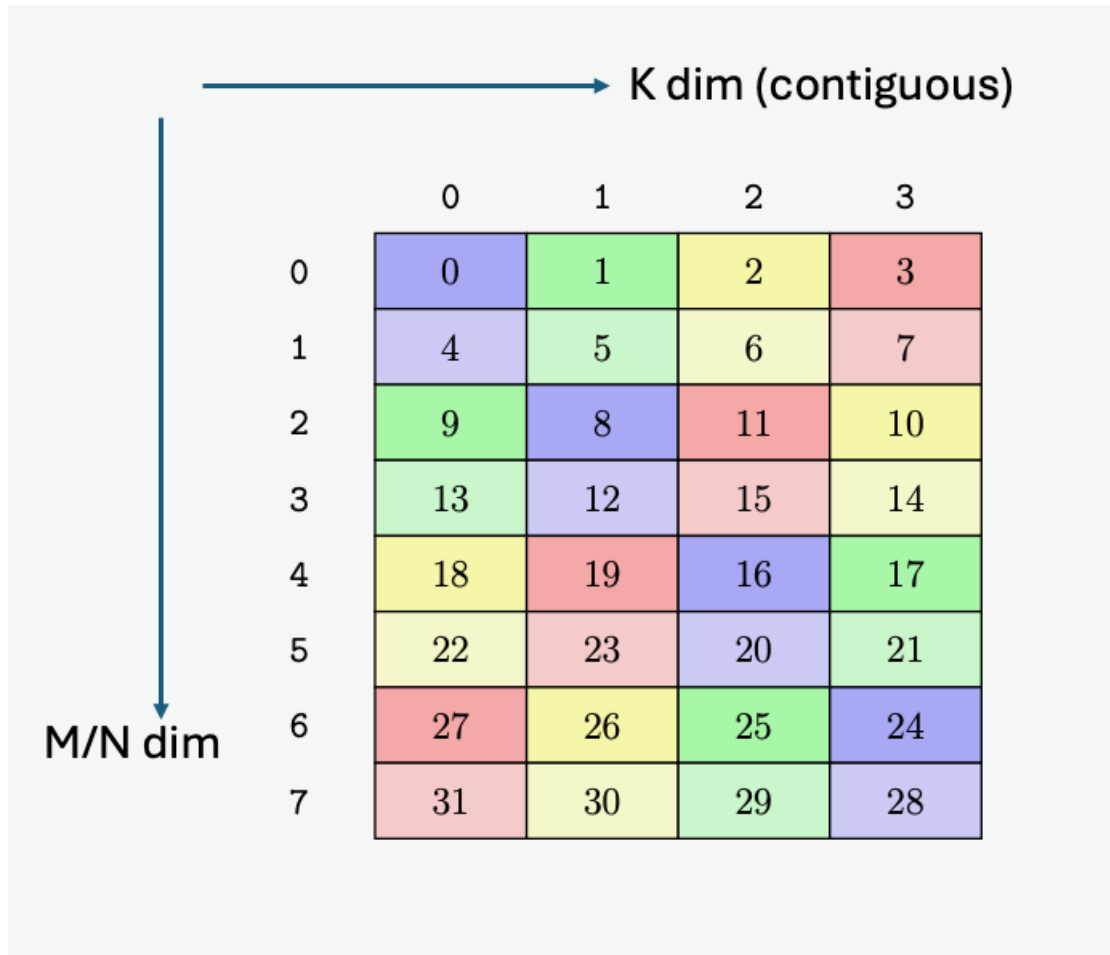


Figure 223: K major 64B swizzling

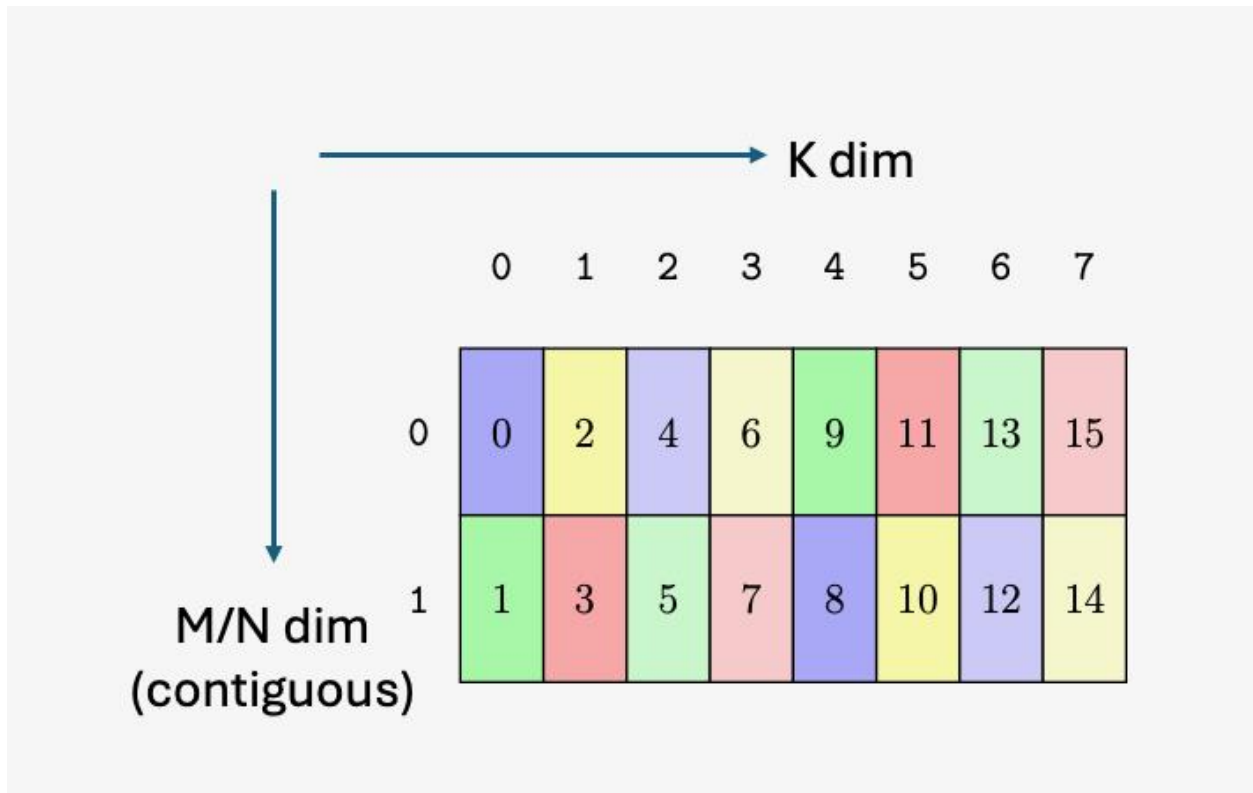


Figure 224: MN major 32B swizzling

9.7.16.10.7.1 Valid combinations of `scale_vectorsize` with types and MMA-Kind

The shape of `scale_A` and `scale_B` matrices depend on the `.scale_vectorsize` as shown in Table 58.

Table 58: Valid combinations of `scale_vectorsize` and shapes

<code>.scale_vectorsize</code>	<code>.kind::*</code>	K	Shape of <code>scale_A</code>	Shape of <code>scale_B</code>
<code>.scale_vec::1X</code>	<code>.kind::mxf8f6f4</code>	All supported values of K	M x 1	1 x N
<code>.scale_vec::2X</code>	<code>.kind::mxf4, .kind::mxf4nvf4</code>	All supported values of K	M x 2	2 x N
<code>.scale_vec::4X</code>	<code>.kind::mxf4nvf4</code>	All supported values of K	M x 4	4 x N
<code>.block16</code>	<code>.kind::mxf4nvf4</code>	K = 96	M x 6	6 x N
		All supported values of K except 96	M x 4	4 x N
<code>.block32</code>	<code>.kind::mxf4, .kind::mxf4nvf4</code>	K = 96	M x 3	3 x N
		All supported values of K except 96	M x 2	2 x N
	<code>.kind::mxf8f6f4</code>	All supported values of K	M x 1	1 x N

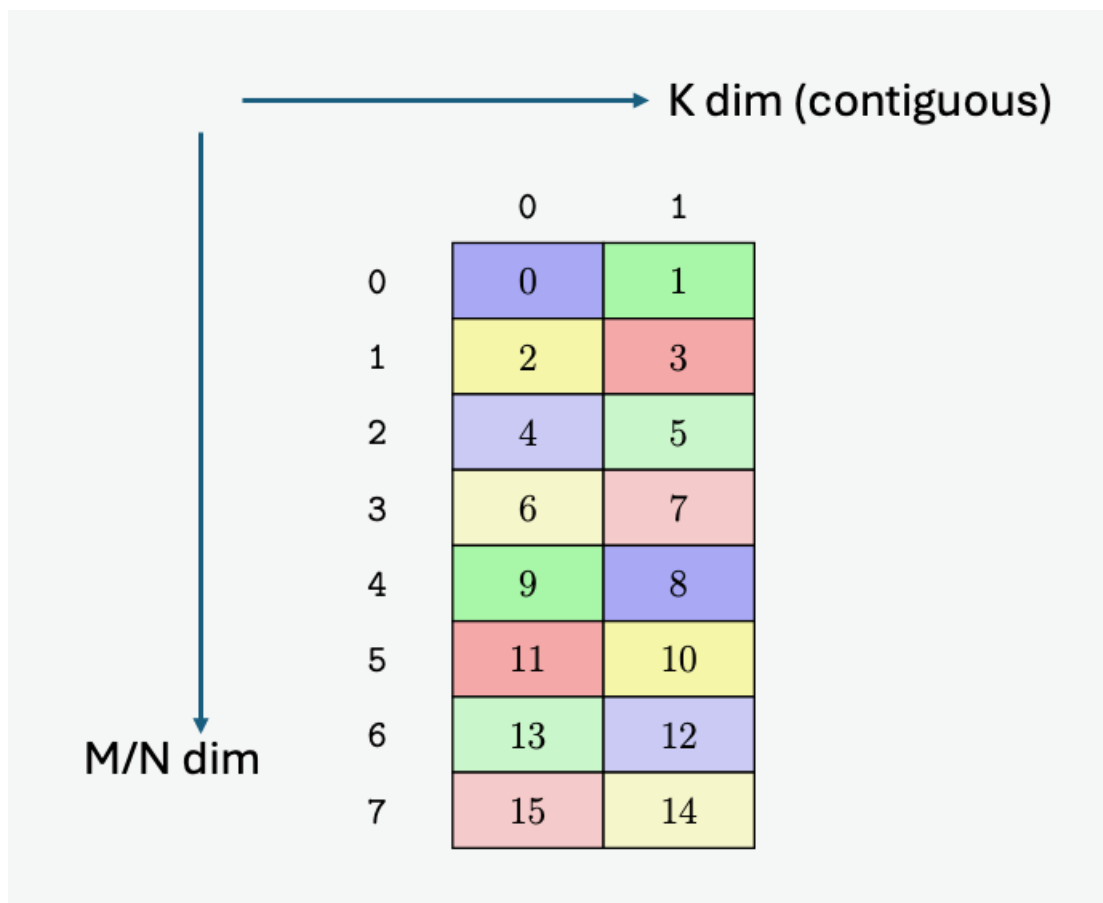


Figure 225: K major 32B swizzling

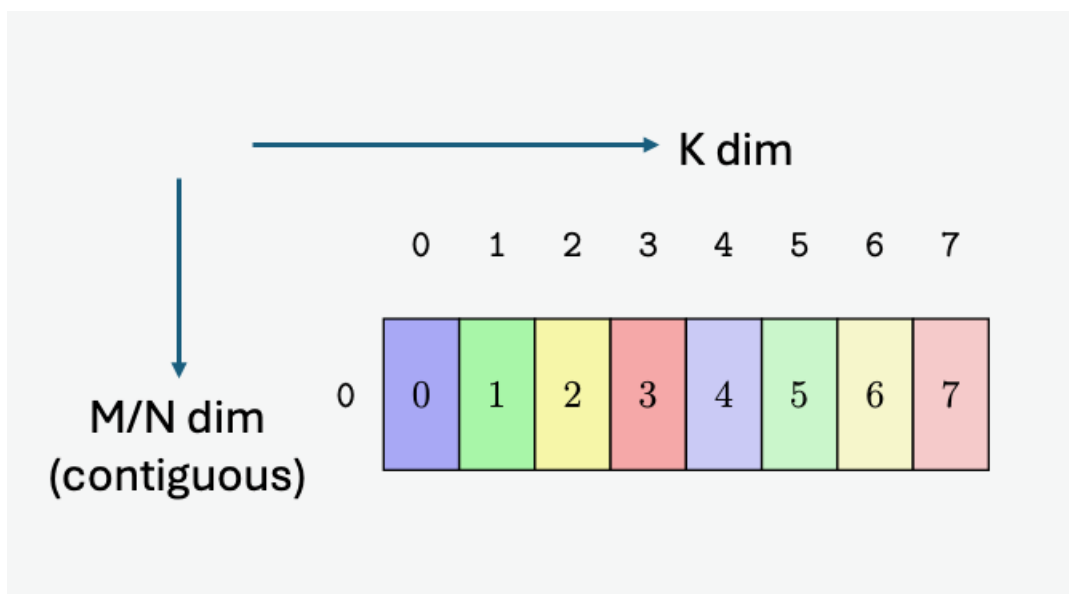


Figure 226: MN major no-swizzling mode

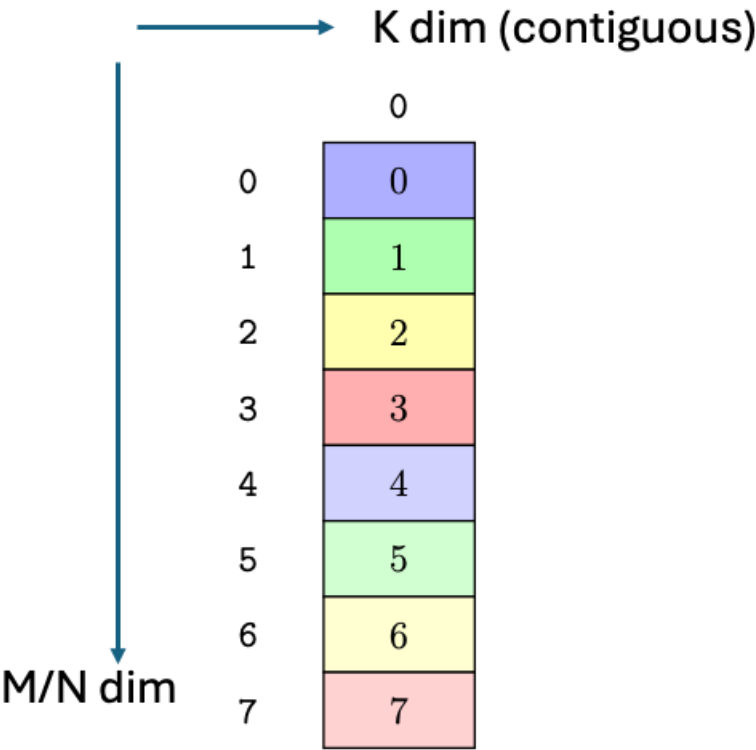


Figure 227: K major no-swizzling mode

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67
2	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103
3	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139
4	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
5	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211
6	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247
7	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283

Figure 228: K major

	0	1	2	3	4	5	6	7
0	0	36	72	108	144	180	216	252
1	1	37	73	109	145	181	217	253
2	2	38	74	110	146	182	218	254
3	3	39	75	111	147	183	219	255
4	4	32	76	104	148	176	220	248
5	5	33	77	105	149	177	221	249
6	6	34	78	106	150	178	222	250
7	7	35	79	107	151	179	223	251
8	8	44	64	100	152	188	208	244
9	9	45	65	101	153	189	209	245
10	10	46	66	102	154	190	210	246
11	11	47	67	103	155	191	211	247
12	12	40	68	96	156	184	212	240
13	13	41	69	97	157	185	213	241
14	14	42	70	98	158	186	214	242
15	15	43	71	99	159	187	215	243
16	16	52	88	124	128	164	200	236
17	17	53	89	125	129	165	201	237
18	18	54	90	126	130	166	202	238
19	19	55	91	127	131	167	203	239
20	20	48	92	120	132	160	204	232
21	21	49	93	121	133	161	205	233
22	22	50	94	122	134	162	206	234
23	23	51	95	123	135	163	207	235
24	24	60	80	116	136	172	192	228
25	25	61	81	117	137	173	193	229
26	26	62	82	118	138	174	194	230
27	27	63	83	119	139	175	195	231
28	28	56	84	112	140	168	196	224
29	29	57	85	113	141	169	197	225
30	30	58	86	114	142	170	198	226
31	31	59	87	115	143	171	199	227

Figure 229: MN major

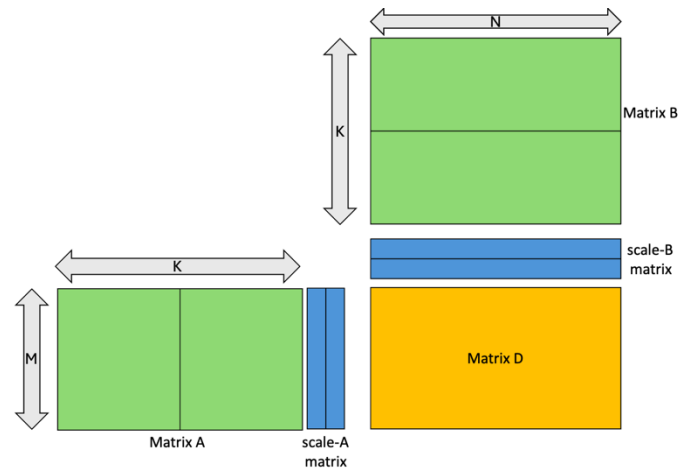


Figure 230: tcgen05.mma with block scaling of scale_vec::2X

The valid combination of the exact element types and the `.scale_vectorsize` are listed in Table 59.

Table 59: Valid combinations of scale_vectorsize with types and MMA-Kind

<code>.kind::*</code>	Element Data Type	Scale Data Type	<code>.scale_vectorsize</code>
<code>.kind::mxfb64</code>	E4M3, E5M2, E2M3	UE8M0	<code>.scale_vec::1X / .block32</code>
<code>.kind::mxfb32</code>	E4M3, E5M2, E2M1	UE8M0	<code>.scale_vec::1X / .block32</code>
<code>.kind::mxvf4</code>	E2M1	UE8M0	<code>.scale_vec::2X / .block32</code>
<code>.kind::mxvf4nvf4</code>	E2M1	UE8M0	<code>.scale_vec::2X / .block32</code> , <code>.scale_vec::4X / .block16</code>
	E2M1	UE4M3	<code>.scale_vec::4X / .block16</code>

New `.blockN` qualifiers are aliases for `.scale_vec::NX` qualifiers as:

- `.block32` is alias for `.scale_vec::1X` or `.scale_vec::2X` based on `.kind` and `K` dimension
- `.block16` is alias for `.scale_vec::4X`

9.7.16.10.7.2 Scale Factor A ID

The value of the scale factor A ID selects the sub-columns in the Tensor Memory to form the scale factor A matrix, which is used to scale the matrix A.

The following shows the scale factor matrix layout for various scale vector sizes:

9.7.16.10.7.3 Layout of the Scale Factor A Matrix for scale_vec::1X/block32 with K=32/K=64

There is one scale factor per row of the A matrix with block size as 32 and the scale factor must be provided in 1-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 231 shows which sub-columns get selected for different values of *SFA_ID*.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	M0 SFO	M0 SFO	M0 SFO	M0 SFO	M32 SFO	M32 SFO	M32 SFO	M32 SFO	M64 SFO	M64 SFO	M64 SFO	M64 SFO	M96 SFO	M96 SFO	M96 SFO	M96 SFO
1	M1 SFO	M1 SFO	M1 SFO	M1 SFO	M33 SFO	M33 SFO	M33 SFO	M33 SFO	M65 SFO	M65 SFO	M65 SFO	M65 SFO	M97 SFO	M97 SFO	M97 SFO	M97 SFO
2	M2 SFO	M2 SFO	M2 SFO	M2 SFO	M34 SFO	M34 SFO	M34 SFO	M34 SFO	M66 SFO	M66 SFO	M66 SFO	M66 SFO	M98 SFO	M98 SFO	M98 SFO	M98 SFO
3	M3 SFO	M3 SFO	M3 SFO	M3 SFO	M35 SFO	M35 SFO	M35 SFO	M35 SFO	M67 SFO	M67 SFO	M67 SFO	M67 SFO	M99 SFO	M99 SFO	M99 SFO	M99 SFO
4	M4 SFO	M4 SFO	M4 SFO	M4 SFO	M36 SFO	M36 SFO	M36 SFO	M36 SFO	M68 SFO	M68 SFO	M68 SFO	M68 SFO	M100 SFO	M100 SFO	M100 SFO	M100 SFO
5	M5 SFO	M5 SFO	M5 SFO	M5 SFO	M37 SFO	M37 SFO	M37 SFO	M37 SFO	M69 SFO	M69 SFO	M69 SFO	M69 SFO	M101 SFO	M101 SFO	M101 SFO	M101 SFO
6	M6 SFO	M6 SFO	M6 SFO	M6 SFO	M38 SFO	M38 SFO	M38 SFO	M38 SFO	M70 SFO	M70 SFO	M70 SFO	M70 SFO	M102 SFO	M102 SFO	M102 SFO	M102 SFO
7	M7 SFO	M7 SFO	M7 SFO	M7 SFO	M39 SFO	M39 SFO	M39 SFO	M39 SFO	M71 SFO	M71 SFO	M71 SFO	M71 SFO	M103 SFO	M103 SFO	M103 SFO	M103 SFO
8	M8 SFO	M8 SFO	M8 SFO	M8 SFO	M40 SFO	M40 SFO	M40 SFO	M40 SFO	M72 SFO	M72 SFO	M72 SFO	M72 SFO	M104 SFO	M104 SFO	M104 SFO	M104 SFO
9	M9 SFO	M9 SFO	M9 SFO	M9 SFO	M41 SFO	M41 SFO	M41 SFO	M41 SFO	M73 SFO	M73 SFO	M73 SFO	M73 SFO	M105 SFO	M105 SFO	M105 SFO	M105 SFO
10	M10 SFO	M10 SFO	M10 SFO	M10 SFO	M42 SFO	M42 SFO	M42 SFO	M42 SFO	M74 SFO	M74 SFO	M74 SFO	M74 SFO	M106 SFO	M106 SFO	M106 SFO	M106 SFO
11	M11 SFO	M11 SFO	M11 SFO	M11 SFO	M43 SFO	M43 SFO	M43 SFO	M43 SFO	M75 SFO	M75 SFO	M75 SFO	M75 SFO	M107 SFO	M107 SFO	M107 SFO	M107 SFO
12	M12 SFO	M12 SFO	M12 SFO	M12 SFO	M44 SFO	M44 SFO	M44 SFO	M44 SFO	M76 SFO	M76 SFO	M76 SFO	M76 SFO	M108 SFO	M108 SFO	M108 SFO	M108 SFO
13	M13 SFO	M13 SFO	M13 SFO	M13 SFO	M45 SFO	M45 SFO	M45 SFO	M45 SFO	M77 SFO	M77 SFO	M77 SFO	M77 SFO	M109 SFO	M109 SFO	M109 SFO	M109 SFO
14	M14 SFO	M14 SFO	M14 SFO	M14 SFO	M46 SFO	M46 SFO	M46 SFO	M46 SFO	M78 SFO	M78 SFO	M78 SFO	M78 SFO	M110 SFO	M110 SFO	M110 SFO	M110 SFO
15	M15 SFO	M15 SFO	M15 SFO	M15 SFO	M47 SFO	M47 SFO	M47 SFO	M47 SFO	M79 SFO	M79 SFO	M79 SFO	M79 SFO	M111 SFO	M111 SFO	M111 SFO	M111 SFO
16	M16 SFO	M16 SFO	M16 SFO	M16 SFO	M48 SFO	M48 SFO	M48 SFO	M48 SFO	M80 SFO	M80 SFO	M80 SFO	M80 SFO	M112 SFO	M112 SFO	M112 SFO	M112 SFO
17	M17 SFO	M17 SFO	M17 SFO	M17 SFO	M49 SFO	M49 SFO	M49 SFO	M49 SFO	M81 SFO	M81 SFO	M81 SFO	M81 SFO	M113 SFO	M113 SFO	M113 SFO	M113 SFO
18	M18 SFO	M18 SFO	M18 SFO	M18 SFO	M50 SFO	M50 SFO	M50 SFO	M50 SFO	M82 SFO	M82 SFO	M82 SFO	M82 SFO	M114 SFO	M114 SFO	M114 SFO	M114 SFO
19	M19 SFO	M19 SFO	M19 SFO	M19 SFO	M51 SFO	M51 SFO	M51 SFO	M51 SFO	M83 SFO	M83 SFO	M83 SFO	M83 SFO	M115 SFO	M115 SFO	M115 SFO	M115 SFO
20	M20 SFO	M20 SFO	M20 SFO	M20 SFO	M52 SFO	M52 SFO	M52 SFO	M52 SFO	M84 SFO	M84 SFO	M84 SFO	M84 SFO	M116 SFO	M116 SFO	M116 SFO	M116 SFO
21	M21 SFO	M21 SFO	M21 SFO	M21 SFO	M53 SFO	M53 SFO	M53 SFO	M53 SFO	M85 SFO	M85 SFO	M85 SFO	M85 SFO	M117 SFO	M117 SFO	M117 SFO	M117 SFO
22	M22 SFO	M22 SFO	M22 SFO	M22 SFO	M54 SFO	M54 SFO	M54 SFO	M54 SFO	M86 SFO	M86 SFO	M86 SFO	M86 SFO	M118 SFO	M118 SFO	M118 SFO	M118 SFO
23	M23 SFO	M23 SFO	M23 SFO	M23 SFO	M55 SFO	M55 SFO	M55 SFO	M55 SFO	M87 SFO	M87 SFO	M87 SFO	M87 SFO	M119 SFO	M119 SFO	M119 SFO	M119 SFO
24	M24 SFO	M24 SFO	M24 SFO	M24 SFO	M56 SFO	M56 SFO	M56 SFO	M56 SFO	M88 SFO	M88 SFO	M88 SFO	M88 SFO	M120 SFO	M120 SFO	M120 SFO	M120 SFO
25	M25 SFO	M25 SFO	M25 SFO	M25 SFO	M57 SFO	M57 SFO	M57 SFO	M57 SFO	M89 SFO	M89 SFO	M89 SFO	M89 SFO	M121 SFO	M121 SFO	M121 SFO	M121 SFO
26	M26 SFO	M26 SFO	M26 SFO	M26 SFO	M58 SFO	M58 SFO	M58 SFO	M58 SFO	M90 SFO	M90 SFO	M90 SFO	M90 SFO	M122 SFO	M122 SFO	M122 SFO	M122 SFO
27	M27 SFO	M27 SFO	M27 SFO	M27 SFO	M59 SFO	M59 SFO	M59 SFO	M59 SFO	M91 SFO	M91 SFO	M91 SFO	M91 SFO	M123 SFO	M123 SFO	M123 SFO	M123 SFO
28	M28 SFO	M28 SFO	M28 SFO	M28 SFO	M60 SFO	M60 SFO	M60 SFO	M60 SFO	M92 SFO	M92 SFO	M92 SFO	M92 SFO	M124 SFO	M124 SFO	M124 SFO	M124 SFO
29	M29 SFO	M29 SFO	M29 SFO	M29 SFO	M61 SFO	M61 SFO	M61 SFO	M61 SFO	M93 SFO	M93 SFO	M93 SFO	M93 SFO	M125 SFO	M125 SFO	M125 SFO	M125 SFO
30	M30 SFO	M30 SFO	M30 SFO	M30 SFO	M62 SFO	M62 SFO	M62 SFO	M62 SFO	M94 SFO	M94 SFO	M94 SFO	M94 SFO	M126 SFO	M126 SFO	M126 SFO	M126 SFO
31	M31 SFO	M31 SFO	M31 SFO	M31 SFO	M63 SFO	M63 SFO	M63 SFO	M63 SFO	M95 SFO	M95 SFO	M95 SFO	M95 SFO	M127 SFO	M127 SFO	M127 SFO	M127 SFO

SFA ID = 00
SFA ID = 01
SFA ID = 10
SFA ID = 11

Figure 231: Layout of scale factor A matrix with scale_vec::1X/block32 with K=32/K=64

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFA_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

9.7.16.10.7.4 Layout of the Scale Factor A Matrix for scale_vec::2X/block32 with K=64/K=128

There are two scale factors per row of the A matrix with block size as 32 and the scale factor must be provided in 2-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the half word offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 232 shows which sub-columns gets selected for different values of *SFA_ID*.

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFA_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	M0 SF0	M0 SF1	M0 SF0	M0 SF1	M32 SF0	M32 SF1	M32 SF0	M32 SF1	M64 SF0	M64 SF1	M64 SF0	M64 SF1	M96 SF0	M96 SF1	M96 SF0	M96 SF1
1	M1 SF0	M1 SF1	M1 SF0	M1 SF1	M33 SF0	M33 SF1	M33 SF0	M33 SF1	M65 SF0	M65 SF1	M65 SF0	M65 SF1	M97 SF0	M97 SF1	M97 SF0	M97 SF1
2	M2 SF0	M2 SF1	M2 SF0	M2 SF1	M34 SF0	M34 SF1	M34 SF0	M34 SF1	M66 SF0	M66 SF1	M66 SF0	M66 SF1	M98 SF0	M98 SF1	M98 SF0	M98 SF1
3	M3 SF0	M3 SF1	M3 SF0	M3 SF1	M35 SF0	M35 SF1	M35 SF0	M35 SF1	M67 SF0	M67 SF1	M67 SF0	M67 SF1	M99 SF0	M99 SF1	M99 SF0	M99 SF1
4	M4 SF0	M4 SF1	M4 SF0	M4 SF1	M36 SF0	M36 SF1	M36 SF0	M36 SF1	M68 SF0	M68 SF1	M68 SF0	M68 SF1	M100 SF0	M100 SF1	M100 SF0	M100 SF1
5	M5 SF0	M5 SF1	M5 SF0	M5 SF1	M37 SF0	M37 SF1	M37 SF0	M37 SF1	M69 SF0	M69 SF1	M69 SF0	M69 SF1	M101 SF0	M101 SF1	M101 SF0	M101 SF1
6	M6 SF0	M6 SF1	M6 SF0	M6 SF1	M38 SF0	M38 SF1	M38 SF0	M38 SF1	M70 SF0	M70 SF1	M70 SF0	M70 SF1	M102 SF0	M102 SF1	M102 SF0	M102 SF1
7	M7 SF0	M7 SF1	M7 SF0	M7 SF1	M39 SF0	M39 SF1	M39 SF0	M39 SF1	M71 SF0	M71 SF1	M71 SF0	M71 SF1	M103 SF0	M103 SF1	M103 SF0	M103 SF1
8	M8 SF0	M8 SF1	M8 SF0	M8 SF1	M40 SF0	M40 SF1	M40 SF0	M40 SF1	M72 SF0	M72 SF1	M72 SF0	M72 SF1	M104 SF0	M104 SF1	M104 SF0	M104 SF1
9	M9 SF0	M9 SF1	M9 SF0	M9 SF1	M41 SF0	M41 SF1	M41 SF0	M41 SF1	M73 SF0	M73 SF1	M73 SF0	M73 SF1	M105 SF0	M105 SF1	M105 SF0	M105 SF1
10	M10 SF0	M10 SF1	M10 SF0	M10 SF1	M42 SF0	M42 SF1	M42 SF0	M42 SF1	M74 SF0	M74 SF1	M74 SF0	M74 SF1	M106 SF0	M106 SF1	M106 SF0	M106 SF1
11	M11 SF0	M11 SF1	M11 SF0	M11 SF1	M43 SF0	M43 SF1	M43 SF0	M43 SF1	M75 SF0	M75 SF1	M75 SF0	M75 SF1	M107 SF0	M107 SF1	M107 SF0	M107 SF1
12	M12 SF0	M12 SF1	M12 SF0	M12 SF1	M44 SF0	M44 SF1	M44 SF0	M44 SF1	M76 SF0	M76 SF1	M76 SF0	M76 SF1	M108 SF0	M108 SF1	M108 SF0	M108 SF1
13	M13 SF0	M13 SF1	M13 SF0	M13 SF1	M45 SF0	M45 SF1	M45 SF0	M45 SF1	M77 SF0	M77 SF1	M77 SF0	M77 SF1	M109 SF0	M109 SF1	M109 SF0	M109 SF1
14	M14 SF0	M14 SF1	M14 SF0	M14 SF1	M46 SF0	M46 SF1	M46 SF0	M46 SF1	M78 SF0	M78 SF1	M78 SF0	M78 SF1	M110 SF0	M110 SF1	M110 SF0	M110 SF1
15	M15 SF0	M15 SF1	M15 SF0	M15 SF1	M47 SF0	M47 SF1	M47 SF0	M47 SF1	M79 SF0	M79 SF1	M79 SF0	M79 SF1	M111 SF0	M111 SF1	M111 SF0	M111 SF1
16	M16 SF0	M16 SF1	M16 SF0	M16 SF1	M48 SF0	M48 SF1	M48 SF0	M48 SF1	M80 SF0	M80 SF1	M80 SF0	M80 SF1	M112 SF0	M112 SF1	M112 SF0	M112 SF1
17	M17 SF0	M17 SF1	M17 SF0	M17 SF1	M49 SF0	M49 SF1	M49 SF0	M49 SF1	M81 SF0	M81 SF1	M81 SF0	M81 SF1	M113 SF0	M113 SF1	M113 SF0	M113 SF1
18	M18 SF0	M18 SF1	M18 SF0	M18 SF1	M50 SF0	M50 SF1	M50 SF0	M50 SF1	M82 SF0	M82 SF1	M82 SF0	M82 SF1	M114 SF0	M114 SF1	M114 SF0	M114 SF1
19	M19 SF0	M19 SF1	M19 SF0	M19 SF1	M51 SF0	M51 SF1	M51 SF0	M51 SF1	M83 SF0	M83 SF1	M83 SF0	M83 SF1	M115 SF0	M115 SF1	M115 SF0	M115 SF1
20	M20 SF0	M20 SF1	M20 SF0	M20 SF1	M52 SF0	M52 SF1	M52 SF0	M52 SF1	M84 SF0	M84 SF1	M84 SF0	M84 SF1	M116 SF0	M116 SF1	M116 SF0	M116 SF1
21	M21 SF0	M21 SF1	M21 SF0	M21 SF1	M53 SF0	M53 SF1	M53 SF0	M53 SF1	M85 SF0	M85 SF1	M85 SF0	M85 SF1	M117 SF0	M117 SF1	M117 SF0	M117 SF1
22	M22 SF0	M22 SF1	M22 SF0	M22 SF1	M54 SF0	M54 SF1	M54 SF0	M54 SF1	M86 SF0	M86 SF1	M86 SF0	M86 SF1	M118 SF0	M118 SF1	M118 SF0	M118 SF1
23	M23 SF0	M23 SF1	M23 SF0	M23 SF1	M55 SF0	M55 SF1	M55 SF0	M55 SF1	M87 SF0	M87 SF1	M87 SF0	M87 SF1	M119 SF0	M119 SF1	M119 SF0	M119 SF1
24	M24 SF0	M24 SF1	M24 SF0	M24 SF1	M56 SF0	M56 SF1	M56 SF0	M56 SF1	M88 SF0	M88 SF1	M88 SF0	M88 SF1	M120 SF0	M120 SF1	M120 SF0	M120 SF1
25	M25 SF0	M25 SF1	M25 SF0	M25 SF1	M57 SF0	M57 SF1	M57 SF0	M57 SF1	M89 SF0	M89 SF1	M89 SF0	M89 SF1	M121 SF0	M121 SF1	M121 SF0	M121 SF1
26	M26 SF0	M26 SF1	M26 SF0	M26 SF1	M58 SF0	M58 SF1	M58 SF0	M58 SF1	M90 SF0	M90 SF1	M90 SF0	M90 SF1	M122 SF0	M122 SF1	M122 SF0	M122 SF1
27	M27 SF0	M27 SF1	M27 SF0	M27 SF1	M59 SF0	M59 SF1	M59 SF0	M59 SF1	M91 SF0	M91 SF1	M91 SF0	M91 SF1	M123 SF0	M123 SF1	M123 SF0	M123 SF1
28	M28 SF0	M28 SF1	M28 SF0	M28 SF1	M60 SF0	M60 SF1	M60 SF0	M60 SF1	M92 SF0	M92 SF1	M92 SF0	M92 SF1	M124 SF0	M124 SF1	M124 SF0	M124 SF1
29	M29 SF0	M29 SF1	M29 SF0	M29 SF1	M61 SF0	M61 SF1	M61 SF0	M61 SF1	M93 SF0	M93 SF1	M93 SF0	M93 SF1	M125 SF0	M125 SF1	M125 SF0	M125 SF1
30	M30 SF0	M30 SF1	M30 SF0	M30 SF1	M62 SF0	M62 SF1	M62 SF0	M62 SF1	M94 SF0	M94 SF1	M94 SF0	M94 SF1	M126 SF0	M126 SF1	M126 SF0	M126 SF1
31	M31 SF0	M31 SF1	M31 SF0	M31 SF1	M63 SF0	M63 SF1	M63 SF0	M63 SF1	M95 SF0	M95 SF1	M95 SF0	M95 SF1	M127 SF0	M127 SF1	M127 SF0	M127 SF1

SFA_ID = 00

SFA_ID = 10

Figure 232: Layout of scale factor A matrix with scale_vec::2X/block32 with K=64/K=128

9.7.16.10.7.5 Layout of the Scale Factor A Matrix for scale_vec::4X/block16 with K=64/K=128

There are four scale factors per row of the A matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. The *SFA_ID* value must be 0 and this specifies that all of the columns (in green) will be used for the scale factor matrix. Figure 233 shows which sub-columns gets selected for different values of *SFA_ID*.

9.7.16.10.7.6 Layout of the Scale Factor A Matrix for block32 with K=96 (Semantically equivalent to scale_vec::3X)

There are three scale factors per row of the A matrix with block size as 32 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 234, Figure 235, Figure 236 and Figure 237 show which sub-columns get selected for different values of *SFA_ID*.

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFA_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

9.7.16.10.7.7 Layout of the Scale Factor A Matrix for block16 with K=96 (Semantically equivalent to scale_vec::6X)

There are six scale factors per row of the A matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 238 and Figure 239 show which sub-columns get selected for different values of *SFA_ID*.

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFA_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

Column X				Column X+1				Column X+2				Column X+3				
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
Lane 0	M0 SF0	M0 SF1	M0 SF2	M0 SF3	M32 SF0	M32 SF1	M32 SF2	M32 SF3	M64 SF0	M64 SF1	M64 SF2	M64 SF3	M96 SF0	M96 SF1	M96 SF2	M96 SF3
1	M1 SF0	M1 SF1	M1 SF2	M1 SF3	M33 SF0	M33 SF1	M33 SF2	M33 SF3	M65 SF0	M65 SF1	M65 SF2	M65 SF3	M97 SF0	M97 SF1	M97 SF2	M97 SF3
2	M2 SF0	M2 SF1	M2 SF2	M2 SF3	M34 SF0	M34 SF1	M34 SF2	M34 SF3	M66 SF0	M66 SF1	M66 SF2	M66 SF3	M98 SF0	M98 SF1	M98 SF2	M98 SF3
3	M3 SF0	M3 SF1	M3 SF2	M3 SF3	M35 SF0	M35 SF1	M35 SF2	M35 SF3	M67 SF0	M67 SF1	M67 SF2	M67 SF3	M99 SF0	M99 SF1	M99 SF2	M99 SF3
4	M4 SF0	M4 SF1	M4 SF2	M4 SF3	M36 SF0	M36 SF1	M36 SF2	M36 SF3	M68 SF0	M68 SF1	M68 SF2	M68 SF3	M100 SF0	M100 SF1	M100 SF2	M100 SF3
5	M5 SF0	M5 SF1	M5 SF2	M5 SF3	M37 SF0	M37 SF1	M37 SF2	M37 SF3	M69 SF0	M69 SF1	M69 SF2	M69 SF3	M101 SF0	M101 SF1	M101 SF2	M101 SF3
6	M6 SF0	M6 SF1	M6 SF2	M6 SF3	M38 SF0	M38 SF1	M38 SF2	M38 SF3	M70 SF0	M70 SF1	M70 SF2	M70 SF3	M102 SF0	M102 SF1	M102 SF2	M102 SF3
7	M7 SF0	M7 SF1	M7 SF2	M7 SF3	M39 SF0	M39 SF1	M39 SF2	M39 SF3	M71 SF0	M71 SF1	M71 SF2	M71 SF3	M103 SF0	M103 SF1	M103 SF2	M103 SF3
8	M8 SF0	M8 SF1	M8 SF2	M8 SF3	M40 SF0	M40 SF1	M40 SF2	M40 SF3	M72 SF0	M72 SF1	M72 SF2	M72 SF3	M104 SF0	M104 SF1	M104 SF2	M104 SF3
9	M9 SF0	M9 SF1	M9 SF2	M9 SF3	M41 SF0	M41 SF1	M41 SF2	M41 SF3	M73 SF0	M73 SF1	M73 SF2	M73 SF3	M105 SF0	M105 SF1	M105 SF2	M105 SF3
10	M10 SF0	M10 SF1	M10 SF2	M10 SF3	M42 SF0	M42 SF1	M42 SF2	M42 SF3	M74 SF0	M74 SF1	M74 SF2	M74 SF3	M106 SF0	M106 SF1	M106 SF2	M106 SF3
11	M11 SF0	M11 SF1	M11 SF2	M11 SF3	M43 SF0	M43 SF1	M43 SF2	M43 SF3	M75 SF0	M75 SF1	M75 SF2	M75 SF3	M107 SF0	M107 SF1	M107 SF2	M107 SF3
12	M12 SF0	M12 SF1	M12 SF2	M12 SF3	M44 SF0	M44 SF1	M44 SF2	M44 SF3	M76 SF0	M76 SF1	M76 SF2	M76 SF3	M108 SF0	M108 SF1	M108 SF2	M108 SF3
13	M13 SF0	M13 SF1	M13 SF2	M13 SF3	M45 SF0	M45 SF1	M45 SF2	M45 SF3	M77 SF0	M77 SF1	M77 SF2	M77 SF3	M109 SF0	M109 SF1	M109 SF2	M109 SF3
14	M14 SF0	M14 SF1	M14 SF2	M14 SF3	M46 SF0	M46 SF1	M46 SF2	M46 SF3	M78 SF0	M78 SF1	M78 SF2	M78 SF3	M110 SF0	M110 SF1	M110 SF2	M110 SF3
15	M15 SF0	M15 SF1	M15 SF2	M15 SF3	M47 SF0	M47 SF1	M47 SF2	M47 SF3	M79 SF0	M79 SF1	M79 SF2	M79 SF3	M111 SF0	M111 SF1	M111 SF2	M111 SF3
16	M16 SF0	M16 SF1	M16 SF2	M16 SF3	M48 SF0	M48 SF1	M48 SF2	M48 SF3	M80 SF0	M80 SF1	M80 SF2	M80 SF3	M112 SF0	M112 SF1	M112 SF2	M112 SF3
17	M17 SF0	M17 SF1	M17 SF2	M17 SF3	M49 SF0	M49 SF1	M49 SF2	M49 SF3	M81 SF0	M81 SF1	M81 SF2	M81 SF3	M113 SF0	M113 SF1	M113 SF2	M113 SF3
18	M18 SF0	M18 SF1	M18 SF2	M18 SF3	M50 SF0	M50 SF1	M50 SF2	M50 SF3	M82 SF0	M82 SF1	M82 SF2	M82 SF3	M114 SF0	M114 SF1	M114 SF2	M114 SF3
19	M19 SF0	M19 SF1	M19 SF2	M19 SF3	M51 SF0	M51 SF1	M51 SF2	M51 SF3	M83 SF0	M83 SF1	M83 SF2	M83 SF3	M115 SF0	M115 SF1	M115 SF2	M115 SF3
20	M20 SF0	M20 SF1	M20 SF2	M20 SF3	M52 SF0	M52 SF1	M52 SF2	M52 SF3	M84 SF0	M84 SF1	M84 SF2	M84 SF3	M116 SF0	M116 SF1	M116 SF2	M116 SF3
21	M21 SF0	M21 SF1	M21 SF2	M21 SF3	M53 SF0	M53 SF1	M53 SF2	M53 SF3	M85 SF0	M85 SF1	M85 SF2	M85 SF3	M117 SF0	M117 SF1	M117 SF2	M117 SF3
22	M22 SF0	M22 SF1	M22 SF2	M22 SF3	M54 SF0	M54 SF1	M54 SF2	M54 SF3	M86 SF0	M86 SF1	M86 SF2	M86 SF3	M118 SF0	M118 SF1	M118 SF2	M118 SF3
23	M23 SF0	M23 SF1	M23 SF2	M23 SF3	M55 SF0	M55 SF1	M55 SF2	M55 SF3	M87 SF0	M87 SF1	M87 SF2	M87 SF3	M119 SF0	M119 SF1	M119 SF2	M119 SF3
24	M24 SF0	M24 SF1	M24 SF2	M24 SF3	M56 SF0	M56 SF1	M56 SF2	M56 SF3	M88 SF0	M88 SF1	M88 SF2	M88 SF3	M120 SF0	M120 SF1	M120 SF2	M120 SF3
25	M25 SF0	M25 SF1	M25 SF2	M25 SF3	M57 SF0	M57 SF1	M57 SF2	M57 SF3	M89 SF0	M89 SF1	M89 SF2	M89 SF3	M121 SF0	M121 SF1	M121 SF2	M121 SF3
26	M26 SF0	M26 SF1	M26 SF2	M26 SF3	M58 SF0	M58 SF1	M58 SF2	M58 SF3	M90 SF0	M90 SF1	M90 SF2	M90 SF3	M122 SF0	M122 SF1	M122 SF2	M122 SF3
27	M27 SF0	M27 SF1	M27 SF2	M27 SF3	M59 SF0	M59 SF1	M59 SF2	M59 SF3	M91 SF0	M91 SF1	M91 SF2	M91 SF3	M123 SF0	M123 SF1	M123 SF2	M123 SF3
28	M28 SF0	M28 SF1	M28 SF2	M28 SF3	M60 SF0	M60 SF1	M60 SF2	M60 SF3	M92 SF0	M92 SF1	M92 SF2	M92 SF3	M124 SF0	M124 SF1	M124 SF2	M124 SF3
29	M29 SF0	M29 SF1	M29 SF2	M29 SF3	M61 SF0	M61 SF1	M61 SF2	M61 SF3	M93 SF0	M93 SF1	M93 SF2	M93 SF3	M125 SF0	M125 SF1	M125 SF2	M125 SF3
30	M30 SF0	M30 SF1	M30 SF2	M30 SF3	M62 SF0	M62 SF1	M62 SF2	M62 SF3	M94 SF0	M94 SF1	M94 SF2	M94 SF3	M126 SF0	M126 SF1	M126 SF2	M126 SF3
31	M31 SF0	M31 SF1	M31 SF2	M31 SF3	M63 SF0	M63 SF1	M63 SF2	M63 SF3	M95 SF0	M95 SF1	M95 SF2	M95 SF3	M127 SF0	M127 SF1	M127 SF2	M127 SF3
0	1	2	3		SFA ID = 00											

Figure 233: Layout of scale factor A matrix with scale_vec::4X/block16 with K=64/K=128

Column X				Column X+1				Column X+2				Column X+3				
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
Lane 0	M0 SFO	M0 SF1	M0 SF2		M32 SFO	M32 SF1	M32 SF2		M64 SFO	M64 SF1	M64 SF2		M96 SFO	M96 SF1	M96 SF2	
1	M1 SFO	M1 SF1	M1 SF2		M33 SFO	M33 SF1	M33 SF2		M65 SFO	M65 SF1	M65 SF2		M97 SFO	M97 SF1	M97 SF2	
2	M2 SFO	M2 SF1	M2 SF2		M34 SFO	M34 SF1	M34 SF2		M66 SFO	M66 SF1	M66 SF2		M98 SFO	M98 SF1	M98 SF2	
3	M3 SFO	M3 SF1	M3 SF2		M35 SFO	M35 SF1	M35 SF2		M67 SFO	M67 SF1	M67 SF2		M99 SFO	M99 SF1	M99 SF2	
4	M4 SFO	M4 SF1	M4 SF2		M36 SFO	M36 SF1	M36 SF2		M68 SFO	M68 SF1	M68 SF2		M100 SFO	M100 SF1	M100 SF2	
5	M5 SFO	M5 SF1	M5 SF2		M37 SFO	M37 SF1	M37 SF2		M69 SFO	M69 SF1	M69 SF2		M101 SFO	M101 SF1	M101 SF2	
6	M6 SFO	M6 SF1	M6 SF2		M38 SFO	M38 SF1	M38 SF2		M70 SFO	M70 SF1	M70 SF2		M102 SFO	M102 SF1	M102 SF2	
7	M7 SFO	M7 SF1	M7 SF2		M39 SFO	M39 SF1	M39 SF2		M71 SFO	M71 SF1	M71 SF2		M103 SFO	M103 SF1	M103 SF2	
8	M8 SFO	M8 SF1	M8 SF2		M40 SFO	M40 SF1	M40 SF2		M72 SFO	M72 SF1	M72 SF2		M104 SFO	M104 SF1	M104 SF2	
9	M9 SFO	M9 SF1	M9 SF2		M41 SFO	M41 SF1	M41 SF2		M73 SFO	M73 SF1	M73 SF2		M105 SFO	M105 SF1	M105 SF2	
10	M10 SFO	M10 SF1	M10 SF2		M42 SFO	M42 SF1	M42 SF2		M74 SFO	M74 SF1	M74 SF2		M106 SFO	M106 SF1	M106 SF2	
11	M11 SFO	M11 SF1	M11 SF2		M43 SFO	M43 SF1	M43 SF2		M75 SFO	M75 SF1	M75 SF2		M107 SFO	M107 SF1	M107 SF2	
12	M12 SFO	M12 SF1	M12 SF2		M44 SFO	M44 SF1	M44 SF2		M76 SFO	M76 SF1	M76 SF2		M108 SFO	M108 SF1	M108 SF2	
13	M13 SFO	M13 SF1	M13 SF2		M45 SFO	M45 SF1	M45 SF2		M77 SFO	M77 SF1	M77 SF2		M109 SFO	M109 SF1	M109 SF2	
14	M14 SFO	M14 SF1	M14 SF2		M46 SFO	M46 SF1	M46 SF2		M78 SFO	M78 SF1	M78 SF2		M110 SFO	M110 SF1	M110 SF2	
15	M15 SFO	M15 SF1	M15 SF2		M47 SFO	M47 SF1	M47 SF2		M79 SFO	M79 SF1	M79 SF2		M111 SFO	M111 SF1	M111 SF2	
16	M16 SFO	M16 SF1	M16 SF2		M48 SFO	M48 SF1	M48 SF2		M80 SFO	M80 SF1	M80 SF2		M112 SFO	M112 SF1	M112 SF2	
17	M17 SFO	M17 SF1	M17 SF2		M49 SFO	M49 SF1	M49 SF2		M81 SFO	M81 SF1	M81 SF2		M113 SFO	M113 SF1	M113 SF2	
18	M18 SFO	M18 SF1	M18 SF2		M50 SFO	M50 SF1	M50 SF2		M82 SFO	M82 SF1	M82 SF2		M114 SFO	M114 SF1	M114 SF2	
19	M19 SFO	M19 SF1	M19 SF2		M51 SFO	M51 SF1	M51 SF2		M83 SFO	M83 SF1	M83 SF2		M115 SFO	M115 SF1	M115 SF2	
20	M20 SFO	M20 SF1	M20 SF2		M52 SFO	M52 SF1	M52 SF2		M84 SFO	M84 SF1	M84 SF2		M116 SFO	M116 SF1	M116 SF2	
21	M21 SFO	M21 SF1	M21 SF2		M53 SFO	M53 SF1	M53 SF2		M85 SFO	M85 SF1	M85 SF2		M117 SFO	M117 SF1	M117 SF2	
22	M22 SFO	M22 SF1	M22 SF2		M54 SFO	M54 SF1	M54 SF2		M86 SFO	M86 SF1	M86 SF2		M118 SFO	M118 SF1	M118 SF2	
23	M23 SFO	M23 SF1	M23 SF2		M55 SFO	M55 SF1	M55 SF2		M87 SFO	M87 SF1	M87 SF2		M119 SFO	M119 SF1	M119 SF2	
24	M24 SFO	M24 SF1	M24 SF2		M56 SFO	M56 SF1	M56 SF2		M88 SFO	M88 SF1	M88 SF2		M120 SFO	M120 SF1	M120 SF2	
25	M25 SFO	M25 SF1	M25 SF2		M57 SFO	M57 SF1	M57 SF2		M89 SFO	M89 SF1	M89 SF2		M121 SFO	M121 SF1	M121 SF2	
26	M26 SFO	M26 SF1	M26 SF2		M58 SFO	M58 SF1	M58 SF2		M90 SFO	M90 SF1	M90 SF2		M122 SFO	M122 SF1	M122 SF2	
27	M27 SFO	M27 SF1	M27 SF2		M59 SFO	M59 SF1	M59 SF2		M91 SFO	M91 SF1	M91 SF2		M123 SFO	M123 SF1	M123 SF2	
28	M28 SFO	M28 SF1	M28 SF2		M60 SFO	M60 SF1	M60 SF2		M92 SFO	M92 SF1	M92 SF2		M124 SFO	M124 SF1	M124 SF2	
29	M29 SFO	M29 SF1	M29 SF2		M61 SFO	M61 SF1	M61 SF2		M93 SFO	M93 SF1	M93 SF2		M125 SFO	M125 SF1	M125 SF2	
30	M30 SFO	M30 SF1	M30 SF2		M62 SFO	M62 SF1	M62 SF2		M94 SFO	M94 SF1	M94 SF2		M126 SFO	M126 SF1	M126 SF2	
31	M31 SFO	M31 SF1	M31 SF2		M63 SFO	M63 SF1	M63 SF2		M95 SFO	M95 SF1	M95 SF2		M127 SFO	M127 SF1	M127 SF2	
per-byte sub-column																
SFA ID = 00																

Figure 234: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=00

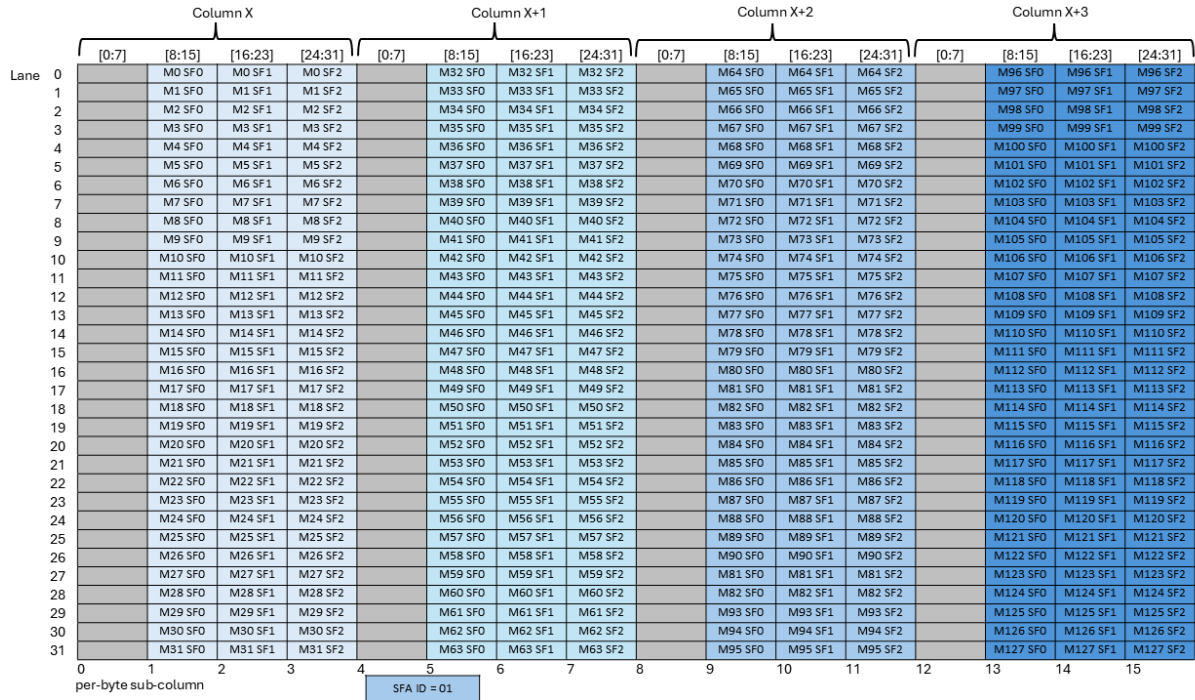


Figure 235: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=01

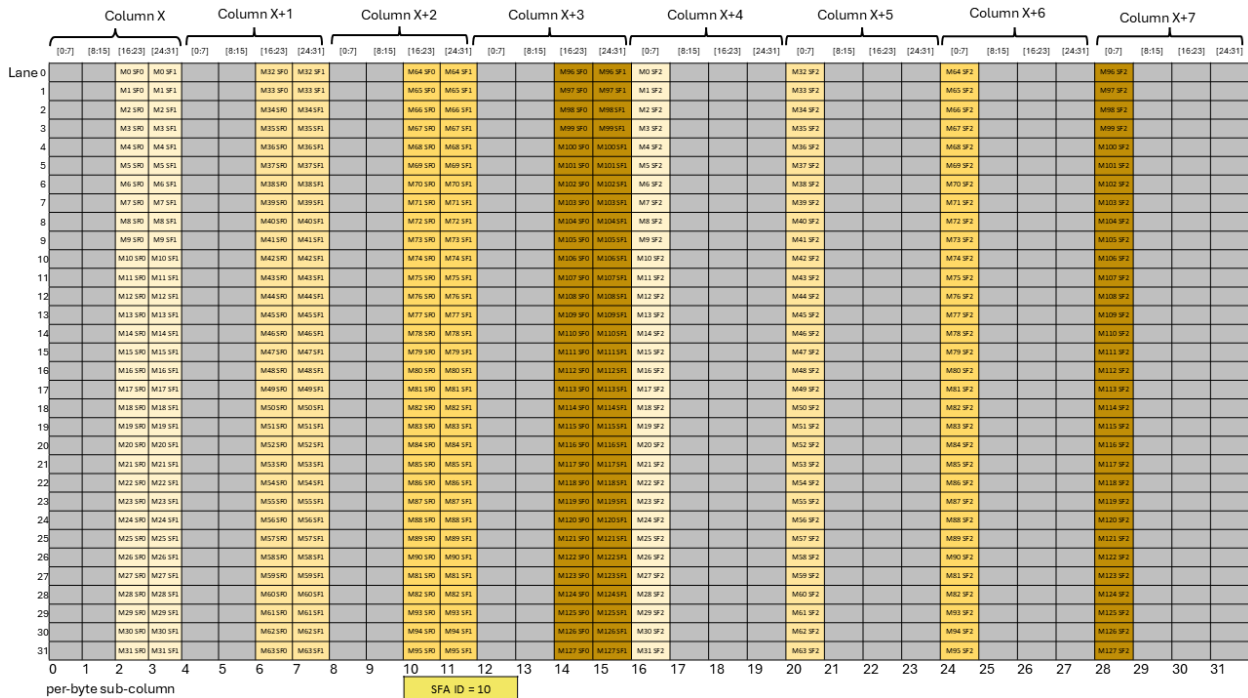


Figure 236: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=10

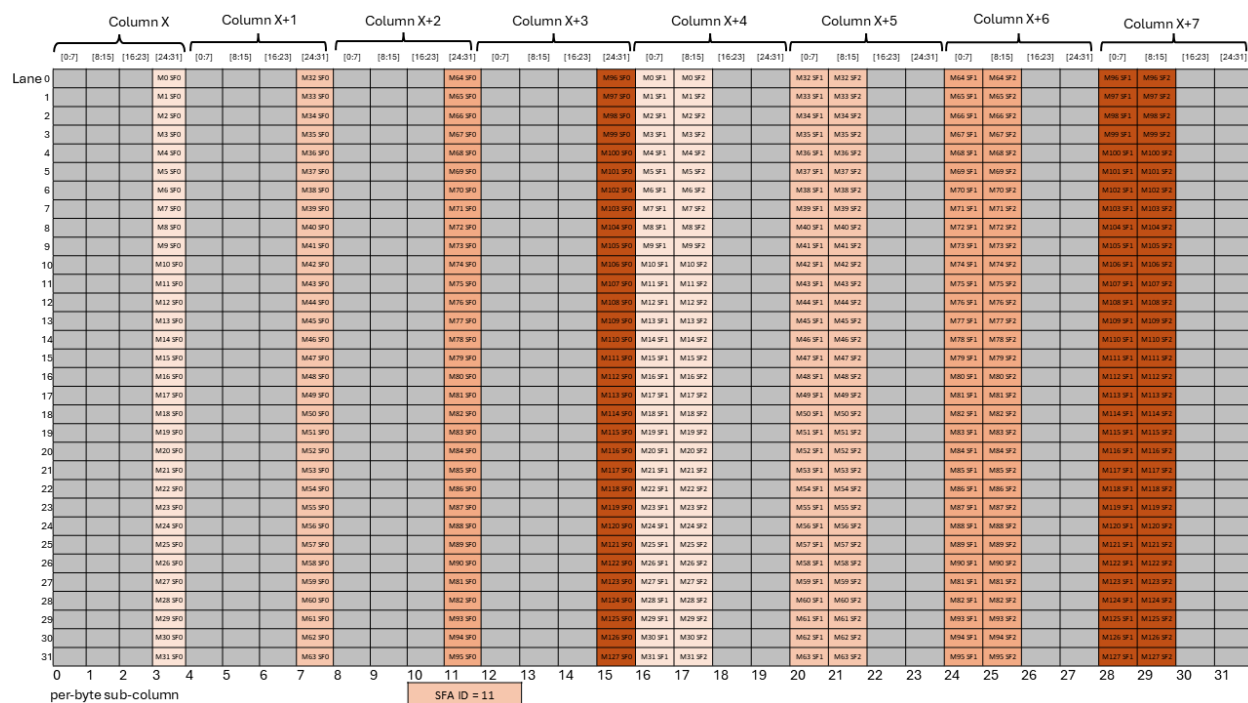


Figure 237: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=11

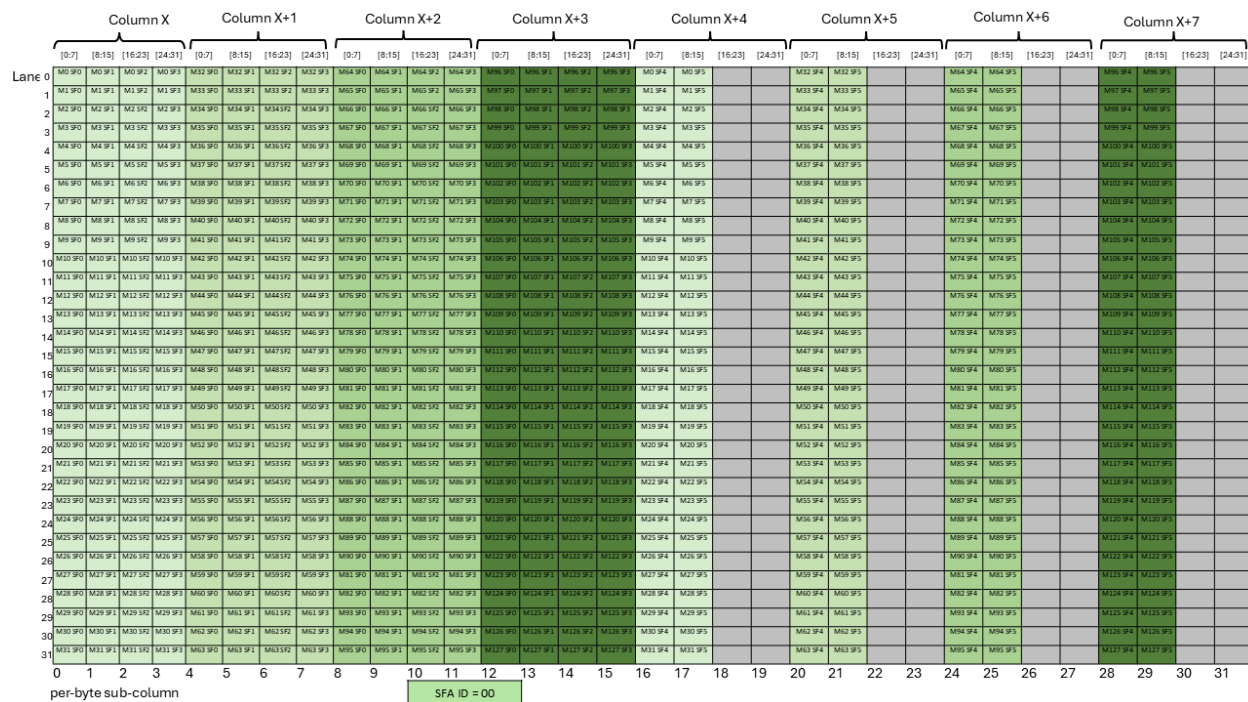


Figure 238: Layout of scale factor A matrix with block16 with K=96 with SFA_ID=00

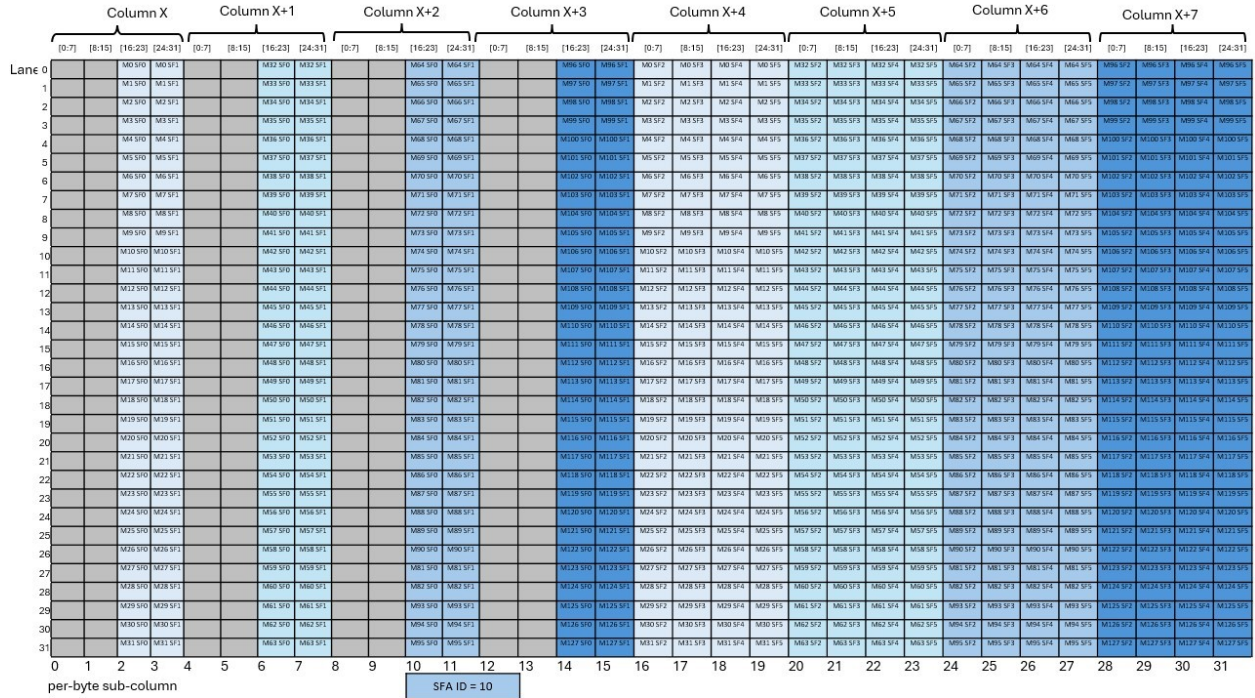


Figure 239: Layout of scale factor A matrix with block16 with K=96 with SFA_ID=10

9.7.16.10.7.8 Scale Factor B ID

The value of the scale factor B ID selects the sub-columns in the Tensor Memory to form the scale factor B matrix, which is used to scale the matrix B.

The following shows the scale factor matrix layout for various scale vector sizes:

9.7.16.10.7.9 Layout of the Scale Factor B Matrix for scale_vec::1X/block32 with K=32/K=64

There is one scale factor per row of the B matrix with block size as 32 and the scale factor must be provided in 1-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 240 shows which sub-columns get selected for different values of *SFB_ID*.

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFB_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

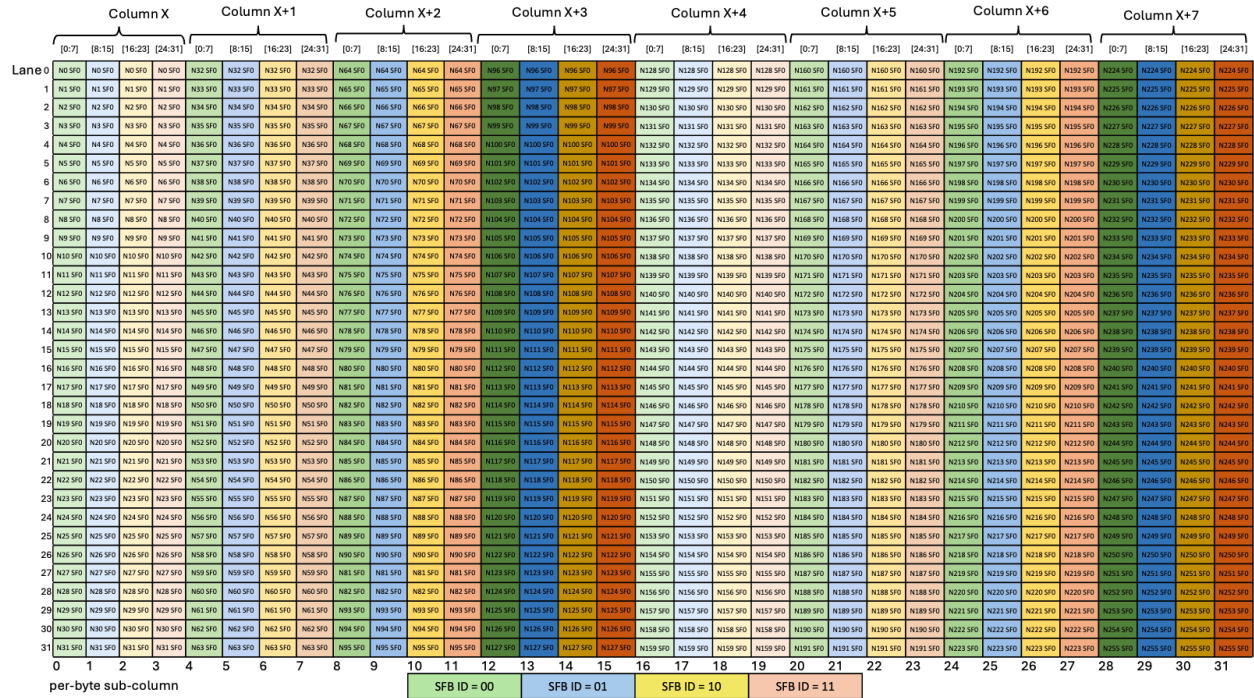


Figure 240: Layout of scale factor B matrix with scale_vec::1X/block32 with K=32/K=64

9.7.16.10.7.10 Layout of the Scale Factor B Matrix for scale_vec::2X/block32 with K=64/K=128

There are two scale factors per row of the B matrix with block size as 32 and the scale factor must be provided in 2-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the half word offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 241 shows which sub-columns get selected for different values of *SFB_ID*.

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFB_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

9.7.16.10.7.11 Layout of the Scale Factor B Matrix for scale_vec::4X/block16 with K=64/K=128

There are four scale factors per row of the B matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. The *SFB_ID* value must be 0 and this specifies that all of the columns (in green) will be used for the scale factor matrix. Figure 242 shows which sub-columns get selected for different values of *SFB_ID*.

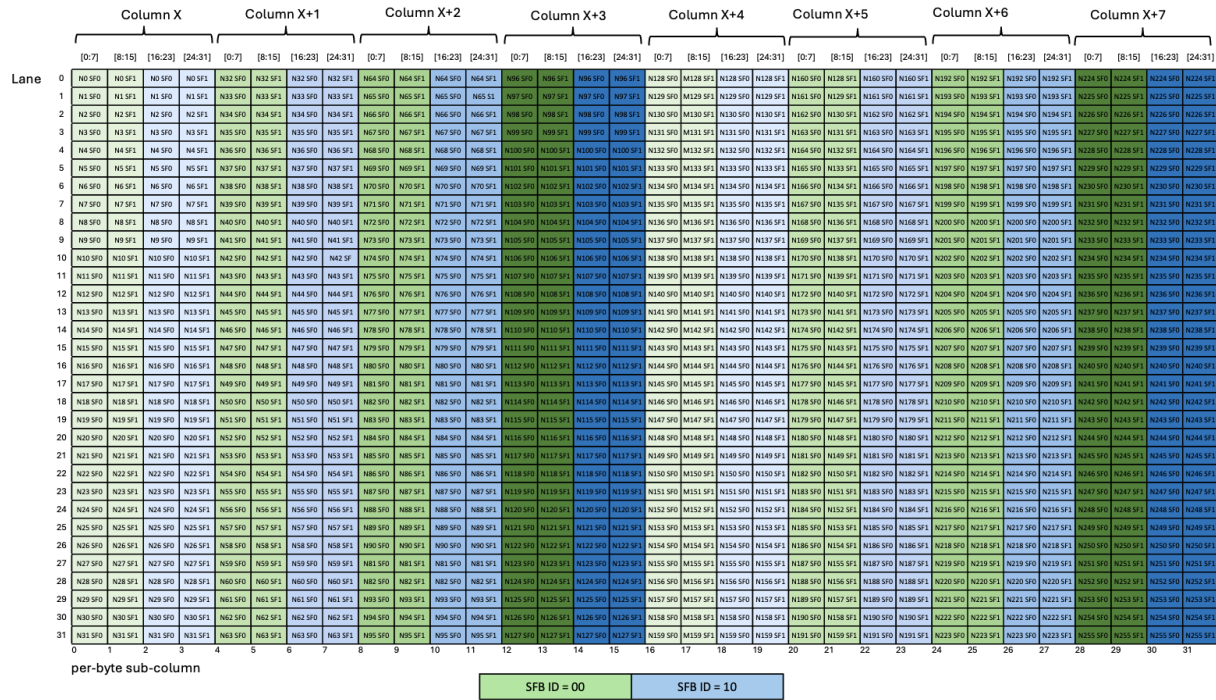


Figure 241: Layout of scale factor B matrix with scale_vec::2X/block32 with K=64/K=128

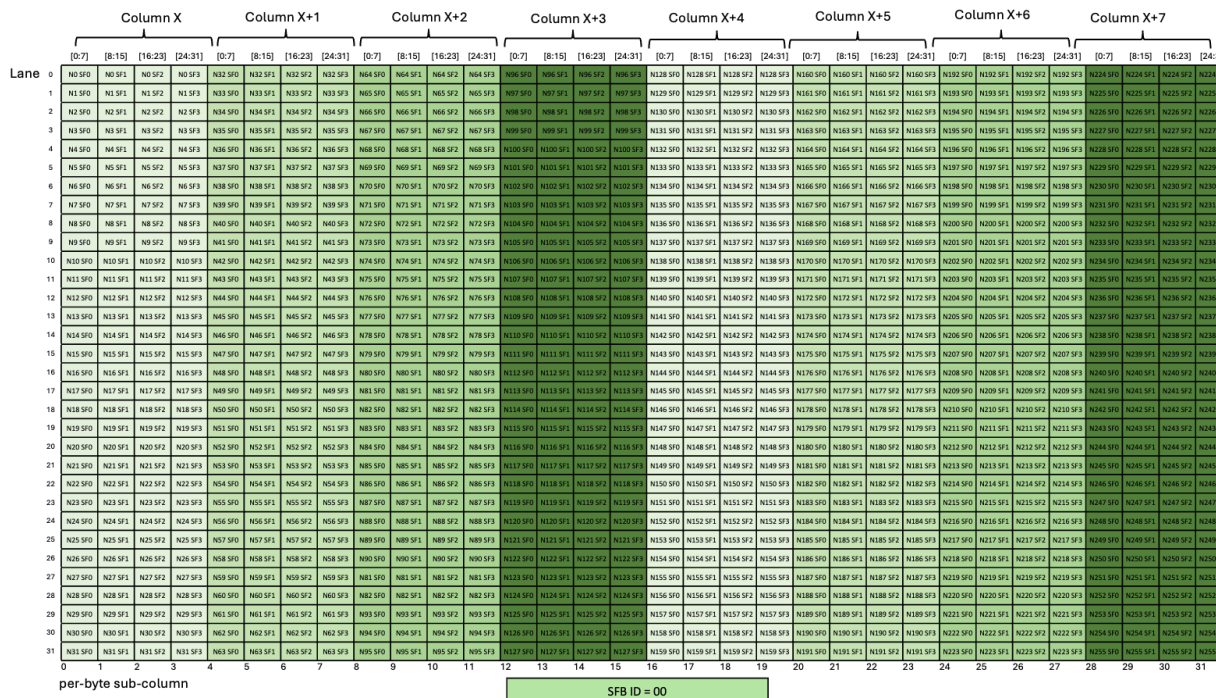


Figure 242: Layout of scale factor B matrix with scale_vec::4X/block16 with K=64/K=128

9.7.16.10.7.12 Layout of the Scale Factor B Matrix for block32 with K=96 (Semantically equivalent to scale_vec::3X)

There are three scale factors per row of the B matrix with block size as 32 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix.

For $N \leq 128$, Figure 243, Figure 244, Figure 245 and Figure 246 show which sub-columns get selected for different values of *SFB_ID*.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	N0 SF0	N0 SF1	N0 SF2		N32 SF0	N32 SF1	N32 SF2		N64 SF0	N64 SF1	N64 SF2		N96 SF0	N96 SF1	N96 SF2	
1	N1 SF0	N1 SF1	N1 SF2		N33 SF0	N33 SF1	N33 SF2		N65 SF0	N65 SF1	N65 SF2		N97 SF0	N97 SF1	N97 SF2	
2	N2 SF0	N2 SF1	N2 SF2		N34 SF0	N34 SF1	N34 SF2		N66 SF0	N66 SF1	N66 SF2		N98 SF0	N98 SF1	N98 SF2	
3	N3 SF0	N3 SF1	N3 SF2		N35 SF0	N35 SF1	N35 SF2		N67 SF0	N67 SF1	N67 SF2		N99 SF0	N99 SF1	N99 SF2	
4	N4 SF0	N4 SF1	N4 SF2		N36 SF0	N36 SF1	N36 SF2		N68 SF0	N68 SF1	N68 SF2		N100 SF0	N100 SF1	N100 SF2	
5	N5 SF0	N5 SF1	N5 SF2		N37 SF0	N37 SF1	N37 SF2		N69 SF0	N69 SF1	N69 SF2		N101 SF0	N101 SF1	N101 SF2	
6	N6 SF0	N6 SF1	N6 SF2		N38 SF0	N38 SF1	N38 SF2		N70 SF0	N70 SF1	N70 SF2		N102 SF0	N102 SF1	N102 SF2	
7	N7 SF0	N7 SF1	N7 SF2		N39 SF0	N39 SF1	N39 SF2		N71 SF0	N71 SF1	N71 SF2		N103 SF0	N103 SF1	N103 SF2	
8	N8 SF0	N8 SF1	N8 SF2		N40 SF0	N40 SF1	N40 SF2		N72 SF0	N72 SF1	N72 SF2		N104 SF0	N104 SF1	N104 SF2	
9	N9 SF0	N9 SF1	N9 SF2		N41 SF0	N41 SF1	N41 SF2		N73 SF0	N73 SF1	N73 SF2		N105 SF0	N105 SF1	N105 SF2	
10	N10 SF0	N10 SF1	N10 SF2		N42 SF0	N42 SF1	N42 SF2		N74 SF0	N74 SF1	N74 SF2		N106 SF0	N106 SF1	N106 SF2	
11	N11 SF0	N11 SF1	N11 SF2		N43 SF0	N43 SF1	N43 SF2		N75 SF0	N75 SF1	N75 SF2		N107 SF0	N107 SF1	N107 SF2	
12	N12 SF0	N12 SF1	N12 SF2		N44 SF0	N44 SF1	N44 SF2		N76 SF0	N76 SF1	N76 SF2		N108 SF0	N108 SF1	N108 SF2	
13	N13 SF0	N13 SF1	N13 SF2		N45 SF0	N45 SF1	N45 SF2		N77 SF0	N77 SF1	N77 SF2		N109 SF0	N109 SF1	N109 SF2	
14	N14 SF0	N14 SF1	N14 SF2		N46 SF0	N46 SF1	N46 SF2		N78 SF0	N78 SF1	N78 SF2		N110 SF0	N110 SF1	N110 SF2	
15	N15 SF0	N15 SF1	N15 SF2		N47 SF0	N47 SF1	N47 SF2		N79 SF0	N79 SF1	N79 SF2		N111 SF0	N111 SF1	N111 SF2	
16	N16 SF0	N16 SF1	N16 SF2		N48 SF0	N48 SF1	N48 SF2		N80 SF0	N80 SF1	N80 SF2		N112 SF0	N112 SF1	N112 SF2	
17	N17 SF0	N17 SF1	N17 SF2		N49 SF0	N49 SF1	N49 SF2		N81 SF0	N81 SF1	N81 SF2		N113 SF0	N113 SF1	N113 SF2	
18	N18 SF0	N18 SF1	N18 SF2		N50 SF0	N50 SF1	N50 SF2		N82 SF0	N82 SF1	N82 SF2		N114 SF0	N114 SF1	N114 SF2	
19	N19 SF0	N19 SF1	N19 SF2		N51 SF0	N51 SF1	N51 SF2		N83 SF0	N83 SF1	N83 SF2		N115 SF0	N115 SF1	N115 SF2	
20	N20 SF0	N20 SF1	N20 SF2		N52 SF0	N52 SF1	N52 SF2		N84 SF0	N84 SF1	N84 SF2		N116 SF0	N116 SF1	N116 SF2	
21	N21 SF0	N21 SF1	N21 SF2		N53 SF0	N53 SF1	N53 SF2		N85 SF0	N85 SF1	N85 SF2		N117 SF0	N117 SF1	N117 SF2	
22	N22 SF0	N22 SF1	N22 SF2		N54 SF0	N54 SF1	N54 SF2		N86 SF0	N86 SF1	N86 SF2		N118 SF0	N118 SF1	N118 SF2	
23	N23 SF0	N23 SF1	N23 SF2		N55 SF0	N55 SF1	N55 SF2		N87 SF0	N87 SF1	N87 SF2		N119 SF0	N119 SF1	N119 SF2	
24	N24 SF0	N24 SF1	N24 SF2		N56 SF0	N56 SF1	N56 SF2		N88 SF0	N88 SF1	N88 SF2		N120 SF0	N120 SF1	N120 SF2	
25	N25 SF0	N25 SF1	N25 SF2		N57 SF0	N57 SF1	N57 SF2		N89 SF0	N89 SF1	N89 SF2		N121 SF0	N121 SF1	N121 SF2	
26	N26 SF0	N26 SF1	N26 SF2		N58 SF0	N58 SF1	N58 SF2		N90 SF0	N90 SF1	N90 SF2		N122 SF0	N122 SF1	N122 SF2	
27	N27 SF0	N27 SF1	N27 SF2		N59 SF0	N59 SF1	N59 SF2		N91 SF0	N91 SF1	N91 SF2		N123 SF0	N123 SF1	N123 SF2	
28	N28 SF0	N28 SF1	N28 SF2		N60 SF0	N60 SF1	N60 SF2		N92 SF0	N92 SF1	N92 SF2		N124 SF0	N124 SF1	N124 SF2	
29	N29 SF0	N29 SF1	N29 SF2		N61 SF0	N61 SF1	N61 SF2		N93 SF0	N93 SF1	N93 SF2		N125 SF0	N125 SF1	N125 SF2	
30	N30 SF0	N30 SF1	N30 SF2		N62 SF0	N62 SF1	N62 SF2		N94 SF0	N94 SF1	N94 SF2		N126 SF0	N126 SF1	N126 SF2	
31	N31 SF0	N31 SF1	N31 SF2		N63 SF0	N63 SF1	N63 SF2		N95 SF0	N95 SF1	N95 SF2		N127 SF0	N127 SF1	N127 SF2	

per-byte sub-column

SFB ID = 00

Figure 243: Layout of scale factor B matrix with block32 with K=96 and $N \leq 128$ with SFA_ID=00

For $N > 128$, Figure 247, Figure 248, Figure 249, Figure 250, Figure 251 and Figure 252 show which sub-columns get selected for different values of *SFB_ID*.

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFB_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

9.7.16.10.7.13 Layout of the Scale Factor B Matrix for block16 with K=96 (Semantically equivalent to scale_vec::6X)

There are six scale factors per row of the B matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix.

For $N \leq 128$, Figure 253 and Figure 254 show which sub-columns get selected for different values of *SFB_ID*.

For $N > 128$, Figure 255, Figure 256, Figure 257 and Figure 258 show which sub-columns get selected for different values of *SFB_ID*.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	N0 SFO	N0 SF1	N0 SF2		N32 SFO	N32 SF1	N32 SF2		N64 SFO	N64 SF1	N64 SF2		N96 SFO	N96 SF1	N96 SF2	
1	N1 SFO	N1 SF1	N1 SF2		N33 SFO	N33 SF1	N33 SF2		N65 SFO	N65 SF1	N65 SF2		N97 SFO	N97 SF1	N97 SF2	
2	N2 SFO	N2 SF1	N2 SF2		N34 SFO	N34 SF1	N34 SF2		N66 SFO	N66 SF1	N66 SF2		N98 SFO	N98 SF1	N98 SF2	
3	N3 SFO	N3 SF1	N3 SF2		N35 SFO	N35 SF1	N35 SF2		N67 SFO	N67 SF1	N67 SF2		N99 SFO	N99 SF1	N99 SF2	
4	N4 SFO	N4 SF1	N4 SF2		N36 SFO	N36 SF1	N36 SF2		N68 SFO	N68 SF1	N68 SF2		N100 SFO	N100 SF1	N100 SF2	
5	N5 SFO	N5 SF1	N5 SF2		N37 SFO	N37 SF1	N37 SF2		N69 SFO	N69 SF1	N69 SF2		N101 SFO	N101 SF1	N101 SF2	
6	N6 SFO	N6 SF1	N6 SF2		N38 SFO	N38 SF1	N38 SF2		N70 SFO	N70 SF1	N70 SF2		N102 SFO	N102 SF1	N102 SF2	
7	N7 SFO	N7 SF1	N7 SF2		N39 SFO	N39 SF1	N39 SF2		N71 SFO	N71 SF1	N71 SF2		N103 SFO	N103 SF1	N103 SF2	
8	N8 SFO	N8 SF1	N8 SF2		N40 SFO	N40 SF1	N40 SF2		N72 SFO	N72 SF1	N72 SF2		N104 SFO	N104 SF1	N104 SF2	
9	N9 SFO	N9 SF1	N9 SF2		N41 SFO	N41 SF1	N41 SF2		N73 SFO	N73 SF1	N73 SF2		N105 SFO	N105 SF1	N105 SF2	
10	N10 SFO	N10 SF1	N10 SF2		N42 SFO	N42 SF1	N42 SF2		N74 SFO	N74 SF1	N74 SF2		N106 SFO	N106 SF1	N106 SF2	
11	N11 SFO	N11 SF1	N11 SF2		N43 SFO	N43 SF1	N43 SF2		N75 SFO	N75 SF1	N75 SF2		N107 SFO	N107 SF1	N107 SF2	
12	N12 SFO	N12 SF1	N12 SF2		N44 SFO	N44 SF1	N44 SF2		N76 SFO	N76 SF1	N76 SF2		N108 SFO	N108 SF1	N108 SF2	
13	N13 SFO	N13 SF1	N13 SF2		N45 SFO	N45 SF1	N45 SF2		N77 SFO	N77 SF1	N77 SF2		N109 SFO	N109 SF1	N109 SF2	
14	N14 SFO	N14 SF1	N14 SF2		N46 SFO	N46 SF1	N46 SF2		N78 SFO	N78 SF1	N78 SF2		N110 SFO	N110 SF1	N110 SF2	
15	N15 SFO	N15 SF1	N15 SF2		N47 SFO	N47 SF1	N47 SF2		N79 SFO	N79 SF1	N79 SF2		N111 SFO	N111 SF1	N111 SF2	
16	N16 SFO	N16 SF1	N16 SF2		N48 SFO	N48 SF1	N48 SF2		N80 SFO	N80 SF1	N80 SF2		N112 SFO	N112 SF1	N112 SF2	
17	N17 SFO	N17 SF1	N17 SF2		N49 SFO	N49 SF1	N49 SF2		N81 SFO	N81 SF1	N81 SF2		N113 SFO	N113 SF1	N113 SF2	
18	N18 SFO	N18 SF1	N18 SF2		N50 SFO	N50 SF1	N50 SF2		N82 SFO	N82 SF1	N82 SF2		N114 SFO	N114 SF1	N114 SF2	
19	N19 SFO	N19 SF1	N19 SF2		N51 SFO	N51 SF1	N51 SF2		N83 SFO	N83 SF1	N83 SF2		N115 SFO	N115 SF1	N115 SF2	
20	N20 SFO	N20 SF1	N20 SF2		N52 SFO	N52 SF1	N52 SF2		N84 SFO	N84 SF1	N84 SF2		N116 SFO	N116 SF1	N116 SF2	
21	N21 SFO	N21 SF1	N21 SF2		N53 SFO	N53 SF1	N53 SF2		N85 SFO	N85 SF1	N85 SF2		N117 SFO	N117 SF1	N117 SF2	
22	N22 SFO	N22 SF1	N22 SF2		N54 SFO	N54 SF1	N54 SF2		N86 SFO	N86 SF1	N86 SF2		N118 SFO	N118 SF1	N118 SF2	
23	N23 SFO	N23 SF1	N23 SF2		N55 SFO	N55 SF1	N55 SF2		N87 SFO	N87 SF1	N87 SF2		N119 SFO	N119 SF1	N119 SF2	
24	N24 SFO	N24 SF1	N24 SF2		N56 SFO	N56 SF1	N56 SF2		N88 SFO	N88 SF1	N88 SF2		N120 SFO	N120 SF1	N120 SF2	
25	N25 SFO	N25 SF1	N25 SF2		N57 SFO	N57 SF1	N57 SF2		N89 SFO	N89 SF1	N89 SF2		N121 SFO	N121 SF1	N121 SF2	
26	N26 SFO	N26 SF1	N26 SF2		N58 SFO	N58 SF1	N58 SF2		N90 SFO	N90 SF1	N90 SF2		N122 SFO	N122 SF1	N122 SF2	
27	N27 SFO	N27 SF1	N27 SF2		N59 SFO	N59 SF1	N59 SF2		N91 SFO	N91 SF1	N91 SF2		N123 SFO	N123 SF1	N123 SF2	
28	N28 SFO	N28 SF1	N28 SF2		N60 SFO	N60 SF1	N60 SF2		N92 SFO	N92 SF1	N92 SF2		N124 SFO	N124 SF1	N124 SF2	
29	N29 SFO	N29 SF1	N29 SF2		N61 SFO	N61 SF1	N61 SF2		N93 SFO	N93 SF1	N93 SF2		N125 SFO	N125 SF1	N125 SF2	
30	N30 SFO	N30 SF1	N30 SF2		N62 SFO	N62 SF1	N62 SF2		N94 SFO	N94 SF1	N94 SF2		N126 SFO	N126 SF1	N126 SF2	
31	N31 SFO	N31 SF1	N31 SF2		N63 SFO	N63 SF1	N63 SF2		N95 SFO	N95 SF1	N95 SF2		N127 SFO	N127 SF1	N127 SF2	

per-byte sub-column

SFB ID = 01

Figure 244: Layout of scale factor B matrix with block32 with K=96 and N<=128 with SFA_ID=01

Lane 0	Column X				Column X+1				Column X+2				Column X+3				Column X+4				Column X+5				Column X+6				Column X+7																																						
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]																																			
0	N0 SFO	N0 SF1			N32 SFO	N32 SF1			N64 SFO	N64 SF1			N96 SFO	N96 SF1	N96 SF2			N128 SFO			N160 SFO					N192 SFO				N224 SFO																																					
1	N1 SFO	N1 SF1	N1 SF2		N33 SFO	N33 SF1	N33 SF2		N65 SFO	N65 SF1	N65 SF2		N97 SFO	N97 SF1	N97 SF2	N97 SF3			N129 SFO	N129 SF1		N161 SFO	N161 SF1	N161 SF2		N193 SFO	N193 SF1	N193 SF2		N225 SFO	N225 SF1	N225 SF2																																			
2	N2 SFO	N2 SF1			N34 SFO	N34 SF1			N66 SFO	N66 SF1			N98 SFO	N98 SF1	N98 SF2	N98 SF3			N130 SFO			N162 SFO				N194 SFO				N226 SFO																																					
3	N3 SFO	N3 SF1	N3 SF2		N35 SFO	N35 SF1	N35 SF2		N67 SFO	N67 SF1	N67 SF2		N99 SFO	N99 SF1	N99 SF2	N99 SF3			N131 SFO	N131 SF1		N163 SFO	N163 SF1	N163 SF2		N195 SFO	N195 SF1	N195 SF2		N227 SFO	N227 SF1	N227 SF2																																			
4	N4 SFO	N4 SF1			N36 SFO	N36 SF1			N68 SFO	N68 SF1			N100 SFO	N100 SF1	N100 SF2	N100 SF3			N132 SFO			N164 SFO				N196 SFO				N228 SFO																																					
5	N5 SFO	N5 SF1	N5 SF2		N37 SFO	N37 SF1	N37 SF2		N69 SFO	N69 SF1	N69 SF2		N101 SFO	N101 SF1	N101 SF2	N101 SF3			N133 SFO	N133 SF1		N165 SFO	N165 SF1	N165 SF2		N197 SFO	N197 SF1	N197 SF2		N229 SFO	N229 SF1	N229 SF2																																			
6	N6 SFO	N6 SF1			N38 SFO	N38 SF1			N70 SFO	N70 SF1			N102 SFO	N102 SF1	N102 SF2	N102 SF3			N134 SFO			N166 SFO				N198 SFO				N230 SFO																																					
7	N7 SFO	N7 SF1	N7 SF2		N39 SFO	N39 SF1	N39 SF2		N71 SFO	N71 SF1	N71 SF2		N103 SFO	N103 SF1	N103 SF2	N103 SF3			N135 SFO	N135 SF1		N167 SFO	N167 SF1	N167 SF2		N199 SFO	N199 SF1	N199 SF2		N231 SFO	N231 SF1	N231 SF2																																			
8	N8 SFO	N8 SF1			N40 SFO	N40 SF1			N72 SFO	N72 SF1			N104 SFO	N104 SF1	N104 SF2	N104 SF3			N136 SFO			N168 SFO				N200 SFO				N232 SFO																																					
9	N9 SFO	N9 SF1	N9 SF2		N41 SFO	N41 SF1	N41 SF2		N73 SFO	N73 SF1	N73 SF2		N105 SFO	N105 SF1	N105 SF2	N105 SF3			N137 SFO	N137 SF1		N169 SFO	N169 SF1	N169 SF2		N201 SFO	N201 SF1	N201 SF2		N233 SFO	N233 SF1	N233 SF2																																			
10	N10 SFO	N10 SF1			N42 SFO	N42 SF1			N74 SFO	N74 SF1			N106 SFO	N106 SF1	N106 SF2	N106 SF3			N138 SFO			N170 SFO				N202 SFO				N234 SFO																																					
11	N11 SFO	N11 SF1	N11 SF2		N43 SFO	N43 SF1	N43 SF2		N75 SFO	N75 SF1	N75 SF2		N107 SFO	N107 SF1	N107 SF2	N107 SF3			N139 SFO	N139 SF1		N171 SFO	N171 SF1	N171 SF2		N203 SFO	N203 SF1	N203 SF2		N235 SFO	N235 SF1	N235 SF2																																			
12	N12 SFO	N12 SF1			N44 SFO	N44 SF1			N76 SFO	N76 SF1			N108 SFO	N108 SF1	N108 SF2	N108 SF3			N140 SFO			N172 SFO				N204 SFO				N236 SFO																																					
13	N13 SFO	N13 SF1	N13 SF2		N45 SFO	N45 SF1	N45 SF2		N77 SFO	N77 SF1	N77 SF2		N109 SFO	N109 SF1	N109 SF2	N109 SF3			N141 SFO	N141 SF1		N173 SFO	N173 SF1	N173 SF2		N205 SFO	N205 SF1	N205 SF2		N237 SFO	N237 SF1	N237 SF2																																			
14	N14 SFO	N14 SF1			N46 SFO	N46 SF1			N78 SFO	N78 SF1			N110 SFO	N110 SF1	N110 SF2	N110 SF3			N142 SFO			N174 SFO				N206 SFO				N238 SFO																																					
15	N15 SFO	N15 SF1	N15 SF2		N47 SFO	N47 SF1	N47 SF2		N79 SFO	N79 SF1	N79 SF2		N111 SFO	N111 SF1	N111 SF2	N111 SF3			N143 SFO	N143 SF1		N175 SFO	N175 SF1	N175 SF2		N207 SFO	N207 SF1	N207 SF2		N239 SFO	N239 SF1	N239 SF2																																			
16	N16 SFO	N16 SF1			N48 SFO	N48 SF1			N80 SFO	N80 SF1			N112 SFO	N112 SF1	N112 SF2	N112 SF3			N144 SFO			N176 SFO				N208 SFO				N240 SFO																																					
17	N17 SFO	N17 SF1	N17 SF2		N49 SFO	N49 SF1	N49 SF2		N81 SFO	N81 SF1	N81 SF2		N113 SFO	N113 SF1	N113 SF2	N113 SF3			N145 SFO	N145 SF1		N177 SFO	N177 SF1	N177 SF2		N209 SFO	N209 SF1	N209 SF2		N241 SFO	N241 SF1	N241 SF2																																			
18	N18 SFO	N18 SF1			N50 SFO	N50 SF1			N82 SFO	N82 SF1			N114 SFO	N114 SF1	N114 SF2	N114 SF3			N146 SFO			N178 SFO				N210 SFO				N242 SFO																																					
19	N19 SFO	N19 SF1	N19 SF2		N51 SFO	N51 SF1	N51 SF2		N83 SFO	N83 SF1	N83 SF2		N115 SFO	N115 SF1	N115 SF2	N115 SF3			N147 SFO	N147 SF1		N179 SFO	N179 SF1	N179 SF2		N211 SFO	N211 SF1	N211 SF2		N243 SFO	N243 SF1	N243 SF2																																			
20	N20 SFO	N20 SF1			N52 SFO	N52 SF1			N84 SFO	N84 SF1			N116 SFO	N116 SF1	N116 SF2	N116 SF3			N148 SFO			N180 SFO				N212 SFO				N244 SFO																																					
21	N21 SFO	N21 SF1	N21 SF2		N53 SFO	N53 SF1	N53 SF2		N85 SFO	N85 SF1	N85 SF2		N117 SFO	N117 SF1	N117 SF2	N117 SF3			N149 SFO	N149 SF1		N181 SFO	N181 SF1	N181 SF2		N213 SFO	N213 SF1	N213 SF2		N245 SFO	N245 SF1	N245 SF2																																			
22	N22 SFO	N22 SF1			N54 SFO	N54 SF1			N86 SFO	N86 SF1			N118 SFO	N118 SF1	N118 SF2	N118 SF3			N150 SFO			N182 SFO				N214 SFO				N246 SFO																																					
23	N23 SFO	N23 SF1	N23 SF2		N55 SFO	N55 SF1	N55 SF2		N87 SFO	N87 SF1	N87 SF2		N119 SFO	N119 SF1	N119 SF2	N119 SF3			N151 SFO	N151 SF1		N183 SFO	N183 SF1	N183 SF2		N215 SFO	N215 SF1	N215 SF2		N247 SFO	N247 SF1	N247 SF2																																			
24	N24 SFO	N24 SF1			N56 SFO	N56 SF1			N88 SFO	N88 SF1			N120 SFO	N120 SF1	N120 SF2	N120 SF3			N152 SFO			N184 SFO				N216 SFO				N248 SFO																																					
25	N25 SFO	N25 SF1	N25 SF2		N57 SFO	N57 SF1	N57 SF2		N89 SFO	N89 SF1	N89 SF2		N121 SFO	N121 SF1	N121 SF2	N121 SF3			N153 SFO	N153 SF1		N185 SFO	N185 SF1	N185 SF2		N217 SFO	N217 SF1	N217 SF2		N249 SFO	N249 SF1	N249 SF2																																			
26	N26 SFO	N26 SF1			N58 SFO	N58 SF1			N90 SFO	N90 SF1			N122 SFO	N122 SF1	N122 SF2	N122 SF3			N154 SFO			N186 SFO				N218 SFO				N250 SFO																																					
27	N27 SFO	N27 SF1	N27 SF2		N59 SFO	N59 SF1	N59 SF2		N91 SFO	N91 SF1	N91 SF2		N123 SFO	N123 SF1	N123 SF2	N123 SF3			N155 SFO	N155 SF1		N187 SFO	N187 SF1	N187 SF2		N219 SFO	N219 SF1	N219 SF2		N251 SFO	N251 SF1	N251 SF2																																			
28	N28 SFO	N28 SF1			N60 SFO	N60 SF1			N92 SFO	N92 SF1			N124 SFO	N124 SF1	N124 SF2	N124 SF3			N156 SFO			N188 SFO				N220 SFO				N252 SFO																																					
29	N29 SFO	N29 SF1	N29 SF2		N61 SFO	N61 SF1	N61 SF2		N93 SFO	N93 SF1	N93 SF2		N125 SFO	N125 SF1	N125 SF2	N125 SF3			N157 SFO	N157 SF1		N189 SFO	N189 SF1	N189 SF2		N221 SFO	N221 SF1	N221 SF2		N253 SFO	N253 SF1	N253 SF2																																			
30	N30 SFO	N30 SF1			N62 SFO	N62 SF1			N94 SFO	N94 SF1			N126 SFO	N126 SF1	N126 SF2	N126 SF3			N158 SFO			N190 SFO				N222 SFO				N254 SFO																																					
31	N31 SFO	N31 SF1	N31 SF2		N63 SFO	N63 SF1	N63 SF2		N95 SFO	N95 SF1	N95 SF2		N127 SFO	N127 SF1	N127 SF2	N127 SF3			N159 SFO	N159 SF1		N191 SFO	N191 SF1	N191 SF2		N223 SFO	N223 SF1	N223 SF2		N255 SFO	N255 SF1	N255 SF2																																			
per-byte sub-column																																SFB ID = 10																																			

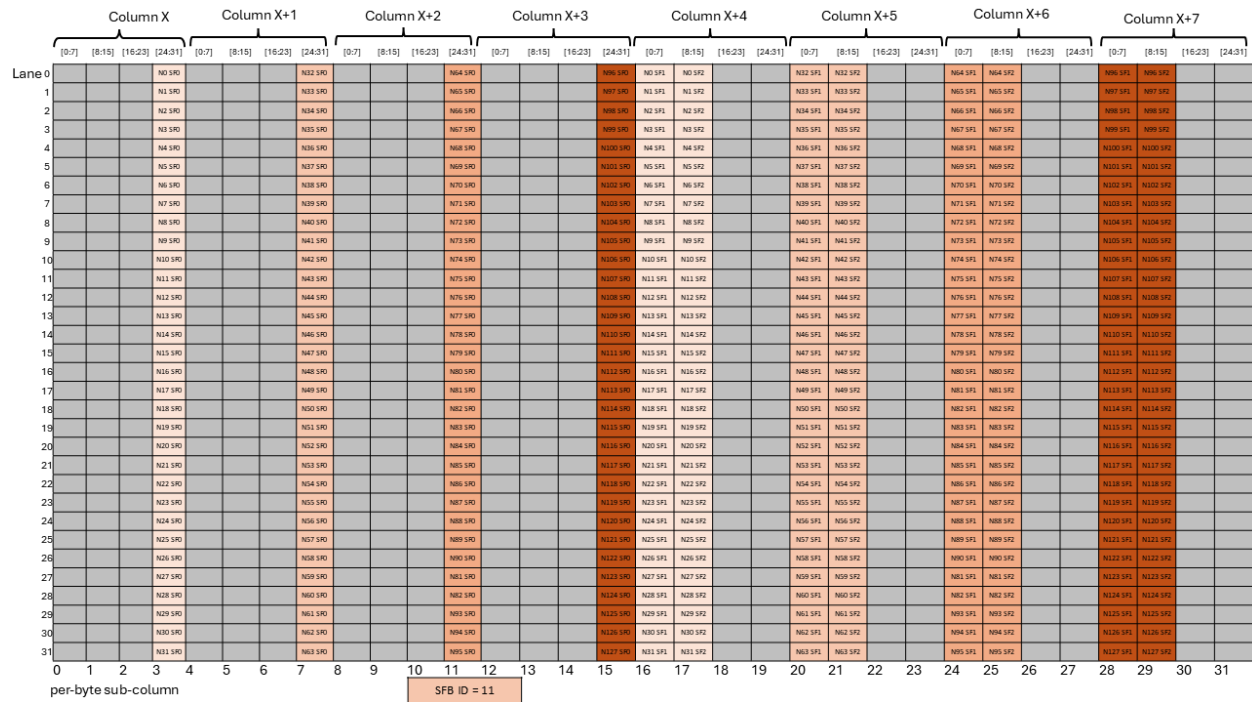


Figure 246: Layout of scale factor B matrix with block32 with K=96 and N≤128 with SFA_ID=11

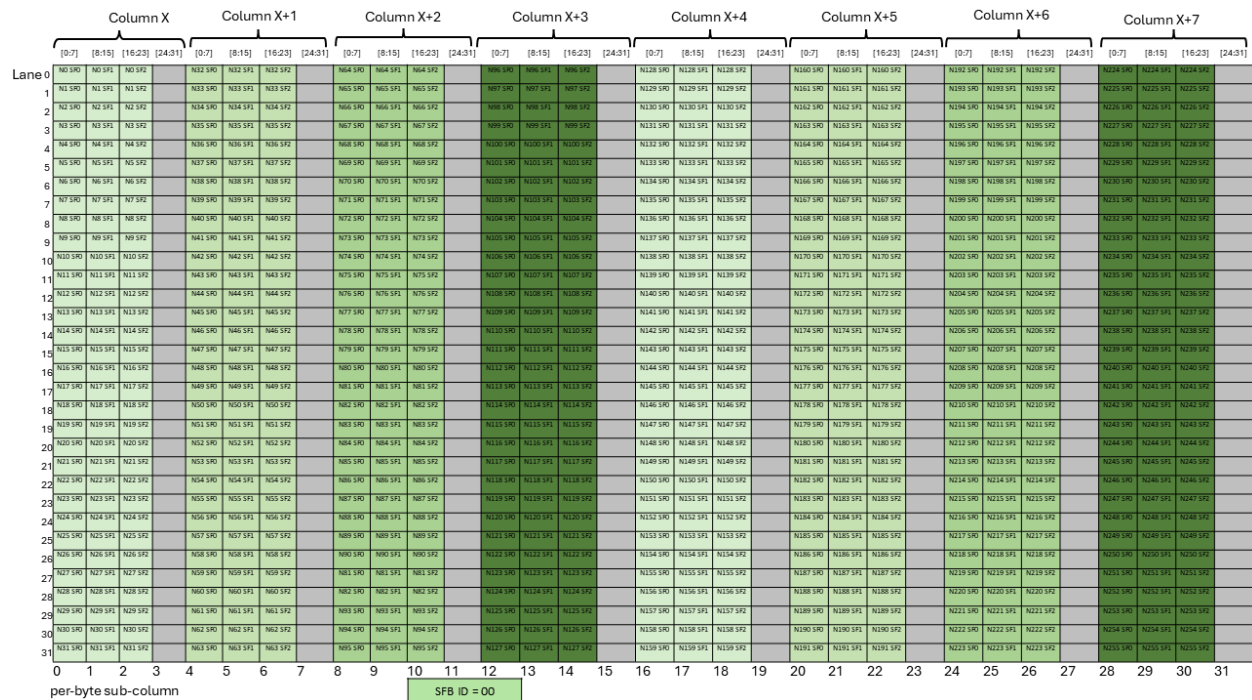


Figure 247: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=00

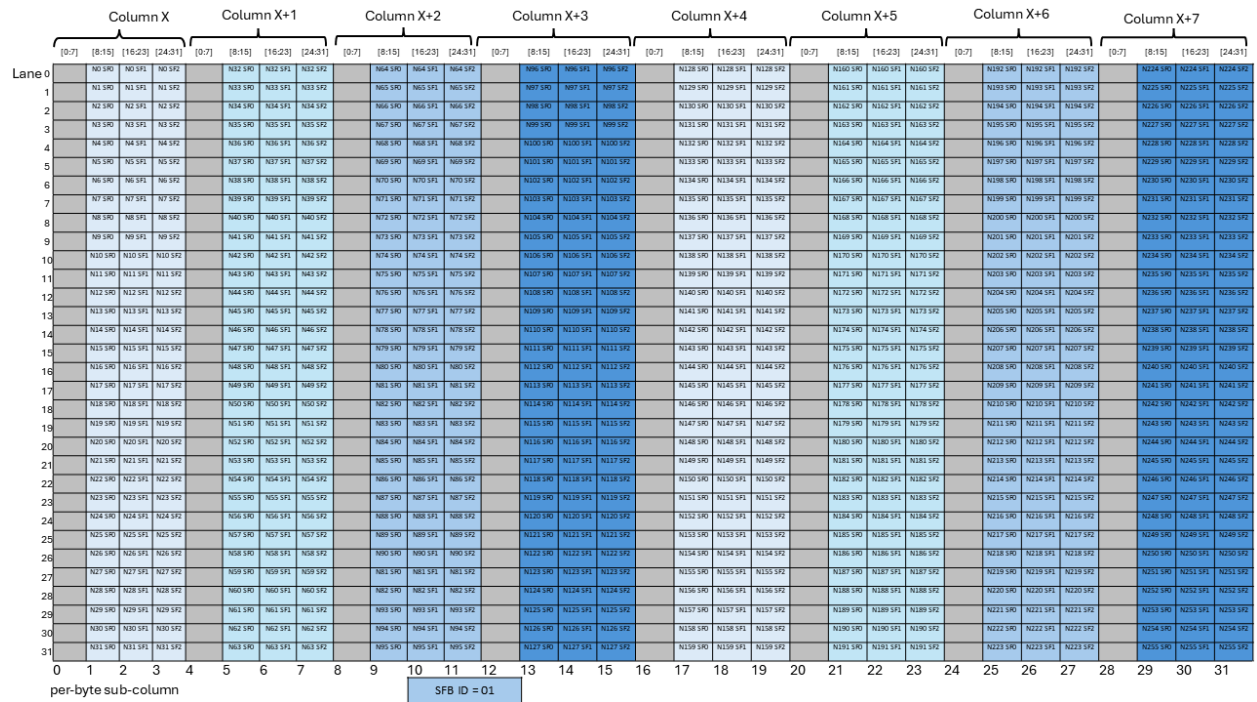


Figure 248: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=01

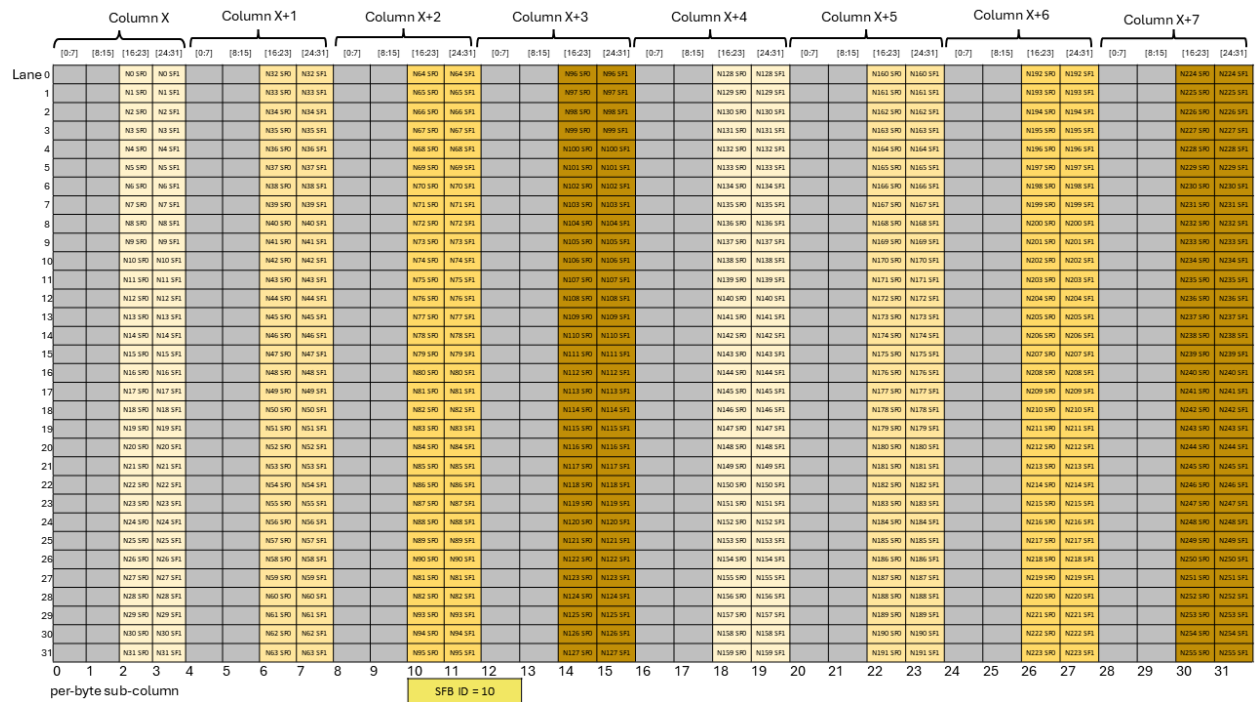


Figure 249: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=10

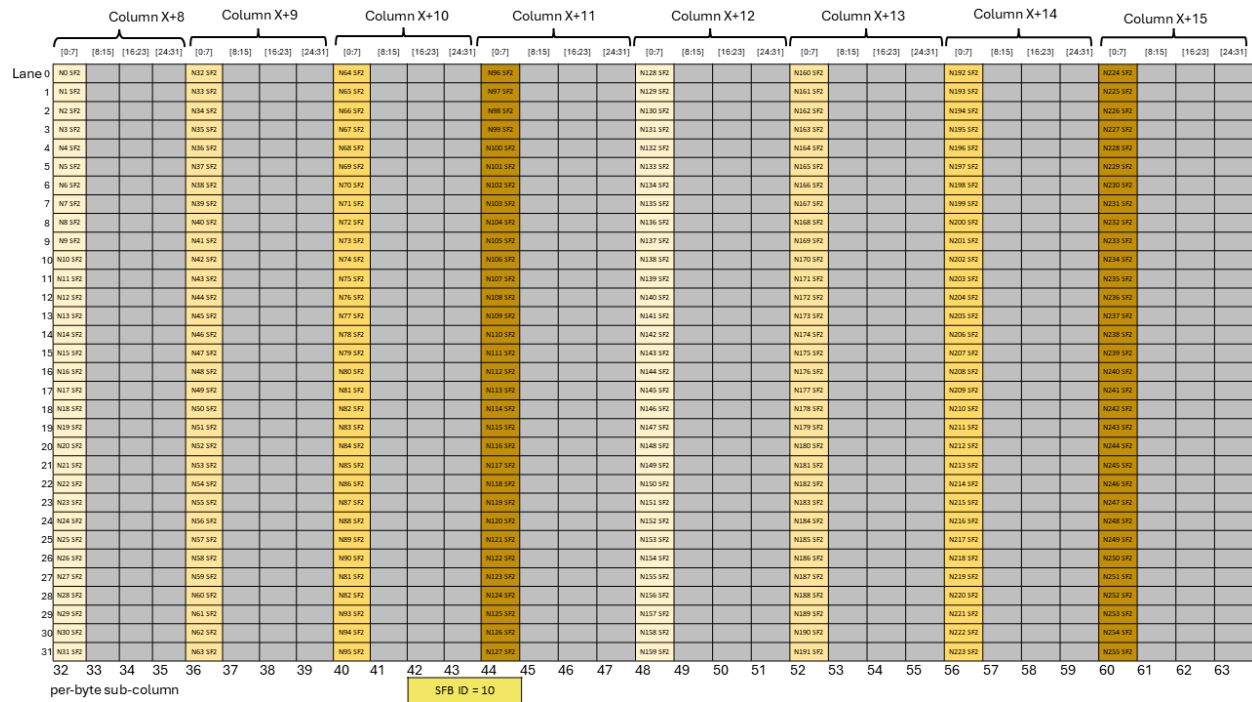


Figure 250: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=10

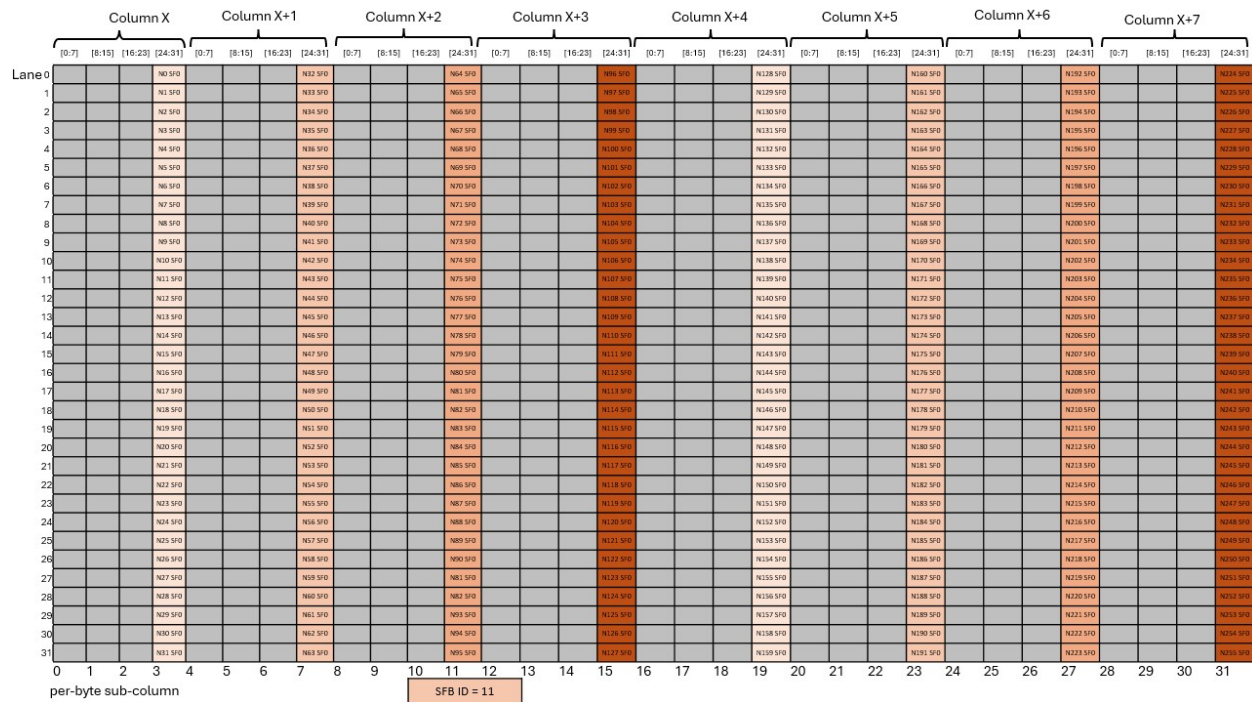


Figure 251: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=11

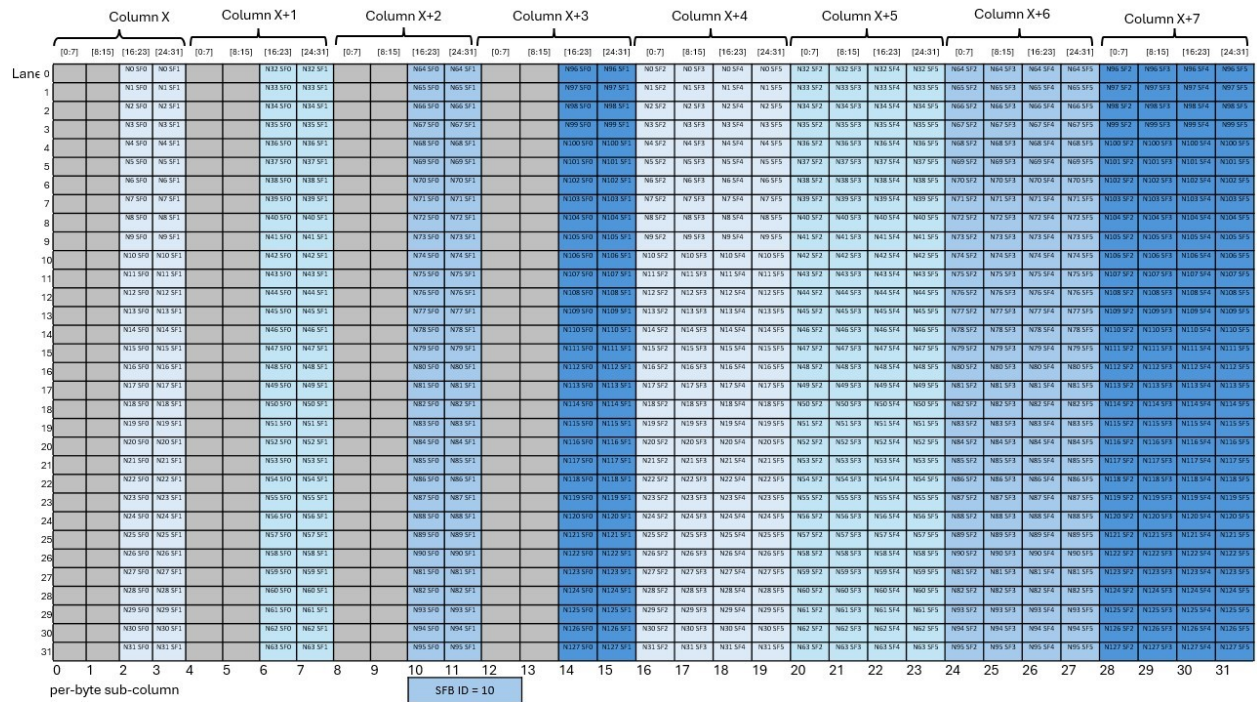


Figure 254: Layout of scale factor B matrix with block 16 with K=96 and N≤128 with SFA_ID=10

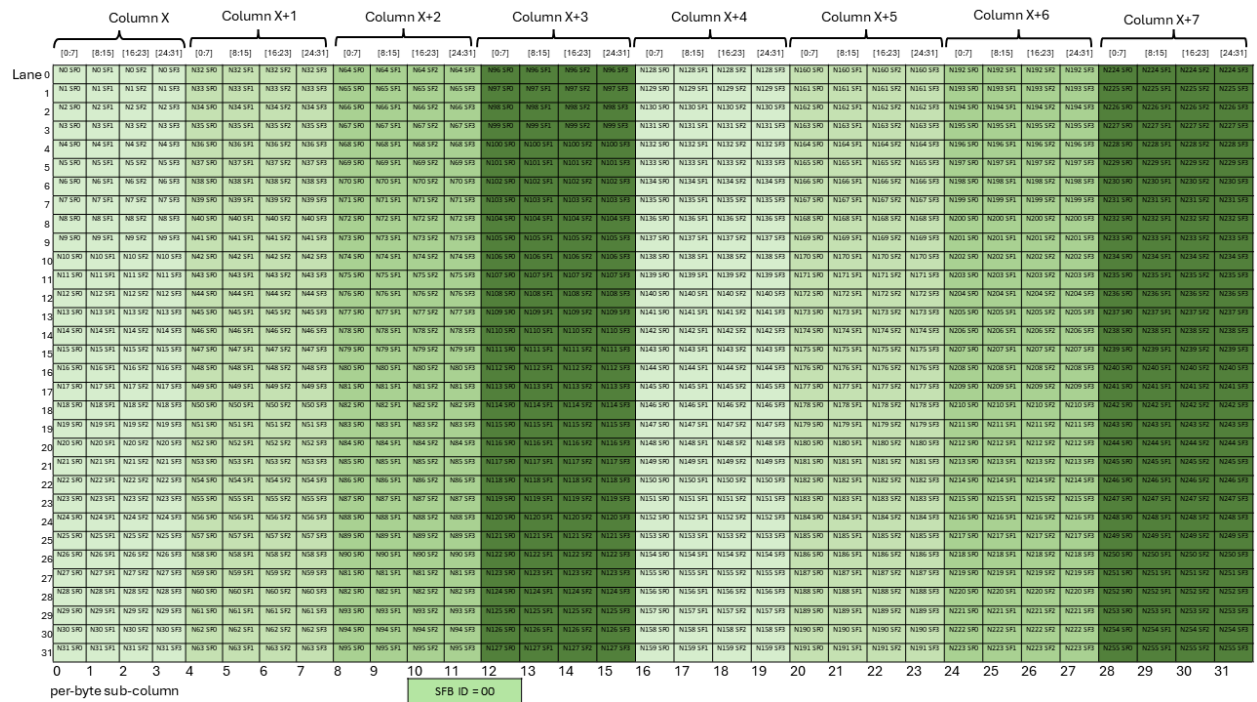


Figure 255: Layout of scale factor B matrix with block 16 with K=96 and N>128 with SFA_ID=00

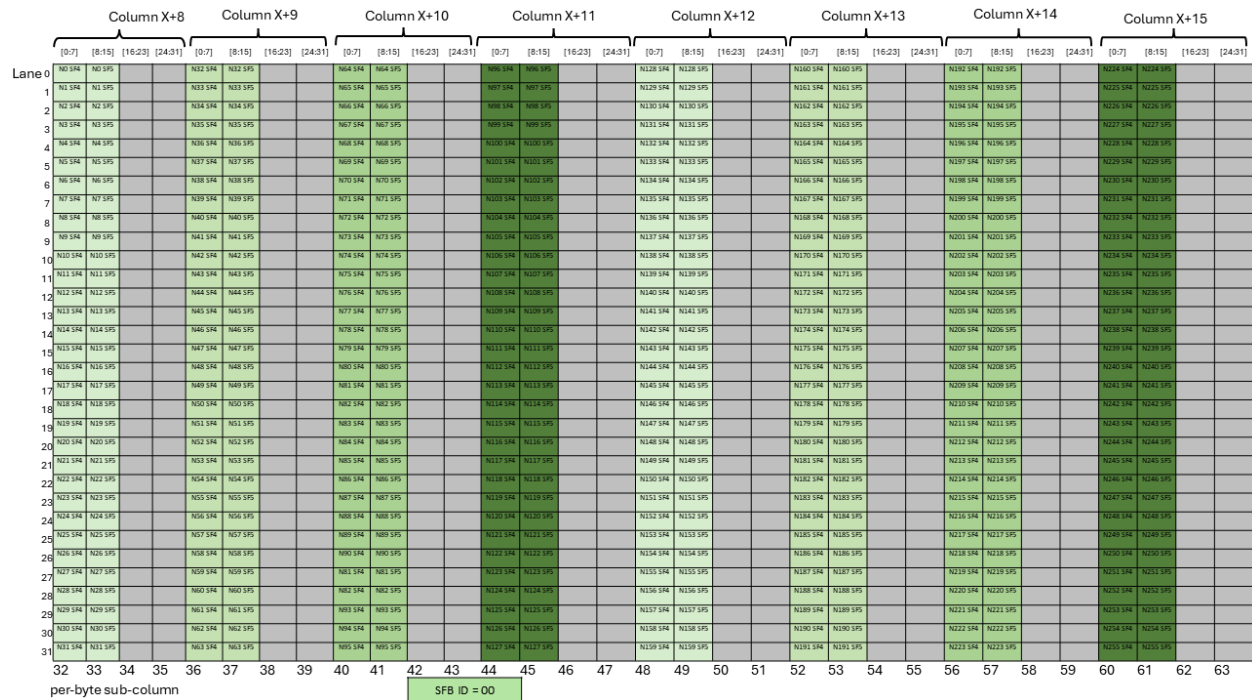


Figure 256: Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=00

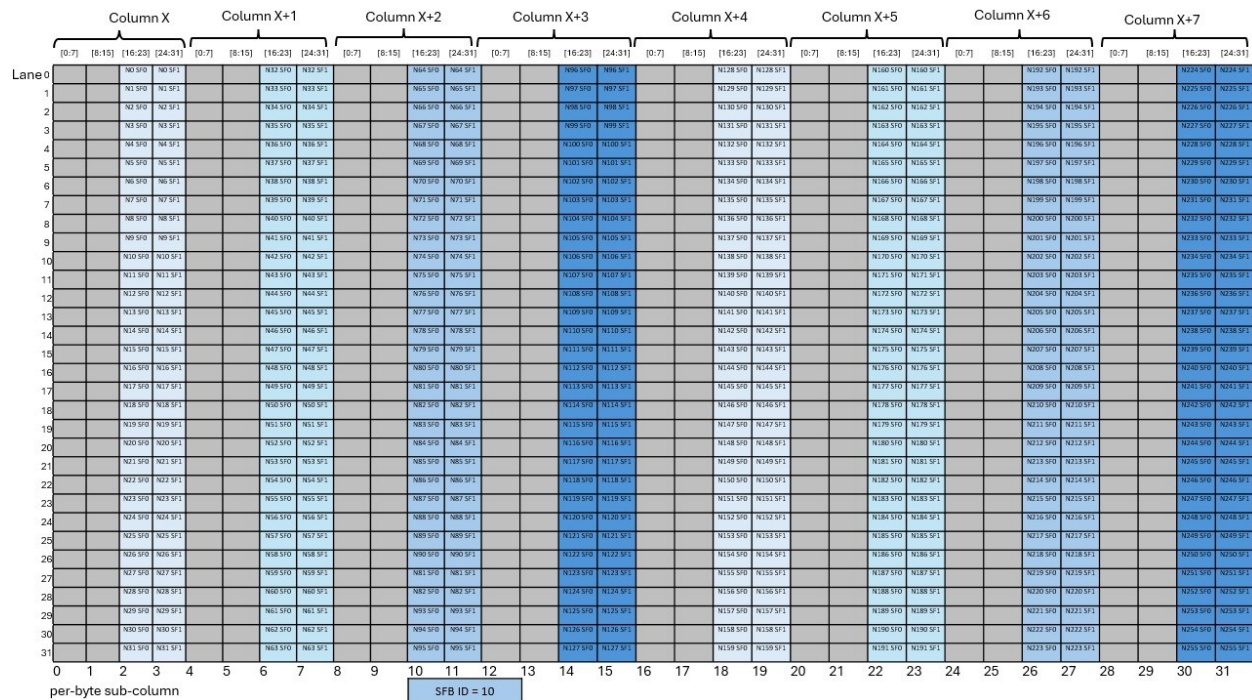


Figure 257: Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=10

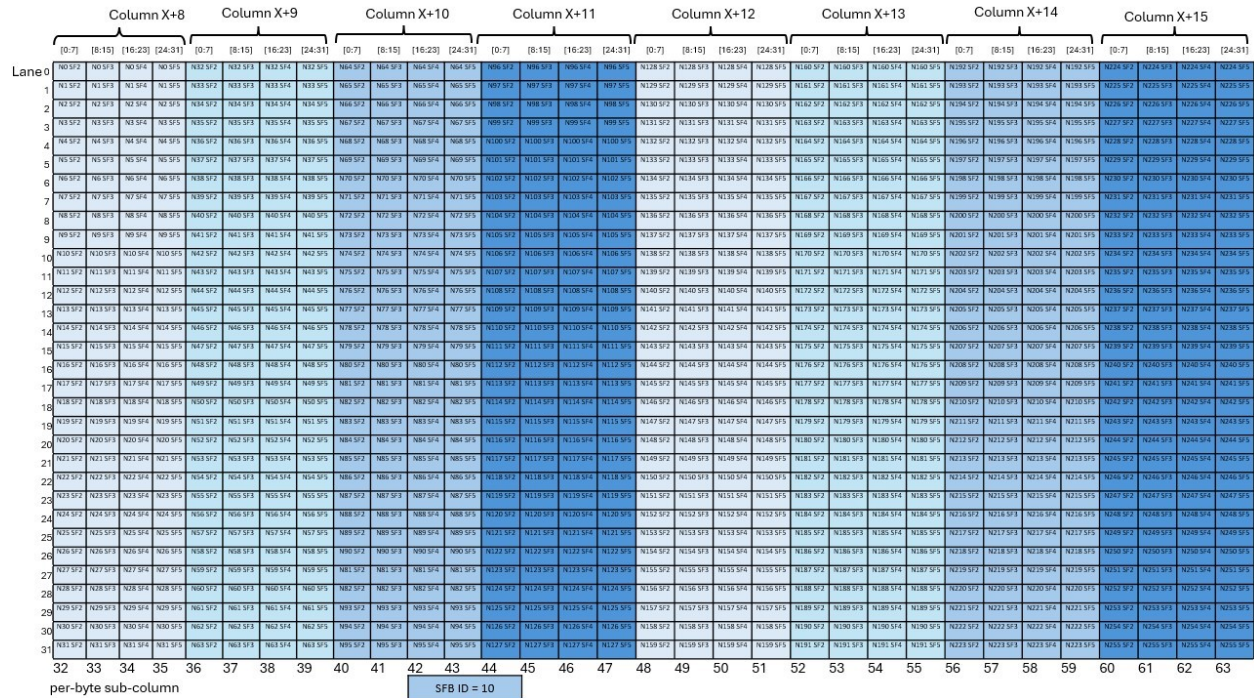


Figure 258: Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=10

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFB_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

9.7.16.10.8 Sparse Matrices

This instruction `tcgen05.mma.sp` can be used when the matrix A is a structured sparse matrix with 50% zeros in each row distributed as per its sparse granularity.

In a $M \times N \times K$ sparse `tcgen05.mma.sp` operation, the matrix A of shape $M \times K$ is stored in a packed form as $M \times (K/2)$ in memory. For each K -wide row of matrix A, 50% of elements are zeros and the remaining $K/2$ non-zero elements are stored in memory. The metadata specifies the mapping of the $K/2$ non-zero elements to the K elements before performing the MMA operation.

Granularity of sparse matrix A is defined as the ratio of the number of non-zero elements in a sub-chunk of the matrix row to the total number of elements in that sub-chunk where the size of the sub-chunk is shape-specific. The following table lists the granularity of different `tcgen05.mma.sp` variants:

.kind of tcgen05.mma	Sparse Granularity
<code>.kind::tf32</code>	1:2
<code>.kind::f16</code>	2:4
<code>.kind::f8f6f4</code>	
<code>.kind::mxf8f6f4</code>	
<code>.kind::i8</code>	
<code>.kind::mxf4</code>	4:8 (in pairs)

9.7.16.10.8.1 Sparse tcgen05.mma.sp with `.kind::tf32`

For `.kind::tf32`, matrix A is structured sparse at a granularity of 1:2. In other words, each chunk of two adjacent elements in a row of matrix A has one zero and one non-zero element. Only the non-zero element is stored in memory and the 4-bit index in the metadata indicates the position of the non-zero element in the two-wide chunk. The only meaningful values of the index are:

- 0b1110
- 0b0100

Rest of the values result in undefined behavior.

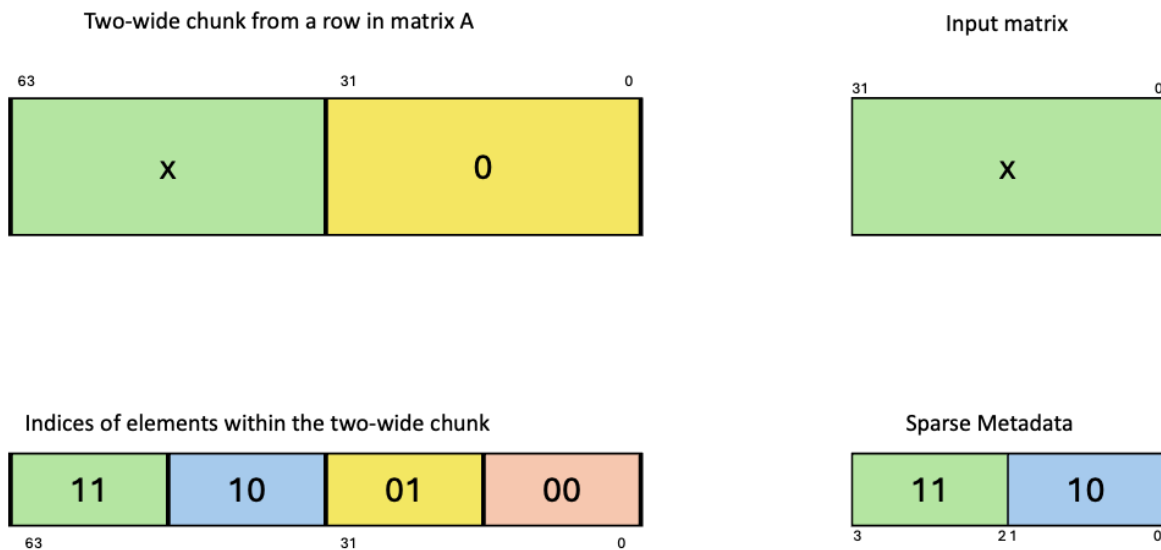


Figure 259: Sparse tcgen05.mma metadata example for tf32 kind

9.7.16.10.8.2 Sparse tcgen05.mma.sp with .kind::f16, .kind::f8f6f4, .kind::mxf8f6f4, .kind::i8

For the following .kind variants of tcgen05.mma:

- .kind::f16
- .kind::f8f8f4
- .kind::mxf8f6f4
- .kind::i8

matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A has two zero and two non-zero elements. Only the non-zero elements are stored in memory and the two 2-bit indices in the metadata indicates the position of the two non-zero elements in the four-wide chunk. The only meaningful values of the index are:

- 0b0100
- 0b1000
- 0b1100
- 0b1001
- 0b1101
- 0b0110
- 0b1110

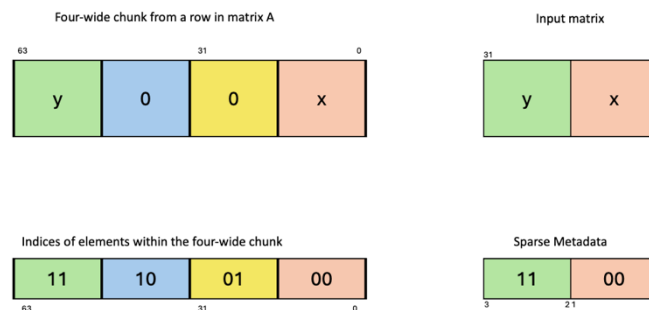


Figure 260: Sparse tcgen05.mma metadata example for f16/f8f6f4/mxf8f6f4 kind

9.7.16.10.8.3 Sparse tcgen05.mma.sp with .kind::mxf4 and .kind::mxf4nvf4

For .kind::mxf4 and .kind::mxf4nvf4, matrix A is pair-wise structured sparse at a granularity of 4:8. In other words, each chunk of eight adjacent elements in a row of matrix A has four zero and four non-zero elements. The zero and non-zero elements are clustered in sub-chunks of two elements each within the eight-wide chunk, so each two-wide sub-chunk within the eight-wide chunk must be all zeros or all non-zeros. Only the four non-zero elements are stored in memory and the two 2-bit indices in the metadata indicates the position of the two two-wide sub-chunks with non-zero values in the eight-wide chunk of a row of matrix A. The only meaningful values of the index are:

- 0b0100
- 0b1000

- ▶ 0b1100
- ▶ 0b1001
- ▶ 0b1101
- ▶ 0b0110
- ▶ 0b1110

Rest of the values result in undefined behavior.

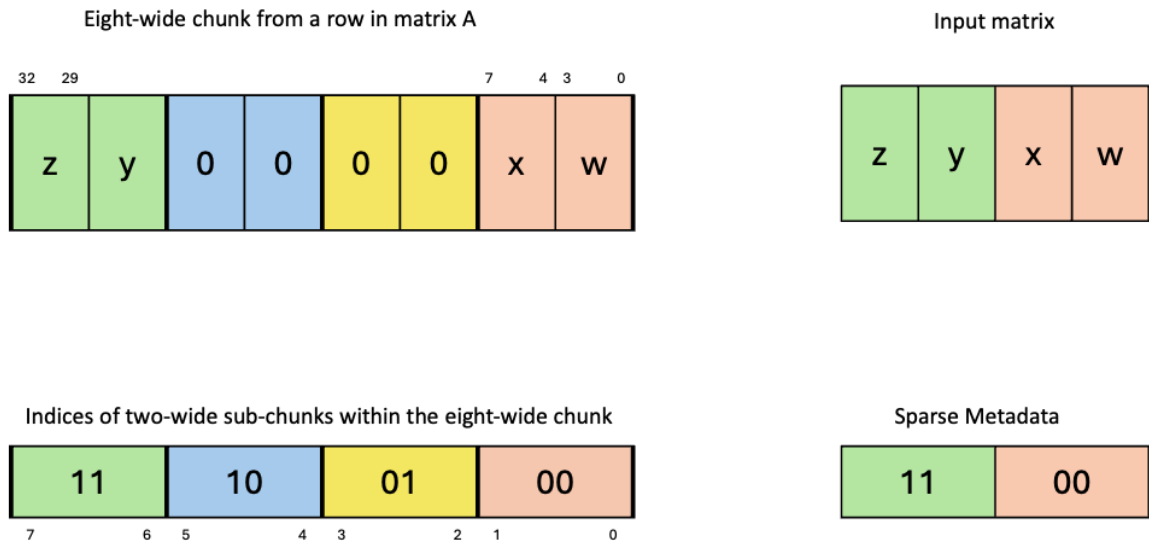


Figure 261: Sparse tcgen05.mma metadata example for mxf4 kind

9.7.16.10.8.4 Sparsity selector

The value of the sparsity selector selects the sub-columns in the Tensor Memory to form the sparsity metadata matrix, which is used with matrix A to form the multiplicand matrix.

The following shows the sparse metadata matrix layout in Tensor Memory for various MMA variants:

9.7.16.10.8.5 Layout of the Sparsity Metadata Matrix for M = 64 for .kind::f16

Figure 262 shows which sub-columns gets selected for different values of Sparsity Selector.

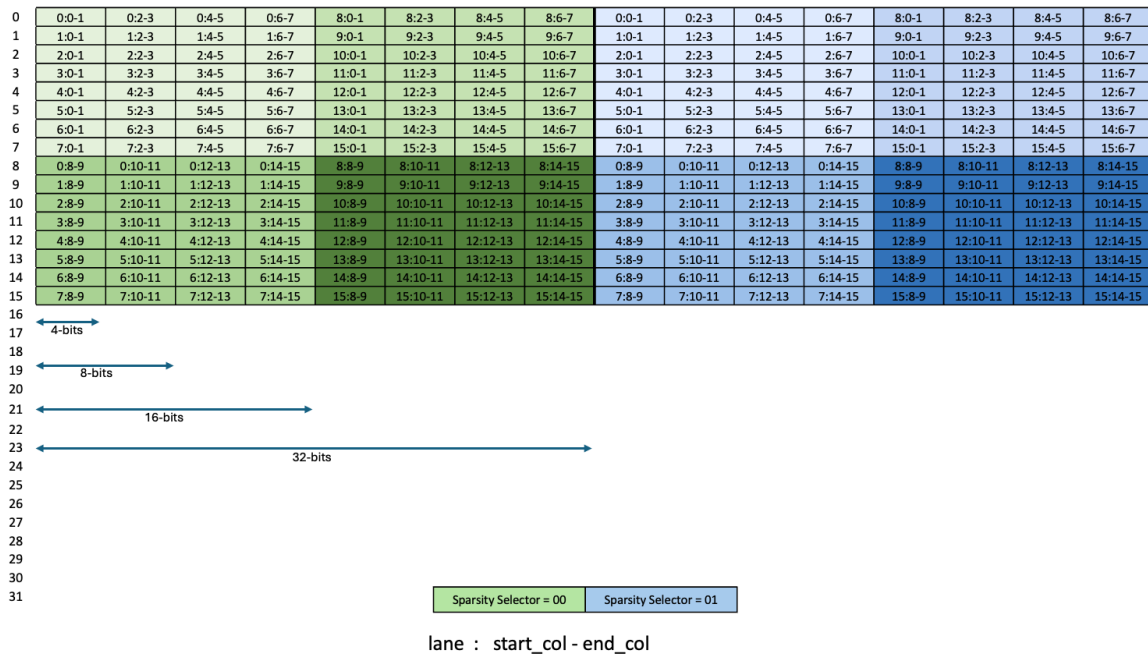


Figure 262: Sparsity Metadata Layout for M = 64 for .kind::f16

9.7.16.10.8.6 Layout of the Sparsity Metadata Matrix for M = 128 / M = 256 for .kind::f16

Figure 263 shows which sub-columns gets selected for different values of Sparsity Selector.

9.7.16.10.8.7 Layout of the Sparsity Metadata Matrix for M = 64 for .kind::tf32

Figure 264 shows which sub-columns gets selected for different values of Sparsity Selector.

9.7.16.10.8.8 Layout of the Sparsity Metadata Matrix for M = 128 / M = 256 for .kind::tf32

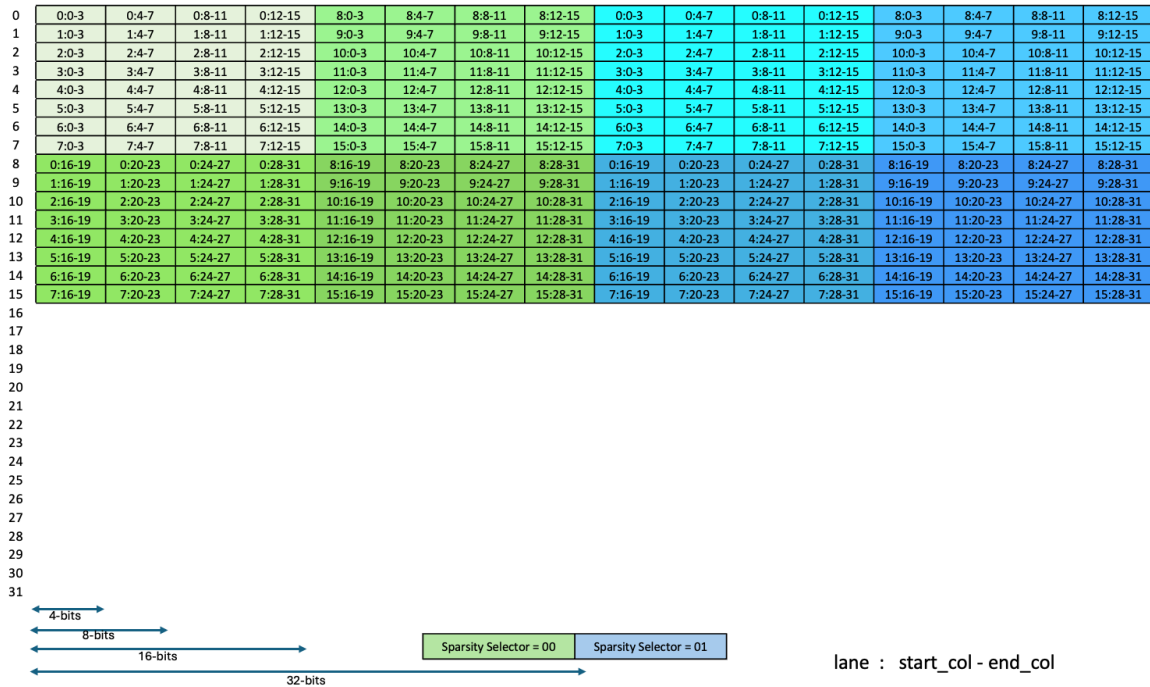
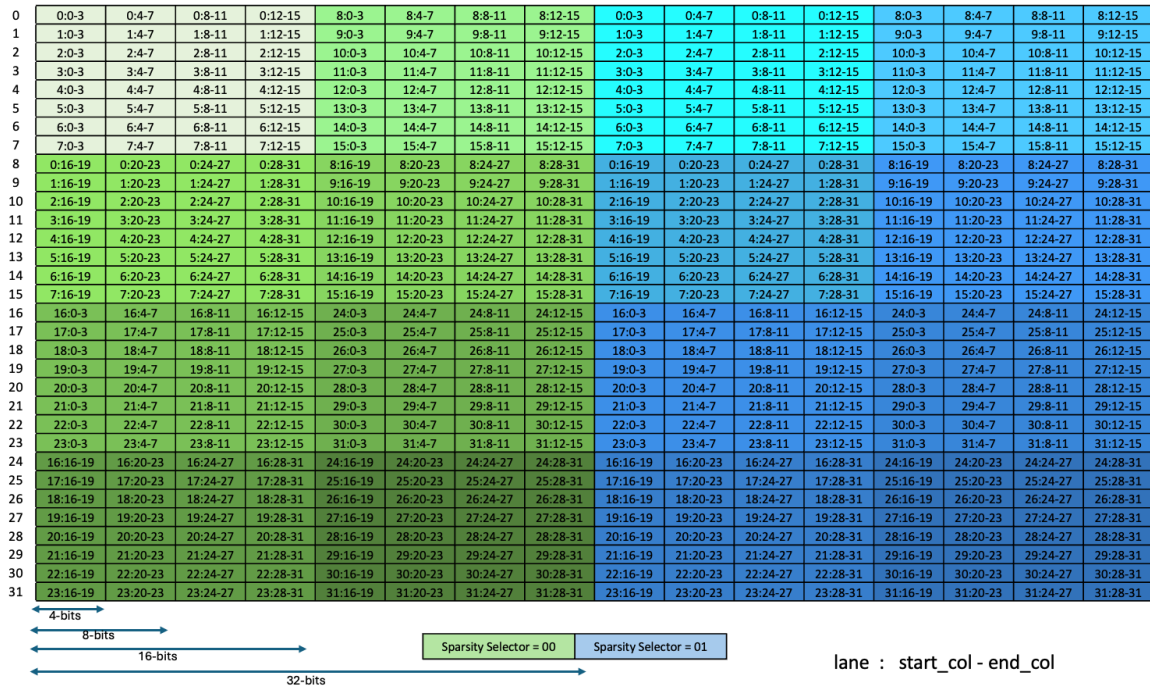
Figure 265 shows which sub-columns gets selected for different values of Sparsity Selector.

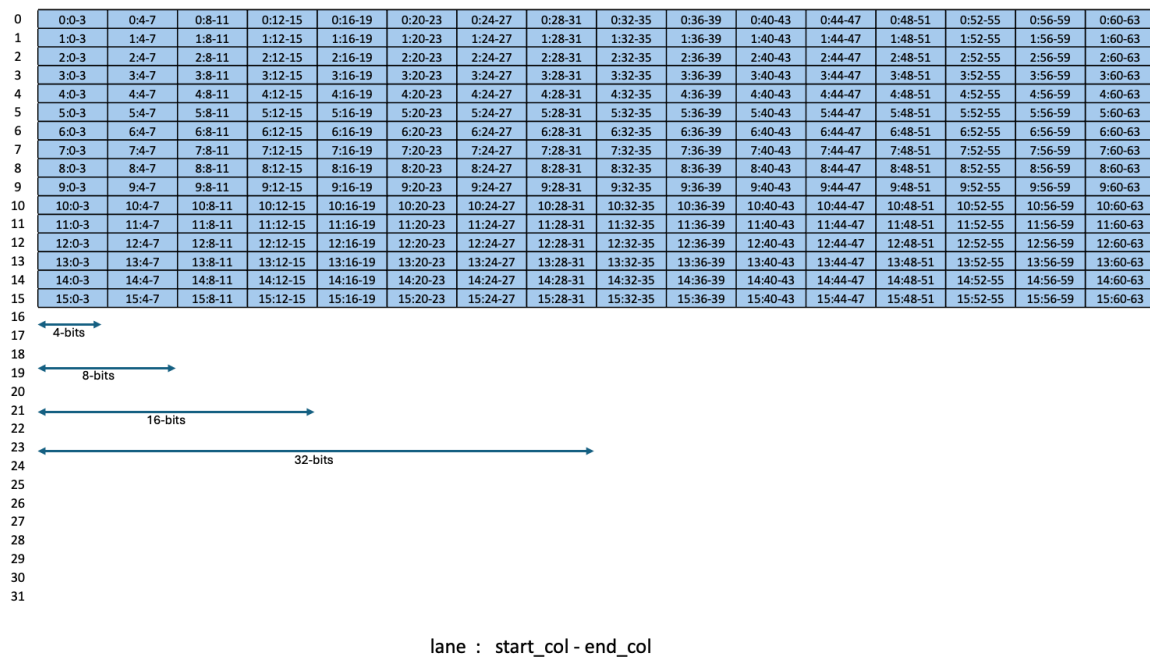
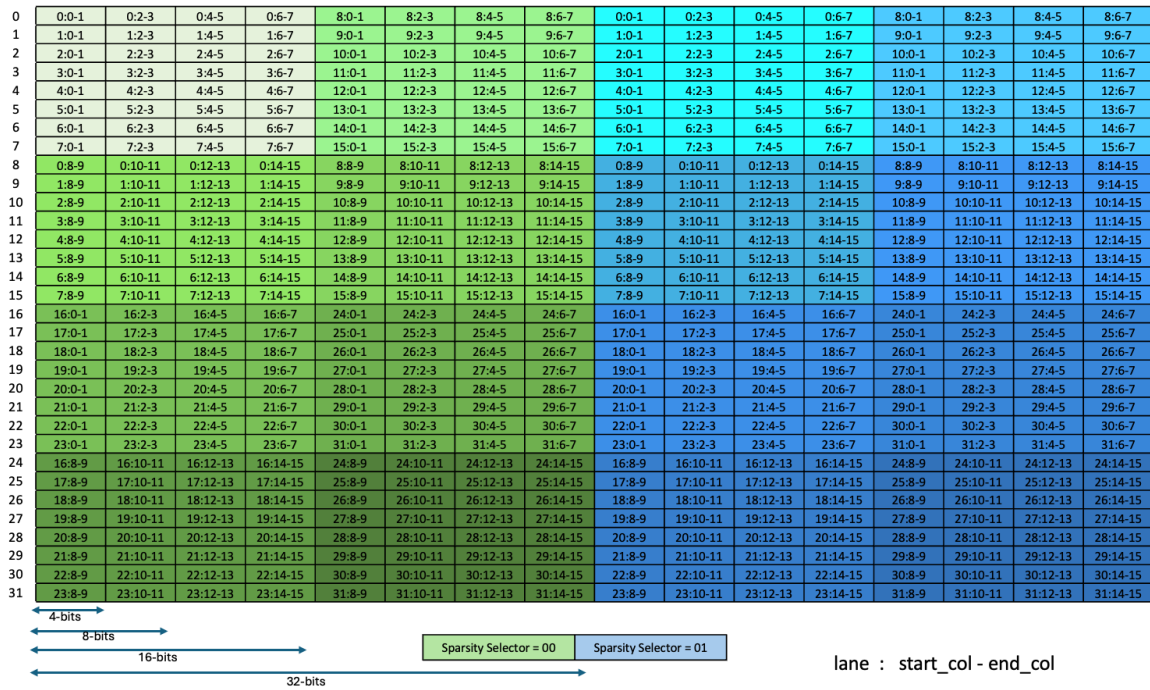
9.7.16.10.8.9 Layout of the Sparsity Metadata Matrix for M = 64 for .kind::f8f6f4, .kind::mxf8f6f4, .kind::i8, .kind::mxf4, .kind::mxf4nrvf4

The value of the sparsity selector:

- must be 0 for .kind::i8 and .kind::f8f6f4
- is assumed to be 0 for .kind::mxf8f6f4, .kind::mxf4 and .kind::mxf4nrvf4

and all of the columns are selected as shown in Figure 266





9.7.16.10.8.10 Layout of the Sparsity Metadata Matrix for M = 128 / M = 256 for .kind::f8f6f4, .kind::mx8f6f4, .kind::i8, .kind::mx4, .kind::mx4nvvf4

The value of the sparsity selector:

- must be 0 for .kind::i8 and .kind::f8f6f4
- is assumed to be 0 for .kind::mx8f6f4, .kind::mx4 and .kind::mx4nvvf4

and all of the columns are selected as shown in Figure 267

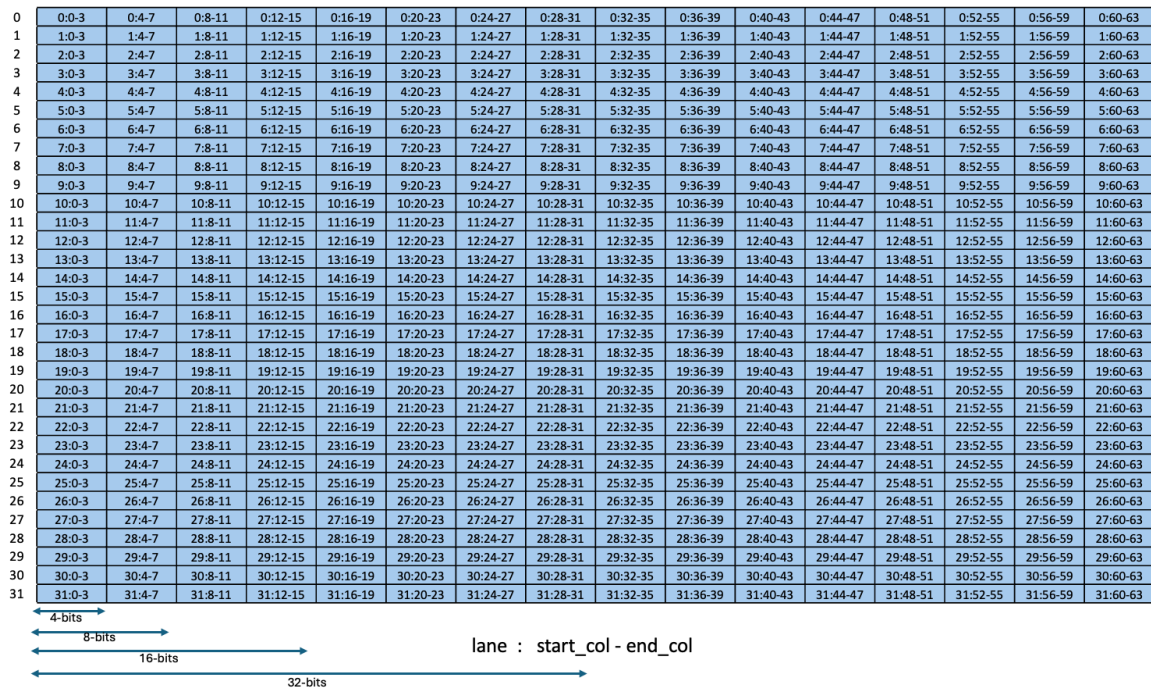


Figure 267: Sparsity Metadata Layout for M = 128 / M = 256 for .kind::f8f6f4, .kind::mx8f6f4, .kind::i8, .kind::mx4, .kind::mx4nvvf4

9.7.16.10.8.11 Alignment restriction

The layouts which utilize only half the datapath lanes as specified in *Data Path Layout Organization*, i.e. *Layout F* and *Layout C*, must use the same alignment across matrices A, D and the sparsity metadata matrix.

9.7.16.10.9 TensorCore 5th Generation of MMA Instructions

9.7.16.10.9.1 TensorCore 5th Generation Instructions: tcgen05.mma

tcgen05.mma

Perform the 5th generation of matrix multiply and accumulate operation.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.cta_group.kind    [d-tmem], a-desc, b-desc, idesc,
                               { disable-output-lane }, enable-input-d {, scale-input-d}
↪;

tcgen05.mma.cta_group.kind    [d-tmem], [a-tmem], b-desc, idesc,
                               { disable-output-lane }, enable-input-d {, scale-input-d}
↪;

.kind      = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.cta_group = { .cta_group::1, .cta_group::2 }

-----

// 2. Floating-point type with block scaling:

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem], a-desc, b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem], [a-tmem], b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

.kind = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.cta_group      = { .cta_group::1, .cta_group::2 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }

-----

// 3. Convolution MMA for floating-point type without block scaling:

tcgen05.mma.cta_group.kind.collector_usage [d-tmem], a-desc, b-desc, idesc,
                                             { disable-output-lane }, enable-input-d {,
↪scale-input-d};

tcgen05.mma.cta_group.kind{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc, idesc,
                                                       { disable-output-lane }, enable-
↪input-d {, scale-input-d};

tcgen05.mma.cta_group.kind.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc, idesc,
                                                       { disable-output-lane }, enable-
↪input-d {, scale-input-d};

.kind      = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.cta_group = { .cta_group::1, .cta_group::2 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op             = { ::fill, ::use, ::lastuse, ::discard* }

-----
```

(continues on next page)

(continued from previous page)

```
// 4. Activation Stationary MMA for floating-point type with block scaling:

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                [d-tmem], a-desc, b-desc, idesc,
                                [scale-A-tmem], [scale-B-tmem], enable-
↪input-d;

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                [d-tmem], [a-tmem], b-desc, idesc,
                                [scale-A-tmem], [scale-B-tmem], enable-
↪input-d;

.cta_group      = { .cta_group::1,   .cta_group::2 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }
.kind           = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op            = { ::fill, ::use, ::lastuse, ::discard* }

-----

// 5. Integer type:

tcgen05.mma.cta_group.kind::i8 [d-tmem], a-desc, b-desc, idesc,
                                { disable-output-lane }, enable-input-d;

tcgen05.mma.cta_group.kind::i8 [d-tmem], [a-tmem], b-desc, idesc,
                                { disable-output-lane }, enable-input-d;

.cta_group = { .cta_group::1,   .cta_group::2 }

-----

// 6. Convolution MMA for integer type:

tcgen05.mma.cta_group.kind::i8.collector_usage [d-tmem], a-desc, b-desc,
↪idesc,
                                { disable-output-lane },
↪enable-input-d;

tcgen05.mma.cta_group.kind::i8.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc,
↪idesc,
                                { disable-output-lane },
↪enable-input-d;

tcgen05.mma.cta_group.kind::i8{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc,
↪idesc,
                                { disable-output-lane },
↪enable-input-d;

.cta_group      = { .cta_group::1,   .cta_group::2 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op            = { ::fill, ::use, ::lastuse, ::discard* }
```

Description

Instruction `tcgen05.mma` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$ where the A matrix is $M \times K$, the B matrix is $K \times N$, and the D matrix is $M \times N$.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The optional immediate argument `scale-input-d` can be specified to scale the input matrix D as follows: $D = A * B + D * (2^{\text{scale-input-d}})$

The valid range of values for argument `scale-input-d` is [0, 15]. The argument `scale-input-d` is only valid for `.kind::tf32` and `.kind::f16`.

The 32-bit register operand `idesc` is the instruction descriptor as described in [Instruction descriptor](#), specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the [Tensor Memory](#) of the executing thread's CTA only. The qualifier `.cta_group::2` specifies that the matrix multiply and accumulate operation is performed on the [Tensor Memory](#) of the executing thread's CTA and its [peer CTA](#).

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section [Issue Granularity](#).

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the [Instruction descriptor](#).

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the [Tensor Memory](#). The address operand `a-tmem` specifies the address of the matrix A in the [Tensor Memory](#). The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in [Matrix Descriptors](#).

The vector operand `disable-output-lane` specifies the lane(s) in the [Tensor Memory](#) that should be not be updated with the resultant matrix D. Elements of the vector operand `disable-output-lane` forms a mask where each bit corresponds to a lane of the [Tensor Memory](#), with least significant bit of the first element of the vector (leftmost in syntax) corresponding to the lane 0 of the [Tensor Memory](#). If a bit in the mask is 1, then the corresponding lane in the Tensor Memory for the resultant matrix D will not be updated. The size of the vector is as follows:

<code>.cta_group</code>	Size of the vector <code>disable-output-lane</code>
<code>::1</code>	4
<code>::2</code>	8

Qualifier `.block_scale` specifies that the matrices A and B are scaled with `scale_A` and `scale_B` matrices respectively before performing the matrix multiply and accumulate operation as specified in the section [Block Scaling for tcgen05.mma.sync](#). The address operand `scale-A-tmem` and `scale-B-tmem` specify the base address the matrices `scale_A` and `scale_B` respectively in the [Tensor Memory](#).

For qualifier `.scale_vectorsize`,

- If `.scale_vec::NX` is specified: N specifies the number of columns in `scale_A` matrix and number of rows in `scale_B` matrix.
- If `.blockN` is specified: N specifies the block size for which single scale factor will be applied. In this form, value of N is same as the K-dimension / (N of `.scale_vec::NX`).

Aliased `.scale_vectorsize` variants:

1. `.block16` is aliased with:
 1. `.scale_vec::4X` when `.kind = .kind::mxf4nvf4` and $K = 64$ or 128
2. `.block32` is aliased with:
 1. `.scale_vec::1X` when `.kind = .kind::mxf8f6f4` for all supported values of K
 2. `.scale_vec::2X` when `.kind = .kind::mxf4` or `.kind::mxf4nvf4` and $K = 64$ or 128

The valid combinations of MMA-kind and `.scale_vectorsize` are described in [Table 58](#). For `.kind::mxf4` when the qualifier `.scale_vectorsize` is not specified, then it defaults to `.block32`. For `.kind::mxf4nvf4`, the qualifier `.scale_vectorsize` must be explicitly specified.

The qualifier `.ashift` shifts the rows of the A matrix down by one row, except for the last row in the [Tensor Memory](#). Qualifier `.ashift` is only allowed with $M = 128$ or $M = 256$.

The qualifier `.collector_usage` specifies the usage of collector buffer for matrix A. Following collector buffer operations can be specified:

<code>.collector_usage</code>	Semantics
<code>.collector::a::fill</code>	Specifies that the A matrix read from the memory should be filled in collector buffer.
<code>.collector::a::use</code>	Specifies that the A matrix can be read from the collector buffer. This requires a previous fill to the collector buffer to be still valid.
<code>.collector::a::lastuse</code>	Specifies that the A matrix can be read from the collector buffer and the contents of the collector buffer can be discarded. This requires a previous fill to the collector buffer to be valid till the collector buffer is read.
<code>.collector::a::discard</code>	Specifies that the contents of the collector buffer for A can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::a::discard`. It is illegal to specify either of `.collector::a::use` or `.collector::a::fill` along with `.ashift`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Qualifier `.kind::mxf4nvf4` introduced in PTX ISA version 8.7.

Qualifiers `.block16` and `.block32` introduced in PTX ISA version 8.8.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Argument `scale-input-d` requires `sm_100a` and is supported on `sm_100f` or higher in the same family from PTX ISA version 8.8.

For `.scale_vectorsize`,

- ▶ `.scale_vec::1X`, `.scale_vec::2X`, `.scale_vec::4X` requires `sm_100a`.
- ▶ `.block16`, `.block32` requires `sm_100f` or `sm_110f`.

For Target ISA details on matrix shape, check [Target ISA Note](#).

For Target ISA details on shared memory descriptor, check [Target ISA Note](#).

Examples

```
tcgen05.mma.cta_group::1.kind::tf32      [taddr0], adesc, bdesc, idesc, {m0, m1, m2,  
↪ m3}, p;  
tcgen05.mma.cta_group::1.kind::mxf8f6f4 [taddr2], [taddr1], bdesc, idesc,  
                                         [tmem_scaleA], [tmem_scaleB], p;  
  
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];  
  
loop:  
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;  
@!p bra loop;
```

9.7.16.10.9.2 TensorCore 5th Generation Instructions: `tcgen05.mma.sp`

`tcgen05.mma.sp`

Perform the 5th generation of matrix multiply and accumulate operation with sparse A matrix.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.sp.cta_group.kind [d-tmem], a-desc, b-desc, [sp-meta-tmem] , idesc,
                               { disable-output-lane }, enable-input-d{, scale-input-
↪d};

tcgen05.mma.sp.cta_group.kind [d-tmem], [a-tmem], b-desc, [sp-meta-tmem] , idesc,
                               { disable-output-lane }, enable-input-d{, scale-input-
↪d};

.kind          = { .kind::f16, , .kind::tf32, .kind::f8f6f4 }
.cta_group     = { .cta_group::1, .cta_group::2 }

-----

// 2. Floating-point type with block scaling:

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}
                                         [d-tmem], a-desc, b-desc , [sp-meta-tmem] ,
↪ idesc,
                                         [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}
                                         [d-tmem], [a-tmem], b-desc , [sp-meta-tmem] ,
↪ idesc,
                                         [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }
.cta_group        = { .cta_group::1, .cta_group::2 }
.kind             = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nfv4 }

-----

// 3. Convolution MMA with floating-point type without block scaling:

tcgen05.mma.sp.cta_group.kind.collector_usage      [d-tmem], a-desc, b-desc,
                                                    [sp-meta-tmem] , idesc,
↪enable-input-d                                     { disable-output-lane },
                                                    {, scale-input-d};

tcgen05.mma.sp.cta_group.kind.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem] , idesc,
↪enable-input-d                                     { disable-output-lane },
```

(continues on next page)

(continued from previous page)

```

                                                    {, scale-input-d};

tcgen05.mma.sp.cta_group.kind{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc,
                                                    [sp-meta-tmem] , idesc,
                                                    { disable-output-lane },
↪enable-input-d
                                                    {, scale-input-d};

.kind          = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op             = { ::fill, ::use, ::lastuse, ::discard* }

-----

// 4. Activation Stationary MMA with floating-point type with block scaling:

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                                    [d-tmem], a-desc, b-desc , [sp-meta-tmem] ,
↪ idesc,
                                                    [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                                    [d-tmem], [a-tmem], b-desc , [sp-meta-tmem] ,
↪ idesc,
                                                    [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

.kind = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }
.collector_usage = { .collector::buffer::op }
::buffer         = { ::a }
::op              = { ::fill, ::use, ::lastuse, ::discard* }

-----

// 5. Integer type:

tcgen05.mma.sp.cta_group.kind::i8 [d-tmem], a-desc, b-desc, [sp-meta-tmem] , idesc,
                                { disable-output-lane }, enable-input-d;

tcgen05.mma.sp.cta_group.kind::i8 [d-tmem], [a-tmem], b-desc, [sp-meta-tmem] , idesc,
                                { disable-output-lane }, enable-input-d;

.cta_group      = { .cta_group::1, .cta_group::2 }

-----

// 6. Convolution MMA with Integer type:

tcgen05.mma.sp.cta_group.kind::i8.collector_usage [d-tmem], a-desc, b-desc,
                                                    [sp-meta-tmem] , idesc,
                                                    { disable-output-lane },
↪enable-input-d;

```

(continues on next page)

(continued from previous page)

```

tcgen05.mma.sp.cta_group.kind::i8.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem], idesc ,
                                                         { disable-output-lane },
↪enable-input-d;

tcgen05.mma.sp.cta_group.kind::i8{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem], idesc ,
                                                         { disable-output-lane },
↪enable-input-d;

.collector_usage = { .collector::buffer::op }
::buffer         = { ::a }
::op              = { ::fill, ::use, ::lastuse, ::discard* }

```

Description

Instruction `tcgen05.mma.sp` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation of the form $D = A * B + D$ where the A matrix is $M \times (K/2)$, the B matrix is $K \times N$, and the D matrix is $M \times N$. [Sparse Matrices](#) describes the details of the sparsity.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The optional immediate argument `scale-input-d` can be specified to scale the input matrix D as follows: $D = A * B + D * (2^{\text{scale-input-d}})$

The valid range of values for argument `scale-input-d` is [0, 15]. The argument `scale-input-d` is only valid for `.kind::tf32` and `.kind::f16`.

The 32-bit register operand `idesc` is the instruction descriptor as described in [Instruction descriptor](#), specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the [Tensor Memory](#) of the executing thread's CTA only. The qualifier `.cta_group::2` specifies that the matrix multiply and accumulate operation is performed on the [Tensor Memory](#) of the executing thread's CTA and its [peer CTA](#).

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma.sp` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma.sp` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section [Issue Granularity](#).

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the [Instruction descriptor](#).

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the [Tensor Memory](#). The address operand `a-tmem` specifies the address of the matrix A in the [Tensor Memory](#). The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in [Matrix Descriptors](#).

The vector operand `disable-output-lane` specifies the lane(s) in the [Tensor Memory](#) that should be not be updated with the resultant matrix D. Elements of the vector operand `disable-output-lane`

forms a mask where each bit corresponds to a lane of the *Tensor Memory*. with least significant bit of the first element of the vector (leftmost in syntax) corresponding to the lane 0 of the Tensor Memory. If a bit in the mask is 1, then the corresponding lane in the Tensor Memory for the resultant matrix D will not be updated. The size of the vector is as follows:

.cta_group	Size of the vector disable-output-lane
::1	4
::2	8

Qualifier `.block_scale` specifies that the matrices A and B are scaled with `scale_A` and `scale_B` matrices respectively before performing the matrix multiply and accumulate operation as specified in the section *Block Scaling for tcgen05.mma.sync*. The address operand `scale-A-tmem` and `scale-B-tmem` specify the base address the matrices `scale_A` and `scale_B` respectively in the *Tensor Memory*.

For qualifier `.scale_vectorsize`,

- If `.scale_vec : :NX` is specified: N specifies the number of columns in `scale_A` matrix and number of rows in `scale_B` matrix.
- If `.blockN` is specified: N specifies the block size for which single scale factor will be applied. In this form, value of N is same as the K-dimension / (N of `.scale_vec : :NX`).

Aliased `.scale_vectorsize` variants:

1. `.block16` is aliased with:
 1. `.scale_vec : :4X` when `.kind = .kind : :mxf4nvf4` and $K = 64$ or 128
2. `.block32` is aliased with:
 1. `.scale_vec : :1X` when `.kind = .kind : :mxf8f6f4` for all supported values of K
 2. `.scale_vec : :2X` when `.kind = .kind : :mxf4` or `.kind : :mxf4nvf4` and $K = 64$ or 128

The valid combinations of MMA-kind and `.scale_vectorsize` are described in *Table 58*. For `.kind : :mxf4` when the qualifier `.scale_vectorsize` is not specified, then it defaults to `.block32`. For `.kind : :mxf4nvf4`, the qualifier `.scale_vectorsize` must be explicitly specified.

The qualifier `.ashift` shifts the rows of the A matrix down by one row, except for the last row in the *Tensor Memory*. Qualifier `.ashift` is only allowed with $M = 128$ or $M = 256$.

The qualifier `.collector_usage` specifies the usage of collector buffer for matrix A. Following collector buffer operations can be specified:

.collector_usage	Semantics
<code>.collector : :a : :fill</code>	Specifies that the A matrix read from the memory should be filled in collector buffer.
<code>.collector : :a : :use</code>	Specifies that the A matrix can be read from the collector buffer. This requires a previous fill to the collector buffer to be still valid.

continues on next page

Table 61 – continued from previous page

.collector_usage	Semantics
<code>.collector::a::lastuse</code>	Specifies that the A matrix can be read from the collector buffer and the contents of the collector buffer can be discarded. This requires a previous fill to the collector buffer to be valid till the collector buffer is read.
<code>.collector::a::discard</code>	Specifies that the contents of the collector buffer for A can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::a::discard`. It is illegal to specify either of `.collector::a::use` or `.collector::a::fill` along with `.ashift`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Qualifier `.kind::mxf4nvf4` introduced in PTX ISA version 8.7.

Qualifiers `.block16` and `.block32` introduced in PTX ISA version 8.8.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8/.kind::mxf4nvf4/.kind::mxf4`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Qualifiers `.kind::mxf4nvf4` and `.kind::mxf4` are supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_103a`
- ▶ `sm_110a`

Argument `scale-input-d` requires `sm_100a` and is supported on `sm_100f` or higher in the same family from PTX ISA version 8.8.

For `.scale_vectorsize`,

- ▶ `.scale_vec::1X`, `.scale_vec::2X`, `.scale_vec::4X` requires `sm_100a`.
- ▶ `.block16`, `.block32` requires `sm_100f` or `sm_110f`.

For Target ISA details on matrix shape, check [Target ISA Note](#).

For Target ISA details on shared memory descriptor, check [Target ISA Note](#).

Examples

```
tcgen05.mma.sp.cta_group::1.kind::f16      [taddr0], adesc, bdesc, [tmem_spmeta0],
↪ idesc, p;

tcgen05.mma.sp.cta_group::1.kind::mxf8f6f4.collector::a:fill
                                         [taddr2], [taddr1], bdesc, [tmem_
↪ spmeta1], idesc,
                                         [tmem_scaleA], [tmem_scaleB], p;

tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;
@!p bra loop;
```

9.7.16.10.9.3 TensorCore 5th Generation Instructions: `tcgen05.mma.ws`

`tcgen05.mma.ws`

Perform the 5th generation of weight stationary convolution matrix multiply and accumulate operation.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.ws.cta_group::1.kind{.collector_usage} [d-tmem], a-desc, b-desc,
↪ idesc,                                           enable-input-d {, zero-column-
↪ mask-desc };

tcgen05.mma.ws.cta_group::1.kind{.collector_usage} [d-tmem], [a-tmem], b-desc,
↪ idesc,                                           enable-input-d {, zero-column-
↪ mask-desc };

.kind = { .kind::f16, .kind::tf32, .kind::f8f6f4 }

-----

// 2. Integer type:
```

(continues on next page)

(continued from previous page)

```

tcgen05.mma.ws.cta_group::1.kind::i8{.collector_usage} [d-tmem], a-desc, b-desc,
↪ idesc,
enable-input-d {, zero-column-
↪ mask-desc};

tcgen05.mma.ws.cta_group::1.kind::i8{.collector_usage} [d-tmem], [a-tmem], b-desc,
↪ idesc,
enable-input-d {, zero-column-
↪ mask-desc};

.collector_usage = { .collector::buffer::op }
::buffer = { ::b0, ::b1, ::b2, ::b3 }
::op = { ::fill, ::use, ::lastuse, ::discard}

```

Description

Instruction `tcgen05.mma.ws` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$ where the A matrix is $M \times K$, the B matrix is $K \times N$, and the D matrix is $M \times N$.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The 32-bit register operand `idesc` is the instruction descriptor as described in *Instruction descriptor*, specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the *Tensor Memory* of the executing thread's CTA only.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma.ws` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma.ws` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section *Issue Granularity*.

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the *Instruction descriptor*.

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the *Tensor Memory*. The address operand `a-tmem` specifies the address of the matrix A in the *Tensor Memory*. The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in *Matrix Descriptors*.

The optional operand `zero-column-mask-desc` is a 64-bit register which specifies the *Zero-Column Mask Descriptor*. The zero-column mask descriptor is used to generate a mask that specifies which columns of B matrix will have zero value for the matrix multiply and accumulate operation regardless of the values present in the shared memory.

The qualifier `.collector_usage` specifies the usage of collector buffer for Matrix B. Following collector buffer operations can be specified:

.collector_usage	Semantics
<code>.collector::bN::fill</code>	Specifies that the B matrix read from the memory should be filled in collector buffer #N.
<code>.collector::bN::use</code>	Specifies that the B matrix can be read from the collector buffer #N. This requires a previous fill to the collector buffer #N to be still valid.
<code>.collector::bN::lastuse</code>	Specifies that the B matrix can be read from the collector buffer #N after which the contents of the collector buffer #N can be discarded. This requires a previous fill to the collector buffer #N to be valid till the collector buffer #N is read.
<code>.collector::bN::discard</code>	Specifies that the contents of the collector buffer #N can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::b0::discard`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Examples

```
tcgen05.mma.ws.cta_group::1.kind::i8.collector::b2:use [taddr2], [taddr1], bdesc,
↳ idesc, p;
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;
@!p bra loop;
```

9.7.16.10.9.4 TensorCore 5th Generation Instructions: `tcgen05.mma.ws.sp`

`tcgen05.mma.ws.sp`

Perform the 5th generation of weight stationary convolution matrix multiply and accumulate operation with sparse A matrix.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.ws.sp.cta_group::1.kind{.collector_usage} [d-tmem], a-desc, b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-column-
↳ mask-desc};

tcgen05.mma.ws.sp.cta_group::1.kind{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-column-
↳ mask-desc};

.kind = { .kind::f16, .kind::tf32, .kind::f8f6f4 }

-----

// 2. Integer type:

tcgen05.mma.ws.sp.cta_group::1.kind::i8{.collector_usage} [d-tmem], a-desc, b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-
↳ column-mask-desc};

tcgen05.mma.ws.sp.cta_group::1.kind::i8{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-
↳ column-mask-desc};

.collector_usage = { .collector::buffer::op }
::buffer = { ::b0, ::b1, ::b2, ::b3 }
::op      = { ::fill, ::use, ::lastuse, ::discard}
```

Description

Instruction `tcgen05.mma.ws.sp` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$ where the A matrix is $M \times (K/2)$, the B matrix is $K \times N$, and the D matrix is $M \times N$. *Sparse Matrices* describes the details of the sparsity.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The 32-bit register operand `idesc` is the instruction descriptor as described in *Instruction descriptor*, specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the Tensor Memory of the executing thread's CTA only.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma.ws.sp` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma.ws.sp` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section *Issue Granularity*.

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the *Instruction descriptor*.

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the *Tensor Memory*. The address operand `a-tmem` specifies the address of the matrix A in the *Tensor Memory*. The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in *Matrix Descriptors*.

The optional operand `zero-column-mask-desc` is a 64-bit register which specifies the *Zero-Column Mask Descriptor*. The zero-column mask descriptor is used to generate a mask that specifies which columns of B matrix will have zero value for the matrix multiply and accumulate operation regardless of the values present in the shared memory.

The qualifier `.collector_usage` specifies the usage of collector buffer for Matrix B. Following collector buffer operations can be specified:

<code>.collector_usage</code>	Semantics
<code>.collector::bN::fill</code>	Specifies that the B matrix read from the memory should be filled in collector buffer #N.
<code>.collector::bN::use</code>	Specifies that the B matrix can be read from the collector buffer #N. This requires a previous fill to the collector buffer #N to be still valid.

continues on next page

Table 63 – continued from previous page

.collector_usage	Semantics
<code>.collector::bN::lastuse</code>	Specifies that the B matrix can be read from the collector buffer #N after which the contents of the collector buffer #N can be discarded. This requires a previous fill to the collector buffer #N to be valid till the collector buffer #N is read.
<code>.collector::bN::discard</code>	Specifies that the contents of the collector buffer #N can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::b0::discard`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Examples

```
tcgen05.mma.ws.sp.cta_group::1.kind::tf32.collector::b1::fill [taddr1], [taddr0],
↳ bdesc,
                                                    [tmem_spmeta0], idesc,
↳ p;
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];
loop:
```

(continues on next page)

(continued from previous page)

```
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;  
@!p bra loop;
```

9.7.16.11 TensorCore 5th Generation Specialized Synchronization Operations

9.7.16.11.1 TensorCore 5th Generation Instructions: `tcgen05.fence`

`tcgen05.fence`

Specialized fence for the asynchronous `tcgen05` operations.

Syntax

```
tcgen05.fence::before_thread_sync ;  
tcgen05.fence::after_thread_sync ;
```

Description

The instruction `tcgen05.fence::before_thread_sync` orders all the prior asynchronous `tcgen05` operations with respect to the subsequent `tcgen05` and the execution ordering operations.

The instruction `tcgen05.fence::after_thread_sync` orders all the subsequent asynchronous `tcgen05` operations with respect to the prior `tcgen05` and the execution ordering operations.

The `tcgen05.fence::*` instructions compose with execution ordering instructions across a thread scope and provide ordering between `tcgen05` instructions across the same scope.

The `tcgen05.fence::before_thread_sync` instructions behave as code motion fence for prior `tcgen05` instructions as they cannot be hoisted across. The `tcgen05.fence::after_thread_sync` instructions behave as code motion fence for subsequent `tcgen05` instructions as they cannot be hoisted across.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
// Producer thread:

tcgen05.cp.cta_group::1.128x256b [taddr0], sdesc0;

tcgen05.fence::before_thread_sync;
st.relaxed.b32 [flag], 1;

// Consumer thread:

loop:
ld.relaxed.b32 r, [flag];
setp.eq.u32 p, r, 1;
@!p bra loop;

tcgen05.fence::after_thread_sync;
tcgen05.mma.cta_group.kind [taddr0], adesc, bdesc, idesc, p;
```

9.7.16.12 TensorCore 5th Generation Async Synchronization Operations

9.7.16.12.1 TensorCore 5th Generation Instructions: `tcgen05.commit`

`tcgen05.commit`

Makes the mbarrier object track the completion of all prior async-tcgen05 operations initiated by the executing thread.

Syntax

```
tcgen05.commit.cta_group.completion_mechanism{.shared::cluster}{.multicast}.b64
                                                [mbar] {, ctaMask};

.completion_mechanism = { .mbarrier::arrive::one }
.cta_group             = { .cta_group::1, .cta_group::2 }
.multicast             = { .multicast::cluster }
```

Description

The instruction `tcgen05.commit` is an asynchronous instruction which makes the mbarrier object, specified by the address operand `mbar`, track the completion of all the prior asynchronous `tcgen05` operations, as listed in *mbarrier based completion mechanism*, initiated by the executing thread. Upon the completion of the tracked asynchronous `tcgen05` operations, the signal specified by the `.completion_mechanism` is triggered by the system on the mbarrier object. This instruction accesses its mbarrier operand using generic-proxy.

The instruction `tcgen05.commit.cta_group::1` tracks for the completion of all prior asynchronous `tcgen05` operations with `.cta_group::1` issued by the current thread. Similarly, the instruction `tcgen05.commit.cta_group::2` tracks for the completion of all prior asynchronous `tcgen05` operations with `.cta_group::2` issued by the current thread.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The qualifier `.mbarrier::arrive::one` indicates that upon the completion of the prior asynchronous `tcgen05` operation issued by the current thread, an arrive-on operation, with the count argument of 1, is signaled on the mbarrier object. The scope of the arrive-on operation is the cluster scope.

The optional qualifier `.multicast::cluster` allows signaling on the mbarrier objects of multiple CTAs in the cluster. Operand `ctaMask` specifies the CTAs in the cluster such that each bit position in the 16-bit `ctaMask` operand corresponds to the `%cluster_ctarank` of the destination CTA. The mbarrier signal is multicast to the same offset as `mbar` in the shared memory of each destination CTA.

If no state space is specified then *Generic Addressing* is used. If the address specified by `mbar` does not fall within the address window of `.shared::cluster` state space then the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
Example 1:
tcgen05.cp.cta_group::1.128x256b          [taddr0], sdesc0;
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj1];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj1], 0;
@!p bra loop;

Example 2:
tcgen05.mma.cta_group::2.kind::tf32      [taddr0], adesc, bdesc, idesc, p;
tcgen05.commit.cta_group::2.mbarrier::arrive::one.b64 [mbarObj2];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj2], 0;
@!p bra loop;
```

9.7.17. Stack Manipulation Instructions

The stack manipulation instructions can be used to dynamically allocate and deallocate memory on the stack frame of the current function.

The stack manipulation instructions are:

- ▶ `stacksave`
- ▶ `stackrestore`
- ▶ `alloca`

9.7.17.1 Stack Manipulation Instructions: `stacksave`

`stacksave`

Save the value of stack pointer into a register.

Syntax

```
stacksave.type d;
.type = { .u32, .u64 };
```

Description

Copies the current value of stack pointer into the destination register `d`. Pointer returned by `stacksave` can be used in a subsequent `stackrestore` instruction to restore the stack pointer. If `d` is modified prior to use in `stackrestore` instruction, it may corrupt data in the stack.

Destination operand `d` has the same type as the instruction type.

Semantics

```
d = stackptr;
```

PTX ISA Notes

Introduced in PTX ISA version 7.3.

Target ISA Notes

stacksave requires sm_52 or higher.

Examples

```
.reg .u32 rd;  
stacksave.u32 rd;  
  
.reg .u64 rd1;  
stacksave.u64 rd1;
```

9.7.17.2 Stack Manipulation Instructions: stackrestore

stackrestore

Update the stack pointer with a new value.

Syntax

```
stackrestore.type a;  
  
.type = { .u32, .u64 };
```

Description

Sets the current stack pointer to source register a.

When `stackrestore` is used with operand a written by a prior `stacksave` instruction, it will effectively restore the state of stack as it was before `stacksave` was executed. Note that if `stackrestore` is used with an arbitrary value of a, it may cause corruption of stack pointer. This implies that the correct use of this feature requires that `stackrestore.type a` is used after `stacksave.type a` without redefining the value of a between them.

Operand a has the same type as the instruction type.

Semantics

```
stackptr = a;
```

PTX ISA Notes

Introduced in PTX ISA version 7.3.

Target ISA Notes

stackrestore requires sm_52 or higher.

Examples

```
.reg .u32 ra;
stacksave.u32 ra;
// Code that may modify stack pointer
...
stackrestore.u32 ra;
```

9.7.17.3 Stack Manipulation Instructions: **alloca**

alloca

Dynamically allocate memory on stack.

Syntax

```
alloca.type ptr, size{, immAlign};

.type = { .u32, .u64 };
```

Description

The **alloca** instruction dynamically allocates memory on the stack frame of the current function and updates the stack pointer accordingly. The returned pointer **ptr** points to local memory and can be used in the address operand of **ld.local** and **st.local** instructions.

If sufficient memory is unavailable for allocation on the stack, then execution of **alloca** may result in stack overflow. In such cases, attempting to access the allocated memory with **ptr** will result in undefined program behavior.

The memory allocated by **alloca** is deallocated in the following ways:

- ▶ It is automatically deallocated when the function exits.
- ▶ It can be explicitly deallocated using **stacksave** and **stackrestore** instructions: **stacksave** can be used to save the value of stack pointer before executing **alloca**, and **stackrestore** can be used after **alloca** to restore stack pointer to the original value which was previously saved with **stacksave**. Note that accessing deallocated memory after executing **stackrestore** results in undefined behavior.

size is an unsigned value which specifies the amount of memory in number of bytes to be allocated on stack. size = 0 may not lead to a valid memory allocation.

Both ptr and size have the same type as the instruction type.

immAlign is a 32-bit value which specifies the alignment requirement in number of bytes for the memory allocated by alloca. It is an integer constant, must be a power of 2 and must not exceed 2^{23} . immAlign is an optional argument with default value being 8 which is the minimum guaranteed alignment.

Semantics

```
alloca.type ptr, size, immAlign:

a = max(immAlign, frame_align); // frame_align is the minimum guaranteed alignment

// Allocate size bytes of stack memory with alignment a and update the stack pointer.
// Since the stack grows down, the updated stack pointer contains a lower address.
stackptr = alloc_stack_mem(size, a);

// Return the new value of stack pointer as ptr. Since ptr is the lowest address of
↪the memory
// allocated by alloca, the memory can be accessed using ptr up to (ptr + size of
↪allocated memory).
stacksave ptr;
```

PTX ISA Notes

Introduced in PTX ISA version 7.3.

Target ISA Notes

alloca requires sm_52 or higher.

Examples

```
.reg .u32 ra, stackptr, ptr, size;

stacksave.u32 stackptr;    // Save the current stack pointer
alloca ptr, size, 8;       // Allocate stack memory
st.local.u32 [ptr], ra;    // Use the allocated stack memory
stackrestore.u32 stackptr; // Deallocate memory by restoring the stack pointer
```


9.7.18. Video Instructions

All video instructions operate on 32-bit register operands. However, the video instructions may be classified as either scalar or SIMD based on whether their core operation applies to one or multiple values.

The video instructions are:

- ▶ vadd, vadd2, vadd4
- ▶ vsub, vsub2, vsub4
- ▶ vmad
- ▶ vavrg2, vavrg4
- ▶ vabsdiff, vabsdiff2, vabsdiff4
- ▶ vmin, vmin2, vmin4
- ▶ vmax, vmax2, vmax4
- ▶ vshl
- ▶ vshr
- ▶ vset, vset2, vset4

9.7.18.1 Scalar Video Instructions

All scalar video instructions operate on 32-bit register operands. The scalar video instructions are:

- ▶ vadd
- ▶ vsub
- ▶ vabsdiff
- ▶ vmin
- ▶ vmax
- ▶ vshl
- ▶ vshr
- ▶ vmad
- ▶ vset

The scalar video instructions execute the following stages:

1. Extract and sign- or zero-extend byte, half-word, or word values from its source operands, to produce signed 33-bit input values.
2. Perform a scalar arithmetic operation to produce a signed 34-bit result.
3. Optionally clamp the result to the range of the destination type.
4. Optionally perform one of the following:
 - ▶ apply a second operation to the intermediate result and a third operand, or
 - ▶ truncate the intermediate result to a byte or half-word value and merge into a specified position in the third operand to produce the final result.

The general format of scalar video instructions is as follows:

```
// 32-bit scalar operation, with optional secondary operation
vop.dtype.atype.btype{.sat}      d, a{.asel}, b{.bsel};
vop.dtype.atype.btype{.sat}.secop d, a{.asel}, b{.bsel}, c;

// 32-bit scalar operation, with optional data merge
vop.dtype.atype.btype{.sat}      d.dsel, a{.asel}, b{.bsel}, c;

.dtype = .atype = .btype = { .u32, .s32 };
.dsel = .asel = .bsel = { .b0, .b1, .b2, .b3, .h0, .h1 };
.secop = { .add, .min, .max };
```

The source and destination operands are all 32-bit registers. The type of each operand (.u32 or .s32) is specified in the instruction type; all combinations of dtype, atype, and btype are valid. Using the atype/btype and asel/bsel specifiers, the input values are extracted and sign- or zero-extended internally to .s33 values. The primary operation is then performed to produce an .s34 intermediate result. The sign of the intermediate result depends on dtype.

The intermediate result is optionally clamped to the range of the destination type (signed or unsigned), taking into account the subword destination size in the case of optional data merging.

```
.s33 optSaturate( .s34 tmp, Bool sat, Bool sign, Modifier dsel ) {
    if ( !sat ) return tmp;

    switch ( dsel ) {
        case .b0, .b1, .b2, .b3:
            if ( sign ) return CLAMP( tmp, S8_MAX, S8_MIN );
            else       return CLAMP( tmp, U8_MAX, U8_MIN );
        case .h0, .h1:
            if ( sign ) return CLAMP( tmp, S16_MAX, S16_MIN );
            else       return CLAMP( tmp, U16_MAX, U16_MIN );
        default:
            if ( sign ) return CLAMP( tmp, S32_MAX, S32_MIN );
            else       return CLAMP( tmp, U32_MAX, U32_MIN );
    }
}
```

This intermediate result is then optionally combined with the third source operand using a secondary arithmetic operation or subword data merge, as shown in the following pseudocode. The sign of the third operand is based on dtype.

```
.s33 optSecOp( Modifier secop, .s33 tmp, .s33 c ) {
    switch ( secop ) {
        .add:    return tmp + c;
        .min:    return MIN( tmp, c );
        .max:    return MAX( tmp, c );
        default: return tmp;
    }
}
```

```
.s33 optMerge( Modifier dsel, .s33 tmp, .s33 c ) {
    switch ( dsel ) {
        case .h0: return ((tmp & 0xffff)          | (0xffff0000 & c));
        case .h1: return ((tmp & 0xffff) << 16) | (0x0000ffff & c);
        case .b0: return ((tmp & 0xff)           | (0xffffffff00 & c));
        case .b1: return ((tmp & 0xff) << 8)    | (0xffff00ff & c);
```

(continues on next page)

(continued from previous page)

```

        case .b2: return ((tmp & 0xff) << 16) | (0xff00ffff & c);
        case .b3: return ((tmp & 0xff) << 24) | (0x00ffffff & c);
        default: return tmp;
    }
}

```

The lower 32-bits are then written to the destination operand.

9.7.18.1.1 Scalar Video Instructions: **vadd**, **vsub**, **vabsdiff**, **vmin**, **vmax**

vadd, **vsub**

Integer byte/half-word/word addition/subtraction.

vabsdiff

Integer byte/half-word/word absolute value of difference.

vmin, **vmax**

Integer byte/half-word/word minimum/maximum.

Syntax

```

// 32-bit scalar operation, with optional secondary operation
vop.dtype.atype.btype{.sat}      d, a{.asel}, b{.bsel};
vop.dtype.atype.btype{.sat}.op2  d, a{.asel}, b{.bsel}, c;

// 32-bit scalar operation, with optional data merge
vop.dtype.atype.btype{.sat} d.dsel, a{.asel}, b{.bsel}, c;

vop    = { vadd, vsub, vabsdiff, vmin, vmax };
.dtype = .atype = .btype = { .u32, .s32 };
.dsel  = .asel  = .bsel  = { .b0, .b1, .b2, .b3, .h0, .h1 };
.op2   = { .add, .min, .max };

```

Description

Perform scalar arithmetic operation with optional saturate, and optional secondary arithmetic operation or subword data merge.

Semantics

```
// extract byte/half-word/word and sign- or zero-extend
// based on source operand type
ta = partSelectSignExtend( a, atype, asel );
tb = partSelectSignExtend( b, btype, bsel );

switch ( vop ) {
    case vadd:    tmp = ta + tb;
    case vsub:    tmp = ta - tb;
    case vabsdiff: tmp = | ta - tb |;
    case vmin:    tmp = MIN( ta, tb );
    case vmax:    tmp = MAX( ta, tb );
}
// saturate, taking into account destination type and merge operations
tmp = optSaturate( tmp, sat, isSigned(dtype), dsel );
d = optSecondaryOp( op2, tmp, c ); // optional secondary operation
d = optMerge( dsel, tmp, c );      // optional merge with c operand
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

vadd, vsub, vabsdiff, vmin, vmax require sm_20 or higher.

Examples

```
vadd.s32.u32.s32.sat    r1, r2.b0, r3.h0;
vsub.s32.s32.u32.sat    r1, r2.h1, r3.h1;
vabsdiff.s32.s32.s32.sat r1.h0, r2.b0, r3.b2, c;
vmin.s32.s32.s32.sat.add r1, r2, r3, c;
```

9.7.18.1.2 Scalar Video Instructions: vshl, vshr

vshl, vshr

Integer byte/half-word/word left/right shift.

Syntax

```
// 32-bit scalar operation, with optional secondary operation
vop.dtype.atype.u32{.sat}.mode      d, a{.asel}, b{.bsel};
vop.dtype.atype.u32{.sat}.mode.op2  d, a{.asel}, b{.bsel}, c;

// 32-bit scalar operation, with optional data merge
vop.dtype.atype.u32{.sat}.mode  d.dsel, a{.asel}, b{.bsel}, c;

vop    = { vshl, vshr };
.dtype = .atype = { .u32, .s32 };
.mode  = { .clamp, .wrap };
.dsel  = .asel = .bsel = { .b0, .b1, .b2, .b3, .h0, .h1 };
.op2   = { .add, .min, .max };
```

Description

vshl

Shift *a* left by unsigned amount in *b* with optional saturate, and optional secondary arithmetic operation or subword data merge. Left shift fills with zero.

vshr

Shift *a* right by unsigned amount in *b* with optional saturate, and optional secondary arithmetic operation or subword data merge. Signed shift fills with the sign bit, unsigned shift fills with zero.

Semantics

```
// extract byte/half-word/word and sign- or zero-extend
// based on source operand type
ta = partSelectSignExtend( a, atype, asel );
tb = partSelectSignExtend( b, .u32, bsel );
if ( mode == .clamp && tb > 32 ) tb = 32;
if ( mode == .wrap )           tb = tb & 0x1f;
switch ( vop ){
    case vshl: tmp = ta << tb;
    case vshr: tmp = ta >> tb;
}
// saturate, taking into account destination type and merge operations
tmp = optSaturate( tmp, sat, isSigned(dtype), dsel );
d = optSecondaryOp( op2, tmp, c ); // optional secondary operation
d = optMerge( dsel, tmp, c );      // optional merge with c operand
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

vshl, vshr require sm_20 or higher.

Examples

```
vshl.s32.u32.u32.clamp  r1, r2, r3;
vshr.u32.u32.u32.wrap   r1, r2, r3.h1;
```

9.7.18.1.3 Scalar Video Instructions: vmad

vmad

Integer byte/half-word/word multiply-accumulate.

Syntax

```
// 32-bit scalar operation
vmad.dtype.atype.btype{.sat}{.scale}    d, {-}a{.ase1}, {-}b{.bse1},
                                           {-}c;
vmad.dtype.atype.btype.po{.sat}{.scale}  d, a{.ase1}, b{.bse1}, c;

.dtype = .atype = .btype = { .u32, .s32 };
.ase1 = .bse1 = { .b0, .b1, .b2, .b3, .h0, .h1 };
.scale = { .shr7, .shr15 };
```

Description

Calculate $(a * b) + c$, with optional operand negates, *plus one* mode, and scaling.

The source operands support optional negation with some restrictions. Although PTX syntax allows separate negation of the a and b operands, internally this is represented as negation of the product $(a * b)$. That is, $(a * b)$ is negated if and only if exactly one of a or b is negated. PTX allows negation of either $(a * b)$ or c.

The plus one mode (.po) computes $(a * b) + c + 1$, which is used in computing averages. Source operands may not be negated in .po mode.

The intermediate result of $(a * b)$ is unsigned if atype and btype are unsigned and the product $(a * b)$ is not negated; otherwise, the intermediate result is signed. Input c has the same sign as the intermediate result.

The final result is unsigned if the intermediate result is unsigned and c is not negated.

Depending on the sign of the a and b operands, and the operand negates, the following combinations of operands are supported for VMAD:

```
(u32 * u32) + u32    // intermediate unsigned; final unsigned
-(u32 * u32) + s32    // intermediate  signed; final  signed
(u32 * u32) - u32    // intermediate unsigned; final  signed
(u32 * s32) + s32    // intermediate  signed; final  signed
```

(continues on next page)

(continued from previous page)

```

-(u32 * s32) + s32 // intermediate signed; final signed
(u32 * s32) - s32 // intermediate signed; final signed
(s32 * u32) + s32 // intermediate signed; final signed
-(s32 * u32) + s32 // intermediate signed; final signed
(s32 * u32) - s32 // intermediate signed; final signed
(s32 * s32) + s32 // intermediate signed; final signed
-(s32 * s32) + s32 // intermediate signed; final signed
(s32 * s32) - s32 // intermediate signed; final signed

```

The intermediate result is optionally scaled via right-shift; this result is sign-extended if the final result is signed, and zero-extended otherwise.

The final result is optionally saturated to the appropriate 32-bit range based on the type (signed or unsigned) of the final result.

Semantics

```

// extract byte/half-word/word and sign- or zero-extend
// based on source operand type
ta = partSelectSignExtend( a, atype, asel );
tb = partSelectSignExtend( b, btype, bsel );
signedFinal = isSigned(atype) || isSigned(btype) ||
               (a.negate ^ b.negate) || c.negate;

tmp[127:0] = ta * tb;

lsb = 0;
if ( .po ) { lsb = 1; } else
if ( a.negate ^ b.negate ) { tmp = ~tmp; lsb = 1; } else
if ( c.negate ) { c = ~c; lsb = 1; }

c128[127:0] = (signedFinal) sext32( c ) : zext ( c );
tmp = tmp + c128 + lsb;
switch( scale ) {
    case .shr7:  result = (tmp >> 7) & 0xffffffffffffffff;
    case .shr15: result = (tmp >> 15) & 0xffffffffffffffff;
}
if ( .sat ) {
    if (signedFinal) result = CLAMP(result, S32_MAX, S32_MIN);
    else             result = CLAMP(result, U32_MAX, U32_MIN);
}

```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

vmad requires sm_20 or higher.

Examples

```
vmad.s32.s32.u32.sat    r0, r1, r2, -r3;
vmad.u32.u32.u32.shr15  r0, r1.h0, r2.h0, r3;
```

9.7.18.1.4 Scalar Video Instructions: vset

vset

Integer byte/half-word/word comparison.

Syntax

```
// 32-bit scalar operation, with optional secondary operation
vset.atype.btype.cmp    d, a{.asel}, b{.bsel};
vset.atype.btype.cmp.op2 d, a{.asel}, b{.bsel}, c;

// 32-bit scalar operation, with optional data merge
vset.atype.btype.cmp d.dsel, a{.asel}, b{.bsel}, c;

.atype = .btype = { .u32, .s32 };
.cmp    = { .eq, .ne, .lt, .le, .gt, .ge };
.dsel   = .asel = .bsel = { .b0, .b1, .b2, .b3, .h0, .h1 };
.op2    = { .add, .min, .max };
```

Description

Compare input values using specified comparison, with optional secondary arithmetic operation or subword data merge.

The intermediate result of the comparison is always unsigned, and therefore destination *d* and operand *c* are also unsigned.

Semantics

```
// extract byte/half-word/word and sign- or zero-extend
// based on source operand type
ta = partSelectSignExtend( a, atype, asel );
tb = partSelectSignExtend( b, btype, bsel );
tmp = compare( ta, tb, cmp ) ? 1 : 0;
d = optSecondaryOp( op2, tmp, c );    // optional secondary operation
d = optMerge( dsel, tmp, c );        // optional merge with c operand
```

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

vset requires sm_20 or higher.

Examples

```
vset.s32.u32.lt    r1, r2, r3;
vset.u32.u32.ne    r1, r2, r3.h1;
```

9.7.18.2 SIMD Video Instructions

The SIMD video instructions operate on pairs of 16-bit values and quads of 8-bit values.

The SIMD video instructions are:

- ▶ vadd2, vadd4
- ▶ vsub2, vsub4
- ▶ vavrg2, vavrg4
- ▶ vabsdiff2, vabsdiff4
- ▶ vmin2, vmin4
- ▶ vmax2, vmax4
- ▶ vset2, vset4

PTX includes SIMD video instructions for operation on pairs of 16-bit values and quads of 8-bit values. The SIMD video instructions execute the following stages:

1. Form input vectors by extracting and sign- or zero-extending byte or half-word values from the source operands, to form pairs of signed 17-bit values.
2. Perform a SIMD arithmetic operation on the input pairs.
3. Optionally clamp the result to the appropriate signed or unsigned range, as determined by the destination type.
4. Optionally perform one of the following:

- a. perform a second SIMD merge operation, or
- b. apply a scalar accumulate operation to reduce the intermediate SIMD results to a single scalar.

The general format of dual half-word SIMD video instructions is as follows:

```
// 2-way SIMD operation, with second SIMD merge or accumulate
vop2.dtype.atype.btype{.sat}{.add} d{.mask}, a{.asel}, b{.bsel}, c;

.dtype = .atype = .btype = { .u32, .s32 };
.mask = { .h0, .h1, .h10 };
.asel = .bsel = { .hxy, where x,y are from { 0, 1, 2, 3 } };
```

The general format of quad byte SIMD video instructions is as follows:

```
// 4-way SIMD operation, with second SIMD merge or accumulate
vop4.dtype.atype.btype{.sat}{.add} d{.mask}, a{.asel}, b{.bsel}, c;

.dtype = .atype = .btype = { .u32, .s32 };
.mask = { .b0,
          .b1, .b10
          .b2, .b20, .b21, .b210,
          .b3, .b30, .b31, .b310, .b32, .b320, .b321, .b3210 };
.asel = .bsel = .bxyzw, where x,y,z,w are from { 0, ..., 7 };
```

The source and destination operands are all 32-bit registers. The type of each operand (.u32 or .s32) is specified in the instruction type; all combinations of dtype, atype, and btype are valid. Using the atype/btype and asel/bsel specifiers, the input values are extracted and sign- or zero-extended internally to .s33 values. The primary operation is then performed to produce an .s34 intermediate result. The sign of the intermediate result depends on dtype.

The intermediate result is optionally clamped to the range of the destination type (signed or unsigned), taking into account the subword destination size in the case of optional data merging.

9.7.18.2.1 SIMD Video Instructions: vadd2, vsub2, vavg2, vabsdiff2, vmin2, vmax2

vadd2, vsub2

Integer dual half-word SIMD addition/subtraction.

vavg2

Integer dual half-word SIMD average.

vabsdiff2

Integer dual half-word SIMD absolute value of difference.

vmin2, vmax2

Integer dual half-word SIMD minimum/maximum.

Syntax

```
// SIMD instruction with secondary SIMD merge operation
vop2.dtype.atype.btype{.sat} d{.mask}, a{.asel}, b{.bsel}, c;

// SIMD instruction with secondary accumulate operation
vop2.dtype.atype.btype.add d{.mask}, a{.asel}, b{.bsel}, c;

vop2 = { vadd2, vsub2, vavrg2, vabsdiff2, vmin2, vmax2 };
.dtype = .atype = .btype = { .u32, .s32 };
.mask = { .h0, .h1, .h10 }; // defaults to .h10
.asel = .bsel = { .hxy, where x,y are from { 0, 1, 2, 3 } };
    .asel defaults to .h10
    .bsel defaults to .h32
```

Description

Two-way SIMD parallel arithmetic operation with secondary operation.

Elements of each dual half-word source to the operation are selected from any of the four half-words in the two source operands a and b using the `asel` and `bsel` modifiers.

The selected half-words are then operated on in parallel.

The results are optionally clamped to the appropriate range determined by the destination type (signed or unsigned). Saturation cannot be used with the secondary accumulate operation.

For instructions with a secondary SIMD merge operation:

- For half-word positions indicated in mask, the selected half-word results are copied into destination d. For all other positions, the corresponding half-word from source operand c is copied to d.

For instructions with a secondary accumulate operation:

- For half-word positions indicated in mask, the selected half-word results are added to operand c, producing a result in d.

Semantics

```
// extract pairs of half-words and sign- or zero-extend
// based on operand type
Va = extractAndSignExt_2( a, b, .asel, .atype );
Vb = extractAndSignExt_2( a, b, .bsel, .btype );
Vc = extractAndSignExt_2( c );

for (i=0; i<2; i++) {
    switch ( vop2 ) {
        case vadd2:      t[i] = Va[i] + Vb[i];
        case vsub2:      t[i] = Va[i] - Vb[i];
        case vavrg2:     if ( ( Va[i] + Vb[i] ) >= 0 ) {
                        t[i] = ( Va[i] + Vb[i] + 1 ) >> 1;
                        } else {
                        t[i] = ( Va[i] + Vb[i] ) >> 1;
                        }
        case vabsdiff2:  t[i] = | Va[i] - Vb[i] |;
        case vmin2:     t[i] = MIN( Va[i], Vb[i] );
        case vmax2:     t[i] = MAX( Va[i], Vb[i] );
    }
    if ( .sat ) {
        if ( .dtype == .s32 ) t[i] = CLAMP( t[i], S16_MAX, S16_MIN );
        else t[i] = CLAMP( t[i], U16_MAX, U16_MIN );
    }
}
// secondary accumulate or SIMD merge
mask = extractMaskBits( .mask );
if ( .add ) {
    d = c;
    for (i=0; i<2; i++) { d += mask[i] ? t[i] : 0; }
} else {
    d = 0;
    for (i=0; i<2; i++) { d |= mask[i] ? t[i] : Vc[i]; }
}
```

PTX ISA Notes

Introduced in PTX ISA version 3.0.

Target ISA Notes

vadd2, vsub2, varvg2, vabsdiff2, vmin2, vmax2 require sm_30 or higher.

Examples

```
vadd2.s32.s32.u32.sat  r1, r2, r3, r1;
vsub2.s32.s32.s32.sat  r1.h0, r2.h10, r3.h32, r1;
vmin2.s32.u32.u32.add  r1.h10, r2.h00, r3.h22, r1;
```

9.7.18.2.2 SIMD Video Instructions: vset2

vset2

Integer dual half-word SIMD comparison.

Syntax

```
// SIMD instruction with secondary SIMD merge operation
vset2.atype.btype.cmp  d{.mask}, a{.asel}, b{.bsel}, c;

// SIMD instruction with secondary accumulate operation
vset2.atype.btype.cmp.add  d{.mask}, a{.asel}, b{.bsel}, c;

.atype = .btype = { .u32, .s32 };
.cmp    = { .eq, .ne, .lt, .le, .gt, .ge };
.mask    = { .h0, .h1, .h10 }; // defaults to .h10
.asel    = .bsel = { .hxy, where x,y are from { 0, 1, 2, 3 } };
    .asel defaults to .h10
    .bsel defaults to .h32
```

Description

Two-way SIMD parallel comparison with secondary operation.

Elements of each dual half-word source to the operation are selected from any of the four half-words in the two source operands a and b using the `asel` and `bsel` modifiers.

The selected half-words are then compared in parallel.

The intermediate result of the comparison is always unsigned, and therefore the half-words of destination d and operand c are also unsigned.

For instructions with a secondary SIMD merge operation:

- For half-word positions indicated in mask, the selected half-word results are copied into destination d. For all other positions, the corresponding half-word from source operand b is copied to d.

For instructions with a secondary accumulate operation:

- For half-word positions indicated in mask, the selected half-word results are added to operand c, producing a result in d.

Semantics

```
// extract pairs of half-words and sign- or zero-extend
// based on operand type
Va = extractAndSignExt_2( a, b, .asel, .atype );
Vb = extractAndSignExt_2( a, b, .bsel, .btype );
Vc = extractAndSignExt_2( c );
for (i=0; i<2; i++) {
    t[i] = compare( Va[i], Vb[i], .cmp ) ? 1 : 0;
}
// secondary accumulate or SIMD merge
mask = extractMaskBits( .mask );
if (.add) {
    d = c;
    for (i=0; i<2; i++) { d += mask[i] ? t[i] : 0; }
} else {
    d = 0;
    for (i=0; i<2; i++) { d |= mask[i] ? t[i] : Vc[i]; }
}
```

PTX ISA Notes

Introduced in PTX ISA version 3.0.

Target ISA Notes

vset2 requires sm_30 or higher.

Examples

```
vset2.s32.u32.lt      r1, r2, r3, r0;
vset2.u32.u32.ne.add  r1, r2, r3, r0;
```

9.7.18.2.3 SIMD Video Instructions: vadd4, vsub4, vavg4, vabsdiff4, vmin4, vmax4

vadd4, vsub4

Integer quad byte SIMD addition/subtraction.

vavg4

Integer quad byte SIMD average.

vabsdiff4

Integer quad byte SIMD absolute value of difference.

vmin4, vmax4

Integer quad byte SIMD minimum/maximum.

Syntax

```
// SIMD instruction with secondary SIMD merge operation
vop4.dtype.atype.btype{.sat} d{.mask}, a{.asel}, b{.bsel}, c;

// SIMD instruction with secondary accumulate operation
vop4.dtype.atype.btype.add d{.mask}, a{.asel}, b{.bsel}, c;
vop4 = { vadd4, vsub4, vavrg4, vabsdiff4, vmin4, vmax4 };

.dtype = .atype = .btype = { .u32, .s32 };
.mask = { .b0,
          .b1, .b10
          .b2, .b20, .b21, .b210,
          .b3, .b30, .b31, .b310, .b32, .b320, .b321, .b3210 };
        defaults to .b3210
.asel = .bsel = .bxyzw, where x,y,z,w are from { 0, ..., 7 };
.asel defaults to .b3210
.bsel defaults to .b7654
```

Description

Four-way SIMD parallel arithmetic operation with secondary operation.

Elements of each quad byte source to the operation are selected from any of the eight bytes in the two source operands a and b using the asel and bsel modifiers.

The selected bytes are then operated on in parallel.

The results are optionally clamped to the appropriate range determined by the destination type (signed or unsigned). Saturation cannot be used with the secondary accumulate operation.

For instructions with a secondary SIMD merge operation:

- For byte positions indicated in mask, the selected byte results are copied into destination d. For all other positions, the corresponding byte from source operand c is copied to d.

For instructions with a secondary accumulate operation:

- For byte positions indicated in mask, the selected byte results are added to operand c, producing a result in d.

Semantics

```
// extract quads of bytes and sign- or zero-extend
// based on operand type
Va = extractAndSignExt_4( a, b, .asel, .atype );
Vb = extractAndSignExt_4( a, b, .bsel, .btype );
Vc = extractAndSignExt_4( c );
for (i=0; i<4; i++) {
    switch ( vop4 ) {
        case vadd4:      t[i] = Va[i] + Vb[i];
        case vsub4:      t[i] = Va[i] - Vb[i];
        case vavrg4:     if ( ( Va[i] + Vb[i] ) >= 0 ) {
                        t[i] = ( Va[i] + Vb[i] + 1 ) >> 1;
                        } else {
                        t[i] = ( Va[i] + Vb[i] ) >> 1;
                        }
        case vabsdiff4:  t[i] = | Va[i] - Vb[i] |;
        case vmin4:     t[i] = MIN( Va[i], Vb[i] );
        case vmax4:     t[i] = MAX( Va[i], Vb[i] );
    }
    if ( .sat ) {
        if ( .dtype == .s32 ) t[i] = CLAMP( t[i], S8_MAX, S8_MIN );
        else                  t[i] = CLAMP( t[i], U8_MAX, U8_MIN );
    }
}
// secondary accumulate or SIMD merge
mask = extractMaskBits( .mask );
if ( .add ) {
    d = c;
    for (i=0; i<4; i++) { d += mask[i] ? t[i] : 0; }
} else {
    d = 0;
    for (i=0; i<4; i++) { d |= mask[i] ? t[i] : Vc[i]; }
}
```

PTX ISA Notes

Introduced in PTX ISA version 3.0.

Target ISA Notes

vadd4, vsub4, varvg4, vabsdiff4, vmin4, vmax4 require sm_30 or higher.

Examples

```
vadd4.s32.s32.u32.sat  r1, r2, r3, r1;
vsub4.s32.s32.s32.sat  r1.b0, r2.b3210, r3.b7654, r1;
vmin4.s32.u32.u32.add  r1.b00, r2.b0000, r3.b2222, r1;
```


9.7.18.2.4 SIMD Video Instructions: vset4

vset4

Integer quad byte SIMD comparison.

Syntax

```
// SIMD instruction with secondary SIMD merge operation
vset4.atype.btype.cmp d{.mask}, a{.asel}, b{.bsel}, c;

// SIMD instruction with secondary accumulate operation
vset4.atype.btype.cmp.add d{.mask}, a{.asel}, b{.bsel}, c;

.atype = .btype = { .u32, .s32 };
.cmp = { .eq, .ne, .lt, .le, .gt, .ge };
.mask = { .b0,
          .b1, .b10
          .b2, .b20, .b21, .b210,
          .b3, .b30, .b31, .b310, .b32, .b320, .b321, .b3210 };
        defaults to .b3210
.asel = .bsel = .bxyzw, where x,y,z,w are from { 0, ..., 7 };
.asel defaults to .b3210
.bsel defaults to .b7654
```

Description

Four-way SIMD parallel comparison with secondary operation.

Elements of each quad byte source to the operation are selected from any of the eight bytes in the two source operands a and b using the `asel` and `bsel` modifiers.

The selected bytes are then compared in parallel.

The intermediate result of the comparison is always unsigned, and therefore the bytes of destination d and operand c are also unsigned.

For instructions with a secondary SIMD merge operation:

- For byte positions indicated in mask, the selected byte results are copied into destination d. For all other positions, the corresponding byte from source operand b is copied to d.

For instructions with a secondary accumulate operation:

- For byte positions indicated in mask, the selected byte results are added to operand c, producing a result in d.

Semantics

```
// extract quads of bytes and sign- or zero-extend
// based on operand type
Va = extractAndSignExt_4( a, b, .asel, .atype );
Vb = extractAndSignExt_4( a, b, .bsel, .btype );
Vc = extractAndSignExt_4( c );
for (i=0; i<4; i++) {
    t[i] = compare( Va[i], Vb[i], cmp ) ? 1 : 0;
}
// secondary accumulate or SIMD merge
mask = extractMaskBits( .mask );
if (.add) {
    d = c;
    for (i=0; i<4; i++) { d += mask[i] ? t[i] : 0; }
} else {
    d = 0;
    for (i=0; i<4; i++) { d |= mask[i] ? t[i] : Vc[i]; }
}
```

PTX ISA Notes

Introduced in PTX ISA version 3.0.

Target ISA Notes

vset4 requires sm_30 or higher.

Examples

```
vset4.s32.u32.lt      r1, r2, r3, r0;
vset4.u32.u32.ne.max  r1, r2, r3, r0;
```

9.7.19. Miscellaneous Instructions

The Miscellaneous instructions are:

- ▶ brkpt
- ▶ nanosleep
- ▶ pmevent
- ▶ trap
- ▶ setmaxnreg

9.7.19.1 Miscellaneous Instructions: brkpt

brkpt

Breakpoint.

Syntax

```
brkpt;
```

Description

Suspends execution.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

brkpt requires sm_11 or higher.

Examples

```
    brkpt;  
@p  brkpt;
```

9.7.19.2 Miscellaneous Instructions: nanosleep

nanosleep

Suspend the thread for an approximate delay given in nanoseconds.

Syntax

```
nanosleep.u32 t;
```

Description

Suspends the thread for a sleep duration approximately close to the delay t , specified in nanoseconds. t may be a register or an immediate value.

The sleep duration is approximated, but guaranteed to be in the interval $[0, 2 \cdot t]$. The maximum sleep duration is 1 millisecond. The implementation may reduce the sleep duration for individual threads within a warp such that all sleeping threads in the warp wake up together.

PTX ISA Notes

`nanosleep` introduced in PTX ISA 6.3.

Target ISA Notes

`nanosleep` requires `sm_70` or higher.

Examples

```
.reg .b32 r;
.reg .pred p;

nanosleep.u32 r;
nanosleep.u32 42;
@p nanosleep.u32 r;
```

9.7.19.3 Miscellaneous Instructions: `pmevent`

`pmevent`

Trigger one or more Performance Monitor events.

Syntax

```
pmevent      a;    // trigger a single performance monitor event
pmevent.mask a;    // trigger one or more performance monitor events
```

Description

Triggers one or more of a fixed number of performance monitor events, with event index or mask specified by immediate operand `a`.

`pmevent` (without modifier `.mask`) triggers a single performance monitor event indexed by immediate operand `a`, in the range `0..15`.

`pmevent.mask` triggers one or more of the performance monitor events. Each bit in the 16-bit immediate operand `a` controls an event.

Programmatic performance monitor events may be combined with other hardware events using Boolean functions to increment one of the four performance counters. The relationship between events and counters is programmed via API calls from the host.

Notes

Currently, there are sixteen performance monitor events, numbered 0 through 15.

PTX ISA Notes

`pmevent` introduced in PTX ISA version 1.4.

`pmevent.mask` introduced in PTX ISA version 3.0.

Target ISA Notes

`pmevent` supported on all target architectures.

`pmevent.mask` requires `sm_20` or higher.

Examples

```
pmevent      1;
@p pmevent    7;
@q pmevent.mask 0xff;
```

9.7.19.4 Miscellaneous Instructions: `trap`

`trap`

Perform trap operation.

Syntax

```
trap;
```

Description

Abort execution and generate an interrupt to the host CPU.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
trap;  
@p trap;
```

9.7.19.5 Miscellaneous Instructions: `setmaxnreg`

`setmaxnreg`

Hint to change the number of registers owned by the warp.

Syntax

```
setmaxnreg.action.sync.aligned.u32 imm-reg-count;  
  
.action = { .inc, .dec };
```

Description

`setmaxnreg` provides a hint to the system to update the maximum number of per-thread registers owned by the executing warp to the value specified by the `imm-reg-count` operand.

Qualifier `.dec` is used to release extra registers such that the absolute per-thread maximum register count is reduced from its current value to `imm-reg-count`. Qualifier `.inc` is used to request additional registers such that the absolute per-thread maximum register count is increased from its current value to `imm-reg-count`.

A pool of available registers is maintained per-CTA. Register adjustments requested by the `setmaxnreg` instructions are handled by supplying extra registers from this pool to the requesting warp or by releasing extra registers from the requesting warp to this pool, depending upon the value of the `.action` qualifier.

The `setmaxnreg.inc` instruction blocks the execution until enough registers are available in the CTA's register pool. After the instruction `setmaxnreg.inc` obtains new registers from the CTA pool, the initial contents of the new registers are undefined. The new registers must be initialized before they are used.

The same `setmaxnreg` instruction must be executed by all warps in a *warpgroup*. After executing a `setmaxnreg` instruction, all warps in the *warpgroup* must synchronize explicitly before executing subsequent `setmaxnreg` instructions. If a `setmaxnreg` instruction is not executed by all warps in the *warpgroup*, then the behavior is undefined.

Operand `imm-reg-count` is an integer constant. The value of `imm-reg-count` must be in the range 24 to 256 (both inclusive) and must be a multiple of 8.

Changes to the register file of the warp always happen at the tail-end of the register file.

The `setmaxnreg` instruction requires that the kernel has been launched with a valid value of maximum number of per-thread registers specified via the appropriate compilation via the appropriate compile-time option or the appropriate performance tuning directive. Otherwise, the `setmaxnreg` instruction may have no effect.

When qualifier `.dec` is specified, the maximum number of per-thread registers owned by the warp prior to the execution of `setmaxnreg` instruction should be greater than or equal to the `imm-reg-count`. Otherwise, the behaviour is undefined.

When qualifier `.inc` is specified, the maximum number of per-thread registers owned by the warp prior to the execution of `setmaxnreg` instruction should be less than or equal to the `imm-reg-count`. Otherwise, the behaviour is undefined.

The mandatory `.sync` qualifier indicates that `setmaxnreg` instruction causes the executing thread to wait until all threads in the warp execute the same `setmaxnreg` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warpgroup must execute the same `setmaxnreg` instruction. In conditionally executed code, `setmaxnreg` instruction should only be used if it is known that all threads in warpgroup evaluate the condition identically, otherwise the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_90a`
- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_120a`
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_120f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Examples

```
setmaxnreg.dec.sync.aligned.u32 64;  
setmaxnreg.inc.sync.aligned.u32 192;
```

Chapter 10. Special Registers

PTX includes a number of predefined, read-only variables, which are visible as special registers and accessed through `mov` or `cvt` instructions.

The special registers are:

- ▶ `%tid`
- ▶ `%ntid`
- ▶ `%laneid`
- ▶ `%warpid`
- ▶ `%nwarpid`
- ▶ `%ctaid`
- ▶ `%nctaid`
- ▶ `%smid`
- ▶ `%nsmid`
- ▶ `%gridid`
- ▶ `%is_explicit_cluster`
- ▶ `%clusterid`
- ▶ `%nclusterid`
- ▶ `%cluster_ctaid`
- ▶ `%cluster_nctaid`
- ▶ `%cluster_ctarank`
- ▶ `%cluster_nctarank`
- ▶ `%lanemask_eq, %lanemask_le, %lanemask_lt, %lanemask_ge, %lanemask_gt`
- ▶ `%clock, %clock_hi, %clock64`
- ▶ `%pm0, ..., %pm7`
- ▶ `%pm0_64, ..., %pm7_64`
- ▶ `%envreg0, ..., %envreg31`
- ▶ `%globaltimer, %globaltimer_lo, %globaltimer_hi`
- ▶ `%reserved_smem_offset_begin, %reserved_smem_offset_end, %reserved_smem_offset_cap, %reserved_smem_offset<2>` %re-

- ▶ %total_smem_size
- ▶ %aggr_smem_size
- ▶ %dynamic_smem_size
- ▶ %current_graph_exec

10.1. Special Registers: %tid

%tid

Thread identifier within a CTA.

Syntax (predefined)

```
.sreg .v4 .u32 %tid;           // thread id vector
.sreg .u32 %tid.x, %tid.y, %tid.z; // thread id components
```

Description

A predefined, read-only, per-thread special register initialized with the thread identifier within the CTA. The %tid special register contains a 1D, 2D, or 3D vector to match the CTA shape; the %tid value in unused dimensions is 0. The fourth element is unused and always returns zero. The number of threads in each dimension are specified by the predefined special register %ntid.

Every thread in the CTA has a unique %tid.

%tid component values range from 0 through %ntid-1 in each CTA dimension.

%tid.y == %tid.z == 0 in 1D CTAs. %tid.z == 0 in 2D CTAs.

It is guaranteed that:

```
0 <= %tid.x < %ntid.x
0 <= %tid.y < %ntid.y
0 <= %tid.z < %ntid.z
```

PTX ISA Notes

Introduced in PTX ISA version 1.0 with type .v4.u16.

Redefined as type .v4.u32 in PTX ISA version 2.0. For compatibility with legacy PTX code, 16-bit mov and cvt instructions may be used to read the lower 16-bits of each component of %tid.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.u32    %r1,%tid.x;  // move tid.x to %rh

// legacy code accessing 16-bit components of %tid
mov.u16    %rh,%tid.x;
cvt.u32.u16 %r2,%tid.z; // zero-extend tid.z to %r2
```

10.2. Special Registers: %ntid

%ntid

Number of thread IDs per CTA.

Syntax (predefined)

```
.sreg .v4 .u32 %ntid;           // CTA shape vector
.sreg .u32 %ntid.x, %ntid.y, %ntid.z; // CTA dimensions
```

Description

A predefined, read-only special register initialized with the number of thread ids in each CTA dimension. The %ntid special register contains a 3D CTA shape vector that holds the CTA dimensions. CTA dimensions are non-zero; the fourth element is unused and always returns zero. The total number of threads in a CTA is ($\%ntid.x * \%ntid.y * \%ntid.z$).

```
%ntid.y == %ntid.z == 1 in 1D CTAs.
%ntid.z ==1 in 2D CTAs.
```

Maximum values of %ntid.{x,y,z} are as follows:

.target architecture	%ntid.x	%ntid.y	%ntid.z
sm_1x	512	512	64
sm_20, sm_3x, sm_5x, sm_6x, sm_7x, sm_8x, sm_9x, sm_10x, sm_12x	1024	1024	64

PTX ISA Notes

Introduced in PTX ISA version 1.0 with type `.v4.u16`.

Redefined as type `.v4.u32` in PTX ISA version 2.0. For compatibility with legacy PTX code, 16-bit `mov` and `cvt` instructions may be used to read the lower 16-bits of each component of `%ntid`.

Target ISA Notes

Supported on all target architectures.

Examples

```
// compute unified thread id for 2D CTA
mov.u32  %r0,%tid.x;
mov.u32  %h1,%tid.y;
mov.u32  %h2,%ntid.x;
mad.u32  %r0,%h1,%h2,%r0;

mov.u16  %rh,%ntid.x;      // legacy code
```

10.3. Special Registers: `%laneid`

`%laneid`

Lane Identifier.

Syntax (predefined)

```
.sreg .u32 %laneid;
```

Description

A predefined, read-only special register that returns the thread's lane within the warp. The lane identifier ranges from zero to `WARP_SZ-1`.

PTX ISA Notes

Introduced in PTX ISA version 1.3.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.u32 %r, %laneid;
```

10.4. Special Registers: %warpid

%warpid

Warp identifier.

Syntax (predefined)

```
.sreg .u32 %warpid;
```

Description

A predefined, read-only special register that returns the thread's warp identifier. The warp identifier provides a unique warp number within a CTA but not across CTAs within a grid. The warp identifier will be the same for all threads within a single warp.

Note that %warpid returns the location of a thread at the moment when read, but its value may change during execution, e.g., due to rescheduling of threads following preemption. For this reason, %ctaid and %tid should be used to compute a virtual warp index if such a value is needed in kernel code; %warpid is intended mainly to enable profiling and diagnostic code to sample and log information such as work place mapping and load distribution.

PTX ISA Notes

Introduced in PTX ISA version 1.3.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.u32  %r, %warpid;
```

10.5. Special Registers: %nwarpid

%nwarpid

Number of warp identifiers.

Syntax (predefined)

```
.sreg .u32 %nwarpid;
```

Description

A predefined, read-only special register that returns the maximum number of warp identifiers.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%nwarpid requires sm_20 or higher.

Examples

```
mov.u32  %r, %nwarpid;
```

10.6. Special Registers: %ctaid

%ctaid

CTA identifier within a grid.

Syntax (predefined)

```
.sreg .v4 .u32 %ctaid;           // CTA id vector
.sreg .u32 %ctaid.x, %ctaid.y, %ctaid.z; // CTA id components
```

Description

A predefined, read-only special register initialized with the CTA identifier within the CTA grid. The `%ctaid` special register contains a 1D, 2D, or 3D vector, depending on the shape and rank of the CTA grid. The fourth element is unused and always returns zero.

It is guaranteed that:

```
0 <= %ctaid.x < %nctaid.x
0 <= %ctaid.y < %nctaid.y
0 <= %ctaid.z < %nctaid.z
```

PTX ISA Notes

Introduced in PTX ISA version 1.0 with type `.v4.u16`.

Redefined as type `.v4.u32` in PTX ISA version 2.0. For compatibility with legacy PTX code, 16-bit `mov` and `cvt` instructions may be used to read the lower 16-bits of each component of `%ctaid`.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.u32 %r0,%ctaid.x;
mov.u16 %rh,%ctaid.y; // legacy code
```

10.7. Special Registers: `%nctaid`

`%nctaid`

Number of CTA ids per grid.

Syntax (predefined)

```
.sreg .v4 .u32 %nctaid           // Grid shape vector
.sreg .u32 %nctaid.x,%nctaid.y,%nctaid.z; // Grid dimensions
```

Description

A predefined, read-only special register initialized with the number of CTAs in each grid dimension. The %nctaid special register contains a 3D grid shape vector, with each element having a value of at least 1. The fourth element is unused and always returns zero.

Maximum values of %nctaid.{x,y,z} are as follows:

.target architecture	%nctaid.x	%nctaid.y	%nctaid.z
sm_1x, sm_20	65535	65535	65535
sm_3x, sm_5x, sm_6x, sm_7x, sm_8x, sm_9x, sm_10x, sm_12x	$2^{31} - 1$	65535	65535

PTX ISA Notes

Introduced in PTX ISA version 1.0 with type .v4.u16.

Redefined as type .v4.u32 in PTX ISA version 2.0. For compatibility with legacy PTX code, 16-bit mov and cvt instructions may be used to read the lower 16-bits of each component of %nctaid.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.u32 %r0,%nctaid.x;
mov.u16 %rh,%nctaid.x;    // legacy code
```

10.8. Special Registers: %smid

%smid

SM identifier.

Syntax (predefined)

```
.sreg .u32 %smid;
```

Description

A predefined, read-only special register that returns the processor (SM) identifier on which a particular thread is executing. The SM identifier ranges from 0 to %nsmid-1. The SM identifier numbering is not guaranteed to be contiguous.

Notes

Note that %smid returns the location of a thread at the moment when read, but its value may change during execution, e.g. due to rescheduling of threads following preemption. %smid is intended mainly to enable profiling and diagnostic code to sample and log information such as work place mapping and load distribution.

PTX ISA Notes

Introduced in PTX ISA version 1.3.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.u32 %r, %smid;
```

10.9. Special Registers: %nsmid

%nsmid

Number of SM identifiers.

Syntax (predefined)

```
.sreg .u32 %nsmid;
```

Description

A predefined, read-only special register that returns the maximum number of SM identifiers. The SM identifier numbering is not guaranteed to be contiguous, so %nsmid may be larger than the physical number of SMs in the device.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%nsmid requires sm_20 or higher.

Examples

```
mov.u32 %r, %nsmid;
```

10.10. Special Registers: %gridid

%gridid

Grid identifier.

Syntax (predefined)

```
.sreg .u64 %gridid;
```

Description

A predefined, read-only special register initialized with the per-grid temporal grid identifier. The %gridid is used by debuggers to distinguish CTAs and clusters within concurrent (small) grids.

During execution, repeated launches of programs may occur, where each launch starts a grid-of-CTAs. This variable provides the temporal grid launch number for this context.

For sm_1x targets, %gridid is limited to the range $[0..2^{16}-1]$. For sm_20, %gridid is limited to the range $[0..2^{32}-1]$. sm_30 supports the entire 64-bit range.

PTX ISA Notes

Introduced in PTX ISA version 1.0 as type `.u16`.

Redefined as type `.u32` in PTX ISA version 1.3.

Redefined as type `.u64` in PTX ISA version 3.0.

For compatibility with legacy PTX code, 16-bit and 32-bit `mov` and `cvt` instructions may be used to read the lower 16-bits or 32-bits of each component of `%gridid`.

Target ISA Notes

Supported on all target architectures.

Examples

```

mov.u64  %s, %gridid; // 64-bit read of %gridid
mov.u32  %r, %gridid; // legacy code with 32-bit %gridid

```

10.11. Special Registers: `%is_explicit_cluster`

`%is_explicit_cluster`

Checks if user has explicitly specified cluster launch.

Syntax (predefined)

```
.sreg .pred %is_explicit_cluster;
```

Description

A predefined, read-only special register initialized with the predicate value of whether the cluster launch is explicitly specified by user.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .pred p;  
mov.pred p, %is_explicit_cluster;
```

10.12. Special Registers: %clusterid

%clusterid

Cluster identifier within a grid.

Syntax (predefined)

```
.sreg .v4 .u32 %clusterid;  
.sreg .u32 %clusterid.x, %clusterid.y, %clusterid.z;
```

Description

A predefined, read-only special register initialized with the cluster identifier in a grid in each dimension. Each cluster in a grid has a unique identifier.

The %clusterid special register contains a 1D, 2D, or 3D vector, depending upon the shape and rank of the cluster. The fourth element is unused and always returns zero.

It is guaranteed that:

```
0 <= %clusterid.x < %nclusterid.x  
0 <= %clusterid.y < %nclusterid.y  
0 <= %clusterid.z < %nclusterid.z
```

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .b32 %r<2>;
.reg .v4 .b32 %rx;

mov.u32    %r0, %clusterid.x;
mov.u32    %r1, %clusterid.z;
mov.v4.u32 %rx, %clusterid;
```

10.13. Special Registers: %nclusterid

%nclusterid

Number of cluster identifiers per grid.

Syntax (predefined)

```
.sreg .v4 .u32 %nclusterid;
.sreg .u32 %nclusterid.x, %nclusterid.y, %nclusterid.z;
```

Description

A predefined, read-only special register initialized with the number of clusters in each grid dimension.

The %nclusterid special register contains a 3D grid shape vector that holds the grid dimensions in terms of clusters. The fourth element is unused and always returns zero.

Refer to the *Cuda Programming Guide* for details on the maximum values of %nclusterid.{x,y,z}.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .b32 %r<2>;  
.reg .v4 .b32 %rx;  
  
mov.u32      %r0, %nclusterid.x;  
mov.u32      %r1, %nclusterid.z;  
mov.v4.u32   %rx, %nclusterid;
```

10.14. Special Registers: %cluster_ctaid

%cluster_ctaid

CTA identifier within a cluster.

Syntax (predefined)

```
.sreg .v4 .u32 %cluster_ctaid;  
.sreg .u32 %cluster_ctaid.x, %cluster_ctaid.y, %cluster_ctaid.z;
```

Description

A predefined, read-only special register initialized with the CTA identifier in a cluster in each dimension. Each CTA in a cluster has a unique CTA identifier.

The %cluster_ctaid special register contains a 1D, 2D, or 3D vector, depending upon the shape of the cluster. The fourth element is unused and always returns zero.

It is guaranteed that:

```
0 <= %cluster_ctaid.x < %cluster_nctaid.x  
0 <= %cluster_ctaid.y < %cluster_nctaid.y  
0 <= %cluster_ctaid.z < %cluster_nctaid.z
```

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .b32 %r<2>;
.reg .v4 .b32 %rx;

mov.u32      %r0, %cluster_ctaid.x;
mov.u32      %r1, %cluster_ctaid.z;
mov.v4.u32   %rx, %cluster_ctaid;
```

10.15. Special Registers: %cluster_nctaid

%cluster_nctaid

Number of CTA identifiers per cluster.

Syntax (predefined)

```
.sreg .v4 .u32 %cluster_nctaid;
.sreg .u32 %cluster_nctaid.x, %cluster_nctaid.y, %cluster_nctaid.z;
```

Description

A predefined, read-only special register initialized with the number of CTAs in a cluster in each dimension.

The %cluster_nctaid special register contains a 3D grid shape vector that holds the cluster dimensions in terms of CTAs. The fourth element is unused and always returns zero.

Refer to the *Cuda Programming Guide* for details on the maximum values of %cluster_nctaid.{x, y, z}.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .b32 %r<2>;  
.reg .v4 .b32 %rx;  
  
mov.u32    %r0, %cluster_nctaid.x;  
mov.u32    %r1, %cluster_nctaid.z;  
mov.v4.u32 %rx, %cluster_nctaid;
```

10.16. Special Registers: %cluster_ctarank

%cluster_ctarank

CTA identifier in a cluster across all dimensions.

Syntax (predefined)

```
.sreg .u32 %cluster_ctarank;
```

Description

A predefined, read-only special register initialized with the CTA rank within a cluster across all dimensions.

It is guaranteed that:

```
0  <=  %cluster_ctarank <  %cluster_nctarank
```

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .b32 %r;  
mov.u32 %r, %cluster_ctarank;
```

10.17. Special Registers: %cluster_nctarank

%cluster_nctarank

Number of CTA identifiers in a cluster across all dimensions.

Syntax (predefined)

```
.sreg .u32 %cluster_nctarank;
```

Description

A predefined, read-only special register initialized with the number of CTAs within a cluster across all dimensions.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.reg .b32 %r;  
mov.u32 %r, %cluster_nctarank;
```

10.18. Special Registers: %lanemask_eq

%lanemask_eq

32-bit mask with bit set in position equal to the thread's lane number in the warp.

Syntax (predefined)

```
.sreg .u32 %lanemask_eq;
```

Description

A predefined, read-only special register initialized with a 32-bit mask with a bit set in the position equal to the thread's lane number in the warp.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%lanemask_eq requires sm_20 or higher.

Examples

```
mov.u32    %r, %lanemask_eq;
```

10.19. Special Registers: %lanemask_le

%lanemask_le

32-bit mask with bits set in positions less than or equal to the thread's lane number in the warp.

Syntax (predefined)

```
.sreg .u32 %lanemask_1e;
```

Description

A predefined, read-only special register initialized with a 32-bit mask with bits set in positions less than or equal to the thread's lane number in the warp.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%lanemask_1e requires sm_20 or higher.

Examples

```
mov.u32    %r, %lanemask_1e
```

10.20. Special Registers: %lanemask_1t

%lanemask_1t

32-bit mask with bits set in positions less than the thread's lane number in the warp.

Syntax (predefined)

```
.sreg .u32 %lanemask_1t;
```

Description

A predefined, read-only special register initialized with a 32-bit mask with bits set in positions less than the thread's lane number in the warp.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%lanemask_lt requires sm_20 or higher.

Examples

```
mov.u32    %r, %lanemask_lt;
```

10.21. Special Registers: %lanemask_ge

%lanemask_ge

32-bit mask with bits set in positions greater than or equal to the thread's lane number in the warp.

Syntax (predefined)

```
.sreg .u32 %lanemask_ge;
```

Description

A predefined, read-only special register initialized with a 32-bit mask with bits set in positions greater than or equal to the thread's lane number in the warp.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%lanemask_ge requires sm_20 or higher.

Examples

```
mov.u32    %r, %lanemask_ge;
```

10.22. Special Registers: %lanemask_gt

%lanemask_gt

32-bit mask with bits set in positions greater than the thread's lane number in the warp.

Syntax (predefined)

```
.sreg .u32 %lanemask_gt;
```

Description

A predefined, read-only special register initialized with a 32-bit mask with bits set in positions greater than the thread's lane number in the warp.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

%lanemask_gt requires sm_20 or higher.

Examples

```
mov.u32    %r, %lanemask_gt;
```

10.23. Special Registers: %clock, %clock_hi

%clock, %clock_hi

%clock

A predefined, read-only 32-bit unsigned cycle counter.

%clock_hi

The upper 32-bits of %clock64 special register.

Syntax (predefined)

```
.sreg .u32 %clock;  
.sreg .u32 %clock_hi;
```

Description

Special register `%clock` and `%clock_hi` are unsigned 32-bit read-only cycle counters that wrap silently.

PTX ISA Notes

`%clock` introduced in PTX ISA version 1.0.

`%clock_hi` introduced in PTX ISA version 5.0.

Target ISA Notes

`%clock` supported on all target architectures.

`%clock_hi` requires `sm_20` or higher.

Examples

```
mov.u32 r1,%clock;  
mov.u32 r2, %clock_hi;
```

10.24. Special Registers: `%clock64`

`%clock64`

A predefined, read-only 64-bit unsigned cycle counter.

Syntax (predefined)

```
.sreg .u64 %clock64;
```

Description

Special register `%clock64` is an unsigned 64-bit read-only cycle counter that wraps silently.

Notes

The lower 32-bits of `%clock64` are identical to `%clock`.

The upper 32-bits of `%clock64` are identical to `%clock_hi`.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

`%clock64` requires `sm_20` or higher.

Examples

```
mov.u64 r1,%clock64;
```

10.25. Special Registers: `%pm0 ... %pm7`

`%pm0 ... %pm7`

Performance monitoring counters.

Syntax (predefined)

```
.sreg .u32 %pm<8>;
```

Description

Special registers `%pm0 ... %pm7` are unsigned 32-bit read-only performance monitor counters. Their behavior is currently undefined.

PTX ISA Notes

%pm0 ... %pm3 introduced in PTX ISA version 1.3.

%pm4 ... %pm7 introduced in PTX ISA version 3.0.

Target ISA Notes

%pm0 ... %pm3 supported on all target architectures.

%pm4 ... %pm7 require sm_20 or higher.

Examples

```
mov.u32  r1,%pm0;
mov.u32  r1,%pm7;
```

10.26. Special Registers: %pm0_64 ... %pm7_64

%pm0_64 ... %pm7_64

64 bit Performance monitoring counters.

Syntax (predefined)

```
.sreg .u64 %pm0_64;
.sreg .u64 %pm1_64;
.sreg .u64 %pm2_64;
.sreg .u64 %pm3_64;
.sreg .u64 %pm4_64;
.sreg .u64 %pm5_64;
.sreg .u64 %pm6_64;
.sreg .u64 %pm7_64;
```

Description

Special registers %pm0_64 ... %pm7_64 are unsigned 64-bit read-only performance monitor counters. Their behavior is currently undefined.

Notes

The lower 32bits of %pm0_64 ... %pm7_64 are identical to %pm0 ... %pm7.

PTX ISA Notes

%pm0_64 ... %pm7_64 introduced in PTX ISA version 4.0.

Target ISA Notes

%pm0_64 ... %pm7_64 require sm_50 or higher.

Examples

```

mov.u32  r1,%pm0_64;
mov.u32  r1,%pm7_64;

```

10.27. Special Registers: %envreg<32>

%envreg<32>

Driver-defined read-only registers.

Syntax (predefined)

```
.sreg .b32 %envreg<32>;
```

Description

A set of 32 pre-defined read-only registers used to capture execution environment of PTX program outside of PTX virtual machine. These registers are initialized by the driver prior to kernel launch and can contain cta-wide or grid-wide values.

Precise semantics of these registers is defined in the driver documentation.

PTX ISA Notes

Introduced in PTX ISA version 2.1.

Target ISA Notes

Supported on all target architectures.

Examples

```
mov.b32    %r1,%envreg0;  // move envreg0 to %r1
```

10.28. Special Registers: %globaltimer, %globaltimer_lo, %globaltimer_hi

%globaltimer, %globaltimer_lo, %globaltimer_hi

%globaltimer

A predefined, 64-bit global nanosecond timer.

%globaltimer_lo

The lower 32-bits of %globaltimer.

%globaltimer_hi

The upper 32-bits of %globaltimer.

Syntax (predefined)

```
.sreg .u64 %globaltimer;  
.sreg .u32 %globaltimer_lo, %globaltimer_hi;
```

Description

Special registers intended for use by NVIDIA tools. The behavior is target-specific and may change or be removed in future GPUs. When JIT-compiled to other targets, the value of these registers is unspecified.

PTX ISA Notes

Introduced in PTX ISA version 3.1.

Target ISA Notes

Requires target sm_30 or higher.

Examples

```
mov.u64 r1,%globaltimer;
```

10.29. Special Registers:

`%reserved_smem_offset_begin`,
`%reserved_smem_offset_end`,
`%reserved_smem_offset_cap`,
`%reserved_smem_offset_<2>`

`%reserved_smem_offset_begin`, `%reserved_smem_offset_end`, `%re-`
`served_smem_offset_cap`, `%reserved_smem_offset_<2>`

%reserved_smem_offset_begin

Start of the reserved shared memory region.

%reserved_smem_offset_end

End of the reserved shared memory region.

%reserved_smem_offset_cap

Total size of the reserved shared memory region.

%reserved_smem_offset_<2>

Offsets in the reserved shared memory region.

Syntax (predefined)

```
.sreg .b32 %reserved_smem_offset_begin;  
.sreg .b32 %reserved_smem_offset_end;  
.sreg .b32 %reserved_smem_offset_cap;  
.sreg .b32 %reserved_smem_offset_<2>;
```

Description

These are predefined, read-only special registers containing information about the shared memory region which is reserved for the NVIDIA system software use. This region of shared memory is not available to users, and accessing this region from user code results in undefined behavior. Refer to *CUDA Programming Guide* for details.

PTX ISA Notes

Introduced in PTX ISA version 7.6.

Target ISA Notes

Require sm_80 or higher.

Examples

```
.reg .b32 %reg_begin, %reg_end, %reg_cap, %reg_offset0, %reg_offset1;

mov.b32 %reg_begin,    %reserved_smem_offset_begin;
mov.b32 %reg_end,      %reserved_smem_offset_end;
mov.b32 %reg_cap,      %reserved_smem_offset_cap;
mov.b32 %reg_offset0,  %reserved_smem_offset_0;
mov.b32 %reg_offset1,  %reserved_smem_offset_1;
```

10.30. Special Registers: %total_smem_size

%total_smem_size

Total size of shared memory used by a CTA of a kernel.

Syntax (predefined)

```
.sreg .u32 %total_smem_size;
```

Description

A predefined, read-only special register initialized with total size of shared memory allocated (statically and dynamically, excluding the shared memory reserved for the NVIDIA system software use) for the CTA of a kernel at launch time.

Size is returned in multiples of shared memory allocation unit size supported by target architecture.

Allocation unit values are as follows:

Target architecture	Shared memory allocation unit size
sm_2x	128 bytes
sm_3x, sm_5x, sm_6x, sm_7x	256 bytes
sm_8x, sm_9x, sm_10x, sm_12x	128 bytes

PTX ISA Notes

Introduced in PTX ISA version 4.1.

Target ISA Notes

Requires sm_20 or higher.

Examples

```
mov.u32 %r, %total_smem_size;
```

10.31. Special Registers: %aggr_smem_size

%aggr_smem_size

Total size of shared memory used by a CTA of a kernel.

Syntax (predefined)

```
.sreg .u32 %aggr_smem_size;
```

Description

A predefined, read-only special register initialized with total aggregated size of shared memory consisting of the size of user shared memory allocated (statically and dynamically) at launch time and the size of shared memory region which is reserved for the NVIDIA system software use.

PTX ISA Notes

Introduced in PTX ISA version 8.1.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
mov.u32 %r, %aggr_smem_size;
```

10.32. Special Registers: %dynamic_smem_size

%dynamic_smem_size

Size of shared memory allocated dynamically at kernel launch.

Syntax (predefined)

```
.sreg .u32 %dynamic_smem_size;
```

Description

Size of shared memory allocated dynamically at kernel launch.

A predefined, read-only special register initialized with size of shared memory allocated dynamically for the CTA of a kernel at launch time.

PTX ISA Notes

Introduced in PTX ISA version 4.1.

Target ISA Notes

Requires sm_20 or higher.

Examples

```
mov.u32 %r, %dynamic_smem_size;
```

10.33. Special Registers: %current_graph_exec

%current_graph_exec

An Identifier for currently executing CUDA device graph.

Syntax (predefined)

```
.sreg .u64 %current_graph_exec;
```

Description

A predefined, read-only special register initialized with the identifier referring to the CUDA device graph being currently executed. This register is 0 if the executing kernel is not part of a CUDA device graph.

Refer to the *CUDA Programming Guide* for more details on CUDA device graphs.

PTX ISA Notes

Introduced in PTX ISA version 8.0.

Target ISA Notes

Requires sm_50 or higher.

Examples

```
mov.u64  r1, %current_graph_exec;
```

Chapter 11. Directives

11.1. PTX Module Directives

The following directives declare the PTX ISA version of the code in the module, the target architecture for which the code was generated, and the size of addresses within the PTX module.

- ▶ `.version`
- ▶ `.target`
- ▶ `.address_size`

11.1.1. PTX Module Directives: `.version`

`.version`

PTX ISA version number.

Syntax

```
.version major.minor    // major, minor are integers
```

Description

Specifies the PTX language version number.

The *major* number is incremented when there are incompatible changes to the PTX language, such as changes to the syntax or semantics. The version major number is used by the PTX compiler to ensure correct execution of legacy PTX code.

The *minor* number is incremented when new features are added to PTX.

Semantics

Indicates that this module must be compiled with tools that support an equal or greater version number.

Each PTX module must begin with a `.version` directive, and no other `.version` directive is allowed anywhere else within the module.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
.version 3.1
.version 3.0
.version 2.3
```

11.1.2. PTX Module Directives: `.target`

`.target`

Architecture and Platform target.

Syntax

```
.target stringlist          // comma separated list of target specifiers
string = { sm_120a, sm_120f, sm_120,          // sm_12x target architectures
           sm_121a, sm_121f, sm_121,          // sm_12x target architectures
           sm_110a, sm_110f, sm_110,          // sm_11x target architectures
           sm_100a, sm_100f, sm_100,          // sm_10x target architectures
           sm_101a, sm_101f, sm_101,          // sm_10x target architectures
           sm_103a, sm_103f, sm_103,          // sm_10x target architectures
           sm_90a, sm_90,                     // sm_9x target architectures
           sm_80, sm_86, sm_87, sm_88, sm_89, // sm_8x target architectures
           sm_70, sm_72, sm_75,               // sm_7x target architectures
           sm_60, sm_61, sm_62,               // sm_6x target architectures
           sm_50, sm_52, sm_53,               // sm_5x target architectures
           sm_30, sm_32, sm_35, sm_37,         // sm_3x target architectures
           sm_20,                             // sm_2x target architectures
           sm_10, sm_11, sm_12, sm_13,         // sm_1x target architectures
           texmode_unified, texmode_independent, // texturing mode
           debug,                             // platform option
           map_f64_to_f32 };                  // platform option
```

Description

Specifies the set of features in the target architecture for which the current PTX code was generated. In general, generations of SM architectures follow an *onion layer* model, where each generation adds new features and retains all features of previous generations. The onion layer model allows the PTX code generated for a given target to be run on later generation devices.

Target architectures with suffix “a”, such as `sm_90a`, include architecture-specific features that are supported on the specified architecture only, hence such targets do not follow the onion layer model. Therefore, PTX code generated for such targets cannot be run on later generation devices. Architecture-specific features can only be used with targets that support these features.

Target architectures with suffix “f”, such as `sm_100f`, include family-specific features that are supported only within the same architecture family. Therefore, PTX code generated for such targets can run only on later generation devices in the same family. Family-specific features can be used with f-targets as well as a-targets of later generation devices in the same family.

Table 64 defines the architecture families.

Table 64: Architecture Families

Family	Target SM architectures included
sm_10x family	sm_100f, sm_103f, future targets in sm_10x family
sm_11x family	sm_110f, sm_101f, future targets in sm_11x family
sm_12x family	sm_120f, sm_121f, future targets in sm_12x family

Semantics

Each PTX module must begin with a `.version` directive, immediately followed by a `.target` directive containing a target architecture and optional platform options. A `.target` directive specifies a single target architecture, but subsequent `.target` directives can be used to change the set of target features allowed during parsing. A program with multiple `.target` directives will compile and run only on devices that support all features of the highest-numbered architecture listed in the program.

PTX features are checked against the specified target architecture, and an error is generated if an unsupported feature is used. The following table summarizes the features in PTX that vary according to target architecture.

Target	Description
sm_120	Baseline feature set for sm_120 architecture.
sm_120f	Adds support for sm_120f family specific features.
sm_120a	Adds support for sm_120a architecture-specific features.
sm_121	Baseline feature set for sm_121 architecture.
sm_121f	Adds support for sm_121f family specific features.
sm_121a	Adds support for sm_121a architecture-specific features.

Target	Description
sm_110	Baseline feature set for sm_110 architecture.
sm_110f	Adds support for sm_110f family specific features.
sm_110a	Adds support for sm_110a architecture-specific features.

Target	Description
sm_100	Baseline feature set for sm_100 architecture.
sm_100f	Adds support for sm_100f family specific features.
sm_100a	Adds support for sm_100a architecture-specific features.
sm_101	Baseline feature set for sm_101 architecture. (Renamed to sm_110)
sm_101f	Adds support for sm_101f family specific features. (Renamed to sm_110f)
sm_101a	Adds support for sm_101a architecture-specific features. (Renamed to sm_110a)
sm_103	Baseline feature set for sm_103 architecture.
sm_103f	Adds support for sm_103f family specific features.
sm_103a	Adds support for sm_103a architecture-specific features.

Target	Description
sm_90	Baseline feature set for sm_90 architecture.
sm_90a	Adds support for sm_90a architecture-specific features.

Target	Description
sm_80	Baseline feature set for sm_80 architecture.
sm_86	Adds support for .xorsign modifier on min and max instructions.
sm_87	Baseline feature set for sm_87 architecture.
sm_88	Baseline feature set for sm_88 architecture.
sm_89	Baseline feature set for sm_89 architecture.

Target	Description
sm_70	Baseline feature set for sm_70 architecture.
sm_72	Adds support for integer multiplicand and accumulator matrices in <code>wmma</code> instructions. Adds support for <code>cvt.pack</code> instruction.
sm_75	Adds support for sub-byte integer and single-bit multiplicand matrices in <code>wmma</code> instructions. Adds support for <code>ldmatrix</code> instruction. Adds support for <code>movmatrix</code> instruction. Adds support for <code>tanh</code> instruction.

Target	Description
sm_60	Baseline feature set for sm_60 architecture.
sm_61	Adds support for <code>dp2a</code> and <code>dp4a</code> instructions.
sm_62	Baseline feature set for sm_61 architecture.

Target	Description
sm_50	Baseline feature set for sm_50 architecture.
sm_52	Baseline feature set for sm_50 architecture.
sm_53	Adds support for arithmetic, comparison and texture instructions for <code>.f16</code> and <code>.f16x2</code> types.

Target	Description
sm_30	Baseline feature set for sm_30 architecture.
sm_32	Adds 64-bit <code>{atom, red}</code> . <code>{and, or, xor, min, max}</code> instructions. Adds <code>shf</code> instruction. Adds <code>ld.global.nc</code> instruction.
sm_35	Adds support for CUDA Dynamic Parallelism.
sm_37	Baseline feature set for sm_35 architecture.

Target	Description
sm_20	Baseline feature set for sm_20 architecture.

Target	Description
sm_10	Baseline feature set for sm_10 architecture. Requires map_f64_to_f32 if any .f64 instructions used.
sm_11	Adds 64-bit {atom, red}. {and, or, xor, min, max} instructions. Requires map_f64_to_f32 if any .f64 instructions used.
sm_12	Adds {atom, red}.shared, 64-bit {atom, red}.global, vote instructions. Requires map_f64_to_f32 if any .f64 instructions used.
sm_13	Adds double-precision support, including expanded rounding modifiers. Disallows use of map_f64_to_f32.

The texturing mode is specified for an entire module and cannot be changed within the module.

The .target debug option declares that the PTX file contains DWARF debug information, and subsequent compilation of PTX will retain information needed for source-level debugging. If the debug option is declared, an error message is generated if no DWARF information is found in the file. The debug option requires PTX ISA version 3.0 or later.

map_f64_to_f32 indicates that all double-precision instructions map to single-precision regardless of the target architecture. This enables high-level language compilers to compile programs containing type double to target device that do not support double-precision operations. Note that .f64 storage remains as 64-bits, with only half being used by instructions converted from .f64 to .f32.

Notes

Targets of the form compute_xx are also accepted as synonyms for sm_xx targets.

Targets sm_{101, 101f, 101a} are renamed to targets sm_{110, 110f, 110a} from PTX ISA version 9.0.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target strings sm_10 and sm_11 introduced in PTX ISA version 1.0.

Target strings sm_12 and sm_13 introduced in PTX ISA version 1.2.

Texturing mode introduced in PTX ISA version 1.5.

Target string sm_20 introduced in PTX ISA version 2.0.

Target string sm_30 introduced in PTX ISA version 3.0.

Platform option debug introduced in PTX ISA version 3.0.

Target string sm_35 introduced in PTX ISA version 3.1.

Target strings sm_32 and sm_50 introduced in PTX ISA version 4.0.

Target strings sm_37 and sm_52 introduced in PTX ISA version 4.1.

Target string sm_53 introduced in PTX ISA version 4.2.

Target string sm_60, sm_61, sm_62 introduced in PTX ISA version 5.0.

Target string sm_70 introduced in PTX ISA version 6.0.

Target string `sm_72` introduced in PTX ISA version 6.1.

Target string `sm_75` introduced in PTX ISA version 6.3.

Target string `sm_80` introduced in PTX ISA version 7.0.

Target string `sm_86` introduced in PTX ISA version 7.1.

Target string `sm_87` introduced in PTX ISA version 7.4.

Target string `sm_88` introduced in PTX ISA version 9.0.

Target string `sm_89` introduced in PTX ISA version 7.8.

Target string `sm_90` introduced in PTX ISA version 7.8.

Target string `sm_90a` introduced in PTX ISA version 8.0.

Target string `sm_100` introduced in PTX ISA version 8.6.

Target string `sm_100f` introduced in PTX ISA version 8.8.

Target string `sm_100a` introduced in PTX ISA version 8.6.

Target string `sm_101` introduced in PTX ISA version 8.6. (Renamed to `sm_110`)

Target string `sm_101f` introduced in PTX ISA version 8.8. (Renamed to `sm_110f`)

Target string `sm_101a` introduced in PTX ISA version 8.6. (Renamed to `sm_110a`)

Target string `sm_103` introduced in PTX ISA version 8.8.

Target string `sm_103f` introduced in PTX ISA version 8.8.

Target string `sm_103a` introduced in PTX ISA version 8.8.

Target string `sm_110` introduced in PTX ISA version 9.0.

Target string `sm_110f` introduced in PTX ISA version 9.0.

Target string `sm_110a` introduced in PTX ISA version 9.0.

Target string `sm_120` introduced in PTX ISA version 8.7.

Target string `sm_120f` introduced in PTX ISA version 8.8.

Target string `sm_120a` introduced in PTX ISA version 8.7.

Target string `sm_121` introduced in PTX ISA version 8.8.

Target string `sm_121f` introduced in PTX ISA version 8.8.

Target string `sm_121a` introduced in PTX ISA version 8.8.

Target ISA Notes

The `.target` directive is supported on all target architectures.

Examples

```
.target sm_10      // baseline target architecture
.target sm_13      // supports double-precision
.target sm_20, texmode_independent
.target sm_90       // baseline target architecture
.target sm_90a      // PTX using architecture-specific features
.target sm_100f     // PTX using family-specific features
```

11.1.3. PTX Module Directives: `.address_size`

`.address_size`

Address size used throughout PTX module.

Syntax

```
.address_size address-size
address-size = { 32, 64 };
```

Description

Specifies the address size assumed throughout the module by the PTX code and the binary DWARF information in PTX.

Redefinition of this directive within a module is not allowed. In the presence of separate compilation all modules must specify (or default to) the same address size.

The `.address_size` directive is optional, but it must immediately follow the `.target` directive if present within a module.

Semantics

If the `.address_size` directive is omitted, the address size defaults to 32.

PTX ISA Notes

Introduced in PTX ISA version 2.3.

Target ISA Notes

Supported on all target architectures.

Examples

```
// example directives
.address_size 32      // addresses are 32 bit
.address_size 64      // addresses are 64 bit

// example of directive placement within a module
.version 2.3
.target sm_20
.address_size 64
...
.entry foo () {
...
}
```

11.2. Specifying Kernel Entry Points and Functions

The following directives specify kernel entry points and functions.

- ▶ `.entry`
- ▶ `.func`

11.2.1. Kernel and Function Directives: `.entry`

`.entry`

Kernel entry point and body, with optional parameters.

Syntax

```
.entry kernel-name ( param-list ) kernel-body
.entry kernel-name kernel-body
```

Description

Defines a kernel entry point name, parameters, and body for the kernel function.

Parameters are passed via `.param` space memory and are listed within an optional parenthesized parameter list. Parameters may be referenced by name within the kernel body and loaded into registers using `ld.param{ : :entry }` instructions.

In addition to normal parameters, opaque `.texref`, `.samplerref`, and `.surfref` variables may be passed as parameters. These parameters can only be referenced by name within texture and surface load, store, and query instructions and cannot be accessed via `ld.param` instructions.

The shape and size of the CTA executing the kernel are available in special registers.

Semantics

Specify the entry point for a kernel program.

At kernel launch, the kernel dimensions and properties are established and made available via special registers, e.g., `%ntid`, `%nctaid`, etc.

PTX ISA Notes

For PTX ISA version 1.4 and later, parameter variables are declared in the kernel parameter list. For PTX ISA versions 1.0 through 1.3, parameter variables are declared in the kernel body.

The maximum memory size supported by PTX for normal (non-opaque type) parameters is 32764 bytes. Depending upon the PTX ISA version, the parameter size limit varies. The following table shows the allowed parameter size for a PTX ISA version:

PTX ISA Version	Maximum parameter size (In bytes)
PTX ISA version 8.1 and above	32764
PTX ISA version 1.5 and above	4352
PTX ISA version 1.4 and above	256

The CUDA and OpenCL drivers support the following limits for parameter memory:

Driver	Parameter memory size
CUDA	256 bytes for sm_1x, 4096 bytes for sm_2x and higher, 32764 bytes for sm_70 and higher
OpenCL	32764 bytes for sm_70 and higher, 4352 bytes on sm_6x and lower

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry cta_fft
.entry filter ( .param .b32 x, .param .b32 y, .param .b32 z )
{
    .reg .b32 %r<99>;
    ld.param.b32 %r1, [x];
    ld.param.b32 %r2, [y];
    ld.param.b32 %r3, [z];
    ...
}

.entry prefix_sum ( .param .align 4 .s32 pitch[8000] )
{
    .reg .s32 %t;
    ld.param::entry.s32 %t, [pitch];
    ...
}
```

11.2.2. Kernel and Function Directives: .func

.func

Function definition.

Syntax

```
.func { .attribute(attr-list) } fname { .noreturn } { .abi_preserve N } { .abi_preserve_
↪ control N } function-body
.func { .attribute(attr-list) } fname (param-list) { .noreturn } { .abi_preserve N } { .abi_
↪ preserve_control N } function-body
.func { .attribute(attr-list) } (ret-param) fname (param-list) { .abi_preserve N } { .abi_
↪ preserve_control N } function-body
```

Description

Defines a function, including input and return parameters and optional function body.

An optional `.noreturn` directive indicates that the function does not return to the caller function. `.noreturn` directive cannot be specified on functions which have return parameters. See the description of `.noreturn` directive in *Performance-Tuning Directives: .noreturn*.

An optional `.attribute` directive specifies additional information associated with the function. See the description of *Variable and Function Attribute Directive: .attribute* for allowed attributes.

Optional `.abi_preserve` and `.abi_preserve_control` directives are used to specify the number of general purpose registers and control registers. See description of [Performance-Tuning Directives: `.abi\_preserve`](#) and [Performance-Tuning Directives: `.abi\_preserve\_control`](#) for more details.

Directives, if any specified, for a function, e.g. `.noreturn`, must be specified consistently between function declaration and definition.

A `.func` definition with no body provides a function prototype.

The parameter lists define locally-scoped variables in the function body. Parameters must be base types in either the register or parameter state space. Parameters in register state space may be referenced directly within instructions in the function body. Parameters in `.param` space are accessed using `ld.param{::func}` and `st.param{::func}` instructions in the body. Parameter passing is call-by-value.

The last parameter in the parameter list may be a `.param` array of type `.b8` with no size specified. It is used to pass an arbitrary number of parameters to the function packed into a single array object.

When calling a function with such an unsized last argument, the last argument may be omitted from the `call` instruction if no parameter is passed through it. Accesses to this array parameter must be within the bounds of the array. The result of an access is undefined if no array was passed, or if the access was outside the bounds of the actual array being passed.

Semantics

The PTX syntax hides all details of the underlying calling convention and ABI.

The implementation of parameter passing is left to the optimizing translator, which may use a combination of registers and stack locations to pass parameters.

Release Notes

For PTX ISA version 1.x code, parameters must be in the register state space, there is no stack, and recursion is illegal.

PTX ISA versions 2.0 and later with target `sm_20` or higher allow parameters in the `.param` state space, implements an ABI with stack, and supports recursion.

PTX ISA versions 2.0 and later with target `sm_20` or higher support at most one return value.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Support for unsized array parameter introduced in PTX ISA version 6.0.

Support for `.noreturn` directive introduced in PTX ISA version 6.4.

Support for `.attribute` directive introduced in PTX ISA version 8.0.

Support for `.abi_preserve` and `.abi_preserve_control` directives introduced in PTX ISA version 9.0.

Target ISA Notes

Functions without unsized array parameter supported on all target architectures.

Unsized array parameter requires sm_30 or higher.

.noreturn directive requires sm_30 or higher.

.attribute directive requires sm_90 or higher.

.abi_preserve and .abi_preserve_control directives require sm_80 or higher.

Examples

```
.func (.reg .b32 rval) foo (.reg .b32 N, .reg .f64 dbl)
{
    .reg .b32 localVar;

    ... use N, dbl;
    other code;

    mov.b32 rval,result;
    ret;
}

...
call (fooval), foo, (val0, val1); // return value in fooval
...

.func foo (.reg .b32 N, .reg .f64 dbl) .noreturn
{
    .reg .b32 localVar;
    ... use N, dbl;
    other code;
    mov.b32 rval, result;
    ret;
}

...
call foo, (val0, val1);
...

.func (.param .u32 rval) bar(.param .u32 N, .param .align 4 .b8 numbers[])
{
    .reg .b32 input0, input1;
    ld.param.b32 input0, [numbers + 0];
    ld.param.b32 input1, [numbers + 4];
    ...
    other code;
    ret;
}

...

.param .u32 N;
.param .align 4 .b8 numbers[8];
st.param.u32 [N], 2;
st.param.b32 [numbers + 0], 5;
st.param.b32 [numbers + 4], 10;
```

(continues on next page)

(continued from previous page)

```
call (rval), bar, (N, numbers);  
...
```

11.2.3. Kernel and Function Directives: `.alias`

`.alias`

Define an alias to existing function symbol.

Syntax

```
.alias fAlias, fAliasee;
```

Description

`.alias` is a module scope directive that defines identifier `fAlias` to be an alias to function specified by `fAliasee`.

Both `fAlias` and `fAliasee` are non-entry function symbols.

Identifier `fAlias` is a function declaration without body.

Identifier `fAliasee` is a function symbol which must be defined in the same module as `.alias` declaration. Function `fAliasee` cannot have `.weak` linkage.

Prototype of `fAlias` and `fAliasee` must match.

Program can use either `fAlias` or `fAliasee` identifiers to reference function defined with `fAliasee`.

PTX ISA Notes

`.alias` directive introduced in PTX ISA 6.3.

Target ISA Notes

`.alias` directive requires `sm_30` or higher.

Examples

```
.visible .func foo(.param .u32 p) {  
    ...  
}  
.visible .func bar(.param .u32 p);  
.alias bar, foo;  
.entry test()  
{
```

(continues on next page)

(continued from previous page)

```

    .param .u32 p;
    ...
    call foo, (p);          // call foo directly
    ...
    .param .u32 p;
    call bar, (p);          // call foo through alias
}
.entry filter ( .param .b32 x, .param .b32 y, .param .b32 z )
{
    .reg .b32 %r1, %r2, %r3;
    ld.param.b32 %r1, [x];
    ld.param.b32 %r2, [y];
    ld.param.b32 %r3, [z];
    ...
}

```

11.3. Control Flow Directives

PTX provides directives for specifying potential targets for `brx.idx` and `call` instructions. See the descriptions of `brx.idx` and `call` for more information.

- ▶ `.branchtargets`
- ▶ `.calltargets`
- ▶ `.callprototype`

11.3.1. Control Flow Directives: `.branchtargets`

`.branchtargets`

Declare a list of potential branch targets.

Syntax

```
Label:    .branchtargets  list-of-labels ;
```

Description

Declares a list of potential branch targets for a subsequent `brx.idx`, and associates the list with the label at the start of the line.

All control flow labels in the list must occur within the same function as the declaration.

The list of labels may use the compact, shorthand syntax for enumerating a range of labels having a common prefix, similar to the syntax described in *Parameterized Variable Names*.

PTX ISA Notes

Introduced in PTX ISA version 2.1.

Target ISA Notes

Requires sm_20 or higher.

Examples

```
.function foo () {
    .reg .u32 %r0;
    ...
    L1:
    ...
    L2:
    ...
    L3:
    ...
    ts: .branchtargets L1, L2, L3;
    @p brx.idx %r0, ts;
    ...
}

.function bar() {
    .reg .u32 %r0;
    ...
    N0:
    ...
    N1:
    ...
    N2:
    ...
    N3:
    ...
    N4:
    ...
    ts: .branchtargets N<5>;
    @p brx.idx %r0, ts;
    ...
}
```

11.3.2. Control Flow Directives: `.calltargets`

`.calltargets`

Declare a list of potential call targets.

Syntax

```
Label:    .calltargets  list-of-functions ;
```

Description

Declares a list of potential call targets for a subsequent indirect call, and associates the list with the label at the start of the line.

All functions named in the list must be declared prior to the `.calltargets` directive, and all functions must have the same type signature.

PTX ISA Notes

Introduced in PTX ISA version 2.1.

Target ISA Notes

Requires sm_20 or higher.

Examples

```
calltgt: .calltargets  fastsin, fastcos;
...
@p    call  (%f1), %r0, (%x), calltgt;
...
```

11.3.3. Control Flow Directives: `.callprototype`

`.callprototype`

Declare a prototype for use in an indirect call.

Syntax

```
// no input or return parameters
label: .callprototype _ .noreturn { .abi_preserve N } { .abi_preserve_control N };
// input params, no return params
label: .callprototype _ (param-list) .noreturn { .abi_preserve N } { .abi_preserve_
↳ control N };
// no input params, // return params
label: .callprototype (ret-param) _ { .abi_preserve N } { .abi_preserve_control N };
// input, return parameters
label: .callprototype (ret-param) _ (param-list) { .abi_preserve N } { .abi_preserve_
↳ control N };
```

Description

Defines a prototype with no specific function name, and associates the prototype with a label. The prototype may then be used in indirect call instructions where there is incomplete knowledge of the possible call targets.

Parameters may have either base types in the register or parameter state spaces, or array types in parameter state space. The sink symbol ' _ ' may be used to avoid dummy parameter names.

An optional `.noreturn` directive indicates that the function does not return to the caller function. `.noreturn` directive cannot be specified on functions which have return parameters. See the description of `.noreturn` directive in [Performance-Tuning Directives: `.noreturn`](#).

Optional `.abi_preserve` and `.abi_preserve_control` directives are used to specify the number of general purpose registers and control registers. See description of [Performance-Tuning Directives: `.abi\_preserve`](#) and [Performance-Tuning Directives: `.abi\_preserve\_control`](#) for more details.

PTX ISA Notes

Introduced in PTX ISA version 2.1.

Support for `.noreturn` directive introduced in PTX ISA version 6.4.

Support for `.abi_preserve` and `.abi_preserve_control` directives introduced in PTX ISA version 9.0.

Target ISA Notes

Requires sm_20 or higher.

`.noreturn` directive requires sm_30 or higher.

`.abi_preserve` and `.abi_preserve_control` directives require sm_80 or higher.

Examples

```
Fproto1: .callprototype _ ;
Fproto2: .callprototype _ (.param .f32 _);
Fproto3: .callprototype (.param .u32 _) _ ;
Fproto4: .callprototype (.param .u32 _) _ (.param .f32 _);
...
@p    call  (%val), %r0, (%f1), Fproto4;
...

// example of array parameter
Fproto5: .callprototype _ (.param .b8 _[12]);

Fproto6: .callprototype _ (.param .f32 _) .noreturn;
...
@p    call  %r0, (%f1), Fproto6;
...

// example of .abi_preserve
Fproto7: .callprototype _ (.param .b32 _) .abi_preserve 10;
...
```

(continues on next page)

(continued from previous page)

```
@p    call %r0, (%r1), Fproto7;
...
```

11.4. Performance-Tuning Directives

To provide a mechanism for low-level performance tuning, PTX supports the following directives, which pass information to the optimizing backend compiler.

- ▶ `.maxnreg`
- ▶ `.maxntid`
- ▶ `.reqntid`
- ▶ `.minnctapersm`
- ▶ `.maxnctapersm` (deprecated)
- ▶ `.pragma`
- ▶ `.abi_preserve`
- ▶ `.abi_preserve_control`

The `.maxnreg` directive specifies the maximum number of registers to be allocated to a single thread; the `.maxntid` directive specifies the maximum number of threads in a thread block (CTA); the `.reqntid` directive specifies the required number of threads in a thread block (CTA); and the `.minnctapersm` directive specifies a minimum number of thread blocks to be scheduled on a single multiprocessor (SM). These can be used, for example, to throttle the resource requirements (e.g., registers) to increase total thread count and provide a greater opportunity to hide memory latency. The `.minnctapersm` directive can be used together with either the `.maxntid` or `.reqntid` directive to trade-off registers-per-thread against multiprocessor utilization without needing to directly specify a maximum number of registers. This may achieve better performance when compiling PTX for multiple devices having different numbers of registers per SM.

Device function directives `.abi_preserve` and `.abi_preserve_control` specify number of data and control registers from callee save registers that a function must preserve for its caller. This can be considered to be the number of general purpose and control registers live in the caller when function is called. Control registers refer to the number of divergent program points that happen in the calltree leading to current function call.

Currently, the `.maxnreg`, `.maxntid`, `.reqntid`, and `.minnctapersm` directives may be applied per-entry and must appear between an `.entry` directive and its body. The directives take precedence over any module-level constraints passed to the optimizing backend. A warning message is generated if the directives' constraints are inconsistent or cannot be met for the specified target device.

A general `.pragma` directive is supported for passing information to the PTX backend. The directive passes a list of strings to the backend, and the strings have no semantics within the PTX virtual machine model. The interpretation of `.pragma` values is determined by the backend implementation and is beyond the scope of the PTX ISA. Note that `.pragma` directives may appear at module (file) scope, at entry-scope, or as statements within a kernel or device function body.

11.4.1. Performance-Tuning Directives: `.maxnreg`

`.maxnreg`

Maximum number of registers that can be allocated per thread.

Syntax

```
.maxnreg n
```

Description

Declare the maximum number of registers per thread in a CTA.

Semantics

The compiler guarantees that this limit will not be exceeded. The actual number of registers used may be less; for example, the backend may be able to compile to fewer registers, or the maximum number of registers may be further constrained by `.maxntid` and `.maxctapersm`.

PTX ISA Notes

Introduced in PTX ISA version 1.3.

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry foo .maxnreg 16 { ... } // max regs per thread = 16
```

11.4.2. Performance-Tuning Directives: `.maxntid`

`.maxntid`

Maximum number of threads in the thread block (CTA).

Syntax

```
.maxntid nx
.maxntid nx, ny
.maxntid nx, ny, nz
```

Description

Declare the maximum number of threads in the thread block (CTA). This maximum is specified by giving the maximum extent of each dimension of the 1D, 2D, or 3D CTA. The maximum number of threads is the product of the maximum extent in each dimension.

Semantics

The maximum number of threads in the thread block, computed as the product of the maximum extent specified for each dimension, is guaranteed not to be exceeded in any invocation of the kernel in which this directive appears. Exceeding the maximum number of threads results in a runtime error or kernel launch failure.

Note that this directive guarantees that the *total* number of threads does not exceed the maximum, but does not guarantee that the limit in any particular dimension is not exceeded.

PTX ISA Notes

Introduced in PTX ISA version 1.3.

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry foo .maxntid 256      { ... } // max threads = 256
.entry bar .maxntid 16,16,4 { ... } // max threads = 1024
```

11.4.3. Performance-Tuning Directives: .reqntid

.reqntid

Number of threads in the thread block (CTA).

Syntax

```
.reqntid nx  
.reqntid nx, ny  
.reqntid nx, ny, nz
```

Description

Declare the number of threads in the thread block (CTA) by specifying the extent of each dimension of the 1D, 2D, or 3D CTA. The total number of threads is the product of the number of threads in each dimension.

Semantics

The size of each CTA dimension specified in any invocation of the kernel is required to be equal to that specified in this directive. Specifying a different CTA dimension at launch will result in a runtime error or kernel launch failure.

Notes

The `.reqntid` directive cannot be used in conjunction with the `.maxntid` directive.

PTX ISA Notes

Introduced in PTX ISA version 2.1.

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry foo .reqntid 256      { ... } // num threads = 256  
.entry bar .reqntid 16,16,4 { ... } // num threads = 1024
```

11.4.4. Performance-Tuning Directives: `.minnctapersm`

`.minnctapersm`

Minimum number of CTAs per SM.

Syntax

```
.minnctapersm ncta
```

Description

Declare the minimum number of CTAs from the kernel's grid to be mapped to a single multiprocessor (SM).

Notes

Optimizations based on `.minnctapersm` need either `.maxntid` or `.reqntid` to be specified as well. If the total number of threads on a single SM resulting from `.minnctapersm` and `.maxntid` / `.reqntid` exceed maximum number of threads supported by an SM then directive `.minnctapersm` will be ignored.

In PTX ISA version 2.1 or higher, a warning is generated if `.minnctapersm` is specified without specifying either `.maxntid` or `.reqntid`.

PTX ISA Notes

Introduced in PTX ISA version 2.0 as a replacement for `.maxnctapersm`.

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry foo .maxntid 256 .minnctapersm 4 { ... }
```

11.4.5. Performance-Tuning Directives: `.maxnctapersm` (deprecated)

`.maxnctapersm`

Maximum number of CTAs per SM.

Syntax

```
.maxnctapersm ncta
```

Description

Declare the maximum number of CTAs from the kernel's grid that may be mapped to a single multiprocessor (SM).

Notes

Optimizations based on `.maxnctapersm` generally need `.maxntid` to be specified as well. The optimizing backend compiler uses `.maxntid` and `.maxnctapersm` to compute an upper-bound on per-thread register usage so that the specified number of CTAs can be mapped to a single multiprocessor. However, if the number of registers used by the backend is sufficiently lower than this bound, additional CTAs may be mapped to a single multiprocessor. For this reason, `.maxnctapersm` has been renamed to `.minnctapersm` in PTX ISA version 2.0.

PTX ISA Notes

Introduced in PTX ISA version 1.3. Deprecated in PTX ISA version 2.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry foo .maxntid 256 .maxnctapersm 4 { ... }
```

11.4.6. Performance-Tuning Directives: `.noreturn`

`.noreturn`

Indicate that the function does not return to its caller function.

Syntax

```
.noreturn
```

Description

Indicate that the function does not return to its caller function.

Semantics

An optional `.noreturn` directive indicates that the function does not return to caller function. `.noreturn` directive can only be specified on device functions and must appear between a `.func` directive and its body.

The directive cannot be specified on functions which have return parameters.

If a function with `.noreturn` directive returns to the caller function at runtime, then the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 6.4.

Target ISA Notes

Requires `sm_30` or higher.

Examples

```
.func foo .noreturn { ... }
```

11.4.7. Performance-Tuning Directives: `.pragma`

`.pragma`

Pass directives to PTX backend compiler.

Syntax

```
.pragma list-of-strings ;
```

Description

Pass module-scoped, entry-scoped, or statement-level directives to the PTX backend compiler.

The `.pragma` directive may occur at module-scope, at entry-scope, or at statement-level.

Semantics

The interpretation of `.pragma` directive strings is implementation-specific and has no impact on PTX semantics. See *Descriptions of .pragma Strings* for descriptions of the pragma strings defined in `ptxas`.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
.pragma "nounroll";    // disable unrolling in backend

// disable unrolling for current kernel
.entry foo .pragma "nounroll"; { ... }
```

11.4.8. Performance-Tuning Directives: `.abi_preserve`

`.abi_preserve`

Specify number of general purpose registers that should be preserved by the callers of this function.

Syntax

```
.abi_preserve N
```

Description

It is an architecture agnostic value specifying actual number of general purpose registers. Internally ABI defines some general purpose registers as preserved (callee save) registers. Integer N specifies the actual number of general purpose registers that should be preserved by the function.

`.abi_preserve` directive can only be specified on device functions and must appear between a `.func` directive and its body.

Semantics

When this directive is specified compiler backend modifies low level ABI components to ensure that number of live data variables in the callers of this function that are stored in the callee save registers are less than specified value.

PTX ISA Notes

Introduced in PTX ISA version 9.0.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
.func bar() .abi_preserve 8

// Indirect call via call prototype
.func (.param .b32 out[30]) foo (.param .b32 in[30]) .abi_preserve 10 { ... }
...
mov.b64 lpfoo, foo;
prot: .callprototype (.param .b32 out[30]) _ (.param .b32 in[30]) .abi_preserve 10;
call (out), lpfoo, (in), prot;
```

11.4.9. Performance-Tuning Directives: `.abi_preserve_control`

`.abi_preserve_control`

Specify number of control registers that should be preserved by the callers of this function.

Syntax

```
.abi_preserve_control N
```

Description

It is an architecture agnostic value specifying the number of divergent program points that happen in the calltree leading to current function call. Internally ABI defines some control registers as preserved (callee save) registers. Integer N specifies the actual number of control registers that should be preserved by the function.

`.abi_preserve_control` directive can only be specified on device functions and must appear between a `.func` directive and its body.

Semantics

When this directive is specified compiler backend modifies low level ABI components to ensure that number of live control variables in the callers of this function that are stored in the callee save control registers are less than specified value.

PTX ISA Notes

Introduced in PTX ISA version 9.0.

Target ISA Notes

Requires `sm_80` or higher.

Examples

```
.func foo() .abi_preserve_control 14

// Indirect call via call prototype
.func (.param .b32 out[30]) bar (.param .b32 in[30]) .abi_preserve_control 10 { ... }
...
mov.b64 lpbar, bar;
prot: .callprototype (.param .b32 out[30]) _ (.param .b32 in[30]) .abi_preserve_
    ↪control 10;
call (out), lpbar, (in), prot;
```

11.5. Debugging Directives

DWARF-format debug information is passed through PTX modules using the following directives:

- ▶ `@@DWARF`
- ▶ `.section`
- ▶ `.file`
- ▶ `.loc`

The `.section` directive was introduced in PTX ISA version 2.0 and replaces the `@@DWARF` syntax. The `@@DWARF` syntax was deprecated in PTX ISA version 2.0 but is supported for legacy PTX ISA version 1.x code.

Beginning with PTX ISA version 3.0, PTX files containing DWARF debug information should include the `.target debug platform` option. This forward declaration directs PTX compilation to retain mappings for source-level debugging.

11.5.1. Debugging Directives: `@@dwarf`

`@@dwarf`

DWARF-format information.

Syntax

```
@@DWARF dwarf-string

dwarf-string may have one of the
.byte  byte-list    // comma-separated hexadecimal byte values
.4byte int32-list   // comma-separated hexadecimal integers in range [0..2^32-1]
.quad  int64-list   // comma-separated hexadecimal integers in range [0..2^64-1]
.4byte label
.quad  label
```

PTX ISA Notes

Introduced in PTX ISA version 1.2. Deprecated as of PTX ISA version 2.0, replaced by `.section` directive.

Target ISA Notes

Supported on all target architectures.

Examples

```
@@DWARF .section .debug_pubnames, "", @progbits
@@DWARF .byte    0x2b, 0x00, 0x00, 0x00, 0x02, 0x00
@@DWARF .4byte   .debug_info
@@DWARF .4byte   0x0000006b5, 0x000000364, 0x61395a5f, 0x5f736f63
@@DWARF .4byte   0x6e69616d, 0x63613031, 0x6150736f, 0x736d6172
@@DWARF .byte    0x00, 0x00, 0x00, 0x00, 0x00
```

11.5.2. Debugging Directives: .section

.section

PTX section definition.

Syntax

```
.section section_name { dwarf-lines }
```

dwarf-lines have the following formats:

.b8	byte-list	// comma-separated list of integers // in range [-128..255]
.b16	int16-list	// comma-separated list of integers // in range [-2 ¹⁵ ..2 ¹⁶ -1]
.b32	int32-list	// comma-separated list of integers // in range [-2 ³¹ ..2 ³² -1]
label:		// Define label inside the debug section
.b64	int64-list	// comma-separated list of integers // in range [-2 ⁶³ ..2 ⁶⁴ -1]
.b32	label	
.b64	label	
.b32	label+imm	// a sum of label address plus a constant integer byte // offset(signed, 32bit)
.b64	label+imm	// a sum of label address plus a constant integer byte // offset(signed, 64bit)
.b32	label1-label2	// a difference in label addresses between labels in // the same dwarf section (32bit)
.b64	label3-label4	// a difference in label addresses between labels in // the same dwarf section (64bit)

PTX ISA Notes

Introduced in PTX ISA version 2.0, replaces @@DWARF syntax.

label+imm expression introduced in PTX ISA version 3.2.

Support for .b16 integers in dwarf-lines introduced in PTX ISA version 6.0.

Support for defining label inside the DWARF section is introduced in PTX ISA version 7.2.

label1-label2 expression introduced in PTX ISA version 7.5.

Negative numbers in dwarf lines introduced in PTX ISA version 7.5.

Target ISA Notes

Supported on all target architectures.

Examples

```
.section .debug_pubnames
{
    .b32    LpubNames_end0-LpubNames_begin0
    LpubNames_begin0:
    .b8     0x2b, 0x00, 0x00, 0x00, 0x02, 0x00
    .b32    .debug_info
    info_label1:
    .b32    0x000006b5, 0x00000364, 0x61395a5f, 0x5f736f63
    .b32    0x6e69616d, 0x63613031, 0x6150736f, 0x736d6172
    .b8     0x00, 0x00, 0x00, 0x00, 0x00
    LpubNames_end0:
}

.section .debug_info
{
    .b32 11430
    .b8 2, 0
    .b32 .debug_abbrev
    .b8 8, 1, 108, 103, 101, 110, 102, 101, 58, 32, 69, 68, 71, 32, 52, 46, 49
    .b8 0
    .b32 3, 37, 176, -99
    .b32 info_label1
    .b32 .debug_loc+0x4
    .b8 -11, 11, 112, 97
    .b32 info_label1+12
    .b64 -1
    .b16 -5, -65535
}
```

11.5.3. Debugging Directives: `.file`

`.file`

Source file name.

Syntax

```
.file file_index "filename" {, timestamp, file_size}
```

Description

Associates a source filename with an integer index. `.loc` directives reference source files by index.

`.file` directive allows optionally specifying an unsigned number representing time of last modification and an unsigned integer representing size in bytes of source file. `timestamp` and `file_size` value can be 0 to indicate this information is not available.

`timestamp` value is in format of C and C++ data type `time_t`.

`file_size` is an unsigned 64-bit integer.

The `.file` directive is allowed only in the outermost scope, i.e., at the same level as kernel and device function declarations.

Semantics

If `timestamp` and `file_size` are not specified, they default to 0.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Timestamp and file size introduced in PTX ISA version 3.2.

Target ISA Notes

Supported on all target architectures.

Examples

```
.file 1 "example.cu"  
.file 2 "kernel.cu"  
.file 1 "kernel.cu", 1339013327, 64118
```


11.5.4. Debugging Directives: `.loc`

`.loc`

Source file location.

Syntax

```
.loc file_index line_number column_position
.loc file_index line_number column_position,function_name label {+ immediate },
↪inlined_at file_index2 line_number2 column_position2
```

Description

Declares the source file location (source file, line number, and column position) to be associated with lexically subsequent PTX instructions. `.loc` refers to `file_index` which is defined by a `.file` directive.

To indicate PTX instructions that are generated from a function that got inlined, additional attribute `.inlined_at` can be specified as part of the `.loc` directive. `.inlined_at` attribute specifies source location at which the specified function is inlined. `file_index2`, `line_number2`, and `column_position2` specify the location at which function is inlined. Source location specified as part of `.inlined_at` directive must lexically precede as source location in `.loc` directive.

The `function_name` attribute specifies an offset in the DWARF section named `.debug_str`. Offset is specified as `label` expression or `label + immediate` expression where `label` is defined in `.debug_str` section. DWARF section `.debug_str` contains ASCII null-terminated strings that specify the name of the function that is inlined.

Note that a PTX instruction may have a single associated source location, determined by the nearest lexically preceding `.loc` directive, or no associated source location if there is no preceding `.loc` directive. Labels in PTX inherit the location of the closest lexically following instruction. A label with no following PTX instruction has no associated source location.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

`function_name` and `inlined_at` attributes are introduced in PTX ISA version 7.2.

Target ISA Notes

Supported on all target architectures.

Examples

```

.loc 2 4237 0
L1:                                // line 4237, col 0 of file #2,
                                // inherited from mov
    mov.u32  %r1,%r2;             // line 4237, col 0 of file #2
    add.u32  %r2,%r1,%r3;        // line 4237, col 0 of file #2
...
L2:                                // line 4239, col 5 of file #2,
                                // inherited from sub
    .loc 2 4239 5
    sub.u32  %r2,%r1,%r3;        // line 4239, col 5 of file #2
    .loc 1 21 3
    .loc 1 9 3, function_name info_string0, inlined_at 1 21 3
    ld.global.u32  %r1, [gg];    // Function at line 9
    setp.lt.s32 %p1, %r1, 8;     // inlined at line 21
    .loc 1 27 3
    .loc 1 10 5, function_name info_string1, inlined_at 1 27 3
    .loc 1 15 3, function_name .debug_str+16, inlined_at 1 10 5
    setp.ne.s32 %p2, %r1, 18;
    @%p2 bra    BB2_3;

.section .debug_str {
info_string0:
    .b8 95 // _
    .b8 90 // z
    .b8 51 // 3
    .b8 102 // f
    .b8 111 // o
    .b8 111 // o
    .b8 118 // v
    .b8 0

info_string1:
    .b8 95 // _
    .b8 90 // z
    .b8 51 // 3
    .b8 98 // b
    .b8 97 // a
    .b8 114 // r
    .b8 118 // v
    .b8 0
    .b8 95 // _
    .b8 90 // z
    .b8 51 // 3
    .b8 99 // c
    .b8 97 // a
    .b8 114 // r
    .b8 118 // v
    .b8 0
}

```

11.6. Linking Directives

- ▶ `.extern`
- ▶ `.visible`
- ▶ `.weak`

11.6.1. Linking Directives: `.extern`

`.extern`

External symbol declaration.

Syntax

```
.extern identifier
```

Description

Declares identifier to be defined external to the current module. The module defining such identifier must define it as `.weak` or `.visible` only once in a single object file. Extern declaration of symbol may appear multiple times and references to that get resolved against the single definition of that symbol.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
.extern .global .b32 foo; // foo is defined in another module
```

11.6.2. Linking Directives: `.visible`

`.visible`

Visible (externally) symbol declaration.

Syntax

```
.visible identifier
```

Description

Declares identifier to be globally visible. Unlike C, where identifiers are globally visible unless declared static, PTX identifiers are visible only within the current module unless declared `.visible` outside the current.

PTX ISA Notes

Introduced in PTX ISA version 1.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
.visible .global .b32 foo; // foo will be externally visible
```

11.6.3. Linking Directives: `.weak`

`.weak`

Visible (externally) symbol declaration.

Syntax

```
.weak identifier
```

Description

Declares identifier to be globally visible but *weak*. Weak symbols are similar to globally visible symbols, except during linking, weak symbols are only chosen after globally visible symbols during symbol resolution. Unlike globally visible symbols, multiple object files may declare the same weak symbol, and references to a symbol get resolved against a weak symbol only if no global symbols have the same name.

PTX ISA Notes

Introduced in PTX ISA version 3.1.

Target ISA Notes

Supported on all target architectures.

Examples

```
.weak .func (.reg .b32 val) foo; // foo will be externally visible
```

11.6.4. Linking Directives: .common

.common

Visible (externally) symbol declaration.

Syntax

```
.common identifier
```

Description

Declares identifier to be globally visible but “common”.

Common symbols are similar to globally visible symbols. However multiple object files may declare the same common symbol and they may have different types and sizes and references to a symbol get resolved against a common symbol with the largest size.

Only one object file can initialize a common symbol and that must have the largest size among all other definitions of that common symbol from different object files.

`.common` linking directive can be used only on variables with `.global` storage. It cannot be used on function symbols or on symbols with opaque type.

PTX ISA Notes

Introduced in PTX ISA version 5.0.

Target ISA Notes

`.common` directive requires `sm_20` or higher.

Examples

```
.common .global .u32 gbl;
```

11.7. Cluster Dimension Directives

The following directives specify information about clusters:

- ▶ `.reqntapercluster`
- ▶ `.explicitcluster`
- ▶ `.maxclusterrank`

The `.reqntapercluster` directive specifies the number of CTAs in the cluster. The `.explicitcluster` directive specifies that the kernel should be launched with explicit cluster details. The `.maxclusterrank` directive specifies the maximum number of CTAs in the cluster.

The cluster dimension directives can be applied only on kernel functions.

11.7.1. Cluster Dimension Directives:

`.reqntapercluster`

`.reqntapercluster`

Declare the number of CTAs in the cluster.

Syntax

```
.reqntapercluster nx
.reqntapercluster nx, ny
.reqntapercluster nx, ny, nz
```

Description

Set the number of thread blocks (CTAs) in the cluster by specifying the extent of each dimension of the 1D, 2D, or 3D cluster. The total number of CTAs is the product of the number of CTAs in each dimension. For kernels with `.reqntapercluster` directive specified, runtime will use the specified values for configuring the launch if the same are not specified at launch time.

Semantics

If cluster dimension is explicitly specified at launch time, it should be equal to the values specified in this directive. Specifying a different cluster dimension at launch will result in a runtime error or kernel launch failure.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
.entry foo .reqntapercluster 2      { . . . }
.entry bar .reqntapercluster 2, 2, 1 { . . . }
.entry ker .reqntapercluster 3, 2   { . . . }
```

11.7.2. Cluster Dimension Directives: `.explicitcluster`

`.explicitcluster`

Declare that Kernel must be launched with cluster dimensions explicitly specified.

Syntax

```
.explicitcluster
```

Description

Declares that this Kernel should be launched with cluster dimension explicitly specified.

Semantics

Kernels with `.explicitcluster` directive must be launched with cluster dimension explicitly specified (either at launch time or via `.reqnctapercluster`), otherwise program will fail with runtime error or kernel launch failure.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.entry foo .explicitcluster      { . . . }
```

11.7.3. Cluster Dimension Directives: `.maxclusterrank`

`.maxclusterrank`

Declare the maximum number of CTAs that can be part of the cluster.

Syntax

```
.maxclusterrank n
```


Description

Declare the maximum number of thread blocks (CTAs) allowed to be part of the cluster.

Semantics

Product of the number of CTAs in each cluster dimension specified in any invocation of the kernel is required to be less or equal to that specified in this directive. Otherwise invocation will result in a runtime error or kernel launch failure.

The `.maxclusterrank` directive cannot be used in conjunction with the `.reqnctapercluster` directive.

PTX ISA Notes

Introduced in PTX ISA version 7.8.

Target ISA Notes

Requires sm_90 or higher.

Examples

```
.entry foo ..maxclusterrank 8      { . . . }
```

11.8. Miscellaneous Directives

PTX provides the following miscellaneous directives:

- `.blocksareclusters`

11.8.1. Miscellaneous Directives: `.blocksareclusters`

`.blocksareclusters`

Specify that CUDA thread blocks are mapped to clusters.

Syntax

```
.blocksareclusters
```

Description

Default behavior of CUDA API is to specify the grid launch configuration by specifying the number of thread blocks and the number of threads per block.

When `.blocksareclusters` directive is specified, it implies that the grid launch configuration for the corresponding `.entry` function is specifying the number of clusters, i.e. the launch configuration is specifying number of clusters instead of the number of thread blocks. In this case, the number of thread blocks per cluster is specified by `.reqnctapercluster` directive and the thread block size is specified with the `.reqntid` directive.

`.blocksareclusters` directive is only allowed for `.entry` functions and also needs `.reqntid` and `.reqnctapercluster` directives to be specified.

Refer to *CUDA Programming Guide* for more details.

PTX ISA Notes

Introduced in PTX ISA version 9.0.

Target ISA Notes

Requires `sm_90` or higher.

Examples

```
.entry foo .reqntid 32, 32, 1 .reqnctapercluster 32, 32, 1 .blocksareclusters { ... }
```

Chapter 12. Descriptions of .pragma Strings

This section describes the .pragma strings defined by ptxas.

12.1. Pragma Strings: "nounroll"

"nounroll"

Disable loop unrolling in optimizing the backend compiler.

Syntax

```
.pragma "nounroll";
```

Description

The "nounroll" pragma is a directive to disable loop unrolling in the optimizing backend compiler.

The "nounroll" pragma is allowed at module, entry-function, and statement levels, with the following meanings:

module scope

disables unrolling for all loops in module, including loops preceding the .pragma.

entry-function scope

disables unrolling for all loops in the entry function body.

statement-level pragma

disables unrolling of the loop for which the current block is the loop header.

Note that in order to have the desired effect at statement level, the "nounroll" directive must appear before any instruction statements in the loop header basic block for the desired loop. The loop header block is defined as the block that dominates all blocks in the loop body and is the target of the loop backedge. Statement-level "nounroll" directives appearing outside of loop header blocks are silently ignored.

PTX ISA Notes

Introduced in PTX ISA version 2.0.

Target ISA Notes

Requires sm_20 or higher. Ignored for sm_1x targets.

Examples

```
.entry foo (...)
.pragma "nounroll"; // do not unroll any loop in this function
{
    ...
}

.func bar (...)
{
    ...
L1_head:
    .pragma "nounroll"; // do not unroll this loop
    ...
    @p    bra L1_end;
L1_body:
    ...
L1_continue:
    bra L1_head;
L1_end:
    ...
}
```

12.2. Pragma Strings: "used_bytes_mask"

"used_bytes_mask"

Mask for indicating used bytes in data of ld operation.

Syntax

```
.pragma "used_bytes_mask mask";
```

Description

The "used_bytes_mask" pragma is a directive that specifies used bytes in a load operation based on the mask provided.

"used_bytes_mask" pragma needs to be specified prior to a load instruction for which information about bytes used from the load operation is needed. Pragma is ignored if instruction following it is not a load instruction.

For a load instruction without this pragma, all bytes from the load operation are assumed to be used.

Operand mask is a 32-bit integer with set bits indicating the used bytes in data of load operation.

Semantics

Each bit in mask operand corresponds to a byte data where each set bit represents the used byte.

Most-significant bit corresponds to most-significant byte of data.

```
// For 4 bytes load with only lower 3 bytes used
.pragma "used_bytes_mask 0x7";
ld.global.u32 %r0, [gbl]; // Higher 1 byte from %r0 is unused

// For vector load of 16 bytes with lower 12 bytes used
.pragma "used_bytes_mask 0xffff";
ld.global.v4.u32 {%r0, %r1, %r2, %r3}, [gbl]; // %r3 unused
```

PTX ISA Notes

Introduced in PTX ISA version 8.3.

Target ISA Notes

Requires sm_50 or higher.

Examples

```
.pragma "used_bytes_mask 0xffff";
ld.global.v4.u32 {%r0, %r1, %r2, %r3}, [gbl]; // Only lower 12 bytes used
```

12.3. Pragma Strings: "enable_smem_spilling"

"enable_smem_spilling"

Enable shared memory spilling for CUDA kernels.

Syntax

```
.pragma "enable_smem_spilling";
```

Description

The "enable_smem_spilling" pragma is a directive that enables register spilling into shared memory. During the spilling process, registers are first spilled into shared memory, and once the allocated shared memory is full, any additional spills are redirected to local memory. This can enhance performance by reducing memory access latency since shared memory accesses are faster than local memory.

The "enable_smem_spilling" pragma is only allowed within the function scope. When applied, it enables shared memory spilling for the specified function.

The usage of pragma is valid only in certain scenarios and specific compilation modes. The usage of pragma is disallowed under following cases and may result in an error:

- ▶ Per-function compilation mode: e.g., Separate Compilation, Device-debug, Whole program with recursive function calls, Extensible-whole-program
- ▶ Kernels utilizing dynamically allocated shared memory
- ▶ Kernels using `setmaxnreg` instruction

Note: If launch bounds are not explicitly specified, the compiler assumes the maximum possible number of threads per CTA to estimate shared memory allocated per CTA and corresponding spill size. However, if the kernel is launched with fewer threads per CTA than estimated, the shared memory allocated per CTA may exceed the compiler estimated size, thereby potentially limiting the number of CTAs that can be launched on an SM. Due to this, using the pragma without launch bounds may lead to performance regressions. Hence it is recommended to use this pragma only when launch bounds are explicitly specified.

PTX ISA Notes

Introduced in PTX ISA version 9.0.

Target ISA Notes

Requires sm_75 or higher.

Examples

```
.entry foo (...)
{
    ...
    .pragma "enable_smem_spilling";    // Enable shared memory spilling for this
    ↪function
    ...
}
```

12.4. Pragma Strings: "frequency"

"frequency"

Specify frequency for basic block execution.

Syntax

```
.pragma "frequency n";
```

Description

The "frequency" pragma is a directive that specifies the number of times a basic block is executed by an executing thread. The optimizing compiler backend treats this pragma as a hint which will be used for optimizations.

Operand *n* is a 64-bit non-negative integer constant that specifies the execution frequency.

Note that in order to have the desired effect of this pragma, it should be specified at the start of the basic block. Basic block is defined as a straight-line sequence of instructions with only one entry point and one exit point.

PTX ISA Notes

Introduced in PTX ISA version 9.0.

Target ISA Notes

Supported on all target architectures.

Examples

```
.entry foo (...)  
{  
    .pragma "frequency 32";  
    ...  
}
```

Chapter 13. Release Notes

This section describes the history of change in the PTX ISA and implementation. The first section describes ISA and implementation changes in the current release of PTX ISA version 9.1, and the remaining sections provide a record of changes in previous releases of PTX ISA versions back to PTX ISA version 2.0.

Table 65 shows the PTX release history.

Table 65: PTX Release History

PTX ISA Version	CUDA Release	Supported Targets
PTX ISA 1.0	CUDA 1.0	sm_{10, 11}
PTX ISA 1.1	CUDA 1.1	sm_{10, 11}
PTX ISA 1.2	CUDA 2.0	sm_{10, 11, 12, 13}
PTX ISA 1.3	CUDA 2.1	sm_{10, 11, 12, 13}
PTX ISA 1.4	CUDA 2.2	sm_{10, 11, 12, 13}
PTX ISA 1.5	driver r190	sm_{10, 11, 12, 13}
PTX ISA 2.0	CUDA 3.0, driver r195	sm_{10, 11, 12, 13}, sm_20
PTX ISA 2.1	CUDA 3.1, driver r256	sm_{10, 11, 12, 13}, sm_20
PTX ISA 2.2	CUDA 3.2, driver r260	sm_{10, 11, 12, 13}, sm_20
PTX ISA 2.3	CUDA 4.0, driver r270	sm_{10, 11, 12, 13}, sm_20
PTX ISA 3.0	CUDA 4.1, driver r285	sm_{10, 11, 12, 13}, sm_20
	CUDA 4.2, driver r295	sm_{10, 11, 12, 13}, sm_20, sm_30
PTX ISA 3.1	CUDA 5.0, driver r302	sm_{10, 11, 12, 13}, sm_20, sm_{30, 35}
PTX ISA 3.2	CUDA 5.5, driver r319	sm_{10, 11, 12, 13}, sm_20, sm_{30, 35}
PTX ISA 4.0	CUDA 6.0, driver r331	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35}, sm_50
PTX ISA 4.1	CUDA 6.5, driver r340	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52}
PTX ISA 4.2	CUDA 7.0, driver r346	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}

continues on next page

Table 65 – continued from previous page

PTX ISA Version	CUDA Release	Supported Targets
PTX ISA 4.3	CUDA 7.5, driver r352	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}
PTX ISA 5.0	CUDA 8.0, driver r361	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}
PTX ISA 6.0	CUDA 9.0, driver r384	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_70
PTX ISA 6.1	CUDA 9.1, driver r387	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_70, sm_72
PTX ISA 6.2	CUDA 9.2, driver r396	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_70, sm_72
PTX ISA 6.3	CUDA 10.0, driver r400	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_70, sm_72, sm_75
PTX ISA 6.4	CUDA 10.1, driver r418	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_70, sm_72, sm_75
PTX ISA 6.5	CUDA 10.2, driver r440	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_70, sm_72, sm_75
PTX ISA 7.0	CUDA 11.0, driver r445	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_80
PTX ISA 7.1	CUDA 11.1, driver r455	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86}
PTX ISA 7.2	CUDA 11.2, driver r460	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86}
PTX ISA 7.3	CUDA 11.3, driver r465	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86}
PTX ISA 7.4	CUDA 11.4, driver r470	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87}
PTX ISA 7.5	CUDA 11.5, driver r495	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87}
PTX ISA 7.6	CUDA 11.6, driver r510	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87}

continues on next page

Table 65 – continued from previous page

PTX ISA Version	CUDA Release	Supported Targets
PTX ISA 7.7	CUDA 11.7, driver r515	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87}
PTX ISA 7.8	CUDA 11.8, driver r520	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_90
PTX ISA 8.0	CUDA 12.0, driver r525	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}
PTX ISA 8.1	CUDA 12.1, driver r530	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}
PTX ISA 8.2	CUDA 12.2, driver r535	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}
PTX ISA 8.3	CUDA 12.3, driver r545	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}
PTX ISA 8.4	CUDA 12.4, driver r550	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}
PTX ISA 8.5	CUDA 12.5, driver r555	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}
	CUDA 12.6, driver r560	
PTX ISA 8.6	CUDA 12.7, driver r565	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}, sm_{100, 100a, 101, 101a}
PTX ISA 8.7	CUDA 12.8, driver r570	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}, sm_{100, 100, 101, 101a}, sm_{120, 120a}
PTX ISA 8.8	CUDA 12.9, driver r575	sm_{10, 11, 12, 13}, sm_20, sm_{30, 32, 35, 37}, sm_{50, 52, 53}, sm_{60, 61, 62}, sm_{70, 72, 75}, sm_{80, 86, 87, 89}, sm_{90, 90a}, sm_{100, 100f, 100a, 101, 101f, 101a, 103, 103f, 103a}, sm_{120, 120f, 120a, 121, 121f, 121a}

continues on next page

Table 65 – continued from previous page

PTX ISA Version	CUDA Release	Supported Targets
PTX ISA 9.0	CUDA 13.0, driver r580	sm_{10,11,12,13}, sm_20, sm_{30,32,35,37}, sm_{50,52,53}, sm_{60,61,62}, sm_{70,72,75}, sm_{80,86,87,88,89}, sm_{90,90a}, sm_{100, 100f,100a,103,103f,103a}, sm_{110,110f, 110a}, sm_{120,120f,120a,121,121f,121a}
PTX ISA 9.1	CUDA 13.1, driver r590	sm_{10,11,12,13}, sm_20, sm_{30,32,35,37}, sm_{50,52,53}, sm_{60,61,62}, sm_{70,72,75}, sm_{80,86,87,88,89}, sm_{90,90a}, sm_{100, 100f,100a,103,103f,103a}, sm_{110,110f, 110a}, sm_{120,120f,120a,121,121f,121a}

Table 66: Arch-specific/ Family-specific PTX Features Release History

Instruction	Variant	PTX ISA Version	Supported Targets
tensormap.replace	Base variant	8.3	sm_90a
		8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
	tensormap.replace.swizzle_atomicity	8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
	.elemtype for .field3 with values 13, 14, 15 for new_val	8.7	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
	.swizzle_mode for .field3 with value 4 for new_val	8.8	sm_103a
wgmma.mma_async, wgmma.mma_async.sp, wgmma.fence, wgmma. commit_group, wgmma. wait_group	Base variant	8.0	sm_90a

continues on next page

Table 66 – continued from previous page

Instruction	Variant	PTX ISA Version	Supported Targets
setmaxnreg	Base variant	8.0	sm_90a
		8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
multimem.ld_reduce, multimem.st,multimem. red	Types .e5m2, .e4m3, .e5m2x2, .e4m3x2, .e4m3x4, .e5m2x4	8.6	sm_100a, sm_120a, sm_121a
		8.8	sm_100f
		9.0	sm_110f
	.acc::f16 qualifier	8.6	sm_100a, sm_120a, sm_121a
		8.8	sm_100f
		9.0	sm_110f
cvt	cvt for .f32 to .e2m1x2/ .e2m3x2/.e3m2x2/.ue8m0x2 AND .e2m1x2/.e2m3x2/.e3m2x2 to .f16x2 AND .ue8m0x2 to .bf16x2 AND .bf16x2 to .ue8m0x2	8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
	cvt with .rs rounding mode	8.7	sm_100a
		8.8	sm_103a
	cvt with .s2f6x2 type	9.1	sm_100a, sm_103a, sm_110a, sm_120a, sm_121a
	cvt for .f16x2 to .e2m1x2/ .e2m3x2/.e3m2x2	9.1	sm_100f, sm_110f, sm_120f
	cvt for .bf16x2 to .e2m1x2/ .e2m3x2/.e3m2x2/.e4m3x2/ .e5m2x2	9.1	sm_100f, sm_110f, sm_120f
cp.async.bulk.tensor	.tile::gather4 and .im2col::w with .shared::cluster as destination state space	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f

continues on next page

Table 66 – continued from previous page

Instruction	Variant	PTX ISA Version	Supported Targets
	.tile::scatter4 and .im2col::w::128	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
	.cta_group	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
cp.async.bulk. prefetch.tensor	.tile::gather4, .im2col::w, . im2col::w::128	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
redux.sync	Type .f32 and .abs, .NaN qualifiers	8.6	sm_100a
		8.8	sm_100f
clusterlaunchcontrol. try_cancel	.multicast::cluster::all	8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
ldmatrix	Shapes .m16n16, .m8n16 AND Type .b8 AND Qualifiers .src_fmt, .dst_fmt	8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
stmatrix	Shapes .m16n8 AND Type .b8 AND	8.6	sm_100a, sm_120a
		8.8	sm_100f, sm_120f
		9.0	sm_110f
tcgen05.alloc, tcgen05. dealloc, tcgen05. relinquish_alloc_permit	Base variant	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
tcgen05.ld, tcgen05.st, tcgen05.wait, tcgen05. cp, tcgen05.fence, tcgen05.commit	Base variant	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
tcgen05.ld.red	Base variant	8.8	sm_103f

continues on next page

Table 66 – continued from previous page

Instruction	Variant	PTX ISA Version	Supported Targets
		9.0	sm_110f
tcgen05.shift	Base variant	8.6	sm_100a
		8.8	sm_103a
		9.0	sm_110a
tcgen05.mma	Base variant	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
	Kind .kind::i8	8.6	sm_100a
		9.0	sm_110a
	Argument scale-input-d	8.6	sm_100a
		8.8	sm_100f
	Qualifiers .scale_vec::1X, .scale_vec::2X, .scale_vec::4X	8.6	sm_100a
	Qualifiers .block16, .block32	8.8	sm_100f, sm_110f
	K shape value 96	8.8	sm_103a
tcgen05.mma.sp	Base variant	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f
	Kind .kind::i8	8.6	sm_100a
		9.0	sm_110a
	Kind .kind::mxf4nvf4 and .kind::mxf4	8.6	sm_100a
		8.8	sm_103a
		9.0	sm_110a
	Argument scale-input-d	8.6	sm_100a
		8.8	sm_100f
	Qualifiers .scale_vec::1X, .scale_vec::2X, .scale_vec::4X	8.6	sm_100a
	Qualifiers .block16, .block32	8.8	sm_100f, sm_110f
tcgen05.mma.ws, tcgen05.mma.ws.sp	Base variant	8.6	sm_100a
		8.8	sm_100f
		9.0	sm_110f

continues on next page

Table 66 – continued from previous page

Instruction	Variant	PTX ISA Version	Supported Targets
	Kind <code>.kind::i8</code>	8.6	sm_100a
		9.0	sm_110a
mma	Types <code>.e3m2</code> , <code>.e2m3</code> , <code>.e2m1</code> AND Qualifiers <code>.kind</code> , <code>.block_scale</code> , <code>.scale_vec_size</code>	8.7	sm_120a
		8.8	sm_120f
mma.sp	Types <code>.e3m2</code> , <code>.e2m3</code> , <code>.e2m1</code> AND Qualifiers <code>.kind</code> , <code>.block_scale</code> , <code>.scale_vec_size</code>	8.7	sm_120a
		8.8	sm_120f
	Kind <code>.kind::mxf4nvf4</code> and <code>.kind::mxf4</code>	8.7	sm_120a, sm_121a

13.1. Changes in PTX ISA Version 9.1

New Features

PTX ISA version 9.1 introduces the following new features:

- ▶ Adds support for `.volatile` qualifier with `.local` state space for `ld` and `st` instructions.
- ▶ Adds support for `.f16x2` and `.bf16x2` source types for `cvt` instruction with destination types `.e2m1x2`, `.e2m3x2`, `.e3m2x2`, `.e4m3x2`, `.e5m2x2`.
- ▶ Adds support for `.scale_vec::4X` with `.ue8m0` as `.stype` with `.kind::mxf4nvf4` for `mma/mma.sp` instructions.
- ▶ Adds support for `.s2f6x2` instruction type for `cvt` instruction.
- ▶ Adds support for `multimem.cp.async.bulk` and `multimem.cp.reduce.async.bulk` instructions.

Semantic Changes and Clarifications

None.

13.2. Changes in PTX ISA Version 9.0

New Features

PTX ISA version 9.0 introduces the following new features:

- ▶ Adds support for `sm_88` target architecture.
- ▶ Adds support for `sm_110` target architecture.

- ▶ Adds support for target `sm_110f` that supports family-specific features.
- ▶ Adds support for target `sm_110a` that supports architecture-specific features.
- ▶ Adds support for pragma `enable_smem_spilling` that is used to enable shared memory spilling for a function.
- ▶ Adds support for pragma `frequency` that is used to specify the execution frequency of a basic block.
- ▶ Adds support for directive `.blocksareclusters` that is used to specify that CUDA thread blocks are mapped to clusters.
- ▶ Extends size operand of `st.bulk` instruction to support 32-bit length.
- ▶ Adds support for performance-tuning directives `.abi_preserve` and `.abi_preserve_control` that are used to specify the number of data and control registers that should be preserved by the callers of a function.

Notes

- ▶ Targets `sm_{101, 101f, 101a}` are renamed to targets `sm_{110, 110f, 110a}` from PTX ISA version 9.0.

Semantic Changes and Clarifications

- ▶ All `tcgen05` instructions(`tcgen05.alloc`, `tcgen05.dealloc`, `tcgen05.relinquish_alloc_permit`, `tcgen05.cp`, `tcgen05.shift`, `tcgen05.mma`, `tcgen05.mma.sp`, `tcgen05.mma.ws`, `tcgen05.mma.ws.sp`, `tcgen05.commit`) within a kernel must specify the same value for the `.cta_group` qualifier.

13.3. Changes in PTX ISA Version 8.8

New Features

PTX ISA version 8.8 introduces the following new features:

- ▶ Adds support for `sm_103` target architecture.
- ▶ Adds support for target `sm_103a` that supports architecture-specific features.
- ▶ Adds support for `sm_121` target architecture.
- ▶ Adds support for target `sm_121a` that supports architecture-specific features.
- ▶ Introduces family-specific target architectures that are represented with “f” suffix. PTX for family-specific targets is compatible with all subsequent targets in same family. Adds support for `sm_100f`, `sm_101f`, `sm_103f`, `sm_120f`, `sm_121f`.
- ▶ Extends `min` and `max` instructions to support three input arguments.
- ▶ Extends `tcgen05.mma` instruction to add support for new `scale_vectorsize` qualifiers `.block16` and `.block32` and K dimension 96.
- ▶ Extends `.field3` of `tensormap.replace` instruction to support 96B swizzle mode.

- ▶ Adds support for `tcgen05.ld.red` instruction.
- ▶ Extends `ld`, `ld.global.nc` and `st` instructions to support 256b load/store operations.
- ▶ [Table 67](#) shows the list of features that are supported on family-specific targets:

Table 67: List of features promoted to family-specific architecture

Feature	Supported targets
.m16n8, .m16n16, .m8n16 shapes and .b8 type for ldma-trix/stmatrix	sm_100f, sm_101f, sm_120f
Shapes for tcgen05 .16x64b .16x128b, .16x256b, .16x32bx2, .32x32b, .4x256b, .32x128b, .64x128b, .128x256b, .128x128b, .31x256b	sm_100f, sm_101f
setmaxnreg	sm_100f, sm_101f, sm_120f
.cta_group modifier	sm_100f, sm_101f
cvt with .e2m1x2, .e3m2x2, .e2m3x2, .ue8m0x2	sm_100f, sm_101f, sm_120f
multimem with .acc::f16 and .e5m2, .e5m2x2, .e5m2x4, .e4m3, .e4m3x2, .e4m3x4 types	sm_100f, sm_101f
tensormap.replace	sm_100f, sm_101f, sm_120f
tcgen05.ld.red	sm_101f, sm_103f
tcgen05.ld/st/fence/ wait/commit/cp/ alloc/dealloc/ relin-quish_alloc_permit	sm_100f, sm_101f
tcgen05.mma{.ws}{.sp} (except kind::mxf4/ kind::mxf4nvf4 for .sp)	sm_100f, sm_101f
tcgen05 .kind::mxf4nvf4, .kind::mxf4, .kind::mxf8f6f4, .kind::f16, .kind::tf32, .kind::f8f6f4	sm_100f, sm_101f
.ashift, .collector_usage modifiers for tcgen05	sm_100f, sm_101f
Modifiers .b8x16, .b6x16_p32, .b4x16_p64	sm_100f, sm_101f, sm_120f
.block_scale modifier	sm_100f, sm_101f, sm_120f
mma{.sp} with .e3m2, .e2m3, .e2m1 types and .kind, .block_scale, .scale_vec_size modifiers (except .sp with mxf4/ mxf4nvf4)	sm_120f
.scale_vec::1X/2X/4X modifiers	sm_120f
.block16/ .block32 modifiers (alias to scale_vec)	sm_100f, sm_101f
.warpx2::02_13, .warpx2::01_23, .warpx4, .pack::16b, .unpack::16b modifiers for tcgen05	sm_100f, sm_101f
clusterlaunchcontrol.try_cancel multicast::cluster::all	sm_100f, sm_101f, sm_120f
.tile::scatter4, .tile::gather4, .im2col::w, .im2col::w::128	sm_100f, sm_101f
redux.f32	sm_100f
scale-input-d for tcgen05	sm_100f

Semantic Changes and Clarifications

- Clarified the behavior of float-to-integer conversions for NaN input.

13.4. Changes in PTX ISA Version 8.7

New Features

PTX ISA version 8.7 introduces the following new features:

- Adds support for `sm_120` target architecture.
- Adds support for target `sm_120a` that supports architecture-specific features.
- Extends `tcgen05.mma` instruction to add support for `.kind::mxf4nvf4` and `.scale_vec::4X` qualifiers.
- Extends `mma` instructions to support `.f16` type accumulator and shape `.m16n8k16` with FP8 types `.e4m3` and `.e5m2`.
- Extends `cvt` instruction to add support for `.rs` rounding mode and destination types `.e2m1x4`, `.e4m3x4`, `.e5m2x4`, `.e3m2x4`, `.e2m3x4`.
- Extends support for `st.async` and `red.async` instructions to add support for `.mmio`, `.release`, `.global` and `.scope` qualifiers.
- Extends `tensormap.replace` instruction to add support for values 13 to 15 for `.elemtype` qualifier.
- Extends `mma` and `mma.sp::ordered_metadata` instructions to add support for types `.e3m2/.e2m3/.e2m1` and qualifiers `.kind`, `.block_scale`, `.scale_vec_size`.

Semantic Changes and Clarifications

- Clarified that in `.tile::gather4`, `.tile::scatter4` modes, tensor coordinates need to be specified as `{col_idx, row_idx0, row_idx1, row_idx2, row_idx3}` i.e. `{x, y0, y1, y2, y3}` instead of `{x0, x1, x2, x3, y}`.
- Updated *Instruction descriptor* of `tcgen05.mma` instruction to clarify the bits that are reserved for future use.

13.5. Changes in PTX ISA Version 8.6

New Features

PTX ISA version 8.6 introduces the following new features:

- Adds support for `sm_100` target architecture.
- Adds support for target `sm_100a` that supports architecture-specific features.
- Adds support for `sm_101` target architecture.

- ▶ Adds support for target `sm_101a` that supports architecture-specific features.
- ▶ Extends `cp.async.bulk` and `cp.async.bulk.tensor` instructions to add `.shared::cta` as destination state space.
- ▶ Extends fence instruction to add support for `.acquire` and `.release` qualifiers.
- ▶ Extends fence and fence.proxy instructions to add support for `.sync_restrict` qualifier.
- ▶ Extends `ldmatrix` instruction to support `.m16n16`, `.m8n16` shapes and `.b8` type.
- ▶ Extends `ldmatrix` instruction to support `.src_fmt`, `.dst_fmt` qualifiers.
- ▶ Extends `stmatrix` instruction to support `.m16n8` shape and `.b8` type.
- ▶ Adds support for `clusterlaunchcontrol` instruction.
- ▶ Extends `add`, `sub` and `fma` instructions to support mixed precision floating point operations with `.f32` as destination operand type and `.f16/.bf16` as source operand types.
- ▶ Extends `add`, `sub`, `mul` and `fma` instructions to support `.f32x2` type.
- ▶ Extends `cvt` instruction with `.tf32` type to support `.satfinite` qualifier for `.rn/.rz` rounding modes.
- ▶ Extends `cp.async.bulk` instruction to support `.cp_mask` qualifier and `byteMask` operand.
- ▶ Extends `multimem.ld_reduce` and `multimem.st` instructions to support `.e5m2`, `.e5m2x2`, `.e5m2x4`, `.e4m3`, `.e4m3x2` and `.e4m3x4` types.
- ▶ Extends `cvt` instruction to support conversions to/from `.e2m1x2`, `.e3m2x2`, `.e2m3x2` and `.ue8m0x2` types.
- ▶ Extends `cp.async.bulk.tensor` and `cp.async.bulk.prefetch.tensor` instructions to support new load_mode qualifiers `.tile::scatter4` and `.tile::gather4`.
- ▶ Extends `tensormap.replace` instruction to add support for new qualifier `.swizzle_atomicity` for supporting new swizzle modes.
- ▶ Extends `mbarrier.arrive`, `mbarrier.arrive_drop`, `.mbarrier.test_wait` and `.mbarrier.try_wait` instructions to support `.relaxed` qualifier.
- ▶ Extends `cp.async.bulk.tensor` and `cp.async.bulk.prefetch.tensor` instructions to support new load_mode qualifiers `.im2col::w` and `.im2col::w::128`.
- ▶ Extends `cp.async.bulk.tensor` instruction to support new qualifier `.cta_group`.
- ▶ Add support for `st.bulk` instruction.
- ▶ Adds support for tcgen05 features and related instructions: `tcgen05.alloc`, `tcgen05.dealloc`, `tcgen05.relinquish_alloc_permit`, `tcgen05.ld`, `tcgen05.st`, `tcgen05.wait`, `tcgen05.cp`, `tcgen05.shift`, `tcgen05.mma`, `tcgen05.mma.sp`, `tcgen05.mma.ws`, `tcgen05.mma.ws.sp`, `tcgen05.fence` and `tcgen05.commit`.
- ▶ Extends `redux.sync` instruction to add support for `.f32` type with qualifiers `.abs` and `.NaN`.

Semantic Changes and Clarifications

None.

13.6. Changes in PTX ISA Version 8.5

New Features

PTX ISA version 8.5 introduces the following new features:

- ▶ Adds support for `mma.sp::ordered_metadata` instruction.

Semantic Changes and Clarifications

- ▶ Values `0b0000`, `0b0101`, `0b1010`, `0b1111` for sparsity metadata (operand `e`) of instruction `mma.sp` are invalid and their usage results in undefined behavior.

13.7. Changes in PTX ISA Version 8.4

New Features

PTX ISA version 8.4 introduces the following new features:

- ▶ Extends `ld`, `st` and `atom` instructions with `.b128` type to support `.sys` scope.
- ▶ Extends integer `wgmma.mma_async` instruction to support `.u8.s8` and `.s8.u8` as `.atype` and `.btype` respectively.
- ▶ Extends `mma`, `mma.sp` instructions to support FP8 types `.e4m3` and `.e5m2`.

Semantic Changes and Clarifications

None.

13.8. Changes in PTX ISA Version 8.3

New Features

PTX ISA version 8.3 introduces the following new features:

- ▶ Adds support for `pragma used_bytes_mask` that is used to specify mask for used bytes for a load operation.
- ▶ Extends `isspacep`, `cvta.to`, `ld` and `st` instructions to accept `::entry` and `::func` sub-qualifiers with `.param` state space qualifier.

- ▶ Adds support for `.b128` type on instructions `ld`, `ld.global.nc`, `ldu`, `st`, `mov` and `atom`.
- ▶ Add support for instructions `tensormap.replace`, `tensormap.cp_fenceproxy` and support for qualifier `.to_proxykind::from_proxykind` on instruction `fence.proxy` to support modifying `tensor-map`.

Semantic Changes and Clarifications

None.

13.9. Changes in PTX ISA Version 8.2

New Features

PTX ISA version 8.2 introduces the following new features:

- ▶ Adds support for `.mmio` qualifier on `ld` and `st` instructions.
- ▶ Extends `lop3` instruction to allow predicate destination.
- ▶ Extends `multimem.ld_reduce` instruction to support `.acc::f32` qualifier to allow `.f32` precision of the intermediate accumulation.
- ▶ Extends the asynchronous warpgroup-level matrix multiply-and-accumulate operation `wgmma.mma_async` to support `.sp` modifier that allows matrix multiply-accumulate operation when input matrix A is sparse.

Semantic Changes and Clarifications

The `.multicast::cluster` qualifier on `cp.async.bulk` and `cp.async.bulk.tensor` instructions is optimized for target architecture `sm_90a` and may have substantially reduced performance on other targets and hence `.multicast::cluster` is advised to be used with `sm_90a`.

13.10. Changes in PTX ISA Version 8.1

New Features

PTX ISA version 8.1 introduces the following new features:

- ▶ Adds support for `st.async` and `red.async` instructions for asynchronous store and asynchronous reduction operations respectively on shared memory.
- ▶ Adds support for `.oob` modifier on half-precision `fma` instruction.
- ▶ Adds support for `.satfinite` saturation modifier on `cvt` instruction for `.f16`, `.bf16` and `.tf32` formats.
- ▶ Extends support for `cvt` with `.e4m3/.e5m2` to `sm_89`.
- ▶ Extends `atom` and `red` instructions to support vector types.

- ▶ Adds support for special register `%aggr_smem_size`.
- ▶ Extends `sured` instruction with 64-bit `min/max` operations.
- ▶ Adds support for increased kernel parameter size of 32764 bytes.
- ▶ Adds support for `multimem` addresses in memory consistency model.
- ▶ Adds support for `multimem.ld_reduce`, `multimem.st` and `multimem.red` instructions to perform memory operations on `multimem` addresses.

Semantic Changes and Clarifications

None.

13.11. Changes in PTX ISA Version 8.0

New Features

PTX ISA version 8.0 introduces the following new features:

- ▶ Adds support for target `sm_90a` that supports architecture-specific features.
- ▶ Adds support for asynchronous warpgroup-level matrix multiply-and-accumulate operation `wgmma`.
- ▶ Extends the asynchronous copy operations with bulk operations that operate on large data, including tensor data.
- ▶ Introduces packed integer types `.u16x2` and `.s16x2`.
- ▶ Extends integer arithmetic instruction `add` to allow packed integer types `.u16x2` and `.s16x2`.
- ▶ Extends integer arithmetic instructions `min` and `max` to allow packed integer types `.u16x2` and `.s16x2`, as well as saturation modifier `.relu` on `.s16x2` and `.s32` types.
- ▶ Adds support for special register `%current_graph_exec` that identifies the currently executing CUDA device graph.
- ▶ Adds support for `elect.sync` instruction.
- ▶ Adds support for `.unified` attribute on functions and variables.
- ▶ Adds support for `setmaxnreg` instruction.
- ▶ Adds support for `.sem` qualifier on `barrier.cluster` instruction.
- ▶ Extends the `fence` instruction to allow opcode-specific synchronization using `op_restrict` qualifier.
- ▶ Adds support for `.cluster` scope on `mbarrier.arrive`, `mbarrier.arrive_drop`, `mbarrier.test_wait` and `mbarrier.try_wait` operations.
- ▶ Adds support for transaction count operations on `mbarrier` objects, specified with `.expect_tx` and `.complete_tx` qualifiers.

Semantic Changes and Clarifications

None.

13.12. Changes in PTX ISA Version 7.8

New Features

PTX ISA version 7.8 introduces the following new features:

- ▶ Adds support for `sm_89` target architecture.
- ▶ Adds support for `sm_90` target architecture.
- ▶ Extends `bar` and `barrier` instructions to accept optional scope qualifier `.cta`.
- ▶ Extends `.shared` state space qualifier with optional sub-qualifier `::cta`.
- ▶ Adds support for `movmatrix` instruction which transposes a matrix in registers across a warp.
- ▶ Adds support for `stmatrix` instruction which stores one or more matrices to shared memory.
- ▶ Extends the `.f64` floating point type `mma` operation with shapes `.m16n8k4`, `.m16n8k8`, and `.m16n8k16`.
- ▶ Extends `add`, `sub`, `mul`, `set`, `setp`, `cvt`, `tanh`, `ex2`, `atom` and `red` instructions with `bf16` alternate floating point data format.
- ▶ Adds support for new alternate floating-point data formats `.e4m3` and `.e5m2`.
- ▶ Extends `cvt` instruction to convert `.e4m3` and `.e5m2` alternate floating point data formats.
- ▶ Adds support for `griddepcontrol` instruction as a communication mechanism to control the execution of dependent grids.
- ▶ Extends `mbarrier` instruction to allow a new phase completion check operation `try_wait`.
- ▶ Adds support for new thread scope `.cluster` which is a set of Cooperative Thread Arrays (CTAs).
- ▶ Extends `fence/membar`, `ld`, `st`, `atom`, and `red` instructions to accept `.cluster` scope.
- ▶ Adds support for extended visibility of shared state space to all threads within a cluster.
- ▶ Extends `.shared` state space qualifier with `::cluster` sub-qualifier for cluster-level visibility of shared memory.
- ▶ Extends `isspacep`, `cvta`, `ld`, `st`, `atom`, and `red` instructions to accept `::cluster` sub-qualifier with `.shared` state space qualifier.
- ▶ Adds support for `mapa` instruction to map a shared memory address to the corresponding address in a different CTA within the cluster.
- ▶ Adds support for `getctarank` instruction to query the rank of the CTA that contains a given address.
- ▶ Adds support for new barrier synchronization instruction `barrier.cluster`.
- ▶ Extends the memory consistency model to include the new cluster scope.
- ▶ Adds support for special registers related to cluster information: `%is_explicit_cluster`, `%clusterid`, `%nclusterid`, `%cluster_ctaid`, `%cluster_nctaid`, `%cluster_ctarank`, `%cluster_nctarank`.

- ▶ Adds support for cluster dimension directives `.reqnctapercluster`, `.explicitcluster`, and `.maxclusterrank`.

Semantic Changes and Clarifications

None.

13.13. Changes in PTX ISA Version 7.7

New Features

PTX ISA version 7.7 introduces the following new features:

- ▶ Extends `isspacep` and `cvta` instructions to include the `.param` state space for kernel function parameters.

Semantic Changes and Clarifications

None.

13.14. Changes in PTX ISA Version 7.6

New Features

PTX ISA version 7.6 introduces the following new features:

- ▶ Support for `szext` instruction which performs sign-extension or zero-extension on a specified value.
- ▶ Support for `bmsk` instruction which creates a bitmask of the specified width starting at the specified bit position.
- ▶ Support for special registers `%reserved_smem_offset_begin`, `%reserved_smem_offset_end`, `%reserved_smem_offset_cap`, `%reserved_smem_offset<2>`.

Semantic Changes and Clarifications

None.

13.15. Changes in PTX ISA Version 7.5

New Features

PTX ISA version 7.5 introduces the following new features:

- ▶ Debug information enhancements to support label difference and negative values in the `.section` debugging directive.
- ▶ Support for `ignore-src` operand on `cp.async` instruction.
- ▶ Extensions to the memory consistency model to introduce the following new concepts:
 - ▶ A *memory proxy* as an abstract label for different methods of memory access.
 - ▶ Virtual aliases as distinct memory addresses accessing the same physical memory location.
- ▶ Support for new `fence.proxy` and `membar.proxy` instructions to allow synchronization of memory accesses performed via virtual aliases.

Semantic Changes and Clarifications

None.

13.16. Changes in PTX ISA Version 7.4

New Features

PTX ISA version 7.4 introduces the following new features:

- ▶ Support for `sm_87` target architecture.
- ▶ Support for `.level::eviction_priority` qualifier which allows specifying cache eviction priority hints on `ld`, `ld.global.nc`, `st`, and `prefetch` instructions.
- ▶ Support for `.level::prefetch_size` qualifier which allows specifying data prefetch hints on `ld` and `cp.async` instructions.
- ▶ Support for `createpolicy` instruction which allows construction of different types of cache eviction policies.
- ▶ Support for `.level::cache_hint` qualifier which allows the use of cache eviction policies with `ld`, `ld.global.nc`, `st`, `atom`, `red` and `cp.async` instructions.
- ▶ Support for `applypriority` and `discard` operations on cached data.

Semantic Changes and Clarifications

None.

13.17. Changes in PTX ISA Version 7.3

New Features

PTX ISA version 7.3 introduces the following new features:

- ▶ Extends `mask()` operator used in initializers to also support integer constant expression.
- ▶ Adds support for stack manipulation instructions that allow manipulating stack using `stacksave` and `stackrestore` instructions and allocation of per-thread stack using `alloca` instruction.

Semantic Changes and Clarifications

The unimplemented version of `alloca` from the older PTX ISA specification has been replaced with new stack manipulation instructions in PTX ISA version 7.3.

13.18. Changes in PTX ISA Version 7.2

New Features

PTX ISA version 7.2 introduces the following new features:

- ▶ Enhances `.loc` directive to represent inline function information.
- ▶ Adds support to define labels inside the debug sections.
- ▶ Extends `min` and `max` instructions to support `.xorsign` and `.abs` modifiers.

Semantic Changes and Clarifications

None.

13.19. Changes in PTX ISA Version 7.1

New Features

PTX ISA version 7.1 introduces the following new features:

- ▶ Support for `sm_86` target architecture.
- ▶ Adds a new operator, `mask()`, to extract a specific byte from variable's address used in initializers.

- ▶ Extends `tex` and `tld4` instructions to return an optional predicate that indicates if data at specified coordinates is resident in memory.
- ▶ Extends single-bit `wmma` and `mma` instructions to support `.and` operation.
- ▶ Extends `mma` instruction to support `.sp` modifier that allows matrix multiply-accumulate operation when input matrix A is sparse.
- ▶ Extends `mbarrier.test_wait` instruction to test the completion of specific phase parity.

Semantic Changes and Clarifications

None.

13.20. Changes in PTX ISA Version 7.0

New Features

PTX ISA version 7.0 introduces the following new features:

- ▶ Support for `sm_80` target architecture.
- ▶ Adds support for asynchronous copy instructions that allow copying of data asynchronously from one state space to another.
- ▶ Adds support for `mbarrier` instructions that allow creation of *mbarrier objects* in memory and use of these objects to synchronize threads and asynchronous copy operations initiated by threads.
- ▶ Adds support for `redux.sync` instruction which allows reduction operation across threads in a warp.
- ▶ Adds support for new alternate floating-point data formats `.bf16` and `.tf32`.
- ▶ Extends `wmma` instruction to support `.f64` type with shape `.m8n8k4`.
- ▶ Extends `wmma` instruction to support `.bf16` data format.
- ▶ Extends `wmma` instruction to support `.tf32` data format with shape `.m16n16k8`.
- ▶ Extends `mma` instruction to support `.f64` type with shape `.m8n8k4`.
- ▶ Extends `mma` instruction to support `.bf16` and `.tf32` data formats with shape `.m16n8k8`.
- ▶ Extends `mma` instruction to support new shapes `.m8n8k128`, `.m16n8k4`, `.m16n8k16`, `.m16n8k32`, `.m16n8k64`, `.m16n8k128` and `.m16n8k256`.
- ▶ Extends `abs` and `neg` instructions to support `.bf16` and `.bf16x2` data formats.
- ▶ Extends `min` and `max` instructions to support `.NaN` modifier and `.f16`, `.f16x2`, `.bf16` and `.bf16x2` data formats.
- ▶ Extends `fma` instruction to support `.relu` saturation mode and `.bf16` and `.bf16x2` data formats.
- ▶ Extends `cvt` instruction to support `.relu` saturation mode and `.f16`, `.f16x2`, `.bf16`, `.bf16x2` and `.tf32` destination formats.
- ▶ Adds support for `tanh` instruction that computes hyperbolic-tangent.

- ▶ Extends ex2 instruction to support .f16 and .f16x2 types.

Semantic Changes and Clarifications

None.

13.21. Changes in PTX ISA Version 6.5

New Features

PTX ISA version 6.5 introduces the following new features:

- ▶ Adds support for integer destination types for half precision comparison instruction set.
- ▶ Extends abs instruction to support .f16 and .f16x2 types.
- ▶ Adds support for cvt.pack instruction which allows converting two integer values and packing the results together.
- ▶ Adds new shapes .m16n8k8, .m8n8k16 and .m8n8k32 on the mma instruction.
- ▶ Adds support for ldmatrix instruction which loads one or more matrices from shared memory for mma instruction.

Removed Features

PTX ISA version 6.5 removes the following features:

- ▶ Support for .satfinite qualifier on floating point wmma.mma instruction has been removed. This support was deprecated since PTX ISA version 6.4.

Semantic Changes and Clarifications

None.

13.22. Changes in PTX ISA Version 6.4

New Features

PTX ISA version 6.4 introduces the following new features:

- ▶ Adds support for .noreturn directive which can be used to indicate a function does not return to it's caller function.
- ▶ Adds support for mma instruction which allows performing matrix multiply-and-accumulate operation.

Deprecated Features

PTX ISA version 6.4 deprecates the following features:

- ▶ Support for `.satfinite` qualifier on floating point `wmma.mma` instruction.

Removed Features

PTX ISA version 6.4 removes the following features:

- ▶ Support for `shf1` and `vote` instructions without the `.sync` qualifier has been removed for `.targetsm_70` and higher. This support was deprecated since PTX ISA version 6.0 as documented in PTX ISA version 6.2.

Semantic Changes and Clarifications

- ▶ Clarified that resolving references of a `.weak` symbol considers only `.weak` or `.visible` symbols with the same name and does not consider local symbols with the same name.
- ▶ Clarified that in `cvt` instruction, modifier `.ftz` can only be specified when either `.atype` or `.dtype` is `.f32`.

13.23. Changes in PTX ISA Version 6.3

New Features

PTX ISA version 6.3 introduces the following new features:

- ▶ Support for `sm_75` target architecture.
- ▶ Adds support for a new instruction `nanosleep` that suspends a thread for a specified duration.
- ▶ Adds support for `.alias` directive which allows defining alias to function symbol.
- ▶ Extends `atom` instruction to perform `.f16` addition operation and `.cas.b16` operation.
- ▶ Extends `red` instruction to perform `.f16` addition operation.
- ▶ The `wmma` instructions are extended to support multiplicand matrices of type `.s8`, `.u8`, `.s4`, `.u4`, `.b1` and accumulator matrices of type `.s32`.

Semantic Changes and Clarifications

- ▶ Introduced the mandatory `.aligned` qualifier for all `wmma` instructions.
- ▶ Specified the alignment required for the base address and stride parameters passed to `wmma.load` and `wmma.store`.
- ▶ Clarified that layout of fragment returned by `wmma` operation is architecture dependent and passing `wmma` fragments around functions compiled for different link compatible SM architectures may not work as expected.
- ▶ Clarified that atomicity for `{atom/red}.f16x2` operations is guaranteed separately for each of the two `.f16` elements but not guaranteed to be atomic as single 32-bit access.

13.24. Changes in PTX ISA Version 6.2

New Features

PTX ISA version 6.2 introduces the following new features:

- ▶ A new instruction `activemask` for querying active threads in a warp.
- ▶ Extends atomic and reduction instructions to perform `.f16x2` addition operation with mandatory `.noftz` qualifier.

Deprecated Features

PTX ISA version 6.2 deprecates the following features:

- ▶ The use of `shfl` and `vote` instructions without the `.sync` is deprecated retrospectively from PTX ISA version 6.0, which introduced the `sm_70` architecture that implements *Independent Thread Scheduling*.

Semantic Changes and Clarifications

- ▶ Clarified that `wmma` instructions can be used in conditionally executed code only if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.
- ▶ In the memory consistency model, the definition of *morally strong operations* was updated to exclude fences from the requirement of *complete overlap* since fences do not access memory.

13.25. Changes in PTX ISA Version 6.1

New Features

PTX ISA version 6.1 introduces the following new features:

- ▶ Support for `sm_72` target architecture.
- ▶ Support for new matrix shapes `32x8x16` and `8x32x16` in `wmma` instruction.

Semantic Changes and Clarifications

None.

13.26. Changes in PTX ISA Version 6.0

New Features

PTX ISA version 6.0 introduces the following new features:

- ▶ Support for `sm_70` target architecture.
- ▶ Specifies the memory consistency model for programs running on `sm_70` and later architectures.
- ▶ Various extensions to memory instructions to specify memory synchronization semantics and scopes at which such synchronization can be observed.
- ▶ New instruction `wmma` for matrix operations which allows loading matrices from memory, performing multiply-and-accumulate on them and storing result in memory.
- ▶ Support for new `barrier` instruction.
- ▶ Extends `neg` instruction to support `.f16` and `.f16x2` types.
- ▶ A new instruction `fns` which allows finding n-th set bit in integer.
- ▶ A new instruction `bar.warp.sync` which allows synchronizing threads in warp.
- ▶ Extends `vote` and `shfl` instructions with `.sync` modifier which waits for specified threads before executing the `vote` and `shfl` operation respectively.
- ▶ A new instruction `match.sync` which allows broadcasting and comparing a value across threads in warp.
- ▶ A new instruction `brx.idx` which allows branching to a label indexed from list of potential targets.
- ▶ Support for unsized array parameter for `.func` which can be used to implement variadic functions.
- ▶ Support for `.b16` integer type in dwarf-lines.
- ▶ Support for taking address of device function return parameters using `mov` instruction.

Semantic Changes and Clarifications

- ▶ Semantics of `bar` instruction were updated to indicate that executing thread waits for other non-exited threads from it's warp.
- ▶ Support for indirect branch introduced in PTX 2.1 which was unimplemented has been removed from the spec.
- ▶ Support for taking address of labels, using labels in initializers which was unimplemented has been removed from the spec.
- ▶ Support for variadic functions which was unimplemented has been removed from the spec.

13.27. Changes in PTX ISA Version 5.0

New Features

PTX ISA version 5.0 introduces the following new features:

- ▶ Support for `sm_60`, `sm_61`, `sm_62` target architecture.
- ▶ Extends atomic and reduction instructions to perform double-precision add operation.
- ▶ Extends atomic and reduction instructions to specify scope modifier.
- ▶ A new `.common` directive to permit linking multiple object files containing declarations of the same symbol with different size.
- ▶ A new `dp4a` instruction which allows 4-way dot product with accumulate operation.
- ▶ A new `dp2a` instruction which allows 2-way dot product with accumulate operation.
- ▶ Support for special register `%clock_hi`.

Semantic Changes and Clarifications

Semantics of cache modifiers on `ld` and `st` instructions were clarified to reflect cache operations are treated as performance hint only and do not change memory consistency behavior of the program.

Semantics of `volatile` operations on `ld` and `st` instructions were clarified to reflect how `volatile` operations are handled by optimizing compiler.

13.28. Changes in PTX ISA Version 4.3

New Features

PTX ISA version 4.3 introduces the following new features:

- ▶ A new `lop3` instruction which allows arbitrary logical operation on 3 inputs.
- ▶ Adds support for 64-bit computations in extended precision arithmetic instructions.
- ▶ Extends `tex.grad` instruction to support cube and acube geometries.
- ▶ Extends `tld4` instruction to support a2d, cube and acube geometries.
- ▶ Extends `tex` and `tld4` instructions to support optional operands for offset vector and depth compare.
- ▶ Extends `txq` instruction to support querying texture fields from specific LOD.

Semantic Changes and Clarifications

None.

13.29. Changes in PTX ISA Version 4.2

New Features

PTX ISA version 4.2 introduces the following new features:

- ▶ Support for `sm_53` target architecture.
- ▶ Support for arithmetic, comparison and texture instructions for `.f16` and `.f16x2` types.
- ▶ Support for `memory_layout` field for surfaces and `suq` instruction support for querying this field.

Semantic Changes and Clarifications

Semantics for parameter passing under ABI were updated to indicate `ld.param` and `st.param` instructions used for argument passing cannot be predicated.

Semantics of `{atom/red}.add.f32` were updated to indicate subnormal inputs and results are flushed to sign-preserving zero for atomic operations on global memory; whereas atomic operations on shared memory preserve subnormal inputs and results and don't flush them to zero.

13.30. Changes in PTX ISA Version 4.1

New Features

PTX ISA version 4.1 introduces the following new features:

- ▶ Support for `sm_37` and `sm_52` target architectures.
- ▶ Support for new fields `array_size`, `num_mipmap_levels` and `num_samples` for Textures, and the `txq` instruction support for querying these fields.
- ▶ Support for new field `array_size` for Surfaces, and the `suq` instruction support for querying this field.
- ▶ Support for special registers `%total_smem_size` and `%dynamic_smem_size`.

Semantic Changes and Clarifications

None.

13.31. Changes in PTX ISA Version 4.0

New Features

PTX ISA version 4.0 introduces the following new features:

- ▶ Support for `sm_32` and `sm_50` target architectures.
- ▶ Support for 64bit performance counter special registers `%pm0_64`, . . . , `%pm7_64`.
- ▶ A new `istypep` instruction.
- ▶ A new instruction, `rsqrt.approx.ftz.f64` has been added to compute a fast approximation of the square root reciprocal of a value.
- ▶ Support for a new directive `.attribute` for specifying special attributes of a variable.
- ▶ Support for `.managed` variable attribute.

Semantic Changes and Clarifications

The `vote` instruction semantics were updated to clearly indicate that an inactive thread in a warp contributes a 0 for its entry when participating in `vote.ballot.b32`.

13.32. Changes in PTX ISA Version 3.2

New Features

PTX ISA version 3.2 introduces the following new features:

- ▶ The texture instruction supports reads from multi-sample and multisample array textures.
- ▶ Extends `.section` debugging directive to include label + immediate expressions.
- ▶ Extends `.file` directive to include timestamp and file size information.

Semantic Changes and Clarifications

The `vavrg2` and `vavrg4` instruction semantics were updated to indicate that instruction adds 1 only if $Va[i] + Vb[i]$ is non-negative, and that the addition result is shifted by 1 (rather than being divided by 2).

13.33. Changes in PTX ISA Version 3.1

New Features

PTX ISA version 3.1 introduces the following new features:

- ▶ Support for `sm_35` target architecture.
- ▶ Support for CUDA Dynamic Parallelism, which enables a kernel to create and synchronize new work.
- ▶ `ld.global.nc` for loading read-only global data through the non-coherent texture cache.
- ▶ A new funnel shift instruction, `shf`.
- ▶ Extends atomic and reduction instructions to perform 64-bit `{and, or, xor}` operations, and 64-bit integer `{min, max}` operations.
- ▶ Adds support for mipmaps.
- ▶ Adds support for indirect access to textures and surfaces.
- ▶ Extends support for generic addressing to include the `.const` state space, and adds a new operator, `generic()`, to form a generic address for `.global` or `.const` variables used in initializers.
- ▶ A new `.weak` directive to permit linking multiple object files containing declarations of the same symbol.

Semantic Changes and Clarifications

PTX 3.1 redefines the default addressing for global variables in initializers, from generic addresses to offsets in the global state space. Legacy PTX code is treated as having an implicit `generic()` operator for each global variable used in an initializer. PTX 3.1 code should either include explicit `generic()` operators in initializers, use `cvta.global` to form generic addresses at runtime, or load from the non-generic address using `ld.global`.

Instruction `mad.f32` requires a rounding modifier for `sm_20` and higher targets. However for PTX ISA version 3.0 and earlier, `ptxas` does not enforce this requirement and `mad.f32` silently defaults to `mad.rn.f32`. For PTX ISA version 3.1, `ptxas` generates a warning and defaults to `mad.rn.f32`, and in subsequent releases `ptxas` will enforce the requirement for PTX ISA version 3.2 and later.

13.34. Changes in PTX ISA Version 3.0

New Features

PTX ISA version 3.0 introduces the following new features:

- ▶ Support for `sm_30` target architectures.
- ▶ SIMD video instructions.
- ▶ A new warp shuffle instruction.
- ▶ Instructions `mad.cc` and `madc` for efficient, extended-precision integer multiplication.
- ▶ Surface instructions with 3D and array geometries.

- ▶ The texture instruction supports reads from cubemap and cubemap array textures.
- ▶ Platform option `.target debug` to declare that a PTX module contains DWARF debug information.
- ▶ `pmevent.mask`, for triggering multiple performance monitor events.
- ▶ Performance monitor counter special registers `%pm4` . . `%pm7`.

Semantic Changes and Clarifications

Special register `%gridid` has been extended from 32-bits to 64-bits.

PTX ISA version 3.0 deprecates module-scoped `.reg` and `.local` variables when compiling to the Application Binary Interface (ABI). When compiling without use of the ABI, module-scoped `.reg` and `.local` variables are supported as before. When compiling legacy PTX code (ISA versions prior to 3.0) containing module-scoped `.reg` or `.local` variables, the compiler silently disables use of the ABI.

The `shfl` instruction semantics were updated to clearly indicate that value of source operand `a` is unpredictable for inactive and predicated-off threads within the warp.

PTX modules no longer allow duplicate `.version` directives. This feature was unimplemented, so there is no semantic change.

Unimplemented instructions `suld.p` and `sust.p.{u32, s32, f32}` have been removed.

13.35. Changes in PTX ISA Version 2.3

New Features

PTX 2.3 adds support for texture arrays. The texture array feature supports access to an array of 1D or 2D textures, where an integer indexes into the array of textures, and then one or two single-precision floating point coordinates are used to address within the selected 1D or 2D texture.

PTX 2.3 adds a new directive, `.address_size`, for specifying the size of addresses.

Variables in `.const` and `.global` state spaces are initialized to zero by default.

Semantic Changes and Clarifications

The semantics of the `.maxntid` directive have been updated to match the current implementation. Specifically, `.maxntid` only guarantees that the total number of threads in a thread block does not exceed the maximum. Previously, the semantics indicated that the maximum was enforced separately in each dimension, which is not the case.

Bit field extract and insert instructions `BFE` and `BFI` now indicate that the `len` and `pos` operands are restricted to the value range `0` . . `255`.

Unimplemented instructions `{atom, red}. {min, max}. f32` have been removed.

13.36. Changes in PTX ISA Version 2.2

New Features

PTX 2.2 adds a new directive for specifying kernel parameter attributes; specifically, there is a new directives for specifying that a kernel parameter is a pointer, for specifying to which state space the parameter points, and for optionally specifying the alignment of the memory to which the parameter points.

PTX 2.2 adds a new field named `force_unnormalized_coords` to the `.samplerref` opaque type. This field is used in the independent texturing mode to override the `normalized_coords` field in the texture header. This field is needed to support languages such as OpenCL, which represent the property of normalized/unnormalized coordinates in the sampler header rather than in the texture header.

PTX 2.2 deprecates explicit constant banks and supports a large, flat address space for the `.const` state space. Legacy PTX that uses explicit constant banks is still supported.

PTX 2.2 adds a new `tlld4` instruction for loading a component (r, g, b, or a) from the four texels comprising the bilinear interpolation footprint of a given texture location. This instruction may be used to compute higher-precision bilerp results in software, or for performing higher-bandwidth texture loads.

Semantic Changes and Clarifications

None.

13.37. Changes in PTX ISA Version 2.1

New Features

The underlying, stack-based ABI is supported in PTX ISA version 2.1 for `sm_2x` targets.

Support for indirect calls has been implemented for `sm_2x` targets.

New directives, `.branchtargets` and `.calltargets`, have been added for specifying potential targets for indirect branches and indirect function calls. A `.callprototype` directive has been added for declaring the type signatures for indirect function calls.

The names of `.global` and `.const` variables can now be specified in variable initializers to represent their addresses.

A set of thirty-two driver-specific execution environment special registers has been added. These are named `%envreg0`..`%envreg31`.

Textures and surfaces have new fields for channel data type and channel order, and the `txq` and `suq` instructions support queries for these fields.

Directive `.minnctapersm` has replaced the `.maxnctapersm` directive.

Directive `.reqntid` has been added to allow specification of exact CTA dimensions.

A new instruction, `rcp.approx.ftz.f64`, has been added to compute a fast, gross approximate reciprocal.

Semantic Changes and Clarifications

A warning is emitted if `.minnctapersm` is specified without also specifying `.maxntid`.

13.38. Changes in PTX ISA Version 2.0

New Features

Floating Point Extensions

This section describes the floating-point changes in PTX ISA version 2.0 for `sm_20` targets. The goal is to achieve IEEE 754 compliance wherever possible, while maximizing backward compatibility with legacy PTX ISA version 1.x code and `sm_1x` targets.

The changes from PTX ISA version 1.x are as follows:

- ▶ Single-precision instructions support subnormal numbers by default for `sm_20` targets. The `.ftz` modifier may be used to enforce backward compatibility with `sm_1x`.
- ▶ Single-precision `add`, `sub`, and `mul` now support `.rm` and `.rp` rounding modifiers for `sm_20` targets.
- ▶ A single-precision fused multiply-add (`fma`) instruction has been added, with support for IEEE 754 compliant rounding modifiers and support for subnormal numbers. The `fma.f32` instruction also supports `.ftz` and `.sat` modifiers. `fma.f32` requires `sm_20`. The `mad.f32` instruction has been extended with rounding modifiers so that it's synonymous with `fma.f32` for `sm_20` targets. Both `fma.f32` and `mad.f32` require a rounding modifier for `sm_20` targets.
- ▶ The `mad.f32` instruction *without rounding* is retained so that compilers can generate code for `sm_1x` targets. When code compiled for `sm_1x` is executed on `sm_20` devices, `mad.f32` maps to `fma.rn.f32`.
- ▶ Single- and double-precision `div`, `rcp`, and `sqrt` with IEEE 754 compliant rounding have been added. These are indicated by the use of a rounding modifier and require `sm_20`.
- ▶ Instructions `testp` and `copysign` have been added.

New Instructions

A *load uniform* instruction, `ldu`, has been added.

Surface instructions support additional `.clamp` modifiers, `.clamp` and `.zero`.

Instruction `sust` now supports formatted surface stores.

A *count leading zeros* instruction, `clz`, has been added.

A *find leading non-sign bit instruction*, `bfind`, has been added.

A *bit reversal* instruction, `brev`, has been added.

Bit field extract and insert instructions, `bfe` and `bfi`, have been added.

A *population count* instruction, `popc`, has been added.

A *vote ballot* instruction, `vote.ballot.b32`, has been added.

Instructions `{atom, red}.add.f32` have been implemented.

Instructions `{atom, red}.shared` have been extended to handle 64-bit data types for `sm_20` targets.

A system-level `membar .sys`, has been added.

The `bar` instruction has been extended as follows:

- ▶ A `bar .arrive` instruction has been added.
- ▶ Instructions `bar .red .popc .u32` and `bar .red .{and, or} .pred` have been added.
- ▶ `bar` now supports optional thread count and register operands.

Scalar video instructions (includes `prmt`) have been added.

Instruction `isspacep` for querying whether a generic address falls within a specified state space window has been added.

Instruction `cvta` for converting global, local, and shared addresses to generic address and vice-versa has been added.

Other New Features

Instructions `ld`, `ldu`, `st`, `prefetch`, `prefetchu`, `isspacep`, `cvta`, `atom`, and `red` now support generic addressing.

New special registers `%nwarpid`, `%nsmid`, `%clock64`, `%lanemask_{eq, le, lt, ge, gt}` have been added.

Cache operations have been added to instructions `ld`, `st`, `suld`, and `sust`, e.g., for `prefetching` to specified level of memory hierarchy. Instructions `prefetch` and `prefetchu` have also been added.

The `.maxnctaper sm` directive was deprecated and replaced with `.minnctaper sm` to better match its behavior and usage.

A new directive, `.section`, has been added to replace the `@@DWARF` syntax for passing DWARF-format debugging information through PTX.

A new directive, `.pragma nounroll`, has been added to allow users to disable loop unrolling.

Semantic Changes and Clarifications

The errata in `cvt .ftz` for PTX ISA versions 1.4 and earlier, where single-precision subnormal inputs and results were not flushed to zero if either source or destination type size was 64-bits, has been fixed. In PTX ISA version 1.5 and later, `cvt .ftz` (and `cvt` for `.target sm_1x`, where `.ftz` is implied) instructions flush single-precision subnormal inputs and results to sign-preserving zero for all combinations of floating-point instruction types. To maintain compatibility with legacy PTX code, if `.version` is 1.4 or earlier, single-precision subnormal inputs and results are flushed to sign-preserving zero only when neither source nor destination type size is 64-bits.

Components of special registers `%tid`, `%ntid`, `%ctaid`, and `%nctaid` have been extended from 16-bits to 32-bits. These registers now have type `.v4.u32`.

The number of samplers available in independent texturing mode was incorrectly listed as thirty-two in PTX ISA version 1.5; the correct number is sixteen.

Chapter 14. Notices

14.1. Notice

This document is provided for information purposes only and shall not be regarded as a warranty of a certain functionality, condition, or quality of a product. NVIDIA Corporation (“NVIDIA”) makes no representations or warranties, expressed or implied, as to the accuracy or completeness of the information contained in this document and assumes no responsibility for any errors contained herein. NVIDIA shall have no liability for the consequences or use of such information or for any infringement of patents or other rights of third parties that may result from its use. This document is not a commitment to develop, release, or deliver any Material (defined below), code, or functionality.

NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice.

Customer should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

NVIDIA products are sold subject to the NVIDIA standard terms and conditions of sale supplied at the time of order acknowledgement, unless otherwise agreed in an individual sales agreement signed by authorized representatives of NVIDIA and customer (“Terms of Sale”). NVIDIA hereby expressly objects to applying any customer general terms and conditions with regards to the purchase of the NVIDIA product referenced in this document. No contractual obligations are formed either directly or indirectly by this document.

NVIDIA products are not designed, authorized, or warranted to be suitable for use in medical, military, aircraft, space, or life support equipment, nor in applications where failure or malfunction of the NVIDIA product can reasonably be expected to result in personal injury, death, or property or environmental damage. NVIDIA accepts no liability for inclusion and/or use of NVIDIA products in such equipment or applications and therefore such inclusion and/or use is at customer’s own risk.

NVIDIA makes no representation or warranty that products based on this document will be suitable for any specified use. Testing of all parameters of each product is not necessarily performed by NVIDIA. It is customer’s sole responsibility to evaluate and determine the applicability of any information contained in this document, ensure the product is suitable and fit for the application planned by customer, and perform the necessary testing for the application in order to avoid a default of the application or the product. Weaknesses in customer’s product designs may affect the quality and reliability of the NVIDIA product and may result in additional or different conditions and/or requirements beyond those contained in this document. NVIDIA accepts no liability related to any default, damage, costs, or problem which may be based on or attributable to: (i) the use of the NVIDIA product in any manner that is contrary to this document or (ii) customer product designs.

No license, either expressed or implied, is granted under any NVIDIA patent right, copyright, or other NVIDIA intellectual property right under this document. Information published by NVIDIA regarding third-party products or services does not constitute a license from NVIDIA to use such products or

services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property rights of the third party, or a license from NVIDIA under the patents or other intellectual property rights of NVIDIA.

Reproduction of information in this document is permissible only if approved in advance by NVIDIA in writing, reproduced without alteration and in full compliance with all applicable export laws and regulations, and accompanied by all associated conditions, limitations, and notices.

THIS DOCUMENT AND ALL NVIDIA DESIGN SPECIFICATIONS, REFERENCE BOARDS, FILES, DRAWINGS, DIAGNOSTICS, LISTS, AND OTHER DOCUMENTS (TOGETHER AND SEPARATELY, "MATERIALS") ARE BEING PROVIDED "AS IS." NVIDIA MAKES NO WARRANTIES, EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE MATERIALS, AND EXPRESSLY DISCLAIMS ALL IMPLIED WARRANTIES OF NONINFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE. TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL NVIDIA BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF NVIDIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. Notwithstanding any damages that customer might incur for any reason whatsoever, NVIDIA's aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms of Sale for the product.

14.2. OpenCL

OpenCL is a trademark of Apple Inc. used under license to the Khronos Group Inc.

14.3. Trademarks

NVIDIA and the NVIDIA logo are trademarks or registered trademarks of NVIDIA Corporation in the U.S. and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright

©2007-2026, NVIDIA Corporation & affiliates. All rights reserved